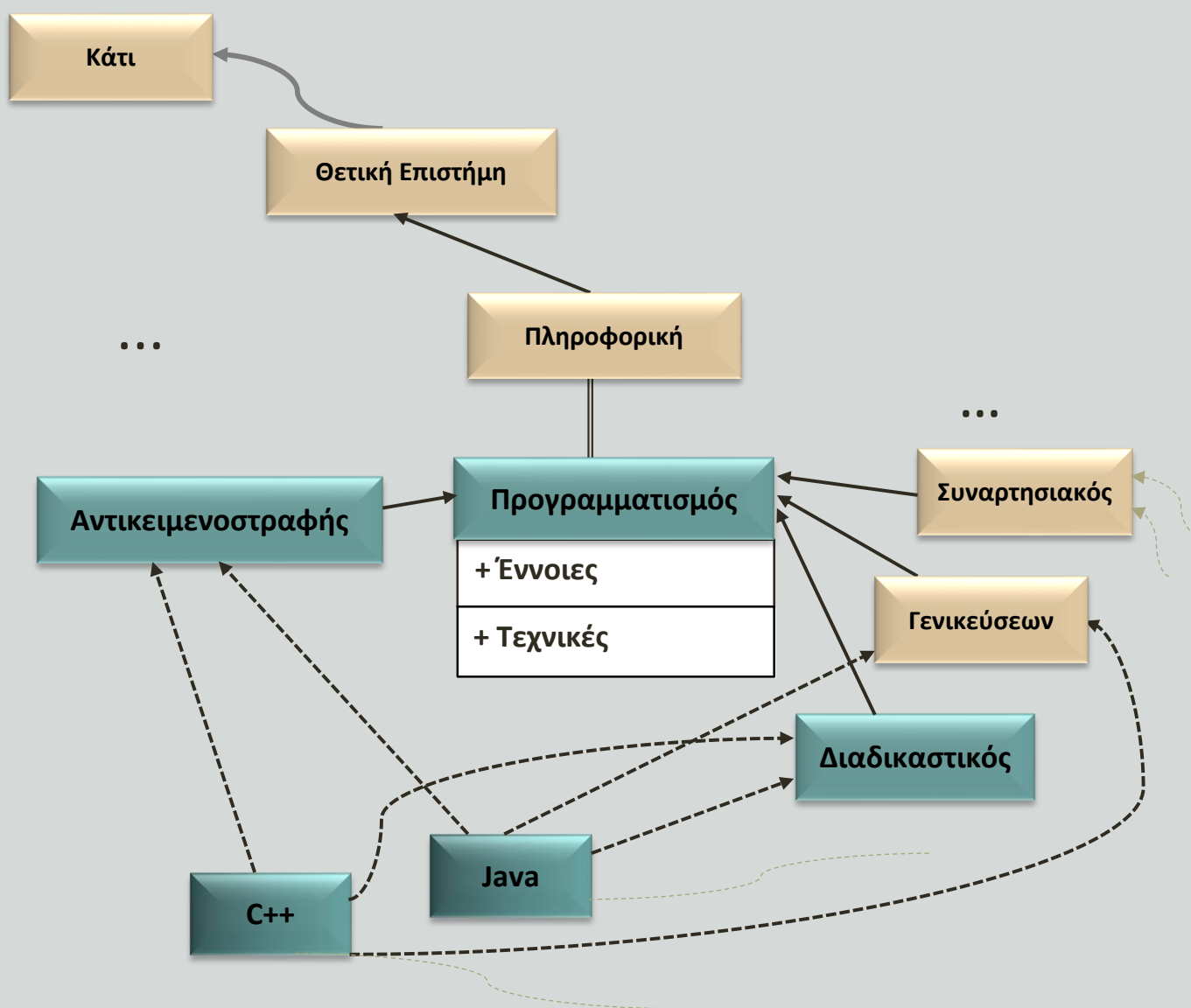


ΙΖΑΜΠΩ Γ. ΚΑΡΑΛΗ

Αντικειμενοστραφής Προγραμματισμός: Έννοιες και Τεχνικές

Ανάδειξή τους με C++ και Java



Αντικειμενοστραφής Προγραμματισμός: Έννοιες και Τεχνικές

Ανάδειξή τους με C++ και Java

Ιζαμπώ Γ. Καράλη
Επίκουρη Καθηγήτρια
Εθνικό και Καποδιστριακό Πανεπιστήμιο Αθηνών

Τίτλος πρωτοτύπου: «Αντικειμενοστραφής Προγραμματισμός: Έννοιες και Τεχνικές»
 Copyright © 2023, ΚΑΛΛΙΠΟΣ, ΑΝΟΙΚΤΕΣ ΑΚΑΔΗΜΑΪΚΕΣ ΕΚΔΟΣΕΙΣ



Το παρόν έργο διατίθεται με τους όρους της άδειας Creative Commons Αναφορά Δημιουργού – Μη Εμπορική Χρήση – Παρόμοια Διανομή 4.0. Για να δείτε τους όρους της άδειας αυτής επισκεφτείτε τον ιστότοπο <https://creativecommons.org/licenses/by-nc-sa/4.0/legalcode.el>

Αν τυχόν κάποιο τμήμα του έργου διατίθεται με διαφορετικό καθεστώς αδειοδότησης, αυτό αναφέρεται ρητά και ειδικώς στην οικεία θέση.

Συντελεστές έκδοσης

Γλωσσική επιμέλεια:

Μαρία Ζουρελίδου

Τεχνική επεξεργασία:

Δημήτριος Καρατζίδης

ΚΑΛΛΙΠΟΣ

Εθνικό Μετσόβιο Πολυτεχνείο

Ηρώων Πολυτεχνείου 9

15780 Ζωγράφου

www.kallipos.gr

Βιβλιογραφική αναφορά:

Καράλη Ι., (2023). *Αντικειμενοστραφής Προγραμματισμός: Έννοιες και Τεχνικές* [Προπτυχιακό Εγχειρίδιο]. Αθήνα: Κάλλιπος, Ανοικτές Ακαδημαϊκές Εκδόσεις

Διαθέσιμο στο:

<http://dx.doi.org/10.57713/kallipos-280>

ISBN:

978-618-228-050-8

ΠΕΡΙΕΧΟΜΕΝΑ

Κατάλογος σχημάτων	vii
Πίνακας συντομεύσεων - ακρωνυμίων	ix
Πρόλογος	xi
1 Εισαγωγή στον Αντικειμενοστραφή Προγραμματισμό	1
1.1 Γενικά Στοιχεία για τον Αντικειμενοστραφή Προγραμματισμό	1
1.2 Διαδικαστικός και Αντικειμενοστραφής Προγραμματισμός	2
1.2.1 Διαδικαστικός ή Αντικειμενοστραφής Προγραμματισμός;	2
1.2.2 Βασικά Χαρακτηριστικά του Αντικειμενοστραφούς Μοντέλου στον Προγραμματισμό	3
1.3 Ονοματολογία του ΟΟΡ, Μηχανισμοί και Ιδιότητες των Γλωσσών του	5
1.3.1 Έννοιες του ΟΟΡ	5
1.3.1.1 Κλάσεις	5
1.3.1.2 Αντικείμενα	7
1.3.1.3 Κληρονομικότητα	7
1.3.2 Μηχανισμοί του ΟΟΡ	8
1.3.3 Ιδιότητες του ΟΟΡ	9
1.4 Πού Συναντάμε το Αντικειμενοστραφές Μοντέλο	11
1.4.1 Ανάλυση και Σχεδίαση Λογισμικού	11
1.4.1.1 Η Γλώσσα Μοντελοποίησης UML	13
1.4.2 Αναπαράσταση Γνώσης	15
1.4.3 Βάσεις Δεδομένων και ΟΟ	18
1.5 Λίγα Λόγια για την Ιστορία	19
1.6 Σύνοψη Κεφαλαίου	26
1.7 Ερωτήσεις	27
1.8 Βιβλιογραφία	28
2 Στοιχειοποίηση και Αφαίρεση στα Δεδομένα	31
2.1 Από τον Διαδικαστικό Προγραμματισμό στον Αντικειμενοστραφή: Ένα Πρώτο Λογικό Βήμα	31

2.1.1	Στοιχειοποίηση	31
2.2	Αφαίρεση στα Δεδομένα	35
2.2.1	Από τους Τύπους στα Αφηρημένα Αντικείμενα	36
2.2.2	Αλγεβρικές Προδιαγραφές των ΑΤΔ	38
2.2.3	Τιμές και Στιγμιότυπα	39
2.3	Τύποι Ορισμένοι από τον Χρήστη	40
2.3.1	Οι Προδιαγραφές του Νέου Τύπου	41
2.3.2	Η Χρήση του Νέου Τύπου	43
2.3.3	Η Υλοποίηση του Νέου Τύπου	44
2.3.4	Αλλαγή της Υλοποίησης του Νέου Τύπου	46
2.3.5	Σχόλια και Συμπεράσματα από το Παράδειγμα	48
2.4	Σύνοψη Κεφαλαίου	50
2.5	Βιβλιογραφία	50
3	Στοιχεία από τον Διαδικαστικό Προγραμματισμό	53
3.1	Γενικές Παρατηρήσεις	53
3.2	Γενικά Στοιχεία για τη Γλώσσα Προγραμματισμού C++	54
3.2.1	Μεταγλώττιση και Αρχεία Επικεφαλίδας	55
3.2.2	Διαδικαστικός Προγραμματισμός με/σε C++	59
3.2.2.1	Τύποι	59
3.2.2.2	Εκφράσεις και Εντολές	61
3.3	Στοιχεία της C++ με Ιδιαίτερο Ενδιαφέρον για τις Κλάσεις	72
3.3.1	Σταθερές	72
3.3.2	Συναρτήσεις	76
3.3.2.1	Συναρτήσεις inline	77
3.3.2.2	Υπερφόρτωση Ονομάτων Συναρτήσεων	79
3.3.2.3	Default Τιμές Ορισμάτων	80
3.3.3	Οι Τελεστές new και delete	81
3.3.4	Τύπος Αναφοράς	82
3.3.4.1	Ο Τύπος Αναφοράς στα Ορίσματα Συνάρτησης	83
3.3.4.2	Μεταβλητές: Απλές, Αναφορές και Δείκτες - Παραδείγματα	86
3.4	Γενικές Παρατηρήσεις περί Μεταγλώττισης και Συνδεσιμότητας Προγραμμάτων C++	87
3.4.1	Συνδεσιμότητα Μεταβλητών	88
3.4.2	Συνδεσιμότητα Συναρτήσεων	90
3.5	Σύνοψη Κεφαλαίου	90
3.6	Ασκήσεις	91
3.7	Βιβλιογραφία	98
4	Η Έννοια της Εμβέλειας	99
4.1	Ονόματα και Εμβέλεια	99
4.1.1	Δηλώσεις Ονομάτων	101
4.1.2	Η Περίπτωση των Μεταβλητών: Εμβέλεια Δήλωσης vs. Χρόνος Ζωής	103
4.2	Χώροι Ονομάτων	109
4.2.1	Η Έννοια του Χώρου Ονομάτων	109
4.2.2	Ανώνυμοι Χώροι Ονομάτων	112
4.2.3	Οργάνωση Προγράμματος που περιέχει Χώρους Ονομάτων	113
4.3	Σύνοψη Κεφαλαίου	114
4.4	Ασκήσεις	115
4.5	Βιβλιογραφία	119

5	Κλάσεις και Αντικείμενα	121
5.1	Αφαίρεση στα Δεδομένα: Ορισμός Νέων Τύπων και Εισαγωγή στις Κλάσεις	121
5.1.1	Τύποι Δεδομένων Ορισμένοι από τον Χρήστη μέσω της Γλωσσικής Δομής <code>struct</code>	123
5.1.1.1	Ολοκληρωμένος Ορισμός και Χρήση του Νέου Τύπου <code>date</code>	126
5.2	Ορισμός Νέου Τύπου Κλάσης	128
5.2.1	Ο Δείκτης <code>this</code>	133
5.3	Ορισμός του Τύπου “Φοιτητής”	138
5.3.1	Οργάνωση του Κώδικα σε Αρχεία	140
5.3.2	<code>inline</code> Συναρτήσεις-μέλη	142
5.4	Αρχικοποίηση - Καταστροφή Αντικειμένων	144
5.4.1	Συναρτήσεις Αρχικοποίησης	145
5.4.1.1	Χρήση Λίστας Αρχικοποίησης Μελών-δεδομένων	147
5.4.2	Συναρτήσεις Καταστροφής	148
5.4.3	Αρχικοποίηση Στιγμιότυπων (συνέχεια): Αρχικοποίηση μέσω Αντιγραφής	154
5.5	Επαναορισμός του Τελεστή Ανάθεσης - Παράδειγμα Αναγκαιότητας του Δείκτη <code>this</code>	160
5.6	Ορίζοντας μια Κλάση	164
5.7	Σύνοψη Κεφαλαίου	165
5.8	Ασκήσεις	166
5.9	Βιβλιογραφία	169
6	Σύνθεση Κλάσεων	171
6.1	Η Έννοια της Σύνθεσης Κλάσεων	171
6.1.1	Ένα Πρώτο Παράδειγμα Σύνθετης Κλάσης	173
6.2	Εκτενέστερη Συζήτηση πάνω στις Συναρτήσεις Αρχικοποίησης	179
6.3	Σύνθετες Κλάσεις: Επαναορισμός του Τελεστή Ανάθεσης	183
6.4	Με Αφορμή τη Σύνθεση Κλάσεων: Εναλλακτικά στους Τύπους των Μελών-δεδομένων	190
6.4.1	Μέλη-δεδομένα Τύπου Δείκτη	190
6.4.2	Επιλογή της Αναπαράστασης των Μελών-δεδομένων	194
6.5	Σύνοψη Κεφαλαίου	203
6.6	Ασκήσεις	204
6.7	Βιβλιογραφία	220
7	Κληρονομικότητα	223
7.1	Εισαγωγή στην Κληρονομικότητα	223
7.1.1	Η Κληρονομικότητα στον Αντικειμενοστραφή Προγραμματισμό	226
7.2	Η Κληρονομικότητα στη C++	227
7.2.1	Διαβαθμίσεις των Δικαιωμάτων Πρόσβασης στα Μέλη μιας Κλάσης	233
7.2.2	Ζητήματα στην Κληρονομικότητα	235
7.2.2.1	Αρχικοποίηση κατά την Κληρονομικότητα	240
7.3	Ορισμός Κλάσεων με Κληρονομικότητα και Σύνθεση	244
7.4	Πολυμορφισμός μέσω Εικονικών Συναρτήσεων και Αφηρημένες Κλάσεις	250
7.4.1	Η Συμπεριφορά των Εικονικών Συναρτήσεων μέσω Παραδειγμάτων	250
7.4.1.1	Ένα Πρώτο Παράδειγμα Εικονικών Συναρτήσεων	251
7.4.1.2	Παράδειγμα: Ιεραρχία με πάνω από Δύο Κλάσεις	255
7.4.1.3	Παράδειγμα: Ιεραρχία Κλάσεων με Μέλη-δεδομένα	257
7.4.1.4	Παράδειγμα: Κλήσεις μεταξύ Εικονικών και μη Εικονικών Συναρτήσεων-μελών	260
7.4.2	Αφηρημένες Κλάσεις	263
7.4.3	Εικονικές Συναρτήσεις Καταστροφής - Γενικό Παράδειγμα στην Κληρονομικότητα	264

7.5	Σύνοψη Κεφαλαίου	268
7.6	Ασκήσεις	271
7.7	Βιβλιογραφία	310
8	Άλλα Γλωσσικά Στοιχεία της C++ Σχετικά με Κλάσεις	311
8.1	Σταθερές και Κλάσεις	312
8.2	Στατικά Μέλη Κλάσεων	312
8.3	Ολοκληρώνοντας Σχετικά με τα Μέλη Κλάσεων	318
8.3.1	Εμφωλευμένες Κλάσεις	319
8.4	Η Έννοια της Φιλίας μεταξύ Εμβλειών	321
8.4.1	Φιλικές Συναρτήσεις	321
8.4.2	Φιλικές Κλάσεις	323
8.5	Κλήση Συναρτήσεων: Πέρασμα Παραμέτρων και Επιστροφή Τιμής	325
8.5.1	Πέρασμα Παραμέτρων και Επιστροφή Τιμής κατά Τιμή	325
8.5.2	Πέρασμα Παραμέτρων και Επιστροφή Τιμής με Χρήση Αναφορών	327
8.5.3	Αναφορές rvalue, Αρχικοποίηση και Ανάθεση με Μεταφορά	328
8.6	Προγραμματισμός Γενικεύσεων	333
8.6.1	Περιγράμματα Συναρτήσεων	334
8.6.2	Περιγράμματα Κλάσεων	336
8.7	Τύποι Βιβλιοθήκης	338
8.8	Το Σύστημα Εισόδου/Εξόδου της C++	341
8.8.1	Αντικειμενοστραφής Προσέγγιση του Συστήματος I/O	341
8.8.1.1	Η Κλάση ios_base	343
8.8.1.2	Έλεγχος της Κατάστασης Ρεύματος	343
8.8.1.3	Ρεύματα Εξόδου	343
8.8.1.4	Ρεύματα Εισόδου	344
8.8.1.5	Είσοδος/Εξοδος επάνω σε Αρχεία	345
8.8.2	Διαμορφωτές	346
8.9	Σύνοψη Κεφαλαίου	346
8.10	Ασκήσεις	348
8.11	Βιβλιογραφία	367
9	Μετάβαση στη Γλώσσα Java	369
9.1	Ιστορικά Στοιχεία της Γλώσσας Java	369
9.1.1	Το Χρονικό μέχρι την Πρώτη Έκδοση της Java	371
9.2	Γενικά Χαρακτηριστικά της Γλώσσας	372
9.3	Οργάνωση και Εκτέλεση των Προγραμμάτων Java	373
9.3.1	Το Περιβάλλον Εκτέλεσης	373
9.3.1.1	Λίγα Λόγια για τη JVM και το class file format	374
9.3.2	Δομή και Μεταγλώττιση Προγραμμάτων Java	374
9.4	Γενικές Παρατηρήσεις για τα Προγράμματα Java	375
9.4.1	Κατηγορίες Κλάσεων στην Java	377
9.4.2	Διαβαθμίσεις Ορατότητας - modifiers για τα Μέλη Κλάσεων	379
9.5	Δεδομένα και Διαδικασίες	380
9.5.1	Μεταβλητές στην Java - Τύποι	380
9.5.1.1	Τύποι Αναφοράς	381
9.5.1.2	Τύποι Αναφοράς και Πίνακες	383
9.5.2	Πώς Γίνονται οι Διαδικασίες	385
9.6	Ορισμός Κλάσεων	386

9.6.1	Αρχικοποίηση Στιγμιотύπων στην Java	389
9.6.1.1	Υπερφορτωμένες Μέθοδοι Αρχικοποίησης	390
9.6.1.2	Χρήση Block Αρχικοποίησης	391
9.6.1.3	Παράδειγμα Αρχικοποίησης Στιγμιотύπων	393
9.6.1.4	Βήματα κατά τη Διαδικασία Αρχικοποίησης Στιγμιотύπων Κλάσεων	395
9.7	Κληρονομικότητα στην Java	395
9.7.1	Γενικά Στοιχεία	396
9.7.2	Αφηρημένες Κλάσεις	398
9.7.3	Παραδείγματα	398
9.7.4	Διαδικασία Αρχικοποίησης Στιγμιотύπων Κλάσεων: η Γενική Περίπτωση	403
9.8	Μεταβλητές Αναφοράς: Αναθέσεις και Χρήση σε Μεθόδους Στιγμιотύπου	403
9.9	Κληρονομικότητα και Σύνθεση: Παραδείγματα	408
9.10	Πολλαπλή Κληρονομικότητα - Τύπος Διεπαφής	413
9.11	Στατικά Μέλη στην Java	416
9.11.1	Χρήση Στατικών Μεθόδων (Μεθόδων Κλάσης)	417
9.11.2	Αρχικοποίηση Κλάσεων	418
9.11.2.1	Διαδικασία Αρχικοποίησης για (μεταβλητές) Κλάση(ς) και Τύπους Διεπαφής	422
9.11.2.2	Παράδειγμα: Στατικά και Μη-στατικά μέλη: Εμβέλεια, Τρόπος Χρήσης, Αρχικοποιήσεις	423
9.11.2.3	Παράδειγμα: Αρχικοποίηση Τύπων Διεπαφής	428
9.12	Τα Πακέτα της Java	429
9.12.1	Παράδειγμα Κώδικα με Χρήση Πακέτων	431
9.13	Άλλα (Κρίσιμα) Θέματα σχετικά με την Java	434
9.13.1	Αρχικοποίηση μέσω Αντιγραφής	434
9.13.2	Περισυλλογή Σκουπιδιών και Διαδικασίες που Σχετίζονται με “Άχρηστα” Αντικείμενα	436
9.13.3	Κάποιες Επισημάνσεις, ακόμα	437
9.14	Σύνοψη Κεφαλαίου	439
9.15	Ασκήσεις	441
9.16	Βιβλιογραφία	454
10	Άλλες Γλώσσες με Αντικειμενοστραφή Χαρακτηριστικά	457
10.1	Η Γλώσσα C#	457
10.1.1	Λίγα Στοιχεία για την Ιστορία της Γλώσσας C# και το Περιβάλλον Εκτέλεσής της	458
10.1.2	Η Οργάνωση των Προγραμμάτων C#	459
10.1.2.1	Επίπεδα Ορατότητας	459
10.1.2.2	Το Αρχικό Σημείο Εκτέλεσης ενός Προγράμματος	460
10.1.3	Δεδομένα και Διαδικασίες	460
10.1.3.1	Δυο Λόγια για τις Διαδικασίες	461
10.1.4	Κλάσεις και Άλλοι Τύποι	462
10.1.4.1	Μέλη Ιδιότητες	464
10.1.4.2	Παραδείγματα Ορισμού Κλάσης	465
10.1.4.3	Τύποι Δομής	473
10.1.5	Κληρονομικότητα	476
10.1.5.1	Πολλαπλή Κληρονομικότητα, Τύποι Διεπαφής	480
10.1.6	Κάποια Άλλα Χρήσιμα Στοιχεία για τη C#	480
10.2	Η Γλώσσα Python	481
10.2.1	Λίγα Λόγια για την Ιστορία της	481

10.2.2	Γενικά Χαρακτηριστικά της Γλώσσας	482
10.2.2.1	Η Μορφή ενός Προγράμματος και η Οργάνωση του Κώδικα	482
10.2.2.2	Δεδομένα και Τύποι	485
10.2.2.3	Δομές Ελέγχου και Συναρτήσεις	487
10.2.3	Αντικειμενοστραφής Προγραμματισμός	488
10.2.3.1	Ορίζοντας μια Απλή Κλάση	488
10.2.3.2	Αρχικοποίηση Αντικειμένων, Ανάθεση και Αρχικοποίηση μέσω Αντι- γραφής	492
10.2.3.3	Μια Υλοποίηση της Έννοιας του “Φοιτητή”	493
10.2.3.4	Σύνθεση Κλάσεων	497
10.2.3.5	Κληρονομικότητα	498
10.2.4	Τελευταίες Παρατηρήσεις	503
10.3	Μια Ματιά σε Γλώσσες Συναρτησιακού Προγραμματισμού	503
10.3.1	Η Γλώσσα Scala	504
10.3.1.1	Βασικά Στοιχεία της Γλώσσας	505
10.3.1.2	Αντικειμενοστραφής Προγραμματισμός	506
10.3.2	Η Γλώσσα F#	510
10.3.2.1	Βασικά Στοιχεία της Γλώσσας	510
10.3.2.2	Επίπεδα Ορατότητας	512
10.3.2.3	Κλάσεις, Αντικείμενα και Κληρονομικότητα στην F#	513
10.4	Σύνοψη Κεφαλαίου	516
10.5	Βιβλιογραφία	517
11	Προβλήματα και Εργασίες Κατάλληλα για Αντικειμενοστραφή Υλοποίηση	519
11.1	Γενικές Παρατηρήσεις επί της Ανάπτυξης	519
11.2	Προβλήματα Μικρής Κλίμακας	521
11.3	Μικρές Εργασίες για Εξάσκηση στην Αφαίρεση στα Δεδομένα	548
11.4	Μικρές Εργασίες στην Αφαίρεση στα Δεδομένα: Δομές Δεδομένων	559
11.5	Μικρές Εργασίες για Εξάσκηση στην Κληρονομικότητα	575
11.6	Μεγαλύτερες Εργασίες	593
11.7	Σύνοψη Κεφαλαίου	608

ΚΑΤΑΛΟΓΟΣ ΣΧΗΜΑΤΩΝ

1.1	Ένα στιγμιότυπο, d, της κλάσης Date	6
1.2	Ένα απλό UML διάγραμμα κλάσεων	14
1.3	Ένα παράδειγμα σημασιολογικού δικτύου	16
1.4	Ένα παράδειγμα οντολογίας στο περιβάλλον Protégé	17
1.5	Ένα παράδειγμα πλαισίων “cat” και “mammal”	18
2.1	Δύο συναρτήσεις	32
2.2	Σύγκρουση ονομάτων: δύο διαφορετικοί ορισμοί για τη συνάρτηση void foo()	32
2.3	Οι συναρτήσεις void foo() μετονομάστηκαν με διαφορετικά ονόματα	33
2.4	Ο κώδικάς μας μέσα σε στοιχεία	34
2.5	Τα στοιχεία με τις διεπαφές τους για την ανταλλαγή πληροφορίας	34
2.6	Οι δύο προγραμματιστές της εφαρμογής και η “συμφωνία” τους για τις προδιαγραφές του τύπου Time	43
2.7	Οι δύο εναλλακτικές αναπαραστάσεις των μεταβλητών του τύπου Time	45
3.1	Διαδικασία μεταγλώττισης προγραμμάτων C++ σε περιβάλλον Unix μορφής	55
3.2	Ανάλυση της φάσης μεταγλώττισης	56
3.3	Το αποτέλεσμα του ορισμού της σταθεράς a	72
3.4	Η γενική εικόνα ενός δείκτη	73
3.5	Δήλωση σταθεράς για το αντικείμενο στο οποίο δείχνει ο δείκτης	73
3.6	Δήλωση σταθεράς για τον δείκτη	74
3.7	Δήλωση σταθεράς και για τον δείκτη και για το αντικείμενο στο οποίο δείχνει	74
3.8	Οι δύο μεταβλητές i και j	82
3.9	Η j είναι αναφορά στην i	83
3.10	Η p είναι δείκτης στην i	83
3.11	Πέρασμα παραμέτρων à la C	84
3.12	Πέρασμα παραμέτρων à la C++	85
3.13	Εξωτερική συνδεσιμότητα για τη μεταβλητή i	88
3.14	Εσωτερική συνδεσιμότητα για τη μεταβλητή i	89
4.1	Εμφωλευμένα περιβάλλοντα	100

4.2	Διακριτά περιβάλλοντα	100
5.1	Οι μεταβλητές <code>today</code> και <code>another_day</code> (αφαιρετικά) στη μνήμη	124
5.2	Η εικόνα των μεταβλητών (αφαιρετικά) στη μνήμη	134
5.3	Η εικόνα των μεταβλητών στην περίπτωση της κλάσης “ <code>Student</code> ” (αφαιρετικά) στη μνήμη	140
5.4	Η εικόνα των μεταβλητών στην περίπτωση μη διάκρισης αρχικοποίησης της κλάσης “ <code>Student</code> ”	145
5.5	Η εικόνα των μεταβλητών στην περίπτωση κλήσης συνάρτησης που δημιουργεί αντικείμενο της κλάσης “ <code>Student</code> ”	149
5.6	Η εικόνα μετά από την επιστροφή από την κλήση της συνάρτησης <code>bla</code>	150
5.7	Αρχικοποίηση μέσω αντιγραφής για τη μεταβλητή <code>s1</code> της κλάσης <code>Student</code>	155
5.8	Το πρόβλημα που προκύπτει από την αρχικοποίηση μέσω αντιγραφής για την κλάση <code>Student</code>	156
5.9	Εικόνα της μνήμης με ορισμό συνάρτησης αρχικοποίησης μέσω αντιγραφής για την κλάση <code>Student</code>	157
6.1	Οι αναπαραστάσεις των μεταβλητών <code>d</code> τύπου <code>Date</code> και <code>t</code> τύπου <code>Time</code>	174
6.2	Η αναπαράσταση της μεταβλητής <code>dd</code> τύπου <code>DetailedDate</code>	177
6.3	Η αναπαράσταση της μεταβλητής <code>s</code> τύπου <code>Student</code>	181
6.4	Η αναπαράσταση της μεταβλητής <code>dd1</code> τύπου <code>DetailedDate</code>	192
6.5	Η αναπαράσταση των μεταβλητών <code>dd1</code> και <code>dd2</code> τύπου <code>DetailedPDate</code>	194
7.1	Η αφαιρετική εικόνα της μνήμης για τη μεταβλητή <code>ws</code> τύπου <code>WorkingStudent</code>	232
7.2	Η κλάση <code>B</code> <i>is a</i> <code>A</code> στη C++	234
7.3	Η αναπαράσταση μιας μεταβλητής <code>y</code> ορισμένη ως <code>Y y;</code>	238
8.1	Η εικόνα των “φοιτητών” της συνάρτησης <code>main</code>	318
8.2	Η ιεραρχία των κλάσεων του συστήματος Εισόδου/Εξόδου της C++	342
9.1	Διαδικασία μεταγλώττισης και εκτέλεσης προγραμμάτων Java	373
9.2	Μεταβλητές αναφοράς και αντικείμενα	383
9.3	Ένα παράδειγμα ιεραρχίας κλάσεων	399
9.4	Η ιεραρχία κλάσεων που χρησιμοποιεί η κλάση <code>Meal</code>	410
9.5	Η ιεραρχία των τύπων διεπαφής και της κλάσης του παραδείγματος	415
11.1	Η τρέχουσα κατάσταση της αλυσίδας	562
11.2	Η κατάσταση της αλυσίδας μετά το χτύπημα του τύμπανου	563
11.3	Παράδειγμα για $N = 5$, κατά το οποίο δημιουργήθηκε ένα αλκάνιο (CH_4), ένα αλκένιο (C_2H_4) και ένα αλκίνιο (C_2H_2)	570
11.4	Αναπαράσταση ενός αλκένιου (C_2H_4)	571
11.5	Ολοκλήρωση ενημέρωσης του γονέα <code>P0</code> από καθηγητή και επανατοποθέτησή του στην ουρά	572
11.6	Ένα παράδειγμα ακολουθίας ακολουθιών δύο τμημάτων	572
11.7	Η δομή δυαδικού δένδρου για τη φύλαξη των κωδικών των φοιτητών	595
11.8	Οι λειτουργίες σε buffers μεγέθους 5	604

ΠΙΝΑΚΑΣ ΣΥΝΤΟΜΕΥΣΕΩΝ - ΑΚΡΩΝΥΜΙΩΝ

ADT	Abstract Data Type
API	Application Programming Interface
AI	Artificial Intelligence
CIL	Common Intermediate Language
CLI	Common Language Infrastructure
CLR	Common Language Runtime
CORBA	Common Object Request Broker Architecture
CTS	Common Type System
DL	Description Logic
FIFO	First In First Out
IDE	Integrated Development Environment
IoT	Internet of Things
JVM	Java Virtual Machine
KDM	Knowledge Discovery Metamodel
KR	Knowledge Representation
LIFO	Last In First Out
LINQ	Language Integrated Query
MSIL	Microsoft Intermediate Language
NCC	Norwegian Computing Center
OMG	Object Management Group
OMT	Object-Modeling Technique
OOM	Object Oriented Model
OOP	Object Oriented Programming
OOSE	Object-Oriented Software Engineering
OR	Operations Research
PDA	Personal Digital Assistant
RMI	Remote Method Invocation
STL	Standard Template Library
UML	Unified Modeling Language
XMI	XML Metadata Interchange
ΑΓ	Αναπαράσταση Γνώσης
ΑΤΔ	Αφηρημένος Τύπος Δεδομένων
ΠΛ	Περιγραφική Λογική
ΣΠ	Συναρτησιακός Προγραμματισμός
TN	Τεχνητή Νοημοσύνη

ΠΡΟΛΟΓΟΣ

Σκεφτόμουν καιρό να γράψω ένα βιβλίο σχετικό με το θέμα αυτό, για την ακρίβεια από το 2007. Τη χρονιά εκείνη, το (σχεδόν) ομότιτλο μάθημα του Τμήματος Πληροφορικής και Τηλεπικοινωνιών του Εθνικού και Καποδιστριακού Πανεπιστημίου Αθηνών είχε πάρει μια ολοκληρωμένη μορφή και οι φοιτητές του εξέφρασαν τη συμβολή του στην εκπαίδευσή τους. Παρότι συνοδευόταν από ένα εγχειρίδιο, το οποίο περιείχε όλα τα προγράμματα που αναπτύχθηκαν στο πλαίσιο του μαθήματος και χρησιμοποιούνταν στο μάθημα, ο σχολιασμός και οι όποιες αναφορές στη θεωρία είχαν επιγραμματική μορφή, κάτι που καθιστούσε αναγκαίο ένα συμπληρωματικό σύγγραμμα με τη μορφή βιβλίου. Στο βιβλίο αυτό, λοιπόν, γίνεται μια πρώτη προσπάθεια με στόχο τη συλλογή και καταγραφή του υλικού αυτού (παραδείγματα, ασκήσεις, εργασίες, θέματα εξετάσεων), καθώς και για μία αποτύπωση της προσέγγισης που ακολουθείται για την ένταξη των φοιτητών στον αντικειμενοστραφή προγραμματισμό στο πλαίσιο του μαθήματος.

Το μάθημα αυτό εντάχθηκε στο Προπτυχιακό Πρόγραμμα Σπουδών του εν λόγω Τμήματος το ακαδημαϊκό έτος 2002-2003. Η απόφαση ήταν να χρησιμοποιηθεί, ως γλώσσα αντικειμενοστραφούς προγραμματισμού, η γλώσσα C++. Καθώς η διδασκαλία του μαθήματος ανατέθηκε σε εμένα, το πρώτο μου μέλημα ήταν να θέσω τον στόχο του και να σχεδιάσω τη μορφή του. Το περιεχόμενο του εμπλουτιζόταν χρόνο με τον χρόνο όλο και περισσότερο. Η όλη προσέγγιση βασιζόταν σε μία σκέψη: πώς θα ήθελα να μου είχαν παρουσιάσει τον αντικειμενοστραφή προγραμματισμό στην πρώτη μου επαφή μαζί του. Η προηγούμενη προγραμματιστική εμπειρία μου ήταν κτισμένη από γλώσσες διαδικαστικού προγραμματισμού, όπως FORTRAN, Cobol, Pascal και C, καθώς και γλώσσες δηλωτικού προγραμματισμού — λογικού και συναρτησιακού. Η συμβολή αυτών των ειδών προγραμματισμού ήταν εμφανής. Αντίθετα, δεν μου ήταν καθόλου καθαρό σε τι θα συνεισέφερε ο αντικειμενοστραφής προγραμματισμός. Στα νεανικά μου χρόνια, όμως, συμμετείχα στην ανάπτυξη μιας εφαρμογής σε μια πλατφόρμα με χρήση της C++, όπου συνέβησαν τα εξής:

1. Η εφαρμογή ήταν περίπλοκη τόσο αλγοριθμικά, όσο και από πλευράς χαρακτηριστικών των δεδομένων (οντοτήτων) της — αφορούσε τον χρονοπρογραμματισμό πληρωμάτων αεροπορικής εταιρείας.
2. Οι συνεργάτες που θα παρείχαν την πλατφόρμα ανάπτυξης της εφαρμογής μάς διέθεσαν τον πηγαίο κώδικά της σε C++, ο οποίος ήταν εξαιρετικά καλογραμμένος.

Όλα, τότε, απέκτησαν νόημα. Σε περιπτώσεις όπως αυτή, ο αντικειμενοστραφής προγραμματισμός έδειξε τη δύναμή του. Επέτρεπε τον διαμοιρασμό των (υπο)προβλημάτων και την ανάπτυξη κώδικα για την επίλυσή τους σε διαφορετικούς προγραμματιστές. Έτσι, έκανε την υλοποίηση γρηγορότερη και ασφαλέστερη. Σε αυτό συνέβαλε και η άμεση απεικόνιση των οντοτήτων των δεδομένων σε κλάσεις. Η χρήση των κλάσεων έκανε τον κώδικα πιο ευανάγνωστο και ευκολότερο στη συντήρησή του. Ο στόχος του μαθήματος, λοιπόν, ήταν

να αναδείξει τη δύναμη αυτή, ξεκινώντας από την περιγραφή των αντικειμενοστραφών εννοιών ανεξαρτήτως γλώσσας και καταλήγοντας στην ανάπτυξη κώδικα όπου ο αντικειμενοστραφής προγραμματισμός κάνει τη διαφορά. Παρότι στη διδασκαλία χρησιμοποιούνταν αρχικά μόνο η C++, από το 2014 εντάχθηκαν στην ύλη του μαθήματος και αντικειμενοστραφή στοιχεία της γλώσσας Java, προκειμένου να αναδειχθεί η ομαλή ένταξη σε διαφορετική γλώσσα του αντικειμενοστραφούς πλαισίου.

Το βιβλίο αυτό βασίζεται στο υλικό του παραπάνω μαθήματος. Έχει ως στόχο να παρουσιάσει τον αντικειμενοστραφή προγραμματισμό, εστιάζοντας στις αρχές και τις βασικές έννοιές του, καθώς και τα γλωσσικά στοιχεία που μια γλώσσα προγραμματισμού χρειάζεται να παρέχει για να μπορεί να χαρακτηριστεί ως αντικειμενοστραφής. Αυτό θα μπορούσε, ίσως, να γίνει ανεξάρτητα από συγκεκριμένη υπαρκτή γλώσσα προγραμματισμού. Τότε, όμως, η παρουσίαση των εννοιών θα ήταν λιγότερο εφαρμόσιμη και, κατά συνέπεια, λιγότερο ελκυστική. Έτσι, μετά τα πρώτα κεφάλαια στα οποία θα γίνει μια εκτενής, αλλά αφαιρετική, παρουσίαση στα αντικειμενοστραφή χαρακτηριστικά και τη χρησιμότητά τους, όλες οι έννοιες θα παρουσιάζονται με χρήση της γλώσσας C++.

Πρέπει, όμως, να γίνει σαφές από την αρχή ότι το βιβλίο δεν θα ικανοποιήσει κάποιον που ενδιαφέρεται κυρίως για τη γλώσσα C++. Δεν θα την καλύψει πλήρως. Υπάρχουν εξαιρετικά βιβλία για τη γλώσσα και εκτενές υλικό στο διαδίκτυο που μπορεί να χρησιμοποιηθεί ως αναφορά. Πέρα από οποιαδήποτε άλλη παράληψη στην παρουσίαση των λεπτομερειών της C++, πρέπει να τονιστεί το εξής. Η γλώσσα αυτή, εκτός από τον αντικειμενοστραφή και τον διαδικαστικό προγραμματισμό, υποστηρίζει και άλλα είδη προγραμματισμού, τα οποία είτε θα παρουσιάσουμε εν συντομία, π.χ. γλωσσικά στοιχεία που αφορούν προγραμματισμό γενικεύσεων, είτε θα παραλείψουμε, π.χ. γλωσσικά στοιχεία που αφορούν συναρτησιακό προγραμματισμό. Από την άλλη μεριά, όμως, παρουσιάζοντας τα στοιχεία τα οποία μας ενδιαφέρουν, συχνά θα προχωράμε σε μεγάλη λεπτομέρεια της συμπεριφοράς της C++, φτάνοντας ακόμα και σε επίπεδο (αφαιρετικής) αναπαράστασης των δεδομένων στη μνήμη, προκειμένου ο αναγνώστης να έχει πλήρη επίγνωση της συμπεριφοράς των προγραμμάτων του.

Στο πλαίσιο της αποσύνδεσης των εννοιών από συγκεκριμένη γλώσσα, θα γίνει μια περιγραφή των γλωσσικών στοιχείων της γλώσσας Java όσον αφορά τον αντικειμενοστραφή χαρακτήρα της, καθώς και μια σύντομη αναφορά στα σχετικά στοιχεία από άλλες γλώσσες αντικειμενοστραφούς προγραμματισμού που είναι διαδεδομένες στις μέρες μας. Ο στόχος είναι, έχοντας μελετήσει το περιεχόμενο του βιβλίου, να μπορεί κάποιος να ενταχθεί εύκολα σε οποιαδήποτε άλλη γλώσσα αντικειμενοστραφούς προγραμματισμού, εστιάζοντας πλέον στα στοιχεία εκείνα που τη διαφοροποιούν από τις άλλες γλώσσες του είδους. Όπως κάθε είδος προγραμματισμού έρχεται να διευκολύνει την ανάπτυξη εφαρμογών με συγκεκριμένα χαρακτηριστικά, έτσι και η κάθε γλώσσα μέσα στο ίδιο είδος έρχεται να προσφέρει κάτι ιδιαίτερο που την κάνει προτιμότερη από άλλες του είδους της για κάποια συγκεκριμένα χαρακτηριστικά εφαρμογών. Η ελπίδα είναι ότι μετά τη μελέτη του βιβλίου, ένας προγραμματιστής να μπορεί να εξοικειώνεται εύκολα με τα αντικειμενοστραφή στοιχεία μιας νέας γι' αυτόν γλώσσας αντικειμενοστραφούς προγραμματισμού, ώστε να εστιάζει τη μελέτη του στα στοιχεία εκείνα της γλώσσας που τη διαφοροποιούν από τις αντικειμενοστραφείς γλώσσες που ήδη ήξερε.

Στο τελευταίο κεφάλαιο του βιβλίου περιλαμβάνονται μια σειρά από ασκήσεις και εργασίες που ενδείκνυται να αναπτυχθούν σε αντικειμενοστραφές περιβάλλον. Το τελικό ζητούμενο είναι η επιτυχής αντιμετώπιση των ασκήσεων και κυρίως των εργασιών αυτών. Παρότι προτείνεται η χρήση της C++, δεν είναι απαραίτητο να χρησιμοποιηθεί η γλώσσα αυτή. Εδώ να συμπληρώσω ότι υπάρχει επιπλέον υλικό το οποίο αναπτύχθηκε στο πλαίσιο του μαθήματος και το οποίο εκτιμώ τα μέγιστα, αλλά καθότι σε αυτό είχαν συμβάλει και συνεργάτες, δεν συμπεριλαμβάνεται στο βιβλίο αυτό. Είναι, όμως, διαθέσιμο στο διαδίκτυο.

Το βιβλίο απευθύνεται κυρίως σε προπτυχιακούς φοιτητές, αλλά και σε κάθε πληροφορικό που θέλει να μυηθεί στον αντικειμενοστραφή προγραμματισμό. Θεωρούμε ότι ο αναγνώστης του είναι ήδη εξοικειωμένος με τον διαδικαστικό προγραμματισμό. Για παράδειγμα, έχει ήδη ολοκληρώσει ένα εισαγωγικό μάθημα προγραμματισμού με μια γλώσσα διαδικαστικού προγραμματισμού, κατά προτίμηση τη C. Παρ' όλα αυτά, ένα από τα αρχικά κεφάλαια είναι αφιερωμένο σε μια γενική παρουσίαση των γλωσσικών στοιχείων του διαδικαστικού προγραμματισμού, με αναφορά στη C++, και εστίαση σε αυτά τα χαρακτηριστικά της γλώσσας που θα μας είναι απαραίτητα στη συνέχεια.

Μερικές οδηγίες, τώρα, για το πώς προτείνεται να μελετηθεί το βιβλίο ή να χρησιμοποιηθεί σε μάθημα. Τα κεφάλαια καλύπτουν το υλικό των διαλέξεων. Μετά τη μελέτη τους, ο αναγνώστης καλείται να “τρέξει” όλα τα προγράμματα και να διασφαλίσει ότι κατανοεί πλήρως τη συμπεριφορά τους. Το επόμενο βήμα είναι να κάνει τις ασκήσεις που συνοδεύουν το κεφάλαιο. Οι ασκήσεις αυτές βοηθούν στην κατανόηση των εννοιών και στην εξάσκηση με τις γλωσσικές δομές που παρουσιάστηκαν στο κεφάλαιο αυτό. Από μόνες τους, όμως, δεν είναι ικανές να αναδείξουν την αξία του αντικειμενοστραφούς προγραμματισμού. Αυτό γίνεται με τις ασκήσεις που περιέχονται στο τελευταίο κεφάλαιο του βιβλίου. Αυτές αφορούν στην ανάπτυξη κώδικα που θα μπορούσε να αποτελέσει είτε υλικό εργαστηρίου είτε παραδοτέες εργασίες. Σε κάθε περίπτωση, η ανάπτυξη κώδικα καλό είναι να γίνεται υπό καθοδήγηση και επίβλεψη στο πλαίσιο του μαθήματος, ώστε να υπάρχει κατεύθυνση όσον αφορά τις σχεδιαστικές επιλογές. Οι εργασίες που προτείνονται στο τελευταίο κεφάλαιο έχουν τέτοια χαρακτηριστικά, ώστε να κάνουν τον αντικειμενοστραφή προγραμματισμό απαραίτητο: απαίτηση για εκτενέστερο κώδικα και περίπλοκες οντότητες. Προτείνεται η εκπόνησή τους από δύο φοιτητές, ώστε να γίνεται διαχωρισμός του κώδικα που πρόκειται να αναπτυχθεί. Οι ασκήσεις ανάπτυξης κώδικα έχουν αρκετή ασάφεια, που να επιτρέπει ευελιξία, και τονίζονται μόνο οι προδιαγραφές εκείνες που αντιμετωπίζονται κομψά από τον αντικειμενοστραφή προγραμματισμό.

Εδώ πρέπει να πω δύο λόγια για τη μετάφραση του όρου “Object Oriented Programming”. Στο βιβλίο έχει υιοθετηθεί η μετάφραση “Αντικειμενοστραφής Προγραμματισμός” μια που, όπως προανέφερα, το υλικό του βασίζεται στο μάθημα του Τμήματος Πληροφορικής και Τηλεπικοινωνιών του Ε.Κ.Π.Α. που έχει τον τίτλο αυτό. Στον όρο έχουν αποδοθεί και άλλες μεταφράσεις. Μια εξ αυτών, πολύ διαδεδομένη στο παρελθόν, είναι το “Αντικειμενοστρεφής Προγραμματισμός” που “ακουγόταν” πιο σωστό. Πολλοί δε αναρωτιόντουσαν, μεταξύ των οποίων και εγώ, ποια από τις δύο μεταφράσεις είναι η σωστή. Στην προσπάθειά μου να πάρω μια απάντηση, κατέφυγα σε μια έμπειρη φιλόλογο της Αγγλικής Γλώσσας, με μεγάλη, όμως, συμμετοχή σε γλωσσολογική ομάδα της Ελληνικής Γλώσσας του Πανεπιστημίου Αθηνών, στο παρελθόν. Εκείνη, αφού πρώτα μου ζήτησε να της εξηγήσω σε τι αναφέρεται ο όρος αυτός για το επιστημονικό μας πεδίο — τι σημαίνει —, συμπέρανε ότι μεταφράζεται ορθότερα περιφραστικά με την έκφραση “Προγραμματισμός προσανατολισμένος σε αντικείμενα” και σίγουρα όχι με κάποιον όρο που να προκύπτει με χρήση του ρήματος “στρέφω”. Ο όρος “Oriented” δηλώνει προσανατολισμό και όχι στροφή. Πράγματι, μελετώντας κάποια λεξικά που έχω στη διάθεσή μου, επιβεβαίωσα ότι το ρήμα “στρέφω” σημαίνει, κυρίως, αλλαγή κατεύθυνσης. Στις μέρες μας, σε κάποιες περιπτώσεις, δηλώνει κατεύθυνση αλλά μόνο μεταφορικά. Όσον δε αφορά το γραμματικώς ορθό, πάλι από τα λεξικά που έχω στη διάθεσή μου και χωρίς να θεωρώ τον εαυτό μου ειδικό σε καμία περίπτωση, είδα ότι τόσο το θέμα στρέφ- όσο και το θέμα στράφ- είναι ενεστωτικά (Λεξικό Δ. Δημητράκου, 1956) — με το στράφ- πιο “αρχαϊκό” —, καθώς και ότι επίθετα υπάρχουν και με τα δύο θέματα (στρέφ- -> εσωστρεφής, στράφ- -> αστραφής). Δεδομένου δε ότι πρόκειται για παθητική μετοχή (ο προγραμματισμός “στρέφεται” στα αντικείμενα, δεν τα “στρέφει”), γραμματικά θα έπρεπε να ακολουθήσουμε παθητική μετοχή, “Αντικειμενοστρεφόμενος” (= στρέφεται διαρκώς) ή “Αντικειμενοστραμμένος” (= στράφηκε — και κατευθύνεται —, π.χ. “Τα βλέμματα όλων είναι στραμμένα στον άνθρωπο στη σκηνή”). Διατήρησα, λοιπόν, τη μετάφραση που “κληρονόμησα” στο Τμήμα μου, μια που θεωρούσα άσκοπο να αλλάξει κάτι λανθασμένο σε κάτι που επίσης δεν είναι σωστό. Προκειμένου να επιτύχουμε μια ορθότερη νοηματικά μετάφραση του όρου, ίσως να χρειαστεί να ξεφύγουμε από μετάφραση της λέξης “oriented”. Για παράδειγμα, οι όροι “όντο(τητο)κεντρικός προγραμματισμός” ή “αντικειμενοκεντρικός προγραμματισμός” θα μπορούσαν να αποδώσουν το νόημα του όρου “Object Oriented Programming” πιο σωστά. Προτιμώ, όμως, να αφήσω τη λύση σε πιο ειδικούς σε θέματα ελληνικής και αγγλικής γλώσσας. Εξάλλου, πολλές μεταφράσεις όρων πληροφορικής είναι ατυχείς. Στο βιβλίο αυτό διατηρώ τη μετάφραση “Αντικειμενοστραφής Προγραμματισμός”, που συχνά, για λόγους συντομίας, θα αναφέρεται απλά με το ακρωνύμιο του αγγλικού όρου, “Object Oriented Programming”, OOP.

Κλείνοντας τον Πρόλογο, θα ήθελα να εκφράσω τις θερμές ευχαριστίες μου σε όλους τους φοιτητές του Τμήματος Πληροφορικής και Τηλεπικοινωνιών του Ε.Κ.Π.Α. που εκτίμησαν και αγάπησαν το μάθημα “Αντικειμενοστραφής Προγραμματισμός” του προπτυχιακού προγράμματος σπουδών τους. Ελπίζω ειλικρινά το μάθημα να βοήθησε την επαγγελματική τους πορεία, συμβάλλοντας στα εφόδια που χρειάζονται για την επιτυχία της. Ιδιαίτερα, ευχαριστώ εκείνους που όντας ακόμα προπτυχιακοί φοιτητές συμμετείχαν εθελοντικά

αλλά και εκείνους που στη συνέχεια, ως μεταπτυχιακοί φοιτητές, το στήριξαν ως συνεργάτες του. Θα σταθώ, όμως, σε κάποιους από αυτούς. Πρώτα θέλω να ευχαριστήσω από καρδιάς τον Νίκο Ποθητό, χρόνια πολύτιμο συνεργάτη του μαθήματος, και τον Μανόλη Πλατάκη. Εύχομαι κάθε νέος να έχει τη σύνεση, την κρίση και τον δυναμισμό τους. Δεν θα ξεχάσω την προσφορά τους ποτέ. Επίσης, ξεχωρίζω τον Γιώργο Καστρίνη, τον Εργκύς Ντόνα, τον Βασίλη-Μάριο Αναστασίου που για χρόνια στήριξαν το μάθημα, βοηθώντας τους φοιτητές να βρουν τον δρόμο τους μέσα σε αυτό. Για τον λόγο αυτόν, ευχαριστώ και τον Ορέστη Πολυχρονίου και τον Αλέξανδρο Καλτσουνίδη. Τελευταίο αφήνω τον εξαίρετο, από άποψη επιστημονικότητας, γνώσεων και ήθους, Στέφανο Σταμάτη: θεωρώ τον εαυτό μου προνομιούχο που έχω συνεργαστεί μαζί του. Να είστε καλά!

Ιζαμπώ Καράλη

ΕΙΣΑΓΩΓΗ ΣΤΟΝ ΑΝΤΙΚΕΙΜΕΝΟΣΤΡΑΦΗ ΠΡΟΓΡΑΜΜΑΤΙΣΜΟ

Σύνοψη

Στο κεφάλαιο αυτό, γίνεται μια γενική εισαγωγή στον αντικειμενοστραφή προγραμματισμό και τις κλάσεις. Παρουσιάζονται συνοπτικά οι έννοιες που συναντώνται σε αυτό το είδος προγραμματισμού, η ονοματολογία τους, οι μηχανισμοί που είναι απαραίτητοι για την υποστήριξή τους, καθώς και ιδιότητες που σχετίζονται με τη χρήση του. Επίσης, γίνεται συζήτηση σχετικά με τη χρήση του αντικειμενοστραφούς μοντέλου γενικότερα. Τέλος, δίνονται ιστορικά στοιχεία για τις γλώσσες αντικειμενοστραφούς προγραμματισμού.

Προαπαιτούμενη Γνώση

Για την κατανόηση του κεφαλαίου, ο αναγνώστης απαιτείται να έχει στοιχειώδεις γνώσεις προγραμματισμού και πληροφορικής.

1.1 Γενικά Στοιχεία για τον Αντικειμενοστραφή Προγραμματισμό

Ο αντικειμενοστραφής προγραμματισμός (object oriented programming - OOP) είναι ένα είδος προγραμματισμού (programming paradigm) που επιτρέπει στον προγραμματιστή να οργανώνει τον κώδικά του σε “κλάσεις” (classes) οι οποίες αντιστοιχούν σε οντότητες ή, γενικότερα, έννοιες της εφαρμογής του. Ένα παράδειγμα κλάσεων είναι οι κατηγορίες δεδομένων της εφαρμογής, όπως οι “αίθουσες”, οι “διδάσκοντες” κ.ά. σε μια εφαρμογή κατασκευής σχολικού ωρολογίου προγράμματος.

Οι κλάσεις μπορεί να συνδέονται μεταξύ τους με μια σχέση εξειδίκευσης. Για παράδειγμα, μια “αίθουσα εργαστηρίου” είναι μια ειδική κατηγορία “αίθουσας”. Ο OOP μας επιτρέπει να δηλώσουμε ρητά τη σχέση αυτή στο πρόγραμμά μας. Ορίζοντας τις κλάσεις του προγράμματος, αν μεταξύ δύο κλάσεων δηλώσουμε σχέση εξειδίκευσης, δεν χρειάζεται να επαναλάβουμε στον ορισμό της ειδικότερης κλάσης ό,τι έχει οριστεί στη γενικότερη, διότι “κληρονομείται” αυτόματα. Η ιδιότητα αυτή αναφέρεται ως “κληρονομικότητα” (inheritance) μεταξύ των κλάσεων.

Αφού οριστούν κλάσεις, ο προγραμματιστής μπορεί να δημιουργεί συγκεκριμένα στιγμιότυπα (instances) των κλάσεων αυτών μέσα στον κώδικά του, π.χ. την αίθουσα “A35” ως στιγμιότυπο “αίθουσας”. Στο πλαίσιο του ΟΟΡ, τα στιγμιότυπα αυτά ονομάζονται “αντικείμενα” (objects). Ο τρόπος με τον οποίο ο προγραμματιστής χειρίζεται τα αντικείμενα μιας κλάσης στο πρόγραμμά του καθορίζεται από τον ορισμό της κλάσης. Συγκεκριμένα, μέσα στον ορισμό μιας κλάσης καθορίζεται η “συμπεριφορά” (behavior) των αντικειμένων της, η οποία προκύπτει από ένα σύνολο “ενεργειών” που μπορούν να εφαρμοστούν στα αντικείμενα αυτά, π.χ. η ενέργεια “ανάθεση μαθήματος” για τα αντικείμενα της “αίθουσας”.

Η ανάγκη για τη δυνατότητα ορισμού κλάσεων από μια γλώσσα προγραμματισμού προέκυψε από τις απαιτήσεις εφαρμογών με περίπλοκες οντότητες. Συγκεκριμένα, επρόκειτο για εφαρμογές προσομοίωσης. Όμως, η τμηματοποίηση του κώδικα που επιτρέπουν οι κλάσεις διευρύναν τη χρήση του ΟΟΡ σε κάθε περίπτωση που απαιτείται ανάπτυξη τμηματοποιημένου ή/και εκτενούς κώδικα. Έτσι, καταλήξαμε στις μέρες μας σχεδόν όλες οι γλώσσες προγραμματισμού να προσφέρουν γλωσσικά στοιχεία ΟΟΡ.

Πλεονεκτήματα του Αντικειμενοστραφούς Προγραμματισμού

Συνοπτικά, οι κλάσεις επιτρέπουν

- απευθείας αναπαράσταση των οντοτήτων ή/και των εννοιών του προβλήματος προς επίλυση στην εφαρμογή και
- την κατάτμηση του κώδικα της εφαρμογής σε τμήματα, τις κλάσεις.

Κατά συνέπεια, τόσο η ανάπτυξη όσο και η συντήρηση της εφαρμογής γίνονται ευκολότερα. Η τμηματοποίηση επιτρέπει διαμοιρασμό του έργου της ανάπτυξης, καθώς και επαναχρησιμοποίηση τμημάτων κώδικα. Και τα δύο παραπάνω χαρακτηριστικά δε, οδηγούν στην παραγωγή πιο ευανάγνωστου κώδικα με ό,τι αυτό συνεπάγεται.

1.2 Διαδικαστικός και Αντικειμενοστραφής Προγραμματισμός

Συγκρίνοντας τον διαδικαστικό προγραμματισμό (procedural programming) με τον αντικειμενοστραφή (διαδικαστικό) προγραμματισμό, μπορούμε να πούμε το εξής. Στον παραδοσιακό διαδικαστικό προγραμματισμό, η έμφαση είναι στον σχεδιασμό των διαδικασιών που επιλύουν ένα πρόβλημα, προσδιορίζοντας την ακολουθία των βημάτων/ενεργειών (actions) που οδηγούν στην επίλυσή του. Οι διαδικασίες επιδρούν σε κάποια δεδομένα (εισόδου) και οι ενέργειες αποσκοπούν στον μετασχηματισμό των δεδομένων αυτών, ώστε να καταλήξουν στο επιθυμητό αποτέλεσμα. Η προσπάθεια, λοιπόν, εστιάζει στον σχεδιασμό και την υλοποίηση των διαδικασιών αυτών.

Στον ΟΟΡ, η εστίαση αλλάζει: η έμφαση στον σχεδιασμό δίνεται στις οντότητες της εφαρμογής, συνήθως τα δεδομένα. Αυτές τις συσχετίζουμε με μια συμπεριφορά που οφείλουν να ακολουθούν όλα τα στιγμιότυπά τους. Η συμπεριφορά αυτή υλοποιείται μέσω διαδικασιών.

Αρα, ενώ στον διαδικαστικό προγραμματισμό, σχεδιάζουμε και υλοποιούμε διαδικασίες (που επιδρούν σε δεδομένα), στον (διαδικαστικό) ΟΟΡ σχεδιάζουμε και υλοποιούμε κλάσεις (που συσχετίζονται με διαδικασίες).

1.2.1 Διαδικαστικός ή Αντικειμενοστραφής Προγραμματισμός;

Το τι είναι κατάλληλο να επιλέξουμε, γλώσσα διαδικαστικού προγραμματισμού (π.χ. C) ή γλώσσα με αντικειμενοστραφή χαρακτηριστικά (π.χ. C++), εξαρτάται από τη φύση του προβλήματος που έχουμε να αντιμετωπίσουμε. Πάρτε, για παράδειγμα, το εξής πρόβλημα: “Γράψτε ένα πρόγραμμα το οποίο να ταξινομεί N ακεραίους”. Εδώ τι έχουμε: η ταξινόμηση αποτελεί μια διαδικασία που βασίζεται σε μια συγκεκριμένη μέθοδο (αλγόριθμο) ταξινόμησης. Αυτή εφαρμόζεται σε κάποια δεδομένα, που είναι απλοί ακέραιοι στο παράδειγμα, και βασίζεται σε μια άλλη διαδικασία που πραγματοποιεί σύγκριση μεταξύ δύο ακεραίων.

Μια άλλη, πιο γενική, κατηγορία αλγοριθμικών προβλημάτων που χειρίζονται απλά δεδομένα είναι η επίλυση αριθμητικών προβλημάτων με χρήση αριθμητικών μεθόδων.

Και στα δύο παραπάνω παραδείγματα προβλημάτων, η έμφαση είναι στην υλοποίηση διαδικασιών οι οποίες εφαρμόζονται πάνω στα δεδομένα. Άρα, χρειαζόμαστε γλώσσες των οποίων οι γλωσσικές δομές (language constructs) διευκολύνουν την αναπαράσταση (υλοποίηση) διαδικασιών. Αν, μάλιστα, όπως στις παραπάνω περιπτώσεις, τα δεδομένα αναπαριστούνται άμεσα από τύπους της γλώσσας, δεν υπάρχει ανάγκη η γλώσσα που θα επιλέξουμε να παρέχει δυνατότητα ορισμού νέων τύπων, ούτε στοιχειωδώς.

Πάρτε, τώρα, ένα παράδειγμα μιας εφαρμογή γραφικών, π.χ. ένα απλοϊκό είδος κινουμένων σχεδίων. Εδώ τι έχουμε: έχουμε κάποιες οντότητες που τις συσχετίζουμε με μια συμπεριφορά. Ως συμπεριφορά, θεωρούμε ένα σύνολο διαδικασιών (στον αντικειμενοστραφή διαδικαστικό προγραμματισμό) που μπορούν να εκτελεστούν αν αυτές οι οντότητες “λάβουν κάποια μηνύματα”. Στο παράδειγμα των κινουμένων σχεδίων, έχουμε διάφορες οντότητες: “σκηνές” που περιέχουν “κινούμενα σχέδια” τα οποία έχουν κάποια γενική συμπεριφορά (π.χ. “μετατόπιση”, “στροφή”) αλλά μερικά έχουν και πιο εξειδικευμένη (π.χ. ένα είδος κινουμένων σχεδίων που να μπορεί επιπλέον να “εκτελεί άλμα” που το μετατοπίζει κάθετα).

Ανάλογο παράδειγμα είναι οι εφαρμογές προσομοίωσης. Για παράδειγμα, θεωρήστε μια προσομοίωση διεξαγωγής αγώνων στίβου προκειμένου να καθοριστούν και να καλυφθούν οι ανάγκες σε υποστηρικτικό προσωπικό. Υπάρχουν τα “αθλήματα”, οι “αθλητές” που συμμετέχουν, ο “εξοπλισμός” τους που χρειάζεται επίβλεψη από το “προσωπικό” κ.ά.

Σε αυτά τα προβλήματα, τώρα, η έμφαση είναι στις οντότητες. Οι διαδικασίες σχετίζονται με αυτές και οφείλουν να τις συνοδεύουν. Άρα, οι γλώσσες που χρειαζόμαστε για την υλοποίησή τους οφείλουν να προσφέρουν γλωσσικές δομές οι οποίες να επιτρέπουν ορισμό οντοτήτων και σύνδεσή τους με τις διαδικασίες που αντιστοιχούν στη συμπεριφορά τους. Επίσης, πρέπει να επιτρέπουν τον συσχετισμό μεταξύ γενικότερων και ειδικότερων οντοτήτων και να υποστηρίζουν τον μηχανισμό που είναι αναγκαίος για την κληρονομικότητα.

Γενικότερα, όμως, σε προβλήματα που η αντιμετώπισή τους απαιτεί διάφορες δομές δεδομένων, όπως στοίβες, ουρές κλπ., και μάλιστα δημιουργία παραπάνω του ενός στιγμιότυπα από την κάθε μία από αυτές, η χρήση του OOP διευκολύνει πολύ. Ως τέτοια παραδείγματα, θεωρήστε τα προβλήματα που συναντώνται στην υλοποίηση λειτουργικών συστημάτων, επεξεργαστών κειμένου κ.ά.

Εδώ πρέπει να επισημάνουμε και κάτι ακόμα. Οι οντότητες έχουν μια εξωτερική συμπεριφορά αλλά δεν επιβάλλουν συγκεκριμένη εσωτερική αναπαράσταση. Καλό, λοιπόν, είναι ο μηχανισμός ορισμού των οντοτήτων να επιτρέπει διάκριση του τι είναι χρησιμοποιήσιμο από τον έξω κόσμο και του τι αποτελεί απόφαση του προγραμματιστή και άρα είναι λεπτομέρεια υλοποίησης. Το δεύτερο πρέπει να “εγκλωβίζεται” και να μην είναι προσβάσιμο από το υπόλοιπο πρόγραμμα. Ο μηχανισμός του εγκλωβισμού (encapsulation) επιτυγχάνει αυτήν τη διάκριση. Έτσι γίνεται δυνατό ανά πάσα στιγμή να αλλάξουμε την υλοποίηση μιας οντότητας χωρίς να αλλάξουμε τίποτα στο υπόλοιπο πρόγραμμα, αρκεί να διατηρήσουμε τη διεπαφή (interface) της συμπεριφοράς της. Ο βασικός μηχανισμός που απαιτείται από μια γλώσσα για να πετύχουμε το προηγούμενο επιτρέπει “απόκρυψη πληροφορίας” (information hiding), που είναι γενικότερα χρήσιμη στην ανάπτυξη κώδικα, κυρίως αν αυτός είναι εκτενής. Οπότε, καταλαβαίνουμε ότι η χρήση του OOP έχει επεκταθεί στην ανάπτυξη εκτενούς κώδικα, διότι επιτρέπει να “πακετάρεται” μια ολόκληρη διαδικασία, ή ένα σύνολο διαδικασιών σε μια κλάση, χρησιμοποιώντας την σαν μηχανισμό απόκρυψης πληροφορίας (π.χ. η προσέγγιση της Java).

Τέλος, λόγω της απόκρυψης πληροφορίας, της καθαρής έκφρασης προδιαγραφών (specifications) μέσω των διεπαφών και της δυνατότητας επαναχρησιμοποίησης (reusability) που επιτυγχάνεται με τη χρήση γλωσσών OOP, οι γλώσσες αυτές προσφέρονται για ανάπτυξη βιβλιοθηκών λογισμικού και επέκτασής τους. Σε αυτό συμβάλλει και η δυνατότητα παροχής “περιβάλλοντος ονομάτων” στα πλαίσια των κλάσεων. Μέσω αυτών, τα ονόματα που δηλώνονται σε μια κλάση συνδέονται με την κλάση επιτρέποντας επιλεκτική χρήση τους.

1.2.2 Βασικά Χαρακτηριστικά του Αντικειμενοστραφούς Μοντέλου στον Προγραμματισμό

Στα προηγούμενα μιλούσαμε για την ανάγκη ορισμού νέων τύπων, προκειμένου να αναπαραστήσουμε τις οντότητες της εφαρμογής που θέλουμε να αναπτύξουμε. Ας πάρουμε τώρα, για παράδειγμα, μια εφαρμογή

γραφικών που σχετίζεται με δισδιάστατα γεωμετρικά σχήματα. Στην εφαρμογή αυτή, έστω ότι έχουμε, π.χ., την έννοια του “Κύκλου” και την έννοια του “Τρίγωνου”. Έστω ότι, τόσο την έννοια του “Κύκλου”, όσο και την έννοια του “Τρίγωνου”, τις συσχετίζουμε

- με το χαρακτηριστικό “χρώμα”, που αναφέρεται στο χρώμα με το οποίο είναι χρωματισμένη η επιφάνειά τους,
- με τον “χρωματισμό” τους όπου δίνουμε ή αλλάζουμε το χρώμα αυτό,
- με τον “υπολογισμό του εμβαδού τους” και
- με την πράξη της “περιστροφής” γύρω από το κέντρο τους (κατά δεδομένες μοίρες).

Έστω ότι θέλουμε να ορίσουμε διάφορα τρίγωνα και κύκλους στην εφαρμογή μας. Άρα, κατ’ αρχήν, χρειαζόμαστε μια γλώσσα που να προσφέρει αφαίρεση στα δεδομένα (data abstraction) για να ορίσουμε τους νέους τύπους, “Κύκλος” και “Τρίγωνο” που να αντιστοιχούν στις έννοιες αυτές συσχετίζοντάς τις με τα παραπάνω, ώστε να μπορούμε, στη συνέχεια, να ορίσουμε μεταβλητές (στιγμιότυπα) των τύπων αυτών.

Παρατηρώντας, όμως, πιο προσεκτικά τις έννοιες “Κύκλος” και “Τρίγωνο”, διαπιστώνουμε ότι και οι δύο αποτελούν ειδικές περιπτώσεις της έννοιας των “Δισδιάστατων γεωμετρικών σχημάτων”. Μάλιστα, ως τέτοιες σχετίζονται με τα χαρακτηριστικά και τις λειτουργίες που αναφέρθηκαν. Ιδανικά, λοιπόν, θα θέλαμε να μπορούμε να ορίσουμε έναν τύπο “Δισδιάστατο γεωμετρικό σχήμα” τον οποίο να συσχετίσουμε με το χαρακτηριστικό “χρώμα” και τις λειτουργίες “χρωματισμός”, “υπολογισμός εμβαδού” και “περιστροφή” και, στη συνέχεια, να ορίσουμε τους τύπους “Τρίγωνο” και “Κύκλος”, δηλώνοντάς τους ως εξειδικεύσεις του τύπου “Δισδιάστατο γεωμετρικό σχήμα”. Στην περίπτωση αυτή, δεν θα πρέπει να χρειάζεται να επαναλάβουμε τα κοινά χαρακτηριστικά και λειτουργίες, παρά να εστιάζουμε μόνο στην ειδοποιό διαφορά τους που τους κάνει να διαφοροποιούνται από τον πιο γενικό τύπο. Στην περίπτωση του παραδείγματος, στον ορισμό των τύπων “Κύκλος” και “Τρίγωνο” πρέπει να προσδιορίσουμε τον τύπο υπολογισμού του εμβαδού τους και τη διαδικασία υπολογισμού του αποτελέσματος της περιστροφής τους.

Η παραπάνω δυνατότητα προσφέρεται από μια γλώσσα προγραμματισμού που επιτρέπει να προσδιοριστεί συσχέτιση γενικού-ειδικότερου μεταξύ τύπων, με σκοπό να μην επαναλαμβάνεται ό,τι έχει οριστεί στον γενικό τύπο, αλλά να κληροδοτείται στον ειδικότερο μέσω ενός μηχανισμού κληρονομικότητας (*inheritance*). Όταν μια γλώσσα προγραμματισμού δίνει τη δυνατότητα του ορισμού νέων τύπων, προσφέροντας αφαίρεση στα δεδομένα, και υποστηρίζει κληρονομικότητα, τότε μιλάμε για μια γλώσσα ΟΟΡ. Οι τύποι που ορίζει ο χρήστης, πλέον, λέγονται κλάσεις και τα στιγμιότυπα/μεταβλητές τους λέγονται αντικείμενα.

Συνοπτικά, το αντικειμενοστραφές μοντέλο σχετίζεται με τη δυνατότητα ορισμού κλάσεων και με την ιδιότητα/μηχανισμό της κληρονομικότητας. Αποδίδοντας κάπως ελεύθερα αυτό που ο ίδιος ο Bjarne Stroustrup, ο δημιουργός της C++, αναφέρει στη Δεύτερη Έκδοση του βιβλίου του “Η Γλώσσα Προγραμματισμού C++” [1], όταν πρόκειται να αναπτύξουμε μια εφαρμογή σε μια γλώσσα αντικειμενοστραφούς προγραμματισμού, πρέπει να αποφασίσουμε ποιες κλάσεις χρειαζόμαστε, να καθορίσουμε το πλήρες σύνολο λειτουργιών για κάθε κλάση και να εντοπίσουμε τα κοινά χαρακτηριστικά και λειτουργίες μεταξύ κλάσεων. Στη συνέχεια, ό,τι κοινό εντοπίστηκε πρέπει να δηλωθεί ρητά, συσχετίζοντάς το με νέες κλάσεις. Ο συσχετισμός του με τις πρωτογενείς κλάσεις γίνεται εφικτός με τη χρήση κληρονομικότητας.

Κλείνοντας την παράγραφο αυτή, προκειμένου να γίνει μια σύνδεση μεταξύ του αντικειμενοστραφούς μοντέλου και των Αφηρημένων Τύπων Δεδομένων (ΑΤΔ) (Abstract Data Types - ADTs), θα παραθέσουμε μια έκφραση από το βιβλίο του Bertrand Meyer [2] που αναφέρεται στην αντικειμενοστραφή σχεδίαση λογισμικού.

“Object Oriented Design is the construction of software systems as structured collections of abstract data type implementations.”

Αποδίδοντας το νόημα της παραπάνω έκφρασης, μπορούμε να περιγράψουμε την αντικειμενοστραφή σχεδίαση ως την κατασκευή συστημάτων λογισμικού οργανωμένων ως σύνολα από υλοποιήσεις ΑΤΔ. Πάνω σε

αυτό, λοιπόν, μπορούμε να κάνουμε την εξής παρατήρηση: οι ΑΤΔ υφίστανται ανεξάρτητα από υλοποίηση, ενώ αναφερόμενοι σε κλάσεις, θεωρούμε ένα σύστημα λογισμικού το οποίο, τελικά, τους υλοποιεί. Επιγραμματικά:

ΑΤΔ: Αφαιρετική περιγραφή τύπων μέσω των ιδιοτήτων των λειτουργιών τους και όχι λεπτομερειών της αναπαράστασής τους.

Κλάσεις: Συγκεκριμένη μοντελοποίηση και υλοποίηση.

1.3 Ονοματολογία του OOP, Μηχανισμοί και Ιδιότητες των Γλωσσών του

Αφού μιλήσαμε γενικά για τον OOP και ποιες ανάγκες ήλθε να καλύψει, προχωράμε με μια παρουσίαση βασικών εννοιών και ιδιοτήτων που σχετίζονται με αυτόν, καθώς και με ένα σύνολο μηχανισμών που είναι απαραίτητοι σε μια γλώσσα προγραμματισμού για την υποστήριξή του. Καθότι συναντώνται διάφοροι όροι στο αντικειμενοστραφές μοντέλο που, πολλές φορές, αναφέρονται στην ίδια ή παρόμοια έννοια, θα προσπαθήσουμε να κάνουμε τις κατάλληλες συνδέσεις. Έχοντας μελετήσει την παράγραφο αυτή, ο στόχος είναι ο αναγνώστης να έχει έλθει σε μια πρώτη επαφή με τα ονόματα και τις έννοιες που συνήθως συναντώνται στις αντικειμενοστραφείς γλώσσες. Η ελπίδα είναι να γίνουν κτήμα του στη συνέχεια του βιβλίου, αρχικά με τη χρήση της C++.

Κατά συνέπεια, κρίνεται κρίσιμο να δοθεί προσοχή στην παράγραφο αυτή, ακόμα και αν, στην παρούσα φάση, οι όροι δεν μπορούν να γίνουν πλήρως κατανοητοί. Να συμπληρώσουμε, επίσης, ότι στο αντικειμενοστραφές μοντέλο συναντώνται και άλλοι όροι πέρα από αυτούς που συζητάμε εδώ οι οποίοι, όμως, αφορούν κυρίως τη φάση της σχεδίασης λογισμικού και όχι προγραμματισμό, οπότε, για λόγους συντομίας και απλότητας, τους παραλείπουμε. Θα ήταν απαραίτητο να συζητούνται σε ένα βιβλίο του οποίου η έμφαση είναι στις αρχές της σχεδίασης των συστημάτων λογισμικού με βάση το αντικειμενοστραφές μοντέλο.

1.3.1 Έννοιες του OOP

Αν θέλαμε να χρησιμοποιήσουμε μια μόνο πρόταση που να συνδέει τις πιο βασικές έννοιες του OOP, θα λέγαμε ότι στον αντικειμενοστραφή προγραμματισμό *ορίζουμε κλάσεις*, όπου από κάποιες στη συνέχεια θα κατασκευάσουμε *αντικείμενα* και τις οποίες κλάσεις μπορούμε να τις συνδέσουμε μεταξύ τους μέσω *κληρονομικότητας*.

Να ξεκινήσουμε περιγράφοντας την έννοια των κλάσεων ή την έννοια των αντικειμένων; Με βάση τη συζήτηση που έχει προηγηθεί, δείχνει πιο κατάλληλο να ξεκινήσουμε με τις κλάσεις παρότι σε πολλά, κυρίως παλαιότερα, συγγράμματα η περιγραφή ξεκινά από τα αντικείμενα.

1.3.1.1 Κλάσεις

Τι είναι, λοιπόν, μια κλάση; Η αλήθεια είναι ότι μπορούμε να την περιγράψουμε με διάφορους τρόπους.

- Με βάση τη συζήτηση που έχει προηγηθεί, μια κλάση αντιπροσωπεύει ένα σύνολο *στοιχείων* που χαρακτηρίζονται από *κοινή συμπεριφορά* και *κοινή δομή*.
- Αν θέλουμε η περιγραφή να συνδέει την έννοια της κλάσης με την έννοια του τύπου δεδομένων, μπορούμε να πούμε ότι μια κλάση είναι η *υλοποίηση ενός νέου τύπου*.
- Μπορούμε ακόμα να πούμε ότι είναι η *μοντελοποίηση μιας οντότητας ή έννοιας* του κόσμου της εφαρμογής μας.
- Τέλος, με βάση την πιο γενική αντίληψη για τις κλάσεις, μια κλάση είναι μια *αφαιρετική μοντελοποίηση μιας ιδέας*, γενικότερα, του κόσμου αυτού.

Στη συνέχεια του βιβλίου, για λόγους πρακτικούς και για λόγους σύνδεσης με το προγραμματιστικό επίπεδο, θα θεωρούμε τις κλάσεις ως το βασικό είδος “τύπου ορισμένου από τον χρήστη”. Οπότε τον όρο “κλάση” θα τον χρησιμοποιούμε ως συνώνυμό του.

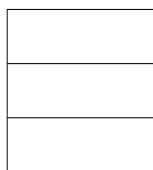
Σε κάθε περίπτωση, ο ορισμός κάθε κλάσης αποτελείται από τα εξής:

1. Από την περιγραφή των δομικών λίθων των αντικειμένων της κλάσης, δηλαδή από τα επιμέρους δεδομένα από τα οποία “κτίζονται” τα στιγμιότυπά της. Αυτή αναφέρεται ως το *δομικό* (structural) τμήμα μιας κλάσης. Στις “διαλέκτους” της βιβλιογραφίας, αλλά και των διαφόρων γλωσσών προγραμματισμού, οι δομικοί αυτοί λίθοι ονομάζονται *μέλη-δεδομένα* (data members), π.χ. στη C++ ή *πεδία* (fields), π.χ. στην Java. Αν τα συσχετίσουμε με τις παρατηρήσιμες ιδιότητες των ΑΤΔ, τις οποίες θα συζητήσουμε αργότερα, θα τα δούμε να ονομάζονται ως *χαρακτηριστικά* (attributes).

Για παράδειγμα, για μια κλάση Date που αντιπροσωπεύει την έννοια της “ημερομηνίας”, οι δομικοί αυτοί λίθοι μπορεί να είναι τρεις ακέραιοι, day, month και year που αντιστοιχούν στην “ημέρα”, τον “μήνα” και το “έτος” της “ημερομηνίας”. Τότε, κάθε στιγμιότυπο, d, της κλάσης Date “κτίζεται” από τους τρεις αυτούς ακραίους. Σαν εικόνα, δείτε το Σχήμα 1.1.

Date d;

d



d.day

d.month

d.year

Σχήμα 1.1: Ένα στιγμιότυπο, d, της κλάσης Date

Εδώ πρέπει να προτρέξουμε λίγο και να πούμε ότι δεδομένα που περιγράφονται σε μια κλάση μπορεί να μην συμμετέχουν στο “κτίσιμο” των στιγμιότυπων της αλλά να αποτελούν δεδομένα που σχετίζονται με την κλάση αυτή καθαυτή. Αυτό θα γίνει καλύτερα κατανοητό στα επόμενα κεφάλαια, όταν θα έχουμε μιλήσει για την έννοια της εμβέλειας (scope) και το γεγονός ότι μια κλάση καθορίζει τη δική της εμβέλεια στην οποία ανήκουν τα ονόματα που δηλώνονται στην κλάση.

2. Από την περιγραφή της συμπεριφοράς που χαρακτηρίζει μια κλάση και την οποία τα αντικείμενά της οφείλουν να ακολουθούν, δηλαδή τη “λειτουργικότητά” της. Αυτή αποτελεί το τμήμα της συμπεριφοράς (behavioral) και περιλαμβάνει τις διαδικασίες μέσω των οποίων προσπελαύνουμε τα στιγμιότυπα της κλάσης. Στις διάφορες “διαλέκτους” των γλωσσών ΟΟΡ θα τις βρούμε να ονομάζονται συχνά ως *συναρτήσεις-μέλη* (member functions), π.χ. στη C++ ή *μέθοδοι* (methods), π.χ. στην Java. Ο όρος “μέθοδοι” είναι εκείνος που σχετίζεται πιο εκτενώς με το αντικειμενοστραφές μοντέλο, μια που ήδη από τις πρώτες γλώσσες με αντικείμενα, π.χ. στη γλώσσα Smalltalk, τα αντικείμενα ανταλλάσσουν μεταξύ τους μηνύματα (messages) στα οποία ανταποκρίνονται εκτελώντας “μεθόδους”, σύμφωνα με τη συμπεριφορά της κλάσης στην οποία ανήκουν. Συσχετίζοντας τις κλάσεις με τους ΑΤΔ, αυτό αφορά τις *λειτουργίες* που καθορίζουν τη συμπεριφορά τους. Πολλές φορές, στο εξής, θα χρησιμοποιούμε τον όρο “λειτουργίες” και για τις κλάσεις.

Όπως και στην περίπτωση του δομικού τμήματος, όμως, ενδέχεται μέρος της λειτουργικότητας που περιγράφεται στο τμήμα της συμπεριφοράς να μην αφορά συγκεκριμένο στιγμιότυπο της κλάσης, αλλά ολόκληρη την κλάση, ακολουθώντας την ίδια λογική που αναφέραμε για τα δεδομένα και θα αναπτύξουμε σε επόμενα κεφάλαια.

1.3.1.2 Αντικείμενα

Μια κλάση μπορεί να θεωρηθεί ως ένα σύνολο αντικειμένων συνοδευόμενο από ένα σύνολο λειτουργιών που αρχικοποιούν, παρατηρούν και μεταβάλλουν τα αντικείμενα αυτά. Εναλλακτικά, τα αντικείμενα είναι τα στιγμιότυπα του τύπου που υλοποιεί η κλάση και, πολλές φορές στη συνέχεια, θα χρησιμοποιούμε τον όρο “στιγμιότυπο” ως συνώνυμο του όρου “αντικείμενο”. Σε κάθε περίπτωση, ένα αντικείμενο μιας κλάσης καταλαμβάνει χώρο στη μνήμη, ο οποίος καθορίζεται από την περιγραφή που δίνεται στο δομικό τμήμα της κλάσης και έχει μια *ταυτότητα* (identity) που το προσδιορίζει μοναδικά. Σε κάθε χρονική στιγμή, το αντικείμενο έχει κάποια *τιμή* (value), η οποία μπορεί να παρατηρηθεί ή να αλλαχθεί από τις λειτουργίες της συμπεριφοράς της κλάσης. Υπάρχει, δηλαδή, η έννοια της *κατάστασης* (state) ενός αντικειμένου, η οποία προκύπτει από την τιμή την οποία έχουν τα χαρακτηριστικά του μια συγκεκριμένη χρονική στιγμή. Στην περίπτωση που η κατάσταση του αντικειμένου δεν είναι δυνατό να αλλάξει, το αντικείμενο θεωρείται *αμετάβλητο* (immutable). Στην περίπτωση αυτή ανήκουν και οι σταθερές. Στην αντίθετη περίπτωση, που η κατάστασή του μπορεί να αλλάξει, το αντικείμενο θεωρείται *μεταβλητό* (mutable). Λειτουργίες της συμπεριφοράς που αλλάζουν την κατάσταση του αντικειμένου μιας κλάσης ονομάζονται *μεταβάλλουσες* (mutators). Λειτουργίες που απλά την παρατηρούν και επιστρέφουν την τιμή του — ή μέρος της — ονομάζονται *αναγνώστριες* (accessors). Τέλος, ιδιαίτερες λειτουργίες πρέπει να συμπεριλαμβάνονται που να *αρχικοποιούν* (initialize) ένα νέο αντικείμενο, προσδίδοντάς του μια αρχική κατάσταση.

Όπως είπαμε παραπάνω, οι ίδιες οι κλάσεις μπορεί να έχουν δικά τους δομικά στοιχεία και συμπεριφορά, ανεξάρτητα από αντικείμενά τους. Επίσης, σε πολλές γλώσσες δίνεται η δυνατότητα και δικής τους αρχικοποίησης.

Παρότι πρόωρο και μάλλον εξεζητημένο, τόσο για λόγους πληρότητας της παρουσίασης των εννοιών, αλλά και για λόγους συσχέτισης με προσεγγίσεις στη χρήση του αντικειμενοστραφούς μοντέλου στην αναπαράσταση γνώσης, να πούμε εδώ ότι μπορούμε να θεωρήσουμε ότι οι κλάσεις μπορούν να αποτελούν στιγμιότυπα κλάσεων οι οποίες καλούνται *μετακλάσεις* (metaclass).

1.3.1.3 Κληρονομικότητα

Πέρα από την αφαίρεση δεδομένων και την υποστήριξη στιγμιότυπων, μια γλώσσα αντικειμενοστραφούς προγραμματισμού πρέπει να υποστηρίζει και την *κληρονομικότητα*. Μέσω αυτής μπορεί να οριστεί μια κλάση προσδιορίζοντας μόνο πώς διαφέρει από κάποιαν άλλη, πιο γενική. Όπως έχουμε πει ήδη, προσδιορίζοντας μια κλάση ως εξειδίκευση μιας άλλης, δεν χρειάζεται να επαναλάβουμε τα κοινά χαρακτηριστικά και λειτουργίες, αλλά εστιάζουμε μόνο στην “ειδοποιό διαφορά” της πιο εξειδικευμένης κλάσης που την κάνει να διαφοροποιείται από τη γενική. Η εξειδικευμένη κλάση συνήθως ονομάζεται *υποκλάση* (subclass) και η πιο γενική κλάση *υπερκλάση* (superclass) — στη C++, ονομάζονται *παραγόμενη κλάση* (derived class) και *βασική κλάση* (base class), αντίστοιχα. Να τονίσουμε ότι κάθε αντικείμενο της υποκλάσης μπορεί να θεωρηθεί αντικείμενο της υπερκλάσης και να χρησιμοποιηθεί σε κάθε περίπτωση που απαιτείται να χρησιμοποιηθεί αντικείμενο της υπερκλάσης. Για παράδειγμα, αν μια κλάση “Car” είναι υποκλάση της κλάσης “Vehicle” και στην κλάση “Vehicle” έχει οριστεί μια λειτουργία “move” που αντιστοιχεί στη μετατόπιση, αν η κλάση “Car” δεν έχει κάποια ιδιαιτερότητα ως προς τη λειτουργία της μετατόπισης σε σχέση με την κλάση “Vehicle”, η λειτουργία “move” δεν χρειάζεται να συμπεριληφθεί στον ορισμό της. Επιπλέον, κάθε αντικείμενο της κλάσης “Car” μπορεί να χρησιμοποιηθεί και όπου απαιτείται αντικείμενο της κλάσης “Vehicle”, π.χ. σε μία έκφραση ανάθεσης (assignment) σε μεταβλητή τύπου “Vehicle”.

Μέσω της σχέσης ιεραρχίας που προκύπτει, μια κλάση μπορεί να κληρονομήσει ιδιότητες και λειτουργίες από κάποιαν ιεραρχικά ανώτερη. Μέσω της ιεραρχίας αυτής, η κληρονομικότητα επιτρέπει να αναπτύσσουμε κώδικα οργανωμένο και καλύτερα κατανοητό, αλλά και να απαλλαγούμε από την επανάληψη κώδικα, πράγμα που αυξάνει την πιθανότητα για λάθη. Θέλει, όμως, προσοχή τόσο στη χρήση κλάσεων που είναι υποκλάσεις άλλων κλάσεων, όσο και στην επέκταση μιας ιεραρχίας κλάσεων με νέες κλάσεις. Για το δεύτερο, ο λόγος είναι ότι ενδέχεται να κληρονομείται συμπεριφορά που ίσως δεν είναι επιθυμητή. Πρέπει, λοιπόν, οι προδιαγραφές

των κλάσεων να είναι ξεκάθαρες και ο προγραμματιστής να τους δίνει την προσοχή που τους οφείλει.

Περισσότερα θέματα που αφορούν κληρονομικότητα θα αναπτύξουμε αργότερα στο ειδικό κεφάλαιο.

1.3.2 Μηχανισμοί του OOP

Στην παράγραφο αυτή, θα δούμε τι χρειαζόμαστε από μια γλώσσα προγραμματισμού για να επιτύχουμε τη λειτουργικότητα που παρουσιάσαμε προηγουμένως. Υπάρχουν ορισμένοι μηχανισμοί που είναι απαραίτητοι για ένα σύστημα προγραμματισμού, προκειμένου αυτό να είναι ικανό να υποστηρίξει OOP.

Οι μηχανισμοί αυτοί έχουν ως εξής:

- Απαιτείται ένας μηχανισμός εγκλωβισμού ο οποίος:
 1. Να συνδέει την κλάση με την αναπαράσταση (των αντικειμένων) της καθώς και με τις λειτουργίες που αποτελούν τη συμπεριφορά της.
 2. Να διασφαλίζει την απόκρυψη της πληροφορίας. Μόνο τα χαρακτηριστικά και οι λειτουργίες των προδιαγραφών της κλάσης πρέπει να γίνονται ορατά και χρησιμοποιήσιμα έξω από την κλάση.

Όσον αφορά το 1., για παράδειγμα στη C παρότι μπορούμε να ορίσουμε νέους τύπους (μέσω struct), η γλώσσα δεν προσφέρει τη δυνατότητα να τους συσχετίσουμε με συναρτήσεις που υλοποιούν τη συμπεριφορά τους. Έστω, για παράδειγμα, μια δομή ακεραίων στην οποία μπορούμε να προσθέσουμε και να αφαιρέσουμε στοιχεία. Θα μπορούσαμε να ορίσουμε στη C έναν νέο τύπο που να την υλοποιεί ως

```
typedef struct int_struct { ... } IStruct;
```

Όμως, δεν επιτρέπεται να συμπεριλάβουμε τη συνάρτηση της πρόσθεσης και τη συνάρτηση της αφαίρεσης μέσα στα άγκιστρα. Θα πρέπει να ορίσουμε τις συναρτήσεις αυτές όπως οποιαδήποτε άλλη συνάρτηση του κώδικά μας, στην οποία έχουμε ένα όρισμα που αντιστοιχεί στο στιγμιότυπο της δομής. Οπότε, διαβάζοντας τον κώδικα, δεν έχουμε τρόπο να διακρίνουμε ότι η συνάρτηση, π.χ. `void add(IStruct* s, int i)`, που αντιστοιχεί στην πρόσθεση στοιχείου/ακεραίου στη δομή, είναι συνάρτηση της συμπεριφοράς του τύπου, μια που το πρωτότυπό της έχει την ίδια μορφή με μια συνάρτηση, π.χ. `void bla(IStruct* s, int i)`, που χρειάζεται στην υλοποίηση του προβλήματός μας και απλά δέχεται ως όρισμα μια μεταβλητή του τύπου.

Δυο λόγια για το 2., τώρα. Κάθε τι που θεωρείται λεπτομέρεια υλοποίησης, όπως η αναπαράσταση των αντικειμένων, καθώς και η υλοποίηση της συμπεριφοράς της κλάσης, πρέπει να επιβάλλεται από τη γλώσσα να παραμένει κρυμμένο μέσα στον εγκλωβισμό. Με τον τρόπο αυτό, όπως είπαμε, επιτυγχάνουμε την απόκρυψη πληροφορίας που είναι τόσο κρίσιμη στην ανάπτυξη εκτενούς κώδικα. Εξίσου κρίσιμη είναι και για προβλήματα που ενδέχεται να εμπλέκουν περίπλοκες οντότητες.

Κάτι ανάλογο στην καθημερινότητά μας συναντάμε συνεχώς: από την αλλαγή ταχυτήτων κατά την οδήγηση αυτοκινήτου, όπου απλά μετακινούμε τον μοχλό, οπότε δεν χρειάζεται και δεν επιτρέπεται να βάλουμε τα χέρια μας μέσα στο σασιόν, έως το τηλεκοντρόλ της τηλεόρασής μας, όπου απλά πατάμε το κουμπί που μας ενδιαφέρει, δεν ασχολούμαστε με το τι γίνεται με το πάτημά του και ούτε μπορούμε να παρέμβουμε. Ακόμα κι αν μετά από κάποια επισκευή είτε του ενός είτε του άλλου, κάτι αλλάξει εσωτερικά, δεν θα αλλάξει κάτι για εμάς τους χρήστες.

Ολοκληρώνοντας, να σημειώσουμε ότι ο μηχανισμός του εγκλωβισμού

- παρέχει στοιχειοποίηση (modularity) επιτρέποντας ανάπτυξη κώδικα με τη λογική του “διαίρει και βασίλευε” (divide and conquer),
- επιτρέπει απόκρυψη πληροφορίας με ό,τι καλό αυτή προσφέρει στην ανάπτυξη και τη συντήρηση κώδικα,

- επιτρέπει επαναχρησιμοποίηση, που είναι πολύ κρίσιμη και για λόγους διευκόλυνσης αλλά και για λόγους μείωσης των σφαλμάτων, κυρίως αν ο κώδικας είναι εκτενής και περίπλοκος.
- Ο επόμενος μηχανισμός που απαιτείται είναι ο *μηχανισμός ελέγχου τύπων* (type checking mechanism). Αυτός πρέπει να εξασφαλίζει αφενός μεν, ότι μια λειτουργία της συμπεριφοράς μιας κλάσης εφαρμόζεται στα αντικείμενα της κλάσης, αφετέρου δε, ότι ο χειρισμός και η παρατήρηση των αντικειμένων της κλάσης γίνεται μέσω των λειτουργιών της κλάσης και μόνο. Για να γίνει το τελευταίο πιο συγκεκριμένο, σκεφτείτε το εξής: έστω ότι δύο κλάσεις έχουν τα ίδια πεδία, δηλαδή τα αντικείμενά τους αναπαριστώνται με τον ίδιο τρόπο. Τότε, αν δώσουμε ως όρισμα μιας συνάρτησης μιας κλάσης αντικείμενο της άλλης κλάσης, ο μηχανισμός ελέγχου τύπων πρέπει να παραγάγει “σφάλμα”.
- Τέλος, προκειμένου να υποστηριχθεί η έννοια των στιγμιотύπων, απαιτείται ένας *μηχανισμός δημιουργίας στιγμιотύπων* (instantiation mechanism). Αυτός πρέπει να είναι ικανός να δημιουργεί αντικείμενα μιας κλάσης και να εκτελεί λειτουργίες αρχικοποίησης (initialization operations) γι’ αυτά.

1.3.3 Ιδιότητες του OOP

Υπάρχουν διάφορες ιδιότητες οι οποίες συνοδεύουν τον OOP και την έννοια των κλάσεων. Πέρα από αυτές που ήδη έχουμε συζητήσει, μια ακόμα είναι εκείνη του *πολυμορφισμού* (polymorphism). Η ιδιότητα αυτή, όπως φαίνεται και από το όνομά της, αναφέρεται στην ικανότητα μιας προγραμματιστικής οντότητας (entity) να έχει “πολλές μορφές”. Στη βιβλιογραφία συναντώνται διάφορες περιπτώσεις που χαρακτηρίζονται ως “πολυμορφισμός”.

Όσον αφορά τους τύπους δεδομένων, σύμφωνα με το [3], οι συμβατικές γλώσσες προγραμματισμού, όπως η Pascal, επιβάλλουν οι προγραμματιστικές οντότητες να είναι ενός και μόνο τύπου. Τέτοιου είδους γλώσσες καλούνται *μονομορφικές* (monomorphic). Οι *πολυμορφικές* (polymorphic) γλώσσες επιτρέπουν κάποιες μεταβλητές και τιμές να είναι πολλών τύπων. Οι *πολυμορφικοί τύποι* είναι τύποι των οποίων οι λειτουργίες εφαρμόζονται σε ορίσματα που το καθένα από αυτά είναι περισσότερο του ενός τύπου. Υπάρχουν διάφορα είδη πολυμορφισμού που εμπίπτουν στην κατηγορία αυτή. Ανάμεσα σ’ αυτά διακρίνουμε την περίπτωση που ένας τύπος (κυρίως ΑΤΔ) μπορεί να είναι *γενικευμένος* (generic)/*παραμετροποιημένος πολυμορφικά* (polymorphic parameterized). Τότε, καθορίζοντας ένα γενικό περίγραμμα, μπορεί να περιγραφεί ένα σύνολο τύπων που προκύπτουν εκχωρώντας διαφορετικούς τύπους ως τιμές σε παραμέτρους τύπου που εμπεριέχονται στον ορισμό. Δηλαδή, δίνουμε έναν μόνο ορισμό τύπου που αναπαριστάνει τη “γενίκευση” τύπων στους οποίους αντιστοιχεί ο ορισμός. Κατά τη χρήση, γίνεται συγκεκριμενοποίησή του, κατά περίπτωση, ανάλογα με τους τύπους που δίνονται ως τιμές στις παραμέτρους. Μιλώντας, στη συνέχεια, για τον προγραμματισμό γενικεύσεων, θα δούμε πιο αναλυτικά την περίπτωση αυτή.

Μια άλλη ιδιότητα, που σύμφωνα με το [3] αποτελεί είδος πολυμορφισμού, είναι η *υπερφόρτωση των συναρτησιακών συμβόλων* (function symbol overloading). Σ’ αυτήν την περίπτωση, σε ένα συναρτησιακό σύμβολο εκχωρούνται πάνω από ένας τύπος. Αυτό μας επιτρέπει να χρησιμοποιούμε το ίδιο σύμβολο (ή όνομα) για να ονομάσουμε διαφορετικές συναρτήσεις οι οποίες διαχωρίζονται από τον τύπο ή/και το πλήθος των ορισμάτων τους. Έτσι μπορούμε να χρησιμοποιήσουμε το σύμβολο/όνομα που θεωρούμε ότι αποδίδει το νόημα του τι κάνει η συνάρτηση αυτή χωρίς να ανησυχούμε για ενδεχόμενη *σύγκρουση ονομάτων* (name conflict). Για παράδειγμα, το σύμβολο “+” εξυπηρετεί στο να εκφράσουμε “συνάθροιση”. Σε ένα προγραμματιστικό περιβάλλον που υποστηρίζει υπερφόρτωση συμβόλων, μπορεί να χρησιμοποιηθεί τόσο για την υλοποίηση της έκφρασης Αριθμός + Αριθμός, όσο και για την έκφραση Λίστα + Λίστα, που αντιστοιχούν στην πρόσθεση αριθμών και στη συνένωση λιστών, αντίστοιχα.

Κλείνοντας τη συζήτηση για τον πολυμορφισμό, πρέπει να επισημάνουμε ότι στο σύγχρονο πλαίσιο του OOP, ο όρος αντιστοιχεί συνήθως σε μια συγκεκριμένη περίπτωση. Αυτή είναι η ικανότητα να χρησιμοποιείται ο κατάλληλος ορισμός κατά την κλήση μιας συνάρτησης, ακόμα και όταν ο τύπος του αντικειμένου πάνω στο οποίο εφαρμόζεται κατά τη φάση της εκτέλεσης είναι υποκλάση του τύπου του ορίσματος της. Γενικότερα, σύμφωνα με το [2], όσον αφορά τον OOP, ο πολυμορφισμός αναφέρεται στην ικανότητα μιας οντότητας να αποτελεί στιγμιотύπο διαφορετικών κλάσεων κατά τη φάση της εκτέλεσης.

Προγραμματισμός Γενικεύσεων (generic programming): Δεδομένου, όμως, ότι έγινε αναφορά στους γενικευμένους τύπους, στο σημείο αυτό θα κάνουμε μια παύση από τη συζήτηση για τον OOP για να κά-
νουμε μια σύντομη παρουσίαση του προγραμματισμού γενικεύσεων (generic programming), μια που
βασίζεται στην έννοια των γενικεύσεων που αναφέραμε προηγουμένως.

Συγκεκριμένα, ο προγραμματισμός γενικεύσεων βασίζεται στην ιδέα της ανάδειξης της γενικευμένης συμπεριφοράς ενός αλγορίθμου ή ενός τύπου, ανεξάρτητα από συγκεκριμένους τύπους δεδομένων πάνω στους οποίους εφαρμόζεται ή, στην περίπτωση των τύπων, χρησιμοποιεί. Οπότε, ο προγραμματιστής, αντί να υλοποιεί πλήρως κάποιες διαδικασίες ή/και νέους τύπους δεδομένων, προσδιορίζει το “περίγραμμα” που περιγράφει τη βασική αυτή συμπεριφορά. Στη συνέχεια, ο ορισμός θα συμπληρωθεί με την πλήρη πληροφορία των τύπων δεδομένων μόνον όταν πρόκειται η γενικευμένη αυτή περιγραφή να χρησιμοποιηθεί για να οριστεί μια πραγματική συνάρτηση (ή τύπος) που θα χρησιμοποιηθεί στο πρόγραμμα. Η παραπάνω αφαιρετική προσέγγιση μας επιτρέπει να δώσουμε μία και μόνο (γενικευμένη) περιγραφή, η οποία θα χρησιμοποιηθεί προκειμένου να δημιουργηθούν συγκεκριμενοποιήσεις της κατά περίπτωση. Με αυτόν τον τρόπο, μπορούμε να εστιάσουμε αποκλειστικά στην ουσία του αλγορίθμου ή του νέου τύπου.

Ας δούμε κάποια παραδείγματα για να γίνουν πιο κατανοητά τα παραπάνω. Για παράδειγμα, ο ΑΤΔ “στοίβα” έχει την ίδια συμπεριφορά ανεξάρτητα από τον τύπο των στοιχείων του. Άρα, αποτελεί καλό υποψήφιο για υλοποίηση μέσω γενικευμένου τύπου. Αν θέλουμε στο πρόγραμμά μας να έχουμε στοίβες διαφορετικού τύπου στοιχείων, π.χ. “ακεραίων”, “πακέτων” κλπ., αντί να δίνουμε πλήρη ορισμό για το κάθε είδος στοίβας, μπορούμε να δώσουμε μόνο έναν ορισμό, γενικευμένο, π.χ.

`stack_of(X),`

ο οποίος, στη συνέχεια, να συγκεκριμενοποιείται κατά περίπτωση, π.χ.

`stack_of(integer), stack_of(package).`

Συνήθως, τη μη συγκεκριμενοποίηση κάποιου τύπου την αναπαριστάνουμε χρησιμοποιώντας μεταβλητές τύπου. Στο παράδειγμα `stack_of(X)`, η μεταβλητή τύπου είναι το `X` και αντιστοιχεί στον εκάστοτε τύπο των στοιχείων της στοίβας.

Με αντίστοιχη λογική, μπορούμε να θεωρήσουμε τη διαδικασία υπολογισμού του μήκους μιας λίστας, π.χ. `length_of(X)`. Δεν έχει σημασία τι τύπου, `X`, είναι τα στοιχεία της λίστας. Είτε είναι “αριθμοί” είτε είναι “ονόματα ανθρώπων”, η διαδικασία είναι η ίδια: έχουμε έναν μετρητή, αρχικά ίσο με μηδέν, και διατρέχουμε τη λίστα, ένα-ένα στοιχείο έως το τέλος της, αυξάνοντας τον μετρητή κατά μία μονάδα για κάθε στοιχείο που επισκεπτόμαστε.

Επίσης, γενικεύσεις συναντάμε διαρκώς και στην καθημερινότητά μας σε τελείως διαφορετικά περιβάλλοντα. Για παράδειγμα, “στοίβες” μπορεί να κτίζονται από “πιάτα” έως “κλήσεις συναρτήσεων”. “Ουρές” μπορεί να περιέχουν “ανθρώπους” που περιμένουν να εξυπηρετηθούν από μία υπηρεσία, έως “διεργασίες” που περιμένουν να τους ανατεθεί κάποιος υπολογιστικός πόρος.

Βασικός συντελεστής του προγραμματισμού γενικεύσεων είναι ο Alexander Stepanov. Εκείνος ονομάτισε αυτό το είδος προγραμματισμού, καθώς και υλοποίησε σε C++ μια βιβλιοθήκη με ένα πλήθος τέτοιων γενικεύσεων, προκειμένου να χρησιμοποιηθούν συγκεκριμενοποιήσεις τους σύμφωνα με τις ανάγκες ενός συγκεκριμένου προγράμματος. Η βιβλιοθήκη αυτή λεγόταν Standard Template Library (STL). Στις μέρες μας, παρότι οι γενικευμένοι τύποι της καθιερωμένης βιβλιοθήκης της C++ δεν ακολουθούν την αρχική υλοποίηση του Stepanov, συχνά, άτυπα, το μέρος της βιβλιοθήκης που τους περιλαμβάνει αποκαλείται “STL”.

Κλείνοντας την παρένθεση αυτή, να πούμε ότι υποστήριξη γενικεύσεων (generics) υπήρχε ήδη από παλαιότερες γλώσσες προγραμματισμού, π.χ. Ada, Eiffel. Επίσης, σιγά-σιγά, επεκτείνεται στις περισσότερες γλώσσες προγραμματισμού, π.χ. Java, C# κ.ά.

1.4 Πού Συναντάμε το Αντικειμενοστραφές Μοντέλο

Στο σημείο αυτό, πρέπει να γίνει μια σημαντική επισήμανση: παρότι οι κλάσεις έγιναν ευρέως γνωστές από την εμφάνισή τους στον προγραμματισμό μέσω των γλωσσών αντικειμενοστραφούς προγραμματισμού, η χρήση τους είναι ευρύτερα διαδεδομένη στον χώρο των υπολογιστικών συστημάτων. Για παράδειγμα, έχουν έντονη παρουσία σε εργαλεία που εμπλέκονται σε άλλες φάσεις της ανάπτυξης λογισμικού, όπως ανάλυση και σχεδίαση με χρήση του αντικειμενοστραφούς μοντέλου. Το ίδιο ισχύει και για την περιοχή της Τεχνητής Νοημοσύνης (ΤΝ) (Artificial Intelligence – AI), όπου η γνώση (knowledge) συχνά αναπαριστάται είτε με τη μορφή κλάσεων είτε με τη χρήση γενικότερων μορφών απεικόνισης οντοτήτων. Ακόμα, συναντάμε το αντικειμενοστραφές μοντέλο στην περιοχή των βάσεων δεδομένων, είτε ως αυτοτελές μοντέλο δεδομένων — το αντικειμενοστραφές μοντέλο δεδομένων (Object Oriented data model) —, ή την αντικειμενοστραφή επέκταση του σχεσιακού μοντέλου (Object-relational data model). Να πούμε, επίσης, ότι αντικειμενοστραφή χαρακτηριστικά έχουν εισαχθεί και σε άλλα είδη προγραμματισμού, όπως ο λογικός προγραμματισμός και ο συναρτησιακός προγραμματισμός, καθώς και έχουν επεκτείνει σχεδόν όλες τις υπάρχουσες γλώσσες διαδικαστικού προγραμματισμού.

Όπως ισχυρίστηκε ο Booch στο [4], το αντικειμενοστραφές μοντέλο μπορεί να διευκολύνει στην αντιμετώπιση της πολυπλοκότητας που εμφανίζεται σε διάφορους τομείς της επιστήμης των υπολογιστών και, ακόμα, να αποτελέσει μια κοινή έννοια που να τους συνδέει. Κατά συνέπεια, σημαντικοί επιστήμονες του χώρου, από τα τέλη της δεκαετίας του '80 - αρχές '90, εργάστηκαν στην επέκταση της χρήσης του και διαπίστωσαν ότι “ήλθε για να μείνει”.

Αναφερόμαστε, όμως, στον όρο “Αντικειμενοστραφές Μοντέλο” (Object Oriented Model - OOM), χωρίς να έχουμε σχολιάσει αν αυτό είναι καλά καθορισμένο, προκειμένου να χρησιμοποιηθεί σε διάφορα εργαλεία. Για τον λόγο αυτό, στο τέλος της δεκαετίας του '80, που ο όρος είχε αρχίσει να χρησιμοποιείται ευρέως, συστάθηκε ο οργανισμός OMG (Object Management Group) [5], με στόχο την τυποποίηση της έννοιας του αντικειμενοστραφούς μοντέλου, προκειμένου να επιτευχθεί επιτυχής αλληλεπίδραση μεταξύ των εργαλείων που το χρησιμοποιούν. Οι δραστηριότητες του οργανισμού εστίασαν στην τυποποίηση εργαλείων που βασίζονται στο μοντέλο. Πολλά από αυτά είναι ιδιαίτερα επιτυχημένα και διαδεδομένα. Παραδείγματα είναι

- η γλώσσα μοντελοποίησης λογισμικού UML (Unified Modeling Language),
- το Common Object Request Broker Architecture (CORBA), που είναι μια τυποποίηση για μια αντικειμενοστραφή μεθοδολογία για ενοποίηση ετερογενών ή/και κατανεμημένων συστημάτων, χωρίς να απαιτείται τα ίδια να βασίζονται στο αντικειμενοστραφές μοντέλο,
- το XML Metadata Interchange (XMI) για ανταλλαγή μεταδεδομένων και
- το Knowledge Discovery Metamodel (KDM), το οποίο επιτρέπει την περιγραφή (σημασιολογικής) μεταπληροφορίας για την αλληλεπίδραση, ακόμα και υπαρχόντων, εργαλείων ανάπτυξης λογισμικού.

Στη συνέχεια, θα κάνουμε μια σύντομη επισκόπηση στη χρήση του αντικειμενοστραφούς μοντέλου σε διάφορους τομείς της πληροφορικής, δίνοντας κάποιες παραπάνω πληροφορίες και αναφορές, ώστε ο αναγνώστης να είναι ενήμερος και, στην περίπτωση που ενδιαφέρεται περισσότερο, να μπορεί να αναζητήσει επιπλέον λεπτομέρειες.

1.4.1 Ανάλυση και Σχεδίαση Λογισμικού

Έχει καθιερωθεί να θεωρούμε ότι η διαδικασία της ανάπτυξης ενός συστήματος λογισμικού διακρίνεται σε διάφορες φάσεις, όπως:

- ανάλυση απαιτήσεων,
- σχεδίαση λογισμικού,

- υλοποίηση,
- δοκιμή συστήματος (testing),
- εγκατάσταση (deployment) και
- συντήρηση (maintenance).

Οι φάσεις αυτές μπορεί να λαμβάνουν χώρα είτε σειριακά, όπως γίνεται στο “μοντέλο του καταρράκτη”, που ακολουθούνταν για την ανάπτυξη λογισμικού ευρέως, ιδίως μέχρι τη δεκαετία του ’80, είτε με επικαλύψεις ή επαναλήψεις. Σε κάθε περίπτωση, στα πρώτα στάδια πρέπει να γίνει μια μοντελοποίηση των απαιτήσεων του προβλήματος, καθώς και του συστήματος που θα αναπτυχθεί για να τις ικανοποιήσει.

Η ανάπτυξη λογισμικού μπορεί να οδηγήσει σε χαοτικές καταστάσεις λόγω της πολυπλοκότητας, αλλά και της ενδεχόμενης ασάφειας του προβλήματος προς αντιμετώπιση. Προκειμένου, λοιπόν, να μπει τάξη στο χάος που μπορεί να δημιουργηθεί, είναι απαραίτητη μια συστηματική και πειθαρχημένη προσέγγιση κατά την ανάπτυξη. Όπως επισημαίνει ο Dijkstra στο [6], η πανάρχαια τεχνική του “διαίρει και βασίλευε” προσφέρει το υπόβαθρο για μεθοδολογίες ικανές να αντιμετωπίσουν το πρόβλημα. Άρα, μια μορφή “αποσύνθεσης” (decomposition) του προβλήματος που θα επιτρέψει τμηματική μοντελοποίηση και ανάπτυξη είναι απαραίτητη. Το ερώτημα, τώρα, είναι τα τμήματα της αποσύνθεσης σε τι θα αναφέρονται; Παρατηρούμε το εξής: κάθε σύστημα αποτελείται από

- οντότητες/αντικείμενα/δεδομένα και
- ενέργειες/διαδικασίες που εφαρμόζονται πάνω σε αυτά.

Η τμηματοποίηση, λοιπόν, να γίνει με βάση τις διαδικασίες ή με βάση τα δεδομένα;

Στην πρώτη περίπτωση, καθορίζουμε τις διαδικασίες του προβλήματος, π.χ. “υπολογισμός τελικού ποσού πληρωμής”, “υπολογισμός διαδρομής”. Αυτές δομούνται χρησιμοποιώντας απλούστερες διαδικασίες, θεωρώντας ότι κάθε μία από αυτές λύνει και από ένα επιμέρους πρόβλημα, π.χ. “υπολογισμός τελικής τιμής προϊόντος 1234”, “υπολογισμός μετακίνησης από την Πόλη1 στην Πόλη2” κ.ο.κ. Αυτής της μορφής την αποσύνθεση ο Booch την αποκαλεί “αλγοριθμική αποσύνθεση” (algorithmic decomposition). Αυτή αποτελεί την αρχή της δομημένης σχεδίασης λογισμικού (structured design) που κυριαρχούσε για χρόνια. Αυτό είναι φυσικό, μια που τα προβλήματα κατά βάση ήταν αλγοριθμικά ή/και ενέπλεκαν απλά δεδομένα. Με την ένταξη των κλάσεων στον προγραμματισμό, όμως, και την υιοθέτηση του αντικειμενοστραφούς μοντέλου, το να κάνουμε αποσύνθεση με βάση τις κλάσεις, π.χ. “τιμολόγιο”, “πελάτης”, “οδικό δίκτυο”, δείχνει πλέον πιο βολικό. Το ποια από τις δύο μεθοδολογίες θα επιλέξουμε εξαρτάται από το πρόβλημά μας, αλλά πλέον, με δεδομένα τα σύγχρονα εργαλεία ανάπτυξης λογισμικού και προγραμματισμού και των χαρακτηριστικών των σύγχρονων προβλημάτων, η αντικειμενοστραφής μεθοδολογία έχει επικρατήσει.

Σημείωση: Εδώ να κάνουμε μια παρατήρηση που δεν σχετίζεται με το θέμα της παραγράφου αυτής, σχετίζεται, όμως, με τη διάκριση σε διαδικασίες και κλάσεις και αφορά τα ονόματα που δίνουμε στα συστατικά των προγραμμάτων μας, π.χ. ονόματα των συναρτήσεων, ονόματα μεταβλητών κ.ά. Ακόμα και στις προτάσεις που διατυπώνουμε σε φυσική γλώσσα, στη στοιχειώδη μορφή τους υποκείμενο-ρήμα-αντικείμενο, διακρίνουμε το “ρήμα” που δηλώνει μια ενέργεια που γίνεται και τις οντότητες που εμπλέκονται στην ενέργεια αυτή: τον δράστη — το υποκείμενο της πρότασης — και τον αποδέκτη της δράσης — το αντικείμενο. Ειδικές μορφές προτάσεων, όπου εμπλέκουν βοηθητικό ρήμα, προσδιορίζουν ιδιότητες του υποκειμένου — ο “Γιάννης έχει γαλανά μάτια” — ή εξειδίκευσή του — “η Γάτα είναι Θηλαστικό” —. Καλό λοιπόν είναι οι διαδικασίες/συναρτήσεις μας να ονοματίζονται με χρήση ρήματος και τα δεδομένα μας με ουσιαστικά και προσδιορισμούς τους.

Χρησιμοποιώντας αντικειμενοστραφή προσέγγιση, στο αρχικό στάδιο κατά την ανάπτυξη ενός συστήματος, έχουμε τη φάση της αντικειμενοστραφούς μοντελοποίησης (object oriented modelling) του συστήματός μας, τόσο από πλευράς ανάλυσης, όσο και από πλευράς σχεδιασμού. Οι κλάσεις που έχουν εντοπιστεί στο

πρόβλημα και εμπεριέχονται στο μοντέλο ενσωματώνουν τα δεδομένα του και τη συμπεριφορά τους. Παρότι αρχικά τα εργαλεία που ακολουθούσαν την αντικειμενοστραφή προσέγγιση εστίαζαν στον καθορισμό των κλάσεων και άλλων στατικών συστατικών, τα σύγχρονα εργαλεία, όπως η UML, επιτρέπουν μοντελοποίηση και δυναμικών συστατικών του συστήματος, όπως η μελέτη περιπτώσεων χρήσης (use cases). Επιπλέον, στα μέσα της δεκαετίας του '90 καθιερώθηκαν σχεδιαστικά πρότυπα (design patterns) για τη σχεδίαση συστημάτων λογισμικού με βάση το αντικειμενοστραφές μοντέλο που προσφέρουν προκατασκευασμένες λύσεις από πλευράς σχεδιασμού για συγκεκριμένες καταστάσεις που συναντώνται σε πληθώρα προβλημάτων.

1.4.1.1 Η Γλώσσα Μοντελοποίησης UML

Η γλώσσα μοντελοποίησης λογισμικού UML (Unified Modeling Language) επιτρέπει την περιγραφή των προδιαγραφών ενός συστήματος με οπτικοποιημένο τρόπο, καθώς και την τεκμηρίωσή τους. Οι πρώτες εκδόσεις της εμφανίστηκαν στα μέσα της δεκαετίας του '90. Έκτοτε έχει αναπτυχθεί περισσότερο και η χρήση της έχει καθιερωθεί. Η αρχική εμφάνισή της είχε ως στόχο να ενοποιήσει μεθόδους σχεδιασμού που βασιζόντουσαν στο αντικειμενοστραφές μοντέλο, και συγκεκριμένα τη μέθοδο του Booch, την τεχνική OMT (Object-Modeling Technique) και την αντικειμενοστραφή τεχνική OOSE (Object-Oriented Software Engineering), και να προτυποποιήσει ένα μοντέλο. Πλέον, αποτελεί ένα πολύ ισχυρό μέσο μέσω του οποίου μπορούν να εκφραστούν πολλές πτυχές ενός συστήματος που ξεπερνούν ακόμα και το λογισμικό του, αλλά και να μοντελοποιήσει συστήματα των οποίων η γλώσσα υλοποίησης είναι μη-αντικειμενοστραφής, όπως Fortran και COBOL.

Η περιγραφή του μοντέλου γίνεται μέσω διαγραμμάτων. Στην τρέχουσα έκδοση της, UML (2.0), παρέχονται 13 τύποι διαγραμμάτων που εντάσσονται σε τρεις κατηγορίες:

1. διαγράμματα στατικών χαρακτηριστικών, π.χ. διάγραμμα κλάσεων (Class Diagram),
2. διαγράμματα συμπεριφοράς, π.χ. διάγραμμα περιπτώσεων χρήσης (Use Case Diagram) και
3. διαγράμματα αλληλεπίδρασης, π.χ. διάγραμμα επικοινωνίας (Communication Diagram).

Στη συνέχεια, θα επικεντρωθούμε στα διαγράμματα κλάσεων. Σε ένα διάγραμμα κλάσεων, ο στόχος μας είναι να καταγράψουμε τις κλάσεις από τις οποίες αποτελείται το σύστημά μας. Αυτές ενδέχεται να μεταφραστούν απευθείας σε κλάσεις του κώδικα που θα αποτελέσει την υλοποίηση του συστήματος.

Κατά την καταγραφή αυτή, σε κάθε κλάση αποτυπώνονται τα δεδομένα της, τα οποία ονομάζονται “χαρακτηριστικά” (attributes), και οι λειτουργίες της (operations). Και τα δύο συνοδεύονται από το επίπεδο ορατότητάς τους (private, protected, global). Επιτρέπεται να προσδιορίσουμε χαρακτηριστικά και λειτουργίες να αναφέρονται τόσο σε στιγμιότυπα, όσο σε κλάσεις. Στο μοντέλο, καταγράφονται και οι συσχετίσεις μεταξύ των κλάσεων. Αυτές μπορεί να έχουν διάφορες μορφές και μπορεί να αφορούν είτε τα στιγμιότυπά τους είτε τις ίδιες τις κλάσεις. Στην πρώτη περίπτωση έχουμε τις συσχετίσεις:

- Γενική Συσχέτιση (Association), που αναφέρεται σε κάθε είδους συσχέτιση που μπορεί να έχουν τα στιγμιότυπα των κλάσεων που συσχετίζονται, π.χ. οι κλάσεις “Πελάτης” και “Κτήριο” με τη συσχέτιση “Πελάτης μπαίνει σε Κτήριο”.
- Εξάρτηση (Dependency), που αναφέρεται σε συσχετίσεις που οδηγούν στο ενδεχόμενο αλλαγές στον ορισμό της μίας (ανεξάρτητης) κλάσης να επιφέρουν αλλαγές στον ορισμό της άλλης (εξαρτημένης) κλάσης. Η ανεξάρτητη κλάση καλείται *προμηθευτής* (supplier) και η εξαρτημένη *πελάτης* (client). Μια περίπτωση Εξάρτησης είναι όταν ένα στιγμιότυπο μιας κλάσης “χρησιμοποιεί” ένα στιγμιότυπο μιας άλλης στον κώδικα μιας λειτουργίας του και το στιγμιότυπο αυτό δεν αποτελεί χαρακτηριστικό της κλάσης, αλλά παράμετρο της λειτουργίας.
- Σύνθεση (Composition).

- Συνάθροιση (Aggregation).

Τόσο η Σύνθεση, όσο και η Συνάθροιση, αντιστοιχούν σε μια σχέση της μορφής “has a”. Η διαφορά είναι ότι στην περίπτωση της Σύνθεσης, τα στιγμιότυπα των κλάσεων δεν έχουν ανεξάρτητη υπόσταση, ενώ στην περίπτωση της Συνάθροισης, τα στιγμιότυπα της κλάσης που συναθροίζεται υφίστανται και ανεξάρτητα από το στιγμιότυπο της κλάσης που τα συναθροίζει.

Στις συσχετίσεις μεταξύ κλάσεων διακρίνουμε τις εξής περιπτώσεις:

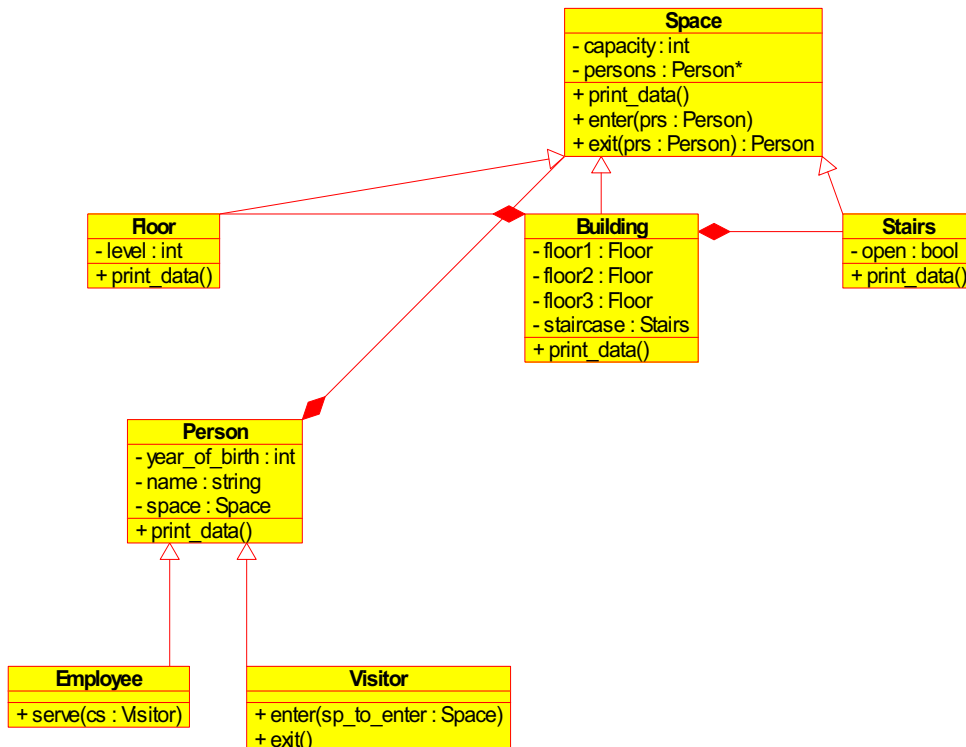
- Γενίκευση/Κληρονομικότητα (Generalization/Inheritance), που αντιστοιχεί στη σχέση εξειδίκευσης κλάσεων “is a”.
- Υλοποίηση (Realization/Implementation) όπου μια κλάση υλοποιεί προδιαγραφές που καθορίζονται σε μια άλλη κλάση ή σε κλάση-διεπαφή (interface).

Τα παραπάνω θα γίνουν πιο κατανοητά αργότερα, έως το Κεφάλαιο 7, οπότε θα έχουμε μιλήσει αναλυτικά για (κάποιες από) τις συσχετίσεις αυτές.

Σε ένα διάγραμμα κλάσεων καθορίζονται και άλλες ακόμα πληροφορίες, όπως, για παράδειγμα, πληροφορίες πληθικότητας μεταξύ στιγμιότυπων κλάσεων, π.χ. 1..* δηλώνει συσχέτιση ενός στιγμιότυπου μιας κλάσης με ένα τουλάχιστον ή παραπάνω στιγμιότυπα της άλλης κλάσης.

Στα διαγράμματα, οι κλάσεις αναπαριστώνται με κουτάκια που χωρίζονται σε τρία μέρη: στο πάνω μέρος αναφέρεται το όνομα της κλάσης, στο μεσαίο τα χαρακτηριστικά της και στο τελευταίο οι λειτουργίες της. Η ορατότητα των χαρακτηριστικών και των λειτουργιών δηλώνεται μέσω των συμβόλων “-”, “#” και “+” (private, protected και global).

Στο Σχήμα 1.2 βλέπετε ένα απλουστευμένο διάγραμμα κλάσεων.



Σχήμα 1.2: Ένα απλό UML διάγραμμα κλάσεων

Στο σχήμα αυτό, οι κλάσεις “Floor”, “Building” και “Stairs” είναι υποκλάσεις μιας κλάσης “Space”. Οι κλάσεις “Employee” και “Visitor” είναι υποκλάσεις της κλάσης “Person”. Στιγμιότυπα της κλάσης “Building” συντίθενται από στιγμιότυπα της κλάσης “Floor” και της κλάσης “Stairs”. Στιγμιότυπα της κλάσης “Person” έχουν στιγμιότυπα της κλάσης “Space”.

1.4.2 Αναπαράσταση Γνώσης

Η Αναπαράσταση Γνώσης (ΑΓ) (Knowledge Representation - KR) εντάσσεται στην περιοχή της Τεχνητής Νοημοσύνης. Η ΤΝ έχει ως στόχο την ανάπτυξη (υπολογιστικών) συστημάτων που επιδεικνύουν νοήμονα συμπεριφορά καθώς και τη μελέτη των προβλημάτων που σχετίζονται με τον στόχο αυτό. Κάθε σύστημα ΤΝ διαχειρίζεται κάποιας μορφής γνώση, μια που η ευφυΐα — ας μιλάμε για ευφυΐα, αντί για νοημοσύνη — είναι άμεσα συσχετισμένη με τη γνώση: κάνουμε συλλογισμούς και καταλήγουμε σε συμπεράσματα, με βάση κάποια γνώση που έχουμε συλλέξει με τα χρόνια. Τα συμπεράσματα στα οποία καταλήγουμε μπορούν να αποτελούν νέα γνώση για μελλοντική χρήση.

Στις μέρες μας, τα συστήματα που επικρατούν περισσότερο στην περιοχή της ΤΝ βασίζονται σε μαθηματικές μεθόδους, π.χ. στατιστικά συστήματα μηχανικής μάθησης και νευρωνικά δίκτυα, μια που έχουν διαθέσιμα πολυάριθμα δεδομένα, τα οποία χρησιμοποιούν για να εξάγουν γνώση. Όμως, πολλές από τις παραδοσιακές, αλλά και διαχρονικές, μεθόδους βασίζονται σε οντοτοκεντρικούς φορμαλισμούς που αναπαριστούν απευθείας τη γνώση αυτή. Σε αυτήν την περίπτωση, η γνώση δεν είναι απαραίτητο να προκύπτει από πραγματικά δεδομένα αλλά μπορεί να είναι απόρροια υποκειμενικών αντιλήψεων του σχεδιαστή του συστήματος.

Στην πραγματικότητα, κάθε γνωστικό πεδίο περιλαμβάνει οντότητες, ακόμα και αν πρόκειται για κάτι αφηρημένο, όπως ιδέες και έννοιες. Αυτές οι οντότητες έχουν κάποιες ιδιότητες, εκτελούν, ενδεχόμενα, κάποιες ενέργειες και συσχετίζονται μεταξύ τους. Στις οντοτοκεντρικές προσεγγίσεις ΑΓ, το επίκεντρο του ενδιαφέροντος είναι οι ίδιες οι οντότητες και όχι οι ενέργειες που, ενδεχόμενα, εκτελούν. Ας πάρουμε για παράδειγμα τον κόσμο των θηλαστικών ζώων. Αυτός θα μπορούσε να αποτελέσει ένα γνωστικό πεδίο. Οι οντότητες είναι τα διάφορα είδη ζώων. Τα είδη των ζώων οργανώνονται σε *ταξινομίες* (taxonomies) που δηλώνουν ποιο είδος ζώου είναι ειδική κατηγορία ποιο άλλου είδους, π.χ. οι τίγρεις είναι ειδική κατηγορία θηλαστικών. Κάθε είδος ζώου έχει κάποιες ιδιότητες, π.χ. αριθμός ποδιών, και μπορεί να σχετίζεται με άλλα είδη και πέρα από τη συσχέτιση της εξειδίκευσης, π.χ. οι τίγρεις τρέφονται με ζώα.

Στο σημείο αυτό πρέπει να αναφέρουμε την έννοια της *οντολογίας* (ontology) που χρησιμοποιείται συχνά για να ονοματίσει μια καταγραφή γνώσης που μπορεί να χρησιμοποιηθεί από ένα υπολογιστικό σύστημα. Να σημειώσουμε ότι θα χρησιμοποιήσουμε τον όρο όπως τον συναντάμε στα υπολογιστικά συστήματα και όχι ως επιστημονικό τομέα, όπως συναντάται στη φιλοσοφία και πηγάζει από τη μελέτη του Αριστοτέλη. Προκειμένου να προσδιορίσουμε τον όρο, λοιπόν, συχνά χρησιμοποιούμε τον ορισμό που δίνεται στο [7]. Εκεί, ο Gruber ορίζει την έννοια της οντολογίας ως “explicit specification of a conceptualization”. Πρόκειται για μια αφαιρετική περιγραφή του πεδίου που μας ενδιαφέρει, θεωρώντας ότι αυτό αποτελείται από αντικείμενα, έννοιες και οντότητες, γενικότερα, καθώς και τις συσχετίσεις που συνδέουν τα παραπάνω μεταξύ τους. Επιγραμματικά, μια οντολογία αποτελεί μια φορμαλιστική καταγραφή των οντοτήτων που είναι κρίσιμες για το γνωστικό πεδίο του προβλήματός μας και των συσχετίσεων μεταξύ τους. Να επισημάνουμε δε την ετυμολογία της λέξης “Ontology”. Η λέξη είναι βασισμένη στην αρχαία ελληνική λέξη “ὄν” που αναφέρεται σε κάθε τι που “είναι” — με την έννοια του “υπάρχει” ή “υφίσταται” — στο γνωστικό πεδίο που αναπαριστάνει.

Κατά συνέπεια, προκειμένου να γίνει εφικτή η αναπαράσταση γνώσης, φορμαλισμοί που διευκολύνουν την αναπαράσταση οντοτήτων ήταν εύλογο να αναπτυχθούν. Στη συνέχεια, θα συνοψίσουμε τα κύρια χαρακτηριστικά των πιο διαδεδομένων από αυτούς, όπου φαίνεται η σύνδεση με το αντικειμενοστραφές μοντέλο. Μάλιστα, ένας από αυτούς τους φορμαλισμούς είναι το ίδιο το αντικειμενοστραφές μοντέλο, με τις κλάσεις, τα αντικείμενα, κληρονομικότητα, μεθόδους κλπ., όπως έχουμε ήδη περιγράψει. Όμως, μια που έχουμε συζητήσει στα προηγούμενα για τα χαρακτηριστικά του, θα εστιάσουμε σε άλλους, προγενέστερους φορμαλισμούς. Να σημειώσουμε ότι αντί για “οντολογία”, θα χρησιμοποιείται ο γενικότερος όρος *βάση γνώσης* (knowledge base) για να αναφερθούμε στην καταγεγραμμένη γνώση προς χρήση από ένα υπολογιστικό σύστημα.

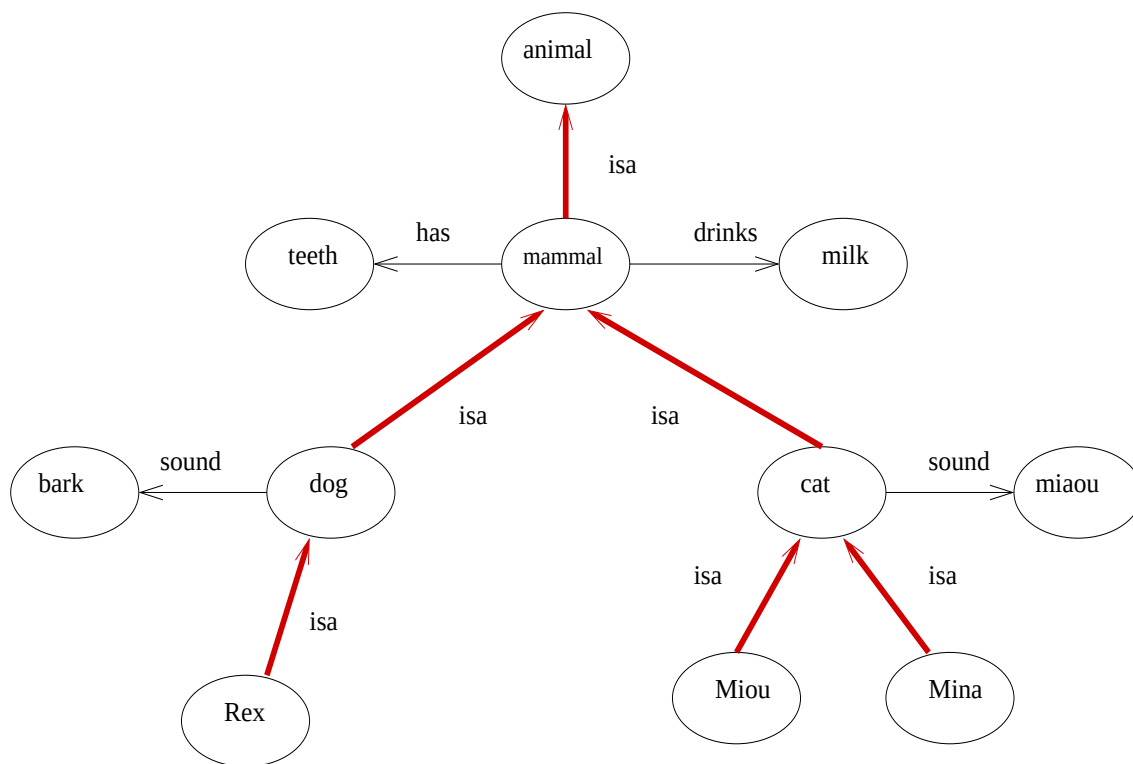
Συγκεκριμένα, θα συζητήσουμε, κυρίως, για

- τα σημασιολογικά δίκτυα (semantic networks) και
- τα πλαίσια (frames).

Να επισημάνουμε ότι όπως θα συζητήσουμε και στη συνέχεια, οι φορμαλισμοί αυτοί έδωσαν τις ιδέες στις

οποίες βασίστηκαν σύγχρονες προσεγγίσεις. Ιστορικά, προέκυψαν από τις ανάγκες περιοχών επεξεργασίας φυσικής γλώσσας (natural language processing), όπως η αυτόματη μετάφραση και η κατανόηση φυσικής γλώσσας (natural language understanding).

Στα **σημασιολογικά δίκτυα** αναπαριστάνουμε τη γνώση του πεδίου που μας ενδιαφέρει ως έναν κατευθυνόμενο γράφο. Ένα παράδειγμα σημασιολογικού δικτύου βλέπετε στο Σχήμα 1.3.



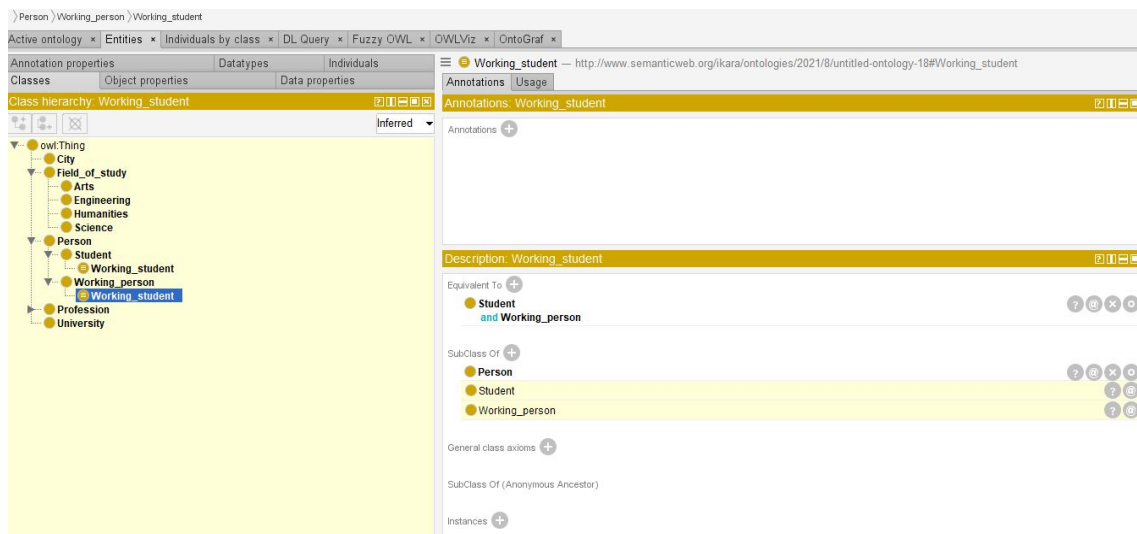
Σχήμα 1.3: Ένα παράδειγμα σημασιολογικού δικτύου

Οι κόμβοι του γράφου αναπαριστούν τις οντότητες που πεδίου μας, π.χ. “dog”, “mammal”, και οι ακμές συσχετίσεις μεταξύ τους. Μια ειδική συσχέτιση είναι εκείνη η οποία προσδιορίζει ότι η οντότητα που αναπαριστάται από έναν κόμβο αποτελεί εξειδίκευση της οντότητας με την οποία τη συσχετίζει, π.χ. η οντότητα “σκύλος” είναι εξειδίκευση της οντότητας “θηλαστικό”. Στην περίπτωση αυτή, η ειδικότερη έννοια/οντότητα κληρονομεί όποια γνώση συνδέεται με τη γενικότερη και δεν είναι άμεσα συσχετισμένη με την ίδια. Στο παράδειγμα, η συσχέτιση αυτή ονοματίζεται “isa”. Δεδομένου ότι, για το παράδειγμα, η οντότητα “dog” “isa” “mammal”, για την οντότητα “dog” καταλαβαίνουμε ότι πέρα από το ότι έχει “sound” “bark”, κληρονομεί ότι “has” “teeth”, “drinks” “milk” και “isa” “animal”. Η κληρονομήση αυτή συνεχίζεται αναδρομικά για όλο το βάθος της ιεραρχίας. Εδώ να κάνουμε μια σημείωση σχετικά με την περίπτωση που μια οντότητα αποτελεί εξειδίκευση παραπάνω της μίας οντοτήτων, σε ίδιο βάθος, π.χ. μια οντότητα “εργαζόμενος φοιτητής” να είναι άμεση εξειδίκευση της οντότητας “φοιτητής” και της οντότητας “εργαζόμενος”. Σε αυτήν την περίπτωση μιλάμε για **πολλαπλή κληρονομικότητα** (multiple inheritance). Στην αναπαράσταση γνώσης, δεν είναι σπάνια η εμφάνιση των περιπτώσεων αυτών. Στον προγραμματισμό, η περίπτωση αυτή δημιουργεί δυσεπίλυτα προβλήματα και καλό είναι να επανεξετάζεται η μοντελοποίηση, μήπως με ένα διαφορετικό μοντέλο μπορεί να αποφευχθεί.

Τα σημασιολογικά δίκτυα προσέφεραν το υπόβαθρο για προσεγγίσεις που αποτέλεσαν τη βάση για λογικούς φορμαλισμούς αναπαράστασης γνώσης οι οποίοι συνοδεύοντουσαν από αποδοτικούς μηχανισμούς συμπερασμού. Στη συνέχεια, από αυτούς πήγασαν τεχνολογίες που χρησιμοποιούνται στις σύγχρονες εφαρμογές αναπαράστασης γνώσης. Συγκεκριμένα, θα αναφέρουμε τον Σημασιολογικό Ιστό (ΣΙ) (Semantic Web) και τις τεχνολογίες του. Ο ΣΙ έχει ως σκοπό τη δυνατότητα οργάνωσης διαφορετικών, κατανεμημένων και, ενδεχόμενα, ετερογενών πηγών δεδομένων/πληροφοριών σε ένα ενιαίο δίκτυο, ώστε αυτές να είναι επεξερ-

γάσιμες από υπολογιστικά συστήματα χωρίς ανθρώπινη παρέμβαση. Προκειμένου να γίνει εφικτή η ενιαία αντιμετώπιση των πηγών, χρειάζεται σημασιολογική πληροφορία σχετικά με το περιεχόμενό τους η οποία να περιγράφει το περιεχόμενο της κάθε πηγής, αλλά και να κάνει εφικτή τη συσχέτιση μεταξύ των περιεχομένων των διαφόρων πηγών.

Για να εξυπηρετηθεί ο σκοπός αυτός, στο πλαίσιο του ΣΙ προσφέρονται διάφορες τεχνολογίες και γλώσσες. Η πιο ισχυρή από τις γλώσσες αναπαράστασης γνώσης του ΣΙ είναι η **OWL** (Web Ontology Language). Αυτή προσφέρει μια σειρά διαλέκτων εκ των οποίων η διάλεκτος **OWL-DL** είναι η πιο διαδεδομένη. Ο λόγος είναι ότι συνδυάζει πλούσια εκφραστική δυνατότητα με αποδοτικό υπολογισμό απαντήσεων σε ερωτήσεις του χρήστη σχετικά με τη γνώση που αναπαρίσταται στην οντολογία/πηγή. Η γλώσσα **OWL-DL** είναι βασισμένη στον φορμαλισμό των Λογικών Περιγραφής (ΛΠ) (Description Logics). Οι ΛΠ αποτελούν ένα ρεπερτόριο φορμαλισμών, με διαφορετικές εκφραστικές δυνατότητες ο καθένας, που επιτρέπουν αποδοτικό υπολογισμό ερωτήσεων. Οι βασικές ιδέες τους προέκυψαν από το σύστημα **KL-ONE**, το οποίο είναι εμπνευσμένο από τα σημασιολογικά δίκτυα. Συγκεκριμένα, στις ΛΠ περιγράφουμε το πεδίο μας με ένα σύνολο από αξιώματα (axioms), στα οποία καταγράφουμε πληροφορία σχετικά με συγκεκριμένα άτομα/στιγμιότυπα του πεδίου μας. Αυτή αναφέρεται σε ποια (νοηματικά) σύνολα ανήκουν τα στιγμιότυπα και πώς συνδέονται μεταξύ τους, π.χ. **Doctor(LIZA)**, **loves(LIZA,NICK)**. Επίσης, μέσω αξιωμάτων καταγράφονται και οι συσχετίσεις εξειδίκευσης μεταξύ των (νοηματικών) συνόλων αυτών, π.χ. **Doctor $\sqsubseteq$ Person**. Επίσης, μπορούμε να διατυπώσουμε εκφράσεις που περιγράφουν νέα σύνολα και να τις χρησιμοποιήσουμε στην περιγραφή μας, π.χ. η έκφραση **≥ 2 hasHouse.Luxury** περιγράφει το σύνολο των ατόμων που έχουν δύο ή και περισσότερα πολυτελή σπίτια. Το τι μπορούμε να διατυπώσουμε εξαρτάται από τη γλώσσα ΛΠ που επιλέγουμε και οι ανάγκες μας θα καθορίσουν την επιλογή μας. Στην ονοματολογία των ΛΠ η “έννοια” που αντιστοιχεί στο νοηματικό σύνολο ονομάζεται *concept* και η σύνδεση μεταξύ ατόμων *role*. Τα άτομα ονομάζονται *individuals*. Στην **OWL-DL**, αντί για τους όρους “concept” και “role” χρησιμοποιούνται οι όροι *class* και *property*. Παρότι διατηρείται ο όρος “individual” και στην **OWL**, στην ονομασία γλωσσικών στοιχείων της που αναφέρονται σε αυτά χρησιμοποιείται ο όρος “object”, π.χ. “object property”. Στο Σχήμα 1.4, βλέπετε ένα παράδειγμα (μέρους) οντολογίας στο περιβάλλον ανάπτυξης οντολογιών **Protégé**, η οποία μπορεί να αναπαρασταθεί ως οντολογία διατυπωμένη σε **OWL-DL**.



Σχήμα 1.4: Ένα παράδειγμα οντολογίας στο περιβάλλον Protégé

Τα **πλαίσια** είναι ένας εναλλακτικός φορμαλισμός αναπαράστασης γνώσης που προσεγγίζει ακόμα περισσότερο το αντικειμενοστραφές μοντέλο. Στη μεθοδολογία αυτή, όλη η γνώση για μία οντότητα του γνωστικού πεδίου που μας ενδιαφέρει συγκεντρώνεται σε ένα **πλαίσιο** (frame) με τη μορφή ενός συνόλου **σχισμών** (slots). Η κάθε σχισμή αντιστοιχεί σε μια ιδιότητα της οντότητας ή σε συσχέτιση με άλλο πλαίσιο. Ειδικές συσχετίσεις προσδιορίζουν σχέση εξειδίκευσης και χρησιμοποιούνται για εκμετάλλευση της κληρονομικότη-

τας, όπως και στα σημασιολογικά δίκτυα και το αντικειμενοστραφές μοντέλο. Από πλευράς αναπαράστασης, όπως φαίνεται και στο Σχήμα 1.5, ένα πλαίσιο μπορεί να θεωρηθεί ως ένα σύνολο εγγραφών (records).

mammal

<i>isa</i>	animal
<i>has</i>	teeth
<i>drinks</i>	milk

cat

<i>isa</i>	mammal
<i>sound</i>	miaou

Σχήμα 1.5: Ένα παράδειγμα πλαισίων “cat” και “mammal”

Η κάθε εγγραφή αναπαριστάει μία σχισμή του πλαισίου. Τα πλαίσια μπορούν να αποτελούν τη βασική μεθοδολογία αναπαράστασης γνώσης ενός συστήματος ή, όπως έχει αναφερθεί στο [8], τη μεθοδολογία υλοποίησης των σημασιολογικών δικτύων.

Όπως είπαμε ήδη, στις μέρες μας έχουν μεγάλη άνθηση συστήματα ΤΝ που βασίζονται σε στατιστικές μεθόδους. Να πούμε, όμως, ότι όπου χρειάζεται σημασιολογική πληροφορία, γνώση που αναπαριστάται με τους παραπάνω φορμαλισμούς είναι κρίσιμη. Αυτή μπορεί να αποτελεί αυτοτελή πηγή γνώσης ή να συμπληρώνει τα εν λόγω συστήματα με στόχο τη βελτίωση της ποιότητας των απαντήσεων τους ή/και την επεξήγησή τους.

Συνοψίζοντας, στην περιοχή της Τεχνητής Νοημοσύνης που αφορά την Αναπαράσταση Γνώσης, πολλές από τις μεθοδολογίες που έχουν υιοθετηθεί βασίζονται σε έννοιες και τεχνικές που σχετίζονται άμεσα με το αντικειμενοστραφές μοντέλο. Με βάση το γεγονός ότι συχνός στόχος των μεθοδολογιών αναπαράστασης γνώσης, καθώς και ο στόχος του αντικειμενοστραφούς μοντέλου, είναι η περιγραφή οντοτήτων και εννοιών και της πληροφορίας που σχετίζονται με αυτές, η παρατήρηση αυτή δεν είναι κάτι που πρέπει να μας ξαφνιάζει.

1.4.3 Βάσεις Δεδομένων και ΟΟ

Το αντικειμενοστραφές μοντέλο εντάχθηκε και στη σχεδίαση των βάσεων δεδομένων. Συγκεκριμένα, συναντάμε τις εξής δύο μορφές:

- Αντικειμενοστραφείς βάσεις δεδομένων (object-oriented databases)
- Αντικειμενοστραφείς-Σχεσιακές βάσεις δεδομένων (object-relational databases)

Στις Αντικειμενοστραφείς βάσεις δεδομένων, όλη η πληροφορία της βάσης μοντελοποιείται μέσω κλάσεων και φυλάσσεται μέσω αντικειμένων. Οι γλώσσες ορισμού του σχήματος της βάσης υποστηρίζουν το αντικειμενοστραφές μοντέλο και με αυτό περιγράφεται η οργάνωση της πληροφορίας σε λογικό επίπεδο. Δηλαδή,

δίνεται η δυνατότητα ορισμού κλάσεων, ιεραρχιών κλάσεων και αντικείμενων κλάσεων. Επίσης, ορίζονται τα χαρακτηριστικά (attributes) των αντικειμένων και οι μέθοδοι με τις οποίες μπορεί να τα χειριστεί κανείς. Σε ένα σύστημα διαχείρισης αντικειμενοστραφούς βάσης δεδομένων, τα πάντα φυλάσσονται ως αυτόνομα αντικείμενα, δεν υπάρχουν πίνακες-σχέσεις με γραμμές, όπως υπάρχουν στο σχεσιακό μοντέλο. Τα αντικείμενα ενδέχεται να συνδέονται μεταξύ τους. Το κάθε αντικείμενο, μαζί με την πληροφορία που το αφορά, φυλάσσεται στη βάση, στον σκληρό δίσκο. Τα αντικείμενα παραμένουν εν ζωή ανεξάρτητα από την εκτέλεση προγράμματος που αλληλεπιδρά μαζί τους και διατηρούν την τελευταία κατάσταση που είχαν αποκτήσει πριν τη λήξη της εκτέλεσης ενός προγράμματος. Οι αντικειμενοστραφείς βάσεις δημιουργήθηκαν προκειμένου να αλληλεπιδράσουν ομαλά με προγράμματα γραμμένα σε γλώσσες αντικειμενοστραφούς προγραμματισμού. Χρησιμοποιώντας μια αντικειμενοστραφή βάση, οι κλάσεις και τα αντικείμενα του αντικειμενοστραφούς προγράμματος θα μπορούσαν να είναι σε άμεση συσχέτιση με οντότητες της βάσης, πράγμα που θα έκανε την επικοινωνία ταχύτερη. Γι' αυτόν τον λόγο, εμφανίστηκαν από πολύ νωρίς. Ήδη, στο τέλος της δεκαετίας του '80, το σύστημα O₂ ήταν ευρέως γνωστό. Στις μέρες μας, χρησιμοποιούνται σε συνδυασμό με προγράμματα γραμμένα σε γλώσσες όπως η C#, η SWIFT, η Java κ.ά. Υπάρχουν πολλά συστήματα διαθέσιμα, άλλα δωρεάν, άλλα εμπορικά, με αρκετά δημοφιλές σύστημα το MongoDB. Παρ' όλα αυτά, οι σχεσιακές βάσεις, λόγω του ότι ήταν ήδη πολύ διαδεδομένες, εξακολουθούν να έχουν τη μεγαλύτερη χρήση.

Ενδιάμεσο μοντέλο στο αντικειμενοστραφές και το σχεσιακό είναι αυτό που προσφέρουν οι Αντικειμενοστραφείς-Σχεσιακές βάσεις δεδομένων. Το μοντέλο αυτό είναι κατά βάση σχεσιακό, μόνο που επιτρέπεται να επεκτείνεται το σύστημα των τύπων που μπορούν να χρησιμοποιηθούν για τον ορισμό των πεδίων μιας σχέσης, μέσω ΑΤΔ που ορίζονται από τον χρήστη. Σε αυτήν την περίπτωση, τα πεδία της σχέσης επεκτείνονται ώστε να συμπεριλάβουν τα χαρακτηριστικά του ΑΤΔ. Επίσης, μπορεί να οριστεί μια σχέση ως εξειδίκευση μιας άλλης. Σε αυτήν την περίπτωση, επαυξάνεται και με τα πεδία της άλλης και απαντήσεις σε ερωτήματα που αναφέρονται στη γενικότερη σχέση συμπεριλαμβάνουν και πλειάδες από την ειδικότερη σχέση, π.χ. αν έχουμε μια σχέση "Employee" και μια εξειδίκευσή της "Manager", τότε στο ερώτημα "Δώσε μου τα ονόματα όλων των Employees", επιστρέφονται και τα ονόματα από τη σχέση "Manager". Γνωστό Αντικειμενοστραφές-Σχεσιακό σύστημα είναι το PostgreSQL — που στην εμπορική έκδοσή του ονομάζεται Illustra — και εμφανίστηκε στη δεκαετία '90.

Κλείνοντας τη συζήτηση για τη μείξη βάσεων δεδομένων και του αντικειμενοστραφούς μοντέλου, θα πρέπει να κάνουμε μια αναφορά στις αντικειμενοστραφείς επεκτάσεις της γλώσσας datalog. Η datalog είναι ένα μοντέλο βάσεων δεδομένων βασισμένο στον λογικό προγραμματισμό — για την ένταξη αντικειμένων στον λογικό προγραμματισμό θα μιλήσουμε σε επόμενη παράγραφο. Ένα πρόγραμμα datalog αποτελείται από γεγονότα (facts) της μορφής, π.χ.

employee(nick).

και κανόνες (rules), π.χ.

colleagues(X,Y) :- works_in(X,C) & works_in(Y,C) & X \= Y.

Τα παραπάνω μπορούμε να πούμε ότι αντιστοιχούν σε σχέσεις μιας σχεσιακής βάσης. Αυτό που προσφέρει η datalog και την κάνει ιδιαίτερα ισχυρή από πλευράς εκφραστικότητας είναι ότι επιτρέπει αναδρομή στους κανόνες. Αυτό καθιστά δυνατό τον υπολογισμό της κλειστότητας μιας σχέσης. Η datalog, όπως και ο λογικός προγραμματισμός, γενικότερα, δεν έχει τύπους, έτσι η ένταξη του αντικειμενοστραφούς μοντέλου δεν δείχνει άμεση επέκταση. Παρ' όλα αυτά, η σχετική έρευνα ξεκίνησε ήδη από τη δεκαετία του '80. Μάλιστα, έχουν κάνει την εμφάνισή τους σύγχρονα συστήματα, όπως το .QL, που συνδυάζουν το αντικειμενοστραφές μοντέλο με τη datalog, καθότι η datalog χρησιμοποιείται σε ένα εύρος δύσκολων εφαρμογών που έλκουν έντονο ενδιαφέρον στις μέρες μας, όπως η ανάλυση προγραμμάτων.

1.5 Λίγα Λόγια για την Ιστορία

Ας κάνουμε μια σύντομη ιστορική αναδρομή για τις γλώσσες OOP. Ο αντικειμενοστραφής προγραμματισμός έγινε ευρέως γνωστός στα μέσα της δεκαετίας του '80 και έως το τέλος της δεκαετίας του '90 είχε πλέον

καθιερωθεί. Η γλώσσα προγραμματισμού C++ συνέβαλε στη μεγάλη εξάπλωση. Η γλώσσα αυτή βασίζεται σε επέκταση του διαδικαστικού προγραμματισμού με γλωσσικές δομές του αντικειμενοστραφούς μοντέλου. Μεταγενέστερα, στη γλώσσα Java το αντικειμενοστραφές μοντέλο είναι το βασικό πλαίσιο μέσα στο οποίο ο προγραμματιστής αναπτύσσει τον κώδικά του. Η Java ενσωμάτωσε ήδη υπάρχουσα εμπειρία από την ανάπτυξη εφαρμογών σε αντικειμενοστραφές περιβάλλον και ενίσχυσε την εξάπλωση του αντικειμενοστραφούς προγραμματισμού. Πολλές νέες γλώσσες αντικειμενοστραφούς προγραμματισμού αναπτύχθηκαν και πολλές από τις ήδη υπάρχουσες γλώσσες αναγκάστηκαν να επεκταθούν με αντικειμενοστραφή χαρακτηριστικά. Γενικά, γλώσσες διαφόρων ειδών προγραμματισμού επεκτάθηκαν ώστε να υποστηρίζουν το αντικειμενοστραφές μοντέλο.

Αναδρομή στην Εξέλιξη της C++ (C with Classes): Για να δούμε, όμως, συνοπτικά τι οδήγησε στη γέννηση της C++. Όπως ο ίδιος ο δημιουργός της, ο Bjarne Stroustrup, έχει δηλώσει [9, 10], η δημιουργία της C++ προήλθε από την ανάγκη για την ανάπτυξη μιας προσομοίωσης για τη μελέτη της εκτέλεσης του πυρήνα (kernel) του λειτουργικού συστήματος UNIX σε καταναμημένο περιβάλλον ενός τοπικού δικτύου (local area network). Η προσπάθεια ξεκίνησε το 1979 στα Bell Labs όπου, τότε, εργαζόταν ο Stroustrup.

Για τον σχεδιασμό της γλώσσας, ήταν καθοριστική η εμπειρία του από τα χρόνια του διδακτορικού του στο Cambridge. Στο πλαίσιο της διατριβής του, έπρεπε να μελετήσει εναλλακτικούς τρόπους για την οργάνωση λογισμικού συστήματος καταναμημένων συστημάτων. Έχοντας ως αρχικό εργαλείο τη γλώσσα SIMULA για να αναπτύξει τις προσομοιώσεις, εκτίμησε την ευκολία με την οποία μπορούσε να αναπτύξει τον κώδικά του και την ασφάλεια από λάθη που του παρείχε το σύστημα τύπων της γλώσσας. Για την ακρίβεια, η δυνατότητα ορισμού κλάσεων του επέτρεψε να αντιστοιχίσει τις έννοιες του προβλήματος της εφαρμογής του απευθείας σε στοιχεία της γλώσσας, κάτι το οποίο τον οδήγησε επιπλέον σε πιο ευανάγνωστο κώδικα. Από την άλλη, η ταχύτητα της εκτέλεσης ήταν απαγορευτική για το έργο του.

Τελικά, για να πετύχει την απόδοση που χρειαζόταν για τη μελέτη του, αναγκάστηκε να ξαναγράψει τον κώδικα για την προσομοίωση στη γλώσσα BCPL (που θεωρείται ο πρόγονος της γλώσσας C). Αυτή είχε χαρακτηριστικά που επέτρεπαν ανάπτυξη λογισμικού χαμηλού επιπέδου, οπότε μπορούσε κανείς να γράψει ταχύτερο κώδικα με την εκμετάλλευσή τους. Η ανάπτυξη του κώδικα αυτού, όμως, τον είχε δυσκολέψει πάρα πολύ. Κατά συνέπεια, όταν προέκυψε η ανάγκη της μελέτης του καταναμημένου πυρήνα του UNIX στα Bell Labs, αποφάσισε να επεκτείνει την ήδη υπάρχουσα, πιο φιλική από την BCPL, γλώσσα C με χαρακτηριστικά της γλώσσας SIMULA τα οποία του είχαν επιτρέψει να αναπτύξει πολύ ευκολότερα τον κώδικα της προσομοίωσης στο πλαίσιο του διδακτορικού του.

Έτσι, το 1980 ήταν έτοιμη η γλώσσα “C with classes” υλοποιημένη μέσω ενός προεπεξεργαστή (pre-processor). Στη συνέχεια, και μόνο όταν η γλώσσα υποστήριξε εικονικές συναρτήσεις (virtual functions) που επέτρεψαν υποστήριξη πολυμορφισμού, το 1983 χαρακτηρίστηκε ως γλώσσα αντικειμενοστραφούς προγραμματισμού. Ονομάστηκε C++ το 1984 και το 1985 εμφανίστηκε η πρώτη εμπορική έκδοσή της (με όνομα Cfront). Από τις αρχές της δεκαετίας του ’90, άρχισαν οι προσπάθειες τυποποίησής της. Η έκδοση του πρώτου ISO προτύπου της έγινε το 1998. Το πρότυπο αυτό είναι γνωστό ως C++98. Σημαντικό πρότυπο της γλώσσας θεωρείται το C++11 στο οποίο ενσωματώθηκαν πολλά χαρακτηριστικά λόγω μεγαλύτερης εμπειρίας από τη χρήση του αντικειμενοστραφούς προγραμματισμού και όχι μόνο. Τρέχον πρότυπο είναι το C++20 (ISO/IEC 14882:2020) με στόχο αναθεώρησης κάθε τρία χρόνια (άρα το επόμενο πρότυπο αναμένεται όχι νωρίτερα από το 2023).

Συνοψίζοντας τις ιδιότητές της, χρησιμοποιώντας εκφράσεις από τον ίδιο τον Stroustrup, η C++ σχεδιάστηκε ώστε να προσφέρει τα εργαλεία που προσφέρει η SIMULA για την οργάνωση κώδικα και ταυτόχρονα να επιτρέπει την ταχύτητα και την ευελιξία που προσφέρει η C για τον προγραμματισμό συστήματος.

Κλάσεις και η Γλώσσα SIMULA: Πριν συνεχίσουμε την ιστορική αναδρομή, μια που στην παράγραφο αυτή χρειάζεται να είμαστε πιο ακριβείς όσον αφορά το όνομα “SIMULA”, πρέπει να κάνουμε την εξής παρατήρηση. Στο παραπάνω, όπως και συχνά, με το όνομα SIMULA αναφερόμαστε, στην πράξη, στη γλώσσα SIMULA 67 που σχεδιάστηκε και υλοποιήθηκε από τους Νορβηγούς επιστήμονες Kristen Nygaard και Ole-Johan Dahl. Είναι, λοιπόν, η γλώσσα SIMULA 67 η “μητέρα” του Αντικειμενοστραφούς Προγραμματισμού;

Η γλώσσα αυτή έδινε στον προγραμματιστή τη δυνατότητα να ορίσει κλάσεις, αντικείμενα, κληρονομικότητα και πολυμορφισμό μέσω εικονικών συναρτήσεων. Συγκεκριμένα, η SIMULA 67 προσφέρει τη γλωσσική δομή

class

για τον ορισμό κλάσεων. Κατόπιν, μπορούν να δημιουργηθούν αντικείμενα της κλάσης. Όμως δεν υπάρχει μηχανισμός προστασίας για τα ονόματα μέσα σε μια κλάση και έτσι ο χρήστης μπορεί να προσπελάσει οποιοδήποτε μέρος της υλοποίησης. Κατά συνέπεια, η αφαίρεση στα δεδομένα που επιτυγχάνεται με τη δυνατότητα ορισμού κλάσεων δεν είναι δυνατόν να προστατευθεί.

Πρέπει να τονιστεί η επιρροή της SIMULA από τη γλώσσα ALGOL 60 από την οποία προέρχεται. Η ALGOL 60 εισήγαγε τον δομημένο προγραμματισμό (structured programming), με στόχο την αποφυγή της διαρκούς χρήσης GO TO εντολών για τον καθορισμό του ελέγχου της εκτέλεσης του προγράμματος. Επίσης, έδωσε τη δυνατότητα “πακεταρίσματος” κώδικα σε blocks, που προσδιορίζονταν με τις λέξεις-κλειδιά begin, end, και έκαναν τον κώδικα ακόμα πιο ευανάγνωστο. Στις πρώτες εκδόσεις της, η γλώσσα SIMULA δεν ήταν τίποτα άλλο από επέκτασή της και ο επιπλέον κώδικας προσημαινόταν με την έκφραση “SIMULA begin”, δηλώνοντας ότι το block που ακολουθεί περιέχει κώδικα της γλώσσας SIMULA.

Πάμε, λοιπόν, πιο πίσω στον χρόνο για να δούμε τι ήταν εκείνο που έδωσε το έναυσμα και την έμπνευση για τη δημιουργία της SIMULA. Πάμε άλλα 20 χρόνια, περίπου, πίσω, πριν την πρώτη σκέψη για τη C με κλάσεις, στο 1961, στη Νορβηγία.

Για την ακρίβεια, αρχικά, πάμε ακόμα νωρίτερα. Στο δεύτερο μισό της δεκαετίας του '50, στο κέντρο που τότε απασχολείτο ο Kristen Nygaard, στο πλαίσιο ενασχόλησης με προβλήματα επιχειρησιακής έρευνας (Operations Research - OR), έκαναν τη διαπίστωση ότι ο πιο κατάλληλος τρόπος αντιμετώπισης προβλημάτων OR μεγάλης κλίμακας ήταν μέσω προσομοίωσης. Ταυτόχρονα, η χρήση των υπολογιστών εξαπλωνόταν. Δεν υπήρχαν, όμως, κατάλληλα εργαλεία για την άμεση αναπαράσταση των εννοιών και των οντοτήτων των προβλημάτων αυτών, κάτι που θα έκανε την ανάπτυξη του κώδικα που θα υλοποιούσε αυτές τις προσομοιώσεις ευκολότερη. Οι ίδιες ανάγκες υπήρχαν και σε προσομοιώσεις που απαιτούντο για την επίλυση προβλημάτων ενδιαφέροντος γενικότερου από την OR.

Οπότε, το 1961, στο Norwegian Computing Center (NCC) πλέον, ως επικεφαλής του, ο Kristen Nygaard, σε συνεργασία με τον Ole-Johan Dahl, ξεκίνησαν να σχεδιάζουν τη SIMULA με στόχο να αναπτύξουν ένα εργαλείο λογισμικού το οποίο να επιτρέπει την έκφραση των εννοιών που συναντώνται στις προσομοιώσεις, αλλά ταυτόχρονα να μπορεί κάποιος και να προγραμματίσει, γενικότερα. Οι σχεδιαστικές επιλογές για την κάλυψη της πρώτης ανάγκης βασιζόταν σε ένα μαθηματικό μοντέλο για δίκτυο διακριτών γεγονότων (discrete event network). Η γλώσσα πάνω στην οποία θα βασιζόταν ήταν η ALGOL 60, γιατί εξυπηρετούσε τις ανάγκες που προαναφέραμε. Στο τέλος του 1964, και αφού είχαν προηγηθεί διάφορες άλλες υλοποιήσεις, ο μεταγλωττιστής της πρώτης έκδοσης της γλώσσας, SIMULA I, ήταν έτοιμος. Στην έκδοση αυτή της γλώσσας, ήταν δυνατό να περιγραφούν δυναμικά και στατικά συστατικά της επίλυσης ενός προβλήματος, καθώς και ένα σύνολο από διεργασίες. Στο πλαίσιο της ορολογίας της γλώσσας, αυτά λεγόταν “stations”, “customers” και “activities”, αντίστοιχα.

Μετά τη χρήση της γλώσσας κατά το διάστημα 1964-1966, άρχισε να υπάρχει ενδιαφέρον από τους δημιουργούς της να την εξελίσουν σε γλώσσα προγραμματισμού γενικού σκοπού. Εντοπίστηκαν, λοιπόν,

οι αδυναμίες της όσον αφορά τον σκοπό αυτό. Ταυτόχρονα, το 1965, ο Hoare έκανε μια δημοσίευση σχετικά με τις “Κλάσεις Εγγραφών” (Record Classes) — θα μιλήσουμε σε λίγο γι’ αυτές — που, τόσο ως έννοια, αλλά και ως υλοποίηση, έλυναν υπάρχοντα προβλήματα της SIMULA. Επίσης, αργότερα, οι “Κλάσεις Εγγραφών” χρησιμοποιούσαν την έννοια των “Υποκλάσεων Εγγραφών” που φαινόταν χρησιμη σε πολλά προβλήματα. Έτσι, τέθηκαν για την εξέλιξη της SIMULA νέες σχεδιαστικές αρχές που ξέφευγαν από το προηγούμενο μαθηματικό μοντέλο και βασιζόντουσαν σε διεργασίες και υποδιεργασίες, γενικότερα, που αλληλεπιδρούν μεταξύ τους. Για την ακρίβεια, εισήχθησαν “Κλάσεις Αντικειμένων”, κατ’ αντιστοιχία των “Κλάσεων Εγγραφών”, και “Υποκλάσεις” τους. Η νέα έκδοση της γλώσσας που βασίστηκε στις έννοιες και τις αρχές αυτές είναι η SIMULA 67, που στη συνέχεια αποτέλεσε τη βασική αναφορά για τις γλώσσες αντικειμενοστραφούς προγραμματισμού. Στο σημείο αυτό να κάνουμε μια σύντομη αναφορά στο γεγονός ότι το σύστημα εισόδου/εξόδου (I/O system) της γλώσσας είναι βασισμένο πάνω σε αυτές ακριβώς τις έννοιες των κλάσεων, των αντικειμένων και των υποκλάσεων. Είναι, δηλαδή, υλοποιημένο με εκμετάλλευση του OOP.

Κλάσεις Εγγραφών (Record Classes): Εν συντομία, λοιπόν, για να μην ξεφύγουμε από την εστίασή μας, θα κάνουμε μια αναφορά στις Κλάσεις Εγγραφών του C. A. R. Hoare, μια που δείχνει να επηρέασαν, εκτός άλλων, την ονοματοδοσία του Αντικειμενοστραφούς Προγραμματισμού. Στο άρθρο με το οποίο παρουσιάστηκαν ([11]), προτείνονται με στόχο να μπορούν να αναπαρασταθούν σύνθετα αντικείμενα στον υπολογιστή. Η γλώσσα η οποία θεωρείται ως υπόβαθρο για την πρόταση είναι, πάλι, η ALGOL 60.

Αυτά τα σύνθετα αντικείμενα είναι εγγραφές που αποτελούνταν από δομικά στοιχεία, πεδία (fields) κλασικών αριθμητικών τύπων, όπως integer και boolean, π.χ. ηλικία, μισθός. Επιπλέον, όμως, μέσω ενός πεδίου ειδικού τύπου αναφοράς, τα αντικείμενα θα μπορούσαν να συσχετιστούν μεταξύ τους μέσω οποιωνδήποτε συσχετίσεων, π.χ. τη συσχέτιση “γονέας” μεταξύ “ανθρώπων”. Κατά την εκτέλεση ενός προγράμματος μπορεί να δημιουργηθεί ένας οποιοσδήποτε αριθμός αντικειμένων, που με βάση αυτές τις συσχετίσεις δομεί ένα δίκτυο. Τα αντικείμενα αποτελούν τιμές μεταβλητών αναφοράς. Η δημιουργία των αντικειμένων αυτών, καθώς και η καταστροφή τους μπορεί να γίνει από το πρόγραμμα ανάλογα με τις απαιτήσεις του προβλήματος.

Κάθε σύνθετο αντικείμενο πρέπει να ανήκει σε μια “Κλάση Εγγραφών” που έχει οριστεί από τον προγραμματιστή, π.χ. “Person”, “Cat”. Σε ένα πρόγραμμα, μπορούν να οριστούν οσεσδήποτε κλάσεις εγγραφών, ξένες μεταξύ τους, κάθε μία από τις οποίες παίρνει ένα όνομα. Λίγο αργότερα, ο Hoare, προτείνει την έννοια των υποκλάσεων μιας κλάσης, με το σκεπτικό ότι τα αντικείμενα της κλάσης μπορεί να ανήκουν σε υποπεριπτώσεις κλάσεων, οι οποίες αναφέρονται κατά τον ορισμό της κλάσης, δηλώνοντας τα ονόματά τους. Μια κλάση η οποία έχει υποκλάσεις καλείται *σύνθετη*, σε αντιδιαστολή με τις άλλες που δεν έχουν υποκλάσεις και καλούνται *απλές*. Παρότι δεν προτείνεται, αρχικά, σύνδεση των κλάσεων εγγραφών με διαδικασίες της γλώσσας, η υποστήριξη αυτού του χαρακτηριστικού αναφέρεται στις δυνατές επεκτάσεις. Το ίδιο ισχύει και για τη δυνατότητα καθορισμού αρχικών τιμών όταν δημιουργείται ένα νέο αντικείμενο-εγγραφή.

Οι περιοχές εφαρμογών των “Κλάσεων Εγγραφών” που προτεινότουσαν από τον Hoare ήταν η κλασική επεξεργασία δεδομένων, π.χ. έκδοση τιμολογίου, η ανάκτηση πληροφοριών, π.χ. επεξεργασία πολύπλοκων πινάκων, η υλοποίηση μεταγλωττιστών, π.χ. χρήση τους για την πληροφορία που μπορεί να συνοδεύει μια μεταβλητή, καθώς και η αποκαλούμενη “αυτοματοποιημένη σχεδίαση”, που πρακτικά αναφερόταν σε εφαρμογές γραφικών, π.χ. γραμμές, πολύγωνα. Δεσπόζουσα θέση στις προτεινόμενες εφαρμογές είχαν οι προσομοιώσεις. Η ιδέα ήταν ότι στις προσομοιώσεις σύνθετων συστημάτων υπάρχει ανάγκη δημιουργίας υπολογιστικών ανάλογων των αντικειμένων του πραγματικού κόσμου, ώστε οι διαδικασίες που εφαρμόζονται σε αυτά να δείχνει ότι λειτουργούν με ρεαλιστικό τρόπο.

Οι Πρώτες Γλώσσες Αντικειμενοστραφούς Προγραμματισμού: Μιλήσαμε ήδη για τη γέννηση της C++ (δεκαετία ’80) καθώς και την επιρροή της από τη γλώσσα SIMULA 67 (δεκαετία ’60). Πήγαμε, όμως,

απευθείας στη C++ από τη SIMULA; Τι έγινε ενδιάμεσα; Πολλά, πάρα πολλά. Απλά, δεν πήραν τόσο μεγάλη έκταση όση προέκυψε μετά την εμφάνιση της C++, η οποία, ας μην ξεχνάμε, βασίστηκε στην, ήδη διαδεδομένη, C.

- Σημαντική εμφάνιση κατά τη δεκαετία του '70 στον χώρο του αντικειμενοστραφούς προγραμματισμού έκανε η γλώσσα **Smalltalk**. Οι πρώτες εκδόσεις της, Smalltalk72 και Smalltalk74, δεν υποστήριζαν κληρονομικότητα. Είχαν, όμως, από την αρχή χαρακτηριστικά που απέβησαν πολύ κρίσιμα για τις μετέπειτα γλώσσες αντικειμενοστραφούς προγραμματισμού: τα πάντα σε ένα πρόγραμμα είναι αντικείμενα (ακόμα και οι ακέραιοι) και έγινε η εισαγωγή του περάσματος μηνυμάτων (message passing) μεταξύ των αντικειμένων και του πολυμορφισμού.

Η έκδοση Smalltalk80 είναι ουσιαστικά η έκδοση η οποία έχει όλα τα χαρακτηριστικά που προσδιορίζουν τη γλώσσα Smalltalk, τα οποία συμπεριλαμβάνουν εκείνα του αντικειμενοστραφούς προγραμματισμού. Δηλαδή, έδινε τη δυνατότητα ορισμού κλάσεων, αντικειμένων και κληρονομικότητας, δυνατότητα ορισμού δεδομένων στιγμιотύπων αλλά και κλάσεων — αντίστοιχα και για μεθόδους — και δυνατότητα ορισμού μετακλάσεων. Επίσης, έδινε τη δυνατότητα χρήσης των κλάσεων για υποστήριξη ταυτόχρονης εκτέλεσης (concurrency).

Όσον αφορά την προέλευσή της, η προσπάθεια που οδήγησε, μεταξύ άλλων, στη δημιουργία της Smalltalk έγινε στο πλαίσιο ενός έργου στο Xerox PARC με επικεφαλής τον Alan Kay, το Dynabook. Αυτό είχε ως στόχο να δημιουργηθεί ένας μικρός προσωπικός υπολογιστής με τη μορφή που έχουν τα σύγχρονα tablets. Παρότι το έργο εντασσόταν σε ένα πρόγραμμα για στρατιωτικούς σκοπούς, η έμπνευσή του προερχόταν από το διδακτορικό του Alan Kay, όπου προτεινόταν κάτι αντίστοιχο για χρήση στην εκπαίδευση. Αξίζει να σημειωθεί ότι η Smalltalk, πέρα από γλώσσα, ήταν ένα ολόκληρο σύστημα που προσέφερε ακόμα και ένα παραθυρικό περιβάλλον ανάπτυξης προγραμμάτων.

- Στο πλαίσιο της ιστορικής αναδρομής, θα θέλαμε να αναφέρουμε και την **Object Pascal**. Αυτή αναπτύχθηκε κατά τη δεκαετία του '80 από την Apple Computer Inc. σε συνεργασία με τον Niklaus Wirth, τον εμπνευστή της Pascal, στο πλαίσιο του αντικειμενοστραφούς συστήματος της εταιρείας. Παρότι σύντομα εγκαταλείφθηκε από την Apple, αποτέλεσε τη βάση για άλλες γλώσσες, όπως η Turbo Pascal. Η Object Pascal είχε πολύ στοιχειώδεις αντικειμενοστραφείς δυνατότητες. Δηλαδή, ενόσω επέτρεπε τον ορισμό κλάσεων, αντικειμένων και κληρονομικότητας, δεν παρείχε εγκλωβισμό στις κλάσεις και της έλειπε η ευρύτερη λειτουργικότητα που είχε, για παράδειγμα, η Smalltalk.
- Η Smalltalk, μεταξύ άλλων πολλών γλωσσών, επηρέασε και τον σχεδιασμό της γλώσσας **Objective-C**. Η γλώσσα αυτή, δεκαετίας του '80, σχεδιάστηκε από τον Brad Cox. Έδινε έμφαση στο πέρασμα μηνυμάτων και ενέταξε αντικειμενοστραφή χαρακτηριστικά στη γλώσσα C. Υιοθετήθηκε, αργότερα, στην ανάπτυξη του λειτουργικού συστήματος NeXTSTEP της εταιρείας NeXT Inc. και, στη συνέχεια, στην ανάπτυξη των συστημάτων της Apple. Η γλώσσα, όμως, δεν εξελίχθηκε ιδιαίτερα και έτσι, το 2014, τη θέση της στην εταιρεία πήρε μια νέα γλώσσα, η Swift που ενσωμάτωσε πολλά χαρακτηριστικά από νεότερες γλώσσες προγραμματισμού και επέτρεψε ανάπτυξη ασφαλέστερου κώδικα με αυτόματη διαχείριση μνήμης και αρχικοποίηση μεταβλητών. Χαρακτηριστικό είναι ότι κατά την παρουσίασή της από την Apple, το 2014, η βάση της δηλώθηκε να είναι μια “Objective C without (the baggage of) C”.
- Δεν μπορούμε, όμως, να μην κάνουμε αναφορά και στη γλώσσα **ADA**. Παρότι η γλώσσα αυτή, στις αρχικές εκδόσεις της, δεν υποστήριζε πλήρως το αντικειμενοστραφές μοντέλο — δεν προσέφερε κληρονομικότητα — πολλά από τα χαρακτηριστικά της επηρέασαν τις μετέπειτα γλώσσες αντικειμενοστραφούς προγραμματισμού. Η γλώσσα αναπτύχθηκε στις αρχές της δεκαετίας του '80. Η αιτία ήταν η ανάγκη του Υπουργείου Άμυνας των ΗΠΑ να βάλει τάξη στη χαοτική κατάσταση που επικρατούσε στο Υπουργείο από την πληθώρα γλωσσών προγραμματισμού και

σχετικών έργων. Για τον σκοπό αυτό, έκαναν αίτημα για προτάσεις, δίνοντας κάποιες προδιαγραφές που συνέταξαν και οι οποίες θεωρούσαν ότι ικανοποιούσαν τις ανάγκες τους σε ευρύ φάσμα. Η γλώσσα που πρότεινε ο Dr. Jean Ichbiah ήταν εκείνη που επελέγη και έτσι προέκυψε η ανάπτυξη της ADA από την ομάδα του στο CII-Honeywell-Bull στη Γαλλία. Το πρώτο πρότυπό της ήταν η ADA 83.

Ένα πολύ βασικό κριτήριο του σχεδιασμού της ADA ήταν η ανάπτυξη ασφαλούς κώδικα. Σε αυτό συνέβαλαν τα πακέτα (packages) που ήταν η βασική καινοτομία της γλώσσας. Αυτά παρείχαν εγκλωβισμό και επέτρεπαν τμηματοποίηση κώδικα. Διέκριναν την πληροφορία τους σε ορατή από τον υπόλοιπο κώδικα και μη ορατή, εγκλωβισμένη στο πακέτο. Ένα πρόγραμμα ADA περιελάμβανε ένα σύνολο από πακέτα. Στις αρχές ανάπτυξης λογισμικού σε ADA προτεινόταν η επαναχρησιμοποίηση κώδικα και αυτό επιτυγχάνετο επαναχρησιμοποιώντας πακέτα. Πρέπει ακόμα να σημειωθεί η δυνατότητα ορισμού γενικεύσεων πακέτων (generic packages) στα οποία παραμετροποιούντουσαν διάφορα χαρακτηριστικά του πακέτου, όπως τύποι, μεγέθη πινάκων κ.ά. Οι παράμετροι παίρνουν τιμές στη χρήση του πακέτου, οπότε και συγκεκριμενοποιείται το πακέτο από τον μεταγλωττιστή.

Στην ADA μπορούμε να ορίσουμε ΑΤΔ. Ένας ΑΤΔ μπορεί να υλοποιηθεί από ένα πακέτο. Τα ορατά μέρη ενός ΑΤΔ δηλώνονται στο ορατό μέρος του πακέτου, ενώ οι λεπτομέρειες υλοποίησης εγκλωβίζονται στο σώμα του πακέτου. Παρότι στις πρώτες εκδόσεις της δεν υποστήριζε κληρονομικότητα, παρείχε τη δυνατότητα ορισμού υποτύπων (subtypes) τύπων που αποτελούν εξειδικεύσεις των τύπων αυτών υπό τη μορφή περιορισμών τιμών. Πρέπει να αναφερθεί, όμως, ότι η ADA 2012 υποστηρίζει πλήρως τον αντικειμενοστραφή προγραμματισμό, ακόμα και πολλαπλή κληρονομικότητα. Η γλώσσα αυτή έχει πολλά χαρακτηριστικά που απαιτούνται από μια σύγχρονη γλώσσα για την ανάπτυξη συστημάτων για την παραγωγή, τα οποία διευκολύνουν την ανάλυση του προγράμματος και οδηγούν σε ασφαλή κώδικα.

- Η γλώσσα **Modula-2** επιτρέπει τον ορισμό αφηρημένων αντικειμένων με παρόμοιο τρόπο. Η γλώσσα αυτή αναπτύχθηκε στο τέλος της δεκαετίας '70 έως τα μέσα της δεκαετίας '80 από τον Niklaus Wirth στο ETH, στη Ζυρίχη. Η Modula-2 εισήγαγε στοιχεία (modules), που προσφέρουν εγκλωβισμό. Έτσι, σε ένα στοιχείο, θα μπορούσαμε να ορίσουμε ΑΤΔ φυλάσσοντας κρυμμένη τη λεπτομερή αναπαράστασή τους και αφήνοντας ορατές μόνο τις λειτουργίες που σχετίζονται με τη συμπεριφορά τους. Να αναφέρουμε ότι η γλώσσα **Modula-3**, που είναι εξέλιξη της Modula-2, εκφράζει πιο άμεσα τα αντικειμενοστραφή χαρακτηριστικά, επιτρέποντας να ορίσουμε τύπους αντικειμένων (object types). Επίσης, ένας τύπος μπορεί να είναι υποτύπος ενός άλλου με τη λογική της κληρονομικότητας. Η γλώσσα αυτή, όμως, παρότι προσέφερε και προσφέρει ένα σημαντικό ερευνητικό υπόβαθρο, δεν έχει χρησιμοποιηθεί ευρέως στην παραγωγή.

Όσες γλώσσες παρουσιάσαμε μέχρι τώρα στην ιστορική αναδρομή αυτή, είναι γλώσσες διαδικαστικού προγραμματισμού. Όμως, αντικειμενοστραφή χαρακτηριστικά εντάχθηκαν σε γλώσσες συναρτησιακού προγραμματισμού (functional programming) αλλά ακόμα και στον λογικό προγραμματισμό (logic programming). Το πρώτο δεν πρέπει να μας ξενίζει, διότι η έννοια των τύπων είναι κρίσιμη για τις γλώσσες συναρτησιακού προγραμματισμού. Ως προς το δεύτερο, όπου τύποι δεν υποστηρίζονται συνήθως από τις γλώσσες του λογικού προγραμματισμού, ο λόγος, μεταξύ άλλων, είναι ότι οι αντικειμενοστραφείς επεκτάσεις διευκόλυναν περισσότερο στην αναπαράσταση γνώσης, στην οποία ο λογικός προγραμματισμός συχνά βρίσκει εφαρμογή.

- Ο **συναρτησιακός προγραμματισμός** είναι ένα είδος προγραμματισμού που βασίζεται στις (μαθηματικές) συναρτήσεις. Δηλαδή, η βασική έννοιά του είναι η συνάρτηση. Συναρτήσεις μπορεί εμφανίζονται ακόμα και ως ορίσματα άλλων συναρτήσεων (υψηλής τάξης - higher order). Ο συναρτησιακός προγραμματισμός έχει μαθηματικό υπόβαθρο τον λάμδα λογισμό (lambda calculus). Εμφανίστηκε τη δεκαετία του '50. Συγκεκριμένα, η πρώτη γλώσσα συναρτησιακού

προγραμματισμού είναι η LISP, που, όμως, το θεωρητικό μοντέλο στο οποίο βασίζεται είναι η θεωρία του Kleene για αναδρομικές συναρτήσεις πρώτης τάξης. Ο δημιουργός της ήταν ο John McCarthy στο MIT και ο σκοπός που σχεδιάστηκε ήταν για να αναπτυχθεί ένα σύστημα υπολογισμού σε συμβολικά δεδομένα (symbolic data), αρχίζοντας με συμβολική παραγοντοποίηση, χρησιμοποιώντας έναν δικό του αλγόριθμο. Αξίζει να σημειωθεί ότι η LISP είναι η δεύτερη παλαιότερη γλώσσα προγραμματισμού, μετά τη FORTRAN, που χρησιμοποιείται ακόμα. Μετά τη δεκαετία του '70, οι περισσότερες γλώσσες συναρτησιακού προγραμματισμού είναι βασισμένες στον λάμδα λογισμό με τύπους (typed lambda calculus).

Ίσως ήταν, λοιπόν, αναμενόμενο ότι, σύντομα, από τη δεκαετία του '70, εμφανίστηκαν γλώσσες συναρτησιακού προγραμματισμού με αντικειμενοστραφή χαρακτηριστικά, όπως η Flavors και η Loops. Τη δεκαετία του '80, εμφανίστηκαν η γλώσσα Common Loops και το σύστημα CLOS.

- Έγιναν, όμως, προσπάθειες ένταξης αντικειμενοστραφών χαρακτηριστικών ακόμα και στον **λογικό προγραμματισμό**. Ο λογικός προγραμματισμός εμφανίστηκε στις αρχές της δεκαετίας του 1970 ως αποτέλεσμα μακροχρόνιας έρευνας πάνω στην αυτόματη απόδειξη θεωρημάτων, κυρίως, καθώς και των προσπαθειών πάνω στην επεξεργασία φυσικής γλώσσας και την τεχνητή νοημοσύνη, γενικότερα. Ο λογικός προγραμματισμός βασίζεται στην (κατηγορηματική) λογική πρώτης τάξης (first order predicate logic). Σημαντική για την άνθησή του ήταν η παρατήρηση που έκανε ο Robert Kowalski ότι κάποιοι τύποι της λογικής αυτής, οι *προτάσεις Horn* (Horn clauses), μπορούν να ερμηνευθούν και δηλωτικά (declaratively) και διαδικαστικά (procedurally) και ανέπτυξε γι' αυτές με την ομάδα του στο Εδιμβούργο μια συγκεκριμένη διαδικασία απόδειξης, την SLD-ανάλυση (SLD-resolution). Έτσι, μπορούμε να θεωρήσουμε ότι ένα σύνολο από τέτοιες προτάσεις αποτελεί γνώση οργανωμένη και δηλωτικά και διαδικαστικά. Οι προτάσεις Horn έχουν τη γενική μορφή ενός κανόνα (rule):

$$A \leftarrow B_1, B_2, \dots, B_n$$

Αυτό μπορεί να διαβαστεί δηλωτικά ως

$$\text{Εάν } B_1 \text{ και } B_2 \text{ και } \dots \text{ και } B_n \text{ τότε } A$$

Ταυτόχρονα, η ίδια πρόταση μπορεί να θεωρηθεί δήλωση μιας διαδικασίας όπου A είναι η επικεφαλίδα της (όνομα, παράμετροι κλπ.) και $B_1, B_2, \dots, B_n$ είναι το σώμα της. Οπότε η ίδια πρόταση μπορεί να διαβαστεί διαδικαστικά ως

$$\text{Για να επιλυθεί το } A \text{ αρκεί να επιλυθεί το } B_1 \text{ και το } B_2 \dots \text{ και το } B_n$$

Ή αλλιώς

$$\text{Η εκτέλεση του } A \text{ μπορεί να αναχθεί στην εκτέλεση του } B_1 \text{ και την εκτέλεση του } B_2 \dots \text{ και την εκτέλεση του } B_n$$

Ο λογικός προγραμματισμός, λοιπόν, μας επιτρέπει να διατυπώνουμε τις σχέσεις και τα χαρακτηριστικά των δεδομένων του προβλήματός μας, δηλαδή, να περιγράψουμε "τι" ισχύει στο πρόβλημά μας. Ταυτόχρονα, όμως, η περιγραφή αυτή είναι εκτελέσιμη. Έτσι, ικανοποιήθηκαν και οι δύο πλευρές μιας διαμάχης, από τη δεκαετία του '60, για το εάν η γνώση που εμπλέκεται στα θέματα τεχνητής νοημοσύνης παριστάνεται καλύτερα δηλωτικά ή διαδικαστικά.

Με βάση τα αποτελέσματα της έρευνας αυτής, στις αρχές του 1970, ο Alain Colmerauer και η ομάδα του στο Πανεπιστήμιο της Μασσαλίας, σε συνεργασία με τον Kowalski, ανέπτυξαν μια γλώσσα προγραμματισμού, στο πλαίσιο της προσπάθειάς τους για την ανάπτυξη συστήματος απόδειξης θεωρημάτων για την επεξεργασία φυσικής γλώσσας. Αυτή η γλώσσα ονομάστηκε *prolog* (programmation en logique) και ενέπλεκε τη διαδικαστική ερμηνεία της λογικής των

προτάσεων Horn που υπέδειξε ο Kowalski. Οι περισσότερες γλώσσες λογικού προγραμματισμού είναι διάλεκτοι της prolog. Τύποι στον λογικό προγραμματισμό δεν υπάρχουν, επομένως κάθε γλωσσική επέκταση που σχετίζεται με αυτούς δεν είναι αναμενόμενη. Όμως, όπως είπαμε και πριν, δεδομένης της χρησιμότητάς του στην αναπαράσταση γνώσης και της καταλληλότητας του αντικειμενοστραφούς μοντέλου για τον σκοπό αυτό, οι αντικειμενοστραφείς επεκτάσεις του μελετήθηκαν έντονα σε ερευνητικό επίπεδο. Επίσης, εντάχθηκαν σε υλοποιήσεις της prolog λόγω της γενικότερης χρησιμότητάς τους στον προγραμματισμό, δεδομένου ότι η prolog είναι πλήρης γλώσσα προγραμματισμού. Χαρακτηριστικά θα αναφέρουμε τις υλοποιήσεις Prolog++ (LPA), Visual Prolog (απόγονο της Turbo Prolog που είχε τύπους), την Logtalk καθώς και την Concurrent Prolog.

Μετά τη C++, τι; Για να ολοκληρώσουμε το ξετύλιγμα του κουβαριού του χρόνου, ας δούμε, σύντομα τι ακολούθησε μετά την εμφάνιση της C++. Δεν θα επεκταθούμε πολύ, μια που ένας σημαντικός σταθμός είναι η εμφάνιση της γλώσσας Java, στην οποία είναι αφιερωμένο ένα μέρος του βιβλίου και, επίσης, συνοπτικά θα παρουσιάσουμε τα χαρακτηριστικά κάποιων σύγχρονων γλωσσών σε ξεχωριστό κεφάλαιο.

Τι ακολούθησε μετά την εμφάνιση της C++, λοιπόν; Η μεγάλη επέκταση του αντικειμενοστραφούς προγραμματισμού ξεκίνησε τη δεκαετία του '90. Σε αυτό συνέβαλαν διάφοροι παράγοντες εκ των οποίων δύο, πολύ σημαντικοί, είναι οι εξής:

- Ο κόσμος γνώριζε ήδη πολλά χρόνια τον αντικειμενοστραφή προγραμματισμό και, λόγω C++, πάρα πολλοί προγραμματίζαν σε αντικειμενοστραφές περιβάλλον. Σε αυτό συνέβαλαν και οι ανάγκες των εφαρμογών, που πλέον είχαν επεκταθεί, οπότε ο διαδικαστικός προγραμματισμός από μόνος του δεν αρκούσε για να καλύψει τη συστηματική ανάπτυξή τους.
- Τη δεκαετία αυτή, εμφανίστηκε η γλώσσα Java που διευκόλυνε την ανάπτυξη κατανεμημένων εφαρμογών πάνω από το διαδίκτυο, ενόσω ταυτόχρονα επεκτείνετο η χρήση του διαδικτύου και λόγω της υπηρεσίας του Παγκόσμιου Ιστού. Επιπροσθέτως, η Java διευκόλυνε την ανάπτυξη εφαρμογών διεπαφών χρηστών και την επαναχρησιμοποίηση κώδικα και λόγω των βιβλιοθηκών που παρείχε.

Η συνέχεια είναι λίγο-πολύ γνωστή: κάθε νέα γλώσσα, σχεδόν, προσφέρει αντικειμενοστραφή χαρακτηριστικά, π.χ. C#, Scala. Αξιοσημείωτο, όμως, είναι ότι και οι περισσότερες από τις ήδη υπάρχουσες γλώσσες εντάσσουν αντικειμενοστραφή χαρακτηριστικά στις νέες εκδόσεις τους. Ενδεικτικά αναφέρουμε τη FORTRAN (έκδοση FORTRAN 2003) και τη COBOL (πρώτη έκδοση COBOL 2002, νεότερη Object Oriented COBOL-2014).

Και εδώ να κλείσουμε την ιστορική αναδρομή. Στο πλαίσιο της είδαμε ότι οι δεκαετίες του '60, του '70, του '80 και του '90 ήταν καθοριστικές για τις γλώσσες προγραμματισμού και τα είδη προγραμματισμού, γενικότερα, που χρησιμοποιούμε στις μέρες μας. Να πούμε ακόμα ότι, με την αλλαγή της χιλιετίας, μεγάλη άνθηση είχαν εργαλεία και γλώσσες κατάλληλα για την επεξεργασία της πληροφορίας του Παγκόσμιου Ιστού. Τι πρόκειται να ακολουθήσει από πλευράς γλωσσών; Το τι γλώσσες θα προκύψουν και τι χαρακτηριστικά θα έχουν θα το δείξει η έρευνα, αλλά κυρίως οι ανάγκες των εφαρμογών που οι γλώσσες θα έλθουν να καλύψουν. Το ποιες θα είναι αυτές οι ανάγκες θα εξαρτηθεί από τα χαρακτηριστικά των εφαρμογών αυτών. Το σημαντικό δε είναι ότι το είδος των εφαρμογών που δύνανται να αναπτυχθούν εξαρτάται άρρηκτα από τις δυνατότητες των υπολογιστικών συστημάτων και των υποδομών, γενικότερα.

1.6 Σύνοψη Κεφαλαίου

- Ο αντικειμενοστραφής προγραμματισμός είναι ένα είδος προγραμματισμού που επιτρέπει στον προγραμματιστή να οργανώνει τον κώδικά του σε κλάσεις που αντιστοιχούν σε οντότητες ή έννοιες της

εφαρμογής του.

- Με τη χρήση κλάσεων καταλήγουμε σε καλύτερα οργανωμένο μοντέλο κώδικα που οδηγεί σε ευκολότερη ανάπτυξη, σε καλύτερη συντήρηση και σε πρόληψη λαθών.
- Παραδείγματα εφαρμογών που επωφελούνται από το αντικειμενοστραφές μοντέλο είναι οι εφαρμογές γραφικών και οι εφαρμογές προσομοίωσης.
- Οι σύγχρονες εφαρμογές και λόγω μεγέθους, αλλά και λόγω προγραμματιστικής πολυπλοκότητας, επωφελούνται τα μέγιστα από τις ιδιότητες του αντικειμενοστραφούς προγραμματισμού.
- Στον αντικειμενοστραφή προγραμματισμό ορίζουμε κλάσεις των οποίων στη συνέχεια, συνήθως, θα κατασκευάσουμε *αντικείμενα*. Τις κλάσεις μπορούμε να τις συνδέσουμε μεταξύ τους με σχέση εξειδίκευσης, μέσω της οποίας επιτυγχάνουμε *κληρονομικότητα*.
- Με τη χρήση κληρονομικότητας οργανώνουμε τον κώδικά μας σε ιεραρχίες κλάσεων. Οι κλάσεις που βρίσκονται χαμηλότερα στην ιεραρχία (υποκλάσεις) κληρονομούν χαρακτηριστικά και λειτουργικότητα από εκείνες που βρίσκονται ψηλότερα στο ίδιο μονοπάτι της ιεραρχίας. Επιβάλλεται να καθορίσουν μόνο εκείνα τα στοιχεία που τις διαφοροποιούν από τις γενικότερές τους.
- Οι γλώσσες αντικειμενοστραφούς προγραμματισμού έχουν ισχυρό *μηχανισμό ελέγχου τύπων* και δυνατότητα υποστήριξης *εγκλωβισμού*. Το δεύτερο προφυλάσσει να παραβιαστεί η *αφαίρεση στα δεδομένα*, που επιτρέπει τον ορισμό νέων τύπων από τον προγραμματιστή.
- Οι κλάσεις μπορούν να λειτουργήσουν και ως εργαλείο για να πετύχουμε *τμηματοποίηση με απόκρυψη πληροφορίας* στον κώδικά μας.
- Οι κλάσεις δίνουν δυνατότητες *πολυμορφισμού*.
- Το αντικειμενοστραφές μοντέλο συναντάται σε διάφορες περιοχές των υπολογιστικών συστημάτων, όπως η μοντελοποίηση κατά την ανάπτυξη λογισμικού, η αναπαράσταση γνώσης στην τεχνητή νοημοσύνη και τα μοντέλα δεδομένων στις βάσεις δεδομένων.
- Ο αντικειμενοστραφής προγραμματισμός βασίστηκε σε έννοιες που εμφανίστηκαν στη δεκαετία του '60, επεκτάθηκαν στη δεκαετία του '70, άνθισαν στη δεκαετία του '80 και καθιερώθηκαν στη δεκαετία του '90.
- Νέες αντικειμενοστραφείς γλώσσες αναπτύχθηκαν μετά την εμφάνιση του αντικειμενοστραφούς προγραμματισμού και παλαιότερες γλώσσες επεκτάθηκαν με αντικειμενοστραφή χαρακτηριστικά. Η ανάπτυξη ή η επέκταση έγινε σε γλώσσες κάθε προγραμματιστικού είδους.

1.7 Ερωτήσεις

- 1.7.1 Αναφέρατε τα βασικά χαρακτηριστικά του διαδικαστικού προγραμματισμού, της αφαίρεσης στα δεδομένα και του αντικειμενοστραφούς προγραμματισμού.
- 1.7.2 Πού εστιάζει ο διαδικαστικός προγραμματισμός και πού ο αντικειμενοστραφής; Δώστε ένα παράδειγμα γλώσσας για τον καθένα. Δώστε, επιγραμματικά, ένα παράδειγμα προβλήματος που ενδείκνυται να υλοποιηθεί με μια γλώσσα διαδικαστικού προγραμματισμού και ένα που ενδείκνυται να υλοποιηθεί με μια γλώσσα αντικειμενοστραφούς προγραμματισμού.

- 1.7.3 Σχολιάστε επιγραμματικά τη δυνατότητα ορισμού συναρτήσεων που παρέχεται από μια γλώσσα διαδικαστικού προγραμματισμού κατ' αναλογία με τη δυνατότητα ορισμού κλάσεων του αντικειμενοστραφούς προγραμματισμού.
- 1.7.4 Αναφέρατε τρεις λέξεις ή εκφράσεις, διαφορετικές από “κλάση” και “αντικείμενο”, που συναντά κανείς στο πλαίσιο του αντικειμενοστραφούς προγραμματισμού και δώστε μια επιγραμματική περιγραφή του νοήματός τους.
- 1.7.5 Γράψτε μια παράγραφο περίπου 30 λέξεων που να περιέχει τις λέξεις “αντικείμενο”, “κλάση”, “κληρονομικότητα” και “αρχικοποίηση” στο νοηματικό πλαίσιο του αντικειμενοστραφούς προγραμματισμού.
- 1.7.6 Τι εννοούμε με τον όρο “αφαίρεση δεδομένων”; Ποιος μηχανισμός είναι απαραίτητος για την επίτευξή του;
- 1.7.7 Τι επιτυγχάνουμε με τον μηχανισμό του εγκλωβισμού στον αντικειμενοστραφή προγραμματισμό;
1. ταχύτητα στην εκτέλεση
 2. αφαίρεση δεδομένων
 3. πολυμορφισμό
 4. κληρονομικότητα
 5. ορισμό αντικειμένων
- 1.7.8 Πού είναι χρήσιμος ο μηχανισμός του εγκλωβισμού στον προγραμματισμό, γενικότερα;
1. διευκολύνει την ανάπτυξη εκτενούς κώδικα
 2. διευκολύνει την υποστήριξη κάθε γλώσσας προγραμματισμού
 3. διευκολύνει την υλοποίηση απλών αριθμητικών προβλημάτων
 4. απαιτείται από το λειτουργικό σύστημα
 5. τίποτα από τα προηγούμενα
- 1.7.9 Στις γλώσσες αντικειμενοστραφούς προγραμματισμού, κατά τον ορισμό μιας κλάσης χωρίζουμε το δημόσιο μέρος της από το ιδιωτικό. Ποιος μηχανισμός απαιτείται από τη γλώσσα προκειμένου να επιτύχουμε τη διάκριση αυτή;
1. ελέγχου τύπων
 2. υποστήριξης τύπων ορισμένων από τον χρήστη
 3. εγκλωβισμού
 4. εμβέλειας χωρίς απόκρυψη πληροφορίας
 5. υποστήριξης ορισμού αντικειμένων

1.8 Βιβλιογραφία

- [1] Bjarne Stroustrup. *The C++ Programming Language*. 2η έκδοση. Addison-Wesley Publishing Company, 1991.
- [2] Bertrand Meyer. *Object-oriented Software Construction*. Prentice-Hall, Inc., 1988.

- [3] L. Cardelli και P. Wegner. “On Understanding Types, Data Abstraction, and Polymorphism”. Στο: *ACM Computing Surveys* 17.4 (Δεκ. 1985).
- [4] Grady Booch. *Object-Oriented Design with Applications*. Benjamin/Cummings, 1991.
- [5] Object Management Group. <https://www.omg.org/>. Ανάκτηση: 2023-5-30.
- [6] Edsger W. Dijkstra. “Programming Considered as a Human Activity”. Στο: *Classics in Software Engineering*. Yourdon Press, 1979, σσ. 1–9.
- [7] Thomas R. Gruber. “A translation approach to portable ontology specifications”. Στο: *Knowledge Acquisition* 5.2 (Ιούν. 1993).
- [8] Fritz Lehmann. “Semantic networks”. Στο: *Computers & Mathematics with Applications* 23.2 (1992), σσ. 1–50.
- [9] Bjarne Stroustrup. “A History of C++: 1979–1991”. Στο: *The Second ACM SIGPLAN Conference on History of Programming Languages*. HOPL-II. Association for Computing Machinery, 1993, σσ. 271–297.
- [10] Bjarne Stroustrup. “Evolving a language in and for the Real World: C++ 1991-2006”. Στο: *Proceedings of the Third ACM SIGPLAN Conference on History of Programming Languages*. HOPL III. Association for Computing Machinery, 2007, σσ. 4–14–59.
- [11] C. A. R. Hoare. “Record Handling”. Στο: *ALGOL Bull.* 21 (Νοέ. 1965), σσ. 39–69.
- [12] Francis G. McCabe. *Logic and Objects*. Prentice Hall International (UK) Ltd., 1992.
- [13] D. A. Turner. “Some History of Functional Programming Languages”. Στο: *Trends in Functional Programming*. Springer Berlin Heidelberg, 2013, σσ. 1–20.
- [14] A. Colmerauer και P. Roussel. “The Birth of Prolog”. Στο: *History of Programming Languages—II*. Association for Computing Machinery, 1996, σσ. 331–367.
- [15] A. Colmerauer, H. Kanoui, R. Pasero και P. Roussel. *Un Système de Communication Homme-Machine en Français*. Report. Université d’ Aix-Marseille II, 1973.
- [16] Robert Kowalski. “Logic Programming in the 1970s”. Στο: *Logic Programming and Nonmonotonic Reasoning*. Springer Berlin Heidelberg, 2013, σσ. 11–22.
- [17] Jan Skansholm. *ADA from the Beginning*. International Computer Science Press, 1988.
- [18] Niklaus Wirth. *Programming in MODULA-2*. 4η έκδοση. Springer-Verlag, 1988.
- [19] L. Cardelli, J. Donahue, L. Glassman, M. Jordan, B. Kalsow και G. Nelson. “Modula-3 Language Definition”. Στο: *ACM SIGPLAN Notices* 27.8 (Αύγ. 1992).
- [20] C.A.R. Hoare. “Record Handling”. Στο: *Programming Languages*. Επιμέλεια υπό F. Genuys. Academic Press, 1968, σσ. 291–347.
- [21] K. Nygaard και O.-J. Dahl. “The Development of the SIMULA Languages”. Στο: *History of Programming Languages*. Association for Computing Machinery, 1978, σσ. 439–480.
- [22] F. Bancilhon, C. Delobel και P. Kanellakis. *Building an Object-Oriented Database System: The Story of O2*. Morgan Kaufmann Publishers, 1992.
- [23] Marvin Minsky. “A Framework for Representing Knowledge”. Στο: *The Psychology of Computer Vision*. Επιμέλεια υπό P. H. Winston. (δημοσίευση ομότιτλης τεχνικής αναφοράς του MIT, 1974). McGraw-Hill, 1975.
- [24] Protégé. <https://protege.stanford.edu/>. Ανάκτηση: 2023-5-30.
- [25] R. Brachman και J. Schmolze. “An Overview of the KL-ONE Knowledge Representation System”. Στο: *Cognitive Science* 9.2 (1985).

- [26] Markus Krötzsch, Frantisek Simancik και Ian Horrocks. *A Description Logic Primer*. 2013. arXiv: [1201.4089](https://arxiv.org/abs/1201.4089).
- [27] *Description Logics*. <http://dl.kr.org/>. Ανάκτηση: 2023-5-30.
- [28] *Web Ontology Language (OWL)*. <https://www.w3.org/OWL/>. Ανάκτηση: 2023-5-30.
- [29] Tim Berners-Lee. *Weaving the Web: The Original Design and Ultimate Destiny of the World Wide Web by Its Inventor*. HarperCollins, 1999.
- [30] N. Guarino, D. Oberle και S. Staab. “What Is an Ontology?” Στο: *Handbook on Ontologies*. Springer Berlin Heidelberg, 2009.
- [31] Dan W. Patterson. *Introduction to Artificial Intelligence and Expert Systems*. Prentice-Hall, Inc., 1990.
- [32] Jeffrey D. Ullman. *Principles of Database and Knowledge-Base Systems, Volume II*. Computer Science Press, 1989.
- [33] S. Ceri, G. Gottlob και L. Tanca. “What You Always Wanted to Know About Datalog (And Never Dared to Ask)”. Στο: *IEEE Transactions on Knowledge and Data Engineering* 1.1 (1989), σσ. 146–166.
- [34] Jeffrey D. Ullman και Jennifer Widom. *A First Course in Database Systems*. USA: Prentice-Hall, Inc., 1997.
- [35] Erich Gamma, Richard Helm, Ralph Johnson και John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional, 1994.
- [36] P.J. Plauger, Meng Lee, David Musser και Alexander A. Stepanov. *C++ Standard Template Library*. Pearson, 2000.
- [37] Ellis Horowitz. *Fundamentals of Programming Languages*. 2η έκδοση. Computer Science Press, 1984.
- [38] Manoochehr Azmoodeh. *Abstract Data Types and Algorithms*. MacMillan Education, 1988.
- [39] Rod Ellis. *Data Abstraction and Program Design*. Pitman Publishing, 1991.
- [40] Johannes J. Martin. *Data Types and Data Structures*. Prentice Hall, 1986.
- [41] Jacques Loeckx. “Algorithmic Specifications: A Constructive Specification Method for Abstract Data Types”. Στο: *ACM Transactions on Programming Languages and Systems* 9.4 (Οκτ. 1987), σσ. 646–685.
- [42] J. Goguen και J. Meseguer. *Unifying Functional, Object-Oriented and Relational Programming with Logical Semantics*. Report SRI-CSL-87-7. Menlo Park: SRI International, Ιούλ. 1987.
- [43] Jim Welsh και John Elder. *Introduction to Modula-2*. Prentice Hall, 1987.
- [44] L. Cardelli, J. Donahue, L. Glassman, M. Jordan, B. Kalsow και G. Nelson. *Modula-3 Report (revised)*. Report. Digital Equipment Corporation, Νοέ. 1989.
- [45] *Miranda System Manual*. 1987.
- [46] G. M. Birtwistle, O.-J. Dahl, B. Myhrhaug και K. Nygaard. *SIMULA: BEGIN*. Auerbach Publisher, Inc., 1973.
- [47] Terrence W. Pratt. *Programming Languages: Design and Implementation*. 2η έκδοση. Prentice-Hall, Inc., 1984.
- [48] Grady Booch, Robert Maksimchuk, Michael Engle, Bobbi Young, Jim Conallen και Kelli Houston. *Object-Oriented Analysis and Design with Applications*. 3η έκδοση. Addison-Wesley Professional, 2007.

ΣΤΟΙΧΕΙΟΠΟΙΗΣΗ ΚΑΙ ΑΦΑΙΡΕΣΗ ΣΤΑ ΔΕΔΟΜΕΝΑ

Σύνοψη

Στο πλαίσιο της μετάβασης από τον διαδικαστικό στον αντικειμενοστραφή προγραμματισμό μέσω λογικών βημάτων, στο κεφάλαιο αυτό, συζητιούνται η ανάγκη κατάτμησης κώδικα, ο προγραμματισμός με στοιχεία καθώς και η αφαίρεση στα δεδομένα εστιάζοντας και στην περίπτωση των Αφηρημένων Τύπων Δεδομένων. Προκειμένου να γίνει πιο καθαρή η ανάγκη ορισμού τύπων από τον προγραμματιστή (τύπων ορισμένων από τον χρήστη – user defined types) σε περιβάλλον που επιτρέπει απόκρυψη πληροφορίας, δίνεται ένα παράδειγμα σε υποθετική γλώσσα, ώστε να αναδειχθούν κοινά χαρακτηριστικά γλωσσών αντικειμενοστραφούς προγραμματισμού όσον αφορά αφαίρεση στα δεδομένα.

Προαπαιτούμενη Γνώση

Για την κατανόηση του κεφαλαίου, ο αναγνώστης απαιτείται να έχει στοιχειώδεις γνώσεις προγραμματισμού και πληροφορικής. Προτείνεται να έχει μελετήσει και το προηγούμενο κεφάλαιο.

2.1 Από τον Διαδικαστικό Προγραμματισμό στον Αντικειμενοστραφή: Ένα Πρώτο Λογικό Βήμα

Το αντικείμενο του βιβλίου είναι ο Αντικειμενοστραφής Προγραμματισμός που κτίζεται πάνω από τον διαδικαστικό προγραμματισμό. Καλό, λοιπόν, θα ήταν να κάνουμε μια διαδρομή ακολουθώντας λογικά βήματα που ξεκινούν από τον διαδικαστικό προγραμματισμό και καταλήγουν στον ΟΟΡ.

2.1.1 Στοιχειοποίηση

Το πρώτο είδος προγραμματισμού που αναπτύχθηκε ήταν ο διαδικαστικός προγραμματισμός και εξυπηρέτούσε την υλοποίηση της επίλυσης αριθμητικών προβλημάτων. Τέτοια προβλήματα συναντάμε σε διάφορες επιστημονικές περιοχές, π.χ. μετεωρολογία. Σε περιπτώσεις που απαιτείται η εκτέλεση πολλών ή/και πολύπλοκων πράξεων, ο άνθρωπος είναι επιρρεπής να κάνει λάθη. Χειρότερα ακόμα, το έργο που προκύπτει μπορεί

πρακτικά να μην μπορεί να διεκπεραιωθεί. Για τον λόγο αυτό, από την αρχαιότητα ήδη, είχε καταφύγει στην κατασκευή μηχανών για να διεκπεραιώνουν τέτοιους υπολογισμούς. Στα μισά του περασμένου αιώνα, όμως, άρχισε η άνθηση των υπολογιστών γενικής χρήσης με παράλληλη ανάπτυξη γλωσσών οι οποίες θα επέτρεπαν τον προγραμματισμό των διαδικασιών που έπρεπε να εκτελεστούν για να επιλυθούν τα προβλήματα. Οι γλωσσικές δομές που παρείχαν οι γλώσσες αυτές διευκόλυναν την υλοποίηση των κατάλληλων αλγορίθμων. Δηλαδή, επέτρεπαν στον προγραμματιστή να εκφράσει εύκολα και κομψά τα υπολογιστικά βήματα που οδηγούσαν στην επίλυση του προβλήματος. Τα δεδομένα ήταν κυρίως αριθμητικά.

Τι γίνεται, όμως, εάν θέλουμε να αναπτύξουμε τον κώδικά μας τμηματικά; Δηλαδή, να υλοποιήσουμε κομμάτια κώδικα ανεξάρτητα το ένα από το άλλο, είτε γιατί η ανάπτυξή τους γίνεται σε απομακρυσμένες χρονικά εποχές, είτε ταυτόχρονα από διαφορετικούς προγραμματιστές.

Ας πάρουμε το εξής παράδειγμα — θα χρησιμοποιήσουμε τη σύνταξη από μια υποθετική γλώσσα προγραμματισμού που μοιάζει με τη C. Έστω, λοιπόν, ότι για τις ανάγκες μιας εφαρμογής απαιτείται η υλοποίηση δύο συναρτήσεων, `void bla1()` και `void bla2()`. Έστω ότι οι υλοποιήσεις τους γίνονται ανεξάρτητα. Ας υποθέσουμε, όμως, ότι στον ορισμό της `void bla2()` καλείται η `void bla1()`. Δηλαδή, συνοπτικά, ας πούμε ότι η κατάσταση είναι όπως φαίνεται στο Σχήμα 2.1.

<pre>void bla1() { ... }</pre>	<pre>void bla2() { ... bla1(); ... }</pre>
------------------------------------	--

Σχήμα 2.1: Δύο συναρτήσεις

Κατά την υλοποίηση της καθεμίας από τις δύο παραπάνω συναρτήσεις, ας υποθέσουμε ότι χρειάζεται να υλοποιηθεί μια ακόμα συνάρτηση, διαφορετική σε κάθε περίπτωση, με τα ίδια ορίσματα και τύπο επιστροφής, όμως. Εν αγνοία της υλοποίησης της μίας από την άλλη, ας υποθέσουμε ότι και στις δύο περιπτώσεις και οι δύο συναρτήσεις ονομάζονται συμπτωματικά με το ίδιο όνομα, έστω `foo`. Χάρη απλότητας του παραδείγματός μας, ας υποθέσουμε ότι δεν δέχονται κανένα όρισμα και δεν επιστρέφουν κάτι. Άρα καταλήγουμε να έχουμε δύο υλοποιήσεις μιας συνάρτησης `void foo()`. Οπότε, συνοπτικά πάλι, η κατάσταση είναι όπως φαίνεται στο Σχήμα 2.2.

<pre>void bla1() { ... foo(); ... }</pre>	<pre>void bla2() { ... bla1(); ... foo(); ... }</pre>
<pre>void foo() { ... }</pre>	<pre>void foo() { ... }</pre>

Σχήμα 2.2: Σύγκρουση ονομάτων: δύο διαφορετικοί ορισμοί για τη συνάρτηση `void foo()`

Στη φάση παραγωγής του εκτελέσιμου προγράμματος της εφαρμογής υπάρχει ανάγκη να συνεννοθούν οι υλοποιήσεις και των δύο συναρτήσεων, `void bla1()` και `void bla2()`. Όμως, με βάση την παραπάνω εικόνα, υπάρχει πρόβλημα πολλαπλού ορισμού της συνάρτησης `void foo()`. Μια λύση θα ήταν, κάθε συνάρτηση με μη προσυμφωνημένο όνομα να παίρνει μια κατάληξη στο όνομά της που να την κάνει μοναδική.

Για παράδειγμα, τη χρονική στιγμή που γράφτηκε ή τα αρχικά του ονοματεπώνυμου του δημιουργού της. Δηλαδή, αντί για, `void foo()`, να τις ονοματίζαμε `void foo_Sep20()` και `void foo_Oct20()` ή `void foo_IK()` και `void foo_AK()`, αντίστοιχα. Το πρώτο ταιριάζει στην περίπτωση που η ανάπτυξη γίνεται από τον ίδιο προγραμματιστή αλλά σε διαφορετικές χρονικές στιγμές. Το δεύτερο ταιριάζει στην περίπτωση που η ανάπτυξη γίνεται από διαφορετικούς προγραμματιστές. Οπότε, για το πρώτο εναλλακτικό, η παραπάνω εικόνα μετασχηματίζεται όπως φαίνεται στο Σχήμα 2.3.

<pre>void bla1() { ... foo_Sep20(); ... }</pre> <pre>void foo_Sep20() { ... }</pre>	<pre>void bla2() { ... bla1(); ... foo_Oct20(); ... }</pre> <pre>void foo_Oct20() { ... }</pre>
---	---

Σχήμα 2.3: Οι συναρτήσεις `void foo()` μετονομάστηκαν με διαφορετικά ονόματα

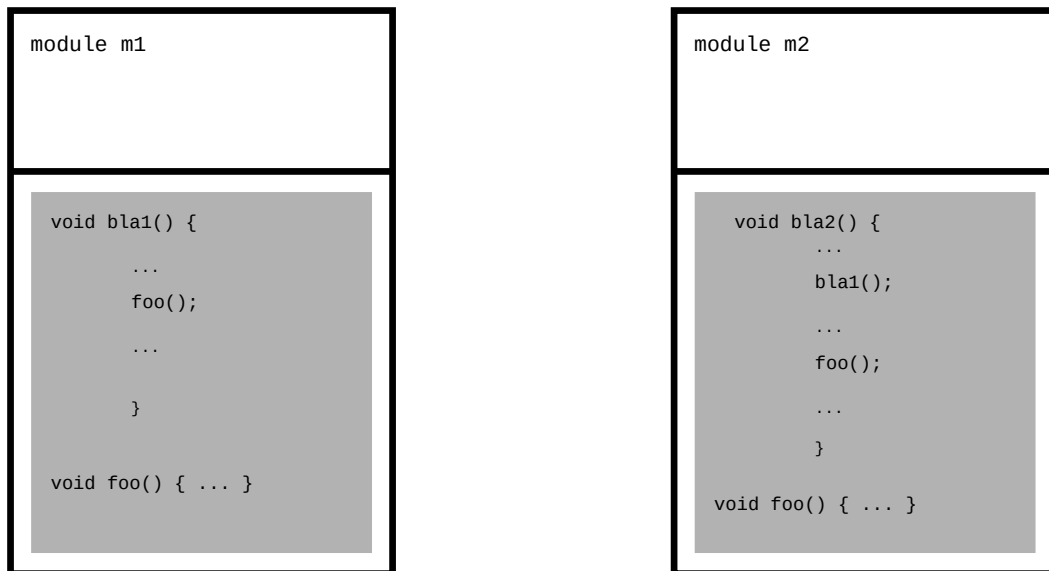
Έτσι, το πρόβλημα των πολλαπλών ορισμών παρακάμπτεται, αλλά καταλήγουμε να έχουμε άκομψα ονόματα συναρτήσεων. Μην ξεχνάμε, όμως, ότι κοιμιά ονόματα που αποδίδουν το νόημα μιας συνάρτησης ή μιας οντότητας του προγράμματός μας, γενικότερα, συμβάλλουν στο να είναι ο κώδικάς μας ευανάγνωστος, χαρακτηριστικό πολύ κρίσιμο κυρίως στη συντήρησή του, ιδιαίτερα αν ο κώδικας είναι εκτενής.

Οπότε ποια είναι η λύση; Η λύση προκύπτει από την επιλογή μιας γλώσσας προγραμματισμού η οποία να επιτρέπει “πακετάρισμα” κώδικα και “εγκλωβισμού” του σε “μαύρα κουτιά” ώστε να απαγορεύεται να χρησιμοποιηθούν ονόματα που ορίζονται/δηλώνονται μέσα στο μαύρο κουτί έξω από το “πακέτο” του. Τέτοια πακέτα συχνά ονομάζονται “στοιχεία” (*modules*). Συχνά δε, αντί για “πακετάρισμα” κώδικα, αναφερόμαστε σε “τμηματοποίηση” κώδικα, μια που με τον τρόπο αυτόν μας επιτρέπεται να αναπτύξουμε τον κώδικά μας τμηματικά.

Έστω, λοιπόν, ότι έχουμε μια γλώσσα προγραμματισμού η οποία μας επιτρέπει να δηλώσουμε ένα τέτοιο πακέτο με μαύρο κουτί χρησιμοποιώντας τη χρήση της λέξης-κλειδιού `module`, συνοδεύοντάς την και από ένα όνομα που αποτελεί το όνομα του στοιχείου. Το παράδειγμά μας, τότε, μπορεί να γραφτεί όπως φαίνεται στο Σχήμα 2.4.

Το πρόγραμμά μας πλέον, έστω P , αποτελείται από ένα σύνολο στοιχείων που περιλαμβάνει και αυτά τα δύο στοιχεία, $m1$ και $m2$. Δηλαδή, $P = \{ m1, m2, \dots \}$. Τα μαύρα κουτιά των στοιχείων θεωρούνται ανεξάρτητα κομμάτια κώδικα, οπότε μπορούμε να ορίσουμε όποια συνάρτηση θέλουμε στο καθένα από αυτά χωρίς να ανησυχούμε ότι θα υπάρξει σύγκρουση ονομάτων.

Πώς, όμως, θα χρησιμοποιηθούν οι συναρτήσεις `void bla1()` και `void bla2()` από τον υπόλοιπο κώδικα; Επίσης, είπαμε, ότι στο σώμα της `void bla2()` καλείται η `void bla1()`. Τώρα, οι συναρτήσεις αυτές είναι ορισμένες σε διαφορετικά μαύρα κουτιά. Πώς θα καταφέρουμε να καλέσουμε τη συνάρτηση `void bla1()` που είναι εγκλωβισμένη στο `module m1` από το σώμα της συνάρτησης `void bla2()` που ορίζεται στο `module m2`; Η απάντηση είναι ότι η γλώσσα, πέρα από “απόκρυψη πληροφορίας” που προσφέρει ο μηχανισμός του εγκλωβισμού, πρέπει να επιτρέπει και “ανταλλαγή πληροφορίας” μεταξύ των στοιχείων, οπότε αυτό είναι επιθυμητό. Η ανταλλαγή αυτή επιτυγχάνεται μέσω “δηλώσεων διεπαφής” (*interface declarations*) με τις οποίες ένα στοιχείο καθορίζει ποια ονόματα μπορούν να γίνουν ορατά και χρησιμοποιήσιμα έξω από το στοιχείο αυτό. Οι δηλώσεις αυτές λέγονται συχνά “δηλώσεις εξαγωγής” (*export declarations*). Για να ολοκληρωθεί η ανταλλαγή αυτή, ένα στοιχείο που επιθυμεί να χρησιμοποιήσει εξαγόμενα ονόματα από ένα άλλο πρέπει να το δηλώσει ρητά μέσω “δηλώσεων εισαγωγής” (*import declarations*) που να εκφράζουν την επιθυμία χρήσης ενός εισαγόμενου ονόματος από συγκεκριμένο στοιχείο (το οποίο εξάγει το όνομα αυτό).



Σχήμα 2.4: Ο κώδικάς μας μέσα σε στοιχεία

Οι δηλώσεις εξαγωγής και εισαγωγής αποτελούν τη διεπαφή ενός στοιχείου και είναι το “παράθυρό” του στον έξω κόσμο. Αν υποθέσουμε ότι οι δηλώσεις εξαγωγής έχουν τη γενική μορφή `export <Name>` και οι δηλώσεις εισαγωγής έχουν τη γενική μορφή `import <Name> from <ModuleName>`, τότε το παράδειγμά μας μπορεί να γραφτεί όπως φαίνεται στο Σχήμα 2.5.



Σχήμα 2.5: Τα στοιχεία με τις διεπαφές τους για την ανταλλαγή πληροφορίας

Στο σημείο αυτό να πούμε ότι τόσο το είδος των δηλώσεων διεπαφής, όσο και η σύνταξή τους είναι ενδεικτικά για να αναδειχθεί η λειτουργικότητα των διεπαφών. Θα μπορούσαν να υπάρχουν και άλλες μορφές δηλώσεων διεπαφής. Για παράδειγμα, “δηλώσεις καθολικότητας” (global declarations), π.χ. `global <Name>`, που να καθορίζουν ονόματα που είναι ορατά και χρησιμοποιήσιμα από όλα τα στοιχεία του κώδικα χωρίς να υπάρχουν αντίστοιχες δηλώσεις εισαγωγής.

Ένα άλλο σημείο που πρέπει να αναφερθεί είναι ότι ο εγκλωβισμός μπορεί να επιδρά σε κάθε όνομα που ορίζεται μέσα σε ένα στοιχείο και όχι μόνο σε ονόματα συναρτήσεων.

Οι τελευταίες αυτές παρατηρήσεις γίνονται τελείως επιγραμματικά και δεν θα αναπτυχθούν παραπάνω, μια που τότε θα ξεφύγουμε από την εστίαση που έχουμε στο βιβλίο αυτό.

Από πλευράς δομής, πολύ γενικά, μπορούμε να πούμε ότι ένα στοιχείο χαρακτηρίζεται από τα εξής:

- Το όνομά του.
- Τις δηλώσεις διεπαφής του.
- Κώδικα που ορίζει ονόματα για χρήση τοπική στο στοιχείο ή ευρύτερη από τον υπόλοιπο κώδικα. Η τελευταία γίνεται δυνατή μέσω κατάλληλων δηλώσεων διεπαφών.

Εδώ να σημειώσουμε ότι, σε κάποιες γλώσσες που δεν υποστηρίζουν τμηματοποίηση θα μπορούσαμε να προσομοιώσουμε την παραπάνω συμπεριφορά μέσω “εργαλείων” εκτός γλώσσας. Για παράδειγμα, στη γλώσσα C θα μπορούσαμε να χρησιμοποιήσουμε αρχεία ως μονάδες εγκλωβισμού, αρκεί οι συναρτήσεις που δεν θέλουμε να γίνουν ορατές και χρησιμοποιήσιμες από άλλα αρχεία να δηλωθούν ως `static` στο αρχείο που ορίζονται. Για να επιτύχουμε την ανταλλαγή πληροφορίας, αρκεί να γράψουμε τις επικεφαλίδες (`headers`) των συναρτήσεων που θέλουμε να διαθέσουμε στον “έξω κόσμο” σε αρχεία επικεφαλίδας (`header files`) τα οποία να “συμπεριλάβουμε” μέσω οδηγιών συμπερίληψης (`#include directives`) σε όποιο άλλο αρχείο θέλουμε να χρησιμοποιηθούν οι συναρτήσεις αυτές.

Κατά την ανάπτυξη μιας εφαρμογής, στη φάση του σχεδιασμού, πρέπει να καθοριστούν τα στοιχεία και οι διεπαφές τους. Δηλαδή, πρέπει να αποφασιστεί ποια είναι τα στοιχεία που χρειάζονται για το πρόγραμμα, ώστε να οριστούν τα δεδομένα και να υλοποιηθούν οι διαδικασίες στα κατάλληλα από αυτά. Όσον αφορά τον καθορισμό των στοιχείων, οι ειδικοί από τον χώρο της τεχνολογίας λογισμικού έχουν θέσει διάφορα κριτήρια για τον διαχωρισμό του κώδικα.

Όταν μια γλώσσα υποστηρίζει στοιχεία, μας δίνει τη δυνατότητα να οργανώσουμε τον κώδικά μας σε τμήματα τα οποία μπορούν να υλοποιηθούν ανεξάρτητα το ένα από το άλλο. Η μόνη απαίτηση είναι να υπάρξει αυστηρή τήρηση των προδιαγραφών των ονομάτων που εμφανίζονται στις δηλώσεις διεπαφής. Αυτό οδηγεί σε οργάνωση του κώδικα η οποία με τη λογική του “διαίρει και βασίλευε” επιτρέπει:

- Ανεξάρτητη ανάπτυξη των στοιχείων, που οδηγεί σε γρηγορότερη ανάπτυξη ή/και ασφαλέστερο κώδικα.
- Πιο ευανάγνωστο κώδικα.
- Ευκολότερη συντήρηση.

Κλείνοντας, να πούμε τα εξής. Οι σύγχρονες εφαρμογές είναι μεγάλης κλίμακας και εμπλέκουν μεγάλη ποικιλία δεδομένων και λειτουργιών. Κατά συνέπεια, όλες οι φάσεις της ανάπτυξης και συντήρησής τους είναι σημαντικά πολυπλοκότερες. Προκειμένου να παρακαμφθούν τα προβλήματα που εισάγει το μεγάλο μέγεθος των εφαρμογών, πρέπει να χρησιμοποιηθούν μηχανισμοί που να δίνουν τη δυνατότητα του “διαίρει και βασίλευε” κατά τον κύκλο ζωής των εφαρμογών. Με την υποστήριξη στοιχείων από μια γλώσσα προγραμματισμού, η δυνατότητα αυτή γίνεται εφικτή. Τα στοιχεία προσφέρουν έναν τρόπο σταδιακής κατασκευής ενός προγράμματος και, γενικότερα, δόμησής του. Πολλές γλώσσες προγραμματισμού, όπως η `Modula-2`, η `ADA`, η `EUCLID`, η `C++`, η `Java` κ.ά., παρέχουν γλωσσικές δομές για την υποστήριξη στοιχειοποίησης.

2.2 Αφαίρεση στα Δεδομένα

Μέχρι στιγμής, λοιπόν, έχουμε μιλήσει για γλώσσες προγραμματισμού που επιτρέπουν να “πακετάρουμε” διαδικασίες — κώδικα, γενικότερα — σε τμήματα, τα στοιχεία, εγκλωβίζοντας στο κάθε στοιχείο ότι δεν είναι απαραίτητο να χρησιμοποιηθεί έξω από αυτό. Ας προχωρήσουμε λίγο παραπέρα τώρα.

Αρχικά οι υπολογιστές χειρίζονταν αριθμητικά δεδομένα. Οι σύγχρονες εφαρμογές, όμως, χειρίζονται δεδομένα διαφόρων τύπων, περίπλοκων οντοτήτων και εννοιών και χαρακτηρίζονται ενδεχόμενα από μεγάλη πολυπλοκότητα. Κατά συνέπεια, έγινε επιτακτική η ανάγκη για μηχανισμούς κατασκευής σύνθετων αντικειμένων από απλούστερα. Αυτοί πρέπει να εξασφαλίζουν αφαίρεση ως προς τις ασήμαντες λεπτομέρειες των

αντικειμένων και δυνατότητα χειρισμού τους μέσω βασικών λειτουργιών που συνδέονται με αυτά. Άρα, πρακτικά, μιλάμε για “τύπους ορισμένους από τον χρήστη” που βασίζονται στην έννοια της “αφαίρεσης στα δεδομένα”.

Διάφορες γλώσσες δίνουν μια στοιχειώδη δυνατότητα ορισμού τύπων περιγράφοντας δομήσεις δεδομένων ώστε να μπορεί ο προγραμματιστής να ορίζει μεταβλητές που δομούνται με τον τρόπο που περιγράφεται στον ορισμό του τύπου. Η C, για παράδειγμα, όπως είπαμε προηγουμένως, το προσφέρει με τη χρήση της γλωσσικής δομής `struct`. Όμως, σε αυτές τις περιπτώσεις, δεν επιτυγχάνεται σύνδεση του τύπου με τη λειτουργικότητά του, δηλαδή τις λειτουργίες που τον συνοδεύουν, κάτι που είναι κρίσιμο για τον καθορισμό του. Για να γίνει η ανάγκη αυτή πιο καθαρή, πάρτε την περίπτωση των κλασικών δομών δεδομένων που μελετάμε στη βιβλιογραφία, για παράδειγμα τη δομή “στοίβα” (`stack`). Όταν αναφερόμαστε στη δομή της στοίβας, αμέσως έρχεται στη σκέψη μας και το σύνολο των λειτουργιών που αποτελούν τη συμπεριφορά της στοίβας. Αυτές είναι οι λειτουργίες `push`, `pop` και `top`. Πιο αναλυτικά, όταν μιλάμε για τη δομή “στοίβα”, στοιχείων τύπου `<type>`, αμέσως στο μυαλό μας έρχονται τα εξής.

- Αποτελείται από ένα σύνολο μεμονωμένων στοιχείων τύπου `<type>`.
- Μπορούμε να μεταβάλουμε τη στοίβα, προσθέτοντας (`push`) ένα στοιχείο τύπου `<type>` πάνω-πάνω, στην κορυφή της στοίβας.
- Μπορούμε να μεταβάλουμε τη στοίβα, αφαιρώντας (`pop`) το στοιχείο που βρίσκεται στην κορυφή της στοίβας.
- Μπορούμε να παρατηρήσουμε (`(get_)top`) το στοιχείο που βρίσκεται στην κορυφή της στοίβας.

Μπορούμε να προσθέσουμε στη λειτουργικότητα της δομής της στοίβας ότι αρχικά μια στοίβα είναι κενή — δεν περιέχει κανένα στοιχείο —, καθώς και τη δυνατότητα ελέγχου αν η στοίβα είναι κενή ή όχι. Πολύ συχνά, αναφερόμαστε στη δομή της στοίβας ως μια LIFO (Last In First Out) γραμμική δομή στοιχείων τύπου `<type>`. Εν ολίγοις, η έννοια της “στοίβας” συνδέεται άρρηκτα με τις ενέργειες που μπορούμε να κάνουμε σε αυτή και τις ιδιότητές τους. Βλέπουμε, λοιπόν, ότι είναι σημαντικό να μπορούμε να συνδέσουμε κατά τον ορισμό του έναν τύπο με τις λειτουργίες του.

2.2.1 Από τους Τύπους στα Αφηρημένα Αντικείμενα

Στο σημείο αυτό, συνεχίζοντας τη συζήτηση σχετικά με τους τύπους δεδομένων, θα γίνει μια αναφορά στους Αφηρημένους Τύπους Δεδομένων (ΑΤΔ), μια που πιστεύουμε ότι θα βοηθήσει και στην καλύτερη κατανόηση των εννοιών του ΟΟΡ. Συζητώντας γενικά περί τύπων δεδομένων και, στη συνέχεια, κάνοντας τη μετάβαση στους ΑΤΔ, γίνεται καθαρότερη η έννοια της αφαίρεσης στα δεδομένα. Επίσης, δίνεται η ευκαιρία για παραδείγματα τυπικών προδιαγραφών λογισμικού (formal specifications), θέμα πολύ ενδιαφέρον, που καλό είναι ο αναγνώστης να το έχει υπόψη του.

Σύμφωνα με το [1], οι τύποι (types) προκύπτουν από την ανάγκη της κατηγοριοποίησης των διαφόρων αντικειμένων ανάλογα με τη χρήση και τη συμπεριφορά τους. Σύνολα αντικειμένων που έχουν ομοιόμορφη συμπεριφορά αποκτούν ονόματα και αποτελούν τους τύπους. Για παράδειγμα, όλοι οι ακέραιοι παρουσιάζουν την ίδια συμπεριφορά αφού συνοδεύονται από το ίδιο σύνολο επιτρεπτών λειτουργιών. Κατά το [2], μια λειτουργία είναι βασικά μια συνάρτηση η οποία όταν εφαρμόζεται σε αντικείμενα ενός τύπου επιστρέφει ένα αντικείμενο κάποιου συγκεκριμένου τύπου, π.χ. ο αριθμητικός τελεστής της πρόσθεσης ακεραίων τιμών, “+”, είναι μια συνάρτηση $+ : integer \times integer \rightarrow integer$.

Τόσο στα μαθηματικά, όσο και στον προγραμματισμό, οι τύποι βάζουν περιορισμούς οι οποίοι βοηθούν στην εξασφάλιση της ορθότητας. Εκδοχές μαθηματικών θεωριών με τύπους, π.χ. η θεωρία συνόλων με τύπους, όπως και γλώσσες προγραμματισμού με τύπους (typed languages), επιβάλλουν περιορισμούς στην αλληλεπίδραση μεταξύ των αντικειμένων. Με αυτόν τον τρόπο, οι τύποι προφυλάσσουν τα αντικείμενα, τα σύνολα στην περίπτωση της θεωρίας συνόλων, από λανθασμένες αλληλεπιδράσεις με άλλα αντικείμενα. Με

λίγα λόγια, πάλι σύμφωνα με το [1], “ένας τύπος προσφέρει ένα κάλυμμα προφύλαξης (για τα αντικείμενα) που κρύβει την αναπαράσταση που βρίσκεται από κάτω και περιορίζει τον τρόπο με τον οποίο τα αντικείμενα αλληλεπιδρούν με άλλα αντικείμενα”.

Μπορούμε να διακρίνουμε δύο είδη γλωσσών προγραμματισμού με τύπους. Υπάρχουν γλώσσες στις οποίες ο τύπος κάθε μεταβλητής ή έκφρασης καθορίζεται στο στάδιο της μετάφρασης. Ακόμα και στην περίπτωση που δεν δίνεται ρητά αρκετή πληροφορία για τον τύπο μιας έκφρασης, αυτός είναι δυνατόν να προσδιοριστεί από ένα σύστημα συμπερασμού τύπων (type inference system). Στη δεύτερη περίπτωση, δεν είναι δυνατόν οι τύποι όλων των εκφράσεων να καθοριστούν στο στάδιο της μετάφρασης, οπότε απαιτείται ένας έλεγχος τύπων (type checking) των εκφράσεων κατά το στάδιο της εκτέλεσης. Και από τις δύο περιπτώσεις γίνεται εμφανές ότι προκειμένου να υποστηριχθούν οι τύποι σε μια γλώσσα προγραμματισμού, χρειάζονται επιπλέον μηχανισμοί με σημαντική πολυπλοκότητα.

Πέρα από τους απλούς τύπους, πολλές γλώσσες προγραμματισμού προσφέρουν περαιτέρω δυνατότητες αφαίρεσης δεδομένων, ομαδοποιώντας αντικείμενα, προκειμένου να απαρτίσουν έναν νέο σύνθετο τύπο. Η εσωτερική υλοποίηση των σύνθετων αυτών τύπων είναι κρυμμένη και η χρήση τους γίνεται μέσω επιτρεπτών λειτουργιών πάνω στις μεταβλητές των τύπων αυτών. Παράδειγμα τέτοιου σύνθετου τύπου είναι ο πίνακας (array). Στον πίνακα προσφέρεται η λειτουργία της επιλογής ενός στοιχείου, συνήθως, μέσω του συμβολισμού “[]”.

Ιδανικά, επιθυμούμε μια γλώσσα προγραμματισμού να προσφέρει κατάλληλες γλωσσικές δομές που να επιτρέπουν στον προγραμματιστή να ορίζει νέους σύνθετους τύπους. Δηλαδή, να ανεβαίνουμε σε ακόμα υψηλότερα επίπεδα αφαίρεσης και να φτιάχνουμε ακόμα πιο πολύπλοκα αντικείμενα τα οποία να χρησιμοποιούμε μέσω προκαθορισμένων λειτουργιών και για τα οποία να αγνοούμε, ή ακόμα, να μην μπορούμε να προσπελάσουμε τις λεπτομέρειες αναπαράστασης και υλοποίησης. Σ’ αυτήν την κατηγορία σύνθετων τύπων ανήκουν και διάφορες μαθηματικές οντότητες, όπως οι γράφοι (graphs), τα δένδρα (trees), η στοίβα (stack), ο σωρός (heap) κλπ. Σε κάθε τέτοια αφαίρεση, η συμπεριφορά ορίζεται μέσω των λειτουργιών που, ιδανικά, μπορούν να καθοριστούν και να επαληθευτούν τυπικά. Αυτά τα σύνθετα αφηρημένα αντικείμενα αποτελούν τους *Αφηρημένους Τύπους Δεδομένων*. Ήδη από το παρελθόν, γλώσσες προγραμματισμού όπως η Miranda επέτρεπαν ορισμούς ΑΤΔ. Ένας κλασικός τρόπος να οριστεί η δομή της “στοίβας” ως ΑΤΔ στη γλώσσα Miranda είναι:

```
abstype stack *
  with empty::stack *
    push::*->stack *->stack *
    isempty::stack *->bool
    top::stack *->*
    pop::stack *->stack *

    stack * == [*]
    empty = []
    push a x = a:x
    isempty x = x=[]
    top (a:x) = a
    pop (a:x) = x
```

Γενικότερα, η υποστήριξη από μια γλώσσα προγραμματισμού της δυνατότητας της αφαίρεσης στα δεδομένα — της οποίας η υλοποίηση ΑΤΔ αποτελεί ένα παράδειγμα — αναφέρεται στη δυνατότητα που μας παρέχει η γλώσσα να ορίζουμε νέους τύπους δεδομένων, χρησιμοποιώντας ήδη υπάρχοντες και για τον κάθε νέο τύπο να προσδιορίσουμε ένα σύνολο λειτουργιών που εφαρμόζονται στα στοιχεία του και είναι άρρηκτα συνδεδεμένες με τον εν λόγω τύπο. Η συσχέτιση των λειτουργιών αυτών με τον τύπο, ώστε ο χρήστης να μην μπορεί να χρησιμοποιήσει τα στοιχεία του τύπου, παρά μόνο μέσω των λειτουργιών του, επιτυγχάνεται μέσω ενός μηχανισμού εγκλωβισμού.

Για να μην χάσει ο αναγνώστης την εστίασή του, υπενθυμίζουμε τον στόχο μας. Σε μια γλώσσα ΟΟΡ, πρέ-

πει να δίνεται η δυνατότητα στον προγραμματιστή να ορίζει δικούς του τύπους που να προσομοιώνουν τις οντότητες της εφαρμογής που καλείται να αναπτύξει, ενσωματώνοντας τη συμπεριφορά τους κατά τον ορισμό. Οι ΑΤΔ αποτελούν μια κατηγορία παραδειγμάτων με την απαίτηση αυτή. Ας συνεχίσουμε, λοιπόν, για λίγο ακόμα, τη συζήτησή μας πάνω στους ΑΤΔ.

2.2.2 Αλγεβρικές Προδιαγραφές των ΑΤΔ

Ο τυπικός ορισμός των ΑΤΔ είναι κρίσιμος για διάφορους λόγους. Αφενός μεν, η τυπική περιγραφή τους χρειάζεται για να γίνει κατανοητή η συμπεριφορά τους και, κατά συνέπεια, η σωστή χρήση τους. Αφετέρου δε, η φύση των μαθηματικών ΑΤΔ συνδέεται άμεσα με την έννοια της τυπικής θεμελίωσης και ταυτόχρονα τη διευκολύνει. Για να γίνει πλήρης καθορισμός της συμπεριφοράς τους, δηλαδή των λειτουργιών τους, πρέπει να αποδοθεί τυπικά και η σημασιολογία τους. Αυτή η σημασιολογική πληροφορία συμπληρώνει την τυπική προδιαγραφή ενός τύπου δεδομένων που ενδέχεται να δίνεται ως *αλγεβρική προδιαγραφή* (algebraic specification) [3].

Το πρώτο μέρος μιας αλγεβρικής προδιαγραφής ενός ΑΤΔ αποτελείται από τη διεπαφή του τύπου αυτού. Σ' αυτή παρουσιάζονται οι λειτουργίες του ΑΤΔ με τη μορφή μαθηματικών συναρτήσεων. Δίνεται δηλαδή, το όνομά τους, το πεδίο ορισμού τους και το πεδίο τιμών τους. Τα σύνολα που εμπλέκονται στα πεδία ορισμού και τιμών εκφράζονται μέσω συμβολικών ονομάτων τα οποία καλούνται *είδη* (sorts) και αντιστοιχούν στους τύπους στην περίπτωση της υλοποίησης μέσω μιας γλώσσας προγραμματισμού. Συνήθως τα είδη που χρησιμοποιούνται στην αλγεβρική προδιαγραφή ενός ΑΤΔ απαριθμούνται στην αρχή της προδιαγραφής. Τα είδη που εμπλέκονται στον ορισμό ενός ΑΤΔ, καθώς και οι λειτουργίες του αποτελούν την *υπογραφή* (signature) του ΑΤΔ. Ας θεωρήσουμε τον ΑΤΔ "στοίβα". Ένα παράδειγμα υπογραφής γι' αυτόν τον ΑΤΔ είναι το εξής:

SORTS	<i>element stack bool</i>
OPERATIONS	
<i>initialize</i>	: $\rightarrow$ <i>stack</i>
<i>push</i>	: <i>stack element</i> $\rightarrow$ <i>stack</i>
<i>pop</i>	: <i>stack</i> $\rightarrow$ <i>stack</i>
<i>top</i>	: <i>stack</i> $\rightarrow$ <i>element</i>
<i>is_empty</i>	: <i>stack</i> $\rightarrow$ <i>bool</i>

Η υπογραφή έχει καθαρά συντακτικό χαρακτήρα και δεν δίνει κάποια σημασιολογική πληροφορία για τον ΑΤΔ. Στην ίδια υπογραφή μπορούν να αποδοθούν πάνω από μία ερμηνείες.

Ακολουθεί το μέρος της προδιαγραφής που αφορά τη σημασιολογία. Προκειμένου να εκφραστεί η σημασιολογία ενός ΑΤΔ, χρησιμοποιούμε εξισώσεις και σύνθεση συναρτήσεων. Μέσω αυτών κατασκευάζονται τα *αξιώματα* (axioms) που διέπουν τις λειτουργίες του ΑΤΔ και αντιστοιχούν στους περιορισμούς που εφαρμόζονται σ' αυτές. Ένα σοβαρό θέμα που αναφέρεται στην κατασκευή αλγεβρικών προδιαγραφών είναι οι *περιπτώσεις λαθών* (error cases). Για παράδειγμα, τι γίνεται αν πάμε να αφαιρέσουμε ένα στοιχείο από μια άδεια στοίβα; Προκειμένου να αντιμετωπιστούν αυτές οι περιπτώσεις, έχουν ακολουθηθεί διάφορες προσεγγίσεις. Η μια από αυτές θεωρεί μια επαύξηση των πεδίων τιμών των συναρτήσεων που αντιστοιχούν σε λειτουργίες με περιπτώσεις λαθών με μια τιμή λάθους, *error*. Αυτό, όμως, οδηγεί σε μια ομοιόμορφη αντιμετώπιση των περιπτώσεων λαθών και των αποδεκτών περιπτώσεων που δεν αποτρέπει να γίνονται κανονικά λειτουργίες, ενόσω έχει προκύψει μια τιμή λάθους. Μια πιο ασφαλής αντιμετώπιση είναι εκείνη που θεωρεί ότι οι λειτουργίες εκφράζονται μέσω μερικών συναρτήσεων. Στην περίπτωση αυτή, οι συναρτήσεις ορίζονται πάνω σ' ένα υποσύνολο του πεδίου ορισμού τους. Για τη στοίβα, η λειτουργία *pop* ορίζεται μόνο για το υποσύνολο των μη κενών στοιβών. Μια μορφή αυτής της προσέγγισης είναι η έκφραση των μερικών συναρτήσεων μέσω *υποείδων* (subsorts). Τα υποείδη είναι συμβολικά ονόματα υποσυνόλων. Για παράδειγμα, το υποείδος *ne_stack* μπορεί να συμβολίσει το υποσύνολο των στοιβών που είναι μη κενές. Τα υποείδη δηλώνονται ως προς το γε-

νικότερο είδος κατά την προδιαγραφή του ΑΤΔ. Μια πλήρης αλγεβρική προδιαγραφή μιας στοίβας, λαμβάνοντας υπόψη τα υποείδη, όπως αυτή δίνεται στο [3], είναι η παρακάτω:

```

STACK
SORTS      element stack bool
SUBSORTS
           ne_stack < stack
OPERATIONS
  initialize : → stack
  push      : stack element → ne_stack
  pop       : ne_stack → stack
  top       : ne_stack → element
  is_empty  : stack → bool
VARIABLES
           s : stack e : element
EQUATIONS
  top(push(s,e)) = e
  pop(push(s,e)) = s
  is_empty(initialize) = true
  is_empty(push(s,e)) = false
    
```

Μια ακολουθία λειτουργιών ενός ΑΤΔ εκφράζεται ως σύνθεση συναρτήσεων. Προκειμένου να υπολογιστεί το αποτέλεσμα της, γίνεται επαναγραφή όρων (term rewriting), θεωρώντας τα αξιώματα ως κανόνες επαναγραφής.

Η αλγεβρική προδιαγραφή ενός ΑΤΔ προσφέρει έναν σαφή τρόπο για τον καθορισμό του τύπου, χωρίς να εμπλέκει λεπτομέρειες και αποφάσεις συγκεκριμένης υλοποίησης. Το πώς όμως θα γίνει κατόπιν μια υλοποίηση που πραγματικά να ικανοποιεί την αλγεβρική προδιαγραφή είναι ένα σοβαρό θέμα. Σε περίπτωση που θέλουμε να αποδείξουμε τυπικά την ορθότητα της υλοποίησης, το έργο είναι πολύ δύσκολο.

Λαμβάνοντας υπόψη τα παραπάνω, κατά το [4], ένας ΑΤΔ είναι ένα σύστημα που αποτελείται από τα εξής μέρη

1. Κάποια σύνολα αντικειμένων.
2. Ένα σύνολο από συντακτικές περιγραφές (πρωτογενών) συναρτήσεων/λειτουργιών.
3. Μια σημασιολογική περιγραφή. Δηλαδή, ένα επαρκώς πλήρες σύνολο εκφράσεων συσχέτισης που καθορίζουν με ποιον τρόπο οι συναρτήσεις/λειτουργίες αλληλεπιδρούν μεταξύ τους. Αυτές αποτελούν τα (αλγεβρικά) αξιώματα του ΑΤΔ.

Ένας άλλος τρόπος έκφρασης των προδιαγραφών των ΑΤΔ δίνεται στο [5] μέσω αλγοριθμικών προδιαγραφών.

2.2.3 Τιμές και Στιγμιότυπα

Ένας ΑΤΔ μπορεί να θεωρηθεί είτε ως ένας χώρος τιμών συνοδευόμενος από ένα σύνολο λειτουργιών που εφαρμόζονται στα στοιχεία αυτού του χώρου είτε ως ένα σύνολο αντικειμένων συνοδευόμενο από ένα σύνολο λειτουργιών που δημιουργούν, παρατηρούν και μεταβάλλουν τα αντικείμενα αυτά. Στη δεύτερη περίπτωση,

μιλάμε για *στιγμιότυπα* ενός ΑΤΔ, τα οποία κάθε χρονική στιγμή έχουν κάποια τιμή, η οποία μπορεί να παρατηρηθεί ή να αλλαχθεί από τις λειτουργίες του τύπου. Κατ' αυτόν τον τρόπο, εισάγεται η έννοια της *κατάστασης*, η οποία παριστάνει την τιμή την οποία έχει ένα στιγμιότυπο μια συγκεκριμένη χρονική στιγμή. Στην περίπτωση της υποστήριξης στιγμιοτύπων, τα στοιχεία των ΑΤΔ αναφέρονται μάλλον σε θέσεις μνήμης, παρά στις τιμές που βρίσκονται μέσα σ' αυτές. Κατά συνέπεια, ακόμα και στην αλγεβρική προδιαγραφή του τύπου, λαμβάνεται υπόψη τόσο ο χώρος φύλαξης (*store*), όσο και η τιμή που βρίσκεται σ' αυτόν. Μια πρόταση για έναν τέτοιο τρόπο προδιαγραφής βρίσκεται στο [4]. Η εισαγωγή της έννοιας του χώρου φύλαξης μέσα στις προδιαγραφές τις κάνει αρκετά πιο πολύπλοκες στη σύνθεση, καθώς και στον συμπερασμό πάνω σ' αυτές.

Υπάρχουν, λοιπόν, δύο προσεγγίσεις σχετικά με τη φύση των στοιχείων ενός ΑΤΔ. Μια στατική, που θεωρεί τα στοιχεία του τύπου ως τιμές, και μια δυναμική που τα θεωρεί ως αντικείμενα με τιμές που μπορούν να αλλαχθούν. Το ποια από τις δύο θα ακολουθηθεί εξαρτάται από διάφορους παράγοντες. Για παράδειγμα, όταν ακολουθούμε την προσέγγιση των στιγμιοτύπων, η αλγεβρική προδιαγραφή ενός ΑΤΔ είναι πολύπλοκη. Αυτό αναφέρεται και στη σύνθεση των αξιωμάτων της και στην εξασφάλιση της ορθότητας για μια υλοποίησή της. Από την άλλη μεριά, αυτή η προσέγγιση είναι πιο αποδοτική από πλευράς υλοποίησης ως προς τον χώρο που καταλαμβάνουν τα δεδομένα. Στην περίπτωση των στατικών τιμών απαιτούνται διαρκώς νέοι χώροι.

Όμως, υπάρχουν περιπτώσεις που η προσέγγιση υπαγορεύεται από τη φύση ενός συγκεκριμένου ΑΤΔ. Υπάρχουν δηλαδή κάποιοι ΑΤΔ των οποίων τα στοιχεία είναι από τη φύση τους δυναμικά και υπάρχουν και άλλοι των οποίων τα στοιχεία είναι από τη φύση τους στατικά. Στατικοί ΑΤΔ μπορούν να θεωρηθούν οι μαθηματικές οντότητες. Ένας τέτοιος ΑΤΔ είναι ο ΑΤΔ “(μαθηματικό) σύνολο”, *set*, με τις συνήθεις λειτουργίες επί των συνόλων. Άλλα παραδείγματα ΑΤΔ με στατική φύση στιγμιοτύπων μπορούμε να πάρουμε από τα μεγέθη της φυσικής, π.χ. ο ΑΤΔ “ταχύτητα”, *velocity*, με μέτρο και φορά.

Από την άλλη μεριά, σε ΑΤΔ όπως η στοίβα, ταιριάζει καλύτερα η προσέγγιση που υποστηρίζει στιγμιότυπα, μια που τα στοιχεία του τύπου αυτού έχουν δυναμική φύση. Προσθέτοντας ένα στοιχείο σε μια στοίβα μεταβάλλεται αυτή καθεαυτή η στοίβα και δεν είναι απόλυτα φυσικό να προκύπτει μια άλλη στοίβα. Άλλο παράδειγμα ΑΤΔ με δυναμική φύση στοιχείων είναι ο ΑΤΔ “αυτοκίνητο”, *car*, με ταχύτητα, μάζα και επιτάχυνση.

SORTS

car velocity number

OPERATIONS

speed : *car* $\rightarrow$ *velocity*

mass : *car* $\rightarrow$ *number*

speedup : *car number* $\rightarrow$ *car*

Θα μπορούσαμε να πούμε ότι η ταχύτητα και η μάζα είναι παρατηρήσιμες ιδιότητες του “αυτοκινήτου”. Υπενθυμίζουμε ότι οι παρατηρήσιμες ιδιότητες αναφέρονται και ως *χαρακτηριστικά* του ΑΤΔ. Αυτές καθορίζουν την κατάσταση των στιγμιοτύπων του.

2.3 Τύποι Ορισμένοι από τον Χρήστη

Στη συνέχεια, θα παρουσιάσουμε ένα παράδειγμα ορισμού νέου τύπου για της ανάγκες μιας εφαρμογής, χρησιμοποιώντας ένα υποθετικό προγραμματιστικό περιβάλλον. Θα αναπτύξουμε ένα απλοϊκό σενάριο το οποίο, όμως, περιέχει πολλές λεπτομέρειες που αναδεικνύουν τα προτερήματα της αφαίρεσης στα δεδομένα και της απόκρυψης πληροφορίας. Δηλαδή, θα δούμε κάποια από αυτά που μπορούμε να επιτύχουμε σε ένα περιβάλλον ανάπτυξης εφαρμογών που επιτρέπει τα παραπάνω, καθώς και τον τρόπο με τον οποίο επιτυγχάνονται.

Θα ξεκινήσουμε από τη συζήτηση για την ανάγκη ορισμού του νέου τύπου. Θα συνεχίσουμε με τον καθορισμό των προδιαγραφών του, ώστε να υπάρχει συμφωνία μεταξύ του ορισμού και της χρήσης του. Όσον αφορά αυτές, θα μείνουμε στο συντακτικό επίπεδο και θα θεωρήσουμε ότι το σημασιολογικό έχει διατυπωθεί σε φυ-

σική γλώσσα, μια που ο νέος τύπος δεν έχει περίπλοκη συμπεριφορά. Θα προχωρήσουμε με την υλοποίηση των προδιαγραφών και τη χρήση του τύπου. Η υλοποίηση θα γίνει χρησιμοποιώντας μια υποθετική γλώσσα. Στη συνέχεια, θα κάνουμε μια συζήτηση για τα ενδεχόμενα σενάρια της χρήσης του τύπου στην εφαρμογή. Στα πλαίσια αυτά, θα αλλάξουμε την υλοποίηση, προκειμένου να αναδειχθεί η ανεξαρτησία της από τη χρήση του τύπου στην υπόλοιπη εφαρμογή.

Ας υποθέσουμε, λοιπόν, ότι έχουμε να αναπτύξουμε μια εφαρμογή που αφορά χρονοπρογραμματισμό προσωπικού για τη στελέχωση αεροπορικών πτήσεων. Κατ' αρχήν, ας δούμε τι χαρακτηρίζει μια πτήση στην εφαρμογή αυτή. Ας υποθέσουμε ότι για μια αεροπορική πτήση, μας ενδιαφέρει, ο αριθμός της, η πόλη αφετηρίας της, η πόλη προορισμού (άφιξης) της, η ώρα αναχώρησής της αλλά και η διάρκειά της, μια που θα πρέπει να ληφθεί υπόψη στον χρονοπρογραμματισμό του μέλους πληρώματος που θα συμμετάσχει σε αυτήν.

Δηλαδή, μια αεροπορική πτήση εννοιολογικά (conceptually) είναι μια δόμηση δεδομένων που αποτελείται από τα πεδία όπως φαίνονται στο παρακάτω:

Αεροπορική Πτήση

Αριθμός Πτήσης	ακέραιος
Πόλη Αφετηρίας	συμβολοσειρά
Πόλη Άφιξης	συμβολοσειρά
Ώρα Αναχώρησης	Χρόνος (?)
Διάρκεια	Χρόνος (?)

Μια αεροπορική πτήση, λοιπόν, απαρτίζεται από τον αριθμό πτήσης της, που για ευκολία ας θεωρήσουμε ότι είναι ένας ακέραιος, την πόλη αφετηρίας και την πόλη άφιξης της, που δεν είναι τίποτα άλλο από απλές συμβολοσειρές, αλλά και την ώρα αναχώρησης και τη διάρκειά της που είναι “Χρόνοι”. Ας υποθέσουμε ότι αντιλαμβανόμαστε τον “Χρόνο” ως εξής: π.χ. διάρκεια “5 ώρες και 30 λεπτά”, αναχώρηση στις “9 η ώρα και 15”. Σε αυτήν την περίπτωση μιλάμε για έναν συμμιγή αριθμό, Χ ώρες και Υ λεπτά. Οπότε, τι τύπου είναι ο “Χρόνος”; Δεν έχουμε τύπο για να αναπαραστήσουμε την έννοια του συμμιγούς, άρα πρέπει να ορίσουμε κάποιον νέο τύπο.

Με μια ματιά, βλέπουμε ότι στην εφαρμογή μας υπάρχει η ανάγκη ορισμού νέων τύπων, είτε με τη μορφή δόμησης δεδομένων, όπως στην περίπτωση των αεροπορικών πτήσεων, είτε με τη μορφή τύπων που συνοδεύονται από λειτουργικότητα, μια που τον τύπο “Χρόνος” πρέπει να τον συνοδεύσουμε με κάποιες πράξεις, π.χ. την πρόσθεση μεταξύ τιμών τύπου “Χρόνος”. Άρα, για την πρώτη περίπτωση, μια γλώσσα σαν τη C καλύπτει τις ανάγκες μας, δεδομένου ότι μας προσφέρει τον ορισμό νέων τύπων ως δομήσεις δεδομένων μέσω της γλωσσικής δομής `struct`. Για τον ορισμό του τύπου “Χρόνος”, όμως, δεν επαρκεί: επιλέγουμε, λοιπόν, μια γλώσσα προγραμματισμού με δυνατότητα αφαίρεσης στα δεδομένα.

2.3.1 Οι Προδιαγραφές του Νέου Τύπου

Πριν προχωρήσουμε στην υλοποίηση του νέου τύπου “Χρόνος”, έστω `Time`, θα πρέπει να δούμε τις προδιαγραφές του σε εννοιολογικό επίπεδο. Θεωρούμε ότι έχει τις εξής πράξεις:

- δύο πράξεις, “read” και “print”, για να επιτυγχάνεται είσοδος και έξοδος από τις καθιερωμένες μονάδες για στιγμιότυπα/μεταβλητές τύπου `Time`,
- δύο πράξεις, “set” και “get”, για να γίνεται ανάθεση και ανάκτηση τιμής, αντίστοιχα, σε στιγμιότυπα/μεταβλητές τύπου `Time` και
- μια πράξη, +, για να γίνεται πρόσθεση τιμών τύπου `Time` και να επιστρέφεται τιμή τύπου `Time`.

Ας υποθέσουμε, ότι η υπογραφή τους είναι η εξής.

```

read  :  → Time
print :  Time →
set   :  integer integer → Time
+     :  Time Time → Time
get   :  Time → (integer, integer)

```

Το παραπάνω είναι διατυπωμένο σε μια μη υπάρχουσα γλώσσα προδιαγραφών, στην οποία η υπογράμμιση δηλώνει στιγμιότυπο/μεταβλητή που συμμετέχει και στο πεδίο ορισμού (αριστερό μέλος) της πράξης και, με την πράξη αυτή, αλλάζει τιμή.

Το νόημα των παραπάνω πράξεων είναι πολύ απλό, οπότε ας αρκεστούμε στη διατύπωση σε φυσική γλώσσα.

Έχοντας διαθέσιμο τον τύπο *Time*, ο ορισμός και η υλοποίηση του τύπου “Αεροπορική Πτήση” (*Flight*) που, όπως είπαμε είναι μια απλή δόμηση δεδομένων, δεν έχει κάτι το ιδιαίτερο και, αν η σύνταξη είναι παρόμοια με της *C*, μπορεί να είναι ως εξής:

```

struct Flight {
    int FlightNo;
    string DepartureCity;
    string ArrivalCity;
    Time DepartureTime;
    Time Duration;
}

```

Συνεχίζοντας το σενάριό μας, έστω ότι στην ανάπτυξη της εφαρμογής εμπλέκονται (τουλάχιστον) δύο προγραμματιστές: ο προγραμματιστής **A** και ο προγραμματιστής **B**. Αυθαίρετα, ας υποθέσουμε ότι ο προγραμματιστής **B** είναι αυτός που θα υλοποιήσει τους νέους τύπους που χρειάζεται η εφαρμογή και ο προγραμματιστής **A** θα είναι εκείνος που υλοποιεί άλλα μέρη της εφαρμογής και θα χρησιμοποιήσει τους τύπους. Άρα, η υλοποίηση του τύπου *Flight*, όπως δόθηκε παραπάνω, ήταν δουλειά του προγραμματιστή **B**. Το ίδιο ισχύει και για την υλοποίηση του τύπου *Time*. Έτσι, λοιπόν, θα αποκαλούμε τον προγραμματιστή **A** “χρήστη του τύπου” και τον προγραμματιστή **B** “προγραμματιστή του τύπου”. Πριν προχωρήσουν, όμως, αυτοί σε υλοποιήσεις, πρέπει να έχει προηγηθεί μια “συμφωνία” όσον αφορά τη σύνταξη στη γλώσσα υλοποίησης των πρωτότυπων των συναρτήσεων/διαδικασιών που θα υλοποιούν τις πράξεις του τύπου *Time*. Έστω ότι η συμφωνία είναι οι πράξεις να υλοποιούνται μέσω των παρακάτω συναρτήσεων/διαδικασιών:

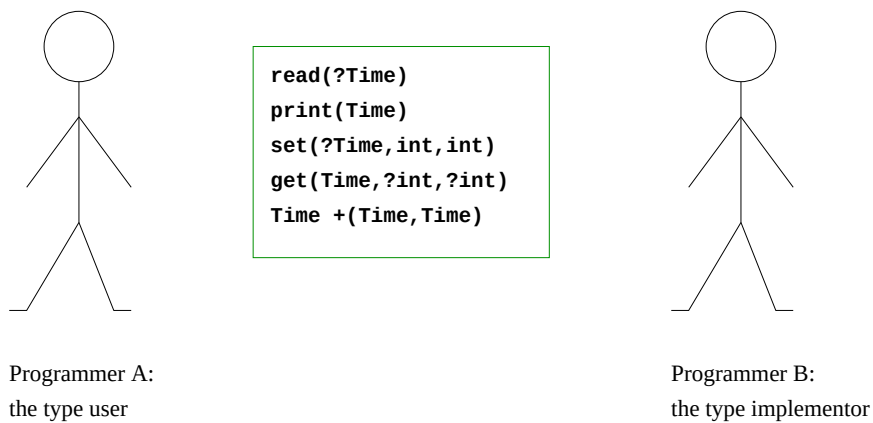
```

read(?Time)          // standard input
print(Time)          // standard output
set(?Time,int,int)   // value assignment
get(Time,?int,?int)  // value retrieval
Time +(Time,Time)    // value addition, result returned in the name of the function

```

Θεωρούμε ότι στην υποθετική γλώσσα μας, η ύπαρξη *?* δηλώνει αλλαγή τιμής για το όρισμα που προσημαίνει. Άρα, η *read* και η *set* αλλάζουν την τιμή του ορίσματος τύπου *Time* και η *get* αλλάζει τα ορίσματά της τύπου *int*.

Αφού καθορίστηκε η “συμφωνία” των προδιαγραφών, οι δύο προγραμματιστές μπορούν να δουλέψουν ανεξάρτητα ο ένας από τον άλλο, με απόλυτη απαίτηση τον σεβασμό της συμφωνίας: ο προγραμματιστής του τύπου πρέπει να υλοποιήσει ακριβώς αυτά που του ζητήθηκαν και ο χρήστης του τύπου να τα χρησιμοποιήσει ακριβώς όπως περιγράφεται στη συμφωνία.



Σχήμα 2.6: Οι δύο προγραμματιστές της εφαρμογής και η “συμφωνία” τους για τις προδιαγραφές του τύπου Time

2.3.2 Η Χρήση του Νέου Τύπου

Ας πάμε, λίγο, στον κόσμο του προγραμματιστή A — χρήστη του τύπου. Έστω ότι γράφει τον παρακάτω κώδικα στην υποθετική γλώσσα μας για να υπολογίσει την ώρα άφιξης μιας συγκεκριμένης πτήσης Αθήνα-Παρίσι.

```

Flight AthensParis;

AthensParis.DepartureCity = "Athens";
AthensParis.ArrivalCity = "Paris";
AthensParis.FlightNo = 314;

set(AthensParis.DepartureTime, 10, 30);

set(AthensParis.Duration, 3, 15);

Time ArrivalTime;

ArrivalTime = AthensParis.DepartureTime + AthensParis.Duration;

print(ArrivalTime);

```

Επεξηγώντας το παραπάνω, έχουμε τα εξής:

`Flight AthensParis;` Ορίζεται μια νέα μεταβλητή τύπου `Flight` με όνομα `AthensParis`.

`AthensParis.DepartureCity = "Athens";` Γίνεται ανάθεση (assignment) τιμής στο πεδίο `DepartureCity` της μεταβλητής `AthensParis` της συμβολοσειράς `"Athens"`.

`AthensParis.ArrivalCity = "Paris";` Γίνεται ανάθεση τιμής στο πεδίο `ArrivalCity` της μεταβλητής `AthensParis` της συμβολοσειράς `"Paris"`.

`AthensParis.FlightNo = 314;` Γίνεται ανάθεση τιμής στο πεδίο `FlightNo` της μεταβλητής `AthensParis` του ακεραίου 314.

`set(AthensParis.DepartureTime, 10, 30);` Γίνεται ανάθεση στο πεδίο `DepartureTime` της μεταβλητής `AthensParis` του συμμιγούς 10ώρες 30λεπτά (δηλαδή, 10 η ώρα και 30).

`set(AthensParis.Duration, 3, 15);` Γίνεται ανάθεση στο πεδίο `Duration` της μεταβλητής `AthensParis` του συμμιγούς 3ώρες 15λεπτά (δηλαδή, 3ώρες και 15λεπτά).

`Time ArrivalTime;` Ορίζεται μια μεταβλητή τύπου `Time`.

`ArrivalTime = AthensParis.DepartureTime + AthensParis.Duration;` Προστίθενται οι τιμές των μεταβλητών τύπου `Time`, `AthensParis.DepartureTime` και `AthensParis.Duration`, και η τιμή, τύπου `Time`, που προκύπτει ανατίθεται ως τιμή της μεταβλητής τύπου `Time`, `ArrivalTime`. Πουθενά δεν έχουμε μιλήσει για ανάθεση τιμής τύπου `Time` σε μεταβλητή τύπου `Time`. Δεν υπάρχει πρόβλημα, όμως. Θεωρούμε ότι αυτό το υποστηρίζει αυτόματα η γλώσσα, κάνοντας επιμέρους αναθέσεις των δομικών στοιχείων της αναπαράστασης του νέου τύπου.

`print(ArrivalTime);` Εκτύπωση της τιμής της μεταβλητής `ArrivalTime` στην καθιερωμένη έξοδο.

Στο παράδειγμα από τον κώδικα του χρήστη του τύπου, συναντάμε τα εξής:

- τη δυνατότητα να ορίζουμε μεταβλητές νέων τύπων,
- την άνεση χρήσης ενός νέου τύπου μέσω της λειτουργικότητάς του (των πράξεων που τον συνοδεύουν),
- τη δυνατότητα υπερφόρτωσης του τελεστή `+`,
- την (ενδεχόμενη) δυνατότητα ανάθεσης σε μεταβλητή του νέου τύπου τιμής του, χωρίς να έχει γίνει κάποια ενέργεια από πλευράς των προγραμματιστών,
- την ανάγκη για πιο αυτοματοποιημένη απόδοση αρχικών τιμών σε μια μεταβλητή νέου τύπου, δεδομένου ότι αυτός μπορεί να δομείται από πολλά πεδία και θα πρέπει να προσέξουμε να τα αρχικοποιήσουμε όλα σωστά.

Στη συνέχεια του παραδείγματος, θα αναδειχθεί περαιτέρω η αξία της αφαίρεσης στα δεδομένα μέσω εγκλωβισμού, που, όπως είπαμε, επιτρέπει — και επιβάλλει — απόκρυψη πληροφορίας.

2.3.3 Η Υλοποίηση του Νέου Τύπου

Τα παραπάνω αφορούσαν μέρος του έργου του χρήστη του τύπου (προγραμματιστής **A**). Ας πάμε λίγο στον προγραμματιστή του τύπου (προγραμματιστής **B**) να δούμε και εκείνου το μέρος του έργου που αφορά την υλοποίηση του τύπου `Time`. Σεβόμενος, πάντα, τη συμφωνία, ουσιαστικά, αυτά που πρέπει να κάνει είναι:

1. να αποφασίσει την αναπαράσταση (των μεταβλητών) του νέου τύπου,
2. με βάση την αναπαράσταση που αποφάσισε, να προχωρήσει στην υλοποίηση των συναρτήσεων/διαδικασιών που αντιστοιχούν στη συμπεριφορά του τύπου και, όπως είπαμε, αποτελούν τις πράξεις που μπορούν να εφαρμοστούν στα στιγμιότυπα/μεταβλητές του τύπου και
3. να υλοποιηθούν όποιες άλλες συναρτήσεις κρίνονται απαραίτητες για την υλοποίηση των συναρτήσεων του 2.

Τόσο το 1, όσο και το 3, αποτελούν αυτό που συχνά αποκαλείται *λεπτομέρειες υλοποίησης* (*implementation details*). Εδώ να σημειώσουμε ότι οι συναρτήσεις που αναφέρονται στο 2 αποτελούν τις χαμηλότερου επιπέδου συναρτήσεις που μπορούν να εφαρμοστούν στις μεταβλητές του τύπου κατά τη χρήση του τύπου. Κάθε άλλη συνάρτηση που θα έχει ως όρισμά της μια μεταβλητή του τύπου θα πρέπει, κατά τον ορισμό της, να καλεί στο σώμα της τις συναρτήσεις αυτές (ή “υψηλότερες” τους, από πλευράς ορισμού).

Ξεκινώντας το έργο του από το 1, ο προγραμματιστής **B** πρέπει να αποφασίσει ποια θα είναι η αναπαράσταση των μεταβλητών του τύπου `Time`. Μην ξεχνάμε ότι, εννοιολογικά, θεωρεί ότι έχουν τιμές συμμιγείς αριθμούς, π.χ. 4 ώρες 45 λεπτά. Ας υποθέσουμε, λοιπόν, ότι εξετάζει τα εξής δύο εναλλακτικά για να αναπαραστήσει μια μεταβλητή τύπου `Time`:

1. να καταχωρήσει χωριστά τις ώρες και τα λεπτά και, επομένως, να την αναπαραστήσει με χρήση δύο μεταβλητών ακεραίων: η μια να αντιστοιχεί στις ώρες και η άλλη στα λεπτά ή
2. να μετατρέψει τις ώρες σε λεπτά, οπότε να χρησιμοποιεί μια μόνο μεταβλητή ακεραίου στην οποία να καταχωρεί τα συνολικά λεπτά που προκύπτουν μετά και τη μετατροπή.

Αν ο προγραμματιστής **B** δεν έχει κάποια πληροφορία για τη χρήση του τύπου, η επιλογή του είναι αυθαίρετη και, έστω, ότι είναι η πρώτη: κάθε μεταβλητή τύπου `Time` θα αναπαριστάνεται από δύο μεταβλητές ακεραίων.

Time t;

1st alternative

<i>int</i>	t.hours
<i>int</i>	t.mins

2nd alternative

<i>int</i>	t.totalMins
------------	--------------------

Σχήμα 2.7: Οι δύο εναλλακτικές αναπαραστάσεις των μεταβλητών του τύπου `Time`

Έστω ότι η υποθετική μας γλώσσα προσφέρει τη γλωσσική δομή `my_type` προκειμένου να οριστεί ένας νέος τύπος. Έστω ότι η γενική μορφή της είναι:

```
my_type <TypeName> {
    <hidden names declarations>

    use_by:

    <specs names declarations>
}
```

Στο παραπάνω, το `<TypeName>` είναι το όνομα του τύπου. Το τμήμα `<specs names declarations>`, που εισάγεται με την έκφραση `use_by:`, περιλαμβάνει τις δηλώσεις των ονομάτων που αναφέρονται στις προδιαγραφές του τύπου και αποτελούν τη “συμφωνία” για το πώς θα χρησιμοποιούνται οι μεταβλητές του τύπου από το υπόλοιπο πρόγραμμα. Τέλος, το τμήμα `<hidden names declarations>` περιλαμβάνει τις δηλώσεις των ονομάτων που είναι απαραίτητα για τον ορισμό του τύπου, αλλά απαγορεύεται να χρησιμοποιηθούν από το υπόλοιπο πρόγραμμα. Μια τέτοια περίπτωση θα μπορούσε να είναι τα ονόματα των μεταβλητών που αποτελούν τα δομικά στοιχεία των μεταβλητών του τύπου — δηλαδή, αντιστοιχούν στην αναπαράστασή του.

Με δεδομένη την παραπάνω γλωσσική δομή για τον ορισμό ενός νέου τύπου, ο ορισμός του τύπου `Time` μπορεί να είναι:

```

my_type Time {
    int hours;
    int mins;
use_by:
    read(?Time);
    print(Time);
    set(?Time,int,int);
    get(Time,?int,?int);
    Time +(Time,Time);
}

```

ο οποίος να συμπληρώνεται με τους απαιτούμενους ορισμούς των συναρτήσεων, που μπορεί να είναι ως εξής:

```

read(?Time t) {
    read(t.hours); read(t.mins);
}

```

```

print(Time t) {
    print(t.hours); print("hours");
    print(t.mins); print("mins");
}

```

```

set(?Time t, int h, int m) {
    t.hours = h; t.mins = m;
}

```

```

get(Time t, ?int h, ?int m) {
    h = t.hours; m = t.mins;
}

```

```

Time +(Time t1, Time t2) {
    Time result;
    result.hours = 0;
    results.mins = t1.mins + t2.mins;
    if (result.mins >= 60) {
        result.mins = result.mins - 60;
        result.hours = results.hours + 1;
    }
    result.hours = result.hours + t1.hours + t2.hours;
    return result;
}

```

Στα παραπάνω υποθέσαμε ότι προσφέρονται από τη γλώσσα συναρτήσεις `read` και `print` που πραγματοποιούν είσοδο και έξοδο ακεραίων από τις καθιερωμένες μονάδες, καθώς και ο τελεστής `+` που πραγματοποιεί πρόσθεση ακεραίων τιμών. Κατά συνέπεια, με τους ορισμούς των συναρτήσεων του τύπου `Time` τους υπερφορτώσαμε.

2.3.4 Αλλαγή της Υλοποίησης του Νέου Τύπου

Ο κώδικας ορισμού του τύπου `Time`, πλέον, μπορεί να συνδυαστεί με τον κώδικα του προγραμματιστή `A` της σελίδας 43 και να καταλήξουμε με πλήρη κώδικα για παραγωγή εκτελέσιμου προγράμματος.

Ας κάνουμε, τώρα, την εξής υπόθεση: ο προγραμματιστής Α έχει στη διάθεσή του τον πηγαίο κώδικα του ορισμού του τύπου `Time`. Βλέπει, λοιπόν, ότι οι μεταβλητές του τύπου “χτίζονται” από δύο μεταβλητές ακεραίων με ονόματα, `hours` και `mins`, οπότε αποφασίζει να τις προσπελάσει απευθείας αλλάζοντας τον κώδικά του ως εξής:

```
Flight AthensParis;

AthensParis.DepartureCity = "Athens";
AthensParis.ArrivalCity = "Paris"
AthensParis.FlightNo = 314;

set(AthensParis.DepartureTime, 10, 30);

set(AthensParis.Duration, 3, 15);

...
AthensParis.Duration.hours++;
....
```

Δηλαδή, πρόσθεσε στον κώδικά του τη γραμμή:

```
AthensParis.Duration.hours++;
```

με την οποία επιδιώκει αύξηση κατά 1 της ώρας της διάρκειας της πτήσης. Ο νόμιμος τρόπος να το επιτύχει είναι μέσω της συνάρτησης `set` για τη μεταβλητή `AthensParis.Duration`, δηλαδή

```
set(AthensParis.Duration, 4, 15);
```

αλλά θεωρώντας, ίσως, ότι αυτό είναι περιττό, επιχειρεί την απευθείας αύξηση. Εδώ, τώρα, έρχεται ο μηχανισμός του “μαύρου κουτιού” του εγκλωβισμού, που παρέχει η γλώσσα, ο οποίος θα επισημάνει “σφάλμα” λόγω απαγορευμένης χρήσης “κρυφού” ονόματος (`hours`) και θα αναστείλει την παραγωγή εκτελέσιμου προγράμματος. Οι “λεπτομέρειες υλοποίησης” οφείλουν να “εγκλωβιστούν” δηλώνοντας τα ονόματά τους στο “κρυφό τμήμα” του ορισμού του τύπου, δηλαδή εκτός του `use_by` τμήματος. Η γλώσσα οφείλει να αναστείλει την παραγωγή εκτελέσιμου προγράμματος για κάθε πηγαίο πρόγραμμα που παραβιάζει τη νόμιμη χρήση ονομάτων. Την αξία αυτού θα τη δούμε στη συνέχεια.

Στο σημείο αυτό, θα κάνουμε μια συζήτηση και για θέματα αποδοτικής αναπαράστασης νέων τύπων: η επιλογή εξαρτάται και από τη χρήση του τύπου. Παρότι η συζήτηση που θα κάνουμε θα αναφέρεται σε υπεραπλουστευμένες περιπτώσεις, οι συλλογισμοί πάνω σε αυτές είναι ουσιαστικοί. Πάμε, λοιπόν, στον τύπο μας, `Time`. Έχοντας διατηρήσει χωριστά τις ώρες και τα λεπτά στην αναπαράστασή του, από τους αντίστοιχους ορισμούς βλέπουμε ότι οι διαδικασίες ανάκτησης τιμής και εκτύπωσής του ολοκληρώνονται άμεσα. Οπότε, αν κατά τη χρήση του τύπου έχουμε ως επί το πλείστον χρήση του μέσω αυτών, η προσέγγισή μας είναι αποδοτική. Αντίθετα, η πράξη της πρόσθεσης, απαιτεί κάποιας μορφής παραπάνω επεξεργασία. Οπότε, αν κατά τη χρήση του τύπου υπερισχύουν πράξεις αυτής της μορφής, καταλληλότερη θα ήταν η αναπαράσταση μέσω μιας και μόνο μεταβλητής ακεραίου στην οποία θα καταχωρούσαμε τον συνολικό χρόνο σε λεπτά. Έστω, λοιπόν, ότι στην εφαρμογή μας ισχύει αυτή η περίπτωση. Άρα, πρέπει να αλλάξουμε την υλοποίηση του τύπου `Time`. Συγκεκριμένα, πρέπει να αλλάξει η αναπαράσταση των μεταβλητών, κάτι που θα συμπαρασύρει την αλλαγή της υλοποίησης των συναρτήσεων. Από την άλλη, τόσο το όνομά του τύπου, όσο και οι δηλώσεις του τμήματος των προδιαγραφών, `use_by`, πρέπει να μείνουν αναλλοίωτα, μια που αυτά αποτελούν τη συμφωνία μεταξύ των προγραμματιστών. Συγκεκριμένα, η υλοποίηση του τύπου γίνεται:

```

my_type Time {
    int totalMins;    // this is the change in the type definition
use_by:
    read(?Time);
    print(Time);
    set(?Time,int,int);
    get(Time,?int,?int);
    Time +(Time,Time);
}

```

η οποία να συμπληρώνεται με τους απαιτούμενους ορισμούς των συναρτήσεων, που μπορεί να είναι ως εξής:

```

read(?Time t) {
    int h, m;
    read(h); read(m);
    t.totalMins = h * 60 + m;
}

print(Time t) {
    print(t.totalMins / 60); print("hours");
    print(t.totalMins % 60); print("mins");
}

set(?Time t, int h, int m) {
    t.totalMins = h * 60 + m;
}

get(Time t, ?int h, ?int m) {
    h = t.totalMins / 60; m = t.totalMins % 60;
}

Time +(Time t1, Time t2) {
    Time result;
    result.totalMins = t1.totalMins + t2.totalMins;
    return result;
}

```

Βλέπουμε, τώρα, ότι αν επιτρέπαμε τη χρήση των κρυφών μεταβλητών, `hours` και `mins`, από τον υπόλοιπο κώδικα της εφαρμογής, το πρόγραμμα θα “κρέμαγε” μετά την αλλαγή αυτή, κάτι που με την απαγόρευση αποτρέπεται. Ο εγκλωβισμός, λοιπόν, μας προφυλάσσει από αυθαίρετες χρήσεις ονομάτων εκτός των προδιαγραφών της συμφωνίας προκειμένου να διαφυλαχθεί η ανεξαρτησία της υλοποίησης του τύπου από τη χρήση του.

2.3.5 Σχόλια και Συμπεράσματα από το Παράδειγμα

Τι είδαμε, λοιπόν, στο παραπάνω παράδειγμα, χρησιμοποιώντας αυτήν την υποθετική γλώσσα που μας επέτρεπε ορισμό νέων τύπων και αφαίρεση στα δεδομένα; Είδαμε:

- Τη δυνατότητα ορισμού νέων τύπων που να προσομοιώνουν τις οντότητες της εφαρμογής μας (ο τύπος `Time`, ο τύπος `Flight`).

- Τη δυνατότητα διαχωρισμού των δηλώσεων των ονομάτων της συμφωνίας των προδιαγραφών (τμήμα που εισάγεται από την έκφραση `use_by`) από τις δηλώσεις των ονομάτων που αφορούν λεπτομέρειες υλοποίησης.
- Την ανεξαρτησία του ορισμού του νέου τύπου από τη χρήση του, στον βαθμό που τηρούνται οι προδιαγραφές του.
- Τη δυνατότητα υπερφόρτωσης συναρτησιακών συμβόλων (επιπλέον ορισμοί για τις συναρτήσεις `read`, `print` και του τελεστή `+`).
- Την επιθυμητή δυνατότητα της αυτόματης επέκτασης των δυνατοτήτων ανάθεσης τιμών για τον νέο τύπο (χρήση του τελεστή `=`).

Να υπενθυμίσουμε, όμως, την ανάγκη για αυτοματοποιημένη απόδοση αρχικών τιμών στις μεταβλητές ενός νέου τύπου δεδομένων, κάτι το οποίο δεν ικανοποιήσαμε με την υποθετική μας γλώσσα αλλά απλά αναδείξαμε με το παράδειγμά μας. Στη γενική περίπτωση των γλωσσών αντικειμενοστραφούς προγραμματισμού δίνεται η δυνατότητα αυτές οι *αρχικοποιήσεις* να γίνονται αυτόματα με διαδικασίες που εκτελούνται μέσω ειδικών συναρτήσεων των οποίων τη συμπεριφορά υλοποιεί ο “προγραμματιστής του τύπου”. Σε επόμενα κεφάλαια θα αναφερθούμε εκτενώς στο θέμα αυτό και μέσω των γλωσσών με τις οποίες θα ασχολούμαστε.

Συνοψίζοντας το τι είδαμε στο παράδειγμα, μπορούμε να πούμε τα εξής. Οι απαιτήσεις για αφαίρεση που έχουν οι σύγχρονες εφαρμογές επεκτείνονται και στο επίπεδο των δεδομένων. Οπότε, υπάρχει μια έντονη ανάγκη για αφαίρεση δεδομένων. Σ’ αυτήν την περίπτωση, οι διάφοροι τύποι δεδομένων περιγράφονται από τα κύρια χαρακτηριστικά τους και επιτρέπεται να τους χρησιμοποιήσει κανείς μόνο μέσω ενός συνόλου προκαθορισμένων λειτουργιών που καθορίζουν τη συμπεριφορά τους νοηματικά. Σύνθετοι τύποι δεδομένων κατασκευάζονται από απλούστερους και προσδιορίζεται ένα σύνολο λειτουργιών που επιτρέπεται να εφαρμοστούν στα στοιχεία καθενός από τους νέους τύπους. Η αναπαράσταση του τύπου, δηλαδή η δομή του και η φύση των δομικών του λίθων, καθώς και η υλοποίηση των επιτρεπτών λειτουργιών του είναι κρυμμένες από τον χρήστη του τύπου.

Η δυνατότητα αφαίρεσης ελαττώνει το μέγεθος της λεπτομέρειας την οποία πρέπει να χειρίζεται ο προγραμματιστής κάθε χρονική στιγμή. Ο χρήστης του τύπου μπορεί να επικεντρωθεί στο τι θέλει να κάνει πάνω στο στιγμιότυπο του νέου τύπου, αντί να ταλαιπωρείται με το πώς θα το πραγματοποιήσει. Το δεύτερο είναι ευθύνη αυτού που υλοποιεί τον νέο τύπο. Σημαντικό είναι ότι δεδομένου ότι ο χρήστης του νέου τύπου δεν ασχολείται με καμία λεπτομέρεια της υλοποίησής του, η εφαρμογή αναπτύσσεται ανεξάρτητα από συγκεκριμένη υλοποίηση του τύπου. Κατ’ αυτόν τον τρόπο, η υλοποίησή του μπορεί να αλλάξει οποιαδήποτε στιγμή, χωρίς να επιβάλλει τροποποιήσεις στην υλοποίηση της εφαρμογής. Σύμφωνα με το [2], δεδομένου ότι η υλοποίηση και η χρήση ενός νέου τύπου (η αναφορά γίνεται σε ΑΤΔ, ειδικότερα) είναι δύο διακριτές δραστηριότητες, η υλοποίηση του τύπου μπορεί να αναβληθεί για αργότερα, αφού έχει ολοκληρωθεί ο σχεδιασμός της εφαρμογής. Έχοντας δεδομένο τον σχεδιασμό, μπορεί να εκτιμηθεί η συχνότητα χρήσης του τύπου, δηλαδή πόσα στιγμιότυπα του δημιουργούνται, ποιες είναι οι λειτουργίες του που εφαρμόζονται συχνότερα κλπ. Οπότε, μπορεί να αποφασιστεί πόσο αποδοτική από πλευράς χώρου ή χρόνου πρέπει να είναι η υλοποίηση του νέου τύπου. Επίσης, δεδομένης της ανεξαρτησίας μεταξύ υλοποίησης και χρήσης του νέου τύπου, μπορούμε να δοκιμάζουμε διαφορετικές μεθοδολογίες υλοποίησης, για να διαλέξουμε εκείνη που κάνει την εφαρμογή πιο αποδοτική.

Στο σημείο αυτό να επισημάνουμε ότι στο παράδειγμά μας δεν ασχοληθήκαμε καθόλου με την έννοια της κληρονομικότητας που είναι τόσο σημαντική στον ΟΟΡ. Ο λόγος είναι ότι θέλουμε να προχωρήσουμε σταδιακά, χωρίς να πολυπλοκεύουμε τα προβλήματα, ώστε να γίνει η κατανόηση των εννοιών ευκολότερα. Εξ άλλου, σε επόμενα κεφάλαια, η έννοια της κληρονομικότητας θα δεσπόζει στα παραδείγματά μας.

2.4 Σύνοψη Κεφαλαίου

- Ενδιαμέσως του διαδικαστικού προγραμματισμού και του αντικειμενοστραφούς, μπορούμε να τοποθετήσουμε την τμηματοποίηση κώδικα μέσω στοιχείων και την αφαίρεση στα δεδομένα, η οποία επιτρέπει τον ορισμό νέων τύπων, χωρίς απαραίτητα να γίνουν ιεραρχίες τους.
- Εκτενείς ή/και περίπλοκες εφαρμογές επωφελούνται από γλώσσες με μηχανισμό εγκλωβισμού που επιτυγχάνει απόκρυψη πληροφορίας. Έτσι, τμήματα κώδικα μπορούν να αναπτυχθούν και να συντηρηθούν ανεξάρτητα το ένα από το άλλο, σεβόμενα τις διεπαφές τους.
- Στις γλώσσες προγραμματισμού που επιτρέπουν αφαίρεση στα δεδομένα, γίνεται ορισμός από τον χρήστη νέων τύπων, η πρόσβαση στα στοιχεία των οποίων γίνεται μέσω συγκεκριμένων λειτουργιών/συναρτήσεων.
- Ένας τύπος είναι η φυσική αναπαράσταση μιας ιδεατής οντότητας. Π.χ. ο τύπος float με τελεστές +, -, *, ... είναι η αναπαράσταση του πραγματικού αριθμού των μαθηματικών. Ένα πρόγραμμα που χρησιμοποιεί τύπους που προσομοιώνουν τις οντότητες της εφαρμογής γίνεται ευκολότερα κατανοητό.
- Προκειμένου να γίνει εφικτή η ορθή χρήση του τύπου, είναι απαραίτητος ο τυπικός ορισμός των συντακτικών του προδιαγραφών. Επίσης, τυπικός ορισμός της σημασιολογίας, π.χ. μέσω αλγεβρικών προδιαγραφών, δίνει τη δυνατότητα για τυπική επαλήθευση της ορθότητας της υλοποίησής του, αλλά δεν είναι πάντα πρακτικά δυνατό να διατυπωθεί.
- Κατά τον ορισμό ενός τύπου, πρέπει να γίνεται διαχωρισμός των λεπτομερειών υλοποίησης του τύπου από τις ιδιότητες και τις λειτουργίες του τύπου που είναι απαραίτητες για τη σωστή χρήση του.
- Υπάρχουν διάφοροι λόγοι για τους οποίους είναι επιθυμητό ένα σύστημα προγραμματισμού να υποστηρίξει τον ορισμό νέων τύπων από τον χρήστη. Η δυνατότητα ορισμού τους και δημιουργίας στιγμιοτύπων τους επιτρέπει τη δήλωση και χρήση αντικειμένων που έχουν συμπεριφορά παρόμοια με τις οντότητες της εφαρμογής στην οποία χρησιμοποιούνται. Αυτό είναι εξαιρετικά χρήσιμο για τις σύγχρονες περιοχές εφαρμογών, στις οποίες τα δεδομένα δεν είναι μόνο αριθμητικά και απαιτείται αποδοτικός και έξυπνος χειρισμός τους.

2.5 Βιβλιογραφία

- [1] L. Cardelli και P. Wegner. "On Understanding Types, Data Abstraction, and Polymorphism". Στο: *ACM Computing Surveys* 17.4 (Δεκ. 1985).
- [2] Manoochehr Azmoodeh. *Abstract Data Types and Algorithms*. MacMillan Education, 1988.
- [3] Rod Ellis. *Data Abstraction and Program Design*. Pitman Publishing, 1991.
- [4] Johannes J. Martin. *Data Types and Data Structures*. Prentice Hall, 1986.
- [5] Jacques Loeckx. "Algorithmic Specifications: A Constructive Specification Method for Abstract Data Types". Στο: *ACM Transactions on Programming Languages and Systems* 9.4 (Οκτ. 1987), σσ. 646–685.
- [6] Niklaus Wirth. *Programming in MODULA-2*. 4η έκδοση. Springer-Verlag, 1988.
- [7] L. Cardelli, J. Donahue, L. Glassman, M. Jordan, B. Kalsow και G. Nelson. "Modula-3 Language Definition". Στο: *ACM SIGPLAN Notices* 27.8 (Αύγ. 1992).
- [8] Ellis Horowitz. *Fundamentals of Programming Languages*. 2η έκδοση. Computer Science Press, 1984.

- [9] J. Goguen και J. Meseguer. *Unifying Functional, Object-Oriented and Relational Programming with Logical Semantics*. Report SRI-CSL-87-7. Menlo Park: SRI International, Ιούλ. 1987.
- [10] Jim Welsh και John Elder. *Introduction to Modula-2*. Prentice Hall, 1987.
- [11] L. Cardelli, J. Donahue, L. Glassman, M. Jordan, B. Kalsow και G. Nelson. *Modula-3 Report (revised)*. Report. Digital Equipment Corporation, Νοέ. 1989.
- [12] *Miranda System Manual*. 1987.
- [13] Bjarne Stroustrup. *The C++ Programming Language*. 2η έκδοση. Addison-Wesley Publishing Company, 1991.
- [14] Bertrand Meyer. *Object-oriented Software Construction*. Prentice-Hall, Inc., 1988.
- [15] Terrence W. Pratt. *Programming Languages: Design and Implementation*. 2η έκδοση. Prentice-Hall, Inc., 1984.
- [16] Grady Booch, Robert Maksimchuk, Michael Engle, Bobbi Young, Jim Conallen και Kelli Houston. *Object-Oriented Analysis and Design with Applications*. 3η έκδοση. Addison-Wesley Professional, 2007.

ΣΤΟΙΧΕΙΑ ΑΠΟ ΤΟΝ ΔΙΑΔΙΚΑΣΤΙΚΟ ΠΡΟΓΡΑΜΜΑΤΙΣΜΟ

Σύνοψη

Στο κεφάλαιο αυτό, αρχικά, θα κάνουμε μια σύντομη σύνοψη των βασικών στοιχείων των γλωσσών του διαδικαστικού προγραμματισμού και πώς αυτά υποστηρίζονται από τη γλώσσα C++. Στη συνέχεια, θα εστιάσουμε σε γλωσσικές δομές και χαρακτηριστικά της C++ που σχετίζονται με τον διαδικαστικό προγραμματισμό και των οποίων η γνώση είναι απαραίτητη για την πλήρη κατανόηση της συμπεριφοράς κάποιων αντικειμενοστραφών χαρακτηριστικών της γλώσσας. Για την καλύτερη κατανόηση του κεφαλαίου, συμπεριλαμβάνονται θέματα που σχετίζονται με τη διαδικασία μεταγλώττισης των προγραμμάτων C++.

Προαπαιτούμενη Γνώση

Για την κατανόηση του κεφαλαίου, ο αναγνώστης απαιτείται να έχει γνώσεις διαδικαστικού προγραμματισμού με χρήση της γλώσσας C.

3.1 Γενικές Παρατηρήσεις

Όπως είπαμε και στα εισαγωγικά κεφάλαια, το αντικείμενό μας είναι ο αντικειμενοστραφής προγραμματισμός πάνω από τον διαδικαστικό. Δηλαδή, το αντικειμενοστραφές μοντέλο προστίθεται στον διαδικαστικό προγραμματισμό, ώστε ο δεύτερος να χρησιμοποιηθεί για να περιγραφούν οι ενέργειες που χαρακτηρίζουν τη συμπεριφορά των κλάσεων. Αυτό επιτυγχάνεται είτε θεωρώντας ότι το πρόγραμμά μας απαρτίζεται αποκλειστικά από κλάσεις, όπως είναι η περίπτωση των προγραμμάτων Java, είτε από συναρτήσεις και κλάσεις, όπως είναι η περίπτωση των προγραμμάτων C++. Όποια και να είναι η περίπτωση, μια αναδρομή στα βασικά συστατικά των γλωσσών διαδικαστικού προγραμματισμού είναι καλό να γίνει.

Στο κεφάλαιο αυτό, θα κάνουμε μια ανασκόπηση των γενικών εννοιών του διαδικαστικού προγραμματισμού εστιάζοντας στη C++. Στην συνέχεια, θα αναφερθούμε, κυρίως, σε γλωσσικά στοιχεία της C++ των οποίων η κατανόηση είναι κρίσιμη για τον ορισμό και τη χρήση των κλάσεων όταν θα παρουσιάζουμε τα αντι-

κειμενοστραφή χαρακτηριστικά της γλώσσας. Στη γενική περίπτωση, οι προγραμματιστές χρησιμοποιούν τα στοιχεία αυτά με βάση κάποιους κανόνες, αγνοώντας ή ξεχνώντας πώς αυτοί έχουν προκύψει. Το πιο ασφαλές και σωστό, όμως, είναι να αντιλαμβάνονται πλήρως τι συμβαίνει στις περιπτώσεις αυτές, μια που υπάρχουν λεπτά σημεία. Σε αυτά, λοιπόν, θα επιμείνουμε.

Το ζητούμενο της πλήρους κατανόησης του κώδικα χαρακτηρίζει και τις συζητήσεις που θα γίνονται στα επόμενα κεφάλαια, ώστε να είναι πλήρως προβλέψιμη η συμπεριφορά ενός προγράμματος. Έτσι, λοιπόν, παρότι ο πρωταρχικός μας στόχος είναι η κατανόηση των εννοιών του αντικειμενοστραφούς προγραμματισμού ως μέσο μοντελοποίησης, θα αναλύουμε κάθε κρίσιμο σημείο των προγραμμάτων που θα παρουσιάζουμε. Σε καθένα από τα επόμενα κεφάλαια, αρχικά θα γίνεται μια αφαιρετική περιγραφή των εννοιών που εισάγονται και της ανάγκης που εξυπηρετούν και, στη συνέχεια, θα αλλάζουμε το ύφος και το επίπεδο περιγραφής, ώστε να γίνεται πλήρως κατανοητή η συμπεριφορά της υποστήριξής τους από τη γλώσσα C++.

Ξεκινάμε, όπως είπαμε, με τις έννοιες του διαδικαστικού προγραμματισμού.

Σε ένα πρόγραμμα γραμμένο σε μια γλώσσα διαδικαστικού προγραμματισμού υλοποιούμε έναν αλγόριθμο. Αυτός εφαρμόζεται σε κάποια δεδομένα. Τα δεδομένα αυτά μπορεί να είναι απλά/στοιχειώδη ή σύνθετα. Στη συνήθη περίπτωση που η γλώσσα προγραμματισμού υποστηρίζει τύπους δεδομένων, τα δεδομένα θα ανήκουν σε κάποιον τύπο. Συνήθως, οι τύποι που προσφέρονται από μια γλώσσα είναι τύποι ακεραίων αριθμών, πραγματικών αριθμών, χαρακτήρων, συμβολοσειρών και πινάκων. Επιπροσθέτως, ενδέχεται, να προσφέρεται μια στοιχειώδης δυνατότητα ορισμού νέων τύπων από τον προγραμματιστή.

Όσον αφορά την υλοποίηση των αλγορίθμων, οι γλώσσες διαδικαστικού προγραμματισμού προσφέρουν τις κατάλληλες γλωσσικές δομές για να εκφράσει ο προγραμματιστής τη ροή του ελέγχου (control) που υλοποιεί τα βήματα του αλγορίθμου. Αυτές, στη γενική περίπτωση, πρέπει να προσφέρουν τη δυνατότητα έκφρασης:

- ακολουθίας (sequence) διαδοχικών βημάτων προς εκτέλεση
- επιλογής (choice) (ομάδας) βημάτων με βάση κάποια συνθήκη
- επανάληψης (repetition/iteration) (ομάδας) βημάτων
- ανάθεσης τιμής σε μεταβλητή (memory assignment)

Επίσης, πρέπει ακόμα να προσφέρει δυνατότητα

- εισόδου και εξόδου τιμών (input/output)

Ιδανικά, απαραίτητα θα λέγαμε, πρέπει να προσφέρεται η δυνατότητα ομαδοποίησης κώδικα με τη μορφή διαδικασιών (procedures) ή συναρτήσεων (functions). Με τον τρόπο αυτό, ο κώδικας είναι πιο δομημένος, ώστε να μπορεί ο προγραμματιστής να λειτουργεί σε επίπεδα αφαίρεσης που του επιτρέπουν μεγαλύτερη άνεση και θα τον αποτρέψουν από σφάλματα.

Τα παραπάνω, όπως είπαμε, αφορούν κάθε γλώσσα διαδικαστικού προγραμματισμού. Ας εστιάσουμε στη γλώσσα C++, τώρα.

3.2 Γενικά Στοιχεία για τη Γλώσσα Προγραμματισμού C++

Ας κάνουμε μια μικρή σύνοψη για τη C++, επαναλαμβάνοντας και σχολιάζοντας εκείνη που κάνει ο ίδιος ο δημιουργός της, ο Bjarne Stroustrup. Το όνομά της δηλώνει επαύξηση (increment) της γλώσσας C, το οποίο δηλώνεται με τη χρήση του τελεστή επαύξησης, ++. Η γλώσσα αυτή σχεδιάστηκε για

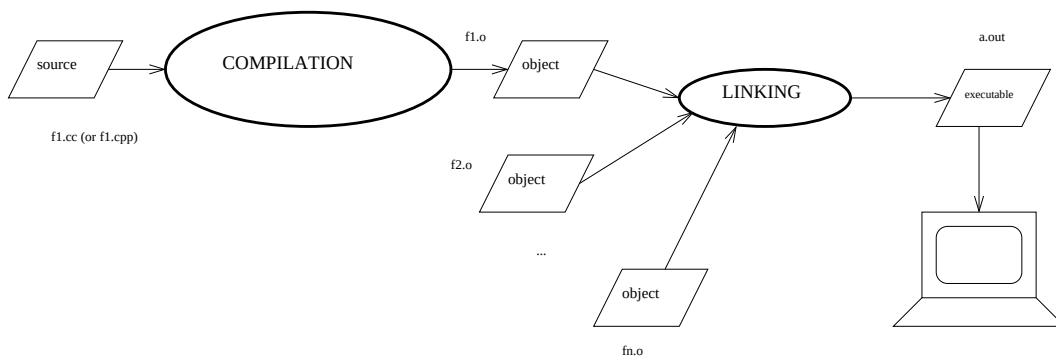
- να είναι μια καλύτερη C, υποστηρίζει διαδικαστικό προγραμματισμό με τη μορφή που τον υποστηρίζει η γλώσσα C, επιπλέον βελτιώνει κάποια χαρακτηριστικά της,
- να υποστηρίζει αφαίρεση δεδομένων, επιτρέποντας τον ορισμό νέων τύπων από τον προγραμματιστή, τις κλάσεις, συνδέοντάς τους με τη συμπεριφορά τους και

- να υποστηρίζει αντικειμενοστραφή προγραμματισμό, επιτρέποντας συσχέτιση των κλάσεων μέσω κληρονομικότητας και κληροδότηση χαρακτηριστικών και συμπεριφοράς.

Η C++ είναι πιο κατάλληλη για ανάπτυξη μεγαλύτερων προγραμμάτων σε σχέση με τη C. Για τον λόγο αυτό, είναι υψίστης σημασίας η έννοια της εμβέλειας ονομάτων-νοημάτων. Βασικό, όμως, είναι ότι έχει αντικειμενοστραφή χαρακτηριστικά. Για να υποστηρίξει επιτυχώς τον αντικειμενοστραφή προγραμματισμό, έχει ένα ισχυρό σύστημα ελέγχου τύπων (type checking system). Η έννοια της “κλάσης” ως τύπος ορισμένος από τον χρήστη είναι εμπνευσμένη από τη SIMULA 67. Στο πλαίσιο, όμως, των δυνατοτήτων ορισμού κλάσεων, η C++ παρέχει διάφορες εξειδικευμένες δυνατότητες.

3.2.1 Μεταγλώττιση και Αρχεία Επικεφαλίδας

Στην παράγραφο αυτή, θα περιγράψουμε τη διαδικασία μεταγλώττισης των προγραμμάτων C++, μια που στις συζητήσεις, στη συνέχεια, θα χρειαστούμε όρους από τη διαδικασία αυτή. Θα κάνουμε μια σύνοψη αυτών που ήδη γνωρίζετε από τη γλώσσα C και αφορούν τη μεταγλώττιση και τα αρχεία επικεφαλίδας. Θα εστιάσουμε στη διαδικασία σε περιβάλλον λειτουργικού Unix μορφής (Linux, MacOS, WSL κλπ.).



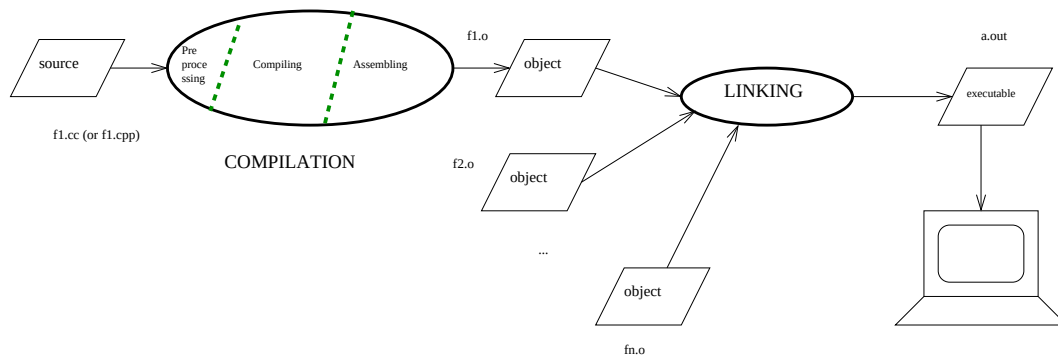
Σχήμα 3.1: Διαδικασία μεταγλώττισης προγραμμάτων C++ σε περιβάλλον Unix μορφής

Γράφουμε το πρόγραμμά μας σε ένα ή περισσότερα αρχεία πηγαίου κώδικα (με κατάληξη, συνήθως, .cc — .cpp σε περιβάλλον Windows). Αυτά δίνονται ως είσοδος στη φάση της μεταγλώττισης (compilation) από την οποία παράγονται τα αντίστοιχα αρχεία αντικειμενικού κώδικα (object code). Στη συνέχεια, συλλέγονται όλα τα απαραίτητα για το πρόγραμμα αρχεία αντικειμενικού κώδικα, τα οποία δίνονται ως είσοδος στη φάση σύνδεσης (linking). Από τη φάση αυτή παράγεται το αρχείο με τον κώδικα του εκτελέσιμου προγράμματος — σε περιβάλλον Unix μορφής, εκτός αν έχει υποδειχθεί κάτι διαφορετικό, το όνομα του αρχείου αυτού είναι το a.out. Το πρόγραμμα αυτό εκτελείται απευθείας από τον υπολογιστή. Η διαδικασία φαίνεται στο Σχήμα 3.1.

Αναλύοντας τη φάση της μεταγλώττισης, όπως φαίνεται στο Σχήμα 3.2, παρατηρούμε ότι ξεκινά από το στάδιο της προεπεξεργασίας (preprocessing), το οποίο οφείλει να προετοιμάζει τα αρχεία πηγαίου κώδικα για τη μεταγλώττιση, π.χ. κάνοντας την απαραίτητη εισαγωγή αρχείων σύμφωνα με τις οδηγίες #include. Στη συνέχεια, η μεταγλώττιση μπορεί να χωριστεί σε δύο επιμέρους στάδια: 1) το στάδιο που θα παραγάγει τον κώδικα assembly για τον υπολογιστή για τον οποίο θα κατασκευαστεί τελικά το εκτελέσιμο και 2) το στάδιο του assembling από το οποίο θα παραχθεί ο αντικειμενικός κώδικας.

Ο λόγος που γίνεται η ανάλυση αυτή, πέρα από μια επανάληψη για τη διαδικασία μεταγλώττισης, είναι για να υπενθυμίσουμε τον ρόλο του προεπεξεργαστή: ετοιμάζει το τελικό προς μεταγλώττιση πηγαίο πρόγραμμα. Στη C, συχνά καταφεύγουμε στη χρήση οδηγιών (directives) του προεπεξεργαστή για να εκφράσουμε στοιχεία του πηγαίου προγράμματος, όπως ο ορισμός σταθερών και απλών συναρτήσεων. Στη C++, όπως θα δούμε, όμως, έχουμε τα απαραίτητα γλωσσικά στοιχεία ώστε να αποφύγουμε το τέχνασμα αυτό και να περιορίσουμε τη χρήση του προεπεξεργαστή αποκλειστικά για τον ρόλο του.

Κάποιες γενικές επαναληπτικές παρατηρήσεις τώρα:



Σχήμα 3.2: Ανάλυση της φάσης μεταγλώττισης

- Στα αρχεία επικεφαλίδας καλό είναι να γράφουμε δηλώσεις (declarations) (συναρτήσεων και μεταβλητών) μόνο. Αυτό προτείνεται ώστε να αποφευχθεί το σφάλμα πολλαπλών ορισμών της ίδιας οντότητας, στην περίπτωση που το αρχείο επικεφαλίδας συμπεριληφθεί σε παραπάνω από ένα αρχεία του προγράμματος. Αυτό δεν ισχύει για ορισμούς συναρτήσεων και μεταβλητών με εσωτερική συνδεσιμότητα. Τέτοιους ορισμούς μπορούμε να τους συμπεριλαμβάνουμε χωρίς ανησυχία.

- Παρατηρήσεις σχετικά με τις οδηγίες του προεπεξεργαστή:

`#include <iostream>`: χρησιμοποιώντας `< >` αναφερόμαστε στον standard include κατάλογο

`#include "my_header.h"`: χρησιμοποιώντας `" "`, η αναζήτηση του αρχείου επικεφαλίδας γίνεται στον τρέχοντα κατάλογο

`#include "/home/users/izambo/c++/includes/hed.h"`: δίνουμε όλο το μονοπάτι ρητά

- Παρατηρήσεις σχετικά με την εντολή μεταγλώττισης:

`g++ -o bla main.cc f.cc`: μεταγλώττιση και παραγωγή εκτελέσιμου προγράμματος από ένα πρόγραμμα που αποτελείται από τα αρχεία πηγαίου κώδικα `main.cc` και `f.cc`. Το αποτέλεσμα της εντολής, το εκτελέσιμο πρόγραμμα, επιθυμούμε να ονομαστεί `bla`.

`g++ -c main.cc`: μεταγλώττιση χωρίς απόπειρα σύνδεσης. Παραγωγή αντικειμενικού κώδικα, στο αρχείο `main.o`.

`g++ -c f.cc`: όμοια με το προηγούμενο.

`g++ -o bla main.o f.o`: σύνδεση των αρχείων αντικειμενικού κώδικα για παραγωγή του εκτελέσιμου προγράμματος το οποίο θα περιέχεται στο αρχείο `bla`.

- Παράμετροι για τη μεταγλώττιση όπως περιγράφονται στο εγχειρίδιο του μεταγλωττιστή `g++`.

`-c` Compile or assemble the source files, but do not link.
The compiler output is an object file corresponding to each source file.

.....

`-E` Stop after the preprocessing stage; do not run the compiler proper. The output is preprocessed source code, which is sent to the standard output.

.....

-o file

Place output in file.

Χρησιμοποιώντας κατάλληλα προγράμματα που θα βρείτε στην συνέχεια, πειραματιστείτε με την παράμετρο -E για να δείτε τι παράγεται ως έξοδος μετά την προεπεξεργασία. Είναι ενδιαφέρον να δείτε τις αντικαταστάσεις που γίνονται λόγω του `#include`, καθώς και λόγω των ορισμών `#define`.

Το ελάχιστο C++ πρόγραμμα αποτελείται από μία συνάρτηση, την `main`. Η εκτέλεση κάθε προγράμματος C++ ξεκινά από την κλήση της συνάρτησης αυτής. Αυτό επιβεβαιώνει ότι η C++ είναι, κατά βάση, μια γλώσσα διαδικαστικού προγραμματισμού, στην οποία προστέθηκε το αντικειμενοστραφές μοντέλο. Η συνάρτηση `main` έχει ακριβώς δύο εναλλακτικές μορφές με πρωτότυπα ίδια με της C.

Το ελάχιστο πρόγραμμα C++ είναι:

```
int main()
{
    return 0;
}
```

Για να έχουμε και κάποια έξοδο, μπορούμε να γράψουμε το εξής πρόγραμμα:

```
#include <iostream>
using namespace std; // We shall discuss this later
// In the general case, we should not do that but
// our programs are simple enough to allow
// using the namespace

int main()
{
    cout << "Hello world" << endl;
    return 0;
}
```

Η έκφραση `#include <iostream>` δηλώνει στον προεπεξεργαστή να συμπεριλάβει τις δηλώσεις που βρίσκονται στο αρχείο επικεφαλίδας `iostream`. Το αρχείο αυτό περιέχει τις δηλώσεις που είναι απαραίτητες για να κάνουμε είσοδο/έξοδο χρησιμοποιώντας τις καθιερωμένες (standard) μονάδες εισόδου/εξόδου.

Το παραπάνω πρόγραμμα εκτυπώνει στην καθιερωμένη μονάδα εξόδου τη συμβολοσειρά "Hello world" και κατόπιν γίνεται αλλαγή γραμμής. Η εκτύπωση πραγματοποιείται με τη χρήση του τελεστή εξόδου, `<<`. Η καθιερωμένη μονάδα εξόδου συνδέεται με το ρεύμα `cout`. Η έκφραση `endl` αντιστοιχεί σε μια συνάρτηση μιας κατηγορίας συναρτήσεων βιβλιοθήκης της C++ που αφορούν είσοδο/έξοδο και λέγονται "διαμορφωτές" (manipulators). Η εν λόγω συνάρτηση αυτό που κάνει είναι να προσθέτει τον χαρακτήρα αλλαγής γραμμής στον αγωγό/προσωρινή μνήμη φύλαξης (buffer) για την έξοδο και στη συνέχεια να αδειάζει (flush) τον αγωγό.

Τι παρατηρούμε στο παραπάνω πρόγραμμα — και γι' αυτό κάναμε και τη συζήτηση επ' αυτού: την υπερφόρτωση του τελεστή `<<`. Η λειτουργικότητά του ως τελεστής μετατόπισης δυαδικών ψηφίων (shift operator) παραμένει. Όταν, όμως, αριστερά του εμφανίζεται ρεύμα εξόδου, λειτουργεί ως συνάρτηση εξόδου.

Αγνοώντας, προς το παρόν, τη δυνατότητα ορισμού κλάσεων, η οργάνωση ενός προγράμματος C++, στη γενική περίπτωση, έχει ως εξής. Γράφουμε τους ορισμούς των συναρτήσεων (και μεταβλητών, ενδεχόμενα) σε αρχεία πηγαίου κώδικα. Ένα αρχείο πηγαίου κώδικα ενδέχεται να συνοδεύεται από ένα αρχείο επικεφαλίδας το οποίο περιέχει εκείνες τις δηλώσεις συναρτήσεων (και μεταβλητών) που το πρώτο έχει ορίσει και που μπορεί να χρησιμοποιηθούν και από άλλα αρχεία πηγαίου κώδικα. Έτσι, αρκεί τα άλλα αυτά αρχεία να συμπεριλάβουν το αρχείο επικεφαλίδας προκειμένου να μπορούν να μεταγλωττιστούν. Μια μικρή υπενθύμιση: η μονάδα με-

ταγλώττισης (compilation unit) είναι ένα αρχείο πηγαίου κώδικα. Δεν είναι δυνατόν να μεταγλωττίσουμε κάτι λιγότερο, π.χ. μια μεμονωμένη συνάρτηση ενός αρχείου.

Για να εξοικειωθείτε λίγο με τη διαδικασία μεταγλώττισης, κάντε δοκιμές μεταγλώττισης χρησιμοποιώντας τα παρακάτω αρχεία

main.cpp:

```
#include <iostream>
#include "p1.h"
using namespace std;

int main()
{
    cout << "HELLO!" << endl;
    f(5);
    cout << "Hi!" << endl;

    return 0;
}
```

p1.cpp:

```
#include <iostream>
using namespace std;

void f(int i)
{
    cout << i << endl;
}
```

p1.h:

```
void f(int);
```

Υπενθύμιση από τη C: η μορφή της συνάρτησης main με ορίσματα (παράδειγμα)

```
#include <iostream>
using namespace std;

int main(int argc, char* argv[])
{
    cout << "argc = " << argc << endl;

    for (int i = 0; i < argc; i++)
        cout << "argv[" << i << "] = " << argv[i] << endl;

    return 0;
}
```

3.2.2 Διαδικαστικός Προγραμματισμός με/σε C++

Αφού, αρχικά, μιλήσαμε γενικά και αφηρημένα για το τι περιμένουμε να μας παρέχει μια γλώσσα διαδικαστικού προγραμματισμού και, στη συνέχεια, πήγαμε στο άλλο άκρο και μιλήσαμε σχετικά με την οργάνωση των προγραμμάτων C++ και τη διαδικασία μεταγλώττισής τους, ας προχωρήσουμε με μια σύνοψη του τι μας προσφέρει η C++ ως γλώσσα διαδικαστικού προγραμματισμού.

Κατ' αρχήν, όσον αφορά τα δεδομένα, η C++ προσφέρει:

- Μια ποικιλία ενσωματωμένων τύπων.
- Μια ποικιλία τύπων από την καθιερωμένη βιβλιοθήκη της. Αυτοί αποτελούν ένα ρεπερτόριο τύπων που συχνά είναι υλοποιήσεις δομών δεδομένων. Αν οι προδιαγραφές τους συμφωνούν με τις απαιτήσεις των τύπων του προβλήματος προς υλοποίηση, διευκολύνουν τον προγραμματιστή στην ανάπτυξη του κώδικά του.
- Γλωσσικά εργαλεία για να ορίζονται νέοι τύποι από τον προγραμματιστή: κάθε νέος τύπος που ορίζεται έχει ως δομικά στοιχεία μεταβλητές που είναι είτε ενσωματωμένων τύπων, είτε τύπων βιβλιοθήκης, είτε άλλων τύπων που ήδη έχουν οριστεί από τον προγραμματιστή.

Κάθε μεταβλητή σε ένα πρόγραμμα C++ θα έχει κάποιον τύπο που θα εμπίπτει σε μια από τις παραπάνω κατηγορίες. Η ανάγκη χρήσης τύπων που εμπίπτουν στην τρίτη, αλλά και στη δεύτερη, περίπτωση θα είναι κυρίως το κριτήριο να επιλέξουμε τη C++ αντί για τη C στην υλοποίησή μας.

Από πλευράς ενεργειών που μπορούν να εφαρμοστούν στα δεδομένα, η C++ προσφέρει:

- Δομικά στοιχεία προκειμένου να διατυπώνονται εκφράσεις προς υπολογισμό.
- Ένα σύνολο εντολών.
- Τη δυνατότητα ορισμού “πακεταρίσματος” κώδικα με τη μορφή συναρτήσεων.
- Ένα σύνολο συναρτήσεων από την καθιερωμένη βιβλιοθήκη.

Εδώ, έχουμε να παρατηρήσουμε ότι τόσο η κατηγοριοποίηση, όσο και η ποικιλία είναι πανομοιότυπες με εκείνες της γλώσσας C.

Στη συνέχεια, τώρα, θα δούμε λίγο πιο αναλυτικά τα παραπάνω, αλλά η παρουσίαση θα παραμείνει συνοπτική και επιφανειακή. Ο αναγνώστης καλείται να αναφερθεί σε ένα οποιοδήποτε εγχειρίδιο της C++ για αναλυτικές περιγραφές και λεπτομέρειες. Παρ' όλα αυτά, σε κάποια σημεία θα επιμείνουμε. Ο λόγος γι' αυτό είναι είτε ότι αυτά αποτελούν κάποια “λεπτά σημεία” που είναι συνήθως πηγή λαθών, είτε ότι η ανάλυσή τους χρειάζεται για την πλήρη κατανόηση κάποιων χαρακτηριστικών που αφορούν τον ορισμό και τη χρήση κλάσεων, όπως ήδη έχουμε προαναγγείλει. Στα δεύτερα θα επανέλθουμε στα αντίστοιχα κεφάλαια, αλλά καλό είναι να έχουν γίνει κατανοητά νωρίτερα, ώστε τότε η έμφαση να δοθεί στα χαρακτηριστικά που αφορούν τις κλάσεις και μόνο.

3.2.2.1 Τύποι

Η γλώσσα C++ προσφέρει ένα ρεπερτόριο ενσωματωμένων βασικών τύπων, όπως:

- Για ακεραίους διαφόρων μεγεθών:
char, short int, int, long int
Ο τύπος char δηλώνει τύπο χαρακτήρων.
- Για πραγματικούς:
float, double, long double

- Για την ερμηνεία του πρώτου bit:
signed/unsigned char
signed/unsigned short int
signed/unsigned int
signed/unsigned long int
- Για τις λογικές μεταβλητές προσφέρεται ο τύπος bool με τιμές: true, false.

Θεωρώντας τους παραπάνω τύπους, η C++ προσφέρει ένα ρεπερτόριο παραγόμενων τύπων με χρήση τελεστών. Αυτοί οι τύποι δηλώνονται με τον εξής τρόπο.

- * δηλώνει δείκτη (prefix)
- & δηλώνει αναφορά (prefix)
- [] δηλώνει array (postfix)
- () δηλώνει συνάρτηση (postfix)
- Παραδείγματα:

```
int* pa;      \\ pa pointer to int
int a; int& ra = a; \\ ra reference to integer variable a
float v[10]; \\ v array of 10 elements of type float
char* p[20]; \\ p array of 20 elements of type pointer to char
void f(int); \\ f function of one argument of type int and no return value
```

Με βάση αυτούς, καθώς και τους τύπους της καθιερωμένης βιβλιοθήκης, στη C++ μπορούμε να ορίσουμε νέους τύπους, είτε με τη μορφή κλάσεων αντικειμένων που έχουν μια δική τους συμπεριφορά, είτε με τη μορφή δομήσεων δεδομένων, είτε με τη μορφή μετονομασίας, είτε απλά με τη μορφή απαρίθμησης.

Όπως και στη C, ο προσδιορισμός void συντακτικά συμπεριφέρεται ως τύπος για να δηλώσει ότι μια συνάρτηση δεν επιστρέφει τιμή, καθώς και ως βασικός τύπος για δείκτες σε αντικείμενα που δεν ξέρουμε και δεν μας ενδιαφέρει ο τύπος τους.

Σχετικά με τους δείκτες, έχουμε δυνατότητα αριθμητικής δεικτών, όπως και στη C. Με βάση αυτή, μπορούμε να χειριστούμε τους πίνακες με χρήση του ονόματός τους ως σταθερό δείκτη που δείχνει στο πρώτο στοιχείο τους και εφαρμόζοντας αριθμητική δεικτών σε αυτό.

Κλείνοντας τα θέματα σχετικά με τους τύπους, χωρίς να εστιάσουμε σε αυτούς από πλευράς αντικειμενοστραφούς προγραμματισμού, μπορούμε να πούμε τα εξής:

- Δίνεται η δυνατότητα μετατροπών τύπων αλλά κάτω από πιο αυστηρά πλαίσια από τη C. Ο προγραμματιστής μπορεί να προσδιορίσει ρητά αν επιθυμεί μετατροπή τύπου και ποια είναι αυτή.
- Δίνεται η δυνατότητα ορισμού νέων τύπων με χρήση της γλωσσικής δομής struct. Οι δυνατότητες που υπάρχουν είναι πολύ πιο εκτεταμένες από αυτές της δομής struct στη C. Όμως, τόσο για λόγους συμβατότητας με τη C, αλλά και για λόγους διάκρισης από τον ορισμό τύπων με αφαίρεση στα δεδομένα, προτείνεται να χρησιμοποιείται για τον ορισμό τύπων που είναι απλές δομές πληροφορίας (με στοιχειώδη λειτουργικότητα αρχικοποιήσεων κλπ.) και των οποίων τα δομικά στοιχεία είναι ορατά και χρησιμοποιήσιμα από όλο το πρόγραμμα. Για παράδειγμα,

```
struct address {
    char* street;
    int number;
    int postcode;
    char* city;
};
```

- Η χρήση της λέξης-κλειδιού `typedef` είναι περιορισμένη και γίνεται για μετονομασία τύπων. Για παράδειγμα,

```
typedef char* Pchar;
Pchar p1;
```

Προσοχή: Η χρήση του `typedef` στη C++ είναι κυρίως:

- για ονομασία παραγόμενων τύπων ώστε να γίνεται ο κώδικας πιο ευκρινής και
- σε περιπτώσεις που δεν έχουμε ακόμα αποφασίσει τον τύπο κάποιων μεταβλητών και ενδέχεται να τον αλλάξουμε αργότερα.
- Η χρήση της λέξης-κλειδιού `union` για ορισμό (εναλλακτικών) ενός τύπου υποστηρίζεται, αλλά είναι πολύ περιορισμένη, λόγω της υποστήριξης του προγραμματισμού γενικεύσεων που προσφέρει η C++ με χρήση περιγραμμάτων (templates).
- Προσφέρεται η δυνατότητα ορισμού απαριθμήσεων (enumerations) με χρήση της λέξης-κλειδιού `enum`, όπως και στη C.

3.2.2.2 Εκφράσεις και Εντολές

Όπως ήδη είπαμε, στη C++, όπως και στη C, μπορούμε να πραγματοποιήσουμε ενέργειες μέσω των εντολών της γλώσσας και να υπολογίσουμε την τιμή εκφράσεων που σχηματίζονται με τη χρήση τελεστών της γλώσσας.

Στο σημείο αυτό, θα κάνουμε μια επιγραμματική αναφορά στους τελεστές της C++. Η C++ είναι πλούσια σε τελεστές, όπως η C, π.χ. `+`, `-`, `*`, `/`, `%`, ... Μερικοί έχουν και νέα σημασία (υπερφόρτωση), π.χ. `<<`. Οι τελεστές

`+, -, *, /, %, &, |, ...`

συμπτύσσονται με τον τελεστή =

π.χ. `a = a + 5; → a += 5;`

Θυμηθείτε ότι η έκφραση `E1 op= E2` είναι ισοδύναμη με την έκφραση `E1 = E1 op (E2)` (με την έκφραση `E1` να υπολογίζεται μια φορά).

Γενικά οι τελεστές “ομαδοποιούνται” από αριστερά, εκτός από τους μοναδιαίους τελεστές και τους τελεστές ανάθεσης που ομαδοποιούνται από δεξιά, π.χ.

`a = b = c → a = ( b = c )`

`a + b + c → ( a + b ) + c`

`*p++ → *(p++)` ΟΧΙ `(*p)++` (Τι σημαίνει αυτό; Μαντέψτε!)

Να παρατηρήσουμε ότι η “ομαδοποίηση” αφορά το σε ποια οντότητα αναφέρεται ο τελεστής και όχι με ποια χρονική σειρά εφαρμόζεται. Δηλαδή, η έκφραση $*(p++)$ δηλώνει ότι η αύξηση αναφέρεται στον δείκτη — αφού πρώτα έχουμε διαβάσει την τιμή της θέσης μνήμης στην οποία δείχνει, αρχικά. Αντίστοιχα, η έκφραση $(*p)++$ δηλώνει ότι η αύξηση αναφέρεται στο αποτέλεσμα της έκφρασης που υπάρχει μέσα στις παρενθέσεις: δηλαδή, πρώτα διαβάζουμε την τιμή της θέσης μνήμης στην οποία δείχνει ο δείκτης και μετά την αυξάνουμε. Θα το κατανοήσετε καλύτερα αν πειραματιστείτε με το παράδειγμα που ακολουθεί παρακάτω.

Υπάρχει καλά ορισμένη προτεραιότητα τελεστών, π.χ.:

$$a + b * c \longrightarrow a + (b * c)$$

(ο τελεστής $*$ έχει μεγαλύτερη προτεραιότητα)

$$a + b - c \longrightarrow (a + b) - c$$

(ίδια προτεραιότητα αλλά αριστερή “ομαδοποίηση”)

Οι τελεστές εμβέλειας, $::$, έχουν τη μεγαλύτερη προτεραιότητα και το κόμμα, $,$, ως ένδειξη ακολουθίας, έχει τη μικρότερη.

Καλό είναι να χρησιμοποιούμε παρενθέσεις, (και), για να κάνουμε σαφή την “ομαδοποίηση” της αποτίμησης. Γενικά η σειρά αποτίμησης δεν είναι εγγυημένη. Οι τελεστές $,$, $\&\&$, $||$ εγγυώνται ότι το αριστερό μέλος τους θα αποτιμηθεί πρώτα, π.χ.

$$b = (a = 2, a + 1) \longrightarrow b = 3 \text{ (και όχι } b = 2)$$

Επιτυγχάνουμε αύξηση και μείωση με τους τελεστές $++$ και $--$, όπως στη C.

- Η έκφραση $++lvalue$ είναι ισοδύναμη με την έκφραση $lvalue += 1$, ισοδύναμα $lvalue = lvalue + 1$, (θεωρώντας ότι το $lvalue$ δεν έχει παρενέργειες).

$$y = ++x \longrightarrow y = (x += 1)$$

$$y = x++ \longrightarrow y = (t = x, x += 1, t)$$

- Κατά την αύξηση/μείωση των δεικτών που αντιστοιχούν σε πίνακες, το αποτέλεσμα αναφέρεται στα στοιχεία του πίνακα (και όχι, π.χ. στις διευθύνσεις). Για παράδειγμα,

```
int p[] = {1, 2, 3, 4};
int* q = p; cout << *q << endl;
q++; cout << *q << endl;
```

μετά την αύξηση, ο q δείχνει στο επόμενο στοιχείο του πίνακα (στον επόμενο ακέραιο) και όχι στην επόμενη θέση μνήμης.

Δείκτες και τελεστές: Εκτελέστε και μελετήστε το παρακάτω πρόγραμμα.

```
////////////////////////////////////
#include <iostream>
using namespace std;

int main()
{
    int p[] = {10, 20, 30, 40};
    int *q = p;
    int x;
```



```
////////////////////////////////////

x = *q;           // x equals to the value of the integer that
                  // q points to (x = 10)
cout << x << endl; // number 10 is printed

////////////////////////////////////

x = *q++;         // x equals to the value of the integer that
                  // q points to (x = 10),
                  // then q points to the next integer, i.e. p[1]
cout << x << endl; // number 10 is printed

////////////////////////////////////

x = *(q++);       // x equals to the value of the integer that
                  // q points to (x = 20),
                  // then q points to the next integer, i.e. p[2]
cout << x << endl; // number 20 is printed

////////////////////////////////////

x = (*q)++;       // x equals to the value of the integer
                  // that q points to (x = 30),
                  // then the value of p[2] is incremented by 1.
                  // No change for q
cout << x << endl; // number 30 is printed

////////////////////////////////////

x = *q;           // x equals to the value of the integer that
                  // q points to (x = 31)
cout << x << endl; // number 31 is printed

////////////////////////////////////

x = *++q;         // First increment q to point to the next integer,
                  // i.e. p[3], and then
                  // assign to x its value (x = 40)
cout << x << endl; // number 40 is printed

////////////////////////////////////

x = ++*q;         // First increment the value of the integer that
                  // q points to and
                  // then assign this value to x (x = 41)
cout << x << endl; // number 41 is printed
```

```
return 0;
}
```

```
////////////////////////////////////
```

Μερικές Παρατηρήσεις

- Δεν υπάρχουν εντολές ανάθεσης και εντολές κλήσης συνάρτησης: η ανάθεση και η κλήση συνάρτησης υπάγονται στην κατηγορία των εκφράσεων.
- Οι λογικοί τελεστές && και || δεν αποτιμούν το δεύτερο όρισμά τους εκτός αν είναι απαραίτητο.
- Όπου γίνονται έλεγχοι τιμών αλήθειας, είναι πιο ευανάγνωστο να γράφουμε ρητά τις λογικές εκφράσεις, με χρήση των λογικών τελεστών, παρά να υπονοείται ο έλεγχος με βάση τις μη μηδενικές τιμές (μην ακολουθούμε την προσέγγιση της C να βασιζόμαστε στη μηδενική ή μη μηδενική τιμή της έκφρασης).
- Προσοχή στις προτεραιότητες! Δείτε το παρακάτω πρόγραμμα (εμπνευσμένο από ένα αντίστοιχο πρόγραμμα του [1]).

```
#include <iostream>
using namespace std;

int main()
{
    int a, b;

    b = 1;
    cout << ((a = --b) ? b : (b = -99))
         << '\n' << " a is " << a << '\n'
         << " b is " << b << endl;

    b = 1;
    cout << (a = --b ? b : (b = -99))
         << '\n' << " a is " << a << '\n'
         << " b is " << b << endl;

    return 0;
}
```

Η εκτύπωση είναι:

```
-99
a is  0
b is -99
-99
a is -99
b is -99
```

Κατά την πρώτη εκτύπωση, οι παρενθέσεις δίνουν προτεραιότητα ώστε η ανάθεση να γίνει μέσα στη συνθήκη. Κατά τη δεύτερη εκτύπωση, η ανάθεση θεωρεί ως δεξί μέλος της το αποτέλεσμα του υπολογισμού της έκφρασης του τριαδικού τελεστή συνθήκης (ternary conditional operator), ?:.

Εντολές

Στη συνέχεια, απλά επαναλαμβάνουμε εντολές που είναι ήδη γνωστές από τη γλώσσα C, μια που ίδιες ακριβώς προσφέρονται και από τη C++. Κατ' αρχήν, να σημειώσουμε ότι, όπως και στη γλώσσα C, η είσοδος και η έξοδος πραγματοποιούνται από συναρτήσεις βιβλιοθήκης. Παρότι μπορεί κανείς να κάνει είσοδο/έξοδο με τις ίδιες συναρτήσεις με εκείνες της C, η C++ προσφέρει και δικό της σύστημα εισόδου/εξόδου, επεκτάσιμο και απόλυτα εναρμονισμένο με το αντικειμενοστραφές μοντέλο. Στο σύστημα αυτό θα αναφερθούμε στο Κεφάλαιο 8.

Η γλώσσα, αυτή καθ' εαυτή, προσφέρει τις παρακάτω κατηγορίες εντολών:

- Εντολές Επιλογής
- Εντολές Επανάληψης
- Εντολές Μεταφοράς Ελέγχου

καθώς και τη δυνατότητα ονομασίας μιας εντολής.

Ας δούμε μία-μία τις κατηγορίες.

Εντολές Επιλογής:

Η γενική μορφή τους είναι

- `if ( expression ) statement`
- `if ( expression ) statement else statement`
- `switch ( expression ) statement`

Στην κατηγορία αυτή, ελέγχεται η τιμή που έχει το *expression* για να μεταφερθεί, ανάλογα, ο έλεγχος της εκτέλεσης του προγράμματος.

Εντολές Επανάληψης:

Η γενική μορφή τους είναι

- `while ( expression ) statement`
- `do statement while ( expression );`
- `for ( init-statement ; expression ; expression ) statement`

Η κατηγορία αυτή εκφράζει την επανάληψη του *statement*.

Εντολές Μεταφοράς Ελέγχου:

Η γενική μορφή τους είναι

- `case constant-expression : statement`
- `default: statement`
- `break;`
- `continue;`
- `return expression ;`
- `goto identifier ;`

Η κατηγορία αυτή εκφράζει μεταφορά ελέγχου. Εκτός από την περίπτωση χρήσης `case`, `default` και `break` σε εντολή `switch`, καθώς και του `return` για την επιστροφή από συνάρτηση, κάθε άλλη ρητή μεταφορά ελέγχου καλό είναι να αποφεύγεται.

Ονομασία Εντολής: Η εντολή *identifier : statement* δίνει το όνομα *identifier* στο *statement*

Ας δούμε λίγες παραπάνω λεπτομέρειες για κάποιες από τις πιο συνηθισμένες εντολές. Ξεκινάμε με τις **εντολές επιλογής**.

- Η επιλογή *if-then* έχει τη συνήθη μορφή:

```
if (expression)
    statement
```

- Η επιλογή *if-then-else* έχει τη συνήθη μορφή:

```
if (expression)
    statement
else
    statement
```

- Η τιμή της έκφρασης *expression* μπορεί να είναι `true` ή `false`.
- Το *statement* μπορεί να αντιστοιχεί σε μια μόνο εντολή ή σε ένα block εντολών.

Παράδειγμα:

```
#include <iostream>
#include <string>      // header file for C++ string manipulation
using namespace std;

int main()
{
    string answer;
    cin >> answer;

    if (answer == "yes")
        cout << "Celebration!" << endl;
    else
        cout << "Tell me: why?" << endl;
}
```

- Η πολλαπλή επιλογή, *switch*, έχει τη συνήθη μορφή:

```
switch (selector)
{
    case int1 : statement; break;
    case int2 : statement; break;
    ....
    default : statement;
}
```

- Η έκφραση *selector* παράγει μια τιμή ακεραίου.
- Ανάλογα με την παραγόμενη τιμή, ο έλεγχος πηγαίνει στο *statement* που ακολουθεί το αντίστοιχο *case*.

- Το `break` χρειάζεται για βγει ο έλεγχος από το `switch`.
- Το `default` υποδεικνύει τι πρέπει να εκτελεστεί στη γενική περίπτωση, όπου κανένα από τα άλλα `case` δεν μπορεί να εφαρμοστεί.

Παράδειγμα:

```
#include <iostream>
using namespace std;

int main()
{
    cout << "What you feel like watching tonight? " << endl;
    cout << "Expected answers are: drama (1),"
         << " comedy (2), adventure (3)" << endl;

    int answer;
    cin >> answer;

    switch (answer) {
        case 1:
            cout << "Suggestion: The Godfather" << endl; break;
        case 2:
            cout << "Suggestion: Home Alone" << endl; break;
        case 3:
            cout << "Suggestion: Indiana Jones and the Raiders"
                 << " of the Lost Ark" << endl; break;
        default:
            cout << "No proper genre!" << endl;
    }
}
```

Ας δούμε, τώρα, λίγο λεπτομερέστερα τις εντολές επανάληψης.

- Η επανάληψη *while* έχει τη συνήθη μορφή:

```
while (expression)
    statement
```

- Η έκφραση *expression* αποτιμάται πριν από κάθε επανάληψη του *statement*.
- Η τιμή της έκφρασης *expression* μπορεί να είναι `true` ή `false`.
- Το *statement* μπορεί να αντιστοιχεί σε μια μόνο εντολή ή σε ένα block εντολών.

Παράδειγμα:

```
#include <iostream>
using namespace std;

int main()
{
```

```

int n;
cout << "Give me a non-zero integer! " << endl;
cin >> n;

int N;
cout << "Give me an upper bound! " << endl;
cin >> N;

int count = 0;
cout << "Starting doubling! " << endl;

while (n < N) {
    cout << n << " " << endl;
    n *= 2; count++;
}

cout << "Finished in " << count << " steps." << endl;
}

```

- Η επανάληψη *for* έχει τη συνήθη μορφή:

```

for (initialization; conditional; statementL)
    statementB

```

- Όταν μπαίνει ο έλεγχος στο *for* για πρώτη φορά, εκτελείται το *initialization* άπαξ.
- Πριν από κάθε επανάληψη, ελέγχεται αν ισχύει το *conditional*.
- Αν είναι το *conditional* αληθές, εκτελείται το *statementB*.
- Στο τέλος κάθε επανάληψης (άμα έχει ολοκληρωθεί η εκτέλεση του *statementB*) εκτελείται το *statementL*.

Παράδειγμα:

```

#include <iostream>
using namespace std;

int main()
{
    cout << "Pos tha ithela na ixa " << endl;
    int si = 1;
    cout << " " << si++ << endl;

    for (int i = si; i < 5; i++) {
        cout << " ke " << i << endl;
    }
    cout << " pedia! " << endl;
}

```

- Η επανάληψη *do-while* έχει τη συνήθη μορφή:

```
do
    statement
while (expression)
```

- Το *statement* εκτελείται τουλάχιστον μια φορά, όταν ο έλεγχος μπει στο *do* για πρώτη φορά.
- Η έκφραση *expression* αποτιμάται μετά την εκτέλεση του *statement*.
- Μια πιθανή χρήση του είναι στην περίπτωση που η έκφραση *expression* εμπλέκει τιμές που διαβάζονται/υπολογίζονται στο *statement*.

Παράδειγμα:

```
#include <iostream>
using namespace std;

int main()
{
    int i;
    do {
        cout << "Enter an integer value: ";
        cin >> i;
        cout << " when doubled is " << 2 * i << endl;
    } while (i != 0);

    cout << " Enough for now! " << endl;
}
```

Ένα απλό παράδειγμα διαδικαστικού προγραμματισμού με C++: διαγραφή χαρακτήρα από μια ακολουθία χαρακτήρων

- Με επιστροφή του αποτελέσματος στην αρχική ακολουθία

```
#include <iostream>
using namespace std;

void remove(char*, char);

int main()
{
    char string1[] = "abagaadagaga";
    char string2[] = "abagaadagaga";
    char string3[] = "ccabbbb";

    remove(string1, 'a');
    cout << string1 << endl;

    remove(string2, 'g');
```

```

    cout << string2 << endl;

    remove(string3, 'a');
    cout << string3 << endl;

    return 0;
}

void remove(char* str, char ch)
{
    int i = 0, j = 0;
    while (str[j] != '\0') {
        while (str[j] != ch && str[j] != '\0')
            str[i++] = str[j++];
        while (str[j] == ch)
            j++;
    }
    str[i] = '\0';
}

```

- Με δημιουργία νέας ακολουθίας

```

#include <iostream>
using namespace std;

int length(const char* str)
{
    int length = 0;

    for (int i = 0; str[i] != '\0'; i++)
        length = i;

    return length;
}

char* remove(const char*, char);

int main()
{
    char strg[] = "abagaadagaga";
    char* string1 = strg;

    cout << remove(string1, 'a') << endl;
    cout << remove(string1, 'g') << endl;

    return 0;
}

```



```

char* remove(const char* str, char ch)
{
    int l = length(str);

    char* res = new char[l+1];

    int j = 0;
    for (int i = 0; i <= l; i++)
        if (str[i] != ch) {
            res[j] = str[i];
            j++;
        }
    res[j] = '\\0';
    return res;
}

```

- Προβληματική υλοποίηση: εσφαλμένη αναφορά σε μνήμη

```

int main()
{
    char strg[] = "abagaadagaga";
    char* string1 = strg;

    cout << remove(string1, 'a') << endl;
    cout << remove(string1, 'g') << endl;

    return 0;
}

char* remove(const char* str, char ch)
{
    int l = length(str);

    char res[l+1];

    int j = 0;
    for (int i = 0; i <= l; i++)
        if (str[i] != ch) {
            res[j] = str[i];
            j++;
        }

    res[j] = '\\0';
    return res; // error: when returning from remove, res is out of
                // scope and its space is "discarded" from the stack
}

```

3.3 Στοιχεία της C++ με Ιδιαίτερο Ενδιαφέρον για τις Κλάσεις

3.3.1 Σταθερές

Στη C++ έχουμε τη δυνατότητα να προσημαίνουμε με τη λέξη-κλειδί `const` έναν προσδιοριστή (identifier). Με αυτόν τον τρόπο, δηλώνουμε ότι αυτό που ονοματίζει δεν πρόκειται να μεταβληθεί, πρόκειται, δηλαδή, για σταθερά. Οι σταθερές του κώδικά μας αποτελούν μέρος των δεδομένων που χειρίζεται το πρόγραμμά μας. Κατά συνέπεια, με χρήση του `const`, αν θέλουμε να ορίσουμε μια σταθερά στο πρόγραμμά μας, δεν υπάρχει λόγος να καταφύγουμε σε χρήση της οδηγίας του προεπεξεργαστή `#define`. Απαλλασσόμαστε, δηλαδή, από την εκμετάλλευσή του για να διαχειριστούμε ιδιότητες του προγράμματος, κάτι που δεν είναι της αρμοδιότητάς του.

Στην περίπτωση που ο προσδιορισμός `const` αναφέρεται σε όρισμα συνάρτησης, η τιμή του ορίσματος απαγορεύεται να μεταβληθεί στο σώμα της συνάρτησης. Αν, παρ' ελπίδα, αυτό συμβεί, στη φάση της μεταγλώττισης παράγεται σφάλμα. Η δυνατότητα να χρησιμοποιήσουμε τον προσδιορισμό `const` σε όρισμα συνάρτησης προσφέρει τα εξής:

- Συμβάλλει στη διασφάλιση της ορθότητας του κώδικα της συνάρτησης. Για να προσδιορίσουμε ένα όρισμα ως `const`, προφανώς δεν θέλουμε να αλλάξει τιμή στο σώμα της συνάρτησης. Αν, λοιπόν, παράγεται σχετικό σφάλμα κατά τη μεταγλώττιση, κάπου στο σώμα της συνάρτησης την αλλάζουμε, άρα κάναμε κάποια ενέργεια που δεν είναι η επιθυμητή.
- Αν ο τύπος ενός ορίσματος δεν είναι κάποιος από τους βασικούς, η συμπεριφορά του περάσματος παραμέτρων κατά τιμή (by value) δεν είναι η επιθυμητή ακόμα και για την περίπτωση που η συνάρτηση απλά “διαβάζει” και χρησιμοποιεί την τιμή του ορίσματος. Θα δούμε στη συνέχεια το γιατί. Θα πρέπει, λοιπόν, το πέρασμα παραμέτρων κατά τιμή να αποφεύγεται και, στη γενική περίπτωση, το πέρασμα παραμέτρων να γίνεται κατά αναφορά (by reference) — το πώς γίνεται το πέρασμα παραμέτρων στη C++ κατά αναφορά, θα το δούμε σε επόμενη παράγραφο. Στην περίπτωση αυτή, ο τρόπος να εκφράσουμε ότι ένα όρισμα πρόκειται να χρησιμοποιηθεί για ανάγνωση τιμής και η τιμή του δεν θα αλλάξει είναι να το προσδιορίσουμε ως `const`.
- Κάνει σαφή τη διεπαφή της συνάρτησης.

Για να δούμε, όμως, λίγο πιο αναλυτικά τι ακριβώς συμβαίνει στην περίπτωση που δηλώνουμε έναν προσδιοριστή ως `const`.

Η λέξη-κλειδί `const` μπορεί να συμπεριληφθεί σε μια δήλωση προσδιοριστή για να τον μετατρέψει σε σταθερά αντί για μεταβλητή. Π.χ.

```
const int a = 5;
```

Το αποτέλεσμα αυτού του ορισμού φαίνεται στο Σχήμα 3.3. Με σκιασμένο περιεχόμενο στις εικόνες θα εκφράζουμε τη μη δυνατότητα αλλαγής τιμής, άρα σταθερή τιμή.

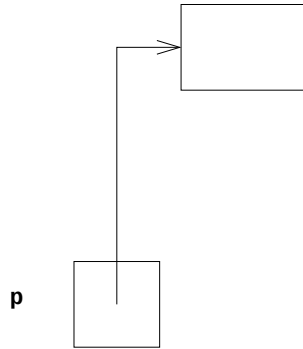


Σχήμα 3.3: Το αποτέλεσμα του ορισμού της σταθεράς `a`

Το περιεχόμενο του `a` δεν μπορεί να αλλάξει. Ακριβώς επειδή το περιεχόμενο δεν μπορεί να αλλάξει, ένας προσδιοριστής που δηλώνεται με `const` πρέπει να αρχικοποιείται. Ενδεχόμενα, ο μεταγλωττιστής μπορεί να εκμεταλλευτεί μια δήλωση `const` με διάφορους τρόπους, ανάλογα με το πόσο “έξυπνος” είναι.

Σταθερές και Δείκτες Μια περίπτωση που πρέπει να εξετάσουμε πιο αναλυτικά στη χρήση της λέξης-κλειδιού `const` είναι η περίπτωση που έχουμε μεταβλητές τύπου δείκτη.

Ένας δείκτης `p` που δείχνει κάπου στη μνήμη, σχηματικά φαίνεται στο Σχήμα 3.4.



Σχήμα 3.4: Η γενική εικόνα ενός δείκτη

Από τη μία έχουμε τη μεταβλητή τύπου δείκτη, αυτή καθαυτή, αλλά από την άλλη έχουμε και τη μεταβλητή στην οποία “δείχνει” ο δείκτης. Οπότε, έχουμε τις εξής περιπτώσεις:

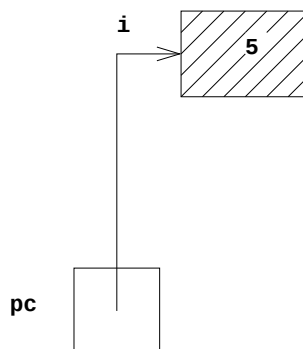
- Δήλωση σταθεράς για το αντικείμενο στο οποίο δείχνει ο δείκτης: η μεταβλητή στην οποία “δείχνει” ο δείκτης δεν μπορεί να αλλάξει τιμή. Ο δείκτης, ο ίδιος, μπορεί να αλλάξει τιμή και να “δείχνει” κάπου αλλού.
- Δήλωση σταθεράς για τον δείκτη: ο δείκτης “δείχνει” πάντα στην ίδια μεταβλητή. Η μεταβλητή αυτή μπορεί να αλλάζει τιμές όμως — οι δείκτες που αντιστοιχούν σε πίνακες ανήκουν σε αυτήν την περίπτωση.
- Δήλωση σταθεράς και για τα δύο: ο δείκτης δείχνει πάντα στον ίδιο χώρο και αυτός έχει τιμή σταθερά.

Ας δούμε, λοιπόν, τι γίνεται σε κάθε μία από τις παραπάνω περιπτώσεις και πώς δηλώνουμε την κάθε περίπτωση. Έστω ότι έχουμε τον ορισμό:

```
int i = 5;
```

- Δήλωση σταθεράς για το αντικείμενο στο οποίο δείχνει ο δείκτης: ο προσδιορισμός `const` προηγείται του τύπου του δείκτη. Το αποτέλεσμα, σχηματικά, φαίνεται στο Σχήμα 3.5.

```
const int* pc = &i;
```



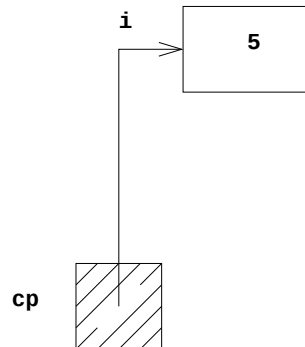
Σχήμα 3.5: Δήλωση σταθεράς για το αντικείμενο στο οποίο δείχνει ο δείκτης

Τότε,

```
*pc = 7; // error
```

- Δήλωση σταθεράς για τον δείκτη: ο προσδιορισμός `const` έπεται του τύπου του δείκτη (προηγείται άμεσα του ονόματός του). Το αποτέλεσμα, σχηματικά, φαίνεται στο Σχήμα 3.6.

```
int *const cp = &i;
```



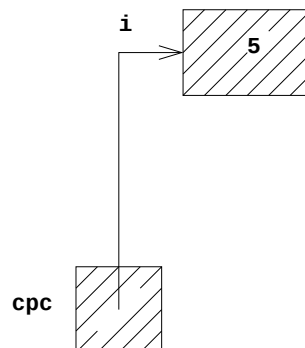
Σχήμα 3.6: Δήλωση σταθεράς για τον δείκτη

Τότε,

```
int j = 10; cp = &j; // error
```

- Δήλωση σταθεράς και για τα δύο. Το αποτέλεσμα, σχηματικά, φαίνεται στο Σχήμα 3.7

```
const int *const cpc = &i;
```



Σχήμα 3.7: Δήλωση σταθεράς και για τον δείκτη και για το αντικείμενο στο οποίο δείχνει

Τότε,

```
*cpc = 7; // error
cpc = &j; // error
```

Για να πειραματιστείτε με τις σταθερές και τους δείκτες, εκτελέστε το παρακάτω πρόγραμμα:

```
#include <iostream>
using namespace std;

int main()
{
    const int a = 10;
    // a = 1000;    // error
    // a++;          // error

    const int arr[] = {1, 2, 3};
    // arr[1] = 2000;    // error

    //////////////////////////////////////

    char abc[] = "abcdef";

    const char* pc = abc;           // pointer to constant
    // pc[3] = 'a';                // error
    abc[3] = 'a';
    pc = "efghijkl";               // ok

    char *const cp = abc;           // constant pointer
    cp[3] = 'a';                   // ok
    // cp = "efgh";                // error

    const char *const cpc = abc;    // constant pointer to constant
    // cpc[3] = 'a';                // error
    // cpc = "efgh";                // error

    return 0;
}
```

Ένα άλλο παράδειγμα με συμβολοσειρές:

```
#include <iostream>
using namespace std;

int main()
{
    char a[] = "string";
    cout << a << endl;
    a[0] = 'd';
    cout << a << endl;

    const char* sa = "literal string";
    cout << sa << endl;
    sa++;
    cout << sa << endl;
}
```

```
// sa[0] = 'd';    // error

return 0;
}
```

Κάποιες παρατηρήσεις για τη χρήση του `const` σε συνδυασμό με δείκτες:

- Ένα αντικείμενο το οποίο είναι σταθερά, όταν προσπελαύνεται μέσω ενός δείκτη, μπορεί να είναι μεταβλητή όταν προσπελαύνεται διαφορετικά. Για παράδειγμα, έστω ότι έχουμε ως όρισμα συνάρτησης δείκτη σε σταθερά. Έστω, όμως, ότι κατά την κλήση περνάμε ως παράμετρο μη σταθερά. Τότε, το σώμα της συνάρτησης δεν μεταβάλλει το αντικείμενο, αλλά το περιβάλλον από το οποίο κλήθηκε η συνάρτηση και στο οποίο θα επιστρέψει ο έλεγχος, όμως, μπορεί.
- Είναι χρήσιμο ως προσδιορισμός για ορίσματα συναρτήσεων. Για παράδειγμα, δηλώνοντας ένα όρισμα που είναι τύπου δείκτη σαν `const`, η συνάρτηση απαγορεύεται να μεταβάλλει το αντικείμενο στο οποίο δείχνει, π.χ.,

```
char* strcpy(char* p, const char* q)
```

η συνάρτηση δεν μπορεί να μεταβάλλει το αντικείμενο στο οποίο δείχνει το `q`. Αυτό, όπως είπαμε, συμβάλλει στη διασφάλιση της ορθότητας του κώδικα και στη σαφήνεια της διεπαφής της συνάρτησης.

Μια τελευταία παρατήρηση: ορισμοί `const` προσδιοριστών ισχύουν μόνο μέσα στο αρχείο που εμφανίζονται, δηλαδή, έχουν εσωτερική συνδεσιμότητα (*internal linkage*). Έτσι, πρέπει να τους συμπεριλαμβάνουμε σε κάθε αρχείο που χρησιμοποιεί τις αντίστοιχες σταθερές. Αυτό, όμως, σημαίνει και ότι μπορούμε να τους γράφουμε σε αρχεία επικεφαλίδας χωρίς φόβο ότι θα προκύψει σφάλμα εξαιτίας πολλαπλών ορισμών, για την περίπτωση που το αρχείο επικεφαλίδας συμπεριληφθεί σε πάνω από ένα αρχεία πηγαίου κώδικα.

3.3.2 Συναρτήσεις

Όπως και στην περίπτωση της γλώσσας C, οι συναρτήσεις είναι ο τρόπος για να κάνει το πρόγραμμά μας κάποια ολοκληρωμένη διαδικασία. Ορίζουμε συναρτήσεις η κάθε μία από τις οποίες υλοποιεί μέρος της επίλυσης του προβλήματός μας. Ο ρόλος τους είναι να παρέχουν μια “ομαδοποίηση” κώδικα, ο οποίος μπορεί να εκτελείται όποτε χρειάζεται. Αυτό πραγματοποιείται με την κλήση της συνάρτησης.

Μια συνάρτηση δεν μπορεί να κληθεί, αν δεν έχει δηλωθεί προηγουμένως. Μια δήλωση συνάρτησης περιλαμβάνει:

- το όνομα της συνάρτησης,
- τον τύπο της τιμής που επιστρέφει και
- το πλήθος και τους τύπους των ορισμάτων της.

Π.χ.

```
int sum(int i, int j);
void swap(int*, int*);
```

Όπως ισχύει και για κάθε άλλο ορισμό οντότητας, μια συνάρτηση πρέπει να ορίζεται μόνο μια φορά. Εδώ να παρατηρήσουμε κάτι που ακόμα δεν μπορεί να γίνει πλήρως κατανοητό, μια που προϋποθέτει να έχουμε συζητήσει για την έννοια της εμβέλειας. Ένας ορισμός επιβάλλεται να είναι μοναδικός στα πλαίσια μιας εμβέλειας. Σε διαφορετικές εμβέλειες, μπορεί να υπάρχουν διαφορετικοί ορισμοί, χωρίς πρόβλημα: αναφέρονται σε διαφορετικές οντότητες.

Ας πάμε τώρα σε ένα λεπτό σημείο που θα γίνει καλύτερα κατανοητό όταν θα έχουμε μιλήσει για κλάσεις και συναρτήσεις αρχικοποίησης. Κατά την κλήση μιας συνάρτησης, η αντιμετώπιση είναι ίδια με την *αρχικοποίηση* των ορισμάτων της (και όχι με την ανάθεση τιμών). Δηλαδή, όταν καλείται μια συνάρτηση, οι χώροι που έχουν φυλαχθεί για τις τυπικές παραμέτρους της *αρχικοποιούνται* χρησιμοποιώντας τα αντίστοιχα ορίσματα.

Το ίδιο ισχύει και για την επιστροφή τιμής από συνάρτηση: ακολουθείται η ίδια διαδικασία με την αρχικοποίηση μεταβλητών. Δηλαδή, μια εντολή `return` λειτουργεί σαν να *αρχικοποιεί* μια μεταβλητή του τύπου που επιστρέφει η συνάρτηση, χρησιμοποιώντας την τιμή της έκφρασης της `return`, πραγματοποιώντας τις ανάλογες μετατροπές τύπων. Παραδείγματα θα δούμε σε επόμενα κεφάλαια.

3.3.2.1 Συναρτήσεις `inline`

Στη C++, μια συνάρτηση μπορεί να δηλωθεί ως `inline`, δίνοντας μια συμβουλή στον μεταγλωττιστή να αντικαταστήσει με τον κώδικα της συνάρτησης τις κλήσεις της συνάρτησης, αντί να χρησιμοποιήσει τον κλασικό μηχανισμό κλήσης συναρτήσεων και επιστροφής αποτελεσμάτων. Για παράδειγμα,

```
inline int max(int i, int j) { return (i > j ? i : j); }
```

τότε, η κλήση, π.χ. `max(5, 10)`, ανάλογα με την “έξυπνάδα” του μεταγλωττιστή, μπορεί να αντιμετωπιστεί σαν να ήταν η έκφραση `(5 > 10 ? 5 : 10)` ή `10` ή μπορεί και να μείνει ανέπαφη, όμως.

Δηλαδή, αν μια συνάρτηση έχει προσδιοριστεί ως `inline`, αντί ο κώδικας που θα παραχθεί για μια κλήση της κατά τη φάση της μεταγλώττισης να είναι ο κώδικας που αντιστοιχεί σε κλήση συνάρτησης, ελέγχεται μήπως μπορεί να παραχθεί κώδικας που αντιστοιχεί στον κώδικα του σώματός της ή ακόμα και να γίνει κάποιος επιμέρους υπολογισμός. Όμως, δεν είναι σίγουρο ότι κάθε `inline` συνάρτηση τελικά θα αντιμετωπιστεί “`inline`” και θα γίνει μια τέτοια αντικατάσταση. Παραδείγματα γι’ αυτό είναι περίπλοκες, αμοιβαία αναδρομικές συναρτήσεις, συναρτήσεις που καλούν τους εαυτούς τους ανάλογα με την είσοδό τους κλπ. Συνήθως, επιλέγουμε να προσδιορίσουμε ως `inline` απλές συναρτήσεις. Συχνά, τέτοιες συναρτήσεις στη C θα τις υλοποιούσαμε μέσω μακρο-ορισμών, με `#define`. Ενόσω σε αυτήν την περίπτωση μπορεί να είχαμε προβλήματα λόγω μη επιθυμητής συμπεριφοράς κατά τη χρήση τους, η υπόδειξη `inline` δεν παραβιάζει την κλασική σημασία της κλήσης συνάρτησης. Δεν έχουμε, δηλαδή, ανεπιθύμητες παρενέργειες. Αυτό θα γίνει απόλυτα κατανοητό με το παρακάτω παράδειγμα.

Παράδειγμα για παρενέργειες της `#define` - Ασφαλής χρήση με `inline` συναρτήσεις:

```
#include <iostream>
using namespace std;

#define MAX1(i, j) (((i) > (j)) ? (i) : (j))

inline int max2(int i, int j)
{ return (i > j ? i : j); }

int main()
{
    int m1, m2, i = 5, j = 10;

    m2 = max2(i++, j++);
    cout << "m2 = " << m2 << endl;
    cout << "i = " << i << endl;
    cout << "j = " << j << endl;
```

```

m1 = MAX1(i++, j++);
cout << "m1 = " << m1 << endl;
cout << "i = " << i << endl;
cout << "j = " << j << endl;

return 0;
}

```

Ας δούμε τη συμπεριφορά κατά την εκτέλεση της συνάρτησης `main` του παραπάνω προγράμματος, αναλύοντας τα κύρια σημεία της.

`int m1, m2, i = 5, j = 10;`: Οι μεταβλητές παίρνουν τις εξής τιμές: `i = 5` και `j = 10` — οι τιμές των μεταβλητών `m1` και `m2` είναι απροσδιόριστες.

`m2 = max2(i++, j++);`: Καλείται η συνάρτηση `max2` για τις τιμές `i = 5` και `j = 10`. Οι τιμές των μεταβλητών `i` και `j` θα αυξηθούν αφότου έχουν διαβαστεί για χρήση κατά την κλήση. Η έκφραση στην επιστροφή της συνάρτησης πρακτικά είναι η $(5 > 10 ? 5 : 10)$. Ο υπολογισμός της οδηγεί στον έλεγχο της συνθήκης $5 > 10$, η οποία είναι ψευδής, οπότε το αποτέλεσμα του υπολογισμού της έκφρασης είναι το 10. Αυτό ανατίθεται στη μεταβλητή `m2`. Άρα, μετά την εκτέλεση αυτής της ανάθεσης, θα έχουμε `m2 = 10`, `i = 6` και `j = 11`. Θα ακολουθήσουν οι εκτυπώσεις των τιμών αυτών.

`m1 = MAX1(i++, j++);`: Η έκφραση αυτή, στο στάδιο της προεπεξεργασίας, έχει αντικατασταθεί από την έκφραση $((i++) > (j++)) ? (i++) : (j++)$ — δείτε το μόνοι σας σταματώντας τη μεταγλώττιση μετά το στάδιο της προεπεξεργασίας (με την παράμετρο `-E` του μεταγλωττιστή `g++`). Οπότε, γίνεται ο έλεγχος της συνθήκης $6 > 11$ αλλά αμέσως μετά οι τιμές των μεταβλητών `i` και `j` θα γίνουν `i = 7` και `j = 12`. Οπότε, η τιμή της επιστροφής της έκφρασης είναι το 12 και αυτή θα ανατεθεί στη μεταβλητή `m1`. Οπότε, έχουμε `m1 = 12`, ενόσω επιθυμούσαμε απλά να υπολογίσουμε και να αναθέσουμε τη μέγιστη από τις τιμές των μεταβλητών `i = 6` και `j = 11`. Δηλαδή, επιθυμούσαμε `m1 = 11`, κάτι που δεν προέκυψε. Τέλος, άπαξ και διαβαστεί η τιμή της μεταβλητής `j` που πρόκειται να επιστραφεί, θα αυξηθεί. Οπότε θα καταλήξει να είναι `j = 13`. Επομένως, καταλήγουμε με `m1 = 11`, `i = 7` και `j = 13` οι τιμές των οποίων και θα εκτυπωθούν.

Βλέπουμε, λοιπόν, ότι ενόσω η κλήση της `inline` συνάρτησης `max2` μας επέστρεψε το επιθυμητό αποτέλεσμα υπολογισμού της μέγιστης τιμής δύο ακεραίων μεταβλητών και μετά αύξηση των τιμών αυτών κατά μία μονάδα, η χρήση μακρο-ορισμού, `#define MAX1(i, j)`, προκάλεσε μη επιθυμητές παρενέργειες που οδήγησαν σε μη αναμενόμενα αποτελέσματα.

Γενικές Παρατηρήσεις: Κάποιες τελευταίες παρατηρήσεις τώρα πάνω στις συναρτήσεις που έχουν προσδιοριστεί ως `inline`.

- Οι ορισμοί συναρτήσεων `inline` πρέπει να υπάρχουν σε κάθε αρχείο (compilation unit) στο οποίο χρησιμοποιούνται. Ο λόγος είναι ότι έχουν εσωτερική συνδεσιμότητα. Αυτό, βέβαια είναι που μας επιτρέπει να βάζουμε τέτοιους ορισμούς σε αρχεία επικεφαλίδας. Θα επανέλθουμε στο τελευταίο μιλώντας για κλάσεις.
- Οι συναρτήσεις `inline` και οι δηλώσεις `const` περιορίζουν τη χρήση οδηγιών `#define` στη C++ σε θέματα που αφορούν την προετοιμασία των αρχείων για τη φάση της μεταγλώττισης. Επιπλέον, επιτρέπουν ιδιότητες που αφορούν το πρόγραμμα, όπως σταθερά δεδομένα, να εκφράζονται αποκλειστικά με στοιχεία που παρέχει η γλώσσα προγραμματισμού καθώς και τη διατήρηση του νοήματος του προγράμματος κατά τις κλήσεις συναρτήσεων.

3.3.2.2 Υπερφόρτωση Ονομάτων Συναρτήσεων

Συζητώντας για τις ιδιότητες του αντικειμενοστραφούς προγραμματισμού, μιλήσαμε για την υπερφόρτωση ονομάτων. Ήδη, από τα πρώτα παραδείγματα κώδικα στη γλώσσα C++, είδαμε υπερφόρτωση του τελεστή <<. Η C++ επιτρέπει υπερφόρτωση ονομάτων συναρτήσεων και τελεστών, ακόμα και σε περιπτώσεις ορισμών τους από τον προγραμματιστή.

Συγκεκριμένα, στην περίπτωση που ένα όνομα δείχνει το κατάλληλο να ονομάσει μια διαδικασία, μπορεί να χρησιμοποιηθεί ως όνομα της συνάρτησης που την υλοποιεί ακόμα κι αν το ίδιο όνομα έχει ήδη χρησιμοποιηθεί για να ονομάσει άλλη συνάρτηση. Αρκεί να μπορεί να γίνει διάκριση μεταξύ των συναρτήσεων από το πλήθος ή/και τον τύπο των ορισμάτων τους. Ο τύπος που επιστρέφει η συνάρτηση δεν παίζει ρόλο για τη διάκριση. Υπάρχουν κανόνες που υποδεικνύουν ποιο είναι το καλύτερο ταίριασμα τύπων κατά την κλήση μιας συνάρτησης προκειμένου να γίνει η επιλογή της συνάρτησης ακόμα και αν δεν υπάρχει ταύτιση στους τύπους των ορισμάτων.

Ας δούμε ένα παράδειγμα υπερφόρτωσης ονόματος συνάρτησης ορισμένης από τον χρήστη. Το όνομα `print` δείχνει επιθυμητό να ονοματίσει συναρτήσεις εκτύπωσης. Μπορούμε να έχουμε διάφορες συναρτήσεις εκτύπωσης, ανάλογα με το τι τύπου είναι το όρισμα που θέλουμε να εκτυπώσουμε. Δηλαδή,

```
void print(int);           // for int
void print(const char*);   // for strings
void print(double); ...
```

Στο παραπάνω παράδειγμα, ανάλογα με τον τύπο του ορίσματος κατά την κλήση της συνάρτησης `print`, θα χρησιμοποιηθεί ο κατάλληλος ορισμός.

Ας δούμε το παράδειγμα πιο ολοκληρωμένα.

```
#include <iostream>
#include <string>
using namespace std;

void print(int i)
{
    cout << "Printing an integer value: " << i << endl;
}

void print(double d)
{
    cout << "Printing a double: " << d << endl;
}

void print(string str)
{
    cout << "Printing a string: " << str << endl;
}

int main()
{
    int i = 10; double d = 3.14; string str = "This is a string";
    print(i); print(d); print(str);

    return 0;
}
```

Για το καθένα από τα `print(i)`, `print(d)` και `print(str)`, ανάλογα με τον τύπο του ορίσματος `i`, `d` και `str`, θα κληθεί η κατάλληλη συνάρτηση.

Η ιδιότητα της υπερφόρτωσης, πέρα από την προφανή χρησιμότητά της, θα μας φανεί ιδιαίτερα χρήσιμη αργότερα, όταν ορίζουμε συναρτήσεις αρχικοποίησης κατά τον ορισμό κλάσεων.

3.3.2.3 Default Τιμές Ορισμάτων

Μια άλλη δυνατότητα που προσφέρει η C++ που, πέρα από τη γενικότερη χρησιμότητά της, θα μας φανεί χρήσιμη αργότερα κυρίως, πάλι, στις συναρτήσεις αρχικοποίησης κλάσεων, είναι εκείνη του προσδιορισμού `default` τιμών στα ορίσματα συναρτήσεων. Ο προσδιορισμός `default` τιμών ορισμάτων είναι χρήσιμος στην περίπτωση που η γενική μορφή της κλήσης μιας συνάρτησης εμπλέκει περισσότερα ορίσματα από τη μορφή των συχότερων κλήσεών της. Δηλαδή, κατά την κλήση της, κάποια από τα ορίσματά της παίρνουν συνήθως συγκεκριμένες τιμές. Τότε οι τιμές αυτές, μπορούν να καθοριστούν ως `default` τιμές των ορισμάτων αυτών κατά τη δήλωση της συνάρτησης. Αν ένα όρισμα συνάρτησης έχει `default` τιμή, η τιμή του μπορεί να παραλειφθεί κατά την κλήση της συνάρτησης και να συμπληρωθεί αυτόματα με την `default` τιμή.

Για παράδειγμα, θα μπορούσαμε να έχουμε μια συνάρτηση εκτύπωσης ακεραίων που επιτρέπει να εμφανίζεται η τιμή τους σε συγκεκριμένο σύστημα αρίθμησης. Άρα, η συνάρτηση θα έχει δύο ορίσματα: ένα για τον ακεραίο που θέλουμε να εκτυπώσουμε και ένα για τη βάση του συστήματος. Συνήθως, όμως, θέλουμε εκτύπωση στο δεκαδικό σύστημα. Άρα, συνήθως, το αντίστοιχο όρισμα έχει την τιμή 10. Για να απαλλαγούμε από τον προσδιορισμό της τιμής αυτής κατά την κλήση, θα μπορούσαμε να υπερφορτώσουμε το όνομα `print` ορίζοντας τις συναρτήσεις:

```
void print(int value);
void print(int value, int base);
```

όπου η `void print(int)` να πραγματοποιεί την εκτύπωση με εμφάνιση της τιμής στο δεκαδικό σύστημα και η `void print(int value, int base)` να καλύπτει και όλες τις άλλες περιπτώσεις. Ο κώδικας του ορισμού και των δύο συναρτήσεων, όμως, είναι παρεμφερής και μπορεί να αποτελέσει έναν ενιαίο κώδικα. Σε τέτοιες περιπτώσεις, το να γραφεί μια και μόνο συνάρτηση που να ορίζεται από αυτόν είναι το πλέον καθαρό και ασφαλές, οπότε και επιθυμητό. Με τη χρήση `default` τιμών ορισμάτων, λοιπόν, μπορεί να οριστεί μία μόνο συνάρτηση που, όμως, έχει `default` τιμή 10 στο όρισμα που αντιστοιχεί στη βάση του συστήματος αρίθμησης. Δηλαδή, η δήλωσή της γίνεται:

```
void print(int value, int base = 10);
```

Σε αυτήν την περίπτωση, για την εμφάνιση της τιμής του ορίσματος του ακεραίου στο δεκαδικό σύστημα, η κλήση

```
print(25);
```

είναι ουσιαστικά ίδια με την κλήση

```
print(25, 10);
```

οπότε, απαλλαχθήκαμε από το να συμπεριλάβουμε την τιμή της βάσης, αφού πρόκειται για την `default` τιμή.

Κάποιες παρατηρήσεις στις `default` τιμές:

- Δεν επιτρέπονται `default` τιμές σε ενδιάμεσα ορίσματα. Π.χ.

```
OXI int g(int, int = 0, int);
ΟΥΤΕ int g(int = 0, int, int);
```

- Ο έλεγχος τύπων των ορισμάτων με `default` τιμές γίνεται στο επίπεδο της δήλωσης, ενώ η αποτίμηση γίνεται στην κλήση της συνάρτησης.

Να συνοψίσουμε εδώ μια άποψη για το εάν πρέπει να επιλέγουμε συνάρτηση με default τιμή(ές) στα ορίσματα ή υπερφορτωμένες συναρτήσεις. Εξαρτάται: αν ο κώδικας είναι παρεμφερής σε όλες τις περιπτώσεις ή μπορεί να συνενωθεί με φυσικό τρόπο, τότε κομψό είναι να οριστεί μία μόνο συνάρτηση. Αν ο κώδικας, όμως, είναι διαφορετικός, τότε προτείνεται υπερφόρτωση.

3.3.3 Οι Τελεστές new και delete

Η δέσμευση χώρου στη μνήμη για τις διάφορες μορφές μεταβλητών έχει ως εξής:

στατικές (static) μεταβλητές: Ο χώρος τους δεσμεύεται από τον ορισμό τους και διατηρείται καθ' όλη τη διάρκεια της εκτέλεσης του προγράμματος. Οι μεταβλητές αυτές φυλάσσονται στον στατικό (static) χώρο.

αυτόματες (automatic) μεταβλητές: Ο χώρος τους ανατίθεται κάθε φορά που ο έλεγχος της εκτέλεσης φτάνει στον ορισμό τους και διατηρείται έως ότου ο έλεγχος βγει από το block μέσα στο οποίο υπάρχει ο ορισμός. Οι μεταβλητές αυτές φυλάσσονται στη στοίβα εκτέλεσης (stack).

δυναμική δέσμευση: Κατά την εκτέλεση ενός προγράμματος, για να δημιουργήσουμε αντικείμενα ρητά τα οποία καταλαμβάνουν κάποιο χώρο έως ότου ρητά να τον αποδεσμεύσουμε, στη C++ χρησιμοποιούμε τους τελεστές new και delete. Χρησιμοποιούμε δυναμική δέσμευση σε περιπτώσεις που, για παράδειγμα, θέλουμε να δημιουργούμε “δομικά στοιχεία” μιας μεγαλύτερης δομής που δεν ξέρουμε το μέγεθός της κατά τη φάση της μεταγλώττισης, όπως κόμβοι δένδρων, στοιχεία λίστας κλπ. Τα αντικείμενα αυτά δημιουργούνται στον σωρό (heap). Δημιουργούμε τέτοια αντικείμενα με τον τελεστή new και τα καταστρέφουμε με τον τελεστή delete. Δεν υπάρχει περισυλλογή σκουπιδιών (garbage collection) για τέτοια αντικείμενα και έτσι πρέπει να προσέχουμε να αποδεσμεύουμε τον χώρο που έχουμε δεσμεύσει, πριν χάσουμε το μέσο πρόσβασης σε αυτόν, π.χ. μεταβλητές τύπου δείκτη στις οποίες έχει ανατεθεί η διεύθυνσή του ως τιμή.

Όσον αφορά τις στατικές και τις αυτόματες μεταβλητές, δεν υπάρχει κάποια διαφορά στη C++ σε σχέση με τις αντίστοιχες της γλώσσας C. Όσον αφορά τη δυναμική δέσμευση (και αποδέσμευση), όμως, η διαφορά είναι ουσιαστική. Πέρα από την πιο φιλική σύνταξη της δημιουργίας μιας μεταβλητής, με χρήση του τελεστή new, η δημιουργία συνοδεύεται αυτόματα από αρχικοποίηση. Το ίδιο ισχύει και για την αποδέσμευση με χρήση του τελεστή delete: πριν αποδεσμευτεί ο χώρος, καλείται αυτόματα η συνάρτηση καταστροφής του τύπου. Αυτά θα γίνουν κατανοητά όταν θα μιλήσουμε για τις κλάσεις, αλλά ας κάνουμε τώρα μια σύνοψη.

Ο τελεστής new επιστρέφει δείκτη σε αντικείμενο του τύπου που προσδιορίζουμε. Π.χ.

```
node* pn = new node;
```

Αν το node έχει οριστεί ως κλάση, κατά τη δέσμευση του χώρου στον σωρό, καλείται η συνάρτηση αρχικοποίησης της κλάσης.

Ο τελεστής delete πρέπει να εφαρμόζεται σε δείκτες που “δείχνουν” σε αντικείμενα που δημιουργήθηκαν από εφαρμογή του τελεστή new, για παράδειγμα, στον δείκτη που ορίστηκε στο παραπάνω.

```
delete pn;
```

Αν το node έχει οριστεί ως κλάση, καλείται η συνάρτηση καταστροφής της κλάσης.

Επίσης, μπορούμε να δημιουργήσουμε/καταστρέψουμε πίνακες από αντικείμενα χρησιμοποιώντας τους τελεστές new και delete. Για παράδειγμα,

```
char* s = new char[10];
delete[] s;
```

Αν τα στοιχεία του πίνακα είναι αντικείμενα κλάσης, για καθένα από αυτά, καλείται η default συνάρτηση αρχικοποίησης της κλάσης.

Για απλές μεταβλητές, δίνεται η δυνατότητα επιλογής της αρχικοποίησης κατά τη δυναμική δημιουργία. Για παράδειγμα,

```
node* pn = new node(10);
```

Προσοχή, όμως, στη σύνταξη.

- Αν έχουμε την εντολή `node* pn = new node(10);`, αυτό σημαίνει ότι η μεταβλητή `pn` είναι δείκτης σε μεταβλητή τύπου `node`, η οποία μεταβλητή αρχικοποιείται με χρήση της τιμής 10. Για να αποδεσμεύσουμε τον χώρο, αρκεί να κάνουμε: `delete pn;`
- Ενώ, αν έχουμε την εντολή `node* pna = new node[10];`, τότε έχουμε τη δημιουργία ενός πίνακα μεταβλητών τύπου `node`, αρχικοποιημένων by default, ο οποίος έχει μέγεθος 10. Για να αποδεσμεύσουμε τον χώρο, πρέπει να κάνουμε: `delete[] pna;`

3.3.4 Τύπος Αναφοράς

Στην παράγραφο αυτή, θα μιλήσουμε για τον τύπο αναφοράς (reference type) που προσφέρει η C++. Αυτός είναι ένας παραγόμενος τύπος που επιτρέπει να δηλώνουμε ένα “συνώνυμο” για μια μεταβλητή. Ο τύπος αυτός δεν προσφέρεται από τη C. Πέρα από τη γενικότερη χρησιμότητα, η χρήση του τύπου αναφοράς στα ορίσματα των συναρτήσεων που θέλουμε να περάσουμε κατά αναφορά κάνει το πέρασμα παραμέτρων σε μια συνάρτηση πολύ πιο “καθαρό”.

Ας δούμε, όμως, πρώτα πώς ορίζεται και πώς συμπεριφέρεται μια αναφορά. Η γενική μορφή του ορισμού της είναι:

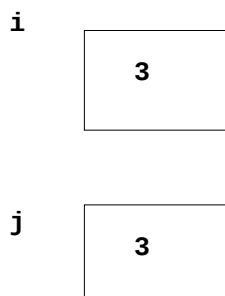
```
<type> &<identifier2> = <identifier1>
```

όπου η μεταβλητή `<identifier1>` πρέπει να έχει ήδη δηλωθεί νωρίτερα και να είναι τύπου `<type>`.

Έστω, λοιπόν, ότι έχουμε τον εξής κώδικα:

```
int i; i = 3; int j = i;
```

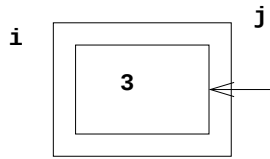
Αυτός οδηγεί στην (αφαιρετική) εικόνα στη μνήμη που φαίνεται στο Σχήμα 3.8.



Σχήμα 3.8: Οι δύο μεταβλητές `i` και `j`

Όμως, αν θέλαμε να ορίσουμε τη μεταβλητή `j` να είναι μια αναφορά στη μεταβλητή `i`, άρα ένα “συνώνυμο” του χώρου που καταλαμβάνει, ο κώδικας θα ήταν ο εξής:

```
int i; i = 3; int& j = i;
```



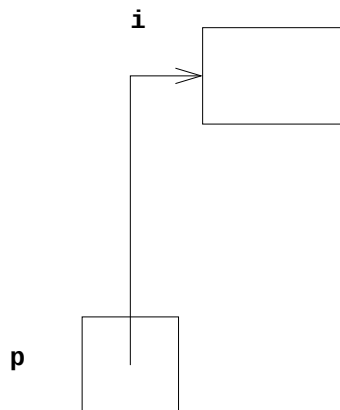
Σχήμα 3.9: Η j είναι αναφορά στην i

Η εικόνα στη μνήμη, τότε, θα ήταν όπως φαίνεται στο Σχήμα 3.9. Σε αυτήν την περίπτωση, το j είναι “συνώνυμο” του i.

Επί της ουσίας, οι αναφορές συμπεριφέρονται ως σταθεροί δείκτες προς τη μεταβλητή με την οποία αρχικοποιούνται (άρα πάντα πρέπει να αρχικοποιούνται) και δεν χρειάζονται dereferencing. Αυτό γίνεται αυτόματα με τη χρήση του ονόματός τους και μόνο. Εδώ να σημειώσουμε ότι, παρότι στον ορισμό μιας αναφοράς χρησιμοποιείται το σύμβολο &, ο τελεστής & εξακολουθεί να χρησιμοποιείται και για να εκφράσουμε τη διεύθυνση μιας μεταβλητής. Δηλαδή, ο κώδικας

```
int i; int* p; p = &i;
```

εξακολουθεί να έχει την ίδια ερμηνεία με εκείνη που έχει στη C και οδηγεί στην εικόνα στη μνήμη του Σχήματος 3.10.



Σχήμα 3.10: Η p είναι δείκτης στην i

3.3.4.1 Ο Τύπος Αναφοράς στα Ορίσματα Συνάρτησης

Έχοντας συζητήσει γενικά περί αναφορών, ας δούμε πώς μπορούμε να χρησιμοποιήσουμε τον τύπο αυτόν για το πέρασμα παραμέτρων σε συναρτήσεις που αλλάζουν τις “εξωτερικές” μεταβλητές των ορισμάτων. Δηλαδή, πέρασμα παραμέτρων το οποίο είναι επιθυμητό να γίνει κατά αναφορά.

Ας πάρουμε ένα παράδειγμα. Έστω ότι θέλουμε να ορίσουμε μια συνάρτηση που αντιμετωπίζει τις τιμές των ορισμάτων της. Η συνάρτηση αυτή αλλάζει τιμή στα ορίσματά της, οπότε χρειαζόμαστε το πέρασμα παραμέτρων της να γίνει κατά αναφορά. Στη γλώσσα C, τέτοια δυνατότητα δεν υπάρχει. Αυτό, λοιπόν, που κάνουμε αν θέλουμε να υλοποιήσουμε τη συνάρτηση στη C, είναι στα ορίσματά της αντί να έχουμε τις μεταβλητές αυτές καθ'εαυτές, έχουμε δείκτες του τύπου των μεταβλητών και κατά την κλήση της συνάρτησης, θα χρησιμοποιήσουμε τις διευθύνσεις των μεταβλητών που μας ενδιαφέρουν. Στο σώμα της συνάρτησης θα έχουμε πρόσβαση στον χώρο των μεταβλητών, έμμεσα, κάνοντας dereference στους δείκτες που θα έχουν δημιουργηθεί στη θέση των ορισμάτων. Με λίγα λόγια, η συνάρτηση της C θα έχει ορίσματα τύπου δείκτη, που περνούν κατά τιμή, αλλά μέσω τεχνάσματος επιτυγχάνουμε την πρόσβαση που θέλουμε και έτσι προσομοιώνουμε το πέρασμα κατά αναφορά. Αντίθετα, στη C++, μπορούμε να χρησιμοποιήσουμε στον τύπο των ορισμάτων αναφορές.

Έτσι, κατά την κλήση της συνάρτησης, θα δημιουργηθούν για τη συνάρτηση “συνώνυμα” των μεταβλητών της κλήσης. Οπότε, αν στο σώμα της χρησιμοποιούμε τα ονόματα των ορισμάτων, στην ουσία αναφερόμαστε στις μεταβλητές αυτές.

Ας δούμε τα παραπάνω πιο καθαρά. Ας ονομάσουμε swap τη συνάρτηση που πραγματοποιεί την αντιμετάθεση των τιμών των ορισμάτων της.

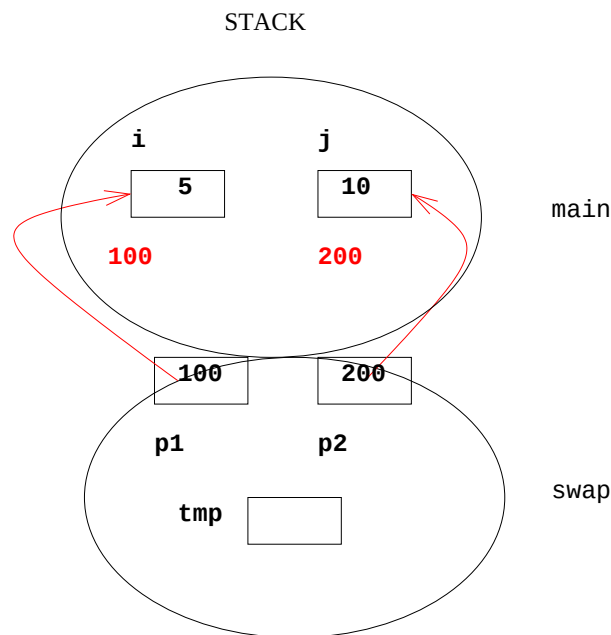
- Πέρασμα παραμέτρων à la C:

```
void swap(int* p1, int* p2)
{
    int tmp = *p1;
    *p1 = *p2;
    *p2 = tmp;
}
```

Κλήση:

```
int i, j; i = 5; j = 10;
swap(&i, &j);
```

Η εικόνα των μεταβλητών όπως (αφαιρετικά) είναι στη μνήμη φαίνεται στο Σχήμα 3.11.



Σχήμα 3.11: Πέρασμα παραμέτρων à la C

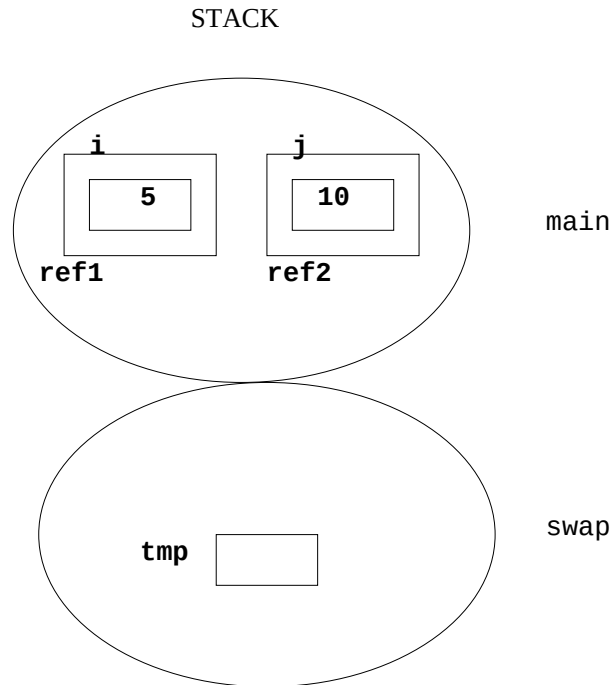
- Πέρασμα παραμέτρων à la C++

```
void swap(int& ref1, int& ref2)
{
    int tmp = ref1;
    ref1 = ref2;
    ref2 = tmp;
}
```

Κλήση:

```
int i, j; i = 5; j = 10;
swap(i, j);
```

Η εικόνα των μεταβλητών όπως (αφαιρετικά) είναι στη μνήμη φαίνεται στο Σχήμα 3.12.



Σχήμα 3.12: Πέρασμα παραμέτρων à la C++

Με τη χρήση του τύπου αναφοράς στα ορίσματα συναρτήσεων, επιτυγχάνουμε τα εξής:

- Κάνουμε πιο καθαρή τη διεπαφή της συνάρτησης: ο τύπος αναφοράς εκφράζει άμεσα ότι πρόκειται για πέρασμα παραμέτρων κατά αναφορά και όχι κατά τιμή.
- Λόγω της απαλλαγής της χρήσης τύπου δείκτη για το πέρασμα κατά αναφορά, η χρήση δείκτη στα ορίσματα συνάρτησης μπορεί πλέον να εκφράζει αποκλειστικά ότι είναι επιθυμητό το όρισμα να είναι τύπου δείκτη, π.χ. για όρισμα που να αποσκοπεί στο πέρασμα συνάρτησης.
- Ο κώδικας στο σώμα της συνάρτησης απαλλάσσεται από dereferences και έτσι γίνεται πιο ευανάγνωστος και ευκολότερα κατανοητός.

Άλλες παρατηρήσεις για τις αναφορές

- Μεγάλα αντικείμενα καλό είναι να τα περνάμε μέσω αναφοράς, για λόγους αποδοτικότητας.
- Ακόμα και αν δεν έχουμε πρόθεση να αλλάξουμε την τιμή τους, το παραπάνω ισχύει. Σε αυτήν την περίπτωση, όμως, καλό είναι να προσδιορίζουμε το όρισμα ως `const`. Π.χ.

```
void f(const tree& t)
{ ... }
```

οπότε, στο σώμα της `f` απαγορεύεται να αλλάξει η τιμή του `t`.

- Οι πίνακες περνούν αυτόματα μέσω αναφορών.

Γενικότερα, δεδομένου ότι η κλήση συνάρτησης οδηγεί σε αρχικοποίηση των ορισμάτων της με χρήση των μεταβλητών για τις οποίες θα κληθεί, το πέρασμα παραμέτρων κατά τιμή:

- οδηγεί σε δημιουργία νέων μεταβλητών,
- οδηγεί σε αρχικοποίηση αυτών των μεταβλητών,
- κατά την καταστροφή τους, μετά τη χρήση τους, ενδέχεται αυτόματα να γίνουν επιπλέον ενέργειες.

Τα παραπάνω σπάνια είναι επιθυμητά για την περίπτωση ορισμάτων τύπου κλάσεων. Στη γενική περίπτωση περάσματος παραμέτρων τέτοιου τύπου, λοιπόν, η πρόταση είναι τα ορίσματα να δηλώνονται ως τύπου αναφοράς. Αν η συνάρτηση δεν πρόκειται να αλλάξει την τιμή τους — οπότε θα τα περνάγαμε κατά τιμή —, απλά να προσδιοριστούν ως `const`.

3.3.4.2 Μεταβλητές: Απλές, Αναφορές και Δείκτες - Παραδείγματα

Ας δούμε το παρακάτω πρόγραμμα:

```
#include <iostream>
using namespace std;

int main()
{
    int i, j; i = 5; j = i;
    cout << j << '\n' << i << '\n';
    cout << &j << '\n' << &i << '\n';

    int ii;
    cout << &ii << '\n';
    ii = 3;
    int& jj = ii;
    cout << jj << '\n' << ii << '\n';
    cout << &jj << '\n' << &ii << '\n';

    int* p; int* q; p = &ii; q = &jj;
    cout << p << '\n' << q << '\n';
    cout << &p << '\n' << &q << '\n';

    return 0;
}
```

Ας δούμε τι γίνεται στο πρόγραμμα.

`int i, j; i = 5; j = i;` Ορίζονται δύο (διαφορετικές) ακέραιες μεταβλητές, `i` και `j`. Στη μεταβλητή `i` ανατίθεται η τιμή 5 και στη μεταβλητή `j` ανατίθεται η τιμή που έχει η `i`, δηλαδή τα 5.

`cout << j << '\n' << i << '\n';` Εκτυπώνονται οι τιμές των μεταβλητών `i` και `j`, δηλαδή τα 5.

`cout << &j << '\n' << &i << '\n';` Εκτυπώνονται οι διευθύνσεις των μεταβλητών `i` και `j` οι οποίες είναι διαφορετικές μεταξύ τους, αφού πρόκειται για διαφορετικές μεταβλητές.

`int ii; cout << &ii << '\n'; ii = 3;` Ορίζεται μια τρίτη ακέραια μεταβλητή, `ii`, και εκτυπώνεται η διεύθυνσή της, που είναι διαφορετική από εκείνες των μεταβλητών `i` και `j`. Στη μεταβλητή `ii` ανατίθεται η τιμή 3.

`int& jj = ii;` Ορίζεται μια αναφορά σε ακέραιο, η `jj`, που “πατάει” επάνω στη μεταβλητή `ii`.

`cout << jj << '\n' << ii << '\n'; cout << &jj << '\n' << &ii << '\n';` Τα ονόματα `ii` και `jj` αναφέρονται στην ίδια μεταβλητή. Άρα, από την πρώτη εκτύπωση, εμφανίζεται η τιμή 3 και τις δύο φορές και από τη δεύτερη εκτύπωση εμφανίζεται η διεύθυνση της μεταβλητής `ii` δύο φορές.

`int* p; int* q; p = ⅈ q = &jj;` Ορίζονται δύο (διαφορετικές) μεταβλητές τύπου δείκτη σε ακέραιο, `p` και `q`. Δεδομένου ότι τα `ii` και `jj` αναφέρονται στην ίδια μεταβλητή, οι αναθέσεις θα οδηγήσουν και τους δύο δείκτες να έχουν την ίδια τιμή, τη διεύθυνση της μεταβλητής `ii`.

`cout << p << '\n' << q << '\n';` Εκτυπώνεται η διεύθυνση της μεταβλητής `ii` και τις δύο φορές.

`cout << &p << '\n' << &q << '\n';` Εκτυπώνονται δύο διαφορετικές διευθύνσεις, μια που οι δύο δείκτες, `p` και `q`, είναι διαφορετικές μεταβλητές.

Παρατηρήστε ότι ακόμα και όταν μιλάμε για τη διεύθυνση της αναφοράς, τελικά προσπελαύνουμε τη διεύθυνση της μεταβλητής με την οποία δεσμεύσαμε την αναφορά.

Κλείνοντας, πειραματιστείτε με το παρακάτω πρόγραμμα.

```
#include <iostream>
using namespace std;

int main()
{
    int i; int& j = i; i = 3;
    // int* p = &i; // Swap with the following line
    int* p = &j;

    cout << i << '\n' << j << '\n' << *p << endl;
    cout << p << '\n' << &j << '\n' << &i << endl;
    cout << &p << endl;

    return 0;
}
```

3.4 Γενικές Παρατηρήσεις περί Μεταγλώττισης και Συνδεσιμότητας Προγραμμάτων C++

Θα κλείσουμε το κεφάλαιο με κάποιες παρατηρήσεις περί μεταγλώττισης και συνδεσιμότητας προγραμμάτων C++, για λόγους επανάληψης. Θα εστιάσουμε σε έννοιες που ήδη ξέρετε από τον διαδικαστικό προγραμματισμό.

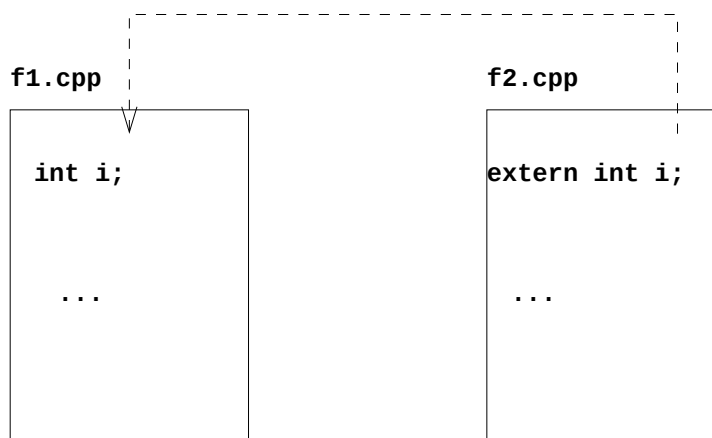
- Ένα C++ πρόγραμμα είναι ένα σύνολο συναρτήσεων με τουλάχιστον μια συνάρτηση, τη συνάρτηση `main`. Οι συναρτήσεις εφαρμόζονται πάνω σε δεδομένα-μεταβλητές που ανήκουν σε τύπους που προσφέρονται από τη γλώσσα ή τύπους ορισμένους από τον χρήστη.
- Η εκτέλεση ενός προγράμματος C++ ξεκινά καλώντας τη συνάρτηση `main`. Η κλήση αυτή εισάγεται αυτόματα κατά την κατασκευή του εκτελέσιμου αρχείου του προγράμματος.

- Το πηγαίο C++ πρόγραμμα μπορεί να είναι γραμμένο σε ένα ή περισσότερα αρχεία.
- Η μονάδα μεταγλώττισης είναι ένα αρχείο (δεν μπορούμε να μεταγλωττίσουμε κάτι λιγότερο, π.χ. μια συνάρτηση: αν θέλουμε να μεταγλωττίσουμε μόνο αυτήν, ας τη γράψουμε σε ένα αρχείο).
- Τα αρχεία τα οποία συμμετέχουν στην εντολή μεταγλώττισης περιέχουν κώδικα συναρτήσεων και ορισμούς μεταβλητών και είναι τα αρχεία κώδικα (κατάληξη `.cc`, `.cpp`).
- Τα αρχεία κώδικα μπορεί να συμπεριλαμβάνουν αρχεία επικεφαλίδας (συνήθως κατάληξη `.h`).
- Στα αρχεία επικεφαλίδας γράφουμε δηλώσεις (όχι κώδικα συναρτήσεων ή ορισμούς μεταβλητών) εκτός από ειδικές περιπτώσεις (π.χ. `const` μεταβλητές, `inline` συναρτήσεις) και ορισμούς τύπων και χώρων ονομάτων.
- Κάθε όνομα (μεταβλητής, συνάρτησης, τύπου κλπ.) πρέπει να έχει δηλωθεί πριν να χρησιμοποιηθεί μέσα σε ένα αρχείο (όχι απαραίτητα να οριστεί).
- Τόσο οι μεταβλητές ενός προγράμματος, όσο και τα σώματα των ορισμών των συναρτήσεων, καταλαμβάνουν χώρο στη μνήμη.
- Ο ορισμός μιας μεταβλητής, δηλαδή η έκφραση εκείνη που οδηγεί σε δέσμευση χώρου στη μνήμη για τη μεταβλητή αυτή, πρέπει να υπάρχει ακριβώς μία φορά σε όλο το πρόγραμμα (εκτός αν πρόκειται για μεταβλητές που ανήκουν σε διαφορετικές εμβέλεις, άρα άλλες μεταβλητές).
- Ο ορισμός μιας συνάρτησης γίνεται με το να οριστεί και το σώμα της συνάρτησης, δηλαδή ο κώδικάς της. Ο ορισμός μιας συνάρτησης πρέπει να υπάρχει ακριβώς μία φορά σε όλο το πρόγραμμα (εκτός αν πρόκειται για υπερφορτωμένους ορισμούς ή ορισμούς συναρτήσεων άλλων εμβελειών).

3.4.1 Συνδεσιμότητα Μεταβλητών

Καθολικές (global) μεταβλητές: Μια καθολική μεταβλητή

- ορίζεται σε ένα αρχείο, έξω από οποιαδήποτε συνάρτηση,
- έχει εξωτερική συνδεσιμότητα, δηλαδή μπορεί να χρησιμοποιηθεί από οπουδήποτε στο πρόγραμμα. Για παράδειγμα, δείτε το Σχήμα 3.13.

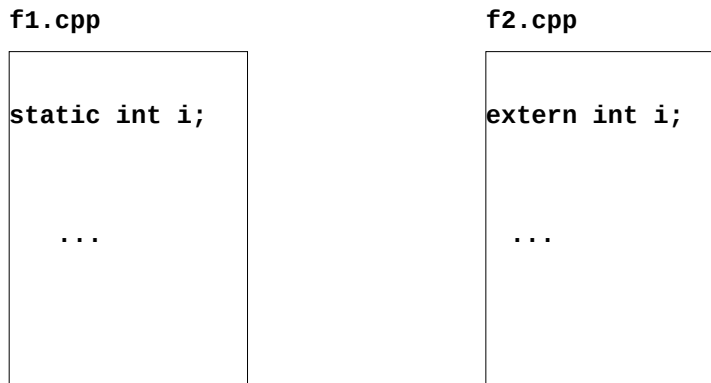


Σχήμα 3.13: Εξωτερική συνδεσιμότητα για τη μεταβλητή `i`

Στο αρχείο `f1.cpp` γίνεται ο ορισμός της μεταβλητής `i`. Στο αρχείο `f2.cpp` υπάρχει απλώς μια δήλωση για τη μεταβλητή `i`.

Στατικές εξωτερικές μεταβλητές: Μια στατική μεταβλητή (με την έννοια της μεταβλητής που είναι τοπική σε αρχείο)

- ορίζεται σε ένα αρχείο, έξω από οποιαδήποτε συνάρτηση και προσημαίνεται με τη λέξη-κλειδί `static`,
- έχει εσωτερική συνδεσιμότητα, δηλαδή μπορεί να χρησιμοποιηθεί μόνο μέσα στο αρχείο. Για παράδειγμα, δείτε το Σχήμα 3.14.



Σχήμα 3.14: Εσωτερική συνδεσιμότητα για τη μεταβλητή `i`

Στο αρχείο `f1.cpp` γίνεται ο ορισμός της μεταβλητής `i`. Στο αρχείο `f2.cpp` υπάρχει μια δήλωση για μια μεταβλητή `i`, όμως για τη μεταβλητή αυτή δεν υπάρχει πουθενά ορισμός.

Μεταβλητές `const`: Μια μεταβλητή `const`

- αν οριστεί έξω από συνάρτηση, έχει εσωτερική συνδεσιμότητα (δεν μπορεί να προσπελασθεί από άλλα αρχεία),
- για να αποκτήσει εξωτερική συνδεσιμότητα πρέπει στη δήλωσή της (και ορισμό της) να προσημανθεί και με τη λέξη-κλειδί `extern`.

Στατικές μεταβλητές τοπικές σε συνάρτηση: Μια στατική μεταβλητή τοπική σε συνάρτηση

- ορίζεται μέσα στο σώμα μιας συνάρτησης, προσημαίνεται με τη λέξη-κλειδί `static` και οφείλει να αρχικοποιείται,
- υφίσταται καθ' όλη τη διάρκεια των κλήσεων της συνάρτησης και διατηρεί την τιμή της ανάμεσα στις κλήσεις.

Αυτόματες (τοπικές) μεταβλητές: Μια αυτόματη μεταβλητή

- ορίζεται μέσα στο σώμα μιας συνάρτησης, ενδεχόμενα και σε εσωτερικό block,
- δημιουργείται προσωρινά, μέσα στη στοίβα εκτέλεσης και καταστρέφεται μόλις ο έλεγχος της εκτέλεσης βγει από τη συνάρτηση (ή το block) όπου έγινε ο ορισμός της,
- δεν έχει καθόλου συνδεσιμότητα, ο linker δεν ασχολείται με τις τοπικές μεταβλητές.

Δυναμική δημιουργία και καταστροφή μεταβλητών κατά την εκτέλεση ενός προγράμματος

- δημιουργούνται με τη χρήση του τελεστή `new`,
- ο χώρος που τους ανατίθεται βρίσκεται στον σωρό,

- πρέπει να καταστραφούν ρητά με χρήση του τελεστή `delete`, αλλιώς δεσμεύουν τον χώρο έως το τέλος εκτέλεσης του προγράμματος.

Παρατηρήσεις:

- Τα παραπάνω δεν κάνουν αναφορά σε μεταβλητές εμβέλειας κλάσης ούτε χώρου ονομάτων.
- Τα στατικά μέλη-δεδομένα των κλάσεων δεσμεύονται στον στατικό χώρο και έχουν εμβέλεια κλάσης.

3.4.2 Συνδεσιμότητα Συναρτήσεων

- Οι συναρτήσεις έχουν εξωτερική συνδεσιμότητα και μπορούν να χρησιμοποιηθούν από κώδικα που βρίσκεται σε οποιοδήποτε αρχείο του προγράμματος.
- Για να αποκτήσει μια συνάρτηση εσωτερική συνδεσιμότητα και να μην μπορεί να χρησιμοποιηθεί έξω από το αρχείο στο οποίο ορίζεται, πρέπει να προσημανθεί με τη λέξη-κλειδί `static`.
- Οι συναρτήσεις `inline` έχουν εσωτερική συνδεσιμότητα.

Παρατήρηση: Τα παραπάνω δεν κάνουν αναφορά σε συναρτήσεις εμβέλειας κλάσης ούτε χώρου ονομάτων.

3.5 Σύνοψη Κεφαλαίου

- Με μια γλώσσα διαδικαστικού προγραμματισμού υλοποιούμε διαδικασίες που μοντελοποιούν αλγορίθμους, οι οποίοι επιδρούν σε δεδομένα.
- Αν η γλώσσα υποστηρίζει τύπους, τα δεδομένα ανήκουν το καθένα σε κάποιον τύπο.
- Προκειμένου να περιγραφούν οι διαδικασίες, η γλώσσα πρέπει να προσφέρει τις κατάλληλες γλωσσικές δομές.
- Η C++ είναι μια γλώσσα που προσθέτει το αντικειμενοστραφές μοντέλο στον διαδικαστικό προγραμματισμό, επεκτείνοντας τη γλώσσα C.
- Η C++ διατηρεί το ρεπερτόριο των εντολών της C και τους τελεστές της, επεκτείνει, όμως, κατά πολύ τις δυνατότητες που σχετίζονται με τύπους.
- Η χρήση της λέξης-κλειδιού `const` για έναν προσδιοριστή και του προσδιορισμού `inline` για μια συνάρτηση, στη C++, απαλλάσσουν τον προεπεξεργαστή από τη διαχείριση χαρακτηριστικών του προγράμματος.
- Κατά την κλήση μιας συνάρτησης στη C++, η διαδικασία οδηγεί σε αρχικοποίηση των ορισμάτων της — όχι σε ανάθεση τιμής. Το ίδιο ισχύει και για την περίπτωση της οντότητας της επιστροφής, αν υπάρχει τέτοια.
- Η C++ επιτρέπει υπερφόρτωση ονομάτων συναρτήσεων και συμβόλων τελεστών.
- Η C++ επιτρέπει προσδιορισμό `default` τιμών στα ορίσματα συναρτήσεων για την περίπτωση που μια συνάρτηση αναμένεται ότι θα καλείται συχνά με αυτές τις τιμές για τα αντίστοιχα ορίσματα.
- Η δυναμική δέσμευση μνήμης στη C++ οφείλει να γίνεται με τη χρήση του τελεστή `new`, ο οποίος διασφαλίζει ότι η οντότητα που θα δημιουργηθεί θα αρχικοποιηθεί. Σε περίπτωση δυναμικής δέσμευσης μνήμης, η αποδέσμευσή της οφείλει να γίνεται με χρήση του τελεστή `delete`, ο οποίος διασφαλίζει ότι θα γίνουν και οι περαιτέρω ενέργειες που έχει προσδιοριστεί ότι χρειάζονται πριν γίνει η αποδέσμευση.

- Για το πέρασμα παραμέτρων συνάρτησης κατά αναφορά, στη C++, ενδείκνυται να χρησιμοποιείται ο τύπος αναφοράς. Αν ένα όρισμα ανήκει σε τύπο ορισμένο από τον χρήστη, αυτό ισχύει γενικότερα. Απλά, αν η συνάρτηση δεν το μεταβάλλει, καλό είναι αυτό να δηλώνεται ρητά με τον προσδιορισμό του ορίσματος ως `const`.
- Οι παραπάνω παρατηρήσεις που αναφέρονται στη γλώσσα C++ είναι κρίσιμες για τη συζήτηση που θα γίνεται αργότερα σχετικά με τις κλάσεις.
- Η κατανόηση της έννοιας των δεικτών που προσφέρει η C++ (και η C) βάζει σε πλεονεκτική θέση τον προγραμματιστή της C++ ως προς την καλύτερη κατανόηση των μεταβλητών σε άλλες αντικειμενοστραφείς γλώσσες και γι' αυτόν τον λόγο το θέμα συζητήθηκε εκτενώς στο κεφάλαιο αυτό.

3.6 Ασκήσεις

3.6.1 Αιτιολογώντας την απάντησή σας, δώστε το αποτέλεσμα της εκτέλεσης του εξής προγράμματος C++:

```
#include <iostream>
using namespace std;

int g(int x, int*& y)
{
    return (--x) + *(++y);
}

void f(int* x, int*& y, int& z)
{
    int p = g(*x, y);
    int q = g(z, x);
    int r = g(*y, x);
    int s = g(*x, y);
    cout << p << " " << q << " " << r << " " << s << endl;
}

int main()
{
    int a[] = {1, 2, 3, 4, 5, 4, 3, 2, 1};
    int* b = &a[0];
    int* c = &a[1];
    int& d = a[2];
    for (int i = 0; i < 4; i++)
        f(c, b, d);

    return 0;
}
```

3.6.2 Αιτιολογώντας την απάντησή σας, δώστε το αποτέλεσμα της εκτέλεσης του εξής προγράμματος C++:

```
#include <iostream>
using namespace std;

void g(int x, int*& y)
{
    x++; y--;
    cout << x << " " << *y << endl;
}

void f(int& b, int*& c, int* d)
{
    cout << b << " " << *c << " " << *d << endl;
    g(b, c); g(b, d);
    cout << b << " " << *c << " " << *d << endl;
}

int main()
{
    int a[] = {5, 6, 7, 8, 9};
    int& p = a[0];
    int* q = &a[4];
    int* r = &a[1];
    f(p, q, r); f(p, r, q);

    return 0;
}
```

3.6.3 Αιτιολογώντας την απάντησή σας, δώστε το αποτέλεσμα της εκτέλεσης του εξής προγράμματος C++:

```
#include <iostream>
using namespace std;

void p(int* x)
{
    cout << *x++ << " " << *x++ << " " << *x++
         << " " << *x++ << " " << *x++ << endl;
}

void s(int* y, int z)
{
    *y = z;
}

void r(int a, int& b, int* c, int& d, int e)
{
    int t;
    s(&t, a); s(&a, b); s(&b, *c);
    s(c, d); s(&d, e); s(&e, t);
}
```

```

}

int main()
{
    int x[] = {10, 20, 30, 40, 50};
    int x0 = x[0];
    int& x1 = x[1];
    int* x2 = &x[2];
    int& x3 = x[3];
    int x4 = x[4];
    p(x);
    r(x0, x1, x2, x3, x4); p(x);
    r(x0, x1, x2, x3, x4); p(x);
    r(x0, x1, x2, x3, x4); p(x);

    return 0;
}

```

3.6.4 Αιτιολογώντας την απάντησή σας, δώστε το αποτέλεσμα της εκτέλεσης του εξής προγράμματος C++:

```

#include <iostream>
using namespace std;

inline void pr(char* m) { cout << m << endl; }

void tr(char x, char& y, char*& z)
{
    y = x-1; z++; *z = *(z-1) + 2;
}

int main()
{
    char mess[] = "adieu";
    char *p0 = &mess[0]; char c1 = mess[1];
    char& c2 = mess[2]; char& c3 = mess[3];
    char c4 = mess[4];
    pr(mess); tr(c1, c2, p0); pr(mess);
    tr(c4, c2, p0); pr(mess);
    tr(c1, c3, p0); pr(mess);
    tr(c4, c3, p0); pr(mess);

    return 0;
}

```

3.6.5 Αιτιολογώντας την απάντησή σας, δώστε το αποτέλεσμα της εκτέλεσης του εξής προγράμματος C++:

```

#include <iostream>
using namespace std;

```

```

char f(char& c, char*& p)
{
    *(p+1) = c-1; c++; p++;

    return *(p-1);
}

int main()
{
    char buff[] = "bbbbbb"; char *p = buff;
    for (int i = 0; i < 4; i++) {
        char& c = buff[i]; buff[i+1] = f(c,p);
    }
    cout << buff << endl;

    return 0;
}

```

- 3.6.6 Ο Μέγιστος Κοινός Διαιρέτης (Μ.Κ.Δ.) δύο ακεραίων είναι ο μέγιστος ακέραιος που διαιρεί και τους δύο. Να γραφεί μία (αναδρομική) συνάρτηση σε C++ που να βρίσκει τον Μ.Κ.Δ. δύο ακεραίων

```
int gcd(int x, int y);
```

Η συνάρτηση να υπολογίζει τον Μ.Κ.Δ. με τον εξής τρόπο: έστω δύο ακέραιοι x και y . Αν ο x διαιρείται ακριβώς από τον y , τότε ο Μ.Κ.Δ. είναι ο y . Αλλιώς, ο Μ.Κ.Δ. είναι ο Μ.Κ.Δ. του y και του υπόλοιπου της διαίρεσης του x δια y (Αλγόριθμος του Ευκλείδη).

- 3.6.7 Να γραφεί μία συνάρτηση σε C++ η οποία, δεδομένου ενός περιττού ακεραίου d , να εκτυπώνει ένα σχήμα διαμαντιού διαγωνίων $d1 = d2 = d$,

```
void diamond(int d);
```

Το σχήμα που εκτυπώνεται να έχει την παρακάτω μορφή, π.χ.

$d = 3$

```

  *
 ***
  *

```

$d = 5$

```

    *
   ***
  *****
 ***
  *

```


$d = 7$

```

      *
     ***
    *****
   ********
  ******
 *****
  ***
   *

```

κοκ.

3.6.8 Να γραφεί μία συνάρτηση σε C++ η οποία να δέχεται δύο ακεραίους m και n , με $m \geq n$, και να εκτυπώνει όλες τις διατάξεις των ακεραίων $1, 2, \dots, m$ ανά n .

```
void permutations(int m, int n);
```

3.6.9 Η εντολή `wc` (word count) του Unix χρησιμοποιείται για να μετράμε το πλήθος των λέξεων (ή χαρακτήρων ή γραμμών) που περιέχει ένα αρχείο (ή κάποια αρχεία). Παράδειγμα:

```

host1> wc test.cpp test2.cpp
      8      11      68 test.cpp
      8      19     101 test2.cpp
     16     30     169 total

```

Να υλοποιηθεί σε C++ η εξής απλουστευμένη εκδοχή της:

- Να δέχεται ως είσοδο οσαδήποτε ονόματα αρχείων.
- Να υπολογίζει και να εκτυπώνει το πλήθος των χαρακτήρων που περιέχονται σε κάθε αρχείο και το σύνολό τους (παράμετρος `-c`), παράδειγμα:

```

host1> wc -c test.cpp test2.cpp
     68 test.cpp
    101 test2.cpp
    169 total

```

- Να υπολογίζει και να εκτυπώνει το πλήθος των λέξεων που περιέχονται σε κάθε αρχείο και το σύνολό τους (παράμετρος `-w`), παράδειγμα:

```

host1> wc -w test.cpp test2.cpp
     11 test.cpp
     19 test2.cpp
     30 total

```

- Να υπολογίζει και να εκτυπώνει το πλήθος των γραμμών που περιέχονται σε κάθε αρχείο και το σύνολό τους (παράμετρος `-l`), παράδειγμα:

```

host1> wc -l test.cpp test2.cpp
      8 test.cpp
      8 test2.cpp
     16 total

```

- Όταν καλείται χωρίς παραμέτρους, να υπολογίζει και να εκτυπώνει όλα τα παραπάνω για κάθε αρχείο (βλέπε αρχικό παράδειγμα).

Σημείωση: Ως “λέξη” θεωρείται κάθε μη κενή ακολουθία χαρακτήρων που καταλήγει σε χαρακτήρα whitespace, π.χ. το κενό, tab, newline κλπ.

3.6.10 Η εντολή `grep` του Unix χρησιμοποιείται για να εντοπίσουμε πρότυπα χαρακτήρων μέσα σε ένα ή περισσότερα αρχεία.

Παράδειγμα: “Αναζήτησε την αλφαριθμητική ακολουθία χαρακτήρων `main` στο περιεχόμενο των αρχείων `test.cpp` και `test2.cpp`”:

```
host1> grep main test.cpp test2.cpp
test.cpp:main()
test2.cpp:main()
```

Να υλοποιηθεί σε C++ η εξής απλουστευμένη εκδοχή της:

- Να δέχεται ως είσοδο οσαδήποτε ονόματα αρχείων.
- Να δέχεται ως είσοδο συγκεκριμένη αλφαριθμητική ακολουθία χαρακτήρων (όχι κανονική έκφραση).
- Να υλοποιεί τις παραμέτρους (το πολύ μία στη γραμμή εντολής):
 - c : Σ’ αυτήν την περίπτωση, να εκτυπώνει μόνο το πλήθος των γραμμών που περιέχουν το πρότυπο. Παράδειγμα:


```
host1> ./grep -c main test.cpp test2.cpp
test.cpp:1
test2.cpp:1
```
 - l : Σ’ αυτήν την περίπτωση, να εκτυπώνει μόνο τα ονόματα των αρχείων μέσα στα οποία εντοπίστηκε το πρότυπο. Παράδειγμα:


```
host1> ./grep -l main test.cpp test2.cpp
test.cpp
test2.cpp
```
 - n : Για το κάθε αρχείο στο οποίο εντοπίστηκε το πρότυπο, να εκτυπώνεται και ο αριθμός γραμμής μέσα στην οποία εντοπίστηκε (ξεκινώντας την αρίθμηση από το 1). Παράδειγμα:


```
host1> ./grep -n main test.cpp test2.cpp
test.cpp:3:main()
test2.cpp:3:main()
```

3.6.11 Να γραφεί μία συνάρτηση σε C++ η οποία να δέχεται μία ακολουθία χαρακτήρων που αν περιέχει και τα παρενθεσιακά σύμβολα `{`, `}`, `(`, `)`, `[`, και `]`, να ελέγχει αν η ακολουθία είναι συντακτικά ορθή από πλευράς ταιριάσματος των παρενθεσιακών αυτών συμβόλων, δηλαδή,

```
bool syntax(const char* str);
```

Στην περίπτωση που δεν είναι συντακτικά ορθή, να εκτυπώνεται ένα μήνυμα με το αντίστοιχο πρόβλημα, για παράδειγμα,

```
"(a[bcd((ef)gh)]ijkl)" ---> correct
```

```
"(a[bcd(())ef))" ---> no proper nesting
```

```
"a[bcd])ef(" ---> closing parenthesis encountered before opening
```

3.6.12 Τα ντετερμινιστικά πεπερασμένα αυτόματα (deterministic finite automata) είναι υποθετικές συσκευές που μας επιτρέπουν να αναγνωρίσουμε αν μία έκφραση είναι αποδεκτή σε σχέση με μία απλή γλώσσα. Η έκφραση — ως ακολουθία συμβόλων από ένα “αλφάβητο” — θεωρείται ως η είσοδος του αυτόματου που διαβάζεται από μία υποθετική ταινία εισόδου. Η ακολουθία διαβάζεται ένα-ένα σύμβολο και τέλος, όταν δεν υπάρχει άλλο σύμβολο στην ακολουθία, αποφασίζεται από το αυτόματο αν η ακολουθία είναι επιτρεπτή (αποδεκτή) σε σχέση με τη γλώσσα που υλοποιεί το αυτόματο ή όχι. Ανά πάσα στιγμή, το αυτόματο βρίσκεται σε μία “κατάσταση” — ξεκινώντας από μία αρχική κατάσταση — και, ανάλογα με το τρέχον σύμβολο εισόδου, μεταβαίνει (μεταπίπτει) από την κατάσταση που βρίσκεται σε μία άλλη. Η ακολουθία εισόδου είναι αποδεκτή, αν μετά το διάβασμα όλων των συμβόλων της ακολουθίας, το αυτόματο βρίσκεται σε κατάσταση που χαρακτηρίζεται ως “τελική”.

Τυπικά, ένα ντετερμινιστικό πεπερασμένο αυτόματο είναι μία πεντάδα

$$M = (K, \Sigma, \delta, s, F)$$

όπου:

- K είναι ένα πεπερασμένο σύνολο από καταστάσεις.
- Σ είναι ένα αλφάβητο.
- $s \in K$ είναι η αρχική κατάσταση.
- $F \subseteq K$ είναι το σύνολο των τελικών καταστάσεων.
- δ είναι η συνάρτηση μεταπτώσεων, δηλαδή, $\delta : K \times \Sigma \rightarrow K$. Η συνάρτηση μεταπτώσεων είναι εκείνη που αναπαριστά τους κανόνες με βάση τους οποίους το αυτόματο μεταβαίνει από τη μία κατάσταση στην άλλη. Δηλαδή, εάν το M βρίσκεται στην κατάσταση k ($\in K$) και το τρέχον σύμβολο από την ακολουθία εισόδου είναι σ ($\in \Sigma$), τότε το $\delta(k, \sigma)$ είναι η κατάσταση στην οποία μεταβαίνει το M , ντετερμινιστικά.

Παράδειγμα: $M = (K, \Sigma, \delta, s, F)$

$$\text{με } K = \{q_0, q_1\}$$

$$\Sigma = \{a, b\}$$

$$s = q_0$$

$$F = q_0$$

$$\delta: (q_0, a) \mapsto q_0$$

$$(q_0, b) \mapsto q_1$$

$$(q_1, a) \mapsto q_1$$

$$(q_1, b) \mapsto q_0$$

Το παραπάνω αυτόματο αναγνωρίζει όλες τις ακολουθίες από a και b που περιέχουν άρτιο αριθμό εμφανίσεων του b , π.χ. $bbbb$, abb , $aabab$, κ.ο.κ.

Να γραφεί ένα πρόγραμμα C++ το οποίο να υλοποιεί ένα ντετερμινιστικό πεπερασμένο αυτόματο. Το πρόγραμμα πρέπει να είναι γενικό, ώστε να λειτουργεί για οποιαδήποτε πεντάδα αυτομάτου.

3.7 Βιβλιογραφία

- [1] Bruce Eckel. *Thinking in C++, Volume I: Introduction to Standard C++*. 2η έκδοση. Prentice Hall, 2000.
- [2] Richard Bornat. *Programming from First Principles*. Prentice Hall International (UK) Ltd., 1987.
- [3] C++ reference. <https://en.cppreference.com/w/>. Ανάκτηση: 2023-5-30.
- [4] cplusplus.com. <http://www.cplusplus.com/>. Ανάκτηση: 2023-5-30.
- [5] Bjarne Stroustrup. *The C++ Programming Language*. 4η έκδοση. Addison-Wesley Professional, 2013.
- [6] Bjarne Stroustrup. *Programming: Principles and Practice Using C++*. 2η έκδοση. Addison-Wesley Professional, 2014.

Η ΕΝΝΟΙΑ ΤΗΣ ΕΜΒΕΛΕΙΑΣ

Σύνοψη

Στο κεφάλαιο αυτό γίνεται συζήτηση για την έννοια της εμβέλειας της σύνδεσης ονομάτων με συγκεκριμένο νόημα μέσα σε ένα πρόγραμμα. Στο πλαίσιο αυτό, παρουσιάζονται οι δηλώσεις ονομάτων στη C++, καθώς και διάφορες μορφές καθορισμού εμβέλειας που προσφέρει η γλώσσα. Για καλύτερη κατανόηση της συμπεριφοράς των προγραμμάτων C++, γίνεται εστίαση επάνω στην εμβέλεια των (δηλώσεων των) ονομάτων μεταβλητών σε σχέση με τον χρόνο ζωής (lifetime) των μεταβλητών αυτών. Ιδιαίτερη έμφαση δίνεται, στη συνέχεια, στην έννοια των χώρων ονομάτων (namespaces), προκειμένου να γίνει μια πρώτη αναφορά σε εμβέλεια που έχει όνομα.

Προαπαιτούμενη Γνώση

Για την κατανόηση του κεφαλαίου, ο αναγνώστης απαιτείται να έχει γνώσεις διαδικαστικού προγραμματισμού με χρήση των γλωσσών C και C++.

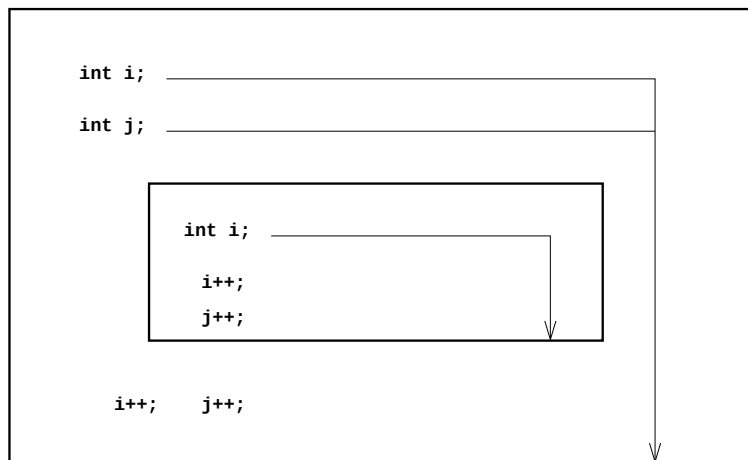
4.1 Ονόματα και Εμβέλεια

Όταν προγραμματίζουμε, χρειαζόμαστε ονόματα. Ονοματίζουμε σχεδόν κάθε τι που ορίζεται στο πρόγραμμά μας. Όσο πιο μεγάλο δε είναι το πρόγραμμα που γράφουμε, τόσο πιο πολλές οντότητες θα ορίσουμε, άρα τόσο πιο πολλά είναι και τα ονόματα που θα χρειαστούμε. Ή μήπως, όχι; Μήπως υπάρχει κάποιος μηχανισμός που περιορίζει — όχι με την έννοια του να αποκρύπτει, απαραίτητα — το νόημα που αποδίδεται σε ένα όνομα, έτσι ώστε το όνομα αυτό να μπορεί να χρησιμοποιηθεί κάπου αλλού στο πρόγραμμά μας, αποκτώντας διαφορετικό νόημα;

Υπάρχει. Είναι η δυνατότητα περιορισμού της σύνδεσης ενός ονόματος με ένα νόημα στο πλαίσιο ενός περιβάλλοντος, μιας εμβέλειας. Με τον όρο εμβέλεια αναφερόμαστε στο μέρος εκείνο του προγράμματος για το οποίο ισχύει η απόδοση νοήματος σε ένα όνομα μέσω μιας δήλωσης. Η εμβέλεια περιορίζει την ισχύ της δήλωσης. Στον κώδικά μας μπορούμε να θεωρήσουμε ότι κάθε του γραμμή ανήκει σε μια εμβέλεια και κάθε

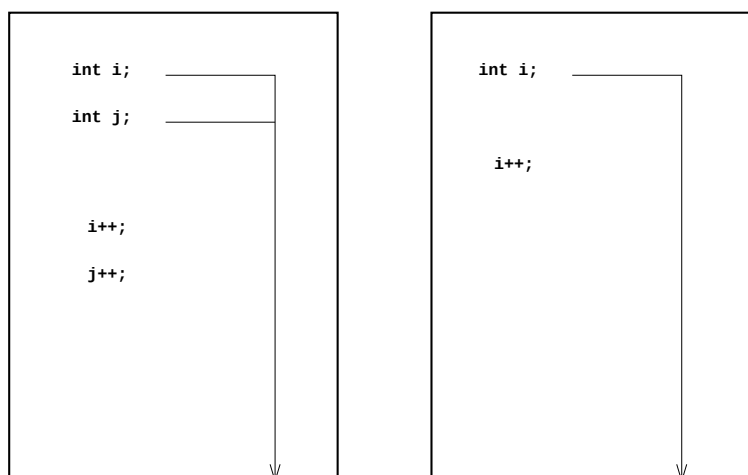
όνομα που χρησιμοποιείται στη γραμμή αυτή έχει νόημα, ανάλογα με το περιβάλλον της εμβέλειας αυτής. Σε διαφορετικά τμήματα του κώδικά μας, το ίδιο όνομα που ονοματίζει, π.χ. μια μεταβλητή, μπορεί να σχετίζεται με διαφορετικές μεταβλητές.

Στα Σχήματα 4.1 και 4.2, με πλαίσια συμβολίζουμε εμβέλειες που προσδιορίζουν περιβάλλοντα. Και στα δύο σχήματα βλέπουμε το παράδειγμα ύπαρξης διαφορετικών μεταβλητών με το ίδιο όνομα, *i*, λόγω ορισμού τους σε διαφορετικές εμβέλειες. Συγκεκριμένα, στο Σχήμα 4.1 έχουμε εμφώλευση εμβελειών. Στην εξωτερική εμβέλεια υπάρχει ορισμός, άρα και δήλωση, δύο ακέραιων μεταβλητών, *i* και *j*. Η δήλωση αυτή είναι σε ισχύ σε όλο το εξωτερικό πλαίσιο του σχήματος. Άρα, όταν αυξάνουμε τις μεταβλητές *i* και *j* στη γραμμή με τις εντολές *i++*; *j++*; αυξάνουμε τις μεταβλητές αυτές. Στο εσωτερικό πλαίσιο, όμως, υπάρχει και νέος ορισμός, άρα και δήλωση, για μια μεταβλητή με όνομα *i*. Οπότε, όταν αυξάνουμε τη μεταβλητή *i* στη γραμμή με την εντολή *i++*; που περιέχεται στο εσωτερικό πλαίσιο, αναφερόμαστε στη νέα μεταβλητή *i* και η αύξηση γίνεται σε αυτήν. Αντίθετα, όταν αυξάνουμε τη μεταβλητή *j* στη γραμμή με την εντολή *j++*; που περιέχεται στο εσωτερικό πλαίσιο, αναφερόμαστε στη μεταβλητή *j* που έχει δηλωθεί στο εξωτερικό πλαίσιο: η δήλωσή της παραμένει σε ισχύ. Σε κάθε περίπτωση, το τι προσδιορίζεται από ένα όνομα προκύπτει από τη δήλωση που εμφανίζεται στην κοντινότερη εμβέλεια.



Σχήμα 4.1: Εμφωλευμένα περιβάλλοντα

Στο Σχήμα 4.2, κάθε πλαίσιο ορίζει διακριτές εμβέλειες, οπότε τα περιβάλλοντα είναι ξένα μεταξύ τους. Κατά συνέπεια, δεν τίθεται θέμα χρήσης ονομάτων δηλωμένων στη μία εμβέλεια από την άλλη.



Σχήμα 4.2: Διακριτά περιβάλλοντα

Ακόμα και στην καθημερινή μας ζωή, το ίδιο όνομα μπορεί να ονοματίζει διαφορετικά στιγμιότυπα της

ίδιας έννοιας. Οπότε, χρησιμοποιούμε ένα όνομα στο πλαίσιο του περιβάλλοντος που βρισκόμαστε μια συγκεκριμένη στιγμή. Για παράδειγμα, υποθέστε ότι ο αδελφός σας ονομάζεται “Γιώργος”. Όμως, και ένας συμφοιτητής σας ονομάζεται “Γιώργος”. Τότε, αν είστε στο σπίτι και φωνάξετε “Γιώργο”, απευθύνεστε στον αδελφό σας. Αντίθετα, αν είστε στη σχολή και φωνάξετε “Γιώργο”, τότε απευθύνεστε στον συμφοιτητή σας. Αν βρεθούν κάπου μαζί, πρέπει να ξεκαθαρίσετε σε ποιον αναφέρεστε από τους δύο. Άλλο παράδειγμα από την καθημερινή ζωή μας αναφέρεται στο μαθητικό και το φοιτητικό περιβάλλον και αφορά γενικότερα νόημα ουσιαστικών. Αν είμαστε μαθητές πανελληνίων και ακούσουμε την έκφραση “βγήκαν οι βάσεις”, το μυαλό μας οδηγείται στο ότι ανακοινώθηκαν οι βάσεις εισαγωγής των πανεπιστημιακών τμημάτων. Η λέξη “βάσεις” αναφέρεται σε αυτήν την έννοια. Αν είμαστε φοιτητές πληροφορικής, όμως, και εξεταστήκαμε στο μάθημα “Βάσεις Δεδομένων”, η έκφραση αυτή θα οδηγήσει το μυαλό μας στο ότι ανακοινώθηκε η βαθμολογία του μαθήματος. Σε αυτό τώρα αναφέρεται η λέξη “βάσεις”. Βλέπουμε, λοιπόν, ότι η συσχέτιση ονόματος με νόημα ανάλογα με το περιβάλλον δεν μας είναι ξένο.

Θα δούμε πιο αναλυτικά κάποιες μορφές εμβέλειας στη C++. Πρέπει εγκαίρως να εξοικειωθούμε με την έννοια αυτή, μια που αργότερα όταν θα μιλάμε για κλάσεις αυτό θα είναι πολύ κρίσιμο. Η προσοχή μας θα πρέπει να είναι διαρκώς σε εγρήγορση σε σχέση με την εμβέλεια που ανήκει κάθε γραμμή κώδικα που γράφουμε ή διαβάζουμε. Λάθος υπόθεση είναι μια από τις πλέον συνήθεις πηγές λαθών: αυξάνουμε, για παράδειγμα, μια μεταβλητή count στο σώμα μιας συνάρτησης μιας κλάσης και άλλη νομίζουμε ότι αυξάνουμε και άλλη αυξάνουμε, στην πραγματικότητα.

Στη C++, ο γενικός τρόπος να ορίσουμε μια εμβέλεια είναι με τη χρήση αγκύλων, { }. Μέσα στις αγκύλες μπορούμε να έχουμε δηλώσεις ονομάτων και να περιορίσουμε την ισχύ τους μέσα σε αυτές. Μπορούμε να έχουμε εμφωλευμένες αγκύλες, οπότε στις εσωτερικές ισχύουν οι δηλώσεις των εξωτερικών, εκτός εάν τα ονόματα έχουν αποκτήσει άλλο νόημα μέσω νέων δηλώσεων. Με τη χρήση εμβελειών βάζουμε σε τάξη και οργάνωση τα ονόματα του προγράμματός μας, κάτι που διευκολύνει την ανάπτυξη κώδικα ακόμα περισσότερο. Ο ίδιος ο Stroustrup, όταν ερωτήθηκε ποιο θεωρεί ότι είναι το πιο “κρίσιμο εργαλείο” που προσφέρει η C++, απάντησε “οι αγκύλες”.

Στη C++ έχουμε τη δυνατότητα να προσδιορίσουμε εμβέλειες που έχουν όνομα. Η πιο απλή περίπτωση είναι εκείνη των χώρων ονομάτων (namespaces). Την περίπτωση αυτή θα εξετάσουμε αργότερα στο κεφάλαιο αυτό. Η άλλη, όμως, περίπτωση, που μας ενδιαφέρει πολύ περισσότερο, αφορά τις κλάσεις. Και αυτές, όπως ήδη είπαμε, πέρα από οτιδήποτε άλλο, προσδιορίζουν εμβέλειες με όνομα. Ένας από τους βασικούς λόγους που θα συζητήσουμε περί χώρων ονομάτων στο βιβλίο αυτό είναι για να είμαστε ήδη εξοικειωμένοι με την περίπτωση των εμβελειών με όνομα, όταν θα μιλάμε για τις κλάσεις.

Κλείνοντας τη σύνοψη αυτή περί εμβέλειας και πώς υποστηρίζεται από τη C++, να πούμε ότι στη γλώσσα αυτή, στην περίπτωση που η χρήση ενός ονόματος δεν καθορίζει μοναδικά την οντότητα στην οποία αναφέρεται, μπορεί να χρησιμοποιηθεί ο τελεστής επίλυσης εμβέλειας (scope resolution operator), ::. Αυτός μας επιτρέπει να διασαφηνίσουμε την εμβέλεια από την οποία μας ενδιαφέρει να επιλέξουμε την οντότητα. Αυτό ισχύει και στη γενική περίπτωση που θέλουμε να κάνουμε επιλεκτική χρήση ονόματος από εμβέλεια. Οι δυνατότητες είναι συγκεκριμένες: είτε μπορούμε να έχουμε πρόσβαση σε ένα καθολικά ορατό όνομα είτε επιλεκτικά από μία εμβέλεια που έχει όνομα. Η γενική μορφή χρήσης του τελεστή επίλυσης εμβέλειας είναι:

`<Scope_name> :: <Identifier>`

όπου `<Identifier>` είναι το όνομα που μας ενδιαφέρει και `<Scope_name>` μπορεί να είναι είτε το όνομα μιας εμβέλειας είτε κενό. Στη δεύτερη περίπτωση αναφερόμαστε στην καθολικά ορατή οντότητα που ονοματίζεται από τον `<Identifier>`.

4.1.1 Δηλώσεις Ονομάτων

Σχεδόν κάθε τι το οποίο χρησιμοποιείται σε ένα πρόγραμμα C++ έχει κάποιο όνομα μέσω του οποίου το χρησιμοποιούμε. Για παράδειγμα, δίνουμε ονόματα στις μεταβλητές, στις συναρτήσεις που ορίζουμε, στους τύπους που ορίζουμε κλπ. Κάθε όνομα, προτού να χρησιμοποιηθεί, πρέπει να έχει δηλωθεί. Σε ένα πρόγραμμα, ή πιο

σωστά σε μία εμβέλεια, πρέπει για κάθε όνομα, να δίνουμε έναν ακριβώς ορισμό, εκτός αν το όνομα αναφέρεται σε οντότητες διαφορετικού είδους, π.χ. να χρησιμοποιήσουμε το ίδιο όνομα για έναν τύπο και μια μεταβλητή, ή επιτρέπεται από τη γλώσσα ρητά, π.χ. υπερφόρτωση ονομάτων συναρτήσεων.

Στο σημείο αυτό, να κάνουμε μια αναφορά στις δηλώσεις σε σχέση με τους ορισμούς ή και την ανάθεση αρχικών τιμών, όσον αφορά την περίπτωση των μεταβλητών. Ας δούμε μία-μία τις περιπτώσεις αυτές.

Δηλώσεις Μια δήλωση αποτελεί μια περιγραφή που συνδέει ένα όνομα μεταβλητής με έναν τύπο, π.χ.

```
extern int error_number;
```

Ορισμοί Με έναν ορισμό γίνεται δέσμευση ενός ονόματος με μια συγκεκριμένη οντότητα, π.χ.

```
char ch;
```

Ένας ορισμός αποτελεί ταυτόχρονα και δήλωση.

Ανάθεση αρχικών τιμών Με τον τρόπο αυτόν, γίνεται ανάθεση αρχικών τιμών στην οντότητα στην οποία αναφέρεται ένα όνομα, π.χ.

```
int count = 1;
const double pi = 3.141592;
```

Η ανάθεση αρχικών τιμών γίνεται σε μια έκφραση ορισμού μιας μεταβλητής, άρα αποτελεί και δήλωσή της.

Γενικά, με μια δήλωση, συνδέουμε ένα όνομα με το είδος της οντότητας την οποία ονομάζει το όνομα. Με τον ορισμό, στην περίπτωση μεταβλητών και συναρτήσεων, δημιουργούμε την οντότητα, δηλαδή δεσμεύουμε τον χώρο στη μνήμη. Μια συνάρτηση μπορεί να δηλωθεί χωρίς να οριστεί, δίνοντας μόνο το πρωτότυπό της. Όπως ήδη είπαμε, για τις μεταβλητές, ένας ορισμός αποτελεί ταυτόχρονα και δήλωση και η ανάθεση αρχικών τιμών σε μια μεταβλητή αποτελεί ταυτόχρονα και ορισμό και δήλωσή της.

Μια δήλωση εισάγει κάποιο όνομα μέσα σε μια εμβέλεια. Η εμβέλεια του νοήματος ενός ονόματος ξεκινά από τη δήλωσή του.

Παραδείγματα (για μεταβλητές και συναρτήσεις)

```
char ch; <---- DEFINITION
```

```
string s; <---- DEFINITION
```

```
int count = 1;
```

```
const double pi = 3.1415;
```

```
extern int ext_number; <---- DECLARATION ONLY !!!!!
```

```
const char* Name = "TheName";
```

```
const char* season[] = {"anixi", "kalokeri", "fthinoporo", "ximonas"};
```

```
int day(Date* pd) { return pd -> d; } <---- DEFINITION
```

```
double sqrt(double); <---- DECLARATION ONLY !!!!!
```


4.1.2 Η Περίπτωση των Μεταβλητών: Εμβέλεια Δήλωσης vs. Χρόνος Ζωής

Επανερχόμαστε στην περίπτωση των μεταβλητών στη C++, μια που η σχέση μεταξύ της εμβέλειας μιας δήλωσης τους και του χρόνου ζωής τους μπορεί να αποτελέσει πηγή σύγχυσης. Θα περιοριστούμε στις μεταβλητές με ονόματα, μια που οι δηλώσεις αφορούν αυτές. Στη C++, οι μεταβλητές που έχουν όνομα εντάσσονται στις εξής κατηγορίες:

Καθολικές μεταβλητές: Ορίζονται εκτός σωμάτων συναρτήσεων. Οι μεταβλητές αυτές είναι, εν δυνάμει, ορατές και χρησιμοποιήσιμες από όλο το πρόγραμμα και έχουν χρόνο ζωής τη διάρκεια εκτέλεσης του προγράμματος. Έχουν εξωτερική συνδεσιμότητα και κατά την εκτέλεση του προγράμματος φυλάσσονται στον στατικό χώρο της μνήμης.

Στατικές εξωτερικές: Αν μια εν δυνάμει καθολική μεταβλητή προσδιορίζεται ως `static`, τότε η εμβέλειά της περιορίζεται έως το τέλος του αρχείου μέσα στο οποίο ορίζεται, άρα και δηλώνεται. Όμως, ο χρόνος ζωής της είναι όλη η διάρκεια εκτέλεσης του προγράμματος. Οι μεταβλητές αυτές έχουν εσωτερική συνδεσιμότητα και κατά την εκτέλεση του προγράμματος φυλάσσονται στον στατικό χώρο της μνήμης.

Στατικές τοπικές σε συνάρτηση: Οι μεταβλητές αυτές ορίζονται (με ταυτόχρονη αρχικοποίηση) μέσα στο σώμα μιας συνάρτησης και έχουν προσδιοριστεί ως `static`. Σε αυτήν την περίπτωση, η εμβέλεια της δήλωσης του ονόματός τους περιορίζεται μέσα στο σώμα της συνάρτησης και μόνο, αλλά ο χρόνος ζωής τους είναι όλη η διάρκεια εκτέλεσης του προγράμματος. Φυλάσσονται στον στατικό χώρο, οπότε η ύπαρξή τους είναι ανεξάρτητη από το εάν το πλαίσιο (`frame`) της συνάρτησης βρίσκεται στη στοίβα εκτέλεσης (μεταβαλλόμενος χώρος) ή όχι — δηλαδή εάν ο έλεγχος της εκτέλεσης του προγράμματος βρίσκεται μέσα στο σώμα της συνάρτησης ή εάν έχει ολοκληρωθεί η εκτέλεση του κώδικά της. Εξ ου και πολλές φορές τις περιγράφουμε ως μεταβλητές “που διατηρούν την τιμή τους μεταξύ των κλήσεων της συνάρτησης”.

Τοπικές σε συνάρτηση: Οι κλασικές αυτόματες μεταβλητές που ορίζονται στο σώμα μιας συνάρτησης. Η εμβέλεια της δήλωσης του ονόματός τους περιορίζεται μέσα στο σώμα της συνάρτησης, αλλά και ο χρόνος ζωής τους σταματά με τη λήξη της εκτέλεσης του κώδικα του σώματος της συνάρτησης.

Τοπικές σε `block { }`: Αυτές είναι μεταβλητές που ορίζονται μέσα σε ένα `block`. Η εμβέλεια της δήλωσής τους περιορίζεται από το `block`. Γενικά, ο χρόνος ζωής τους περιορίζεται από την εκτέλεση του `block`.

Στατικές τοπικές σε `block`: Αν μια μεταβλητή που οριστεί μέσα σε ένα `block` προσδιορίζεται ως `static`, ο χρόνος ζωής της είναι όλη η διάρκεια της εκτέλεσης του προγράμματος. Ισχύει ό,τι έχουμε πει για την περίπτωση των μεταβλητών που είναι στατικές τοπικές σε συνάρτηση.

Τα παρακάτω παραδείγματα έχουν σκοπό να σας εξασκήσουν στην καλύτερη κατανόηση της σχέσης μεταξύ εμβέλειας ονόματος και διάρκειας ζωής των μεταβλητών.

Παράδειγμα: Συνδυάστε καθένα από τα αρχεία `f1.cpp`, `f2.cpp` και `f3.cpp` με το αρχείο `main.cpp`, τρέξτε το εκτελέσιμο που προκύπτει κάθε φορά και αιτιολογήστε τα αποτελέσματα της εκτέλεσης.

```
// File: main.cpp
#include <iostream>
using namespace std;

extern int x;
void f();
```

```
int main()
{
    cout << x << endl;
    f();
    return 0;
}
```

```
// File: f1.cpp
#include <iostream>
using namespace std;

int x = 10000;

void f()
{
    int x;
    x = 1;
    cout << ::x << endl;
    {
        int x;
        x = 2;
        cout << x << endl;    // Prints inner x = 2
    }
    cout << x << endl;    // Prints outer x = 1
}
```

```
// File: f2.cpp
#include <iostream>
using namespace std;

int x = 20000;

void f()
{
    int x = 2;
    cout << x << endl;    // Prints local x
    cout << ::x << endl;    // Prints global x
}
```

```
// File: f3.cpp
#include <iostream>
using namespace std;

int x = 300000;

void f()
{
    int y = x;    // Assigns the value of global x
    int x = 2;    // Defines a new local x
}
```

```

    cout << x << endl;    // Prints new x
    cout << y << endl;    // Prints y which has the value of global x
}

```

Παράδειγμα: Συνδυάστε καθένα από τα αρχεία f1.cpp, f2.cpp, f3.cpp και f4.cpp με το αρχείο main.cpp, τρέξτε το εκτελέσιμο που προκύπτει κάθε φορά και αιτιολογήστε τα αποτελέσματα της εκτέλεσης.

```

// File: main.cpp
#include <iostream>
using namespace std;

extern int x;

int f();

int main()
{
    cout << "                x = " << x << endl;
    cout << "1st output of f()    " << f() << endl;
    cout << "2nd output of f()    " << f() << endl;

    return 0;
}

```

```

// File: f1.cpp
int x = 10000;

int f()
{
    int x = ::x;
    return x++;
}

```

```

// File: f2.cpp
int x = 10000;

int f()
{
    static int x = ::x;
    return x++;
}

```

```
// File: f3.cpp
static int x = 10000; // CAREFUL: x local to current file!

int f()
{
    int x = ::x;
    return x++;
}
```

```
// File: f4.cpp
#include <iostream>
using namespace std;

int x = 10000;

int f()
{
    static int x = ::x;
    {
        static int x = 8;
        cout << x++ << endl;
    }
    cout << x++ << endl;
    return x;
}
```

Μεταβλητές στον στατικό και στον μεταβαλλόμενο χώρο: Πειραματιστείτε εκτελώντας τα παρακάτω προγράμματα και αιτιολογήστε τα αποτελέσματα (“σχεδιάστε” τους χώρους της μνήμης).

- **Παράδειγμα:**

```
int i = 1000;    // global scope, allocated in static space

void g()
{
    int i = 5;    // scope within g, automatic variable
}

void f()
{
    int i = 50;    // scope within f, automatic variable
    static int j = 500; // scope within f, allocated in static space

    g();
}
```

```
int main()
{
    int i = 5000; // scope within main, automatic variable

    f();

    return 0;
}
```

• Παράδειγμα:

```
// File: vars.cpp
#include <iostream>
using namespace std;

int g = 1000;

void p(int i)
{
    int j = 30;
    cout << "i in p is : " << i << endl;
    cout << "j in p is : " << j << "\n\n\n" << endl;
}

void f(int i)
{
    int j = 20;
    static int k = 100;

    cout << "i in f is : " << i << endl;
    cout << "j in f is : " << j << endl;
    cout << "k in f is : " << k << endl;
    cout << "g in f is : " << g << "\n\n" << endl;

    i++; j++; k++; g++;

    p(i);
}

int main()
{
    int i = 5; cout << "i in main is : " << i << '\n' << endl;
    f(i);      cout << "i in main is : " << i << '\n' << endl;
    f(i++);    cout << "i in main is : " << i << '\n' << endl;
    f(++i);    cout << "i in main is : " << i << '\n' << endl;

    cout << "g in main is : " << g << '\n' << endl;

    return 0;
}
```

```
}

```

Δέσμευση στον ελεύθερο χώρο: Πειραματιστείτε εκτελώντας τα παρακάτω προγράμματα και αιτιολογήστε τα αποτελέσματα (“σχεδιάστε” τους χώρους της μνήμης).

- **Παράδειγμα:** (τροποποίηση του προηγούμενου ώστε να γίνεται δυναμική δέσμευση χώρου στο σώμα της συνάρτησης p)

```
// Similar to vars.cpp but space allocated
// within the body of function p
#include <iostream>
using namespace std;

int g = 1000;

void p(int i)
{
    int* j = new int(30);    // now j points to heap
    cout << "i in p is : " << i << endl;
    cout << "j in p points to : " << *j << "\n\n" << endl;
    delete j;
}

void f(int i)
{
    int j = 20;
    static int k = 100;
    cout << "i in f is : " << i << endl;
    cout << "j in f is : " << j << endl;
    cout << "k in f is : " << k << endl;
    cout << "g in f is : " << g << "\n\n" << endl;

    i++; j++; k++; g++;

    p(i);
}

int main()
{
    int i = 5;
    cout << "i in main is :" << i << '\n' << endl;

    f(i);    cout << "i in main is :" << i << '\n' << endl;
    f(i++);  cout << "i in main is :" << i << '\n' << endl;
    f(++i);  cout << "i in main is :" << i << '\n' << endl;

    cout << "g in main is :" << g << '\n' << endl;
}
```

```
    return 0;
}
```

- **Παράδειγμα:** Χώρος που δεσμεύτηκε δυναμικά αποδεσμεύεται, αλλά ο δείκτης ο οποίος αρχικοποιήθηκε να δείχνει σε αυτόν παραμένει εν ζωή στην εμβέλεια.

```
#include <iostream>
using namespace std;

void p()
{
    int k = 300;
    int* kk = &k;
    cout << "kk points to :" << *kk << "\n\n\n" << endl;

    int* j = new int(30);
    cout << "j in p points to : " << *j << "\n\n\n" << endl;

    delete j;    // Deallocate storage where j pointed to;

    j = kk;    // j is NOT out of scope, now points where kk points to
    cout << "kk points to :" << *kk << "\n\n\n" << endl;
    cout << "j after deletion points to :" << *j << "\n\n\n" << endl;
}

int main()
{
    p(); p();

    return 0;
}
```

4.2 Χώροι Ονομάτων

Στα προηγούμενα είδαμε ότι μέσω ενός ονόματος μπορούμε να έχουμε πρόσβαση είτε στην οντότητα που σχετίζεται με το όνομα αυτό στην εμβέλεια που βρισκόμαστε είτε, με χρήση του τελεστή επίλυσης εμβέλειας με κενό αριστερά του, στην οντότητα που είναι καθολικά ορατή. Μπορούμε, όμως, να έχουμε πρόσβαση σε μια οντότητα μιας εμβέλειας επιλεκτικά, αρκεί η εμβέλεια να έχει όνομα — και να μην έχει απαγορευτεί η πρόσβαση. Στην περίπτωση αυτή, αρκεί αριστερά από τον τελεστή επίλυσης εμβέλειας να συμπεριλάβουμε το όνομα της εμβέλειας αυτής.

4.2.1 Η Έννοια του Χώρου Ονομάτων

Στη C++ δίνεται η δυνατότητα να “πακετάρουμε” δηλώσεις σε μια εμβέλεια με όνομα, μέσω των *χώρων ονομάτων*. Συγκεκριμένα, ένας χώρος ονομάτων αποτελεί μια εμβέλεια με όνομα. Ένα παράδειγμα ορισμού ενός χώρου ονομάτων είναι το εξής:

```
namespace Geometry {
    const double PI = 3.1415;
    double L = 1.0;        // Don't include definitions, in the general case
    double square_surface(double);
    double rectang_surface(double, double);
    double circle_surface(double);
}
```

Στο παραπάνω παράδειγμα, έχουμε ορίσει έναν χώρο ονομάτων με όνομα `Geometry`, στον οποίο περιέχονται δηλώσεις ονομάτων που αφορούν τον χώρο της γεωμετρίας. Συγκεκριμένα, δηλώνεται η σταθερά π και συναρτήσεις που υπολογίζουν το εμβαδόν διαφόρων γεωμετρικών σχημάτων, όπως του τετραγώνου, του ορθογωνίου παραλληλογράμμου και του κύκλου. Επίσης, συμπεριλάβαμε και μια μεταβλητή, L , την τιμή της οποίας μπορεί να χρειαστούμε ως μήκος πλευράς.

Αν θέλουμε να επιλέξουμε ένα από τα ονόματα αυτά εκτός του χώρου ονομάτων, αρκεί με χρήση του τελεστή επίλυσης εμβέλειας, `::`, να προσδιορίσουμε το όνομα του χώρου. Για παράδειγμα, η έκφραση:

```
exPI = Geometry::PI;
```

κάνει ανάθεση στη μεταβλητή `exPI` της τρέχουσας εμβέλειας την τιμή που έχει η σταθερά π που έχει οριστεί στον χώρο ονομάτων `Geometry`.

Τα ονόματα ενός χώρου ονομάτων γίνονται διαθέσιμα έξω από αυτόν χωρίς να επιβάλεται η χρήση του τελεστή επίλυσης εμβέλειας με τη χρήση της έκφρασης `using namespace`. Από τη γραμμή που εμφανίζεται η έκφραση και έως το τέλος της εμβέλειας μέσα στην οποία βρίσκεται αυτή, πλέον, δεν χρειάζεται να χρησιμοποιηθεί ο τελεστής επίλυσης εμβέλειας για να προσδιοριστεί το όνομα του χώρου. Για παράδειγμα,

```
double Var1 = Geometry::PI;
```

```
using namespace Geometry;
```

```
double Var2 = PI;
```

Η επίδραση του `using` ισχύει έως το τέλος της τρέχουσας εμβέλειας, όπως είπαμε. Δηλαδή, η απαλλαγή από τη χρήση του τελεστή `::` ισχύει έως το τέλος του τρέχοντος block ή, αν το `using` εμφανίζεται έξω από block, έως τέλος του αρχείου. Προτείνεται να μην χρησιμοποιούμε εκφράσεις `using` μέσα σε αρχεία επικεφαλίδας, διότι είναι πολύ επικίνδυνο. Ο λόγος είναι ότι εισάγουν τις δηλώσεις του χώρου ονομάτων στην εμβέλεια του αρχείου που θα τα συμπεριλάβει και αυτό μπορεί να μην είναι επιθυμητό. Η έκφραση `using` μπορεί να αναφέρεται σε ολόκληρο το `namespace` ή μόνο σε κάποια από τα ονόματα που δηλώνει. Υπάρχουν πολλές δυνατότητες που σχετίζονται με τους χώρους ονομάτων, αλλά είναι εκτός του στόχου μας, οπότε θα τις παραλείψουμε.

Ολοκληρώνοντας, όμως, το παράδειγμα του χώρου ονομάτων `Geometry`, θα κάνουμε μια παρατήρηση που θα φανεί χρήσιμη αργότερα, όταν θα μιλάμε για κλάσεις: στα σώματα των συναρτήσεων που δηλώνονται στον χώρο ονομάτων βρισκόμαστε στην εμβέλεια του χώρου ονομάτων. Παρατηρήστε στο παράδειγμα που ακολουθεί ότι στον ορισμό της συνάρτησης `double circle_surface(double)` αναφερόμαστε στο `PI` του χώρου ονομάτων `Geometry` χωρίς ρητό προσδιορισμό με χρήση του τελεστή `::`. Αξίζει ακόμα να παρατηρήσετε ότι εφόσον οι συναρτήσεις `double square_surface(double)`, `double rectang_surface(double, double)` και `double circle_surface(double)` του χώρου ονομάτων `Geometry` ορίζονται εκτός του χώρου, προσδιορίζεται ότι πρόκειται για τις συναρτήσεις της εμβέλειας αυτής με χρήση του τελεστή επίλυσης εμβέλειας και το όνομα του χώρου ονομάτων.

Ακολουθεί το παράδειγμα ορισμού και χρήσης του χώρου ονομάτων `Geometry` ολοκληρωμένο.

Παράδειγμα ορισμού και χρήσης χώρου ονομάτων

```

#include <iostream>
using namespace std;

double L = 100.0;

namespace Geometry {
    const double PI = 3.1415;
    double L = 1.0;      // Don't include definitions, in the general case
    double square_surface(double);
    double rectang_surface(double, double);
    double circle_surface(double);
}

int main()
{
    cout << L << endl;

    cout << Geometry::PI << endl;

    cout << "Geometry::L is: " << Geometry::L << endl;

    cout << Geometry::square_surface(L) << endl;

    using namespace Geometry;    // till the end of the block of
                                   // the main function, no need
                                   // to use scope resolution operator,
                                   // unless ambiguity exists

    cout << circle_surface(Geometry::L) << endl;
                                   // Geometry::circle_surface() but L is ambiguous
    cout << ::L << endl;

    return 0;
}

double Geometry::square_surface(double l)
{
    return l*l;
}

double Geometry::rectang_surface(double l1, double l2)
{
    return l1*l2;
}

```

```
double Geometry::circle_surface(double l)
{
    return PI*l*l;
}
```

Κλείνοντας, πρέπει να σημειώσουμε ότι ενδείκνυται να ορίζουμε χώρους ονομάτων στην περίπτωση που θεωρούμε ότι τα ονόματα που δηλώνονται είναι ειδικού ενδιαφέροντος, οπότε όποιος ενδιαφέρεται να τα χρησιμοποιήσει μπορεί να αναφερθεί σε αυτά επιλεκτικά. Μια τέτοια περίπτωση είναι τα ονόματα που ορίζονται σε βιβλιοθήκες. Το πιο χαρακτηριστικό παράδειγμα της περίπτωσης αυτής είναι η καθιερωμένη βιβλιοθήκη της C++: τα ονόματά της είναι δηλωμένα σε έναν χώρο ονομάτων, τον `std`. Για τον λόγο αυτό, όποτε πρόκειται να χρησιμοποιήσουμε ονόματα από την καθιερωμένη βιβλιοθήκη είτε τα κάνουμε (άμεσα) ορατά μέσω της έκφρασης `using namespace std`; είτε προσδιορίζουμε την εμβέλεια με χρήση του τελεστή `::`, π.χ. `std::cout`. Στη γενική περίπτωση, προτιμάται το δεύτερο μια που το πρώτο ενδέχεται να οδηγήσει σε ασάφεια νοήματος κατά τη χρήση ονομάτων, αν υπάρχουν δηλώσεις για κάποια από τα ονόματα της βιβλιοθήκης και στον κώδικά μας. Στους κώδικες του βιβλίου, όμως, ακολουθούμε το πρώτο μια που πρόκειται για κώδικες περιορισμένης έκτασης, με λίγες δηλώσεις ονομάτων, οπότε ο προσδιορισμός της εμβέλειας της βιβλιοθήκης σε κάθε χρήση ονόματός της πιο πολύ θα “φόρτωνε” το κείμενο του κώδικα παρά θα διασφάλιζε αποφυγή ενδεχόμενων προβλημάτων στη μεταγλώττισή του.

4.2.2 Ανώνυμοι Χώροι Ονομάτων

Μια ενδιαφέρουσα δυνατότητα που μας προσφέρουν οι χώροι ονομάτων δίνεται από τους *ανώνυμους χώρους ονομάτων*. Αυτοί είναι χώροι ονομάτων χωρίς όνομα. Ο ορισμός ενός ανώνυμου χώρου ονομάτων κάνει όλα τα ονόματα που δηλώνει άμεσα ορατά έως το τέλος του αρχείου μέσα στο οποίο ορίζεται ο χώρος και μόνο. Έτσι αποφεύγεται να τα προσδιορίσουμε ως `static`, για να περιορίσουμε τη χρήση τους στο αρχείο αυτό. Κατά συνέπεια, απαλλασσόμαστε από τη χρήση του προσδιορισμού `static` για θέματα που αφορούν εμβέλεια, οπότε μπορούμε να περιορίσουμε τη χρήση του σε περιπτώσεις που αφορούν χρόνο ζωής μεταβλητών.

```
// Declarations in the unnamed namespace are COMPLETELY hidden
// outside this file. However the names can be used within the file
// as if they are "imported" by a "using" directive
// Effect: no need to declare this information as "static"

#include <iostream>
using namespace std;

namespace {
    const double PI = 3.1415;
    double L = 1.0;
    double circle_surface(double l)
    {
        return PI*l*l;
    }
}

int main()
{
    cout << L << endl;
    cout << "The namespace L: " << L << endl;
    circle_surface(L);
}
```

```

    return 0;
}

```

4.2.3 Οργάνωση Προγράμματος που περιέχει Χώρους Ονομάτων

Αν θέλουμε να ορίσουμε έναν χώρο ονομάτων, η οργάνωση των αρχείων που προτείνεται είναι η εξής. Ορίζουμε τον χώρο ονομάτων σε ένα αρχείο επικεφαλίδας και γράφουμε τους ορισμούς για τα ονόματα που δηλώνει σε ένα αρχείο πηγαίου κώδικα. Στο αρχείο αυτό συμπεριλαμβάνουμε το αρχείο επικεφαλίδας που περιέχει τον ορισμό του χώρου ονομάτων. Το αρχείο επικεφαλίδας αυτό πρέπει να συμπεριληφθεί σε κάθε αρχείο στο οποίο θέλουμε να χρησιμοποιηθούν ονόματα από τον χώρο ονομάτων.

Για το προηγούμενο παράδειγμα, η οργάνωση μπορεί να είναι η εξής.

```

// File: namespace.h
namespace Geometry {
    const double PI = 3.1415;
// double L = 1.0;      // The problem with this becomes obvious now!
    extern double L;      // Declaration only
                        // Geometry::L will be defined in a source file
    double square_surface(double);
    double rectang_surface(double, double);
    double circle_surface(double);
}

```

```

// File namespace.cpp
#include <iostream>
#include "namespace.h"
using namespace std;

double Geometry::L = 1.0;

double Geometry::square_surface(double l)
{
    return l*l;
}

double Geometry::rectang_surface(double l1, double l2)
{
    return l1*l2;
}

double Geometry::circle_surface(double l)
{
    return PI*l*l;
}

```

```
// File main.cpp
#include <iostream>
#include "namespace.h"
using namespace std;

double L = 100.0;

int main()
{
    cout << L << endl;

    cout << Geometry::PI << endl;

    cout << "Geometry::L is: " << Geometry::L << endl;

    cout << Geometry::square_surface(L) << endl;

    using namespace Geometry;

    cout << circle_surface(Geometry::L) << endl;
        // Geometry::circle_surface() but L is ambiguous
    cout << ::L << endl;

    return 0;
}
```

Με προτεινόμενη αυτήν την οργάνωση των αρχείων στην περίπτωση κώδικα που περιέχει ορισμούς χώρων ονομάτων, καλό είναι μέσα στους χώρους ονομάτων να περιλαμβάνουμε δηλώσεις, καθώς και ορισμούς οντοτήτων με εσωτερική συνδεσιμότητα, π.χ. σταθερών και συναρτήσεων `inline`. Ο λόγος είναι ότι γράφοντας τον ορισμό του χώρου ονομάτων σε ένα αρχείο επικεφαλίδας το οποίο θα συμπεριληφθεί, ενδεχόμενα, από διαφορετικά αρχεία πηγαίου κώδικα, σε κάθε άλλη περίπτωση θα είχαμε για τα ονόματα που ορίζονται στον χώρο πολλαπλή εμφάνιση ορισμών για το ίδιο είδος οντότητας με το ίδιο όνομα στην εμβέλειά του.

Κλείνοντας τη συζήτηση για τους χώρους ονομάτων, συνοψίζουμε γιατί κάναμε την αναφορά σε αυτούς.

- Ορίζουν εμβέλεια με όνομα.
 - Ορίζουμε έναν χώρο ονομάτων ώστε επιλεκτικά με χρήση του τελεστή `::` να αναφερόμαστε σε δηλώσεις που ισχύουν σε αυτή.
 - Στα σώματα των συναρτήσεων που δηλώνονται σε έναν χώρο ονομάτων λειτουργούμε μέσα στην εμβέλεια του χώρου αυτού.

Τα παραπάνω θα μας φανούν χρήσιμα όταν, αργότερα, θα μιλάμε για κλάσεις, μια που και αυτές ορίζουν εμβέλειες με όνομα.

- Με τη χρήση ανώνυμων χώρων ονομάτων, απαλλασσόμαστε από τη χρήση της λέξης `static`, στη C++, για θέματα που αφορούν εμβέλεια.

4.3 Σύνοψη Κεφαλαίου

- Η έννοια της εμβέλειας είναι κρίσιμη για ένα οργανωμένο πρόγραμμα.

- Μια δήλωση εισάγει ένα όνομα σε μια εμβέλεια μέσα στην οποία το όνομα αυτό συνδέεται με το νόημα που προσδίδεται από τη δήλωση.
- Στη C++ συνήθης τρόπος να προσδιορίσουμε εμβέλειες είναι με χρήση αγκυλών, {}, που ορίζουν blocks.
- Στην περίπτωση των μεταβλητών είναι επιθυμητό να γίνει κατανοητή η διάκριση μεταξύ της εμβέλειας του ονόματός τους και της διάρκειας ζωής τους.
- Στη C++ μπορούμε επιλεκτικά να χρησιμοποιήσουμε ονόματα όπως δηλώνονται σε μια συγκεκριμένη εμβέλεια, χρησιμοποιώντας τον τελεστή επίλυσης εμβέλειας, ::.
- Στη C++ μπορούμε να “πακετάρουμε” δηλώσεις σε μία εμβέλεια με όνομα, χρησιμοποιώντας την έννοια του *χώρου ονομάτων*.
- Ορισμός χώρων ονομάτων ενδείκνυται στην περίπτωση που αναπτύσσουμε εκτενή κώδικα ή/και κώδικα εξειδικευμένου ενδιαφέροντος.
- Εξοικείωση με θέματα εμβέλειας με όνομα χρησιμοποιώντας τους χώρους ονομάτων προετοιμάζει το έδαφος για την εξοικείωση με θέματα εμβέλειας που αφορούν κλάσεις.

4.4 Ασκήσεις

4.4.1 Αιτιολογώντας την απάντησή σας, δώστε το αποτέλεσμα της εκτέλεσης του εξής προγράμματος C++:

```
#include <iostream>
using namespace std;

int i = 100;

int main()
{
    cout << i++ << endl;

    int j = 20; int i = 10;

    cout << i++ << endl;
    cout << j++ << endl;

    {
        int i = 5;
        cout << i++ << endl;
        cout << j++ << endl;
    }

    cout << i++ << endl;
    cout << j++ << endl;

    return 0;
}
```

4.4.2 Αιτιολογώντας την απάντησή σας, δώστε το αποτέλεσμα της εκτέλεσης του εξής προγράμματος C++:

```
#include <iostream>
using namespace std;

int i = 100;

int bla(int bi)
{
    int i = bi++;
    cout << i++ << endl;

    return i;
}

int main()
{
    cout << i++ << endl;

    int j = 20; int i = 10;

    cout << i++ << endl;
    cout << j++ << endl;

    i = bla(i);
    cout << i++ << endl;

    i = bla(::i);
    cout << i++ << endl;

    {
        int i = ::i;
        cout << i++ << endl;
        cout << j++ << endl;
    }

    ::i = i;
    cout << ::i++ << endl;
    cout << j++ << endl;

    return 0;
}
```

4.4.3 Αιτιολογώντας την απάντησή σας, δώστε το αποτέλεσμα της εκτέλεσης του εξής προγράμματος C++:

```
#include <iostream>
using namespace std;

int i = 100;
```

```

int bla(int i)
{
    cout << i++ << endl;
    return i++;
}

int bloo(int bi)
{
    cout << i++ << endl;

    static int i = bi;
    return i++;
}

int main()
{
    cout << i++ << endl;

    int j = 20; int i = 10;

    cout << i++ << endl;
    cout << j++ << endl;

    i = bla(i);
    i = bla(::i);

    i = bloo(i);
    i = bloo(::i);

    cout << i++ << endl;
    cout << j++ << endl;

    return 0;
}

```

4.4.4 Κατ' αναλογία με το παράδειγμα από τον χώρο της Γεωμετρίας, σκεφτείτε ένα πεδίο για το οποίο θα θέλατε να δώσετε δηλώσεις σε έναν χώρο ονομάτων, π.χ. μεγεθών και σταθερών από μια περιοχή της Φυσικής, και υλοποιήστε το.

4.4.5 Δίνονται τα παρακάτω αρχεία:

unn.cc:

```

#include <iostream>
using namespace std;

int bla(int);

namespace {
    int i = 100;
}

```

```

}

int main()
{
    cout << i << endl;
    cout << bla(i) << endl;

    int i = 10;
    cout << i << endl;
    cout << bla(i) << endl;

    {
        int i = 1;
        cout << i << endl;
        cout << bla(i) << endl;
    }

    cout << i << endl;
    cout << bla(i) << endl;

    return 0;
}

```

unn1.cc:

```

#include <iostream>
using namespace std;

int i = 200;

int bloo(int);

int bla(int bi)
{
    cout << i << endl;
    i = bi;

    return bloo(i);
}

```

unn2.cc:

```

#include <iostream>
using namespace std;

extern int i;

int bloo(int bi)
{
    cout << i << endl;
    i = bi;
}

```



```
    return i;  
}
```

Μπορεί να προκύψει εκτελέσιμο πρόγραμμα από τα αρχεία αυτά; Αν ναι, αιτιολογώντας την απάντησή σας, δώστε το αποτέλεσμα της εκτέλεσής του.

4.5 Βιβλιογραφία

- [1] Bruce Eckel. *Thinking in C++, Volume I: Introduction to Standard C++*. 2η έκδοση. Prentice Hall, 2000.
- [2] C++ reference. <https://en.cppreference.com/w/>. Ανάκτηση: 2023-5-30.
- [3] cplusplus.com. <http://www.cplusplus.com/>. Ανάκτηση: 2023-5-30.
- [4] Bjarne Stroustrup. *The C++ Programming Language*. 4η έκδοση. Addison-Wesley Professional, 2013.
- [5] Bjarne Stroustrup. *Programming: Principles and Practice Using C++*. 2η έκδοση. Addison-Wesley Professional, 2014.

ΚΕΦΑΛΑΙΟ 5

ΚΛΑΣΕΙΣ ΚΑΙ ΑΝΤΙΚΕΙΜΕΝΑ

Σύνοψη

Στο κεφάλαιο αυτό, γίνεται εστίαση στην έννοια της αφαίρεσης στα δεδομένα και στον ορισμό νέων τύπων στη C++. Συγκεκριμένα, περιγράφεται ο τρόπος με τον οποίο γίνεται ο ορισμός των κλάσεων και, κατόπιν, η χρήση τους για τον ορισμό αντικειμένων και εκτέλεση λειτουργιών επάνω σε αυτά. Παρουσιάζονται οι προσδιορισμοί ορατότητας των ονομάτων που δηλώνονται σε μια κλάση και συζητείται η σωστή χρήση τους. Ιδιαίτερη έμφαση δίνεται στη διαδικασία αρχικοποίησης των αντικειμένων μέσω συναρτήσεων αρχικοποίησης (constructors) καθώς και στη λειτουργικότητα των συναρτήσεων καταστροφής/αποδέσμευσης (destructors). Η παρουσίαση εστιάζει στη C++. Ολοκληρώνεται με τη διαδικασία αρχικοποίησης δι' αντιγραφής (copy construction) καθώς και τη διαδικασία ανάθεσης τιμών μεταξύ αντικειμένων, τονίζοντας τη διαφορά μεταξύ των δύο.

Προαπαιτούμενη Γνώση

Για την κατανόηση του κεφαλαίου, ο αναγνώστης απαιτείται να έχει γνώσεις διαδικαστικού προγραμματισμού και να έχει μελετήσει τις έννοιες του αντικειμενοστραφούς προγραμματισμού από τα προηγούμενα κεφάλαια του βιβλίου.

5.1 Αφαίρεση στα Δεδομένα: Ορισμός Νέων Τύπων και Εισαγωγή στις Κλάσεις

Ένας τύπος είναι η φυσική αναπαράσταση μιας ιδεατής οντότητας. Για παράδειγμα, ο τύπος `float` με τις αριθμητικές πράξεις να τον συνοδεύουν, `+`, `-`, `*` κλπ., είναι η αναπαράσταση του πραγματικού αριθμού των μαθηματικών. Όπως τονίσαμε στα προηγούμενα, ένα πρόγραμμα που χρησιμοποιεί τύπους που προσομοιώνουν τις οντότητες της εφαρμογής γίνεται ευκολότερα κατανοητό. Ο στόχος μας στον αντικειμενοστραφή προγραμματισμό, τελικά, είναι ο ορισμός από τον χρήστη νέων τύπων, η πρόσβαση στα στοιχεία των οποίων γίνεται μέσω συγκεκριμένων συναρτήσεων. Έτσι επιτυγχάνουμε διαχωρισμό των λεπτομερειών υλοποίησης του τύπου από τις ιδιότητες/λειτουργίες του τύπου, που είναι απαραίτητες για τη σωστή χρήση του. Οι νέοι

τύποι που ορίζει ο προγραμματιστής στον αντικειμενοστραφή προγραμματισμό μοντελοποιούνται ως κλάσεις. Μια υπενθύμιση ακόμα, οι κλάσεις μπορούν να λειτουργήσουν και ως εργαλείο για να πετύχουμε *τμηματοποίηση* του κώδικά μας με *απόκρυψη πληροφορίας*, ώστε να έχουμε καλύτερη οργάνωση του προγράμματός μας.

Στο κεφάλαιο αυτό, καθώς και στα αμέσως επόμενα, θα εξετάσουμε τα βασικά χαρακτηριστικά του αντικειμενοστραφούς προγραμματισμού μέσα από μια συγκεκριμένη γλώσσα προγραμματισμού, τη C++. Να επισημάνουμε ότι οι οντότητες που θα επιλέξουμε να υλοποιήσουμε ως κλάσεις στα παραδείγματά μας εννοιολογικά είναι πολύ απλές και δεν αναδεικνύουν άμεσα την αναγκαιότητα του αντικειμενοστραφούς προγραμματισμού. Όμως, η υλοποίησή τους μέσω κλάσεων είναι αρκούντως πλούσια ώστε να αναδεικνύει τα θέματα που πρέπει να αντιμετωπίσουμε, όταν αναπαριστάνουμε μια οντότητα του προβλήματός μας ως νέο τύπο τον οποίο υλοποιούμε ως κλάση. Η υλοποίηση δε, είναι τέτοια ώστε να αναδεικνύει λεπτά σημεία που πρέπει να προσέξουμε υλοποιώντας κλάσεις στη C++. Ενδιαφέρουσες περιπτώσεις προβλημάτων κατάλληλων προς αντιμετώπιση σε αντικειμενοστραφές περιβάλλον και υλοποίηση εννοιών τους μέσω κλάσεων, ο αναγνώστης καλείται να αντιμετωπίσει μέσω των προβλημάτων που περιλαμβάνονται στο Κεφάλαιο 11.

Ας δούμε, λοιπόν, ποιες είναι οι βασικές οντότητες/έννοιες που θα υλοποιούμε ως κλάσεις στη συνέχεια του κεφαλαίου αυτού:

η έννοια της “Ημερομηνίας”: Στην περίπτωση αυτή αναφερόμαστε σε μια οντότητα που εκφράζει την ημερομηνία και την οποία αντιλαμβανόμαστε ως τρεις ακεραίους αριθμούς που αντιστοιχούν στην ημέρα, στον μήνα και στο έτος. Από πλευράς συμπεριφοράς, ας υποθέσουμε ότι έχουμε μια απλή λειτουργική οντότητα η οποία είναι:

- η δυνατότητα ανάθεσης τιμών για την ημέρα, μήνα και έτος μιας ημερομηνίας, συνολικά και
- η δυνατότητα ταυτόχρονης ανάκτησης των παραπάνω τιμών.

η έννοια του “Φοιτητή”: Στην περίπτωση αυτή, η οντότητα αντιστοιχεί στην έννοια του φοιτητή. Ας υποθέσουμε ότι ενδιαφερόμαστε για το ονοματεπώνυμό του, τον αριθμό μητρώου του και το έτος σπουδών του και ότι η λειτουργικότητα αφορά:

- τη δυνατότητα αναθέσεων τιμών για το ονοματεπώνυμο, τον αριθμό μητρώου και το έτος σπουδών, κατά περίπτωση, και
- τη δυνατότητα ανάκτησης των παραπάνω τιμών, πάλι κατά περίπτωση.

Στη συνέχεια, ξεκινώντας από την πιο απλή περίπτωση του τύπου της “ημερομηνίας”, θα δούμε τα θέματα που πρέπει να αντιμετωπίζουμε προκειμένου να ορίσουμε μια κλάση που θα υλοποιεί την οντότητά μας.

Επίσης, να κάνουμε μια επισήμανση. Για λόγους ομαλής μετάβασης στην ορολογία του αντικειμενοστραφούς προγραμματισμού, θα συνδέσουμε όρους του με όρους ήδη γνωστούς από τον προγραμματισμό. Για την περίπτωση της C++ ειδικότερα, θα λέγαμε ότι οι όροι αυτοί είναι συνώνυμοι, σαν να είχαμε δύο διαλέκτους: μια διάλεκτο που σχετίζεται με τον προγραμματισμό, γενικότερα, και μια διάλεκτο που αναφέρεται στον αντικειμενοστραφή προγραμματισμό, ειδικότερα. Στο πλαίσιο αυτό, στα κεφάλαια που ακολουθούν και αφορούν τη C++, θα αντιμετωπίζουμε συχνά ως συνώνυμα τον όρο “*τύπος ορισμένος από τον χρήστη*” με τον όρο “*κλάση*” καθώς και τον όρο “*μεταβλητή*” (τέτοιου τύπου) με τον όρο “*αντικείμενο*”.

Για κάθε θέμα που θα συζητάμε, αρχικά θα δίνουμε την εννοιολογική περιγραφή του και στη συνέχεια θα προχωράμε σε παράδειγμα με κώδικα C++. Για να γίνει η συμπεριφορά του κώδικα σαφής, πολύ συχνά, θα κάνουμε αναφορά στην εικόνα της μνήμης κατά την εκτέλεσή του. Προσέξτε, όμως, η εικόνα αυτή θα είναι αφαιρετική και δεν θα συμφωνεί σε λεπτομέρειες με την πραγματική εικόνα της μνήμης. Δηλαδή, θα είναι τέτοια ώστε να μπορεί να αναδείξει τις λεπτομέρειες της συμπεριφοράς της εκτέλεσης, παραβιάζοντας τεχνικές λεπτομέρειες, όχι όμως όσον αφορά εκείνες που είναι κρίσιμες για τις εν λόγω συζητήσεις.

Ξεκινάμε, λοιπόν, την περιήγησή μας στον αντικειμενοστραφή προγραμματισμό με όχημα τη C++. Αρχικά, μπορούμε να δώσουμε μια σύνοψη, λέγοντας τα εξής. Η θεμελιώδης έννοια του τύπου που ορίζεται από

τον χρήστη στη C++ είναι αυτή της κλάσης. Η πρόσβαση στα αντικείμενα μιας κλάσης γίνεται μέσω συναρτήσεων που είναι συνδεδεμένες με την κλάση, των *συναρτήσεων-μελών* (member functions) της κλάσης. Κάθε αντικείμενο που δημιουργείται “κτίζεται” από μεταβλητές που περιγράφονται στον ορισμό μιας κλάσης, τα *μέλη-δεδομένα* (data members) της κλάσης. Στην αναπαράσταση ενός αντικειμένου μιας κλάσης, συμμετέχουν όλα τα μέλη-δεδομένα που περιγράφονται στην κλάση (εκτός από εκείνα που προσδιορίζονται ως *στατικά*, τα οποία είναι μέλη που απλά είναι εμβέλειας κλάσης και δεν αφορούν συγκεκριμένο αντικείμενο). Κατά τη δημιουργία αντικειμένων μιας κλάσης, γίνεται αυτόματα κλήση συγκεκριμένων συναρτήσεων που ενδεχόμενα δηλώνονται στην κλάση και λέγονται *συναρτήσεις αρχικοποίησης/κατασκευής*. Κατ’ αναλογία, ορίζονται *συναρτήσεις καταστροφής* που καλούνται αυτόματα όταν πρόκειται να γίνει καταστροφή των αντικειμένων των κλάσεων και απελευθέρωση του χώρου που τους έχει ανατεθεί. Ένα αντικείμενο μιας κλάσης μπορεί να δημιουργηθεί όπως μια οποιαδήποτε άλλη μεταβλητή της C++, δηλαδή είτε ως αυτόματη μεταβλητή, είτε ως στατική, είτε δυναμικά στον σωρό.

Στη συνέχεια, θα αναλύσουμε τα παραπάνω.

5.1.1 Τύποι Δεδομένων Ορισμένοι από τον Χρήστη μέσω της Γλωσσικής Δομής struct

Ας ξεκινήσουμε από το παράδειγμα της ημερομηνίας. Ας επανέλθουμε στο παράδειγμα των αεροπορικών πτήσεων του Κεφαλαίου 2. Να υπενθυμίσουμε ότι, στο παράδειγμα αυτό, για τις αεροπορικές πτήσεις μας ενδιέφερε ο Αριθμός Πτήσης, η Πόλη Αφετηρίας, η Πόλη Άφιξης, η Ώρα Αναχώρησης και η Διάρκεια. Ας υποθέσουμε, όμως, ότι έχουμε μια παραλλαγή της έννοιας των αεροπορικών πτήσεων όπου μας ενδιαφέρει και η ημερομηνία μιας αεροπορικής πτήσης (δεν αναφερόμαστε, δηλαδή, σε πτήσεις επαναλαμβανόμενες, π.χ. σε καθημερινή βάση, αλλά πτήσεις που αφορούν συγκεκριμένες ημερομηνίες), δηλαδή:

Αεροπορική Πτήση

Αριθμός Πτήσης	<i>ακέραιος</i>
Πόλη Αφετηρίας	<i>συμβολοσειρά</i>
Πόλη Άφιξης	<i>συμβολοσειρά</i>
Ώρα Αναχώρησης	<i>Χρόνος</i>
Διάρκεια	<i>Χρόνος</i>
Ημερομηνία Πτήσης	<i>Ημερομηνία (?)</i>

Οπότε, θα πρέπει να υλοποιήσουμε έναν τύπο “Ημερομηνία” που αναπαριστάει την έννοια της ημερομηνίας. Ας υποθέσουμε ότι αυτή ακολουθεί την περιγραφή που δώσαμε προηγουμένως.

Ο Νέος Τύπος à la C

Αν η γλώσσα υλοποίησης ήταν η C, θα μπορούσαμε να κάνουμε την υλοποίηση ως νέο τύπο υλοποιημένο με χρήση της γλωσσικής δομής struct με τρία πεδία τύπου ακεραίου — ένα για την ημέρα (day), ένα για τον μήνα (month) και ένα για το έτος (year). Δηλαδή:

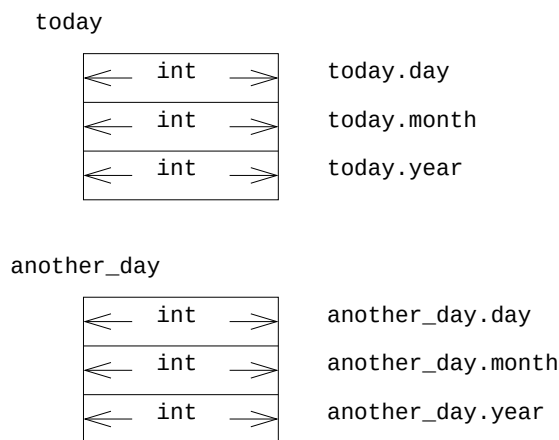
```
struct date { int day; int month; int year; };
```

Με τον παραπάνω ορισμό, μπορούμε να ορίσουμε μεταβλητές του τύπου date, π.χ.

```
...
date today;
date another_day;
..
```

Καθεμία από τις μεταβλητές today και another_day είναι μια διαφορετική οντότητα (στη μνήμη). Η καθεμία από αυτές “κτίζεται” από τρεις ακεραίους, όπως φαίνεται στο Σχήμα 5.1.

Στη συνέχεια, με δεδομένη αυτήν την αναπαράσταση, θα μπορούσαμε να ορίσουμε τις συναρτήσεις που υλοποιούν τη συμπεριφορά του τύπου “Ημερομηνία”. Έστω ότι αυτές έχουν καθοριστεί να έχουν ονόματα



Σχήμα 5.1: Οι μεταβλητές `today` και `another_day` (αφαιρετικά) στη μνήμη

`set_date` για ανάθεση τιμής και `get_date` για ανάκτηση τιμής, με τρία ορίσματα η καθεμία, τα οποία να αντιστοιχούν στην ημέρα, μήνα και έτος της ημερομηνίας που μας ενδιαφέρει. Μια πιθανή υλοποίησή τους θα ήταν:

```
void set_date(date* dt, int d, int m, int y) {
    dt->day = d; dt->month = m; dt->year = y;
}

void get_date(date* dt, int* d, int* m, int* y) {
    *d = dt->day; *m = dt->month; *y = dt->year;
}
```

Τι προβλήματα βλέπουμε στην παραπάνω υλοποίηση; Ένα πρόβλημα από το οποίο μπορούμε να ξεκινήσουμε είναι ότι δεν γίνεται καθαρό σε κάποιον που διαβάζει το πρόγραμμα ότι οι συναρτήσεις `set_date` και `get_date` έχουν ειδική σημασία για τον τύπο `date` και αντιστοιχούν στη συμπεριφορά του. Δηλαδή, παρότι έχουν ένα όρισμα που αντιστοιχεί στον τύπο αυτόν, δεν τις διαφοροποιεί από κάθε άλλη συνάρτηση που απλά δέχεται ως όρισμα μια μεταβλητή του τύπου δείκτη σε `date`, π.χ.

```
void bla(date*, ...);
```

Ο Νέος Τύπος à la C++

Αντίθετα, στη C++, οι συναρτήσεις που υλοποιούν τη συμπεριφορά του τύπου μπορούν να δηλωθούν μέσα στον ορισμό του τύπου, απαλλαγμένες από το όρισμα που αντιστοιχεί στη μεταβλητή του τύπου. Με τον τρόπο αυτόν δηλώνεται ότι εφαρμόζονται σε μια μεταβλητή του τύπου, η οποία όμως δεν αναφέρεται ρητά, αλλά υπονοείται.

Με βάση αυτό, ο ορισμός του τύπου στη C++ μπορεί να γραφτεί όπως παρακάτω:

```
struct date {
    int day, month, year;
    void set_date(int, int, int);
    void get_date(int*, int*, int*);
};
```

Σε αυτήν την περίπτωση, εκφράσαμε τη συνάρτηση `set_date` ως *συνάρτηση-μέλος* του τύπου `date` και εννοείται ότι, κατά τη χρήση της, εφαρμόζεται απαραίτητα σε μια μεταβλητή του τύπου. Το ίδιο ισχύει και για τη συνάρτηση `get_date`.

Πώς, όμως, χρησιμοποιούμε αυτές τις συναρτήσεις; Αφού δεν έχουμε όρισμα που να αντιστοιχεί στη μετα-

βλητή του τύπου, πώς αναφερόμαστε σε αυτή κατά την κλήση τους; Η σύνταξη της κλήσης μιας συνάρτησης-μέλους ακολουθεί τη σύνταξη της χρήσης ενός πεδίου μιας μεταβλητής που είναι τύπου που έχει οριστεί μέσω `struct`: η μεταβλητή γράφεται αριστερά από την κλήση διαχωρίζοντάς την με τελεία, ., π.χ.

```
today.set_date(22,5,2022)
```

αντί να είχαμε την κλήση

```
set_date(&today,22,5,2022)
```

που θα είχαμε χρησιμοποιώντας τη C και τον ορισμό της συνάρτησης που δώσαμε για τη γλώσσα αυτή.

Στο σημείο αυτό να κάνουμε και μια σημαντική παρατήρηση. Κατά τον ορισμό ενός τύπου μέσω `struct`, ορίζεται και μία εμβέλεια. Η εμβέλεια αυτή έχει όνομα το όνομα του τύπου και της ανήκουν όλες οι δηλώσεις ονομάτων που περιέχονται στον ορισμό του τύπου. Δεν είναι, λοιπόν, απαραίτητο να χρησιμοποιούμε το συνθετικό `_date` στα ονόματα των συναρτήσεων. Μπορούμε να ονοματίσουμε τις συναρτήσεις `set_date` και `get_date` με τα ονόματα `set` και `get`, αντίστοιχα, μια που έχουν μοναδικό νόημα στην εμβέλεια `date`.

Τέλος, μια παρατήρηση άνευ σημασίας για τον ορισμό τύπων, αλλά μεγάλης σημασίας για πέρασμα παραμέτρων με αναφορά στη C++: τα ορίσματα που αντιστοιχούν στις τιμές της ημέρας, του μήνα και του έτους στη συνάρτηση `get` είναι πιο κατάλληλο να έχουν τύπο `int&`, αντί για `int*`.

Συνοψίζοντας το τι έχουμε πετύχει έως τώρα, στη C++, ο ορισμός του νέου τύπου `date` και η χρήση του μπορεί να είναι ως εξής:

Ορισμός του τύπου (δήλωση των συναρτήσεων-μελών του)

```
struct date {
    int day, month, year;
    void set(int, int, int);
    void get(int&, int&, int&);
};
```

Κλήση συναρτήσεων-μελών για μια μεταβλητή του νέου τύπου

```
date today;
int d, m, y;
today.set(1,2,2003);
today.get(d,m,y);
cout << "day " << d << ", month " << m << ", year " << y << endl;
```

Ορισμός συναρτήσεων-μελών

Τι απομένει για να ολοκληρώσουμε τον ορισμό του τύπου ώστε οι παραπάνω κλήσεις να έχουν νόημα; Ο ορισμός των συναρτήσεων-μελών. Εδώ, τώρα, τίθεται το ερώτημα. Αφού δεν έχουμε όρισμα που να αντιστοιχεί στη μεταβλητή του τύπου που εφαρμόζεται η συνάρτηση-μέλος (και κατά την κλήση γράφουμε αριστερά, πριν την τελεία .), πώς θα αναφερθούμε σε αυτή μέσα στο σώμα του ορισμού της συνάρτησης-μέλους; Η απάντηση είναι ότι, συνήθως, αναφερόμαστε όχι σε ολόκληρη τη μεταβλητή αλλά σε συγκεκριμένα πεδία της, οπότε αρκεί να αναφέρουμε το όνομά τους. Για το παράδειγμά μας, ο ορισμός της συνάρτησης-μέλους `get` μπορεί να είναι:

```
void get(int& i, int& j, int& k)
{ i = day; j = month; k = year; }
```

Αν αυτός ο ορισμός γραφεί εκτός του ορισμού του τύπου δεν υπάρχει σύνδεση του ονόματος της συνάρτησης με τον τύπο. Θεωρείται, δηλαδή, ότι το όνομα ανήκει στην εμβέλεια (εν δυνάμει) ολόκληρου του προγράμματος. Όμως, είπαμε ότι κατά τον ορισμό του τύπου ορίζεται και μια εμβέλεια με το όνομά του. Οπότε, ο τρόπος να κάνουμε τη σύνδεση είναι να χρησιμοποιήσουμε τον τελεστή επίλυσης εμβέλειας, ::, και να προσδιορίσουμε τον τύπο.

Με βάση τις παραπάνω παρατηρήσεις, με τον παρακάτω κώδικα ολοκληρώνεται ο ορισμός του νέου τύπου και των συναρτήσεων-μελών του.

```
void date::set(int i, int j, int k)
{
    day = i;
    month = j;
    year = k;
}

void date::get(int& i, int& j, int& k)
{
    i = day;
    j = month;
    k = year;
}
```

Στο παραπάνω, όπως είπαμε, τα ονόματα `day`, `month` και `year` στο σώμα μιας συνάρτησης-μέλους αναφέρονται στα αντίστοιχα πεδία της μεταβλητής στην οποία θα εφαρμοστεί η συνάρτηση κατά την κλήση της. Γενικά, μια συνάρτηση-μέλος χρησιμοποιεί τα ονόματα άλλων μελών χωρίς να αναφέρει ρητά κάποιο αντικείμενο, οπότε υπονοείται το αντικείμενο για το οποίο καλείται η συνάρτηση-μέλος. Τι γίνεται, όμως, αν δεν θέλω να αναφερθώ σε συγκεκριμένα πεδία του αντικειμένου; Τι γίνεται, δηλαδή, αν θέλω να αναφερθώ στην τιμή ολόκληρου του αντικειμένου ή στο αντικείμενο το ίδιο ως φυσική οντότητα; Θα δούμε στη συνέχεια ότι στις συναρτήσεις-μέλη υπονοείται ένα όρισμα τύπου δείκτη του τύπου του οποίου είναι μέλος η συνάρτηση — αυτό που “ξεφορτωθήκαμε”, δηλαδή — και το όνομά του είναι μια δεσμευμένη λέξη, η `this`. Θα μιλήσουμε γι’ αυτό αργότερα, αναλυτικά.

Απόκρυψη Πληροφορίας

Έτσι, ολοκληρώθηκε ο ορισμός του τύπου `date` και έγινε σαφής η χρήση του. Όμως, μέχρι στιγμής, δεν έχουμε πετύχει *απόκρυψη πληροφορίας*. Αν υποθέσουμε ότι δεν αφορά τον χρήστη του τύπου `date` η αναπαράσταση των μεταβλητών του τύπου παρά μόνο η χρήση τους μέσω των συναρτήσεων `set` και `get`, ο παραπάνω ορισμός δεν διασφαλίζει ότι ο χρήστης θα περιοριστεί στη χρήση αυτή. Όλα τα ονόματα που έχουν δηλωθεί στον τύπο είναι *δημόσια* (`public`), είναι δηλαδή χρησιμοποιήσιμα από όλο το πρόγραμμα. Για παράδειγμα, ο χρήστης του τύπου μπορεί να έχει πρόσβαση και στα μέλη-δεδομένα του τύπου. Προκειμένου να αποτρέψουμε τον χρήστη του τύπου να έχει πρόσβαση σε κάποια ονόματα μελών του τύπου, τα ονόματα αυτά πρέπει να έχουν δηλωθεί ως *ιδιωτικά* (`private`). Στη C++, αυτό γίνεται με το να έχει προηγηθεί των δηλώσεών τους, η λέξη-κλειδί `private`. Με τον τρόπο αυτό, περιορίζεται η ορατότητά τους και, κατ’ επέκταση η χρήση τους, μόνο μέσα στον τύπο. Αυτό ισχύει για τα ονόματα όλων των δηλώσεων που ακολουθούν τον προσδιορισμό *πρόσβασης/ορατότητας* (`access specifier`) `private`, εκτός αν υπάρξει άλλος προσδιορισμός που θα αλλάξει την ορατότητα. Με τη χρήση του προσδιορισμού ορατότητας `private`, περιορίζουμε την ορατότητα των ονομάτων μόνο μέσα στην εμβέλεια του τύπου και έτσι έχουμε απόκρυψη πληροφορίας.

5.1.1.1 Ολοκληρωμένος Ορισμός και Χρήση του Νέου Τύπου `date`

Στο σημείο αυτό, παραθέτουμε τον κώδικα στον οποίο καταλήξαμε μετά από όλες τις παρατηρήσεις που κάναμε. Ο κώδικας αυτός αναδεικνύει:

- τον ορισμό νέου τύπου, τον `date`, με χρήση `struct`,
- τη σύνδεση των συναρτήσεων που υλοποιούν τη συμπεριφορά του με τον τύπο, δηλώνοντάς τις ως συναρτήσεις-μέλη του τύπου,

- τη διάκριση ορατότητας ονομάτων των μελών του σε δημόσια και ιδιωτικά,
- τον ορισμό των συναρτήσεων-μελών εκτός του τύπου και χρήση του τελεστή επίλυσης εμβέλειας για συσχέτισή τους με τον τύπο,
- τον ορισμό των συναρτήσεων-μελών και την πρόσβαση στα μέλη της μεταβλητής του τύπου στην οποία εφαρμόζεται,
- τον ορισμό μεταβλητών του τύπου και
- την εφαρμογή συναρτήσεων-μελών σε μεταβλητές του τύπου (και χρήση μελών γενικότερα, στην περίπτωση που τα ονόματά τους είναι ορατά και χρησιμοποιήσιμα εκτός του τύπου).

Ο κώδικας είναι ο εξής.

```
#include <iostream>
using namespace std;

struct date {
    void set(int, int, int);
    void get(int&, int&, int&);
private:
    int day, month, year;
};

int main()
{
    date today;
    int d, m, y;

    today.set(31,12,1999);

    // cout << today.month << endl;
    // If the above is uncommented, an error occurs

    today.get(d,m,y);

    cout << "day " << d << ", month " << m << ", year " << y << endl;

    return 0;
}

void date::set(int i, int j, int k)
{
    day = i;
    month = j;
    year = k;
}

void date::get(int& i, int& j, int& k)
{
```

```

    i = day;
    j = month;
    k = year;
}

```

Πειραματιστείτε με τον αποσχολιασμό της γραμμής:

```
// cout << today.month << endl;
```

Αν την αποσχολιάσετε, προκύπτει λάθος κατά τη μεταγλώττιση που αναφέρεται στην προσπάθεια πρόσβασης σε ιδιωτικό μέλος. Αν η γραμμή που περιέχει τον προσδιορισμό ορατότητας `private`: ήταν σχολιασμένη, θα είχατε μια (συντακτικά) σωστή πρόσβαση στο μέλος-δεδομένο (το “τουβλάκι”) `month` της μεταβλητής `today`.

Με τον παραπάνω τρόπο δώσαμε τον πρώτο ορισμό τύπου ορισμένου από τον χρήστη, τον τύπο `date`, στη C++. Ορίσαμε, δηλαδή, την πρώτη μας κλάση. Όμως, δεν χρησιμοποιήσαμε πουθενά στον κώδικά μας μια γλωσσική δομή με αντίστοιχο όνομα. Προκειμένου να ορίσουμε τον νέο μας τύπο, χρησιμοποιήσαμε τη γλωσσική δομή `struct` της C++. Αυτή μας επέτρεπε, πέρα από τις δηλώσεις των μελών-δεδομένων, των δομικών στοιχείων, δηλαδή, των μεταβλητών του τύπου που θα οριστούν, να συμπεριλάβουμε και τις δηλώσεις των συναρτήσεων-μελών που υλοποιούν τη συμπεριφορά του τύπου. Τα ονόματά τους, όμως, είναι δημόσια και εν δυνάμει χρησιμοποιήσιμα από όλο το πρόγραμμα. Αν θέλουμε να περιορίσουμε τη χρήση τους μόνο μέσα στην εμβέλεια του τύπου, αυτό πρέπει να το δηλώσουμε ρητά. Την ίδια λειτουργικότητα μπορούμε να επιτύχουμε με χρήση της γλωσσικής δομής `class` για τον ορισμό του τύπου. Η διαφορά είναι ότι, στην περίπτωση αυτή, τα ονόματα που δηλώνονται στον ορισμό του τύπου είναι ιδιωτικά. Για να γίνουν δημόσια πρέπει να έχει προηγηθεί των δηλώσεών τους ο προσδιορισμός ορατότητας `public` — και να μην έχει ανααιρεθεί η ορατότητά τους από άλλον προσδιορισμό.

Στη συνέχεια, θα χρησιμοποιούμε τη γλωσσική δομή `class`, αντί για την `struct`, προκειμένου να ορίσουμε τους τύπους ορισμένους από τον χρήστη, τις κλάσεις. Παρότι για τη C++ και οι δύο τρόποι προσφέρουν αντίστοιχη λειτουργικότητα, προτείνουμε τον περιορισμό της χρήσης του `struct` για ορισμό τύπων που αντιστοιχούν σε δομήσεις δεδομένων, υιοθετώντας την δυνατότητα ορισμών τύπων που προσφέρεται από τη γλώσσα C. Για τον ορισμό τύπων που αντιστοιχούν σε Αφηρημένους Τύπους Δεδομένων και υλοποιούν κλάσεις, προτείνεται η χρήση της γλωσσικής δομής `class`. Τα παραπάνω, πλέον, δείχνει να προτείνονται ευρύτερα από τους εμπλεκόμενους στον χώρο.

Στη συνέχεια, προκειμένου να οριστεί ένας νέος τύπος με τη μορφή κλάσης, θα χρησιμοποιούμε αποκλειστικά τη γλωσσική δομή `class`, οπότε η διευκρίνιση θα παραλείπεται.

5.2 Ορισμός Νέου Τύπου Κλάσης

Η γενική μορφή ορισμού ενός νέου τύπου (μιας κλάσης) είναι η εξής:

```

class <όνομα κλάσης> {
    <δηλώσεις_1>
public:
    <δηλώσεις_2>
};

```

Αρχικά δίνουμε το όνομα του νέου τύπου ως όνομα της κλάσης, <όνομα κλάσης>, και ακολουθούν οι δηλώσεις των μελών της κλάσης. Στη γενική περίπτωση, τα μέλη μπορούν να είναι μεταβλητές (μέλη-δεδομένα), συναρτήσεις (συναρτήσεις-μέλη) ή ακόμα και νέες κλάσεις. Εδώ, να τονίσουμε το εξής: μια κλάση ορίζει και μια νέα εμβέλεια. Η εμβέλεια αυτή έχει όνομα το οποίο είναι το όνομα της κλάσης. Τα ονόματα των μελών μιας κλάσης ανήκουν στην εμβέλεια της κλάσης και μπορούμε να περιορίσουμε τη χρήση τους μόνο μέσα στην εμβέλεια της κλάσης (με την εξαίρεση “φιλικών” εμβελειών συναρτήσεων και άλλων κλάσεων). Αν ο ορισμός

μιας κλάσης ακολουθεί την παραπάνω γενική μορφή, τα ονόματα που δηλώνονται από τις δηλώσεις στο μέρος `<δηλώσεις_1>` δεν μπορούν να χρησιμοποιηθούν εκτός της εμβέλειας της κλάσης. Αντίθετα, τα ονόματα που δηλώνονται από τις δηλώσεις στο μέρος `<δηλώσεις_2>` μπορούν να χρησιμοποιηθούν από οποιοδήποτε μέρος του προγράμματος. Ο λόγος είναι ότι έχει προηγηθεί των δηλώσεων αυτών ο προσδιορισμός ορατότητας `public`.

Εν ολίγοις, αν μια κλάση έχει οριστεί χρησιμοποιώντας τη γλωσσική δομή `class`, τα ονόματα που δηλώνονται χωρίς να έχει προηγηθεί των δηλώσεών τους κάποιος προσδιορισμός ορατότητας είναι *ιδιωτικά* και είναι ορατά μόνο μέσα στην εμβέλεια της κλάσης. Προκειμένου να προσδιορίσουμε τα δημόσια ονόματα μελών της κλάσης, οι δηλώσεις τους πρέπει να ακολουθούν τον προσδιορισμό ορατότητας `public`.

Στα πρώτα κεφάλαια, συζητήσαμε και αναδείξαμε, μέσω παραδείγματος στο Κεφάλαιο 2, την ανάγκη αφαίρεσης στα δεδομένα και απόκρυψης πληροφορίας προκειμένου να διαχωριστεί το μέρος του ορισμού του τύπου που είναι καθοριστικό για τον τύπο και αντιστοιχεί στη συμπεριφορά του και το μέρος του ορισμού του τύπου που περιέχει τις λεπτομέρειες υλοποίησής του. Τα ονόματα των μελών που συμμετέχουν στο πρώτο οφείλουν να έχουν δηλωθεί ως δημόσια και τα ονόματα των μελών που συμμετέχουν στο δεύτερο οφείλουν να έχουν δηλωθεί ως ιδιωτικά. Δεδομένου ότι είναι πιο καθαρό να φαίνονται πρώτα τα ονόματα μέσω των οποίων μπορούμε να μεταχειριστούμε τις μεταβλητές του νέου τύπου, συχνά αυτά δηλώνονται στην αρχή του τύπου, προκειμένου να φαίνεται καθαρά η *διεπαφή* του τύπου. Στην περίπτωση αυτή, κατά τον ορισμό ενός νέου τύπου στη C++ μέσω `class`, η γενική μορφή του ορισμού μιας κλάσης γίνεται:

```
class <όνομα κλάσης> {
public:
    <δηλώσεις_2>
private:
    <δηλώσεις_1>
};
```

Τι, όμως, συμπεριλαμβάνουμε στον ορισμό μιας κλάσης; Ορίζοντας μια κλάση πρέπει να δηλώσουμε, κατ' αρχήν, τα εξής:

- τα *δομικά στοιχεία* που κτίζουν κάθε αντικείμενο της κλάσης — άτυπα, τα λέμε τα “τουβλάκια” (building blocks) του αντικειμένου της κλάσης,
- μια σειρά από συναρτήσεις-μέλη που αναθέτουν/μεταβάλλουν τις τιμές στα δομικά στοιχεία των αντικειμένων, οπότε είναι *μεταβάλλουσες*,
- μια σειρά από συναρτήσεις-μέλη που ανακτούν τις τιμές αυτές, τις *αναγνώστριες* — στην περίπτωση αυτή καλό είναι να τις προσδιορίζουμε ως *σταθερές* με χρήση της λέξης-κλειδιού `const`.

Με βάση τις παραπάνω παρατηρήσεις, ο ορισμός του τύπου `date` χρησιμοποιώντας τη γλωσσική δομή `class`, μπορεί να γραφεί ως εξής:

```
class date {
public:
    void set(int, int, int);
    void get(int&, int&, int&) const;
private:
    int day, month, year;
};
```

Προσέξτε ότι προκειμένου να δηλώσουμε ότι η συνάρτηση-μέλος `get` υλοποιεί αναγνώστρια, το πρωτότυπό της γίνεται:

```
void get(int&, int&, int&) const
```

ακολουθείται, δηλαδή, από τη λέξη-κλειδί `const` προκειμένου να δηλωθεί ότι το αντικείμενο στο οποίο θα εφαρμοστεί δεν αλλάζει τιμή. Συντακτικά, δείχνει περιέργο αλλά δεν είναι: σε κάθε συνάρτηση-μέλος υπονοείται ένα επιπλέον όρισμα, αυτό που αντιστοιχεί (σε δείκτη) στη μεταβλητή της κλάσης και το οποίο “έξαφανίσαμε”. Άρα, δεν έχουμε θέση στα ορίσματα για τον προσδιορισμό της μεταβλητής αυτής ως σταθερά. Έτσι, ο τρόπος που ακολούθησε η C++ για τον προσδιορισμό αυτόν είναι ο προσδιορισμός `const` να γραφεί εκτός ορισμάτων, συμπληρώνοντας το πρωτότυπο. Προσέξτε: πρέπει να γίνει η αντίστοιχη αλλαγή και στον ορισμό της συνάρτησης-μέλος `get` και αυτός να γίνει:

```
void date::get(int& i, int& j, int& k) const
{
    i = day;
    j = month;
    k = year;
}
```

Έχοντας ορίσει μια κλάση, αυτό που μπορούμε να θεωρήσουμε ότι έχουμε επιτύχει είναι ότι επεκτείναμε το σύνολο των τύπων που είναι διαθέσιμοι για να αποτελέσουν τον τύπο μιας μεταβλητής που ορίζεται, στη συνέχεια. Μια μεταβλητή τύπου ορισμένου από τον χρήστη — ένα αντικείμενο της κλάσης — μπορεί να έχει οποιαδήποτε φύση, σύμφωνα με την κατηγοριοποίηση μεταβλητών που κάναμε στην Παράγραφο 3.3.3. Επίσης, μπορούμε να χρησιμοποιούμε μια κλάση ως τύπο για μια μεταβλητή τύπου δείκτη. Προκειμένου να έχουμε πρόσβαση σε ένα μέλος-δεδομένο ενός αντικειμένου μέσω δείκτη σε αυτό (αν οι δηλώσεις ορατότητας το επιτρέπουν), χρησιμοποιούμε την ίδια σύνταξη με την πρόσβαση στα πεδία ενός τύπου ορισμένου μέσω `struct`, δηλαδή μέσω του τελεστή `->`. Για παράδειγμα, αν έχουμε μια μεταβλητή τύπου δείκτη ορισμένη ως

```
date* pd;
```

τότε, αν επιτρεπόταν η πρόσβαση στο μέλος-δεδομένο `day`, θα γράφαμε

```
pd->day
```

Η ίδια σύνταξη ακολουθείται και για την εφαρμογή συναρτήσεων-μελών σε αντικείμενο μιας κλάσης μέσω δείκτη του τύπου της κλάσης, π.χ.

```
pd->set(12,12,1912);
```

Παράδειγμα: Στο παρακάτω βλέπουμε το παράδειγμα του νέου τύπου `date`, ορισμένο ως κλάση και περιπτώσεις χρήσης του.

```
#include <iostream>
using namespace std;

class date {
public:
    void set(int, int, int);
    void get(int&, int&, int&) const;
private:
    int day, month, year;
};

date TODAY; // Global variable

int main()
{
```

```

date someday;           // Local variable

int d, m, y;

someday.set(18,3,2005);
someday.get(d,m,y);
cout << "day " << d << ", month " << m << ", year " << y << endl;

date nextday;           // Another instance
nextday.set(19,3,2005);
nextday.get(d,m,y);
cout << "day " << d << ", month " << m << ", year " << y << endl;

////////////////////////////////////

TODAY.set(19,11,2019);
TODAY.get(d,m,y);
cout << "day " << d << ", month " << m << ", year " << y << endl;

////////////////////////////////////

date* psomeday = &someday;    // Pointer to local date

cout << "The size of a variable of type int " << sizeof(int) << endl;
cout << "The size of a variable of type date "
    << sizeof(someday) << endl;
cout << "The size of a variable of type pointer to date "
    << sizeof(psome day) << endl;

psomeday->get(d,m,y);
cout << "day " << d << ", month " << m << ", year " << y << endl;
(*psomeday).get(d,m,y);
cout << "day " << d << ", month " << m << ", year " << y << endl;

date* pbreakdate = new date(); // Pointer to date in heap
                                // (dynamic allocation)
pbreakdate->set(23,12,2019);
pbreakdate->get(d,m,y);
cout << "day " << d << ", month " << m << ", year " << y << endl;

delete pbreakdate;
}

void date::set(int i, int j, int k)
{
    day = i;
    month = j;
    year = k;
}
    
```

```

void date::get(int& i, int& j, int& k) const
{
    i = day;
    j = month;
    k = year;
}

```

Στο παράδειγμα αυτό, πριν από οτιδήποτε άλλο, ορίζεται μια καθολική/στατική μεταβλητή τύπου `date`, η `TODAY`.

Μέσα στο σώμα της συνάρτησης `main`, αρχικά ορίζεται μια τοπική/αυτόματη μεταβλητή τύπου `date`, η `someday`. Με τις εντολές:

```

someday.set(18,3,2005);
someday.get(d,m,y);

```

εφαρμόζονται οι συναρτήσεις-μέλη `set` και `get` της κλάσης `date` στη μεταβλητή/αντικείμενο `someday`. Με την κλήση της `set` δίνουμε μια τιμή στη μεταβλητή η οποία, στην προκειμένη περίπτωση, είναι η αρχική τιμή του.

Με τις εντολές:

```

date nextday;           // Another instance
nextday.set(19,3,2005);
nextday.get(d,m,y);

```

ορίζεται μια άλλη τοπική μεταβλητή τύπου `date`, η `nextday`, και στη συνέχεια έχουμε κλήση των συναρτήσεων-μελών `set` και `get` της κλάσης `date` για τη μεταβλητή αυτή.

Με τις εντολές που ακολουθούν:

```

TODAY.set(19,11,2019);
TODAY.get(d,m,y);

```

οι συναρτήσεις-μέλη `set` και `get` της κλάσης `date` εφαρμόζονται στην καθολική μεταβλητή `TODAY`.

Ακολουθεί ορισμός μιας μεταβλητής τύπου δείκτη σε `date`, η `psomeday`. Αυτή αρχικοποιείται με τη διεύθυνση της τοπικής μεταβλητής `someday`, άρα, “δείχνει” σε τοπική μεταβλητή.

```
date* psomeday = &someday; // Pointer to local date
```

Επομένως, μπορούμε να προσπελάσουμε τη μεταβλητή `someday` είτε απευθείας χρησιμοποιώντας το όνομά της, όπως κάναμε προηγουμένως, είτε μέσω του δείκτη `psomeday`, π.χ.

```
psomeday->get(d,m,y);
```

ή, ισοδύναμα,

```
(*psomeday).get(d,m,y);
```

Και στις δύο περιπτώσεις, εφαρμόζεται η συνάρτηση-μέλος `get` της κλάσης `date` στη μεταβλητή που “δείχνει” ο δείκτης `psomeday`, δηλαδή την `someday`.

Προσέξτε τις εντολές εκτύπωσης που μεσολαβούν. Αυτές εκτυπώνουν μεγέθη μεταβλητών με χρήση του τελεστή `sizeof` και έχουν ως στόχο να κάνουν καθαρή τη διαφορά των μεγεθών του δείκτη σε σχέση με μεταβλητή του τύπου του. Καθότι ο τύπος `date` έχει τρία μέλη-δεδομένα και τα οποία είναι τύπου `int`, οι εκτυπώσεις αυτές δίνουν την εικόνα και της σχέσης αυτής.

Τέλος, με την εντολή:

```
date* pbreakdate = new date();
```

ορίζεται μια ακόμα μεταβλητή τύπου δείκτη σε `date`, η `pbreakdate`, η οποία αρχικοποιείται με τη διεύθυνση ενός αντικειμένου που δημιουργείται δυναμικά στον σωρό και επιστρέφεται από τη συνάρτηση δημιουργίας

του, new. Το αντικείμενο αυτό παίρνει (αρχική) τιμή εφαρμόζοντας τη συνάρτηση-μέλος set της κλάσης date μέσω του δείκτη pbreakdate,

```
pbreakdate->set(23,12,2019);
```

Η τιμή αυτή ανακτάται αμέσως μετά με εφαρμογή της συνάρτησης-μέλους get με τον ίδιο τρόπο,

```
pbreakdate->get(d,m,y);
```

Με βάση τον κώδικα του παραδείγματος, ο μόνος τρόπος να χειριστούμε το αντικείμενο αυτό είναι μέσω της μεταβλητής pbreakdate.

Με την εντολή

```
delete pbreakdate;
```

καταστρέφουμε το αντικείμενο. Η μεταβλητή pbreakdate δεν έχει καταστραφεί, απλά η τιμή της αφορά διεύθυνση που πλέον δεν ανήκει σε χώρο που ελέγχει το πρόγραμμα. Η ίδια η μεταβλητή pbreakdate θα καταστραφεί όταν τελειώσει η εκτέλεση της συνάρτησης main οπότε και θα αφαιρεθεί το πλαίσιο (frame) της main από τη στοίβα.

Η εικόνα της μνήμης πριν την εκτέλεση της εντολής

```
delete pbreakdate;
```

αφαιρετικά, φαίνεται στο Σχήμα 5.2. Στο σχήμα αυτό, καθώς και σε αντίστοιχα σχήματα που θα παραθέσουμε στη συνέχεια, παρουσιάζουμε μια εικονική εκδοχή όσον αφορά τους χώρους της μνήμης και άλλα τεχνικά στοιχεία χαμηλού επιπέδου, προκειμένου να εστιάσουμε στα στοιχεία που αφορούν το δικό μας θέμα. Οπότε, απεικονίζουμε (δισδιάστατα) τους χώρους της μνήμης. Στο Σχήμα 5.2 απεικονίζεται το πλαίσιο της συνάρτησης main και βλέπουμε:

τη στοίβα (stack), όπου φυλάσσονται οι αυτόματες μεταβλητές:

- someday και nextday, τύπου date,
- οι ακέραιες μεταβλητές d, m και y και
- οι μεταβλητές τύπου δείκτη (σε date) psomeday και pbreakdate.

τον στατικό χώρο (static), όπου φυλάσσεται η στατική (καθολική) μεταβλητή TODAY.

τον σωρό (heap), όπου δεσμεύεται δυναμικά χώρος και δημιουργείται ένα αντικείμενο της κλάσης/τύπου date.

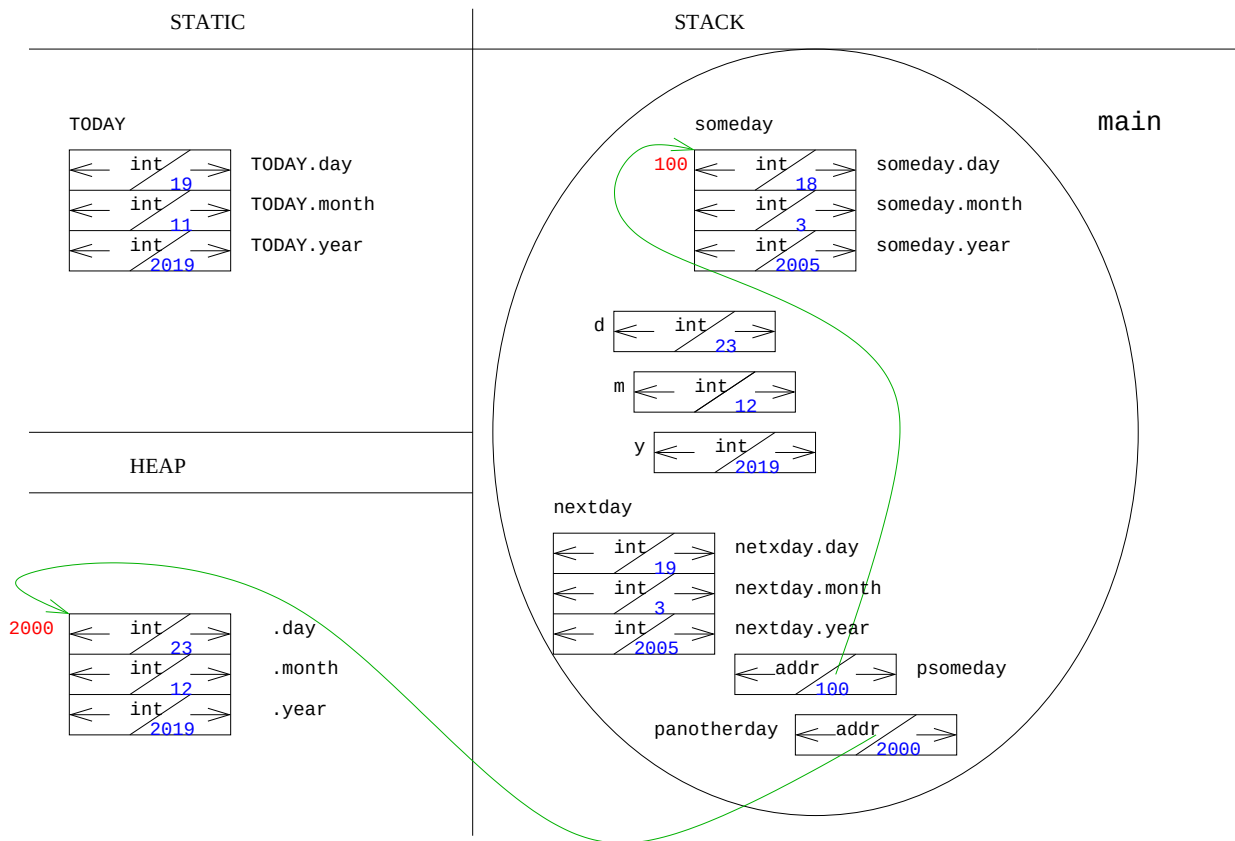
Δίπλα στις θέσεις μνήμης εμφανίζονται με μαύρο χρώμα τα ονόματα μέσω των οποίων μπορούν άμεσα να προσπελαστούν. Με κόκκινο χρώμα εμφανίζονται υποθετικοί αριθμοί που αντιστοιχούν στις διευθύνσεις τους. Εσωτερικά, αναγράφεται ο τύπος τους και με μπλε χρώμα η τιμή τους. Στην περίπτωση που έχουμε μεταβλητές τύπου δείκτη, για να γίνει πιο άμεση η συσχέτιση της τιμής τους με τη μεταβλητή στην οποία αντιστοιχεί η διεύθυνση αυτή, η συσχέτιση απεικονίζεται με ένα πράσινο βέλος.

5.2.1 Ο Δείκτης this

Επανερχόμαστε για λίγο στις συναρτήσεις-μέλη. Να υπενθυμίσουμε ότι το πρωτότυπο μιας συνάρτησης η οποία δεν αποτελεί μέλος μιας κλάσης οφείλει να έχει ορίσματα για όλα τα δεδομένα τα οποία χρειάζεται από το περιβάλλον από το οποίο καλείται. Έτσι, προκειμένου να αναφερθεί σε κάποια μεταβλητή ορίσματος, αρκεί να χρησιμοποιήσει το όνομα της παραμέτρου. Στις συναρτήσεις-μέλη μιας κλάσης, όμως, το όρισμα που αντιστοιχεί στο αντικείμενο της κλάσης παραλείπεται. Δηλαδή, για να ανατρέξουμε στο αρχικό παράδειγμα της “Ημερομηνίας”, στην υλοποίησή του τύπου date à la C, η ανάθεση τιμής των μεταβλητών τύπου date υλοποιείτο από μια συνάρτηση με πρωτότυπο:

```
void set_date(date* dt, int d, int m, int y)
```

και υλοποίηση:



Σχήμα 5.2: Η εικόνα των μεταβλητών (αφαιρετικά) στη μνήμη

```
void set_date(date* dt, int d, int m, int y) {
    dt->day = d; dt->month = m; dt->year = y;
}
```

Η αντίστοιχη συνάρτηση-μέλος στην υλοποίηση του τύπου στη C++ έχει πρωτότυπο:

```
void date::set(int i, int j, int k)
```

Παραλείψαμε, δηλαδή, το όρισμα που αντιστοιχεί στο αντικείμενο της κλάσης `date` πάνω στο οποίο θα εφαρμοστεί η `set` κατά την κλήση της. Στην κλήση της, έχουμε πει ότι η μεταβλητή θα δηλωθεί προθεματικά, π.χ. `TODAY.set(26, 5, 2022)`. Αφού, όμως, το όρισμα παραλείπεται, πώς θα αναφερθούμε σε αυτό στον ορισμό της συνάρτησης-μέλους; Από τον ορισμό των συναρτήσεων-μελών της κλάσης `date`, είδαμε την πρόσβαση στη μεταβλητή μέσω των ονομάτων των μελών-δεδομένων που την απαρτίζουν, π.χ.

```
void date::set(int i, int j, int k)
{
    day = i;
    month = j;
    year = k;
}
```

Τι γίνεται, όμως, αν η πρόσβαση μέσω των ονομάτων των μελών-δεδομένων δεν επαρκεί ή δεν είναι κατάλληλη;

Η απάντηση είναι ότι στο σώμα μιας συνάρτησης-μέλους μπορούμε να χρησιμοποιήσουμε τη λέξη-κλειδί `this` θεωρώντας ότι είναι ένας δείκτης στο αντικείμενο πάνω στο οποίο εφαρμόζεται η συνάρτηση-μέλος. Δηλαδή, μπορούμε να υποθέσουμε ότι κάθε συνάρτηση-μέλος έχει ένα επιπλέον όρισμα το οποίο υπονοείται.

Αυτό είναι τύπου δείκτη στον τύπο της κλάσης. Ο δείκτης αυτός έχει όνομα `this`. Κατά την κλήση της συνάρτησης-μέλους, ο δείκτης αυτός παίρνει ως τιμή τη διεύθυνση του αντικειμένου στο οποίο εφαρμόζεται η συνάρτηση-μέλος. Μπορούμε, λοιπόν, να ξαναγράψουμε τον παραπάνω ορισμό της συνάρτησης-μέλους `set` ως εξής:

```
void date::set(int i, int j, int k)
{
    this->day = i;
    this->month = j;
    this->year = k;
}
```

Η αναλογία με τον ορισμό à la C είναι εμφανής.

Ουσιαστικά, οι συναρτήσεις-μέλη λειτουργούν σαν να έχουν κρυμμένο ένα όρισμα τύπου δείκτη ο οποίος για μια κλάση `X` μπορεί να θεωρηθεί ότι είναι τύπου `X *const this`.

Η χρήση του δείκτη `this` στο σώμα του ορισμού μιας συνάρτησης-μέλους έχει νόημα σε διάφορες περιπτώσεις. Για παράδειγμα,

- όταν μια συνάρτηση-μέλος παίρνει ως όρισμα μεταβλητές της ίδιας της κλάσης (π.χ. `void append(list*)`; για συνένωση λιστών), για ευκρίνεια στον κώδικα,
- υπάρχει ανάγκη για πρόσβαση στον δείκτη ή/και σε ολόκληρο το αντικείμενο και χειρισμού του μέσα από το σώμα της συνάρτησης,
- υπάρχει ανάγκη επιστροφής της τιμής του αντικειμένου ή της διεύθυνσής του από τη συνάρτηση.

Σε περίπτωση που μια συνάρτηση-μέλος δεν πρόκειται να τροποποιήσει το αντικείμενο για το οποίο καλείται, τότε στη δήλωσή της, όπως προαναφέραμε, συμπεριλαμβάνουμε τη λέξη-κλειδί `const`, π.χ. `char* get_name() const`; Σ' αυτήν την περίπτωση, για μια κλάση `X`, ο τύπος του ορίσματος των συναρτήσεων-μελών που υπονοείται, μπορεί να θεωρηθεί ότι είναι `const X *const this`.

Απλό παράδειγμα χρήσης του δείκτη `this`:

Ας δούμε ένα παράδειγμα με χρήση του δείκτη `this` στο παράδειγμα του τύπου `date`:

```
#include <iostream>
using namespace std;

class date {
private:
    int day, month, year;
public:
    void set(int, int, int);
    void get(int&, int&, int&) const;
};

int main()
{
    date today;
    int d, m, y;
    today.set(18,3,2005);
    today.get(d,m,y);
    cout << "day " << d << ", month " << m << ", year " << y << endl;
```

```

    return 0;
}

void date::set(int i, int j, int k)
{
    this->day = i;
    this->month = j;
    this->year = k;
}

void date::get(int& i, int& j, int& k) const
{
    // set(10,1,100);      // What a silly thing to do!
                          // (erase "const" to compile)
    // this->set(10,1,100); // Same as above.
    i = this->day;
    j = this->month;
    k = this->year;
}

```

Το παραπάνω παράδειγμα, επιπλέον, δείχνει ότι μέσα στα σώματα των συναρτήσεων-μελών μπορείτε να καλέσετε (και άλλες) συναρτήσεις-μέλη. Αν παραλείψετε το αντικείμενο, οι συναρτήσεις εφαρμόζονται στη μεταβλητή που δείχνει ο δείκτης `this` (του ορίσματος) της συνάρτησης-μέλους. Για παράδειγμα, η κλήση:

```
set(10,1,100);
```

του παραπάνω κώδικα είναι ισοδύναμη με την κλήση:

```
this->set(10,1,100);
```

Παράδειγμα χρήσης του δείκτη `this` για ευκρίνεια:

Στο παραπάνω παράδειγμα, η χρήση του `this` κάνει σαφές ότι πρόκειται για χρήση μελών της κλάσης του αντικειμένου στο οποίο εφαρμόζεται η συνάρτηση-μέλος (και στο οποίο “δείχνει” ο `this`). Παρ’ όλα αυτά, δεν είναι τόσο απαραίτητη από πλευράς αναγνωσιμότητας, όσο στο παρακάτω παράδειγμα. Συγκεκριμένα, στο παράδειγμα αυτό, προσθέσαμε στον τύπο `date` άλλη μια συνάρτηση-μέλος, την

```
void copy_values_from(const date& anotherdt)
```

η οποία αντιγράφει την τιμή της μεταβλητής τύπου `date` που δέχεται ως όρισμα, την `anotherdt`, στη μεταβλητή τύπου `date` πάνω στην οποία εφαρμόζεται. Στην περίπτωση αυτή, είναι σαφώς πιο κομψό το να διακρίνεται εμφανώς και το κάθε αντικείμενο, μια που τα ονόματα των μελών είναι κοινά.

```

#include <iostream>
using namespace std;

class date {
public:
    void set(int, int, int);
    void get(int&, int&, int&) const;
    void copy_values_from(const date& anotherdt);
private:
    int day, month, year;
};

```

```

int main()
{
    date today;
    date* ptoday = &today;

    int d, m, y;

    today.set(18,3,2005);
    today.get(d,m,y);
    cout << "day " << d << ", month " << m << ", year " << y << endl;

    ptoday->get(d,m,y);
    cout << "day " << d << ", month " << m << ", year " << y << endl;

    date newdate;
    newdate.set(1,2,3);
    today.copy_values_from(newdate);

    today.get(d,m,y);
    cout << "day " << d << ", month " << m << ", year " << y << endl;

    return 0;
}

void date::set(int i, int j, int k)
{
    day = i;
    month = j;
    year = k;
}

void date::get(int& i, int& j, int& k) const
{
    i = day;
    j = month;
    k = year;
}

void date::copy_values_from(const date& anotherdt)
{
    /* All the following have the same behavior */
    /*
        day = anotherdt.day;
        month = anotherdt.month;
        year = anotherdt.year;
    */

    ///*
    this->day = anotherdt.day;

```

```

    this->month = anotherdt.month;
    this->year = anotherdt.year;
    /**/

    /*
    (*this).day = anotherdt.day;
    (*this).month = anotherdt.month;
    (*this).year = anotherdt.year;
    */
}

```

Στο σώμα του ορισμού της συνάρτησης-μέλους `copy_values_from` είναι αποσχολιασμένος ο κώδικας:

```

this->day = anotherdt.day;
this->month = anotherdt.month;
this->year = anotherdt.year;

```

με τον οποίο γίνεται η αντιγραφή των τιμών των πεδίων της μεταβλητής του ορίσματος στα αντίστοιχα της μεταβλητής στην οποία εφαρμόζεται η συνάρτηση-μέλος. Στον κώδικα αυτόν, γίνεται χρήση του δείκτη `this`. Ο κώδικας είναι ισοδύναμος τόσο με τον σχολιασμένο κώδικα που δεν τον αναφέρει ρητά, όσο και με τον κώδικα που κάνει ρητά το dereference του δείκτη.

Παρ' όλα αυτά, ακόμα και στο παραπάνω παράδειγμα, η χρήση του `this` απλά συμβάλλει στην ευκρίνεια του κώδικα. Δεν είναι απαραίτητη. Θα δούμε, όμως, ότι υπάρχουν περιπτώσεις που η χρήση του είναι αναπόφευκτη διότι δεν μπορούμε να εκφράσουμε διαφορετικά αυτό που πρέπει να γίνει στη διαδικασία που περιγράφεται από τον ορισμό της συνάρτησης-μέλους. Θα δούμε ένα παράδειγμα μιας τέτοιας περίπτωσης αργότερα κατά τον (επανα)ορισμό του τελεστή ανάθεσης (assignment operator) για τις μεταβλητές ενός νέου τύπου.

5.3 Ορισμός του Τύπου ``Φοιτητής''

Ας προχωρήσουμε με το δεύτερο παράδειγμα τύπου, μέσω του οποίου θα συζητήσουμε τα διάφορα θέματα προς αντιμετώπιση όταν ορίζουμε έναν νέο τύπο. Ο τύπος αυτός, όπως είπαμε, είναι ο τύπος του "Φοιτητή", που αντιστοιχεί στην έννοια του φοιτητή. Ενδιαφερόμαστε για το ονοματεπώνυμο ενός φοιτητή, τον αριθμό μητρώου του και το έτος σπουδών του. Η λειτουργικότητα του τύπου αφορά:

- τη δυνατότητα αναθέσεων τιμών για το ονοματεπώνυμο, τον αριθμό μητρώου και το έτος σπουδών, κατά περίπτωση,
- τη δυνατότητα ανάκτησης των παραπάνω τιμών, πάλι κατά περίπτωση.

Οπότε, ορίζουμε, μια κλάση `Student` με συναρτήσεις-μέλη με δημόσια ορατότητα οι οποίες θα αντιστοιχούν στη συμπεριφορά του τύπου. Έστω ότι τα ονόματά τους είναι:

`set_name`: για να γίνεται ανάθεση του ονόματος του αντικειμένου της κλάσης `Student`,

`set_no`: για να γίνεται ανάθεση του αριθμού μητρώου του αντικειμένου της κλάσης `Student`,

`set_year_of_studies`: για να γίνεται ανάθεση του έτους σπουδών του αντικειμένου της κλάσης `Student`,

`get_name`: για να γίνεται ανάκτηση του ονόματος του αντικειμένου της κλάσης `Student`,

`get_no`: για να γίνεται ανάκτηση του αριθμού μητρώου του αντικειμένου της κλάσης `Student` και

`get_year_of_studies`: για να γίνεται ανάκτηση του έτους σπουδών του αντικειμένου της κλάσης `Student`.

Στη συνέχεια, πρέπει να δούμε τις λεπτομέρειες υλοποίησης του τύπου. Αυτές, κατ' αρχήν, αφορούν:

- την αναπαράσταση των αντικειμένων της κλάσης και
- την υλοποίηση των συναρτήσεων-μελών, με δεδομένη την αναπαράσταση που έχει αποφασιστεί.

Όσον αφορά το πρώτο, έστω ότι αποφασίζουμε ότι η κλάση θα έχει τρία μέλη-δεδομένα, τα οποία θα αποτελούν τα δομικά στοιχεία κάθε αντικειμένου της κλάσης, τα:

`name`: μια συμβολοσειρά που αντιστοιχεί στο ονοματεπώνυμο του φοιτητή,

`no`: ένας αριθμός που αντιστοιχεί στον αριθμό μητρώου του φοιτητή και

`year_of_studies`: ένας αριθμός που αντιστοιχεί στο έτος σπουδών του φοιτητή.

Όσον αφορά τους τύπους των παραπάνω μεταβλητών, ο τύπος των μελών-δεδομένων `no` και `year_of_studies` ας είναι `int`, για λόγους απλότητας. Ο τύπος της μεταβλητής `name`, ως συμβολοσειρά, στη γενική περίπτωση προτείνεται να είναι `string` που είναι τύπος που προσφέρεται από την καθιερωμένη βιβλιοθήκη της C++, είναι δηλαδή τύπος βιβλιοθήκης. Όμως, στο παράδειγμά μας θα ακολουθήσουμε μια πιο “παραδοσιακή” αναπαράσταση συμβολοσειράς, à la C, τον τύπο `char*` και θα δεσμεύσουμε δυναμικά τον χώρο για την τιμή της. Ο λόγος είναι ότι ο τύπος δείκτη θα μας εξυπηρετήσει να κάνουμε πιο κατανοητά τα προβλήματα που θα συζητήσουμε αργότερα. Τα ονόματα των μελών που αποτελούν αποφάσεις υλοποίησης οφείλουν να είναι ιδιωτικά, επομένως οι δηλώσεις τους πρέπει να ακολουθούν προσδιορισμό ορατότητας `private`.

Με βάση τις παραπάνω παρατηρήσεις, ο ορισμός της κλάσης `Student` γίνεται ως εξής:

```
class Student {
private:
    char* name;           // +
    int no;               // |      Building blocks
    int year_of_studies;  // +
public:
    void set_name(const char*); // +
    void set_no(int);           // |      Mutators
    void set_year_of_studies(int); // +
    char* get_name() const;     // +
    int get_no() const;         // |      Accessors
    int get_year_of_studies() const; // +
};
```

Παρατηρήσετε τη διάκριση των συναρτήσεων-μελών που ανακτούν την τιμή του αντικειμένου: έχουν προσδιοριστεί ως `const`.

Ένα παράδειγμα της χρήσης του τύπου `Student` φαίνεται στην παρακάτω συνάρτηση `main`.

```
int main()
{
    Student s;
    int year, no;

    s.set_name("FName LName");
```

```

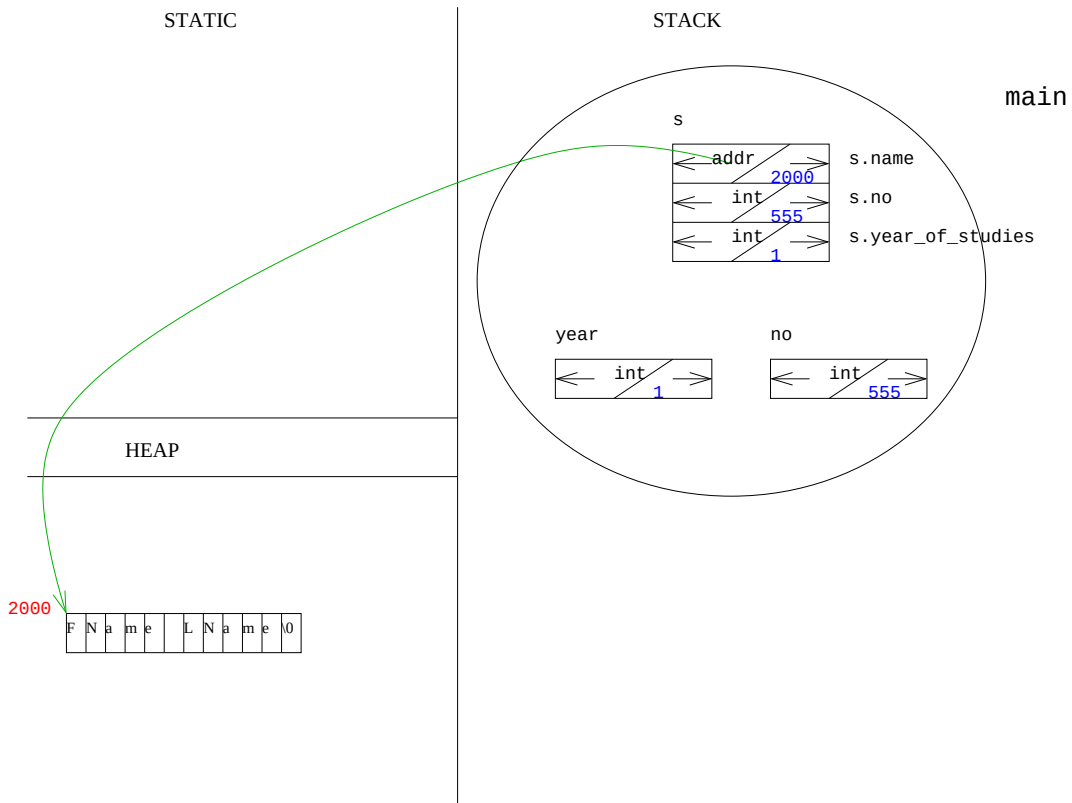
s.set_no(555);
s.set_year_of_studies(1);

year = s.get_year_of_studies();
no = s.get_no();

return 0;
}

```

Η αφαιρετική εικόνα της μνήμης πριν την επιστροφή από τη συνάρτηση `main` φαίνεται στο Σχήμα 5.3.



Σχήμα 5.3: Η εικόνα των μεταβλητών στην περίπτωση της κλάσης “Student” (αφαιρετικά) στη μνήμη

5.3.1 Οργάνωση του Κώδικα σε Αρχεία

Στο σημείο αυτό θα συζητήσουμε το πώς οργανώνουμε τον κώδικά μας σε αρχεία, στην περίπτωση που περιλαμβάνει τύπους ορισμένους από τον χρήστη με τη μορφή κλάσεων. Για λόγους ευχρηστίας και επαναχρησιμοποίησης, διαχωρίζουμε τον κώδικα ορισμού μιας κλάσης από τον κώδικα χρήσης της, γράφοντάς τους σε διαφορετικά αρχεία. Ο κώδικας ορισμού μιας κλάσης αποτελείται από τον κώδικα ορισμού/δήλωσης της κλάσης και τον κώδικα ορισμού των μελών της, όποτε αυτό είναι απαραίτητο να γίνεται εκτός ορισμού της κλάσης. Πριν από τη χρήση της κλάσης, πρέπει να έχει προηγηθεί η δήλωσή της. Συνεπώς, καλό είναι ο ορισμός της κλάσης να γραφεί σε ένα αρχείο επικεφαλίδας το οποίο να συμπεριλαμβάνεται σε κάθε αρχείο που ο κώδικας του χρησιμοποιεί τα ονόματά της. Οι ορισμοί των συναρτήσεων-μελών — γενικότερα, ορισμοί οντοτήτων που είναι μέλη της κλάσης — πρέπει να εμφανίζονται άπαξ σε όλο τον κώδικα του προγράμματος. Άρα, όπως συνήθως κάνουμε στις περιπτώσεις αυτές ήδη από τα προγράμματα της C, οι ορισμοί των μελών αυτών της κλάσης θα πρέπει να γραφούν σε ένα αρχείο πηγαίου κώδικα το οποίο να συμμετέχει κατά τη μεταγλώττιση.

Παράδειγμα: Στο παρακάτω παράδειγμα έχουμε ένα πρόγραμμα που συμπεριλαμβάνει τον πλήρη ορισμό της κλάσης Student και αναδεικνύει έναν διαχωρισμό του κώδικα σε αρχεία. Τα αρχεία αυτά έχουν ως εξής:

- Ορισμός νέου τύπου:

- student.h: ορισμός του νέου τύπου (αρχείο επικεφαλίδας)

```
// File: student.h
class Student {
private:
    char* name;
    int no;
    int year_of_studies;
public:
    void set_name(const char*);
    void set_no(int);
    void set_year_of_studies(int);
    char* get_name() const;
    int get_no() const;
    int get_year_of_studies() const;
};
```

- student.cpp: ορισμός των συναρτήσεων-μελών του νέου τύπου (αρχείο κώδικα)

```
// File: student.cpp
#include <cstring>
#include <iostream>
#include "student.h"
using namespace std;

void Student::set_name(const char* nam)
{
    name = new char[strlen(nam)+1];
    strcpy(name, nam);
}

void Student::set_no(int n)
{
    no = n;
}

void Student::set_year_of_studies(int y)
{
    year_of_studies = y;
}

char* Student::get_name() const
{
    return name;
}
```

```

int Student::get_no() const
{
    return no;
}

int Student::get_year_of_studies() const
{
    return year_of_studies;
}

```

- classtest.cpp: χρήση του νέου τύπου στον ορισμό μεταβλητών

```

// File: classtest.cpp
#include <iostream>
#include "student.h"
using namespace std;

int main()
{
    Student s;
    int year, no;

    // s.no = 13; // error: student::no is private
    s.set_name("First Last");
    s.set_no(100);
    s.set_year_of_studies(1);
    s.set_name("NFirst Last");

    year = s.get_year_of_studies();
    no = s.get_no();
    cout << "\nYear of studies is " << year
         << " for student with no " << no
         << " and name " << s.get_name() << endl;

    return 0;
}

```

5.3.2 inline Συναρτήσεις-μέλη

Προγραμματίζοντας με κλάσεις, συχνά χρησιμοποιούμε πολύ μικρές συναρτήσεις. Το κόστος κλήσης μιας συνάρτησης είναι μεγάλο. Στο Κεφάλαιο 3, αναφέραμε ότι μπορούμε, ενδεχόμενα, να απαλλαγούμε από το κόστος κλήσης μιας συνάρτησης δηλώνοντας τη συνάρτηση ως `inline`. Είναι, λοιπόν, κρίσιμο να επισημάνουμε ότι μια συνάρτηση-μέλος που ορίζεται — και όχι απλώς δηλώνεται — στον ορισμό μιας κλάσης θεωρείται ότι είναι `inline`. Για παράδειγμα, αν αλλάζαμε τον ορισμό της κλάσης `Student` ώστε να συμπεριλάβουμε και τους ορισμούς των συναρτήσεων-μελών, όπως φαίνεται στο παρακάτω περίγραμμα, οι συναρτήσεις-μέλη της θα ήταν σαν να είχαν προσδιοριστεί `inline`.


```
class Student {
    char* name;
    // ...
public:
    // ...
    char* get_name() const { return name; }    // INLINE
    // ...
};
```

Με βάση τα παραπάνω, και η κλάση `date` έχει συναρτήσεις-μέλη που καλό είναι να οριστούν μέσα στην κλάση. Στο παρακάτω παράδειγμα, βλέπουμε τον νέο ορισμό της και τη χρήση της. Στη χρήση της, δεν αλλάζει τίποτα από αυτά που είχαμε συζητήσει στα προηγούμενα.

```
#include <iostream>
using namespace std;

class date {
    int day, month, year;
public:
    void set(int d, int m, int y)                // inline definition
    { day = d; month = m; year = y; }

    void get(int& d, int& m, int& y) const        // inline definition
    { d = day; m = month; y = year; }
};

int main()
{
    date today;
    date* ptoday = &today;

    cout << "The size of a variable of type date "
         << sizeof(today) << endl;
    cout << "The size of a variable of type pointer to date "
         << sizeof(ptoday) << endl;

    int d, m, y;

    today.set(18,3,2005);
    today.get(d,m,y);
    cout << "day " << d << ", month " << m << ", year " << y << endl;
    // cout << "day " << today.day << endl; // ERROR: date::day is private

    ptoday->get(d,m,y);
    cout << "day " << d << ", month " << m << ", year " << y << endl;

    return 0;
}
```

Να υπενθυμίσουμε ότι μια `inline` συνάρτηση έχει εσωτερική συνδεσιμότητα. Κατά συνέπεια, εξακολουθούμε να μπορούμε να γράφουμε τον ορισμό των κλάσεων σε αρχεία επικεφαλίδας και να τα συμπεριλαμβάνουμε σε κάθε αρχείο που χρησιμοποιούμε την κλάση. Ακόμα και στην περίπτωση που στον ορισμό της συμπεριλαμβάνονται ορισμοί συναρτήσεων-μελών, δεν φοβόμαστε ότι θα υπάρξουν πολλαπλοί ορισμοί της ίδιας οντότητας λόγω της εσωτερικής συνδεσιμότητας των συναρτήσεων αυτών.

Παρότι όλες οι συναρτήσεις-μέλη της κλάσης `Student` του παραδείγματος που συζητάμε είναι υποψήφιος για χαρακτηρισμό ως `inline` και, άρα, ορισμό εντός του ορισμού της κλάσης, για λόγους αναγνωσιμότητας, δεν θα τις ορίσουμε έτσι: θα θεωρούμε ότι τις ορίζουμε σε ένα ξεχωριστό αρχείο πηγαίου κώδικα και στον ορισμό της κλάσης απλά θα συμπεριλαμβάνουμε τις δηλώσεις τους (τα πρωτότυπά τους).

5.4 Αρχικοποίηση - Καταστροφή Αντικειμένων

Στο Κεφάλαιο 1 μιλήσαμε για την έννοια της κατάστασης των αντικειμένων, η οποία μπορεί είτε να παρατηρηθεί, είτε να αλλάξει, με λειτουργίες της συμπεριφοράς της κλάσης στην οποία ανήκει. Στην κλασική βιβλιογραφία του αντικειμενοστραφούς μοντέλου, τα αντικείμενα συχνά θεωρούνται ως *μηχανές με κατάσταση* (*state machines*). Ως εκ τούτου, θεωρείται ότι, ανά πάσα στιγμή, ένα αντικείμενο βρίσκεται σε μία από τις δυνατές καταστάσεις, η οποία μέσω μιας μεθόδου μπορεί να αλλάξει ή να παρατηρηθεί. Για λόγους ορθότητας του προγράμματος, λοιπόν, καλό είναι ένα αντικείμενο να βρίσκεται πάντα σε *καλά ορισμένη κατάσταση* καθ' όλη τη διάρκεια του χρόνου ζωής του, είτε πρόκειται για την *αρχική κατάσταση*, είτε για *μεταγενέστερη*.

Η κατάσταση ενός αντικειμένου σε μια χρονική στιγμή αντιπροσωπεύεται από την τιμή που έχει το αντικείμενο τη χρονική στιγμή αυτή. Η τιμή αυτή μπορούμε να θεωρήσουμε ότι προκύπτει από τις τιμές των δομικών στοιχείων του. Στις προηγούμενες παραγράφους, τόσο για την κλάση `date` όσο και για την κλάση `Student`, επικεντρωθήκαμε σε συναρτήσεις-μέλη που είτε αλλάζουν, είτε ανακτούν την τιμή ενός αντικειμένου της κλάσης. Προκειμένου να αναθέσουμε μια τιμή σε ένα αντικείμενο χρησιμοποιούσαμε μια μεταβάλλουσα συνάρτηση-μέλος. Προκειμένου ένα αντικείμενο να αποκτήσει μια αρχική τιμή, ήταν ευθύνη του χρήστη του τύπου να το εξασφαλίσει κάνοντας ο ίδιος τις κατάλληλες αναθέσεις τιμών. Στις συναρτήσεις `main` των παραδειγμάτων μας μέχρι τώρα, πριν κάνουμε οποιαδήποτε ανάκτηση τιμής μιας μεταβλητής τύπου κλάσης, είχαμε κάνει αναθέσεις μέσω εφαρμογής των κατάλληλων συναρτήσεων-μελών στη μεταβλητή αυτή.

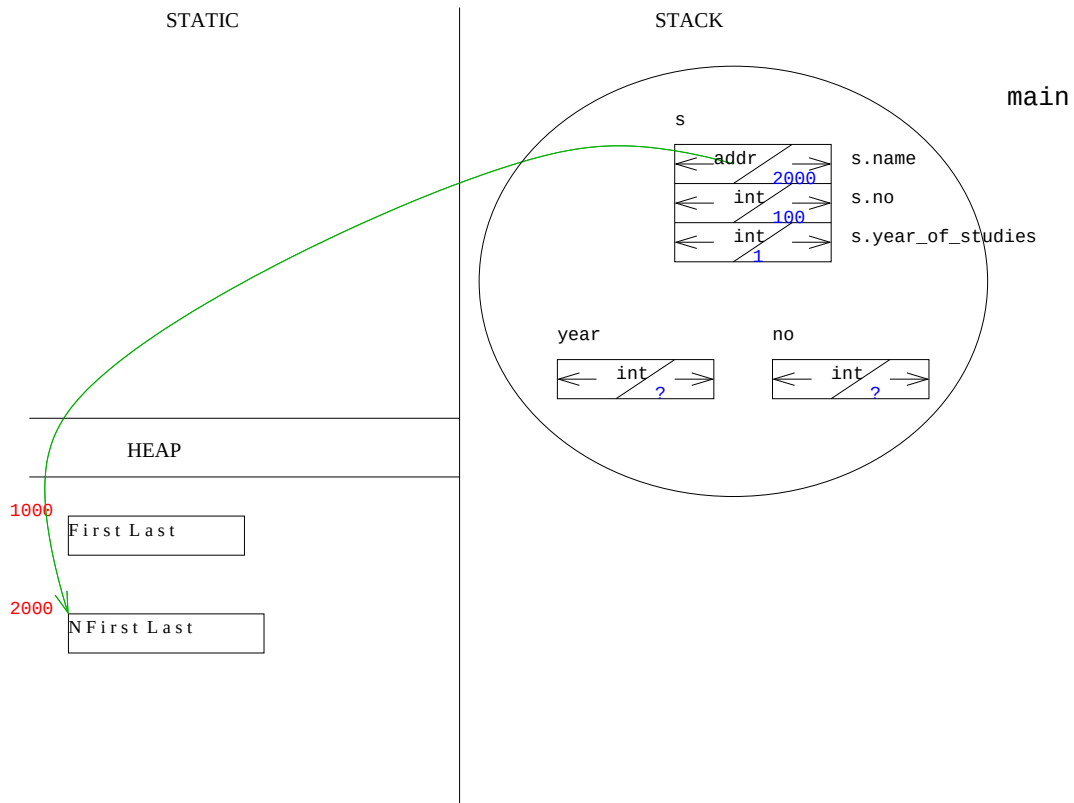
Όμως, το να επαφίεται η διαδικασία απόδοσης αρχικής τιμής ενός αντικειμένου μιας κλάσης στον χρήστη της κλάσης, δηλαδή ο χρήστης να είναι υπεύθυνος για την εξασφάλιση αποδεκτής αρχικής κατάστασης για ένα αντικείμενο, και γενικότερα *αρχικοποίησής* του, θέτει επιπλέον φόρτο στον χρήστη του τύπου και κάνει πιο πιθανό να προκύψουν σφάλματα ή κακές πρακτικές. Για παράδειγμα, ας δούμε τον ορισμό του τύπου `Student` και τη χρήση του από τη συνάρτηση `main` της σελίδας 142. Η αφαιρετική εικόνα της μνήμης μετά την εκτέλεση της εντολής

```
s.set_name("NFirst Last");
```

φαίνεται στο Σχήμα 5.4.

Βλέπουμε ότι ο χώρος τον οποίο καταλαμβάνει η συμβολοσειρά `"First Last"` εξακολουθεί να είναι δεσμευμένος αλλά δεν είναι πλέον προσβάσιμος με κανέναν τρόπο, αφού η μεταβλητή `s.name` που "έδειχνε" σε αυτόν πλέον έχει άλλη τιμή. Καλό θα ήταν, λοιπόν, η συνάρτηση, πριν κάνει τη νέα ανάθεση στον δείκτη του ονόματος, `name`, να αποδεσμεύει τον χώρο στον οποίο "δείχνει" και μετά την ανάθεση, παρότι θα εξακολουθεί να είναι δεσμευμένος για το πρόγραμμα, δεν θα είναι πλέον προσπελάσιμος. Όμως, αυτό δεν ισχύει για την ανάθεση ονόματος κατά την αρχικοποίηση του αντικειμένου, μια που αυτό δεν έχει τιμή. Επιβάλλεται, λοιπόν, διαφοροποίηση της διαδικασίας στην περίπτωση που πρόκειται για αντικείμενο που αρχικοποιείται κατά τη διαδικασία αυτή — και άρα δεν έχει κανένα παρελθόν για το οποίο πρέπει να προνοήσουμε — ή πρόκειται για αντικείμενο το οποίο έχει αρχικοποιηθεί πρωτύτερα και τώρα, κατά τη διαδικασία, απλά χρησιμοποιείται. Σε κάθε περίπτωση, καταλαβαίνουμε ότι ο ορισμός της κλάσης `Student`, όπως έχει τώρα, δεν επαρκεί.

Η διαδικασία αρχικοποίησης καλό είναι να συμπεριλαμβάνεται στον ορισμό της κλάσης και να καθορίζεται από τον προγραμματιστή που ορίζει τον τύπο ώστε να διασφαλίζεται η ορθότητα της αρχικής κατάστασης



Σχήμα 5.4: Η εικόνα των μεταβλητών στην περίπτωση μη διάκρισης αρχικοποίησης της κλάσης “Student”

του αντικειμένου, σύμφωνα με τις προδιαγραφές του τύπου. Στην περίπτωση που είναι επιθυμητό να υπάρχουν εναλλακτικά στις αρχικοποιήσεις ενός τύπου, αυτό να πραγματοποιείται μέσω παραμετροποίησης της διαδικασίας και ο χρήστης απλά να έχει να επιλέξει τις παραμέτρους και τις τιμές τους.

5.4.1 Συναρτήσεις Αρχικοποίησης

Στη C++, η απόδοση αρχικής κατάστασης ενός αντικειμένου και, γενικότερα η διαδικασία αρχικοποίησης, γίνεται μέσω ειδικών συναρτήσεων-μελών, οι οποίες λέγονται *συναρτήσεις αρχικοποίησης (constructors)*. Αυτές καλούνται αυτόματα κατά τη δημιουργία του αντικειμένου, δηλαδή όταν ορίζεται μια μεταβλητή του τύπου της κλάσης.

Συντακτικά, οι συναρτήσεις αυτές προσδιορίζονται από το ότι έχουν το ίδιο όνομα με την κλάση και στο πρωτότυπό τους δεν υπάρχει τύπος επιστροφής. Για παράδειγμα, στο παρακάτω περίγραμμα κώδικα, έχουμε τέσσερις διαφορετικές δηλώσεις συναρτήσεων αρχικοποίησης για την κλάση `date`. Δεν συμπεριλάβαμε τους ορισμούς τους, αλλά τα σχόλια δίπλα από την κάθε δήλωση δίνουν την ερμηνεία του ορίσματος που παραλείπεται: όταν λείπει ένα όρισμα, η ημερομηνία της μεταβλητής αφορά το τρέχον έτος, όταν λείπουν δύο ορίσματα, αφορά τον τρέχοντα μήνα και το τρέχον έτος και όταν λείπουν και τα τρία, αφορά τη “σημερινή” ημερομηνία. Βλέπουμε, λοιπόν, ότι μπορούμε να έχουμε υπερφορτωμένους ορισμούς της συνάρτησης αρχικοποίησης.

```
class date {
// ...
public:
    date(int, int, int);
    date(int, int);           // this year
    date(int);                // this year and this month
    date();                   // today
```

```
// ...
};
```

Στη συνέχεια, αρκεί κατά τον ορισμό μιας μεταβλητής να δώσουμε τιμές για ορίσματα τα οποία να ταιριάζουν με τα ορίσματα από ένα από τα πρωτότυπα των συναρτήσεων αρχικοποίησης. Για παράδειγμα,

```
int main()
{
    // ...
    date day(14);
    date day_month(14,3);
    date today;
    // ...
}
```

Δεν χρειάζεται ούτε να καλέσουμε ρητά μια συνάρτηση αρχικοποίησης, ούτε να αναφέρουμε το όνομά της κατά τον ορισμό μιας μεταβλητής. Η συνάρτηση αρχικοποίησης της οποίας η δήλωση ταιριάζει με τα ορίσματα που ακολουθούν το όνομα της μεταβλητής κατά τον ορισμό της καλείται αυτόματα από τη γλώσσα, μετά τη δέσμευση του χώρου για τη δημιουργία του αντικειμένου. Στην περίπτωση δημιουργίας αντικειμένου μέσω δυναμικής δέσμευσης χώρου στον σωρό, τα ορίσματα ακολουθούν τον τύπο της κλάσης στον οποίο εφαρμόζεται ο τελεστής `new`. Για παράδειγμα, η κλήση

```
new date(30,5,2000);
```

δημιουργεί στον σωρό ένα αντικείμενο τύπου `date`, το οποίο αρχικοποιεί με χρήση της συνάρτησης αρχικοποίησης `date(int,int,int)`.

Η συνάρτηση αρχικοποίησης που καλείται χωρίς να έχουν προσδιοριστεί ορίσματα (τιμές τους) αναφέρεται ως *default συνάρτηση αρχικοποίησης* (*default constructor*). Για την κλάση `date`, όπως ορίστηκε παραπάνω, αυτή είναι η συνάρτηση `date()`. Ισχύει ότι, μπορεί να δοθούν `default` τιμές για τα ορίσματα των συναρτήσεων αρχικοποίησης, όπως για οποιαδήποτε άλλη συνάρτηση. Οπότε, για την κλάση `date` θα μπορούσαμε να είχαμε τη δήλωση

```
date(int d = 1, int m = 1, int y = 2000);
```

και με αυτή να επιτρέπεται να οριστούν μεταβλητές τύπου `date` με τέσσερις διαφορετικούς τρόπους, προσδιορίζοντας καμία, μία, δύο ή και τις τρεις τιμές των ορισμάτων της συνάρτησης. Δηλαδή,

```
class date {
    // ...
public:
    date(int d = 1, int m = 1, int y = 2000);
    // ...
};

int main()
{
    // ...

    date day_default;
    // day_default.day = 1
    // day_default.month = 1
    // day_default.year = 2000

    date day_month(14,3);
    // day_month.day = 14
```

```
// day_month.month = 3
// day_month.year = 2000

date day_explicit(5,10,2030);
// day_explicit.day = 5
// day_explicit.month = 10
// day_explicit.year = 2030
// ...
}
```

Στην περίπτωση αυτή, η default συνάρτηση αρχικοποίησης προκύπτει από τη χρήση των default τιμών σε όλα τα ορίσματα της συνάρτησης αρχικοποίησης.

5.4.1.1 Χρήση Λίστας Αρχικοποίησης Μελών-δεδομένων

Στην περίπτωση που θέλουμε να αρχικοποιήσουμε τα μέλη-δεδομένα μιας κλάσης, στον ορισμό των συναρτήσεων αρχικοποίησης της κλάσης με τις οποίες θέλουμε να το επιτύχουμε, χρησιμοποιούμε *λίστα αρχικοποίησης μελών-δεδομένων* (*member initializer list*). Συντακτικά, μια λίστα αρχικοποίησης έχει την εξής μορφή:

```
: <member name>( <values to be used for initialization> ), ... ,
   <member name>( <values to be used for initialization> )
```

Με τον τρόπο αυτό, δηλώνεται ότι κατά την αρχικοποίηση του μέλους <member name> να χρησιμοποιηθούν οι τιμές που εμφανίζονται στο <values to be used for initialization>. Για παράδειγμα, ένας απλός ορισμός συνάρτησης αρχικοποίησης για την κλάση `date`, η οποία by default να αρχικοποιεί τα μέλη-δεδομένα με μηδενικές τιμές, μπορεί να είναι η εξής:

```
date(int d = 0, int m = 0, int y = 0)
    : month(m), day(d), year(y)    // initializer list
{   cout << "Creating a date!" << endl;
    // month = m; day = d; year = y; // Assignment:
                                     // do not use it for initialization!
}
```

Προσέξτε: η αρχικοποίηση των μελών-δεδομένων προηγείται της εκτέλεσης του σώματος της συνάρτησης αρχικοποίησης. Οπότε, στο σώμα της θεωρείται ότι αυτά έχουν ήδη αρχικοποιηθεί. Για τον λόγο αυτό, υπάρχει ο σχολιασμένος κώδικας με τις αναθέσεις στο σώμα της συνάρτησης αρχικοποίησης. Στη γενική περίπτωση, αναθέσεις γίνονται σε μεταβλητές που έχουν οριστεί, άρα και αρχικοποιηθεί, προηγουμένως, οπότε δεν πρέπει να χρησιμοποιούνται για αρχικοποίηση. Για μεταβλητές βασικών τύπων της γλώσσας, όπως `int`, δεν υπάρχει διαφορά στο αποτέλεσμα μεταξύ των δύο. Οπότε, στην περίπτωση της αρχικοποίησης των μελών-δεδομένων της κλάσης `date`, δεν θα είχαμε διαφορετικό αποτέλεσμα μεταξύ χρήσης λίστας αρχικοποίησης για τα μέλη της και ανάθεσης στο σώμα της συνάρτησης.¹

Η σειρά με την οποία εμφανίζονται οι αρχικοποιήσεις στη λίστα αρχικοποίησης δεν παίζει ρόλο. Η σειρά με την οποία γίνονται οι αρχικοποιήσεις είναι η σειρά με την οποία γράφουμε τις δηλώσεις των μελών-δεδομένων μέσα στην κλάση.

Προσθέτοντας και τη συνάρτηση αρχικοποίησης, ο ορισμός της κλάσης `date` γίνεται ως εξής:

¹Από την έκδοση C++11 και έπειτα, η γλώσσα προσφέρει δυνατότητα αρχικοποίησης των μελών-δεδομένων μιας κλάσης κατά τη δήλωσή τους. Αυτή μπορεί να χρησιμοποιηθεί στην περίπτωση που μια συνάρτηση αρχικοποίησης της κλάσης δεν περιλαμβάνει λίστα αρχικοποίησης μελών-δεδομένων. Στο επίπεδο αυτό, πλέον, προσφέρονται πολλές δυνατότητες. Στον βαθμό που ο αναγνώστης έχει κατανοήσει τη διάκριση της φάσης της αρχικοποίησης από την υπόλοιπη ζωή του αντικειμένου μετά τη συζήτηση αυτή, καλείται να διερευνήσει τα εναλλακτικά που προσφέρει η κάθε έκδοση μόνος του.

```
// File: day_class.h
#include <iostream>
using namespace std;

class date {
private:
    int day, month, year;
public:
    date(int d = 0, int m = 0, int y = 0)
        : month(m), day(d), year(y)    // initializer list
    { cout << "Creating a date!" << endl; }

    void set(int i, int j, int k)
    { day = i; month = j; year = k; }

    void get(int& i, int& j, int& k) const
    { i = day; j = month; k = year; }
};
```

Στην περίπτωση της κλάσης Student, καλό θα ήταν, λοιπόν, να έχουμε ορίσει τουλάχιστον μία συνάρτηση αρχικοποίησης που να αρχικοποιεί το μέλος-δεδομένο name με μια συμβολοσειρά που θα αντιστοιχεί στο όνομά του. Δηλαδή, να προσθέσουμε στον ορισμό της κλάσης μια συνάρτηση-μέλος

Student(const char*)

της οποίας ο ορισμός, για παράδειγμα, είναι:

```
Student::Student(const char* nam)
{
    name = new char[strlen(nam)+1];
    strcpy(name, nam);
}
```

Στη συνέχεια, όταν καλείται η συνάρτηση-μέλος set_name, ξέρουμε ότι το μέλος-δεδομένο name έχει ήδη αρχικοποιηθεί και “δείχνει” σε μια συμβολοσειρά. Οπότε, ο ορισμός της μπορεί να αλλάξει ώστε να κάνει την κατάλληλη αποδέσμευση χώρου και να γίνει:

```
void Student::set_name(const char* nam)
{
    delete[] name;
    name = new char[strlen(nam)+1];
    strcpy(name, nam);
}
```

5.4.2 Συναρτήσεις Καταστροφής

Η αντίστροφη διαδικασία της κατασκευής αντικειμένων είναι εκείνη της καταστροφής αντικειμένων. Πολλοί τύποι χρειάζονται να οριστεί μια συνάρτηση-μέλος για να διασφαλιστεί η ασφαλής απελευθέρωση του χώρου που καταλαμβάνει ένα στιγμιότυπό τους. Αυτό αναφέρεται κυρίως σε ενέργειες που πρέπει να γίνουν πριν χαθεί η δυνατότητα πρόσβασης στον χώρο που καταλαμβάνει το στιγμιότυπο.

Ας υποθέσουμε ότι έχουμε την κλάση Student, όπως διαμορφώθηκε με την προσθήκη της συνάρτησης

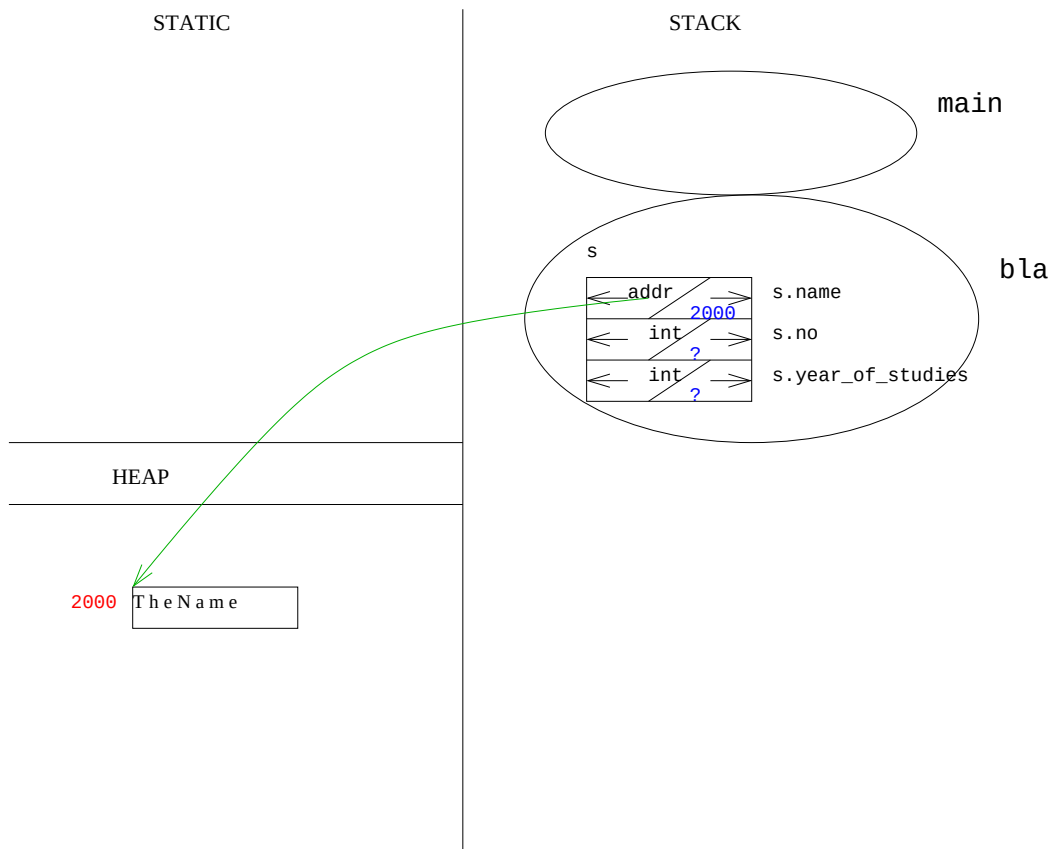
αρχικοποίησης και την αλλαγή του ορισμού της συνάρτησης-μέλους `set_name`. Για να δούμε τι πρόβλημα μπορεί, και πάλι, να προκύψει. Θεωρήστε ότι έχουμε τις εξής συναρτήσεις:

```
void bla()
{
    Student s("TheName");
}

int main()
{
    bla();

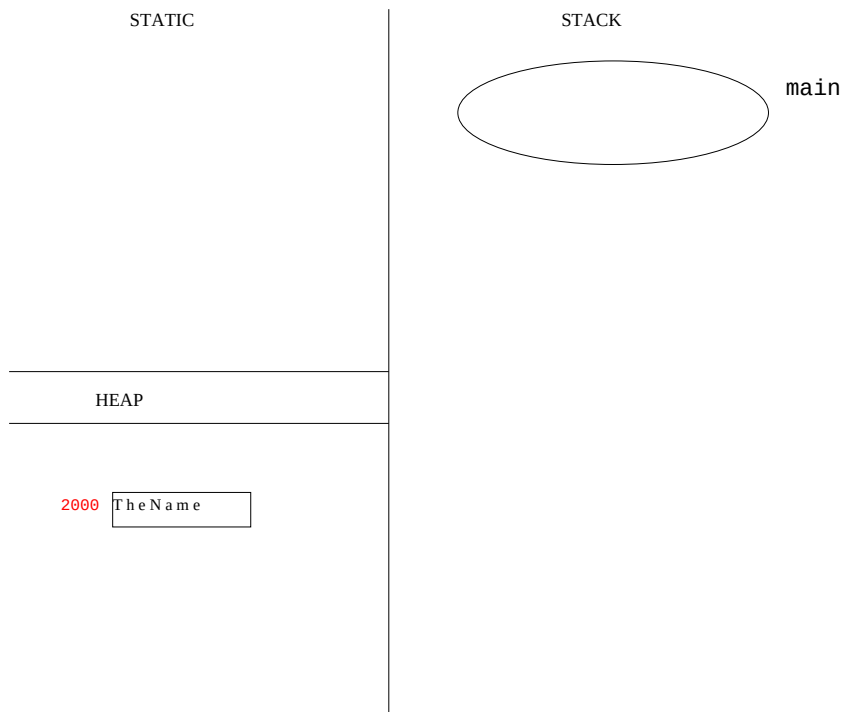
    // ...

    return 0;
}
```



Σχήμα 5.5: Η εικόνα των μεταβλητών στην περίπτωση κλήσης συνάρτησης που δημιουργεί αντικείμενο της κλάσης "Student"

Η αφαιρετική εικόνα στη μνήμη κατά την κλήση της `bla` φαίνεται στο Σχήμα 5.5. Μετά την επιστροφή από την κλήση της, η εικόνα φαίνεται στο Σχήμα 5.6. Έχει μείνει, δηλαδή, ένα "σκουπίδι" στον σωρό: ο χώρος όπου φυλάσσεται η συμβολοσειρά "TheName". Η μεταβλητή `s` της οποίας το μέλος-δεδομένο `name`, το `s.name` δηλαδή, είχε πρόσβαση στον χώρο της συμβολοσειράς "TheName" στον σωρό, καταστράφηκε. Ο χώρος αυτός, λοιπόν, είναι δεσμευμένος αλλά όχι προσβάσιμος. Θα έπρεπε, πριν καταστραφεί η μεταβλητή `s` να έχουμε αποδεσμεύσει τον χώρο αυτό. Πώς θα μπορούσαμε να το κάνουμε; Τι μπορούμε να κάνουμε σε



Σχήμα 5.6: Η εικόνα μετά από την επιστροφή από την κλήση της συνάρτησης `bla`

περιπτώσεις σαν και αυτή; Περιπτώσεις, δηλαδή, όπου θέλουμε να γίνουν κάποιες ενέργειες πριν καταστραφεί ένα αντικείμενο, διότι υπάρχουν πόροι που είναι συνυφασμένοι με τη ζωή του αντικειμένου και όταν αυτό καταστραφεί πλέον, δεν είναι διαχειρίσιμοι. Πώς μπορούμε να ορίζουμε, λοιπόν, μια διαδικασία, η οποία έχει ως σκοπό όταν εκτελεστεί να κάνει ένα “συνμάζεμα” πόρων που συνδέονται με το αντικείμενο, πριν την καταστροφή του αντικειμένου; Είναι γεγονός ότι το πρόβλημα είναι πολύ σύνηθες και κρίσιμο, κυρίως όσον αφορά διαχείριση μνήμης που έχει δεσμευθεί δυναμικά.

Η C++ δίνει τη δυνατότητα να ορίσουμε μια συνάρτηση-μέλος, τη *συνάρτηση καταστροφής* (*destructor*), η οποία καλείται αυτόματα από τη γλώσσα πριν καταστραφεί ένα αντικείμενο. Συντακτικά, η γενική μορφή της, για μια κλάση *X*, είναι:

$$\sim X()$$

Η ιδέα είναι ότι το σύμβολο $\sim$ υπονοεί “συμπλήρωμα” (“αντίθετο”) των συναρτήσεων αρχικοποίησης.

Έτσι, λοιπόν, πρέπει να επαυξήσουμε τον ορισμό της κλάσης *Student* και με τον ορισμό μιας συνάρτησης καταστροφής

$$\sim \text{Student}()$$

η οποία να αποδεσμεύει τον χώρο της συμβολοσειράς του ονόματος, δηλαδή, ο ορισμός της να είναι:

```
Student::~~Student()
{
    delete[] name;
}
```

Όταν ένα αντικείμενο πρόκειται να καταστραφεί, καλείται αυτόματα η συνάρτηση καταστροφής της κλάσης του, εφαρμόζοντάς την σε αυτό. Προσοχή: στη C++, βγαίνοντας από το σώμα της *main*, δεν σταματά η εκτέλεση: καλούνται και οι συναρτήσεις καταστροφής όλων των αντικειμένων που δημιουργήθηκαν (εκτός εκείνων που βρίσκονται στον σωρό). Έτσι, συχνά ακούγεται η έκφραση ότι η C++ “κάνει πράγματα behind the scenes”. Η απάντηση στην παρατήρηση αυτή είναι ότι δεν είναι καθόλου “πίσω από την πλάτη” του προγραμματιστή αυτό που συμβαίνει. Η κλήση των συναρτήσεων αρχικοποίησης και καταστροφής απλά γίνεται αυτόματα από τη γλώσσα, χωρίς παρέμβαση του προγραμματιστή και μάλιστα για διευκόλυνση και διασφά-

λίσή του. Είναι δική του ευθύνη να έχει κατανοήσει τη συμπεριφορά τους, ώστε να αντιλαμβάνεται τι ακριβώς συμβαίνει κατά την εκτέλεση του προγράμματός του.

Σύνοψη του κώδικα του παραδείγματος του “Φοιτητή” μέχρι τώρα:

Ο ορισμός της κλάσης Student, όπως έχει διαμορφωθεί, επαυξημένος με συναρτήσεις κατασκευής-καταστροφής δίνεται στη συνέχεια, συνολικά. Παρατηρήστε την προσθήκη εκτύπωσης μηνυμάτων στις συναρτήσεις αυτές ώστε να γίνεται αντιληπτή η εκτέλεσή τους.

```
// File: student_constr.h
class Student {
private:
    char* name;
    int no;
    int year_of_studies;
public:
    Student(const char*);
    ~Student();

    void set_name(const char*);
    void set_no(int);
    void set_year_of_studies(int);

    char* get_name() const;
    int get_no() const;
    int get_year_of_studies() const;
};
```

```
// File: student_constr.cpp
#include <cstring>
#include <iostream>
#include "student_constr.h"
using namespace std;

////////////////////////////////////

void Student::set_name(const char* nam)
{
    delete[] name;
    name = new char[strlen(nam)+1];
    strcpy(name, nam);
}

void Student::set_no(int n)
{
    no = n;
}

void Student::set_year_of_studies(int y)
{

```

```

    year_of_studies = y;
}

////////////////////////////////////

char* Student::get_name() const
{
    return name;
}

int Student::get_no() const
{
    return no;
}

int Student::get_year_of_studies() const
{
    return year_of_studies;
}

////////////////////////////////////

Student::~~Student()
{
    cout << "Deleting student with name " << name << endl;
    delete[] name;
}

////////////////////////////////////

Student::Student(const char* nam)
{
    name = new char[strlen(nam)+1];
    strcpy(name,nam);
    cout << "A student was just constructed " << endl;
}

```

Δοκιμάστε να συνδυάσετε τον ορισμό της κλάσης Student με την παρακάτω συνάρτηση main.

```

// File: classtest_constr.cpp
#include <iostream>
#include "student_constr.h"
using namespace std;

int main()
{
    Student s("First Last");
    // Student s1(s);
}

```

```
// Student s1 = s;
cout << "This is the first output of the main function " << endl;

s.set_no(100);
s.set_year_of_studies(1);
s.set_name("NFirst Last");

int year, no;

year = s.get_year_of_studies();
no = s.get_no();
cout << "\nYear of studies is " << year
      << " for student with no " << no
      << " and name " << s.get_name() << endl;

////////////////////////////////////

cout << "Just exiting the main function ...." << endl;

return 0;
}
```

Το αποτέλεσμα της εκτέλεσης είναι:

A student was just constructed

This is the first output of the main function

Year of studies is 1 for student with no 100 and name NFirst Last

Just exiting the main function

Deleting student with name NFirst Last

Παρατηρήστε ότι δεδομένου ότι ο ορισμός της μεταβλητής τύπου Student, s, προηγείται της εκτύπωσης του μηνύματος “This is the first output of the main function”, πριν την εκτύπωση αυτή εμφανίζεται η εκτύπωση από το σώμα της συνάρτησης αρχικοποίησης της Student, “A student was just constructed”. Αντίστοιχα, παρότι η εκτύπωση του μηνύματος “Just exiting the main function” προκύπτει από την τελευταία εντολή του σώματος της συνάρτησης main, δεν είναι το τελευταίο μήνυμα που εμφανίζεται κατά την εκτέλεση. Ακολουθεί το μήνυμα από τη συνάρτηση καταστροφής της κλάσης Student, “Deleting student with name NFirst Last” καθώς αποδεσμεύοντας τον χώρο (του πλαισίου) της συνάρτησης main, καταστρέφονται όποιες μεταβλητές είχαν οριστεί σε αυτήν, άρα και η s, οπότε, σύμφωνα με αυτά που έχουμε πει, πριν την καταστροφή της, καλείται η συνάρτηση καταστροφής του τύπου της εφαρμοσμένη σε αυτήν.

Να κάνουμε εδώ ένα γενικό σχόλιο. Εκτός περιπτώσεων για τις οποίες μας ικανοποιεί η default συμπεριφορά της C++ — θα δούμε κάποιες από αυτές στη συνέχεια —, γενικά, χρειάζεται να ορίσουμε συναρτήσεις αρχικοποίησης σε κάθε κλάση που ορίζουμε προκειμένου να έχουν τα αντικείμενά της μια καλά ορισμένη αρχική κατάσταση. Αντίθετα, ο ορισμός συνάρτησης καταστροφής είναι απαραίτητος μόνο για τις κλάσεις με τα χαρακτηριστικά που προαναφέραμε, οπότε δεν είναι πάντα απαραίτητος. Για παράδειγμα, για την κλάση date δεν υπάρχει λόγος ορισμού μιας συνάρτησης καταστροφής, εκτός και αν θέλουμε απλά να εκτυπώνεται κάποιο μήνυμα, ώστε να αντιλαμβανόμαστε πότε ένα αντικείμενό της πρόκειται να καταστραφεί, π.χ.

```
~date() { cout << "Destroying a date with values: " << day << " "
          << month << year << endl; }
```

Σε ένα τελικό σύστημα, όμως, αυτό δεν είναι απαραίτητο.

Θα δούμε στη συνέχεια ότι στην περίπτωση που για μια κλάση χρειάζεται να οριστεί συνάρτηση καταστροφής, χρειάζεται να συμπληρωθεί ο ορισμός της και με κάποιες άλλες συναρτήσεις, με στόχο να αντιμετωπιστούν προβλήματα που σχετίζονται με τα ίδια χαρακτηριστικά της κλάσης. Ο ορισμός της κλάσης `Student`, όπως έχει καταλήξει να είναι, δηλαδή έχοντας συνάρτηση αρχικοποίησης και συνάρτηση καταστροφής, είναι επαρκής, ώστε να μην έχουμε σφάλματα ή μη ελεγχόμενες καταστάσεις; Ακόμα, όχι. Προσέξτε τις δύο σχολιασμένες γραμμές της συνάρτησης `main` του προηγούμενου παραδείγματος!

5.4.3 Αρχικοποίηση Στιγμιотύπων (συνέχεια): Αρχικοποίηση μέσω Αντιγραφής

Ορίζοντας έναν νέο τύπο στη C++, ο στόχος είναι, ουσιαστικά, να έχουμε διευρύνει το σύνολο των τύπων που είναι διαθέσιμοι στον προγραμματιστή για να χρησιμοποιήσει. Ιδανικά, λοιπόν, ό,τι μπορούμε να κάνουμε με τους βασικούς τύπους της γλώσσας, θα πρέπει να μπορούμε να κάνουμε χρησιμοποιώντας και έναν τύπο κλάσης.

Όσον αφορά τους βασικούς τύπους, μπορούμε να ορίσουμε μια νέα μεταβλητή ενός τύπου, αρχικοποιώντας την με την τιμή μιας ήδη υπάρχουσας. Δηλαδή, για παράδειγμα, μπορούμε να έχουμε τον παρακάτω κώδικα:

```
int i = 5;
int j = i;
```

Στον κώδικα αυτό, ορίζουμε μια ακέραια μεταβλητή, την `i`, αρχικοποιώντας την με την τιμή 5 και, στη συνέχεια, ορίζουμε μια δεύτερη ακέραια μεταβλητή, την `j`, αρχικοποιώντας την με την τιμή που έχει η ήδη υπάρχουσα μεταβλητή `i`. Δηλαδή, σύμφωνα με τις δύο αυτές γραμμές, οι ακέραιες μεταβλητές `i` και `j` θα καταλήξουν να έχουν και οι δύο την τιμή 5.

Με βάση την παρατήρηση που κάναμε, θέλουμε να μπορούμε να κάνουμε την ίδια διαδικασία και για τους τύπους που ορίζουμε εμείς ως κλάσεις. Ας πάρουμε την κλάση `Student` και τη συνάρτηση `main` του προηγούμενου παραδείγματος. Έχοντας ορίσει (και αρχικοποιήσει) μια μεταβλητή τύπου `Student`, την `s`, με την εντολή

```
Student s("First Last");
```

θα θέλαμε να μπορούμε να ορίσουμε μια νέα μεταβλητή τύπου `Student` αρχικοποιώντας την με βάση την τιμή της υπάρχουσας μεταβλητής `s`. Αυτό, πράγματι μπορούμε να το κάνουμε. Δηλαδή, θεωρώντας το προηγούμενο παράδειγμα, μπορούμε να αποσχολιάσουμε τη γραμμή

```
Student s1 = s;
```

οπότε ορίζουμε μια νέα μεταβλητή τύπου `Student`, την `s1`, την οποία αρχικοποιούμε “κατ’ εικόνα” της μεταβλητής `s` χρησιμοποιώντας την τιμή της. Τελείως ισοδύναμα, θα μπορούσαμε να αποσχολιάσουμε την άλλη γραμμή,

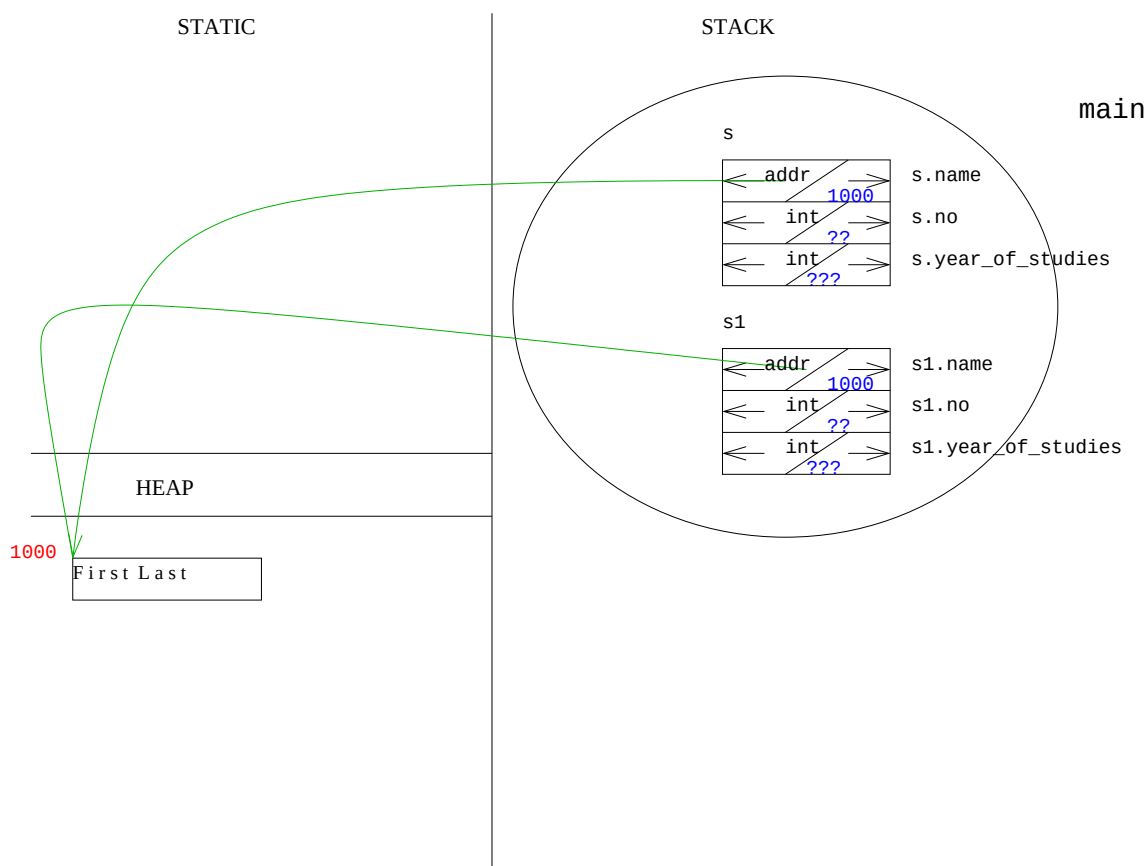
```
Student s1(s);
```

και να επιτύχουμε το ίδιο αποτέλεσμα. Δηλαδή, εκτός αν εμείς έχουμε υποδείξει κάτι διαφορετικό, η C++ αρχικοποιεί τη νέα μεταβλητή `s1` αντιγράφοντας πιστά στα μέλη-δεδομένα της τις τιμές των μελών-δεδομένων της μεταβλητής `s`. Γίνεται, δηλαδή, *επιμέρους αντιγραφή τιμών* (*bitwise copy*). Η αφαιρετική εικόνα της μνήμης μετά την εκτέλεση αυτών των εντολών φαίνεται στο Σχήμα 5.7.

Από την εικόνα του Σχήματος 5.7 βλέπουμε ήδη το πρόβλημα: με την απλή αντιγραφή του περιεχομένου των μελών-δεδομένων, οι δύο μεταβλητές τύπου `Student` έχουν το ίδιο όνομα “δείχνοντας” στην ίδια συμβολοσειρά. Κάτι που δεν είναι αυτό που πραγματικά θα θέλαμε και επιπλέον είναι προβληματικό. Βλέπετε στη συνέχεια ότι με την εκτέλεση της εντολής:

```
s.set_name("NFirst Last");
```

ανατίθεται νέα συμβολοσειρά ως όνομα της μεταβλητής `s` και καταστρέφεται η προηγούμενη. Η εικόνα στη



Σχήμα 5.7: Αρχικοποίηση μέσω αντιγραφής για τη μεταβλητή s1 της κλάσης Student

μνήμη μεταβάλλεται όπως φαίνεται στο Σχήμα 5.8: το μέλος-δεδομένο name της μεταβλητής s1, το s1.name, “δείχνει” σε χώρο που έχει απελευθερωθεί από το πρόγραμμα, άρα η πρόσβαση σε αυτόν θα οδηγήσει σε σφάλμα.

Τι μπορούμε να κάνουμε, λοιπόν;

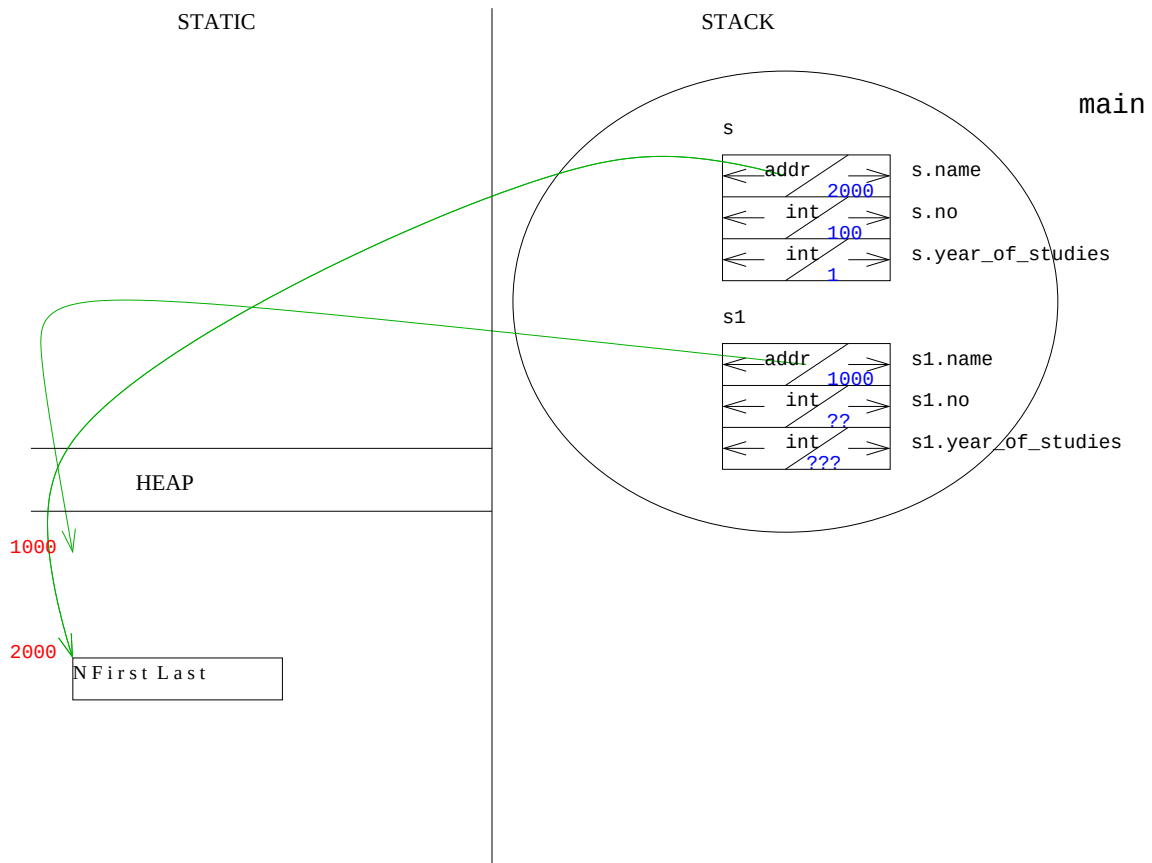
Η απάντηση είναι ότι η C++ επιτρέπει, για κάθε κλάση που η αυτόματη συμπεριφορά της γλώσσας για την περίπτωση αυτή δεν είναι η επιθυμητή, να οριστεί μια συνάρτηση αρχικοποίησης μέσω αντιγραφής (copy constructor). Το πρωτότυπο της συνάρτησης αυτής, για μια κλάση X, είναι:

$X::X(\text{const } X\&)$

Μια επισήμανση: προσέξτε τον τύπο του ορίσματος. Είναι τύπου αναφοράς. Υπενθυμίζουμε ότι το πέρασμα παραμέτρων στις συναρτήσεις της C++ αντιστοιχεί με αρχικοποίηση των ορισμάτων. Αν δεν είχαμε αναφορά και το πέρασμα παραμέτρου ήταν με τιμή, θα δημιουργούνταν νέο αντικείμενο της κλάσης X, το οποίο θα αρχικοποιείτο με χρήση του αντικειμένου στη θέση του ορίσματος. Αυτό, όμως, οδηγεί στη χρήση της ίδιας συνάρτησης αρχικοποίησης μέσω αντιγραφής για το αντικείμενο της παραμέτρου. Οπότε, ούτε επιθυμητό ούτε ορθό είναι το πέρασμα παραμέτρου κατά τιμή στο όρισμα της συνάρτησης αρχικοποίησης μέσω αντιγραφής. Δεδομένου δε ότι το όρισμα δεν θα αλλάξει από το σώμα της συνάρτησης, προσδιορίζεται ως σταθερά με χρήση του const. Στο σημείο αυτό, πρέπει να τονίσουμε ότι η χρήση του τελεστή ανάθεσης στον ορισμό μεταβλητής κλάσης ΔΕΝ είναι έκφραση ανάθεσης, υπονοεί αρχικοποίηση μέσω αντιγραφής, δηλαδή κλήση της παραπάνω συνάρτησης.

Προκειμένου, λοιπόν, να επιτύχουμε το επιθυμητό αποτέλεσμα για τα αντικείμενα της κλάσης Student, θα μπορούσαμε να ορίσουμε μια συνάρτηση αρχικοποίησης μέσω αντιγραφής για την κλάση με τον εξής τρόπο:

```
Student::Student(const Student& s)
{
    name = new char[strlen(s.name)+1];
```



Σχήμα 5.8: Το πρόβλημα που προκύπτει από την αρχικοποίηση μέσω αντιγραφής για την κλάση Student

```
strcpy(name, s.name);
// name = s.name;    Careful: this is the result of default copy
no = s.no;
year_of_studies = s.year_of_studies;
cout << "A student was just created by copying ... " << endl;
}
```

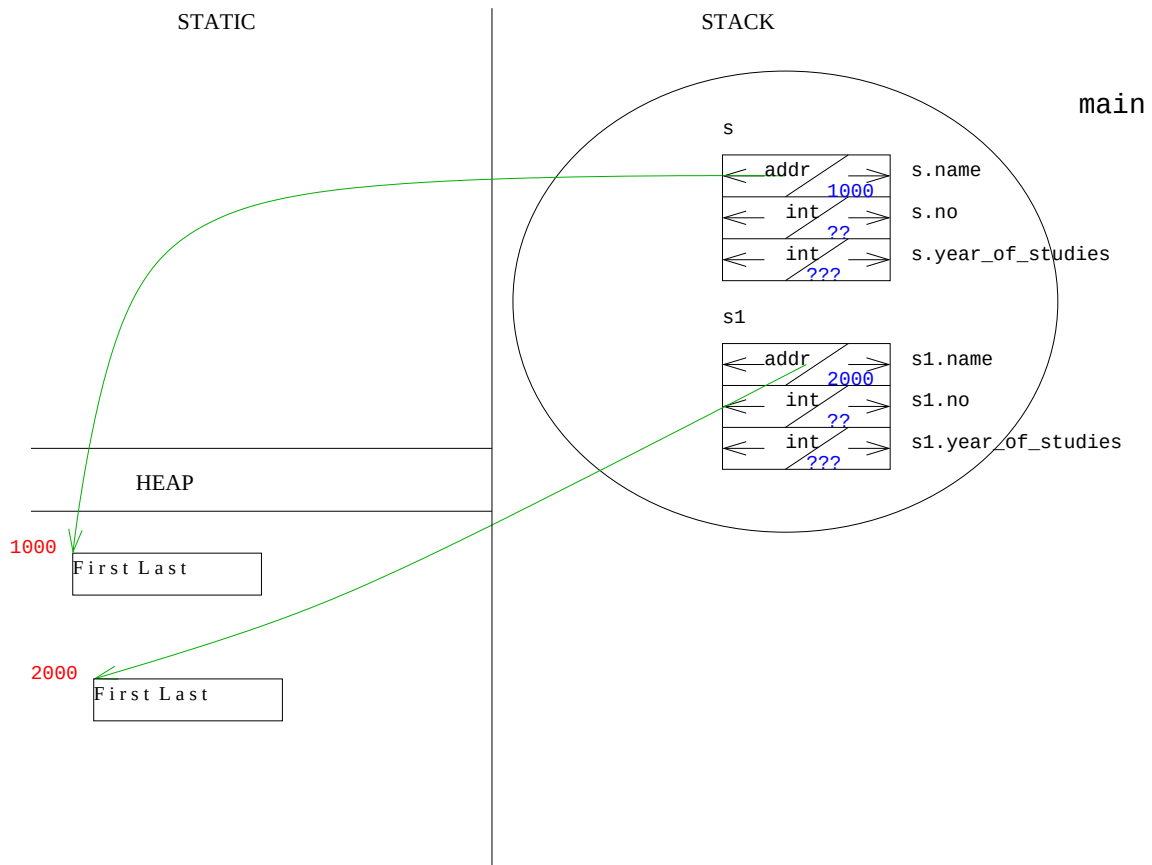
Η τιμή του μέλους-δεδομένου `name` του αντικειμένου που αρχικοποιείται προκύπτει από τον κώδικα:

```
name = new char[strlen(s.name)+1];
strcpy(name, s.name);
```

Αν ο ορισμός της κλάσης `Student` συμπεριλάβει αυτόν τον ορισμό της συνάρτησης αρχικοποίησης μέσω αντιγραφής, η εικόνα στη μνήμη που προκύπτει ως αποτέλεσμα των παρακάτω ορισμών μεταβλητών, αποσχολιάζοντας είτε τη μία γραμμή είτε την άλλη γραμμή, φαίνεται στο Σχήμα 5.9.

```
Student s("First Last");
// Student s1(s);
// Student s1 = s;
```

Στη σχολιασμένη γραμμή του ορισμού της συνάρτησης αρχικοποίησης απεικονίζεται ενέργεια που θα οδηγούσε στο αποτέλεσμα της αυτόματης αρχικοποίησης του μέλους-δεδομένου `name` και η οποία αντικαθίσταται γιατί δημιουργεί το πρόβλημα. Για τα μέλη-δεδομένα `no` και `year_of_studies`, αυτή η απλή αντιγραφή τιμών είναι επαρκής. Ορίζοντας, όμως, δική μας συνάρτηση αρχικοποίησης μέσω αντιγραφής, πρέπει να συμπεριλάβουμε τις αντιγραφές αυτές. Πρέπει, δηλαδή, να περιγράψουμε όλη τη διαδικασία που θέλουμε να



Σχήμα 5.9: Εικόνα της μνήμης με ορισμό συνάρτησης αρχικοποίησης μέσω αντιγραφής για την κλάση Student

λαμβάνει χώρα κατά την αρχικοποίηση, ακόμα και για τις περιπτώσεις που οι ενέργειες που θα γινόντουσαν αυτόματα είναι οι επιθυμητές.

Κλείνοντας τη συζήτηση για τον ορισμό συνάρτησης αρχικοποίησης μέσω αντιγραφής, να πούμε ότι και εδώ το σωστό είναι να κάνουμε τις αρχικοποιήσεις των μελών-δεδομένων με χρήση λίστας αρχικοποίησης, παρότι για μέλη βασικών τύπων δεν υπάρχει διαφορά στο αποτέλεσμα. Οπότε, ο ορισμός της συνάρτησης αρχικοποίησης μέσω αντιγραφής για την κλάση Student γίνεται:

```
Student::Student(const Student& s)
    : no(s.no), year_of_studies(s.year_of_studies)
{
    name = new char[strlen(s.name)+1];
    strcpy(name, s.name);
    // name = s.name;    Careful: this is the result of default copy

    cout << "A student was just created by copying ... " << endl;
}
```

Παρότι κάναμε την παρατήρηση αυτή για την αρχικοποίηση των μελών-δεδομένων με χρήση λίστας αρχικοποίησης, για ιστορικούς λόγους, στα παρακάτω θα διατηρήσουμε τον προηγούμενο ορισμό της συνάρτησης αρχικοποίησης μέσω αντιγραφής για την κλάση Student.

Νέα σύνοψη του κώδικα του παραδείγματος του “Φοιτητή”:

Ας συνοψίσουμε τον ορισμό της κλάσης Student, επαυξημένο με συναρτήσεις αρχικοποίησης και κατα-

στροφής καθώς και συνάρτηση αρχικοποίησης μέσω αντιγραφής.

```
// File: student_constr.h
class Student {
private:
    char* name;
    int no;
    int year_of_studies;
public:
    Student(const char*);
    Student(const Student&);

    ~Student();

    void set_name(const char*);
    void set_no(int);
    void set_year_of_studies(int);

    char* get_name() const;
    int get_no() const;
    int get_year_of_studies() const;
};
```

```
// File: student_constr.cpp
#include <cstring>
#include <iostream>
#include "student_constr.h"
using namespace std;

////////////////////////////////////

void Student::set_name(const char* nam)
{
    delete[] name;
    name = new char[strlen(nam)+1];
    strcpy(name,nam);
}

void Student::set_no(int n)
{
    no = n;
}

void Student::set_year_of_studies(int y)
{
    year_of_studies = y;
}
```



```
////////////////////////////////////

char* Student::get_name() const
{
    return name;
}

int Student::get_no() const
{
    return no;
}

int Student::get_year_of_studies() const
{
    return year_of_studies;
}

////////////////////////////////////

Student::~~Student()
{
    cout << "Deleting student with name " << name << endl;
    delete[] name;
}

////////////////////////////////////

Student::Student(const char* nam)
{
    name = new char[strlen(nam)+1];
    strcpy(name,nam);
    cout << "A student was just constructed " << endl;
}

Student::Student(const Student& s)
{
    name = new char[strlen(s.name)+1];
    strcpy(name,s.name);
    // name = s.name;    Careful: this is the result of default copy
    no = s.no;
    year_of_studies = s.year_of_studies;
    cout << "A student was just created by copying ... " << endl;
}
```

5.5 Επαναορισμός του Τελεστή Ανάθεσης - Παράδειγμα Αναγκαιότητας του Δείκτη `this`

Στα προηγούμενα, χρησιμοποιήσαμε το σύμβολο `=` και κάναμε συζήτηση σχετικά με τη χρήση του σε ορισμό μεταβλητής, σε αντιδιαστολή με τη χρήση του ως τελεστή ανάθεσης, όπου συνήθως το χρησιμοποιούμε. Στη δεύτερη περίπτωση, οι μεταβλητές που συμμετέχουν στην έκφραση έχουν οριστεί (και αρχικοποιηθεί) πρωτότερα.

Στα παραδείγματά μας, κάναμε αναθέσεις μεταξύ μεταβλητών βασικών τύπων της C++. Δεν σχολιάσαμε, όμως, τη συμπεριφορά αναθέσεων μεταξύ μεταβλητών τύπου κλάσης. Όπως έχουμε πει, ορίζοντας έναν τύπο, επεκτείνουμε το σύνολο των διαθέσιμων τύπων, οπότε η βασική λειτουργικότητα της γλώσσας επεκτείνεται και για τους τύπους αυτούς. Κατά συνέπεια, όπως μπορούσαμε να έχουμε τον παρακάτω κώδικα για ακέραιες μεταβλητές:

```
int i = 5;
int j = 10;
j = i;
```

μπορούμε να έχουμε και αντίστοιχο κώδικα για μεταβλητές τύπου κλάσης, για παράδειγμα:

```
Student s1("Name1");
Student s2("Name2");
s2 = s1; // assignment
```

Τι γίνεται, όμως, στην περίπτωση αυτή; Η απάντηση είναι ότι γίνεται επιμέρους ανάθεση τιμών για τα μέλη-δεδομένα της κλάσης όπου, αν δεν έχει οριστεί κάτι διαφορετικό, είναι “τυφλή” αντιγραφή τιμών (bitwise copy). Στο παραπάνω παράδειγμα, θα καταλήξουν οι μεταβλητές `s1` και `s2` να έχουν ακριβώς τις ίδιες τιμές στα μέλη-δεδομένα τους, αυτές της μεταβλητής `s1`. Και να που το πρόβλημα με το μέλος-δεδομένο `name` εμφανίζεται και πάλι! Τι μπορούμε να κάνουμε;

Η C++ μας επιτρέπει να επαναορίσουμε τον τελεστή ανάθεσης ώστε να έχει την επιθυμητή συμπεριφορά. Στην περίπτωση της κλάσης `Student`, για το μέλος-δεδομένο `name` δεν μας ενδιαφέρει η αντιγραφή της διεύθυνσης της συμβολοσειράς του ονόματος αλλά η αντιγραφή της συμβολοσειράς. Δεδομένου ότι αριστερά του ο τελεστής ανάθεσης δέχεται μεταβλητή της κλάσης, μπορεί να δηλωθεί ως συνάρτηση-μέλος της κλάσης. Το όρισμά της είναι πάλι τύπου της κλάσης, το οποίο, δεδομένου ότι δεν αλλάζει κατά την ανάθεση, μπορεί να δηλωθεί ως `const`. Χρειάζεται, τώρα, λίγο συζήτηση για το ποιος μπορεί να είναι ο τύπος επιστροφής της. Μην ξεχνάμε ότι για τους βασικούς τύπους μπορούμε να έχουμε αλυσιδωτές αναθέσεις. Για παράδειγμα, αν έχουμε τρεις ακέριες μεταβλητές, `i`, `j` και `k`, μπορούμε να έχουμε την έκφραση:

```
i = j = k;
```

Αυτή ερμηνεύεται ως

```
i = ( j = k );
```

που σημαίνει: “να γίνει η ανάθεση της τιμής της μεταβλητής `k` στη μεταβλητή `j` και (να επιστραφεί αυτή η νέα τιμή, ώστε) στη συνέχεια η τιμή αυτή να ανατεθεί στη μεταβλητή `i`. Για να είναι συμβατή η συμπεριφορά του επαναορισμένου τελεστή ανάθεσης για μια κλάση με την παρατήρηση αυτή, ο τύπος επιστροφής του πρέπει να είναι του τύπου της κλάσης (θα είναι αναφορά για λόγους που θα εξηγήσουμε σε μεταγενέστερο κεφάλαιο και αφορά τον τύπο επιστροφής συναρτήσεων γενικότερα).

Με βάση τα παραπάνω, ο ορισμός της κλάσης `Student` επεκτείνεται με την παρακάτω δήλωση συνάρτησης-μέλους:²

```
Student& operator=(const Student& s);
```

Ας δούμε τον ορισμό της τώρα. Αυτός, κατ’ ελάχιστον πρέπει να περιέχει τις επιθυμητές ενέργειες που θέλουμε να γίνονται κατά την ανάθεση, δηλαδή:

²Μια λεπτομέρεια: απαιτείται το πρόθεμα `operator` για τη διευκόλυνση του μεταγλωττιστή.

```

name = new char[strlen(s.name)+1];
strcpy(name,s.name);
no = s.no;
year_of_studies = s.year_of_studies;

```

Για να μην αφήνουμε τις παλιές συμβολοσειρές ονομάτων σαν “σκουπίδια”, θα πρέπει, πριν γίνει η ανάθεση της νέας τιμής του μέλους `name`, να κάνουμε αποδέσμευση του χώρου της παλιάς, δηλαδή:

```
delete[] name;
```

Όμως υπάρχει το εξής πρόβλημα. Σε μεταβλητές βασικών τύπων της C++, δεν απαγορεύεται να κάνουμε ανάθεση μιας μεταβλητής στον εαυτό της, δηλαδή, για παράδειγμα:

```
i = i;
```

Στην περίπτωση αυτή, το όρισμα του τελεστή και το αριστερό μέλος — η μεταβλητή πάνω στην οποία εφαρμόζεται ως συνάρτηση-μέλος — είναι το ίδιο. Όμως, με βάση τον κώδικα που γράψαμε προηγουμένως προκειμένου να ορίσουμε τις σωστές ενέργειες που πρέπει να γίνουν κατά την ανάθεση μεταβλητών τύπου `Student`, πριν κάνουμε τη νέα ανάθεση, αποδεσμεύουμε τον χώρο και καταστρέφουμε τη συμβολοσειρά του ονόματος. Άρα ο κώδικας αυτός δεν πρέπει να εκτελεστεί σε περιπτώσεις που το όρισμα και το αριστερό μέλος είναι το ίδιο αντικείμενο. Και να που εμφανίζεται η ανάγκη να αναφερθούμε στο αντικείμενο που εφαρμόζεται μια συνάρτηση-μέλος — ο τελεστής ανάθεσης — ως φυσική οντότητα και όχι μέσω των μελών-δεδομένων του. Ποιος είναι ο τρόπος να έχουμε πρόσβαση στη διεύθυνση του αντικειμένου αυτού; Αυτή είναι η τιμή του δείκτη `this`. Οπότε, ο κώδικας του επαναορισμού του τελεστή ανάθεσης για την κλάση `Student` επαυξάνεται με τον έλεγχο εάν το όρισμα έχει διαφορετική διεύθυνση από την τιμή του `this`. Μόνον τότε πρέπει να γίνονται οι παραπάνω ενέργειες, διαφορετικά να μην γίνει τίποτα. Δηλαδή, ο κώδικας του σώματος του ορισμού γίνεται:

```

if (this != &s) { // Careful not to assign to itself, especially if
                  // some storage reclaim was made within the body

    delete[] name;
    name = new char[strlen(s.name)+1];
    strcpy(name,s.name);
    no = s.no;
    year_of_studies = s.year_of_studies;
}

```

Ολοκληρώνοντας, μένει να συζητήσουμε τι επιστρέφει η συνάρτηση. Είπαμε ότι αυτή είναι η τιμή του αντικειμένου όπως έχει διαμορφωθεί μετά την ανάθεση. Πώς μπορούμε να έχουμε πρόσβαση στην τιμή αυτή; Κάνοντας `dereference` τον δείκτη `this`. Άρα η έκφραση επιστροφής της συνάρτησης είναι:

```
return *this;
```

Συνολικά, ο επαναορισμός του τελεστή ανάθεσης για την κλάση `Student` γίνεται:

```

Student& Student::operator=(const Student& s)
{
    if (this != &s) { // Careful not to assign to itself, especially if
                      // some storage reclaim was made within the body

        delete[] name;
        name = new char[strlen(s.name)+1];
        strcpy(name,s.name);
        no = s.no;
        year_of_studies = s.year_of_studies;
        cout << "I just performed a student ASSIGNMENT ... " << endl;
    }
    return *this;
}

```

```
}

```

Προσθέσαμε την εκτύπωση ενός σχετικού μηνύματος προκειμένου, και πάλι, να κάνουμε αντιληπτή την εκτέλεση του σώματος της συνάρτησης.

Ο τελικός κώδικας του παραδείγματος του “Φοιτητή”:

Στο παρακάτω, συνοψίζουμε τον ορισμό της κλάσης Student, επαυξημένο με συναρτήσεις αρχικοποίησης και καταστροφής, συνάρτηση αρχικοποίησης μέσω αντιγραφής και επαναορισμό του τελεστή ανάθεσης.

Αρχείο επικεφαλίδας student_constr.h

```
// File: student_constr.h
class Student {
private:
    char* name;
    int no;
    int year_of_studies;
public:
    Student(const char*);
    Student(const Student&);

    ~Student();

    Student& operator=(const Student& s);

    void set_name(const char*);
    void set_no(int);
    void set_year_of_studies(int);

    char* get_name() const;
    int get_no() const;
    int get_year_of_studies() const;
};

```

Αρχείο κώδικα student_constr.cpp

```
// File: student_constr.cpp
#include <cstring>
#include <iostream>
#include "student_constr.h"
using namespace std;

////////////////////////////////////
void Student::set_name(const char* nam)
{
    delete[] name;
    name = new char[strlen(nam)+1];
    strcpy(name,nam);
}

```

```

void Student::set_no(int n)
{
    no = n;
}

void Student::set_year_of_studies(int y)
{
    year_of_studies = y;
}

////////////////////////////////////
char* Student::get_name() const
{
    return name;
}

int Student::get_no() const
{
    return no;
}

int Student::get_year_of_studies() const
{
    return year_of_studies;
}

////////////////////////////////////
Student::~Student()
{
    cout << "Deleting student with name " << name << endl;
    delete[] name;
}

////////////////////////////////////
Student::Student(const char* nam)
{
    name = new char[strlen(nam)+1];
    strcpy(name,nam);
    cout << "A student was just constructed " << endl;
}

Student::Student(const Student& s)
{
    name = new char[strlen(s.name)+1];
    strcpy(name,s.name);
    // name = s.name;    Careful: this is the result of default copy
    no = s.no;
    year_of_studies = s.year_of_studies;
    cout << "A student was just created by copying ... " << endl;
}
    
```

```

}

////////////////////////////////////
Student& Student::operator=(const Student& s)
{
    if (this != &s) { // Careful not to assign to itself,
                      // especially if some storage reclaim
                      // was made within the body
        // name = s.name;    Careful: this is the result
                            // of default assignment

        delete[] name;
        name = new char[strlen(s.name)+1];
        strcpy(name,s.name);
        no = s.no;
        year_of_studies = s.year_of_studies;
        cout << "I just performed a student ASSIGNMENT ... " << endl;
    }
    return *this;
}

```

5.6 Ορίζοντας μια Κλάση

Στην παράγραφο αυτή θα συνοψίσουμε τα σημεία που πρέπει να προσέξουμε ορίζοντας έναν νέο τύπο ως κλάση.

- Κατ' αρχήν, πρέπει να συμφωνηθεί το όνομα του τύπου και οι δηλώσεις των ονομάτων των συναρτήσεων-μελών που αντιστοιχούν στη συμπεριφορά του. Τα ονόματα αυτά πρέπει να είναι δημόσια.
- Πρέπει να αποφασιστεί η αναπαράσταση των μεταβλητών του τύπου.
- Με δεδομένη την αναπαράσταση, προχωράμε στον ορισμό των συναρτήσεων-μελών. Όποιο όνομα αφορά λεπτομέρεια αναπαράστασης ή υλοποίησης πρέπει να δηλωθεί στο ιδιωτικό τμήμα της κλάσης και να είναι κρυφό και μη χρησιμοποιήσιμο από το υπόλοιπο πρόγραμμα.
- Πρέπει να καθοριστούν οι διάφορες συναρτήσεις αρχικοποίησης. Συναρτήσεις αρχικοποίησης που επιτρέπεται να χρησιμοποιηθούν εκτός της εμβέλειας της κλάσης πρέπει να δηλωθούν στο δημόσιο τμήμα της κλάσης.
- Πρέπει να μελετηθεί αν η ζωή της μεταβλητής είναι συνυφασμένη με την πρόσβαση σε άλλους πόρους. Αν ναι, πρέπει να οριστούν κατάλληλες συναρτήσεις καταστροφής και αρχικοποίησης μέσω αντιγραφής και να γίνει επαναορισμός του τελεστή ανάθεσης. Στη γενική περίπτωση, αν χρειάζεται να οριστεί το ένα από αυτά, χρειάζεται να οριστούν όλα. Αυτό στη βιβλιογραφία αναφέρεται και ως *κανόνας των τριών*.³ Προσοχή, διότι στην περίπτωση που ο προγραμματιστής επαναορίζει τη συμπεριφορά αυτών των διαδικασιών, πρέπει να συμπεριλάβει όλες τις ενέργειες που πρέπει να γίνουν, διότι ο έλεγχος πηγαίνει αποκλειστικά στην επαναορισμένη διαδικασία.

³Μετά την έκδοση C++11, με δεδομένη επιπλέον λειτουργικότητα, ο κανόνας αναφέρεται ως “κανόνας των πέντε”, όπως θα σχολιάσουμε αργότερα.

5.7 Σύνοψη Κεφαλαίου

- Η θεμελιώδης έννοια του τύπου που ορίζεται από τον χρήστη στη C++ είναι αυτή της *κλάσης*.
- Μέσα από ένα πρόγραμμα C++, η πρόσβαση στα αντικείμενα μιας κλάσης γίνεται μέσω των *συναρτήσεων-μελών* της κλάσης που αντιστοιχούν στη συμπεριφορά του τύπου. Συντακτικά, το αντικείμενο πάνω στο οποίο εφαρμόζεται μια συνάρτηση-μέλος κατά την κλήση της αναγράφεται αριστερά από το όνομα της συνάρτησης και χωρίζεται με τελεία, . .
- Κάθε αντικείμενο που δημιουργείται “κτίζεται” από όλα τα *μέλη-δεδομένα* της κλάσης του (εκτός των στατικών).
- Ένα αντικείμενο μιας κλάσης μπορεί να δημιουργηθεί είτε ως στατική μεταβλητή, είτε ως αυτόματη μεταβλητή, είτε δυναμικά στον σωρό μέσω του τελεστή new.
- Το *δημόσιο μέρος* μιας κλάσης αποτελεί τη διεπαφή της κλάσης μέσω της οποίας χειριζόμαστε τα αντικείμενα της κλάσης έξω από την κλάση.
- Τα ονόματα του *ιδιωτικού μέρους* της κλάσης μπορούν να χρησιμοποιηθούν μόνο από τις συναρτήσεις-μέλη της κλάσης (γενικότερα, μόνο από την εμβέλεια της κλάσης) (Σημείωση: υπάρχει και άλλη μια δυνατότητα, μέσω φιλικών συναρτήσεων και φιλικών κλάσεων).
- Τα μέλη των τύπων που ορίζονται μέσω struct είναι by default δημόσια, ενώ αν ο ορισμός γίνει με χρήση class, by default είναι ιδιωτικά.
- Συνήθως, χρησιμοποιούμε διαφορετικά αρχεία για τη χρήση ενός τύπου από εκείνα του ορισμού του. Ο ορισμός μιας κλάσης, συνήθως, γράφεται σε αρχείο επικεφαλίδας και ο ορισμός των συναρτήσεων-μελών της σε αρχείο πηγαίου κώδικα. Στον ορισμό μιας κλάσης, όμως, μπορούμε να συμπεριλάβουμε τον ορισμό συναρτήσεων-μελών που θα χαρακτηρίζαμε ως inline.
- Στην περίπτωση που μέσα στο σώμα του ορισμού μιας συνάρτησης-μέλους επιθυμούμε να κάνουμε ρητή αναφορά στο αντικείμενο πάνω στο οποίο εφαρμόζεται η συνάρτηση κατά την κλήση της, χρησιμοποιούμε ένα όρισμα που υπονοείται και είναι τύπου δείκτη στην κλάση, με όνομα this.
- Κατά τη δημιουργία αντικειμένων μιας κλάσης γίνεται κλήση συγκεκριμένων συναρτήσεων που λέγονται *συναρτήσεις αρχικοποίησης*.
- Μπορεί να γίνει αρχικοποίηση αντικειμένων μιας κλάσης με αντιγραφή άλλων αντικειμένων της. Σε αυτήν την περίπτωση καλείται η *συνάρτηση αρχικοποίησης μέσω αντιγραφής*.
- Όταν πρόκειται να γίνει καταστροφή ενός αντικειμένου μιας κλάσης (οπότε θα γίνει απελευθέρωση του χώρου που του έχει ανατεθεί), καλείται μια συνάρτηση που λέγεται *συνάρτηση καταστροφής*.
- Η C++ καλεί αυτόματα συναρτήσεις αρχικοποίησης ή καταστροφής, κατά περίπτωση. Αν δεν έχει γίνει επαναορισμός τους από τον προγραμματιστή του τύπου, αυτές έχουν μια προκαθορισμένη σαφή συμπεριφορά.
- Όταν ορίζουμε μια κλάση, πρέπει να ορίσουμε συναρτήσεις αρχικοποίησης, εκτός ορισμένων περιπτώσεων που η συμπεριφορά εκείνων που συνθέτει η C++ επαρκεί.
- Υπάρχει δυνατότητα επαναορισμού από τον προγραμματιστή του τύπου των διαδικασιών: 1) της αρχικοποίησης μέσω αντιγραφής, 2) της συνάρτησης καταστροφής και 3) του τελεστή ανάθεσης, αν η αυτόματη συμπεριφορά της C++ δεν είναι η κατάλληλη για τον τύπο. Όταν γίνει επαναορισμός τους, πρέπει να συμπεριληφθούν όλες οι ενέργειες που πρέπει να γίνουν, διότι ο έλεγχος λαμβάνει υπόψη του

αποκλειστικά τους ορισμούς του προγραμματιστή. Ο “κανόνας των τριών” υποδεικνύει ότι, αν πρέπει να επαναοριστεί η μία από τις παραπάνω διαδικασίες, πρέπει να επαναοριστούν όλες.

5.8 Ασκήσεις

- 5.8.1** Να υλοποιηθεί ένας τύπος “Ονοματεπώνυμο”, `name`, με λειτουργικότητα: απόκτηση τιμής, διαβάζοντας την από την καθιερωμένη είσοδο, (`read`) και εκτύπωση τιμής στην καθιερωμένη έξοδο (`print`). Προσέξτε ότι δεν δίνεται καμία προδιαγραφή για την αναπαράστασή του.
- 5.8.2** Ορίστε, αντίστοιχα με την κλάση `Student`, μια κλάση `Employee` η οποία να αναπαριστά την έννοια του “Υπαλλήλου”. Σκεφτείτε τι χαρακτηριστικά μπορεί να έχει και ορίστε κατάλληλες συναρτήσεις-μέλη αναγνώστριες και μεταβάλλουσες για καθένα από αυτά. Προσθέστε και μέλη-δεδομένα των οποίων η τιμή να μην είναι άμεσα προσβάσιμη από το υπόλοιπο πρόγραμμα, π.χ. η χρονολογία αποφοίτησης από το Λύκειο, προκειμένου να υπολογίζεται πόσα χρόνια αργότερα ο υπάλληλος προσελήφθη.
- 5.8.3** Πειραματιστείτε με την κλάση `date` της σελίδας 143, γράφοντας τον ορισμό της κλάσης σε ένα αρχείο επικεφαλίδας και επεκτείνοντας τον κώδικα του παραδείγματος και με μια άλλη συνάρτηση `void bla()`, η οποία στο σώμα της να χρησιμοποιεί την κλάση `date`. Ορίστε την συνάρτηση `bla` σε ξεχωριστό αρχείο πηγαίου κώδικα και καλέστε την από τη συνάρτηση `main`. Για να εξασκηθείτε παραπάνω, “ζωγραφίστε” τους χώρους των μεταβλητών στη μνήμη, όπως κάναμε για το παράδειγμα της σελίδας 130.
- 5.8.4** Ορίστε μέσω κατάλληλων γλωσσικών δομών της C++ το παράδειγμα των πτήσεων του Κεφαλαίου 2. Πειραματιστείτε με την αλλαγή της αναπαράστασης του τύπου `Time`.
- 5.8.5** Να υλοποιήσετε ως κλάση τη δομή δεδομένων στοίβας ακεραίων με τη συνήθη λειτουργικότητα, αποκρύπτοντας τις αποφάσεις αναπαράστασης και υλοποίησης.
- 5.8.6** Θεωρώντας τον ορισμό της κλάσης `Student`, όπως δόθηκε στη σελίδα 151, και την παρακάτω συνάρτηση `main`, προβλέψτε το αποτέλεσμα της εκτέλεσης κάθε εντολής του προγράμματος και ζωγραφίστε την αφαιρετική εικόνα της μνήμης όπως εξελίσσεται κατά τη ροή της.

```
#include <iostream>
#include "student_constr.h"
using namespace std;

Student GS("First GlobalLast"); // Global Student

int main()
{
    Student s("First Last");      // Local Student
    cout << "This is the first output of the main function " << endl;

    Student * ps = new Student("TheDyn Student");
                                // Dynamic allocation

    Student s2("FLast");          // A second local Student

    delete ps; // Student in heap destroyed
```



```

ps = &s;    // Pointer points to local Student

s.set_no(100);
s.set_year_of_studies(1);
s.set_name("NFirst Last");

int year, no;

year = s.get_year_of_studies();
no = s.get_no();
cout << "\nYear of studies is " << year
      << " for student with no " << no
      << " and name " << s.get_name() << endl;

year = ps->get_year_of_studies();
no = ps->get_no();
cout << "\nYear of studies is " << year
      << " for student with no " << no
      << " and name " << s.get_name() << endl;

GS.set_no(101);
GS.set_year_of_studies(1);
year = GS.get_year_of_studies();
no = GS.get_no();
cout << "\nYear of studies is " << year
      << " for student with no " << no
      << " and name " << GS.get_name() << endl;

cout << "Just exiting the main function ...." << endl;

return 0;
}

```

5.8.7 Θεωρώντας τον ορισμό της κλάσης Student, όπως δόθηκε στη σελίδα 162, και την παρακάτω συνάρτηση main, προβλέψτε το αποτέλεσμα της εκτέλεσης κάθε εντολής του προγράμματος και ζωγραφίστε την αφαιρετική εικόνα της μνήμης όπως εξελίσσεται κατά τη ροή της.

```

#include <iostream>
#include "student_constr.h"
using namespace std;

int main()
{
    Student s("First Last");
    cout << "This is the first output of the main function " << endl;

    s.set_no(100);
    s.set_year_of_studies(1);
    s.set_name("NFirst Last");
}

```

```

    int year, no;

    year = s.get_year_of_studies();
    no = s.get_no();
    cout << "\nYear of studies is " << year
          << " for student with no " << no
          << " and name " << s.get_name() << endl;

    Student scopied(s);          // Student creation
    Student sassgncopied = s;    // Student creation

    sassgncopied.set_name("NewName");
    scopied = sassgncopied;      // Student assignment

    cout << "Just exiting the main function ...." << endl;

    return 0;
}

```

5.8.8 Δίνεται το παρακάτω πρόγραμμα C++:

```

#include <iostream>
using namespace std;

class A {
    int data;
public:
    A(int dt = 0) : data(dt)
    { cout << "Creating an A" << endl; }

    A(const A& a) : data(a.data)
    { cout << "Creating an A by Copying" << endl; }

    ~A()
    { cout << "Destroying an A with data" << data << endl; }

    void inc()
    { data++; }
};

int main()
{
    A a1; A a2(10); A a3(a2); A a4 = a2; A& a5 = a1; a5 = a2;
    a1.inc(); a2.inc(); a3.inc(); a4.inc(); a5.inc();
    return 0;
}

```

Αιτιολογώντας την απάντησή σας, δώστε το αποτέλεσμα της εκτέλεσής του. Για κάθε εκτύπωση, ανα-

φέρατε από ποια κλήση συνάρτησης προέκυψε.

5.8.9 Έστω ότι έχουμε το παρακάτω πρόγραμμα:

```
#include <iostream>
using namespace std;

class A {
    int data;
public:
    A(int datav = 0) : data(datav)
    { cout << "An A was just constructed" << endl; }

    A(A& a)
    { cout << "An A was just constructed by copying but ..."
      << endl;
      data = a.data; }

    A& operator=(const A&)
    { cout << "I do nothing at all" << endl; return *this; }

    ~A()
    { cout << "Destructing an A with value: " << data << endl; }

    void put(int datav)
    { data = datav; }
};

int main()
{
    A a1; A a2(a1); A a3 = a2; a1.put(5); a3 = a1;
    return 0;
}
```

Τι εκτυπώνεται κατά την εκτέλεσή του; Δώστε το πρωτότυπο της συνάρτησης που καλείται κατά την εκτέλεση κάθε εντολής της `main`.

5.9 Βιβλιογραφία

- [1] Bruce Eckel. *Thinking in C++, Volume I: Introduction to Standard C++*. 2η έκδοση. Prentice Hall, 2000.
- [2] C++ reference. <https://en.cppreference.com/w/>. Ανάκτηση: 2023-5-30.
- [3] cplusplus.com. <http://www.cplusplus.com/>. Ανάκτηση: 2023-5-30.
- [4] Bjarne Stroustrup. *The C++ Programming Language*. 4η έκδοση. Addison-Wesley Professional, 2013.
- [5] Bjarne Stroustrup. *Programming: Principles and Practice Using C++*. 2η έκδοση. Addison-Wesley Professional, 2014.
- [6] Stanley B. Lippman. *Inside the C++ Object Model*. Addison-Wesley Professional, 1996.

- [7] Stanley B. Lippman, Josée Lajoie και Barbara Moo. *C++ Primer*. 5η έκδοση. Addison-Wesley Professional, 2012.

ΣΥΝΘΕΣΗ ΚΛΑΣΕΩΝ

Σύνοψη

Στο κεφάλαιο αυτό, γίνεται συζήτηση σχετικά με τη χρήση των κλάσεων ως επέκταση των διαθέσιμων στον προγραμματιστή τύπων, εστιάζοντας στη σύνθεση κλάσεων (class composition). Συγκεκριμένα, παρουσιάζεται η συμπεριφορά των αντικειμένων ως μέλη κλάσεων στη C++, δηλαδή η ορατότητα, η αρχικοποίηση, η καταστροφή τους, και η μέριμνα που πρέπει να δείξουμε για τη σωστή ανάθεση τιμών μεταξύ των αντικειμένων της νέας κλάσης που τα περιέχει. Κλείνουμε με μια συζήτηση σχετικά με την επιλογή τύπου για τα μέλη-δεδομένα μιας νέας κλάσης που συμμετέχουν στην αναπαράσταση των αντικειμένων της κλάσης.

Προαπαιτούμενη Γνώση

Για την κατανόηση του κεφαλαίου, ο αναγνώστης απαιτείται να έχει γνώσεις διαδικαστικού προγραμματισμού και να έχει μελετήσει τις έννοιες του αντικειμενοστραφούς προγραμματισμού και της C++ από τα προηγούμενα κεφάλαια του βιβλίου.

6.1 Η Έννοια της Σύνθεσης Κλάσεων

Όπως έχουμε πει, έχοντας ορίσει μια κλάση στη C++, στην πράξη, έχουμε επεκτείνει το σύνολο των τύπων που έχει στη διάθεσή του ένας προγραμματιστής για να χρησιμοποιήσει στον κώδικα που θέλει να αναπτύξει. Μια κλάση, δηλαδή, μπορεί να χρησιμοποιηθεί σε διάφορες περιπτώσεις όπου χρειαζόμαστε τύπο. Ήδη συζητήσαμε τη χρήση κλάσεων ως τύπο σε περιπτώσεις όπως:

- ορισμός μεταβλητών εκτός συναρτήσεων,
- ορισμός μεταβλητών τοπικών σε συνάρτηση και
- μεταβλητές δημιουργημένες στον σωρό.

Επίσης, είδαμε ότι οι κλάσεις μπορούν να χρησιμοποιηθούν στις συναρτήσεις τόσο ως τύπος ορισμάτων όσο και ως τύπος επιστροφής. Αναλυτικά, θα συζητήσουμε τις περιπτώσεις αυτές σε επόμενο κεφάλαιο.

Στο τρέχον κεφάλαιο, θα συζητήσουμε τη χρήση μιας κλάσης στον τύπο μελών-δεδομένων μιας νέας κλάσης. Στην περίπτωση αυτή, ο τύπος ενός μέλους-δεδομένου μιας νέας κλάσης μπορεί να είναι κλάση ή δείκτης σε κλάση ή αναφορά σε κλάση. Στην πρώτη περίπτωση, τα αντικείμενα της νέας κλάσης έχουν ως μέλη αντικείμενα της άλλης, “κτίζονται”, δηλαδή, από αυτά.

Όταν μια κλάση χρησιμοποιείται στον τύπο ενός μέλους-δεδομένου μιας νέας κλάσης με τέτοιον τρόπο ώστε κατά τη δημιουργία ενός αντικειμένου της δεύτερης να δημιουργείται ένα αντικείμενο της πρώτης του οποίου η ζωή να είναι άρρηκτα συνδεδεμένη με τη ζωή του πρώτου λέμε ότι η δεύτερη (νέα) κλάση “έχει ένα” (*has a*) αντικείμενο της πρώτης (υπάρχουσας) κλάσης και μιλάμε για *σύνθεση κλάσεων*. Η νέα κλάση καλείται *σύνθετη*. Δεν είναι απαραίτητο η σύνθεση να αφορά ένα μέλος-δεδομένο μόνο, μπορεί και παραπάνω, μπορεί και όλα τα μέλη.

Στη C++, η σύνθεση κλάσεων εκφράζεται άμεσα με τη δήλωση του τύπου ενός (ή παραπάνω) μέλους-δεδομένου μιας νέας κλάσης να είναι (υπάρχουσα) κλάση. Σε αυτήν την περίπτωση, μια νέα κλάση έχει ως μέλη αντικείμενα άλλης κλάσης. Τι πρέπει να διασαφηνίσουμε στην περίπτωση αυτή;

1. Ποια είναι η αναπαράσταση των αντικειμένων της σύνθετης κλάσης;
2. Τι δικαιώματα πρόσβασης έχει η εμβέλεια της νέας κλάσης στα ονόματα της εμβέλειας των κλάσεων των μελών της;
3. Πώς γίνεται η αρχικοποίηση των αντικειμένων της νέας κλάσης και με ποια σειρά σε σχέση με τα μέλη τους;
4. Πώς γίνεται η διαδικασία καταστροφής των αντικειμένων της νέας κλάσης και με ποια σειρά σε σχέση με τα μέλη τους;

Όσον αφορά το πρώτο ερώτημα, η απάντηση είναι άμεση: όταν δημιουργείται ένα αντικείμενο της σύνθετης κλάσης, δημιουργούνται και αντικείμενα για τα “τουβλάκια” του που αντιστοιχούν σε μέλη-δεδομένα τύπου κλάσης.

Από πλευράς δικαιωμάτων πρόσβασης, η νέα κλάση δεν έχει παραπάνω δικαιώματα από το υπόλοιπο πρόγραμμα: έχει πρόσβαση μόνο στα δημόσια ονόματα των κλάσεων που χρησιμοποιεί ως τύπους μελών της.

Όσον αφορά τη διαδικασία αρχικοποίησης, πρώτα αρχικοποιούνται τα αντικείμενα που είναι μέλη της κλάσης και στη συνέχεια το αντικείμενο της κλάσης. Οι συναρτήσεις αρχικοποίησης που θα χρησιμοποιηθούν για τα αντικείμενα-μέλη δηλώνονται σε λίστα αρχικοποίησης στις συναρτήσεις αρχικοποίησης της σύνθετης κλάσης. Όπως έχουμε πει, η σειρά με την οποία αρχικοποιούνται τα μέλη-δεδομένα είναι η σειρά με την οποία δηλώνονται στον ορισμό της κλάσης και όχι η σειρά στη λίστα αρχικοποίησης. Είναι υποχρεωτικό, λοιπόν, να ορίσουμε συναρτήσεις αρχικοποίησης; Η απάντηση, κατ’ αρχήν, είναι όχι. Αν δεν έχει οριστεί κάποια συνάρτηση αρχικοποίησης στη σύνθετη κλάση, η C++ δημιουργεί μία που χρησιμοποιεί τις default συναρτήσεις αρχικοποίησης των μελών της — η συνάρτηση αρχικοποίησης κατηγοριοποιείται τότε ως “non-trivial synthesized”. Στην περίπτωση που οι κλάσεις των μελών-δεδομένων δεν έχουν default συναρτήσεις αρχικοποίησης, πρέπει να οριστούν συναρτήσεις αρχικοποίησης στη σύνθετη κλάση, κατ’ ελάχιστο για να περάσουν τις τιμές των παραμέτρων στις συναρτήσεις αρχικοποίησης των μελών μέσω λίστας αρχικοποίησης.

Κατά την καταστροφή αντικειμένων, ακολουθείται η αντίστροφη σειρά από εκείνη της δημιουργίας και της αρχικοποίησης. Δηλαδή, πρώτα εκτελείται το σώμα της συνάρτησης καταστροφής της σύνθετης κλάσης και μετά εκείνες των μελών της, με την αντίστροφη σειρά της δήλωσής τους στον ορισμό της κλάσης.

Μπορούμε να έχουμε σύνθεση κλάσεων σε οποιοδήποτε βάθος. Δηλαδή, σύνθετες κλάσεις μπορούν να χρησιμοποιηθούν ως τύποι μελών-δεδομένων νέων σύνθετων κλάσεων. Οι παραπάνω παρατηρήσεις ισχύουν για οποιοδήποτε βάθος σύνθεσης.

6.1.1 Ένα Πρώτο Παράδειγμα Σύνθετης Κλάσης

Για να γίνουν τα παραπάνω πιο κατανοητά, θα τα σχολιάσουμε μέσω ενός παραδείγματος. Το παράδειγμα είναι αρκετά τεχνητό αλλά αναδεικνύει τα σημεία τα οποία θέλουμε να διασαφηνίσουμε.

Ας θεωρήσουμε ότι έχουν ήδη οριστεί — οπότε είναι διαθέσιμοι — ένας τύπος “Ημερομηνίας” και ένας τύπος “Χρονικής Στιγμής”, με πολύ απλή λειτουργικότητα, μέσω των κλάσεων `Date` και `Time`, αντίστοιχα. Συγκεκριμένα, ο τύπος της “Ημερομηνίας” αναπαριστάνει την έννοια της ημερομηνίας (ημέρα, μήνας, έτος) και δίνεται η δυνατότητα εκτύπωσης της τιμής μιας “Ημερομηνίας” η οποία έχει δοθεί κατά την αρχικοποίηση. Έστω, ότι η αρχικοποίηση επιδέχεται default τιμών οι οποίες είναι η τιμή “1” για την ημέρα, “1” για τον μήνα και “2006” για το έτος. Ο τύπος της “Χρονικής Στιγμής” αναπαριστάνει την έννοια της χρονικής στιγμής (ώρες, λεπτά). Η λειτουργικότητά του ας είναι αντίστοιχη με εκείνην του τύπου της “Ημερομηνίας”, όπου οι default τιμές κατά την αρχικοποίηση είναι “0” για τις ώρες και “0” για τα λεπτά.

Οι υλοποιήσεις των τύπων μπορεί να είναι πολύ απλές: για κάθε επιμέρους πληροφορία, να υπάρχει ένα μέλος-δεδομένο, του οποίου ο τύπος του ας είναι `int`. Κατά συνέπεια, δεν υπάρχει λόγος ορισμού συναρτήσεων καταστροφής. Εμείς θα ορίσουμε, όμως, ώστε να αντιλαμβανόμαστε την επικείμενη καταστροφή των αντικειμένων λόγω της εκτέλεσης του σώματός τους.

Ας υποθέσουμε, λοιπόν, ότι οι ορισμοί των κλάσεων `Date` και `Time` είναι οι εξής:

Ο τύπος της “Ημερομηνίας”, `Date`:

```
class Date {

    int day;
    int month;
    int year;

public:

    Date(int d = 1, int m = 1, int y = 2006)
        : day(d), month(m), year(y) // non-class types,
                                     // i.e. initializer list
                                     // alternative to assignment
                                     // in the body
    {
        // day = d; month = m; year = y;
        cout << "A Date was just created with value: "
              << day << ' ' << month << ' ' << year << endl; }

    ~Date()
    { cout << '\n' << "A Date is to be destroyed with value: "
      << day << ' ' << month << ' ' << year << endl; }

    void print() const
    { cout << day << ' ' << month << ' ' << year << endl; }
};
```

Ο τύπος της “Χρονικής Στιγμής”, `Time`:

```
class Time {

    int hours;
```

```

    int mins;

public:
    Time(int h = 0, int m = 0) : hours(h), mins(m)
    {   cout << "A Time was just created with value: "
        << hours << ' ' << mins << endl; }

    ~Time()
    {   cout << '\n' << "A Time is to be destroyed with value: "
        << hours << ' ' << mins << endl; }

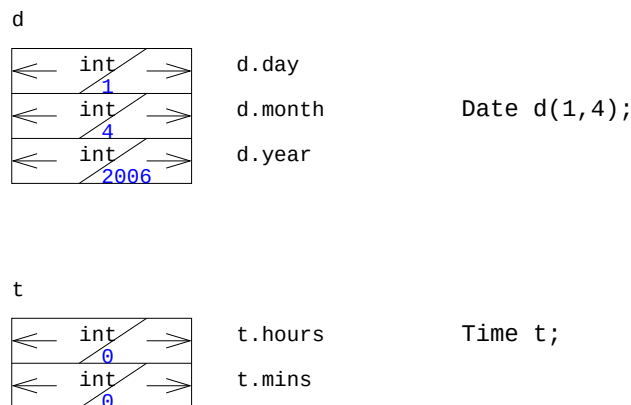
    void print() const
    {   cout << hours << " hours   and " << mins << " mins " << endl; }
};

```

Ας υποθέσουμε ότι έχουμε μια μεταβλητή *d* τύπου *Date* και *t* τύπου *Time*, ορίζοντάς τις με τον εξής τρόπο:

Date *d*(1,4); Time *t*;

Οι αναπαραστάσεις τους φαίνονται στο Σχήμα 6.1.



Σχήμα 6.1: Οι αναπαραστάσεις των μεταβλητών *d* τύπου *Date* και *t* τύπου *Time*

Έστω, τώρα, ότι πρόκειται να αναπτύξουμε μια εφαρμογή στην οποία υπάρχει ανάγκη ορισμού τύπου “Λεπτομερούς Ημερομηνίας”, η οποία αναπαριστάει την πλήρη πληροφορία ημερομηνίας και χρονικής στιγμής. Ας υποθέσουμε ότι η λειτουργικότητα του τύπου είναι στοιχειώδης: απλά μπορεί να εκτυπωθεί η τιμή με την οποία έχει γίνει η αρχικοποίηση. Σε αυτήν την περίπτωση, μπορούμε να ορίσουμε μια κλάση, *DetailedDate*, και για την αναπαράσταση των αντικειμένων της να χρησιμοποιήσουμε δύο μέλη-δεδομένα που να αντιστοιχούν στην πληροφορία ημερομηνίας και την πληροφορία της χρονικής στιγμής, αντίστοιχα. Για την υλοποίηση των μελών μπορούμε να χρησιμοποιήσουμε τις ήδη υπάρχουσες κλάσεις *Date* και *Time*. Να ορίσουμε δηλαδή μια σύνθετη κλάση, την *DetailedDate*, η οποία έχει ένα (“has a”) μέλος-δεδομένο τύπου κλάσης *Date* και έχει ένα (“has a”) μέλος-δεδομένο τύπου κλάσης *Time*.

Στην περίπτωση αυτή, η υλοποίηση της κλάσης *DetailedDate* μπορεί να είναι ως εξής:


```

class DetailedDate {

    Date date;
    Time time;

public:

    DetailedDate()
    // The default constructors of the data members are used
    {   cout << "DetailedDate was just created with value:"
        << endl;

    // NO VISIBILITY in private part
    //   cout << date.day << ' ' << date.month << ' '
    //       << date.year << endl;
    //   cout << time.hours << " hours and "
    //       << time.mins << " mins " << endl;

        date.print();
        time.print();
    }

    // DetailedDate(int d, int mo, int y, int h, int mi) :
    DetailedDate(int d, int mo = 0, int y = 0, int h = 0, int mi = 0) :
        date(d,mo,y), time(h,mi)
    // Initializer list is required to pass the values
    // to the constructors of data members.
    {   cout << "DetailedDate was just created with value:"
        << '\n' << endl;

        date.print();
        time.print();
    }

    ~DetailedDate()
    {   cout << '\n'
        << "A DetailedDate is to be destroyed with value: " << endl;
        date.print();
        time.print();
    }

    void print() const
    {   cout << "I am printing a DetailedDate " << endl;
        date.print();
        time.print();
    }
};
    
```

Στον ορισμό της κλάσης `DetailedDate` παρατηρούμε τα εξής:

- Έχει μόνο δύο μέλη-δεδομένα: το μέλος `date` τύπου `Date` και το μέλος `time` τύπου `Time`.
- Στην εμφάνισή της δεν υπάρχει δικαίωμα πρόσβασης στα ιδιωτικά ονόματα των κλάσεων `Date` και `Time`. Για παράδειγμα, αν αποσχολιάσουμε τον κώδικα:

```
// cout << date.day << ' ' << date.month << ' '
//      << date.year << endl;
// cout << time.hours << " hours and "
//      << time.mins << " mins " << endl;
```

θα προκύψει σφάλμα, διότι από την εμφάνιση της κλάσης `DetailedDate`, γίνεται προσπάθεια πρόσβασης σε μέλη-δεδομένα των κλάσεων `Date` και `Time` των οποίων τα ονόματα είναι ιδιωτικά. Όπως είπαμε, η εμφάνιση της κλάσης `DetailedDate` μπορεί να χρησιμοποιήσει μόνο τα δημόσια ονόματα των κλάσεων `Date` και `Time`, οπότε προκειμένου να εκτυπωθούν οι επιμέρους τιμές της πληροφορίας, αυτό μπορεί να γίνει μόνο μέσω κλήσεων των συναρτήσεων-μελών `print` των κλάσεων αυτών.

- Δεδομένου ότι οι κλάσεις `Date` και `Time` έχουν default συναρτήσεις αρχικοποίησης, δεν είναι απαραίτητο να ορίσουμε συναρτήσεις αρχικοποίησης στην κλάση `DetailedDate`. Στην περίπτωση αυτή, όταν θα δημιουργούνται ένα αντικείμενο της κλάσης, θα καλούντο οι συναρτήσεις αρχικοποίησης των κλάσεων `Date` και `Time` με τις default τιμές των ορισμάτων τους για να αρχικοποιήσουν τα μέλη-δεδομένα. Αν, λοιπόν, αυτή η συμπεριφορά μας είναι επαρκής, θα μπορούσαμε να μην είχαμε ορίσει καμία συνάρτηση αρχικοποίησης στην κλάση `DetailedDate`.
- Στον ορισμό της κλάσης `DetailedDate`, όμως, έχουμε ορισμό default συνάρτησης αρχικοποίησης και ξεχωριστό ορισμό για non-default. Στο σχολιασμένο πρωτότυπο του κώδικα της non-default συνάρτησης, δεν περιλαμβάνονται default τιμές. Στον μη σχολιασμένο, περιλαμβάνονται και κάποιες default τιμές, διαφορετικές, όμως, από εκείνες των κλάσεων των μελών. Σε κάθε περίπτωση, οι τιμές των ορισμάτων περνούν στις συναρτήσεις αρχικοποίησης των μελών μέσω της λίστας αρχικοποίησης που, για το κάθε μέλος, προσδιορίζει ποια συνάρτηση αρχικοποίησης θα χρησιμοποιηθεί, καθώς και τις τιμές των ορισμάτων της.
- Ορισμός συνάρτησης καταστροφής δεν απαιτείται για καμία από τις συγκεκριμένες κλάσεις. Παρ' όλα αυτά, ορίστηκε και για την κλάση `DetailedDate`, για λόγους εξοικείωσης με τη σειρά κλήσης των συναρτήσεων καταστροφής στην περίπτωση σύνθετων κλάσεων. Εκτυπώνεται ένα μήνυμα προκειμένου να γίνεται αντιληπτή η εκτέλεση του σώματός της και καλούνται συναρτήσεις που εφαρμόζονται στα μέλη-δεδομένα της κλάσης, ώστε να γίνει σαφές ότι αυτά δεν έχουν καταστραφεί όταν εκτελείται η συνάρτηση.

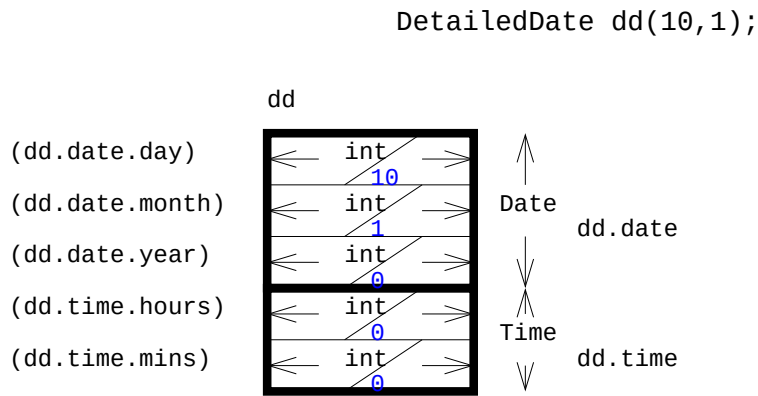
Ας υποθέσουμε ότι έχουμε μια μεταβλητή `dd` τύπου `DetailedDate`, ορίζοντάς την με τον εξής τρόπο:

```
DetailedDate dd(10,1);
```

Η αναπαράστασή της φαίνεται στο Σχήμα 6.2.

Παρατηρήστε την αναπαράσταση της μεταβλητής `dd` μέσω δύο και μόνο “τουβλακιών”, `dd.date` και `dd.time`, άσχετα αν καθένα από αυτά αποτελείται από επιμέρους “τουβλάκια”.

Ας συνδυάσουμε, τώρα, τους παραπάνω ορισμούς των κλάσεων `Date`, `Time` και `DetailedDate` με τη συνάρτηση `main` που ακολουθεί.



Σχήμα 6.2: Η αναπαράσταση της μεταβλητής `dd` τύπου `DetailedDate`

```
int main()
{
    DetailedDate dd1;

    dd1.print();

    return 0;
}
```

Η εκτύπωση που προκύπτει από την εκτέλεσή της είναι:

A Date was just created with value: 1 1 2006

A Time was just created with value: 0 0

DetailedDate was just created with value:

1 1 2006

0 hours and 0 mins

I am printing a DetailedDate

1 1 2006

0 hours and 0 mins

A DetailedDate is to be destroyed with value:

1 1 2006

0 hours and 0 mins

A Time is to be destroyed with value: 0 0

A Date is to be destroyed with value: 1 1 2006

Η ανάλυση των εκτυπώσεων έχει ως εξής:

- Τα μηνύματα:

A Date was just created with value: 1 1 2006

A Time was just created with value: 0 0

προκύπτουν από την εκτέλεση των σωμάτων των default συναρτήσεων αρχικοποίησης των κλάσεων `Date` και `Time`, για να αρχικοποιηθούν τα μέλη-δεδομένα της μεταβλητής `dd1`. Καλείται πρώτα

η συνάρτηση αρχικοποίησης της `Date` και στη συνέχεια της `Time`, διότι στον ορισμό της κλάσης `DetailedDate` τα μέλη δηλώνονται ως εξής:

```
Date date;
Time time;
```

- Στη συνέχεια, τα μηνύματα:

```
DetailedDate was just created with value:
1 1 2006
0 hours and 0 mins
```

προκύπτουν από την εκτέλεση του σώματος της default συνάρτησης αρχικοποίησης της κλάσης `DetailedDate`. Η πρώτη εκτύπωση προκύπτει από το ίδιο το σώμα και οι άλλες δύο από τις κλήσεις των συναρτήσεων-μελών `print` των κλάσεων `Date` και `Time`, εφαρμοσμένες στα, ήδη αρχικοποιημένα, μέλη-δεδομένα της μεταβλητής `dd1` που αρχικοποιείται.

- Στη συνέχεια, τα μηνύματα:

```
I am printing a DetailedDate
1 1 2006
0 hours and 0 mins
```

προκύπτουν από την εκτέλεση της συνάρτησης-μέλους `print` της κλάσης `DetailedDate`, που καλείται από τη συνάρτηση `main`, εφαρμοζόμενη στη μεταβλητή `dd1`.

- Ακολουθούν τα μηνύματα από τις συναρτήσεις καταστροφής που καλούνται λόγω της καταστροφής της μεταβλητής `dd1`. Η σειρά εκτέλεσης είναι η αντίστροφη της σειράς της αρχικοποίησης. Πρώτα καλείται η συνάρτηση καταστροφής της `DetailedDate` και εκτυπώνονται τα μηνύματα:

```
A DetailedDate is to be destroyed with value:
1 1 2006
0 hours and 0 mins
```

Το πρώτο μήνυμα εκτυπώνεται απευθείας από το σώμα της συνάρτησης και τα άλλα δύο από τις κλήσεις των συναρτήσεων-μελών `print` των κλάσεων `Date` και `Time`, εφαρμοσμένες στα μέλη-δεδομένα της μεταβλητής `dd1` και τα οποία ακόμα δεν έχουν καταστραφεί.

Τα μηνύματα:

```
A Time is to be destroyed with value: 0 0
```

```
A Date is to be destroyed with value: 1 1 2006
```

προκύπτουν από τις συναρτήσεις καταστροφής των κλάσεων των μελών-δεδομένων της μεταβλητής `dd1` που εκτελούνται με την αντίστροφη σειρά της δήλωσης των μελών στην κλάση `DetailedDate`.

Για να εξασκηθείτε λίγο ακόμα, πειραματιστείτε, συνδυάζοντας τους παραπάνω ορισμούς των κλάσεων `Date`, `Time` και `DetailedDate` με τη συνάρτηση `main` που ακολουθεί, βγάζοντας σταδιακά τα σχόλια των ορισμών των μεταβλητών στο σώμα της. Παρατηρήστε τις διαφορές στις `default` τιμές των ορισμάτων των συναρτήσεων αρχικοποίησης μεταξύ του τύπου `DetailedDate` και των τύπων `Date` και `Time`.

Αποσχολιάζοντας παραπάνω από μία γραμμές, παρατηρήστε τη σειρά που καλούνται οι συναρτήσεις καταστροφής: η μεταβλητή που ορίστηκε τελευταία στη συνάρτηση `main` θα είναι και εκείνη που θα καταστραφεί πρώτη. Γενικότερα, οι μεταβλητές καταστρέφονται με την αντίστροφη σειρά με την οποία δημιουργήθηκαν. Η σειρά αυτή γίνεται φανερή από τις εκτυπώσεις από τα σώματα των συναρτήσεων καταστροφής.

```
int main()
{
    // Date d1;
    // Date d2(3);
    // Date d3(3,4);
    // Date d4(3,4,2005);

    // Time t1;
    // Time t2(5);
    // Time t3(2,35);

    // dd1, dd2 and dd3 are instances/objects of DetailedDate

    DetailedDate dd1;
    // DetailedDate dd2(10);
    // DetailedDate dd3(10,1);
    // DetailedDate dd4(10,1,2007);
    // DetailedDate dd5(10,1,2007,11);
    // DetailedDate dd6(25,10,2005,3,45);

    dd1.print();

    return 0;
}
```

6.2 Εκτενέστερη Συζήτηση πάνω στις Συναρτήσεις Αρχικοποίησης

Στο παραπάνω παράδειγμα, είχαμε μια σύνθετη κλάση, την `DetailedDate`, της οποίας όλα τα μέλη-δεδομένα ήταν τύπου κλάσης. Ας πάρουμε, τώρα, ένα πιο γενικό παράδειγμα όπου έχουμε μια σύνθετη κλάση της οποίας τα μέλη-δεδομένα είναι τόσο τύπου κλάσης, όσο και βασικών τύπων. Με την ευκαιρία του παραδείγματος αυτού, θα τονίσουμε περισσότερο την ανάγκη για προσοχή στον ορισμό των συναρτήσεων αρχικοποίησης.

Ας προχωρήσουμε με το παράδειγμά μας. Ας υποθέσουμε ότι έχουμε ορισμένο έναν τύπο `date` που αναπαριστά την έννοια της “Ημερομηνίας”, όπως την θεωρούσαμε στο προηγούμενο κεφάλαιο. Ας υποθέσουμε ότι ο ορισμός του τύπου `date` είναι ο εξής:

```
// File: day_class.h
#include <iostream>
using namespace std;
```

```

class date {
private:
    int day, month, year;
public:
    date() : month(0), day(0), year(0)
    { cout << "Creating a date!" << endl; }

    ~date()
    { cout << "Destroying a date with: " << day
      << " " << month << " " << year << endl; }

    void set(int i, int j, int k)
    { day = i; month = j; year = k; }

    void get(int& i, int& j, int& k) const
    { i = day; j = month; k = year; }
};

```

Πέρα από τη συνηθισμένη λειτουργικότητα, όπως την έχουμε συζητήσει για τον τύπο αυτό, προσέξτε ότι έχουμε μόνο default συνάρτηση αρχικοποίησης. Επίσης, έχουμε και συνάρτηση καταστροφής, παρότι δεν χρειαζόταν στην πραγματικότητα.

Έστω ότι έχουμε την ανάγκη αναπαράστασης μιας έννοιας “Φοιτητή” η οποία, πέρα από τη φύση και τη λειτουργικότητα όπως την έχουμε συζητήσει έως τώρα, εμπεριέχει και την πληροφορία της ημερομηνίας πρώτης εγγραφής στον κύκλο σπουδών του φοιτητή και επέκταση της λειτουργικότητας για την πληροφορία αυτή. Στην περίπτωση αυτή, θα μπορούσαμε να προσθέσουμε ένα επιπλέον μέλος-δεδομένο για την πρόσθετη πληροφορία και να χρησιμοποιήσουμε τον τύπο `date` ως τύπο του. Δηλαδή, ο ορισμός της κλάσης `Student` που αντιστοιχεί στην νέα έννοια του “Φοιτητή” θα μπορούσε να είναι:

```

// File: student_constr.h
#include "day_class.h"

class Student {
private:
    char* name;
    int no;
    int year_of_studies;
    date RegistrationDate;
public:
    Student(const char*);
    Student(const Student&);

    ~Student();

    void set_name(const char*);
    void set_no(int);
    void set_year_of_studies(int);
    void set_date(int, int, int);

```

```

char* get_name() const;
int get_no() const;
int get_year_of_studies() const;
void get_date(int&, int&, int&) const;
};

```

Το επιπλέον μέλος-δεδομένο είναι το μέλος `RegistrationDate`. Για την ανάθεση και ανάκτηση της σχετικής πληροφορίας, προσθέσαμε στο δημόσιο μέρος της κλάσης τις συναρτήσεις-μέλη:

```
void set_date(int, int, int);
```

και

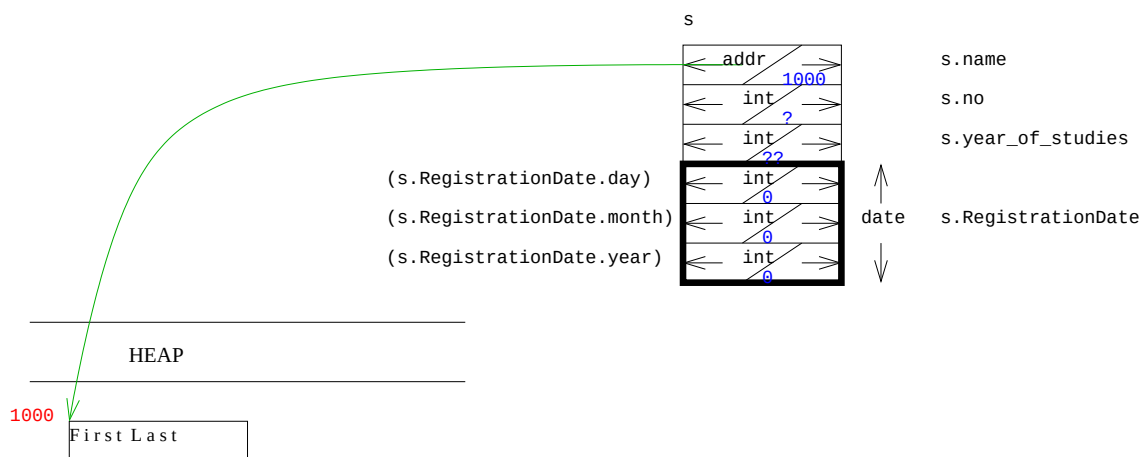
```
void get_date(int&, int&, int&) const;
```

αντίστοιχα.

Με βάση τους παραπάνω ορισμούς των κλάσεων `date` και `Student`, η αναπαράσταση μιας μεταβλητής `s` τύπου `Student` που ορίζεται ως

```
Student s("First Last");
```

φαίνεται στο Σχήμα 6.3.



Σχήμα 6.3: Η αναπαράσταση της μεταβλητής `s` τύπου `Student`

Για να ολοκληρώσουμε τον ορισμό της κλάσης `Student`, πρέπει να προσθέσουμε τους ορισμούς των επιπλέον συναρτήσεων-μελών. Δηλαδή,

```

void Student::set_date(int d, int m, int y)
{
    RegistrationDate.set(d,m,y);
}

void Student::get_date(int& d, int& m, int& y) const
{
    RegistrationDate.get(d,m,y);
}

```

Αρκεί, όμως; Τι γίνεται με την αρχικοποίηση των αντικειμένων της κλάσης `Student`; Οι συναρτήσεις αρχικοποίησης της κλάσης `Student`, αν παραμείνουν να είναι ορισμένες ως:

```

Student::Student(const char* nam)
{
    name = new char[strlen(nam)+1];
}

```

```

    strcpy(name, nam);
    cout << "A student was just constructed " << endl;
}

Student::Student(const Student& s)
{
    name = new char[strlen(s.name)+1];
    strcpy(name, s.name);
    no = s.no;
    year_of_studies = s.year_of_studies;
    cout << "A student was just created by copying ... " << endl;
}

```

λειτουργούν ικανοποιητικά;

Η συνάρτηση αρχικοποίησης `Student::Student(const char* nam)`, με βάση τον παραπάνω ορισμό, υπονοεί default αρχικοποίηση για το μέλος-δεδομένο `RegistrationDate`. Αυτή είναι εφικτή μέσω χρήσης της default συνάρτησης αρχικοποίησης της κλάσης `date` η οποία αρχικοποιεί τα μέλη του με μηδενικές τιμές. Επομένως, default αρχικοποίηση για το μέλος-δεδομένο `RegistrationDate` είναι εφικτή και αν η συμπεριφορά της είναι ικανοποιητική, δεν χρειάζεται να κάνουμε καμία αλλαγή.

Ας δούμε τη συνάρτηση αρχικοποίησης `Student::Student(const Student& s)` τώρα. Έτσι όπως είναι ορισμένη, δεν κάνει καμία υπόδειξη αρχικοποίησης για το μέλος-δεδομένο `RegistrationDate` της νέας μεταβλητής. Επομένως, θα γίνει default αρχικοποίηση για το μέλος αυτό με το αποτέλεσμα που περιγράψαμε προηγουμένως: το μέλος-δεδομένο `RegistrationDate` της νέας μεταβλητής θα αρχικοποιηθεί με μηδενικές τιμές. Αυτό, βέβαια, δεν είναι επιθυμητό, εφόσον αναφερόμαστε στον ορισμό της συνάρτησης αρχικοποίησης μέσω αντιγραφής: θέλουμε και το μέλος-δεδομένο `RegistrationDate` της νέας μεταβλητής να αρχικοποιηθεί μέσω αντιγραφής. Πρέπει, λοιπόν, να το δηλώσουμε ρητά. Το μέσον για να το κάνουμε αυτό είναι η λίστα αρχικοποίησης. Έτσι, η συνάρτηση αρχικοποίησης μέσω αντιγραφής για την κλάση `Student` πρέπει να αλλάξει και να γίνει, για παράδειγμα, ως εξής:

```

Student::Student(const Student& s)
: RegistrationDate(s.RegistrationDate)
    // Copy constructor to be used (even
    // if not defined by the user).
    // If this is in comments, the default
    // constructor is used for initializing
    // the data member RegistrationDate
{
    name = new char[strlen(s.name)+1];
    strcpy(name, s.name);
    no = s.no;
    year_of_studies = s.year_of_studies;
    cout << "A student was just created by copying ... " << endl;
}

```

Ακόμα και αν δεν έχουμε ορίσει — και καλώς — συνάρτηση αρχικοποίησης μέσω αντιγραφής για την κλάση `date`, εφόσον ορίζεται συνάρτηση αρχικοποίησης μέσω αντιγραφής στη σύνθετη κλάση `Student`, πρέπει να προσδιορίσουμε αρχικοποίηση μέσω αντιγραφής για το μέλος `RegistrationDate` στη λίστα αρχικοποίησης της συνάρτησης.

Γενικότερα, αν ορίζεται ρητά συνάρτηση αρχικοποίησης μέσω αντιγραφής για μια σύνθετη κλάση, τα μέλη τύπου κλάσης πρέπει να αρχικοποιούνται μέσω αντιγραφής, προσδιορίζοντάς το στη λίστα αρχικοποίησης της

συνάρτησης.

Για να εξασκηθείτε με τα παραπάνω, θεωρήστε τους ορισμούς των τύπων `date` και `Student` όπως τους ολοκληρώσαμε προηγουμένως και την παρακάτω συνάρτηση `main`. “Ζωγραφίστε” την αναπαράσταση των μεταβλητών της αφαιρετικά στη μνήμη και πειραματιστείτε με τα εναλλακτικά της χρήσης ή όχι αρχικοποίησης μέσω αντιγραφής στο μέλος-δεδομένο `RegistrationDate` στον ορισμό της συνάρτησης αρχικοποίησης της κλάσης `Student`.

```
#include <iostream>
#include "student_constr.h"
using namespace std;

int main()
{
    Student s("First Last");
    cout << "This is the first output of the main function " << endl;

    s.set_no(100);
    s.set_year_of_studies(1);
    s.set_name("NFirst Last");
    s.set_date(20,11,2005);

    int year, no;

    year = s.get_year_of_studies();
    no = s.get_no();
    cout << "\nYear of studies is " << year
         << " for student with no " << no
         << " and name " << s.get_name() << endl;

    int rday, rmonth, ryear;
    s.get_date(rday,rmonth,ryear);
    cout << "and the registration date is:" << rday << " "
         << rmonth << " " << ryear << endl;

    Student s1(s);

    cout << "Just exiting the main function ...." << endl;

    return 0;
}
```

6.3 Σύνθετες Κλάσεις: Επαναορισμός του Τελεστή Ανάθεσης

Μιλώντας για τις λεπτομέρειες που πρέπει να διαχειριστούμε ορίζοντας σύνθετες κλάσεις, θα πρέπει να αναφερθούμε και στην περίπτωση επαναορισμού του τελεστή ανάθεσης. Όπως είπαμε, κατά τον ορισμό της κλάσης `Student`, έχει χρειαστεί ο επαναορισμός του τελεστή ανάθεσης, ώστε να έχουμε την επιθυμητή και ορθή ανάθεση του μέλους-δεδομένου `Student::name` που είναι τύπου `char*`.

Στη νέα, σύνθετη, κλάση `Student`, αντίστοιχα με τον ορισμό της συνάρτησης αρχικοποίησης μέσω αντι-

γραφής της κλάσης, στον επαναορισμό του τελεστή ανάθεσης πρέπει να λάβουμε υπόψη μας και το νέο μέλος-δεδομένο. Η παρατήρηση αυτή είναι ανεξάρτητη από το εάν αυτό το μέλος είναι τύπου κλάσης ή όχι. Στο σώμα του επαναορισμού του τελεστή ανάθεσης, δηλαδή, πρέπει να συμπεριλάβουμε και ανάθεση μεταξύ των μελών-δεδομένων `RegistrationDate`. Αυτό που πρέπει να επισημανθεί είναι ότι εάν έχει επαναοριστεί ο τελεστής ανάθεσης στην κλάση `date`, θα κληθεί ο ορισμός αυτός.

Στον παρακάτω κώδικα, είναι συγκεντρωμένοι οι πλήρεις ορισμοί των κλάσεων `date` και `Student`. Στην κλάση `date` αλλάξαμε τη συνάρτηση αρχικοποίησης, ώστε να δέχεται και non default αρχικοποιήσεις και συμπεριλάβαμε, παρότι δεν ήταν απαραίτητο, επαναορισμό του τελεστή ανάθεσης.

Ο τύπος `date`:

```
// File: day_class.h
#include <iostream>
using namespace std;

class date {
private:
    int day, month, year;
public:

    date(int i = 0, int j = 0, int k = 0) : month(j), day(i), year(k)
    { cout << "Creating a date!" << endl; }

    date(const date& d) : day(d.day), month(d.month), year(d.year)
    { cout << "Creating a date by copying!" << endl; }

    ~date()
    { cout << "Destroying a date!" << endl; }

    // Experiment with keeping and deleting the assignment redefinition
    date& operator=(const date& d)
    { day = d.day; year = d.year; month = d.month;
      cout << "I just performed a Date Assignment ... " << endl;
      return *this; }

    void set(int i, int j, int k)
    { day = i; month = j; year = k; }

    void get(int& i, int& j, int& k) const
    { i = day; j = month; k = year; }
};
```

Ο τύπος `Student`:

```
//File: student_constr.h
#include "day_class.h"

class Student {
private:
    char* name;
```

```

    int no;
    int year_of_studies;
    date RegistrationDate;
public:
    Student(const char*);
    ~Student();

    Student(const Student&);

// Experiment with keeping and deleting the assignment redefinition
// - in case of deletion, don't forget to comment out
// the definition of the assignment operator in the source file -
    Student& operator=(const Student& s);

    void set_name(const char*);
    void set_no(int);
    void set_year_of_studies(int);
    void set_date(int, int, int);

    char* get_name() const;
    int get_no() const;
    int get_year_of_studies() const;
    void get_date(int&, int&, int&) const;
};

```

```

// File: student_constr.cpp
#include <cstring>
#include <iostream>
#include "student_constr.h"
using namespace std;

////////////////////////////////////
void Student::set_name(const char* nam)
{
    delete[] name;
    name = new char[strlen(nam)+1];
    strcpy(name,nam);
}

void Student::set_no(int n)
{
    no = n;
}

void Student::set_year_of_studies(int y)
{
    year_of_studies = y;
}

```

```

void Student::set_date(int d, int m, int y)
{
    RegistrationDate.set(d,m,y);
}

////////////////////////////////////
char* Student::get_name() const
{
    return name;
}

int Student::get_no() const
{
    return no;
}

int Student::get_year_of_studies() const
{
    return year_of_studies;
}

void Student::get_date(int& d, int& m, int& y) const
{
    RegistrationDate.get(d,m,y);
}

////////////////////////////////////
Student::~Student()
{
    cout << "Deleting student with name " << name << endl;
    delete[] name;
}

////////////////////////////////////
Student::Student(const char* nam)
{
    name = new char[strlen(nam)+1];
    strcpy(name,nam);
    cout << "A student was just constructed " << endl;
}

////////////////////////////////////
Student::Student(const Student& s)
// For experimentation, comment out the following initialization,
// or keep one at a time
    : RegistrationDate(s.RegistrationDate)
// : RegistrationDate(5,10)
// : RegistrationDate(5,10,2007)
{

```

```

        name = new char[strlen(s.name)+1];
        strcpy(name,s.name);
// name = s.name;    Careful: this is the result of default copy
        no = s.no;
        year_of_studies = s.year_of_studies;
// RegistrationDate = s.RegistrationDate; // Not a good idea!
// Assignment!

        cout << "A student was just created by copying ... " << endl;
    }

    //////////////////////////////////////
Student& Student::operator=(const Student& s)
{
// name = s.name;    Careful: this is the result
//                    of default assignment

    if (this != &s) { // Careful not to assign to itself,
                     // especially if some storage reclaim
                     // was made within the body

        delete[] name;
        name = new char[strlen(s.name)+1];
        strcpy(name,s.name);
        no = s.no;
        year_of_studies = s.year_of_studies;
        RegistrationDate = s.RegistrationDate; // Experiment with
                                                // commenting out this line

        cout << "I just performed a student ASSIGNMENT ... " << endl;
    }
    return *this;
}

```

Συνδυάστε τους παραπάνω ορισμούς με τη συνάρτηση main που ακολουθεί.

```

#include <iostream>
#include "student_constr.h"
using namespace std;

int main()
{
    Student s("First Last");
    cout << "This is the first output of the main function " << endl;

    s.set_no(100);
    s.set_year_of_studies(1);
    s.set_name("NFirst Last");
    s.set_date(20,11,2005);

    int year, no;
}

```

```

year = s.get_year_of_studies();
no = s.get_no();
cout << "\nYear of studies is " << year
      << " for student with no " << no
      << " and name " << s.get_name() << endl;

int rday, rmonth, ryear;
s.get_date(rday,rmonth,ryear);
cout << "and the registration date is:" << rday << " "
      << rmonth << " " << ryear << endl;

////////////////////////////////////
Student s1("The NEW STUDENT");
s1.set_date(2,11,2010);
s1.get_date(rday,rmonth,ryear);
cout << "and the new student registration date is:"
      << rday << " " << rmonth << " " << ryear << endl;

////////////////////////////////////
Student scopied(s1);
scopied.get_date(rday,rmonth,ryear);
cout << "and the scopied registration date is:"
      << rday << " " << rmonth << " " << ryear << endl;

scopied.set_date(21,12,2115);
scopied.get_date(rday,rmonth,ryear);
cout << "and the scopied registration date is now:"
      << rday << " " << rmonth << " " << ryear << endl;

////////////////////////////////////
scopied = s;

scopied.get_date(rday,rmonth,ryear);
cout << "and the scopied registration date is:"
      << rday << " " << rmonth << " " << ryear << endl;

////////////////////////////////////

cout << "Just exiting the main function ...." << endl;

return 0;
}

```

Στη συνάρτηση main, ορίζονται τρεις μεταβλητές τύπου Student, οι s, s1 και scopied. Οι μεταβλητές s και s1 αρχικοποιούνται με χρήση συμβολοσειρών, όπως φαίνεται από τους ορισμούς τους:

```

Student s("First Last");
Student s1("The NEW STUDENT");

```

Η μεταβλητή scopied αρχικοποιείται μέσω αντιγραφής από τη μεταβλητή s1, όπως φαίνεται από τον ορισμό της:

```
Student scopied(s1);
```

Μέσα στο σώμα της main γίνεται η ανάθεση:

```
scopied = s;
```

Προκειμένου να κατανοήσετε καλύτερα τη συμπεριφορά του τελεστή ανάθεσης καθώς και των αρχικοποιήσεων, πειραματιστείτε σχολιάζοντας και αποσχολιάζοντας σχολιασμένες γραμμές.

Συγκεκριμένα, για να κατανοήσετε τη συμπεριφορά της ανάθεσης `scopied = s;`, εξετάστε τις εξής περιπτώσεις:

- Έχοντας επαναορίσει τον τελεστή ανάθεσης στην κλάση `date` και συμπεριλαμβάνοντας την ανάθεση `RegistrationDate = s.RegistrationDate;`

στον ορισμό του

```
Student& Student::operator=(const Student& s)
```

θα γίνει η ανάθεση της μεταβλητής `s.RegistrationDate` στη μεταβλητή `scopied.RegistrationDate` με χρήση του επαναορισμένου τελεστή ανάθεσης της κλάσης `date`. Θα δούμε να εκτυπώνεται και το μήνυμα `I just performed a Date Assignment ....`

- Σχολιάζοντας τον επαναορισμό του τελεστή ανάθεσης στην κλάση `date`, αλλά έχοντας συμπεριλάβει την ανάθεση

```
RegistrationDate = s.RegistrationDate;
```

στον ορισμό του

```
Student& Student::operator=(const Student& s)
```

θα γίνει η ανάθεση της μεταβλητής `s.RegistrationDate` στη μεταβλητή `scopied.RegistrationDate` από την αυτόματη διαδικασία της C++ — με ικανοποιητικό αποτέλεσμα από πλευράς τελικών τιμών.

- Αν παραλείψετε την ανάθεση

```
RegistrationDate = s.RegistrationDate;
```

στον ορισμό του

```
Student& Student::operator=(const Student& s)
```

σε καμία από τις παραπάνω περιπτώσεις δεν θα γίνει η ανάθεση της μεταβλητής `s.RegistrationDate` στη μεταβλητή `scopied.RegistrationDate` οπότε, ενόσω οι τιμές όλων των άλλων μελών-δεδομένων της μεταβλητής `scopied` θα αλλάξουν, η τιμή του μέλους-δεδομένου `scopied.RegistrationDate` θα παραμείνει η προηγούμενη.

Με βάση τους παραπάνω ορισμούς των κλάσεων, έχετε περισσότερες ευκαιρίες να εξετάσετε την αρχικοποίηση μέσω αντιγραφής. Συγκεκριμένα, στη συνάρτηση αρχικοποίησης

```
Student::Student(const Student& s)
```

αποσχολιάζοντας, μία τη φορά, τις γραμμές

```
// : RegistrationDate(s.RegistrationDate)
// : RegistrationDate(5,10)
// : RegistrationDate(5,10,2007)
```

θα παρατηρήσετε ότι με τον ορισμό `Student scopied(s1);`, η μεταβλητή `scopied.RegistrationDate` αρχικοποιείται ως εξής:

- Αποσχολιάζοντας το

`RegistrationDate(s.RegistrationDate)`

η μεταβλητή `scopied.RegistrationDate` αρχικοποιείται με την τιμή της μεταβλητής `s1.RegistrationDate`.

- Αποσχολιάζοντας το `RegistrationDate(5,10)`, η μεταβλητή `scopied.RegistrationDate` αρχικοποιείται ως εξής:
 - `scopied.RegistrationDate.day` αρχικοποιείται με 5,
 - `scopied.RegistrationDate.month` αρχικοποιείται με 10 και
 - `scopied.RegistrationDate.year` αρχικοποιείται με 0.
- Αποσχολιάζοντας το `RegistrationDate(5,10,2007)`, η αρχικοποίηση της μεταβλητής `scopied.RegistrationDate` γίνεται ως εξής:
 - `scopied.RegistrationDate.day` αρχικοποιείται με 5,
 - `scopied.RegistrationDate.month` αρχικοποιείται με 10 και
 - `scopied.RegistrationDate.year` αρχικοποιείται με 2007.

Με τα παραπάνω ολοκληρώσαμε τη συζήτηση όσον αφορά κάποια βασικά σημεία στη σύνθεση κλάσεων και κάποιες λεπτομέρειες που πρέπει να προσέξουμε σε σχέση με αυτά.

6.4 Με Αφορμή τη Σύνθεση Κλάσεων: Εναλλακτικά στους Τύπους των Μελών-δεδομένων

Στα προηγούμενα, μιλήσαμε για την περίπτωση σύνθεσης κλάσεων στη C++ όπου τα αντικείμενα των σύνθετων κλάσεων έχουν ως μέλη-δεδομένα αντικείμενα άλλων κλάσεων. Είπαμε, όμως, ότι αν έχουμε ορίσει μια κλάση, στην πράξη έχουμε επεκτείνει το σύνολο των διαθέσιμων τύπων. Επομένως, ένα μέλος-δεδομένο μπορεί να οριστεί ως δείκτης τύπου της κλάσης. Τότε το μέλος-δεδομένο δεν είναι αντικείμενο, αλλά απλά μια μεταβλητή που έχει μια επιτρεπτή τιμή τύπου δείκτη: διεύθυνση ή τη μηδενική τιμή δείκτη (ανάλογα με την έκδοση της C++, `NULL` ή `nullptr`). Επίσης, ένα μέλος-δεδομένο μπορεί να οριστεί ως αναφορά τύπου κλάσης. Τότε, το μέλος αυτό δεν είναι ένα νέο αντικείμενο που συμμετέχει στο “κτίσιμο” των αντικειμένων της νέας κλάσης και αρχικοποιείται κατά την αρχικοποίησή τους. Απλά, αναφέρεται/“πατάει” σε ένα αντικείμενο του τύπου της αναφοράς, το οποίο οφείλει να υπάρχει ήδη. Το αντικείμενο αυτό συνήθως δίνεται ως παράμετρος στις συναρτήσεις αρχικοποίησης της νέας κλάσης.

Από πλευράς ορατότητας, επαναλαμβάνουμε ότι αν ένα μέλος-δεδομένο μιας νέας κλάσης είναι τύπου κλάσης ή τύπου δείκτη σε κλάση ή τύπου αναφοράς σε κλάση, στην εμβέλεια της νέας κλάσης μπορούμε να χρησιμοποιήσουμε μόνο τα δημόσια ονόματα της κλάσης του μέλους.

Στη συνέχεια θα επιμείνουμε στο θέμα της αρχικοποίησης μελών-δεδομένων τύπου δείκτη, μια που πλημμελείς αρχικοποιήσεις τέτοιων μελών είναι συνήθεις αιτίες σφαλμάτων. Θα ολοκληρώσουμε την παράγραφο, κάνοντας μια συζήτηση, μέσω παραδείγματος, για την επιλογή αναπαράστασης ως προς το είδος του τύπου των μελών-δεδομένων.

6.4.1 Μέλη-δεδομένα Τύπου Δείκτη

Αν ορίζοντας μια νέα κλάση, ο τύπος ενός μέλους-δεδομένου είναι δείκτης σε κλάση, τότε το μέλος δεν είναι αντικείμενο. Παρότι ισχύουν οι περιορισμοί ορατότητας που συζητήσαμε αν το μέλος είναι αντικείμενο, όσον αφορά την αναπαράσταση του αντικειμένου της νέας κλάσης και την αρχικοποίηση του μέλους του, οι δύο περιπτώσεις μελών διαφέρουν ριζικά, μια που πρόκειται για μεταβλητές διαφορετικού είδους.

Ας δούμε τα παραπάνω με ένα παράδειγμα. Θα χρησιμοποιήσουμε τους τύπους `Date` και `Time` της σελίδας 173. Για λόγους σύγκρισης με το προηγούμενο παράδειγμα, παρότι καθόλα ακατάλληλη ως αναπαράσταση της έννοιας της “Λεπτομερούς Ημερομηνίας”, θα ορίσουμε την κλάση `DetailedDate` να έχει μέλη-δεδομένα τύπου δείκτη σε αντικείμενα των κλάσεων `Date` και `Time` αντί για τα ίδια τα αντικείμενα. Δηλαδή:

```
class DetailedDate {
// Not a good representation, anyway, as data members do not exist
// outside the object
    Date* date;
    Time* time;

public:
    DetailedDate()
    {   cout << "DetailedDate was just created with value:"
        << '\n' << endl;
//      CAREFUL: no space has been allocated
//              or initialized for Date and Time

//      this->date->print();    // error
//      this->time->print();    // error
    }

    ~DetailedDate()
    {   cout << '\n' << "A DetailedDate is to be destroyed with value: "
        << endl;
//      date->print();
//      time->print();
    }
};
```

Με δεδομένο τον παραπάνω ορισμό, η μεταβλητή `dd1` που προκύπτει από τον ορισμό:

```
DetailedDate dd1;
```

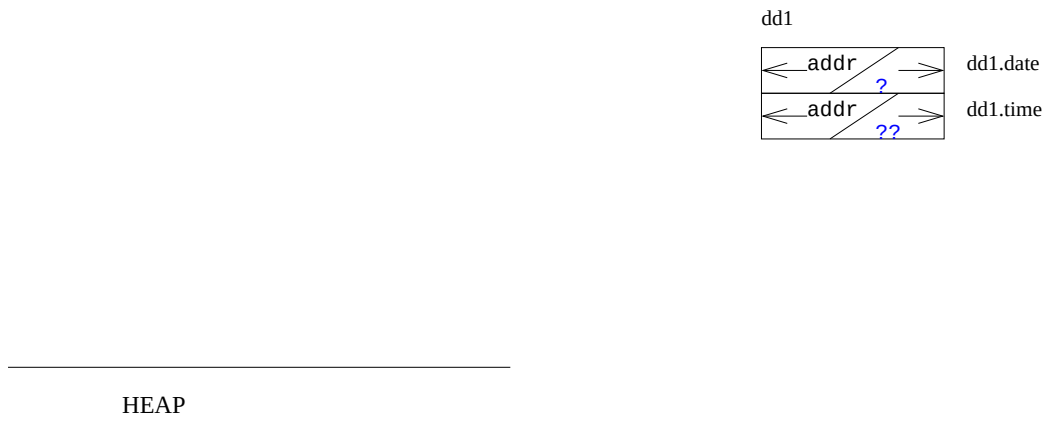
“κτίζεται” από δύο “τουβλάκια” τύπου δείκτη, δηλαδή μεγέθους τέτοιου ώστε να “χωράνε” μια διεύθυνση. Η αφαιρετική εικόνα της μεταβλητής όπως προκύπτει μετά τον ορισμό αυτό φαίνεται στο Σχήμα 6.4.

Όπως αναμενόταν και βλέπουμε, δεν έχουν δημιουργηθεί αντικείμενα των κλάσεων `Date` και `Time`, αφού αυτό δεν έχει ζητηθεί κατά την αρχικοποίηση των αντικειμένων της νέας κλάσης ούτε μέσω λίστας αρχικοποίησης, ούτε στο σώμα της συνάρτησης αρχικοποίησης της κλάσης αυτής.

Εξαιτίας της μη δημιουργίας αντικειμένων των κλάσεων `Date` και `Time`, θα προκύψει σφάλμα αν αποσχολιάσουμε τις σχολιασμένες γραμμές:

```
//      this->date->print();    // error
//      this->time->print();    // error
```

Στο σημείο αυτό, να κάνουμε μια παρένθεση για μια παρατήρηση που αφορά τις δηλώσεις των κλάσεων `Date` και `Time`, προκειμένου να χρησιμοποιηθούν από την κλάση `DetailedDate`. Αν δεν χρησιμοποιούσαμε ονόματα από τις εμβέλεις των κλάσεων αυτών στον ορισμό της κλάσης `DetailedDate`, δεδομένου ότι τα μέλη-δεδομένα της κλάσης είναι τύπου δείκτη και όχι αντικείμενα των κλάσεων `Date` και `Time`, δεν θα ήταν απαραίτητο να έχει προηγηθεί ο πλήρης ορισμός των κλάσεων αυτών. Θα ήταν αρκετό να είχαν προηγηθεί *προ-δηλώσεις* (*forward declarations*) όπου απλά δηλώνουμε τα ονόματά τους και ότι πρόκειται περί κλάσεων. Για τις κλάσεις `Date` και `Time`, αυτές θα ήταν:



Σχήμα 6.4: Η αναπαράσταση της μεταβλητής dd1 τύπου DetailedDate

```
class Date;
class Time;
```

Επανερχόμενοι στην συζήτησή μας τώρα, η αρχικοποίηση των μελών της κλάσης DetailedDate πρέπει να γίνει προσεκτικά ώστε να δείχνουν σε αντικείμενα των κλάσεων Date και Time όταν δημιουργείται ένα αντικείμενο της κλάσης. Άρα, στη συνάρτηση αρχικοποίησης της κλάσης DetailedDate, πρέπει να δημιουργήσουμε τα αντικείμενα αυτά και να διασφαλίσουμε ότι τα μέλη-δεδομένα date και time, αντίστοιχα, θα “δείξουν” σε αυτά.

Μια σωστή αρχικοποίηση μπορεί να γίνει, για παράδειγμα, με τους εξής τρόπους:

- μέσω δυναμικής δημιουργίας των αντικειμένων στο σώμα της συνάρτησης αρχικοποίησης και ταυτόχρονη ανάθεση των διευθύνσεών τους στα μέλη-δεδομένα της μεταβλητής που αρχικοποιείται ή
- μέσω δυναμικής δημιουργίας τους κατά την αρχικοποίηση μέσω λίστας αρχικοποίησης

Ο ορισμός της κλάσης με τη διορθωμένη συνάρτηση αρχικοποίησης (κλάση DetailedPDate) είναι:

```
class DetailedPDate {
// Not a good representation, anyway
    Date* date;
    Time* time;

public:
    DetailedPDate() // : date(new Date), time(new Time)
                  // Alternative to
                  // assignment in the body
// Replace the above with:
// DetailedPDate() : time(new Time), date(new Date)
    { cout << "DetailedPDate was just created with value:"
      << '\n' << endl;
      date = new Date;
      time = new Time;
// Replace the above with:
//     time = new Time;
```

```

//    date = new Date;
    date->print();        // actually: this->date->print()
    time->print();
}

DetailedPDate(int d, int mo, int y, int h, int mi)
//    : date(new Date(d,mo,y)), time(new Time(h,mi)) // Alternative
//                                                    to assignment in the body
{    cout << "DetailedPDate was just created with value:"
        << '\n' << endl;

    date = new Date(d,mo,y);
    time = new Time(h,mi);
    date->print();
    time->print();
}

~DetailedPDate()
{    cout << '\n' << "A DetailedPDate is to be destroyed with value: "
        << endl;

    date->print();
    time->print();

    delete date;
    delete time;
}
};

```

Όπως έχουμε συζητήσει, στην περίπτωση που η αρχικοποίηση γίνεται μέσω λίστας αρχικοποίησης, η σειρά με την οποία θα δημιουργηθούν τα αντικείμενα των κλάσεων Date και Time είναι η σειρά με την οποία έχουμε γράψει τις δηλώσεις των μελών-δεδομένων στον ορισμό της κλάσης, γιατί με τη σειρά αυτή γίνεται η δημιουργία και άρα αρχικοποίηση των μελών-δεδομένων. Στο παράδειγμά μας, όπου έχουμε:

```

Date* date;
Time* time;

```

πρώτα θα δημιουργηθεί ένα αντικείμενο της κλάσης Date και μετά θα δημιουργηθεί ένα αντικείμενο της κλάσης Time.

Αντίθετα, στην περίπτωση, που έχουμε ρητή δημιουργία και ανάθεση στο σώμα των συναρτήσεων αρχικοποίησης, η σειρά εξαρτάται από τη σειρά που έχουμε γράψει τις εντολές στο σώμα της συνάρτησης, μια που πρόκειται για εντολές κατά την εκτέλεση του σώματός της. Αν, δηλαδή, στο παράδειγμά μας, στο σώμα μιας συνάρτησης αρχικοποίησης που εκτελείται έχουμε τον κώδικα:

```

date = new Date;
time = new Time;

```

πρώτα θα δημιουργηθεί ένα αντικείμενο της κλάσης Date και μετά θα δημιουργηθεί ένα αντικείμενο της κλάσης Time. Αν, όμως, σχολιάσουμε αυτές τις γραμμές και αποσχολιάσουμε τις γραμμές:

```

// time = new Time;
// date = new Date;

```

είναι προφανές ότι η σειρά δημιουργίας των αντικειμένων των κλάσεων Date και Time θα είναι ανάποδα.

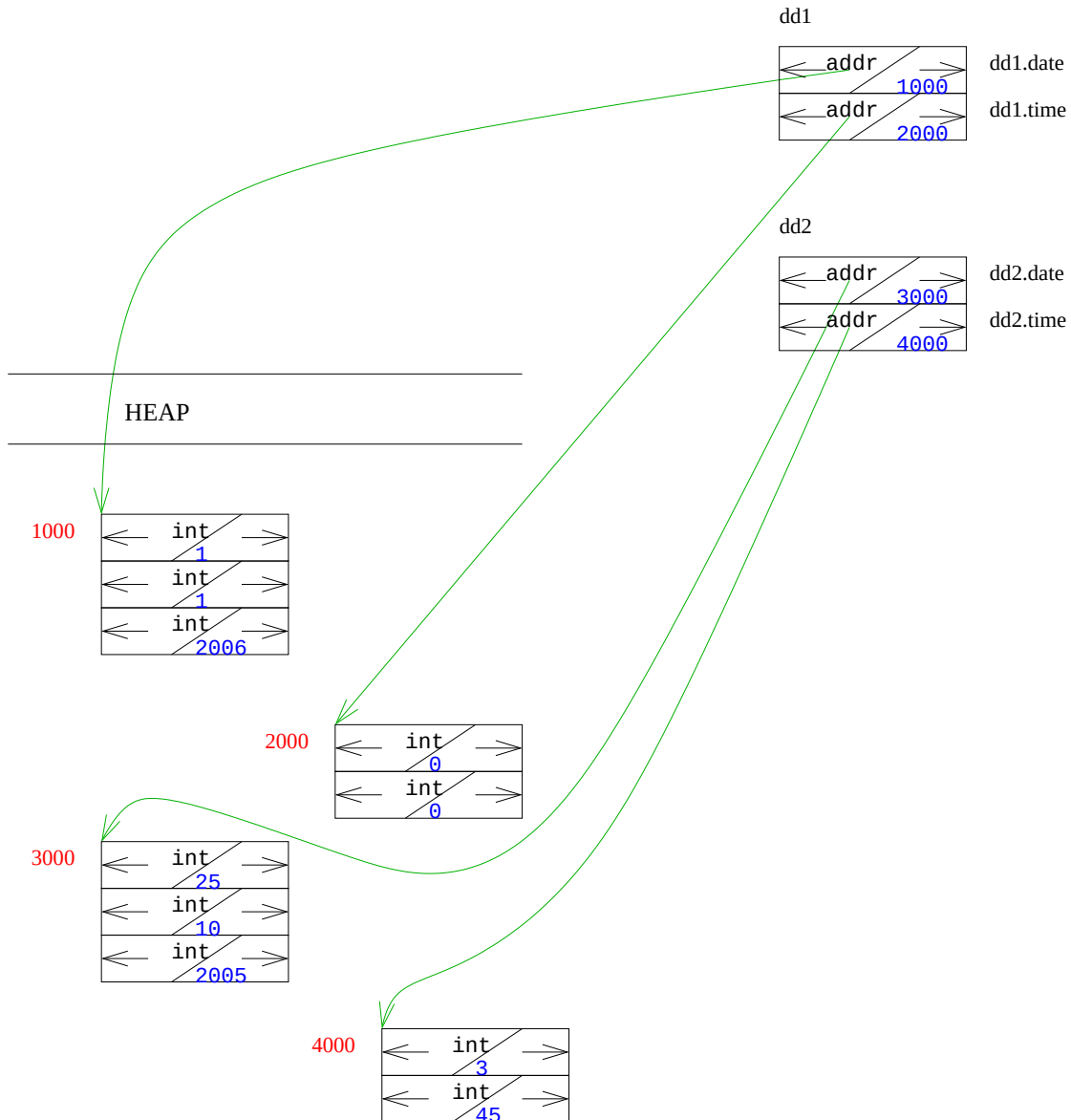
Θεωρήστε τους παρακάτω ορισμούς μεταβλητών της κλάσης DetailedPDate:

```
DetailedPDate dd1;
```

```
DetailedPDate dd2(25,10,2005,3,45);
```

Πειραματιστείτε σχολιάζοντας και αποσχολιάζοντας τα παραπάνω εναλλακτικά ώστε να επιβεβαιώσετε την παραπάνω συμπεριφορά.

Στο Σχήμα 6.5, φαίνεται η αφαιρετική εικόνα στη μνήμη που προκύπτει μετά τους ορισμούς αυτούς. Τα αντικείμενα των κλάσεων Date και Time έχουν δημιουργηθεί δυναμικά στον σωρό και τα μέλη-δεδομένα των μεταβλητών dd1 και dd2 απλά έχουν ως τιμή τις διευθύνσεις τους.



Σχήμα 6.5: Η αναπαράσταση των μεταβλητών `dd1` και `dd2` τύπου `DetailedPDate`

6.4.2 Επιλογή της Αναπαράστασης των Μελών-δεδομένων

Έχοντας ολοκληρώσει την ανάλυση των βασικών σημείων με τα οποία πρέπει να ασχοληθεί ο προγραμματιστής κατά τον ορισμό κλάσεων και την παρουσίαση της σύνθεσης κλάσεων, αξίζει να γίνει μια συζήτηση που

αφορά τις αποφάσεις αναπαράστασης των αντικειμένων μιας κλάσης. Συγκεκριμένα, θα εστιάσουμε στη διερεύνηση του είδους του τύπου των μελών-δεδομένων μιας νέας κλάσης: Θα είναι μια μεταβλητή ενός τύπου (ορισμένου από τον χρήστη ή δεδομένου από τη C++); Θα είναι τύπου δείκτη; Θα είναι τύπου αναφοράς;

Θα εξετάσουμε τα παραπάνω, πάλι, μέσω παραδείγματος υλοποίησης μιας κλάσης. Ας πάμε πίσω στη δεκαετία του '60 και ας θεωρήσουμε την έννοια της τυπικής “Οικογένειας” της εποχής εκείνης.

Μια “Οικογένεια” αυτής της μορφής θεωρούμε ότι συμπεριλαμβάνει δύο “Πρόσωπα”, τους συζύγους, και μια “Ημερομηνία” που αντιστοιχεί στην ημερομηνία γάμου τους. Τα στιγμιότυπα του “Προσώπου” που αντιστοιχούν στους συζύγους προϋπάρχουν του στιγμιότυπου της “Οικογένειας” και υφίστανται ανεξάρτητα από αυτό. Αντίθετα, η ημερομηνία γάμου αποτελεί αποκλειστικά συστατικό της “Οικογένειας”, άρα ένα στιγμιότυπο μιας “Οικογένειας” έχει ως δομικό στοιχείο του ένα στιγμιότυπο της έννοιας της “Ημερομηνίας”.

Μια οικογένεια της εποχής εκείνης είχε το πολύ ένα αυτοκίνητο. Επίσης, μπορεί να είχε κανένα, ένα ή παραπάνω παιδιά τα οποία θα δημιουργηθούν στο πλαίσιο της “Οικογένειας” και είναι στιγμιότυπα “Προσώπων”. Τέλος, ένα αυτοκίνητο είναι στιγμιότυπο της έννοιας του “Αυτοκινήτου”.

Θα προχωρήσουμε υλοποιώντας την έννοια της “Οικογένειας” ως κλάση. Πρώτα θα διανθίσουμε σταδιακά τις λεπτομέρειες των επιμέρους εννοιών της “Ημερομηνίας”, του “Προσώπου” και του “Αυτοκινήτου” και θα τις υλοποιήσουμε.

Η έννοια της “Ημερομηνίας”: Ας ξεκινήσουμε από την έννοια της “Ημερομηνίας”. Ακολουθώντας μια προσέγγιση αντίστοιχη με αυτήν που ακολουθούσαμε συνήθως στα προηγούμενα, η υλοποίησή της ως κλάση, έστω Date, μπορεί να έχει την εξής μορφή:

```
class Date {
    int day, month, year;
public:
    Date(int d, int m, int y) : day(d), month(m), year(y) {}
    int get_year() const { return year; }
    int get_month() const { return month; }
    int get_day() const { return day; }
};
```

Η έννοια του “Προσώπου”: Ας προχωρήσουμε με την έννοια του “Προσώπου”. Έστω ότι ένα “Πρόσωπο” έχει ονοματεπώνυμο, ημερομηνία γέννησης και φύλο και η συμπεριφορά περιορίζεται σε ανάκτηση και ανάθεση των πληροφοριών αυτών. Η ανάθεση της ημερομηνίας γέννησης και του φύλου γίνεται με τη δημιουργία των στιγμιότυπων των προσώπων. Έστω ότι η έννοια του “Προσώπου” υλοποιείται από μια κλάση Person.

Θεωρώντας την έννοια του “Φύλου” σαν διακριτές τιμές, μπορούμε να ορίσουμε έναν τύπο Sex ως απαρίθμηση. Για παράδειγμα, για τη δεκαετία του '60 ο ορισμός μπορεί να ήταν:

```
enum Sex {M, F};
```

Τα δημόσια ονόματα της κλάσης Person θα αντιστοιχούν στη λειτουργικότητά της, που έστω ότι υλοποιείται από τις συναρτήσεις-μέλη:

```
void set_name(const char* nam);
char* get_name() const;
Date get_date_of_birth() const;
Sex get_sex() const;
```

Χρησιμοποιώντας τους τύπους που προαναφέραμε, τα μέλη-δεδομένα της κλάσης Person, μπορεί να αναπαρασταθούν ως:

```
char* name;
```

```
Date date_of_birth;
Sex sex;
```

Επίσης, με βάση την παρατήρηση περί αρχικοποίησης κατά τη δημιουργία, μια συνάρτηση αρχικοποίησης μπορεί να είναι η:

```
Person(int d, int m, int y, Sex sx) :
    date_of_birth(d,m,y), sex(sx), name(NULL) {}
```

Δεδομένης της υλοποίησης του ονοματεπώνυμου, για τους λόγους που συζητήσαμε διεξοδικά κατά την υλοποίηση της έννοιας του “Φοιτητή”, καταλαβαίνουμε ότι θα χρειαστεί να ορίσουμε και συνάρτηση αρχικοποίησης μέσω αντιγραφής:

```
Person(const Person& p)
```

και συνάρτηση καταστροφής:

```
~Person()
```

καθώς και να επαναορίσουμε τον τελεστή ανάθεσης:

```
Person& operator=(const Person& p)
```

Με βάση τις παρατηρήσεις αυτές, ο ολοκληρωμένος ορισμός της κλάσης Person γίνεται:

```
class Person {
    char* name;
    Date date_of_birth;
    Sex sex;
public:
    Person(int d, int m, int y, Sex sx)
        : date_of_birth(d,m,y), sex(sx), name(NULL) {}

    Person(const Person& p)
        : date_of_birth(p.date_of_birth), sex(p.sex)
    {
        if (p.name != NULL) {
            name = new char[strlen(p.name)+1];
            strcpy(name,p.name);
        }
        else
            name = NULL;
    }

    ~Person()
    {
        if (name!=NULL) {
            cout << "Deleting " << name << endl;
            delete[] name;
        }
        else
            cout << "Deleting Anonymous Person " << endl;
    }
};
```

```

    }

    Person& operator=(const Person& p)
    {
        if (this != &p) {
            if (p.name != NULL) {
                delete[] name;
                name = new char[strlen(p.name)+1];
                strcpy(name, p.name);
            }
            else
                name = NULL;
            date_of_birth = p.date_of_birth;
            sex = p.sex;
        }
        return *this;
    }

    void set_name(const char* nam)
    {
        if (name != NULL)
            delete[] name;
        name = new char[strlen(nam)+1];
        strcpy(name, nam);
    }

    char* get_name() const
    {
        return name;
    }

    Date get_date_of_birth() const
    {
        return date_of_birth;
    }

    Sex get_sex() const
    {
        return sex;
    }
};

```

Η έννοια του “Αυτοκινήτου”: Συνεχίζοντας με την έννοια του “Αυτοκινήτου”, ας υποθέσουμε ότι αυτή δεν είναι παρά μια δόμηση των επιμέρους πληροφοριών ενός στιγμιοτύπου, που είναι ο αριθμός κυκλοφορίας του, το μοντέλο του και ο κυβισμός της μηχανής του. Άρα, ας υλοποιήσουμε την έννοια του “Αυτοκινήτου” χρησιμοποιώντας struct με τον παρακάτω απλό ορισμό:

```

struct Car {
    int licence_no;

```

```

char* model;
int cc;
Car(int no, const char* mdl, int c) : licence_no(no), cc(c)
{
    model = new char[strlen(mdl)+1];
    strcpy(model, mdl);
}
~Car()
{
    cout << "Deleting car " << licence_no << endl;
    delete model;
} // Copy Constructor and Assignment Operator also needed!
};

```

Σημείωση: Προσέξτε ότι, για τους λόγους που έχουμε συζητήσει, εξαιτίας του τύπου του μέλους-δεδομένου `model` θα πρέπει να οριστεί και συνάρτηση αρχικοποίησης μέσω αντιγραφής και να επαναοριστεί ο τελεστής ανάθεσης αλλά για λόγους οικονομίας το παραλείπουμε.

Η έννοια της “Οικογένειας”:

Ας προχωρήσουμε τώρα με την υλοποίηση της έννοιας της “Οικογένειας”. Ας υποθέσουμε ότι αυτή υλοποιείται από μια κλάση `Family`. Να υπενθυμίσουμε τις υποθέσεις που κάναμε για την έννοια της “Οικογένειας”. Είπαμε ότι ένα στιγμιότυπο “Οικογένειας” συμπεριλαμβάνει δύο συζύγους, ο καθένας από τους οποίους είναι στιγμιότυπο της έννοιας “Πρόσωπο”. Τα στιγμιότυπα των συζύγων προϋπάρχουν του στιγμιότυπου της “Οικογένειας” και υφίστανται ανεξάρτητα από αυτό. Με βάση την παρατήρηση αυτή, ένα στιγμιότυπο της “Οικογένειας” πρέπει να δομείται από αναφορές στα ήδη υπάρχοντα αυτά στιγμιότυπα. Αντίθετα, η ημερομηνία γάμου αποτελεί αποκλειστικά συστατικό της “Οικογένειας”, άρα ένα στιγμιότυπο μιας “Οικογένειας” έχει ως δομικό στοιχείο του ένα στιγμιότυπο της έννοιας της “Ημερομηνίας”. Με βάση τα παραπάνω, η υλοποίηση της αναπαράστασης των μελών-δεδομένων της κλάσης `Family` που αντιστοιχούν στους συζύγους και την ημερομηνία γάμου μπορεί να είναι:

```

Person& husband;
Person& wife;
Date date_of_wedding;

```

Είπαμε ακόμα ότι μια οικογένεια της εποχής εκείνης είχε το πολύ ένα αυτοκίνητο. Άρα το μέλος-δεδομένο της κλάσης `Family` που αντιστοιχεί στο αυτοκίνητο είτε πρέπει να συνδέεται με ένα (υπάρχον) αυτοκίνητο είτε να έχει μηδενική τιμή. Οπότε, μια υλοποίηση του μέλους αυτού μπορεί να είναι τύπου δείκτη στο στιγμιότυπο του “Αυτοκινήτου” ώστε να επιδέχεται μηδενικής τιμής:

```

Car* owned_car;

```

Τέλος, είπαμε ότι μια “Οικογένεια” είχε κανένα, ένα ή παραπάνω παιδιά τα οποία θα δημιουργηθούν στα πλαίσια της “Οικογένειας” και είναι στιγμιότυπα “Προσώπων”. Οπότε η σύνδεση με μια λίστα “Προσώπων” στο μέλος-δεδομένο που αναπαριστάνει τα παιδιά και η δυνατότητα το μέλος να έχει μηδενική τιμή ικανοποιεί τα παραπάνω. Αν υποθέσουμε ότι οι κόμβοι της λίστας των παιδιών υλοποιούνται από μια δόμηση:

```

struct ChildrenN {
    Person* person;
    ChildrenN* next;
    ChildrenN(Person* p, ChildrenN* n) : person(p), next(n) {};

```



```

~ChildrenN()
{
    delete person; //assuming dynamic creation
    if (next != NULL)
        delete next;
} // Copy Constructor and Assignment Operator also needed!
};

```

Τότε, το μέλος-δεδομένο που αντιστοιχεί στα παιδιά της κλάσης Family που υλοποιεί την έννοια της “Οικογένειας” μπορεί να οριστεί ως:

```
ChildrenN* children;
```

Όταν δημιουργείται μια οικογένεια, χρειάζεται τους συζύγους και τα στοιχεία της ημερομηνίας γάμου. Επίσης, ας θεωρήσουμε ότι δεν έχει αυτοκίνητο ούτε παιδιά. Οπότε μια συνάρτηση αρχικοποίησης μπορεί να είναι:

```

Family(int d, int m, int y, Person& h, Person& w)
: date_of_wedding(d,m,y), husband(h), wife(w),
  children(NULL), owned_car(NULL) {};

```

Επίσης, πρέπει να ορίσουμε συνάρτηση καταστροφής, αν μη τι άλλο για τους κόμβους της λίστας των παιδιών

```
~Family();
```

Θα χρειαστεί να ορίσουμε συνάρτηση αρχικοποίησης μέσω αντιγραφής και να επαναορίσουμε τον τελεστή ανάθεσης, ανάλογα με το πώς επιθυμούμε να είναι η συμπεριφορά στις λειτουργίες αυτές.¹ Δεν θα ασχοληθούμε με αυτά, όμως, εδώ, για λόγους οικονομίας έκτασης.

Ας δούμε κάποια λειτουργικότητα που μπορούμε να αποδώσουμε στην “Οικογένεια”. Έστω ότι έχουμε τις παρακάτω συναρτήσεις-μέλη στο δημόσιο μέρος της κλάσης Family:

- Person& birth(int d, int m, int y, Sex sx); προκειμένου να δημιουργήσουμε, με ημερομηνία γέννησης d/m/y και φύλο sx, ένα παιδί της “Οικογένειας” το οποίο και επιστρέφουμε,
- void buy_car(Car& cr); προκειμένου να αποκτήσει η “Οικογένεια” το αυτοκίνητο cr,
- bool has_children() const; ελέγχει αν η “Οικογένεια” έχει παιδιά,
- bool has_car() const; ελέγχει αν η “Οικογένεια” έχει αυτοκίνητο,
- int no_of_children() const; υπολογίζει και επιστρέφει το πλήθος των παιδιών της “Οικογένειας”,
- int diff() const; υπολογίζει και επιστρέφει το διάστημα που μεσολάβησε από το έτος γέννησης του πρώτου παιδιού έως το έτος γέννησης του τελευταίου.

Θεωρώντας τα παραπάνω, ένας ολοκληρωμένος ορισμός της κλάσης Family είναι ο εξής:

¹Από την έκδοση της C++11 και έπειτα, θα μπορούσαμε να απαγορεύσουμε να γίνονται αυτές οι διαδικασίες.

```

class Family {
    Person& husband;
    Person& wife;
    Date date_of_wedding;
    ChildrenN* children;
    Car* owned_car;

    Person* first_child() const;
    Person* last_child() const;

public:
    Family(int d, int m, int y, Person& h, Person& w)
        : date_of_wedding(d,m,y), husband(h), wife(w),
          children(NULL), owned_car(NULL) {};

    ~Family()
    {
        if (has_children())
            delete children;
    } // dynamic creation assumed
    // Copy Constructor and Assignment Operator may be also needed!

    Person& birth(int, int, int, Sex);

    void buy_car(Car& cr)
    {
        owned_car = &cr;
    }

    bool has_children() const
    {
        return children != NULL;
    }

    bool has_car() const
    {
        return owned_car != NULL;
    }

    int no_of_children() const;

    int diff() const
    {
        if (has_children())
            return last_child()->get_date_of_birth().get_year() -
                   first_child()->get_date_of_birth().get_year();
    }
};

```

Προσέξτε τη δήλωση των συναρτήσεων-μελών

```
Person* first_child() const;
Person* last_child() const;
```

στο ιδιωτικό μέρος της κλάσης. Οι συναρτήσεις αυτές χρειάζονται για την υλοποίηση των δημόσιων συναρτήσεων-μελών, αλλά δεν αποτελούν μέρος της συμπεριφοράς των στιγμιotypών της κλάσης Family. Επομένως, οι δηλώσεις τους δεν πρέπει να συμπεριληφθούν στο δημόσιο μέρος της κλάσης.

Η υλοποίηση ολοκληρώνεται με την υλοποίηση και των υπόλοιπων συναρτήσεων-μελών.

```
Person& Family::birth(int d, int m, int y, Sex sx)
{
    Person* pChild = new Person(d,m,y,sx);
    ChildrenN* pChildN = new ChildrenN(pChild,children);
    children = pChildN;

    return *pChild;
}

Person* Family::last_child() const
{
    if (children != NULL)
        return children->person;
    else {
        cout << "Careful, no children" << endl;
        return NULL;
    };
}

Person* Family::first_child() const
{
    if (children != NULL) {
        ChildrenN* tmpp = children;
        while(tmpp->next != NULL)
            tmpp = tmpp->next;
        return tmpp->person;
    }
    else {
        cout << "Careful, no children" << endl;
        return NULL;
    };
}

int Family::no_of_children() const
{
    int count = 0;
    if (children != NULL) {
        ChildrenN* tmpp = children;
        do {
            count++;
        } while (tmpp->next != NULL);
    }
}
```

```

        tmpp = tmpp->next;
    } while(tmpp != NULL);
}
return count;
}

```

Χρήση της κλάσης Family:

Με δεδομένους τους προηγούμενους ορισμούς κλάσεων, μια ενδεικτική χρήση της κλάσης Family φαίνεται στην παρακάτω συνάρτηση main.

```

#include <iostream>
#include "family.h"
using namespace std;

int main()
{
    Person h(1,3,1930,M);
    Person w(2,6,1935,F);

    h.set_name("Giorgos Papas");
    w.set_name("Nitsa Labrou");

    cout << "/////////////////////////////////////////" << endl;

    ///// Set up a family
    Family f(1,1,1960,h,w);

    ///// Buy car
    Car acar(1234,"TheCarModel",1200);
    f.buy_car(acar);

    ///// Have children
    Person& child1 = f.birth(1,1,1962,M);
    // Person child1 = f.birth(1,1,1962,M); // Swap with the
                                           // above to check behavior

    Person& child2 = f.birth(1,1,1964,F);
    Person& child3 = f.birth(1,1,1966,F);

    ///// Give names to their children
    child1.set_name("nikos");
    child2.set_name("marika");
    child3.set_name("lilika");

    ///// Check how many children they have
    cout << f.no_of_children() << endl;
    ///// Check the children's age difference
    cout << f.diff() << endl;

    cout << "This is the last output of the main function" << endl;
}

```

```

    return 0;
}

```

Με το παραπάνω παράδειγμα, δόθηκε και η ευκαιρία να δούμε και μία υλοποίηση δομής λίστας.

Μια μικρή παραλλαγή του παραδείγματος: Αν εκτελέσετε, όμως, το παραπάνω παράδειγμα θα δείτε ότι η σειρά καταστροφής των αντικειμένων δεν ανταποκρίνεται στο φυσιολογικό. Ας κάνουμε κάποιες αλλαγές, λοιπόν, ώστε να κάνουμε την “Οικογένειά” μας πιο ρεαλιστική. Στο πλαίσιο αυτό, στη λειτουργικότητα της “Οικογένειας” θα συμπεριλάβουμε και μια διαδικασία “ενηλικίωσης” του μεγαλύτερου παιδιού. Δηλαδή, θα επαυξήσουμε το δημόσιο μέρος της κλάσης Family με μία ακόμα συνάρτηση-μέλος, έστω:

```
Person* child_adult();
```

η οποία υλοποιεί την “ενηλικίωση” του μεγαλύτερου παιδιού το οποίο και επιστρέφει μέσω δείκτη. Ας υποθέσουμε ότι η διαδικασία αυτή δεν είναι τίποτα άλλο από το να μηδενίσουμε τον δείκτη στο παιδί του τελευταίου κόμβου της λίστας των παιδιών. Δηλαδή:

```

Person* Family::child_adult()
{
    if (children != NULL) {
        ChildrenN* tmp = children;
        while(tmp->next != NULL && tmp->next->person != NULL)
            tmp = tmp->next;
        Person* ad = tmp->person;
        tmp->person = NULL;
        if (ad == NULL)
            cout << "Empty Nesters! " << endl;
        return ad;
    }
    else {
        cout << "Careful, no children" << endl;
        return NULL;
    }
};
}

```

Με τον παραπάνω ορισμό, επιστρέφεται κάθε φορά το μεγαλύτερο μη ενήλικο παιδί και η θέση του στη λίστα των παιδιών μηδενίζεται. Δηλαδή, όταν θα έχουν ενηλικιωθεί όλα τα παιδιά, η λίστα των παιδιών στις θέσεις των παιδιών θα έχει μηδενικές τιμές (“άδεια φωλιά”, λοιπόν). Προσέξτε: δεδομένου ότι στους κόμβους της λίστας ενδέχεται να έχετε μηδενικές τιμές στους δείκτες person, πρέπει να αλλάξετε και τη συνάρτηση καταστροφής της ChildrenN.

Αυτό που μένει είναι να αλλάξει κατάλληλα η συνάρτηση main. Προσπαθήσετε εσείς να την αλλάξετε κατάλληλα, ώστε να καθορίσετε εσείς τη σειρά της καταστροφής. Υπόδειξη: κάντε δυναμική δημιουργία των στιγμιotypών των συζύγων της οικογένειας και, για παράδειγμα, εκμεταλλευτείτε τη διαδικασία ενηλικίωσης (child_adult) ώστε να έχετε πρόσβαση στα παιδιά μέσω δεικτών. Στη συνέχεια, κάντε δυναμική καταστροφή των στιγμιotypών με σειρά που ανταποκρίνεται στο φυσιολογικό.

6.5 Σύνοψη Κεφαλαίου

- Στη σύνθεση κλάσεων μια κλάση έχει μέλη άρρηκτα συνδεδεμένα με αντικείμενα άλλων κλάσεων.
- Από πλευράς δικαιωμάτων πρόσβασης, η νέα σύνθετη κλάση δεν έχει παραπάνω δικαιώματα από το

υπόλοιπο πρόγραμμα: έχει πρόσβαση μόνο στα δημόσια ονόματα των κλάσεων που χρησιμοποιεί ως τύπους μελών της.

- Αρχικοποίηση αντικειμένων της σύνθετης κλάσης: Πρώτα αρχικοποιούνται τα αντικείμενα που είναι μέλη της κλάσης (με τη σειρά που δηλώνονται στον ορισμό της κλάσης και όχι με τη σειρά που εμφανίζονται στη λίστα αρχικοποίησης στη συνάρτηση αρχικοποίησης της κλάσης) και έπειτα αρχικοποιείται το αντικείμενο της κλάσης.
- Καταστροφή αντικειμένων της σύνθετης κλάσης: Ακολουθείται η αντίστροφη σειρά από εκείνη της αρχικοποίησης, δηλαδή, πρώτα εκτελείται το σώμα της συνάρτησης καταστροφής της κλάσης και μετά εκείνες των μελών της (με την αντίστροφη σειρά της δήλωσης).
- Με βάση τα χαρακτηριστικά του κόσμου του προβλήματος, ο τύπος ενός μέλους-δεδομένου μπορεί να είναι μια νέα μεταβλητή ενός υπάρχοντος τύπου, ένας δείκτης σε μια μεταβλητή ή αναφορά σε υπάρχουσα. Σε κάθε περίπτωση απαιτείται προσοχή ώστε οι αρχικοποιήσεις των αντικειμένων της νέας κλάσης να γίνουν σωστά.

6.6 Ασκήσεις

- 6.6.1** Να υλοποιήσετε ως κλάση τη δομή δεδομένων λίστας ακεραίων με τη συνήθη λειτουργικότητα, αποκρύπτοντας τις αποφάσεις αναπαράστασης και υλοποίησης.
- 6.6.2** Να υλοποιήσετε ως κλάση τη δομή δεδομένων ουράς ακεραίων με τη συνήθη λειτουργικότητα, αποκρύπτοντας τις αποφάσεις αναπαράστασης και υλοποίησης.
- 6.6.3** Αιτιολογώντας την απάντησή σας, δώστε το αποτέλεσμα της εκτέλεσης του εξής προγράμματος C++:

```
#include <iostream>
using namespace std;

class Block {
    int data;
public:
    Block(int i = 10) : data(i)
    { cout << "Creating a Block " << endl; }

    Block(const Block& blk) : data(blk.data)
    { cout << "Creating a Block by copying with data:" <<
      data << endl; }

    ~Block()
    { cout << "Destroying a Block with " << data << endl; }

    void inc()
    { data++; } };

class A {
    Block& block1;
```

```
    Block block2;
public:
    A(Block& blk) : block1(blk), block2(blk)
    { cout << "Creating an A " << endl; }

    ~A()
    { cout << "Destroying an A " << endl; }

    void inc()
    { block1.inc(); block2.inc(); }

    A& operator=(const A& a)
    { if (this != &a) {
        block1 = a.block1;
        cout << "Not a wise assignment!" << endl;
    }
    return *this; } };

class Fat {
    A a;
    A& ra;
    A* pa;
public:
    Fat(A& da) : a(da), ra(da)
    { a = da; pa = new A(da);
      cout << "Fat just created !" << endl; }

    ~Fat()
    { delete pa;
      cout << "Fat to be destroyed !" << endl; }

    void inc()
    { a.inc(); ra.inc(); pa->inc(); } };

int main()
{
    Block block; A a(block);
    Fat fat(a); fat.inc();
    return 0;
}
```

6.6.4 Δίνεται το παρακάτω πρόγραμμα C++:

```
#include <iostream>
using namespace std;

class A {
    int data;
public:
```

```

A(int i = 100) : data(i)
{ cout << "Just constructed an A with: " << data << endl; }

A(const A& a) : data(a.data)
{ cout << "Just constructed an A by copying with data "
  << data << endl; }

~A()
{ cout << "Destructing an A with data " << data << endl; }

int get_data()
{ return data; }

void change()
{ data += 100; } };

class FatA {
    A data1;
    A data2;
public:
    FatA(A& a) : data1(a), data2(a)
    { cout << "Just constructed a FatA with: "
      << data1.get_data() << " "
      << data2.get_data() << endl; }

    ~FatA()
    { cout << "Destructing a FatA " << endl; }

    void change()
    { data1.change(); data2.change(); } };

class BigFatA {
    FatA& data1;
    FatA& data2;
public:
    BigFatA(FatA& fatA) : data1(fatA), data2(fatA)
    { cout << "Just constructed a BigFatA " << endl; }

    ~BigFatA()
    { cout << "Destructing a BigFatA " << endl; }

    void change()
    { FatA& rdata1 = data1; rdata1.change();
      FatA cdata2 = data2; cdata2.change(); } };

int main()
{
    A a;

```



```

    FatA fatA(a); fatA.change();

    BigFatA bigFatA(fatA);
    bigFatA.change();

    return 0;
}

```

Δώστε το αποτέλεσμα της εκτέλεσής του, αιτιολογώντας την απάντησή σας.

6.6.5 Αιτιολογώντας την απάντησή σας, δώστε το αποτέλεσμα της εκτέλεσης του εξής προγράμματος C++:

```

#include <iostream>
using namespace std;

class Block {
    int data;
public:
    Block(int datav = 0) : data(datav)
    { cout << "Constructing a Block with " << datav << endl; }

    ~Block()
    { cout << "Destructing a Block with: " << data << endl; }

    void double_it()
    { data = data * 2; }

    void print()
    { cout << data << endl; } };

class B {
    Block data1;
    Block* data2;
    Block* data3;
public:
    B() : data2(NULL), data3(NULL)
    { cout << "Constructing a B" << endl; }

    B(int value1, Block* value2) : data1(value1)
    { cout << "Constructing a B with given values " << endl;
      data2 = new Block(value1); data3 = value2; }

    ~B()
    { cout << "Destructing a B " << endl;
      if (data2 == NULL)
          cout << "with NULL data2" << endl;
      else {
          cout << "with data2: " << endl; delete data2; }
      if (data3 == NULL)
          cout << "with NULL data3" << endl;
    }
}

```

```

        else {
            cout << "with data3: " << endl; data3->print(); } }

void double_it()
{ data1.double_it(); data2->double_it(); data3->double_it(); }

void double_data1()
{ data1.double_it(); }

void double_data2()
{ data2->double_it(); }

void double_data3()
{ data3->double_it(); } };

int main()
{
    Block block(20); B b(10,&block);
    block.double_it(); b.double_it(); b.double_data1();
    b.double_data2(); b.double_data3(); block.double_it();
    return 0;
}

```

6.6.6 Δίνεται το παρακάτω πρόγραμμα C++:

```

#include <iostream>
using namespace std;

class A {
    int data;
public:
    A(int i = 100) : data(i)
    { cout << "Constructing an A with: " << data << endl; }

    A(const A& a) : data(a.data)
    { cout << "Constructing an A by copying with data "
      << data << endl; }

    ~A()
    { cout << "Destructing an A with data " << data << endl; }

    int get_data()
    { return data; }

    void change()
    { data += 100; } };

class FatA {
    A& data1;

```

```
A& data2;
public:
    FatA(A& a) : data1(a), data2(a)
    {   cout << "Constructing a FatA with: "
        << data1.get_data() << " "
        << data2.get_data() << endl; }

    ~FatA()
    {   cout << "Destructing a FatA " << endl; }

    void change()
    {   data1.change(); data2.change(); } };

class BigFatA {
    FatA& data1;
    FatA& data2;
public:
    BigFatA(FatA& fatA) : data1(fatA), data2(fatA)
    {   cout << "Constructing a BigFatA " << endl; }

    ~BigFatA()
    {   cout << "Destructing a BigFatA " << endl; }

    void change()
    {   FatA cdata1 = data1; cdata1.change();
        FatA cdata2 = data2; cdata2.change(); }
};

int main()
{
    A a;
    FatA fatA(a); fatA.change();

    BigFatA bigFatA(fatA);
    bigFatA.change();

    return 0;
}
```

Δώστε το αποτέλεσμα της εκτέλεσής του, αιτιολογώντας την απάντησή σας.

6.6.7 Αιτιολογώντας την απάντησή σας, δώστε το αποτέλεσμα της εκτέλεσης του εξής προγράμματος C++:

```
#include <iostream>
using namespace std;

class Block {
    int data;
```

```

public:
    Block(int i = 10) : data(i)
    {   cout << "Creating a Block " << endl; }

    ~Block()
    {   cout << "Destroying a Block with " << data << endl; }

    void inc()
    {   data++; } };

class A {
    Block& block1;
    Block block2;
public:
    A(Block& blk) : block1(blk), block2(blk)
    {   cout << "Creating an A " << endl; }

    A(const A& a) : block1(a.block1), block2(a.block2)
    {   cout << "Creating an A by copying but"
        << " I will also do bad things" << endl;
        block1.inc(); block2.inc(); }

    ~A()
    {   cout << "Destroying an A " << endl; }

    void inc()
    {   block1.inc(); block2.inc(); } };

class Fat {
    A a;
    A& ra;
    A* pa;
public:
    Fat(A& da) : a(da), ra(da)
    {   pa = new A(da);
        cout << "Fat just created !" << endl; }

    ~Fat()
    {   delete pa;
        cout << "Fat to be destroyed !" << endl; }

    void inc()
    {   a.inc(); ra.inc(); pa->inc(); } };

int main()
{
    Block block; A a(block); Fat fat(a);
    fat.inc();
}

```

```

    return 0;
}

```

6.6.8 Αιτιολογώντας την απάντησή σας, δώστε το αποτέλεσμα της εκτέλεσης του εξής προγράμματος C++:

```

#include <iostream>
using namespace std;

class A {
    int data;
public:
    A(int di = 10) : data(di)
    { cout << "Creating an A with " << data << endl; }

    A(const A& a) : data(a.data)
    { cout << "Creating an A by Copying" << endl; }

    ~A()
    { cout << "Deleting an A with " << data << endl; }

    void inc()
    { data++; } };

class B {
    A data1;
    A* data2;
public:
    B(const A& a) : data1(a), data2(NULL)
    { cout << "Creating a B " << endl; }

    B(const B& b) : data1(b.data1), data2(b.data2)
    { cout << "Creating a B by Copying" << endl; }

    ~B()
    { cout << "Deleting a B " << endl; }

    void inc()
    { data1.inc(); if (data2 != NULL) data2->inc(); }

    void set(A* pa)
    { data2 = pa; } };

class C {
    B data1;
    B& data2;
public:
    C(B& b) : data1(b), data2(b)
    { cout << "Creating a C " << endl; }

```

```

C(const C& c) : data1(c.data1), data2(c.data2)
{ cout << "Creating a C by Copying" << endl; }

~C()
{ cout << "Deleting a C " << endl; }

void inc()
{ data1.inc(); data2.inc(); } };

class D {
    C& data1;
    C& data2;
public:
    D(C& c1, C& c2) : data1(c1), data2(c2)
    { cout << "Creating a D " << endl; }

    ~D()
    { cout << "Deleting a D " << endl; }

    void inc()
    { data1.inc(); data2.inc(); } };

int main()
{
    A a; B b1(a); B b2(b1); C c1(b1); C c2(c1); D d(c1,c2);
    a.inc(); b1.inc(); b2.inc(); c1.inc(); c2.inc(); d.inc();
    b1.set(&a);
    a.inc(); b1.inc(); b2.inc(); c1.inc(); c2.inc(); d.inc();
    return 0;
}

```

6.6.9 Αιτιολογώντας την απάντησή σας, δώστε το αποτέλεσμα της εκτέλεσης του εξής προγράμματος C++:

```

#include <iostream>
using namespace std;

class A {
    int data;
public:
    A(int i = 10) : data(i)
    { cout << "An A just created" << endl; }

    A(const A& a) : data(a.data)
    { cout << "An A just created with CP" << endl; }

    ~A()
    { cout << "An A to be destroyed with data" << data << endl; }
}

```

```

    void increase(int i)
    { data *= i; } };

class A1 {
    A data1;
    A& data2;
public:
    A1(A& a) : data1(a), data2(a)
    { cout << "An A1 just created" << endl; }

    ~A1()
    { cout << "An A1 to be destroyed" << endl; }

    void process()
    { data1.increase(2); data2.increase(2); } };

class A2 {
    A data1;
    A* data2;
public:
    A2(const A& a) : data1(a)
    { data2 = new A(a);
      cout << "An A2 just created" << endl; }

    A2(const A2& a2) : data1(a2.data1)
    { data2 = new A(*(a2.data2));
      cout << "An A2 just created with CP" << endl; }

    ~A2()
    { cout << "An A2 to be destroyed" << endl; delete data2; }

    void process()
    { data1.increase(10); data2->increase(10); } };

class B {
    A1& data1;
    A2 data2;
public:
    B(A1& a1, A2& a2) : data1(a1), data2(a2)
    { cout << "A B just created" << endl; }

    ~B()
    { cout << "A B to be destroyed" << endl; }

    void process()
    { data1.process(); data2.process(); } };
    
```

```

int main()
{
    A a; A1 a1(a); A2 a2(a);
    B b1(a1,a2); B b2(b1);
    b1.process(); b2.process(); a1.process(); a2.process();
    return 0;
}

```

6.6.10 Αιτιολογώντας την απάντησή σας, δώστε το αποτέλεσμα της εκτέλεσης του παρακάτω προγράμματος C++. Στη συνέχεια, αποσχολιάστε το σχολιασμένο κομμάτι κώδικα. Δώστε πάλι το αποτέλεσμα της εκτέλεσης, αιτιολογώντας την απάντησή σας.

```

#include <iostream>
using namespace std;

class A {
    int data1;
    int& data2;
public:
    A(int d1, int& d2) : data1(d1), data2(d2)
    { cout << "An A just constructed" << endl; }

    ~A()
    { cout << "An A to be destructed with: "
      << data1 << " and " << data2 << endl; }

    A& operator=(const A& a)
    { cout<< "I am performing an A assignment" << endl;
      data1 = a.data1; data2 = a.data2;
      return *this; }

    void inc()
    { data1++; data2++; }

    void print()
    { cout << data1 << "    " << data2 << endl; } };

class B {
    int data1;
    int* data2;
    A& data3;
public:
    B(int d1, int* pd2, A& a) : data1(d1), data2(pd2), data3(a)
    { cout << "A B just constructed" << endl; }

    ~B()
    { cout << "A B to be destructed with: "
      << data1 << " and " << *data2 << " and " << endl;
      data3.print(); }
}

```



```

B& operator=(const B& b)
{  cout << "I am performing a B assignment" << endl;
   data1 = b.data1; data2 = b.data2; data3 = b.data3;
   return *this; }

void inc()
{  data1++; ++(*data2); data3.inc(); } };

int main()
{
   int i1 = 10; int i2 = 20;
   A a1(i1, i1); A a2(i2, i2);
   B b1(i1, &i2, a1); B b2(i2, &i1, a2); B b3(b1);

   a1.inc(); a2.inc();
   b1.inc(); b2.inc(); b3.inc();
   cout << "i1=" << i1 << "      " << "i2=" << i2 << endl;
/*
   b2 = b3; b3.inc();
   cout << "i1=" << i1 << "      " << "i2=" << i2 << endl;
*/
   return 0;
}

```

6.6.11 Αιτιολογώντας την απάντησή σας, δώστε το αποτέλεσμα της εκτέλεσης του εξής προγράμματος C++:

```

#include <iostream>
using namespace std;

class Block {
   int data;
public:
   Block(int datav = 0) : data(datav)
   {  cout << "Constructing a Block with " << datav << endl; }

   ~Block()
   {  cout << "Destructing a Block with: " << data << endl; }

   void double_it()
   {  data = data * 2; }

   void print()
   {  cout << data << endl; } };

class B {
   Block data1;
   Block* data2;

```

```

    Block* data3;
public:
    B() : data2(NULL), data3(NULL)
    { cout << "Constructing a B" << endl; }

    B(int value1, Block* value2) : data1(value1)
    { cout << "Constructing a B with given values " << endl;
      data2 = new Block(value1); data3 = value2; }

    ~B()
    { cout << "Destructing a B " << endl;
      if (data2 == NULL)
        cout << "with NULL data2" << endl;
      else {
        cout << "with data2: " << endl; delete data2; }
      if (data3 == NULL)
        cout << "with NULL data3" << endl;
      else {
        cout << "with data3: " << endl; data3->print(); } }

    void double_it()
    { data1.double_it(); data2->double_it(); data3->double_it(); }

    void double_data1()
    { data1.double_it(); }

    void double_data2()
    { data2->double_it(); }

    void double_data3()
    { data3->double_it(); } };

int main()
{
    Block block(20); B b(10,&block);
    block.double_it(); b.double_it(); b.double_data1();
    b.double_data2(); b.double_data3(); block.double_it();
    return 0;
}

```

6.6.12 Αιτιολογώντας την απάντησή σας, δώστε το αποτέλεσμα της εκτέλεσης του εξής προγράμματος C++.

```

#include <iostream>
using namespace std;

class A {
    int data;

```

```

public:
    A(int d = 10) : data(d)
    { cout << "An A just constructed" << endl; }

    A(const A& a) : data(a.data)
    { cout << "An A just constructed by CP" << endl; }

    ~A()
    { cout << "An A to be destructed with: " << data << endl; }

    void inc()
    { data++; }

    int get()
    { return data; } };

class B {
    A data1;
    A& data2;
    A* data3;
public:
    B(const A& d1, A& d2, const A& d3) : data1(d1), data2(d2)
    { data3 = new A(d3);
      cout << "A B just constructed" << endl; }

    B(const B& b) : data1(b.data1), data2(b.data2)
    { data3 = new A(*(b.data3));
      cout << "A B just constructed by CP " << endl; }

    ~B()
    { cout << "A B to be destructed with: " << data1.get()
      << " and " << data2.get() << " and "
      << data3->get() << endl;
      delete data3; }

    B& operator=(const B& b)
    { cout << "I am performing a B assignment " << endl;
      if (this != &b) {
          delete data3;
          data1 = b.data1;
          data2 = b.data2;
          data3 = new A(*(b.data3)); }
      return *this; }

    void inc()
    { data1.inc(); data2.inc(); data3->inc(); } };

void fun(B b1, B& b2, B* pb3)
{

```

```

    b1.inc(); b2.inc(); pb3->inc();
    b2 = *pb3; b1 = b2;
    b1.inc(); b2.inc(); pb3->inc(); }

int main()
{
    A a1; A a2(100);
    B b1(a1, a2, a1); B b2(a2, a1, a2);
    fun(b1, b2, &b1);
    return 0;
}

```

6.6.13 Δίνεται το παρακάτω πρόγραμμα C++:

```

#include <iostream>
using namespace std;

class Block {
    int i;
public:
    Block(int ii = 10) : i(ii)
    { cout << "Creating a Block with: " << i << endl; }

    Block(const Block& blk)
    { i = blk.i * 2;
      cout << "The new Block data is: " << i << endl; }

    ~Block()
    { cout << "Destroying a Block with: " << i << endl; }

    int get_data()
    { return i; } };

class A {
    Block block;
    Block* pblock;
public:
    A(const Block& blk) : block(blk)
    { pblock = new Block(block); Block tblock; tblock = *pblock; }

    ~A()
    { cout << "Destroying an A with " << block.get_data() << " and "
      << pblock->get_data() << endl;
      delete pblock; } };

int main()
{

```

```

    Block block; A a(block);
    return 0;
}

```

Αιτιολογώντας την απάντησή σας, δώστε το αποτέλεσμα της εκτέλεσής του, αναφέροντας τα πρωτότυπα των συναρτήσεων που καλούνται.

6.6.14 Αιτιολογώντας την απάντησή σας, δώστε το αποτέλεσμα της εκτέλεσης του εξής προγράμματος C++:

```

#include <iostream>
using namespace std;

class Block {
    int i;
public:
    Block(int ii = 0) : i(ii)
    { cout << "Creating a Block with: " << i << endl; }

    Block(const Block& blk)
    { i = blk.i;
      cout << "Creating a Block by copying with: " << i << endl; }

    ~Block()
    { cout << "Destroying a Block with: " << i << endl; }

    void inc()
    { i++; }

    int get_data()
    { return i; } };

class A {
    Block& data1;
    Block* data2;
    Block data3;
public:
    A(Block& blk) : data1(blk), data3(blk)
    { data2 = &data3;
      cout << "Creating an A with :" << data1.get_data()
        << " and " << data2->get_data() << " and "
        << data3.get_data() << endl; }

    ~A()
    { cout << "Destroying an A with :" << data1.get_data()
      << " and " << data2->get_data() << " and "
      << data3.get_data() << endl; }

    void process()
    { data1.inc(); data2->inc(); data3.inc(); } };

```

```

class FatA {
    A data1;
    A* data2;
public:
    FatA(A& a) : data1(a)
    { data2 = new A(a); cout << "Creating a FatA " << endl; }

    FatA(const FatA& fa) : data1(fa.data1)
    { data2 = new A(*(fa.data2));
      cout << "Creating a FatA by copying" << endl; }

    ~FatA()
    { cout << "Destroying a FatA " << endl; delete data2; }

    void process()
    { data1.process(); data2->process(); } };

void process(FatA fa)
{
    fa.process();
}

int main()
{
    Block block; A a(block); FatA fa(a);
    cout << "-----" << endl;
    fa.process();
    cout << "-----" << endl;
    process(fa);
    cout << "-----" << endl;
    block.inc();
    return 0;
}

```

Εργασίες Για περισσότερη εξάσκηση στην ανάπτυξη κώδικα στην περιοχή της αφαίρεσης στα δεδομένα και σύνθεσης κλάσεων για πιο σύνθετα προβλήματα, μπορείτε να ασχοληθείτε με τις εργασίες των Παραγράφων 11.3 και 11.4.

6.7 Βιβλιογραφία

- [1] Bruce Eckel. *Thinking in C++, Volume I: Introduction to Standard C++*. 2η έκδοση. Prentice Hall, 2000.
- [2] C++ reference. <https://en.cppreference.com/w/>. Ανάκτηση: 2023-5-30.
- [3] cplusplus.com. <http://www.cplusplus.com/>. Ανάκτηση: 2023-5-30.
- [4] Bjarne Stroustrup. *The C++ Programming Language*. 4η έκδοση. Addison-Wesley Professional, 2013.

- [5] Bjarne Stroustrup. *Programming: Principles and Practice Using C++*. 2η έκδοση. Addison-Wesley Professional, 2014.
- [6] Stanley B. Lippman. *Inside the C++ Object Model*. Addison-Wesley Professional, 1996.
- [7] Stanley B. Lippman, Josée Lajoie και Barbara Moo. *C++ Primer*. 5η έκδοση. Addison-Wesley Professional, 2012.

ΚΛΗΡΟΝΟΜΙΚΟΤΗΤΑ

Σύνοψη

Στο κεφάλαιο αυτό, συζητείται η έννοια της κληρονομικότητας μεταξύ κλάσεων. Επίσης, τονίζεται η διαφορά μεταξύ κληρονομικότητας και σύνθεσης κλάσεων, παρότι και οι δύο έννοιες αποτελούν περιπτώσεις επαναχρησιμοποίησης. Στη συνέχεια, το κεφάλαιο εστιάζει στον τρόπο ορισμού κληρονομικότητας στη C++, ξεκινώντας από τη διεύρυνση των προσδιορισμών ορατότητας για τα κληροδοτούμενα μέλη των κλάσεων. Κατόπιν, αναλύονται θέματα που προκύπτουν και πρέπει να ληφθούν υπόψη όταν ο κώδικάς μας περιέχει κλάσεις συσχετισμένες με κληρονομικότητα. Έμφαση δίνεται στη διαδικασία αρχικοποίησης αντικειμένων και στη διαδικασία καταστροφής τους όταν είναι αντικείμενα κλάσεων που συμμετέχουν σε μια ιεραρχία που προκύπτει από τη συσχέτιση αυτή. Στο κεφάλαιο γίνεται ιδιαίτερη αναφορά στην εφαρμογή συναρτήσεων σε αντικείμενα μέσω δεικτών, στον ρόλο των εικονικών συναρτήσεων (virtual functions), στη χρήση αφηρημένων κλάσεων (abstract classes) ως πρότυπα συμπεριφοράς και βασικής υλοποίησης και στην επίτευξη πολυμορφισμού. Τέλος, η κληρονομικότητα και η σύνθεση κλάσεων χρησιμοποιούνται σε ένα ενιαίο πλαίσιο C++ κώδικα, μέσω παραδείγματος, σχολιάζοντας τη συμπεριφορά του.

Προαπαιτούμενη Γνώση

Για την κατανόηση του κεφαλαίου, ο αναγνώστης απαιτείται να έχει γνώσεις διαδικαστικού προγραμματισμού και να έχει μελετήσει τις έννοιες του αντικειμενοστραφούς προγραμματισμού και της C++ από τα προηγούμενα κεφάλαια του βιβλίου.

7.1 Εισαγωγή στην Κληρονομικότητα

Από το πρώτο κεφάλαιο του βιβλίου, κάναμε σαφές πόσο σημαντική είναι η ιδιότητα της κληρονομικότητας για να χαρακτηριστεί μια γλώσσα προγραμματισμού ως γλώσσα OOP. Στον αντικειμενοστραφή προγραμματισμό, δεν αρκεί να έχουμε τη δυνατότητα να ορίζουμε δικούς μας τύπους ως κλάσεις των οποίων στο πρόγραμμά μας να ορίζουμε αντικείμενα, αλλά πρέπει να έχουμε τη δυνατότητα να προσδιορίζουμε συσχέτιση

“γενικότερης-ειδικότερης” μεταξύ κλάσεων. Τότε, με τον μηχανισμό της κληρονομικότητας, ό,τι έχει οριστεί στη γενικότερη κλάση κληροδοτείται στην ειδικότερη και δεν χρειάζεται να επαναλαμβάνεται στον ορισμό της. Στην ειδικότερη κλάση, απλά προσδιορίζεται η “ειδοποιός διαφορά” που τη διακρίνει από τη γενικότερη, είτε από πλευράς χαρακτηριστικών, είτε από πλευράς γενικότερης λειτουργικότητας.

Από πλευράς ανάπτυξης μιας εφαρμογής σε αντικειμενοστραφές περιβάλλον, αρχικά πρέπει να εντοπίσουμε ποιες έννοιες θα αναπαρασταθούν ως κλάσεις, καθορίζοντας τη λειτουργικότητά τους. Κοινή λειτουργικότητα μεταξύ κλάσεων πρέπει να αποδοθεί σε πιο γενικές κλάσεις. Αυτές δεν αναπαριστούν κατ’ ανάγκην έννοιες άμεσα εμφανείς μέσα από την περιγραφή των απαιτήσεων της εφαρμογής. Σε κάθε περίπτωση, ο συσχετισμός γενικότερου-ειδικότερου μεταξύ των κλάσεων εκμεταλλεύεται τον μηχανισμό της κληρονομικότητας.

Προκειμένου να γίνει καλύτερα κατανοητή η έννοια της κληρονομικότητας, θα ξεκινήσουμε μέσω ενός παραδείγματος σε πλαίσιο γενικότερο του προγραμματισμού. Ας θεωρήσουμε τον κόσμο των περιοδικών αυτοκινήτων στις αρχές της δεκαετίας ’00. Φυλλομετρώντας ένα περιοδικό τα χρόνια εκείνα, μπορούσες να δεις περιγραφές διαφόρων μοντέλων αυτοκινήτων που ήταν σε κυκλοφορία. Οι περιγραφές συμπεριλάμβαναν απαριθμήσεις των διάφορων χαρακτηριστικών τους, σχολιασμούς, συγκρίσεις μοντέλων κ.ά. Παρατηρώντας όμως την οργάνωση ενός περιοδικού, έβλεπες ότι ήταν χωρισμένο σε τμήματα και τα μοντέλα ήταν ομαδοποιημένα σε κατηγορίες, μία ανά τμήμα: αυτοκίνητα λιμουζίνες, αυτοκίνητα hatchback, αυτοκίνητα 4x4 κ.ά. Μάλιστα, κάποια από τα περιοδικά αυτά είχαν και διαφορετικό τμήμα για μοτοσυκλέτες. Και πώς “ταιριάζει” να συμπεριλαμβάνονται μοτοσυκλέτες σε ένα περιοδικό αυτοκινήτων; Μα γιατί τόσο τα αυτοκίνητα, όσο και οι μοτοσυκλέτες είναι ειδικές κατηγορίες οχημάτων, οπότε έχουν “κοινά”. Αυτό που βλέπουμε, λοιπόν, άμεσα στο περιοδικό είναι μοντέλα. Παρατηρώντας πιο προσεκτικά, όμως, με βάση κοινά στοιχεία και ιδιότητες, τα μοντέλα ομαδοποιούνται σε κατηγορίες που οργανώνονται ιεραρχικά με πιο γενική κατηγορία εκείνη που αντιστοιχεί στην έννοια του οχήματος.

Με βάση το παράδειγμα του κόσμου των περιοδικών αυτών, ας δούμε μια απλοϊκή εκδοχή του κόσμου των οχημάτων της δεκαετίας του ’00. Πέρα από τα αυτοκίνητα και τις μοτοσυκλέτες της εποχής, θα λάβουμε υπόψη μας και τα ποδήλατα. Σύμφωνα, λοιπόν, με τον κόσμο αυτό, τα “οχήματα” (VEHICLE) έχουν “τροχούς” (wheels), ανά πάσα στιγμή βρίσκονται σε μία “θέση” (position) και έχουν δυνατότητα “μετακίνησης” (move). Διακρίνονται δε, σε:

- “αυτοκίνητα” (CAR)
- “ποδήλατα” (BICYCLE)
- “μοτοσυκλέτες” (MOTORBIKE)

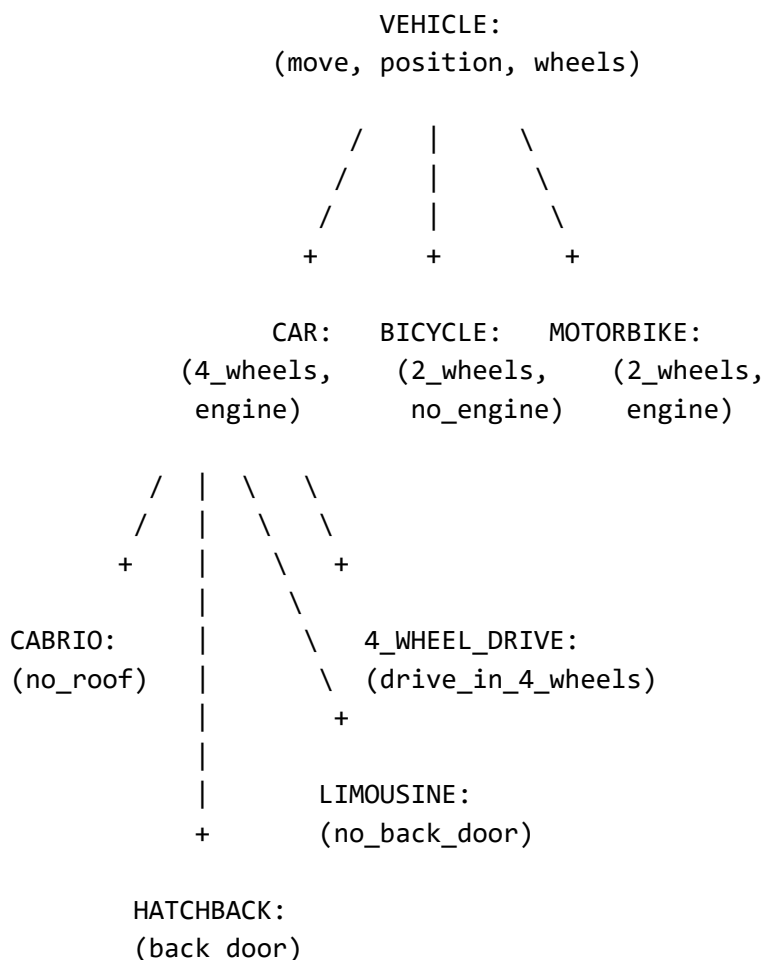
Τα “αυτοκίνητα” είναι “οχήματα” τα οποία έχουν “4 τροχούς” (4_wheels) και “μηχανή” (engine). Τα “ποδήλατα” είναι “οχήματα” τα οποία έχουν “2 τροχούς” (2_wheels) και “δεν έχουν μηχανή” (no_engine). Οι “μοτοσυκλέτες” είναι “οχήματα” τα οποία έχουν “2 τροχούς” (2_wheels) και “μηχανή” (engine).

Τα “αυτοκίνητα” διακρίνονται σε:

- “κάμπριο” (CABRIO)
- “χάτσμπак” (HATCHBACK)
- “λιμουζίνες” (LIMOUSINE)
- “4x4” (4_WHEEL_DRIVE)

Τα “κάμπριο” είναι “αυτοκίνητα” τα οποία “δεν έχουν οροφή” (no_roof). Τα “χάτσμπак” είναι “αυτοκίνητα” τα οποία έχουν “πίσω πόρτα” (back_door). Οι “λιμουζίνες” είναι “αυτοκίνητα” τα οποία “δεν έχουν πίσω πόρτα” (no_back_door). Τα “4x4” είναι “αυτοκίνητα” τα οποία έχουν “κίνηση στους 4 τροχούς” (drive_in_4_wheels).

Με βάση την παραπάνω περιγραφή, μπορούμε να απεικονίσουμε τον κόσμο των οχημάτων σχηματικά με ένα δένδρο, όπου σε ψηλότερους κόμβους απεικονίζουμε πιο γενικές κατηγορίες και σε χαμηλότερους κόμβους εξειδικεύσεις τους. Στον βασικό κόμβο (ρίζα του δένδρου), VEHICLE, συμπληρώσαμε και τη λειτουργικότητά του. Στους χαμηλότερους κόμβους συμπληρώσαμε, για τον καθένα κόμβο, την “ειδοποιό διαφορά” που τον εξειδικεύει σε σχέση με τον γενικότερό του. Έτσι, το δένδρο αυτό μπορεί να είναι το εξής.



Με βάση την οργάνωση αυτού του σχήματος, οφείλουμε να συμπεράνουμε τα εξής. Ένα “αυτοκίνητο”, πέρα από το ότι έχει “4 τροχούς”, αφού είναι “όχημα”, ανά πάσα στιγμή βρίσκεται σε μία “θέση” και έχει δυνατότητα “μετακίνησης”. Ένα “ποδήλατο”, πέρα από το ότι έχει “2 τροχούς” και “δεν έχει μηχανή”, αφού είναι “όχημα”, ανά πάσα στιγμή βρίσκεται σε μία “θέση” και έχει δυνατότητα “μετακίνησης”. Μια “μοτοσυκλέτα”, πέρα από το ότι έχει “2 τροχούς” και “μηχανή”, αφού είναι “όχημα”, ανά πάσα στιγμή βρίσκεται σε μία “θέση” και έχει δυνατότητα “μετακίνησης”.

Ένα “κάμπριο”, πέρα από το ότι “δεν έχει οροφή”, αφού είναι “αυτοκίνητο”, “έχει 4 τροχούς” και ανά πάσα στιγμή βρίσκεται σε μία “θέση” και έχει δυνατότητα “μετακίνησης”. Ένα “χάτσμπак”, πέρα από το ότι έχει “πίσω πόρτα”, αφού είναι “αυτοκίνητο”, “έχει 4 τροχούς” και ανά πάσα στιγμή βρίσκεται σε μία “θέση” και έχει δυνατότητα “μετακίνησης”. Μια “λιμουζίνα”, πέρα από το ότι “δεν έχει πίσω πόρτα”, αφού είναι “αυτοκίνητο”, “έχει 4 τροχούς” και ανά πάσα στιγμή βρίσκεται σε μία “θέση” και έχει δυνατότητα “μετακίνησης”. Ένα “4x4”, πέρα από το ότι έχει “κίνηση στους 4 τροχούς”, αφού είναι “αυτοκίνητο”, “έχει 4 τροχούς” και ανά πάσα στιγμή βρίσκεται σε μία “θέση” και έχει δυνατότητα “μετακίνησης”.

Βλέπουμε, λοιπόν, ότι κάθε οντότητα/έννοια “κληρονομεί” και τη λειτουργικότητα της γονεϊκής της, εκτός από εκείνη την οποία επαναορίζει — στο παράδειγμά μας, η λειτουργικότητα που αφορά τους “τροχούς” επαναορίζεται στις υποκατηγορίες των οχημάτων. Επίσης δε, η κληρονόμηση αυτή αφορά όλο το βάθος της ιεραρχίας. Συσσωρεύεται, δηλαδή, (μη επαναορισθείσα) λειτουργικότητα από όλους τους “προγόνους” της έννοιας/οντότητας.

Στο σχήμα αυτό, βλέπουμε τις πιο γενικευμένες οντότητες και τις εξειδικεύσεις τους. “Διαβάζουμε” το παραπάνω σχήμα από πάνω προς τα κάτω. Το σχεδιάζουμε ανάποδα. Βρίσκουμε, δηλαδή, κοινά χαρακτηριστικά στις έννοιες/οντότητες του κόσμου μας και φτιάχνουμε νέες, πιο γενικευμένες, οντότητες συσχετίζοντάς τις με αυτά.

Στο σχήμα, βλέπουμε μια ιεραρχική οργάνωση εννοιών — θα μπορούσαμε να την αναφέρουμε και ως *ταξινομία*. Πολλά προβλήματα εμπεριέχουν κόσμους των οποίων οι έννοιες μπορούν να οργανωθούν κατ’ αυτόν τον τρόπο.

7.1.1 Η Κληρονομικότητα στον Αντικειμενοστραφή Προγραμματισμό

Δεδομένου, όμως, ότι ο στόχος μας είναι ο προγραμματισμός, ας εστιάσουμε σε αυτόν, συγκεκριμενοποιώντας τα παραπάνω στο πεδίο του. Έχουμε πει ότι οι οντότητες/έννοιες ενός προβλήματος το οποίο πρόκειται να αντιμετωπιστεί χρησιμοποιώντας αντικειμενοστραφή προγραμματισμό αναπαριστώνται από κλάσεις. Άρα, λοιπόν, σκεφτείτε τα παραπάνω αντικαθιστώντας τους όρους “έννοια” και “οντότητα” με τον όρο “κλάση”. Στον αντικειμενοστραφή προγραμματισμό, μπορούμε να ορίσουμε μια κλάση ως εξειδίκευση μιας άλλης, οπότε δεν χρειάζεται να επαναλάβουμε ό,τι ορίζεται στην πιο γενική: απλά κληρονομείται από αυτήν. Στον ορισμό της εξειδίκευσης, απλά καθορίζουμε την επιπλέον λειτουργικότητα, καθώς και λειτουργικότητα που μπορεί να επαναορίζεται. Ο μηχανισμός της κληρονομικότητας διασφαλίζει ότι ό,τι έχει οριστεί στη γενικότερη κλάση, θα κληροδοτηθεί στην ειδικότερη. Επίσης, πρέπει να τονιστεί ότι αν μια κλάση *είναι* εξειδίκευση μιας άλλης κλάσης, τα αντικείμενα της πρώτης *είναι* αντικείμενα και της δεύτερης. Κατά συνέπεια, όπου απαιτείται αντικείμενο τύπου της πιο γενικής κλάσης, μπορεί να χρησιμοποιηθεί αντικείμενο της πιο εξειδικευμένης κλάσης. Προσοχή, το ανάποδο δεν ισχύει: ένα αντικείμενο τύπου “κάμπριο” είναι αντικείμενο τύπου “αυτοκινήτου”, ένα αντικείμενο τύπου “αυτοκινήτου”, όμως δεν είναι απαραίτητα τύπου “κάμπριο”, οπότε δεν μπορεί να χρησιμοποιηθεί σε περιπτώσεις που απαιτούνται αντικείμενα του τύπου αυτού.

Ένα μεγάλο ζήτημα στην κληρονομικότητα είναι η *πολλαπλή κληρονομικότητα* (multiple inheritance). Αυτή αναφέρεται στην περίπτωση που μια έννοια/οντότητα αποτελεί άμεση εξειδίκευση παραπάνω από μίας έννοιας/οντότητας. Ας δούμε το πρόβλημα που υπάρχει μέσω παραδείγματος. Έστω ότι έχουμε τις αντίστοιχες κλάσεις για την έννοια του “Φοιτητή” και την έννοια του “Εργαζόμενου”. Αν θεωρήσουμε ότι κάποιος ως φοιτητής έχει μηδενικό εισόδημα, ενώ κάποιος εργαζόμενος έχει εισόδημα που καθορίζεται από τις απολαβές του, μπορούμε να θεωρήσουμε ότι οι κλάσεις που αναπαριστούν τις παραπάνω έννοιες συμπεριλαμβάνουν στη λειτουργικότητά τους τον καθορισμό του εισοδήματος με τον ανάλογο τρόπο. Έστω, τώρα, η έννοια του “Εργαζόμενου Φοιτητή” που αναφέρεται στους φοιτητές οι οποίοι εργάζονται. Η έννοια αυτή είναι εξειδίκευση τόσο της έννοιας του “Φοιτητή”, όσο και της έννοιας του “Εργαζόμενου”. Το πρόβλημα, όμως, είναι ότι, στην περίπτωση αυτή, δεν είναι καλά ορισμένο τι σημαίνει η “κληρονομηση” από τις γενικότερες κλάσεις της όσον αφορά τη λειτουργικότητά της. Εμείς, ως άνθρωποι, καταλαβαίνουμε ότι ένας “Εργαζόμενος Φοιτητής” κληρονομεί τη λειτουργικότητα του “εισοδήματος” από τη φύση του ως “Εργαζόμενος”, αλλά αυτό δεν προκύπτει από αυτά που συζητήσαμε σχετικά με την κληρονομικότητα. Έχουν προταθεί διάφορες λύσεις για περιπτώσεις σαν αυτή. Η πιο άμεση είναι, σε περίπτωση ασάφειας, κατά τη χρήση μιας λειτουργίας, να καθορίζεται ρητά η εμβέλεια από την οποία θα επιλεγεί ο ορισμός της. Άλλη προσέγγιση είναι να δίνεται προτεραιότητα μεταξύ των γενικότερων κλάσεων κατά την εξειδίκευση. Άλλη προσέγγιση είναι να κληρονομείται η πιο άμεσα κληρονομημένη από πλευράς βάθους κληρονομιάς και να γίνεται προσδιορισμός μόνο αν προκύπτει από διαφορετικά μονοπάτια κληρονομιάς ίδιου βάθους. Ανάλογα με τη φύση της γλώσσας προγραμματισμού, μπορεί να υπάρξουν και άλλα εναλλακτικά, όπως η συνένωση της λειτουργικότητας, σε περιβάλλοντα λογικού προγραμματισμού. Ως γενική παρατήρηση, όμως, θα προτείναμε η πολλαπλή κληρονομικότητα να αποφεύγεται στον προγραμματισμό, αναθεωρώντας το μοντέλο της υλοποίησης ώστε να επιτευχθεί ένα πιο καθαρό, στο οποίο τέτοιες καταστάσεις δεν εμφανίζονται. Κατά παρέκκλιση, μπορεί να έχουμε πολλαπλή κληρονομικότητα εάν είναι απόλυτα απαραίτητο, ή αυτή δεν οδηγεί σε καταστάσεις στις οποίες υπάρχει ασάφεια.

Στο σημείο αυτό, να κάνουμε μια επισήμανση: τόσο με την κληρονομικότητα μεταξύ κλάσεων, όσο και με τη σύνθεση κλάσεων, επιτυγχάνουμε *επαναχρησιμοποίηση* κλάσεων. Και στις δύο περιπτώσεις, σε ορισμούς

νέων κλάσεων χρησιμοποιούμε ήδη υπάρχουσες κλάσεις. Όμως, ως έννοιες είναι τελείως διαφορετικές, μια που αναπαριστούν τελείως διαφορετικές συσχετίσεις:

- Η *κληρονομικότητα* σχετίζεται με τη συσχέτιση **isa**: ένα “αυτοκίνητο” είναι “όχημα”. Ένα αντικείμενο της εξειδικευμένης κλάσης είναι και αντικείμενο της πιο γενικής.
- Η *σύνθεση* σχετίζεται με τη συσχέτιση **has a**: ένα “αυτοκίνητο” έχει “μηχανή”. Ένα αντικείμενο της νέας κλάσης (εμπερι)έχει ένα αντικείμενο της υπάρχουσας.

Καλό, λοιπόν, είναι η διάκριση αυτή να γίνει κατανοητή, ώστε κατά περίπτωση να χρησιμοποιείται η κατάλληλη συσχέτιση μεταξύ των κλάσεων. Αυτό συμβάλλει στο να επιτευχθεί η καθαρότητα της οργάνωσης του κώδικα, στην οποία αποσκοπεί, μεταξύ άλλων, ο αντικειμενοστραφής προγραμματισμός.

Ολοκληρώνοντας την εισαγωγή, ας δούμε λίγο την ορολογία που χρησιμοποιείται στην περίπτωση που μια κλάση αποτελεί εξειδίκευση μιας άλλης. Στην περίπτωση αυτή, η πιο γενική κλάση ονομάζεται *υπερκλάση* (superclass) ή *βασική κλάση* (base class). Πιο άτυπα, την αποκαλούμε και “γονεϊκή” ή “μητρική” κλάση. Η εξειδικευμένη κλάση αποκαλείται *υποκλάση* (subclass) ή *παραγόμενη κλάση* (derived class). Άτυπα, την αποκαλούμε “θυγατρική” κλάση ή “κλάση-παιδί”.

7.2 Η Κληρονομικότητα στη C++

Στη C++, νέες κλάσεις μπορούν να οριστούν από ήδη υπάρχουσες ενσωματώνοντας τα χαρακτηριστικά και τη συμπεριφορά τους ή επαναορίζοντάς τα όπου είναι απαραίτητο. Ο μηχανισμός που το επιτυγχάνει αυτό είναι εκείνος της κληρονομικότητας. Στην περίπτωση αυτή, η νέα κλάση είναι *παραγόμενη* της ήδη υπάρχουσας κλάσης, η οποία θεωρείται η *βασική* της.

Μια παραγόμενη κλάση μπορεί να προσθέσει μέλη-δεδομένα και συναρτήσεις-μέλη, επαυξάνοντας τον ορισμό της (εξειδικεύοντας όμως τα στοιχεία της). Όπως είπαμε, όμως, μπορεί και να επαναορίσει μέλη. Στην περίπτωση αυτή, ο τελεστής επίλυσης εμβέλειας μπορεί να χρησιμοποιηθεί για να αναφερθούμε στην εκδοχή που ορίζεται σε μια βασική κλάση, αντί για την επαναορισθείσα στην παραγόμενη — αρκεί η πρόσβαση να μην έχει απαγορευθεί.

Υπάρχουν διάφορα ζητήματα που αφορούν την κληρονομικότητα και πρέπει να καθοριστεί η συμπεριφορά των περιπτώσεων που τα εμπλέκουν. Πριν τα συζητήσουμε εκτενώς, ας δούμε μέσω ενός παραδείγματος πώς εκφράζεται κληρονομικότητα στη C++. Στο παράδειγμα αυτό, καθώς και σε ολόκληρη τη συζήτηση στο βιβλίο αυτό, αναφερόμαστε στη σχέση **isa** μεταξύ κλάσεων — εκτός αν κάπου αναφερθούμε σε κάτι διαφορετικό, οπότε θα γίνει σαφής διευκρίνιση.

Στο παράδειγμα, έχουμε μια κλάση *Student* η οποία αναπαριστά την έννοια του “Φοιτητή” και είναι μια (περιορισμένη) παραλλαγή της κλάσης *Student* που χρησιμοποιούσαμε στα προηγούμενα κεφάλαια. Έστω ότι ο ορισμός της είναι ο εξής:

```
class Student {
private:
    char* name;
    int no;
public:
    Student(const char*);
    ~Student();
    void set_no(int);
    char* get_name() const;
    int get_no() const;
    void print() const;
};
```

Η κλάση `Student` έχει μόνο δύο μέλη-δεδομένα, `name` και `no`. Η τιμή του `name` ανατίθεται κατά την αρχικοποίηση και μόνο. Η κλάση έχει, επιπλέον, μία συνάρτηση-μέλος, την `print`, για την εκτύπωση της πληροφορίας ενός στιγμιοτύπου της στην καθιερωμένη έξοδο. Έστω ότι δεν έχουμε ορισμένη συνάρτηση αρχικοποίησης μέσω αντιγραφής, ούτε επαναορισμό του τελεστή ανάθεσης.

Μια υλοποίηση των παραπάνω συναρτήσεων-μελών μπορεί να είναι η εξής:

```
void Student::set_no(int n)
{
    no = n;
}

char* Student::get_name() const
{
    return name;
}

int Student::get_no() const
{
    return no;
}

void Student::print() const
{
    cout << "The name of the student is: " << name << endl;
    cout << "The AM of the student is: " << no << endl;
}

////////////////////////////////////
Student::Student(const char* nam)
{
    name = new char[strlen(nam)+1];
    strcpy(name,nam);
    cout << "A student was just constructed " << endl;
}

////////////////////////////////////
Student::~Student()
{
    cout << "Deleting student with name " << name << endl;
    delete[] name;
}
```

Ας υποθέσουμε, τώρα, ότι θέλουμε να ορίσουμε μια κλάση που να αναπαριστάνει την έννοια του “Εργαζόμενου Φοιτητή” ως εξειδίκευση της έννοιας του “Φοιτητή”. Δηλαδή, ένας “Εργαζόμενος Φοιτητής” *isa* “Φοιτητής”. Μπορούμε, τότε, να ορίσουμε μια κλάση `WorkingStudent` ως υποκλάση της κλάσης `Student`. Ο τρόπος να δηλώσουμε τη συσχέτιση αυτή στη C++ είναι να την προσδιορίσουμε στον ορισμό της κλάσης `WorkingStudent` ως εξής:

```
class WorkingStudent : public Student
```

Ας υποθέσουμε ότι ένας “Εργαζόμενος Φοιτητής”, πέρα από τη φύση του ως “Φοιτητής”, έχει “μισθό” και, με βάση τους “μήνες” που έχει δουλέψει, προκύπτει το “εισόδημά” του. Οπότε, στη συμπεριφορά του

“Εργαζόμενου Φοιτητή” προστίθεται η δυνατότητα υπολογισμού και ανάκτησης του εισοδήματος. Επίσης, υπάρχει και πάλι διαδικασία εκτύπωσης της πληροφορίας ενός “Εργαζόμενου Φοιτητή”. Αυτή εκτυπώνει στην καθιερωμένη έξοδο την επιπλέον πληροφορία του “Εργαζόμενου Φοιτητή”, καθώς και την πληροφορία λόγω της φύσης του ως “Φοιτητής”. Με βάση αυτήν την περιγραφή, ένας ορισμός της κλάσης `WorkingStudent` θα μπορούσε να είναι ως εξής:

```
class WorkingStudent : public Student {
    int salary;
    int months;
public:
    WorkingStudent(const char*, int);
    void set_months(int);
    int get_income() const;
    void print() const;
};
```

Βλέπουμε στον ορισμό αυτό τα μέλη-δεδομένα που προσθέτει η κλάση `WorkingStudent` σε αυτά που έχει κληρονομήσει από τη γονεϊκή της κλάση, `Student`, και τα οποία είναι τα:

```
int salary;
int months;
```

Κάθε αντικείμενο, λοιπόν, της κλάσης `WorkingStudent` “κτίζεται” από τη συνένωση των μελών-δεδομένων και των δύο κλάσεων, δηλαδή τα:

```
char* name;
int no;
int salary;
int months;
```

Από πλευράς, όμως, ορατότητας, η εμβέλεια της κλάσης `WorkingStudent` δεν έχει δικαίωμα πρόσβασης στα ονόματα που είναι “ιδιωτικά” στην εμβέλεια της κλάσης `Student`. Άρα, στην εμβέλεια της κλάσης `WorkingStudent` μπορούμε να χρησιμοποιήσουμε μόνο ονόματα που είναι δηλωμένα στο `public` μέρος της κλάσης `Student`.

Όσον αφορά την αρχικοποίηση των αντικειμένων της κλάσης `WorkingStudent`, τα μέλη-δεδομένα που κληρονομούνται από την κλάση `Student` αρχικοποιούνται με χρήση συνάρτησης αρχικοποίησης της κλάσης `Student`. Αυτή προσδιορίζεται στη συνάρτηση αρχικοποίησης της κλάσης `WorkingStudent` μέσω λίστας αρχικοποίησης.

Με βάση τις παρατηρήσεις αυτές, οι συναρτήσεις-μέλη της κλάσης `WorkingStudent` θα μπορούσαν να οριστούν ως εξής:

```
WorkingStudent::WorkingStudent(const char* name, int sal)
    : Student(name), salary(sal)
{
    cout << "A Working Student has been created with salary: "
          << salary << endl;
}

void WorkingStudent::set_months(int mons)
{
    months = mons;
}
```

```

int WorkingStudent::get_income() const
{
    return months * salary;
}

void WorkingStudent::print() const
{
    Student::print();
    cout << "The Income is: " << get_income() << endl;
}

```

Στους ορισμούς αυτούς, προσέξτε τα παρακάτω σημεία.

Στον ορισμό της συνάρτησης αρχικοποίησης της κλάσης `WorkingStudent`,

```

WorkingStudent::WorkingStudent(const char* name, int sal)
    : Student(name), salary(sal)

```

συμπεριλάβαμε, μέσω λίστας αρχικοποίησης, τον προσδιορισμό της συνάρτησης αρχικοποίησης της κλάσης `Student`

`Student(name)`

που επιλέγεται να χρησιμοποιηθεί ώστε να γίνουν διαδικασίες αρχικοποίησης και για τα μέλη που κληρονομούνται από την κλάση αυτή. Αυτό επιτυγχάνεται με χρήση του ονόματος της κλάσης και ορίσματα αντίστοιχα με τα ορίσματα της επιθυμητής συνάρτησης αρχικοποίησης της κλάσης αυτής. Όπως αναμένεται, οι τιμές των παραμέτρων τους προκύπτουν από την ύπαρξη κατάλληλων ορισμάτων στη συνάρτηση αρχικοποίησης της κλάσης `WorkingStudent`.

Ένα δεύτερο σημείο που πρέπει να προσέξετε στους παραπάνω ορισμούς αφορά τη συνάρτηση-μέλος `void WorkingStudent::print() const`. Η συνάρτηση αυτή επιθυμούμε να εκτυπώνει στην καθιερωμένη έξοδο τις πληροφορίες ενός συγκεκριμένου “Εργαζόμενου Φοιτητή”. Όσον αφορά μέλη της κλάσης `WorkingStudent`, προφανώς δεν υπάρχει κάποιος περιορισμός χρήσης των ονομάτων τους. Όσον αφορά, όμως, κληρονομημένα μέλη, προκειμένου να επιτύχουμε τον σκοπό μας, πρέπει να καλέσουμε τη συνάρτηση-μέλος `print` της εμβέλειας της κλάσης `Student`. Ο τρόπος να το προσδιορίσουμε αυτό είναι με χρήση του τελεστή επίλυσης εμβέλειας, `::`. Να σημειώσουμε ότι, χωρίς προσδιορισμό θα είχαμε (αναδρομική) κλήση της συνάρτησης-μέλους `print` της εμβέλειας της κλάσης `WorkingStudent`, μια που στην κλάση `WorkingStudent` η συνάρτηση επαναορίζεται.

Ας δούμε, τώρα, τη χρήση της κλάσης `WorkingStudent`, ορίζοντας μια μεταβλητή του τύπου αυτού σε μια συνάρτηση `main` όπως η παρακάτω:

```

int main()
{
    WorkingStudent ws("TheName", 1000); // Object
                                        // creation/initialization

    ws.set_no(100);                     // Inherited function
    ws.set_months(12);                  // Added function
    ws.print();                          // Redefined function

    return 0;
}

```

Στον παραπάνω κώδικα έχουμε τα εξής:

`WorkingStudent ws("TheName", 1000);` Ορισμός μιας μεταβλητής με όνομα `ws` τύπου `WorkingStudent`: δημιουργία αντικειμένου της κλάσης και αρχικοποίησή του.

`ws.set_no(100);` Εφαρμογή στο αντικείμενο της κλάσης `WorkingStudent` συνάρτησης-μέλους κληρονομημένης από την κλάση `Student`.

`ws.set_months(12);` Εφαρμογή στο αντικείμενο της κλάσης `WorkingStudent` συνάρτησης-μέλους ορισμένης στην κλάση `WorkingStudent`.

`ws.print();` Εφαρμογή στο αντικείμενο της κλάσης `WorkingStudent` συνάρτησης-μέλους επαναορισμένης στην κλάση `WorkingStudent`.

Από την εκτέλεση του παραπάνω προγράμματος προκύπτει η εξής εκτύπωση:

```
A student was just constructed
A Working Student has been created with salary: 1000
The name of the student is: TheName
The AM of the student is: 100
The Income is: 12000
Deleting student with name TheName
```

Στην εκτύπωση αυτή, ιδιαίτερο ενδιαφέρον έχουν οι δύο πρώτες γραμμές. Η πρώτη γραμμή προκύπτει από την εκτέλεση της συνάρτησης αρχικοποίησης της κλάσης `Student` και η δεύτερη γραμμή από την εκτέλεση της συνάρτησης αρχικοποίησης της κλάσης `WorkingStudent`. Ο λόγος είναι ότι στην περίπτωση κληρονομικότητας μεταξύ κλάσεων, όταν δημιουργείται ένα αντικείμενο της υποκλάσης, πρώτα καλείται η συνάρτηση αρχικοποίησης της υπερκλάσης και στη συνέχεια καλείται η συνάρτηση αρχικοποίησης της υποκλάσης. Το αντίστροφο συμβαίνει κατά την καταστροφή. Στο παράδειγμά μας, δεν είχαμε ορίσει συνάρτηση καταστροφής στην κλάση `WorkingStudent` — δεν χρειαζόταν —, οπότε βλέπουμε μόνο την εκτύπωση από τη συνάρτηση καταστροφής της κλάσης `Student`,

Deleting student with name TheName

Η αφαιρετική εικόνα στη μνήμη πριν την ολοκλήρωση της εκτέλεσης φαίνεται στο Σχήμα 7.1.

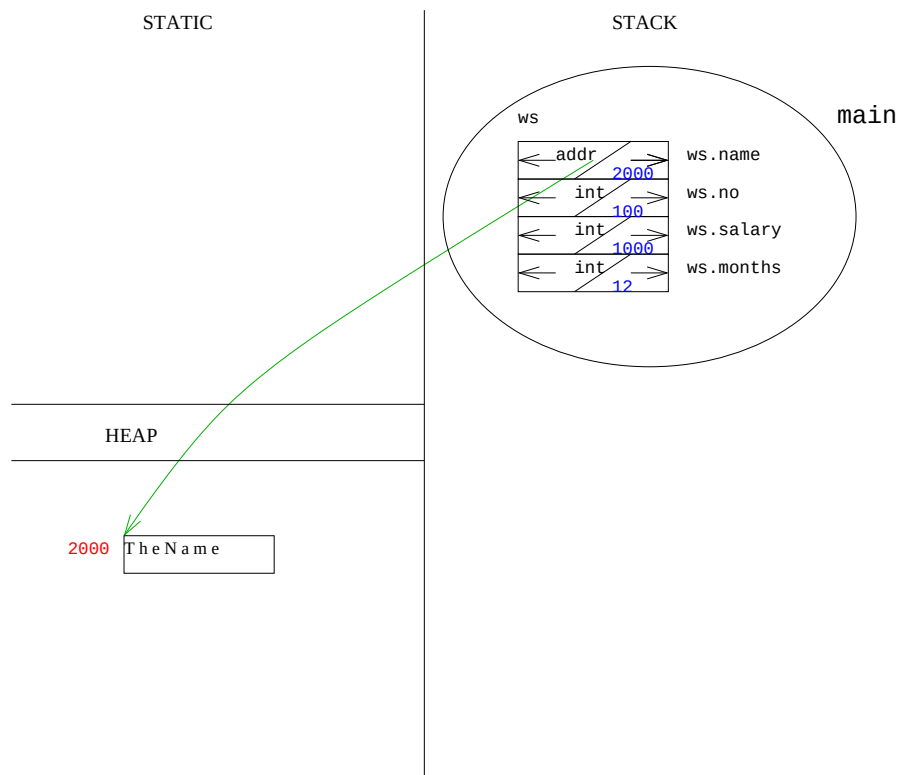
Προσέξτε την οργάνωση της μεταβλητής `ws` τύπου `WorkingStudent`: αποτελείται από τέσσερα μέλη-δεδομένα τα:

```
ws.name
ws.no
ws.salary
ws.months
```

Τα δύο πρώτα μέλη-δεδομένα είναι αυτά που κληρονομούνται από την κλάση `Student` και τα δύο τελευταία είναι αυτά που προστίθενται από την υποκλάση `WorkingStudent`. Κάθε αντικείμενο της κλάσης `WorkingStudent`, όμως, “κτίζεται” από αυτά τα διακριτά τέσσερα μέλη-δεδομένα.

Στην περίπτωση αντικειμένων κλάσης που είναι παραγόμενη άλλης κλάσης, τα μέλη-δεδομένα που κληρονομούνται από τη βασική, στη βιβλιογραφία αναφέρονται και ως *υποαντικείμενο* (subobject) της βασικής. Εμείς, προκειμένου να παραμείνει σαφές το διακριτό των μελών-δεδομένων, αν θελήσουμε να αναφερθούμε στα κληρονομημένα μέλη-δεδομένα συνολικά, θα αναφερόμαστε σε αυτά ως η “φύση” του αντικειμένου ως *αντικείμενο της βασικής*.

Επαναλαμβάνουμε ότι ένα αντικείμενο υποκλάσης μπορεί να θεωρηθεί και ως αντικείμενο της υπερκλάσης της και να χρησιμοποιηθεί οποτεδήποτε χρειάζεται τέτοιο. Παραδείγματα τέτοιων περιπτώσεων είναι εντολές ανάθεσης σε αντικείμενο υπερκλάσης, πέρασμα παραμέτρων του τύπου της υπερκλάσης, χρήση της διεύθυνσής του αντικειμένου ως τιμή δείκτη τύπου υπερκλάσης κλπ. Σε κάποιες από αυτές τις περιπτώσεις, θα επανέλθουμε αργότερα.



Σχήμα 7.1: Η αφαιρετική εικόνα της μνήμης για τη μεταβλητή `ws` τύπου `WorkingStudent`

Ας συνδυάσουμε τους προηγούμενους ορισμούς κλάσεων `WorkingStudent` και `Student` με την παρακάτω συνάρτηση `main`. Στη συνάρτηση αυτή, ορίζουμε τέσσερις μεταβλητές:

1. μία μεταβλητή τύπου `WorkingStudent`, την `ws`,
2. μία μεταβλητή τύπου `Student`, την `s`,
3. μία μεταβλητή τύπου δείκτη σε `Student`, την `ps`, και
4. μία μεταβλητή τύπου δείκτη σε `WorkingStudent`, την `pws`.

```
int main()
{
    WorkingStudent ws("TheName", 1000);
    cout << "Printing from Main:" << endl;
    Student s("FirstLast");

    Student* ps;
    WorkingStudent* pws;

    pws = &ws;
    ps = &s;

    ps = &ws; // ok
    // pws = &s; // error

    s = ws; // Student operator = is called (Student::=)
```

```

        // CAREFUL: as no redefinition
        // exists, not the correct behavior
// ws = s;    // error

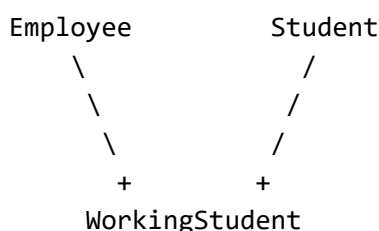
    ps = pws; // ok
// pws = ps; // error

    return 0;
}

```

Στη συνάρτηση αυτή, διακρίνονται καθαρά ποιες αναθέσεις είναι επιτρεπτές και ποιες όχι: ένα αντικείμενο της κλάσης `WorkingStudent` *isa* αντικείμενο και της κλάσης `Student`, το αντίθετο δεν ισχύει. Η σχέση αυτή αφορά και τις μεταβλητές τύπου δείκτη στις αντίστοιχες κλάσεις. Στο παραπάνω, προσέξτε το λεπτό σημείο της ανάθεσης `s = ws`; εφόσον στον ορισμό της κλάσης `Student` του παραδείγματος δεν έχουμε επαναορισμό του τελεστή ανάθεσης, `=`, γίνεται bitwise copy των μελών-δεδομένων της κλάσης `Student`, από τη μεταβλητή `ws` στη μεταβλητή `s`, κατά συνέπεια το πρόβλημα με το μέλος `Student::name` που έχουμε συζητήσει αναλυτικά στα προηγούμενα, επανεμφανίζεται.

Κλείνοντας την παράγραφο αυτή, θα κάνουμε μια σύντομη αναφορά στην πολλαπλή κληρονομικότητα στη C++. Θα μπορούσαμε να είχαμε την παρακάτω ιεραρχία:



Στην περίπτωση αυτή, θα θέλαμε να δηλώσουμε την κλάση `WorkingStudent` ως παραγόμενη κλάση τόσο της κλάσης `Student`, όσο και μιας που αναπαριστάει τον “Εργαζόμενο”, την κλάση `Employee`.

Ο τρόπος να δηλώσουμε πολλαπλή κληρονομικότητα στη C++ είναι στον ορισμό μιας κλάσης να παραθέσουμε όλες τις συσχετίσεις με τις βασικές της κλάσεις. Στο παράδειγμά μας, αυτό θα ήταν:

```

class WorkingStudent : public Student, public Employee {
// ...
};

```

Σε αυτήν την περίπτωση, η κλάση `WorkingStudent` κληρονομεί τα μέλη τόσο από την κλάση `Student`, όσο και από την κλάση `Employee`. Σε περίπτωση σύγκρουσης των κληρονομημένων ονομάτων, στην εμβέλεια της κλάσης `WorkingStudent` γίνεται χρήση του τελεστή επίλυσης εμβέλειας, `::`, προκειμένου να προσδιοριστεί η βασική κλάση από την οποία θα χρησιμοποιηθεί το όνομα. Στο παράδειγμά μας, ας υποθέσουμε ότι υπάρχει συνάρτηση-μέλος `bool submit_tax_declaration()` και στις δύο βασικές κλάσεις, όπου η `Student::submit_tax_declaration` επιστρέφει `false` και η `Employee::submit_tax_declaration` επιστρέφει `true`. Αν υποθέσουμε ότι για την έννοια του “Εργαζόμενου Φοιτητή”, που αναπαριστάει η κλάση `WorkingStudent`, η επιθυμητή συμπεριφορά είναι εκείνη της συνάρτησης της κλάσης `Employee`, τότε για τα αντικείμενα της κλάσης `WorkingStudent`, θα πρέπει να προσδιορίζουμε ρητά τη συνάρτηση ως `Employee::submit_tax_declaration`.

7.2.1 Διαβαθμίσεις των Δικαιωμάτων Πρόσβασης στα Μέλη μιας Κλάσης

Στα προηγούμενα, διευκρινίσαμε ότι μέσα από την εμβέλεια της υποκλάσης `WorkingStudent` δεν έχουμε πρόσβαση στα ιδιωτικά ονόματα της εμβέλειας της κλάσης `Student`. Έτσι, στην εμβέλεια της κλάσης

WorkingStudent χρησιμοποιούσαμε μόνο δημόσια ονόματα της κλάσης Student. Θα ήταν πολύ περιοριστικό, όμως, μια υποκλάση μιας κλάσης να μην έχει παραπάνω δικαιώματα πρόσβασης στα μέλη της υπερκλάσης της από ό,τι το υπόλοιπο πρόγραμμα.

Στη C++, προσφέρεται ένας επιπλέον προσδιορισμός ορατότητας: χαρακτηρισμός ονόματος μέλους ως **protected**. Αν ένα μέλος μιας κλάσης έχει δηλωθεί σε παράγραφο που έχει προσδιορισμό ορατότητας **protected**, το μέλος μπορεί να χρησιμοποιηθεί από την εμβέλεια υποκλάσεων για αντικείμενά τους. Για το υπόλοιπο πρόγραμμα θεωρείται ιδιωτικό. Τονίζουμε ότι, ακόμα και από την εμβέλεια μιας υποκλάσης, μπορεί να χρησιμοποιηθεί μόνο για τα δικά της αντικείμενα κι όχι για αντικείμενα υπερκλάσεών της.

Συνοπτικά, από πλευράς ορατότητας, ένα μέλος μιας κλάσης μπορεί να είναι:

private: Οπότε, το όνομά του μπορεί να χρησιμοποιηθεί μόνο από την εμβέλεια της κλάσης (και από φιλικές συναρτήσεις και φιλικές κλάσεις της κλάσης — την έννοια της “φιλίας” θα τη συζητήσουμε στο επόμενο κεφάλαιο).

protected: Οπότε, το όνομά του μπορεί να χρησιμοποιηθεί από την εμβέλεια της κλάσης (και από φιλικές συναρτήσεις και φιλικές κλάσεις της κλάσης), καθώς και από την εμβέλεια παραγόμενων κλάσεων (και από φιλικές συναρτήσεις και φιλικές κλάσεις παραγόμενων κλάσεων) για τα αντικείμενα των υποκλάσεων και μόνο, όμως.

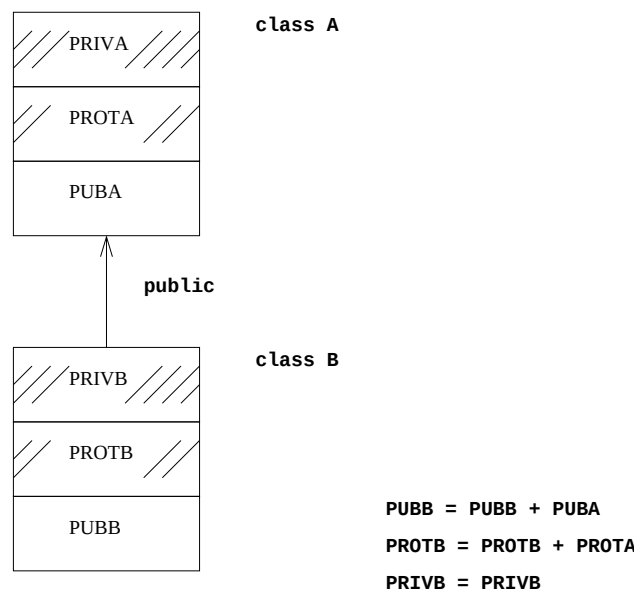
public: Το όνομά του μπορεί να χρησιμοποιηθεί από οποιαδήποτε εμβέλεια του προγράμματος.

Σχηματικά, η κληρονομικότητα μεταξύ δύο κλάσεων A και B που δηλώνεται ως:

```
class B : public A
```

φαίνεται στο Σχήμα 7.2.

```
class B : public A {...};
```



Σχήμα 7.2: Η κλάση B *isa* A στη C++

Στο σχήμα, η κλάση B είναι υποκλάση της κλάσης A και συνοψίζονται οι εξής παρατηρήσεις:

- τα δημόσια ονόματα της κλάσης B (PUBB), επαυξάνονται με τα δημόσια ονόματα της κλάσης A (PUBA),
- τα προστατευόμενα (protected) ονόματα της κλάσης B (PROTB), επαυξάνονται με τα προστατευόμενα ονόματα της κλάσης A (PROTA) και

- ιδιωτικά ονόματα της κλάσης B (PRIVB) παραμένουν μόνο όσα δηλώθηκαν μέσα στην κλάση B.

Οι παραπάνω παρατηρήσεις είναι σημαντικές για την κατανόηση της κληροδότησης κληρονομιάς. Δηλαδή, έστω ότι υπάρχει μια τρίτη κλάση, π.χ. η C, η οποία έχει δηλωθεί ως υποκλάση της κλάσης B. Το πώς αντιμετωπίζονται τα ονόματα που κληροδοτούνται από την κλάση A στην κλάση B ως προς τα αντικείμενα και την εμβέλεια της κλάσης C εξαρτάται από την κατηγορία στην οποία ανήκουν τα ονόματα αυτά στην κλάση A. Η σύνοψη που κάναμε προηγουμένως περιγράφει τη συμπεριφορά της κληροδότησης και, αναδρομικά, για κάθε βάθος ιεραρχίας.

7.2.2 Ζητήματα στην Κληρονομικότητα

Υπάρχουν διάφορα ερωτήματα που πρέπει να μας απασχολήσουν στην περίπτωση που μια κλάση είναι υποκλάση μιας άλλης. Στο παρακάτω, κάνουμε μια σύνοψη κάποιων βασικών ερωτημάτων και την προσέγγιση που ακολουθεί η C++ ως προς αυτά.

- Το πρώτο θέμα το έχουμε ήδη αναλύσει και αφορά την *αναπαράσταση* των αντικειμένων της παραγόμενης κλάσης. Αυτά “κτίζονται” από τη συνένωση των μελών-δεδομένων της βασικής κλάσης και των δικών της.
- Το επόμενο θέμα αφορά τα *θέματα εμβέλειας*. Συγκεκριμένα,
 - Πώς (μέσω ποιων ονομάτων) χειριζόμαστε τα αντικείμενα της παραγόμενης κλάσης από τον υπόλοιπο κώδικα;
Στην περίπτωση της καθαρής “isa” σχέσης κληρονομικότητας (αυτό που η C++ αποκαλεί *public inheritance*), πέρα από τα ονόματα που είναι δηλωμένα σε δημόσιο τμήμα της παραγόμενης κλάσης, μπορούμε να χρησιμοποιήσουμε και κάθε όνομα που είναι σε δημόσιο τμήμα της βασικής κλάσης. Στην περίπτωση που μια συνάρτηση-μέλος έχει επαναοριστεί στην παραγόμενη, ο ορισμός που υπάρχει στη βασική δεν είναι (άμεσα) ορατός.
 - Πώς (μέσω ποιων ονομάτων) χειριζόμαστε τα αντικείμενα της παραγόμενης κλάσης από το περιβάλλον της κλάσης αυτής (π.χ. στα σώματα των ορισμών των συναρτήσεων-μελών της παραγόμενης κλάσης);
Όσον αφορά κληρονομούμενα ονόματα από τη βασική κλάση, μπορούμε να χρησιμοποιήσουμε τα ονόματα που είναι δηλωμένα είτε στο *public* είτε στο *protected* τμήμα της.
 - Πώς (μέσω ποιων ονομάτων) χειριζόμαστε τα αντικείμενα της βασικής κλάσης από το περιβάλλον της παραγόμενης;
Όπως ακριβώς θα τα χειριζόμασταν από οποιοδήποτε άλλο μέρος του κώδικα, δηλαδή μέσω του δημόσιου τμήματός τους.
- Το επόμενο θέμα αφορά τις διαδικασίες *αρχικοποίησης* και *καταστροφής* των αντικειμένων της παραγόμενης κλάσης:
 - Όταν δημιουργείται ένα αντικείμενο μιας παραγόμενης κλάσης, μια συνάρτηση αρχικοποίησης της βασικής κλάσης πρέπει να κληθεί για να αρχικοποιήσει τη “φύση” του ως αντικείμενο της βασικής, π.χ. να αρχικοποιήσει εκείνα τα μέλη του στιγμιότυπου που προέρχονται από τη βασική κλάση.
 - Στις συναρτήσεις αρχικοποίησης της παραγόμενης κλάσης πρέπει να προσδιορίζεται η συνάρτηση αρχικοποίησης της βασικής που είναι επιθυμητό να χρησιμοποιηθεί και να περαστούν οι τιμές στα ορίσματά της (αυτό γίνεται με τη χρήση λίστας αρχικοποίησης). Διαφορετικά, καλείται η *default* συνάρτηση αρχικοποίησης της βασικής κλάσης.
 - Οι συναρτήσεις αρχικοποίησης δεν κληρονομούνται.

- Κατά την κατασκευή ενός αντικειμένου, πρώτα καλείται η συνάρτηση αρχικοποίησης της βασικής κλάσης και μετά της παραγόμενης της.
- Στην καταστροφή, καλούνται οι συναρτήσεις καταστροφής και των δύο κλάσεων, αντίστροφα.
- Αντίστοιχα συμπεριφέρεται η διαδικασία *αρχικοποίησης μέσω αντιγραφής* μεταξύ αντικειμένων της παραγόμενης κλάσης. Με την ίδια σειρά με τη σειρά κλήσης των συναρτήσεων αρχικοποίησης, είτε καλούνται οι ορισμένες συναρτήσεις από τον χρήστη όποτε αυτές έχουν επαναοριστεί είτε γίνεται bitwise copy μεταξύ των αντικειμένων οποτεδήποτε δεν υπάρχουν επαναορισμοί. Προσοχή: αν οριστεί η συνάρτηση αρχικοποίησης μέσω αντιγραφής στην παραγόμενη κλάση, πρέπει μέσω λίστας αρχικοποίησης, να δηλωθεί ρητά η χρήση της συνάρτησης αρχικοποίησης μέσω αντιγραφής της βασικής.
- Ανάλογη είναι και η συμπεριφορά κατά την *ανάθεση* μεταξύ αντικειμένων της παραγόμενης. Σε περιπτώσεις που δεν έχει γίνει επαναορισμός του τελεστή ανάθεσης, γίνεται bitwise copy. Προσοχή: εάν επαναοριστεί ο τελεστής ανάθεσης για την παραγόμενη κλάση, πρέπει στο σώμα του να κληθεί ρητά και ο τελεστής ανάθεσης για τη βασική για να γίνει η ανάθεση μεταξύ των “υποαντικειμένων” της βασικής κλάσης.
- Το τελευταίο θέμα αφορά την *κληροδότηση κληρονομιάς*. Δηλαδή, τι κληροδοτείται στους απογόνους μιας παραγόμενης κλάσης από αυτά που κληρονόμησε από τη βασική της. Στην περίπτωση της καθαρής “isa” σχέσης κληρονομικότητας¹, η απάντηση είναι ότι:
 - τα δημόσια ονόματα μιας παραγόμενης κλάσης επαυξάνονται με τα δημόσια της βασικής της και
 - τα προστατευόμενα ονόματα μιας παραγόμενης κλάσης επαυξάνονται με τα προστατευόμενα ονόματα της βασικής της.
 - Τα ιδιωτικά ονόματα τόσο της παραγόμενης, όσο και της βασικής υπόκεινται στους περιορισμούς που έχουμε αναφέρει.

Τα παραπάνω ισχύουν αναδρομικά για κάθε βάθος ιεραρχίας.

Στη συνέχεια, θα διασαφηνίσουμε τα παραπάνω μέσω παραδειγμάτων.

Παράδειγμα: Αρχικά, θα δούμε ένα παράδειγμα εμπνευσμένο από ένα ανάλογο από το [1]. Θεωρήστε ότι έχουμε την παρακάτω κλάση, X:

```
class X {
protected:
    int i;
public:
    X() : i(10) {}

    void set(int ii)
    { cout << "This is X::set" << endl;
      i = ii; }
```

¹Στο σημείο αυτό να σημειώσουμε ότι η C++ προσφέρει άλλα δύο εναλλακτικά συσχετίσεων μεταξύ κλάσεων τα οποία αποκαλεί *protected inheritance* και *private inheritance*. Στην πρώτη περίπτωση, η συσχέτιση δηλώνεται ως: `class B : protected A ... ;` και αυτό που επιτυγχάνεται είναι να επαυξηθούν τα προστατευόμενα ονόματα της κλάσης B όχι μόνο με τα προστατευόμενα ονόματα της A αλλά και με τα δημόσια ονόματά της. Όμως, τα δημόσια ονόματα της B περιορίζονται σε αυτά που δηλώνονται στα δημόσια τμήματά της. Ακόμα πιο περιοριστική είναι η δεύτερη περίπτωση που δηλώνεται ως: `class B : private A ... ;`. Σε αυτήν την περίπτωση, απλά τα προστατευόμενα και τα δημόσια ονόματα της κλάσης A μπορούν να χρησιμοποιηθούν σαν να ήταν ιδιωτικά στην κλάση B. Οι συσχετίσεις αυτές διευκολύνουν την προσβασιμότητα ονομάτων κατά την υλοποίηση ενός συστήματος, αλλά δεν απεικονίζουν την “isa” σχέση στην οποία αναφέρεται η κληρονομικότητα στο αντικειμενοστραφές μοντέλο, οπότε δεν τις λαμβάνουμε καθόλου υπόψη στο βιβλίο αυτό.

```

int get() const
{ return i; }

int double_val()
{ cout << "Printing from X::double_val: " << i << endl;
  return i = i * 2; }
};

```

Η κλάση X έχει:

- Ένα ακέραιο μέλος-δεδομένο, με προστατευόμενο όνομα `i`.
- Μια default συνάρτηση αρχικοποίησης, η οποία αρχικοποιεί το παραπάνω μέλος-δεδομένο με την τιμή 10.
- Μια μεταβάλλουσα συνάρτηση-μέλος με δημόσιο όνομα `set`, η οποία αναθέτει τιμή στο μέλος-δεδομένο.
- Μια αναγνώστρια συνάρτηση-μέλος με δημόσιο όνομα `get`, η οποία επιστρέφει την τιμή του μέλους-δεδομένου.
- Μια μεταβάλλουσα συνάρτηση με δημόσιο όνομα `double_val`, η οποία διπλασιάζει την τιμή του μέλους-δεδομένου και επιστρέφει τη νέα τιμή του.

Έστω, τώρα ότι έχουμε και μια υποκλάση της X, την Y, η οποία ορίζεται ως εξής:

```

class Y : public X {
    int i;

public:
    Y() : i(0) {}

    int change()           // new function
    { i = double_val(); // inherited function
      return i; }

    void set(int ii)       // redefined function
    { i = ii;
      X::set(ii);          // inherited function
      cout << "This is Y::set" << endl; }

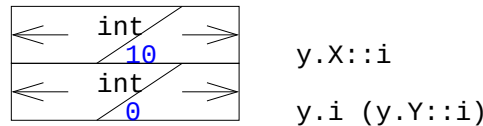
    void print() const
    {
//      X x;
//      cout << x.X::i << endl; // CAREFUL: This is not allowed:
//                               // x is superclass object

      cout << "X::i " << X::i << endl;
//      cout << "X::i " << get() << endl;
//      cout << "X::i " << X::get() << endl;
      cout << "Y::i " << i << endl; }
};

```

Η κλάση Υ:

- Προσθέτει ένα ακέραιο μέλος-δεδομένο με ιδιωτικό όνομα i . Στην εμβέλειά της, χρησιμοποιώντας μόνο το όνομα i , αναφερόμαστε στη μεταβλητή αυτή. Αν για ένα αντικείμενο της θέλουμε να αναφερθούμε στο κληρονομημένο μέλος, θα πρέπει να το προσδιορίσουμε ρητά ως $X : i$. Η αναπαράσταση ενός αντικειμένου της κλάσης Y , έστω y , φαίνεται στο Σχήμα 7.3.



Σχήμα 7.3: Η αναπαράσταση μιας μεταβλητής y ορισμένη ως $Y = y$;

- Έχει μια default συνάρτηση αρχικοποίησης, η οποία αρχικοποιεί το παραπάνω επιπρόσθετο μέλος-δεδομένο με την τιμή 0.
- Προσθέτει μια μεταβάλλουσα συνάρτηση-μέλος `change`. Αυτή αναθέτει στο άμεσα ορατό μέλος-δεδομένο `i`, άρα το επιπρόσθετο `Y::i`, την τιμή που προκύπτει από την επιστροφή της κληρονομημένης συνάρτησης-μέλους `double_val`. Προσοχή: το σώμα της συνάρτησης `double_val` ανήκει στην εμβέλεια της κλάσης `X`, άρα η εκτέλεσή του θα μεταβάλει την τιμή της `i` της εμβέλειας αυτής, άρα της `X::i`. Επομένως, η συνάρτηση-μέλος `change` μεταβάλλει τις τιμές και των δύο μελών, κάνοντάς τις ίσες με το διπλάσιο της προηγούμενης τιμής του `X::i`.
- Επαναορίζει τη μεταβάλλουσα συνάρτηση-μέλος με δημόσιο όνομα `set`, η οποία αναθέτει νέα τιμή τόσο στο επιπρόσθετο όσο και στο κληρονομημένο μέλος-δεδομένο. Προκειμένου να επιτύχει το δεύτερο, χρησιμοποιεί τη δημόσια μεταβάλλουσα συνάρτηση `set` της κλάσης `X`. Δεδομένου ότι η συνάρτηση `set` έχει επαναοριστεί στην κλάση `Y`, η συνάρτηση αυτή πρέπει να προσδιοριστεί ρητά ως `X::set`. Να σημειώσουμε ότι η ανάθεση στο κληρονομημένο μέλος-δεδομένο δεν είναι απαραίτητο να γίνει μέσω κλήσης δημόσιας συνάρτησης-μέλους: το `X::i` έχει προστατευόμενο όνομα, επομένως είναι προσβάσιμο από τη θυγατρική κλάση `Y`, κάτι που αναδεικνύεται στη συνάρτηση-μέλος `print`.
- Προσθέτει μια αναγνώστρια συνάρτηση-μέλος `print`, η οποία εκτυπώνει τις τιμές των μελών-δεδομένων ενός αντικειμένου της. Επιτυγχάνουμε πρόσβαση στο κληρονομημένο μέλος και ανάκτηση της τιμής του με τα εξής εναλλακτικά:

```
X::i
get()
X::get()
```

Όπως είπαμε, η πρόσβαση στο κληρονομημένο μέλος-δεδομένο είναι επιτρεπτή, διότι το όνομά του είναι δηλωμένο ως `protected`. Προκειμένου να γίνει σαφές ότι κάτι τέτοιο είναι επιτρεπτό στην εμβέλεια της κλάσης `Y`, αλλά μόνο για δικά της αντικείμενα, παρατηρήστε ότι αν αποσχολιαστούν οι παρακάτω δύο γραμμές του σώματος της συνάρτησης, θα προκύψει σφάλμα.

```
// X x;  
// cout << x.X::i << endl; // CAREFUL: This is not allowed:  
// x is superclass object
```


Έστω, τώρα, ότι έχουμε και την παρακάτω συνάρτηση `main` στην οποία ορίζεται μια μεταβλητή `obj` τύπου `Y`. Αυτή η μεταβλητή, σύμφωνα με τα παραπάνω, “κτίζεται” από δύο μέλη ακεραίων τα: `obj.X::i` και `obj.i` (το οποίο κατ’ ουσία είναι `obj.Y::i`). Αυτά παίρνουν αρχικές τιμές 10 και 0, αντίστοιχα, λόγω της εκτέλεσης των default συναρτήσεων αρχικοποίησης που είναι ορισμένες στις αντίστοιχες κλάσεις.

Το σώμα της συνάρτησης `main` είναι οργανωμένο σε σχολιασμένα “τμήματα”, BLOCK X, ώστε να αποσχο- λιάζουμε ένα “τμήμα” τη φορά και να μελετάμε τη συμπεριφορά της εκτέλεσης.

```
int main()
{
    /* //// BLOCK A ////
       cout << "sizeof(X) = " << sizeof(X) << endl;
       cout << "sizeof(Y) = " << sizeof(Y) << endl;
    */

    Y obj;

    /* //// BLOCK B ////
       obj.change();
       obj.print();
    */

    /* //// BLOCK C ////
       obj.double_val();
       obj.print();
    */

    /* //// BLOCK D ////
       obj.set(120);
       obj.print();
       obj.X::set(12);
       obj.print();
    */
    return 0;
}
```

Αποσχο- λιάζοντας ένα “τμήμα” τη φορά, παρατηρούμε τα παρακάτω:

BLOCK A: Αποσχο- λιάζοντας το “τμήμα” αυτό, επιβεβαιώνεται ότι το μέγεθος μιας μεταβλητής τύπου `Y` είναι διπλάσιο του μεγέθους μιας μεταβλητής τύπου `X`. Για παράδειγμα, αν σε έναν υπολογιστή, το μέγεθος για τον τύπο `int` είναι 4 bytes, έχουμε τα παρακάτω αποτελέσματα:

```
sizeof(X) = 4
sizeof(Y) = 8
```

BLOCK B: Αποσχο- λιάζοντας το “τμήμα” αυτό, καλείται συνάρτηση-μέλος που προστέθηκε από την κλάση `Y`, η `change`, και παρατηρούμε ότι άλλαξαν τιμή και τα δύο μέλη-δεδομένα της μεταβλητής `obj`. Η νέα τιμή τους είναι το διπλάσιο της τιμής που είχε το `obj.X::i`, η οποία ήταν η (αρχική) τιμή 10. Η συνολική εκτύπωση είναι:

```
Printing from X::double_val: 10
```

```
X::i 20
Y::i 20
```

BLOCK C: Αποσχολιάζοντας το “τμήμα” αυτό, καλείται συνάρτηση-μέλος που κληροδοτήθηκε στην κλάση Y από την κλάση X, η `double_val`. Όπως είπαμε, αυτή λειτουργεί μέσα στην εμβέλεια της κλάσης X. Από την εκτύπωση, παρατηρούμε ότι άλλαξε η τιμή μόνο του μέλους-δεδομένου `obj.X::i`

```
Printing from X::double_val: 10
X::i 20
Y::i 0
```

BLOCK D: Αποσχολιάζοντας το “τμήμα” αυτό, αρχικά καλείται η συνάρτηση-μέλος `set` της εμβέλειας της κλάσης του αντικειμένου `obj`, η οποία είναι η Y, άρα η `Y::set`. Είπαμε ότι η συνάρτηση αυτή αλλάζει τις τιμές και των δύο μελών-δεδομένων της μεταβλητής στην οποία εφαρμόζεται, καλώντας και τη συνάρτηση `X::set`. Κατόπιν, κατά την κλήση της συνάρτησης-μέλους `X::set` εφαρμοσμένη στη μεταβλητή `obj`, αλλάζει μόνο το μέλος `obj.X::i`. Η κλήση αυτή δεν έχει καμία επίδραση στο μέλος `obj.Y::i`, οπότε η τιμή του δεν αλλάζει. Η συνολική εκτύπωση που προκύπτει από την εκτέλεση μετά τον αποσχολισμό του “τμήματος” αυτού είναι:

```
This is X::set
This is Y::set
X::i 120
Y::i 120
This is X::set
X::i 12
Y::i 120
```

7.2.2.1 Αρχικοποίηση κατά την Κληρονομικότητα

Έχοντας δώσει μια σύνοψη των ζητημάτων που προκύπτουν στην περίπτωση κληρονομικότητας μεταξύ κλάσεων, στα προηγούμενα, συζητήσαμε αρκετά περιπτώσεις δικαιωμάτων πρόσβασης και ορατότητας κλάσεων ως προς τα ονόματα της εμβέλειας των γονεϊκών τους, καθώς και αναπαράστασης και χρήσης αντικειμένων κλάσεων που αποτελούν υποκλάσεις άλλων. Στο σημείο αυτό, θα εστιάσουμε στη διαδικασία αρχικοποίησης και στη διαδικασία καταστροφής των αντικειμένων αυτών.

Έχουμε πει ότι όταν δημιουργείται ένα αντικείμενο κλάσης που είναι υποκλάση μιας άλλης, πρώτα καλείται μια συνάρτηση αρχικοποίησης της γονεϊκής και στη συνέχεια καλείται η συνάρτηση αρχικοποίησης της κλάσης του αντικειμένου. Η δεύτερη προσδιορίζεται από τον ορισμό της μεταβλητής. Η πρώτη επιλέγεται σύμφωνα με τη λίστα αρχικοποίησης της δεύτερης.

Κατά την καταστροφή ενός αντικειμένου κλάσης που είναι υποκλάση μιας άλλης, πρώτα καλείται η συνάρτηση καταστροφής της κλάσης του αντικειμένου και στη συνέχεια η συνάρτηση καταστροφής της γονεϊκής της.

Τα παραπάνω ισχύουν για οποιοδήποτε βάθος ιεραρχίας.

Προκειμένου να γίνουν πιο κατανοητά τα παραπάνω, αλλά και για να πειραματιστούμε με ένα παράδειγμα κληρονομικότητας πιο κοντινό στον πραγματικό κόσμο, θεωρήστε ότι έχουμε την κλάση `Student`, όπως αυτή έχει οριστεί στη σελίδα 162. Έστω ότι έχουμε και την έννοια “Μεταπτυχιακός Φοιτητής”, η οποία είναι εξειδίκευση της έννοιας του “Φοιτητή”, που αναπαριστάνεται από την κλάση `Student`, προσθέτοντας επιπλέον την πληροφορία του “έτους αποφοίτησης” των προπτυχιακών σπουδών του. Η πληροφορία αυτή ας θεωρήσουμε ότι είναι συνυφασμένη εξαρχής με έναν “Μεταπτυχιακό Φοιτητή” και ότι θα θέλαμε να πληροφορούμαστε την τιμή της οποτεδήποτε το επιθυμούμε.

Επομένως, μπορούμε να ορίσουμε μια κλάση PGStudent, η οποία να αναπαριστάει την έννοια του “Μεταπτυχιακού Φοιτητή” και την οποία να δηλώσουμε ως υποκλάση της κλάσης Student. Όσον αφορά τον ορισμό της, η κλάση PGStudent έχει τα εξής:

- Προκειμένου να αναπαρασταθεί η πληροφορία του “έτους αποφοίτησης”, προσθέτει ένα ακέραιο μέλος-δεδομένο, `gyear`.
- Χρειάζεται οπωσδήποτε συνάρτηση αρχικοποίησης, κατ’ ελάχιστον για πέρασμα της τιμής του “ονόματος” στο όρισμα της συνάρτησης αρχικοποίησης της κλάσης Student. Επιπλέον, σύμφωνα με τις προδιαγραφές, πρέπει οπωσδήποτε να αρχικοποιείται και το μέλος-δεδομένο `gyear`. Αν δεν δίνεται τιμή, η default τιμή είναι το 0.
- Χρειάζεται μια αναγνώστρια συνάρτηση-μέλος, προκειμένου να ανακτούμε την τιμή του μέλους-δεδομένου `gyear`, έστω `get_year`.
- Δεδομένης της φύσης της επιπλέον πληροφορίας που προσθέσαμε στην κλάση που αναπαριστάει την έννοια του “Μεταπτυχιακού Φοιτητή”, ούτε συνάρτηση καταστροφής, ούτε συνάρτηση αρχικοποίησης μέσω αντιγραφής θα ήταν απαραίτητες. Όμως, επειδή, όπως είπαμε, θέλουμε να δούμε τη συμπεριφορά των διαδικασιών αρχικοποίησης και καταστροφής αντικειμένων, στην περίπτωση που ανήκουν σε υποκλάσεις άλλων κλάσεων, προσθέσαμε και τις συναρτήσεις αυτές. Παρότι η λειτουργικότητά της δεν έχει κάτι ιδιαίτερο, να επαναλάβουμε το λεπτό σημείο στον επαναορισμό της συνάρτησης αρχικοποίησης μέσω αντιγραφής. Εφόσον επαναορίζουμε τη συνάρτηση στην κλάση PGStudent, δεν πρέπει να παραλείψουμε στη λίστα αρχικοποίησής της να υποδείξουμε τη χρήση της συνάρτησης αρχικοποίησης μέσω αντιγραφής της κλάσης Student για την αρχικοποίηση της φύσης του αντικείμενου που αρχικοποιείται ως αντικείμενο της κλάσης Student. Ως όρισμα της συνάρτησης αρχικοποίησης μέσω αντιγραφής της κλάσης Student, σαφώς και μπορούμε να χρησιμοποιήσουμε το όρισμα της συνάρτησης αρχικοποίησης μέσω αντιγραφής της κλάσης PGStudent, μια που ένα αντικείμενο της κλάσης PGStudent *isa* και αντικείμενο της κλάσης Student.

Στο παρακάτω βλέπετε τον ορισμό της κλάσης PGStudent ως υποκλάση της κλάσης Student:

```
class PGStudent : public Student {
    int gyear;    // graduation year
public:
    PGStudent(const char* nam, int gy = 0) : Student(nam), gyear(gy)
    { cout << "A PGStudent was just constructed"
      << " with Graduation year: " << gyear << endl; }

    /**
     * PGStudent(const PGStudent& pgs)
     *   : Student(pgs), // Is there any other possibility
     *                   // for initialization by copying?
     *   gyear(pgs.gyear)
     * { cout << "A PostGraduate Student was just created by copying"
     *   << endl; }

    /**/
    ~PGStudent()
    { cout << "Destroying a PGStudent" << endl; }

    int get_year() const
    { return gyear; }
};
```

Ας δούμε λίγο τη χρήση της κλάσης PGStudent. Στην παρακάτω συνάρτηση main, ορίζουμε τρεις μεταβλητές τύπου PGStudent, τις pgs1, pgs2 και pgs3. Οι δύο πρώτες αρχικοποιούνται απευθείας, η πρώτη με την default τιμή για το “έτος αποφοίτησης”. Η τρίτη μεταβλητή, pgs3, αρχικοποιείται μέσω αντιγραφής με χρήση της μεταβλητής pgs1. Πειραματιστείτε, σχολιάζοντας τη συνάρτηση αρχικοποίησης μέσω αντιγραφής της κλάσης PGStudent. Θα δείτε ότι στην περίπτωση που σχολιάσετε τον ορισμό της, η συνάρτηση αρχικοποίησης μέσω αντιγραφής της κλάσης Student καλείται αυτόματα. Για το επιπλέον μέλος-δεδομένο, gyear, γίνεται bitwise copy.

```
int main()
{
    PGStudent pgs1("FN1");

    cout << "Printing the graduation year from Main: "
          << pgs1.get_year() << endl;

    // Inherited functions:
    pgs1.set_no(123);
    cout << "Printing the no from Main: " << pgs1.get_no() << endl;

    PGStudent pgs2("FN2",2004);
    cout << "Printing the graduation year from Main: "
          << pgs2.get_year() << endl;

    PGStudent pgs3(pgs1); // Copy constructor invoked.
                          // IN CASE NO DEFINITION
                          // of COPY CONSTRUCTOR:
                          // - copy constructor of Student is used
                          // to copy the Student subobject
                          // - bitwise copy for the rest
    cout << "Printing from Main: " << pgs3.get_year() << endl;

    return 0;
}
```

Κατά την εκτέλεση της παραπάνω συνάρτησης main, προκύπτουν οι εξής εκτυπώσεις:

- Από τον ορισμό της μεταβλητής PGStudent pgs1("FN1"); προκύπτουν οι εκτυπώσεις:

A student was just constructed

A PGStudent was just constructed with Graduation year: 0

Η πρώτη οφείλεται στην εκτέλεση της συνάρτησης αρχικοποίησης Student::Student(const char*), η οποία καλείται πρώτη, μια που η κλάση PGStudent είναι υποκλάση της κλάσης Student. Η δεύτερη εκτύπωση προκύπτει από την εκτέλεση της συνάρτησης αρχικοποίησης της κλάσης PGStudent.

- Στη συνέχεια, προκύπτουν οι εκτυπώσεις:

Printing the graduation year from Main: 0

Printing the no from Main: 123

Προκειμένου να προκύψουν οι εκτυπώσεις αυτές, έχουν εφαρμοστεί στη μεταβλητή `pgs1` είτε συναρτήσεις-μέλη που έχουν οριστεί στην κλάση της, την `PGStudent`, είτε συναρτήσεις-μέλη που έχουν κληρονομηθεί από την υπερκλάση της, την `Student` — οι συναρτήσεις `get_year` και `set_no/get_no`, αντίστοιχα.

- Οι επόμενες εκτυπώσεις που προκύπτουν είναι οι:

```
A student was just constructed
A PGStudent was just constructed with Graduation year: 2004
Printing the graduation year from Main: 2004
```

Στις εκτυπώσεις αυτές διακρίνουμε συμπεριφορά ανάλογη με την προηγούμενη περίπτωση.

- Καθότι έπεται στο σώμα της συνάρτησης `main` ο ορισμός `PGStudent pgs3(pgs1);`, που δηλώνει αρχικοποίηση του αντικειμένου `pgs3` μέσω αντιγραφής από το αντικείμενο `pgs1`, ακολουθούν οι εκτυπώσεις από τις συναρτήσεις αρχικοποίησης μέσω αντιγραφής, πρώτα της κλάσης `Student` και στη συνέχεια της κλάσης `PGStudent`.

```
A student was just created by copying ...
A PostGraduate Student was just created by copying
```

- Μετά την τελευταία εκτύπωση από την εκτέλεση του σώματος της συνάρτησης `main`,

```
Printing from Main: 0
```

ακολουθούν οι εκτυπώσεις από τα σώματα των συναρτήσεων καταστροφής. Οι μεταβλητές που ορίστηκαν στο σώμα μιας συνάρτησης, της `main` εν προκειμένω, καταστρέφονται με την αντίστροφη σειρά της δημιουργίας τους.

1. Πρώτη γίνεται η καταστροφή του αντικειμένου `pgs3`, που δημιουργήθηκε τελευταίο. Οι εκτυπώσεις από τις συναρτήσεις καταστροφής που εκτελούνται “για χάρη του” — με “όρισμα” το αντικείμενο αυτό — είναι:

```
Destroying a PGStudent
Deleting student with name FN1
```

Πρώτα βλέπουμε την εκτύπωση από το σώμα της συνάρτησης καταστροφής της κλάσης `PGStudent` και στη συνέχεια την εκτύπωση από το σώμα της συνάρτησης καταστροφής της κλάσης `Student`.

2. Στη συνέχεια γίνεται η καταστροφή του αντικειμένου `pgs2`. Οι εκτυπώσεις που προκύπτουν με τρόπο ανάλογο με τον προηγούμενο είναι:

```
Destroying a PGStudent
Deleting student with name FN2
```

3. Τέλος, γίνεται η καταστροφή του αντικειμένου `pgs1`. Οι εκτυπώσεις είναι ανάλογες:

```
Destroying a PGStudent
Deleting student with name FN1
```

7.3 Ορισμός Κλάσεων με Κληρονομικότητα και Σύνθεση

Έχουμε τονίσει ότι έχοντας ορίσει μια κλάση, πρακτικά, έχουμε επεκτείνει με την κλάση αυτή το σύνολο των διαθέσιμων τύπων προς χρήση στα προγράμματά μας. Στην περίπτωση που αυτό αφορά χρήση κλάσης ως τύπου σε μέλη-δεδομένα νέας κλάσης, έχουμε σύνθεση κλάσεων.

Στα προηγούμενα παραδείγματα, τα μέλη-δεδομένα των κλάσεων που ορίζαμε ως υποκλάσεις ήταν βασικών τύπων. Ας σκεφτούμε, όμως, την περίπτωση ο “Μεταπτυχιακός φοιτητής” μας να μην εμπεριείχε την πληροφορία του “έτους αποφοίτησης”, που αναπαραστήσαμε με μια ακέραια μεταβλητή, αλλά να μας ενδιέφερε η πλήρης ημερομηνία αποφοίτησής του.

Έστω, ότι έχουμε ήδη ορισμένη μια κλάση `Date` η οποία αναπαριστάνει την έννοια της “Ημερομηνίας”. Η κλάση είναι μια απλή παραλλαγή της κλάσης που χρησιμοποιούμε για να αναπαριστάνουμε “ημερομηνίες”.

```
class Date {
    int day; int month; int year;
public:
    Date(int d = 1, int m = 1, int y = 2005)
        : day(d), month(m), year(y)
    { cout << "A Date was just created with value: "
        << day << ' ' << month << ' ' << year << endl; }

    Date(const Date& d) : day(d.day), month(d.month), year(d.year)
    { cout << "A Date was just created by copying" << endl; }

    ~Date()
    { cout << '\n' << "Destroying a Date with value: "
        << day << ' ' << month << ' ' << year << endl; }

    void print() const
    { cout << day << ' ' << month << ' ' << year << endl; }
};
```

Με δεδομένες, τώρα, τις κλάσεις `Date` και `Student`, θεωρούμε ότι έχουμε τον παρακάτω ορισμό της κλάσης `PGStudent`. Ο ορισμός είναι ανάλογος με εκείνον της σελίδας 241, με τη διαφορά ότι το επιπλέον μέλος-δεδομένο, `gdate`, είναι τύπου κλάσης, της `Date`.

```
class PGStudent : public Student {
    Date gdate;    // graduation date
public:
    PGStudent(const char* nam) : Student(nam)
    { cout << "A PGStudent was just constructed " << endl;
      cout << "The Graduation date was :" << endl;
      this -> gdate.print(); }

    PGStudent(const char* nam, int gd, int gm)
        : Student(nam), gdate(gd, gm)
    { cout << "A PGStudent was just constructed " << endl;
      cout << "The Graduation date was :" << endl;
      this -> gdate.print(); }

    ~PGStudent()
    { cout << "Destroying a PGStudent" << endl; }
```

```

Date get_date() const
{ return gdate; }
// const Date& get_date() const { return gdate; }
// CAREFUL: direct access
// to privately named area
};

```

Υπάρχουν κάποια σημεία που αξίζει να σημειώσουμε με αφορμή τον παραπάνω ορισμό.

- Μια διαφορά βλέπουμε στις επικεφαλίδες και στις λίστες αρχικοποίησης των συναρτήσεων αρχικοποίησης: στην πρώτη συνάρτηση, αφήνουμε να γίνει η default αρχικοποίηση του μέλους-δεδομένου gdate, η οποία ανάγεται στη χρήση της συνάρτησης αρχικοποίησης

```
Date::Date(int d = 1, int m = 1, int y = 2005)
```

με χρήση των default τιμών: 1, 1 και 2005. Στη δεύτερη συνάρτηση αρχικοποίησης, παρατηρούμε ότι αφήνουμε το τελευταίο όρισμα της παραπάνω συνάρτησης να πάρει την default τιμή 2005. Όλα τα στιγμιότυπα, λοιπόν, της κλάσης PGStudent που ορίζονται έχουν έτος αποφοίτησης το έτος 2005.

- Κάτι πιο γενικού ενδιαφέροντος τώρα: παρατηρήστε ότι στις συναρτήσεις αρχικοποίησης της κλάσης PGStudent έχουμε την κλήση:

```
this -> gdate.print();
```

Αυτή είναι πλήρως νόμιμη: το αντικείμενο `this->gdate`, που είναι μέλος-δεδομένο του αντικειμένου που αρχικοποιείται, έχει ήδη δημιουργηθεί και αρχικοποιηθεί, οπότε μπορούμε να το χρησιμοποιήσουμε εδώ χωρίς να παράγεται σφάλμα.

- Το τελευταίο που θα θέλαμε να σχολιάσουμε είναι ο τύπος επιστροφής της συνάρτησης-μέλους `get_date`. Αν είναι `Date`, τότε, στη γενική περίπτωση, δημιουργείται και αρχικοποιείται ένα αντικείμενο τύπου `Date`. Αν κάτι τέτοιο δεν είναι επιθυμητό, καλύτερα ο τύπος επιστροφής να είναι `Date&`. Κατ' αυτόν τον τρόπο, όμως, δίνουμε δημόσια πρόσβαση σε χώρο που κατ' όνομα είναι ιδιωτικός. Προκειμένου να αποτρέψουμε αλλαγή τιμής κατ' αυτόν τον τρόπο στον χώρο αυτό, προσδιορίζουμε τη μεταβλητή επιστροφής της `get_date` ως σταθερά, προκειμένου να επιβληθεί να χρησιμοποιείται μόνο σε περιπτώσεις που αναμένεται σταθερά και άρα δεν θα αλλάξει το περιεχόμενό του. Στα θέματα αυτά, όμως, θα επανέλθουμε στο επόμενο κεφάλαιο.

Θεωρώντας τους προηγούμενους ορισμούς κλάσεων, μπορούμε να ορίσουμε μεταβλητές τύπου `PGStudent`, όπως στην παρακάτω συνάρτηση `main`.

```

int main()
{
    PGStudent pgs1("FN");
    // PGStudent pgs2("NFN", 22, 5);

    PGStudent pgs3(pgs1);

    // pgs1.get_date().print(); // Careful:
    // invokes copy constructor and
    // destructor of Date for the
    // temporary return object of get_date

    cout << "Ending main" << endl;
}

```

```

    return 0;
}

```

Στο σώμα της συνάρτησης, έχουμε σχολιάσει τον ορισμό της μεταβλητής `pgs2`, μια που δεν θα προσθέσει κάτι ενδιαφέρον από πλευράς εκτυπώσεων επιπλέον αυτών που θα δούμε στη συνέχεια.

Επίσης, έχουμε σχολιάσει την κλήση της συνάρτησης `Date::print`, που εφαρμόζεται στο αντικείμενο που επιστρέφει η συνάρτηση `PGStudent::get_date`, η οποία εφαρμόζεται στο αντικείμενο `pgs1`. Ο λόγος είναι ότι θα έκανε τις εκτυπώσεις πιο πολύπλοκες, ενόσω δεν αφορούν το θέμα μας αυτήν τη στιγμή. Παρ' όλα αυτά, στην έκφραση αυτή παρατηρήστε την ακολουθία προσδιορισμού ενεργειών στο πλαίσιο χειρισμού του αντικειμένου `pgs1`.

Εκτελώντας τη συνάρτηση `main` με τα σχόλια ως έχουν, παρατηρούμε τις εξής εκτυπώσεις:

- Αρχικά, έχουμε την εκτύπωση του μηνύματος

```
A student was just constructed
```

λόγω της εκτέλεσης της συνάρτησης αρχικοποίησης της κλάσης `Student`, προκειμένου να αρχικοποιηθεί η μεταβλητή `pgs1`, η οποία είναι τύπου υποκλάσης της κλάσης `Student`.

- Στη συνέχεια αρχικοποιείται η “εξειδικευμένη” φύση της μεταβλητής `pgs1`, ως αντικείμενο της κλάσης `PGStudent`. Το μέλος-δεδομένο, όμως, `pgs1.gdate` είναι αντικείμενο κλάσης, της `Date`, και σύμφωνα με τον ορισμό της μεταβλητής `pgs1`, θα κληθεί η `default` συνάρτηση αρχικοποίησης της κλάσης `Date`. Από την εκτέλεσή της θα εκτυπωθεί το μήνυμα:

```
A Date was just created with value: 1 1 2005
```

Ακολουθεί, κατά τα γνωστά, η εκτέλεση του σώματος της συνάρτησης αρχικοποίησης της κλάσης `PGStudent`, οπότε εκτυπώνονται τα παρακάτω:

```
A PGStudent was just constructed
```

```
The Graduation date was :
```

```
1 1 2005
```

- Ακολουθεί ο ορισμός της μεταβλητής `pgs3` τύπου `PGStudent`, η οποία αρχικοποιείται μέσω αντιγραφής από τη μεταβλητή `pgs1`. Η σειρά με την οποία γίνονται οι κλήσεις των συναρτήσεων αρχικοποίησης για την αρχικοποίηση του αντικειμένου `pgs3` είναι η ίδια με τη σειρά με την οποία ακολουθήθηκε για την αρχικοποίηση του αντικειμένου `pgs1`. Για την κλάση `PGStudent`, όμως, δεν έχουμε ορίσει συνάρτηση αρχικοποίησης μέσω αντιγραφής, έτσι εκτελείται αυτή που συνθέτει η γλώσσα. Επομένως, θα δούμε μηνύματα από συναρτήσεις αρχικοποίησης μέσω αντιγραφής της κλάσης `Student` και της κλάσης `Date`. Οι εκτυπώσεις, λοιπόν, που προκύπτουν είναι:

```
A student was just created by copying ...
```

```
A Date was just created by copying
```

- Ακολουθεί η τελευταία εκτύπωση από το σώμα της `main`, που είναι

```
Ending main
```

- Στη συνέχεια, καταστρέφονται τα αντικείμενα με την αντίστροφη σειρά της δημιουργίας τους και βλέπουμε τις εκτυπώσεις από τις συναρτήσεις καταστροφής των κλάσεών τους. Άρα,

1. Κατά την καταστροφή του αντικειμένου `pgs3`, βλέπουμε αρχικά το μήνυμα από τη συνάρτηση καταστροφής της κλάσης `PGStudent`.

```
Destroying a PGStudent
```


Ακολουθεί το μήνυμα από τη συνάρτηση καταστροφής της κλάσης Date, λόγω της καταστροφής του μέλους-δεδομένου pgs3.gdate, που είναι τύπου Date.

Destroying a Date with value: 1 1 2005

Ακολουθεί το μήνυμα από τη συνάρτηση καταστροφής της κλάσης Student, η οποία είναι η υπερκλάση της κλάσης PGStudent.

Deleting student with name FN

Και εφόσον αυτή ούτε σύνθετη κλάση είναι, ούτε υποκλάση κάποιας άλλης, ολοκληρώνεται η ακολουθία των εκτελέσεων συναρτήσεων καταστροφών που προκλήθηκαν, λόγω της καταστροφής της μεταβλητής pgs3.

2. Στη συνέχεια, καταστρέφεται και η μεταβλητή pgs1, οπότε η εκτέλεση ολοκληρώνεται με την εκτέλεση της ίδιας ακολουθίας συναρτήσεων καταστροφής για τη μεταβλητή αυτή, όπως και στην περίπτωση της καταστροφής της μεταβλητής pgs3. Οι εκτυπώσεις που προκύπτουν είναι:

Destroying a PGStudent

Destroying a Date with value: 1 1 2005

Deleting student with name FN

Στην παρακάτω γενική μορφή, συνοψίζεται η σειρά κλήσης συναρτήσεων αρχικοποίησης-καταστροφής, στη γενική περίπτωση που έχουμε αντικείμενα κλάσεων, στις οποίες έχουμε και κληρονομικότητα και σύνθεση:

Κατασκευή:

bmcs
bc
dmcs
dc

Καταστροφή:

dd
dmds
bd
bmcs

Όπου:

bmcs: συναρτήσεις αρχικοποίησης των μελών-δεδομένων της βασικής κλάσης, με τη σειρά που εμφανίζονται στον ορισμό της κλάσης

bc: συνάρτηση αρχικοποίησης της βασικής κλάσης

dmcs: συναρτήσεις αρχικοποίησης των μελών-δεδομένων της παραγόμενης κλάσης, με τη σειρά που εμφανίζονται στον ορισμό της κλάσης

dc: συνάρτηση αρχικοποίησης της παραγόμενης κλάσης

dd: συνάρτηση καταστροφής της παραγόμενης κλάσης

dmds: συναρτήσεις καταστροφής των μελών-δεδομένων της παραγόμενης κλάσης, με την αντίστροφη σειρά που εμφανίζονται στον ορισμό της κλάσης

bd: συνάρτηση καταστροφής της βασικής κλάσης

bmds: συναρτήσεις καταστροφής των μελών-δεδομένων της βασικής κλάσης, με την αντίστροφη σειρά που εμφανίζονται στον ορισμό της κλάσης

Γενικό Παράδειγμα για εξάσκηση: Για περισσότερη εξάσκηση σχετικά με τη συμπεριφορά των συναρτήσεων αρχικοποίησης και καταστροφής, στη γενική περίπτωση που έχουμε σύνθεση αλλά και κληρονομικότητα μεταξύ κλάσεων, πειραματιστείτε σχολιάζοντας και αποσχολιάζοντας γραμμές στον παρακάτω κώδικα.

Στον κώδικα αυτό, έχουμε μια σύνθετη κλάση, την κλάση A, της οποίας τα μέλη-δεδομένα είναι αντικείμενα των κλάσεων A1 και A2. Και οι τρεις κλάσεις έχουν συναρτήσεις αρχικοποίησης, συναρτήσεις αρχικοποίησης μέσω αντιγραφής και συναρτήσεις καταστροφής.

Επίσης, έχουμε άλλη μια κλάση, την B, η οποία είναι σύνθετη κλάση μια που ένα μέλος-δεδομένο της είναι αντικείμενο κλάσης, της κλάσης B1. Η κλάση B συνδέεται μέσω κληρονομικότητας με την κλάση A, μια που έχει οριστεί ως υποκλάση της. Τόσο η κλάση B, όσο και η κλάση B1, έχουν συναρτήσεις αρχικοποίησης, συναρτήσεις αρχικοποίησης μέσω αντιγραφής και συναρτήσεις καταστροφής.

Στη συνάρτηση main, ορίζονται αντικείμενα των κλάσεων A και B. Μελετήστε τη συμπεριφορά των συναρτήσεων αρχικοποίησης και των συναρτήσεων καταστροφής, κάνοντας πειραματισμούς, σχολιάζοντας ή αποσχολιάζοντας μέρη του κώδικα. Με βάση τις προηγούμενες συζητήσεις, θα πρέπει να είστε σε θέση να προβλέπετε σωστά τη συμπεριφορά του προγράμματος κατά την εκτέλεσή του.

```
#include <iostream>
using namespace std;

// Add comments gradually to the copy constructor definitions
// of the following classes starting from the derived class
// to see copy construction initialization under inheritance
// and composition

class A1 {
public:
    A1()
    { cout << "Constructing from A1" << endl; }

    A1(const A1& a1)
    { cout << "Copy Constructing from A1" << endl; }

    ~A1()
    { cout << "Destructing from A1" << endl; }
};

class A2 {
public:
    A2()
    { cout << "Constructing from A2" << endl; }

    A2(const A2& a2)
    { cout << "Copy Constructing from A2" << endl; }
```

```

    ~A2()
    {   cout << "Destructing from A2" << endl; }
};

class A {
    A1 a1;
    A2 a2;
public:
    A()
    {   cout << "Constructing from A" << endl; }

    // A(const A& a) : a1(a.a1), a2(a.a2)
    // {   cout << "Copy Constructing from A" << endl; }
    // Swap with the following copy constructor

    A(const A& a)
    {   cout << "Copy Constructing from A" << endl; }

    ~A()
    {   cout << "Destructing from A" << endl; }
};

class B1 {
public:
    B1()
    {   cout << "Constructing from B1" << endl; }

    B1(const B1& b1)
    {   cout << "Copy Constructing from B1" << endl; }

    ~B1()
    {   cout << "Destructing from B1" << endl; }
};

class B : public A {
    B1 b1;
public:
    B()
    {   cout << "Constructing from B" << endl; }

    // Uncomment the following lines:
    // B(const B& b)
    //     : A(b)
    // {   cout << "Copy Constructing from B" << endl; }

    ~B()
    {   cout << "Destructing from B" << endl; }
};

```

```

int main()
{
    B b1; B b2(b1);
    /*
    A a1; A a2(a1);
    */
    return 0;
}

```

7.4 Πολυμορφισμός μέσω Εικονικών Συναρτήσεων και Αφηρημένες Κλάσεις

Έχει γίνει κατανοητό από τα προηγούμενα ότι σε ένα μονοπάτι ιεραρχίας κλάσεων, ένα αντικείμενο υποκλάσης “είναι” και αντικείμενο όλων των υπερκλάσεων της στο μονοπάτι της ιεραρχίας. Επίσης, μπορεί να χρησιμοποιηθεί οποτεδήποτε αναμένεται μεταβλητή των τύπων των υπερκλάσεων αυτών. Η δυνατότητα αυτή αφορά και τις μεταβλητές τύπου δείκτη και, κατ’ επέκταση, τις μεταβλητές αναφοράς. Για την περίπτωση αναθέσεων με χρήση μεταβλητών κλάσεων και υπερκλάσεων τους, καθώς και δεικτών τέτοιων κλάσεων, είδαμε και ένα μικρό παράδειγμα στη σελίδα 232.

Σχετικά με τα παραπάνω, έχουμε τονίσει ότι ένα αντικείμενο υπερκλάσης δεν μπορεί να θεωρηθεί και αντικείμενο υποκλάσης. Οπότε, δεν επιτρέπεται η ανάθεση στιγμιοτύπου (κλπ.) της βασικής κλάσης σε στιγμιοτύπο (κλπ.) της παραγόμενης κλάσης. Ένας πρόσθετος λόγος που αυτό απαγορεύεται αφορά θέματα υλοποίησης και είναι ότι, στην περίπτωση αυτή, παραμένουν μη ορισμένα τα επιπλέον μέλη-δεδομένα της παραγόμενης κλάσης. Η απαγόρευση αυτή, βέβαια, μπορεί να παρακαμφθεί, είτε επαναορίζοντας τον τελεστή ανάθεσης, είτε/και ορίζοντας συνάρτηση μετατροπής (cast) από τον τύπο της υπερκλάσης στον τύπο της υποκλάσης.

Με τη δυνατότητα να προσπελαύνουμε αντικείμενα παραγόμενων κλάσεων μέσω δεικτών (ή αναφορών) σε υπερκλάσεις επιτυγχάνουμε πολυμορφισμό στη C++. Η χρήση δεικτών στη βασική κλάση (έστω και αν σε αυτόν έχει ανατεθεί ένας δείκτης σε παραγόμενη κλάση) παραπέμπει στην πρόσβαση των συναρτήσεων-μελών της (εμβέλειας της) βασικής κλάσης. Αυτό, όμως, πολλές φορές δεν είναι επιθυμητό. Ποια είναι η λύση: οι συναρτήσεις για τις οποίες η συμπεριφορά αυτή δεν είναι η κατάλληλη να έχουν προσδιοριστεί ως `virtual`, οπότε να είναι *εικονικές συναρτήσεις* (virtual functions). Τότε, αν η εφαρμογή τους σε ένα αντικείμενο είναι μέσω δείκτη και όχι άμεσα στο αντικείμενο αυτό, ο ορισμός που χρησιμοποιείται είναι αυτός της συνάρτησης που είναι ορατή στην εμβέλεια της κλάσης του αντικειμένου και όχι της κλάσης του δείκτη. Τα παραπάνω ισχύουν και στην περίπτωση που, αντί για δείκτη, έχουμε αναφορά. Να κάνουμε σαφές ότι στην περίπτωση που μια συνάρτηση-μέλος εφαρμόζεται απευθείας πάνω σε αντικείμενο, ο προσδιορισμός `virtual` δεν παίζει κανένα ρόλο: η συνάρτηση-μέλος που χρησιμοποιείται είναι εκείνη που είναι απευθείας ορατή στην εμβέλεια της κλάσης του αντικειμένου.

7.4.1 Η Συμπεριφορά των Εικονικών Συναρτήσεων μέσω Παραδειγμάτων

Προκειμένου τα παραπάνω να γίνουν πιο καθαρά, ας συνεχίσουμε με παραδείγματα. Τα ονόματα των κλάσεων είναι εμπνευσμένα από τον κόσμο των μουσικών οργάνων. Τον κόσμο αυτό χρησιμοποιεί και το [1] στο αντίστοιχο πρόβλημα.

Στο παράδειγμά μας, έχουμε μια κλάση `I` που αναπαριστάει “Μουσικά Όργανα” και μια κλάση `W` που αναπαριστάει τα “Πνευστά”. Τα “Πνευστά” είναι εξειδίκευση των “Μουσικών Οργάνων”, οπότε η κλάση `W` δηλώνεται ως υποκλάση της κλάσης `I`. Και οι δύο κλάσεις έχουν ορισμένη μια συνάρτηση-μέλος

```
void play() const
```

προκειμένου να αναπαρασταθεί η λειτουργικότητά τους. Προφανώς, υπάρχει εξειδίκευση των “Πνευστών” ως προς τη λειτουργικότητα αυτή, σε σχέση με τα “Μουσικά Όργανα”.

7.4.1.1 Ένα Πρώτο Παράδειγμα Εικονικών Συναρτήσεων

Ας υποθέσουμε ότι έχουμε τους παρακάτω ορισμούς:

```
class I {
public:
    // virtual void play() const = 0; // pure virtual
    // Makes the class to be abstract:
    // see later
    // virtual void play() const
    void play() const
    { cout << "I::play" << endl; }
};

class W : public I {
public:
    void play() const
    { cout << "W::play" << endl; }
};
```

Στους παραπάνω ορισμούς, στις συναρτήσεις-μέλη `play`, απλά, έχουμε εκτυπώσεις για να κάνουμε εμφανές ποια είναι η κλάση στην οποία ανήκει η συνάρτηση που εκτελείται όταν την καλούμε.

Ας υποθέσουμε ότι έχουμε την παρακάτω συνάρτηση `main`, στην οποία ορίζουμε τις εξής μεταβλητές:

- Μια μεταβλητή `instr` τύπου `I`.
- Μια μεταβλητή `flute` τύπου `W`.
- Μια μεταβλητή `pflute` τύπου δείκτη σε `I`, δηλαδή τύπου `I*`.

Θα συζητήσουμε τη συμπεριφορά της εκτέλεσης κατά την κλήση των συναρτήσεων-μελών, εστιάζοντας στην περίπτωση κατά την οποία η μεταβλητή δείκτη, `pflute`, “δείχνει” στη μεταβλητή `flute`. Στη συνάρτηση, κάνουμε κάποιες αναθέσεις μεταξύ αντικειμένων ή δεικτών, προκειμένου να θυμηθείτε τι είναι επιτρεπτό και τι δεν είναι. Δεν θα τις συζητήσουμε πάλι, όμως.

```
int main()
{
    I instr;
    instr.play();

    W flute;
    flute.play();

    instr = flute;
    // flute = instr; // error

    I* pflute = &instr;
    pflute->play();

    pflute = &flute;
    pflute->play();
}
```

```

    return 0;
}

```

Ξεκινώντας από τις πρώτες κλήσεις, εφόσον η μεταβλητή `instr` είναι αντικείμενο τύπου `I`, είναι καθορισμένο ότι η κλήση:

```
instr.play();
```

αναφέρεται στη συνάρτηση `I::play`, οπότε η εκτύπωση που θα προκύψει είναι:

```
I::play
```

Το ίδιο ισχύει για την κλήση:

```
flute.play();
```

Η εκτύπωση που θα προκύψει είναι:

```
W::play
```

για τον ίδιο λόγο με την προηγούμενη περίπτωση.

Ας συνεχίσουμε με τις γραμμές:

```

I* pflute = &instr;
pflute->play();

```

Εδώ, η πρόσβαση στο αντικείμενο προκειμένου να εφαρμοστεί η συνάρτηση-μέλος `play` γίνεται μέσω δείκτη. Όμως, ο τύπος του δείκτη `pflute` είναι ο ίδιος με τον τύπο του αντικειμένου στο οποίο δείχνει, το `instr`. Ο τύπος αυτός είναι η κλάση `I`, οπότε η εκτύπωση που προκύπτει είναι:

```
I::play
```

Στη συνέχεια, όμως, βλέπουμε ότι στη μεταβλητή `pflute` ανατίθεται η διεύθυνση του αντικειμένου `flute`:

```
pflute = &flute;
```

Η ανάθεση είναι επιτρεπτή, διότι ο τύπος του δείκτη `pflute` είναι υπερκλάση του τύπου του αντικειμένου στο οποίο δείχνει. Ακολουθεί, όμως, η κλήση:

```
pflute->play();
```

Στην περίπτωση αυτή, η πρόσβαση στο αντικείμενο `flute`, προκειμένου να εφαρμοστεί η συνάρτηση-μέλος `play`, γίνεται μέσω του δείκτη `pflute`. Όμως, το αντικείμενο `flute` είναι τύπου `W` και ο δείκτης `pflute` είναι τύπου `I`. Οπότε, ποια συνάρτηση θα κληθεί; Η `W::play`, που είναι (άμεσα) ορατή στην κλάση του αντικειμένου ή η `I::play`, που είναι ορατή στην κλάση του δείκτη;

Αν αφήσουμε το πρόγραμμα ως έχει, οπότε η επικεφαλίδα της συνάρτησης-μέλους `I::play` είναι:

```
void play() const;
```

η συνάρτηση που θα κληθεί είναι εκείνη του τύπου του δείκτη, άρα η `I::play` και η εκτύπωση είναι:

```
I::play
```

Τι κάνουμε, όμως, αν αυτό δεν είναι το επιθυμητό και θέλουμε να κληθεί εκείνη του τύπου του αντικειμένου;

Εάν θέλουμε ο τύπος του αντικειμένου να καθορίζει τον ορισμό που θα χρησιμοποιηθεί στην κλήση μιας συνάρτησης-μέλους, ανεξάρτητα εάν εφαρμόζεται μέσω δείκτη (ή αναφοράς) ή όχι, τότε η συνάρτηση-μέλος πρέπει να προσδιοριστεί ως `virtual` στη γονεϊκή κλάση από την οποία — και χαμηλότερα στην ιεραρχία — θέλουμε να ισχύει αυτό. Οπότε, για το παραπάνω πρόγραμμα, θα πρέπει να αλλάξουμε την επικεφαλίδα της συνάρτησης `I::play` και αντί να είναι:

```
void play() const;
```

να γίνει

```
virtual void play() const
```

— ανταλλάξτε τους σχολιασμούς στον ορισμό της —. Σε αυτήν την περίπτωση, αν η εφαρμογή της συνάρτησης γίνει μέσω δείκτη (ή αναφοράς) τύπου `I`, η απόφαση για το ποιος ορισμός θα εκτελεστεί δεν επιλύεται στη φάση της μεταγλώττισης, αλλά αναβάλλεται για τη φάση της εκτέλεσης και καθορίζεται από τον ορισμό που είναι άμεσα ορατός στην κλάση του αντικειμένου που “δείχνει” ο δείκτης.

Συνοψίζοντας, με τη δυνατότητα προσδιορισμού μιας συνάρτησης-μέλους σε μια (υπερ)κλάση ως `virtual`, η `C++` δίνει τη λύση στην πρόσβαση στον σωστό ορισμό αν έχουμε κλήση της συνάρτησης μέσω

δείκτη (ή αναφοράς) στην (υπερ)κλάση αυτή, ενώ επιθυμούμε να εκτελεστεί ο ορισμός της κλάσης του αντικειμένου που “δείχνει” ο δείκτης, ακόμα και αν αυτή είναι υποκλάση της κλάσης του δείκτη (ή αναφοράς).

Διαφορετικά, θα μπορούσαμε να κάναμε χρήση πεδίου τύπου στην υπερκλάση και περιπτωσιολογία στις αντίστοιχες συναρτήσεις για όλες τις τιμές των πεδίων τύπου. Αυτό και άκομψο είναι αλλά, κυρίως, μη επεκτάσιμο, μια που προϋποθέτει ότι ξέρουμε εκ των προτέρων όλες τις παραγόμενες κλάσεις. Οπότε, η χρήση συναρτήσεων-μελών που έχουν προσδιοριστεί ως `virtual`, μας βοηθάει να σχεδιάζουμε και να υλοποιούμε συστήματα που είναι εύκολα επεκτάσιμα.

Να υπενθυμίσουμε και να επισημάνουμε το εξής: όταν η κλήση σε μια συνάρτηση-μέλος γίνεται μέσω του ονόματος του αντικειμένου και του τελεστή `.`, είτε είναι εικονική είτε όχι, η αναφορά επιλύεται σε φάση μεταγλώττισης και η συνάρτηση που καλείται είναι αυτή που είναι άμεσα ορατή στην κλάση του αντικειμένου.

Για τις εικονικές συναρτήσεις ισχύουν τα εξής:

- Άπαξ και μια συνάρτηση-μέλος δηλωθεί ως `virtual` σε μια κλάση, παραμένει εικονική για όλη την ιεραρχία των κλάσεων που βρίσκονται κάτω από την κλάση αυτή.
- Μπορεί και να μην επαναορίζεται σε κάποια παραγόμενη κλάση.
- Δεν μπορούμε να έχουμε εικονικές συναρτήσεις κατασκευής αντικειμένων. Μπορούμε, όμως, να έχουμε εικονικές συναρτήσεις καταστροφής.
- ΠΡΟΣΟΧΗ: οι default τιμές εικονικών συναρτήσεων αντικαθίστανται σε φάση μεταγλώττισης, οπότε μπορεί να μην είναι οι επιθυμητές.
- Αρκεί η δήλωση μιας συνάρτησης ως `virtual` σε μια κλάση για να διαπιστωθεί ότι τα αντικείμενα της ιεραρχίας από την κλάση αυτή και κάτω χρειάζονται επιπλέον πληροφορίες κατά την αναπαράστασή τους, οι οποίες θα χρησιμοποιηθούν, αν χρειαστεί, στη φάση εκτέλεσης.

Σχετικά με το τελευταίο σημείο, θα θέλαμε να προσθέσουμε τα εξής. Όταν μια κλάση έχει εικονικές συναρτήσεις, αποκτά, από τον μεταγλωττιστή, έναν “πίνακα συναρτήσεων `virtual`” και τα αντικείμενά της αποκτούν έναν δείκτη στον πίνακα αυτό. Κάθε κλάση της ιεραρχίας από την πρώτη εμφάνιση συνάρτησης `virtual` και χαμηλότερα αποκτά τον δικό της πίνακα συναρτήσεων `virtual`. Για την κάθε συνάρτηση `virtual` μιας κλάσης έχει αντιστοιχηθεί μια θέση του πίνακα συναρτήσεων `virtual`, της οποίας η τιμή οδηγεί στον κατάλληλο ορισμό της συνάρτησης για την κλάση αυτή. Η θέση στον πίνακα του δείκτη ο οποίος αντιστοιχεί σε μια συνάρτηση `virtual` παραμένει η ίδια για όλη την ιεραρχία.

Στον ορισμό της κλάσης `I` υπάρχει ακόμα μια σχολιασμένη γραμμή, στην οποία δεν αναφερθήκαμε καθόλου. Πρόκειται για την:

```
virtual void play() const = 0;
```

Προς το παρόν, δεν θα κάνουμε κάποια συζήτηση σχετικά με την περίπτωση αυτή. Τη συμπεριλάβαμε στο παράδειγμα, μια που έχει τον χαρακτηρισμό `virtual`, αλλά το τι εκφράζει είναι θέμα που θα συζητήσουμε αργότερα.

Ας συνεχίσουμε, όμως, τη συζήτησή μας σχετικά με τις εικονικές συναρτήσεις, προκειμένου να κατανοήσουμε τι μπορούν να μας προσφέρουν και τι όχι. Θεωρήστε ότι έχουμε την παρακάτω συνάρτηση `main`.

```
int main()
{
    I instr;
    W flute;

    I* pflute = &flute;
    pflute->play();
}
```

```

    (*pflute).play();

// (*pflute).W::play(); // error
// pflute->W::play();    // error

    W flute2;
// flute2 = *pflute;     //error

    return 0;
}

```

Στη συνάρτηση αυτή, έχουμε ορίσει τις τρεις μεταβλητές που είχαμε στην προηγούμενη περίπτωση και μία ακόμα μεταβλητή, την `flute2` τύπου `W`. Η `pflute` είναι τύπου `I*` και “δείχνει” στην `flute` τύπου `W`.

Να σχολιάσουμε τα εξής:

- Η εντολή `(*pflute).play()`; έχει ακριβώς την ίδια συμπεριφορά με την εντολή `pflute->play()`; και εξαρτάται από το εάν η συνάρτηση-μέλος `I::play` έχει δηλωθεί ως `virtual` ή όχι.
- Και οι δύο εντολές:

```

(*pflute).W::play();
pflute->W::play();

```

παράγουν σφάλμα, διότι η κλάση του δείκτη `pflute` είναι υπερκλάση της κλάσης `W` και στην υπερκλάση δεν είναι ορατός ορισμός μέλους υποκλάσης. Αυτό ελέγχεται στη φάση μεταγλώττισης και δεν έχει σημασία ο τύπος του αντικειμένου που “δείχνει” ο δείκτης στη φάση εκτέλεσης.

- Επανερχόμενοι στη γενικότερη συζήτηση περί αναθέσεων σε περίπτωση κληρονομικότητας, η ανάθεση:

```
flute2 = *pflute;
```

δεν είναι επιτρεπτή: ο δείκτης `pflute` είναι τύπου υπερκλάσης του τύπου του αντικειμένου `flute2`. Ακόμα και αν “δείχνει” σε αντικείμενο υποκλάσης της κλάσης του, κάνοντας `dereference` τον δείκτη αναφερόμαστε στη φύση του ως αντικείμενο της υπερκλάσης.

Παραλλαγή του παραδείγματος: Το επόμενο παράδειγμα είναι μια παραλλαγή του προηγούμενου παραδείγματος, ώστε να γίνει πλήρως κατανοητό ότι ο δείκτης παραμένει πάντα του τύπου που δηλώθηκε. Στο παράδειγμα αυτό, έχουμε μια παραλλαγή της κλάσης `W` ώστε να έχει και ένα μέλος-δεδομένο, το `int i`. Παρότι αντίθετο με καλές πρακτικές, για λόγους επίδειξης, συμπεριλαμβάνουμε τη δήλωσή του στο δημόσιο τμήμα της κλάσης `W`.

```

class I {
public:
    virtual void play() const
    { cout << "I::play" << endl; }
};

class W : public I {
public:
    W() : i(33) {}

```



```

void play() const
{   cout << "W::play" << endl; }

int i;    // public data member, not a good idea
};

```

Στην παρακάτω συνάρτηση main, ορίζουμε τις μεταβλητές flute και pflute, όπως προηγουμένως, και αναθέτουμε στη μεταβλητή pflute ως τιμή τη διεύθυνση του αντικειμένου flute.

```

int main()
{
    W flute;

    I* pflute = &flute;
    pflute->play();

    cout << flute.i << endl;
    cout << pflute->i << endl;    //error
    cout << (*pflute).i << endl; //error

    return 0;
}

```

Το αντικείμενο flute, ως αντικείμενο της κλάσης W, έχει ένα ακέραιο μέλος-δεδομένο, το flute.i, και το οποίο μπορούμε να προσπελάσουμε με το όνομα αυτό. Στην περίπτωση, όμως, που η πρόσβαση στη μεταβλητή flute γίνεται μέσω δείκτη που είναι τύπου της υπερκλάσης I, στην οποία δεν υπάρχει δηλωμένο αυτό το μέλος, η μεταγλώττιση παράγει σφάλμα.

Η παρατήρηση αυτή δεν αφορά μόνο μέλη-δεδομένα, αλλά μέλη γενικότερα. Στην περίπτωση που έχουμε δείκτη τύπου υπερκλάσης να “δείχνει” σε αντικείμενο υποκλάσης, η πρόσβαση στο αντικείμενο μέσω του δείκτη αυτού πρέπει να γίνει μέσω μελών που είναι ορατά στον τύπο του δείκτη. Δεν θα μπορούσαμε, δηλαδή, να καλέσουμε συνάρτηση-μέλος που είναι ορισμένη στην κλάση του αντικειμένου, αλλά δεν υπάρχει ορατός ορισμός στην κλάση του δείκτη. Στο παράδειγμά μας, αν δεν είχαμε ορισμένη τη συνάρτηση-μέλος I::play, η εντολή pflute->play(); (ή η (*pflute).play();) θα παρήγαγε σφάλμα κατά τη μεταγλώττιση.

7.4.1.2 Παράδειγμα: Ιεραρχία με πάνω από Δύο Κλάσεις

Στο επόμενο παράδειγμα έχουμε ιεραρχία τριών κλάσεων. Συγκεκριμένα, πέρα από τις κλάσεις I και W, έχουμε και μια κλάση “Πηγής Ήχου”, S, η οποία είναι υπερκλάση της κλάσης I. Έστω ότι και αυτή έχει ορισμένη μια συνάρτηση-μέλος S::play. Έστω ότι οι ορισμοί των κλάσεων είναι οι εξής:

```

class S {
public:
    // virtual void play() const
    void play() const
    {   cout << "S::play" << endl; }
};

class I : public S {
public:
    // virtual void play() const

```

```

    void play() const
    { cout << "I::play" << endl; }
};

class W : public I {
public:
    void play() const
    { cout << "W::play" << endl; }
};

```

Έστω ότι έχουμε την παρακάτω συνάρτηση `main`, στην οποία ορίζουμε μια μεταβλητή τύπου `W`, την `flute`, και τρεις μεταβλητές τύπου δείκτη, τις `wpflute`, `ipflute` και `spflute`, τύπων `W*`, `I*` και `S*`, αντίστοιχα. Θα αναθέσουμε και στις τρεις μεταβλητές τύπου δείκτη ως τιμή τη διεύθυνση της μεταβλητής `flute` και θα συζητήσουμε τη συμπεριφορά της εκτέλεσης του προγράμματος σε διάφορες περιπτώσεις προσδιορισμού των συναρτήσεων `play` των κλάσεων ως `virtual`.

```

int main()
{
    W flute;
    flute.play();

    W* wpflute = &flute;
    wpflute->play();

    S* spflute = &flute;
    spflute->play();

    I* ipflute = &flute;
    ipflute->play();

    return 0;
}

```

Κατ' αρχήν, να πούμε ότι όσον αφορά τις δύο πρώτες κλήσεις της συνάρτησης `play`,

`flute.play();`

και

`wpflute->play();`

με βάση τους ορισμούς των μεταβλητών `flute` και `wpflute`, σε κάθε περίπτωση εκτυπώνεται το μήνυμα:

`W::play`

Ενδιαφέρον έχουν οι επόμενες κλήσεις της συνάρτησης `play`:

`spflute->play();`

και

`ipflute->play();`

Στις κλήσεις αυτές, η κλάση του δείκτη είναι ψηλότερα στην ιεραρχία από την κλάση του αντικειμένου στο οποίο "δείχνει". Θα διακρίνουμε τις εξής περιπτώσεις:

- Η συνάρτηση `S::play` να έχει δηλωθεί ως `virtual`, δηλαδή:

```
virtual void S::play() const;
```

Από τη στιγμή που η κλάση `S` βρίσκεται ψηλότερα στην ιεραρχία από τις άλλες δύο κλάσεις, τόσο η συνάρτηση `I::play`, όσο και η συνάρτηση `W::play`, έμμεσα είναι προσδιορισμένες ως `virtual`.

Στην περίπτωση αυτή, και από τις δύο αυτές κλήσεις η εκτύπωση είναι:

`W::play`

διότι εκτελείται η συνάρτηση `play` που είναι άμεσα ορατή στην κλάση του αντικειμένου που “δείχνουν” οι δείκτες και το οποίο είναι τύπου `W`.

- Ας δούμε την περίπτωση όπου έχουμε τις εξής δηλώσεις:

`void S::play() const;`

και

`virtual void I::play() const;`

Στην περίπτωση αυτή, η συνάρτηση `play` δεν είναι `virtual` στην κλάση του δείκτη `spflute` που είναι η κλάση `S`. Επομένως, η κλήση:

`spflute->play();`

θα οδηγήσει στην εκτέλεση της συνάρτησης `play` που είναι άμεσα ορατή στην κλάση του δείκτη, άρα την `S::play`, και η εκτύπωση που προκύπτει είναι:

`S::play`

Δεδομένου, όμως, ότι η συνάρτηση `play` είναι `virtual`, στην κλάση του δείκτη `ipflute`, η κλήση:

`ipflute->play();`

θα οδηγήσει στην εκτέλεση της συνάρτησης `W::play`, για τους γνωστούς λόγους, οπότε η εκτύπωση που προκύπτει είναι:

`W::play`

- Στην περίπτωση που η συνάρτηση `play` δεν έχει δηλωθεί ως `virtual`, ούτε στην κλάση `S`, ούτε στην κλάση `I`, από τις κλήσεις αυτές εκτελούνται οι ορισμοί της συνάρτησης `play` που είναι άμεσα ορατοί στην κλάση του εκάστοτε δείκτη, οπότε προκύπτουν οι εκτυπώσεις:

`S::play`

`I::play`

Στα επόμενα παραδείγματά μας θα εστιάσουμε σε θέματα ορατότητας ονομάτων μέσα στις εμβέλεις των σωματών των συναρτήσεων-μελών.

7.4.1.3 Παράδειγμα: Ιεραρχία Κλάσεων με Μέλη-δεδομένα

Στο παράδειγμα που ακολουθεί, έχουμε τις δύο κλάσεις, `I` και `W`. Η κάθε μία από αυτές, τώρα, έχει ένα μέλος-δεδομένο `int i`.

Να υπενθυμίσουμε εδώ ότι, ενώ οι μεταβλητές τύπου `I` δομούνται από μια μεταβλητή ακεραίου, την `I::i`, οι μεταβλητές τύπου `W` δομούνται από δύο: το κληρονομημένο μέλος, `I::i`, και αυτό που πρόσθεσε η κλάση `W`, το `W::i`.

Στην κλάση `I`, το μέλος `I::i` αρχικοποιείται με την τιμή 10. Στην κλάση `W`, το μέλος `W::i` αρχικοποιείται με την τιμή 20 και το κληρονομημένο μέλος, `I::i`, αφήνεται να αρχικοποιηθεί με τον default τρόπο που προσδιορίζει η κλάση `I`, άρα με την τιμή 10.

Οι συναρτήσεις-μέλη `play` των κλάσεων επαυξάνονται, ώστε να εκτυπώνουν την τιμή του ορατού στην εμβέλειά τους μέλους `i` και στη συνέχεια αυξάνουν την τιμή του.

Οι ορισμοί των κλάσεων `I` και `W` είναι οι εξής:

```
class I {
    int i;
public:
```

```

    I() : i(10) {}

    virtual void play()
// void play()
    { cout << "I::play" << endl;
      cout << i++ << endl; }
};

class W : public I {
    int i;
public:
    W() : i(20) {}

    void play()
    { cout << "W::play" << endl;
      cout << i++ << endl; }
};

```

Ας θεωρήσουμε, τώρα, μια συνάρτηση `main` στην οποία ορίζουμε μια μεταβλητή τύπου `I`, την `instr`, μια μεταβλητή τύπου `W`, την `flute`, και μια μεταβλητή τύπου δείκτη σε `I`, την `pflute`.

```

int main()
{
    I instr;
    instr.play();

    W flute;
    flute.play();

    I* pflute = &instr;
    pflute->play();

    pflute = &flute;
    pflute->play();

    return 0;
}

```

Στη συνάρτηση `main`, αρχικά εφαρμόζεται η συνάρτηση `play` απευθείας στα αντικείμενα `instr` και `flute`, οπότε, σε κάθε περίπτωση, από τις κλήσεις αυτές θα προκύψει η παρακάτω εκτύπωση:

```

I::play
10
W::play
20

```

Για τις πρώτες δύο γραμμές της εκτύπωσης, δεν υπάρχει κάτι να προβληματίσει. Βλέπουμε την εκτύπωση του μηνύματος από το σώμα της συνάρτησης `I::play`, καθώς και την εκτύπωση της τιμής της `instr.i` (`instr.I::i`), η οποία είναι 10 — η οποία στη συνέχεια θα αυξηθεί σε 11.

Στις επόμενες δύο γραμμές, βλέπουμε την εκτύπωση του μηνύματος από το σώμα της συνάρτησης `W::play`, καθώς και την εκτύπωση της τιμής της `flute.i` (`flute.W::i`), η οποία είναι 20, και η οποία

στη συνέχεια θα αυξηθεί σε 21. Θέλουμε να επιμείνουμε λίγο στην 4η γραμμή. Στο σώμα των συναρτήσεων `play` και των δύο κλάσεων γίνεται χρήση της μεταβλητής `i`. Η μεταβλητή `i` που είναι άμεσα ορατή μέσα στο σώμα της συνάρτησης `W::play` είναι το μέλος-δεδομένο `i` της κλάσης `W`. Επομένως, όταν εκτελείται η συνάρτηση `W::play`, εκτυπώνεται και αυξάνεται η τιμή αυτού του “τουβλακιού” της μεταβλητής στην οποία εφαρμόζεται η συνάρτηση.

Μεγαλύτερο ενδιαφέρον, όμως, έχουν οι εκτυπώσεις που προκύπτουν από τον παρακάτω κώδικα:

```
I* pflute = &instr;
pflute->play();
```

```
pflute = &flute;
pflute->play();
```

Θα διακρίνουμε τις εξής περιπτώσεις:

- Στην πρώτη περίπτωση έχουμε δηλώσει τη συνάρτηση `I::play` ως `virtual`. Δηλαδή, θεωρούμε ότι ισχύει η δήλωση:

```
virtual void I::play();
```

Από τον κώδικα:

```
I* pflute = &instr;
pflute->play();
```

προκύπτει η εκτύπωση:

```
I::play
11
```

όπου, πάλι, δεν υπάρχει κάτι ιδιαίτερο να συζητήσουμε. Πρόκειται για το μήνυμα από το σώμα της `I::play` καθώς και την τιμή της μεταβλητής `instr.i` — η οποία και πάλι μετά θα έχει αυξηθεί.

Όσον αφορά τον κώδικα:

```
pflute = &flute;
pflute->play();
```

που ακολουθεί, η συμπεριφορά του είναι ανάλογη με την προηγούμενη περίπτωση και η εκτύπωση είναι:

```
W::play
21
```

Ο λόγος είναι ότι η συνάρτηση `I::play` έχει δηλωθεί ως `virtual`, ο δείκτης `pflute` είναι τύπου `I` και “δείχνει” στη μεταβλητή `flute` που είναι τύπου `W`. Κατά συνέπεια, εκτελείται η συνάρτηση `W::play` και εφαρμόζεται στη μεταβλητή `flute`.

- Προσοχή θέλει η περίπτωση που δεν έχουμε δηλώσει τη συνάρτηση `I::play` ως `virtual`. Δηλαδή, θεωρούμε ότι ισχύει η δήλωση:

```
void I::play();
```

Στην περίπτωση αυτή, ενώ η εκτέλεση μέχρι τον κώδικα:

```
pflute = &flute;
pflute->play();
```

παραμένει ίδια, η συμπεριφορά αλλάζει για τις γραμμές αυτές. Αφού η συνάρτηση `I::play` δεν είναι `virtual`, θα εκτελεστεί η συνάρτηση `I::play`, εφαρμοζόμενη στη μεταβλητή `flute`. Οπότε εμφανίζεται το μήνυμα από το σώμα της:

```
I::play
```

και ακολουθεί η τιμή της μεταβλητής `flute.i`, που δεν έχει τροποποιηθεί μέχρι τότε κατά την εκτέλεση. Οπότε εκτυπώνεται η τιμή της η οποία είναι:

```
10
```

και η οποία, στη συνέχεια, θα αυξηθεί.

7.4.1.4 Παράδειγμα: Κλήσεις μεταξύ Εικονικών και μη Εικονικών Συναρτήσεων-μελών

Θα συνεχίσουμε τη συζήτηση πάνω σε θέματα ορατότητας ονομάτων μέσα στις εμβέλειες των σωμάτων των συναρτήσεων-μελών με ένα πιο περίπλοκο παράδειγμα.

Στο παράδειγμα αυτό, προσθέσαμε στις κλάσεις `I` και `W` μια επιπλέον συνάρτηση-μέλος, την `foo`. Και στις δύο κλάσεις, η συνάρτηση `foo`, πέρα από ένα μήνυμα που δηλώνει την εμβέλεια μέσα στην οποία εκτελείται, καλεί τη συνάρτηση `play` που είναι άμεσα ορατή στην εμβέλεια αυτή.

Ας θεωρήσουμε, λοιπόν, τους παρακάτω ορισμούς για τις κλάσεις `I` και `W`.

```
class I {
    int i;
public:
    I() : i(10) {}

    // Take appropriate comments out, to examine the behavior
    // virtual void foo()
    void foo()
    { cout << "I::foo" << endl;
      play(); } // Bear in mind that this is: this->play()

    // virtual void play()
    void play()
    { cout << "I::play" << endl;
      cout << i++ << endl; }
};

class W : public I {
    int i;
public:
    W() : i(20) {}

    // Try to comment out the foo() definition that follows
    // combining with I::play() virtual
    /**
    void foo()
    { cout << "W::foo" << endl;
      play(); }
```

```

/**/
// Try to comment the play() definition:
// the inherited will access the I::i
/**/
    void play()
    {   cout << "W::play" << endl;
        cout << i++ << endl; }
/**/
};

```

Θεωρούμε ότι έχουμε τη συνάρτηση `main` του προηγούμενου παραδείγματος. Για λόγους ευχρηστίας, την παραθέτουμε στη συνέχεια.

```

int main()
{
    I instr; instr.foo();
    W flute; flute.foo();

    I* pflute = &instr; pflute->foo();
    pflute = &flute; pflute->foo();

    return 0;
}

```

Στην εκτέλεση της `main`, στις τρεις πρώτες κλήσεις της συνάρτησης `foo` καλούνται οι συναρτήσεις που είναι ορατές στην εμβέλεια της κλάσης του αντικειμένου στο οποίο εφαρμόζεται η συνάρτηση `foo`. Αυτό ισχύει και για την κλήση της `play`, την οποία προκαλεί η κλήση τους. Οι λόγοι για τα παραπάνω είναι ότι η εφαρμογή των συναρτήσεων γίνεται είτε άμεσα στο αντικείμενο είτε μέσω δείκτη ίδιου τύπου με τον τύπο του αντικειμένου.

Ας εστιάσουμε, τώρα, στη γραμμή:

```
pflute = &flute; pflute->foo();
```

Εδώ, έχουμε διαφορά στον τύπο του δείκτη, `I* pflute`, από τον τύπο της μεταβλητής στην οποία “δείχνει”, `W flute`. Οπότε, κατά τα γνωστά, το ποιος ορισμός θα επιλεγεί, εξαρτάται από το εάν η συνάρτηση `I::foo` έχει δηλωθεί ως `virtual` ή όχι. Όμως, στην περίπτωση που επιλεγεί η `I::foo`, είτε λόγω μη δήλωσής της ως `virtual` είτε διότι έχει σχολιαστεί ο ορισμός της στην κλάση `W`, οπότε κληροδοτείται σε αυτήν, υπάρχει ερώτημα σε σχέση με το ποιος ορισμός θα επιλεγεί κατά την κλήση:

```
play();
```

που υπάρχει στο σώμα της. Η απάντηση είναι ότι εξαρτάται από το εάν έχει δηλωθεί η συνάρτηση `I::play` ως `virtual` ή όχι. Για να γίνει αυτό πιο αντιληπτό, μην ξεχνάτε ότι ουσιαστικά η κλήση αυτή είναι η:

```
this->play();
```

οπότε αν ισχύει η δήλωση:

```
void I::play();
```

στον κώδικά μας θα κληθεί η `I::play` — και θα χρησιμοποιηθεί η μεταβλητή `I::i` —, ενώ αν ισχύει η δήλωση:

```
virtual void I::play();
```

στον κώδικά μας θα κληθεί η `W::play` — και θα χρησιμοποιηθεί η μεταβλητή `W::i` —.

Για παράδειγμα, για την περίπτωση που έχουμε,

```
void I::foo();
```

και

```
virtual void I::play();
```

η εκτύπωση είναι:

```
I::foo
I::play
10
W::foo
W::play
20
I::foo
I::play
11
I::foo
W::play
21
```

Στις τελευταίες τρεις γραμμές που αφορούν τη συζήτησή μας, βλέπετε ότι ενώ στην τρίτη από το τέλος, το μήνυμα προκύπτει από την εκτέλεση του σώματος της συνάρτησης `I::foo`, οι δύο τελευταίες εκτυπώσεις προκύπτουν από την εκτέλεση του σώματος της συνάρτησης `W::play`.

Προσεξτε, όμως, αν δεν έχουμε ορισμένη τη συνάρτηση `play` στην κλάση `W`, κληρονομείται από την κλάση `I`. Στην περίπτωση αυτή, η εκτύπωση από την εκτέλεση του προγράμματος είναι:

```
I::foo
I::play
10
W::foo
I::play
10
I::foo
I::play
11
I::foo
I::play
11
```

Δείτε τη διαφορά στις δύο τελευταίες γραμμές της εκτύπωσης. Ο λόγος είναι ότι κατά την κλήση `pflute->foo()` εκτελείται η (κληρονομημένη) συνάρτηση `I::play` και γίνεται πρόσβαση στη μεταβλητή `flute.I::i`. Η ίδια εκτέλεση είχε γίνει και κατά την κλήση:

```
flute.foo();
```

οπότε έχουμε και την πρώτη πρόσβαση στη μεταβλητή `flute.I::i` — η παρατήρηση αυτή αναδεικνύεται και από τις γραμμές της εκτύπωσης:

```
W::foo
I::play
10
```

(4η, 5η και 6η γραμμή).

Δοκιμάστε να εκτελέσετε το πρόγραμμα, σχολιάζοντας και αποσχολιάζοντας τον ανάλογο κώδικα, ώστε να δείτε τη συμπεριφορά στις διάφορες περιπτώσεις και να βεβαιωθείτε ότι μπορείτε να την αιτιολογήσετε, με βάση τις παρατηρήσεις που κάναμε στην προηγούμενη συζήτηση.

7.4.2 Αφηρημένες Κλάσεις

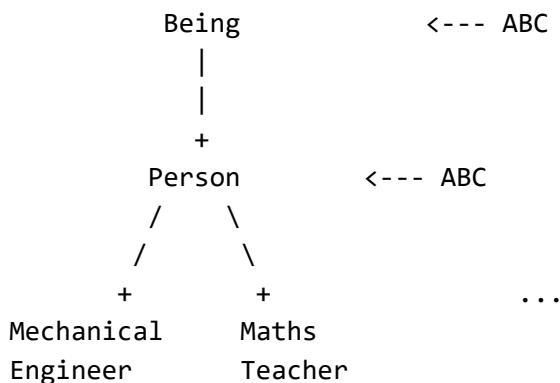
Στον ορισμό της κλάσης `I` του παραδείγματος στη σελίδα 251 υπάρχει μια σχολιασμένη γραμμή, στην οποία δεν αναφερθήκαμε καθόλου στα προηγούμενα. Πρόκειται για την:

```
virtual void play() const = 0;
```

Βλέπουμε ότι πρόκειται για μια δήλωση συνάρτησης, της `void play() const`, η οποία χαρακτηρίζεται ως `virtual`, αλλά η δήλωση ακολουθείται από την ισότητα `= 0`. Αν μια συνάρτηση δηλωθεί ως `virtual` με τον τρόπο αυτό, χαρακτηρίζεται ως *πλήρως εικονική* (pure virtual) και το αποτέλεσμα είναι η κλάση η οποία τη δηλώνει να προσδιορίζεται ως *Αφηρημένη (Βασική) Κλάση* (Abstract Base Class). Όταν μια κλάση είναι αφηρημένη κλάση, απαγορεύεται να δημιουργηθούν αντικείμενά της. Αν μια παραγόμενη κλάση μιας αφηρημένης δεν ορίζει όλες τις πλήρως εικονικές συναρτήσεις της αφηρημένης κλάσης, τότε και η κλάση αυτή είναι αφηρημένη.

Ουσιαστικά, οι αφηρημένες κλάσεις χρησιμεύουν μόνο για κληρονομικότητα προδιαγραφών και υλοποίησης από άλλες (παραγόμενες) κλάσεις. Προσφέρουν, δηλαδή, μια γενική συμπεριφορά την οποία οι παραγόμενες κλάσεις καλούνται να υλοποιήσουν ή/και, ενδεχόμενα, να εξειδικεύσουν.

Για παράδειγμα, μπορεί να έχουμε την κλάση `Person`, της οποίας η συμπεριφορά περιέχει μια συνάρτηση εκτύπωσης της πληροφορίας της, `print`. Αν θεωρήσουμε ότι στο πρόβλημά μας καμία μεταβλητή/αντικείμενο δεν πρέπει να είναι στιγμιότυπο της κλάσης `Person`, γιατί κάθε “Πρόσωπο” ανήκει σε κάποια πιο εξειδικευμένη κατηγορία, π.χ. με βάση την επαγγελματική ενασχόλησή του, τότε η κλάση `Person` πρέπει να χαρακτηριστεί ως αφηρημένη κλάση. Δεδομένου ότι με βάση την ενασχόληση αυτή, θα υπάρχει και επιπλέον/εξειδικευμένη πληροφορία για εκτύπωση, η συνάρτηση `Person::print` θα πρέπει να δηλωθεί ως πλήρως εικονική. Οι κλάσεις στις οποίες αντιστοιχούν οι επαγγελματικές ομάδες θα είναι υποκλάσεις της κλάσης `Person` και θα πρέπει να ορίζουν τη συνάρτηση `print`. Αν προσθέσουμε μια επιπλέον γενική κατηγορία για τα “Όντα” ως αφηρημένη κλάση, `Being`, σχηματικά, αυτός ο κόσμος μπορεί να είναι:



Στο σχήμα αυτό, με `ABC` συμβολίζουμε το ότι η κλάση είναι αφηρημένη.

Παρόλο που δεν μπορούμε να ορίσουμε αντικείμενα μιας αφηρημένης κλάσης, μπορούμε να ορίσουμε δείκτες σε αφηρημένες κλάσεις και αναφορές σε αφηρημένες κλάσεις, για να πετύχουμε πολυμορφικό χειρισμό αντικειμένων παραγόμενων κλάσεων. Αυτός ο χειρισμός μπορεί να αφορά σύνολα αντικειμένων και αποτελεί πολύ συνηθισμένη χρήση των αφηρημένων κλάσεων.

Για να επανέλθουμε στο παράδειγμα της σελίδας 251, κάθε “Μουσικό Όργανο” εμπίπτει σε μια κατηγορία, π.χ. “Πνευστά”, “Εγχόρδα” κλπ. Κατά συνέπεια, η κλάση `I` είναι κατάλληλη να χαρακτηριστεί ως αφηρημένη κλάση. Οπότε, ο κώδικάς της μπορεί να αλλάξει, ώστε η συνάρτηση `play` να δηλωθεί ως πλήρως εικονική.

Ας υποθέσουμε, λοιπόν, ότι ο ορισμός των κλάσεων γίνεται:

```
class I {
public:
    virtual void play() const = 0;
};
```

```
class W : public I {
public:
    void play() const
    { cout << "W::play" << endl; }
};
```

Τότε, στην παρακάτω συνάρτηση main δεν θα μπορούσαμε να ορίσουμε μεταβλητές τύπου I, επομένως η γραμμή με την εντολή `I instr;` πρέπει να σχολιαστεί.

```
int main()
{
    // I instr; instr.play();

    W flute; flute.play();

    I* pflute = &flute;
    pflute->play();
    // pflute->W::play();

    return 0;
}
```

Όμως, όπως είπαμε, μπορούμε να ορίσουμε μεταβλητές τύπου δείκτη σε I, οπότε ο ασχολίαστος κώδικας μεταγλωττίζεται και εκτελείται κανονικά και η εκτύπωσή του είναι:

```
W::play
W::play
```

Σχολιάζοντας τα πλεονεκτήματα της χρήσης αφηρημένων κλάσεων, να πούμε ότι μας επιτρέπουν να δίνουμε γενικές προδιαγραφές και ο κώδικας να είναι επεκτάσιμος ανά πάσα στιγμή με κλάσεις που υλοποιούν τις προδιαγραφές τους ή επεκτείνουν τις υλοποιήσεις τους. Έτσι, δεν υπάρχει λόγος χρήσης περιπτωσιολογίας που επιβάλλει να ξέρουμε εκ των προτέρων όλες τις υποκατηγορίες τους.

Κλείνοντας τη συζήτηση σχετικά με τις αφηρημένες κλάσεις, ας δούμε μια συνήθη χρήση τους μέσω μιας μεταφοράς από τον κόσμο των μουσικών οργάνων. Στιγμιότυπα συγκεκριμένων ειδών μουσικών οργάνων, π.χ. “βιολιών”, “φλάουτων” κλπ., συχνά απαρτίζουν ως σύνολο μια “ορχήστρα” στην οποία το καθένα “παίζει” το μέρος του έργου που το αφορά καθ’ υπόδειξη του “διευθυντή” της. Άρα μπορούμε να αναπαραστήσουμε την “ορχήστρα” ως ένα σύνολο αναφορών σε “μουσικά όργανα” μέσω των οποίων εφαρμόζουμε την ενέργεια του “παιξίματος” σε στιγμιότυπα υποκατηγοριών “μουσικών οργάνων”. Δεν υπάρχουν στιγμιότυπα της γενικής κατηγορίας “μουσικών οργάνων”, όμως. Η γενική κατηγορία αυτή υπάρχει για να προσφέρει τη γενική συμπεριφορά του “παιξίματος” την οποία οι συγκεκριμένες υποκατηγορίες προσδιορίζουν κατά περίπτωση. Το αποτέλεσμα κατά την εκτέλεση ενός “μουσικού έργου” προκύπτει από την εκτέλεση του ιδιαίτερου “παιξίματος” των υποκατηγοριών των “μουσικών οργάνων” από στιγμιότυπά τους. Σημείωση: στο παράδειγμα της μεταφοράς αυτής, και η κατηγορία των “πνευστών” θα αναπαριστανόταν ως αφηρημένη κλάση σε αντίθεση με τα προηγούμενα παραδείγματα της παραγράφου.

7.4.3 Εικονικές Συναρτήσεις Καταστροφής - Γενικό Παράδειγμα στην Κληρονομικότητα

Ας συνοψίσουμε τα σημεία που παρουσιάσαμε στο κεφάλαιο αυτό μέσω ενός παραδείγματος και να συζητήσουμε ένα τελευταίο θέμα: τις εικονικές συναρτήσεις καταστροφής.

Ας θεωρήσουμε την έννοια του “Φοιτητή” και την έννοια του “Μεταπτυχιακού Φοιτητή” ως εξειδίκευσή

της. Ας υποθέσουμε ότι για έναν “Μεταπτυχιακό Φοιτητή” μάς ενδιαφέρει και η πληροφορία του “έτους αποφοίτησής” του αλλά και του τίτλου του “πρώτου πτυχίου” του.

Προκειμένου να αναδείξουμε τα σημεία που επιθυμούμε, προσθέτουμε στον ορισμό της κλάσης `Student` της σελίδας 162 μια συνάρτηση `print`, η οποία απλά εκτυπώνει το “ονοματεπώνυμο” του “Φοιτητή” στην καθιερωμένη έξοδο. Οπότε, θα επαναξήσουμε και την κλάση που αναπαριστάει την έννοια του “Μεταπτυχιακού Φοιτητή”, έστω `PGStudent`, ώστε να επαναορίζει τη συνάρτηση-μέλος `print`, προκειμένου να εκτυπώνει και πληροφορίες που σχετίζονται με την εξειδίκευση της κλάσης, π.χ. το “πρώτο πτυχίο” του.

Επίσης, και πάλι για χάρη της συζήτησης, στην κλάση `PGStudent`, ας αναπαραστήσουμε την πληροφορία του “πρώτου πτυχίου” μέσω ενός μέλους-δεδομένου, έστω `first_degree`, τύπου `char*`.

Σχολιάζοντας τις επεκτάσεις που κάνει η κλάση `PGStudent`, μπορούμε να πούμε τα εξής:

- Ο επαναορισμός της συνάρτησης `print` πρέπει να συμπεριλάβει την κλήση της συνάρτησης `Student::print`, ώστε να επιτύχει την επέκταση της εκτύπωσης.
- Με δεδομένη την αναπαράσταση του μέλους-δεδομένου `first_degree`, στην κλάση `PGStudent`, θα χρειαστεί να γίνει ορισμός συνάρτησης καταστροφής, επαναορισμός της συνάρτησης αρχικοποίησης μέσω αντιγραφής καθώς και επαναορισμός του τελεστή ανάθεσης. Ως προς το τελευταίο, καθώς πλέον σε αναθέσεις μεταξύ μεταβλητών τύπου `PGStudent` θα καλείται ο ορισμός αυτός, θα πρέπει να συμπεριλάβουμε και την ανάθεση μεταξύ τους ως αντικείμενα της κλάσης `Student`, χρησιμοποιώντας τον τελεστή `Student::operator=`. Κατά την κλήση του τελεστή αυτού, θα εκτελεστεί ο ορισμός του από την κλάση `Student` — αν δεν υπήρχε τέτοιος, θα εκτελείτο η διαδικασία που αυτόματα θα συντίθετο από τη `C++`.

Ας πάμε, τώρα, στην κλάση `Student`. Σύμφωνα με το τι περιμένουμε από μια συνάρτηση εκτύπωσης, καταλαβαίνουμε ότι ενδεχόμενες υποκλάσεις της κλάσης `Student` μπορεί να επαναορίζουν τη συνάρτηση `print`. Ας θεωρήσουμε επιπλέον το ενδεχόμενο να θέλουμε να χειριστούμε αντικείμενα υποκλάσεων της μέσω δεικτών (ή αναφορών) τύπου `Student`, είτε π.χ. για λόγους μαζικής επεξεργασίας αντικειμένων διαφορετικών υποκλάσεων είτε λόγω περάσματος παραμέτρων σε συναρτήσεις (με ορίσματα τύπου αναφοράς ή δείκτη τύπου `Student`). Οπότε, θα πρέπει να χαρακτηρίσουμε τη συνάρτηση `print` ως `virtual`. Τέλος, αν δεν επιθυμούμε να δημιουργούμε στιγμιότυπα της γενικής έννοιας “Φοιτητής”, άρα αντικείμενα της κλάσης `Student`, τότε μπορούμε να προσδιορίσουμε τη συνάρτηση `print` ως πλήρως εικονική, ώστε μέσω αυτής να χαρακτηριστεί η κλάση `Student` ως αφηρημένη κλάση. Αυτό μπορεί να γίνει χρησιμοποιώντας τον συμβολισμό `= 0` στη δήλωσή της, ανεξάρτητα αν δώσουμε στη συνέχεια ορισμό της ή όχι. Στην περίπτωση του παραδείγματος, στη συνάρτηση `virtual void Student::print() const = 0` δίνεται, εν συνεχεία, και ορισμός.

Στο σημείο αυτό, όμως, θα συζητήσουμε κάτι που προηγουμένως δεν είχε καθόλου αναφερθεί. Έστω ότι κάνουμε δυναμική αποδέσμευση χώρου που καταλαμβάνει αντικείμενο υποκλάσης μέσω δείκτη τύπου υπερκλάσης. Τότε, ο μόνος τρόπος να γίνει η αποδέσμευση με τέτοιον τρόπο, ώστε να κληθεί η συνάρτηση καταστροφής της υποκλάσης, είναι η συνάρτηση καταστροφής της υπερκλάσης να έχει δηλωθεί ως `virtual`. Μάλιστα, αν μια κλάση αποτελεί παραγόμενη κλάση μιας βασικής η οποία έχει εικονική συνάρτηση καταστροφής, η συνάρτηση καταστροφής της, είτε έχει ρητά οριστεί είτε όχι, θεωρείται και αυτή εικονική.

Με βάση τα παραπάνω, η συνάρτηση καταστροφής της κλάσης `Student` πρέπει να δηλωθεί ως `virtual` για να έχουμε κλήση της συνάρτησης καταστροφής της κλάσης `PGStudent` σε περίπτωση δυναμικής αποδέσμευσης χώρου αντικειμένου της μέσω δείκτη τύπου `Student`. Με βάση τον ορισμό της κλάσης `PGStudent`, λόγω του μέλους-δεδομένου `first_degree`, αυτό είναι απαραίτητο.

Οπότε ο ορισμός της κλάσης `Student` έχει ως εξής:

```
class Student {
private:
    char* name;
    int no;
    int year_of_studies;
```

```

public:
    Student(const char*);
    Student(const Student&);

    // ~Student();
    virtual ~Student();

    Student& operator=(const Student& s);

    void set_name(const char*);
    void set_no(int);
    void set_year_of_studies(int);

    char* get_name() const;
    int get_no() const;
    int get_year_of_studies() const;

    // void print() const;
    // virtual void print() const;
    virtual void print() const = 0;
};

```

Προσέξτε τις γραμμές που σχολιάσαμε. Αφήσαμε την τελική εκδοχή των δηλώσεων που προέκυψε από την ανάλυση που κάναμε προηγουμένως.

Όσον αφορά τον ορισμό των συναρτήσεων-μελών, για να μην επαναλαμβανόμαστε, παρακάτω δίνεται μόνο ο ορισμός της συνάρτησης-μέλους `Student::print`. Οι υπόλοιποι είναι ίδιοι με εκείνους του παραδείγματος της σελίδας 162.

```

void Student::print() const
{
    cout << "The Student's name is: " << name << endl;
}

```

Ο ορισμός της κλάσης `PGStudent` είναι ο εξής:

```

class PGStudent : public Student {

    int gyear;
    char* first_degree;

public:
    PGStudent(const char* nam, const char* fd, int gy = 0)
        : Student(nam), gyear(gy)
    { first_degree = new char[strlen(fd)+1];
      strcpy(first_degree, fd);
      cout << "A PGstudent was just constructed " << endl; }

    // CAREFUL: copy constructor is needed
    PGStudent(const PGStudent& pgs) : Student(pgs), gyear(pgs.gyear)
    { first_degree = new char[strlen(pgs.first_degree)+1];
      strcpy(first_degree, pgs.first_degree);
    }
}

```

```

        cout << "A PGstudent was just created by copying ... "
              << endl; }

// CAREFUL: assignment needs redefinition
//          (alternative implementation if visibility is redesigned
//          Student data members were qualified as "protected")
PGStudent& operator=(const PGStudent& pgs)
{   if (this != &pgs) {
        this->Student::operator=(pgs);
        gyear = pgs.gyear;
        delete[] first_degree;
        first_degree = new char[strlen(pgs.first_degree)+1];
        strcpy(first_degree,pgs.first_degree);
        cout << "I just performed a PGstudent ASSIGNMENT ... "
              << endl;
    }
    return *this; }

~PGStudent()
{   cout << "Deleting postgraduate student with name "
        << get_name() << endl;
    delete[] first_degree; }

int get_year() const
{   return gyear; }

void print() const
{   Student::print();
    cout << "First degree is: " << first_degree << endl; }
};

```

Δεν έχουμε να παρατηρήσουμε τίποτα περισσότερο από αυτά που αναλύσαμε προηγουμένως και ό,τι έχουμε συζητήσει διεξοδικά στα προηγούμενα. Αυτό που απομένει είναι να χρησιμοποιήσουμε τις κλάσεις μας, ώστε να εξετάσουμε τη συμπεριφορά τους. Η παρακάτω συνάρτηση `main` συνοψίζει πολλές από τις παρατηρήσεις που έχουμε κάνει. Αποσχολιάζοντας κατάλληλες γραμμές του κώδικά της, βεβαιωθείτε ότι κατανοείτε τη συμπεριφορά της.

```

int main()
{
    PGStudent pgs("FN1","FDEGR");

    PGStudent pgs2(pgs);

    pgs2.set_name("The New Name");
    pgs = pgs2;

    // Student* ps= new PGStudent("FN2","FDEGR2");
    // delete ps;

    cout << "Printing from Main:" << endl;
}

```

```

// Student s("FirstLast");

// s.print();
// pgs.print();
// pgs.Student::print();

////////////////////////////////////
// Student* ps=&s;
// PGStudent* ppgs=&pgs;

// ps->print();
// ppgs->print();
// ppgs->Student::print();

////////////////////////////////////
// Student* ss[2];

// ss[0] = &s;
// ss[1] = &pgs;

// ss[0]->print();
// ss[1]->print();

// ss[0] = new Student("TestedSt");
// ss[1] = new PGStudent("TestPSt","FDeg");

// delete ss[0];
// delete ss[1];

////////////////////////////////////
// Student* p1 = new Student("TestSt");
// Student* p2 = new PGStudent("TestPSt","FDeg");

// delete p1;
// delete p2;

return 0;
}

```

7.5 Σύνοψη Κεφαλαίου

- Μια έννοια (ή οντότητα) μπορεί να είναι εξειδίκευση μιας άλλης.
- Οι κλάσεις που αντιστοιχούν στις έννοιες/οντότητες αυτές συνδέονται μεταξύ τους μέσω κληρονομικότητας: η πιο ειδική κληρονομεί τα χαρακτηριστικά και τη συμπεριφορά από τη γενικότερη, προσδιορίζοντας μόνο την “ειδοποιό διαφορά” της ως προς αυτήν.

- Από την περιγραφή του προβλήματός μας βρίσκουμε κοινά στοιχεία στις έννοιες/οντότητες της περιγραφής και φτιάχνουμε νέες, πιο γενικευμένες, συνδέοντάς τις με αυτά, ώστε οι αρχικές να εκφραστούν ως εξειδικεύσεις τους.
- Πολλαπλή κληρονομικότητα εμφανίζεται στην περίπτωση όπου μια έννοια/οντότητα είναι άμεση εξειδίκευση παραπάνω από μιας έννοιας/οντότητας.
- Στον αντικειμενοστραφή προγραμματισμό εκφράζουμε τη συσχέτιση εξειδίκευσης μεταξύ κλάσεων μέσω της σχέσης *isa*. Μέσω της σχέσης αυτής νέες κλάσεις μπορούν να οριστούν από ήδη υπάρχουσες, ενσωματώνοντας τα χαρακτηριστικά και τη συμπεριφορά τους, ή επαναορίζοντάς τα όπου είναι απαραίτητο.
- Οι κλάσεις που αναπαριστάνουν τις γενικές έννοιες/οντότητες αποκαλούνται *υπερκλάσεις* ή *βασικές κλάσεις*. Οι κλάσεις που αναπαριστάνουν τις εξειδικεύσεις τους αποκαλούνται *υποκλάσεις* ή *παραγόμενες κλάσεις*.
- Στη C++ η κληρονομικότητα μέσω της σχέσης *isa* μεταξύ δύο κλάσεων, π.χ. B και A, δηλώνεται με την έκφραση

```
class B : public A
```

Τότε, B *isa* A. Κάθε αντικείμενο της κλάσης B μπορεί να θεωρηθεί και ως αντικείμενο της κλάσης A.

- Η χρήση του τελεστή επίλυσης εμβέλειας επιτρέπει να χρησιμοποιηθεί η εκδοχή ενός ονόματος που ορίζεται σε μια βασική κλάση, αντί για την επαναορισθείσα στην παραγόμενη.
- Ένα μέλος μιας κλάσης μπορεί να δηλωθεί ως *private*, *public* ή *protected*. Στην τελευταία περίπτωση μπορεί να χρησιμοποιηθεί και από την εμβέλεια των παραγόμενων κλάσεων της, αλλά μόνο για δικά τους αντικείμενα.
- Τα αντικείμενα μιας παραγόμενης κλάσης “κτίζονται” από όλα τα μέλη-δεδομένα των κλάσεων (του μονοπατιού) της ιεραρχίας πάνω από την κλάση.
- Έξω από την εμβέλεια μιας παραγόμενης κλάσης, μπορούμε να χειριστούμε τα αντικείμενά της χρησιμοποιώντας τα δημόσια ονόματα όλων των κλάσεων (του μονοπατιού) της ιεραρχίας πάνω από την κλάση.
- Κατά την αρχικοποίηση ενός αντικειμένου μιας παραγόμενης κλάσης, πρώτα αρχικοποιείται η “φύση του ως αντικείμενο της βασικής”. Πρώτα, δηλαδή, καλείται μια συνάρτηση αρχικοποίησης της βασικής κλάσης και μετά της δικής του. Ο προσδιορισμός της συνάρτησης αρχικοποίησης της βασικής κλάσης που θα χρησιμοποιηθεί γίνεται μέσω λίστας αρχικοποίησης στη συνάρτηση αρχικοποίησης της παραγόμενης.
- Κατά την καταστροφή ενός αντικειμένου η διαδικασία γίνεται αντίστροφα: πρώτα καλείται η συνάρτηση καταστροφής της κλάσης του αντικειμένου και μετά της βασικής της.
- Οι παραπάνω διαδικασίες για την αρχικοποίηση και την καταστροφή εφαρμόζονται αναδρομικά για όλες τις κλάσεις του μονοπατιού της ιεραρχίας πάνω από μια παραγόμενη κλάση.
- Σε περίπτωση που τουλάχιστον ένα μέλος-δεδομένο κλάσης ή/και υπερκλάσης είναι αντικείμενο κλάσης, έχουμε και κληρονομικότητα και σύνθεση. Τα δικαιώματα πρόσβασης και η αναπαράσταση καθορίζονται από την κάθε σχέση μεμονωμένα. Η αρχικοποίηση ενός αντικειμένου μιας παραγόμενης κλάσης γίνεται με την εξής σειρά: πρώτα καλούνται οι συναρτήσεις αρχικοποίησης των μελών-δεδομένων της βασικής κλάσης και στη συνέχεια καλείται η συνάρτηση αρχικοποίησης της βασικής κλάσης —

αυτή που έχει προσδιοριστεί από τη συνάρτηση αρχικοποίησης της παραγόμενης κλάσης —. Ακολουθεί η εκτέλεση των συναρτήσεων αρχικοποίησης των μελών-δεδομένων της παραγόμενης κλάσης και, τελευταία, καλείται η συνάρτηση αρχικοποίησης της παραγόμενης κλάσης. Κατά την καταστροφή ενός αντικείμενου παραγόμενης κλάσης η σειρά είναι η αντίστροφη. Τα παραπάνω ισχύουν για κάθε βάθος και ως προς τις δύο σχέσεις.

- Όταν δύο κλάσεις συνδέονται μέσω κληρονομικότητας, ένα αντικείμενο υποκλάσης είναι και αντικείμενο της υπερκλάσης του. Αντίθετα, στη σύνθεση κλάσεων ένα αντικείμενο της σύνθετης κλάσης έχει (ως μέλος του) ένα αντικείμενο της άλλης κλάσης.
- Από τη στιγμή που κάθε αντικείμενο υποκλάσης μπορεί να χρησιμοποιηθεί οποτεδήποτε χρειάζεται ένα αντικείμενο υπερκλάσης της, εάν η πρόσβαση στο αντικείμενο γίνεται μέσω δείκτη (ή αναφοράς) τύπου της υπερκλάσης, τίθεται το ερώτημα από ποια εμβέλεια προκύπτει ένα όνομα μέλους: της κλάσης του αντικείμενου ή της κλάσης του δείκτη (της αναφοράς); Αν πρόκειται για όνομα μέλους-δεδομένου, το όνομα προκύπτει από την εμβέλεια της κλάσης του δείκτη (της αναφοράς). Αν πρόκειται για όνομα συνάρτησης-μέλους, εξαρτάται από το εάν η συνάρτηση θεωρείται στην υπερκλάση ως εικονική ή όχι. Σε κάθε περίπτωση πρέπει να υπάρχει τέτοια συνάρτηση ορατή στην υπερκλάση. Αν δεν θεωρείται εικονική, ο ορισμός προκύπτει από την εμβέλεια της κλάσης του δείκτη (της αναφοράς). Αν θεωρείται εικονική, ο ορισμός προκύπτει από την εμβέλεια της κλάσης του αντικείμενου που “δείχνει” ο δείκτης (η αναφορά) και η απόφαση για το ποιος είναι αναβάλλεται για τη φάση της εκτέλεσης.
- Στη C++ μια συνάρτηση-μέλος προσδιορίζεται ως εικονική δίνοντας τον χαρακτηρισμό `virtual` κατά τη δήλωσή της, π.χ.

```
virtual void bla();
```

Από τη στιγμή που μια συνάρτηση δηλωθεί ως εικονική σε μία κλάση, παραμένει εικονική για όλη την ιεραρχία των κλάσεων που βρίσκονται κάτω από την κλάση αυτή.

- Δεν ορίζονται εικονικές συναρτήσεις αρχικοποίησης — δεν έχει νόημα. Είναι δυνατόν να χαρακτηριστούν ως εικονικές συναρτήσεις καταστροφής, προκειμένου να ακολουθείται πάντα η κατάλληλη διαδικασία κατά τη δυναμική αποδέσμευση.
- Επισημαίνουμε ότι οι συναρτήσεις-μέλη κληροδοτούνται στις θυγατρικές κλάσεις, ανεξάρτητα αν έχουν δηλωθεί ως `virtual` ή όχι στη γονεϊκή κλάση. Ο χαρακτηρισμός μιας συνάρτησης-μέλους ως `virtual` παίζει ρόλο μόνο στην περίπτωση που η συνάρτηση εφαρμόζεται σε ένα αντικείμενο μέσω δεικτών (ή αναφορών) τύπου της υπερκλάσης της κλάσης του αντικείμενου. Αν μια συνάρτηση-μέλος εφαρμόζεται άμεσα στο αντικείμενο, η απόφαση για το ποιος είναι ο ορισμός της παίρνεται σε φάση μεταγλώττισης και εξαρτάται αποκλειστικά από τον τύπο του αντικείμενου.
- Στη C++ επεκτείνοντας με `= 0` τη δήλωση μιας εικονικής συνάρτησης-μέλους, π.χ.

```
virtual void bla() = 0;
```

προσδιορίζουμε τη συνάρτηση ως *πλήρως εικονική*. Αν δηλώσουμε μια συνάρτηση-μέλος μιας κλάσης ως πλήρως εικονική, τότε η κλάση χαρακτηρίζεται ως *αφηρημένη κλάση*, οπότε δεν μπορούμε να δημιουργήσουμε αντικείμενά της.

- Αφηρημένες κλάσεις ορίζονται ώστε να προσφέρουν προδιαγραφές συμπεριφοράς ή/και γενικές υλοποιήσεις, τις οποίες οι υποκλάσεις τους για τις οποίες θέλουμε να ορίζονται αντικείμενα, πρέπει να έχουν υλοποιήσει ή επαυξήσει.

7.6 Ασκήσεις

7.6.1 Υλοποιήστε μια κλάση `Employee` και μια κλάση `Manager` ως εξειδίκευσή της που να αντιστοιχούν στις έννοιες του “Εργαζόμενου” και του “Μάνατζερ”, αντίστοιχα. Αποφασίστε εσείς κάποια λειτουργικότητα για την έννοια του “Εργαζόμενου”. Επαυξήστε την στην έννοια του “Μάνατζερ” με λειτουργικότητα που σχετίζεται με το πλήθος “Εργαζόμενων” για τους οποίους είναι υπεύθυνος. Στη συνέχεια, υλοποιήστε και μια ακόμα εκδοχή της έννοιας του “Μάνατζερ” στην οποία να συμπεριλαμβάνονται και οι εργαζόμενοι αυτοί καθαυτοί — όχι μόνο το πλήθος τους. Υπόδειξη: για την εκδοχή αυτή, χρησιμοποιείτε δείκτες.

7.6.2 Έστω ότι έχουμε το παρακάτω C++ πρόγραμμα:

```
#include <iostream>
using namespace std;

class A {
    int data;
public:
    A(int dat = 0) : data(dat)
    { cout << "Constructing an A" << endl; }

    VVVV void print()
    { cout << "The A data is: " << data << endl; } };

class B : public A {
    int data;
public:
    B(int dat = 0) : data(dat)
    { cout << "Constructing a B" << endl; }

    void print()
    { A::print(); cout << "The B data is: " << data << endl; } };

int main()
{
    A a(10); B b(5); A* pa = &a; B* pb = &b;
    a.print(); b.print(); pa->print(); pb->print();
    pa = pb; pa->print(); a = b; a.print();
    return 0;
}
```

Ποιο είναι το αποτέλεσμα της εκτέλεσής του στην περίπτωση που αντικαταστήσουμε το `VVVV` με το κενό και ποιο στην περίπτωση που το αντικαταστήσουμε με τη λέξη `virtual`; Αιτιολογήστε την απάντησή σας.

7.6.3 Αιτιολογώντας την απάντησή σας, δώστε το αποτέλεσμα της εκτέλεσης του παρακάτω προγράμματος:

```

#include <iostream>
using namespace std;

class Mathima {
    int ECTS;
    int grade;
public:
    Mathima(int e = 6) : ECTS(e)
    { cout << "A Mathima was just constructed with: "
      << ECTS << " ECTS" << endl; }

    /*
    Mathima(const Mathima& m) : ECTS(m.ECTS), grade(m.grade)
    { cout << "A Mathima was just constructed with CP! " << endl; }
    */

    virtual bool pass()
    { if (grade > 4) {
      cout << "Success" << endl;
      return true;
    }
    else {
      cout << "Fail" << endl;
      return false; } }

    void take_exam(int gr) { grade = gr; } };

class Kormos : public Mathima {
public:
    Kormos(int e = 8) : Mathima(e)
    { cout << "A Kormos was just constructed " << endl; }

    bool pass()
    { if (Mathima::pass()) {
      cout << "One of the must-tasks is over! " << endl;
      return true;
    }
    else {
      cout << "Don't give up! " << endl;
      return false; } } };

void Exam(Mathima& object) { object.pass(); }

int main()
{
    Kormos k1; Kormos k2(8); Kormos k3(k1);
    k1.take_exam(3); k2.take_exam(8); k3.take_exam(10);
    Exam(k1); Exam(k2); Exam(k3);
}

```

```

    return 0;
}

```

Κατόπιν, αποσχολιάστε τον σχολιασμένο κώδικα και επαναλάβετε.

7.6.4 Δίνεται το παρακάτω πρόγραμμα C++:

```

#include <iostream>
using namespace std;

class A {
    void set_data3(int i)
    { data3 = i; }

    int data1;
protected:
    void set_data1(int i)
    { data1 = i; }

    int data2;
public:
    A(int i = 0) : data1(i), data2(i), data3(i) {}

    ~A()
    { cout << "Destroying an A with data " << data1 << " "
      << data2 << " " << data3 << endl; }

    int data3;

    void set_data2(int i)
    { data2 = i; } };

class B : public A {
    int data1;
public:
    B(int i = 0) : A(i), data1(i) {}

    ~B()
    { cout << "Destroying a B with data " << data1 << endl; }

    void set_data1(int i)
    { data1 = i; }

    void process(int i)
    { DATA = i; SETDATA(i); } };

int main()
{
    B b; b.process(500);
}

```

```

    return 0;
}

```

Με ποια από τα ονόματα των μελών-δεδομένων των παραπάνω κλάσεων είναι επιτρεπτό να αντικατασταθεί η λέξη DATA; Ποιες είναι οι επιτρεπτές κλήσεις συναρτήσεων-μελών που μπορούν να αντικαταστήσουν την έκφραση SETDATA(i); Για κάθε συνδυασμό τέτοιων αντικαταστάσεων, αιτιολογώντας την απάντησή σας, δώστε το αποτέλεσμα της εκτέλεσης του προγράμματος.

7.6.5 Έστω ότι έχουμε το παρακάτω C++ πρόγραμμα:

```

class P {
    void priv() {}
protected:
    void prot() {}
public:
    void pub() {} };

class C : public P {
    P p;
    C* pc;
public:
    void mem()
    { XX(); p.YY(); pc -> ZZ(); } };

int main()
{
    C c; c.WW();
    return 0;
}

```

Με ποια από τα ονόματα των συναρτήσεων priv, prot και pub είναι επιτρεπτό να αντικαταστήσουμε τα XX, YY, ZZ και WW; Τι έχετε να παρατηρήσετε σε σχέση με τις επιτρεπτές τιμές των XX και ZZ; Πώς το αιτιολογείτε;

7.6.6 Έστω ότι έχουμε τις παρακάτω C++ κλάσεις:

```

class C1 {
    int data;
public:
    XXX C1(int dat = 0) : data(dat) {}

    YYY ~C1()
    { cout << "Deleting a C1" << endl; }

    ZZZ void print()
    { cout << data << endl; } };

class C2 : public C1 {
    int* data;
public:

```

```

WWW C2(int dat = 0)
{   if (dat == 0)
        data = NULL;
    else
        data = new int(dat); }

VVV ~C2()
{   cout << "Deleting C2 in all cases" << endl; delete data; }

UUU void print()
{   if (data != NULL)
        cout << *data << endl; } };

```

Αν υποθέσουμε ότι τα XXX, YYY, ZZZ, WWW, VVV και UUU μπορούν να αντικατασταθούν είτε με το κενό είτε με τη λέξη `virtual`, ποιες αντικαταστάσεις δεν μπορούν να γίνουν και ποιες ενδείκνυται να γίνουν; Αιτιολογήστε την απάντησή σας.

7.6.7 Έστω ότι έχουμε το παρακάτω C++ πρόγραμμα:

```

#include <iostream>
using namespace std;

class A {
public:
    A()
    {   cout << "Constructing an A" << endl; }

    A(A& a)
    {   cout << "Copy Constructing an A" << endl; } };

class B : public A {
public:
    B()
    {   cout << "Constructing a B" << endl; }

    B(B& b) : A(b)
    {   cout << "Copy Constructing a B" << endl; } };

void bla(XXX data)
{   cout << "OK" << endl; }

int main()
{
    YYY object;
    bla(object);
    return 0;
}

```

Αν οι ενδεχόμενες τιμές για το XXX είναι A, B, A&, B&, A* και B*, ποιες είναι οι επιτρεπτές τιμές για το YYY στην κάθε περίπτωση και γιατί; Τι εκτυπώνεται κατά την εκτέλεση του κάθε επιτρεπτού συνδυασμού

και γιατί;

7.6.8 Δίνεται το παρακάτω πρόγραμμα C++:

```
#include <iostream>
using namespace std;

class A {
    int data;
public:
    A(int i = 10) : data(i)
    { cout << "Constructing an A with " << data << endl; }

    ~A()
    { cout << "Deleting an A with " << data << endl; }

    // virtual void bla()
    // { cout << "This is A::bla" << endl; data++; }
    // void bla()
    // { cout << "This is A::bla" << endl; data++; }
    // virtual void bloo()
    // { cout << "This is A::bloo()" << endl; bla(); }
    // void bloo()
    // { cout << "This is A::bloo()" << endl; bla(); }
};

class B : public A {
    int data;
public:
    B(int i = 100) : data(i)
    { cout << "Constructing a B with " << data << endl; }

    ~B()
    { cout << "Deleting a B with " << data << endl; }

    void bla()
    { cout << "This is B::bla" << endl; data++; }

    void bloo()
    { cout << "This is B::bloo()" << endl; bla(); } };

int main()
{
    A a; B b; A* pa;
    pa = &b;
    pa->bla(); pa->bloo();
    return 0;
}
```

Αποσχοιάζοντας δύο από τις παραπάνω σχολιασμένες συναρτήσεις τη φορά ώστε το πρόγραμμα να

μεταγλωττίζεται, δώστε το αποτέλεσμα της εκτέλεσης, αιτιολογώντας την απάντησή σας. Αυτό να γίνει για όλους τους δυνατούς συνδυασμούς αποσχολισμού.

7.6.9 Έστω ότι έχουμε το παρακάτω C++ πρόγραμμα:

```
#include <iostream>
using namespace std;

class Athlete {
protected:
    int speed;
public:
    Athlete(int spd = 0) : speed(spd) {}

    virtual void run() = 0;

    void hit()
    { cout << "I was hit" << endl; stop(); }

    virtual void stop()
    { if (speed != 0) {
        speed = 0; cout << "I just stopped";
    } } };

class StrongAthlete : public Athlete {
public:
    StrongAthlete(int spd) : Athlete(spd) {}

    void run()
    { speed = speed + 30; cout << " I run fast" << endl; }

    void hit()
    { cout << "Nothing happens" << endl; } };

class HeavyAthlete : public StrongAthlete {
public:
    HeavyAthlete(int spd) : StrongAthlete(spd) {}

    void run()
    { speed = speed + 10; cout << " I run not so fast" << endl; }

    void stop()
    { if (speed != 0)
        speed = speed - 10;
        cout << "I cannot stop" << endl; } };

class DelicateAthlete : public Athlete {
public:
    DelicateAthlete(int spd = 0) : Athlete(spd) {}
```

```

    void run()
    {   speed = speed + 20; cout << " I am agile" << endl; } };

void train(AAAA what)
{   what.run(); what.hit(); }

int main()
{
    BBBB what(10); train(what);
    return 0;
}

```

Με τι μπορείτε να αντικαταστήσετε τα AAAA και BBBB; Ποιο είναι το αποτέλεσμα της εκτέλεσης για τον κάθε συνδυασμό αντικαταστάσεων; Αιτιολογήστε την απάντησή σας (Σημείωση: όπου είναι απαραίτητο, στη συνάρτηση train, αντικαταστήστε το . με -. Επίσης, όπου είναι απαραίτητο, στη συνάρτηση main, αντικαταστήστε το what με &what.)

7.6.10 Δίνονται οι παρακάτω C++ κλάσεις:

```

class Top {
    int data;
public:
    Top(int i = 0) : data(i)
    {   cout << "Creating a Top with data:" << data << endl; }

    ~Top()
    {   cout << "Destroying a Top with data:" << data << endl; }

    int find()
    {   cout << "This Top data is:" << data << endl;
        return data; }

    virtual void change(int i)
    {   data = data + i; }

    virtual void tricky() = 0; };

class Middle1 : public Top {
    int data;
public:
    Middle1(int i = 10) : data(i)
    {   cout << "Creating a Middle1 with data:" << data << endl; }

    ~Middle1()
    {   cout << "Destroying a Middle1 with data:" << data << endl; }

    virtual int find()
    {   Top::find();
        cout << "This Middle1 data is:" << data << endl;
        return data; }
}

```



```
void change(int i)
{ data = data - i; } };

class Middle2 : public Top {
    int data;
public:
    Middle2(int i = 20) : data(i)
    { cout << "Creating a Middle2 with data:" << data << endl; }

    ~Middle2()
    { cout << "Destroying a Middle2 with data:" << data << endl; }

    virtual void tricky()
    { change(find()); } };

class Bottom1 : public Middle1 {
    int data;
public:
    Bottom1(int i = 100) : data(i)
    { cout << "Creating a Bottom1 with data:" << data << endl; }

    ~Bottom1()
    { cout << "Destroying a Bottom1 with data:" << data << endl; }

    virtual int find()
    { Middle1::find();
      cout << "This Bottom1 data is:" << data << endl;
      return data; }

    virtual void tricky()
    { change(find()); } };

class Bottom2 : public Middle2 {
    int data;
public:
    Bottom2(int i = 200) : data(i)
    { cout << "Creating a Bottom2 with data:" << data << endl; }

    ~Bottom2()
    { cout << "Destroying a Bottom2 with data:"
      << data << endl; } };
```

Με τι επιτρέπεται να αντικαταστήσετε το CLASS1 στην παρακάτω συνάρτηση main και ποιο είναι το αποτέλεσμα της κάθε εκτέλεσης; Αιτιολογήστε τις απαντήσεις σας.

```
int main()
{
    CLASS1 object;
    object.find();
}
```

```

    object.change(500);
    object.tricky();
    return 0;
}

```

Αν η συνάρτηση main ήταν η εξής:

```

int main()
{
    CLASS1 object; CLASS2* pobject = &object;
    pobject->find();
    pobject->change(500);
    pobject->tricky();
    return 0;
}

```

με βάση τις επιτρεπτές αντικαταστάσεις του CLASS1, με ποια από τα Top, Middle1, Bottom1 μπορούμε να αντικαταστήσουμε το CLASS2; Συνδυάζοντας τις αντικαταστάσεις αυτές, δώστε το αποτέλεσμα της εκτέλεσης, αιτιολογώντας την απάντησή σας.

7.6.11 Δίνεται η παρακάτω ιεραρχία κλάσεων:

```

class Super {
public:
    P1 Super()
    { cout << "Constructing a Super" << endl; }

    P2 ~Super()
    { cout << "Destructing a Super" << endl; }

    void Fun()
    { cout << "Calling Super::Fun" << endl; }

    virtual void Done() = 0; };

class Middle : public Super {
public:
    P3 Middle()
    { cout << "Constructing a Middle" << endl; }

    P4 ~Middle()
    { cout << "Destructing a Middle" << endl; }

    void Fun()
    { cout << "Calling Middle::Fun" << endl; } };

class Bottom : public Middle {
public:
    Bottom()
    { cout << "Constructing a Bottom" << endl; }
}

```

```

~Bottom()
{ cout << "Destructing a Bottom" << endl; }

void Done()
{ cout << "Calling Bottom::Done" << endl; } };

```

Με ποιο από τη λέξη `virtual` ή το κενό μπορούμε να αντικαταστήσουμε καθένα από τα P1, P2, P3 και P4 και γιατί;

Δίνεται και η παρακάτω συνάρτηση `main`:

```

int main()
{ C inst;
  inst.Fun();
  inst.Done();
  return 0;
}

```

Τι τιμές μπορεί να πάρει το C; Αιτιολογώντας την απάντησή σας, δώστε το αποτέλεσμα της εκτέλεσης της `main`, αντικαθιστώντας με όλες τις επιτρεπτές τιμές τα C, P1, P2, P3 και P4.

Αν η συνάρτηση `main` έχει την παρακάτω μορφή:

```

int main()
{ A* pinst = new B;
  pinst->Fun();
  pinst->Done();
  delete pinst;
  return 0;
}

```

Τι τιμές μπορούν να πάρουν τα A και B; Αιτιολογώντας την απάντησή σας, δώστε το αποτέλεσμα της εκτέλεσης της `main` αντικαθιστώντας με όλες τις επιτρεπτές τιμές τα A, B, P1, P2, P3 και P4.

7.6.12 Δίνεται το παρακάτω πρόγραμμα C++:

```

#include <iostream>
using namespace std;

class A {
    int var;
public:
    A(int i = 10) : var(i)
    { cout << "Constructing an A with " << var << endl; }

    ~A()
    { cout << "An A will be destroyed with " << var << endl; }

    void bla()
    { cout << "I am A::bla() " << endl; var++; }
}

```

```

    void bloo()
    {   cout << "I am A::blo()" << endl; bla(); } };

class B : public A {
    int var;
public:
    B(int i = 100) : var(i)
    {   cout << "Constructing a B with " << var << endl; }

    ~B()
    {   cout << "A B will be destroyed with " << var << endl; }

    void bla()
    {   cout << "I am B::bla() " << endl; var++; } };

class C : public A {
    int var;
public:
    C(int i = 200) : var(i)
    {   cout << "Constructing a C with " << var << endl; }

    ~C()
    {   cout << "A C will be destroyed with " << var << endl; }

    virtual void bla()
    {   cout << "I am C::bla() " << endl; var++; } };

class D : public B {
    int var;
public:
    D(int i = 1000) : var(i)
    {   cout << "Constructing a D with " << var << endl; }

    ~D()
    {   cout << "A D will be destroyed with " << var << endl; } };

class E : public C {
    int var;
public:
    E(int i = 2000) : var(i)
    {   cout << "Constructing an E with " << var << endl; }

    ~E()
    {   cout << "An E will be destroyed with " << var << endl; }

    void bla()
    {   cout << "I am E::bla() " << endl; var++; } };

class F : public C {

```

```

    int var;
public:
    F(int i = 2222) : var(i)
    { cout << "Constructing a F with " << var << endl; }

    ~F()
    { cout << "A F will be destroyed with " << var << endl; } };

int main()
{
    A* pa; B* pb; C* pc; A a; D d; E e; F f;
    a.bla(); a.bloo(); d.bla(); d.bloo();
    e.bla(); e.bloo(); f.bla(); f.bloo();

    pa=&a; pa->bla(); pa->bloo();
    pa=&d; pa->bla(); pa->bloo();
    pa=&e; pa->bla(); pa->bloo();
    pa=&f; pa->bla(); pa->bloo();

    pb=&a; pb->bla(); pb->bloo();
    pb=&d; pb->bla(); pb->bloo();

    pc=&d; pc->bla(); pc->bloo();
    pc=&e; pc->bla(); pc->bloo();
    pc=&f; pc->bla(); pc->bloo();

    return 0;
}

```

Το πρόγραμμα αυτό δεν μεταγλωττίζεται. Εξηγήστε γιατί. Χωρίς να αλλάξετε τους ορισμούς των κλάσεων, σχολιάστε τις ελάχιστες γραμμές ώστε αυτό να μεταγλωττίζεται. Στη συνέχεια, αιτιολογώντας την απάντησή σας, δώστε το αποτέλεσμα της εκτέλεσής του.

7.6.13 Δίνεται το παρακάτω πρόγραμμα C++:

```

#include <iostream>
using namespace std;

class A {
protected:
    int data;
public:
    A(int i = 0) : data(i)
    { cout << "Constructing an A with " << i << endl; }

    ~A()
    { cout << "Destructing an A with " << data << endl; }

    virtual void bla()
    { cout << "Calling A::bla" << endl; data++; }
}

```

```

    void bloo()
    { cout << "Calling A::bloo" << endl; }

    void fun()
    { cout << "Calling A::fun" << endl; data++; } };

class B : public A {
    int data;
public:
    B(int i = 100) : data(i)
    { cout << "Constructing a B with " << i << endl; }

    ~B()
    { cout << "Destructing a B with" << data << endl; }

    void bla()
    { cout << "Calling B::bla" << endl; data++; }

    void bloo()
    { cout << "Calling B::bloo " << endl; A::data++; } };

// void foo(A param) {
// void foo(A& param) {
// void foo(B param) {
// void foo(B& param) {
    param.bla(); param.bloo(); param.fun(); }

int main()
{
// A data; foo(data);
// B data; foo(data);
    return 0;
}

```

Αποσχοιάζοντας μια από τις δύο σχολιασμένες γραμμές στο σώμα της `main` τη φορά, ποια/ες γραμμές μπορούν να αποσχολιαστούν από τη συνάρτηση `foo` για να μπορεί να εκτελεστεί το πρόγραμμα; Αιτιολογώντας την απάντησή σας, δώστε το αποτέλεσμα της εκτέλεσης.

7.6.14 Δίνεται η παρακάτω ιεραρχία κλάσεων:

```

class Student {
protected:
    bool _attend;
    bool _practice;
    bool _study;
public:
    virtual void attend()
    { _attend = true;
      cout << "I am a student who attends" << endl; }

```

```
void practice()
{ _practice = true;
  cout << "I am a student who practices" << endl; }

virtual void study()
{ _study = true;
  cout << "I am a student who studies" << endl; }

void graduate()
{ if (_attend == true && _practice == true && _study == true)
  cout << "I am student who will graduate" << endl;
  else
    cout << "I am student in a very bad position" << endl; } };

class Undergraduate : public Student {
public:
  void attend()
  { _attend = true;
    cout << "I am an undergrad who attends" << endl; }

  virtual void practice()
  { _practice = true;
    cout << "I am an undergrad who practices" << endl; }

  void study()
  { _study = true;
    cout << "I am an undergrad who studies" << endl; } };

class FirstYearStudent : public Undergraduate {
public:
  void attend()
  { _attend = true;
    cout << "I am a freshstudent who attends" << endl; }

  void practice()
  { _practice = true;
    cout << "I am a freshstudent who practices" << endl; }

  void study()
  { _study = true;
    cout << "I am a freshstudent who studies" << endl; }

  void graduate()
  { cout << "I am an optimistic freshstudent" << endl; } };
```

Δίνονται και οι συναρτήσεις:

```
void bla(Student* S)
{ S->attend();
```

```

        S->study();
        S->practice();
        S->graduate();
    }

    void bloo(Undergraduate* S)
    {
        S->attend();
        S->study();
        S->practice();
        S->graduate();
    }

    void blee(FirstYearStudent* S)
    {
        S->attend();
        S->study();
        S->practice();
        S->graduate();
    }

    int main()
    {
        FirstYearStudent s1;
        bla(&s1); bloo(&s1); blee(&s1);
        Undergraduate s2;
        bla(&s2); bloo(&s2);
        Student* ps1 = new Student;
        bla(ps1);
        Student* ps2 = new Undergraduate;
        bla(ps2);
        Student* ps3 = new FirstYearStudent;
        bla(ps3);
        return 0;
    }

```

Αιτιολογώντας την απάντησή σας, δώστε το αποτέλεσμα της εκτέλεσης της main.

7.6.15 Δίνεται το παρακάτω πρόγραμμα C++:

```

#include <iostream>
using namespace std;

class Food {
public:
    Food()
    { cout << "Constructing food" << endl; }

    void prepare()
    { cout << "I am preparing food" << endl; }

    virtual void evaluate() = 0; };

```



```
class Starter : public Food {
public:
    Starter()
    { cout << "Constructing starter" << endl; }

    virtual void evaluate() = 0; };

class Soup : public Starter {
public:
    Soup()
    { cout << "Constructing soup" << endl; }

    virtual void evaluate()
    { cout << "I am evaluating a soup" << endl; } };

class MainCourse : public Food {
public:
    MainCourse()
    { cout << "Constructing main course" << endl; }

    void prepare()
    { cout << "I am preparing a main course" << endl; }

    virtual void evaluate() = 0; };

class Meat : public MainCourse {
public:
    Meat()
    { cout << "Constructing meat" << endl; }

    void prepare()
    { cout << "I am preparing meat" << endl; }

    virtual void evaluate()
    { cout << "I am evaluating meat" << endl; } };

class Fish : public MainCourse {
public:
    Fish()
    { cout << "Constructing fish" << endl; }

    virtual void evaluate()
    { cout << "I am evaluating fish" << endl; } };

class Dessert : public Food {
public:
    Dessert()
    { cout << "Constructing dessert" << endl; }
```

```

    void prepare()
    { cout<< "I am preparing dessert" << endl; }

    virtual void evaluate()
    { cout << "I am evaluating dessert" << endl; } };

class IceCream : public Dessert {
public:
    IceCream()
    { cout << "Constructing ice cream" << endl; }

    void prepare()
    { cout << "I am preparing ice cream" << endl; }

    virtual void evaluate()
    { cout << "I am evaluating ice cream" << endl; } };

class Menu {
    Starter* starter;
    MainCourse& maincourse;
    Dessert* dessert;
public:
    Menu(MainCourse& mc, Starter* ps = 0, Dessert* pd = 0)
        : maincourse(mc)
    { starter = ps; dessert = pd; }

    void eat()
    { maincourse.prepare(); maincourse.evaluate();
      if (starter != 0) {
          starter->prepare(); starter->evaluate();
      }
      if (dessert != 0) {
          dessert->prepare(); dessert->evaluate();
      } } };

int main()
{
    /* // Part A
       Menu m1(maincourse);
       m1.eat();          */

    /* // Part B
       Menu m2(maincourse, &firstcourse);
       m2.eat();          */

    /* // Part C
       Menu m3(maincourse, &firstcourse, &thirdcourse);
       m3.eat();          */

```

```

    return 0;
}

```

Αφαιρώντας τα σχόλια `/* */`, ένα κάθε φορά, στο σώμα της `main`, συμπληρώστε τα Part A, Part B και Part C για να μπορεί να εκτελεστεί η `main`. Αιτιολογώντας την απάντησή σας, δώστε το αποτέλεσμα της εκτέλεσης.

7.6.16 Δίνεται το παρακάτω πρόγραμμα C++:

```

#include <iostream>
using namespace std;

class Place {
public:
    Place()
    { cout << "Constructing a Place" << endl; }

    ~Place()
    { cout << "Destructing a Place" << endl; }

    void enter()
    { cout << "Arriving..." << endl; }

    virtual void exit()
    { cout << "Departing..." << endl; }

    void visit()
    { enter(); exit(); } };

class Public : public Place {
public:
    void exit()
    { cout << "Exiting a crowded place" << endl; } };

class Port : public Public {
public:
    Port()
    { cout << "Constructing a Port" << endl; }

    ~Port()
    { cout << "Destructing a Port" << endl; } };

class Taverna : public Public {
public:
    Taverna()
    { cout << "Constructing a Taverna" << endl; }

    ~Taverna()
    { cout << "Destructing a Taverna" << endl; }
}

```

```

void enter()
{ Public::enter(); cout << "Nice taverna..." << endl; }

void exit()
{ cout << "I forgot to pay..." << endl; }

void eat()
{ enter(); cout << "Nice food..." << endl; exit(); } };

class Beach : public Public {
public:
    Beach()
    { cout << "Constructing a Beach" << endl; }

    ~Beach()
    { cout << "Destructing a Beach" << endl; }

    void swim()
    { enter(); cout << "Swimming..." << endl; exit(); } };

class Private : public Place {
public:
    void enter()
    { cout << "Quiet Place!" << endl; } };

class Hotel : public Private {
public:
    Hotel()
    { cout << "Constructing a Hotel" << endl; }

    ~Hotel()
    { cout << "Destructing a Hotel" << endl; }

    void sleep()
    { enter(); cout << "Sleeping..." << endl; exit(); } };

class HolidayIsland {
    Port p;
    Taverna& t;
    Beach b;
    Hotel* h;
public:
    HolidayIsland(Taverna& t1, Hotel* h1) : h(h1), t(t1) {}

    void arrive()
    { p.enter(); cout << "Just arrived" << endl; }

    void depart()
    { p.exit(); cout << "Just departed" << endl; }

```

```

void weekend()
{ arrive(); b.swim(); t.eat(); h->sleep(); depart(); }

void visit()
{ Place* place[] = {&p, &t, &b, h};
  for(int i = 0; i < 4; i++)
    place[i]->visit(); } };

int main()
{
    Taverna t1; Hotel h1;
    HolidayIsland isle1(t1,&h1);
    // isle1.weekend();
    // isle1.visit();
    return 0;
}

```

Αποσχολιάζοντας μια γραμμή κάθε φορά στο σώμα της main, δώστε το αποτέλεσμα της εκτέλεσης, αιτιολογώντας την απάντησή σας.

- 7.6.17 Δίνεται το παρακάτω πρόγραμμα C++. Αιτιολογώντας την απάντησή σας, δώστε το αποτέλεσμα της εκτέλεσής του, αποσχολιάζοντας μια γραμμή κάθε φορά στον ορισμό της κλάσης Musical.

```

#include <iostream>
using namespace std;

class Actor {
protected:
    int strength;
public:
    Actor(int s = 10) : strength(s)
    { cout << "Creating an Actor" << endl; }

    Actor(const Actor& actor)
    { cout << "Creating a Copy of an Actor" << endl;
      strength = actor.strength; }

    virtual ~Actor()
    { cout << "Disappearing ..." << strength << endl; }

    int get_strength()
    { return strength; }

    void tiring()
    { if (strength > 0)
        strength--; }

    virtual void act()
    { cout << "words, words, words ..." << endl; tiring(); } };

```

```

class ActorDancer : public Actor {
    int strength;
public:
    ActorDancer(int as = 5, int s = 100) : Actor(as), strength(s)
    { cout << "Creating an Actor who Dances" << endl; }

    virtual ~ActorDancer()
    { if (get_strength() > 80)
        cout << "Exiting but no problem at all" << endl;
        else {
            if (get_strength() > 50)
                cout << "Exiting but still keeping" << endl;
            else
                cout << "Exiting exhausted" << endl; }
        cout << strength << endl; }

    int get_strength()
    { return Actor::get_strength() + strength; }

    void tiring()
    { cout << strength << endl;
        if (strength > 20)
            strength -= 20;
        cout << strength << endl; }

    virtual void act()
    { cout << "I am dancing in the sun!" << endl; tiring(); } };

class ActorSingerDancer : public ActorDancer {
public:
    ActorSingerDancer(int astrength = 40, int dstrength = 50) :
        ActorDancer(astrength, dstrength)
    { cout << "Creating an Actor who Sings and Dances" << endl; }

    virtual void act()
    { ActorDancer::act(); cout << "I am singing !" << endl;
        tiring(); Actor::tiring(); } };

class Musical {
    ActorSingerDancer& starring;
    ActorDancer& dancer1; ActorDancer& dancer2; ActorDancer& dancer3;
    Actor& actor1; Actor& actor2; Actor& actor3;
public:
    Musical(ActorSingerDancer& s, ActorDancer& d1, ActorDancer& d2,
        ActorDancer& d3, Actor& a1, Actor& a2, Actor& a3)
        : starring(s), dancer1(d1), dancer2(d2), dancer3(d3),
          actor1(a1), actor2(a2), actor3(a3)
    { cout << "The show starts!" << endl;

```

```

        scene(s,d1,d2); scene(d1,d2,d3); scene(a1,a2,a3); }

~Musical()
{   cout << "The End" << endl; }

// void scene(Actor a1, Actor a2, Actor a3)
// void scene(Actor& a1, Actor& a2, Actor& a3)
{   a1.act(); a2.act(); a3.act(); } };

int main()
{
    ActorSingerDancer MS; Actor PB, SS, CF;
    ActorDancer AD1, AD2, AD3;
    Musical mamma_mia(MS, AD1, AD2, AD3, PB, SS, CF);
    return 0;
}

```

7.6.18 Αιτιολογώντας την απάντησή σας, δώστε το αποτέλεσμα της εκτέλεσης του εξής προγράμματος C++:

```

#include <iostream>
using namespace std;

class drink {
protected:
    int value;
public:
    drink(int val = 5) : value(val)
    {   cout << "offering a drink" << endl; }

    virtual void serve() = 0;

    virtual int get_value()
    {   return value; } };

class hot : public drink {
public:
    hot()
    {   cout << " a hot one " << endl; }

    void serve()
    {   cout << "Careful it's hot! " << "Please pay "
        << get_value() << " euros " << endl; } };

class milk : public hot {
    int value;
public:
    milk(int val = 3) : value(val)
    {   cout << " milk!! " << endl; }
}

```

```

    int get_value()
    { return value; }

    void serve()
    { value += drink::value; hot::serve(); } };

class chocolate : public hot {
public:
    chocolate()
    { cout << " a chocolate! " << endl; } };

class cold : public drink {
public:
    cold()
    { cout << " a cold one " << endl; }

    void serve()
    { cout << "Please pay " << get_value()
      << " euros " << endl; } };

class refreshment : public cold {
public:
    refreshment()
    { cout << " a refreshment!" << endl; } };

class cocktail : public cold {
    int value;
public:
    cocktail(int val = 5) : value(val)
    { cout << " a cocktail!" << endl; }

    int get_value()
    { return value; }

    void serve()
    { value += drink::value; cold::serve(); } };

class bar {
    drink** drks;
public:
    bar()
    { drks = new drink*[6];
      drks[0] = new hot(); drks[1] = new cold();
      drks[2] = new milk(); drks[3] = new chocolate();
      drks[4] = new refreshment(); drks[5] = new cocktail(); }

    void serve()
    { for (int i = 0; i < 6; i++)
      drks[i]->serve(); } };

```



```

int main()
{
    bar br; br.serve();
    return 0;
}

```

7.6.19 Δίνεται το παρακάτω πρόγραμμα C++:

```

#include <iostream>
using namespace std;

class Book {
protected:
    const int pages;
    int time_per_page;
public:
    Book(int p = 100, int tpp = 10) : pages(p), time_per_page(tpp)
    { cout << "New Book with: " << pages << " and "
      << time_per_page << endl; }

    ~Book()
    { cout << " Book to be destroyed! " << endl; }

    // int get_time_needed()
    // { return pages * time_per_page; }
    // virtual int get_time_needed()
    // { return pages * time_per_page; }
    // virtual int modify_fatigue(int init_fatig)
    // { return init_fatig *= 2; }
    // int modify_fatigue(int init_fatig)
    // { return init_fatig *= 2; }
    virtual void announcement() = 0; };

class Novel : public Book {
public:
    Novel(int p) : Book(p)
    { cout << " New Novel just created! " << endl; }

    ~Novel()
    { cout << " Novel to be destroyed! " << endl; }

    int modify_fatigue(int init_fatig)
    { return init_fatig; }

    void announcement()
    { cout << " I am reading a Novel!" << endl; } };

class Science : public Book {

```

```

public:
    Science(int p) : Book(p)
    { cout << " New Science just created! " << endl; }

    ~Science()
    { cout << " Science to be destroyed! " << endl; }

    int get_time_needed()
    { time_per_page *= 10; return Book::get_time_needed(); }

    int modify_fatigue(int init_fatig)
    { return init_fatig *= 10; }

    void announcement()
    { cout << " I am reading Science!" << endl; } };

class Cartoon : public Book {
public:
    Cartoon(int p) : Book(p)
    { cout << " New Cartoon just created! " << endl; }

    ~Cartoon()
    { cout << " Cartoon to be destroyed! " << endl; }

    int get_time_needed()
    { time_per_page /= 2; return Book::get_time_needed(); }

    int modify_fatigue(int init_fatig)
    { return init_fatig /= 5; }

    void announcement()
    { cout << " What a funny Cartoon!" << endl; } };

class Reader {
    int available_time;
    int fatigue;
    Book& book1;
    Book& book2;
    Book& book3;
    void read(Book& book)
    { book.announcement();
      available_time -= book.get_time_needed();
      fatigue = book.modify_fatigue(fatigue); }
public:
    Reader(int at, int f, Book& b1, Book& b2, Book& b3)
      : available_time(at), fatigue(f),
        book1(b1), book2(b2), book3(b3)
    { cout << " A New Reader has been created! " << endl; }

```

```

~Reader()
{ cout << " A Reader to be destroyed with! " << available_time
  << " and " << fatigue << endl; }

void read()
{ read(book1);
  if (available_time >= fatigue) {
    read(book2);
    if (available_time >= fatigue) {
      read(book3);
      cout << " I finished reading! " << endl;
    }
    else
      cout << " I give up!" << endl; }
  else
    cout << "Too many books!" << endl; } };

int main() {
  Novel the_great_gatsby(20);
  Science calculus(50);
  Cartoon asterix(100);
  Reader r1(1000,600,the_great_gatsby,calculus,asterix);
  Reader r2(200000,50,calculus,asterix,the_great_gatsby);
  r1.read(); r2.read();
  return 0;
}

```

Αποσχολιάστε σχολιασμένες γραμμές με τρόπο ώστε το πρόγραμμα να μεταγλωττίζεται και, αιτιολογώντας την απάντησή σας, δώστε το αποτέλεσμα της εκτέλεσής του, κάθε φορά.

7.6.20 (Παραλλαγή της προηγούμενης άσκησης) Δίνεται το παρακάτω πρόγραμμα C++:

```

#include <iostream>
using namespace std;

class Book {
protected:
    const int pages;
    int time_per_page;
public:
    Book(int p = 100, int tpp = 10) : pages(p), time_per_page(tpp)
    { cout << "New Book with: " << pages << " and "
      << time_per_page << endl; }

    ~Book()
    { cout << " Book to be destroyed! " << endl; }

    // int get_time_needed()
    // { return pages * time_per_page; }

```

```

// virtual int get_time_needed()
// { return pages * time_per_page; }
// int modify_fatigue(int init_fatig)
// { return init_fatig *= 2; }
// virtual int modify_fatigue(int init_fatig)
// { return init_fatig *= 2; }

void announcement()
{ cout << "I am reading a Book" << endl; } };

class Novel : public Book {
public:
    Novel(int p) : Book(p)
    { cout << " New Novel just created! " << endl; }

    ~Novel()
    { cout << " Novel to be destroyed! " << endl; } };

class Science : public Book {
public:
    Science(int p) : Book(p)
    { cout << " New Science just created! " << endl; }

    ~Science()
    { cout << " Science to be destroyed! " << endl; }

    int get_time_needed()
    { time_per_page *= 10; return Book::get_time_needed(); }

    int modify_fatigue(int init_fatig)
    { return init_fatig *= 10; } };

class Cartoon : public Book {
public:
    Cartoon(int p) : Book(p)
    { cout << " New Cartoon just created! " << endl; }

    ~Cartoon()
    { cout << " Cartoon to be destroyed! " << endl; }

    int modify_fatigue(int init_fatig)
    { return init_fatig /= 5; } };

class Reader {
    int available_time;
    int fatigue;
    Book& book1;
    Book& book2;
    Book& book3;

```

```

void read(Book& book)
{ book.announcement();
  available_time -= book.get_time_needed();
  fatigue = book.modify_fatigue(fatigue); }
public:
    Reader(int at, int f, Book& b1, Book& b2, Book& b3)
        : available_time(at), fatigue(f),
          book1(b1), book2(b2), book3(b3)
    { cout << " A New Reader has been created! " << endl; }

    ~Reader()
    { cout << " A Reader to be destroyed with! " << available_time
      << " and " << fatigue << endl; }

    void read()
    { read(book1);
      cout << available_time << " " << fatigue << endl;
      if (available_time >= fatigue) {
          read(book2);
          cout << available_time << " " << fatigue << endl;
          if (available_time >= fatigue) {
              read(book3);
              cout << available_time << " " << fatigue << endl;
              cout << " I finished reading! " << endl;
          }
          else
              cout << " I give up!" << endl;
      }
      else
          cout << "Too many books!" << endl; } };

int main()
{
    Novel the_great_gatsby(20);
    Science calculus(50);
    Cartoon asterix(100);
    Reader r1(1000,600,the_great_gatsby,calculus,asterix);
    Reader r2(200000,50,calculus,asterix,the_great_gatsby);
    r1.read(); r2.read();

    return 0;
}
    
```

Αποσχολιάστε σχολιασμένες γραμμές με τρόπο ώστε το πρόγραμμα να μεταγλωττίζεται και, αιτιολογώντας την απάντησή σας, δώστε το αποτέλεσμα της εκτέλεσής του, κάθε φορά.

7.6.21 Δίνεται το παρακάτω ημιτελές πρόγραμμα C++:

```

#include <iostream>
using namespace std;
    
```

```

class Ingredient {
    int quantity;
public:
    Ingredient(int q) : quantity(q)
    { cout << "I am using :" << q << " of "; id(); }

    Ingredient& mix(Ingredient& ingr)
    { cout << "Adding " << ingr.quantity << " of "; ingr.id();
      cout << " to " << endl; id();
      Ingredient* pin = new Ingredient(quantity + ingr.quantity);
      return *pin; }

    virtual void id()
    { cout << "an ingredient " << endl; } };

class Sugar : public Ingredient {
public:
    Sugar(int q) : Ingredient(q)
    { cout<< "sugar" << endl; }

    void id()
    { cout << "sugar" << endl; } };

class Eggs : public Ingredient {
public:
    Eggs(int q) : Ingredient(q)
    { cout << "eggs" << endl; }

    void id()
    { cout << "eggs" << endl; } };

class Butter : public Ingredient {
public:
    Butter(int q) : Ingredient(q)
    { cout << "butter" << endl; }

    void id()
    { cout << "butter" << endl; } };

class Powder : public Ingredient {
public:
    Powder(int q) : Ingredient(q)
    { cout << "powder" << endl; }

    void id()
    { cout << "powder" << endl; } };

class Cocoa : public Ingredient {

```

```

public:
    Cocoa(int q) : Ingredient(q)
    {   cout << "cocoa" << endl; }

    void id()
    {   cout << "cocoa" << endl; } };

class Cheese : public Ingredient {
public:
    Cheese(int q) : Ingredient(q)
    {   cout << "cheese" << endl; }

    void id()
    {   cout << "cheese" << endl; } };

class Food {
protected:
    int Mins;
public:
    Food(int mins = 0) : Mins(mins) {}

    virtual void bake()
    {   cout << " Put to oven for " << endl; }

    virtual void prepare() = 0; };

class Cake : public Food {
    Sugar& sugar;
    Eggs& eggs;
    Butter& butter;
    Powder& powder;
public:
    Cake(Sugar& s, Eggs& e, Butter& b, Powder& pow)
        : Food(90), sugar(s), eggs(e), butter(b), powder(pow)
    {   cout << " I am going to prepare a cake " << endl; }

    void mix()
    {   powder.mix(butter.mix(sugar.mix(eggs))); }

    virtual void bake ()
    {   Food::bake(); cout << Mins << " mins" << endl; }

    virtual void prepare()
    {   mix(); bake(); } };

class ChocoCake : public Cake {
    Cocoa& cocoa;
public:

```

```

    ChocoCake(Sugar& s, Eggs& e, Butter& b, Powder& p, Cocoa& co)
        : Cake(s,e,b,p), cocoa(co)
    {   cout << " with cocoa " << endl; }

    void add_speciality()
    {   cout << "adding cocoa" << endl; }

    virtual void prepare()
    {   mix(); add_speciality(); bake(); } };

class CheesePie : public Food {
    Eggs& eggs;
    Butter& butter;
    Cheese& cheese;
public:
    CheesePie(Eggs& e, Butter& b, Cheese& ch)
        : Food(120), eggs(e), butter(b), cheese(ch)
    {   cout << " I am going to prepare a cheesepie " << endl; }

    void mix()
    {   cheese.mix(butter.mix(eggs)); }

    virtual void bake ()
    {   Food::bake(); cout << Mins << " mins" << endl; }

    void wrap()
    {   cout << "adding pastry " << endl; }

    virtual void prepare()
    {   mix(); wrap(); bake(); } };

void prepare(Food& fagito)
    {   fagito.prepare(); }

int main()
{
    // Part A
    // TYPE fagito(PARAMS);
    prepare(fagito);
    return 0;
}

```

Τι τιμές μπορεί να πάρει το TYPE; Για καθεμία από αυτές αφαιρέστε τα σχόλια από το σώμα της main και αντικαταστήστε με τους κατάλληλους ορισμούς το Part A και κατάλληλες παραμέτρους το PARAMS για να μπορεί να εκτελεστεί η main. Αιτιολογώντας την απάντησή σας, δώστε το αποτέλεσμα της εκτέλεσης.

7.6.22 Δίνεται το παρακάτω ημιτελές πρόγραμμα C++:

```
#include <iostream>
```



```

using namespace std;

class Investment {
    double money;
public:
    Investment(double m) : money(m)
    { cout << "I can invest :" << m << " money "<< endl; }

    Investment& invest(Investment& inv)
    { cout << "Investing " << inv.money << " to " << money << endl;
      Investment* pinv = new Investment((money + inv.money)*2);
      return *pinv; }

    double money_from_investment()
    { return money; } };

class Person {
public:
    Person() {}
    virtual double earn() = 0; };

class Employee : public Person {
    double salary;
public:
    Employee(double money) : salary(money)
    { cout << " A Employee is created earning: "
      << salary << endl; }

    virtual double earn()
    { return salary; } };

class FreeLance : public Person {
    double income;
    double expenses;
public:
    FreeLance(double moneyin, double moneyout)
        : income(moneyin),expenses(moneyout)
    { cout << " A FreeLance is created getting: "
      << income << " and spending: " << expenses << endl; }

    virtual double earn()
    { return income-expenses; } };

class HouseHold {
    double _expenses;
public:
    HouseHold(double exp) : _expenses(exp)
    { cout << " I just constructed a HouseHold spending : "
      << _expenses << endl; }

```

```

    virtual double expenses()
    { return _expenses; }

    virtual double investments() = 0;

    virtual double fortune() = 0; };

class Family : public Household {
    Person& husband;
    Person& wife;
    int NoOfKids;
public:
    Family(int noofkids, Person& h, Person& w, double exp)
        : Household(exp), husband(h), wife(w), NoOfKids(noofkids)
    { cout << "I just constructed a Family with: "
        << NoOfKids << " Kids " << endl; }

    virtual double expenses()
    { return Household::expenses() + 10 * NoOfKids; }

    virtual double investments()
    { Investment inv1(0.1*wife.earn());
      Investment inv2(0.1*husband.earn());
      return inv1.invest(inv2).money_from_investment(); }

    virtual double fortune()
    { return husband.earn() + wife.earn() +
        investments() - expenses(); } };

class SinglePerson : public Household {
    Person& person;
public:
    SinglePerson(Person& p, double exp) : Household(exp), person(p)
    { cout << " I just created a Single Person Household "
        << endl; }

    virtual double investments()
    { Investment zeroinv(0);
      Investment inv(0.3*person.earn());
      return inv.invest(zeroinv).money_from_investment(); }

    virtual double fortune()
    { return person.earn()+investments()-expenses(); } };

    void fortune(HouseHold& nikokirio)
    { cout << nikokirio.fortune() << endl; }

```

```
int main()
{
    // Part A
    // TYPE nikokirio(PARAMS);
    fortune(nikokirio);
    return 0;
}
```

Τι τιμές μπορεί να πάρει το TYPE; Για καθεμία από αυτές αφαιρέστε τα σχόλια από το σώμα της `main` και αντικαταστήστε με τους κατάλληλους ορισμούς το Part A και κατάλληλες παραμέτρους το PARAMS για να μπορεί να εκτελεστεί η `main`. Αιτιολογώντας την απάντησή σας, δώστε το αποτέλεσμα της εκτέλεσης.

7.6.23 Αιτιολογώντας την απάντησή σας, δώστε το αποτέλεσμα της εκτέλεσης του εξής προγράμματος C++:

```
#include <iostream>
using namespace std;

enum Efrt {zero, little, enough, hard};
enum WTh {positive, negative};

class Person {
protected:
    WTh way_of_thinking;
    Efrt effort;
public:
    Person(Efrt efrt, WTh wth=positive)
        : effort(efrt), way_of_thinking(wth) {}

    virtual bool is_good() = 0;

    virtual void earn_living() = 0;

    virtual void socialize(Person* pp)
    { way_of_thinking = pp->way_of_thinking; };

    void improve(Person* pp)
    { if (effort <= pp->effort)
        effort = pp->effort;
      else
        pp->effort = effort; } };

class GoodPerson : public Person {
public:
    GoodPerson(Efrt efrt): Person(efrt) {}

    bool is_good()
    { return true; }
```

```

void socialize(Person* pp)
{ if (pp->is_good())
    improve(pp);
  else
    pp->Person::socialize(this); }

void earn_living()
{ switch(effort) {
    case hard :
        cout << "I earn a lot of money" << endl; break;
    case enough :
        cout << "I earn enough money" << endl; break;
    case little :
        cout << "I earn little money" << endl; break;
    default:
        cout << "I need a good influence" << endl; } } };

class WickedPerson : public Person {
public:
    WickedPerson() : Person(zero,negative) {}

    bool is_good()
    { return false; }

    void earn_living()
    { switch(way_of_thinking) {
        case positive :
            cout << "I'll make an honest effort" << endl; break;
        default :
            cout << "You'd better not ask me" << endl; } } };

int main()
{
    GoodPerson gp1(zero); GoodPerson gp2(little);
    GoodPerson gp3(enough); GoodPerson gp4(hard);
    WickedPerson wp;

    Person* persons[] = {&gp1, &gp2, &gp3, &gp4, &wp};
    for(int i = 0; i < 5; i++)
        persons[i]->earn_living();

    for(int i = 0; i < 4; i++)
        persons[i+1]->socialize(persons[i]);
    for(int i = 0; i < 5; i++)
        persons[i]->earn_living();

    for(int i = 4; i > 0; i--)
        persons[i-1]->socialize(persons[i]);
}

```

```

    for(int i = 0; i < 5; i++)
        persons[i]->earn_living();

    return 0;
}

```

7.6.24 Δίνεται το παρακάτω πρόγραμμα C++:

```

#include <iostream>
using namespace std;

class Dish {
protected:
    int cost;
    int cal;
public:
    Dish(int cst = 0, int cl = 150) : cost(cst), cal(cl)
    { cout << "A Dish was created " << endl; }

    ~Dish()
    { cout << "A Dish will be destroyed" << endl; }

    virtual void eat()
    { cout << "Eating a Dish with cal " << cal
      << " and cost " << cost << endl; }

    int get_cost()
    { return cost; }

    int get_cal()
    { return cal; } };

class FirstCourse : public Dish {
public:
    FirstCourse(int cst, int cl) : Dish(cst,cl)
    { cout << "A FirstCourse was created with " <<
      cost << " and " << cal << endl; }

    ~FirstCourse()
    { cout << "A FirstCourse will be destroyed" << endl; } };

class Starter : public FirstCourse {
public:
    Starter(int cst, int cl) : FirstCourse(cst,cl)
    { cout << "A Starter was created " << endl; }

    ~Starter()
    { cout << "A Starter will be destroyed" << endl; }
}

```

```

    void eat()
    {   FirstCourse::eat();
        cout << "Eating Starter " << endl; } };

class Salad : public FirstCourse {
public:
    Salad(int cst, int cl) : FirstCourse(cst,cl)
    {   cout << "A Salad was created " << endl; }

    ~Salad()
    {   cout << "A Salad will be destroyed" << endl; }

    void eat()
    {   FirstCourse::eat();
        cout << "Eating Salad " << endl; } };

class MainCourse : public Dish {
public:
    MainCourse(int cst, int cl) : Dish(cst,cl)
    {   cout << "A MainCourse was created with " <<
        cost << " and " << calcs << endl; }

    ~MainCourse()
    {   cout << "A MainCourse will be destroyed" << endl; } };

class Dessert : public Dish {
public:
    Dessert(int cst, int cl) : Dish(cst,cl)
    {   cout << "A Dessert was created " << endl; }

    ~Dessert()
    {   cout << "A Dessert will be destroyed" << endl; }

    void eat()
    {   Dish::eat();
        cout << "Eating Dessert " << endl; } };

class Meal {
    static const int service = 1;
    Dish starter;
    Dish salad;
    Dish maincourse;
    Dish dessert;
public:
    Meal()
    {   cout << "A Meal has been created " << endl; }

    ~Meal()
    {   cout << "A Meal will be destroyed" << endl; }

```

```

void order()
{ starter = *(new Starter(4, 100));
  salad = *(new Salad(6, 50));
  maincourse = *(new MainCourse(10, 500));
  dessert = *(new Dessert(4, 450)); }

void eat()
{ starter.eat(); salad.eat(); maincourse.eat(); dessert.eat(); }

int pay_bill()
{ return starter.get_cost() + salad.get_cost() +
  maincourse.get_cost() + dessert.get_cost() + service; }

int evaluate_cals()
{ return starter.get_cals() + salad.get_cals() +
  maincourse.get_cals() + dessert.get_cals(); } };

int main()
{
    Meal meal;

    meal.order();
    meal.eat();

    cout << "You are going to pay " << meal.pay_bill()
          << " euros " << endl;
    cout << "You are going to get " << meal.evaluate_cals()
          << " calories " << endl;
    return 0;
}

```

Δώστε το αποτέλεσμα της εκτέλεσής του, αιτιολογώντας την απάντησή σας.

7.6.25 Θεωρήστε τις κλάσεις και τη συνάρτηση `main` της προηγούμενης άσκησης με τη διαφορά ότι ο ορισμός της κλάσης `Meal` είναι ο εξής:

```

class Meal {
    static const int service = 1;
    Dish* starter;
    Dish* salad;
    Dish* maincourse;
    Dish* dessert;
public:
    Meal()
    { cout << "A Meal has been created " << endl; }

    ~Meal()
    { cout << "A Meal will be destroyed" << endl; }
}

```

```

void order()
{   starter = new Starter(4, 100);
    salad = new Salad(6, 50);
    maincourse = new MainCourse(10, 500);
    dessert = new Dessert(4, 450); }

void eat()
{   starter->eat(); salad->eat();
    maincourse->eat(); dessert->eat(); }

int pay_bill()
{   return starter->get_cost() + salad->get_cost() +
        maincourse->get_cost() + dessert->get_cost() +
        service; }

int evaluate_cals()
{   return starter->get_cals() + salad->get_cals() +
        maincourse->get_cals() + dessert->get_cals(); } };

```

Δώστε το αποτέλεσμα της εκτέλεσης του προγράμματος, αιτιολογώντας την απάντησή σας.

Εργασίες Για περισσότερη εξάσκηση στην ανάπτυξη κώδικα στην περιοχή της κληρονομικότητας για πιο σύνθετα προβλήματα, μπορείτε να ασχοληθείτε με τις εργασίες της Παραγράφου 11.5.

7.7 Βιβλιογραφία

- [1] Bruce Eckel. *Thinking in C++, Volume I: Introduction to Standard C++*. 2η έκδοση. Prentice Hall, 2000.
- [2] C++ reference. <https://en.cppreference.com/w/>. Ανάκτηση: 2023-5-30.
- [3] cplusplus.com. <http://www.cplusplus.com/>. Ανάκτηση: 2023-5-30.
- [4] Bjarne Stroustrup. *The C++ Programming Language*. 4η έκδοση. Addison-Wesley Professional, 2013.
- [5] Bjarne Stroustrup. *Programming: Principles and Practice Using C++*. 2η έκδοση. Addison-Wesley Professional, 2014.
- [6] Stanley B. Lippman. *Inside the C++ Object Model*. Addison-Wesley Professional, 1996.
- [7] Stanley B. Lippman, Josée Lajoie και Barbara Moo. *C++ Primer*. 5η έκδοση. Addison-Wesley Professional, 2012.

ΑΛΛΑ ΓΛΩΣΣΙΚΑ ΣΤΟΙΧΕΙΑ ΤΗΣ C++ ΣΧΕΤΙΚΑ ΜΕ ΚΛΑΣΕΙΣ

Σύνοψη

Στο κεφάλαιο αυτό, συζητούνται θέματα σχετικά με τις κλάσεις τα οποία είτε δεν έχουν συζητηθεί στα προηγούμενα είτε σχετίζονται αποκλειστικά με τη γλώσσα C++. Αρχικά, γίνεται μια συζήτηση περί σταθερών της C++ στο πλαίσιο των κλάσεων. Στη συνέχεια, παρουσιάζονται τα στατικά μέλη κλάσεων της C++. Δίνεται μεγάλη βαρύτητα στην παρουσίασή τους, διότι, πέρα από το άμεσο όφελος από τη λειτουργικότητά τους, με τη χρήση τους γίνεται σταδιακή μετάβαση από τη χρήση της έννοιας της κλάσης ως τύπου ορισμένου από τον χρήστη στη χρήση της ως εργαλείου τμηματοποίησης κώδικα/στοιχειοποίησης. Έτσι, στο επόμενο κεφάλαιο, θα οδηγηθούμε ομαλά στη μετάβαση στη γλώσσα Java και την κατανόηση της λειτουργικότητας των γλωσσικών δομών της. Κατόπιν, ολοκληρώνεται η συζήτηση περί μελών κλάσεων, παρουσιάζοντας τις εμφωλευμένες κλάσεις (nested classes). Στο κεφάλαιο αυτό, θα συζητήσουμε και άλλα θέματα που αφορούν κλάσεις και τα οποία δεν πρέπει να παραλειφθούν. Ένα από αυτά αφορά τη δυνατότητα εκχώρησης δικαιωμάτων ορατότητας μέσω φιλικών συναρτήσεων ή/και κλάσεων και περιπτώσεις χρήσης τους. Επίσης, θα συζητηθούν η χρήση των κλάσεων στα ορίσματα ή/και στον τύπο επιστροφής συνάρτησης και η αποδοτική αντιμετώπισή της. Στη συνέχεια, θα γίνει μια αναφορά στην υποστήριξη Προγραμματισμού Γενικεύσεων στη C++ μέσω περιγραμμάτων κλάσεων και περιγραμμάτων συναρτήσεων. Θα ακολουθήσει μια συνοπτική παρουσίαση των τύπων βιβλιοθήκης που προσφέρει η γλώσσα. Τέλος, θα συζητηθεί το σύστημα εισόδου/εξόδου της C++, εστιάζοντας στα αντικειμενοστραφή χαρακτηριστικά του.

Προαπαιτούμενη Γνώση

Για την κατανόηση του κεφαλαίου, ο αναγνώστης απαιτείται να έχει γνώσεις διαδικαστικού προγραμματισμού και να έχει μελετήσει τις έννοιες του αντικειμενοστραφούς προγραμματισμού και της C++ από τα προηγούμενα κεφάλαια του βιβλίου.

8.1 Σταθερές και Κλάσεις

Θα ξεκινήσουμε με κάτι απλό που θα μπορούσαμε να το είχαμε συζητήσει νωρίτερα. Αφορά τον χαρακτηρισμό “σταθερά” σε κάτι που σχετίζεται με κλάσεις.

Συναρτήσεις-μέλη const: Ήδη μιλήσαμε για τον χαρακτηρισμό συναρτήσεων-μελών ως `const` όταν δεν μεταβάλλουν το αντικείμενο πάνω στο οποίο εφαρμόζονται, π.χ.

```
char* Student::get_name() const;
```

Ουσιαστικά, έτσι δηλώνουμε ότι μια συνάρτηση-μέλος είναι αναγνώστρια συνάρτηση.

Σταθερά Μέλη-Δεδομένα: Μπορεί να θέλουμε να δηλώσουμε ότι ένα μέλος-δεδομένο είναι σταθερά. Για παράδειγμα, ο “αριθμός μητρώου” ενός φοιτητή έχει τιμή που ανατίθεται εξ αρχής και δεν αλλάζει. Αν στην κλάση `Student` των παραδειγμάτων μας θέλουμε να εκφράσουμε αυτήν την ιδιότητα του “αριθμού μητρώου”, τότε πρέπει να αλλάξουμε τη δήλωση του μέλους-δεδομένου `no`, σε:

```
const int no;
```

Εξυπακούεται ότι πρέπει να αφαιρέσουμε τη συνάρτηση-μέλος `void Student::set_no(int)`. Επιπλέον, πρέπει να αρχικοποιήσουμε το μέλος `Student::no` στις συναρτήσεις αρχικοποίησης μέσω λίστας αρχικοποίησης.

Συνοπτικά, λοιπόν, να πούμε ότι μέλη-δεδομένα μπορούν να δηλωθούν ως `const`, επιβάλλοντας η τιμή τους να μην αλλάξει κατά τη διάρκεια ζωής του αντικειμένου. Η τιμή αυτή δίνεται μέσω λίστας αρχικοποίησης από τις συναρτήσεις αρχικοποίησης.

Αντικείμενα const: Κάποια αντικείμενα δεν θέλουμε να μεταβληθούν κατά τη διάρκεια της ζωής τους. Η χρήση της λέξης-κλειδιού `const` κατά τον ορισμό τους δηλώνει ότι το αντικείμενο δεν θα μεταβληθεί (και κάθε ενέργεια προς το αντίθετο δημιουργεί συντακτικό λάθος), π.χ.

```
const Date millennium(1,1,2000);
```

Σε αυτήν την περίπτωση, οι μόνες συναρτήσεις-μέλη που μπορούν να εφαρμοστούν σε αυτά τα αντικείμενα είναι εκείνες που έχουν δηλωθεί ως `const`, οπότε δεν μεταβάλλουν τα αντικείμενα στα οποία εφαρμόζονται. Να σημειώσουμε ότι μπορούμε να έχουμε υπερφορτωμένους ορισμούς συναρτήσεων-μελών `const` με `non-const` εκδοχές τους. Το ποια συνάρτηση θα κληθεί αποφασίζεται από τον μεταγλωττιστή, με βάση το εάν εφαρμόζεται σε αντικείμενο `const` ή όχι.

8.2 Στατικά Μέλη Κλάσεων

Στην παράγραφο αυτή θα αναφερθούμε στα στατικά μέλη κλάσεων. Αυτά είναι μέλη που δεν είναι απαραίτητο να εφαρμοστούν σε αντικείμενο της κλάσης διότι είναι ανεξάρτητα αντικειμένου. Δηλαδή, όσον αφορά τα στατικά μέλη-δεδομένα, αυτά δεν συμμετέχουν στην κατασκευή αντικειμένων της κλάσης και, όσον αφορά τις στατικές συναρτήσεις-μέλη, αυτές δεν λειτουργούν πάνω σε αντικείμενο της κλάσης.

Ένα C++ πρόγραμμα αποτελείται από συναρτήσεις και, ενδεχόμενα, κλάσεις. Το ελάχιστο πρόγραμμα αποτελείται από μία συνάρτηση, την `main`. Κατά συνέπεια, επιτρέπεται κώδικας ενός προγράμματος να βρίσκεται σε συναρτήσεις που δεν είναι απαραίτητα μέλη κλάσεων. Στις καθαρά αντικειμενοστραφείς γλώσσες προγραμματισμού, όμως, όπου όλος ο κώδικας πρέπει να είναι οργανωμένος σε κλάσεις, αυτό δεν ισχύει. Κάθε συνάρτηση πρέπει να είναι μέλος μιας κλάσης, ανεξάρτητα εάν εφαρμόζεται σε αντικείμενο της κλάσης ή όχι. Στις γλώσσες αυτές, οι κλάσεις μπορούν να λειτουργούν αποκλειστικά για την παροχή εμβέλειας και απόκρυψης πληροφορίας και όχι απαραίτητα ως περιγραφή νέων τύπων οι οποίοι αναπαριστούν οντότητες/έννοιες του προβλήματος και των οποίων μεταβλητές ορίζονται στον κώδικα. Κατά συνέπεια, είναι απαραίτητη η δυνατότητα να “αποσυνδέουμε” τα μέλη κλάσεων από την ύπαρξη αντικειμένων τους. Αυτό επιτυγχάνεται με το να τα δηλώνουμε ως στατικά.

Όσον αφορά τα στατικά μέλη-δεδομένα, στη C++, αυτό επιτυγχάνεται με τη χρήση της λέξης-κλειδιού `static` κατά τη δήλωση της μεταβλητής στον ορισμό της κλάσης. Με τον τρόπο αυτό, δηλώνουμε ότι η μεταβλητή ανήκει απλά στην εμβέλεια της κλάσης και δεν συμμετέχει στην κατασκευή των αντικειμένων της. Τα στατικά μέλη κλάσεων αρχικοποιούνται (και μόνο μία φορά) σε επίπεδο εμβέλειας κλάσης.

Αντίστοιχα, μπορούμε να δηλώσουμε μια συνάρτηση-μέλος ως στατική με τη χρήση της λέξης-κλειδιού `static` κατά τη δήλωση της συνάρτησης στον ορισμό της κλάσης. Κατά τον τρόπο αυτό, δηλώνουμε ότι η συνάρτηση απλά ανήκει στην εμβέλεια της κλάσης και δεν είναι απαραίτητο να εφαρμοστεί σε αντικείμενό της. Συντακτικά, μπορεί να κληθεί εφαρμοζόμενη σε αντικείμενο — ή δείκτη — τύπου της κλάσης, αλλά τότε το αντικείμενο — ή ο δείκτης — λειτουργεί σαν “οδηγός”, για να υποδείξει τη χρήση της εμβέλειας της κλάσης του.

Τα στατικά μέλη-δεδομένα μιας κλάσης υπάρχουν και μπορούν να χρησιμοποιηθούν ακόμα και αν δεν έχουν δημιουργηθεί αντικείμενα της κλάσης. Αντίστοιχα μπορούν να χρησιμοποιηθούν και οι στατικές συναρτήσεις-μέλη. Όπως αναμένεται, εκτός εμβέλειας της κλάσης, μπορούμε να έχουμε πρόσβαση στα δημόσια στατικά μέλη μιας κλάσης. Συντακτικά, αυτό επιτυγχάνεται:

- είτε μέσω ενός αντικειμένου της κλάσης,
- είτε με χρήση του ονόματος της κλάσης και του τελεστή επίλυσης εμβέλειας, `::` (υποχρεωτικά αν δεν έχουν δημιουργηθεί αντικείμενα της κλάσης).

Εκτός κλάσης, μπορούμε να έχουμε πρόσβαση στα στατικά ιδιωτικά ή στατικά προστατευόμενα μέλη μέσω δημοσίων συναρτήσεων-μελών (ή φιλικών συναρτήσεων — θα αναφερθούμε σε αυτές αργότερα).

Από τη φύση των στατικών συναρτήσεων-μελών, ως ανεξάρτητες αντικειμένου, είναι αναμενόμενο ότι δεν μπορούμε να χρησιμοποιήσουμε τον δείκτη `this` στο σώμα μιας στατικής συνάρτησης-μέλους μιας κλάσης. Επίσης, για τον ίδιο λόγο, δεν μπορούμε να δηλώσουμε μια στατική συνάρτηση-μέλος ως `const`. Αντίθετα, τα στατικά μέλη-δεδομένα μπορούν να δηλωθούν ως `const`.

Επιγραμματικά, μπορούμε να πούμε ότι τα στατικά μέλη ουσιαστικά είναι συμβατικές εξωτερικές μεταβλητές και συναρτήσεις οι οποίες απλά είναι εμβέλειας κλάσης.

Ας δούμε ένα παράδειγμα που συγκεντρώνει διάφορες παρατηρήσεις για τα στατικά μέλη — αλλά και για τα μέλη που έχουν δηλωθεί ως `const`.

Στο παράδειγμα αυτό, θεωρούμε μια έννοια “Φοιτητής” με κάπως διαφορετικές προδιαγραφές από αυτές που είχε έως τώρα. Συγκεκριμένα, θεωρούμε ότι μας ενδιαφέρει η πληροφορία του “αριθμού μητρώου” ενός “Φοιτητή”, καθώς και η “βαθμολογία των επιτυχώς εξετασθέντων μαθημάτων” του, ώστε να μπορούμε να “υπολογίσουμε τον μέσο όρο” του. Επίσης, ενδιαφέρει να μπορούμε να αποφασίσουμε εάν έχει “ολοκληρώσει τις σπουδές” του ή όχι. Προκειμένου να αποφασίσουμε τα παραπάνω, χρειάζεται να έχουμε καθορίσει το “πλήθος μαθημάτων που απαιτούνται” για την ολοκλήρωση των σπουδών, καθώς και το ποια βαθμολογία αποτελεί τη “βάση” για τη βαθμολογία επιτυχώς εξετασθέντος μαθήματος.

Τόσο ο “αριθμός μητρώου”, όσο και η “βαθμολογία των επιτυχώς εξετασθέντων μαθημάτων”, αποτελούν πληροφορία που αφορά συγκεκριμένο “Φοιτητή”. Το “πλήθος μαθημάτων που απαιτούνται”, όμως, και η “βάση” της βαθμολογίας είναι πληροφορία που αφορά όλους τους φοιτητές, οπότε σχετίζεται με την έννοια του “Φοιτητή” γενικότερα. Αν υποθέσουμε ότι, για στατιστικούς λόγους, μας ενδιαφέρει και η “μέγιστη τιμή μέσου όρου” βαθμολογίας φοιτητών που ολοκλήρωσαν τις σπουδές τους, η πληροφορία αυτή εμπίπτει στην ίδια περίπτωση. Πληροφορία που σχετίζεται με την έννοια του “Φοιτητή” γενικότερα είναι και το “πλήθος” των φοιτητών που υπάρχουν μια δεδομένη στιγμή.

Ας υποθέσουμε ότι η έννοια του “Φοιτητή” αναπαριστάται από μια κλάση `Student`. Με βάση τις παραπάνω προδιαγραφές, η υλοποίησή του μπορεί να είναι η εξής:

```
class Student {
public:
    Student(int nov);
    ~Student();
```

```

int get_no() const
{ return no; }

bool graduate() const
{ return no_courses == TOTAL_COURSES;
    // const static together
    // with instance var
}

int evaluate_GPA() const;

void record_grade(int grade);

static int get_highest_GPA() // static function definition
{ return highest_GPA; }    // static var access

static int get_no_students() // static function definition
{ return no_students; }     // static var access

static const int PASS_GRADE = 5; // static const var definition

static const int TOTAL_COURSES = 40; // static const var
                                     // definition

private:
    static int no_students; // static var declaration
    static int highest_GPA; // static var declaration

    const int no;
    int no_courses;
    int completed_courses[TOTAL_COURSES];
};

int Student::no_students = 0; // static var definition

int Student::highest_GPA = 0; // static var definition

Student::Student(int nov) : no(nov), no_courses(0)
{
    no_students++; // static var access
    cout << "Student with no " << no << " created!" << endl;
    cout << "There are " << no_students << " Students now!" << endl;
}

Student::~~Student()
{
    no_students--; // static var access
    cout << "Student with no " << no << " to be destroyed!" << endl;
}

```

```

    cout << "There are " << no_students // static var access
         << " Students now!" << endl;
}

void Student::record_grade(int grade)
{
    if (grade >= PASS_GRADE) {           // const static together
                                         // with instance var
        completed_courses[no_courses++] = grade;
    };
}

int Student::evaluate_GPA() const
{
    int curr_GPA = 0;
    if (no_courses > 0) {
        for (int i = 0; i < no_courses; i++)
            curr_GPA += completed_courses[i];
        if ((curr_GPA % no_courses) < (no_courses * 0.5))
            curr_GPA = curr_GPA / no_courses;
        else
            curr_GPA = ((curr_GPA / no_courses) + 1);
        if (graduate())
            if (curr_GPA > highest_GPA) // static var access
                highest_GPA = curr_GPA; // static var access
    }
    return curr_GPA;
}

```

Για την αναπαράσταση των αντικειμένων, έχουμε μέλη-δεδομένα στιγμιότυπου τα:

`const int no:` το οποίο αναπαριστάει τον “αριθμό μητρώου” του φοιτητή. Σύμφωνα με παρατήρηση που έγινε στην προηγούμενη παράγραφο, αυτός είναι σταθερά, οπότε παίρνει υποχρεωτικά τιμή κατά τη δημιουργία ενός αντικειμένου της κλάσης.

`int no_courses:` το οποίο αναπαριστάει το πλήθος των “επιτυχώς εξετασθέντων μαθημάτων” ενός στιγμιότυπου της έννοιας “Φοιτητής”.

`int completed_courses[TOTAL_COURSES]:` το οποίο αναπαριστάει τη “βαθμολογία των επιτυχώς εξετασθέντων μαθημάτων” ενός στιγμιότυπου της έννοιας “Φοιτητής”.

Βλέπουμε, επίσης τα παρακάτω στατικά μέλη-δεδομένα (μέλη-δεδομένα κλάσης):

`static int no_students:` η μεταβλητή εμβέλειας κλάσης `Student`, προκειμένου να καταχωρούμε το πλήθος των στιγμιότυπων της κλάσης που είναι εν ζωή.

`static int highest_GPA:` η μεταβλητή εμβέλειας κλάσης `Student`, προκειμένου να καταχωρούμε τον τρέχοντα μέγιστο μέσο όρο που έχει επιτύχει φοιτητής, του οποίου έχει υπολογιστεί ο μέσος όρος και

έχει ολοκληρώσει τις υποχρεώσεις λήψης πτυχίου¹.

Η συμπεριφορά των στιγμιotypών της κλάσης `Student` καθορίζεται από τις παρακάτω συναρτήσεις-μέλη:

`Student(int nov)`: Η συνάρτηση αρχικοποίησης. Χρειάζεται οπωσδήποτε για να αρχικοποιήσει το μέλος-δεδομένο που έχει δηλωθεί ως `const`. Επίσης, στο σώμα της αυξάνει την τιμή της στατικής μεταβλητής `Student::no_students` κατά ένα, μια που για να εκτελείται, σημαίνει ότι δημιουργήθηκε μόλις ένα νέο αντικείμενο της κλάσης `Student`.

`~Student()`: Η συνάρτηση καταστροφής. Δεν θα ήταν απαραίτητη, αν δεν έπρεπε να καταμετράμε τα εν ζωή στιγμιότυπα της κλάσης `Student`. Επομένως, στο σώμα της μειώνουμε την τιμή της στατικής μεταβλητής `Student::no_students` κατά ένα, μια, που για να εκτελείται, σημαίνει ότι πρόκειται να καταστραφεί ένα αντικείμενο της κλάσης `Student`.

`int get_no() const`: Αναγνώστρια για το μέλος-δεδομένο που αντιστοιχεί στον “αριθμό μητρώου” του φοιτητή.

`bool graduate() const`: Ελέγχει εάν ο φοιτητής έχει ολοκληρώσει τις υποχρεώσεις λήψης πτυχίου, χρησιμοποιώντας τη στατική σταθερά που αντιστοιχεί στο απαιτούμενο πλήθος μαθημάτων, `TOTAL_COURSES`.

`int evaluate_GPA() const`: Υπολογίζει τον τρέχοντα μέσο όρο της βαθμολογίας του φοιτητή. Εάν έχει ολοκληρώσει τις υποχρεώσεις λήψης πτυχίου του, ελέγχει μήπως ο μέσος όρος του υπερβαίνει τον τρέχοντα καλύτερο μέσο όρο, ώστε να γίνει ενημέρωση της αντίστοιχης στατικής μεταβλητής, `highest_GPA`.

`void record_grade(int grade)`: μεταβάλλουσα συνάρτηση-μέλος προκειμένου να επιχειρήσουμε να καταχωρήσουμε έναν βαθμό του φοιτητή. Προκειμένου να επιτευχθεί αυτό, γίνεται έλεγχος με τη στατική σταθερά που αντιστοιχεί στη “βάση”, `PASS_GRADE`.

Επίσης, έχουμε και τις παρακάτω συναρτήσεις εμβέλειας κλάσης `Student` (στατικές συναρτήσεις):

`static int get_highest_GPA()`: Ανακτά την τιμή του μέγιστου μέσου όρου όλων των φοιτητών.

`static int get_no_students()`: Ανακτά το πλήθος των εν ζωή στιγμιotypών φοιτητών.

Τέλος, έχουμε τον ορισμό στατικών σταθερών, τις εξής:

`static const int PASS_GRADE = 5`: Πρόκειται για τη σταθερά που δηλώνει ποια είναι η “βάση” της βαθμολογίας.

`static const int TOTAL_COURSES = 40`: Πρόκειται για τη σταθερά που δηλώνει ποιο είναι το πλήθος των μαθημάτων που πρέπει να έχει εξεταστεί επιτυχώς, προκειμένου ο (οποιοσδήποτε) φοιτητής να θεωρείται ότι ολοκλήρωσε τις σπουδές του.

Στην παρακάτω συνάρτηση `main` έχουμε ένα σενάριο χρήσης της κλάσης `Student`. Βλέπετε τη χρήση των στατικών μελών, τόσο μέσω χρήσης του τελεστή επίλυσης εμβέλειας, όσο και μέσω αντικειμένου της κλάσης, αλλά ακόμα και μέσω δείκτη της κλάσης. Επίσης, όπως τονίσαμε, γίνεται εμφανές ότι τα στατικά μέλη μπορούν να χρησιμοποιηθούν, ανεξάρτητα ύπαρξης αντικειμένου της κλάσης.

¹Παρότι στο παράδειγμά μας, για λόγους ευκρίνειας, κάνουμε την αρχικοποίηση με την τιμή 0 στις δύο αυτές στατικές μεταβλητές, να σημειώσουμε ότι από το πρότυπο της C++ διασφαλίζεται ότι η αρχικοποίηση των στατικών μεταβλητών είναι με μηδενική τιμή ούτως ή άλλως.

```

int main()
{
    cout << Student::get_no_students()    //    static function call
        << endl;                          //    no instances yet

    Student s1(100);
    Student s2(200);

    cout << Student::get_no_students()    //    static function call
        << endl;
    cout << s2.get_no_students() << endl; //    static function call
                                           //    using instance

    s1.record_grade(9);
    s1.record_grade(4);
    s1.record_grade(7);
    s1.record_grade(10);
    cout << "The student with no; " << s1.get_no()
        << " has current GPA: " << s1.evaluate_GPA() << endl;

    cout << "The total courses will be: "
        << s1.TOTAL_COURSES << endl;    //    static const var access
                                           //    using instance

    cout << "The pass grade is: "
        << Student::PASS_GRADE << endl; //    static const var access

    Student* ps3 = new Student(300);
    cout << ps3->get_no_students() << endl; //    static function call
                                           //    using pointer

    Student* ps4;
    cout << ps4->get_no_students() << endl; //    static function call
                                           //    using pointer

    delete ps3;

    return 0;
}

```

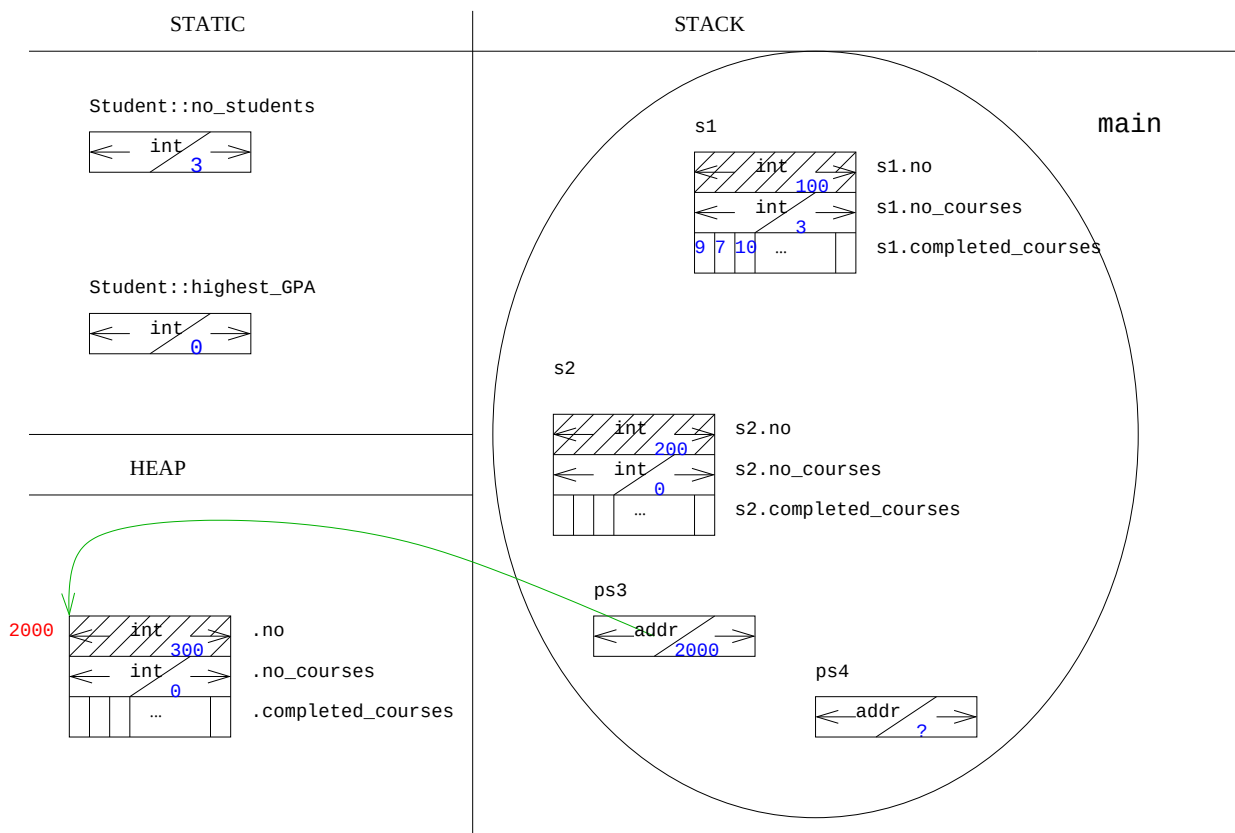
Κατά την εκτέλεση της συνάρτησης αυτής, δημιουργούνται τρία αντικείμενα της κλάσης Student. Η αφαιρετική εικόνα της μνήμης με την αναπαράστασή τους πριν την εκτέλεση της εντολής `delete ps3`; φαίνεται στο Σχήμα 8.1.

Η εκτύπωση που προκύπτει από την εκτέλεση του παραπάνω προγράμματος είναι η εξής:

```

0
Student with no 100 created!
There are 1 Students now!
Student with no 200 created!
There are 2 Students now!

```

Σχήμα 8.1: Η εικόνα των “φοιτητών” της συνάρτησης `main`

```

2
2
The student with no; 100 has current GPA: 9
The total courses will be: 40
The pass grade is: 5
Student with no 300 created!
There are 3 Students now!
3
3
Student with no 300 to be destroyed!
There are 2 Students now!
Student with no 200 to be destroyed!
There are 1 Students now!
Student with no 100 to be destroyed!
There are 0 Students now!

```

8.3 Ολοκληρώνοντας Σχετικά με τα Μέλη Κλάσεων

Συνοψίζοντας, είδαμε ότι μια κλάση μπορεί να έχει μέλη τα οποία να είναι:

- είτε μέλη *στιγμιότυπου*, τα οποία συμβάλλουν στην αναπαράσταση ενός αντικειμένου, αν πρόκειται για μέλη-δεδομένα, ή εφαρμόζονται απαραίτητα σε στιγμιότυπο — που προσπελαύνεται άμεσα μέσω του δείκτη `this` — αν πρόκειται για συνάρτηση-μέλος,
- είτε *στατικά* μέλη (μέλη κλάσης), τα οποία είναι ανεξάρτητα αντικειμένου, απλώς ανήκουν στην εμβέ-

λεια της κλάσης και προσπελαύνονται άμεσα με χρήση του τελεστή επίλυσης εμβέλειας, `::`, ή με χρήση αντικειμένου, ή δείκτη, της κλάσης, προκειμένου να υποδειχθεί η κλάση μέσω αυτού.

Αλήθεια, όμως, τι να σημαίνει μια κενή κλάση, δηλαδή αν η κλάση δεν έχει κανένα μέλος; Εκτελέστε το παρακάτω:

```
#include <iostream>
using namespace std;

class A {};

int main()
{
    A a;
    cout << sizeof(a) << endl;

    A b;
    cout << &a << '\n' << &b << endl;    // Different addresses

    return 0;
}
```

Δεσμεύεται ο ελάχιστος χώρος που μπορεί να διευθυνσιοδοτηθεί για τα αντικείμενά της!

8.3.1 Εμφωλευμένες Κλάσεις

Στην περίπτωση που μια κλάση δεν είναι ευρέος ενδιαφέροντος, ή συνδέεται με το περιβάλλον μιας κλάσης ή, ακόμα, αποτελεί λεπτομέρεια υλοποίησης μιας κλάσης, τότε η κλάση αυτή μπορεί να δηλωθεί ως μέλος της κλάσης και να οριστεί ως τέτοιο. Δηλαδή,

```
class A {
    // ...
    class B { ... };
    // ...
};
```

Στην παραπάνω περίπτωση, η κλάση B είναι *εμφωλευμένη κλάση* της εξωτερικής κλάσης A, κάτι που δηλώνει ότι:

- Η εμφωλευμένη κλάση ανήκει στην εμβέλεια της κλάσης. Η ορατότητα του ονόματός της ακολουθεί τους κανόνες ορατότητας των μελών των κλάσεων. Αν αποτελεί λεπτομέρεια υλοποίησης, καλό είναι να δηλωθεί ως `private`.
- Η εμφωλευμένη κλάση δεν συνδέεται με αντικείμενο της εξωτερικής. Απλά ανήκει στην εμβέλειά της.
- Από την εμβέλεια της εμφωλευμένης κλάσης έχουμε πρόσβαση σε όλα τα ονόματα της εμβέλειας της εξωτερικής. Το ανάποδο δεν ισχύει. Η εξωτερική κλάση έχει πρόσβαση μόνο στα δημόσια ονόματα της εμφωλευμένης.

Εκμεταλλεόμενοι τη δυνατότητα αυτή, τύποι που είναι περιορισμένου ενδιαφέροντος στο πλαίσιο μιας κλάσης, καλό είναι να δηλώνονται ως εμφωλευμένοι στην κλάση αυτή και όχι σε επίπεδο καθολικής εμβέλειας. Ένα τέτοιο παράδειγμα αποτελεί ο τύπος του κόμβου της λίστας, `ChildrenN`, που αναπαριστάνει τα παιδιά μιας οικογένειας στο παράδειγμα της σελίδας 198. Ο τύπος αυτός έχει οριστεί απλά για να εξυπηρετήσει

την υλοποίηση της ακολουθίας των παιδιών της οικογένειας — μέλος `children` της κλάσης `Family`. Κατά συνέπεια, θα ήταν πιο κατάλληλο ο τύπος αυτός να είναι ορισμένος ως εμφωλευμένος στην κλάση `Family`, η οποία αναπαριστάει την “Οικογένεια” και μάλιστα να δηλωθεί στο ιδιωτικό τμήμα της.

Με βάση τα παραπάνω, ένας πιο καθαρός ορισμός της κλάσης `Family` είναι ο εξής:

```
class Family {

    struct ChildrenN {
        Person* person;
        ChildrenN* next;
        ChildrenN(Person* p, ChildrenN* n) : person(p), next(n) {};
        ~ChildrenN()
        {   if (person != NULL)
            delete person; // Assuming dynamic creation
            if (next != NULL)
                delete next; } // Copy constructor and
                                // assignment operator also needed!
    };

    Person& husband;
    Person& wife;
    Date date_of_wedding;
    ChildrenN* children;
    Car* owned_car;

    Person* first_child() const;
    Person* last_child() const;

public:
    Family(int d, int m, int y, Person& h, Person& w)
        : date_of_wedding(d,m,y), husband(h), wife(w),
          children(NULL), owned_car(NULL) {};

    ~Family()
    {
        if (has_children())
            delete children;
    } // Dynamic creation assumed
        // Copy constructor and assignment operator may be also needed!

    Person& birth(int, int, int, Sex);
    Person* child_adult();

    void buy_car(Car& cr)
    {
        owned_car = &cr;
    }
}
```

```

bool has_children() const
{
    return children != NULL;
}

bool has_car() const
{
    return owned_car != NULL;
}

int no_of_children() const;

int diff() const
{
    if (has_children())
        return last_child()->get_date_of_birth().get_year() -
            first_child()->get_date_of_birth().get_year();
}
};

```

8.4 Η Έννοια της Φιλίας μεταξύ Εμβελειών

Έχουμε τονίσει ότι στα ιδιωτικά μέλη κλάσεων έχουμε πρόσβαση μόνο από την εμβέλεια της ίδιας της κλάσης που τα δηλώνει — καθώς και τις εμβέλειες που είναι εσωτερικές σε αυτή, όπως τα σώματα των συναρτήσεων-μελών και οι εμβέλειες των εμφωλευμένων κλάσεων. Όμως, αρκετά συχνά στα προηγούμενα, κάναμε αναφορά στην έννοια των “φιλικών συναρτήσεων” και των “φιλικών κλάσεων” από τις εμβέλειες των οποίων μπορεί να γίνει πρόσβαση και στα ιδιωτικά ονόματα μιας κλάσης. Στην παράγραφο αυτή, θα ασχοληθούμε με την έννοια αυτή της “φιλίας”.

Μια κλάση μπορεί να καθορίσει ότι μια συνάρτηση, είτε αυτή είναι μέλος άλλης κλάσης είτε απλή συνάρτηση, είναι “φιλική”, δίνοντάς της δικαίωμα πρόσβασης στα ιδιωτικά ονόματά της. Το ίδιο μπορεί να γίνει και για μια ολόκληρη κλάση. Μπορεί να αναρωτηθεί κανείς μήπως έτσι παραβιάζουμε την απόκρυψη πληροφορίας. Η απάντηση είναι ότι η “παραβίαση” δεν γίνεται αυθαίρετα από τον έξω κόσμο, η ίδια η κλάση δίνει το δικαίωμα. Είναι όπως εμείς μπορεί να έχουμε δώσει το κλειδί του σπιτιού μας σ’ έναν συγκεκριμένο γείτονα, για να ποτίζει τα φυτά μας κατά καιρούς, να “έχει τον νου” του όταν λείπουμε κλπ.

8.4.1 Φιλικές Συναρτήσεις

Μια κλάση μπορεί να δηλώσει μια συνάρτηση που δεν είναι συνάρτηση-μέλος της ως *φιλική* με τη χρήση της λέξης-κλειδιού `friend`. Τότε, η συνάρτηση αυτή, παρόλο που δεν ανήκει στην εμβέλεια της κλάσης, έχει δικαίωμα να έχει πρόσβαση στο ιδιωτικό μέρος της κλάσης. Αυτό είναι ιδιαίτερα χρήσιμο στην περίπτωση ορισμού συναρτήσεων που αποτελούν “εργαλεία” για τις κλάσεις, όπως π.χ. τελεστές και επαναλήπτες (iterators). Η δήλωση μπορεί να γίνει σε οποιοδήποτε σημείο μέσα στον ορισμό της κλάσης.

Ως παράδειγμα δήλωσης φιλικών συναρτήσεων, θεωρήστε πάλι το παράδειγμα της σελίδας 198. Όταν ορίζονται νέες κλάσεις, συχνά στη λειτουργικότητά τους, θέλουμε να έχουμε δυνατότητες εκτύπωσης για τα αντικείμενά τους. Ένας συνήθης τρόπος να επιτύχουμε αυτό είναι υπερφορτώνοντας τον τελεστή εξόδου, <<, ώστε να δέχεται στο δεξί μέλος του μεταβλητές του νέου αυτού τύπου. Όπως θα συζητήσουμε πιο στοχευμένα όταν θα μιλήσουμε για το σύστημα Εισόδου/Εξόδου της C++, η γενική μορφή του πρωτότυπου του τελεστή αυτού για μεταβλητές τύπου <Type> είναι:

```
ostream& operator<<(ostream&, const <Type>&)
```

Αρκεί, λοιπόν, να δώσουμε έναν ορισμό του για τον τύπο `<Type>` που μας ενδιαφέρει. Αντίθετα με τον τελεστή ανάθεσης, `=`, όμως, ο τελεστής εξόδου στο αριστερό του μέλος δέχεται ρεύμα εξόδου και όχι το αντικείμενο της κλάσης για την οποία θα επαναοριστεί. Επομένως, δεν μπορεί να δηλωθεί ως συνάρτηση-μέλος της κλάσης. Ο ορισμός θα πρέπει να δοθεί εκτός εμβέλειας κλάσης και, κατά συνέπεια, να χρησιμοποιηθούν μόνο τα δημόσια μέλη της, κάτι που και περιοριστικό είναι και, ενδεχόμενα μη επαρκές και, τελικά ίσως και μη αποδοτικό. Η λύση σε αυτήν την περίπτωση είναι ο τελεστής να δηλωθεί ως `friend` από την εμβέλεια της κλάσης `<Type>`.

Για παράδειγμα, για την υπερφόρτωση του τελεστή `<<` ώστε να δέχεται μεταβλητές του τύπου `Date`, η δήλωση φιλίας από την κλάση `Date` και ο ορισμός του μπορεί να είναι ως εξής:

Δήλωση φιλίας από την κλάση `Date`:

```
class Date {
    int day, month, year;

public:
    Date(int d, int m, int y) : day(d), month(m), year(y) {}

    int get_year() const
    { return year; }

    int get_month() const
    { return month; }

    int get_day() const
    { return day; }

    friend ostream& operator<<(ostream&, const Date&);
};
```

Ορισμός του τελεστή `<<` για μεταβλητές του τύπου `Date`

```
ostream& operator<<(ostream& os, const Date& date)
{
    os << date.day << " " <<
        date.month << " " <<
        date.year << endl;
    return os;
}
```

Ως πιο πολύπλοκο ορισμό του τελεστή `<<` για τις κλάσεις του παραδείγματος της “Οικογένειας”, παραθέτουμε έναν ορισμό του για αντικείμενα της κλάσης `Family`. Θεωρούμε τον ορισμό της κλάσης όπως διαμορφώθηκε στην προηγούμενη παράγραφο, δηλαδή έχοντας ορίσει τον τύπο `ChildrenN` ως εμφωλευμένο. Θεωρώντας δεδομένους τους ορισμούς για τους τελεστές:

```
ostream& operator<<(ostream&, const Date&)
ostream& operator<<(ostream&, const Person&)
ostream& operator<<(ostream&, const Car&)
```

ο ορισμός του τελεστή `<<` για αντικείμενα της κλάσης `Family` μπορεί να είναι ο εξής:

```
ostream& operator<<(ostream& os, const Family& f)
{ os << f.husband << "married with: "
  << f.wife << "on " << f.date_of_wedding;

  if (f.children == NULL )
    os << "and have no children" << endl;
  else {
    Family::ChildrenN* tmp = f.children;
    os << "have children: " << endl;
    while (tmp->next != NULL && tmp->person != NULL) {
      os << *(tmp->person); tmp = tmp->next; }
    if (tmp->person != NULL)
      os << *(tmp->person) << endl;
  }
  if (f.owned_car != NULL)
    os << "and bought the car :" << *(f.owned_car) << endl;
  return os;
}
```

Στον παραπάνω κώδικα, παρατηρήστε στον ορισμό:

```
Family::ChildrenN* tmp = f.children;
```

τον προσδιορισμό του ονόματος ChildrenN με χρήση του τελεστή επίλυσης εμβέλειας και το όνομα της κλάσης Family. Στην περίπτωση που ο τύπος ChildrenN δεν ήταν εμφωλευμένος, είχαμε, δηλαδή, τον αρχικό ορισμό της σελίδας 198, ο τύπος θα ανήκε στην καθολική εμβέλεια οπότε, στον παραπάνω ορισμό, θα αρκούσε μόνο το όνομά του για να προσδιοριστεί.

8.4.2 Φιλικές Κλάσεις

Ως φιλικές μπορούν να δηλωθούν και ολόκληρες κλάσεις. Αυτό γίνεται, πάλι, με τη χρήση της λέξης-κλειδιού friend. Π.χ.

```
class NewClass {
  // ...
  friend class FriendClass;
  // ...
};
```

Παρακάτω δίνουμε κάποια γενικά παραδείγματα περιπτώσεων δήλωσης φιλίας μεταξύ κλάσεων.

Μια περίπτωση δήλωσης φιλίας μεταξύ κλάσεων είναι εκείνη, κατά την οποία όλες οι συναρτήσεις-μέλη μιας κλάσης θα ήταν επιθυμητό να δηλωθούν ως φιλικές από μια άλλη, διότι αποτελούν συναρτήσεις “εργαλεία” για την κλάση αυτή.

Κάπως ανάλογη είναι η περίπτωση όπου έχουμε μια κλάση “Manipulator” των μελών μιας άλλης κλάσης.

```
class Manipulator {
  // ...
public:
  // <functions to use globally and which actually manipulate members
  // of any class that declares it "friend">
};
```

Δηλαδή, λειτουργεί πάνω στα μέλη της εφαρμόζοντας κάποιες διαδικασίες. Τότε, μια κλάση που επιθυμεί να της δώσει το δικαίωμα πρόσβασης να χειριστεί τα μέλη της, τη δηλώνει ως friend, π.χ.

```
class JustAClass {
    // ...
    friend class Manipulator;
    // ...
};
```

Πλέον, η κλάση Manipulator έχει πρόσβαση σε όλα τα μέλη της κλάσης JustAClass.

Κλείνοντας την αναφορά στη φιλία, ας κάνουμε κάποιες γενικές παρατηρήσεις σχετικά με αυτήν.

- Οι φιλικές συναρτήσεις δεν αποτελούν συναρτήσεις-μέλη της κλάσης (απλώς παίρνουν ένα αντικείμενο της κλάσης στις παραμέτρους τους).
- Δηλώσεις friend μπορούν να εμφανιστούν οπουδήποτε μέσα στον ορισμό μιας κλάσης (δεν επηρεάζει αν είναι μέσα στο private ή στο public μέρος τους)
- Η φιλία δεν είναι ούτε συμμετρική, ούτε μεταβατική μεταξύ των κλάσεων, ούτε κληρονομείται.

Παρότι η φιλία δεν κληρονομείται, μια συνάρτηση-μέλος την οποία κληρονομεί μια παραγόμενη κλάση από μια φιλική μιας τρίτης μπορεί να λειτουργήσει στο περιβάλλον της τελευταίας, χωρίς να παράγεται σφάλμα ορατότητας. Ο λόγος είναι ότι παρότι είναι απλά ορατή στην παραγόμενη κλάση, η συνάρτηση-μέλος, αφού κληρονομείται από τη βασική κλάση, βρίσκεται στην εμβέλεια της βασικής, η οποία έχει δηλωθεί ως φιλική.

Επιβεβαιώστε την παρατήρηση αυτή με το παρακάτω παράδειγμα:

```
class C;

class A {
public:
    void bla(C c);
};

class B : public A {};

class C {
    int i;
    friend class A;
public:
    C() : i(0) {};
};

int main()
{
    C c; A a;
    a.bla(c);

    B b; b.bla(c);

    return 0;
}
```

```
void A::bla(C c)
{ c.i++; }
```

8.5 Κλήση Συναρτήσεων: Πέρασμα Παραμέτρων και Επιστροφή Τιμής

Στη σελίδα 77, είχαμε αναφέρει ότι τόσο το πέρασμα παραμέτρων κατά την κλήση συνάρτησης, όσο και η επιστροφή τιμής από την κλήση της, οδηγούν σε διαδικασίες αρχικοποίησης των εμπλεκόμενων μεταβλητών. Μάλιστα, είχαμε επισημάνει τη διάκριση σε σχέση με τη διαδικασία ανάθεσης τιμών, λέγοντας ότι θα επανέλθουμε αργότερα στη διάκριση αυτή, όταν θα έχουμε ήδη μιλήσει σχετικά με τις κλάσεις.

Μετά από τις συζητήσεις που έχουμε κάνει στα προηγούμενα κεφάλαια, η διάκριση είναι σαφής. Όταν καλείται μια συνάρτηση, οι χώροι που έχουν φυλαχθεί για τις τυπικές παραμέτρους της αρχικοποιούνται χρησιμοποιώντας τα αντίστοιχα ορίσματα. Αντίστοιχα, μια εντολή `return` λειτουργεί σαν να αρχικοποιεί μια μεταβλητή του τύπου που επιστρέφει η συνάρτηση, με χρήση της τιμής της έκφρασης της `return`. Εξυπακούεται, λοιπόν, ότι γίνεται χρήση της συνάρτησης αρχικοποίησης μέσω αντιγραφής του τύπου του κάθε ορίσματος, ή του τύπου της επιστροφής, ανάλογα.

Για να γίνουν τα παραπάνω πιο καθαρά, ας θεωρήσουμε τον ορισμό της κλάσης `Student` της σελίδας 162.

8.5.1 Πέρασμα Παραμέτρων και Επιστροφή Τιμής κατά Τιμή

Θεωρήστε και τις παρακάτω συναρτήσεις:

```
void bla(Student s)
{
    cout << "Body of the bla function" << endl;
}

Student blaS(Student s) // C++11: better define move constructor
                        // and move assignment operator
{
    cout << "Body of the blaS function" << endl;
    return s;
}
```

Η συνάρτηση

```
void bla(Student s)
```

δέχεται ως όρισμα μια μεταβλητή τύπου `Student`. Το όρισμα περνάει κατά τιμή. Οπότε, σύμφωνα με αυτά που προαναφέραμε, κατά την κλήση της συνάρτησης, η νέα μεταβλητή τύπου `Student` που δημιουργείται αρχικοποιείται με χρήση της μεταβλητής `Student` που δίνεται ως τιμή κατά την κλήση. Η αρχικοποίηση γίνεται μέσω αντιγραφής.

Η συνάρτηση

```
Student blaS(Student s)
```

δέχεται ως όρισμα μια μεταβλητή τύπου `Student` και επιστρέφει μεταβλητή τύπου `Student`. Και οι δύο μεταβλητές αρχικοποιούνται μέσω αντιγραφής, η μία με χρήση της μεταβλητής που δίνεται ως τιμή κατά την κλήση και η άλλη με χρήση της μεταβλητής που εμφανίζεται στην εντολή:

```
return s;
```

δηλαδή το όρισμα της συνάρτησης.

Ας θεωρήσουμε τώρα την παρακάτω συνάρτηση main.

```
int main()
{
    Student s("First Last");
    cout << "This is the first output of the main function " << endl;

    s.set_no(100);
    s.set_year_of_studies(1);

    int year, no;

    year = s.get_year_of_studies();
    no = s.get_no();
    cout << "Year of studies is " << year
         << " for student with no " << no
         << " and name " << s.get_name() << endl;

    // Student scopied(s); // Student creation
    //
    //
    // bla(s);
    // blaS(s);

    // scopied = blaS(s); // Temporary objects are destroyed at the end
                        // of the full expression they are part of

    cout << "Just exiting the main function ...." << endl;

    return 0;
}
```

Αποσχολιάστε κατάλληλες γραμμές για να επιβεβαιώσετε τη συμπεριφορά που περιγράψαμε.

Στη συνάρτηση main, ενδιαφέρον έχουν οι κλήσεις:

bla(s); Κατά την κλήση αυτή, δημιουργείται ένα αντικείμενο της κλάσης Student, το οποίο αρχικοποιείται από το αντικείμενο του περιβάλλοντος της main. Η εκτύπωση που προκύπτει από την κλήση αυτή είναι:

```
A student was just created by copying ...
Body of the bla function
Deleting student with name First Last
```

Η τελευταία γραμμή προκύπτει από το σώμα της συνάρτησης καταστροφής της κλάσης Student, η οποία εκτελείται λόγω καταστροφής της μεταβλητής τύπου Student του ορίσματος της συνάρτησης bla.

blaS(s); Κατά την κλήση αυτή, δημιουργείται ένα αντικείμενο της κλάσης Student, το οποίο αρχικοποιείται από το αντικείμενο του περιβάλλοντος της main. Δημιουργείται και δεύτερο αντικείμενο της κλάσης Student, λόγω του τύπου επιστροφής της συνάρτησης blaS, το οποίο αρχικοποιείται μέσω αντιγραφής με τη χρήση της μεταβλητής τύπου Student που εμφανίζεται στην εντολή:


```
return s;
```

Η εκτύπωση που προκύπτει από την κλήση αυτή είναι:

```
A student was just created by copying ...
Body of the blaS function
A student was just created by copying ...
Deleting student with name First Last
Deleting student with name First Last
```

Παρατηρήστε την καταστροφή των δύο αντικειμένων μετά το τέλος της εκτέλεσης του σώματος της συνάρτησης.

`scopied = blaS(s);` Κατά την κλήση αυτή, ενδιαφέρον έχει να παρατηρήσετε τα παραπάνω σε συνδυασμό με την κλήση του τελεστή ανάθεσης. Τα προσωρινά αντικείμενα διατηρούνται έως την ολοκλήρωση της εκτέλεσης της εντολής. Η εκτύπωση που προκύπτει από την εντολή αυτή είναι:

```
A student was just created by copying ...
Body of the blaS function
A student was just created by copying ...
I just performed a student ASSIGNMENT ...
Deleting student with name First Last
Deleting student with name First Last
```

8.5.2 Πέρασμα Παραμέτρων και Επιστροφή Τιμής με Χρήση Αναφορών

Αν το πέρασμα παραμέτρων ή/και η επιστροφή τιμής από συνάρτηση γίνεται κατά αναφορά, δεν δημιουργούνται αντικείμενα, οπότε, προφανώς, δεν τίθεται θέμα αρχικοποιήσεων. Για να το επιβεβαιώσετε, θεωρήστε τις παρακάτω συναρτήσεις σε συνδυασμό με τη συνάρτηση `main` που ακολουθεί.

```
void bla(const Student& s)
{
    cout << "Body of the bla function" << endl;
}

Student& blaS(Student& s)
// Student& blaS(Student s) // CAREFUL: don't do this!
{
    cout << "Body of the blaS function" << endl;
    return s;
}

int main()
{
    Student s("First Last");
    // Student &rs = s; // Just to remind you about
    // the behavior of references
    cout << "This is the first output of the main function " << endl;
    int year, no;

    s.set_no(100);
```

```

s.set_year_of_studies(1);

year = s.get_year_of_studies();
no = s.get_no();
cout << "\nYear of studies is " << year
      << " for student with no " << no
      << " and name " << s.get_name() << endl;

Student scopied(s);

bla(s);
blaS(s);
scopied = blaS(s);

cout << "Just exiting the main function ...." << endl;

return 0;
}

```

Κατά την εκτέλεση του προγράμματος αυτού, τα μόνα αντικείμενα που δημιουργούνται είναι εκείνα του περιβάλλοντος της συνάρτησης `main`.

Με την ευκαιρία του παραπάνω παραδείγματος, να κάνουμε την εξής παρατήρηση: αν μια συνάρτηση επιστρέφει μια μεταβλητή κατά αναφορά, το ασφαλές είναι η μεταβλητή αυτή να υφίσταται στο περιβάλλον της καλούσας συνάρτησης. Για παράδειγμα, στην περίπτωση που αποσχολιάζαμε την επικεφαλίδα:

`Student& blaS(Student s)`

η κλήση της συνάρτησης αυτής επιστρέφει μια αναφορά σε “τοπική” μεταβλητή της συνάρτησης `blaS` το οποίο, στη γενική περίπτωση, δεν είναι ασφαλές.

8.5.3 Αναφορές *rvalue*, Αρχικοποίηση και Ανάθεση με Μεταφορά

Σε κάποιες περιπτώσεις για τις οποίες δεν είναι καθορισμένη η συμπεριφορά από το πρότυπο της C++, ανάλογα με την “έξυπνάδα” του μεταγλωττιστή, για λόγους βελτιστοποίησης, κατά την κλήση συνάρτησης ενδέχεται να μην δημιουργηθούν (προσωρινές) μεταβλητές αν δεν είναι απαραίτητο. Έτσι, το ίδιο πρόγραμμα ενδέχεται να συμπεριφέρεται διαφορετικά όταν εκτελείται σε διαφορετικούς υπολογιστές, όπου σε κάποια εκτέλεσιμα μπορεί να γίνονται πολλές περιττές αρχικοποιήσεις.

Από την έκδοση C++11, όμως, δίνεται η δυνατότητα στον προγραμματιστή να επιβάλει να μην γίνεται σπατάλη πόρων ακόμα και στις περιπτώσεις που δημιουργηθούν αυτές οι μεταβλητές. Αυτό επιτυγχάνεται με τον ορισμό μιας επιπλέον συνάρτησης αρχικοποίησης μέσω αντιγραφής και τον επαναορισμό του τελεστή ανάθεσης με τέτοιον τρόπο ώστε να πραγματοποιιούν *μετακίνηση* τιμών, αντί για *αντιγραφή*. Οι συναρτήσεις αυτές καλούνται στην περίπτωση που στα ορίσματά τους περνούν *rvalues*. Συνοπτικά, να πούμε ότι ως *rvalue* χαρακτηρίζεται οτιδήποτε μπορεί να χρησιμοποιηθεί στο δεξί μέρος μιας έκφρασης ανάθεσης και *lvalue* οτιδήποτε μπορεί να χρησιμοποιηθεί στο αριστερό.

Από την έκδοση C++11 και κατόπιν, με τον συμβολισμό `&&` μπορούμε να δηλώσουμε μια μεταβλητή ως αναφορά σε *rvalue*, κάτι που με την απλή αναφορά δεν ήταν δυνατό. Εκμεταλλευόμενοι τη δυνατότητα αυτή, μπορούμε, για μια κλάση `X`, να ορίσουμε μια επιπλέον συνάρτηση αρχικοποίησης με πρωτότυπο:

`X(X&&)`

καθώς και να υπερφορτώσουμε τον τελεστή ανάθεσης ορίζοντας τον:

`X& operator=(X&&)`

Οι συναρτήσεις αυτές ονομάζονται *συνάρτηση αρχικοποίησης με μεταφορά* (move constructor) και *τελεστής*

ανάθεσης με μεταφορά (move assignment operator), αντίστοιχα. Οι συναρτήσεις πρέπει να είναι έτσι ορισμένες, ώστε να πραγματοποιούν μετακίνηση πόρων από το αντικείμενο του ορίσματος στο αντικείμενο που εφαρμόζονται. Όταν έχουν οριστεί αυτές οι συναρτήσεις, κατά την εκτέλεση του προγράμματος μπορούμε να χειριστούμε με αποδοτικό τρόπο τα προσωρινά αντικείμενα που δημιουργούνται λόγω της κλήσης μιας συνάρτησης. Συνήθως, η ανάγκη προκύπτει για το αντικείμενο της επιστροφής της συνάρτησης.

Ας δούμε τα παραπάνω επαυξάνοντας την κλάση Student της σελίδας 162 ως εξής:

```
class Student {
private:
    char* name;
    int no;
    int year_of_studies;
public:
    Student(const char*);
    Student(const Student&);
    Student(Student&&);

    ~Student();

    Student& operator=(const Student& s);
    Student& operator=(Student&& s);

    void set_name(const char*);
    void set_no(int);
    void set_year_of_studies(int);

    char* get_name() const;
    int get_no() const;
    int get_year_of_studies() const;
};
```

Στο παρακάτω, συγκεντρώνουμε μόνο τους ορισμούς των συναρτήσεων αρχικοποίησης και καταστροφής καθώς και του τελεστή ανάθεσης. Οι υπόλοιποι ορισμοί των συναρτήσεων-μελών, καθότι δεν παρουσιάζουν κάποιο ενδιαφέρον ή αλλαγή, για λόγους οικονομίας παραλείπονται.

```
Student::~~Student()
{
    if (name != nullptr)
    { cout << "Deleting student with name " << name << endl;
      delete[] name; }
    else
        cout << "Deleting student with no name" << endl;
}

Student::Student(const char* nam)
{
    name = new char[strlen(nam)+1];
    strcpy(name,nam);
    cout << "A student was just constructed " << endl;
}
```

```

Student::Student(const Student& s)
{
    name = new char[strlen(s.name)+1];
    strcpy(name,s.name);
    // name = s.name;    Careful: this is the result of default copy
    no = s.no;
    year_of_studies = s.year_of_studies;
    cout << "A student was just created by copying ... " << endl;
}

Student::Student(Student&& s)
{
    name = s.name;    // We use the result of default copy
    no = s.no;
    year_of_studies = s.year_of_studies;

    s.name = nullptr;
    s.no = 0;
    s.year_of_studies = 0;
    cout << "A student was just created by moving ... " << endl;
}

Student& Student::operator=(const Student& s)
{
    if (this != &s) { // Careful not to assign to itself, especially
                    // if some storage reclaim was made within the body
        delete[] name;
        name = new char[strlen(s.name)+1];
        strcpy(name,s.name);
        no = s.no;
        year_of_studies = s.year_of_studies;
        cout << "I just performed a student ASSIGNMENT ... " << endl;
    }
    return *this;
}

Student& Student::operator=(Student&& s)
{
    if (this != &s) { // Careful: not perform "move" actions to itself
                    // --- C++11 std::move function may cause
                    //      *this to be considered an rvalue
        delete[] name;

        name = s.name;
        no = s.no;
        year_of_studies = s.year_of_studies;

        s.name = nullptr;

```

```

        s.no = 0;
        s.year_of_studies = 0;

        cout << "I just performed a student MOVE ASSIGNMENT ... " << endl;
    }

    return *this;
}

```

Προσέξτε τη χρήση τύπου αναφοράς σε rvalue στη νέα συνάρτηση αρχικοποίησης και στον υπερφορτωμένο ορισμό του τελεστή ανάθεσης. Επίσης, προσέξτε τη διαφορά στην υλοποίηση των σωμάτων τους: δεν φτιάχνουμε νέο αντίγραφο της συμβολοσειράς του ονόματος, αφού ο δείκτης του υπάρχοντος θα “μηδενιστεί”. (Σημείωση: ο “μηδενισμός” του δείκτη γίνεται με ανάθεση της τιμής nullptr, η οποία προσφέρεται από την έκδοση C++11 και η οποία είναι σταθερά τύπου δείκτη που αναπαριστάει τη μηδενική τιμή δεικτών). Συνοπτικά, να πούμε ότι στις συναρτήσεις αυτές, γίνεται αντιγραφή των τιμών του ορίσματος της συνάρτησης προς το αντικείμενο που εφαρμόζεται, αλλά με τη συμπεριφορά μεταφοράς: οι όποιοι πόροι “μεταφέρονται” από το αντικείμενο του ορίσματος και αυτό αποκτά μια κατάσταση με “μηδενικές” τιμές. Όπως καταλαβαίνουμε, λόγω των παραπάνω, απαιτείται να γίνει και η κατάλληλη περιπτωσιολογία στο σώμα της συνάρτησης καταστροφής.

Ας θεωρήσουμε τους ορισμούς των συναρτήσεων bla και blaS της Παραγράφου 8.5.1 και την παρακάτω συνάρτηση main.

```

int main()
{
    Student s("First Last");
    cout << "This is the first output of the main function " << endl;

    s.set_no(100);
    s.set_year_of_studies(1);

    int year, no;

    year = s.get_year_of_studies();
    no = s.get_no();
    cout << "Year of studies is " << year
        << " for student with no " << no
        << " and name " << s.get_name() << endl;

    // Student scoped(s); // Student creation
    //
    // bla(blaS(s)); // move constructor used

    // scoped = blaS(s); // Temporary objects are destroyed at the end
                        // of the full expression they're part of
                        // move assignment used

    cout << "Just exiting the main function ...." << endl;
}

```

```

    return 0;
}

```

Με βάση τον παραπάνω ορισμό της κλάσης `Student`, αποσχολιάζοντας στη συνάρτηση `main` την κλήση:
`bla(blaS(s));`

κατά την εκτέλεση, η εκτύπωση είναι:

```

A student was just constructed
This is the first output of the main function
Year of studies is 1 for student with no 100 and name First Last
A student was just created by copying ...
Body of the blaS function
A student was just created by moving ...
Body of the bla function
Deleting student with name First Last
Deleting student with no name
Just exiting the main function ....
Deleting student with name First Last

```

Από την κλήση

```

bla(blaS(s));

```

προκύπτουν οι γραμμές:

```

A student was just created by copying ...
Body of the blaS function
A student was just created by moving ...
Body of the bla function
Deleting student with name First Last
Deleting student with no name

```

Παρατηρήστε την εκτύπωση από τη συνάρτηση αρχικοποίησης με μεταφορά της κλάσης `Student` που καλείται όταν πρόκειται να δημιουργηθεί το αντικείμενο τύπου `Student` από την επιστροφή της κλήσης `blaS(s)` και το οποίο χρησιμοποιείται ως όρισμα κατά την κλήση της συνάρτησης `bla`. Επίσης, παρατηρήστε και την εκτύπωση της συνάρτησης καταστροφής της κλάσης `Student`, όταν καλείται κατά την καταστροφή του αντικειμένου του ορίσματος της κλήσης `blaS(s)`: το “όνομα” του αντικειμένου έχει μηδενιστεί.

Αν αποσχολιάσουμε τη γραμμή δημιουργίας του αντικειμένου `scopied`, δηλαδή τη γραμμή:

```

Student scopied(s);

```

καθώς και τη γραμμή στην οποία υπάρχει η ανάθεση:

```

scopied = blaS(s);

```

η εκτύπωση είναι η εξής:

```

A student was just constructed
This is the first output of the main function
Year of studies is 1 for student with no 100 and name First Last
A student was just created by copying ...
A student was just created by copying ...
Body of the blaS function
A student was just created by moving ...

```

```

I just performed a student MOVE ASSIGNMENT ...
Deleting student with no name
Deleting student with no name
Just exiting the main function ....
Deleting student with name First Last
Deleting student with name First Last

```

Στην παραπάνω εκτύπωση, προσέξτε τις γραμμές που προκύπτουν από την εντολή με την ανάθεση. Αυτές είναι:

```

A student was just created by copying ...
Body of the blaS function
A student was just created by moving ...
I just performed a student MOVE ASSIGNMENT ...
Deleting student with no name
Deleting student with no name

```

Η κλήση της συνάρτησης `blaS` αρχικοποιεί το αντικείμενο της επιστροφής της με μεταφορά. Η ανάθεση οδηγεί στη χρήση του ορισμού του τελεστή ανάθεσης που πραγματοποιεί μεταφορά. Έτσι, βλέπουμε τα δύο αντικείμενα που καταστρέφονται στο τέλος — αυτό του ορίσματος της συνάρτησης `blaS` και αυτό της επιστροφής της — να έχουν “μηδενικές” τιμές στο “όνομά” τους.

8.6 Προγραμματισμός Γενικεύσεων

Στο Κεφάλαιο 1, κάναμε μια σύντομη αναφορά στον Προγραμματισμό Γενικεύσεων, μια που σχετίζεται με ιδιότητες του αντικειμενοστραφούς προγραμματισμού, όπως η δυνατότητα ορισμού νέων τύπων και η υποστήριξη πολυμορφισμού. Ως προς το τελευταίο, το βασικό χαρακτηριστικό του προγραμματισμού γενικεύσεων είναι ότι μας επιτρέπει ο κώδικάς μας να έχει μια πολυμορφική συμπεριφορά.

Ο προγραμματισμός γενικεύσεων βασίζεται στην ιδέα της περιγραφής της γενικευμένης συμπεριφοράς ενός αλγορίθμου ή ενός τύπου, ανεξάρτητα από συγκεκριμένους τύπους δεδομένων για τα δεδομένα τα οποία εμπλέκει. Οι τελευταίοι πρέπει να πληρούν κάποιες απαιτήσεις προκειμένου να μπορούν να χρησιμοποιηθούν για να συγκεκριμενοποιηθεί η γενίκευση, αλλά πέρα από αυτό, η γενικευμένη συμπεριφορά δεν εξαρτάται από κάποιο άλλο χαρακτηριστικό τους. Οι τύποι αυτοί δίνονται ως παράμετροι στους ορισμούς των γενικευμένων συναρτήσεων ή των γενικευμένων τύπων. Κατά την ανάπτυξη του κώδικα ενός συγκεκριμένου προγράμματος, οι παράμετροι αυτές θα πρέπει να συγκεκριμενοποιηθούν από υπάρχοντες τύπους, προκειμένου να κατασκευαστεί ο πλήρης κώδικας που αντιστοιχεί στον ορισμό της συνάρτησης ή του τύπου. Ουσιαστικά, ο προγραμματιστής προσδιορίζει μόνο το “περίγραμμα” που περιγράφει τη συμπεριφορά. Στη συνέχεια, ο ορισμός θα συμπληρωθεί με την πλήρη πληροφορία των τύπων δεδομένων, μόνον όταν πρόκειται η γενικευμένη αυτή περιγραφή να χρησιμοποιηθεί για να οριστεί μια πραγματική συνάρτηση ή τύπος που θα χρησιμοποιηθεί στο πρόγραμμα.

Η υποστήριξη γενικεύσεων υπήρχε ήδη από παλαιότερες γλώσσες προγραμματισμού, π.χ. Ada, Eiffel. Η βασική ιδέα τους προέρχεται από τον συναρτησιακό προγραμματισμό. Πλέον, σιγά-σιγά, επεκτείνεται στις περισσότερες γλώσσες προγραμματισμού. Στη C++ δόθηκε η δυνατότητα υποστήριξης του προγραμματισμού γενικεύσεων με την ένταξη των *περιγραμμάτων* (*templates*) στη γλώσσα, το 1991. Βασικός υπέρμαχος του προγραμματισμού γενικεύσεων είναι ο Alexander Stepanov. Εκείνος ονομάτισε αυτό το είδος προγραμματισμού. Η πιο γνωστή συμβολή του, όμως, είναι ότι υλοποίησε σε C++ μια βιβλιοθήκη με ένα πλήθος τέτοιων γενικεύσεων. Η βιβλιοθήκη αυτή λεγόταν Standard Template Library (STL) [1].

Στη C++, ορίζοντας περιγράμματα συναρτήσεων ή κλάσεων με παραμέτρους τύπων, ορίζουμε ένα σύνολο σχετιζόμενων συναρτήσεων ή ένα σύνολο από σχετιζόμενες κλάσεις, αντίστοιχα. Κατά τη μεταγλώττιση, ανάλογα με τους τύπους που εμφανίζονται στη χρήση ενός περιγράμματος, παράγεται κώδικας για την κάθε μια

περίπτωση. Οι παράμετροι τύπων πρέπει να είναι μοναδικές στο κάθε περίγραμμα. Ο τρόπος να δηλώσουμε μια παράμετρο τύπου είναι με χρήση της λέξης-κλειδιού `typename` ή με χρήση της λέξης-κλειδιού `class`. Μπορούμε να θεωρήσουμε ότι και τα δύο είναι ισοδύναμα αν και το πρώτο σε κάποιες περιπτώσεις είναι πιο γενικό.

Έχουν προταθεί διάφορα μοντέλα μεταγλώττισης των περιγραμμάτων. Ένα λεπτό ζήτημα προς αντιμετώπιση αφορά την επίλυση ονόματος που χρησιμοποιείται μέσα σε ένα περίγραμμα: το νόημά του προέρχεται από την εμβέλεια του ορισμού ενός περιγράμματος ή της χρήσης του;

Ας δούμε λίγο τα περιγράμματα συναρτήσεων και κλάσεων πιο εστιασμένα.

8.6.1 Περιγράμματα Συναρτήσεων

Σε ένα περίγραμμα συνάρτησης, παράμετροι τύπων μπορούν να εμφανίζονται:

- στα τυπικά ορίσματα της συνάρτησης
- στον τύπο που επιστρέφει η συνάρτηση
- μέσα στο σώμα της συνάρτησης

Η πιο απλή γενική μορφή δήλωσης ενός περιγράμματος συνάρτησης είναι:

```
template <typename T, ...> <function header>
```

ή

```
template <class T, ...> <function header>
```

Παράγεται κώδικας μόνο για τις συναρτήσεις που καλούνται.

Ας δούμε ένα παράδειγμα ορισμού και χρήσης ενός περιγράμματος συνάρτησης. Έστω ο παρακάτω ορισμός περιγράμματος μιας συνάρτησης που απλά συγκρίνει τις τιμές μεταβλητών ενός (οποιοδήποτε) τύπου, `T`.

```
// File: mpls.h
template <typename T> bool is_smaller(T& v1, T& v2)
{
    if (v1 < v2)
        return true;
    else
        return false;
}
```

Το συγκεκριμένο περίγραμμα για να μπορεί να χρησιμοποιηθεί, η “απαίτηση” που έχει είναι να υπάρχει ορισμός του τελεστή `<` για τον τύπο που θα συγκεκριμενοποιήσει την παράμετρο `T`. Δηλαδή, επανερχόμενοι στο παράδειγμα της “Οικογένειας” της σελίδας 198, ας υποθέσουμε ότι για τις κλάσεις `Date` και `Person`, έχουμε επαναορίσει τον τελεστή `<` ως εξής:

Ορισμός συνάρτησης-μέλους < της κλάσης `Date`: Παλαιότερες ημερομηνίες θεωρούνται “μικρότερες”.

```
bool operator<(Date& d)
{
    if (year > d.year || (year == d.year && month > d.month)
        || (year == d.year && month == d.month && day >= d.day))
        return false;
    else
        return true;
}
```


Ορισμός συνάρτησης-μέλους < της κλάσης Person: Νεότερα πρόσωπα θεωρούνται ότι είναι “μικρότερα”.

```
bool operator<(Person& p)
{
    if (p.date_of_birth < date_of_birth)
        return true;
    else
        return false;
}
```

Τότε, η παρακάτω συνάρτηση main εκτελείται ομαλά.

```
#include "tpls.h"

int main()
{
    Person p1(1,1,1999,M);
    Person p2(1,1,2000,F);
    Person p3(1,2,2000,M);
    Person p4(5,2,2000,F);

    if (is_smaller(p1,p2))
        cout << "Test Success!" << endl;
    else
        cout << "Test Failed!" << endl;

    if (is_smaller(p3,p2))
        cout << "Test Success!" << endl;
    else
        cout << "Test Failed!" << endl;

    if (is_smaller(p1,p1))
        cout << "Test Success!" << endl;
    else
        cout << "Test Failed!" << endl;

    cout << "/////////////////////////////////" << endl;

    int i1 = 2; int i2 = 3;
    if (is_smaller(i1,i2))
        cout << "Test Success!" << endl;
    else
        cout << "Test Failed!" << endl;

    float f1 = 2.5; float f2 = 0.3;
    if (is_smaller(f1,f2))
        cout << "Test Success!" << endl;
    else
        cout << "Test Failed!" << endl;
```

```

char c1 = 'a'; char c2 = 'a';
if (is_smaller(c1,c2))
    cout << "Test Success!" << endl;
else
    cout << "Test Failed!" << endl;

return 0;
}

```

Η εκτύπωση που προκύπτει είναι η εξής:

```

Test Failed!
Test Success!
Test Failed!
////////////////////
Test Success!
Test Failed!
Test Failed!
Deleting Anonymous Person
Deleting Anonymous Person
Deleting Anonymous Person
Deleting Anonymous Person

```

Ένα ακόμα παράδειγμα μπορούμε να δούμε από τους ορισμούς της παλιομοδίτικης “Οικογένειάς” μας. Ας υποθέσουμε ότι ορίζουμε το μήκος μιας λίστας μέσω περιγράμματος, όπου ο τύπος του κόμβου είναι παράμετρος τύπου. Ο ορισμός θα μπορούσε να είναι ο εξής:

```

// File: mpls_length.h
template <typename T> int length(T* varN)
{
    int count = 0;
    if (varN != NULL) {
        T* tmpp = varN;
        do {
            count++; tmpp = tmpp->next;
        } while(tmpp != NULL); }
    return count;
}

```

Εδώ η “απαίτηση” που υπάρχει είναι να υπάρχει ορατό όνομα μέλους-δεδομένου `next` στην εμβέλεια του τύπου στον οποίο θα συγκεκριμενοποιηθεί η παράμετρος `T`.

Έχοντας το παραπάνω περίγραμμα διαθέσιμο, ο ορισμός της συνάρτησης που υπολογίζει και επιστρέφει το πλήθος των παιδιών μιας οικογένειας αρκεί να χρησιμοποιήσει το περίγραμμα αυτό. Οπότε, η συνάρτηση-μέλος `no_of_children` της κλάσης `Family` μπορεί να αλλάξει σε:

```
int no_of_children() const { return length(children); }
```

8.6.2 Περιγράμματα Κλάσεων

Παραμετρικοί τύποι μπορούν να οριστούν ως περιγράμματα κλάσεων. Στην περίπτωση αυτή, δίνονται γενικοί ορισμοί κλάσεων που στη συνέχεια θα συγκεκριμενοποιηθούν (όχι εξειδικευθούν) με βάση συγκεκριμένους

τύπους. Μια παρατήρηση: ένα πολύπλοκο θέμα που προκύπτει είναι η συμπεριφορά της κληρονομικότητας στα περιγράμματα κλάσεων.

Η πιο απλή γενική μορφή δήλωσης ενός περιγράμματος κλάσης είναι:

```
template <typename T, ... > <class definition>
```

ή

```
template <class T, ... > <class definition>
```

Στην επικεφαλίδα μιας κλάσης μπορούν να χρησιμοποιηθούν και παράμετροι που δεν είναι τύποι, π.χ.

```
template<class T, int elements>
class Stack {
// ...
}
```

τότε, η συγκεκριμενοποίησή του γίνεται, π.χ.

```
Stack<double,100> inst_stck;
```

Ο κώδικας που θα δημιουργηθεί από τη συγκεκριμενοποίηση του περιγράμματος αφορά μόνο τις συναρτήσεις-μέλη του περιγράμματος που χρησιμοποιούνται.

Στη συνέχεια, παραθέτουμε ένα παράδειγμα ορισμού ενός περιγράμματος κλάσης για μια απλή υλοποίηση στοίβας και χρήσης του.

Ορισμός του περιγράμματος:

```
// File: stackT.h
template<class T>
class StackT {
    static const int MAX_LENGTH = 100;

    int top_i;
    T stack[MAX_LENGTH];

public:
    StackT() : top_i(-1) {}

    bool is_empty() const
    { return top_i == 0; }

    bool is_full() const
    { return top_i == MAX_LENGTH - 1; }

    T top() const
    { return stack[top_i]; }

    bool push(const T& val)
    { if (!is_full()) {
        stack[++top_i] = val;
        return true;
    }
    else
        return false;
    }
}
```

```

    bool pop(T& val)
    {   if (!is_full()) {
            val = stack[top_i--];
            return true;
        }
        else
            return false;
    }
};

```

Χρήση του περιγράμματος: Συγκεκριμενοποίηση του περιγράμματος για στοίβα ακεραίων. Σχολιάστε τις γραμμές του βρόχου, ώστε να δείτε τον κώδικα assembly που παράγεται στις δύο περιπτώσεις: κλήση και των δύο συναρτήσεων push και pop, κλήση της συνάρτησης push και όχι της pop.

```

#include "stackT.h"
#include <iostream>
using namespace std;

int no_elements = 10;

int main()
{
    StackT<int> int_stackv;

    for (int i = 0; i < no_elements; i++)
        int_stackv.push(i);

    int val;
    // Add - remove comments to the following loop, to check whether
    // the assembly code for pop() is produced
    for (int k = 0; k < no_elements; k++) {
        int_stackv.pop(val);
        cout << val << endl;
    }

    return 0;
}

```

8.7 Τύποι Βιβλιοθήκης

Έχοντας κάνει την αναφορά στον προγραμματισμό γενικεύσεων, στην παράγραφο αυτή θα κάνουμε μια σύνοψη των τύπων βιβλιοθήκης που παρέχει η C++. Αυτοί είναι εμπνευσμένοι από ένα σύνολο των τύπων των οποίων βασικός εμπνευστής και προγραμματιστής ήταν ο Stepanov και τους οποίους παρείχε στη βιβλιοθήκη STL. Οι περισσότεροι από τους τύπους βιβλιοθήκης της C++ παρέχουν την ίδια λειτουργικότητα με αυτούς. Οι τύποι βιβλιοθήκης είναι Αφηρημένοι Τύποι Δεδομένων, οι οποίοι έχουν μια συγκεκριμένη συμπεριφορά και λειτουργικότητα, με στόχο να διευκολύνουν τον προγραμματιστή να γράφει κώδικα, ο οποίος επιπροσθέτως είναι πιο ευανάγνωστος, πιο ασφαλής αλλά και αποδοτικός. Στην πλειοψηφία τους, είναι γενικευμένοι τύποι. Να σημειώσουμε ότι δεν είναι όλοι οι τύποι βιβλιοθήκης υλοποιημένοι με χρήση περιγραμμάτων.

Οι τύποι βιβλιοθήκης της C++ “περιέχουν” στοιχεία άλλων τύπων, γι’ αυτό και ο αγγλικός τους όρος είναι *containers* - περιέκτες. Ο τύπος των στοιχείων αυτών, στη γενική περίπτωση, είναι παραμετρικός. Οι τύποι βιβλιοθήκης διακρίνονται σε δύο βασικές κατηγορίες:

- τους *ακολουθιακούς τύπους* (sequential containers)
- τους *τύπους συσχέτισης* (associative containers)

Στους ακολουθιακούς τύπους, η πρόσβαση στα στοιχεία τους γίνεται με βάση τη θέση τους στον (στιγμιότυπο του) περιέκτη. Στους τύπους συσχέτισης, η πρόσβαση γίνεται με χρήση ενός κλειδιού (key). Δηλαδή, θεωρούμε ότι υπάρχει μια συσχέτιση (κλειδί, τιμή) με βάση την οποία γίνεται η ανάκτηση της τιμής δίνοντας το κλειδί.

Στο σημείο αυτό, να κάνουμε μια αναφορά σε ένα “εργαλείο” που συνδέεται με την έννοια των περιεκτών. Αυτό είναι οι τύποι *επαναληπτών* (iterators). Οι περισσότεροι από τους τύπους βιβλιοθήκης συνδέονται με τύπους επαναληπτών που επιτρέπουν πρόσβαση στα στοιχεία τους. Μια αναλογία σχετικά με τους περιέκτες και τους επαναλήπτες τους είναι ότι οι δεύτεροι είναι για τους πρώτους, στο επίπεδο τύπων βιβλιοθήκης, ό,τι είναι για τους πίνακες οι δείκτες, στο επίπεδο βασικών τύπων της γλώσσας. Η χρήση των επαναληπτών στοχεύει στο να γίνει η πρόσβαση στα στοιχεία ενός περιέκτη μια ασφαλής διαδικασία. Οι πίνακες της γλώσσας αποτελούν το αποδοτικότερο μέσο για την πρόσβαση σε μια ακολουθία στοιχείων στη μνήμη, αλλά δεν είναι ασφαλές, μια που σχετίζεται με τα ίδια προβλήματα που σχετίζονται με τη χρήση δεικτών.

Οι πράξεις που μπορούμε να εφαρμόσουμε στους περιέκτες αφορούν εισαγωγή, διαγραφή και πρόσβαση στοιχείων, εξειδικευμένες κατά το είδος του περιέκτη. Επίσης, ανά είδος περιέκτη, ενδέχεται να υπάρχουν και επιπλέον εξειδικευμένες πράξεις. Μια πολύ καλή σύνοψη της λειτουργικότητας των περιεκτών μπορείτε να βρείτε στο [2].

Στην παράγραφο αυτή, απλά θα απαριθμήσουμε τους βασικούς τύπους βιβλιοθήκης της C++, τονίζοντας την κύρια σκοπιμότητά τους, μια που λεπτομέρειες για τη συμπεριφορά τους και τη χρήση τους μπορεί να δει κανείς στα περισσότερα εγχειρίδια της γλώσσας C++.

Ξεκινάμε από τους ακολουθιακούς τύπους. Στους τύπους αυτούς, τα στοιχεία τοποθετούνται μέσα στον περιέκτη με τη σειρά που προστέθηκαν. Η πρόσβαση σε ένα στοιχείο γίνεται προσδιορίζοντας τη θέση του.

vector: Ο τύπος αυτός είναι περιέκτης που αναπαριστάνει πίνακες των οποίων το μέγεθος μπορεί να μεγαλώνει σύμφωνα με τις τρέχουσες ανάγκες της εκτέλεσης του προγράμματος. Η φύλαξη των στοιχείων γίνεται ακολουθιακά σε συνεχή χώρο στη μνήμη, οπότε η πρόσβαση στα στοιχεία του μπορεί να γίνει και μέσω κανονικών δεικτών και εξίσου αποδοτικά. Όμως, η απαίτηση αυτή οδηγεί στο να χρειάζεται ενδεχόμενα επανατοποθέτηση των στοιχείων στην περίπτωση που ο ήδη δεσμευμένος χώρος δεν επαρκεί. Η διαδικασία αυτή προσθέτει κόστος, οπότε κάθε φορά που επαναλαμβάνεται, δεσμεύεται επιπλέον χώρος, ώστε να ελαχιστοποιηθεί η ανάγκη να επαναληφθεί στη συνέχεια.

array: Ο τύπος αυτός είναι περιέκτης που αναπαριστάνει πίνακες σταθερού μεγέθους. Αφού το μέγεθος δεν μπορεί να αλλάξει, ο περιέκτης δεν μπορεί να επεκταθεί ή να συρρικνωθεί, ανάλογα με τις τρέχουσες ανάγκες. Τα στοιχεία που περιέχει είναι γραμμικά διατεταγμένα.

deque: Το όνομα προκύπτει από το “double-ended queue” που εκφράζει ουρά με δύο άκρα. Στην ουσία, πρόκειται για έναν ακολουθιακό τύπο περιέκτη μεταβλητού μεγέθους, που μπορεί να επεκταθεί (ή να συρρικνωθεί) και από τα δύο άκρα. Υπ’ αυτήν την έννοια, παρέχει παρόμοια λειτουργικότητα με τον τύπο **vector**. Όμως δεν είναι εγγυημένο ότι τα στοιχεία του περιέκτη θα φυλαχθούν σε συνεχόμενες θέσεις στη μνήμη, άρα δεν είναι ασφαλές να γίνει πρόσβαση σε αυτά μέσω απόστασης από τη θέση της τιμής συγκεκριμένου δείκτη.

list: Ο τύπος αυτός είναι ακολουθιακός περιέκτης που επιτρέπει να γίνει εισαγωγή ή διαγραφή στοιχείου σε οποιαδήποτε θέση σε σταθερό χρόνο. Επίσης, μπορούμε να τον διατρέξουμε και προς τις δύο κατευθύνσεις. Είναι υλοποιημένος σαν διπλά συνδεδεμένη λίστα.

forward_list: Ο τύπος αυτός είναι ακολουθιακός περιέκτης που επιτρέπει να γίνει εισαγωγή ή διαγραφή στοιχείου σε οποιαδήποτε θέση σε σταθερό χρόνο. Όμως, μπορούμε να διατρέξουμε τα στοιχεία του μόνο κατά τη μία φορά. Είναι υλοποιημένος ως απλά συνδεδεμένη λίστα.

Στο σημείο αυτό, θα κάνουμε αναφορά σε κάποιους τύπους βιβλιοθήκης που είναι προσαρμογές των ακολουθιακών αυτών τύπων περιεκτών. Γενικά, οι προσαρμογές περιεκτών (*container adaptors*) είναι κλάσεις που υλοποιούνται με χρήση πιο γενικών περιεκτών, εγκλωβίζοντας την υλοποίησή τους και προσφέροντας ένα συγκεκριμένο σύνολο συναρτήσεων που αποτελεί τη συμπεριφορά τους.

Οι προσαρμογές ακολουθιακών περιεκτών είναι:

stack: Αναπαριστάνει τον ΑΤΔ “στοίβα” (δομή LIFO - Last-In-First-Out). Τα στοιχεία προστίθενται και αφαιρούνται μόνο από το ένα άκρο του περιέκτη.

queue: Αναπαριστάνει τον ΑΤΔ “ουρά” (δομή FIFO - First-In-First-Out). Τα στοιχεία προστίθενται από το ένα άκρο και αφαιρούνται από το άλλο.

priority_queue: Αναπαριστάνει τον ΑΤΔ “ουρά προτεραιότητας”. Είδος ουράς στην οποία το πρώτο στοιχείο είναι πάντα εκείνο με τη μεγαλύτερη προτεραιότητα, με βάση κάποια διάταξη.

Θα συνεχίσουμε τώρα με μια απαρίθμηση των βασικών τύπων συσχέτισης. Να υπενθυμίσουμε ότι η πρόσβαση στα στοιχεία των περιεκτών αυτών δεν γίνεται μέσω της θέσης τους στον περιέκτη, αλλά μέσω της τιμής ενός κλειδιού.

map: Ο τύπος αυτός είναι περιέκτης στοιχείων με τη μορφή ζευγαριών κλειδί-τιμή, όπου με βάση την τιμή του κλειδιού, ανακτούμε την τιμή του στοιχείου. Τα στοιχεία είναι διατεταγμένα με βάση μια διάταξη του κλειδιού. Η τιμή του κλειδιού προσδιορίζει μοναδικά μια τιμή στοιχείου.

multimap: Ο τύπος αυτός είναι περιέκτης στοιχείων παρόμοιος με τον περιέκτη **map**, με τη διαφορά ότι το κλειδί δεν καθορίζει μοναδικά μια τιμή στοιχείου του περιέκτη. Επιτρέπεται, δηλαδή, η ίδια τιμή κλειδιού να είναι συσχετισμένη με παραπάνω από μια τιμές στοιχείων. Η διαφορά αυτή έχει ως συνέπεια και άλλες διαφορές στη συμπεριφορά του.

set: Ο τύπος αυτός είναι περιέκτης στοιχείων που αναπαριστάνει την έννοια του “συνόλου”. Η χρήση του ενδείκνυται στην περίπτωση που μας ενδιαφέρει η ύπαρξη (ή όχι) ενός στοιχείου στον περιέκτη. Τα στοιχεία είναι μοναδικά. Η φύλαξή τους γίνεται με βάση κάποια διάταξη και είναι οργανωμένα με βάση ένα δυαδικό δένδρο αναζήτησης.

multiset: Ο τύπος αυτός είναι περιέκτης στοιχείων παρόμοιος με τον περιέκτη **set**, με τη διαφορά ότι οι τιμές των στοιχείων δεν είναι μοναδικές.

Υπάρχουν και μη διατεταγμένες εκδοχές των προηγούμενων τύπων.

Με βάση τις ανάγκες του συγκεκριμένου προβλήματος, γίνεται η επιλογή του τύπου, λαμβάνοντας υπόψη τη λειτουργικότητά του. Μια ενδιαφέρουσα σύνοψη για την καταλληλότητα του καθενός από τους παραπάνω τύπους γίνεται στο [3].

Στο σημείο αυτό, να αναφέρουμε την ύπαρξη ενός τύπου βιβλιοθήκης ο οποίος μας επιτρέπει να χειριζόμαστε ακολουθίες από bits, τον τύπο **bitset**. Αυτός παρέχει πράξεις σχετιζόμενες με bits. Επίσης, επιτρέπει μετατροπές από και προς ακεραίους, καθώς και συμβολοσειρές bits.

Τελευταίος στην επισκόπηση αλλά πρώτος από πλευράς έκτασης χρήσης είναι ο τύπος βιβλιοθήκης **string**. Ο τύπος αυτός αναπαριστάνει ακολουθίες χαρακτήρων και είναι ο τύπος που προτείνεται για την υψηλού επιπέδου και ασφαλή αναπαράσταση συμβολοσειρών σε ένα πρόγραμμα. Στα προηγούμενα κεφάλαια, χρησιμοποιούσαμε την προσέγγιση της γλώσσας C για την αναπαράσταση συμβολοσειρών, **char***, διότι θεωρείτο οικεία και διευκόλυνε στην ανάδειξη των θεμάτων που συζητούσαμε. Όμως, είχαμε ήδη αναφέρει την ύπαρξη κατάλληλου τύπου βιβλιοθήκης, ο οποίος, στη γενική περίπτωση, προτιμάται. Ο τύπος **string** συνοδεύεται από ένα σύνολο πράξεων πάνω σε συμβολοσειρές όπως: προσθήκη χαρακτήρα, διαγραφή χαρακτήρα ή χαρακτήρων, συνένωση συμβολοσειρών, αντικατάσταση μέρους συμβολοσειράς κ.ά.

8.8 Το Σύστημα Εισόδου/Εξόδου της C++

Στην παράγραφο αυτή, θα ασχοληθούμε με το σύστημα Εισόδου/Εξόδου (Input/Output - I/O) της C++. Παρότι, ούτως ή άλλως, είναι χρήσιμο να γίνει μια αναφορά στη λειτουργικότητά του, ο κύριος λόγος είναι η παρουσίαση των βασικών χαρακτηριστικών του σχεδιασμού του, καθότι αυτά ταιριάζουν απόλυτα με το αντικειμενοστραφές περιβάλλον. Επιπλέον, το ίδιο αποτελεί ένα πολύ καλό παράδειγμα κομψής αντικειμενοστραφούς μοντελοποίησης συστήματος.

Στις “παραδοσιακές” γλώσσες προγραμματισμού, το σύστημα I/O όφειλε να υποστηρίξει ένα συγκεκριμένο σύνολο από ενσωματωμένους τύπους. Στη C++ το σύνολο των τύπων είναι επεκτάσιμο από τους τύπους που ορίζει ο χρήστης. Η C++ υποστηρίζει δυνατότητες εισόδου/εξόδου με δύο τρόπους. Ο ένας είναι εκείνος της γλώσσας C και δεν θα ασχοληθούμε με αυτόν. Επιπλέον, όμως, προσφέρει και ένα δικό της σύστημα I/O, προσαρμοσμένο στις ανάγκες ενός αντικειμενοστραφούς περιβάλλοντος. Τα βασικά κριτήρια του σχεδιασμού του συστήματος I/O της C++ είναι να είναι εύκολο στη χρήση, πλήρες, αποτελεσματικό, αλλά και ασφαλές, στον χειρισμό τύπων, ευέλικτο και επεκτάσιμο.

Το σύστημα I/O της C++ αποτελείται από ένα σύνολο τύπων, αντικειμένων και συναρτήσεων βιβλιοθήκης. Όπως και στη γλώσσα C, λειτουργεί μέσω ρευμάτων (*streams*). Ένα ρεύμα είναι μια λογική συσκευή που, είτε παράγει είτε καταναλώνει πληροφορία. Ένα ρεύμα συνδέεται με μια φυσική συσκευή, μέσα από το σύστημα I/O. Στη C++, τα ρεύματα είναι στιγμιότυπα κλάσεων, οι οποίες έχουν συμπεριφορά αντίστοιχη με τη λειτουργικότητα των ρευμάτων. Μπορούμε να κάνουμε λειτουργίες I/O τόσο στις προκαθορισμένες μονάδες εισόδου/εξόδου, όσο και σε αρχεία ή στη μνήμη (in RAM I/O).

Έχουμε τη δυνατότητα για μορφοποιημένη I/O, καθώς και για I/O σε επίπεδο χαρακτήρων. Όσον αφορά τη μορφοποίηση εισόδου/εξόδου, αυτή επιτυγχάνεται με δύο εναλλακτικούς τρόπους:

1. Με χρήση συναρτήσεων-μελών μιας γενικής κλάσης ρευμάτων και χειρισμό σημαιών (*flags*).
2. Με χρήση καθολικών συναρτήσεων, οι οποίες ονομάζονται *διαμορφωτές* (*manipulators*) και λειτουργούν πάνω σε ρεύματα.

Γενικότερα οι διαμορφωτές συνδυάζονται με τον τελεστή εισόδου (>>) ή τον τελεστή εξόδου (<<) και επιτελούν διάφορες λειτουργίες πάνω στο ρεύμα που εφαρμόζονται, όπως μορφοποίηση, αλλαγή τιμών για τις σημαίες της κατάστασης του ρεύματος κ.ά.

8.8.1 Αντικειμενοστραφής Προσέγγιση του Συστήματος I/O

Το σύστημα I/O της C++ είναι υλοποιημένο μέσω μιας ιεραρχικής οργάνωσης κλάσεων, περιγραμμάτων κλάσεων και συγκεκριμενοποιήσεων τους (*template instantiations*). Στην οργάνωση αυτή, μάλιστα, συναντάμε ένα εξαιρετικό παράδειγμα πολλαπλής κληρονομικότητας.

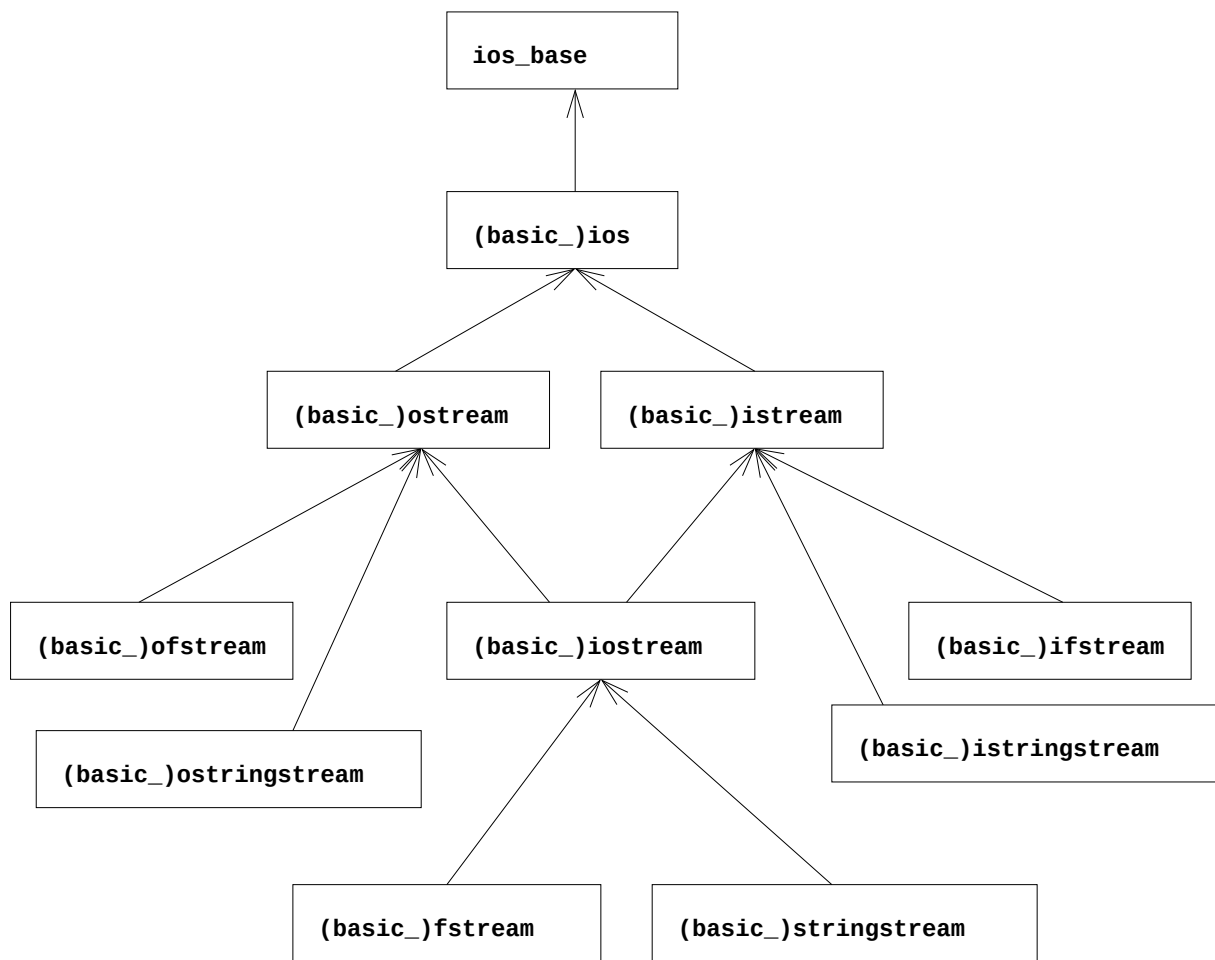
Μέσω των περιγραμμάτων κλάσεων, επιτυγχάνεται η λειτουργικότητα όλων των ρευμάτων του συστήματος με τρόπο ανεξάρτητο από συγκεκριμένο τύπο χαρακτήρων. Παρ’ όλα αυτά, λόγω της συνήθους χρήσης των τύπων `char` και `wchar_t`, προσφέρονται από τη βιβλιοθήκη και συγκεκριμενοποιήσεις των περιγραμμάτων για τους τύπους αυτούς.

Η εικόνα της οργάνωσης φαίνεται στο Σχήμα 8.2.

Η ρίζα της ιεραρχίας είναι μια κλάση, η `ios_base`, η οποία προσφέρει τη βασική λειτουργικότητα όλων των ειδών των ρευμάτων, π.χ. πληροφορίες και διαχείριση της κατάστασης ενός ρεύματος. Εξειδικεύσή της είναι το περίγραμμα `basic_ios`, που αντιστοιχεί στην έννοια του ρεύματος που συνδέεται σε μια φυσική συσκευή και ανάμεσά τους μεσολαβεί μια προσωρινή μνήμη φύλαξης (*buffer*). Η επικεφαλίδα της είναι:

```
template<class CharT, class Traits = std::char_traits<CharT>>
class basic_ios : public std::ios_base
```

Στη συνέχεια, στο σχήμα φαίνονται τα περιγράμματα που είναι εξειδικεύσεις. Τα ονόματα που αναγράφονται έχουν το μέρος `basic_` σε παρένθεση, διότι το όνομα χωρίς το πρόθεμα αυτό ονοματίζει τη συγκεκριμενο-



Σχήμα 8.2: Η ιεραρχία των κλάσεων του συστήματος Εισόδου/Εξόδου της C++

ποίηση του περιγράμματος για τον τύπο `char`, που είναι και ο πλέον συνηθισμένος κατά τη χρήση. Για παράδειγμα, για το περίγραμμα `basic_ios`, προσφέρεται από τη βιβλιοθήκη ο τύπος `ios`, ο οποίος είναι ορισμένος ως

```
basic_ios<char>
```

Ο τύπος `basic_ostream`, που είναι εξειδίκευση του τύπου `basic_ios`, είναι ο γενικός τύπος που αναπαριστάει ρεύματα εξόδου. Οι τύποι `basic_ofstream` και `basic_ostreamstream` είναι εξειδικεύσεις του για έξοδο σε αρχεία και συμβολοσειρές, αντίστοιχα.

Ο τύπος `basic_istream`, που είναι εξειδίκευση του τύπου `basic_ios`, είναι ο γενικός τύπος που αναπαριστάει ρεύματα εισόδου. Οι τύποι `basic_ifstream` και `basic_istreamstream` είναι εξειδικεύσεις του για είσοδο σε αρχεία και συμβολοσειρές, αντίστοιχα.

Ο τύπος `basic_iostream` είναι εξειδίκευση και του τύπου `basic_ostream` και του τύπου `basic_istream`. Άρα, έχουμε πολλαπλή κληρονομικότητα. Είναι ο γενικός τύπος που αναπαριστάει ρεύματα εισόδου/εξόδου. Εξειδικεύσεις του είναι οι τύποι `basic_fstream` και `basic_stringstream`, για είσοδο/έξοδο σε αρχεία και συμβολοσειρές, αντίστοιχα.

Όταν ένα πρόγραμμα C++ ξεκινά την εκτέλεσή του, “ανοίγουν” τα ενσωματωμένα ρεύματα:

<code>cin:</code>	Αντικείμενο της κλάσης <code>istream</code> . Συνδέεται με την προκαθορισμένη είσοδο.
<code>cout:</code>	Αντικείμενο της κλάσης <code>ostream</code> . Συνδέεται με την προκαθορισμένη έξοδο.
<code>cerr, clog:</code>	Αντικείμενα της κλάσης <code>ostream</code> . Συνδέονται με την προκαθορισμένη έξοδο σφαλμάτων (standard error).

Το πρώτο αντιστοιχεί στο πληκτρολόγιο και τα άλλα στην οθόνη, εκτός αν έχει γίνει ανακατεύθυνση.

8.8.1.1 Η Κλάση `ios_base`

Στο σημείο αυτό, θα δώσουμε ενδεικτικά κάποια σημεία από τη λειτουργικότητα της κλάσης αυτής. Η κλάση `ios_base`, μεταξύ άλλων στοιχειωδών λειτουργιών για τα ρεύματα, προσφέρει λειτουργίες για μορφοποιημένη είσοδο και έξοδο μέσω σημαιών. Παραδείγματα σημαιών μορφοποίησης είναι:

`showbase`: προκειμένου να εμφανίζεται η βάση του συστήματος αρίθμησης όταν εκτυπώνονται αριθμητικές τιμές

`showpos`: προκειμένου να εμφανίζεται το σύμβολο `+` όταν εκτυπώνονται θετικοί αριθμοί

`showpoint`: προκειμένου να εμφανίζεται η υποδιαστολή σε κάθε περίπτωση εμφάνισης αριθμών

`skipws`: προκειμένου να μη ληφθούν υπόψη κενοί χαρακτήρες κατά την είσοδο

`left/right/internal`: έξοδος στοιχισμένη αριστερά/δεξιά ή στα δύο άκρα με ενδιάμεσο “γέμισμα” (στις κατάλληλες περιπτώσεις)

`dec/oct/hex`: επιλογή της αντίστοιχης βάσης στο σύστημα αρίθμησης

...

Ο χειρισμός των σημαιών μορφοποίησης γίνεται με χρήση συναρτήσεων-μελών της κλάσης `ios_base`. Για παράδειγμα, η συνάρτηση `setf` χρησιμοποιείται για να θέσει τιμές, π.χ.

```
cout.setf(ios::hex);
cout.setf(ios::showbase);
```

Άλλο παράδειγμα είναι η συνάρτηση-μέλος `precision`, με την οποία καθορίζουμε την ακρίβεια των δεκαδικών ψηφίων κατά την έξοδο των πραγματικών. Για περισσότερες λεπτομέρειες σχετικά με την κλάση αυτή, μπορείτε να απευθυνθείτε στα σχετικά εγχειρίδια.

8.8.1.2 Έλεγχος της Κατάστασης Ρεύματος

Ανά πάσα στιγμή, κάθε ρεύμα βρίσκεται σε μια “κατάσταση”. Η κατάσταση αυτή μπορεί να ελεγχθεί μέσω των παρακάτω συναρτήσεων-μελών της `(basic_)ios`:

`bool good() const`: η επόμενη ενέργεια ενδέχεται να πετύχει

`bool eof() const`: η είσοδος έφτασε στο τέλος

`bool fail() const`: η επομένη ενέργεια θα αποτύχει

`bool bad() const`: το ρεύμα έχει χαλάσει

Όταν ένα ρεύμα χρησιμοποιείται σε μία συνθήκη, ουσιαστικά, ελέγχεται η κατάστασή του με βάση κλήση στη συνάρτηση `fail()`.

8.8.1.3 Ρεύματα Εξόδου

Στο περίγραμμα `basic_ostream`, ορίζεται ο τελεστής `<<` για να χειρίζεται την έξοδο των ενσωματωμένων τύπων της C++. Στη συνέχεια, για πρακτικούς λόγους, θα αναφερόμαστε στη συγκεκριμενοποίησή του `ostream`.

Για νέους τύπους ορισμένους από τον χρήστη, αρκεί ο χρήστης να ορίσει μια συνάρτηση

```
ostream& operator<<(ostream&, const <Type>&)
```

όπου `<Type>` ο νέος τύπος.

Ο τύπος επιστροφής του τελεστή << είναι αναφορά σε ostream, για να μπορεί ο τελεστής να εφαρμοστεί ξανά πάνω στο αποτέλεσμα της κλήσης του (ομαδοποίηση από αριστερά προς τα δεξιά).

Για παράδειγμα, στη σελίδα 322 δώσαμε ορισμούς του τελεστή << για τις κλάσεις Date και Family με τα παρακάτω πρωτότυπα:

```
ostream& operator<<(ostream&, const Date&)
ostream& operator<<(ostream&, const Family&)
```

Οι παραπάνω ορισμοί καλύπτουν όλες τις περιπτώσεις ρευμάτων που είναι στιγμιότυπα εξειδικεύσεων της κλάσης ostream, π.χ. καθιερωμένη έξοδο, αρχεία.

Η έξοδος σε επίπεδο χαρακτήρων επιτυγχάνεται μέσω ενός συνόλου συναρτήσεων-μελών του περιγράμματος (basic_)ostream, π.χ. put, write, flush, seekr κλπ.

8.8.1.4 Ρεύματα Εισόδου

Κατ' αναλογία με την έξοδο, στο περίγραμμα (basic_)istream, ορίζεται ο τελεστής >> για να χειρίζεται την είσοδο των ενσωματωμένων τύπων της C++.

Για νέους τύπους οριζόμενους από τον χρήστη, αρκεί ο χρήστης να ορίσει μια συνάρτηση

```
istream& operator>>(istream&, <Type>&)
```

όπου <Type> ο νέος τύπος.

Στην έκφραση εισόδου ή εξόδου, δεν δίνεται καμιά πληροφορία για τον τύπο της μεταβλητής. Το επιθυμητό αποτέλεσμα επιτυγχάνεται λόγω των υπερφορτωμένων ορισμών των τελεστών << και >>.

Δεδομένου ότι οι τελεστές << και >> επιστρέφουν ρεύμα εξόδου και ρεύμα εισόδου, αντίστοιχα, μπορούμε να τους χρησιμοποιήσουμε περισσότερες από μια φορά σε μια έκφραση. Για παράδειγμα, έστω ότι έχουμε τον παρακάτω κώδικα:

```
int i;
cin >> i;

float f;
cin >> f;

char c;
cin >> c;

char buf[10];
cin >> buf;

cout << "i = " << i << endl;
cout << "f = " << f << endl;
cout << "c = " << c << endl;
cout << "buf = " << buf << endl;
```

Το ίδιο επιτυγχάνουμε με το παρακάτω:

```
int i; float f;
char c; char buf[10];

cin >> i >> f >> c >> buf;

cout << "i = " << i << endl;
cout << "f = " << f << endl;
```

```
cout << "c = " << c << endl;
cout << "buf = " << buf << endl;
```

Η είσοδος σε επίπεδο χαρακτήρων επιτυγχάνεται μέσω ενός συνόλου συναρτήσεων-μελών του περιγράμματος (basic_)istream, π.χ. get, getline, read, ignore κλπ.

8.8.1.5 Είσοδος/Εξοδος επάνω σε Αρχεία

Η είσοδος από αρχείο και η έξοδος σε αρχείο γίνεται με χρήση των (συγκεκριμενοποιήσεων των) περιγραμμάτων basic_ifstream και basic_ofstream, αντίστοιχα (ή basic_fstream, αν πρόκειται για αρχείο πάνω στο οποίο θα γίνεται και είσοδος και έξοδος). Ό,τι μπορούμε να εφαρμόσουμε σε ένα στιγμιότυπο της (basic_)istream και της (basic_)ostream, μπορούμε να εφαρμόσουμε και σε στιγμιότυπα της (basic_)ifstream και (basic_)ofstream, αντίστοιχα, καθώς και στα στιγμιότυπα της (basic_)fstream, συνολικά. Μάλιστα, η λειτουργικότητα της (basic_)ios μπορεί να εφαρμοστεί και στα αρχεία.

Πιο εξειδικευμένα, τα ρεύματα που συνδέονται με αρχεία είναι στιγμιότυπα των κλάσεων ifstream, ofstream ή fstream. “Ανοίγουμε” ένα αρχείο συνδέοντάς το με ένα ρεύμα, με χρήση της συνάρτησης-μέλους open των basic_ifstream ή basic_ofstream ή basic_fstream, ανάλογα με τη λειτουργικότητα του αρχείου. Αν αποτύχει η open, το ρεύμα παίρνει την τιμή μηδέν. Αυτός είναι ο κλασικός τρόπος ελέγχου αν όλα πήγαν καλά στη διαδικασία — εναλλακτικά, μπορεί να χρησιμοποιηθεί η συνάρτηση is_open για το ρεύμα.

Επειδή οι κλάσεις ofstream, ifstream και fstream έχουν συναρτήσεις αρχικοποίησης που καλούν την αντίστοιχη συνάρτηση open, τις περισσότερες φορές δεν είναι ανάγκη να “ανοίγουμε” τα αρχεία ρητά. Αρκεί να κάνουμε, για παράδειγμα:

```
ifstream datastr("DATAFILE");
```

οπότε με αυτό ταυτόχρονα επιτύχαμε:

- δήλωση για ρεύμα datastr
- σύνδεση με το αρχείο DATAFILE
- “άνοιγμα” για διάβασμα

Για να “κλείσουμε” ένα αρχείο, χρησιμοποιούμε τη συνάρτηση-μέλος close(). Όμως, δεν είναι υποχρεωτικό να το κάνουμε ρητά, μια που αυτό γίνεται αυτόματα από τη συνάρτηση καταστροφής των ρευμάτων όταν βγαίνουμε από την εμβέλεια της δήλωσης του ρεύματος.

Ας δούμε ένα παράδειγμα εξόδου σε αρχείο. Ας υποθέσουμε ότι έχουμε ορίσει τον τελεστή << για όλες τις κλάσεις του παραδείγματος της “Οικογένειας” της σελίδας 198. Στο παρακάτω, μπορούμε να δούμε τη χρήση του ως παράδειγμα εξόδου μεταβλητών σε αρχείο.

```
Person h(1,3,1930,M);
Person w(2,6,1935,F);
Family f(1,1,1960,h,w);
////////////////////////////////////
///// Printing to file "outfile" /////
////////////////////////////////////
ofstream out("outfile");
if (!out) {
    cout << "Cannot open file.\n";
    return 1;
}
```

```
out << h << endl;
out << w << endl;
out << f << endl;
```

Προσέξτε να συμπεριλάβετε στον κώδικά σας το κατάλληλο αρχείο επικεφαλίδας, κάθε φορά που θέλετε να πραγματοποιήσετε είσοδο/έξοδο πάνω σε αρχεία.

8.8.2 Διαμορφωτές

Από την καθιερωμένη βιβλιοθήκη της C++, παρέχονται και συναρτήσεις καθολικής εμβέλειας που σχετίζονται με διαδικασίες εισόδου/εξόδου. Αυτοί είναι οι διαμορφωτές. Χρησιμοποιούνται για να γράφουμε στην ίδια γραμμή εισόδου ή εξόδου, μαζί με τη χρήση των αντίστοιχων τελεστών και τις λειτουργίες που θέλουμε να γίνουν. Για παράδειγμα, πολύ γνωστός είναι ο διαμορφωτής `endl` που προσθέτει τον χαρακτήρα αλλαγής γραμμής στη συμβολοσειρά προς έξοδο και ωθεί το περιεχόμενο της προσωρινής μνήμης φύλαξης προς την έξοδο (flushes the buffer). Παρότι προσφέρεται ένα σύνολο ενσωματωμένων διαμορφωτών, μπορούν να οριστούν και νέοι από τον χρήστη, λόγω υποστήριξης υπερφόρτωσης.

Παραδείγματα ενσωματωμένων διαμορφωτών με τον χαρακτηρισμό τους ως εισόδου ή εξόδου είναι:

`dec`: είσοδος/έξοδος δεδομένων σε δεκαδική μορφή (εισόδου/εξόδου)

`hex`: είσοδος/έξοδος δεδομένων σε δεκαεξαδική μορφή (εισόδου/εξόδου)

`oct`: είσοδος/έξοδος δεδομένων σε οκταδική μορφή (εισόδου/εξόδου)

`endl`: προσθέτει τον χαρακτήρα `'\n'` στο ρεύμα εξόδου και ωθεί το περιεχόμενο της προσωρινής μνήμης προς την έξοδο (εξόδου)

`ends`: προσθέτει τον χαρακτήρα `'\0'` στο ρεύμα εξόδου (εξόδου)

`flush`: ωθεί το περιεχόμενο της προσωρινής μνήμης εξόδου στην έξοδο (εξόδου)

`ws`: παράλειψη κενών χαρακτήρων (εισόδου)

`resetiosflags(long f)`: επαναφορά αρχικών τιμών σημαιών (εισόδου/εξόδου)

`setbase(int base)`: θέτει τη βάση του συστήματος αρίθμησης για την εκτύπωση ακεραίων (εξόδου)

`setfill(int ch)`: αλλαγή του χαρακτήρα “γεμίματος” στον χαρακτήρα `ch` (εξόδου)

`setiosflags(long f)`: θέση τιμών των σημαιών της κλάσης `ios_base` (εισόδου/εξόδου)

`setprecision(int p)`: θέτει την ακρίβεια στην έξοδο πραγματικών αριθμών (εξόδου)

8.9 Σύνοψη Κεφαλαίου

- Στη C++ μπορούμε να δηλώσουμε μέλη-δεδομένα κλάσεων ως σταθερές. Σε αυτήν την περίπτωση πρέπει να πάρουν τιμή κατά την αρχικοποίηση των αντικειμένων της, μέσω λίστας αρχικοποίησης.
- Μπορούμε να δηλώσουμε αντικείμενα μιας κλάσης ως σταθερές. Σε αυτήν την περίπτωση μπορούμε να τα χειριστούμε μόνο μέσω συναρτήσεων-μελών που έχουν δηλωθεί ως `const`.
- Τα στατικά μέλη κλάσεων είναι μέλη που δεν είναι απαραίτητο να εφαρμοστούν σε αντικείμενο της κλάσης, διότι είναι ανεξάρτητα αντικειμένου. Πρόκειται για συναρτήσεις ή μεταβλητές που απλά ανήκουν στην εμβέλεια της κλάσης.

- Τα στατικά μέλη-δεδομένα δεν συμμετέχουν στην κατασκευή αντικειμένων της κλάσης. Οι στατικές συναρτήσεις-μέλη δεν λειτουργούν πάνω σε αντικείμενο της κλάσης. Μπορούν να προσπελαστούν, χωρίς να χρησιμοποιηθεί αντικείμενο της κλάσης, προσδιορίζοντας την κλάση με τον τελεστή επίλυσης εμβέλειας.
- Τα στατικά μέλη καλούνται και μέλη κλάσης και είναι ιδιαίτερα σημαντικά για γλώσσες όπου όλος ο κώδικας πρέπει να είναι ενταγμένος σε κλάσεις, όπως στην Java.
- Το πέρασμα παραμέτρων κατά την κλήση συνάρτησης και η επιστροφή της τιμής της οδηγούν σε διαδικασίες αρχικοποίησης των εμπλεκόμενων μεταβλητών.
- Όταν ένα όρισμα πρόκειται να χρησιμοποιηθεί μόνο για ανάγνωση, καλό είναι να δηλωθεί ως αναφορά και να προσδιοριστεί ως `const`.
- Η χρήση αναφοράς κατά την επιστροφή τιμής είναι επικίνδυνη και πρέπει να χρησιμοποιείται μόνο σε περιπτώσεις που ο χώρος στον οποίο αναφέρεται η αναφορά ανήκει στο περιβάλλον της καλούσας συνάρτησης.
- Για λόγους οικονομίας πόρων, στις περιπτώσεις όπου ισχύει ο “κανόνας των 3”, ενδείκνυται να ορίζεται συνάρτηση αρχικοποίησης με μεταφορά και να ορίζεται ο τελεστής ανάθεσης για μεταφορά (“κανόνας των 5”). Αυτό γίνεται με χρήση αναφορών σε *rvalues* και προσφέρεται από την έκδοση C++11 (και τις μεταγενέστερες).
- Στη C++ υποστηρίζεται ο προγραμματισμός γενικεύσεων, με χρήση περιγραμμάτων. Στα περιγράμματα χρησιμοποιούνται παράμετροι τύπου, για να υποστηριχθεί η γενίκευση του αλγορίθμου ή/και του τύπου. Κατά τη χρήση τα περιγράμματα συγκεκριμενοποιούνται, δίνοντας τιμές στις παραμέτρους συγκεκριμένων τύπων.
- Μπορούμε να ορίσουμε περιγράμματα συναρτήσεων. Παράγεται εκτελέσιμος κώδικας μόνο για τις συγκεκριμενοποιήσεις των περιγραμμάτων συναρτήσεων.
- Μπορούμε να ορίσουμε περιγράμματα κλάσεων. Παράγεται εκτελέσιμος κώδικας μόνο για τις συναρτήσεις-μέλη που χρησιμοποιούνται.
- Η C++ προσφέρει ένα σύνολο τύπων δεδομένων ως *τύπους βιβλιοθήκης*. Οι τύποι αυτοί είναι περιέκτες στοιχείων, τα οποία, συνήθως, δεν ανήκουν σε συγκεκριμένο τύπο. Πρόκειται, δηλαδή, για γενικευμένους Αφηρημένους Τύπους Δεδομένων.
- Οι τύποι βιβλιοθήκης της C++ διακρίνονται σε δύο βασικές κατηγορίες: τους ακολουθιακούς τύπους και τους τύπους συσχέτισης.
- Στους ακολουθιακούς τύπους η πρόσβαση στα στοιχεία τους γίνεται με βάση τη θέση τους στον (στιγμιότυπο του) περιέκτη. Παραδείγματα είναι οι τύποι: `vector`, `array`, `deque`, `list` κ.ά.
- Στους τύπους συσχέτισης η πρόσβαση στα στοιχεία της μεταβλητής του τύπου γίνεται με χρήση ενός κλειδιού. Θεωρούμε ότι υπάρχει μια συσχέτιση με βάση την οποία γίνεται η ανάκτηση της τιμής του στοιχείου, δίνοντας την τιμή του κλειδιού. Παραδείγματα είναι οι τύποι: `map`, `multimap`, `set`, `multiset` κ.ά.
- Προσφέρονται από την καθιερωμένη βιβλιοθήκη της C++ προσαρμογές περιεκτών, όπου με χρήση πιο γενικών περιεκτών υλοποιείται συγκεκριμένη λειτουργικότητα. Προσαρμογές ακολουθιακών περιεκτών είναι οι τύποι: `stack`, `queue` και `priority_queue`.

- Η C++ υποστηρίζει δυνατότητες εισόδου/εξόδου όπως εκείνες της γλώσσας C. Επιπλέον, όμως, προσφέρει και ένα *σύστημα εισόδου/εξόδου κατάλληλο* για αντικειμενοστραφές περιβάλλον: ασφαλές, ευέλικτο και επεκτάσιμο.
- Το σύστημα I/O της C++ βασίζεται σε μια ιεραρχική οργάνωση τύπων που είναι είτε κλάσεις, είτε περιγράμματα κλάσεων, είτε συγκεκριμενοποιήσεις περιγραμμάτων.
- Οι διαδικασίες εισόδου/εξόδου θεωρούμε, όπως και στη γλώσσα C, ότι γίνονται πάνω σε ρεύματα. Τα ρεύματα είναι αντικείμενα των κλάσεων της ιεραρχίας. Οι λειτουργίες που μπορούμε να εφαρμόσουμε σε αυτά καθορίζονται από την κλάση τους και είναι εκείνες που είτε ορίζονται σε αυτή είτε κληρονομούνται από άλλες. Ρεύματα που θα χρησιμοποιηθούν τόσο για είσοδο, όσο και για έξοδο, είναι αντικείμενα κλάσεων με πολλαπλή κληρονομικότητα.
- Μπορούμε να κάνουμε είσοδο/έξοδο είτε στις καθιερωμένες μονάδες, είτε σε αρχεία, είτε σε συμβολοσειρές.
- Πέρα από τις συναρτήσεις-μέλη των κλάσεων, μπορούμε να χειριστούμε τα ρεύματα και με μια σειρά καθολικών συναρτήσεων που λέγονται διαμορφωτές. Οι διαμορφωτές συνδυάζονται με τους τελεστές εισόδου/εξόδου, >> και <<. Συμμετέχουν στην ίδια έκφραση, επηρεάζοντας την κατάσταση του ρεύματος στο οποίο εφαρμόζονται οι τελεστές.

8.10 Ασκήσεις

8.10.1 Αιτιολογώντας την απάντησή σας, δώστε το αποτέλεσμα της εκτέλεσης του παρακάτω προγράμματος:

```
#include <iostream>
using namespace std;

class B {
    static int data;
public:
    static void print() { cout << data << endl; }

    void vrt() const { cout << "B::vrt()" << endl; } };

class D1 : public B {
    static int data;
public:
    static void print() { cout << data << endl; }

    void vrt() const { cout << "D1::vrt()" << endl; } };

class D2 : public D1 {
public:
    void vrt() const { cout << "D2::vrt()" << endl; } };

int B::data = 10;
int D1::data = 100;
```

```

int main()
{
    D2 d2; d2.vrt(); d2.print();
    D1* pd1 = &d2; pd1->vrt(); pd1->print();
    B* pb = pd1; pb->vrt(); pb->print();

    return 0;
}

```

8.10.2 Να οριστούν οι τύποι της λίστας και της ουράς με χρήση εμφωλευμένων τύπων για τον τύπο των κόμβων τους.

8.10.3 Αιτιολογήστε τα αποτελέσματα της εκτέλεσης του παρακάτω προγράμματος.

```

#include <iostream>
using namespace std;

class A1 {
public:
    A1()
    { cout << "Constructing an A1" << endl; }

    A1(int i)
    { cout << "Constructing an A1 with :"
      << i << endl; }

    ~A1()
    { cout << "Destroying an A1" << endl; } };

class A2 {
public:
    A2()
    { cout << "Constructing an A2" << endl; }

    A2(int i)
    { cout << "Constructing an A2 with :"
      << i << endl; }

    ~A2()
    { cout << "Destroying an A2" << endl; } };

class B1 {
public:
    B1()
    { cout << "Constructing a B1" << endl; }

    ~B1()
    { cout << "Destroying a B1" << endl; } };

```

```

class A {
    A1 a1;
    A2 a2;
public:
    A()
    { cout << "Constructing an A" << endl; }

    A(int i, int j) : a2(i), a1(j)
    { cout << "Constructing an A" << endl; }

    ~A()
    { cout << "Destroying an A" << endl; } };

class B : public A {
    B1 b1;
public:
    B()
    { cout << "Constructing a B" << endl; }

    B(int i, int j) : A(i,j)
    { cout << "Constructing a B" << endl; }

    ~B()
    { cout << "Destroying a B" << endl; } };

void bloo(B* b)
{ cout << "Printing in bloo" << endl; }

void bla(B b)
{ cout << "Printing in bla" << endl;
  bloo(&b); }

int main()
{
    B object1;
    B object2(5,15);
    cout << "First printing in main " << endl;

    bla(object1);

    bloo(&object2);

    cout << "Last printing in main " << endl;

    return 0;
}

```


8.10.4 Έστω ότι έχουμε το παρακάτω πρόγραμμα:

```
#include <iostream>
using namespace std;

class Student {
public:
    Student() {}
    Student(const Student& s)
    { cout << "Just copied" << endl; } };

YYYY bla() {
    Student* ps = new Student;
    return XXXX; }

int main()
{
    bla();
    cout << "OK" << endl;
    return 0;
}
```

Θεωρούμε ότι μπορούμε να αντικαταστήσουμε το XXXX είτε με *ps είτε με το ps. Θεωρούμε ότι μπορούμε να αντικαταστήσουμε το YYYY είτε με το Student, είτε με το Student&, είτε με το Student*. Ποιοι συνδυασμοί είναι επιτρεπτοί και τι εκτυπώνεται στην κάθε περίπτωση;

8.10.5 Αιτιολογώντας την απάντησή σας, δώστε το αποτέλεσμα του παρακάτω προγράμματος C++:

```
#include <iostream>
using namespace std;

class Date {
    int day, month, year;
public:
    Date(int d = 0, int m = 0, int y = 0)
        : day(d), month(m), year(y)
    { cout << "Creating a Date!" << endl; }

    Date(const Date& anotherd)
        : day(anotherd.day), month(anotherd.month),
          year(anotherd.year)
    { cout << "Creating a Date by COPYING!" << endl; } };

class Student {
    Date RegistrationDate;
public:
    Student()
    { cout << "Creating a Student" << endl; }
```

```

    Student(const Student& s)
    { cout << "Just copied" << endl; } };

Student bla()
{ Student s1;
  Student& s2 = s1;
  Student* ps = new Student;
  return *ps; }

int main()
{
    bla();
    cout << "OK" << endl;
    return 0;
}

```

8.10.6 Έστω ότι έχουμε το παρακάτω πρόγραμμα:

```

#include <iostream>
using namespace std;

class A {
    int data;
public:
    A(int i) : data(i)
    { cout << "Constructing an A" << endl; }

    A(A& a) : data(a.data)
    { cout << "Copy Constructing an A" << endl; }

    int inc()
    { return data++; }

    int get_data()
    { return data; }
};

class Fat {
    A data;
public:
    Fat(int i) : data(i) {}

    YYY get_data()
    { return XXX; } };

int main()
{ Fat f1(5); Fat f2(f1);
  cout << f1.get_data().inc() << endl;
}

```

```

    if (f1.get_data().get_data() == f2.get_data().get_data())
        cout << "Equal!" << endl;
    else
        cout << "Different!" << endl;
    return 0;
}

```

Θεωρούμε ότι μπορούμε να αντικαταστήσουμε το XXX είτε με το data είτε με το &data. Θεωρούμε ότι μπορούμε να αντικαταστήσουμε το YYY είτε με το A, είτε με το A&, είτε με το A*. Ποιοι συνδυασμοί είναι επιτρεπτοί και τι εκτυπώνεται στην κάθε περίπτωση και γιατί; (υπονοείται ότι γίνονται και οι αντίστοιχες αλλαγές όπου χρειάζονται, π.χ. η τελεία . σε βέλος ->)

8.10.7 Έστω ότι έχουμε το παρακάτω πρόγραμμα:

```

#include <iostream>
using namespace std;

class A {
    int i;
public:
    A(int ival = 0): i(ival)
    { cout << "Costructing an A with: " << ival << endl; }

    A(const A& a): i(a.i)
    { cout << "Costructing an A by copying" << endl; }

    A& operator=(const A& a)
    { cout << "Do I really change anything?" << endl;
      return *this; }

    ~A()
    { cout << "Destructing an A with: " << i << endl; } };

A& bla(A& pa)
{ A blaa(5); A blab(pa); A blac = pa; A& blad = pa; blad = blaa;
  return XXXX; }

int main()
{
    A a; bla(a);
    return 0;
}

```

Με ποιο/α από τα blaa, blab, blac, blad είναι ασφαλές να αντικαταστήσουμε το XXXX και γιατί; Αντικαθιστώντας, πείτε τι εκτυπώνεται κατά την εκτέλεση του προγράμματος και δώστε το πρωτότυπο της συνάρτησης που καλείται κατά την εκτέλεση κάθε εντολής της main και της bla.

8.10.8 Αιτιολογώντας την απάντησή σας, δώστε το αποτέλεσμα της εκτέλεσης του εξής προγράμματος C++:

```

#include <iostream>
using namespace std;

```

```

class Block {
    int data;
public:
    Block(int dt = 0) : data(dt)
    { cout << "Creating a Block with " << data << endl; }

    Block(const Block& bk) : data(bk.data)
    { cout << "Creating a Block by Copying with "
      << bk.data << endl; }

    ~Block()
    { cout << "Destroying a Block with " << data << endl; }

    int get()
    { return data; }

    void set(int dt)
    { data = dt; } };

class B {
    Block& data1;
    Block data2;
    Block* data3;
public:
    B(Block& blk1, Block& blk2, Block* pblk3 = NULL)
      : data1(blk1), data2(blk2)
    { cout << "Creating a B " << endl;
      if (pblk3 != NULL)
        data3 = new Block(*pblk3);
      else
        data3 = NULL; }

    B(const B& b) : data1(b.data1), data2(b.data2)
    { cout << "Creating a B by copying " << endl;
      if (b.data3 != NULL)
        data3 = new Block(*(b.data3));
      else
        data3 = NULL; }

    ~B()
    { cout << "Destroying a B" << endl;
      if (data3 != NULL)
        delete data3; }

    void set(int dt)
    { data1.set(dt); data2.set(dt);
      if (data3 != NULL)
        data3->set(dt); }

```

```

    void print()
    {   cout << "The B data are " << data1.get() << " "
        << data2.get() << " " << data3->get() << endl; } };

B& first(B& b)
{   b.set(10); b.print(); return b; }

B second(B& b)
{   b.set(5); b.print(); return b; }

B third(B b)
{   b.set(1000); b.print(); return b; }

int main()
{
    Block block; B b1(block,block,&block);
    b1.print(); first(b1); second(b1); third(b1); block.set(300);
    return 0;
}

```

8.10.9 Αιτιολογώντας την απάντησή σας, δώστε το αποτέλεσμα της εκτέλεσης του εξής προγράμματος C++:

```

#include <iostream>
using namespace std;

class A {
    int data;
public:
    A(int dt = 0) : data(dt)
    {   cout << "Creating an A" << endl; }

    A(const A& a) : data(a.data)
    {   cout << "Creating an A by Copying" << endl; }

    ~A()
    {   cout << "Destroying an A with data " << data << endl; }

    void inc()
    {   data++; }

    A naughty1()
    {   return *this; }

    A& naughty2()
    {   return *this; } };

class BigA {
    A a;

```

```

    A& ra;
    A* pa;
public:
    BigA(const A& a1, A& a2, A* pa3=NULL)
        : a(a1), ra(a2), pa(pa3)
    { cout << "Constructing a BigA" << endl; }
    A& inc()
    { a.inc(); ra.inc();
      if (pa != NULL)
        pa->inc();
      return a; } };

int main()
{
    A data; BigA big_a(data,data,&data);
    big_a.inc();
    data.inc(); big_a.inc().naughty1();
    data.inc(); big_a.inc().naughty2();
    big_a.inc().naughty1().naughty2();
    big_a.inc().naughty2().naughty1();
    return 0;
}

```

8.10.10 Αιτιολογώντας την απάντησή σας, δώστε το αποτέλεσμα της εκτέλεσης του εξής προγράμματος C++:

```

#include <iostream>
using namespace std;

class A {
    int data;
public:
    A(int i = 10) : data(i)
    { cout << "An A was just constructed with " << i << endl; }

    ~A()
    { cout << "An A will be destroyed with " << data << endl; }

    void inc()
    { data++; }

    int get()
    { return data; }

    void print()
    { cout << "The A data is: " << data << endl; } };

class B1 {
    A data;

```

```

public:
    B1(const A& a) : data(a)
    { cout << "A B1 was just constructed " << endl; }

    ~B1()
    { cout << "A B1 will be destroyed with "
      << data.get() << endl; }

    B1 process()
    { data.inc(); return *this; }

    void print()
    { cout << "The B1 data is: " << data.get() << endl; } };

class B2 {
    A& data;
public:
    B2(A& a) : data(a)
    { cout << "A B2 was just constructed " << endl; }

    ~B2()
    { cout << "A B2 will be destroyed with "
      << data.get() << endl; }

    B2& operator=(const B2& b2)
    { cout<< "I am performing a B2 assignment" << endl;
      data = b2.data; return *this; }

    B2 process()
    { data.inc(); return *this; }

    void print()
    { cout << "The B2 data is: " << data.get() << endl; } };

class B3 {
    A* data;
public:
    B3(const A& a)
    { data = new A(a);
      cout << "A B3 was just constructed " << endl; }

    B3(const B3& b3)
    { data = new A(*(b3.data));
      cout << "A B3 was just constructed by copying " << endl; }

    ~B3()
    { cout << "A B3 will be destroyed " << endl;
      delete data; }
    
```

```

    B3& operator=(const B3& b3)
    {   cout<< "I am performing a B3 assignment" << endl;
        delete data; data = new A(*(b3.data)); return *this; }

    B3 process()
    {   data->inc(); return *this; }

    void print()
    {   cout << "The B3 data is: " << data->get() << endl; } };

class C {
    B1 data1;
    B2 data2;
    B3 data3;
public:
    C(const B1& b1, const B2& b2, const B3& b3)
        : data1(b1), data2(b2), data3(b3)
    {   cout << "A C was just constructed " << endl; }

    ~C()
    {   cout << "A C will be destroyed! " << endl; }

    B1 process1()
    {   return data1.process(); }

    B2 process2()
    {   return data2.process(); }

    B3 process3()
    {   return data3.process(); } };

int main()
{
    A a; B1 b1(a); B2 b2(a); B3 b3(a); C c(b1,b2,b3);
    a.print(); b1.print(); b2.print(); b3.print();
    b1 = c.process1(); b2 = c.process2();
    c.process2(); b3 = c.process3();
    cout << "/////////////////////////////////" << endl;
    a.print(); b1.print(); b2.print(); b3.print();
    return 0;
}

```

8.10.11 Δίνεται το παρακάτω πρόγραμμα C++:

```

#include <iostream>
using namespace std;

class A1 {
    int data;

```



```

public:
    A1(int i = 10) : data(i)
    { cout << "Constructing an A1 with: " << i << endl; }

    A1(const A1& a1) : data(a1.data)
    { cout << "Copy constructing an A1" << endl; }

    ~A1()
    { cout << "Destroying an A1 with: " << data << endl; }

    void change()
    { data = data * 10; } };

class A2 {
    int data;
public:
    A2(int i = 20) : data(i)
    { cout << "Constructing an A2 with: " << i << endl; }

    A2(const A2& a2) : data(a2.data)
    { cout << "Copy constructing an A2" << endl; }

    ~A2()
    { cout << "Destroying an A2 with: " << data << endl; }

    void change()
    { data = data * 20; } };

class A3 {
public:
    A3()
    { cout << "Constructing an A3" << endl; }

    A3(const A3& a3)
    { cout << "Copy constructing an A3?" << endl; }

    ~A3()
    { cout << "Destroying an A3" << endl; }

    void change()
    { cout << "Nothing to change" << endl; } };

class A {
    A1 a1;
    A2 a2;
    A3 a3;
public:
    A()
    { cout << "Constructing an A" << endl; }

```

```

A(const A& a) : a1(a.a1)
{ cout << "Copy constructing an A" << endl; }

~A()
{ cout << "Destroying an A" << endl; }

A& operator=(const A& a)
{ cout << "I am performing a stupid assignment between As"
  << endl;
  if (this != &a)
    a1 = a.a1;
  return *this; }

void change()
{ a1.change(); a2.change(); a3.change(); } };

class BigA {
  A data1;
  A& data2;
public:
  BigA(A& a) : data1(a), data2(a)
  { cout << "Constructing a BigA" << endl; }

  ~BigA()
  { cout << "Destroying a BigA" << endl; }

  A get(int index)
  { if (index == 1)
    return data1;
    else return data2; } };

// BigA volta(BigA& biga)
// BigA& volta(BigA& biga)
{ cout << "Volta ta data?" << endl;
  return biga; }

int main()
{
  A first; A second(first); A third = first;
  third.change(); second = third;

  BigA biga(first);
  volta(biga).get(2).change();

  return 0;
}

```

Αποσχοιάζοντας μια από τις δύο σχολιασμένες γραμμές τη φορά, δώστε το αποτέλεσμα της εκτέλεσης,

αιτιολογώντας την απάντησή σας.

8.10.12 Αιτιολογώντας την απάντησή σας, δώστε το αποτέλεσμα της εκτέλεσης του εξής προγράμματος C++:

```
#include <iostream>
using namespace std;

class A {
    int i;
public:
    A(int ii = 10) : i(ii)
    { cout << "Constructing an A with" << i << endl; }

    A(const A& a) : i(a.i)
    { cout << "Constructing an A by COPYING with "
      << i << endl; }

    ~A()
    { cout << "Destroying an A with " << i << endl; }

    int inc()
    { return ++i; }

    int get_data() const
    { return i; } };

class B {
    A& a;
public:
    B(A& aa) : a(aa)
    { cout << "Constructing a B with " << a.get_data() << endl; }

    B(const B& b) : a(b.a)
    { cout << "Constructing a B by COPYING with "
      << a.get_data() << endl; }

    ~B()
    { cout << "Destroying a B" << endl; }

    int inc()
    { return a.inc(); }

    int get_data() const
    { return a.get_data(); } };

class C {
    A a;
    B b;
public:
```

```

    C(const A& aa, B& bb) : a(aa), b(bb)
    { cout << "Constructing a C" << endl; }

    C(const C& c) : a(c.a), b(c.b)
    { cout << "Constructing a C by COPYING" << endl; }

    ~C()
    { cout << "Destroying a C" << endl; }

    void inc()
    { a.inc(); b.inc(); }

    B get_b()
    { return b; } };

class D {
    A* a;
    B b;
    C& c;
public:
    D(A& aa, const B& bb, C& cc) : a(&aa), b(bb), c(cc)
    { cout << "Constructing a D" << endl; }

    D(const D& d) : a(d.a), b(d.b), c(d.c)
    { cout << "Constructing a D by COPYING" << endl; }

    ~D()
    { cout << "Destroying a D " << endl; }

    void inc()
    { a->inc(); b.inc(), c.inc(); }

    B get_b()
    { return b; } };

int main()
{
    A a(20); B b(a); C c(a,b); D d(a,b,c);
    a.inc(); b.inc(); c.inc(); d.inc();
    c.get_b().inc(); d.get_b().inc();
    return 0;
}

```

8.10.13 Αιτιολογώντας την απάντησή σας, δώστε το αποτέλεσμα της εκτέλεσης του παρακάτω προγράμματος C++, αποσχολιάζοντας τις σχολιασμένες γραμμές κώδικα, μια κάθε φορά:

```

#include <iostream>
using namespace std;

```

```

class Block {
    int data;
public:
    Block(int datav = 0) : data(datav)
    { cout << "Constructing a Block with " << datav << endl; }

    ~Block()
    { cout << "Destructing a Block with: " << data << endl; }

    void double_it()
    { data = data * 2; } };

class B1 {
    Block* data;
public:
    B1(int value)
    { cout << "Constructing a B1 with value " << value << endl;
      data = new Block(value); }

    B1(const B1& b1)
    { data = new Block(*(b1.data)); }

    ~B1()
    { cout << "Destructing a B1 with block " << endl;
      delete data; } };

class B2 {
    Block data;
public:
    B2(int datav) : data(datav)
    { cout << "Constructing a B2 with value: " << datav << endl; }

    ~B2()
    { cout << "Destructing a B2 with block " << endl; }

    Block safe()
    { return data; }

    Block& unsafe()
    { return data; } };

class B {
    B1 data1;
    B2 data2;
public:
    B(int datav) : data2(datav), data1(datav)
    { cout << "Constructing a B with: " << datav << endl; }
    
```

```

~B()
{ cout << "Destructing a B" << endl; }

B2 reveal1()
{ return data2; }

B2& reveal2()
{ return data2; } };

int main()
{
    B ba(10);
    // B bb(ba);

    // ba.reveal1().safe().double_it();
    // ba.reveal1().unsafe().double_it();
    // ba.reveal2().safe().double_it();
    // ba.reveal2().unsafe().double_it();

    return 0;
}

```

8.10.14 Αιτιολογώντας την απάντησή σας, δώστε το αποτέλεσμα της εκτέλεσης του εξής προγράμματος C++:

```

#include <iostream>
using namespace std;

class Block {
    int data;
public:
    Block(int dat = 0) : data(dat)
    { cout << "Building a Block with data :" << dat << endl; }

    int inc()
    { return data++; }

    void add(int i)
    { data = data + i; }

    void print()
    { cout << "Block data is :" << data << endl; } };

class B {
    Block b1;
    Block& b2;
    Block* b3;
public:

```

```

B(Block& blk1, Block& blk2 , Block* pblk=NULL)
    : b1(blk1), b2(blk2), b3(pblk)
{   cout << "Constructing a B" << endl; }

void inc()
{   b1.inc(); b2.inc(); if (b3 != NULL) b3->inc(); }

void add(int i)
{   b1.add(i); if (b3 != NULL) b3->add(i); }

B naughty()
{   return *this; }

void print()
{   b1.print(); b2.print(); if (b3 != NULL) b3->print(); } };

int main()
{
    Block block(10); B b(block, block, &block); b.print();
    b.inc(); b.print(); block.print();
    b.add(3); b.print(); block.print();
    b.add(block.inc()); b.print(); block.print();
    b.naughty().inc(); b.print(); block.print();
    return 0;
}

```

8.10.15 Αιτιολογώντας την απάντησή σας, δώστε το αποτέλεσμα της εκτέλεσης του εξής προγράμματος C++:

```

#include <iostream>
using namespace std;

class Block {
    int data;
public:
    Block(int dat = 0) : data(dat)
    {   cout << "Constructing a Block" << endl; }

    void inc()
    {   data++; }

    void print()
    {   cout << "The Block data is: " << data << endl; } };

class B1 {
    Block b1;
    Block& b2;
    Block b3;
}

```

```

public:
    B1(Block& blk1, Block& blk2, Block& blk3)
        : b1(blk1), b2(blk2), b3(blk3)
    { cout << "Constructing a B1" << endl; }

    void inc()
    { b1.inc(); b2.inc(); b3.inc(); }

    void print()
    { b1.print(); b2.print(); b3.print(); } };

class B2 {
    Block* pb1;
    Block* pb2;
    Block* pb3;
public:
    B2(Block* pblk1 = NULL, Block* pblk2 = NULL, Block* pblk3 = NULL)
        : pb1(pblk1), pb2(pblk2), pb3(pblk3)
    { cout << "Constructing a B2" << endl; }

    void inc()
    { if (pb1 != NULL)
        pb1->inc();
      if (pb2 != NULL)
        pb2->inc();
      if (pb3 != NULL)
        pb3->inc(); }

    void print()
    { if (pb1 != NULL)
        pb1->print();
      if (pb2 != NULL)
        pb2->print();
      if (pb3 != NULL)
        pb3->print(); } };

class B {
    B1 b1;
    B2 b2;
public:
    B(B1& db1, B2& db2) : b1(db1), b2(db2)
    { cout << "Constructing a B" << endl; }

    void inc()
    { b1.inc(); b2.inc(); }

    void print()
    { b1.print(); b2.print(); }

```



```
B tricky()
{ inc(); return *this; } };

int main()
{
    Block block(10); B1 b1(block, block, block);
    B2 b2(&block, &block, &block); B b(b1,b2);
    block.print(); b1.print(); b2.print(); b.print();

    block.inc(); b1.inc(); b2.inc(); b.inc();
    block.print(); b1.print(); b2.print(); b.print();

    b.tricky().inc();
    block.print(); b1.print(); b2.print(); b.print();

    return 0;
}
```

8.10.16 Ορίστε τον τελεστή << για τους τύπους που έχετε ορίσει στην Άσκηση 8.10.2.

8.10.17 Ορίστε τους τύπους λίστας και ουράς που έχετε ορίσει στην Άσκηση 8.10.2 με χρήση περιγραμμάτων.

8.11 Βιβλιογραφία

- [1] P.J. Plauger, Meng Lee, David Musser και Alexander A. Stepanov. *C++ Standard Template Library*. Pearson, 2000.
- [2] Bjarne Stroustrup. *Programming: Principles and Practice Using C++*. 2η έκδοση. Addison-Wesley Professional, 2014.
- [3] Stanley B. Lippman, Josée Lajoie και Barbara Moo. *C++ Primer*. 5η έκδοση. Addison-Wesley Professional, 2012.
- [4] Bruce Eckel. *Thinking in C++, Volume I: Introduction to Standard C++*. 2η έκδοση. Prentice Hall, 2000.
- [5] C++ reference. <https://en.cppreference.com/w/>. Ανάκτηση: 2023-5-30.
- [6] cplusplus.com. <http://www.cplusplus.com/>. Ανάκτηση: 2023-5-30.
- [7] Bjarne Stroustrup. *The C++ Programming Language*. 4η έκδοση. Addison-Wesley Professional, 2013.
- [8] Stanley B. Lippman. *Inside the C++ Object Model*. Addison-Wesley Professional, 1996.

ΜΕΤΑΒΑΣΗ ΣΤΗ ΓΛΩΣΣΑ JAVA

Σύνοψη

Έχοντας παρουσιάσει την υποστήριξη του αντικειμενοστραφούς προγραμματισμού μέσω της C++, στο κεφάλαιο αυτό θα γίνει η μετάβαση στη γλώσσα Java. Αρχικά, θα γίνει ανασκόπηση των ιστορικών στοιχείων της γλώσσας, καθώς και των γενικών χαρακτηριστικών και της λειτουργικότητάς της. Η εισαγωγή θα συμπληρωθεί με το περιβάλλον εκτέλεσης των προγραμμάτων της και κάποιες γενικές παρατηρήσεις πάνω σε αυτά. Στα πλαίσια αυτών, θα αναφέρουμε τις διαβαθμίσεις ορατότητας και τα είδη κλάσεων που υποστηρίζονται στην Java. Θα ακολουθήσει η παρουσίαση στοιχείων από τον διαδικαστικό προγραμματισμό, όπως μεταβλητές και τύποι. Η έμφαση θα είναι στις διαφορές με τη C++. Θα προχωρήσουμε με θέματα που αφορούν το αντικειμενοστραφές μοντέλο. Αρχικά, θα συζητήσουμε για τα είδη αρχικοποιήσεων στην Java. Θα συνεχίσουμε με την κληρονομικότητα, οπότε θα γίνει αναφορά στις αφηρημένες κλάσεις και στους τύπους διεπαφής (interfaces). Σημαντικό βάρος του κεφαλαίου θα δοθεί στη διάκριση μεταξύ μελών στιγμιοτύπου και μελών κλάσης. Θα συμπεριλάβουμε ακόμα την παρουσίαση της κατάτμησης κώδικα μέσω πακέτων (packages). Θα κλείσουμε κάνοντας σύντομες επισημάνσεις σχετικά με κάποια επιπλέον λειτουργικότητα της Java, προκειμένου ο αναγνώστης να την έχει υπόψη του.

Προαπαιτούμενη Γνώση

Για την κατανόηση του κεφαλαίου, ο αναγνώστης απαιτείται να έχει γνώσεις διαδικαστικού προγραμματισμού και να έχει μελετήσει τις έννοιες του αντικειμενοστραφούς προγραμματισμού και τη γλώσσα C++ από τα προηγούμενα κεφάλαια του βιβλίου.

9.1 Ιστορικά Στοιχεία της Γλώσσας Java

Θα ξεκινήσουμε δίνοντας κάποια στοιχεία για την προέλευση της γλώσσας Java. Αυτό κρίνεται απαραίτητο, όχι μόνο για λόγους γενικότερης εγκυκλοπαιδικής κατάρτισης, αλλά και λόγω του ότι μέσω αυτών θα επισημάνουμε τους σχεδιαστικούς της στόχους, τι ήλθε, δηλαδή, η Java να εξυπηρετήσει. Με τον τρόπο αυτό,

θα εντοπίσουμε τα βασικά χαρακτηριστικά της, τα οποία μπορεί να αποτελούν το κριτήριο στην επιλογή της ως γλώσσα υλοποίησης μιας εφαρμογής. Όπως έχει ήδη τονιστεί, στο βιβλίο αυτό δίνεται ιδιαίτερη έμφαση στην κατανόηση και την ανάπτυξη κριτικής σκέψης, με απώτερο στόχο αυτά να οδηγήσουν σε μια ώριμη και αναλυτική στάση απέναντι σε οποιοδήποτε πρόβλημα ή λύση. Το να έχουν, λοιπόν, συζητηθεί τα παραπάνω συμβάλλει στον στόχο αυτό.

Προκειμένου να παρουσιάσουμε την ιστορία της προέλευσης της Java, θα χρησιμοποιήσουμε στοιχεία που ο ίδιος ο βασικός σχεδιαστής της, ο James Gosling, έχει αναφέρει σε σχετικές παρουσιάσεις και συνεντεύξεις του [1, 2, 3].

Στις αρχές της δεκαετίας του '90, στην SUN Microsystems, μια ομάδα μηχανικών και τεχνοκρατών, στην οποία συμμετείχε ο James Gosling συνειδητοποίησε ότι η χρήση υπολογιστών ως εξάρτημα ηλεκτρονικών συσκευών έχει διαδοθεί ευρέως από τις αντίστοιχες επιχειρήσεις, κυρίως στην Ασία. Αυτό σήμαινε ότι υπήρχε έντονη δραστηριότητα σχετική με υπολογιστές και η οποία λάμβανε χώρα εκτός του χώρου των επιχειρήσεων των υπολογιστών. Η ομάδα ξεκίνησε να διερευνά τις προσπάθειες των επιχειρήσεων των ηλεκτρονικών κάνοντας ταξίδια για επισκέψεις στις επιχειρήσεις αυτές και συζητήσεις με μηχανικούς τους. Κατά τη διερεύνησή τους, διαπίστωσαν ότι οι τελευταίοι κατέληγαν στα ίδια αποτελέσματα ή έκαναν τα ίδια λάθη που είχαν ήδη προκύψει από τις προσπάθειες που είχαν προηγηθεί από τον χώρο των υπολογιστών και κυρίως της **δικτύωσής** τους (computer networking). Επομένως, υπήρχε έντονη ανάγκη συμβολής των μηχανικών των υπολογιστών στον χώρο της ανάπτυξης σύγχρονων ηλεκτρονικών συσκευών. Η χρήση δικτύων ήταν στα βασικά “πιστεύω” της SUN, κατά συνέπεια οι επιστήμονες τους οποίους προσέλκυε, όπως ο Gosling, είχαν την ίδια άποψη και είχαν πολύ καλή γνώση του χώρου αυτού, επομένως είχαν πολλά να προσφέρουν στο αναδυόμενο πεδίο εφαρμογής της.

Κατά τη διερεύνηση, μια ακόμα διαπίστωση που έκανε η ομάδα ήταν η δέσμευση των ηλεκτρονικών συσκευών με τον επεξεργαστή που χρησιμοποιείτο. Ενώ κάθε άλλο ανταλλακτικό ενός προϊόντος, π.χ. πυκνωτές, μπαταρίες κ.ά., μπορούσε να προέρχεται από οποιαδήποτε εταιρεία ή/και μπορούσε να γίνει αλλαγή προμηθευτή του ανταλλακτικού ανά πάσα στιγμή, αυτό δεν ίσχυε για τους επεξεργαστές. “Πριν καν γραφεί η πρώτη γραμμή κώδικα” για το προϊόν, η επιχείρηση που το παρήγαγε δεσμευόταν απόλυτα ως προς τον προμηθευτή του επεξεργαστή του. Ακόμα και στην περίπτωση που ο προμηθευτής δεν άλλαζε, θα μπορούσε να αλλάξει ριζικά η μορφή των επεξεργαστών που παράγει κάνοντάς το δύσκολο για το προϊόν να ακολουθήσει την εξέλιξη αυτή. Κατά συνέπεια, η **μεταφερισιμότητα** (portability) του κώδικα ως προς το περιβάλλον εκτέλεσης ήταν υψίστης σημασίας για τα προϊόντα αυτά.

Το ζητούμενο, λοιπόν, ήταν η εκτέλεση ενός προγράμματος να μην εξαρτάται από συγκεκριμένη αρχιτεκτονική ενός επεξεργαστή. Αυτό μπορεί να επιτευχθεί, αν ο κώδικας αναπτύσσεται για μια **εικονική μηχανή** (virtual machine) η οποία, και μόνο, να έχει υλοποιηθεί για διάφορους επεξεργαστές. Με τον τρόπο αυτό, επιτυγχάνεται η μεταφερισιμότητα του κώδικα, η εκτέλεση κώδικα σε κατανεμημένα ετερογενή περιβάλλοντα, αλλά και η δυνατότητα ανάπτυξης κώδικα για “μηχανές που ακόμα δεν έχουν ανακαλυφθεί”. Κατά συνέπεια, ο ίδιος κώδικας μπορεί να εκτελείται σε διαφορετικά περιβάλλοντα, π.χ. με διαφορετικές αριθμητικές, και όμως πάντα να παράγει τα ίδια αποτελέσματα.

Όσον αφορούσε το θέμα αυτό, ο Gosling ήταν εκείνος που υιοθέτησε την ιδέα της εικονικής μηχανής. Αυτή προέκυψε από την εμπειρία που είχε από τα μεταπτυχιακά του χρόνια σχετικά με μια αντίστοιχη μηχανή που υλοποίησε, η οποία είχε ως κώδικα μηχανής τις εντολές p-code. Τα χρόνια εκείνα, ο επιβλέπων του είχε προγράμματα γραμμένα σε USD pascal, τα οποία έτρεχαν σε μηχανήματα των οποίων το υλικό (hardware) εκτελούσε εντολές p-code της USD pascal. Κάποια στιγμή, όμως, τα προγράμματα αυτά έπρεπε να μεταφερθούν σε νέα μηχανήματα, DEC VAX. Αναζητώντας την πιο “συμπαγέμενη” λύση, ο Gosling οδηγήθηκε στη δημιουργία μιας εικονικής μηχανής η οποία να τρέχει στα DEC VAX μηχανήματα και να μπορεί να εκτελεί εντολές p-code. Η προσέγγιση αυτή, λοιπόν, φάνηκε να έχει εφαρμογή και στην περίπτωση των προβλημάτων του χώρου των ηλεκτρονικών.

Ένα κρίσιμο θέμα για τον χώρο των ηλεκτρονικών συσκευών είναι το θέμα της **ασφάλειας**. Δεν υπάρχει ανοχή στα σφάλματα του λογισμικού, μια που ένα σφάλμα κατά την εκτέλεση κώδικα σε μια τέτοια συσκευή μπορεί να είναι μοιραίο. Αν ένα σφάλμα, για παράδειγμα, οδηγεί στο να πάρει φωτιά μια τοστιέρα, τότε μπορεί

να εξελιχθεί σε πυρκαγιά, με ό,τι αυτό συνεπάγεται. Από την άλλη, σε υπάρχουσες τεχνολογίες, όπως της C και της C++, όπου η ταχύτητα εκτέλεσης έχει μεγαλύτερη προτεραιότητα σε σχέση με την ασφάλεια, η πιθανότητα σφάλματος δεν είναι μηδενική και μάλιστα οδηγεί σε βεβαιότητα αυτό να συμβεί μέσα σε ένα πεπερασμένο χρονικό διάστημα.

Όσον αφορά το προγραμματιστικό πλαίσιο, οι αντικειμενοστραφείς προσεγγίσεις, όπως της γλώσσας C++, έδειχνε να είναι ιδανικές για την ανάπτυξη των διαφόρων τμημάτων των κατανεμημένων εφαρμογών. Προσφέρουν μια αυστηρή μεθοδολογία σχετικά με τις διεπαφές και κάνουν σαφές το πώς τα συστατικά ενός συστήματος συνδέονται το ένα με το άλλο. Όμως, οι υπάρχουσες τεχνολογίες είχαν προβλήματα. Πολλά από τα σφάλματα κατά την εκτέλεση, και μάλιστα αυτά που είναι δύσκολο να εντοπιστούν, οφείλονταν σε ανεξέλεγκτη συμπεριφορά δεικτών. Επίσης, λόγω έλλειψης περισυλλογής σκουπιδιών υπήρχαν γενικότερα προβλήματα μνήμης και διαχείρισής της. Τέλος, δεν ήταν εύκολη η υλοποίηση των κατανεμημένων συστημάτων και, βέβαια, δεν προσεφέρετο η επιθυμητή μεταφερσιμότητα. Έτσι, “μια C++” που να ικανοποιεί τα προηγούμενα έδειχνε να είναι η κατάλληλη τεχνολογία για να αναπτύσσονται εφαρμογές που να λειτουργούν σε ένα πλήθος ετερογενών ή/και κατανεμημένων συσκευών.

9.1.1 Το Χρονικό μέχρι την Πρώτη Έκδοση της Java

Οι φάσεις που ακολούθησαν σχετικά με τη δραστηριότητα της ομάδας στη SUN Microsystems, που αποτελείτο κυρίως από τους James Gosling, Mike Sheridan και Patrick Naughton, έχουν ως εξής.

1990-1992: Από το 1990, άρχισε η δραστηριότητα ενός έργου εμπνευσμένου από τις διαπιστώσεις που αναφέρθηκαν στην προηγούμενη παράγραφο, το Green Project. Ο στόχος του ήταν, με βάση τις διαπιστώσεις αυτές, να αναπτυχθούν νέες τεχνολογίες που θα διευκόλυναν τον προγραμματισμό για ευφυείς συσκευές της επόμενης γενεάς. Αυτό διήρκεσε έως το 1992. Βασικός συντελεστής του έργου ήταν ο James Gosling. Ο ίδιος χαρακτηρίζει το έργο αυτό ως έργο εκμάθησης και έρευνας, ώστε να γίνουν καλύτερα κατανοητά τα προβλήματα που αναφέρθηκαν προηγουμένως και να σχεδιαστούν λύσεις. Βασικό αποτέλεσμα της προσπάθειας αυτής ήταν η γλώσσα Oak (1992). Για την ιστορία, το όνομα του έργου, “Green”, οφειλόταν στο χρώμα της πόρτας χώρου της ομάδας και το όνομα “Oak” από μια βελανιδιά που φαινόταν από το παράθυρο του γραφείου του Gosling, όνομα που χρησιμοποίησε για να ονοματίσει τον κατάλογο που τοποθετούσε υλικό για το έργο αυτό. Στο πλαίσιο του έργου, κατασκευάστηκε και μιας μορφής tablet, το Star7, για να λειτουργεί σαν προσωπικός ψηφιακός βοηθός (Personal Digital Assistant, PDA), και στο οποίο εκτελείτο ο κώδικας της γλώσσας. Το σύστημα διεπαφής του ήταν βασισμένο στα γραφικά (graphical user interface). Ο Gosling είχε ήδη εμπειρία στα παραθυρικά συστήματα επικοινωνίας, λόγω προηγούμενης ενασχόλησής του τόσο στην SUN, όσο και, παλαιότερα, στην IBM.

1992-1994: Το προϊόν του έργου, όμως, χαρακτηριζόταν “demo”, δεν ήταν εμπορικό προϊόν. Το 1992, μια εταιρία καλωδιακής τηλεόρασης φάνηκε να έχει ανάγκη από ένα προϊόν του οποίου οι προδιαγραφές ταίριαζαν απόλυτα στα χαρακτηριστικά του προϊόντος της ομάδας και να ταυτίζεται με τις απόψεις της σχετικά με τις λύσεις των προβλημάτων με τα οποία ασχολείται. Τον Σεπτέμβριο του 1992, λοιπόν, άρχισε η μετάβαση από ένα δοκιμαστικό προϊόν σε ένα εμπορικό. Κατά τη διάρκεια της ανάπτυξης, όμως, έγινε μια καθοριστική διαπίστωση: παρ’ όλες τις αρχικές προδιαγραφές που είχε θέσει η επιχείρηση, στην ουσία, αυτό που επιθυμούσε στην πράξη δεν συμφωνούσε με τη βασική λειτουργικότητα του προϊόντος. Συγκεκριμένα, ο τομέας της διαφήμισης (advertising) ήταν υψίστης σημασίας για την επιχείρηση, ενόσω η βασική λειτουργικότητα όφειλε να εξυπηρετεί τη δημοσιοποίηση περιεχομένου (publish content).

Ταυτόχρονα, στα μέσα της δεκαετίας του ’90, η χρήση του Παγκόσμιου Ιστού και του διαδικτύου είχε ευρέως διαδοθεί. Έτσι, το 1994, η ομάδα άλλαξε τον προσανατολισμό της και από ανάπτυξη προϊόντος για τις επιχειρήσεις καλωδιακής σύνδεσης, μεταστράφηκε σε ανάπτυξη τεχνολογιών και προϊόντων για το διαδίκτυο και τον Παγκόσμιο Ιστό.

1994-1995: Το διάστημα αυτό ήταν καθοριστικό για τα αποτελέσματα της ομάδας. Η προσέγγισή τους επανασχεδιάστηκε για το διαδίκτυο. Οι τεχνολογίες τους δεν στόχευαν σε ανάπτυξη “desktop εφαρμογών” αλλά “web εφαρμογών”.

Στις αρχές του 1995, στο συνέδριο TED, μετά από παρότρυνση του John Gage, συναδέλφου του Gosling στην SUN, κατά τη διάρκεια ομιλίας του πρώτου, έγινε από τον Gosling επίδειξη πλοηγού (browser) ο οποίος εκτελούσε applet. Επρόκειτο για ένα τρισδιάστατο μοντέλο μορίου το οποίο μπορούσε να περιστραφεί με κλικάρισμα του χρήστη. Μέχρι τότε, οι πλοηγοί εμφάνιζαν απλές σελίδες, στις οποίες ο χρήστης μπορούσε να μεταβεί από τη μία στην άλλη, κλικάροντας. Στην καλύτερη περίπτωση, οι ιστοσελίδες περιείχαν εικόνες. Κατά την επίδειξη αυτή, παρουσιάστηκε για πρώτη φορά πλοηγός να εμφανίζει σελίδες των οποίων το περιεχόμενο ήταν κινούμενο και μπορούσε να αλληλεπιδράσει με τον χρήστη. Η δυνατότητα να διαμοιράζεται μέσω διαδικτύου δυναμικό και διαδραστικό περιεχόμενο και να προβάλεται μέσω πλοηγού ήταν πραγματικότητα. Σε συνδυασμό με μια εντυπωσιακή ομιλία από τον Gage, η απήχηση ήταν τεράστια.

Η πρώτη έκδοση της Java έγινε το 1995. Το όνομα επιλέχθηκε από ένα σύνολο από εναλλακτικά που είχε προτείνει η ομάδα και το οποίο είχε τη νομική δυνατότητα να υιοθετηθεί. Συνδυάζοντας ένα ωριμότερο και καθαρότερο αντικειμενοστραφές μοντέλο, μεταφερσιμότητα, χαρακτηριστικά κατάλληλα για την ανάπτυξη κατανεμημένων εφαρμογών (διαδικτύου) και ευκολία ανάπτυξης φιλικών διεπαφών χρήστη, η αποδοχή και η διάδοσή της ήταν εντυπωσιακές και είναι γνωστές σε όλους.

Το 2009 η Oracle αγόρασε την SUN Microsystems και έτσι η Java διατίθεται πλέον από την Oracle.

9.2 Γενικά Χαρακτηριστικά της Γλώσσας

Έχοντας προηγηθεί μεγάλο χρονικό διάστημα από την πρώτη εμφάνιση αντικειμενοστραφών χαρακτηριστικών σε γλώσσα προγραμματισμού και λόγω της διάδοσης της χρήσης της C++, η λειτουργικότητά τους είχε ωριμάσει. Η Java είναι μια *αντικειμενοστραφής γλώσσα προγραμματισμού* που υιοθέτησε μια πιο κατασταλαγμένη μορφή των χαρακτηριστικών αυτών και ένα πλήρως αντικειμενοστραφές πλαίσιο. Η έκφραση κληρονομικότητας έγινε πιο καθαρή και άλλα χαρακτηριστικά, όπως οι δυνατότητες για αρχικοποιήσεις αντικειμένων, εμπλουτίστηκαν. Σε ένα πρόγραμμα Java, τα πάντα ανήκουν σε κλάσεις. Οι κλάσεις αυτές δομούν μια ιεραρχία, η οποία έχει ως βασική κλάση την κλάση `Object`, η οποία προσφέρεται από τη γλώσσα. Όλες, λοιπόν, οι κλάσεις, είτε οι προσφερόμενες από τη γλώσσα είτε εκείνες που ορίζει ο προγραμματιστής, είναι υποκλάσεις της κλάσης `Object`.

Σε πολλά σημεία, η σύνταξη της Java ακολουθεί εκείνη της C++. Οι διαδικαστικές γλωσσικές δομές τους έχουν απόλυτη συνάφεια. Όμως, για λόγους ασφάλειας, όπως συζητήθηκε προηγουμένως, δίνονται πολύ λιγότερες δυνατότητες διαχείρισης χαμηλού επιπέδου. Η διαχείριση της μνήμης γίνεται από τη γλώσσα και ο προγραμματιστής ούτε πρέπει, ούτε μπορεί να επέμβει ρητά. Μη δίνοντας τη δυνατότητα διαχείρισης δεικτών, η ποσότητα λαθών περιορίζεται και μάλιστα λαθών που δεν είναι εύκολο να εντοπιστούν. Έτσι, παρότι ο χρόνος ανάπτυξης του πρωτότυπου ενός συστήματος ενδέχεται να μην είναι μικρότερος, ο χρόνος μέχρι την παράδοση του προϊόντος μειώνεται σημαντικά. Πέρα από τη C++, ο σχεδιασμός της γλώσσας είναι επηρεασμένος από γλώσσες όπως η Eiffel και η SmallTalk.

Η Java ξεκίνησε ως μια νέα τεχνολογία προγραμματισμού για έξυπνες συσκευές. Κατέληξε ως πλαίσιο κατάλληλο για ανάπτυξη εφαρμογών για *κατανεμημένα περιβάλλοντα* σε/ή *ετερογενείς πλατφόρμες* που διευκολύνει την *επικοινωνία μεταξύ προγραμμάτων*. Τα προγράμματα Java μπορούν να επεκταθούν δυναμικά ή να συνδεθούν και να επικοινωνήσουν εύκολα μεταξύ τους μέσω δικτύου. Συνοπτικά, όπως δηλώνει ο ίδιος ο Gosling, η Java κατασκευάστηκε για το περιβάλλον που τώρα αποκαλούμε “*Διαδίκτυο των Πραγμάτων*” (Internet of Things - IoT). Η εκτέλεση ενός προγράμματος Java είναι ανεξάρτητη από επεξεργαστή. Το πιο διαφημιζόμενο χαρακτηριστικό της, πράγματι, είναι ότι ένα πρόγραμμα Java, μπορεί να “τρέξει” σε οποιονδήποτε επεξεργαστή — για τον οποίο έχει γίνει υλοποίηση της εικονικής μηχανής που θα παρουσιάσουμε στη συνέχεια. Για

τον λόγο αυτό, η SUN είχε υιοθετήσει και προβάλλει τη φράση: “*Write Once, Run Anywhere*” η οποία, όμως, κατά τον Gosling, υπεραπλουστεύει την πραγματικότητα, κάτι που καλό είναι να έχει κανείς υπόψη του.

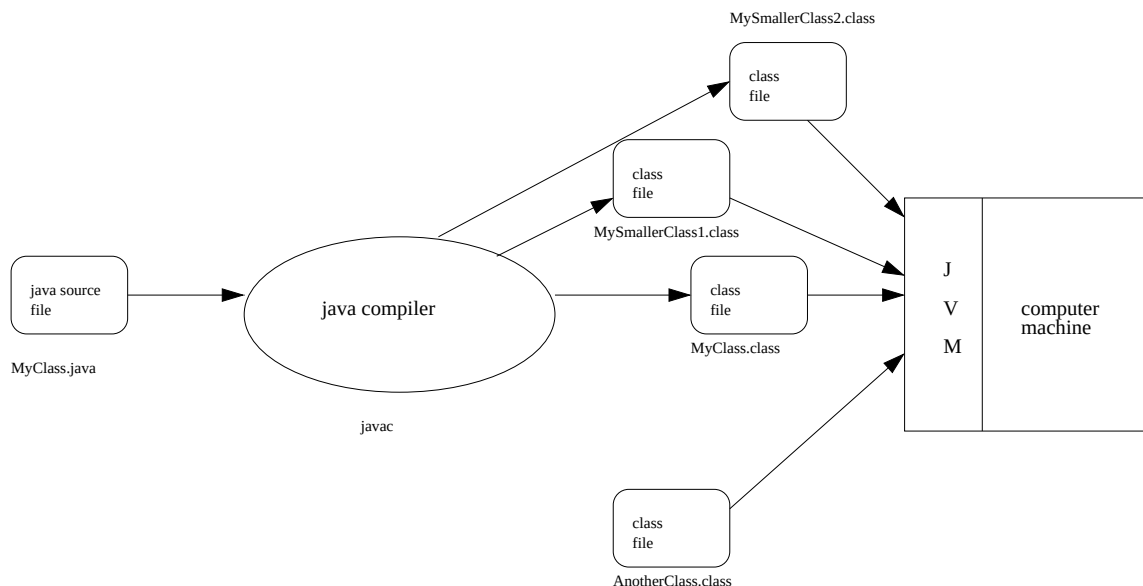
Τέλος, η Java επικεντρώνεται στον χρήστη των εφαρμογών και προσφέρει ποικιλία δυνατοτήτων για ανάπτυξη συστημάτων φιλικής επικοινωνίας ανθρώπου-υπολογιστή, κυρίως γραφικών. Θέτοντας τον χρήστη στο κέντρο των πραγμάτων, διευκολύνει την ανάπτυξη εφαρμογών που στοχεύουν σε αυτού του είδους την αλληλεπίδραση.

Παρότι η γλώσσα έχει πολλές λεπτομέρειες που παρουσιάζουν ενδιαφέρον, στη συνέχεια, θα επικεντρωθούμε στα βασικά αντικειμενοστραφή χαρακτηριστικά της, καθώς και σε κάποια σημεία που διαφοροποιείται σε σχέση με τη C++, όπως το περιβάλλον εκτέλεσης, οι μεταβλητές της, οι διαβαθμίσεις ορατότητας και η κληρονομικότητα.

9.3 Οργάνωση και Εκτέλεση των Προγραμμάτων Java

Στα προηγούμενα, συζητήσαμε αρκετά τα περί μεταφερισιμότητας των προγραμμάτων Java. Στην παράγραφο αυτή, πλέον, θα γίνουμε πιο συγκεκριμένοι ως προς το πώς εκτελείται ένα πρόγραμμα Java, οπότε θα γίνει πιο καθαρό το πώς επιτυγχάνεται η μεταφερισιμότητα αυτή. Ας δούμε, λοιπόν, τη διαδικασία που πρέπει να ακολουθηθεί για την εκτέλεση, ξεκινώντας από το πηγαίο πρόγραμμα. Η διαδικασία αυτή παρουσιάζεται στο Σχήμα 9.1.

9.3.1 Το Περιβάλλον Εκτέλεσης



Σχήμα 9.1: Διαδικασία μεταγλώττισης και εκτέλεσης προγραμμάτων Java

Το πηγαίο πρόγραμμα γράφεται σε ένα σύνολο αρχείων που περιέχουν κώδικα Java — ένα εκ των οποίων πρέπει να περιέχει την κύρια κλάση, για την οποία θα μιλήσουμε παρακάτω. Τα αρχεία πηγαίου κώδικα Java έχουν κατάληξη .java. Τα αρχεία αυτά μεταγλωττίζονται από τον μεταγλωττιστή (javac) σε αρχεία κλάσεων (class file), τα οποία έχουν κατάληξη .class. Κάθε κλάση του πηγαίου κώδικα μεταγλωττίζεται σε ένα ξεχωριστό αρχείο κλάσης, το οποίο περιέχει την αναπαράσταση/πληροφορία για την κλάση αυτή σε *bytecodes* (one-byte opcodes). Τα *bytecodes* είναι εντολές για την εικονική μηχανή *Java Virtual Machine* (JVM). Με τη χρήση των *bytecodes*, γίνεται η αναπαράσταση ενός προγράμματος ως ακολουθία από bytes.

Στον υπολογιστή στον οποίο θέλουμε να τρέξει το πρόγραμμά μας, εκτελούμε τη JVM. Τα αρχεία κλάσεων

που προκύπτουν από τη μεταγλώττιση του πηγαίου προγράμματος, καθώς και όποια ακόμα αρχεία κλάσεων είναι απαραίτητα για την εκτέλεση του προγράμματός μας, φορτώνονται αυτόματα σε αυτήν προσδιορίζοντας μόνο το αρχείο κλάσης που έχει προκύψει από την κύρια κλάση. Η εκτέλεση του προγράμματος γίνεται από τη JVM, η οποία, όπως είπαμε, είναι το μόνο πρόγραμμα που εκτελείται άμεσα στον υπολογιστή.

Η πλατφόρμα της Java (Java platform) έχει δύο συστατικά: τη JVM και το Application Programming Interface (API). Το API είναι μια μεγάλη συλλογή έτοιμου λογισμικού που παρέχει πλήθος λειτουργιών. Αποτελεί τη βασική βιβλιοθήκη (core library) της γλώσσας.

Στην Java, λογισμικό συναφούς λειτουργικότητας ομαδοποιείται σε πακέτα (*packages*). Το πακέτο `java.lang` της πλατφόρμας περιέχει κλάσεις που είναι θεμελιώδεις για τον σχεδιασμό της Java.

Επειδή συνήθως τα προγράμματα Java είναι εκτενή, αναπτύσσονται κυρίως σε περιβάλλοντα που διευκολύνουν την ανάπτυξη κώδικα, τα Integrated Development Environments (IDEs), π.χ. Eclipse, NetBeans.

9.3.1.1 Λίγα Λόγια για τη JVM και το class file format

Η JVM είναι μια αφηρημένη υπολογιστική μηχανή (abstract computing machine), η οποία αναγνωρίζει το συγκεκριμένο δυαδικό format των αρχείων κλάσεων, το *class file format*. Δεν σχετίζεται αποκλειστικά με την Java. Υπάρχουν και άλλες γλώσσες που μεταγλωττίζονται σε bytecode, για να μπορούν να εκτελεστούν στη JVM, π.χ. η Scala. Η JVM δεν αντιλαμβάνεται και ούτε την αφορά από ποια γλώσσα προέρχεται ένα αρχείο κλάσης.

Το περιεχόμενο ενός αρχείου κλάσης είναι μια ακολουθία από 8-bit bytes. Κάθε εντολή (instruction) που αναγνωρίζει η JVM αναπαριστάται από έναν αριθμό, *opcode* (operation code), ο οποίος, στην περίπτωση που η εντολή έχει τελούμενα (operands), ακολουθείται από ένα ή παραπάνω bytes. Κάθε bytecode, δηλαδή, αποτελείται από ένα byte που αντιστοιχεί στο opcode της εντολής και ακολουθείται από bytes που αντιστοιχούν στα τελούμενά της. Τα bytecodes της Java είναι μια ειδική περίπτωση εντολών p-code για αφηρημένες μηχανές, με τις οποίες επιτυγχάνεται μεταφερσιμότητα — το όνομα “p-code” προέρχεται από την έκφραση “portable code”.

Σε ένα αρχείο κλάσης, προσδιορίζεται όλη η πληροφορία που αφορά την κλάση. Για παράδειγμα, προσδιορίζεται το όνομα της κλάσης, δηλώσεις που την αφορούν, π.χ. ορατότητα της κλάσης, οι σταθερές της, σχέσεις που μπορεί να έχει με άλλες κλάσεις, π.χ. την υπερκλάση της, εσωτερικές (inner) κλάσεις ή περιβάλλουσες (enclosing) κλάσεις κ.ά. Όπως είναι αναμενόμενο, περιγράφονται επίσης τα πεδία της και οι μέθοδοί της.

9.3.2 Δομή και Μεταγλώττιση Προγραμμάτων Java

Κάθε εφαρμογή σε Java πρέπει να έχει μια κύρια κλάση. Κύρια κλάση είναι μια οποιαδήποτε κλάση που περιέχει μια μέθοδο `main` με την εξής επικεφαλίδα:

```
public static void main(String[] args)
```

Η παράμετρος `args` είναι τύπου `String[]` και περιέχει τα ορίσματα που δίνονται από τη γραμμή εντολής όταν εκτελούμε το πρόγραμμα. Κάθε πρόγραμμα Java πρέπει να έχει μια τέτοια μέθοδο `main`. Η κλήση της αποτελεί το αρχικό σημείο της εκτέλεσης του προγράμματος αυτού.

Το ελάχιστο πρόγραμμα Java, λοιπόν, είναι, για παράδειγμα το:

```
// file: java.java
class NameDoesNotMatter {
    public static void main(String[] args) {}
}
```

Στο σημείο αυτό, να κάνουμε και μια μικρή επισήμανση που αφορά τη σύνταξη των προγραμμάτων Java: η ορατότητα ενός μέλους γίνεται με δήλωση ορατότητας απευθείας στο μέλος, αντίθετα με τις δηλώσεις ορατότητας στη C++, που αφορούν μια ομάδα μελών. Προσέξτε, λοιπόν, τη δήλωση ορατότητας για τη συνάρτηση `main` της κύριας κλάσης: υποχρεωτικά πρέπει να είναι δηλωμένη ως `public`. Οπότε, το παρακάτω πρό-

γραμμα, παρότι μεταγλωττίζεται, δεν θεωρείται αυτόνομο, οπότε δεν μπορεί να εκτελεστεί.

```
// Careful main should be "public"
// The following compiles but does not execute
class SomeClass {
    static void main(String[] args) {}
}
```

Από την πρώτη ματιά στη γλώσσα, γίνεται ξεκάθαρο ότι δεν πρόκειται για μια γλώσσα διαδικαστικού προγραμματισμού στην οποία δίνεται η δυνατότητα χρήσης αντικειμενοστραφών χαρακτηριστικών, όπως στην περίπτωση της C++. Στην Java, ακόμα και το ελάχιστο πρόγραμμα οφείλει να περιέχει μία κλάση. Δεν υπάρχει “χώρος” να οριστούν μεταβλητές ή/και συναρτήσεις, παρά μόνο ως μέλη κλάσεων — στα σώματα των συναρτήσεων, όμως μπορούμε να ορίζουμε τοπικές μεταβλητές. Οι κλάσεις της Java μπορούν να ομαδοποιηθούν σε πακέτα. Αν δεν δηλωθεί ρητά το περιεχόμενο του αρχείου ως μέρος ενός πακέτου (δηλαδή, μη ύπαρξη δήλωσης `package` στην πρώτη γραμμή του αρχείου), τότε θεωρείται ότι αυτό είναι μέρος ενός unnamed πακέτου.¹

Για να έχουμε και κάποια έξοδο μπορούμε να γράψουμε το εξής πρόγραμμα:

```
class FirstApp {
    public static void main(String[] args) {
        System.out.println("Hello World!");
    }
}
```

Το πρόγραμμα αυτό εκτυπώνει στην καθιερωμένη έξοδο τη συμβολοσειρά `Hello World!`. Ο τρόπος που γίνεται αυτό είναι μέσω της κλήσης της μεθόδου `println`, με όρισμα τη συμβολοσειρά και εφαρμογή στο αντικείμενο `out` της εμβέλειας της κλάσης `System`. Η κλάση `System` είναι μια κλάση που προσφέρεται από την Java — περιλαμβάνεται στο API — και περιέχει ως μέλη της διάφορες χρήσιμες μεταβλητές και μεθόδους. Για παράδειγμα, μέσω της κλάσης αυτής προσφέρεται η λειτουργικότητα για είσοδο/έξοδο στις καθιερωμένες μονάδες. Η μεταβλητή `out` είναι ένα στατικό πεδίο της κλάσης `System`, τύπου `PrintStream`. Η κλάση `PrintStream` είναι μια εξειδίκευση της κλάσης `OutputStream`, η οποία είναι η γονεϊκή όλων των κλάσεων εξόδου χαρακτήρων. Η μέθοδος `void println(String x)` της κλάσης `PrintStream` εκτυπώνει τη συμβολοσειρά `x` στο αντικείμενο της `PrintStream` στο οποίο εφαρμόζεται, ολοκληρώνοντας τη γραμμή προς έξοδο.

Ένα παράδειγμα προγράμματος Java που περιλαμβάνει και είσοδο από τη γραμμή εντολής δίνεται στη σελίδα 385. Στο παράδειγμα αυτό, γίνονται και οι απαραίτητες μετατροπές τύπων που συνοδεύουν την είσοδο.

9.4 Γενικές Παρατηρήσεις για τα Προγράμματα Java

Στην παράγραφο αυτή κάνουμε κάποιες παρατηρήσεις σχετικά με έννοιες που είναι κρίσιμες για την ανάπτυξη κώδικα σε Java.

Πακέτα και Κλάσεις: Η οργάνωση του κώδικα των προγραμμάτων Java γίνεται σε *πακέτα*. Τα πακέτα ορίζουν εμβέλειες με όνομα και περιέχουν ορισμούς *κλάσεων*. Τα δεδομένα των κλάσεων και τα δομικά στοιχεία των αντικειμένων τους αναφέρονται ως *πεδία*. Η συμπεριφορά των κλάσεων και των αντικειμένων τους υλοποιούνται από συναρτήσεις που αποκαλούνται *μέθοδοι*. Μπορούμε να ορίσουμε *αντικείμενα* των κλάσεων, είτε ως τιμή των μεταβλητών των πεδίων είτε ως τιμή μεταβλητών στις μεθόδους.

Μέλη Στιγμιοτύπων και Μέλη Κλάσεων: Τόσο τα πεδία, όσο και οι μέθοδοι μιας κλάσης, μπορούν να σχετίζονται με κάποιο αντικείμενο της κλάσης, ή να είναι ανεξάρτητα από αντικείμενο και να αφορούν την

¹Για λόγους απλότητας, ο κώδικας των παραδειγμάτων του κεφαλαίου αυτού δεν θα ανήκει σε συγκεκριμένο πακέτο. Θα θεωρείται ότι ανήκει στο unnamed πακέτο, εκτός εάν δηλώνεται ρητά κάτι διαφορετικό.

κλάση και μόνο. Στην πρώτη περίπτωση, μιλάμε για μέλη στιγμιοτύπου (instance members). Αυτά, αν πρόκειται για πεδία, συμμετέχουν στο “κτίσιμο” ενός αντικειμένου της κλάσης, είναι δομικά στοιχεία του αντικειμένου. Αν πρόκειται για μεθόδους, εφαρμόζονται απαραίτητα σε αντικείμενο. Στη δεύτερη περίπτωση, μιλάμε για μέλη κλάσης (class members). Αυτά συμπεριφέρονται όπως τα στατικά μέλη κλάσεων της C++. Δηλώνονται με τη λέξη-κλειδί `static`.

Ως προς τι Δηλώνουμε Ορατότητα; Δηλώνουμε ορατότητα τόσο για τα ονόματα των μελών των κλάσεων, όσο και για τα ονόματα των ιδίων των κλάσεων. Οι *διαβαθμίσεις ορατότητας* περιορίζουν τη χρήση των ονομάτων, είτε ως προς τις άλλες κλάσεις είτε ως προς τα άλλα πακέτα. Στη δεύτερη περίπτωση, έχουμε τη δυνατότητα να περιορίσουμε τη χρήση κλάσεων ή/και μελών τους, μόνο στην εμβέλεια του πακέτου μέσα στο οποίο έχουν οριστεί.

Κληρονομικότητα: Όλες οι κλάσεις της Java, είτε αυτές είναι ορισμένες από τον προγραμματιστή είτε παρέχονται από το API της γλώσσας, είναι οργανωμένες ιεραρχικά, μέσω *κληρονομικότητας*. Η κλάση που βρίσκεται στη ρίζα (root) της ιεραρχίας είναι η κλάση `Object`. Κάθε κλάση ή τύπος πίνακα (array) είναι υποκλάση της κλάσης `Object`. Κάθε κλάση μπορεί να δηλωθεί ως άμεση υποκλάση το πολύ μιας κλάσης (χρησιμοποιώντας τη λέξη-κλειδί `extends`). Αν δεν δηλωθεί ρητά υπερκλάση, υπονοείται ότι η κλάση είναι άμεση υποκλάση της κλάσης `Object`. Για να υποστηριχθεί κάποιας μορφής πολλαπλή κληρονομικότητα με “καθαρό” τρόπο, η Java προσφέρει έναν τύπο, εναλλακτικό των κλάσεων, τους *τύπους διεπαφής* (interfaces). Στη γενική περίπτωση, οι τύποι διεπαφής περιέχουν μόνο δηλώσεις μεθόδων. Κάθε κλάση που μπορεί να θεωρηθεί ότι “αντιπροσωπεύεται” από τις προδιαγραφές που εκφράζουν οι δηλώσεις αυτές, μπορεί να δηλώσει ότι *υλοποιεί* τον τύπο (η λέξη-κλειδί που χρησιμοποιείται είναι `implements`) και οφείλει να ορίσει τις μεθόδους του. Μια κλάση μπορεί να υλοποιήσει πάνω από έναν τύπο διεπαφής.

Η Χρήση της Λέξης-κλειδιού `final`: Ενδιαφέρον είναι να σχολιαστεί η χρήση της λέξης-κλειδιού `final`. Δείχνει να εκφράζει το νόημα “τελικό”/“τελεία και παύλα”! Μπορεί να χρησιμοποιηθεί ως προσδιοριστής σε μια κλάση. Σε αυτήν την περίπτωση, απαγορεύεται να οριστούν υποκλάσεις της κλάσης αυτής. Μπορεί να χρησιμοποιηθεί ως προσδιοριστής σε μια μέθοδο κλάσης. Σε αυτήν την περίπτωση, η μέθοδος αυτή απαγορεύεται να επαναοριστεί στις υποκλάσεις της κλάσης. Μπορεί να χρησιμοποιηθεί ως προσδιοριστής σε ένα πεδίο κλάσης. Στην περίπτωση αυτή, η τιμή του πεδίου είναι σταθερή. Τέλος, μπορεί να χρησιμοποιηθεί ως προσδιοριστής μεταβλητών γενικότερα, δηλώνοντας πάλι σταθερά. Κάτι που πρέπει να αναφέρουμε εδώ, παρότι ο όρος δεν έχει ακόμα παρουσιαστεί, στην περίπτωση που προσδιορίζεται ως `final` μια μεταβλητή που είναι αναφορά (reference), η ίδια η μεταβλητή δεν μπορεί να “δείξει”/αναφερθεί κάπου αλλού, το αντικείμενο, όμως, στο οποίο αναφέρεται μπορεί να αλλάζει τιμή. Δεν γίνεται άμεσα δήλωση σταθερών αντικειμένων: πρέπει να οριστεί κατάλληλη κλάση με μη μεταβαλλόμενα δεδομένα.

Επισημειώσεις: Καλό είναι να αναφέρουμε τη χρήση *επισημειώσεων* (annotations) στα προγράμματα Java, για λόγους περιγραφής του κώδικα. Οι επισημειώσεις είναι μεταδεδομένα του προγράμματος και όχι μέρος του κώδικα και, στη γενική περίπτωση, δεν έχουν επίδραση κατά την εκτέλεση του προγράμματος. Εισάγονται με τον χαρακτήρα `@`, δηλαδή η πιο απλή γενική μορφή τους είναι `@Entity`. Παρότι διάφορες πληροφορίες σχετικά με τον κώδικα μπορούν να εκφραστούν μέσω επισημειώσεων, π.χ. ο συγγραφέας του, πολύ συχνά συναντάμε την επισημείωση `@Override` σε μεθόδους, ώστε να επισημειώσουμε ότι η μέθοδος που ακολουθεί επαναορίζει κληρονομημένη μέθοδο, π.χ.

```
@Override
void MyMethod() {System.out.println("Not from the parent method");}
```

Τέλος, μπορεί να οριστούν και κατηγορίες επισημειώσεων από τον ίδιο τον χρήστη, ώστε να χρησιμοποιηθούν στον κώδικά του.

Η Γενική Μορφή του Περιεχομένου μιας Απλής Κλάσης: Σε μια απλή μορφή, μια κλάση Java περιμένουμε να έχει:

- κάποια πεδία
- κάποιες μεθόδους
- κάποιες μεθόδους (συναρτήσεις) αρχικοποίησης

Ένα τέτοιο παράδειγμα είναι:

```
class MySimpleClass {
    private int data1;           // field
    private int data2;           // field

    void myMethod() {            // method
        data1++; data2++;
        System.out.println("The data1 value is: " + data1 +
                           " The data2 value is: " + data2);
    }

    MySimpleClass() {}           // constructor (default)

    MySimpleClass(int d1, int d2) { // constructor
        data1 = d1; data2 = d2;
    }
}
```

Μπορούμε να δηλώσουμε για μια κλάση ότι απαγορεύεται να δημιουργηθούν αντικείμενά της, είναι δηλαδή *αφηρημένη κλάση*. Αυτό γίνεται προσδιορίζοντάς την ως `abstract`.

9.4.1 Κατηγορίες Κλάσεων στην Java

Έχοντας κάνει μια γενική συζήτηση σχετικά με τις κλάσεις της Java, στο σημείο αυτό ας δούμε σε ποιες κατηγορίες μπορεί να εμπίπτει μια κλάση. Ξεκινώντας από πλευράς *διαβάθμισης ορατότητας*, μια (εξωτερική) κλάση μπορεί να είναι:

- Καθολική: δηλώνοντας μια κλάση ως `public` μπορεί να προσπελαστεί και έξω από το πακέτο στο οποίο ορίζεται.
- Τοπική στο πακέτο μέσα στο οποίο ορίστηκε. Στην περίπτωση αυτή είναι ορατή και χρησιμοποιήσιμη από τον κώδικα μόνο του πακέτου αυτού.

Η παραπάνω διάκριση αφορά μόνο τις “εξωτερικές” κλάσεις. Στην Java, όμως, μπορεί να οριστούν κλάσεις που είναι *εμφωλευμένες*, δηλαδή είναι ορισμένες εντός μιας άλλης κλάσης. Σε αυτήν την περίπτωση, οι διαβαθμίσεις ορατότητας μπορεί να είναι όλες εκείνες που αφορούν μέλη κλάσεων.

Ας δούμε, όμως, καλύτερα τα είδη *εμφωλευμένων* κλάσεων, μια που στην Java προσφέρονται σε μεγαλύτερη ποικιλία. Όπως είπαμε, οι *εμφωλευμένες* κλάσεις είναι κλάσεις που ορίζονται μέσα σε μια άλλη κλάση. Αποτελούν μέλη της κλάσης στην οποία ορίζονται. Διακρίνονται σε δύο γενικές κατηγορίες:

1. στατικές (static) και
2. μη-στατικές (non-static)

Στην πρώτη περίπτωση, η εμφωλευμένη κλάση προσδιορίζεται ως `static`. Οι κλάσεις αυτές καλούνται *στατικές εμφωλευμένες κλάσεις* (static nested classes). Μπορούν να δημιουργηθούν στιγμιότυπα μιας τέτοιας κλάσης με απευθείας χρήση της, χωρίς να πρέπει να γίνει μέσω αντικειμένων της εξωτερικής της κλάσης (οπότε και να απαιτείται να έχουν δημιουργηθεί τέτοια). Για τον λόγο αυτό, μια στατική εμφωλευμένη κλάση δεν μπορεί να έχει πρόσβαση σε μη στατικά μέλη της εξωτερικής της. Ένα μικρό παράδειγμα εμφωλευμένης κλάσης είναι το εξής:

```
class MyClass {
    public static void main(String[] args) {

//      SInClass Var3 = new SInClass(); // error: cannot find
//                                     // symbol in MyClass

        Out.SInClass Var3 = new Out.SInClass();
        Out var1 = new Out();
    }
}

class Out {
    static class SInClass {
        int data = 100;
        SInClass() {
            System.out.println("Printing from SInClass with data: " +
                               data); }
    }

    SInClass var1 = new SInClass();
    SInClass var2 = new SInClass();
}
```

Η κλάση `SInClass` είναι στατική εμφωλευμένη. Η εξωτερική της κλάση είναι η `Out`. Πέρα από τις παρατηρήσεις που κάναμε, προσέξτε τη χρήση της κλάσης `SInClass` από την κυρία κλάση, `MyClass`: μπορούμε να τη χρησιμοποιήσουμε, αφού η ορατότητά της δεν περιορίζεται μέσα στην εξωτερική κλάση, αλλά για να τη χρησιμοποιήσουμε, οφείλουμε να χρησιμοποιήσουμε το όνομα της εξωτερικής κλάσης, δηλαδή `Out.SInClass()`.

Στη δεύτερη περίπτωση εμφωλευμένων κλάσεων ανήκουν οι *εσωτερικές κλάσεις* (inner classes). Τα αντικείμενα των κλάσεων αυτών είναι συσχετισμένα με αντικείμενο της εξωτερικής τους. Δηλαδή, υφίστανται “εντός” ενός αντικειμένου της εξωτερικής. Καθότι οι κλάσεις αυτές δεν είναι ανεξάρτητες αντικειμένου της εξωτερικής, μπορούν να έχουν πρόσβαση σε όλα τα μέλη της. Ένα αφαιρετικό παράδειγμα ορισμού αντικειμένων των εσωτερικών κλάσεων είναι το εξής:

```
class Window {                // outer class
    // ...
    class Button {            // inner class
        // ...
    }
}
...
Window win1 = new Window();
```

```
Window.Button butt1 = win1.new Button();
...
```

Στο παράδειγμα, βλέπουμε ότι για να δημιουργήσουμε ένα αντικείμενο της εσωτερικής κλάσης, πρέπει να έχουμε ήδη ορίσει ένα αντικείμενο της εξωτερικής. Τότε, ορίζουμε το αντικείμενο της εσωτερικής “εντός” ενός αντικειμένου της εξωτερικής, όπως φαίνεται στον ορισμό του αντικειμένου `butt1` του παραπάνω παραδείγματος.

Να παρατηρήσουμε εδώ ότι ένας από τους λόγους που η Java προσφέρει μεγάλη ποικιλία εμφωλευμένων κλάσεων είναι ότι μέσα στους στόχους του σχεδιασμού της είναι και η ευκολία ανάπτυξης γραφικών διεπαφών χρήστη. Για τον χώρο αυτό, οι διαφορετικές περιπτώσεις εμφωλευμένων κλάσεων είναι χρήσιμες. Για παράδειγμα, η περίπτωση ορισμού εσωτερικών κλάσεων όπου τα αντικείμενά τους “υφίστανται” στο πλαίσιο της ύπαρξης ενός αντικειμένου της εξωτερικής κλάσης εξυπηρετεί στην υλοποίηση παραθυρικών περιβαλλόντων.

Υπάρχουν δύο ακόμα κατηγορίες εμφωλευμένων κλάσεων στην Java, οι οποίες, σύμφωνα με την περιγραφή της γλώσσας, θεωρούνται ειδικές κατηγορίες εσωτερικών κλάσεων. Αυτές είναι οι εξής:

1. Οι *ανώνυμες κλάσεις* (anonymous classes), οι οποίες είναι κλάσεις που δεν έχουν όνομα. Η λειτουργικότητά τους επιτρέπει να ορίζεται μια κλάση κατά τον ορισμό ενός αντικειμένου της κλάσης αυτής. Έτσι, επιτρέπουν τη χρήση τους για μία φορά, την κατασκευή ενός μόνο αντικειμένου. Ουσιαστικά, πρόκειται για τη δυνατότητα ορισμού υποκλάσης μιας ήδη υπάρχουσας κλάσης (ή τύπου-διεπαφής), που θα χρησιμοποιηθεί για τον ορισμό ενός και μόνο αντικειμένου. Με τον τρόπο αυτό, δεν χρειάζεται ξεχωριστή κλάση και, επιπλέον, η περιγραφή που εξειδικεύει το αντικείμενο συνοδεύει τον ορισμό του.
2. Οι *τοπικές* (local) κλάσεις, οι οποίες είναι κλάσεις που είναι ορισμένες μέσα σε block. Ένα block προσδιορίζεται από ένα ζευγάρι αγκύλων, { }. Κλάσεις που χαρακτηρίζονται ως τοπικές ορίζονται συνήθως στα σώματα μεθόδων.

Για να κλείσουμε τη σύνοψη των κατηγοριοποιήσεων των κλάσεων, θα επαναλάβουμε τη διάκριση σε σχέση με τη *δυνατότητα δημιουργίας αντικειμένων*. Στην περίπτωση αυτή, έχουμε κλάσεις των οποίων μπορούμε να δημιουργήσουμε αντικείμενα και αφηρημένες κλάσεις. Στις αφηρημένες κλάσεις, θα αναφερθούμε πιο αναλυτικά αργότερα. Τέλος, σε σχέση με τη δυνατότητα “*επέκτασής*” τους μέσω υποκλάσεων, στην περίπτωση που δηλωθούν ως `final`, όπως είπαμε, δεν επιτρέπεται να οριστούν υποκλάσεις τους.

9.4.2 Διαβαθμίσεις Ορατότητας - modifiers για τα Μέλη Κλάσεων

Στην παράγραφο αυτή, θα παραθέσουμε συνοπτικά τα εναλλακτικά όσον αφορά τις διαβαθμίσεις ορατότητας των μελών των κλάσεων.

Τα πεδία, οι μέθοδοι και τα άλλα μέλη μιας κλάσης, σε σχέση με την ορατότητα έξω από την εμβέλεια της κλάσης, μπορούν να δηλωθούν ως:

- **public**: μπορούν να προσπελαστούν και έξω από το πακέτο που ορίζεται η κλάση
- **private**: μπορούν να χρησιμοποιηθούν μόνο από την κλάση μέσα στην οποία ορίζονται
- **protected**: μπορούν να προσπελαστούν μέσα από το πακέτο στο οποίο ορίζεται η κλάση και από τις υποκλάσεις της κλάσης μέσα στην οποία ορίζονται, ακόμα και αν οι υποκλάσεις αυτές είναι ορισμένες σε άλλο πακέτο
- **χωρίς προσδιορισμό**: μπορούν να προσπελαστούν μόνο μέσα από το πακέτο στο οποίο ορίζεται η κλάση

Άλλοι συνήθεις προσδιορισμοί μέσω modifiers που μπορούμε να έχουμε στις δηλώσεις πεδίων μιας κλάσης είναι:

- **static**: Δηλώνεται ότι το πεδίο είναι μια μεταβλητή εμβέλειας κλάσης. Δεν είναι απαραίτητο να έχει οριστεί αντικείμενο της κλάσης για να αναφερθούμε σε αυτό.

- **final**: Το πεδίο δεν μπορεί να αλλάξει τιμή (άμεσα) μετά την αρχικοποίηση.

Άλλοι συνήθεις προσδιορισμοί μέσω **modifiers** που μπορούμε να έχουμε στις δηλώσεις μεθόδων μιας κλάσης είναι:

- **static**: Δηλώνεται ότι η μέθοδος δεν χρειάζεται να εφαρμοστεί σε αντικείμενο. Οι στατικές μέθοδοι δεν μπορούν να καλέσουν μεθόδους που δεν είναι στατικές.
- **final**: Δηλώνεται ότι η μέθοδος δεν μπορεί να επαναοριστεί σε υποκλάση — δεν μπορεί να γίνει **override**.
- **abstract**: Δηλώνεται ότι η μέθοδος δεν έχει υλοποίηση (απλά, δηλώνεται μόνο). Τότε και η κλάση πρέπει να δηλωθεί ως **abstract**.

Εδώ, να υπενθυμίσουμε ότι η ορατότητα των εξωτερικών κλάσεων (δηλαδή κλάσεων που δεν είναι εσωτερικές σε μια άλλη κλάση) μπορεί να περιοριστεί μέσα στο πακέτο που ορίζεται η κλάση, ή να είναι δημόσια. Εάν ένα αρχείο περιέχει πολλούς ορισμούς κλάσεων, μόνο μια από αυτές μπορεί να δηλωθεί ως **public**. Στην περίπτωση που το αρχείο αποτελεί μονάδα μεταγλώττισης ενός πακέτου, η κλάση αυτή πρέπει να έχει το ίδιο όνομα με το αρχείο.

Θεωρούνται ορατά, και μπορούν να χρησιμοποιηθούν, ονόματα από διάφορα πακέτα του API.

9.5 Δεδομένα και Διαδικασίες

Ας ξεκινήσουμε να εξετάζουμε τα βασικά γλωσσικά στοιχεία της Java, τα οποία ο προγραμματιστής μπορεί να χρησιμοποιήσει κατά τη συγγραφή του προγράμματός του. Θα εστιάσουμε, κυρίως, στα δεδομένα, περιγράφοντας τους τύπους τους και τη φιλοσοφία της Java σχετικά με αυτούς. Όσον αφορά τις διαδικασίες, η αναφορά θα είναι πολύ σύντομη, μια που οι εντολές και οι εκφράσεις της γλώσσας δεν διαφέρουν ουσιαστικά από εκείνες της C και της C++.

9.5.1 Μεταβλητές στην Java - Τύποι

Σε ένα πρόγραμμα Java, συναντάμε μεταβλητές με τους εξής ρόλους:

- Πεδίο. Διακρίνουμε τις εξής κατηγορίες πεδίων:

Πεδία (εμβέλειας) κλάσης: Αυτά είναι πεδία που έχουν δηλωθεί ως **static**. Λειτουργούν όπως τα στατικά μέλη-δεδομένα της C++: δεν συμμετέχουν στην κατασκευή αντικειμένου, αλλά αποτελούν μεταβλητές που ανήκουν στην εμβέλεια της κλάσης. Λαμβάνοντας υπόψη ότι στην Java κάθε κλάση μπορεί να θεωρηθεί με τη σειρά της στιγμιότυπο μιας μετακλάσης, αυτές είναι τα πεδία της. Ο λόγος που κάνουμε αυτήν την αναφορά είναι ότι, όπως θα δούμε πιο αναλυτικά αργότερα, οι αρχικοποιήσεις των πεδίων κλάσης είναι μέρος της διαδικασίας αρχικοποίησης της κλάσης.

Πεδία στιγμιότυπου: Αυτά είναι απλά πεδία που συμμετέχουν στην κατασκευή κάθε αντικειμένου της κλάσης. Δηλαδή, για κάθε αντικείμενο της κλάσης, δημιουργείται διαφορετική μεταβλητή, σύμφωνα με τη δήλωση του πεδίου.

- Παράμετρος συνάρτησης (μεθόδου) για τα ορίσματά της.
- Τοπικές μεταβλητές συναρτήσεων (μεθόδων), οι οποίες ορίζονται στα σώματα των μεθόδων.

Στην Java οι τύποι χωρίζονται σε δύο κατηγορίες:

- στους *πρωτογενείς τύπους* (**primitive data types**) και

- στους *τύπους αναφοράς* (reference types)

Με βάση τη διάκριση αυτή, οι μεταβλητές μπορεί να είναι:

- είτε απλές μεταβλητές, των οποίων ο τύπος είναι ένας από τους πρωτογενείς τύπους της Java
- είτε *μεταβλητές αναφοράς* (reference variables), των οποίων ο τύπος είναι ένας *τύπος αναφοράς* (reference type)

Η Java προσφέρει ένα σύνολο τύπων που αποκαλεί *πρωτογενείς τύπους*, όπως: `byte` (8-bit signed two's complement integer), `short`, `int`, `long`, `float`, `double`, `boolean`, `char` (16-bit Unicode character). Οι τύποι αυτοί εντάσσονται στις εξής δύο γενικές κατηγορίες: `boolean` και `numeric`. Για παράδειγμα, ένας ορισμός ακέрайας μεταβλητής είναι:

```
int i;
```

Πρέπει να σημειώσουμε ότι τα πεδία αποκτούν αυτόματα αρχικές τιμές. Στην περίπτωση που ο τύπος τους είναι πρωτογενής, η τιμή είναι η μηδενική τιμή του τύπου τους. Στην περίπτωση μεταβλητών αναφοράς, η τιμή είναι `null`. Προσοχή: αν οι μεταβλητές δεν είναι μεταβλητές-πεδία, τότε δεν αποκτούν αρχικές τιμές. Για παράδειγμα, ο παρακάτω κώδικας παράγει σφάλμα:

```
class App {
    public static void main(String[] args) {
        bla();
    }

    static void bla() {
        int i;
        System.out.println("The output of bla() is " + i);
        // error: variable i might not have been initialized
    }
}
```

9.5.1.1 Τύποι Αναφοράς

Όσον αφορά τους τύπους αναφοράς, αυτοί μπορεί να είναι ένας από τους εξής:

1. *τύπος κλάσης* (class type)
2. *τύπος πίνακα* (array type)
3. *τύπος διεπαφής* (interface type)

Όπως είπαμε, οι τύποι αυτοί αποτελούν τους τύπους των μεταβλητών αναφοράς. Οι τιμές τους είναι αναφορές σε στιγμιότυπα κλάσεων και πινάκων. Μια ειδική τιμή αναφοράς είναι η τιμή `null`. Αυτή αναπαριστάει αναφορά σε κανένα αντικείμενο και αποτελεί την default τιμή που αποδίδεται σε μια μεταβλητή αναφοράς.

Για παράδειγμα, ένας ορισμός μεταβλητής αναφοράς είναι:

```
Food fd;
```

όπου `Food` είναι μια κλάση.

Προσέξτε όμως, σε αντίθεση με τη C++, στον παραπάνω ορισμό δεν έχει γίνει δημιουργία αντικειμένου. Το μόνο το οποίο θα δημιουργηθεί με τον ορισμό αυτό είναι μια μεταβλητή αναφοράς με όνομα `fd`, η οποία

θα μπορεί να “αναφερθεί”/“δείξει” σε αντικείμενο τύπου `Food`. Το αντικείμενο πρέπει να δημιουργηθεί ρητά με χρήση του τελεστή `new`, συνοδευόμενο από το όνομα της κλάσης.

Ας δούμε, όμως, πώς γίνεται η δήλωση μιας μεταβλητής και, στην περίπτωση μεταβλητής αναφοράς, πώς αυτή συνδέεται με αντικείμενο.

Η μορφή της δήλωσης μεταβλητής είναι, για παράδειγμα:

```
type t;
```

Με τον τρόπο αυτό, δηλώνεται στον μεταγλωττιστή ότι το όνομα `t` πρόκειται να χρησιμοποιηθεί ως όνομα δεδομένου τύπου `type`. Αν το `type` είναι πρωτογενής τύπος, τότε κατά τη δήλωση αυτή δεσμεύεται ο απαιτούμενος χώρος για τη μεταβλητή του τύπου.

Αν το `type` είναι τύπος αναφοράς (κλάση, διεπαφή ή πίνακας), τότε η τιμή της μεταβλητής δεν αναφέρεται σε αντικείμενο με τη δήλωση αυτή. Αυτό μπορεί να πραγματοποιηθεί μόνο όταν δημιουργηθεί ένα αντικείμενο τύπου `type` και της ανατεθεί. Όπως είπαμε, δημιουργία στιγμιότυπου/αντικειμένου μιας κλάσης γίνεται με χρήση του τελεστή `new`. Η κλήση του δεσμεύει στη μνήμη τον κατάλληλο χώρο για το αντικείμενο και επιστρέφει μια αναφορά σε αυτό. Επίσης, καλεί την (κατάλληλη) συνάρτηση αρχικοποίησης. Συνολικά, δήλωση μεταβλητής αναφοράς και δημιουργίας αντικειμένου ανατεθειμένου σε αυτή οφείλει να γίνει με τον εξής τρόπο:

```
type t = new type(5,10,20);
```

Στο παράδειγμα, ορίζεται μια μεταβλητή αναφοράς, `t`, τύπου `type` καθώς και ένα αντικείμενο τύπου `type` το οποίο αρχικοποιείται με κλήση της συνάρτησης αρχικοποίησης με ορίσματα τρεις ακεραίους. Η μεταβλητή αναφοράς `t` παίρνει τιμή και “αναφέρεται” στο αντικείμενο αυτό. Μια μεταβλητή τύπου `type` μπορεί να πάρει τιμή μια αναφορά σε ένα στιγμιότυπο του τύπου `type`, ή υποκλάσης του, ή να έχει την τιμή `null`.

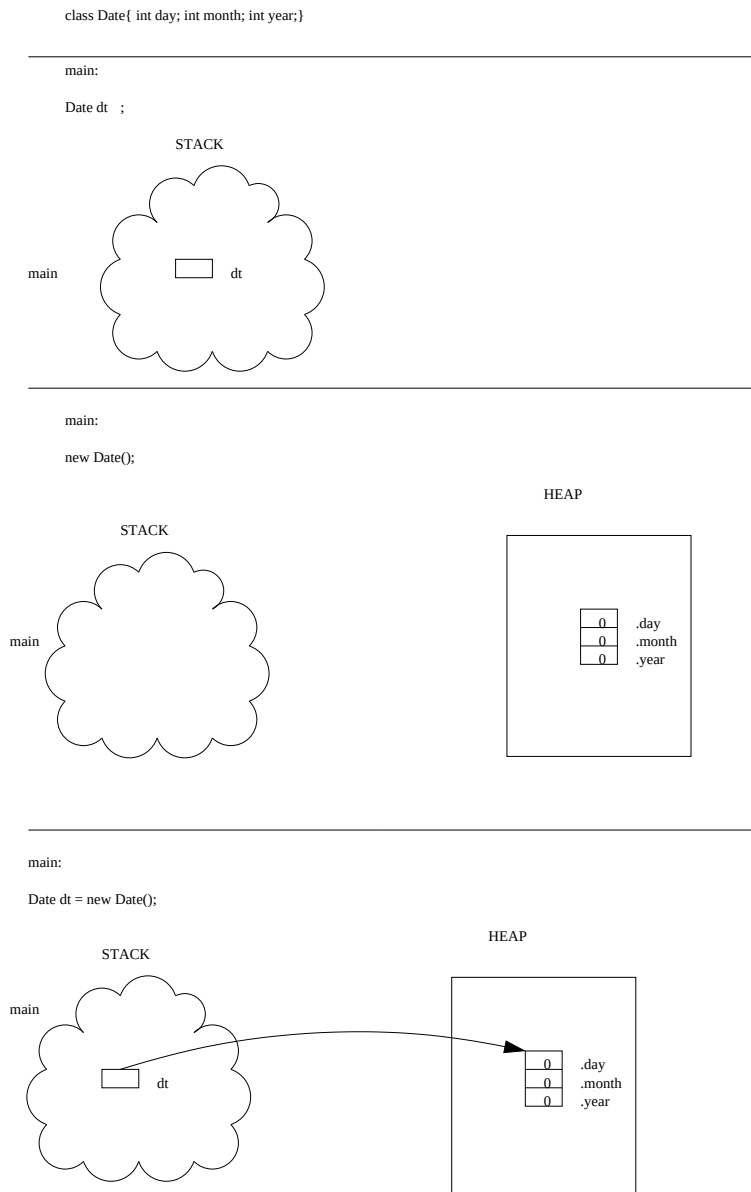
Οι μεταβλητές αναφοράς είναι δείκτες σε αντικείμενο του τύπου τους (ή υποκλάσης του) που πάντα γίνονται *dereference*. Δείχνει, λοιπόν, η συμπεριφορά τους να είναι αντίστοιχη με εκείνη των αναφορών της C++. Αντίθετα, όμως, με αυτές, δεν είναι σταθεροί δείκτες, μπορεί να “δείξουν” σε διαφορετικό αντικείμενο. Επίσης, μπορεί να μην αναφέρονται σε κανένα αντικείμενο (ενδέχεται να έχουν την τιμή `null`). Μια υπενθύμιση: αν μια μεταβλητή που έχει δηλωθεί `final` περιέχει αναφορά σε αντικείμενο, η μεταβλητή θα αναφέρεται πάντα στο ίδιο αντικείμενο, αλλά η τιμή (κατάσταση) του αντικειμένου μπορεί να αλλάξει.

Για να γίνει πιο καθαρή η εικόνα σχετικά με τις μεταβλητές αναφοράς και τα αντικείμενα, δείτε το Σχήμα 9.2. Να επισημάνουμε ότι, στο σχήμα αυτό, θεωρείται μια υποθετική αναπαράσταση αντικειμένων, η οποία προκύπτει από τα πεδία στιγμιότυπου της κλάσης. Αυτό δεν ισχύει στη γενική περίπτωση, αλλά είναι κατάλληλο για τους σκοπούς της παρουσίασης των παραδειγμάτων μας και περιέχει την πληροφορία που μας εξυπηρετεί.²

Στο Σχήμα 9.2, θεωρούμε δεδομένη μια κλάση `Date` της οποίας τα αντικείμενα δομούνται από τρεις ακεραίους. Ας υποθέσουμε ότι δεν προσδιορίζεται αρχική τιμή για τα πεδία της.

Στο πρώτο μέρος του σχήματος, απλά ορίζουμε στη μέθοδο `main` μια μεταβλητή αναφοράς τύπου `Date`, την `dt`. Βλέπουμε, λοιπόν, σε μια αφαιρετική εικόνα της μνήμης, η μεταβλητή να φυλάσσεται στη στοίβα και ο χώρος που δεσμεύει να είναι τέτοιος ώστε να “χωρά” μια αναφορά (ένας δείκτης). Στη δεύτερη περίπτωση, δεν ορίζεται καμία μεταβλητή αναφοράς, απλά στη μέθοδο `main` δημιουργείται ένα αντικείμενο τύπου `Date`. Βλέπουμε, λοιπόν, για το αντικείμενο αυτό, να δεσμεύεται χώρος στον σωρό που να “χωράει” μια δόμηση από τρεις ακεραίους. Παρότι δεν είναι συνηθισμένο να δημιουργείται ένα αντικείμενο στο οποίο δεν αναφέρεται κάποια μεταβλητή αναφοράς, αυτό μπορεί να συμβεί. Πάρτε το παράδειγμα δημιουργίας αντικειμένου για προσωρινή χρήση σε μία έκφραση μιας εντολής. Στην τελευταία περίπτωση του Σχήματος 9.2, η οποία είναι πολύ συνηθισμένη, βλέπουμε να ορίζεται στη μέθοδο `main` ταυτόχρονα μια μεταβλητή αναφοράς τύπου `Date`, η `dt`, και ένα αντικείμενο τύπου `Date` και να ανατίθεται ως τιμή της μεταβλητής `dt` η διεύθυνση του

²Να σημειώσουμε ότι στο πρότυπο της JVM δεν επιβάλλεται συγκεκριμένη αναπαράσταση για τα αντικείμενα, παρότι σε μερικές υλοποιήσεις της από την Oracle, ένα αντικείμενο αναπαριστάται από δύο δείκτες. Ο ένας δείκτης δείχνει σε έναν πίνακα με δύο στοιχεία: το ένα περιέχει τις μεθόδους του αντικειμένου και το άλλο έναν δείκτη στο αντικείμενο κλάσης που αναπαριστάνει τον τύπο του αντικειμένου. Ο δεύτερος δείκτης δείχνει στον χώρο στον σωρό ο οποίος έχει ανατεθεί στα δεδομένα του αντικειμένου. Για λόγους παρουσίασης των διαφόρων περιπτώσεων, μας αρκεί το τελευταίο και για λόγους απλότητας θα θεωρούμε ένα αντικείμενο να αναπαριστάται μόνο από αυτό.



Σχήμα 9.2: Μεταβλητές αναφοράς και αντικείμενα

αντικειμένου αυτού. Δηλαδή, η `dt` αναφέρεται στο αντικείμενο αυτό. Η τρίτη και η πρώτη περίπτωση είναι οι πιο συνηθισμένες.

9.5.1.2 Τύποι Αναφοράς και Πίνακες

Στο σημείο αυτό θα κάνουμε μια σύνοψη των τύπων αναφοράς της Java, καθώς και τη θέση των πινάκων στην κατηγοριοποίησή τους. Να υπενθυμίσουμε ότι οι όλες οι κλάσεις της Java θεωρούμε ότι εντάσσονται σε μια ιεραρχία, στην οποία η κλάση-ρίζα είναι η κλάση `Object` που προσφέρεται από το API της γλώσσας. Κάθε κλάση ή διεπαφή ή πίνακας είναι υποκλάση της κλάσης `Object`.

Οι πίνακες αναπαριστούν “συλλογές” αντικειμένων που περιέχουν σταθερό πλήθος τιμών συγκεκριμένου τύπου. Οι πίνακες στην Java θεωρούνται στιγμιότυπα της κλάσης `Arrays`, η οποία είναι υποκλάση της κλάσης `Object`. Συγκεκριμένα, στο API της Java, η κλάση `Arrays` δηλώνεται ως:

```
public class Arrays extends Object
```

Το μήκος ενός πίνακα καθορίζεται από το πλήθος των στοιχείων που συμμετέχουν ή προσδιορίζεται κατά τη δημιουργία του. Κάθε τύπος πίνακα καθορίζεται από τον τύπο που συνθέτει τον πίνακα. Αυτός, με τη σειρά

του, μπορεί να είναι τύπος πίνακα κοκ. Τελικά, θα καταλήξει σε τύπο που θα είναι είτε πρωτογενής, είτε κλάση, είτε διεπαφή. Αυτός ο τύπος θεωρείται ότι είναι ο τύπος των στοιχείων του.

Παραδείγματα δήλωσης πινάκων είναι τα εξής:

```
int[] intArray;           // integer array (reference variable) declaration
intArray = new int[3];    // integer array creation, of length 3,
                          // and array variable value assignment
```

```
intArray[0] = 10; // Assign value to first element
intArray[1] = 20; // Assign value to second element
intArray[2] = 30; // Assign value to last element
```

Στο παράδειγμα αυτό, κατ' αρχήν ορίζουμε μια μεταβλητή αναφοράς τύπου πίνακα ακεραίων, την `intArray`. Στη συνέχεια, της δίνουμε τιμή έναν πίνακα ακεραίων τριών στοιχείων, που δημιουργείται στην ίδια εντολή. Τέλος, δίνουμε τιμές σε ένα-ένα από τα στοιχεία του. Όπως και στις γλώσσες C και C++, η θέση ενός στοιχείου του πίνακα εντοπίζεται ως απόσταση από την αρχή του, δηλαδή το 1ο στοιχείο του πίνακα με χρήση του "0", το δεύτερο με χρήση του "1" κοκ. Το ίδιο τελικό αποτέλεσμα με τον παραπάνω κώδικα, πιο συνοπτικά, θα μπορούσαμε να πετύχουμε ως εξής:

```
int[] intArray = {10,20,30}; // array variable declaration and initialization
```

Στον κώδικα αυτό, γίνεται ταυτόχρονα η δημιουργία της μεταβλητής αναφοράς του πίνακα, η δημιουργία του ίδιου του πίνακα και οι αρχικοποιήσεις τους. Τα παραπάνω παραδείγματα αφορούν μονοδιάστατους πίνακες. Μπορούμε να ορίσουμε, όμως, πίνακες οποιασδήποτε διάστασης με ανάλογους τρόπους.

Υπάρχουν διάφορες διαδικασίες που μπορούν να εφαρμοστούν σε πίνακες όπως, εύρεση του μήκους τους, αντιγραφή πινάκων, ταξινόμηση, αναζήτηση στοιχείου κ.ά. Αυτές προσφέρονται από την Java ως μέθοδοι της κλάσης `Arrays`.

Κάτι που είναι ενδιαφέρον και πρέπει, επίσης, να συζητηθεί είναι ότι οι πίνακες αναπαριστάνονται και υποστηρίζονται απευθείας από τη JVM με κατάλληλες εντολές δημιουργίας, πρόσβασης και φόρτωσης. Αυτό συμβάλλει στην ανάδειξη της κρισιμότητας της αποφυγής σφαλμάτων ως απαιτούμενο από τον σχεδιασμό της γλώσσας.

Θα ήταν παράλειψη, όμως, να μην κάνουμε μια αναφορά στον τρόπο που αναπαριστάνουμε συμβολοσειρές σε ένα πρόγραμμα Java μια που εμφανίζονται ήδη από τη μέθοδο `main` της κύριας κλάσης, ως τύπος του πίνακα του ορίσμάτος της. Για την αναπαράσταση συμβολοσειρών, η Java προσφέρει μια κλάση, την `String`, με την οποία αναπαριστά ακολουθία χαρακτήρων. Αυτές είναι ακολουθίες Unicode code points, δηλαδή ακολουθίες αριθμών όπως αυτοί αντιστοιχίζονται στους χαρακτήρες από το πρότυπο Unicode. Τα αντικείμενα της κλάσης `String` έχουν σταθερές τιμές. Δεν πρέπει να εκλαμβάνονται ως πίνακες χαρακτήρων.

Η κλάση `String` συνοδεύεται από μεθόδους για τη διεκπεραίωση διάφορων διαδικασιών που σχετίζονται με συμβολοσειρές, όπως συγκρίσεις συμβολοσειρών, αναζήτηση χαρακτήρων κ.ά., καθώς και μετατροπές κεφαλαίων-πεζών. Πρέπει, όμως, να σχολιάσουμε τον ρόλο του τελεστή συνένωσης συμβολοσειρών `+`, ο οποίος διεκπεραιώνει και τη μετατροπή αντικειμένων άλλων τύπων σε συμβολοσειρές, κάτι που έχουμε συναντήσει στα ορίσματα της μεθόδου `println` που χρησιμοποιήσαμε για τις εκτυπώσεις.

Ένα παράδειγμα ορισμού μεταβλητής συμβολοσειράς με τιμή συμβολοσειρά είναι:

```
String str = "myString";
```

Ο παρακάτω κώδικας οδηγεί στο να καταλήξουμε με την ίδια τιμή για τη μεταβλητή `str`:

```
char strchars[] = {'m','y','S','t','r','i','n','g'};
String str = new String(strchars);
```

Στην περίπτωση αυτή, η δημιουργία του αντικειμένου της συμβολοσειράς ζητήθηκε ρητά, κάνοντας την αρχικοποίησή του με χρήση πίνακα χαρακτήρων.

Ένα παράδειγμα χρήσης των συμβολοσειρών βλέπουμε στον παρακάτω κώδικα με τον οποίο πραγματοποιούμε είσοδο από τη γραμμή εντολής.

```

class App {
    public static void main(String[] args) {
        // Should check for proper number and type of args though!

        for (String s: args) {    // Use the enhanced for statement
                                // as args is array
            System.out.println(s);
        }

        // Parse string args to int
        // parse<NumType>: convert a String representing a number

        int N = args.length; // length is a public final field of array

        int[] iargs = new int[N];

        for (int i = 0; i < N; i++) {
            iargs[i] = Integer.parseInt(args[i]);
                                // Integer.parseInt parses the
                                // string argument as a signed
                                // decimal integer

            System.out.println(iargs[i]);
        }
    }
}

```

Στο παραπάνω παρατηρήστε τα εξής:

- Οι παράμετροι από τη γραμμή εντολής δίνονται ως τιμές σε έναν πίνακα συμβολοσειρών, τον `args`.
- Γίνεται χρήση μιας συμπυκμένης μορφής δομής επανάληψης (`for loop`), η οποία μπορεί να χρησιμοποιηθεί για πίνακες


```
for (String s: args) {}
```
- Χρησιμοποιείται το πεδίο `length` ενός πίνακα.
- Χρησιμοποιείται η μέθοδος `static int parseInt(String s)` της wrapper κλάσης `Integer` που δέχεται ως όρισμα μια συμβολοσειρά και επιστρέφει τον (μη προσημασμένο) δεκαδικό ακέραιο που της αντιστοιχεί. Σχετικά με τις wrapper κλάσεις θα μιλήσουμε στο τέλος του κεφαλαίου.

9.5.2 Πώς Γίνονται οι Διαδικασίες

Όσα συζητούσαμε στην προηγούμενη παράγραφο αφορούσαν δεδομένα. Ας δούμε, τώρα, τι γλωσσικά στοιχεία έχουμε για να περιγράψουμε διαδικασίες. Θα τα χρειαστούμε για να ορίσουμε τις μεθόδους των κλάσεων του προγράμματός μας. Όσον αφορά τις εκφράσεις που μπορούμε να διατυπώσουμε στην Java, καθώς και τις εντολές, το γενικό πλαίσιο ακολουθεί αυτό της C. Δηλαδή έχουμε το ρεπερτόριο εντολών της C — το περιγράψαμε στο αντίστοιχο κεφάλαιο της C++ — και χρησιμοποιούμε αντίστοιχους τελεστές για να κατασκευάζουμε εκφράσεις. Οι εντολές, και εδώ, ομαδοποιούνται σε `blocks`. Μια διαφορά που πρέπει να σημειώσουμε είναι ότι, αντίθετα με τη C++, δεν επιτρέπεται υπερφόρτωση των τελεστών. Επίσης, το ρεπερτόριο των εντολών έχει επεκταθεί με κάποιες παραλλαγές τους, ως επί το πλείστον συντακτικές.

Κάτι που είναι κρίσιμο να επισημανθεί είναι ότι το πέρασμα παραμέτρων συναρτήσεων γίνεται κατά τιμή μόνο. Για να αλλάξουμε τιμή σε ένα όρισμα πρωτογενούς τύπου, πρέπει να το κάνουμε είτε μέσω wrapping σε έναν αντίστοιχο τύπο/κλάση είτε, για παράδειγμα, με τον παρακάτω τρόπο.

```

/*
Primitive data type variables can only be passed by value.
In case you want a function to change the value of a
primitive, just return the new value and reassign it.
In more sophisticated cases, use wrapper classes.
*/

class First {

    public static void main(String[] args) {
        int i = 1;
        System.out.println(i);
        i = change(i);
        System.out.println(i);
    }

    static int change(int i) { return ++i; }
}

```

Με αφορμή το παραπάνω, να σημειώσουμε ότι για κάθε πρωτογενή τύπο, έχουμε τον αντίστοιχο wrapper τύπο, ο οποίος τον αναπαριστάει με τη μορφή κλάσης. Η κλάση αυτή έχει ένα πεδίο που ανήκει στον αντίστοιχο πρωτογενή τύπο. Για παράδειγμα, για τον πρωτογενή τύπο `int` των ακεραίων, η wrapper κλάση είναι η `Integer`. Ένα αντικείμενο της κλάσης αυτής περιέχει ένα μόνο πεδίο τύπου `int`.

Στη συνέχεια του κεφαλαίου, θα ασχοληθούμε με το πώς ορίζουμε τις δικές μας κλάσεις στην Java. Θα συζητήσουμε σχετικά με το πώς είναι συνήθως οργανωμένες, τις δυνατότητες για τις αρχικοποιήσεις των στιγμιότυπων τους, καθώς και τα εναλλακτικά που προσφέρονται ως προς την κληρονομικότητα. Θα ολοκληρώσουμε τη συζήτηση σχετικά με τις κλάσεις περιγράφοντας τα στατικά μέλη των κλάσεων (τα μέλη κλάσης).

9.6 Ορισμός Κλάσεων

Ξεκινάμε την παρουσίαση για το πώς ορίζουμε δικές μας κλάσεις στην Java, ορίζοντας μια απλή κλάση, ικανή, όμως, να αναδείξει τα βασικά συστατικά μιας κλάσης.

Στον παρακάτω κώδικα, πέρα από την κύρια κλάση, ορίζεται μια κλάση Dish. Η κύρια κλάση έχει δηλωθεί ως public. Η κλάση Dish έχει ορατότητα μόνο μέσα στο πακέτο στο οποίο έχει οριστεί.

```
public class App {  
  
    public static void main(String[] args) {  
        // Dish d = new Dish;           // error  
        // Dish d = new Dish();         // error, unless:  
                                         // - no constructor existed or  
                                         // - default constructor exists as well  
        Dish d = new Dish(100,"giaourti");  
        // System.out.println(d.name);  // If field is "private",  
                                         // this case causes error  
    }  
}
```

```
        System.out.println(d.getCalories());
    }
}

class Dish {

    // fields: class visibility

    private int calories;
    private String name;

    // constructor: package visibility

    Dish(int cal, String nam) { // CAREFUL: no use of declaring
                                // public, as class Dish has
                                // package level visibility!

        calories = cal;
        name = nam;
        System.out.println("The new dish named: " + name +
                           " contains " + calories +
                           " calories!");
    }

    // methods:
    public void eat() { // CAREFUL: no use of declaring public,
                        // as class Dish has package level visibility!

        System.out.println("Eating " + name );
    }

    public int getCalories() { // CAREFUL: no use of declaring
                                // public, as class Dish has
                                // package level visibility!

        return calories;
    }
}
```

Ας δούμε την κλάση Dish λίγο καλύτερα. Θα εστιάσουμε μόνο σε στοιχεία που κρίνονται απαραίτητα — οτιδήποτε θεωρείται προφανές, θα παραλείπεται.

Η κλάση Dish έχει:

- Δύο πεδία, ένα τύπου ακεραίου και ένα τύπου συμβολοσειράς, τα:

```
// fields: class visibility

private int calories;
private String name;
```

Προσέξτε ότι ο προσδιορισμός ορατότητας συνοδεύει το καθένα από αυτά χωριστά. Συγκεκριμένα, καθένα από αυτά έχει προσδιοριστεί ως `private`, οπότε η ορατότητά τους έχει περιοριστεί μόνο μέσα στην εμβέλεια της κλάσης.

- Μια συνάρτηση (μέθοδο) αρχικοποίησης. Η μέθοδος έχει ορατότητα πακέτου, ακολουθώντας το επίπεδο ορατότητας της κλάσης. Η μέθοδος απαιτεί δύο ορίσματα, κάτι που καταργεί τη δυνατότητα του ορισμού αντικειμένων της κλάσης Dish με default αρχικοποίηση. Αν παραλείπαμε τη συνάρτηση αυτή, η default αρχικοποίηση θα ήταν δυνατή και τα πεδία του αντικειμένου θα είχαν τις μηδενικές τιμές του τύπου τους, στην προκείμενη περίπτωση 0 και null.

```
// constructor: package visibility

Dish(int cal, String nam) { // CAREFUL: no use of declaring
                            // public, as class Dish has
                            // package level visibility!

    calories = cal;
    name = nam;
    System.out.println("The new dish named: " + name +
                        " contains " + calories +
                        " calories!");
}
```

Η μέθοδος, στο σώμα της, απλά αναθέτει στα πεδία τις τιμές των ορισμάτων της και εκτυπώνει τις τιμές τους.

Στο σώμα της main, μέσω της εντολής:

```
Dish d = new Dish(100, "giaourti");
```

ορίζεται ένα αντικείμενο της κλάσης Dish με τιμές πεδίων 100 και "giaourti" (new Dish(100, "giaourti")) και μια μεταβλητή αναφοράς τύπου Dish, η d, (Dish d) η οποία αναφέρεται στο αντικείμενο αυτό.

- Δύο μεθόδους, τη μέθοδο eat και τη μέθοδο getCalories:

```
// methods:
public void eat() { // CAREFUL: no use of declaring public,
                   // as class Dish has package level visibility!

    System.out.println("Eating " + name );
}

public int getCalories() { // CAREFUL: no use of declaring
                           // public, as class Dish has
                           // package level visibility!

    return calories;
}
```

Ενόσω η κύρια κλάση έχει δηλωθεί ως public, η κλάση Dish έχει ορατότητα μόνο μέσα στο πακέτο στο οποίο έχει οριστεί. Επομένως, για τον κώδικα αυτό ως έχει, το ότι έχουν δηλωθεί οι μέθοδοι αυτές ως public είναι περιττό, μια που στιγμιότυπα της κλάσης Dish μπορούν να οριστούν μόνο μέσα στο πακέτο.

Στο σώμα της main, βλέπουμε την εφαρμογή της μεθόδου getCalories() πάνω στο αντικείμενο στο οποίο αναφέρεται η μεταβλητή d, κατά την κλήση της μεθόδου εκτύπωσης:

```
System.out.println(d.getCalories());
```

9.6.1 Αρχικοποίηση Στιγμιότυπων στην Java

Στο προηγούμενο παράδειγμα, είδαμε συνοπτικά τα βασικά συστατικά μιας κλάσης, κάνοντας κάποια σχόλια σχετικά με την αρχικοποίηση των αντικειμένων. Ας δούμε, όμως, το θέμα της αρχικοποίησης πιο αναλυτικά, μια που η γλώσσα προσφέρει διάφορες δυνατότητες.

Μέθοδοι Αρχικοποίησης: Η αρχικοποίηση των αντικειμένων μπορεί να γίνει με χρήση μεθόδων αρχικοποίησης, κλασικά. Οι μέθοδοι αρχικοποίησης καλούνται αυτόματα με τη χρήση του τελεστή `new`. Αν δεν έχει οριστεί κάποια μέθοδος αρχικοποίησης σε μια κλάση, η Java προσφέρει μια μέθοδο αρχικοποίησης χωρίς ορίσματα, την *default μέθοδο αρχικοποίησης* (default constructor). Αυτή καλεί τη μέθοδο αρχικοποίησης της γονεϊκής κλάσης. Αν η κλάση δεν είναι δηλωμένη ως υποκλάση, καλείται η μέθοδος αρχικοποίησης της κλάσης `Object`.

Μέσα από το σώμα μιας μεθόδου αρχικοποίησης, μπορούμε να καλέσουμε μια άλλη με χρήση της λέξης-κλειδιού `this`, προσδιορίζοντας μέσω παραμέτρων ποια εννοούμε. Ένας (συντακτικός) περιορισμός που υπάρχει είναι ότι η κλήση πρέπει να γίνει πριν από οποιονδήποτε άλλο κώδικα του σώματος της μεθόδου αρχικοποίησης. Να παρατηρήσουμε εδώ ότι με τη χρήση του `this`, κατά τον τρόπο αυτό, μπορούμε να προσομοιώσουμε τη χρήση default τιμών στις συναρτήσεις αρχικοποίησης της C++: αρκεί να ορίσουμε μια default μέθοδο αρχικοποίησης στο σώμα της οποίας να καλούμε μέσω του `this` τη μέθοδο αρχικοποίησης με τις τιμές των ορισμάτων που θα θεωρούσαμε default τιμές, π.χ:

```
Dish() { this(600, "Variety"); }
```

Αρχικοποίηση Πεδίων κατά τη Δήλωσή τους: Πέρα, όμως, από τον ορισμό μεθόδων αρχικοποίησης, αρχικοποίηση των πεδίων μιας κλάσης μπορεί να γίνει κατά τη δήλωσή τους. Αυτό μπορεί να γίνει είτε με απευθείας ανάθεση αρχικών τιμών, π.χ.

```
private int calories = 1000;
private String name = "Junk food";
```

είτε μέσω χρήσης `final` μεθόδου (*final method initialization*). Για παράδειγμα:

```
varType myVar = initializeInstanceVariable();
```

όπου η μέθοδος έχει δηλωθεί ως:

```
protected final varType initializeInstanceVariable()
```

— αυτό είναι χρήσιμο αν οι υποκλάσεις θέλουν να χρησιμοποιήσουν τη μέθοδο.

Υπενθυμίζουμε ότι αν δεν έχουν δοθεί αρχικές τιμές, τα πεδία μιας κλάσης αρχικοποιούνται με τις μηδενικές τιμές του τύπου τους.

Block Αρχικοποίησης: Μια ακόμα δυνατότητα που προσφέρει η Java σχετική με τις αρχικοποιήσεις είναι η *block αρχικοποίηση* (*block initialization*), προκειμένου να μην επαναλαμβάνουμε κώδικα κοινό για κάθε μέθοδο αρχικοποίησης στο σώμα της καθεμιάς. Συγκεκριμένα, στην Java, έχουμε τη δυνατότητα να εσωκλείσουμε κώδικα μέσα σε ένα block που ορίζεται μέσω αγκύλων, `{ }` το οποίο να συμπεριλάβουμε απευθείας στον ορισμό της κλάσης. Τότε, ο μεταγλωττιστής αντιγράφει τον κώδικα του block στην αρχή οποιασδήποτε μεθόδου αρχικοποίησης υποδειχθεί κατά τη δημιουργία ενός αντικειμένου και έτσι εκτελείται πριν από τον κώδικα της μεθόδου.

Θα δούμε, τώρα, κάποια παραδείγματα για τα εναλλακτικά που μας προσφέρει η Java, σχετικά με τις αρχικοποιήσεις. Για λόγους απλότητας, στα παραδείγματά μας, πλέον, όλες οι κλάσεις που θα ορίζουμε και τα μέλη τους θα είναι ορατότητας πακέτου.

9.6.1.1 Υπερφορτωμένες Μέθοδοι Αρχικοποίησης

Στο επόμενο παράδειγμα έχουμε μια παραλλαγή του προηγούμενου κώδικα, όπου, όμως, έχουμε υπερφόρτωση στις μεθόδους αρχικοποίησης.

```
// Multiple constructors

class App {
    public static void main(String[] args) {
        Dish d = new Dish();
        Dish d1 = new Dish("hot dog");
        Dish d2 = new Dish(100, "giaourti");
    }
}

class Dish {

    private int calories = 1000;
    private String name = "Junk food";

    /*
    Dish() {
        System.out.println("The new dish named: " + name +
            " contains " + calories + " calories!");
    }

    */
    /**
    // If default construction with default values needed
    Dish() {
        this(400, "LightFood");
        System.out.println("Again: The new dish named: " + name +
            " contains " + calories + " calories!");
    }
    /**/

    Dish(String nam) {
        name = nam;
        System.out.println("The new dish named: " + name +
            " contains " + calories + " calories!");
    }

    Dish(int cal, String nam) {
        calories = cal;
        name = nam;
        System.out.println("The new dish named: " + name +
            " contains " + calories + " calories!");
    }
}
```



```

    void eat() {
        System.out.println("Eating " + name );
    }

    int getCalories() {
        return calories;
    }
}

```

Συγκεκριμένα, ορίστηκαν, τώρα, τρεις μέθοδοι αρχικοποίησης, οι:

```

        Dish();
        Dish(String nam);

```

και

```

        Dish(int cal, String nam);

```

Κατά συνέπεια, στη μέθοδο main της κυρίας κλάσης και τα τρία παρακάτω εναλλακτικά είναι επιτρεπτά:

```

Dish d = new Dish();
Dish d1 = new Dish("hot dog");
Dish d2 = new Dish(100, "giaourti");

```

Στο παράδειγμα, η default μέθοδος αρχικοποίησης

```

Dish() {
    this(400, "LightFood");
    System.out.println("Again: The new dish named: " + name +
        " contains " + calories + " calories!");
}

```

χρησιμοποιεί τη μέθοδο αρχικοποίησης με δύο ορίσματα δίνοντάς τους σταθερές ως τιμές. Αν σχολιάσουμε αυτόν τον ορισμό default αρχικοποίησης, όμως, και αποσχολιάσουμε τον ορισμό

```

Dish() {
    System.out.println("The new dish named: " + name +
        " contains " + calories + " calories!");
}

```

εκτελώντας το πρόγραμμα, θα δείτε ότι οι τιμές των πεδίων που εκτυπώνονται είναι αυτές που προέκυψαν από τις αρχικοποιήσεις:

```

private int calories = 1000;
private String name = "Junk food";

```

9.6.1.2 Χρήση Block Αρχικοποίησης

Στο παρακάτω πρόγραμμα, αναδεικνύεται η συμπεριφορά της χρήσης block αρχικοποίησης.

```

class App {
    public static void main(String[] args) {
        Dish d = new Dish(100, "giaourti");
        d.eat();
        System.out.println("Actually you got: " +
            d.getCalories() + " calories!");
    }
}

```

```

}

class Dish {
    int calories = 2000;
    String name = "What you have to eat";

    // Block initialization
    { System.out.println("The new dish named: " + name +
                        " contains " + calories + " calories!"); }

    // NOT WHAT WE WANTED - it is copied in the beginning

    Dish(int cal, String nam) {
        calories = cal;
        name = nam;
        System.out.println("The new dish named: " + name +
                        " contains " + calories + " calories!");
    }

    void eat() {
        System.out.println("Eating " + name);
    }

    int getCalories() {
        return calories;
    }
}

```

Στο παράδειγμα αυτό, έχουμε μια άλλη παραλλαγή του παραδείγματος στην οποία:

- Γίνεται αρχικοποίηση των πεδίων της κλάσης Dish κατά τη δήλωσή τους:

```

int calories = 2000;
String name = "What you have to eat";

```

- Στην κλάση Dish υπάρχει block αρχικοποίησης, το:

```

{ System.out.println("The new dish named: " + name +
                    " contains " + calories + " calories!"); }

```

- Στην κλάση Dish υπάρχει μια μέθοδος αρχικοποίησης:

```

Dish(int cal, String nam) {
    calories = cal;
    name = nam;
    System.out.println("The new dish named: " + name +
                        " contains " + calories + " calories!");
}

```

Να υπενθυμίσουμε ότι η εκτέλεση του κώδικα του block αρχικοποίησης προηγείται του κώδικα του σώματος της μεθόδου αρχικοποίησης, άρα οι τιμές των πεδίων που θα εκτυπωθούν δεν θα είναι εκείνες που θα

ανατεθούν ενδεχόμενα από τη μέθοδο, αλλά εκείνες που έχουν προκύψει από τις δηλώσεις των πεδίων — συγκεκριμένες ή οι μηδενικές.

Επομένως, η εκτύπωση που προκύπτει από την εκτέλεση του προγράμματος είναι:

The new dish named: What you have to eat contains 2000 calories!

The new dish named: giaourti contains 100 calories!

Eating giaourti

Actually you got: 100 calories!

Η εκτύπωση αυτή προκύπτει ως εξής. Κατά τον ορισμό του αντικειμένου της κλάσης Dish (new Dish(100, "giaourti")), εξαιτίας της κλήσης του block αρχικοποίησης και με βάση τις τιμές με τις οποίες έχουν αρχικοποιηθεί τα πεδία κατά τη δήλωση, προκύπτει η εκτύπωση

The new dish named: What you have to eat contains 2000 calories!

Στη συνέχεια, από την εκτύπωση από το σώμα της μεθόδου αρχικοποίησης προκύπτει η εκτύπωση

The new dish named: giaourti contains 100 calories!

Κατόπιν, η κλήση d.eat(); παράγει την εκτύπωση

Eating giaourti

λόγω της τρέχουσας τιμής του πεδίου name του αντικειμένου στο οποίο αναφέρεται η μεταβλητή d. Τέλος, η κλήση d.getCalories() επιστρέφει την τιμή του πεδίου calories, η οποία εμφανίζεται στην εκτύπωση

Actually you got: 100 calories!

9.6.1.3 Παράδειγμα Αρχικοποίησης Στιγμιοτύπων

Ας δούμε, τώρα, μέσω παραδείγματος, μια πιο πολύπλοκη διαδικασία αρχικοποίησης στιγμιοτύπων. Στην περίπτωση του παραδείγματος που ακολουθεί, έχουμε:

- αρχικοποίηση πεδίων κατά τη δήλωσή τους
- block αρχικοποίησης
- πολλαπλές μεθόδους αρχικοποίησης
- μετάβαση σε άλλη μέθοδο αρχικοποίησης με χρήση της λέξης-κλειδιού this

```
// Block Initialization
// Multiple constructors and constructor switching

class App {
    public static void main(String[] args) {
        Dish d = new Dish();
    }
}

class Dish {

// Initialization with no default values:
    int calories = 1000;
    String name = "Junk food";
    { System.out.println("From block: The new dish named: " + name +
        " contains " + calories + " calories!"); }

    // NOT WHAT WE WANTED - it is copied in the beginning
```

```
// If default construction with default values needed
Dish() {
    this(400, "LightFood");
    System.out.println("Again: The new dish named: " + name +
        " contains " + calories + " calories!");
}

Dish(String nam) {
    name = nam;
    System.out.println("The new dish named: " + name +
        " contains " + calories + " calories!");
}

Dish(int cal, String nam) {
    calories = cal;
    name = nam;
    System.out.println("The new dish named: " + name +
        " contains " + calories + " calories!");
}

void eat() {
    System.out.println("Eating " + name );
}

int getCalories() {
    return calories;
}
}
```

Κατά την εκτέλεση του παραπάνω προγράμματος, η εκτύπωση που προκύπτει είναι:

From block: The new dish named: Junk food contains 1000 calories!

The new dish named: LightFood contains 400 calories!

Again: The new dish named: LightFood contains 400 calories!

Το πρόγραμμά μας το μόνο που κάνει είναι να δημιουργεί ένα αντικείμενο τύπου Dish, αρχικοποιώντας το με χρήση της μεθόδου αρχικοποίησης Dish(), και να το αναθέτει ως τιμή στη μεταβλητή αναφοράς τύπου Dish, την d. Με βάση, όμως, αυτά που μόλις συζητήσαμε, δεδομένου ότι στο σώμα της εν λόγω μεθόδου αρχικοποίησης υπάρχει η χρήση του this(400, "LightFood"), θα κληθεί και η μέθοδος αρχικοποίησης Dish(int cal, String nam). Δεδομένου, όμως, ότι υπάρχει και block αρχικοποίησης, πριν την εκτέλεση του σώματος της τελευταίας μεθόδου, θα εκτελεστεί και ο κώδικας που υπάρχει στο block. Τα πεδία του στιγμιότυπου έχουν ήδη αρχικοποιηθεί με συγκεκριμένες τιμές, όπως φαίνονται στις δηλώσεις:

```
int calories = 1000;
String name = "Junk food";
```

Οπότε,

- η εκτύπωση
From block: The new dish named: Junk food contains 1000 calories!

προκύπτει από την εκτέλεση του block με τρέχουσες τιμές των πεδίων τις τιμές που δίνονται κατά τη δήλωσή τους,

- η εκτύπωση

The new dish named: LightFood contains 400 calories!

προκύπτει από την εκτέλεση του σώματος της μεθόδου αρχικοποίησης Dish(int cal, String nam) για τιμές ορισμάτων 400 και "LightFood"

- και η εκτύπωση

Again: The new dish named: LightFood contains 400 calories!

προκύπτει από την εκτέλεση του σώματος της μεθόδου αρχικοποίησης Dish().

9.6.1.4 Βήματα κατά τη Διαδικασία Αρχικοποίησης Στιγμιότυπων Κλάσεων

Κατά τη δημιουργία ενός αντικειμένου/στιγμιότυπου μιας κλάσης, η οποία δεν είναι δηλωμένη ρητά ως υποκλάση, τα βήματα κατά την αρχικοποίησή του έχουν ως εξής:

1. Αρχικά, δίνονται στα πεδία του οι default μηδενικές τιμές των τύπων τους.
2. Στη συνέχεια, γίνεται υπολογισμός των ορισμάτων της μεθόδου αρχικοποίησης που προσδιορίζεται (η αρχική είναι εκείνη που χρησιμοποιείται κατά τη δημιουργία του μέσω του τελεστή new) .
3. Κατόπιν, στην περίπτωση που υπάρχει εμφάνιση της λέξης-κλειδιού this, οδηγούμαστε στο Βήμα 2 για τη μέθοδο αρχικοποίησης που προσδιορίζεται από τη χρήση του this.
4. Όταν δεν υπάρχει εμφάνιση της λέξης-κλειδιού this, γίνεται αρχικοποίηση των πεδίων σύμφωνα με τις δηλώσεις τους, αν περιέχουν ρητές αρχικοποιήσεις και, στη συνέχεια, εκτελείται ο κώδικας από τα block αρχικοποίησης με τη σειρά που εμφανίζονται στον ορισμό της κλάσης.
5. Ακολουθεί η εκτέλεση του κώδικα των μεθόδων αρχικοποίησης με αντίστροφη σειρά. Δηλαδή, η εκτέλεση ξεκινά από το σώμα εκείνης που προσδιορίστηκε πιο πρόσφατα και καταλήγει στο σώμα εκείνης που χρησιμοποιήθηκε στην έκφραση που εμφανίστηκε ο τελεστής new για τη δημιουργία του αντικειμένου.

9.7 Κληρονομικότητα στην Java

Στην Java η έννοια της κληρονομικότητας είναι θεμελιώδης. Η γλώσσα προσφέρει, μέσω του API, μια έτοιμη ιεραρχία κλάσεων όπου όλες οι κλάσεις είναι υποκλάσεις της κλάσης Object. Ο προγραμματιστής ορίζει τις δικές του κλάσεις ως υποκλάσεις κλάσεων. Πιο συγκεκριμένα, ορίζει μια κλάση είτε δηλώνοντάς την ρητά ως υποκλάση μιας άλλης κλάσης, είτε δεν υπάρχει τέτοια σύνδεση στον ορισμό της, οπότε η κλάση θεωρείται άμεση υποκλάση της κλάσης Object. Τελικά, κάθε κλάση θεωρείται (άμεση ή έμμεση) υποκλάση της κλάσης Object.

Παρότι δεν επιμείναμε σε αυτό κατά την παρουσίαση της C++, χρησιμοποιώντας άλλες λέξεις-κλειδιά (protected, private), πέρα από τη λέξη-κλειδί public, η ονομασία "κληρονομικότητα" στη C++ επεκτείνεται και σε συσχετίσεις κλάσεων που δεν απεικονίζουν εννοιολογική συσχέτιση γενίκευσης-εξειδίκευσης, απλά διευκολύνουν στην υλοποίηση. Αντίθετα, στην Java η κληρονομικότητα αναφέρεται αποκλειστικά στην εννοιολογική συσχέτιση που απεικονίζει τη σχέση *isa*.

9.7.1 Γενικά Στοιχεία

Η λέξη-κλειδί που χρησιμοποιείται, στην Java, για να δηλώσουμε ότι μια κλάση είναι υποκλάση μιας άλλης είναι `extends`. Για παράδειγμα, με το παρακάτω

```
class B extends A
```

ορίζουμε μια κλάση B ως υποκλάση της κλάσης A, εννοώντας ότι

```
class B isa class A
```

Με την παραπάνω συσχέτιση, κάθε αντικείμενο της κλάσης B μπορεί να θεωρηθεί ως αντικείμενο της κλάσης A. Μεταξύ άλλων, αυτό επιτρέπει:

- το αντικείμενο να δοθεί ως τιμή μιας μεταβλητής αναφοράς τύπου A, δηλαδή:

```
A varA = new B();
```
- στον βαθμό που επιτρέπεται από τις δηλώσεις ορατότητας, μέσα από την εμβέλεια της κλάσης B να χειριστούμε μέλη της κλάσης A και
- από τον “έξω κόσμο”, δηλαδή από περιβάλλοντα άλλων κλάσεων, να εφαρμόσουμε μεθόδους της κλάσης A στα αντικείμενα της κλάσης B.

Πρέπει να επιμείνουμε λίγο σε ό,τι αφορά την εφαρμογή των μεθόδων στα αντικείμενα. Στη γενική περίπτωση, εφαρμόζουμε μια μέθοδο σε ένα αντικείμενο, χρησιμοποιώντας μια μεταβλητή αναφοράς. Τι γίνεται, όμως, στην περίπτωση που πρόκειται για μια μη-στατική μέθοδο όπου ο τύπος της μεταβλητής αναφοράς είναι υπερκλάση της κλάσης του αντικειμένου και η κλάση του αντικειμένου έχει επαναορίσει τη μέθοδο; Για παράδειγμα,

```
class App {
    public static void main(String[] args) {
        A var = new B();
        var.bla();
    }
}

class A {
    void bla() { System.out.println("Executing the A bla()"); }
}

class B extends A {
    void bla() { System.out.println("Executing the B bla()"); }
}
```

Στη C++, όταν η εφαρμογή μιας συνάρτησης-μέλους γίνεται μέσω δεικτών ή αναφορών τύπου υπερκλάσης, γίνεται διάκριση σε σχέση με το εάν η συνάρτηση που είναι ορατή στην υπερκλάση έχει δηλωθεί ως `virtual` ή όχι. Αντίθετα, στην Java, όταν καλείται μια μη στατική μέθοδος, πάντα γίνεται `method dispatch` και εκτελείται η μέθοδος που είναι ορατή στην κλάση του αντικειμένου. Είναι, δηλαδή, σαν να έχει δηλωθεί `virtual` στην υπερκλάση και έτσι έχουμε αυτόματα πολυμορφισμό. Οπότε, κατά την εκτέλεση του παραπάνω παραδείγματος, η εκτύπωση είναι:

Executing the B bla()

Να σημειώσουμε ότι για τις στατικές μεθόδους, όμως, πολυμορφισμός δεν υφίσταται, διότι δεν θεωρείται ότι εφαρμόζονται σε συγκεκριμένο αντικείμενο.

Να διευκρινίσουμε εδώ τον τρόπο με τον οποίο αναφερόμαστε σε μια επαναορισμένη μη-στατική μέθοδο υπερκλάσης από το σώμα του ορισμού μεθόδου που ανήκει σε υποκλάση που την επαναορίζει. Στη C++ χρησιμοποιούσαμε τον τελεστή επίλυσης εμβέλειας, `::`, και προσδιορίζοντας τη γονεϊκή κλάση, είχαμε πρόσβαση

στη δική της συνάρτηση-μέλος. Στην Java, η πρόσβαση σε μεθόδους γίνεται με πιο περιορισμένο τρόπο: δεν μπορούμε να χρησιμοποιούμε ονόματα κλάσεων για να έχουμε άμεση πρόσβαση σε μεθόδους τους, εκτός αν πρόκειται για στατικές μεθόδους. Οπότε, τι κάνουμε στην παραπάνω περίπτωση; Μέσα στο σώμα μιας μεθόδου της υποκλάσης, μπορούμε να χρησιμοποιήσουμε τη λέξη-κλειδί `super` ώστε να κληθεί η επαναορισμένη μέθοδος της άμεσης υπερκλάσης της. Αυτή είναι και η μόνη περίπτωση πρόσβασης σε επαναορισμένη μη στατική μέθοδο σε μια κλάση. Στο παραπάνω παράδειγμα, αν θέλαμε να κληθεί η μέθοδος `bla` της κλάσης `A` από το σώμα του ορισμού της `bla()` στην κλάση `B`, ο κώδικας θα ήταν:

```
class App {
    public static void main(String[] args) {
        A var = new B();
        var.bla();
    }
}

class A {
    void bla() { System.out.println("Executing the A bla()"); }
}

class B extends A {
    void bla() {
        super.bla();    // Calling superclass bla()
        System.out.println("Executing the B bla()");
        super.bla();    // Calling superclass bla()
    }
}
```

Η εκτύπωση που θα προέκυπτε από την εκτέλεση του παραπάνω προγράμματος είναι:

```
Executing the A bla()
Executing the B bla()
Executing the A bla()
```

Κάποια παρατήρηση που πρέπει να γίνει είναι ότι δεν μπορούμε να επαναορίσουμε σε μια υποκλάση `public` μεθόδους, χωρίς να τις δηλώσουμε πάλι ως `public`. Γενικότερα, δεν μπορούμε να επαναορίσουμε τις μεθόδους στις υποκλάσεις, δίνοντας ασθενέστερα δικαιώματα ορατότητας. Αναμενόμενο, επίσης, είναι να μην επιτρέπεται να χρησιμοποιηθεί το ίδιο όνομα για στατικά και μη στατικά μέλη στην ίδια κλάση ή σε επαναορισμούς στις υποκλάσεις της.

Θα κάνουμε, εδώ, μια αναφορά στη σημασία κάποιων όρων που συναντάμε στην Java στο πλαίσιο της κληρονομικότητας. Στην ορολογία της Java συναντάμε τους όρους *overriding* και *hiding*, εννοώντας την αλλαγή ορατότητας λόγω επαναορισμού μεθόδων:

overriding: μεταξύ μη στατικών μεθόδων που επαναορίζονται σε υποκλάση: το αν καλείται η μέθοδος της υποκλάσης ή της υπερκλάσης εξαρτάται από τον τύπο του αντικειμένου πάνω στο οποίο εφαρμόζεται η μέθοδος

hiding: μεταξύ στατικών μεθόδων που επαναορίζονται σε υποκλάση — σε περίπτωση που η κλήση γίνεται με χρήση μεταβλητής αναφοράς, το αν καλείται η μέθοδος της υποκλάσης ή της υπερκλάσης εξαρτάται από τον τύπο της μεταβλητής (και όχι του αντικειμένου).

Με την ευκαιρία αυτή, αναφέρουμε και τον όρο *shadowing*, που αφορά την αλλαγή κατάστασης ορατότητας μεταξύ μεταβλητών στενότερης και ευρύτερης εμβέλειας.

9.7.2 Αφηρημένες Κλάσεις

Υπάρχει περίπτωση να θέλουμε να ορίσουμε μια κλάση, αλλά να μην θέλουμε να δημιουργήσουμε στιγμιότυπά της. Στην περίπτωση αυτή, όπως ήδη έχουμε πει, μιλάμε για *αφηρημένες κλάσεις* και τις χρησιμοποιούμε κυρίως ως κοινές προδιαγραφές μιας γενικής κατηγορίας κλάσεων και αποφυγή επανάληψης κώδικα. Στη C++, οι αφηρημένες κλάσεις είδαμε να ορίζονται με έμμεσο τρόπο, δηλώνοντας τουλάχιστον μια συνάρτηση-μέλος ως πλήρως εικονική. Στην Java, όμως, ο προσδιορισμός μιας κλάσης ως αφηρημένη γίνεται άμεσα στη δήλωσή της, με χρήση της λέξης-κλειδιού `abstract`. Για παράδειγμα, ορίζοντας μια κλάση `AbsClas` ως:

```
abstract class AbsClas{... }
```

απαγορεύουμε να δημιουργηθούν αντικείμενά της. Δηλαδή, η εντολή

```
new AbsClas(); // ERROR!!!!
```

θα προκαλέσει σφάλμα μεταγλώττισης. Το σώμα μιας αφηρημένης κλάσης μπορεί να περιέχει κώδικα που θα περιείχε μια κοινή κλάση. Όμως, μπορούμε επιπλέον να συμπεριλάβουμε και μεθόδους των οποίων δίνουμε μόνο το πρωτότυπό τους και όχι το σώμα του ορισμού τους. Σε αυτήν την περίπτωση, πρέπει να προσημάνουμε τη δήλωση της μεθόδου με τη λέξη-κλειδί `abstract`. Για παράδειγμα,

```
abstract class AbsClas {
    abstract void commonMethod();
}
```

Αν μια κλάση περιέχει αφηρημένες (`abstract`) μεθόδους, υποχρεωτικά η κλάση πρέπει να δηλωθεί ως `abstract`. Αυτό είναι αναμενόμενο: δεν θα ήταν λογικό να ορίσουμε αντικείμενα μιας κλάσης η οποία έχει μεθόδους στιγμιότυπου χωρίς καθορισμένη συμπεριφορά. Όσον αφορά στατικές μεθόδους, όμως, αυτές δεν μπορούν να δηλωθούν ως `abstract` (και να μην οριστούν), κάτι που, και πάλι, είναι αναμενόμενο, μια που δεν σχετίζονται με αντικείμενα, αλλά με την ίδια την κλάση.

Μια αφηρημένη κλάση υπόκειται στον ίδιο περιορισμό με τις κοινές κλάσεις σε σχέση με την κληρονομικότητα: μια κλάση είναι άμεση υποκλάση ακριβώς μίας κλάσης, ανεξάρτητα αν αυτή είναι αφηρημένη ή όχι.

Αναφέροντας επιγραμματικά κάποια σημεία που αφορούν τις αφηρημένες κλάσεις να πούμε ότι

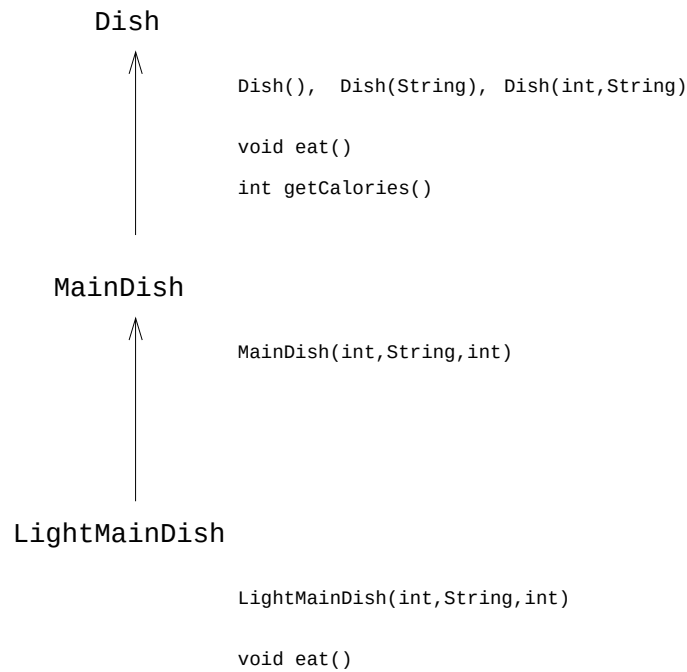
- δεν μπορούμε να ορίσουμε αντικείμενά τους,
- μπορούμε να έχουμε μεθόδους που δεν έχουν υλοποίηση (`abstract` μεθόδους), αλλά και μεθόδους που έχουν σώμα,
- τα πεδία μπορεί να είναι οποιασδήποτε κατηγορίας (στατικά - μη στατικά, `final` - όχι `final`) και ορατότητας και
- οι μέθοδοι μπορεί να είναι οποιασδήποτε κατηγορίας και ορατότητας

Μια παρατήρηση: αν μια αφηρημένη κλάση έχει μια αφηρημένη μέθοδο, μια υποκλάση της είτε πρέπει να δίνει ορισμό στη μέθοδο, είτε και αυτή να προσδιοριστεί ως αφηρημένη κλάση. Αναμενόμενο, επίσης, είναι μια αφηρημένη μέθοδος να μην μπορεί να δηλωθεί ως `final`.

9.7.3 Παραδείγματα

Προκειμένου να γίνουν κατανοητά όσα συζητήσαμε προηγουμένως, ας πειραματιστούμε με κάποια παραδείγματα. Θα χρησιμοποιήσουμε και πάλι την κλάση `Dish` και θα την “επεκτείνουμε” ορίζοντας μια υποκλάση της, την `MainDish`. Επίσης, θα ορίσουμε και μια ακόμα κλάση, την `LightMainDish`, σαν υποκλάση της κλάσης `MainDish`. Συνοπτικά, οι κλάσεις, οι μέθοδοί τους και η ιεραρχία τους απεικονίζονται στο Σχήμα 9.3.

Η κλάση `MainDish` προσθέτει ένα ακόμα πεδίο στιγμιότυπου και η κλάση `LightMainDish` επαναορίζει τη μέθοδο `void eat()`. Και οι δύο νέες κλάσεις έχουν ορίσει μεθόδους αρχικοποίησης.



Σχήμα 9.3: Ένα παράδειγμα ιεραρχίας κλάσεων

Στο παράδειγμα που ακολουθεί, προσέξτε στον ορισμό της μεθόδου `void eat()` της κλάσης `LightMainDish` τη χρήση της λέξης-κλειδιού `super`. Όπως είπαμε, αυτός είναι ο τρόπος να κληθεί η μέθοδος που είναι ορατή στη γονεϊκή κλάση.

Και οι δύο νέες κλάσεις έχουν ορίσει μεθόδους αρχικοποίησης. Προσέξτε πάλι τη χρήση της λέξης-κλειδιού `super` στα σώματά τους. Αυτήν τη φορά εκφράζει χρήση της μεθόδου αρχικοποίησης της γονεϊκής κλάσης. Συγκεκριμένα, δηλώνει τη χρήση εκείνης της μεθόδου αρχικοποίησης της γονεϊκής που ταιριάζει με το πλήθος και τον τύπο των ορισμάτων που χρησιμοποιούνται. Έτσι, γίνεται και το πέρασμα τιμών στη μέθοδο για την αρχικοποίηση της φύσης του αντικειμένου ως αντικείμενο της γονεϊκής κλάσης. Στην περίπτωση που δεν εμφανιζόταν η λέξη-κλειδί `super` στα σώματα των μεθόδων αρχικοποίησης των υποκλάσεων, υπονοείται η χρήση της default αρχικοποίησης της υπερκλάσης.

```

class App {
    public static void main(String[] args) {
        Dish d = new Dish();
        MainDish md = new MainDish(600, "mbrizola", 8);
        LightMainDish lmd = new LightMainDish(300, "Rizogalo", 2);

        md.eat();
        lmd.eat();
        // Dish d;
        d = lmd;
        d.eat();
    }
}

// abstract class Dish {
class Dish {
    int calories = 1000;

```

```

String name = "Junk food";

Dish() {
    System.out.println("The new dish named: " + name +
        " contains " + calories + " calories!");
}

Dish(String nam) {
    name = nam;
    System.out.println("The new dish named: " + name +
        " contains " + calories + " calories!");
}

Dish(int cal, String nam) {
    calories = cal;
    name = nam;
    System.out.println("The new dish named: " + name +
        " contains " + calories + " calories!");
}

void eat() {
    System.out.println("Eating " + name );
}

int getCalories() {
    return calories;
}
}

class MainDish extends Dish {
    int cost = 6;

    MainDish(int cal, String nam, int cst) {
        super(cal, nam);
        cost = cst;
        System.out.println("The cost of the main dish is " + cost);
    }
}

class LightMainDish extends MainDish {
    LightMainDish(int cal, String nam, int cst) {
        super(cal, nam, cst);
        System.out.println("A LightMainDish has been created");
    }

    void eat() {
        super.eat();
        System.out.println("Eating a Light Main Dish");
    }
}

```

}

Στη μέθοδο `main` της κύριας κλάσης, ορίζουμε τρία αντικείμενα: ένα της κλάσης `Dish`, ένα της κλάσης `MainDish` και ένα της κλάσης `LightMainDish` και τα αναθέτουμε ως τιμές των μεταβλητών αναφοράς, `d`, `md` και `lmd`, του αντίστοιχου τύπου.

Κατόπιν, καλούμε τη μέθοδο `eat()`, εφαρμόζοντάς την διαδοχικά στις μεταβλητές αναφοράς `md` και `lmd`. Κατά τη διαδικασία αυτή έχουμε:

- την εκτέλεση της μεθόδου `eat()` της κλάσης `Dish`, μια που κληρονομείται από την κλάση `MainDish`, και
- την εκτέλεση της μεθόδου `eat()` της κλάσης `LightMainDish`, η οποία προκαλεί την εκτέλεση της μεθόδου `eat()` που είναι ορατή στην υπερκλάση της, την `MainDish`, που, όπως μόλις είπαμε, είναι η μέθοδος `eat()` της κλάσης `Dish`.

Τέλος, αναθέτουμε στη μεταβλητή αναφοράς `d` το αντικείμενο στο οποίο αναφέρεται η μεταβλητή `lmd`. Αυτό είναι τύπου `LightMainDish`, κάτι που οδηγεί στο να κληθεί η μέθοδος `eat()` της κλάσης `LightMainDish` κατά την τελευταία κλήση του σώματος της `main`, `d.eat()`;

Στον κώδικα του παραδείγματος, βλέπουμε σχολιασμένες

1. τη γραμμή

```
// Dish d;
```

2. και τη γραμμή

```
// abstract class Dish {
```

Αν αποσχολιάσουμε τη δεύτερη — και σχολιάσουμε την εναλλακτική επικεφαλίδα της κλάσης —, τότε δηλώνουμε ότι η κλάση `Dish` είναι αφηρημένη. Οπότε, δεν μπορούμε να ορίσουμε αντικείμενά της. Επομένως, η έκφραση `new Dish()` προκαλεί σφάλμα, οπότε η γραμμή

```
Dish d = new Dish();
```

πρέπει να σχολιαστεί. Προκειμένου να λειτουργήσει το πρόγραμμα, αρκεί να αποσχολιαστεί η πρώτη γραμμή. Στην κατάσταση που προκύπτει λόγω της ανάθεσης `d = lmd`, έχουμε μεταβλητή αναφοράς τύπου αφηρημένης κλάσης να αναφέρεται σε αντικείμενο υποκλάσης της. Αυτή είναι μια από τις χρήσεις των αφηρημένων κλάσεων, ιδίως αν θέλουμε να επεξεργαστούμε αντικείμενα διαφορετικών υποκλάσεων συλλογικά.

Στο σημείο αυτό, ας συνοψίσουμε τη χρήση των λέξεων-κλειδιών `this` και `super`.

Η λέξη-κλειδί `this`: αναφέρεται στο τρέχον αντικείμενο.

- Μπορεί να εμφανίζεται στο σώμα μιας μεθόδου αρχικοποίησης, για να δηλώσει χρήση άλλης μεθόδου αρχικοποίησης της ίδιας κλάσης.
- Μπορεί να εμφανίζεται στο σώμα (μη στατικών) μεθόδων, για διάκριση του αντικειμένου από άλλα του τύπου του.

Η λέξη-κλειδί `super`: αναφέρεται στο τρέχον αντικείμενο, βλέποντάς το ως στιγμιότυπο της υπερκλάσης της κλάσης στην οποία ανήκει.

- Μπορεί να εμφανίζεται στο σώμα μιας μεθόδου αρχικοποίησης, για να δηλώσει χρήση μεθόδου αρχικοποίησης της γονεϊκής κλάσης.
- Μπορεί να χρησιμοποιηθεί για πρόσβαση σε μέλη της γονεϊκής κλάσης (μεθόδους και πεδία) που έχουν επαναοριστεί στην κλάση του.

Οι λέξεις-κλειδιά `this` και `super` δεν μπορούν να χρησιμοποιηθούν από στατικά μέλη. Επίσης, δεν είναι επιτρεπτό στο σώμα του ορισμού μιας μεθόδου αρχικοποίησης μιας κλάσης να χρησιμοποιούνται και τα δύο: αν είναι επιθυμητή η ρητή έκφραση χρήσης άλλης μεθόδου αρχικοποίησης, είτε το `this` θα εμφανίζεται είτε το `super`.

Ας κάνουμε κάποιες ακόμα παρατηρήσεις. Στον κώδικα που ακολουθεί βλέπετε μια παραλλαγή του ορισμού της κλάσης `LightMainDish` που ορίσαμε προηγουμένως.

```
class LightMainDish extends MainDish {

    LightMainDish(int cal, String nam, int cst) {
        super(cal, nam, cst);
        System.out.println("A LightMainDish has been created");
    }

    void eat() {
        System.out.println("Mother class field: " +
                           super.calories); // OK
//      System.out.println("Mother class field: " +
//                          MainDish.calories); // Syntax NOT OK:
//                                          // no static member
        super.eat(); // OK inherited method call

//      super.super.getCalories(); // NOT ALLOWED!
//                               // MainDish should have defined
//                               // a method to call method
//                               // getCalories() of Dish
        System.out.println("Eating a Light Main Dish");
    }
}
```

Στον κώδικα αυτό μπορούμε να παρατηρήσουμε τα εξής:

- Στην εντολή

```
System.out.println("Mother class field: " + super.calories); // OK
```

υπάρχει η έκφραση `super.calories` η οποία δηλώνει ρητά την πρόσβαση στο πεδίο `calories` που είναι ορατό στη γονεϊκή κλάση `MainDish`.

- Αν αποσχολιάζαμε, όμως, τον κώδικα:

```
//      System.out.println("Mother class field: " +
//                          MainDish.calories); // Syntax NOT OK:
//                                          // no static member
```

η έκφραση `MainDish.calories` θα προκαλούσε σφάλμα μεταγλώττισης, διότι στην κλάση `MainDish` δεν υπάρχει ορατό στατικό μέλος με όνομα `calories` — και δεν θα μπορούσε, αφού έχουμε πεδίο στιγμιοτύπου με το ίδιο όνομα.

Δεν είναι δυνατόν να έχουμε πρόσβαση σε επαναορισμένη προγονική μέθοδο στιγμιοτύπου. Για παράδειγμα, η έκφραση

```
super.super.getCalories(); // NOT ALLOWED!
```

προκαλεί σφάλμα. Στην περίπτωση που μια κλάση η οποία επαναορίζει μια κληρονομημένη μέθοδο θέλει να δώσει τη δυνατότητα στις υποκλάσεις της να μπορούν να χρησιμοποιήσουν και τη μέθοδο της γονεικής, θα πρέπει να ορίσει ξεχωριστή μέθοδο, η οποία θα την καλεί, με χρήση της λέξης-κλειδιού `super`. Για παράδειγμα, για να μπορεί να χρησιμοποιηθεί από τις υποκλάσεις της κλάσης `LightMainDish` η μέθοδος `void eat()` της κλάσης `MainDish`, θα μπορούσαμε να ορίζαμε μια άλλη μέθοδο στην κλάση `LightMainDish`, την:

```
• void eatFromMainDish() {
    super.eat();
}
```

9.7.4 Διαδικασία Αρχικοποίησης Στιγμιότυπων Κλάσεων: η Γενική Περίπτωση

Στην παράγραφο αυτή θα συνοψίσουμε τη γενική διαδικασία που ακολουθείται κατά την αρχικοποίηση στιγμιότυπων κλάσεων, λαμβάνοντας υπόψη και την κληρονομικότητα.

- Όταν δημιουργείται ένα στιγμιότυπο μιας κλάσης, δεσμεύεται χώρος ώστε να χωράνε όλα τα μη στατικά πεδία του — οι μεταβλητές στιγμιότυπου —, τόσο αυτά που δηλώνονται στην κλάση αυτή, όσο και αυτά που δηλώνονται σε όλη την ιεραρχία των υπερκλάσεων της.
- Τα πεδία αυτά αρχικοποιούνται με τις default τιμές του τύπου τους.
- Στη συνέχεια, εκτελείται η παρακάτω διαδικασία:
 1. Γίνεται υπολογισμός και ανάθεση ορισμάτων στις παραμέτρους της μεθόδου αρχικοποίησης που χρησιμοποιείται (αρχικά εκείνης που χρησιμοποιείται μέσω του τελεστή `new`).
 2. Αν το σώμα της μεθόδου αρχικοποίησης ξεκινά με κλήση (άλλης) μεθόδου αρχικοποίησης της κλάσης του στιγμιότυπου (χρησιμοποιώντας τη λέξη-κλειδί `this`), εφάρμοσε τη διαδικασία για τη μέθοδο αυτή από το βήμα 1 και, κατόπιν, μεταπήδησε στο βήμα 5 για την αρχική μέθοδο.
 3. Αν δεν ισχύει το παραπάνω και η κλάση δεν είναι η `Object`, τότε η εκτέλεση του σώματος της μεθόδου αρχικοποίησης ξεκινά με κλήση μεθόδου αρχικοποίησης της υπερκλάσης (είτε έμμεσα είτε άμεσα χρησιμοποιώντας τη λέξη-κλειδί `super`) ξεκινώντας τη διαδικασία αρχικοποίησης γι' αυτήν τη μέθοδο αρχικοποίησης από το βήμα 1.
 4. Εκτελούνται οι αρχικοποιήσεις των πεδίων στιγμιότυπου και οι συναρτήσεις αρχικοποίησης για τα πεδία του στιγμιότυπου. Τηρείται η σειρά με την οποία εμφανίζονται στο κείμενο του ορισμού της κλάσης. Εκτελούνται τα `block` αρχικοποίησης.
 5. Εκτελείται το υπόλοιπο σώμα της μεθόδου αρχικοποίησης.
- Μετά την παραπάνω διαδικασία αρχικοποίησης, επιστρέφεται η αναφορά στο αντικείμενο αυτό.

9.8 Μεταβλητές Αναφοράς: Αναθέσεις και Χρήση σε Μεθόδους Στιγμιότυπου

Στο σημείο αυτό θα μιλήσουμε σχετικά με τις επιτρεπτές αναθέσεις τιμών σε μεταβλητές αναφοράς και για τις επιτρεπτές εφαρμογές μεθόδων στιγμιότυπου μέσω αυτών των μεταβλητών. Η συζήτηση θα γίνει με χρήση παραδείγματος.

Στο παράδειγμα, αλλάζουμε λίγο την ιεραρχία των κλάσεων που χρησιμοποιούμε και τους ορισμούς τους. Πέρα από την κύρια κλάση, έχουμε μόνο την κλάση `Dish` και την υποκλάση της `MainDish`. Επίσης, υπάρχει μια κλάση `Bla` που βρίσκεται εκτός της ιεραρχίας των παραπάνω κλάσεων.

Στο παράδειγμα αυτό, η κλάση `MainDish` επαναορίζει τη μέθοδο `void eat()` και προσθέτει και μία μέθοδο `void onDiet()`. Θεωρούμε, δηλαδή, ότι στην κλάση `Dish` δεν είναι ορατή η μέθοδος `void onDiet()`.

Με το παράδειγμα αυτό, θα πειραματιστούμε με τα εξής:

- τις επιτρεπτές αναθέσεις τιμών σε μεταβλητές
- τι προκύπτει από μια επιτρεπτή ανάθεση
- ποιες μεθόδους στιγμιοτύπου μπορούμε να εφαρμόσουμε στις μεταβλητές αναφορών

Συνοπτικά, σχετικά με τα παραπάνω:

- Μια μεταβλητή αναφοράς
 - μπορεί να αναφερθεί σε αντικείμενο της κλάσης της, άρα και σε αντικείμενο οποιασδήποτε υποκλάσης της, και
 - μπορεί να πάρει την τιμή μιας άλλης μεταβλητής αναφοράς της κλάσης της ή υποκλάσης της, μέσω ανάθεσης.
- Μετά από μια επιτρεπτή ανάθεση σε μια μεταβλητή αναφοράς, η μεταβλητή χάνει την προηγούμενη τιμή και αποκτά νέα τιμή. Άρα, αν αναφερόταν σε αντικείμενο, παύει πλέον να αναφέρεται σε αυτό. Αν στο αντικείμενο αυτό δεν αναφέρεται καμία άλλη μεταβλητή αναφοράς, αυτό είναι υποψήφιο για “καταστροφή” και ανάκτηση του χώρου του κατά την περισυλλογή σκουπιδιών.
 - Αν το δεξί μέλος της ανάθεσης είναι αντικείμενο, η μεταβλητή πλέον αναφέρεται στο αντικείμενο αυτό.
 - Αν το δεξί μέλος της ανάθεσης είναι μια άλλη μεταβλητή αναφοράς η οποία αναφέρεται σε αντικείμενο, η μεταβλητή πλέον αναφέρεται και εκείνη στο αντικείμενο αυτό.
- Στις μεταβλητές αναφορών μπορούμε να εφαρμόσουμε μόνο μεθόδους με όνομα ορατό μέσα από την κλάση τους. Δηλαδή, αν εφαρμόσουμε μια μέθοδο στιγμιοτύπου σε ένα αντικείμενο μέσω μιας μεταβλητής αναφοράς, δεν αρκεί η μέθοδος να έχει όνομα ορατό στην κλάση του αντικειμένου, πρέπει το όνομα να είναι ορατό και στην κλάση της μεταβλητής. Το αποτέλεσμα μιας επιτρεπτής εφαρμογής μεθόδου στιγμιοτύπου το έχουμε συζητήσει: θα κληθεί η μέθοδος που είναι ορατή στην κλάση του αντικειμένου στο οποίο αναφέρεται η μεταβλητή αναφοράς.

Ας δούμε, λοιπόν, τα παραπάνω μέσα από το παράδειγμα. Ως άσκηση, πειραματιστείτε αποσχολιάζοντας το κατάλληλο σύνολο γραμμών, ώστε να δείτε το σφάλμα, στην περίπτωση μη επιτρεπτής πράξης. Στην περίπτωση που ο κώδικας μεταγλωττίζεται και εκτελείται χωρίς σφάλμα, εκτελέστε το πρόγραμμα και αιτιολογήστε το αποτέλεσμα της εκτέλεσης.

```
//References to class pointing to objects of subclass and vice versa
class App {
    public static void main(String[] args) {

        Dish d = new Dish();
        d.eat();

        // Dish ff = new MainDish(50, "light", 0); // OK!
        // MainDish ff = new Dish(50, "light"); // Incompatible types
        // MainDish ff = d; // Incompatible types

        Dish dmd = new MainDish(600, "mbrizola", 15); // OK!
```

```
//      dmd.onDiet();           // Error: dmd of type Dish
//                               // => no onDiet() in scope of Dish

    dmd.eat();

//      Bla b = new Bla();
//      dmd = b;                // Incompatible types

//      MainDish md1 = dmd;     // Incompatible types: dmd of type Dish

//      MainDish md1;
//      md1 = d;                // Incompatible types

    System.out.println("////////////////////");

    dmd = d;    // Now dmd refers to a Dish,
               // no reference to "mbrizola" any longer

    dmd.eat();
    dmd.eatMore(200);

    d = dmd;    // nothing actually changes
    d.eat();

    System.out.println("////////////////////");

    MainDish md1 = new MainDish(700, "patates", 3);
    MainDish md2 = new MainDish(400, "fakes", 1);

    md1 = md2;    // refs to same object

//      md2 = dmd;    // Error: incompatible types
//                   // Dish cannot be converted to MainDish

    md1.eatMore(3000);
    System.out.println(md1.getCalories());
    System.out.println(md2.getCalories());
}
}

class Bla {
    int i;
}

// abstract class Dish {
class Dish {

    int calories = 1000;
    String name = "Junk food";
}
```

```

Dish() {
    System.out.println("The new dish named: " + name +
        " contains " + calories + " calories!");
}

Dish(String nam) {
    name = nam;
    System.out.println("The new dish named: " + name +
        " contains " + calories + " calories!");
}

Dish(int cal, String nam) {
    calories = cal;
    name = nam;
    System.out.println("The new dish named: " + name +
        " contains " + calories + " calories!");
}

void eat() {
    System.out.println("Eating " + name );
}

void eatMore(int cal) {
    calories += cal;
    System.out.println("Eating more calories " + calories );
}

int getCalories() {
    return calories;
}
}

class MainDish extends Dish {
    int cost = 6;
    MainDish(int cal, String nam, int cst) {
        super(cal, nam);
        cost = cst;
        System.out.println("The cost of the main dish is " + cost);
    }

    void eat() {
        System.out.println("Eating Main Dish " + name );
    }

    void onDiet() {
        System.out.println("Sorry, I have to refuse it! ");
    }
}

```


Στο πρόγραμμα αυτό, βλέπουμε:

- επιτρεπτές αναθέσεις τιμών σε μεταβλητές αναφοράς, διότι στο δεξί μέλος εμφανίζονται μεταβλητές ή αντικείμενα του ίδιου τύπου ή υποκλάσης του τύπου, π.χ.

```
- Dish d = new Dish(); ίδιοι τύποι
- Dish dmd = new MainDish(600, "mbrizola", 15); το αντικείμενο προς αναφορά είναι
  τύπου υποκλάσης
- dmd = d; ίδιοι τύποι — μετά την ανάθεση και οι δύο μεταβλητές αναφέρονται στο ίδιο αντι-
  κείμενο
```

- μη επιτρεπτές αναθέσεις τιμών σε μεταβλητές αναφοράς, διότι στο δεξί μέλος εμφανίζονται μεταβλητές ή αντικείμενα είτε υπερκλάσης τους είτε εκτός του δικού τους μονοπατιού στην ιεραρχία των κλάσεων, π.χ.

```
- MainDish ff = d; η μεταβλητή d είναι τύπου Dish, που είναι υπερκλάση της κλάσης
  MainDish
- MainDish ff = new Dish(50, "light"); ο τύπος του αντικειμένου είναι υπερκλάση του
  τύπου της μεταβλητής
- Τύποι “άσχετοι” μεταξύ τους:
```

```
Bla b = new Bla();
dmd = b;    // Incompatible types
```

- αποδεκτές εφαρμογές μεθόδων, από πλευράς ορατότητας, π.χ.
 - `d.eat();` υπάρχει μέθοδος `void eat()` ορατή στην κλάση της μεταβλητής `d`, που είναι η κλάση `Dish`
 - `md1.eatMore(3000);` υπάρχει μέθοδος `void eatMore()` ορατή στην κλάση της μεταβλητής `md1`, που είναι η κλάση `MainDish`, διότι κληρονομείται από την κλάση `Dish`
- μη αποδεκτές εφαρμογές μεθόδων, από πλευράς ορατότητας, π.χ.


```
dmd.onDiet();
```

στην κλάση `Dish` που είναι ο τύπος της μεταβλητής `dmd` δεν είναι ορατή η μέθοδος `void onDiet()` — παρότι είναι ορατή στην κλάση του αντικειμένου.

Όπως είπαμε, μετά από μια ανάθεση τιμής μεταξύ δύο μεταβλητών αναφοράς που αναφέρονται σε αντικείμενα, οι δύο μεταβλητές αναφέρονται στο ίδιο αντικείμενο, αυτό στο οποίο αναφέρεται η μεταβλητή που εμφανίζεται στο δεξί μέρος της ανάθεσης. Ο παρακάτω κώδικας, το επιβεβαιώνει:

```
MainDish md1 = new MainDish(700, "patates", 3);
MainDish md2 = new MainDish(400, "fakes", 1);

md1 = md2;    // refs to same object
```

```
md1.eatMore(3000);
System.out.println(md1.getCalories());
System.out.println(md2.getCalories());
```

Το αποτέλεσμα της εκτύπωσης που προκύπτει από την εκτέλεσή του είναι:

```

The new dish named: patates contains 700 calories!
The cost of the main dish is 3
The new dish named: fakes contains 400 calories!
The cost of the main dish is 1
Eating more calories 3400
3400
3400

```

Μετά την ανάθεση `md1 = md2`, και οι δύο μεταβλητές αναφοράς, `md1` και `md2`, αναφέρονται στο αντικείμενο με όνομα "fakes" και καμία, πλέον, δεν αναφέρεται στο αντικείμενο με όνομα "patates".

9.9 Κληρονομικότητα και Σύνθεση: Παραδείγματα

Στα παραδείγματά μας έως τώρα είχαμε ορίσει κλάσεις των οποίων τα πεδία ήταν είτε πρωτογενών τύπων, π.χ. `int`, είτε μεταβλητές αναφοράς τύπου κλάσης από το API της Java, π.χ. `String`. Στα παραδείγματα που ακολουθούν, θα ορίσουμε κλάσεις των οποίων τα πεδία ενδέχεται να είναι μεταβλητές αναφοράς τύπου κλάσης που έχουμε ορίσει εμείς. Γενικά, μπορούμε να απεικονίσουμε τη σχέση *has a* μεταξύ κλάσεων, με περιπτώσεις που τα πεδία μιας κλάσης είναι μεταβλητές αναφοράς που αναφέρονται σε αντικείμενα μιας άλλης, μια που κατ'αυτόν τον τρόπο, η κλάση "έχει" πεδίο τύπου (αναφορά σε) αντικείμενο της άλλης κλάσης.

Το παρακάτω είναι ένα πολύ απλό παράδειγμα. Πέρα από την κυρία κλάση, ορίζεται μια κλάση `Dish` και μια υποκλάση της `MainDish`. Επίσης, ορίζεται και μια κλάση `AirMeal` της οποίας τα πεδία είναι μεταβλητές αναφοράς τύπου `MainDish` και `Dish`. Αυτές αρχικοποιούνται με αντικείμενα των κλάσεών τους κατά τη δήλωσή τους.

Η κλάση `AirMeal` δεν έχει ορισμένη κάποια μέθοδο. Έχοντας αφήσει ελεύθερη την ορατότητα σε επίπεδο πακέτου, οι μέθοδοι των κλάσεων `MainDish` και `Dish` μπορούν να εφαρμοστούν στα πεδία της `AirMeal` από την κύρια κλάση.

Η κλάση `AirMeal` δεν έχει ορισμένη κάποια μέθοδο αρχικοποίησης. Το παράδειγμα, όμως, κάνει καθαρή τη διαδικασία αρχικοποίησης των πεδίων κατά τη δήλωσή τους: όταν δημιουργηθεί ένα αντικείμενο της κλάσης `AirMeal`, θα κληθούν οι μέθοδοι κατασκευής των κλάσεων `MainDish` και `Dish` με τις τιμές που εμφανίζονται στις δηλώσεις των πεδίων της κλάσης, προκειμένου να γίνει η αρχικοποίηση των αντικειμένων στα οποία θα αναφερθούν τα πεδία του.

```

class App {
    public static void main(String[] args) {
        AirMeal am = new AirMeal(); // AirMeal by default
        System.out.println("////////////////////////////////////////");
        am.var1.eat();
        am.var2.eat();
    }
}

class Dish {

    int calories = 1000;
    String name = "Junk food";
}

```

```
Dish() {
    this(2000, "FatAndSpices");
    System.out.println("The new dish named: " + name +
        " contains " + calories + " calories!");
}

Dish(String nam) {
    name = nam;
    System.out.println("The new dish named: " + name +
        " contains " + calories + " calories!");
}

Dish(int cal, String nam) {
    calories = cal;
    name = nam;
    System.out.println("The new dish named: " + name +
        " contains " + calories + " calories!");
}

void eat() {
    System.out.println("Eating " + name );
}

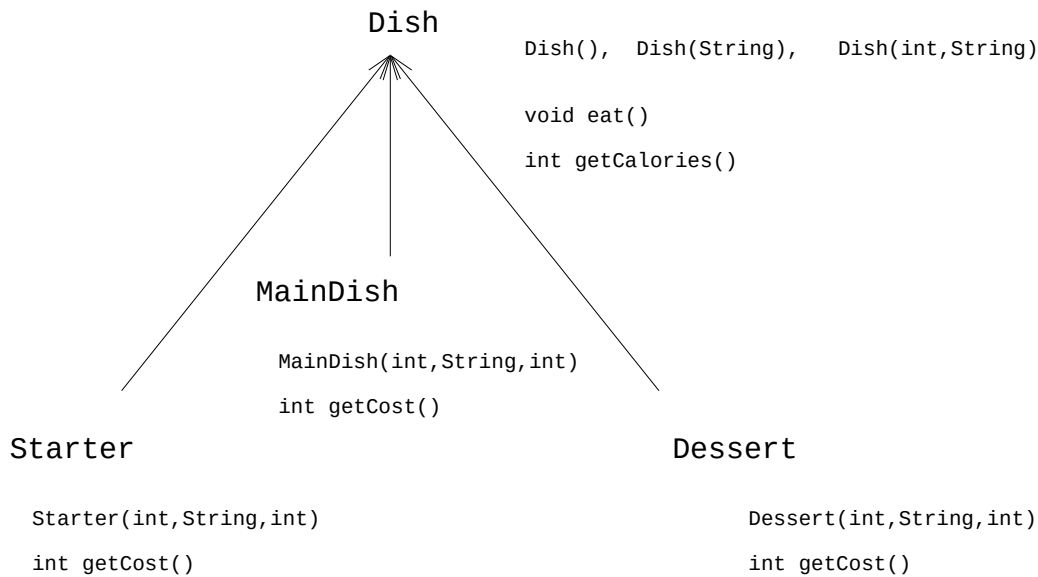
int getCalories() {
    return calories;
}
}

class MainDish extends Dish {
    int cost = 6;
    MainDish(int cal, String nam, int cst) {
        super(cal, nam);
        cost = cst;
        System.out.println("The cost of the main dish is " + cost);
    }
}

class AirMeal {
    MainDish var1 = new MainDish(0, "JustAGlassofWater", 0);
    Dish var2 = new Dish(0, "JustAnotherAGlassofWater");
}
```

Όπως είπαμε στο παράδειγμα, δεν είχε οριστεί καμία μέθοδος αρχικοποίησης για την κλάση `AirMeal`. Η default αρχικοποίηση πραγματοποιείται μέσω των αρχικοποιήσεων που καθορίζονται κατά τη δήλωση των πεδίων της κλάσης.

Ακολουθεί ένα παράδειγμα κλάσεων που μας επιτρέπει να σχολιάσουμε τα εναλλακτικά για την αρχικοποίηση των πεδίων κατά τη διαδικασία default αρχικοποιήσεων των αντικειμένων μιας κλάσης. Στο παράδειγμα αυτό, ορίζεται μια κλάση `Meal`, της οποίας τα πεδία είναι μεταβλητές αναφοράς τύπων που δομούν την ιεραρχία κλάσεων που φαίνεται στο Σχήμα 9.4.



Σχήμα 9.4: Η ιεραρχία κλάσεων που χρησιμοποιεί η κλάση Meal

Η κλάση Meal έχει δύο μεθόδους αρχικοποίησης:

1. μία default μέθοδο και
2. μία μέθοδο με πρωτότυπο:

```
Meal(int cal1, String nam1, int cst1,
     int cal2, String nam2, int cst2,
     int cal3, String nam3, int cst3)
```

Αν υποθέσουμε ότι θεωρούμε ότι by default το γεύμα μας είναι ομελέτα, θα θέλαμε να καταλήξουμε η default αρχικοποίηση αντικειμένου τύπου Meal, μέσω της έκφρασης `new Meal()`; να δημιουργεί αντικείμενα με κενές αναφορές στα πεδία `starter` και `dessert` και αναφορά σε αντικείμενο της κλάσης `MainDish`, αρχικοποιημένο, για παράδειγμα, μέσω `MainDish(500, "Omelette", 4)`, ως τιμή του πεδίου `maindish`. Αυτό μπορούμε να το πετύχουμε με τους εξής δύο τρόπους:

1. Στη μία περίπτωση, κάνουμε την αρχικοποίηση του πεδίου κατά τη δήλωσή του και ορίζουμε μία default μέθοδο αρχικοποίησης για την κλάση Meal, στο σώμα της οποίας δεν κάνουμε τίποτα για το πεδίο αυτό, δηλαδή για παράδειγμα:

```
MainDish = new MainDish(500, "Omelette", 4); // Not a good idea!
// It will always initialize

Meal() {}
```

2. Στην άλλη περίπτωση, δεν κάνουμε καμιά ρητή αρχικοποίηση στη δήλωση του πεδίου και ορίζουμε μία default μέθοδο αρχικοποίησης για την κλάση Meal, στο σώμα της οποίας κάνουμε την αρχικοποίησή του με τον επιθυμητό τρόπο, δηλαδή:

```
MainDish maindish;

Meal() { maindish = new MainDish(500, "Omelette", 4); }
```

Αν έχουμε και εναλλακτικές αρχικοποιήσεις γευμάτων, δηλαδή, πέρα από την default έχουμε και άλλες μεθόδους αρχικοποίησης για την κλάση `Meal`, θα πρέπει να ακολουθήσουμε το δεύτερο εναλλακτικό και να ορίσουμε default μέθοδο αρχικοποίησης στο σώμα της οποίας κάνουμε την αρχικοποίηση του πεδίου `maindish`. Διαφορετικά, θα ξεκινάγαμε κάθε γεύμα με μια ομελέτα, κάτι που μάλλον δεν είναι επιθυμητό!

Το παρακάτω πρόγραμμα το κάνει σαφές. Δοκιμάστε να το εκτελέσετε, σχολιάζοντας τις κατάλληλες γραμμές για το καθένα εναλλακτικό κάθε φορά. Στην περίπτωση της μεταβλητής `m1` που αναφέρεται σε αντικείμενο της κλάσης `Meal`, που αρχικοποιείται by default, δεν υπάρχει διαφορά στο αποτέλεσμα. Στην περίπτωση της μεταβλητής `m2`, όμως, όταν έχουμε αποσχολιάσει την αρχικοποίηση κατά τη δήλωση του πεδίου `maindish`, η αρχικοποίηση θα γίνει, ούτως ή άλλως, πριν την εκτέλεση του σώματος της μη-default μεθόδου αρχικοποίησης της `Meal`. Όπως σχολιάσαμε χιουμοριστικά προηγουμένως, στην περίπτωσή μας αυτό δεν είναι επιθυμητό. Σε άλλες περιπτώσεις, όμως, θα μπορούσε να ήταν.

Ας κάνουμε εδώ μια παρατήρηση σχετικά με τη C++: από το πρότυπο C++11 και έπειτα επιτρέπονται αρχικοποιήσεις κατά τις δηλώσεις των μελών-δεδομένων και στη γλώσσα αυτή. Αυτές, όμως, λαμβάνονται υπόψη κατά την αρχικοποίηση ενός αντικειμένου, μόνον αν δεν γίνονται αρχικοποιήσεις των μελών-δεδομένων αυτών από τη μέθοδο αρχικοποίησης που χρησιμοποιείται.

```
class App {
    public static void main(String[] args) {

        Meal m1 = new Meal();

        Meal m2 = new Meal(100, "salata", 4,
                           600, "mbrizola", 12,
                           200, "crema karamele", 4);
    }
}

class Dish {
    int calories = 1000;
    String name = "Junk food";
    Dish() {
        System.out.println("The new dish named: " + name +
                           " contains " + calories + " calories!");
    }

    Dish(String nam) {
        name = nam;
        System.out.println("The new dish named: " + name +
                           " contains " + calories + " calories!");
    }

    Dish(int cal, String nam) {
        calories = cal;
        name = nam;
        System.out.println("The new dish named: " + name +
                           " contains " + calories + " calories!");
    }
}
```

```

    void eat() {
        System.out.println("Eating " + name );
    }

    int getCalories() {
        return calories;
    }
}

class Starter extends Dish {
    int cost = 4;
    Starter(int cal, String nam, int cst) {
        super(cal, nam);
        cost = cst;
        System.out.println("The cost of the starter is " + cost);
    }

    int getCost() { return cost; }
}

class MainDish extends Dish {
    int cost = 9;
    MainDish(int cal, String nam, int cst) {
        super(cal, nam);
        cost = cst;
        System.out.println("The cost of the main dish is " + cost);
    }

    int getCost() { return cost; }
}

class Dessert extends Dish {
    int cost = 3;
    Dessert(int cal, String nam, int cst) {
        super(cal, nam);
        cost = cst;
        System.out.println("The cost of the dessert is " + cost);
    }

    int getCost() { return cost; }
}

class Meal {

    Starter starter;
    MainDish maindish;
    // MainDish maindish = new MainDish(500, "Omelette", 4);
    // Not a good idea!
    // It will always initialize

```

```

Dessert dessert;

Meal() { maindish = new MainDish(500, "Omelette", 4); }
                                // Better like this
                                // if omelette is the default
// Meal() {}

Meal(int cal1, String nam1, int cst1,
     int cal2, String nam2, int cst2,
     int cal3, String nam3, int cst3) {

    starter = new Starter(cal1,nam1,cst1);
    maindish = new MainDish(cal2,nam2,cst2);
    dessert = new Dessert(cal3,nam3,cst3);

    int energy = starter.getCalories() +
                 maindish.getCalories() + dessert.getCalories();
    int bill = starter.getCost() + maindish.getCost() +
               dessert.getCost();
    System.out.println("You are about to eat a meal of " +
                       energy + " calories and pay : " +
                       bill + " Euros");
}
}

```

9.10 Πολλαπλή Κληρονομικότητα - Τύπος Διεπαφής

Αφού μια κλάση είναι άμεση υποκλάση ακριβώς μίας κλάσης, υποστηρίζεται πολλαπλή κληρονομικότητα στην Java; Και αν ναι, πώς; Η απάντηση δίνεται κάνοντας αναφορά σε μια ειδική κατηγορία τύπων, τους *τύπους διεπαφής*.

Ένας τύπος διεπαφής έχει παρόμοια μορφή με κλάση και μεταγλωττίζεται σε αρχείο κλάσης, αλλά έχει ουσιαστικές διαφορές με αυτές. Βασικά, αποτελεί έναν τύπο που απλά αποτελεί *προδιαγραφή* τύπων, την οποία οι τύποι κλάσης πρέπει να καθορίσουν, να *υλοποιήσουν*, όπως λέγεται, χρησιμοποιώντας τη λέξη-κλειδί `implements`.

Στη δήλωση των τύπων διεπαφής, χρησιμοποιούμε τη λέξη-κλειδί `interface` αντί για `class`. Στην αρχική τους εκδοχή, οι τύποι διεπαφής αποτελούντο μόνο από:

- πεδία και
- προδιαγραφές μεθόδων

Στις πιο πρόσφατες εκδόσεις της Java, υπάρχουν διάφορες αλλαγές, όμως. Θα σχολιάσουμε κάποιες από αυτές, αργότερα. Τα πεδία των τύπων διεπαφής θεωρούνται: `public`, `static` και `final`. Οι μέθοδοί τους, στη γενική περίπτωση, θεωρούνται `public` και `abstract` — δίνουμε μόνο την επικεφαλίδα τους και όχι σώμα.

Ένα παράδειγμα τύπου διεπαφής είναι:

```

interface Interf {
    int CONST = 100;
    void interfMethod();
}

```

Μπορούμε να έχουμε κληρονομικότητα μεταξύ τύπων διεπαφής. Τη σχέση αυτή την εκφράζουμε και εδώ με χρήση της λέξης-κλειδιού `extends`. Για παράδειγμα,

```
interface SubInterf extends Interf {
    int ANOTHER_CONST = 200;
    void subinterfMethod();
}
```

Ένας τύπος διεπαφής μπορεί να συνδέεται άμεσα μέσω κληρονομικότητας με παραπάνω από έναν τύπους διεπαφής.

Δεν μπορούμε να ορίσουμε αντικείμενα των τύπων διεπαφής. Χρησιμοποιούνται μόνο για κληρονομικότητα. Κρίσιμη χρήση τους είναι η έκφραση πολλαπλής κληρονομικότητας. Η λέξη-κλειδί `implements` χρησιμοποιείται από μια κλάση για να δηλώσει ότι “υλοποιεί” έναν τύπο διεπαφής. Στην περίπτωση αυτή, η κλάση πρέπει να ορίσει όλες τις μεθόδους του τύπου διεπαφής. Μια κλάση μπορεί να υλοποιεί οποιονδήποτε αριθμό από τύπους διεπαφής, δίνοντας ορισμούς στις μεθόδους τους. Οι τύποι διεπαφής που υλοποιεί, όμως, απαγορεύεται να έχουν προδιαγραφές μεθόδων που διαφέρουν μόνο στον τύπο επιστροφής.

Στο σημείο αυτό, πρέπει να κάνουμε κάποια επισήμανση. Μετά την έκδοση SE8 της Java, υπάρχουν αρκετές αλλαγές στις δυνατότητες των τύπων διεπαφής. Αυτό οφείλεται σε μεγάλο βαθμό στην εμπειρία που είχε αποκτηθεί από τη χρήση της γλώσσας και τη γνώση που είχε προκύψει. Για παράδειγμα, πλέον επιτρέπεται να ορίζονται και στατικές μέθοδοι στις διεπαφές ή/και να δίνονται και ορισμοί στις μεθόδους στιγμιότυπου, προσδιορίζοντάς τις ως `default`. Η δυνατότητα ορισμού μεθόδων στους τύπους διεπαφής ήλθε να λύσει ένα πρόβλημα που προέκυπτε, όχι κατά την αρχική ανάπτυξη ενός συστήματος κλάσεων, αλλά κατά την επέκτασή του. Στην περίπτωση που υπάρχει ανάγκη επέκτασης του ορισμού ενός τύπου διεπαφής με νέες μεθόδους, προκειμένου να καλύψει καλύτερα επιπρόσθετες ανάγκες ενός συστήματος, αντί να οριστεί νέος τύπος διεπαφής ο οποίος να κάνει `extend` τον υπάρχοντα, απλά προστίθεται στον υπάρχοντα η επιπλέον λειτουργικότητα. Αυτό επιτυγχάνει την αποφυγή ορισμού νέων τύπων διεπαφής, αν οι επεκτεινόμενοι δεν κρίνονται πλέον απαραίτητοι για μελλοντικές επεκτάσεις του συστήματος. Σε αυτήν την περίπτωση, όμως, ήδη υπάρχουσες κλάσεις που “υλοποιούν” τον υπάρχοντα τύπο διεπαφής δεν δίνουν ορισμούς για τη νέα λειτουργικότητα, κάτι που θα δημιουργούσε σφάλμα. Δίνοντας, λοιπόν, τη δυνατότητα δήλωσης μεθόδων με `default` ορισμούς, δηλώνεται ότι αυτές οι μέθοδοι δεν είναι αναγκαίο να (επανα)οριστούν και έτσι το πρόβλημα παρακάμπτεται. Όμως, ενδέχεται να εμφανίζονται προβλήματα που σχετίζονται με την πολλαπλή κληρονομικότητα και που με τους τύπους διεπαφής αποφεύγονταν. Οπότε, η δυνατότητα ορισμού μεθόδων με `default` ορισμούς καλό είναι να αποφεύγεται στην αρχική κατασκευή ενός συστήματος κλάσεων, που οφείλει να υλοποιεί έναν προσεγμένο σχεδιασμό, και να χρησιμοποιείται με φειδώ σε ενδεχόμενη επέκτασή του.

Ας δούμε ένα μικρό παράδειγμα χρήσης τύπων διεπαφής. Στο παράδειγμα, ορίζονται τρεις τύποι διεπαφής, οι `Dish`, `MainDish` και `HealthyDishTraits`, και υποθέτουμε ότι ο τύπος `MainDish` είναι εξειδίκευση του τύπου `Dish`. Έχουμε, δηλαδή

```
interface MainDish extends Dish
```

οπότε συνδέονται μέσω κληρονομικότητας.

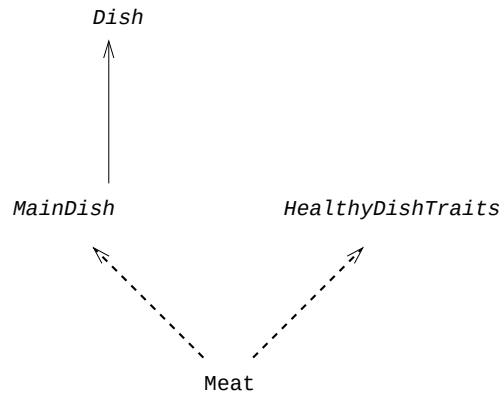
Στη συνέχεια, έχουμε μια κλάση, την `Meat`, η οποία “υλοποιεί” τους τύπους διεπαφής `MainDish` και `HealthyDishTraits`.

```
class Meat implements MainDish, HealthyDishTraits
```

Η κλάση `Meat` οφείλει να δώσει ορισμούς για όλες τις μεθόδους που είναι ορατές στους τύπους διεπαφής `MainDish` και `HealthyDishTraits`, είτε γιατί είναι δηλωμένες σε αυτούς είτε διότι έχουν κληρονομηθεί από γενικότερο τύπο, όπως είναι για τον τύπο `MainDish` η περίπτωση των μεθόδων:

```
void eat();
int getCalories();
```

Το παράδειγμα έχει ως εξής:



Σχήμα 9.5: Η ιεραρχία των τύπων διεπαφής και της κλάσης του παραδείγματος

```

class App {
    public static void main(String[] args) {
        Meat mt = new Meat(600, "mbrizola", 12);
        mt.eat();
        System.out.println(mt.getCalories());
        System.out.println(mt.isLate());
        System.out.println(mt.givesStrength());
    }
}

interface Dish {
    void eat();           // public
    int getCalories();    // public
}

interface MainDish extends Dish {
    int MEALTIME = 14;    // public, static, final
    boolean isLate();
}

interface HealthyDishTraits {
    int LOWCALORIES = 300;
    int givesStrength();
}

class Meat implements MainDish, HealthyDishTraits {
    int calories;
    String name;
    int time;
    Meat(int cal, String nam, int t) {
        calories = cal;
        name = nam;
    }
}

```

```

        time = t;
        System.out.println("You are eating :" + name +
                           " at " + time + " having " +
                           calories + " calories");
    }

    public void eat() {                                // public
        System.out.println("Eating " + name);
    }

    public int getCalories() {                          // public
        return calories;
    }

    public boolean isLate() {
        return ((time > MEALTIME) ? true : false);
    }

    public int givesStrength() {
        return (calories - LOWCALORIES);
    }
}

```

Με βάση τον κώδικα αυτό, μπορούμε να δημιουργούμε αντικείμενα της κλάσης `Meat` και να τα χειριζόμαστε με τις μεθόδους `void eat()`, `int getCalories()`, `boolean isLate()` και `int givesStrength()`. Προσέξτε τον προσδιορισμό ορατότητας `public` στο σώμα της κλάσης `Meat`. Οι μέθοδοι που δηλώνονται στους τύπους διεπαφής θεωρούνται `public`, οπότε στην κλάση που τις υλοποιεί πρέπει να το εκφράσουμε ρητά.

9.11 Στατικά Μέλη στην Java

Από την πρώτη φορά που είδαμε κώδικα Java, για να παρουσιάσουμε το ελάχιστο πρόγραμμα, αναφερθήκαμε σε στατικά μέλη. Είπαμε ότι το ελάχιστο πρόγραμμα αποτελείται από μια κλάση με τουλάχιστον ένα μέλος, τη μέθοδο `public static void main(String[] args)`, μια στατική μέθοδο, από την κλήση της οποίας ξεκινάει η εκτέλεση του προγράμματος.

Τα στατικά μέλη, ή μέλη κλάσης, όπως αναφέρονται, είναι υψίστης σημασίας για την Java, μια που δεν υπάρχει “χώρος” για να γράψουμε κώδικα εκτός κλάσεων. Στη C++ αυτό δεν ισχύει: οποτεδήποτε μας ενδιαφέρει να υλοποιήσουμε μια διαδικασία που δεν αποτελεί μέρος της συμπεριφοράς μιας κλάσης, αυτή μπορεί να οριστεί ως συνάρτηση εκτός κλάσεων. Το ίδιο ισχύει και για τις μεταβλητές. Αν κάποια μεταβλητή δεν αποτελεί δομικό στοιχείο ενός αντικειμένου, μπορεί να οριστεί εκτός κλάσεων. Στατικά μέλη, συνεπώς, θα ορίσουμε στη C++ μόνο εάν θέλουμε να σχετίσουμε τη συνάρτηση ή τη μεταβλητή με την εμβέλεια μιας κλάσης.

Στην Java, κάθε τι σχετίζεται με την εμβέλεια μιας κλάσης, οπότε, αν κάποια μεταβλητή δεν αποτελεί πεδίο στιγμιότυπου και μια μέθοδος δεν αποτελεί μέρος της συμπεριφοράς των αντικειμένων της κλάσης, θα οριστεί μέσα σε μια κλάση και θα χαρακτηριστεί ως `static` για την κλάση αυτή.

Να υπενθυμίσουμε ότι ένα μέλος κλάσης μπορεί να προσπελαστεί, είτε χρησιμοποιώντας το όνομα της κλάσης απευθείας είτε μέσω μιας μεταβλητής αναφοράς. Στη δεύτερη περίπτωση, παρότι δεν παίζει ρόλο ο τύπος του αντικειμένου στο οποίο αναφέρεται, η μεταβλητή πρέπει να έχει πάρει τιμή, έστω και την τιμή `null`.

9.11.1 Χρήση Στατικών Μεθόδων (Μεθόδων Κλάσης)

Θα κάνουμε μια σύντομη συζήτηση για τη συμπεριφορά κατά τη χρήση μεθόδων κλάσης, χρησιμοποιώντας ένα απλό παράδειγμα. Στον παρακάτω κώδικα, πέρα από την κύρια κλάση, έχουμε δύο κλάσεις, την κλάση `Tt` και την κλάση `Ttm`. Η κλάση `Ttm` είναι υποκλάση της `Tt`. Και οι δύο κλάσεις ορίζουν μια στατική συνάρτηση, την `static void bla()`. Να επισημάνουμε ότι η κλάση `Ttm` δεν θα μπορούσε να δώσει ορισμό σε μια συνάρτηση με πρωτότυπο `void bla()`, χωρίς να την προσδιορίσει ως `static`.

```
class Bla {
    public static void main(String[] args) {
        // Tt t = new Tt();
        // Tt t = new Ttm();
        // Tt t = null; // variable must be initialized

        Tt.bla();
        Ttm.bla();

        // Ttm t = new Ttm();

        // t.bla();
    }
}

class Tt {
    static void bla() {
        System.out.println("Tt.bla()");
    }
}

class Ttm extends Tt {
    static void bla()
    // void bla() // compilation error
    { System.out.println("Ttm.bla()"); }
}
```

Παρατηρούμε ότι στο σώμα της μεθόδου `main` της κύριας κλάσης έχουμε κλήσεις της μεθόδου `bla()` με τους εξής τρόπους:

- είτε προσδιορίζοντας κλάση ρητά, όπως στις κλήσεις:

```
Tt.bla();
Ttm.bla();
```

- είτε χρησιμοποιώντας μεταβλητή αναφοράς, όπως στην κλήση
`t.bla();`

Στη δεύτερη περίπτωση, ο τύπος της μεταβλητής και μόνο καθορίζει την εμβέλεια της οποίας η `bla()` θα εκτελεστεί. Δηλαδή, αν αποσχολιάσουμε τη γραμμή

```
// t.bla();
```

όποια από τις παρακάτω περιπτώσεις και να αποσχολιάσουμε

```
// Tt t = new Tt();
// Tt t = new Ttm();
// Tt t = null;
```

το αποτέλεσμα της εκτέλεσης του προγράμματος θα είναι:

```
Tt.bla()
Ttm.bla()
Tt.bla()
```

Αντίθετα, αν αποσχολιάσουμε τη γραμμή

```
// Ttm t = new Ttm();
```

το αποτέλεσμα της εκτέλεσης του προγράμματος θα είναι:

```
Tt.bla()
Ttm.bla()
Ttm.bla()
```

9.11.2 Αρχικοποίηση Κλάσεων

Τα στατικά μέλη μιας κλάσης σχετίζονται και με μια άλλη δυνατότητα που προσφέρει η Java: την *αρχικοποίηση κλάσης*. Όταν μια κλάση πρόκειται να χρησιμοποιηθεί — άρα και να φορτωθεί στη JVM — είτε, για παράδειγμα, γιατί χρησιμοποιείται κάποιο πεδίο της, είτε γιατί καλείται μια μεθοδός της, ή γιατί πρόκειται να δημιουργηθεί αντικείμενό της, πριν να πραγματοποιηθεί η διαδικασία που εμπεριέχει τη χρήση της, η κλάση *αρχικοποιείται*. Δηλαδή, στην περίπτωση που η κλάση περιέχει στατικά πεδία, αυτά αρχικοποιούνται σύμφωνα με τις δηλώσεις τους. Μάλιστα, η διαδικασία αρχικοποίησης μιας κλάσης δεν αφορά μόνο την κλάση που χρησιμοποιείται άμεσα, αλλά και όλη την ιεραρχία των κλάσεων των οποίων είναι υποκλάση. Δηλαδή, θα αρχικοποιηθούν όλες οι μεταβλητές κλάσης των κλάσεων αυτών. Η διαδικασία αυτή θα γίνει άπαξ, την πρώτη φορά που θα χρησιμοποιηθεί η κλάση.

Εννοιολογικά, μπορούμε να συσχετίσουμε τη διαδικασία αυτή με το γεγονός ότι και οι ίδιες οι κλάσεις αποτελούν στιγμιότυπα μιας μετακλάσης, όπως αναφέραμε στην σελίδα 380, οπότε, πριν τη χρήση τους, καλό είναι να αποκτήσουν μια καλά ορισμένη αρχική κατάσταση. Από αυτήν την οπτική γωνία, μπορούμε να κατανοήσουμε τη λειτουργικότητα μιας ακόμα γλωσσικής δομής που μας παρέχει η Java, τα *στατικά block αρχικοποίησης*. Αυτά, συντακτικά, είναι ίδια με τα block αρχικοποίησης που συζητήσαμε προηγουμένως, απλά προσημαίνονται με τη λέξη-κλειδί *static*. Ο κώδικάς τους δεν εκτελείται κάθε φορά που δημιουργείται αντικείμενο της κλάσης, αλλά συμμετέχει στη διαδικασία αρχικοποίησής της. Δηλαδή, κατά τη διαδικασία αρχικοποίησης μιας κλάσης, πέρα από την αρχικοποίηση των μεταβλητών κλάσης, θα εκτελεστούν και τα στατικά block αρχικοποίησης της κλάσης. Στο σώμα τους μπορούμε να περιγράψουμε διαδικασίες που θα θέλαμε να εκτελεστούν κατά την αρχικοποίηση και δεν είναι δυνατόν αυτό να γίνει με επιμέρους κλήσεις μεθόδων κατά την αρχικοποίηση των στατικών μεταβλητών της κλάσης.

Ας δούμε τα παραπάνω μέσω ενός παραδείγματος. Στο παράδειγμα, εκτός από την κύρια κλάση, έχουμε τις κλάσεις AD, BD, A, B και C. Η κλάση B είναι υποκλάση της κλάσης A και η κλάση C είναι υποκλάση της κλάσης B. Οι κλάσεις A, B και C περιέχουν στατικά μέλη. Οι κλάσεις A και B έχουν στατικά πεδία που είναι μεταβλητές αναφοράς των τύπων AD και BD, αντίστοιχα, και αρχικοποιούνται σε αντικείμενα αντίστοιχων τύπων. Επίσης, περιλαμβάνουν στατικά block αρχικοποίησης. Η κλάση C ορίζει μόνο μια μέθοδο κλάσης.

Στη μέθοδο *main* της κύριας κλάσης, αποσχολιάζοντας τις κατάλληλες γραμμές, μπορούμε να δούμε τις εξής περιπτώσεις ενεργοποίησης της διαδικασίας αρχικοποίησης κλάσεων:

- λόγω δημιουργίας αντικειμένου μιας κλάσης
- λόγω κλήσης στατικής μεθόδου απευθείας, χρησιμοποιώντας το όνομα της κλάσης

- λόγω κλήσης στατικής μεθόδου με χρήση μεταβλητής αναφοράς (ανεξάρτητα αν η τιμή της είναι μη-δενική ή αναφέρεται σε αντικείμενο)
- λόγω αναφοράς σε στατικό πεδίο μιας κλάσης, χρησιμοποιώντας το όνομα της κλάσης
- λόγω αναφοράς σε στατικό πεδίο μιας κλάσης, χρησιμοποιώντας μεταβλητή αναφοράς

Να παρατηρήσουμε το εξής: όταν γίνεται χρήση στατικού μέλους, ενεργοποιείται η διαδικασία αρχικοποίησης μόνο για την κλάση στην οποία ορίζεται το μέλος (και τις υπερκλάσεις της). Αν, δηλαδή, δεν είναι ορισμένη στην κλάση μέσω της οποίας γίνεται η πρόσβαση, αλλά απλά κληρονομείται σε αυτήν, η κλάση αυτή δεν αρχικοποιείται — ούτε και οι υπερκλάσεις της έως την υπερκλάση του ορισμού.

Για λόγους σύγκρισης, στην αρχή του σώματος της μεθόδου `main`, ορίζουμε κάποια αντικείμενα των κλάσεων που δεν περιέχουν στατικά μέλη.

```
class App {
    public static void main(String[] args) {
        /**
         * Creation of instances of classes with no class members:
         * AD addfc = new AD();
         * AD ad = new AD(55);
         * BD bd = new BD(555);
         */

        // Creation of instance of class in a hierarchy with static members:
        // C rc = new C(); // static initialization of superclasses occurs
        //
        // C.bla();

        // However, initialization occurs "locally" when:
        // System.out.println("The A var is " + C.ad.getADdata());
        // System.out.println("The B var is " + C.bd.data);
        // C.bloo();

        // Access can be done by reference variable:
        // C rc = null;
        //
        // rc.bla();

        // Compare, again, the above with:
        // System.out.println("A data is " + rc.ad.getADdata());
        //                                     // static variable method call
        // System.out.println("B data is " + rc.bd.data);
        //                                     // static variable field access
        // rc.bloo();
    }
}

class AD {

    int data;
```

```

    { System.out.println("The initial AD data is " + data); }

    { System.out.println("Now AD constructor, if any, " +
        "will start executing!"); }

    AD() {}

    AD(int d) {
        data = d;
        System.out.println("The AD data is " + data);
    }

    int getADdata() { return data; }
}

class BD {

    int data;

    BD(int d) {
        data = d;
        System.out.println("The BD data is: " + data);
    }
}

class A {
    static { System.out.println("Static block init " +
        "of class A executing!"); }

    static AD ad = new AD(100);

    static { System.out.println("Second static block init " +
        "of class A executing!"); }

    static void bloo() { System.out.println("Bloo of A called"); }
}

class B extends A {
    static { System.out.println("Static block init " +
        "of class B executing!"); }

    static BD bd = new BD(1000);
}

class C extends B {
    static void bla() { System.out.println("C.bla called"); }
}

```

Κατά την εκτέλεση του παραπάνω προγράμματος, αποσχοιάζοντας ένα από τα παρακάτω σχολιασμένα

ζεύγη εντολών:

```
//    C rc = new C(); // static initialization of superclasses occurs}
//
//    C.bla();}

ή

//    C rc = null;
//
//    rc.bla();
```

γίνεται αρχικοποίηση όλων των κλάσεων στην ιεραρχία πάνω από την κλάση C. Στη μία περίπτωση, λόγω δημιουργίας αντικειμένου της κλάσης C και στη δεύτερη περίπτωση, επειδή η μέθοδος void bla() ορίζεται στην κλάση C. Οι εκτυπώσεις από την εκτέλεση των αρχικοποιήσεων της ιεραρχίας είναι:

```
Static block init of class A executing!
The initial AD data is 0
Now AD constructor, if any, will start executing!
The AD data is 100
Second static block init of class A executing!
Static block init of class B executing!
The BD data is: 1000
```

Οι εκτυπώσεις αυτές προκύπτουν ως εξής.

Static block init of class A executing!: εκτύπωση από την εκτέλεση του πρώτου στατικού block της κλάσης A

The initial AD data is 0: από την αρχικοποίηση του στατικού πεδίου AD ad της κλάσης A και δημιουργία αντικειμένου της κλάσης AD (μηδενική αρχικοποίηση του πεδίου data της κλάσης A και εκτέλεση του πρώτου block αρχικοποίησης στιγμιοτύπου της κλάσης AD)

Now AD constructor, if any, will start executing!: από την αρχικοποίηση του στατικού πεδίου AD ad της κλάσης A και δημιουργία αντικειμένου της κλάσης AD (εκτέλεση του δεύτερου block αρχικοποίησης στιγμιοτύπου της κλάσης AD)

The AD data is 100: από την αρχικοποίηση του στατικού πεδίου AD ad της κλάσης A και δημιουργία αντικειμένου της κλάσης AD (εκτύπωση από τη μέθοδο αρχικοποίησης στιγμιοτύπου της κλάσης AD)

Second static block init of class A executing!: εκτύπωση από την εκτέλεση του δεύτερου στατικού block της κλάσης A

Static block init of class B executing!: εκτύπωση από την εκτέλεση του στατικού block της κλάσης B

The BD data is: 1000: από την αρχικοποίηση του στατικού πεδίου BD bd της κλάσης B και δημιουργία αντικειμένου της κλάσης BD (εκτύπωση από τη μέθοδο αρχικοποίησης στιγμιοτύπου της κλάσης BD)

Σε όλες τις παρακάτω περιπτώσεις, όμως, εάν δεν έχει δημιουργηθεί αντικείμενο της κλάσης C, μόνο μέρος της ιεραρχίας των κλάσεων αρχικοποιείται.

- Λόγω πρόσβασης σε πεδίο κληρονομημένο στην κλάση C από την κλάση A, αν αποσχολιάσουμε μια από τις παρακάτω γραμμές

```
//    System.out.println("The A var is " + C.ad.getADdata());
//    System.out.println("A data is " + rc.ad.getADdata());
// static variable method call
```

αρχικοποιείται μόνο η κλάση A, μια που το πεδίο ad κληρονομείται στην κλάση C από αυτήν.

- Λόγω πρόσβασης σε πεδίο κληρονομημένο στην κλάση C από την κλάση B, αν αποσχολιάσουμε μια από τις παρακάτω γραμμές

```
//    System.out.println("The B var is " + C.bd.data);
//    System.out.println("B data is " + rc.bd.data);
// static variable field access
```

αρχικοποιείται μόνο η κλάση B (και η γονεϊκή της), μια που το πεδίο bd κληρονομείται στην κλάση C από αυτήν.

- Λόγω πρόσβασης σε μέθοδο κληρονομημένη στην κλάση C από την κλάση A, αν αποσχολιάσουμε μια από τις παρακάτω γραμμές

```
//    C.bloo();
//    rc.bloo();
```

αρχικοποιείται μόνο η κλάση A, μια που η μέθοδος void bloo() κληρονομείται στην κλάση C από αυτήν.

9.11.2.1 Διαδικασία Αρχικοποίησης για (μεταβλητές) Κλάση(ς) και Τύπους Διεπαφής

Στο σημείο αυτό θα συνοψίσουμε τη διαδικασία αρχικοποίησης κλάσης. Η διαδικασία αρχικοποίησης αφορά και τους τύπους διεπαφής, αλλά η διαδικασία γι' αυτούς διαφέρει σε κάποια σημεία. Στα παρακάτω, η περιγραφή θα αφορά και τις δύο κατηγορίες τύπων.

- Η αρχικοποίηση μιας κλάσης έγκειται στο να εκτελεστούν οι static block διαδικασίες αρχικοποίησης και να γίνουν οι αρχικοποιήσεις των στατικών πεδίων της.
- Η αρχικοποίηση ενός τύπου διεπαφής έγκειται στο να εκτελεστούν οι διαδικασίες αρχικοποίησης για τα πεδία του τύπου.
- Οι παραπάνω διαδικασίες εκτελούνται με τη σειρά που εμφανίζονται στον ορισμό της κλάσης (ή του τύπου διεπαφής).
- Πριν γίνει η αρχικοποίηση μιας κλάσης, γίνεται η αρχικοποίηση της άμεσης υπερκλάσης της, αλλά όχι των τύπων διεπαφής που υλοποιεί.
- Επίσης, όταν γίνεται αρχικοποίηση ενός τύπου διεπαφής, δεν γίνεται αρχικοποίηση των τύπων διεπαφής τους οποίους εξειδικεύει (μέσω extends).
- Η αρχικοποίηση κλάσεων ή των τύπων διεπαφής γίνεται αμέσως πριν συμβεί κάποιο από τα εξής (το πρώτο χρονικά εξ αυτών):
 - Αν πρόκειται για κλάση, δημιουργία στιγμιοτύπου της.
 - Αν πρόκειται για κλάση, καλείται μια στατική μέθοδός της.
 - Γίνεται ανάθεση σε στατικό πεδίο της κλάσης.

- Ένα στατικό πεδίο του τύπου χρησιμοποιείται (εκτός αν πρόκειται για πεδίο με πρωτογενή τύπο ή τύπο `String`, αρχικοποιημένου σε σταθερά).
- Μια αναφορά στατικού μέλους προκαλεί αρχικοποίηση μόνο της κλάσης η οποία το δηλώνει, ακόμα και αν η αναφορά γίνεται με χρήση υποκλάσης ή, αν πρόκειται για τύπο διεπαφής, κλάσης που το υλοποιεί.
- Η διαδικασία αρχικοποίησης για ένα στατικό πεδίο αποτιμάται άπαξ και η ανάθεση γίνεται επίσης άπαξ, όταν γίνεται η αρχικοποίηση της κλάσης του.
- Στη φάση της εκτέλεσης, πρώτα αρχικοποιούνται τα στατικά πεδία που έχουν δηλωθεί `final` και αρχικοποιούνται με απλές εκφράσεις (τυπικά: `constant expressions`).

9.11.2.2 Παράδειγμα: Στατικά και Μη-στατικά μέλη: Εμβέλεια, Τρόπος Χρήσης, Αρχικοποιήσεις

Στο παρακάτω παράδειγμα, θα δούμε τη χρήση όσων συζητήσαμε προηγουμένως, μέσω ενός πιο ρεαλιστικού παραδείγματος. Στο παράδειγμα, πέρα από την κύρια κλάση, έχουμε τις κλάσεις `Date`, `Person` και `Employee`.

Η κλάση `Date`, εκτός από προφανή πεδία για την πληροφορία μιας ημερομηνίας, έχει μια μέθοδο αρχικοποίησης που αποδίδει στα πεδία αρχικές τιμές καθώς και μια μέθοδο στιγμιοτύπου για την εκτύπωση των τιμών τους (`void print()`).

Η κλάση `Employee` είναι υποκλάση της κλάσης `Person`. Τόσο η κλάση `Person`, όσο και η κλάση `Employee`, έχουν πεδίο στιγμιοτύπου μια αναφορά τύπου `Date` με το όνομα `date`. Στην κλάση `Person`, το πεδίο αυτό αντιστοιχεί στην ημερομηνία γέννησης του στιγμιοτύπου. Στην κλάση `Employee`, το πεδίο αυτό αντιστοιχεί στην ημερομηνία πρόσληψης του στιγμιοτύπου. Στην περίπτωση των στιγμιοτύπων της κλάσης `Employee`, λοιπόν, καταλαβαίνουμε ότι αυτά θα έχουν δύο πεδία που είναι αναφορές τύπου `Date`. Επίσης, και οι δύο κλάσεις `Person` και `Employee` έχουν ορισμένη μέθοδο στιγμιοτύπου με πρωτότυπο `void print()`, η οποία εκτυπώνει την τιμή του άμεσα ορατού πεδίου `date`, καλώντας την κατάλληλη μέθοδο. Η μέθοδος `print` της κλάσης `Employee`, αρχικά καλεί τη μέθοδο `print` και για το κληρονομημένο πεδίο, με χρήση της λέξης-κλειδιού `super` για να το προσδιορίσει.

Τέλος, οι κλάσεις `Person` και `Employee` έχουν στατικά μέλη και στατικά block αρχικοποίησης. Συγκεκριμένα, έχουν και οι δύο ένα στατικό πεδίο ακεραίου με το ίδιο όνομα, `population`, και μια στατική μέθοδο για την ανάκτηση της τιμής του, `int get_population()`. Θεωρούμε ότι το στατικό αυτό πεδίο έχει τιμή το πλήθος των στιγμιοτύπων της αντίστοιχης κλάσης που έχουν δημιουργηθεί. Στον κώδικά μας θα χρησιμοποιήσουμε τόσο τα στατικά πεδία, όσο και τις στατικές μεθόδους, είτε μέσω του ονόματος της κλάσης είτε μέσω μεταβλητής αναφοράς.

Μια παρατήρηση: αν θέλουμε να έχουμε πρόσβαση σε επαναορισμένο πεδίο από εμβέλεια υποκλάσης, μπορούμε να αναθέσουμε το αντικείμενο της υποκλάσης σε μεταβλητή αναφοράς της υπερκλάσης από την εμβέλεια της οποίας θέλουμε να προκύπτει το πεδίο και, στη συνέχεια, να χρησιμοποιήσουμε τη μεταβλητή αυτή για την πρόσβαση.

Να υπενθυμίσουμε ότι για λόγους απλότητας του παραδείγματος, αφήνουμε όλα τα μέλη να είναι ορατά σε εμβέλεια πακέτου.

Αποσχολιάστε κατάλληλες γραμμές και εκτελέστε το πρόγραμμα, κάθε φορά, για να βεβαιωθείτε ότι καταλαβαίνετε τη συμπεριφορά της εκτέλεσης.

```
class App {
    public static void main(String[] args) {

//      Employee.get_population();
//      Person.get_population();

//      Person p1 = new Person(2,3,1972);
```

```

/*
    p1.print();
    p1.get_population();
*/

// Employee e1 = new Employee(2,3,1972,1,1,1995);
/*
    e1.print();
    e1.get_population();

    e1.date.print();
    System.out.println(e1.population);
*/
System.out.println("////////////////////////////////////////");
/*
    p1 = e1;

    p1.print();           // instance method call
    p1.get_population();  // class (static) method call

    p1.date.print();      // field access
    System.out.println(p1.population); // static field access

    System.out.println(Person.population);
                        // static field explicit access
    System.out.println(Employee.population);
                        // static field explicit access
*/
}
}

class Date {           // no static members
    int day;
    int month;
    int year;
    Date(int d, int m, int y) {
        day = d;
        month = m;
        year = y;
        System.out.println("A new Date was created " + day + " " +
                           month + " " + year);
    }

    void print() {
        System.out.println(" " + day + " " + month + " " + year);
    }
}

```

```

class Person {
// both static and non static members, static block initialization
    static int population;
    Date date;
// static block initialization
    static { System.out.println("The initial population is: " +
                                population); }

    Person(int d, int m, int y) {
        date = new Date(d,m,y);
        population++;
        System.out.println("A new person was born in ");
        date.print();
    }

    static int get_population() {
        System.out.println("People population is " + population);
        return population;
    }

    void print() {
        System.out.println("Printing birth date ");
        date.print();
        System.out.println("Age should be non zero");
    }
}

class Employee extends Person { // both static and non static members,
                                // static block initialization

    static int population;
    Date date;
    static { System.out.println("The initial Employee population is: " +
                                population); }

    Employee(int d1, int m1, int y1, int d2, int m2, int y2) {
        super(d1,m1,y1);
        date = new Date(d2,m2,y2);
        population++;
        System.out.println("A new Employee in ");
        date.print();
    }

    static int get_population() {
        System.out.println("Employee population is " + population);
        return population;
    }

    void print() {
        System.out.println("Printing employee birth date ");
    }
}
    
```

```

    super.date.print();    // inherited field date
    System.out.println("Printing employment date ");
    date.print();
    System.out.println("Age should be over 18");
}
}

```

Συνοπτικά, στον κώδικά μας

- Με την κλήση:

```
Employee.get_population();
```

ή την κλήση

```
Person.get_population();
```

έχουμε κλήση της στατικής μεθόδου `int get_population()`, προσδιορίζοντας την κλάση από την εμβέλεια της οποίας θα επιλεγεί. Επίσης, γίνονται και οι αρχικοποιήσεις των κλάσεων `Person` και `Employee` ή της κλάσης `Person`, αντίστοιχα.

- Πέρα από ορισμό της μεταβλητής `p1`, η εντολή:

```
Person p1 = new Person(2,3,1972);
```

- δημιουργεί ένα αντικείμενο της κλάσης `Person`,
- αρχικοποιεί την κλάση `Person`, στην περίπτωση που δεν έχει ήδη αρχικοποιηθεί: θα αρχικοποιηθεί το στατικό πεδίο της και θα εκτελεστεί και ο κώδικας του στατικού block αρχικοποίησης.

- Υποθέτοντας ότι έχουμε ορίσει και αρχικοποιήσει ήδη τη μεταβλητή `Person p1` όπως προηγουμένως, στον παρακάτω κώδικα

```
p1.print();
p1.get_population();
```

τόσο η μέθοδος στιγμιοτύπου `void print()`, όσο και η στατική μέθοδος `int get_population()`, καλούνται μέσω μεταβλητής αναφοράς. Στην περίπτωση της μεθόδου στιγμιοτύπου, με βάση την αρχικοποίηση που κάναμε στη μεταβλητή `p1`, θα κληθεί η μέθοδος `void print()` της κλάσης `Person`. Στην κλήση της στατικής μεθόδου `int get_population()`, καλείται εκείνη της κλάσης `Person`, σε κάθε περίπτωση.

- Πέρα από ορισμό της μεταβλητής `e1`, η εντολή:

```
Employee e1 = new Employee(2,3,1972,1,1,1995);
```

- δημιουργεί ένα αντικείμενο της κλάσης `Employee`,
- αρχικοποιεί την κλάση `Employee`, στην περίπτωση που δεν έχει ήδη αρχικοποιηθεί: θα αρχικοποιηθεί η υπερκλάση της, `Person`, αν αυτό δεν έχει ήδη γίνει, και στη συνέχεια θα αρχικοποιηθεί το στατικό πεδίο της και θα εκτελεστεί και ο κώδικας του στατικού block αρχικοποίησής της.

- Για τον κώδικα:

```
e1.print();
e1.get_population();
```

που περιέχει τη μεταβλητή αναφοράς `Employee e1`, οι παρατηρήσεις είναι ανάλογες με εκείνες για τον αντίστοιχο κώδικα για τη μεταβλητή αναφοράς `Person p1`.

- Στην κλήση:

```
e1.date.print();
```

έχουμε εφαρμογή της μεθόδου `void print()` της κλάσης `Date` στο πεδίο με όνομα `date` του αντικείμενου στο οποίο αναφέρεται η μεταβλητή `Employee e1`. Το πεδίο αυτό είναι εκείνο που είναι ορατό στην εμβέλεια της κλάσης `Employee`.

- Στην κλήση:

```
System.out.println(e1.population);
```

γίνεται εκτύπωση της τιμής του στατικού πεδίου `population` της κλάσης `Employee`. Η πρόσβαση σε αυτό γίνεται μέσω της χρήσης της μεταβλητής αναφοράς `e1` που είναι τύπου `Employee`.

- Προσέξτε τώρα: θεωρώντας ότι έχει προηγηθεί ο κώδικας που έχει περιγραφεί προηγουμένως, η εντολή:

```
p1 = e1;
```

αναθέτει το αντικείμενο στο οποίο αναφέρεται η μεταβλητή `e1` ως τιμή στη μεταβλητή αναφοράς `Person p1`. Το αντικείμενο αυτό, όμως, είναι τύπου `Employee`. Οπότε:

- Στην περίπτωση της κλήσης

```
p1.print();
```

έχουμε κλήση μεθόδου στιγμιοτύπου, επομένως θα κληθεί η μέθοδος `print` της εμβέλειας του τύπου του στιγμιοτύπου, άρα της `Employee`.

- Στην περίπτωση της κλήσης

```
p1.get_population();
```

έχουμε κλήση μεθόδου κλάσης μέσω μεταβλητής αναφοράς, επομένως θα κληθεί η μέθοδος `get_population` της εμβέλειας του τύπου της μεταβλητής, άρα της `Person`.

- Στην περίπτωση της κλήσης

```
p1.date.print();
```

παρότι το αντικείμενο στο οποίο αναφέρεται η μεταβλητή `p1` είναι τύπου `Employee`, έχουμε πρόσβαση στο πεδίο στιγμιοτύπου `date` της εμβέλειας της κλάσης `Person` μια που η μεταβλητή `p1` είναι του τύπου αυτού. Έτσι πετύχαμε πρόσβαση στο πεδίο που έχει επαναοριστεί.

- Στην περίπτωση της κλήσης

```
System.out.println(p1.population);
```

γίνεται πρόσβαση στατικού πεδίου μέσω μεταβλητής αναφοράς.

- Μπορούμε να έχουμε πρόσβαση στα στατικά πεδία των κλάσεων με χρήση των ονομάτων των κλάσεων, ώστε να είναι ανεξάρτητη δήλωσης μεταβλητών (ή αντικειμένων τους). Αυτό πραγματοποιούν, για παράδειγμα, οι παρακάτω εκφράσεις:

```
System.out.println(Person.population);
```

```
// static field explicit access
```

```
System.out.println(Employee.population);
```

```
// static field explicit access
```

9.11.2.3 Παράδειγμα: Αρχικοποίηση Τύπων Διεπαφής

Χωρίς να θέλουμε να δώσουμε ιδιαίτερη έμφαση στο θέμα αυτό, μια που ήδη κάναμε αναφορά στην αρχικοποίηση των τύπων διεπαφής, περιγράφοντας τη διαδικασία αρχικοποίησης κλάσεων, παραθέτουμε ένα απλό παράδειγμα για να πειραματιστείτε με την πρόσβαση μελών και την αρχικοποίηση των τύπων διεπαφής.

```
class App {
    public static void main(String[] args) {
//      C rc1 = new C(); // If just instance creation,
//                          // no interface initialization occurs

//      System.out.println("The Int2 var is " + rc1.IMPD2.getData());
//      System.out.println("The CInt1 var is " + rc1.IMPP.data);
//                          // just CInt1 gets initialized

//      System.out.println("The Int1 var is " + rc1.IMPD.getData());
//                          // Error: ambiguity
//      System.out.println("The Ambig var is " + Int1.IMPD.getData());
//                          // use this to resolve it
    }
}

class AD {
    int data;
    { System.out.println("The initial ADate data is " + data ); }

    AD() {}

    AD(int d) {
        data = d;
        System.out.println("The AD data is " + data);
    }

    int getData() { return data; }
}

class BD {
    int data;
    BD(int d) {
        data = d;
        System.out.println("The BD data is: " + data); }
}

interface Int1 {
    AD IMPD = new AD(5);
}

interface CInt1 extends Int1 {
    BD IMPP = new BD(100); // public, static, final
}
```

```
interface Int2 {
    AD IMPD = new AD();
    AD IMPD2 = new AD(1);
}

class C implements Int2, CInt1 {}
```

Στο παραπάνω πρόγραμμα, έχουμε τρεις κλάσεις, AD, BD και C, καθώς επίσης και τρεις τύπους διεπαφής, Int1, CInt1 (extends Int1) και Int2. Οι τύποι διεπαφής αυτοί έχουν (στατικά) πεδία μεταβλητές αναφοράς τύπου AD και BD, αρχικοποιημένες σε αντικείμενα αντίστοιχων τύπων. Η κλάση C υλοποιεί τους τύπους Int2 και CInt1.

Αφού κάνετε κατάλληλους αποσχολιασμούς, μελετώντας και εκτελώντας το παραπάνω πρόγραμμα, παρατηρήστε πότε συμβαίνουν αρχικοποιήσεις τύπων διεπαφής και πότε όχι. Στην περίπτωση που γίνονται αρχικοποιήσεις, παρατηρήστε ποιοι τύποι αρχικοποιούνται. Τέλος, δείτε πάλι ένα πρόβλημα ασάφειας που προκύπτει από χρήση ονόματος και πώς μπορεί να επιλυθεί.

Συγκεκριμένα:

- Κατά τον υπολογισμό της έκφρασης

```
C rc1 = new C();
```

δεν γίνεται καμία αρχικοποίηση τύπου διεπαφής, παρότι η κλάση C υλοποιεί δύο τέτοιους τύπους.

- Αντίθετα, στις περιπτώσεις

```
System.out.println("The Int2 var is " + rc1.IMPD2.getData());
```

και

```
System.out.println("The CInt1 var is " + rc1.IMPP.data);
```

έχουμε αρχικοποιήσεις των τύπων διεπαφής στους οποίους δηλώνονται τα ονόματα που χρησιμοποιούνται. Προσέξτε, όμως, τη δεύτερη περίπτωση, όπου ο τύπος είναι ειδική περίπτωση γενικότερου: ο γενικότερος δεν αρχικοποιείται.

- Με την εντολή:

```
System.out.println("The Int1 var is " + rc1.IMPD.getData());
```

γίνεται προσπάθεια χρήσης πεδίου των τύπων διεπαφής που υλοποιεί η κλάση C, για την προέλευση του οποίου, όμως, υπάρχει ασάφεια. Τότε, πρέπει να δηλώσουμε ρητά τον τύπο από την εμβέλεια του οποίου θέλουμε να προκύπτει το πεδίο. Δηλαδή, π.χ.,

```
System.out.println("The Ambig var is " + Int1.IMPD.getData());
```

Αν θέλετε να χρησιμοποιήσετε τύπους διεπαφής, καλείστε να αναφερθείτε οπωσδήποτε στα εγχειρίδια της Java για την έκδοση που χρησιμοποιείτε, μια που οι προδιαγραφές τους μεταξύ των εκδόσεων έχουν διαφορές.

9.12 Τα Πακέτα της Java

Από τις πρώτες μας συζητήσεις σχετικά με την οργάνωση των προγραμμάτων Java, κάναμε αναφορά στην έννοια των *πακέτων*. Ένα πακέτο ομαδοποιεί ένα σύνολο κλάσεων. Η ορατότητα των κλάσεων μπορεί να περιοριστεί στο πακέτο. Επίσης, όπως έχουμε τονίσει, έχουμε τη δυνατότητα να περιορίσουμε τη χρήση των μελών μιας κλάσης μέσα στο πακέτο στο οποίο ορίζεται η κλάση. Αυτό δηλώνεται μη προσδιορίζοντας ορατότητα (public, protected ή private) κατά τη δήλωση του μέλους στην κλάση.

Παρότι τα πακέτα διαφέρουν ριζικά από τις κλάσεις, έχουν κοινό το ότι αποτελούν εργαλείο οργάνωσης κώδικα και παρέχουν δική τους εμβέλεια στην οποία μπορούμε να περιορίσουμε τη χρήση ονομάτων. Θα μπορούσαμε να δούμε μια συνάφεια με τους χώρους ονομάτων της C++, ως προς την ομαδοποίηση ονομάτων μέσα σε εμβέλεια. Στους χώρους ονομάτων, όμως, δεν δίνεται η δυνατότητα απόκρυψης πληροφορίας.

Ας συνοψίσουμε λίγο την περιγραφή σχετικά με τα πακέτα, χωρίς να επεκταθούμε σε λεπτομέρειες. Ο στόχος είναι να δούμε τη βασική λειτουργικότητά τους.

- Κάθε πακέτο έχει τον δικό του χώρο ονομάτων τύπων.
- Τα μέλη των πακέτων είναι τύποι, για παράδειγμα κλάσεις και τύποι διεπαφής.
- Οι τύποι που συνιστούν ένα πακέτο αποκαλούνται *μέλη του πακέτου* (package members) και μπορούμε να περιορίσουμε τη χρήση τους μόνο στο πακέτο αυτό.
- Στη γενική περίπτωση, αναφερόμαστε σε ένα όνομα που ορίζεται μέσα σε ένα πακέτο, χρησιμοποιώντας το πλήρες όνομα το οποίο συμπεριλαμβάνει το όνομα του πακέτου, π.χ.

```
p1.p11.BlaClass.bla();
```

Στο παράδειγμα, καλείται η μέθοδος `bla`, της κλάσης `BlaClass` η οποία ανήκει στο πακέτο `p11` που ιεραρχικά βρίσκεται κάτω από το πακέτο `p1` (το πακέτο `p11` αποτελείται από κώδικα που είναι γραμμένος στα αρχεία του (υπο)καταλόγου `p1.p11`).

- Μπορούμε, όμως να κάνουμε δηλώσεις εισαγωγής με χρήση της λέξης-κλειδιού `import`. Μια δήλωση εισαγωγής μπορεί να αναφέρεται σε ολόκληρο πακέτο ή σε συγκεκριμένο μέλος του, π.χ.

```
import p1.p11.*;
```

δηλαδή, χρήση wildcard για να αναφερθούμε σε όλα τα ονόματα των τύπων του πακέτου, ή

```
import p1.p11.BlaClass
```

χρήση ονόματος συγκεκριμένου τύπου. Όταν γίνει μια δήλωση εισαγωγής, επιτρέπεται να χρησιμοποιούνται τύποι από το άλλο πακέτο με το απλό όνομά τους μόνο και όχι το `fully qualified`, στο οποίο αναφέρεται ρητά το πακέτο, π.χ. αν έχουμε μια δήλωση:

```
import p1.p11.*;
```

μετά, μπορούμε να αναφερθούμε απευθείας στην μέθοδο του παραδείγματος ως:

```
BlaClass.bla();
```

- Ένα πακέτο περιλαμβάνει μία ή παραπάνω μονάδες μεταγλώττισης. Κάθε μονάδα μεταγλώττισης έχει άμεση πρόσβαση σε όλους τους τύπους του πακέτου στο οποίο ανήκει, καθώς και στους `public` τύπους του πακέτου της γλώσσας `java.lang`, που προσφέρει κλάσεις που είναι θεμελιώδεις για τη γλώσσα.
- Μια μονάδα μεταγλώττισης αποτελείται από τρία (προαιρετικά) μέρη:
 1. Μια δήλωση πακέτου, που περιλαμβάνει το *πλήρες όνομα* (fully qualified name) του πακέτου στο οποίο συμμετέχει η μονάδα, π.χ.


```
package p1.p11;
```

 Αν παραλείπεται η δήλωση, η μονάδα είναι μέρος του πακέτου χωρίς όνομα, (*unnamed* πακέτου).
 2. Δηλώσεις εισαγωγής
 3. Δηλώσεις τύπων
- Για να χρησιμοποιήσουμε ένα `public` μέλος πακέτου εκτός πακέτου, πρέπει να κάνουμε ένα από τα εξής:
 - είτε να χρησιμοποιήσουμε το `fully qualified` όνομά του, που συμπεριλαμβάνει και το όνομα του πακέτου

- είτε να το “εισαγάγουμε” μέσω μιας δήλωσης `import`, η οποία αναφέρει ρητά το όνομα του μέλους, οπότε απαλλασσόμαστε από τη χρήση του ονόματος του πακέτου
- είτε να “εισαγάγουμε” μέσω μιας δήλωσης `import` όλα τα ονόματα του πακέτου (με χρήση `wild-card`), οπότε η απαλλαγή αφορά όλα τα ονόματα των μελών του πακέτου
- Υπάρχει η εκδοχή `static import` για να “εισαγάγουμε” τα στατικά μέλη κλάσεων ενός πακέτου. Στην περίπτωση αυτή, δεν είναι καν απαραίτητη η χρήση του ονόματος της κλάσης. Για παράδειγμα, αν έχουμε τη δήλωση:

```
import static p1.p11.BlaClass;
```

τότε, αν δεν υπάρχει ασάφεια, μπορούμε να γράψουμε απευθείας

```
bla();
```

Γενικά, όμως, η χρήση του `import static` δεν προτείνεται.

Η οργάνωση του κώδικα σε πακέτα συνδέεται με την κατανομή του κώδικα του πηγαίου προγράμματος σε αρχεία. Συγκεκριμένα, ισχύουν τα παρακάτω.

- Η οργάνωση των ονομάτων των πακέτων είναι ιεραρχική.
- Η ιεραρχία στην οργάνωση των πακέτων δεν έχει κάποια σχέση με τα δικαιώματα πρόσβασης.
- Παρόλο που το πρότυπο της Java δεν απαιτεί ιεραρχικό σύστημα αρχείων, συνήθως οργανώνουμε τα αρχεία του κώδικά μας ως εξής:
 - Γράφουμε τον κώδικά κάθε κλάσης (τύπου, γενικότερα) σε ένα αρχείο με όνομα το όνομα της κλάσης και κατάληξη `.java`
 - Τοποθετούμε το αρχείο αυτό σε έναν κατάλογο με όνομα το όνομα του πακέτου που ανήκει ο τύπος.
 - Αντίστοιχη οργάνωση έχουν και τα αρχεία `.class` που προκύπτουν από τη μεταγλώττιση.
 - Το πλήρες μονοπάτι του καταλόγου κάτω από τον οποίο βρίσκεται η ιεραρχία των `.class` αρχείων λέγεται *μονοπάτι κλάσεων* (class path) και προσδιορίζεται από τη μεταβλητή περιβάλλοντος/συστήματος (CLASSPATH).

9.12.1 Παράδειγμα Κώδικα με Χρήση Πακέτων

Στο παρακάτω παράδειγμα, μπορείτε να δείτε ένα πρόγραμμα που αποτελείται από ένα σύνολο πακέτων. Σχολιασμένο μέσα στον κώδικα περιλαμβάνεται και το όνομα του αρχείου στο οποίο βρίσκεται ο κώδικας, ώστε να γίνει εμφανής η εκμετάλλευση της ιεραρχικής δομής των καταλόγων. Για απλότητα στη χρήση, οι κλάσεις των πακέτων περιέχουν μόνο στατικές μεθόδους. Με βάση το παράδειγμα, μπορούμε να δούμε τα εξής:

- Ένα πακέτο, το `p1`, να εκτείνεται σε παραπάνω από ένα αρχεία. Το αντίθετο δεν επιτρέπεται: μέσα στο ίδιο αρχείο δεν μπορούμε να έχουμε κώδικα και δεύτερου πακέτου.
- Χρήση του πλήρους ονόματος για πρόσβαση σε μέλη άλλων πακέτων. Για παράδειγμα, στην κλάση `cp2` του πακέτου `p1`, η κλήση

```
p1.inner.cp2.fun();
```

από το σώμα της μεθόδου `static void foo()` αναφέρεται στην `public static` μέθοδο της κλάσης `cp2` του πακέτου `p1.inner`. Στην περίπτωση που η μέθοδος δεν είχε δηλωθεί ως `public`, αυτή η κλήση θα προκαλούσε σφάλμα μεταγλώττισης.

- Χρήση δηλώσεων εισαγωγής και τι αντίκτυπο έχουν. Για παράδειγμα, η δήλωση εισαγωγής
`import p1.*;`

μας επιτρέπει να έχουμε πρόσβαση στη μέθοδο `public static void bla()` της κλάσης `cp1` που είναι ορισμένη στο πακέτο `p1`, απευθείας, γράφοντας
`cp1.bla();`

- Προσέξτε τα ονόματα των `public` κλάσεων: είναι τα ίδια με τα ονόματα των αρχείων.

Αποσχολιάζοντας κατάλληλες γραμμές, κάντε δοκιμές μεταγλώττισης και εκτέλεσης, ώστε να δείτε τη συμπεριφορά.

```
// directory: ./p1
// file: cp1.java

//class cp1 is public, should be declared in a file named cp1.java
package p1;

public class cp1 {
    public static void main(String[] args) {
        System.out.println("These tricky packages!!!!!!");
        p1.inner.cp2.fun(); // ERROR:fun() is not public in cp2;
                           // cannot be accessed from outside package
                           // Note that there are two alternative
                           // method headers in that file

        cp2.foo();
        p2.cp2.blee();
    }

    public static void bla() {
        System.out.println("This is p1.cp1.bla() executing");
    }
}

////////////////////////////////////
// directory: ./p1
// file: cp2.java

package p1;

public class cp2 {

    public static void bloo() {
        System.out.println("This is p1.cp2.bloo() executing");
    }

    static void foo() {
        System.out.println("Inside p1.cp2.foo()");
        p1.inner.cp2.fun();
        bla();
    }
}
```

```

    private static void bla() {
        System.out.println("This is p1.cp2.bla() executing");
    }
}

// package p2; // Not allowed

////////////////////////////////////
// directory: ./p1/inner
// file: cp2.java

package p1.inner;

// import p1; // Cannot be written like that
import p1.*; // import (public) types from p1
              // to use without name qualification

public class cp2 {

    // static void fun() {
    public static void fun() {
        System.out.println("This is p1.inner.cp2.fun() executing");
    //     p1.cp1.bla();
        cp1.bla();
    }
}

////////////////////////////////////
// directory: ./p2
// file: cp2.java

package p2;

import p1.*;

public class cp2 {

    public static void blee() {
        System.out.println("This is p2.cp2.blee() executing");
        p1.cp2.bloo(); // Needs full path qualification
        p1.inner.cp2.fun();
    }
}

```

9.13 Άλλα (Κρίσιμα) Θέματα σχετικά με την Java

Θα κλείσουμε τη σύννοσή μας για τη γλώσσα Java κάνοντας μια σύντομη αναφορά σε κάποια στοιχεία της που κρίνουμε σκόπιμο να θέσουμε υπόψη στον αναγνώστη. Δεν θα γίνει λεπτομερής ανάλυση. Όντας ενήμερος, ο αναγνώστης μπορεί να ανατρέξει στη βιβλιογραφία, αν ενδιαφέρεται περαιτέρω για κάποια από αυτά.

9.13.1 Αρχικοποίηση μέσω Αντιγραφής

Στην Java δεν υπάρχει αρχικοποίηση μέσω αντιγραφής. Το αποτέλεσμα μιας συνάρτησης αρχικοποίησης μέσω αντιγραφής το επιτυγχάνουμε υλοποιώντας τη διεπαφή `Cloneable`, ώστε να επαναορίσουμε τη μέθοδο `clone` της κλάσης `Object`.

Ακολουθεί ένα παράδειγμα ορισμού μεθόδου αρχικοποίησης μέσω αντιγραφής για μια κλάση, την `PersonOrder`, της οποίας στιγμιότυπα επιθυμούμε να μπορούμε να δημιουργήσουμε αρχικοποιώντας τα μέσω αντιγραφής. Όπως μπορείτε να παρατηρήσετε στον κώδικα της κλάσης, αυτή δηλώνεται να υλοποιεί τον τύπο διεπαφής `Cloneable`.

```
class PersonOrder implements Cloneable
```

και δίνει ορισμό στη μέθοδο

```
public Object clone()
```

τέτοιον ώστε να επιτυγχάνουμε το επιθυμητό αποτέλεσμα.

```
class App {
    public static void main(String[] args) {

        Dish d = new Dish(20, "Ice Cream");
        PersonOrder sallysorder = new PersonOrder(d);
        PersonOrder ladysorder = (PersonOrder) sallysorder.clone();
        System.out.println(sallysorder.dish ==
                           ladysorder.dish); // not the same Dish object

        Dish d1 = new Dish(400, "Cake");
        sallysorder.dish = d1;

        sallysorder.dish.eat();
        ladysorder.dish.eat(); // ladysorder didn't change

        PersonOrder childsoorder = new PersonOrder(ladysorder.dish);
        System.out.println(childsoorder.dish ==
                           ladysorder.dish); // the same Dish object

        ladysorder.dish = new Dish(600, "SuperCake");
        ladysorder.dish.eat();
        childsoorder.dish.eat(); // childsoorder refers to the old dish
                                // of ladysorder

        System.out.println(childsoorder.dish ==
                           ladysorder.dish);
                                // not the same Dish object any more
    }
}
```

```
class Dish {
    int calories = 1000;
    String name = "Junk food";
    Dish() {
        System.out.println("The new dish named: " + name +
            " contains " + calories + " calories!");
    }

    Dish(String nam) {
        name = nam;
        System.out.println("The new dish named: " + name +
            " contains " + calories + " calories!");
    }

    Dish(int cal, String nam) {
        calories = cal;
        name = nam;
        System.out.println("The new dish named: " + name +
            " contains " + calories + " calories!");
    }

    void eat() {
        System.out.println("Eating " + name );
    }

    int get_calories() {
        return calories;
    }
}

class PersonOrder implements Cloneable {
    Dish dish;
    PersonOrder(Dish d) {
        dish = d;
        System.out.println("A new PersonOrder is created which is: ");
        dish.eat();
    }

    public Object clone() {
        Dish d = new Dish(this.dish.calories, this.dish.name);
        PersonOrder person = new PersonOrder(d);
        return person;
    }
}
```

9.13.2 Περισυλλογή Σκουπιδιών και Διαδικασίες που Σχετίζονται με “Αχρηστα” Αντικείμενα

Από την αρχή της συζήτησής μας σχετικά με τις βασικές σχεδιαστικές αρχές της Java, είπαμε ότι μια από τις απαιτήσεις ήταν η αποφυγή σφαλμάτων που μπορεί να προκύψουν σε ένα πρόγραμμα. Στα προγράμματα C/C++, μια από τις συνήθεις πηγές σφαλμάτων προέρχεται από τη βασική δύναμή τους που είναι το ότι παρέχουν στον προγραμματιστή δυνατότητες για διαδικασίες χαμηλού επιπέδου. Μία από αυτές είναι η δυνατότητα διαχείρισης της μνήμης μέσω δεικτών. Αυτή, παρόλα τα οφέλη που προσφέρει, είναι και ένα από τα προβλήματά τους, λόγω της πολυπλοκότητας στη χρήση, καθώς και βασική πηγή σφαλμάτων των προγραμμάτων τους.

Η Java αποφεύγει το πρόβλημα αυτό αναλαμβάνοντας η ίδια τη διαχείριση της μνήμης. Ο προγραμματιστής ορίζει μεταβλητές και δημιουργεί αντικείμενα, αλλά η ανάκτηση του χώρου που καταλαμβάνεται από οντότητες οι οποίες πλέον δεν είναι προς χρήση επαφίεται στη γλώσσα. Όσον αφορά αντικείμενα στα οποία πλέον δεν υπάρχει κάποια αναφορά, υπάρχει διαδικασία περισυλλογής σκουπιδιών της γλώσσας, η οποία, όταν ενεργοποιηθεί, ανακτά τους χώρους τους. Έτσι, ο προγραμματιστής ούτε πρέπει, ούτε μπορεί να ασχοληθεί με τη διαδικασία αυτή.

Για να δούμε λίγο καλύτερα τι εννοούμε με το παραπάνω. Κατά τη διάρκεια εκτέλεσης ενός προγράμματος Java, δημιουργούμε αντικείμενα. Συνήθως, τα αντικείμενα αυτά προσπελαύνονται με χρήση μεταβλητών αναφοράς που αναφέρονται σε αυτά. Ενδέχεται, όμως, από ένα σημείο και ύστερα, αυτά να μην είναι δυνατόν να χρησιμοποιηθούν, διότι δεν υπάρχει καμιά αναφορά σε αυτά. Μπορεί, δηλαδή, είτε οι μεταβλητές που αναφερόντουσαν σε ένα αντικείμενο να άλλαξαν τιμή και, για παράδειγμα, να αναφέρονται πλέον σε άλλο αντικείμενο είτε το αντικείμενο να δημιουργήθηκε προσωρινά στο πλαίσιο μιας έκφρασης. Τότε, δεν πρέπει να δεσμεύει πλέον χώρο στη μνήμη (συγκεκριμένα στον σωρό). Στην Java, η ανάκτηση του χώρου σε αυτές τις περιπτώσεις γίνεται αυτόματα από μια διαδικασία που ενεργοποιείται από τη JVM, την περισυλλογή σκουπιδιών. Αυτή η διαδικασία αναλαμβάνει να ελέγξει αν υπάρχουν αντικείμενα στη μνήμη τα οποία δεν χρησιμοποιούνται, ελέγχοντας αν υπάρχουν αναφορές σε αυτά. Για κάθε αντικείμενο για το οποίο αυτό ισχύει, κάποια στιγμή ο χώρος του “επιστρέφεται” στο σύστημα, γίνεται, δηλαδή, διαθέσιμος προς χρήση.

Με βάση τα παραπάνω, καταλαβαίνουμε ότι η αναγκαιότητα ορισμού συνάρτησης καταστροφής μιας κλάσης από τον προγραμματιστή της κλάσης είναι εξαιρετικά περιορισμένη, έως μηδενική. Στην περίπτωση αποδέσμευσης της μνήμης που καταλαμβάνει μια οντότητα, η ανάκτηση των χώρων στους οποίους μπορεί να αναφερόταν η οντότητα θα επιτευχθεί χωρίς παρέμβαση του προγραμματιστή. Για τον λόγο αυτό, στην Java δεν προσφέρεται λειτουργικότητα αντίστοιχη με τη C++.

Παρ’ όλα αυτά, η Java διαθέτει κάποιες μεθοδολογίες που μπορούν να ακολουθηθούν, προκειμένου να γίνουν κάποιες διαδικασίες όταν πλέον ένα αντικείμενο είναι υποψήφιο για καταστροφή. Δεδομένου ότι πλέον δεν υπάρχει ανάγκη διαχείρισης μνήμης, οι διαδικασίες αυτές αφορούν κυρίως αποδέσμευση άλλων πόρων.

Η πιο διαδεδομένη περίπτωση ήταν ο ορισμός μιας μεθόδου της κλάσης `Object`, της:

`protected void finalize()`

Η μέθοδος αυτή καλείται από τη διαδικασία περισυλλογής σκουπιδιών για ένα αντικείμενο στο οποίο δεν υπάρχουν πλέον αναφορές. Κατά συνέπεια, μπορούμε να την ορίσουμε με τέτοιον τρόπο, ώστε να γίνονται όποιες διαδικασίες, κυρίως αποδεσμεύσεις, πρέπει να γίνουν πριν “καταστραφεί” το αντικείμενο αυτό, μέσω του οποίου και μόνο είχαμε πρόσβαση σε κάποιους πόρους. Η χρήση της μεθόδου αυτής, όμως, απεδείχθη ότι δημιουργούσε προβλήματα που οδηγούσαν σε σφάλματα, οπότε η χρήση της αντενδείκνυται. Επιπροσθέτως, δεν ήταν εγγυημένο το πότε θα γίνει η κλήση της. Έτσι, από την έκδοση SE9, η χρήση της αντιπροτείνεται (με σχετική επισήμειωση στη δήλωσή της — `@Deprecated(since="9")`). Αντί αυτής, αν κρίνεται αναγκαίο, προτείνεται:

- Η χρήση της μεθόδου

`public Cleaner.Cleanable register(Object obj, Runnable action)`

της κλάσης `Cleaner`. Πρακτικά, μπορούμε να προσδιορίσουμε τη διαδικασία, `action`, που θέλουμε

να εκτελεστεί, όταν το αντικείμενο `obj` βρεθεί σε μια “ανενεργή” κατάσταση (γίνει “phantom reachable”).

- Ή/και να χαρακτηρίσουμε αντικείμενα ως `PhantomReferences`.
- Στην περίπτωση κλάσεων των οποίων τα στιγμιότυπα είναι συνδεδεμένα με πόρους εκτός σωρού, υλοποίηση για τον τύπο διεπαφής `AutoCloseable`.

Για περισσότερες λεπτομέρειες σχετικά με τα παραπάνω, μπορείτε να αναφερθείτε στα εγχειρίδια της Java. Σε κάθε περίπτωση, η χρήση τους είναι πολύ τεχνική και εξειδικευμένη.

9.13.3 Κάποιες Επισημάνσεις, ακόμα

Στο σημείο αυτό θα επισημάνουμε κάποια επιπλέον σημεία από τη λειτουργικότητα που προσφέρει η Java. Θα τα παραθέσουμε επιγραμματικά, ώστε ο αναγνώστης να είναι ενήμερος για την ύπαρξή τους. Αν ενδιαφέρεται περαιτέρω, μπορεί να αναζητήσει λεπτομέρειες από βιβλιογραφία σχετική με τη γλώσσα, καθώς και από τα εγχειρίδιά της.

Autoboxing και Unboxing: Για κάθε πρωτογενή τύπο, η Java προσφέρει από το API της μια αντίστοιχη *wrapper* κλάση. Αυτές είναι οι κλάσεις `Boolean`, `Character`, `Integer`, `Long`, `Float` και `Double`. Οι κλάσεις αυτές έχουν ένα πεδίο το οποίο είναι μεταβλητή του αντίστοιχου πρωτογενούς τύπου. Ο σκοπός τους είναι να δίνεται η δυνατότητα να χειριζόμαστε τιμές πρωτογενών τύπων σαν να ήταν αντικείμενα.

Η διαδικασία μετατροπής πρωτογενών τύπων στους αντίστοιχους *wrapper* τύπους λέγεται *autoboxing*. Η αντίστροφη λειτουργία κατά την οποία ένα αντικείμενο μιας *wrapper* κλάσης μετατρέπεται σε τιμή του αντίστοιχου πρωτογενούς τύπου λέγεται *unboxing*. Οι μετατροπές αυτές γίνονται αυτόματα από τον μεταγλωττιστή και συμβαίνουν στις εξής περιπτώσεις:

- κατά το πέρασμα παραμέτρων μιας τιμής πρωτογενούς τύπου σε ένα όρισμα του αντίστοιχου *wrapper* τύπου (ή το αντίστροφο)
- κατά την ανάθεση σε μεταβλητή *wrapper* τύπου τιμής πρωτογενούς τύπου (ή το αντίστροφο)

Generics: Στην Java έχουμε δυνατότητες προγραμματισμού γενικεύσεων με χρήση κλάσεων που αποκαλούνται *γενικεύσεις* (*generics*). Οι γενικεύσεις επιτρέπουν να χρησιμοποιήσουμε παραμέτρους τύπων κατά τον ορισμό τύπων (κλάσεων ή τύπων διεπαφής) ή/και μεθόδων τους. Προκειμένου να χρησιμοποιήσουμε τις γενικεύσεις, πρέπει να δώσουμε συγκεκριμένους τύπους ως τιμή στις παραμέτρους τους. Τύποι γενικεύσεων προσφέρονται από το API της γλώσσας. Μπορούμε, όμως, να ορίσουμε και δικούς μας.

Εννοιολογικά και συντακτικά, αντιστοιχούν στα περιγράμματα της C++. Έχουν, όμως, και αρκετές διαφορές. Υπάρχουν διαφορές που σχετίζονται με την υλοποίησή τους. Στη C++, για κάθε γενίκευση που χρησιμοποιούμε, δημιουργείται ένα διαφορετικό στιγμιότυπό της, αντικαθιστώντας τις παραμέτρους με τους τύπους που δίνουμε ως τιμή. Επίσης, φτιάχνονται στιγμιότυπα μόνο για τις συναρτήσεις που χρησιμοποιούνται. Στην Java αυτό δεν ισχύει: μόνο μια κλάση παράγεται από τη μεταγλώττιση. Αυτή λειτουργεί για κάθε τιμή παραμέτρων τύπου. Αυτό, κατ’ ουσίαν, γίνεται δυνατό λόγω του ότι όλοι οι τύποι κλάσεων αποτελούν υποκλάσεις της κλάσης `Object`. Μια ακόμα διαφορά που πρέπει να αναφερθεί, και σχετίζεται με την προηγούμενη παρατήρηση, είναι ότι ενόσω οι παράμετροι τύπου των περιγραμμάτων της C++ μπορούν να χρησιμοποιούνται και σε στατικά μέλη, στους τύπους γενικεύσεων της Java αυτό δεν είναι δυνατό.

Εξαιρέσεις (Exceptions): Κρίσιμη λειτουργικότητα μιας γλώσσας για τον καθορισμό της ορθής λειτουργίας και χρήσης των προγραμμάτων είναι η δυνατότητα ορισμού και χειρισμού *εξαιρέσεων*. Οι εξαιρέσεις αναφέρονται σε συμβάντα κατά την εκτέλεση ενός προγράμματος που δεν αναμένονται στην περίπτωση ομαλής εξέλιξής της. Η Java προσφέρει στον προγραμματιστή τη δυνατότητα χειρισμού τέτοιων συμβάντων. Αυτό, μάλιστα, γίνεται εντάσσοντας τα εργαλεία που προσφέρει στη γενικότερη ιεραρχία κλάσεων της γλώσσας.

Συγκεκριμένα, εξαιρέσεις μπορεί να καθοριστούν σε μεθόδους, χρησιμοποιώντας την εντολή `throw`. Αυτή χρειάζεται ένα όρισμα, το οποίο είναι αντικείμενο της κλάσης `Throwable`. Υποκλάσεις της κλάσης είναι οι κλάσεις `Error` και `Exception`. Στα προγράμματά μας, χειριζόμαστε εξαιρέσεις ανταλλάσσοντας αντικείμενα κυρίως της δεύτερης κλάσης με χρήση των γλωσσικών στοιχείων `throw` και `catch`. Η Java προσφέρει διάφορες κλάσεις που είναι υποκλάσεις της κλάσης `Exception`, στις οποίες μπορεί να ανήκουν τα αντικείμενα. Μπορούμε, επίσης, να ορίσουμε και δικές μας εξαιρέσεις, ορίζοντας τις αντίστοιχες κλάσεις ως υποκλάσεις της `Exception`. Η κλάση `Error` αφορά προβλήματα που αφορούν τη JVM. Κώδικας κατά την εκτέλεση του οποίου ενδέχεται να προκύψουν προβλήματα πρέπει να εμπεριέχεται μέσα σε `try-catch blocks`.

Νήματα (Threads): Η έννοια του *ταυτόχρονου προγραμματισμού* (*concurrent programming*) είναι θεμελιώδης για την Java. Έτσι, κάνουμε μια αναφορά στα *νήματα εκτέλεσης*, μια που αποτελούν το βασικό εργαλείο του για την Java. Ο προγραμματισμός νημάτων είναι το ένα από τα δύο εναλλακτικά που προσφέρονται για ταυτόχρονο προγραμματισμό. Το δεύτερο είναι οι *διεργασίες* (*processes*). Ένα νήμα αποτελεί μια μονάδα εκτέλεσης. Τα νήματα λειτουργούν μέσα σε διεργασίες και έχουν κοινούς πόρους με αυτές. Κάθε διεργασία έχει τουλάχιστον ένα νήμα, όπως, κατ' επέκταση και κάθε εφαρμογή.

Κλείνουμε την παράγραφο, συμπεριλαμβάνοντας κάποιες επιπλέον κρίσιμες επισημάνσεις. Όπως αναφέραμε εξ αρχής, η Java προσφέρει πλούσια λειτουργικότητα ως έτοιμο λογισμικό μέσω του API. Θα ήταν παράλειψη να μην κάνουμε αναφορά σε κάποια διαχρονικά στοιχεία της που συνέβαλαν ευρέως και στην εξάπλωση της γλώσσας.

Το πρώτο αφορά την *ανάπτυξη κατανεμημένων εφαρμογών πελατών-εξυπηρετή* (*client-server*) και επιτυγχάνεται με χρήση του πακέτου RMI (*Remote Method Invocation*). Αυτό διευκόλυνε πολύ την ανάπτυξη κατανεμημένων εφαρμογών πάνω στο διαδίκτυο. Με το λογισμικό του πακέτου αυτού δίνεται η δυνατότητα από ένα πρόγραμμα που εκτελείται σε μια JVM να καλέσουμε μεθόδους εφαρμοσμένες σε αντικείμενα άλλου προγράμματος που εκτελείται από άλλη JVM σε απομακρυσμένο υπολογιστή. Στη συνέχεια, το πρόγραμμα που εκτελείται στη δεύτερη μπορεί να επιστρέψει αποτελέσματα στην πρώτη. Η επικοινωνία γίνεται είτε μέσω του πρωτοκόλλου HTTP είτε μέσω του πρωτοκόλλου Java Serialization. Γενικά, η ανταλλαγή πληροφορίας γίνεται σε ένα υψηλό, αφαιρετικό επίπεδο εναρμονισμένο με την ίδια τη γλώσσα και τα αντικειμενοστραφή χαρακτηριστικά της.

Στην ίδια λογική αφαιρετικότητας κινείται και η λειτουργικότητα ενός άλλου πακέτου που επιτρέπει πρόσβαση σε δεδομένα οργανωμένα σε πίνακες, κυρίως σε σχεσιακές βάσεις δεδομένων. Πρόκειται για το πακέτο JDBC. Αυτό προσφέρει τα μέσα για σύνδεση με μια βάση, υποβολή ερωτημάτων στη βάση αυτή και ανάκτηση και επεξεργασία αποτελεσμάτων. Και τα τρία στάδια περιγράφονται σε υψηλό επίπεδο, φιλικό στον προγραμματιστή και απαλλαγμένο από πολλές τεχνικές και εξειδικευμένες λεπτομέρειες.

Τέλος, μια άλλη σημαντική δυνατότητα που πρέπει να αναφέρουμε είναι η *ευκολία ανάπτυξης γραφικών διεπαφών χρήστη* μέσω των πακέτων που προσφέρει η γλώσσα. Αυτά ξεκινούν από το παλαιότερο AWT πάνω στο οποίο βασίστηκε το μεταγενέστερο πακέτο Swing και φτάνουν μέχρι το πιο σύγχρονο πακέτο JavaFX. Όλα προσφέρουν μεγάλες δυνατότητες για εύκολη ανάπτυξη γραφικών διεπαφών με βελτίωση στην απόδοσή τους ανά τον χρόνο. Το πακέτο Swing έχει χρησιμοποιηθεί ευρέως αλλά οι νεότερες εφαρμογές χρησιμοποιούν συχνά το πακέτο JavaFX επιτρέποντας ο κώδικας του συστήματος να μεταφερθεί και σε Android και iOS.

9.14 Σύνοψη Κεφαλαίου

- Η έμπνευση για την Java προήλθε από τον χώρο των ευφών ηλεκτρονικών συσκευών και ο σχεδιασμός της ξεκίνησε στη SUN microsystems το 1990, ως έργο που στόχευε σε συσκευές δικτυωμένες με παραδοσιακές μεθόδους.
- Λόγω της ανάπτυξης του διαδικτύου και του Παγκόσμιου Ιστού, καθώς και της καταλληλότητας του πλαισίου αυτού για τους στόχους της ομάδας του έργου, οι προσπάθειες στράφηκαν στην ανάπτυξη ενός εργαλείου το οποίο να διευκολύνει την ανάπτυξη εφαρμογών για τον χώρο αυτό, με βασικές απαιτήσεις του σχεδιασμού του:
 - τη μεταφερισιμότητα του κώδικα,
 - την αποφυγή σφαλμάτων,
 - τη διευκόλυνση στην ανάπτυξη κατανεμημένων εφαρμογών (ακόμα και) σε ετερογενείς πλατφόρμες,
 - τη διευκόλυνση στην ανάπτυξη φιλικών (γραφικών) διεπαφών ανθρώπου-υπολογιστή.
- Το 1995 εμφανίστηκε η πρώτη έκδοση της γλώσσας Java.
- Τα προγράμματα Java μεταγλωττίζονται σε αρχεία κλάσεων, ένα για κάθε κλάση τους. Τα αρχεία κλάσεων περιέχουν την περιγραφή των κλάσεων σε *bytecodes*.
- Η μεταφερισιμότητα των προγραμμάτων Java επιτυγχάνεται από το γεγονός ότι τα μεταγλωττισμένα προγράμματα δεν εκτελούνται απευθείας από τον υπολογιστή, αλλά από μία αφηρημένη υπολογιστική μηχανή, την *Java Virtual Machine (JVM)*, η οποία εκτελείται σε αυτόν και η οποία “ερμηνεύει” τα *bytecodes*.
- Η σύνταξη της Java ακολουθεί εκείνη της C και της C++, όμως τα προγράμματα Java είναι πλήρως αντικειμενοστραφή: ο κώδικας ενός προγράμματος Java είναι όλος γραμμένος σε κλάσεις. Αυτές αποτελούν μέρος μιας ιεραρχίας με ρίζα την κλάση *Object* της γλώσσας.
- Η γλώσσα προσφέρει πολλές έτοιμες κλάσεις μέσω του *Application Programming Interface (API)* που τη συνοδεύει.
- Επιπλέον, οι κλάσεις στην Java είναι οργανωμένες σε *πακέτα* που ενδεχομένως περιορίζουν την ορατότητά τους (αν μια κλάση δεν δηλωθεί να ανήκει σε ένα συγκεκριμένο πακέτο, θεωρείται ότι ανήκει στο *unnamed* πακέτο).
- Ένα πρόγραμμα Java περιέχει τουλάχιστον μία κλάση, την *κύρια κλάση*. Αυτή χαρακτηρίζεται από το ότι περιέχει μια στατική μέθοδο με πρωτότυπο:


```
public static void main(String[] args)
```

 Η εκτέλεση ενός προγράμματος Java ξεκινά με κλήση της μεθόδου αυτής.
- Μια κλάση Java έχει ως μέλη, κυρίως, *πεδία* και *μεθόδους*. Αυτά διακρίνονται σε *μέλη κλάσης* και *μέλη στιγμιοτύπου*. Στην πρώτη περίπτωση πρόκειται για μεταβλητές (πεδία) και συναρτήσεις (μεθόδους) που είναι ανεξάρτητες αντικειμένου και απλά είναι εμβέλειας κλάσης.
- Μπορούν να οριστούν και κλάσεις ως μέλη κλάσεων. Οι *εμφωλευμένες* αυτές κλάσεις διακρίνονται σε *διάφορες κατηγορίες*.
- Η ορατότητα ενός μέλους μιας κλάσης μπορεί να:

- είναι περιορισμένη στην κλάση (δήλωση μέλους με χρήση της λέξης-κλειδιού `private`),
 - εκτείνεται και στο πακέτο στο οποίο ανήκει η κλάση (χωρίς προσδιορισμό ορατότητας),
 - εκτείνεται και στις υποκλάσεις της κλάσης, ανεξαρτήτως πακέτου (δήλωση μέλους με χρήση της λέξης-κλειδιού `protected`),
 - εκτείνεται σε όλον τον κώδικα του προγράμματος (δήλωση μέλους με χρήση της λέξης-κλειδιού `public`).
- Μπορούμε να περιορίσουμε την εμβέλεια του ονόματος μιας μη εμφωλευμένης (εξωτερικής) κλάσης μόνο μέσα στο πακέτο που ορίστηκε η κλάση.
 - Μπορούμε να απαγορεύσουμε να δημιουργούνται αντικείμενα μιας κλάσης δηλώνοντάς την ως *αφηρημένη*. Σε αφηρημένες κλάσεις επιτρέπεται να έχουμε δηλώσεις μεθόδων χωρίς ορισμό (αφηρημένες μεθόδους).
 - Η κληρονομικότητα στην Java εκφράζει τη σχέση “*isa*” μεταξύ τύπων.
 - Μια κλάση είναι άμεση υποκλάση ακριβώς μίας κλάσης. Η κλάση αυτή είτε δηλώνεται ρητά με χρήση της λέξης-κλειδιού `extends`, είτε είναι η κλάση `Object`, αν δεν υπάρχει τέτοια δήλωση.
 - Ειδική μορφή “κλάσης” είναι οι *τύποι διεπαφής*. Αυτοί, βασικά, αποτελούνται από προδιαγραφές τις οποίες οι κλάσεις που αντιπροσωπεύονται από αυτές οφείλουν να υλοποιήσουν. Μια κλάση μπορεί να υλοποιήσει μία ή παραπάνω προδιαγραφές τύπων (με χρήση της λέξης-κλειδιού `implements`). Μέσω αυτών προσφέρεται μια μορφή πολλαπλής κληρονομικότητας.
 - Οι τύποι δεδομένων στην Java διακρίνονται σε:
 - πρωτογενείς τύπους και
 - τύπους αναφοράς.

Στην πρώτη περίπτωση ανήκουν οι βασικοί τύποι που συναντώνται συνήθως στις σύγχρονες γλώσσες προγραμματισμού, π.χ. `int`. Στη δεύτερη περίπτωση ανήκουν οι τύποι κλάσης, οι τύποι διεπαφής και οι πίνακες.

- Στην περίπτωση που μια μεταβλητή είναι τύπου αναφοράς, δεν δημιουργείται αυτόματα αντικείμενο στο οποίο να αναφέρεται η τιμή της.
- Ένα αντικείμενο δημιουργείται ρητά με χρήση του τελεστή `new`. Η κλήση του επιστρέφει αναφορά, η οποία μπορεί να αποτελέσει την τιμή μεταβλητής αναφοράς τύπου της κλάσης του (ή υπερκλάσης της).
- Αντικείμενα τα οποία δεν είναι εν χρήσει και στα οποία δεν αναφέρεται καμία μεταβλητή αναφοράς είναι υποψήφια για “καταστροφή”. Τότε, ο χώρος που καταλαμβάνουν “επιστρέφεται” για να χρησιμοποιηθεί ξανά εκ νέου, στη συνέχεια. Ο εντοπισμός των αντικειμένων αυτών και η “επιστροφή” του χώρου που καταλαμβάνουν γίνεται από μια διαδικασία που ονομάζεται *διαδικασία περισυλλογής σκουπιδιών*. Αυτή ενεργοποιείται αυτόματα από το σύστημα, οπότε ο προγραμματιστής απαλλάσσεται από το να ασχοληθεί με άχρηστα αντικείμενα.
- Μια λέξη-κλειδί της γλώσσας που χρησιμοποιείται σε διάφορες περιπτώσεις είναι η λέξη `final`. Το κοινό νόημα που μπορεί να αποδοθεί, χαριτολογώντας, στη λέξη για όλες αυτές τις περιπτώσεις είναι “τελεία και παύλα”.

- Όσον αφορά τις διαδικαστικές γλωσσικές δομές της Java και τις εκφράσεις που μπορούν να διατυπωθούν, αυτές έχουν μεγάλη ομοιότητα με τις αντίστοιχες της C και της C++. Δεν μπορεί, όμως, να γίνει υπερφόρτωση τελεστών.
- Όταν γίνεται *αρχικοποίηση στιγμιοτύπων* (αντικειμένων), η διαδικασία που εκτελείται συμπεριλαμβάνει:
 - αρχικοποίηση πεδίων, μέσω δηλώσεων,
 - εκτέλεση μεθόδων αρχικοποίησης,
 - εκτέλεση κώδικα σε block αρχικοποίησης.
- Η λέξη-κλειδί *this* δηλώνει είτε το αντικείμενο είτε τη μέθοδο αρχικοποίησης (της εμβέλειας) της κλάσης του αντικειμένου.
- Η λέξη-κλειδί *super* δηλώνει είτε το αντικείμενο ως αντικείμενο της υπερκλάσης του είτε τη μέθοδο αρχικοποίησης της υπερκλάσης της κλάσης του αντικειμένου.
- Δεν μπορούμε να έχουμε (άμεση) πρόσβαση σε μέλη στιγμιοτύπου τυχαίας υπερκλάσης μιας κλάσης.
- Η χρήση (στατικών) μεθόδων κλάσης γίνεται προσδιορίζοντας είτε την ίδια την κλάση είτε μέσω μεταβλητής αναφοράς της κλάσης αυτής.
- Όταν μια κλάση πρωτοχρησιμοποιείται (φορτώνεται στη JVM), *αρχικοποιείται* ακολουθώντας μια διαδικασία που περιλαμβάνει:
 - αρχικοποίηση στατικών πεδίων,
 - εκτέλεση κώδικα στατικών block αρχικοποίησης.

Πριν αρχικοποιηθεί μια κλάση, αρχικοποιείται η άμεση υπερκλάση της (αναδρομική διαδικασία), όχι όμως οι τύποι διεπαφής που υλοποιεί.

- Τα πακέτα, στα οποία ομαδοποιούνται (σχετιζόμενες) κλάσεις, έχουν ιεραρχική δομή η οποία, όμως, δεν αντανakλά δικαιώματα ορατότητας, αλλά σχετίζεται με την οργάνωση του κώδικα σε καταλόγους αρχείων.
- Οι δηλώσεις εισαγωγής (με χρήση της λέξης-κλειδιού *import*) που εμφανίζονται στα πακέτα απλά επιτρέπουν να αναφερθούμε στα μέλη τους απευθείας, χωρίς τη χρήση του ονόματος του πακέτου.
- Η Java προσφέρει πολλές δυνατότητες τις οποίες δεν αναπτύξαμε ή δεν αναφέραμε και τις οποίες ο αναγνώστης καλείται να μελετήσει ανατρέχοντας στη βιβλιογραφία.

9.15 Ασκήσεις

- 9.15.1** Δοκιμάστε να “τρέξετε” τα παραδείγματα του κεφαλαίου πειραματιζόμενοι με το περιεχόμενό τους σχολιάζοντας ή/και αποσχολιάζοντας κατάλληλες γραμμές.
- 9.15.2** Υλοποιήστε σε Java τα παραδείγματα και τις ασκήσεις από τα Κεφάλαια 5, 6, 7 και 8. Δείτε και σχολιάστε ποια σημεία τους ή ποιες προδιαγραφές τους δεν μπορούν να ικανοποιηθούν.
- 9.15.3** Δίνεται το παρακάτω πρόγραμμα Java:

```

public class MainClass {
    public static void main(String[] args) {
        A a1 = new A(); A a2 = new A(10);
        B b1 = new B(); B b2 = new B(100);
        System.out.println("////////////////////////////////////////");
        a1.print(); a2.print(); b1.print(); b2.print();
        System.out.println("////////////////////////////////////////");
        a1 = b1; a1.print(); b1 = b2; a1.print();
    }
}

class A {
    private int data;
    { System.out.println("A new A was just created"); }

    public A() {
        System.out.println("The A data is " + data); }

    public A(int d) {
        data = d;
        System.out.println("The A data is " + data);
    }

    public void print() {
        System.out.println("The A data is " + data);
    }
}

class B extends A {
    private int data;
    { System.out.println("A new B was just created"); }

    public B() {
        System.out.println("The B data is " + data);
    }

    public B(int d) {
        data = d;
        System.out.println("The B data is " + data);
    }

    public void print() {
        super.print();
        System.out.println("The B data is " + data);
    }
}

```

Επίσης, δίνεται και το παρακάτω πρόγραμμα C++:

```
#include <iostream>
using namespace std;

class A {
    int data;
public:
    A(int d = 0)
    { cout << "A new A was just created" << endl;
      data = d;
      cout << "The A data is " << data << endl; }

    void print()
    { cout << "The A data is " << data << endl; }
};

class B : public A {
    int data;
public:
    B(int d = 0)
    { cout << "A new B was just created" << endl;
      data = d;
      cout << "The B data is " << data << endl; }

    void print()
    { A::print();
      cout << "The B data is " << data << endl; }
};

int main()
{
    A a1; A a2(10); B b1; B b2(100);
    cout << "/////////////////////////////////" << endl;
    a1.print(); a2.print(); b1.print(); b2.print();
    cout << "/////////////////////////////////" << endl;
    a1 = b1; a1.print(); b1 = b2; a1.print();
    return 0;
}
```

Τα προγράμματα δείχνουν παρόμοια, όμως, δεν έχουν τα ίδια αποτελέσματα. Αιτιολογώντας την απάντησή σας, δώστε τα αποτελέσματα του καθενός. Επίσης, εξηγήστε συνοπτικά πού και γιατί τα αποτελέσματα διαφέρουν.

9.15.4 Δίνεται το παρακάτω πρόγραμμα Java:

```
public class MainClass {
    public static void main(String[] args) {
        A a;
        a = new C(); a.print(); a = new B(); a.print();
    }
}
```

```

        a = new A(); a.print();
    }
}

class A {
    private int data1;
    { System.out.println("The initial value of A data is: " +
        data1); }

    A() {
        this(5);
    }

    A(int i) {
        data1 = i;
        System.out.println("The given value of A data is: " + data1);
    }

    void print() {
        System.out.println("The current value of A data is: " +
            data1);
    }
}

class B extends A {
    B() {
        super(10);
    }

    B(int i) {
        super(i);
    }
}

class C extends B {
    private int data2;
    C() {
        this(100);
    }

    C(int i) {
        super(i);
        data2 = i;
        System.out.println("The given value of C data is: " + data2);
    }

    void print() {
        super.print();
    }
}

```

```

        System.out.println("The current value of C data is: " +
                           data2);
    }
}

```

Επίσης, δίνεται και το παρακάτω πρόγραμμα C++:

```

#include<iostream>
using namespace std;

class A {
    int data1;
public:
    A() : data1(5)
    { cout << "The given value of A data is: " << data1 << endl; }

    A(int i) : data1(i)
    { cout << "The given value of A data is: " << data1 << endl; }

    void print()
    { cout << "The current value of A data is: " << data1 << endl; }
};

class B : public A {
public:
    B() : A(10) {}

    B(int i) : A(i) {}
};

class C : public B {
    int data2;
public:
    C() : data2(100), B(100)
    { cout << "The given value of C data is: " << data2 << endl; }

    C(int i) : data2(i), B(i)
    { cout << "The given value of C data is: " << data2 << endl; }

    void print()
    { B::print();
      cout << "The current value of C data is: " << data2 << endl; }
};

int main()
{
    A* pa;
    pa = new C(); pa->print(); pa = new B(); pa->print();
    pa = new A(); pa->print();
}

```

```

    return 0;
}

```

Δώστε τα αποτελέσματα των δύο προγραμμάτων, αιτιολογώντας την απάντησή σας. Τα αποτελέσματα των δύο προγραμμάτων διαφέρουν. Αιτιολογήστε γιατί.

9.15.5 Δίνεται το παρακάτω πρόγραμμα Java:

```

public class Main {
    public static void main(String[] args) {
        System.out.println("Initial A count: ");
        A.print_count();
        System.out.println("Initial B count: " );
        B.print_count();

        A a1 = new A();
        a1.print(); a1.print_count(); a1.scope();
        B b1 = new B();
        b1.print(); b1.print_count(); b1.scope();
        a1 = b1;
        a1.print(); a1.print_count(); a1.scope();
    }
}

class A {
    private int data;
    private static int count;

    static { System.out.println("Initializing A "); }

    public A() {
        this(10);
    }

    public A(int i) {
        data = i;
        System.out.println("An A just constructed with data: " +
                           data);
    }

    public void print() {
        System.out.println("Printing message for A data: " + data);
        count++;
    }

    public static void print_count() {
        System.out.println(count);
    }
}

```



```

    public void scope() {
        System.out.println("Being in A's scope!");
    }
}

class B extends A {
    private int data;
    private static int count;

    static { System.out. println("Initializing B "); }

    public B() {
        this(100,200);
    }

    public B(int i, int j) {
        super(i);
        data = j;
        System.out.println("A B just constructed with data: " + data);
    }

    public void print() {
        super.print();
        System.out.println("Printing message for B data: " + data);
        count++;
    }

    public static void print_count() {
        System.out.println(count);
    }

    public void scope() {
        System.out.println("Being in B's scope!");
    }
}

```

Επίσης, δίνεται και το παρακάτω πρόγραμμα C++:

```

#include <iostream>
using namespace std;

class A {
    int data;
    static int count;
public:
    A(int i = 10) : data(i)
    { cout << "An A just constructed with data: "
      << data << endl; }
}

```

```

    void print()
    {   cout << "Printing message for A data: "
        << data << endl;
        count++; }

    static void print_count()
    {   cout << count << endl; }

    void scope()
    {   cout << "Being in A's scope!" << endl; } };

class B : public A {
    int data;
    static int count;
public:
    B(int i = 100, int j = 200) : A(i), data(j)
    {   cout << "A B just constructed with data: "
        << data << endl; }

    void print()
    {   A::print();
        cout << "Printing message for B data: "
            << data << endl;
        count++; }

    static void print_count()
    {   cout << count << endl; }

    void scope()
    {   cout << "Being in B's scope!" << endl; } };

int A::count = 0;
int B::count = 0;

int main()
{
    cout << "Initial A count: " << endl;
    A::print_count();
    cout << "Initial B count: " << endl;
    B::print_count();

    A* a1 = new A();
    a1->print(); a1->print_count(); a1->scope();

    B* b1 = new B();
    b1->print(); b1->print_count(); b1->scope();

    a1 = b1;
    a1->print(); a1->print_count(); a1->scope();
}

```

```

    return 0;
}

```

Δώστε τα αποτελέσματα των δύο προγραμμάτων, αιτιολογώντας την απάντησή σας. Τα αποτελέσματα των δύο προγραμμάτων διαφέρουν. Αιτιολογήστε γιατί.

9.15.6 Δίνεται το παρακάτω πρόγραμμα Java:

```

public class MainClass {
    public static void main(String[] args) {
        A a1 = new A(); A a2 = new A(10);
        B b1 = new B(); B b2 = new B(100);
        System.out.println("////////////////////////////////////////");
        a1.print(); a2.print();
        b1.print(); b2.print();
        System.out.println("////////////////////////////////////////");
        a1 = b1; a1.print(); } }

class A {
    private int data;
    { System.out.println("A new A was just created"); }

    public A() {
        System.out.println("The A data is " + data);
    }

    public A(int d) {
        data = d;
        System.out.println("The A data is " + data);
    }

    public void print() {
        System.out.println("The A data is " + data);
    }
}

class B extends A {
    private int data;
    { System.out.println("A new B was just created"); }

    public B() {
        System.out.println("The B data is " + data);
    }

    public B(int d) {
        super(d);
        data = d;
        System.out.println("The B data is " + data);
    }
}

```

```

    public void print() {
        super.print();
        System.out.println("The B data is " + data);
    }
}

```

Επίσης, δίνεται και το παρακάτω πρόγραμμα C++:

```

#include <iostream>
using namespace std;

class A {
    int data;
public:
    A(int d = 0)
    { cout << "A new A was just created" << endl;
      data = d;
      cout << "The A data is " << data << endl; }

    virtual void print()
    { cout << "The A data is " << data << endl; } };

class B : public A {
    int data;
public:
    B(int d = 0) : A(d)
    { cout << "A new B was just created" << endl;
      data = d;
      cout << "The B data is " << data << endl; }

    void print()
    { A::print();
      cout << "The B data is " << data << endl; } };

int main()
{
    A a1; A a2(10); B b1; B b2(100);
    cout << "/////////////////////////////////" << endl;
    a1.print(); a2.print(); b1.print(); b2.print();
    cout << "/////////////////////////////////" << endl;
    a1 = b1; a1.print();
    return 0;
}

```

Δώστε τα αποτελέσματα του καθενός, αιτιολογώντας την απάντησή σας. Πού διαφέρουν και γιατί;

9.15.7 Δίνεται το παρακάτω πρόγραμμα C++:

```

#include <iostream>
using namespace std;

```

```

class A {
    int data;
public:
    A(int i = 100) : data(i)
    { cout << "I just constructed an A with: " << data << endl; }

    A(const A& a) : data(a.data)
    { cout << "I just constructed an A by copying with data "
      << data << endl; }

    ~A()
    { cout << "Destructing an A with data " << data << endl; }

    int get_data()
    { return data; }

    void change()
    { data += 100; } };

class FatA {
    A data1;
    A* data2;
    A& data3;
public:
    FatA(A& a) : data1(a), data3(a)
    { data2 = new A(a);
      cout << "I just constructed a FatA with: "
        << data1.get_data() << " " << data2->get_data() << " "
        << data3.get_data() << endl; }

    FatA(FatA& fa) : data1(fa.data1), data3(fa.data3)
    { data2 = new A(*(fa.data2)); }

    ~FatA()
    { cout << "Destructing a FatA " << endl; delete data2; }

    void change()
    { data1.change(); data2->change(); data3.change(); } };

int main()
{
    A a;
    FatA f1(a); FatA f2 = f1;
    f1.change(); f2.change();
    return 0;
}
    
```

Επίσης, δίνεται και το παρακάτω πρόγραμμα Java:

```
public class first {
    public static void main(String[] args) {
        A a = new A(100);
        FatA f1 = new FatA(a); FatA f2 = f1;
        f1.change(); f2.change();
        System.out.println("Printing f1 data"); f1.print();
        System.out.println("Printing f2 data"); f2.print();
    }
}

class A {
    private int data;
    A(int i) {
        data = i;
        System.out.println("I just constructed an A with " + data);
    }

    int get_data() {
        return data;
    }

    void change() {
        data += 100;
    }
}

class FatA {
    private A data1;
    private A data2;
    private A data3;

    FatA(A a) {
        data1 = a; data2 = a; data3 = a;
        System.out.println("I just constructed a FatA with: " +
            data1.get_data() + " " + data2.get_data() + " " +
            data3.get_data());
    }

    void change() {
        data1.change(); data2.change(); data3.change();
    }

    void print() {
        System.out.println(" The FatA data are: " +
            data1.get_data() + " " +
            data2.get_data() + " " +
            data3.get_data());
    }
}
```

```

    }
}

```

Δώστε τα αποτελέσματα της εκτέλεσης των παραπάνω προγραμμάτων, αιτιολογώντας τα. Εντοπίστε τις ουσιαστικές διαφορές στα αποτελέσματα αυτά και αιτιολογήστε τις.

9.15.8 Δίνονται τα παρακάτω προγράμματα C++ και Java:

Πρόγραμμα C++:

```

#include <iostream>
using namespace std;

class A {
    int data;
public:
    A(int i = 10) : data(i)
    {   cout << "An A was just constructed with: "
        << data << endl; }

    void inc()
    {   data++; }

    void print()
    {   cout << data << endl; } };

class B : public A {
    int data;
public:
    B(int i = 100) : A(100), data(i)
    {   cout << "A B was just constructed with: "
        << data << endl; }

    void inc()
    {   data++; }

    void print()
    {   A::print(); cout << data << endl; } };

int main()
{
    B b; b.inc(); b.print();
    A* pab = new B(); pab->inc(); pab->print();
    return 0;
}

```

Πρόγραμμα Java:

```

public class Main {
    public static void main(String[] args) {
        B.inc(); A.print_count(); B.print_count();
    }
}

```

```

        B b = new B(); b.inc(); b.print_count();
        A a = new B(); a.inc(); a.print_count();
    }
}

class A {
    private static int count;
    static { System.out.println("Initializing A "); }

    public static void inc() {
        count++;
    }

    public static void print_count() {
        System.out.println(count);
    }
}

class B extends A {
    private static int count;
    static { System.out.println("Initializing B "); }

    public static void inc() {
        count++;
    }

    public static void print_count() {
        System.out.println(count);
    }
}

```

Δώστε τα αποτελέσματα της εκτέλεσης των παραπάνω προγραμμάτων, αιτιολογώντας τα.

9.16 Βιβλιογραφία

- [1] *Oral History of James Gosling*. <https://www.youtube.com/watch?v=TJ6XHroNewc>. Ανάκτηση: 2023-5-30.
- [2] *James Gosling: Java, JVM, Emacs, and the Early Days of Computing | Lex Fridman Podcast*. https://www.youtube.com/watch?v=IT__Nrr3PNI. Ανάκτηση: 2023-5-30.
- [3] *How The JVM Spec Came To Be*. <https://www.infoq.com/presentations/gosling-jvm-lang-summit-keynote/>. Ανάκτηση: 2023-5-30.
- [4] *Timeline of key Java milestones*. https://www.oracle.com/java/moved-by-java/timeline/?source=ow:o:p:po::RC_WWMK200728P00026:DevLiveJava_PromoOcomJava&intcmp=:ow:o:p:po::RC_WWMK200728P00026:DevLiveJava_PromoOcomJava. Ανάκτηση: 2023-5-30.
- [5] *The Java Tutorials*. <https://docs.oracle.com/javase/tutorial/java/index.html>. Ανάκτηση: 2023-5-30.

- [6] *Java Platform, SE8, API Specification*. <https://docs.oracle.com/javase/8/docs/api/index.html>. Ανάκτηση: 2023-5-30.
- [7] *Java Platform SE8 Documentation*. <https://docs.oracle.com/javase/8/docs/>. Ανάκτηση: 2023-5-30.
- [8] Bruce Eckel. *Thinking In Java*. 4η έκδοση. Pearson, 2006.
- [9] Russel Winder και Graham Roberts. *Developing Java Software*. 3η έκδοση. Wiley, 2006.
- [10] Eric Roberts. *The Art & Science of Java: An Introduction to Computer Science*. Pearson/Addison Wesley, 2008.
- [11] Ken Arnold, James Gosling και David Holmes. *The Java™ Programming Language*. 4η έκδοση. Addison-Wesley Professional, 2005.
- [12] J. Gosling, B. Joy, G. Steele, G. Bracha και A. Buckley. *The Java Language Specification, Java SE 8 Edition*. Report. Oracle, 2015.
- [13] J. Gosling και H. McGilton. *The Java Language Environment —A White Paper*. <https://www.oracle.com/java/technologies/language-environment.html>. Ανάκτηση: 2023-5-30.
- [14] T. Lindholm, F. Yellin, G. Bracha και A. Buckley. *The Java Virtual Machine Specification, Java SE 8 Edition*. Report. Oracle, 2015.
- [15] James Gosling. “Java Intermediate Bytecodes”. Στο: *1995 ACM SIGPLAN Workshop on Intermediate Representations (IR’95)*. Association for Computing Machinery, 1995.

ΑΛΛΕΣ ΓΛΩΣΣΕΣ ΜΕ ΑΝΤΙΚΕΙΜΕΝΟΣΤΡΑΦΗ ΧΑΡΑΚΤΗΡΙΣΤΙΚΑ

Σύνοψη

Στο κεφάλαιο αυτό, θα συνοψίσουμε τα αντικειμενοστραφή χαρακτηριστικά άλλων γλωσσών που χρησιμοποιούνται ευρέως στις μέρες μας. Συγκεκριμένα, θα αναφερθούμε στις γλώσσες C# και Python. Η C# έχει ένα μοντέλο εκτέλεσης που βασίζεται σε ένα σύστημα εκτέλεσης, εκείνο του πλαισίου .NET, κατ' αναλογία με τη JVM για τη γλώσσα Java. Η Python είναι μια γλώσσα που έχει ευρύτατη χρήση. Ο στόχος δεν είναι η αναλυτική παρουσίαση των γλωσσών αυτών, αλλά η εστίαση στη λειτουργικότητα των γλωσσικών δομών τους που σχετίζονται με τον αντικειμενοστραφή προγραμματισμό. Επίσης, θα γίνει μια σύντομη αναφορά στις γλώσσες αντικειμενοστραφούς συναρτησιακού προγραμματισμού Scala και F#. Η Scala μεταφράζεται σε JVM bytecode εντολές. Η F# είναι συναρτησιακή γλώσσα για το πλαίσιο .NET.

Προαπαιτούμενη Γνώση

Για την κατανόηση του κεφαλαίου, ο αναγνώστης απαιτείται να έχει γνώσεις διαδικαστικού προγραμματισμού και να έχει μελετήσει τις έννοιες του αντικειμενοστραφούς προγραμματισμού, καθώς και τις γλώσσες C++ και Java από τα προηγούμενα κεφάλαια του βιβλίου.

10.1 Η Γλώσσα C#

Η C# είναι μια αντικειμενοστραφής γλώσσα, κατά βάση, με ισχυρό σύστημα τύπων, που υποστηρίζει διάφορα είδη προγραμματισμού, όπως διαδικαστικό, συναρτησιακό, προγραμματισμό γενικεύσεων κ.ά. Η γλώσσα έχει αναπτυχθεί από τη Microsoft και οι εφαρμογές που αναπτύσσονται σε αυτήν εκτελούνται στο περιβάλλον της Microsoft .NET.

Από μια επιφανειακή άποψη, η C# έχει κάποια κοινά στοιχεία με τη γλώσσα Java. Η εκτέλεση των προγραμμάτων έγκειται στην κλήση μεθόδων. Όλος ο κώδικας είναι οργανωμένος σε κλάσεις που βρίσκονται μέσα σε μια ιεραρχία ορισμένη από τη γλώσσα. Υπάρχουν τύποι διεπαφής προκειμένου να προσφερθεί μια

λειτουργικότητα πολλαπλής κληρονομικότητας. Δεν δίνεται δυνατότητα ρητής διαχείρισης μνήμης από τον προγραμματιστή και υπάρχει αυτόματη διαδικασία περισυλλογής σκουπιδιών. Τα προγράμματα εκτελούνται μέσω ενός περιβάλλοντος εκτέλεσης και όχι απευθείας από τον υπολογιστή.

Όμως, ο ισχυρισμός των σχεδιαστών της γλώσσας είναι ότι οι ομοιότητες είναι μεγαλύτερες με τη C++ και, σε κάθε περίπτωση, ο στόχος ήταν η βελτίωση αυτής της γλώσσας. Και πράγματι, παρατηρώντας τα χαρακτηριστικά της C#, θα δει κανείς ότι όντως δίνει μια πιο αντικειμενοστραφή μορφή στη C++, μια που εξακολουθεί να διατηρεί πολλά από τα χαρακτηριστικά της. Για παράδειγμα, δεν είναι υποχρεωτικό όλες οι μεταβλητές να είναι μεταβλητές αναφοράς. Σε κάθε περίπτωση, πολλά από τα χαρακτηριστικά της C# δείχνουν οικεία σε έναν προγραμματιστή C++ ή/και έναν προγραμματιστή Java και έτσι, με δεδομένες τις περιγραφές των γλωσσών αυτών που έχουν γίνει στα προηγούμενα, για λόγους έκτασης, η περιγραφή της C# δεν θα δίνει λεπτομέρειες, επεξηγήσεις και παραδείγματα σε θέματα που μπορούν να γίνουν εύκολα αντιληπτά.

Καθότι ο στόχος μας δεν είναι να παρουσιάσουμε τη γλώσσα στο σύνολό της, θα δώσουμε κάποια βασικά στοιχεία. Η εστίασή μας θα είναι σε στοιχεία που αφορούν αντικειμενοστραφή προγραμματισμό.

10.1.1 Λίγα Στοιχεία για την Ιστορία της Γλώσσας C# και το Περιβάλλον Εκτέλεσής της

Η γλώσσα C# δημιουργήθηκε, αναπτύχθηκε και υποστηρίζεται από τη Microsoft. Ο δημιουργός της και βασικός συντελεστής της είναι ο Δανός μηχανικός Anders Hejlsberg. Πριν την απασχόλησή του στη Microsoft, ο Hejlsberg είχε ήδη μια επιτυχημένη πορεία στην ανάπτυξη γλωσσών προγραμματισμού, με πιο γνωστό παράγωγο των προσπαθειών του την Turbo Pascal, στην οποία βασίστηκε και η γλώσσα Delphi. Γενικότερα, είχε ήδη μια επιτυχημένη δραστηριότητα στον χώρο των γλωσσών προγραμματισμού, τόσο λόγω προσωπικής ενασχόλησης, όσο και στο πλαίσιο της απασχόλησής του στην Borland. Το 1996 μετακινήθηκε από την Borland στη Microsoft.

Η C# δημιουργήθηκε το 2000. Το τελικό αποτέλεσμα ήταν προϊόν ομαδικής συστηματικής προσπάθειας στη Microsoft, προκειμένου να αναπτυχθεί μια αντικειμενοστραφής γλώσσα προγραμματισμού, η οποία να συνοδεύει το πλαίσιο .NET. Ο στόχος ήταν να βελτιώνει τη C++ και άλλες γλώσσες από πλευράς διαφόρων κρίσιμων ιδιοτήτων, όπως η παραγωγικότητα κατά την ανάπτυξη εφαρμογών και η ασφάλεια κατά την εκτέλεσή τους. Κατά την ανάπτυξή της, είχαν εμπλακεί επιστήμονες με εμπειρία σε διαφορετικά πεδία γλωσσών προγραμματισμού και υπήρχε διαρκής διάδραση με χρήστες της. Απέκτησε τα πρώτα πρότυπα το 2002 (ECMA) και το 2003 (ISO/IEC). Η γλώσσα διαρκώς εξελίσσεται και βελτιώνεται σε νέες εκδόσεις, που παράγονται με μεγάλη συχνότητα.

Η C# είναι άρρηκτα συνδεδεμένη με το περιβάλλον ανάπτυξης και εκτέλεσης εφαρμογών της Microsoft .NET. Το περιβάλλον .NET αποτελείται από ένα σύνολο βιβλιοθηκών και ένα σύστημα εκτέλεσης. Χρησιμοποιώντας ένα ολοκληρωμένο περιβάλλον ανάπτυξης, π.χ. το Visual Studio ή το .NET CLI — προσοχή, δεν είναι το ίδιο με το ομόηχο πρότυπο —, οι εφαρμογές κωδικοποιούνται σε μια γλώσσα προγραμματισμού, όπως η C#, η F# κ.ά. Ο κώδικάς τους μεταφράζεται σε μια ενδιάμεση γλώσσα, σύμφωνη με το πρότυπο του Common Language Infrastructure (CLI), ώστε, στη συνέχεια, να εκτελεστεί από το σύστημα εκτέλεσης του .NET, το Common Language Runtime (CLR). Η ενδιάμεση αυτή γλώσσα ονομάζεται Common Intermediate Language (CIL) — ή Microsoft Intermediate Language (MSIL). Το πρότυπο του CLI προτείνει μια μορφή για τα εκτελέσιμα προγράμματα και τις προδιαγραφές του περιβάλλοντος για την εκτέλεσή τους. Το CLI βασίζεται στο σύστημα τύπων Common Type System (CTS), το οποίο μοιράζονται όλες οι γλώσσες που στοχεύουν σε εκτέλεση από ένα περιβάλλον που υπακούει στο CLI. Το CTS επιτρέπει συνεργασία μεταξύ γλωσσών, ασφάλεια τύπων και ταχύτητα κατά την εκτέλεση. Αυτά επιτυγχάνονται με τον καθορισμό κανόνων για τον ορισμό τύπων και διαχείρισή τους από το CLR, καθώς και την παροχή μιας βιβλιοθήκης με τους συνήθεις πρωτογενείς τύπους. Επίσης, το CTS καθορίζει κανόνες στους οποίους υπακούουν τα συστήματα τύπων διαφόρων γλωσσών, προκειμένου να γίνεται εφικτή η αλληλεπίδραση μεταξύ αντικειμένων διαφορετικών γλωσσών. Το όλο πλαίσιο βασίζεται σε ένα αντικειμενοστραφές μοντέλο πάνω στο οποίο υλοποιούνται πολλές γλώσσες προγραμματισμού.

Κατά την εκτέλεση του ενδιάμεσου παράγωγου ενός προγράμματος μεταφρασμένου σε CLI, το σύστημα

εκτέλεσης CLR επιτελεί “επιτόπου μεταγλώττιση” (Just-In-Time compiling) του κώδικά του, προκειμένου το παράγωγο της διαδικασίας να εκτελεστεί πραγματικά από τον επεξεργαστή. Στο σημείο αυτό, είναι πολύτιμη η παρέμβαση του CLR, μια που είναι σε θέση να γνωρίζει τι κάνει ο κώδικας και να κάνει τις απαραίτητες παρεμβάσεις, π.χ. να εκτελέσει τη διαδικασία περισυλλογής σκουπιδιών. Επίσης, το CLR κάνει διαθέσιμες, σε υψηλό επίπεδο, έννοιες συστήματος, όπως νήματα και εξαιρέσεις. Υλοποιήσεις του CLR είναι διαθέσιμες για διάφορα συστήματα, κατά συνέπεια μια εφαρμογή που έχει αναπτυχθεί σε .NET, λογικά, θα πρέπει να μπορεί να μεταφερθεί σε οποιοδήποτε από αυτά.

10.1.2 Η Οργάνωση των Προγραμμάτων C#

Η έννοια της εμβέλειας είναι υψίστης σημασίας για τη C#. Όλα τα ονόματα που χρησιμοποιούνται σε ένα πρόγραμμα ανήκουν σε μια εμβέλεια. Βασικές έννοιες όσον αφορά την οργάνωση ενός προγράμματος είναι οι χώροι ονομάτων, οι τύποι, τα μέλη των τύπων και τα *assemblies*. Τα τελευταία αποτελούν το αποτέλεσμα μεταγλώττισης προγραμμάτων C# και είναι είτε εκτελέσιμα εφαρμογών (με κατάληξη *.exe*), είτε βιβλιοθηκών (οπότε έχουν κατάληξη *.dll*). Παραδείγματα τύπων είναι οι κλάσεις, οι τύποι διεπαφής και οι τύποι δομής. Παραδείγματα μελών τους είναι τα πεδία, οι μέθοδοι, οι ιδιότητες (*properties*) και τα συμβάντα (*events*).

Ένα πρόγραμμα C# γράφεται σε ένα σύνολο αρχείων και περιέχει ένα σύνολο από χώρους ονομάτων. Οι χώροι ονομάτων ορίζουν εμβέλειες με όνομα. Σε έναν χώρο ονομάτων, ορίζονται κλάσεις, τύποι διεπαφής, τύποι δομής — χρησιμοποιώντας τη γλωσσική δομή *struct* —, *απαριθμήσεις* και *delegates*. Επίσης, σε κάποιο βαθμό, δίνεται η δυνατότητα ορισμού εμφωλευμένων χώρων ονομάτων. Τα ονόματα ενός χώρου ονομάτων γίνονται προσβάσιμα χρησιμοποιώντας το όνομα του χώρου συνοδευόμενο από τελεία, .. Μπορούν να γίνουν και άμεσα προσβάσιμα, αν προηγηθεί της χρήσης τους η οδηγία *using<Namespace_name>*. Δεν υπάρχουν περιορισμοί πρόσβασης στο όνομα ενός χώρου ονομάτων: είναι καθολικά προσβάσιμο. Επίσης, δεν υπάρχει μηχανισμός περιορισμού πρόσβασης ενός ονόματος μέσα σε έναν χώρο ονομάτων και μόνο.

Όταν δεν έχει οριστεί ρητά κάποιος χώρος ονομάτων, η γλώσσα θεωρεί αυτόματα έναν ανώνυμο χώρο ονομάτων, του οποίου τα ονόματα είναι καθολικά προσβάσιμα από όλο το πρόγραμμα. Επίσης, η γλώσσα προσφέρει έναν χώρο ονομάτων, τον *System*, μέσα στον οποίο ορίζονται βασικές κλάσεις προς χρήση κατά την ανάπτυξη κώδικα. Μια τέτοια κλάση είναι η κλάση *Console*, στην οποία ορίζονται τα ρεύματα για είσοδο/έξοδο από τις καθιερωμένες μονάδες.

Να σημειώσουμε ότι δίνονται διάφορες δυνατότητες από τη C# σχετικά με τον καθορισμό και τη χρήση των χώρων ονομάτων.

10.1.2.1 Επίπεδα Ορατότητας

Η διάκριση των επιπέδων ορατότητας στη C# δεν αναφέρεται στους χώρους ονομάτων, αλλά στα ονόματα τύπων ή των μελών τους. Ο προσδιορισμός ορατότητας των μελών πρέπει να δηλώνεται απευθείας στο μέλος. Η C# προσφέρει 6 επίπεδα ορατότητας για τα ονόματα τύπων ενός προγράμματος ή μελών τους:

Δημόσια: Το όνομα είναι προσβάσιμο από όλον τον κώδικα του ίδιου *assembly* ή άλλου στο οποίο υπάρχει αναφορά.

Ιδιωτικά: Το όνομα είναι προσβάσιμο μόνο από την κλάση που το ορίζει.

Προστατευόμενα: Το όνομα είναι προσβάσιμο μόνο από την κλάση που το ορίζει και τις υποκλάσεις της.

Εσωτερικά (internal): Το όνομα είναι προσβάσιμο από όλον τον κώδικα του ίδιου *assembly*, άλλα κανενός άλλου.

Προστατευόμενα εσωτερικά (protected internal): Το όνομα είναι προσβάσιμο από όλον τον κώδικα του ίδιου *assembly* και κώδικα υποκλάσεων της κλάσης, ακόμα και από άλλο *assembly*.

Ιδιωτικά προστατευόμενα (private protected): Το όνομα είναι προσβάσιμο μόνο από την κλάση που το ορίζει και τις υποκλάσεις της που είναι ορισμένες στο ίδιο assembly.

10.1.2.2 Το Αρχικό Σημείο Εκτέλεσης ενός Προγράμματος

Η εκτέλεση ενός προγράμματος C# ξεκινά με την κλήση μιας μεθόδου `Main`. Από την έκδοση 9 της γλώσσας και ύστερα, αυτή δεν είναι υποχρεωτικό να οριστεί ρητά από τον προγραμματιστή: μέσα σε ένα αρχείο, ο προγραμματιστής γράφει τον κώδικα που θα αντιστοιχούσε στο σώμα της. Στην περίπτωση που οριστεί μέθοδος `Main`, αυτή πρέπει να χαρακτηριστεί ως `static` στην κλάση (ή `struct`) που θα έχει οριστεί. Επίσης, μπορεί ή όχι να παίρνει παράμετρο τύπου `string[]`, που αντιστοιχεί στα ορίσματα που δίνονται στη γραμμή εντολής. Η υπογραφή της (signature) — το πρωτότυπο της χωρίς τύπο επιστροφής — μπορεί να έχει διάφορες μορφές. Η μέθοδος `Main` δεν είναι υποχρεωτικό να είναι μοναδικά ορισμένη σε ένα πρόγραμμα.

Στα παραδείγματά μας θα ορίζουμε ρητά τη συνάρτηση `Main`.

10.1.3 Δεδομένα και Διαδικασίες

Σε ένα πρόγραμμα C#, έχουμε δεδομένα πάνω στα οποία εφαρμόζονται κάποιες διαδικασίες ή συμμετέχουν σε αυτές. Κάθε τι σε ένα πρόγραμμα C# πρέπει να έχει κάποιον τύπο. Κατά συνέπεια, κάθε μεταβλητή (ή σταθερά) ανήκει σε κάποιον τύπο. Σε κάποιες περιπτώσεις μεταβλητών (τοπικές σε μέθοδο), ο τύπος τους μπορεί να μην δηλωθεί από τον προγραμματιστή (να χρησιμοποιηθεί η λέξη-κλειδί `var`), αλλά να εξαχθεί από τον μεταγλωττιστή. Μπορούμε να δηλώσουμε μια μεταβλητή ως σταθερά με χρήση του προσδιορισμού `const`, αλλά μόνο εάν πρόκειται για μεταβλητές ενσωματωμένων τύπων της γλώσσας. Προκειμένου να προσομοιώσουμε τη συμπεριφορά σταθεράς για γενικότερα πεδία της κλάσης, αυτά πρέπει να έχουν προσδιοριστεί ως `readonly`, ώστε να μπορούν να πάρουν τιμές μόνο κατά την αρχικοποίηση.

Οι τύποι που προσφέρει η C# ανήκουν σε δύο κατηγορίες:

- τους τύπους αναφοράς και
- τους τύπους τιμής.

Οι μεταβλητές που ανήκουν σε τύπους τιμής εμπεριέχουν την τιμή τους. Οι μεταβλητές που ανήκουν σε τύπους αναφοράς αποθηκεύουν αναφορά στην τιμή τους, η οποία είναι ένα αντικείμενο του τύπου τους. Παρότι παραπάνω από μία μεταβλητές αναφοράς μπορεί να αναφέρονται στο ίδιο αντικείμενο, αυτό δεν είναι δυνατό με τις μεταβλητές τιμής. Κάθε μια από αυτές έχει στον δικό της χώρο το αντικείμενο της τιμής της και το οποίο φυλάσσεται στη στοίβα. Τα αντικείμενα στα οποία αναφέρονται οι μεταβλητές αναφοράς φυλάσσονται στον σωρό.

Τύποι αναφοράς είναι οι κλάσεις, οι τύποι διεπαφής, οι πίνακες, οι συμβολοσειρές και οι τύποι `delegates`. Οι υπόλοιποι τύποι είναι τύποι τιμής. Αυτοί είναι οι απλοί τύποι της γλώσσας, οι απαριθμήσεις, οι τύποι δομής, τύποι πλειάδων κ.ά. Ως απλοί τύποι, προσφέρονται οι συνηθισμένοι τύποι ακεραίων, πραγματικών, λογικών μεταβλητών κ.ά. Το ενδιαφέρον είναι ότι για κάθε τύπο τιμής που προσφέρεται από τη γλώσσα, προσφέρεται και ένας ακόμα που επιδέχεται της τιμής `null`. Ο τύπος αυτός ονοματίζεται με την προσθήκη του συμβόλου `?`. Για παράδειγμα, για τον τύπο `int`, ο αντίστοιχος επεκτεταμένος τύπος είναι `int?`.

Όλο το σύστημα των τύπων της γλώσσας ανήκει σε μια ιεραρχία με ρίζα την κλάση `object`. Το σύστημα αυτό ονομάζεται *Common Type System*, (CTS). Μπορούμε να μεταχειριστούμε μια τιμή μεταβλητής τύπου τιμής σαν τιμή μεταβλητής τύπου αναφοράς, εφαρμόζοντας στη μεταβλητή τη διαδικασία του *boxing*. Τότε, δημιουργείται στον σωρό ένα αντικείμενο με τιμή την τιμή της μεταβλητής. Προκειμένου να κάνουμε την αντίστροφη διαδικασία, κάνουμε *unboxing*. Τότε, μια τιμή μεταβλητής τύπου αναφοράς δίνεται ως τιμή μεταβλητής τύπου τιμής (αρκεί η πρώτη να έχει προκύψει από *boxing*). Για παράδειγμα, για μια μεταβλητή, `i`, τύπου `int` μπορούμε να κάνουμε:

```
int i = 5;
```

```
object o = i;    // boxing
i = (int)o;      // unboxing
```

Η διαδικασία του boxing έγινε αυτόματα κατά τον ορισμό της μεταβλητής `o` του τύπου `object`, ο οποίος είναι τύπου αναφοράς.

Για τις μεταβλητές τύπου αναφοράς, η ισότητα μεταξύ τους δεν προκύπτει από την ισότητα των αντικειμένων στα οποία αναφέρονται. Στην περίπτωση που για έναν τύπο αναφοράς θα θέλαμε να ισχύει το αντίθετο και, γενικότερα, για τις μεταβλητές του θα θέλαμε να τις χειριζόμαστε κατά τιμή, θα πρέπει τον τύπο αυτό να τον ορίσουμε ως *τύπο εγγραφής* (record). Ο τύπος αυτός προσφέρεται από την έκδοση 9 της C# και ύστερα, προκειμένου να δίνεται η δυνατότητα ορισμού τύπων αναφοράς οι οποίοι, όμως, να συμπεριφέρονται κατά τιμή. Μάλιστα, εκτός αν προσδιοριστεί το αντίθετο, οι τιμές των τύπων αυτών είναι αμετάβλητες. Στην έκδοση 10 εντάχθηκε και ο τύπος `record struct`, προκειμένου να οριστούν τύποι δομής με τη συμπεριφορά των τύπων εγγραφής.

Σε περιπτώσεις που χρειάζεται μετατροπή τύπων, π.χ. ανάθεσης ή περάσματος παραμέτρων, αν οι εμπλεκόμενοι τύποι είναι διαφορετικοί, γίνεται αυτόματη μετατροπή από τη C#, όταν δεν υπάρχει απώλεια πληροφορίας. Στην αντίθετη περίπτωση, η μετατροπή πρέπει να ζητηθεί ρητά από τον προγραμματιστή μέσω `cast`.

10.1.3.1 Δυο Λόγια για τις Διαδικασίες

Όπως συνήθως, στις γλώσσες διαδικαστικού προγραμματισμού, οι διαδικασίες που πρόκειται να διεκπεραιωθούν αποτελούνται από ένα σύνολο ενεργειών που εκφράζονται μέσω εντολών και υπολογισμούς τιμών εκφράσεων. Το ρεπερτόριο των εντολών της C#, καθώς και το ρεπερτόριο δυνατοτήτων για τη διατύπωση εκφράσεων, είναι παρεμφερή με εκείνα των γλωσσών C/C++. Ακόμα, δίνεται και η δυνατότητα υπερφόρτωσης των τελεστών.

Θα μείνουμε όμως στις συναρτήσεις και τις μεθόδους, μια που υπάρχουν κάποια σημεία που πρέπει να επισημάνουμε. Ένα θέμα αφορά το πέρασμα παραμέτρων. Αυτό γίνεται κατά τιμή. Για τις μεταβλητές τύπου αναφοράς, βέβαια, αυτό ισχύει για το περιεχόμενο της μεταβλητής και όχι για την τιμή του αντικειμένου στο οποίο αναφέρεται. Μπορούμε όμως να επιτύχουμε πέρασμα παραμέτρων κατά αναφορά, προσημαίνοντας την παράμετρο (και το όρισμα) με τη λέξη-κλειδί `ref`. Επιπλέον, ένα όρισμα μπορεί να έχει απλά τη φύση μεταβλητής εξόδου προσημαίνοντάς το με τη λέξη-κλειδί `out`. Σε αυτήν την περίπτωση, πρόκειται για ένα όρισμα που περνά κατά αναφορά, απλά δεν είναι υποχρεωτικό να έχει τιμή. Κατά την κλήση μιας μεθόδου, μπορούμε να προσδιορίσουμε τα ορίσματα με το όνομά τους και να μην βασιζόμαστε στη σειρά που εμφανίζονται στη δήλωσή τους. Να πούμε ακόμα ότι υποστηρίζεται η δυνατότητα απόδοσης default τιμών στα ορίσματα. Επίσης, υποστηρίζεται η υπερφόρτωση των ορισμών των μεθόδων. Μπορούν να οριστούν και εμφωλευμένες συναρτήσεις σε μεθόδους, καθώς και λάμδα-εκφράσεις ή ανώνυμες μέθοδοι. Για τον ορισμό των λάμδα-εκφράσεων, χρησιμοποιείται ο τελεστής `=>`. Ο τελεστής αυτός χρησιμοποιείται και στην περίπτωση που θέλουμε να ορίσουμε μικρές συναρτήσεις/μεθόδους, που απλά επιστρέφουν την τιμή (που προκύπτει από τον υπολογισμό) μιας έκφρασης. Τέλος, μπορούμε να παραλείψουμε τον ορισμό μιας μεθόδου, δηλώνοντάς την ως *αφηρημένη*.

Οι μέθοδοι στη C# δεν είναι αυτόματα εικονικές. Στην περίπτωση που επιθυμούμε μια μέθοδος να είναι εικονική, πρέπει να την προσδιορίσουμε ως `virtual`. Σε αυτήν την περίπτωση, παραμένει εικονική για όλη την κατώτερη ιεραρχία των κλάσεων. Αν σε μια κατώτερη κλάση πρόκειται να την επαναορίσουμε, πρέπει η νέα μέθοδος να δηλώνεται με χρήση της λέξης-κλειδιού `override`. Αν μια μέθοδος δεν είναι εικονική, κάθε φορά που πρόκειται να επαναοριστεί στην κατώτερη ιεραρχία, πρέπει να δηλώνεται με χρήση της λέξης-κλειδιού `new`. Αν δεν επιθυμούμε μια εικονική μέθοδος να επαναοριστεί σε κλάσεις της κατώτερης ιεραρχίας, πρέπει να δηλωθεί ως `sealed`.

Προκειμένου να επιτευχθεί λειτουργικότητα που στις C/C++ θα επιτυγχάναμε μέσω δεικτών σε συναρτήσεις, στη C# μπορούμε να ορίσουμε τύπους `delegate`. Αυτοί αναπαριστούν αναφορές σε μεθόδους με συγκεκριμένη λίστα παραμέτρων και τύπο επιστροφής.

Στις μεθόδους θα επανέλθουμε.

10.1.4 Κλάσεις και Άλλοι Τύποι

Ένα πρόγραμμα C# αποτελείται από ορισμούς τύπων μέσω δηλώσεων τύπων. Μια δήλωση τύπου καθορίζει το όνομά του και τα μέλη του. Τα είδη τύπων που μπορεί να ορίσει ο προγραμματιστής είναι: κλάσεις, τύποι δομής, τύποι διεπαφής, απαριθμήσεις, *delegates* και τύποι πλειάδων. Εναλλακτικά, για τις κλάσεις και τους τύπους δομής, μπορεί να ορίσει τύπους εγγραφής. Στην περίπτωση που χρειάζεται να οριστεί ένας “ελαφρύς” τύπος για προσωρινή χρήση εντός μεθόδων, μπορούμε να ορίσουμε ανώνυμους τύπους. Επίσης, χρησιμοποιώντας παραμέτρους, μπορούμε να ορίσουμε γενικευμένους τύπους.

Στη συνέχεια, θα επικεντρωθούμε στις κλάσεις και θα δώσουμε και κάποια στοιχεία για τους τύπους δομής και τους τύπους διεπαφής.

Ο ορισμός μιας κλάσης δηλώνεται με τη λέξη-κλειδί `class`, π.χ. `class MyNewClass { ... }`. Η δημιουργία αντικειμένων μιας κλάσης γίνεται με χρήση του τελεστή `new`, π.χ. `new MyClass()` και συνήθως συνοδεύεται από ανάθεση σε μια μεταβλητή αναφοράς του ίδιου (ή ανώτερου στην ιεραρχία κλάσεων) τύπου, π.χ. `MyClass varMyClass = new MyClass();`.

Μια κλάση μπορεί να δηλωθεί ως *αφηρημένη*, οπότε απαγορεύεται να δημιουργηθούν αντικείμενά της. Μπορούμε να απαγορεύσουμε να οριστούν υποκλάσεις μιας κλάσης, προσδιορίζοντάς την ως *κλειδωμένη* (*sealed*). Κλάσεις μπορούν να οριστούν και ως *στατικές*, δηλώνοντάς τις ως `static`. Στην περίπτωση αυτή, δεν μπορούν να δημιουργηθούν αντικείμενά τους. Παρότι και στις αφηρημένες κλάσεις δεν μπορούμε να ορίσουμε (απευθείας) στιγμιότυπα, η φύση και η λειτουργικότητα των στατικών κλάσεων είναι τελείως διαφορετικές. Οι στατικές κλάσεις πρέπει να περιέχουν μόνο στατικά μέλη και απαγορεύεται να έχουν υποκλάσεις — είναι κλειδωμένες.

Από πλευράς ορατότητας, οι εξωτερικές κλάσεις και οι τύποι δομής μπορούν να δηλωθούν ως `public` ή `internal`. Αν δεν υπάρχει προσδιορισμός ορατότητας, θεωρούνται *εσωτερικές*. Αν δεν υπάρχει προσδιορισμός ορατότητας για τα μέλη των κλάσεων, αυτά θεωρούνται *ιδιωτικά*.

Τα μέλη μιας κλάσης μπορεί να είναι είτε *στατικά μέλη* είτε *μέλη στιγμιοτύπου*. Ως συνήθως, τα στατικά μέλη είναι συνυφασμένα με την κλάση, ενώ τα μέλη στιγμιοτύπου με συγκεκριμένο αντικείμενο της κλάσης.

Η C# διακρίνει τα μέλη μιας κλάσης στις παρακάτω κατηγορίες:

Σταθερές: Σταθερές συνυφασμένες με την κλάση.

Πεδία: Μεταβλητές στιγμιοτύπου ή στατικές.

Μέθοδοι: Συναρτήσεις στιγμιοτύπου ή στατικές.

Ιδιότητες (Properties): Ενέργειες που αποσκοπούν στην ανάκτηση και αλλαγή τιμής ιδιοτήτων των στιγμιοτύπων της κλάσης ή της ίδιας της κλάσης (αν έχουν δηλωθεί στατικές).

Δεικτοδότες (Indexers): Ενέργειες που επιτυγχάνουν δεικτοδότηση ενός αντικειμένου σαν να ήταν πίνακας.

Συμβάντα (Events): Ειδοποιήσεις που μπορούν να παραχθούν από την κλάση.

Τελεστές: Μετατροπές και τελεστές που υποστηρίζονται για την κλάση.

Μέθοδοι αρχικοποίησης: Διαδικασίες για την αρχικοποίηση αντικειμένων της κλάσης ή την αρχικοποίηση της ίδιας της κλάσης (με τη δήλωση `static`).

Μέθοδοι καταστροφής (Finalizers): Ενέργειες που πρέπει να γίνουν πριν καταστραφεί ένα αντικείμενο της κλάσης.

Τύποι: Νέοι τύποι εμφωλευμένοι στην κλάση.

Σχετικά με τα παραπάνω, έχουμε να αναφέρουμε τα εξής:

- Ένα πεδίο που έχει δηλωθεί ως σταθερά μέσω `const` είναι, ουσιαστικά, στατικό.
- Τα στατικά μέλη μπορούν να προσπελαστούν χρησιμοποιώντας μόνο το όνομα της κλάσης.
- Στατική μέθοδος αρχικοποίησης μπορούμε να ορίσουμε και σε μη-στατικές κλάσεις, αν κρίνουμε ότι στατικά πεδία τους χρειάζονται μια ιδιαίτερη διαδικασία αρχικοποίησης.
- Οι στατικές μέθοδοι αρχικοποίησης:
 - Είναι μοναδικές σε μια κλάση.
 - Δεν έχουν παραμέτρους.
 - Δεν επιδέχονται προσδιορισμό ορατότητας.
 - Καλούνται αυτόματα πριν τη χρήση στατικών πεδίων της κλάσης ή τη δημιουργία στιγμιότυπου της και μετά από τυχόν αρχικοποιήσεις στατικών πεδίων που περιγράφονται στον ορισμό της κλάσης. Οι τελευταίες γίνονται με τη σειρά που γράφονται οι δηλώσεις των πεδίων στον ορισμό της κλάσης.

Καθότι η στατική μέθοδος αρχικοποίησης επιβάλλει επιπλέον κόστος στην εκτέλεση, καλό είναι να ορίζεται μόνο σε περιπτώσεις που οι αρχικοποιήσεις των στατικών μελών κατά τη δήλωσή τους στον ορισμό της κλάσης δεν επαρκούν.

- Αν δεν έχει δηλωθεί αρχικοποίηση ενός στατικού πεδίου, αυτό αποκτά αυτόματα τις default τιμές του τύπου του, κατά περίπτωση.
- Προκειμένου να αναφερθούμε στο αντικείμενο στο οποίο εφαρμόζεται μια μέθοδος, μπορούμε να χρησιμοποιήσουμε τη λέξη-κλειδί `this`.
- Για τις μεθόδους αρχικοποίησης στιγμιότυπων μπορούμε να πούμε τα εξής:
 - Δεν υπάρχει αυτόματη διαδικασία αρχικοποίησης μέσω αντιγραφής από τη `C#`.
 - Μπορούν να οριστούν μέθοδοι αρχικοποίησης μέσω αντιγραφής, ορίζοντας μέθοδο αρχικοποίησης στιγμιότυπου με πρωτότυπο `X(X)` για μια κλάση με όνομα `X`.
 - Αν επιθυμούμε να ορίσουμε διαδικασίες αρχικοποίησης μέσω αντιγραφής και να δημιουργούμε και το αντικείμενο ταυτόχρονα από την ίδια μέθοδο, τότε μπορούμε να υλοποιήσουμε τον τύπο διεπαφής `ICloneable`, ορίζοντας τη μέθοδο `Clone`. Στην περίπτωση που θέλουμε να κάνουμε ρηχή αντιγραφή (shallow copy) κατά την αρχικοποίηση του νέου αντικειμένου, μπορούμε να χρησιμοποιήσουμε τη μέθοδο `Object.MemberwiseClone`.
 - Με χρήση της λέξης-κλειδιού `this` στον ορισμό μιας μεθόδου αρχικοποίησης, `X(...):this(...){...}`, μπορούμε να παραπέμψουμε στην εκτέλεση της μεθόδου αρχικοποίησης που ταιριάζει με τα ορίσματα που ακολουθούν το `this` πριν την εκτέλεση του σώματος της μεθόδου.
- Δίνεται η δυνατότητα να γίνει αρχικοποίηση των πεδίων ενός αντικειμένου κατά τη δημιουργία του, χωρίς αυτό να γίνεται από τη μέθοδο αρχικοποίησης που καλείται. Αυτό επιτυγχάνεται με χρήση *αρχικοποιητών αντικειμένων* (object initializers). Για παράδειγμα, αν η κλάση `MyClass` έχει πεδία (ή ιδιότητες) `public int Prop1;` και `public int Prop2;`, μπορούμε να δημιουργήσουμε και να αρχικοποιήσουμε ένα αντικείμενό της με τον εξής τρόπο:

```
MyClass varMyClass = new MyClass { Prop1 = 10, Prop2 = 100 };
```

Μπορεί, ακόμα, να γίνει συνδυασμός αρχικοποίησης μέλους με χρήση μεθόδου αρχικοποίησης. Για παράδειγμα:

```
MyClass varMyClass = new MyClass(10) { Prop2 = 100 };
```

Δεδομένου ότι τα πεδία μιας κλάσης μπορεί να είναι τύπου κλάσης, η σύνθεση κλάσεων επιτυγχάνεται άμεσα και δεν έχει κάποια ιδιαιτερότητα στην αντιμετώπισή της. Η αρχικοποίηση των μελών τα οποία αντιστοιχούν σε μια *has a* σχέση υπόκειται στους γενικότερους κανόνες περί αρχικοποίησης μελών. Το ίδιο και για την πρόσβαση στα μέλη αυτά: ακολουθεί τους γενικότερους κανόνες ορατότητας και πρόσβασης.

Ακόμα, να αναφέρουμε τη δυνατότητα μερικού ορισμού κλάσεων ή/και μεθόδων που μπορούν να επεκταθούν σε άλλο assembly ή/και κλάση. Επίσης, δίνεται η δυνατότητα ορισμού *ανώνυμων τύπων*, προκειμένου να οριστεί κάποιος “ελαφρύς” τύπος που πρόκειται να χρησιμοποιηθεί τοπικά. Τέλος, να πούμε ότι η δυνατότητα *ταιριάσματος προτύπων* (pattern matching) συνδυάζεται με την έννοια των αντικειμένων, προσφέροντας πολύτιμη λειτουργικότητα, π.χ. ταίριασμα ως προς την κατάσταση ενός αντικειμένου.

10.1.4.1 Μέλη ιδιότητες

Παρότι όλες οι κατηγορίες μελών παρουσιάζουν ενδιαφέρον, θα μείνουμε λίγο στις ιδιότητες, μια που αποτελούν ένα πρακτικό και συνηθισμένο εργαλείο χειρισμού αντικειμένων. Ορίζοντας μια ιδιότητα σε μια κλάση, δίνεται η δυνατότητα να χειριστούμε αφαιρετικά ένα ιδιωτικό πεδίο της κλάσης, το οποίο αποκαλείται *υποστηρικτικό πεδίο* (backing field) της ιδιότητας. Ο χειρισμός του γίνεται με ορισμό των μεθόδων *get* και *set* της ιδιότητας, οι οποίες στην ορολογία της C#, αποκαλούνται *accessors*. Ο accessor *get* σχετίζεται με την ανάκτηση τιμής του πεδίου και ο accessor *set* με ανάθεση ή αλλαγή της τιμής του. Από την έκδοση 9 της C# έχει ενταχθεί και ο accessor *init*, προκειμένου να αποδοθεί αρχική τιμή στο πεδίο σε μια διαδικασία αρχικοποίησης (σε αυτήν την περίπτωση, συμπεριφέρεται σαν να ήταν ορισμένος ο accessor *set*). Στην περίπτωση των *accessors set* και *init*, προκειμένου να αναφερθούμε στην τιμή που θα χρησιμοποιηθεί κατά την ανάθεση, χρησιμοποιούμε τη λέξη-κλειδί *value*.

Κατά τον ορισμό μιας ιδιότητας, δηλώνουμε ποιοι *accessors* τη συνοδεύουν και, αν θέλουμε, δίνουμε και ορισμούς γι’ αυτούς. Αν κατά τον ορισμό ενός *accessor*, απλά έχουμε τον υπολογισμό μιας έκφρασης, μπορούμε να τον ορίσουμε με χρήση του τελεστή *=>*. Ένα πολύ απλό παράδειγμα ορισμού ιδιότητας είναι:

```
public string FName
{
    get => _fname;
    set => _fname = value;
}
```

Στο παραπάνω, ορίζεται η ιδιότητα *FName* για το υποστηρικτικό πεδίο *_fname* που πρέπει να έχει δηλωθεί στην κλάση. Η ιδιότητα αυτή δίνει ορισμούς για τον accessor *get* και για τον accessor *set*.

Στο παράδειγμα, το υποστηρικτικό πεδίο είναι ένα πεδίο που έχει δηλωθεί μέσα στην κλάση και αναφέρεται ρητά στον ορισμό της ιδιότητας που του αντιστοιχεί. Επιτρέπεται, όμως, και να μην εμφανίζεται καθόλου στον ορισμό της κλάσης. Τότε, απλά δηλώνουμε την ιδιότητα και τους *accessors* που τη συνοδεύουν και ο μεταγλωττιστής δημιουργεί το πεδίο και τους *accessors*. Στην περίπτωση αυτή, μιλάμε για *αυτο-υλοποιούμενες ιδιότητες* (auto-implemented properties) και τις χρησιμοποιούμε όταν δεν υπάρχει κάτι ιδιαίτερο στον κώδικα των *accessors*, πέρα από την ανάκτηση ή/και ανάθεση τιμής κατά περίπτωση. Στο παραπάνω παράδειγμα, θα μπορούσαμε να μην έχουμε ορίσει πεδίο *_fname* στην κλάση και να είχαμε ορίσει την *FName* ως αυτο-υλοποιούμενη ιδιότητα, δηλαδή:

```
public string FName
{ get; set; }
```

Μια τελευταία παρατήρηση. Δεν είναι απαραίτητο να έχουν όλοι οι accessors το ίδιο επίπεδο ορατότητας, ούτε να είναι όλοι δηλωμένοι. Αν έχει δηλωθεί μόνο get accessor, πρακτικά στο πεδίο επιτρέπεται μόνο ανάγνωση. Αν έχουν δηλωθεί και ο get και ο set, επιτρέπεται και η ανάκτηση και η αλλαγή τιμής. Αν είναι δηλωμένος μόνο ο init, πρακτικά πρόκειται για ένα αμετάβλητο πεδίο.

10.1.4.2 Παραδείγματα Ορισμού Κλάσης

Στη συνέχεια, θα δώσουμε κάποια παραδείγματα ορισμού κλάσης στη C#, εμπνευσμένα από τον τύπο του “Φοιτητή” της Παραγράφου 8.2. Όπως είπαμε, για λόγους έκτασης, δεν θα δώσουμε λεπτομερείς επεξηγήσεις, παρά μόνο επισημάνσεις, όπου κρίνεται απαραίτητο.

Ένα πρώτο παράδειγμα ορισμού κλάσης

```
using System;
// using System.Linq; // if Sum() is used

public class Test
{
    public static void Main()
    {
        Console.WriteLine("The number of courses is: " +
                           Student.TOTAL_COURSES);
        // class const access
        Console.WriteLine("The number of created students so far is: " +
                           Student.get_no_students());
        // static method call

        Console.WriteLine("Testing a Single Class!");

        Student s1 = new Student(100); // instance creation
        Student s2 = new Student(200); // --- >> ---
        Student s3 = new Student(300); // --- >> ---

        s1.record_grade(6); // instance method call
        s1.record_grade(4); // --- >> ---
        s1.record_grade(10); // --- >> ---
        s1.record_grade(8); // --- >> ---
        s1.record_grade(9); // --- >> ---

        if (s1.graduate()) // --- >> ---
        {
            Console.WriteLine("YES!");
        }
        else
        {
            Console.WriteLine("NOT YET!");
        }
    }
};
```

```

        Console.WriteLine("The GPA of Student with no: " +
            s1.get_no_student() + // instance method call
            " is: " + s1.evaluate_GPA());
            // instance method call
        Console.WriteLine(Student.get_no_students());
            // static method call
        Console.WriteLine(Student.get_highest_GPA());
            // static method call
    }
}

class Student
{
    //// class constants
    public const int PASS_GRADE = 5;
    public const int TOTAL_COURSES = 40;

    //// static constructor:
    static Student()
    {
        Console.WriteLine("Initializing the class!");
    }

    //// constructor:
    public Student(int nov)
    {
        Console.WriteLine("Constructing a Student! ");

        no = nov; // instance field assigned
        no_completed_courses = 0; // instance field assigned

        Console.WriteLine("The Student No is: " + no);
            // instance field accessed

        Student.no_students = Student.no_students + 1;
            // static field accessed and assigned

        Console.WriteLine("Created Students so far are: " +
            Student.no_students); // static field accessed
    }

    //// public instance methods
    public int get_no_student() => no;
        // defined by expression (use of operator ==>)

    public void record_grade(int grade)
    {
        if (grade >= PASS_GRADE) // const access
        {

```

```

        completed_courses[no_completed_courses++] = grade;
        // instance field accessed and assigned
    }
}

// In the following, the commented body of the function
// would have reached the same result
public bool graduate() => no_completed_courses == TOTAL_COURSES;
        // instance method

//{
//    return no_completed_courses == TOTAL_COURSES;
//}

public int evaluate_GPA()
{
    int curr_GPA = 0;        // local variable
    if (no_completed_courses > 0)
    {
        // The following is a C-like loop
        // for (int i = 0; i < no_completed_courses; i++)
        //    curr_GPA += completed_courses[i];
        // OR
        // In the following, Sum() method from LINQ is used
        // curr_GPA = completed_courses.Sum();
        // OR
        foreach (int grade in completed_courses)
            curr_GPA += grade;
        if ((curr_GPA % no_completed_courses) <
            (no_completed_courses * 0.5))
            curr_GPA = curr_GPA / no_completed_courses;
        else
            curr_GPA = ((curr_GPA / no_completed_courses) + 1);
        if (graduate())
            if (curr_GPA > highest_GPA) // static field access
                highest_GPA = curr_GPA; // static field assignment
    }
    return curr_GPA;
}

//// public static methods
public static int get_highest_GPA() => highest_GPA;
public static int get_no_students() => no_students;

//// private static fields
// number of created Students:
private static int no_students = 0;
// current highest GPA of all students:
private static int highest_GPA = 0;

```

```

    //// private instance fields
    private int no;    // specific student's no
    // student's grades:
    private int[] completed_courses = new int[TOTAL_COURSES];
    // student's number of passed courses:
    private int no_completed_courses = 0;    // Instance field
}

```

Στο παραπάνω πρόγραμμα έχουμε ορισμό κλάσης που περιέχει: σταθερές, στατική μέθοδο αρχικοποίησης, (δημόσια) μέθοδο αρχικοποίησης στιγμιοτύπων, (δημόσιες) μεθόδους στιγμιοτύπου, (δημόσιες) στατικές μεθόδους, (ιδιωτικά) στατικά πεδία, (ιδιωτικά) πεδία στιγμιοτύπου. Επίσης, βλέπετε τον ορισμό απλών μεθόδων επιστροφής τιμών εκφράσεων με χρήση του τελεστή =>.

Η εκτύπωση κατά την εκτέλεση του παραπάνω προγράμματος είναι:

```

The number of courses is: 40
Initializing the class!
The number of created students so far is: 0
Testing a Single Class!
Constructing a Student!
The Student No is: 100
Created Students so far are: 1
Constructing a Student!
The Student No is: 200
Created Students so far are: 2
Constructing a Student!
The Student No is: 300
Created Students so far are: 3
NOT YET!
The GPA of Student with no: 100 is: 8
3
0

```

Στην παραπάνω εκτύπωση, παρατηρήστε ότι δεν πραγματοποιείται αρχικοποίηση της κλάσης Student λόγω χρήσης της σταθεράς της TOTAL_COURSES. Αντίθετα, βλέπουμε την εκτύπωση από το σώμα της στατικής μεθόδου αρχικοποίησης, Initializing the class!, πριν από την εκτύπωση που προκαλείται από την εντολή εκτύπωσης που εμπλέκει κλήση της στατικής μεθόδου get_no_students() της κλάσης.

Παράδειγμα ορισμού κλάσης με χρήση ιδιοτήτων

Στο παρακάτω παράδειγμα, έχουμε μια παραλλαγή της έννοιας του “Φοιτητή”, κατά την οποία μας ενδιαφέρει και το ονοματεπώνυμό του. Επίσης, θέλουμε να επιβάλουμε ο αριθμός μητρώου του να μην είναι αρνητικός αριθμός, σε περίπτωση αλλαγής του. Προκειμένου να χειριστούμε τα πεδία στιγμιοτύπου, έχουμε ορίσει ιδιότητες. Οι ιδιότητες αυτές είναι:

- Οι αυτο-υλοποιούμενες ιδιότητες FName και LName που αναπαριστούν το όνομα και το επώνυμο του “Φοιτητή”, αντίστοιχα. Επιτρέπουν ανάκτηση και αλλαγή/ανάθεση τιμής.
- Η ιδιότητα Name που αναπαριστάνει το πλήρες ονοματεπώνυμο του “Φοιτητή”. Η ιδιότητα αυτή επιτρέπει μόνο ανάγνωση τιμής, η οποία προκύπτει από επιστροφή της τιμής μιας απλής έκφρασης που χρησιμοποιεί τις προηγούμενες ιδιότητες.
- Η ιδιότητα No που αναπαριστάνει τον αριθμό μητρώου του “Φοιτητή”. Αυτή επιτρέπει ανάκτηση και αλλαγή τιμής και σχετίζεται με ένα ιδιωτικό υποστηρικτικό πεδίο που δηλώνεται στην κλάση, το _no.

Προκειμένου να αναδειχθεί η αυτόνομη ύπαρξη του πεδίου αυτού, στη συνάρτηση αρχικοποίησης στιγμιοτύπων της κλάσης, κάνουμε απευθείας ανάθεση/ανάκτηση σε αυτό.

Στο παράδειγμα αυτό έχουμε ακολουθήσει την προτεινόμενη γραφή για τα ονόματα της C#, όπου προτείνεται τα δημόσια ονόματα να ξεκινούν με κεφαλαίο γράμμα, τα ονόματα των ιδιωτικών πεδίων στιγμιοτύπου με τον χαρακτήρα `_` και τα ονόματα των ιδιωτικών στατικών πεδίων με `s_`. Οι λέξεις μέσα στο όνομα διακρίνονται μεταξύ τους από χρήση κεφαλαίου γράμματος στην έναρξη λέξης, εκτός από τα ιδιωτικά ονόματα στα οποία το πρώτο γράμμα του ονόματος είναι πεζό. Έτσι, ορίζουμε μια νέα κλάση `Student` ως εξής:

```
using System;

public class Test
{
    public static void Main()
    {
        Console.WriteLine("Testing a Single Class with Properties!");

        Student s1 = new Student("FN1", "LN1", 100);
        Student s2 = new Student("FN2", "LN2", 200);
        Student s3 = new Student("FN3", "LN3", 300);

        s1.RecordGrade(6);
        s1.RecordGrade(4);
        s1.RecordGrade(10);
        s1.RecordGrade(8);
        s1.RecordGrade(9);

        Console.WriteLine("The GPA of the Student with first name: " +
                           s1.FName + ", last name: " +      // property use
                           s1.LName + " and No: " +          // property use
                           s1.No + " so far is: " +          // property use
                           s1.EvaluateGPA());

        Console.WriteLine("The GPA of the Student with name: " +
                           s2.Name + " and No: " +           // property use
                           s2.No + " so far is: " + s2.EvaluateGPA());
    }
}

class Student
{
    // class constants
    public const int PASS_GRADE = 5;
    public const int TOTAL_COURSES = 40;

    // static members
    private static int s_noStudents = 0;
    private static int s_highestGPA = 0;

    public static int GetHighestGPA() => s_highestGPA;
```

470 ΑΛΛΕΣ ΓΛΩΣΣΕΣ ΜΕ ΑΝΤΙΚΕΙΜΕΝΟΣΤΡΑΦΗ ΧΑΡΑΚΤΗΡΙΣΤΙΚΑ


```

        Student.s_noStudents = Student.s_noStudents + 1;

        Console.WriteLine("Created Students so far are: " +
                           Student.s_noStudents);
    }

    public void RecordGrade(int grade)
    {
        if (grade >= PASS_GRADE)
        {
            _completedCourses[_noCompletedCourses++] = grade;
        }
    }

    public bool Graduate() => _noCompletedCourses == TOTAL_COURSES;

    public int EvaluateGPA()
    {
        int currGPA = 0;
        if (_noCompletedCourses > 0)
        {
            foreach (int grade in _completedCourses)
                currGPA += grade;
            if ((currGPA % _noCompletedCourses) <
                (_noCompletedCourses * 0.5))
                currGPA = currGPA / _noCompletedCourses;
            else
                currGPA = ((currGPA / _noCompletedCourses) + 1);
            if (Graduate())
                if (currGPA > s_highestGPA)
                    s_highestGPA = currGPA;
        }
        return currGPA;
    }
}

```

Η εκτύπωση από την εκτέλεση του παραπάνω προγράμματος, καταλήγει ως εξής:

...

The GPA of the Student with first name: FN1, last name: LN1 and No: 100 so far is: 8
 The GPA of the Student with name: FN2 LN2 and No: 200 so far is: 0

Τα δεδομένα από τις παραπάνω εκτυπώσεις προκύπτουν από τη χρήση των ιδιοτήτων FName, LName, Name και No.

Παράδειγμα απευθείας αρχικοποίησης

Ας επανέξουμε τον παραπάνω ορισμό της κλάσης Student, με μια default μέθοδο αρχικοποίησης. Για παράδειγμα:

```
public Student()
{
    Console.WriteLine("Student constructor doing nothing executing!");
}
```

Η μέθοδος αυτή, κακώς, δεν αυξάνει το στατικό πεδίο που αντιστοιχεί στον μετρητή των στιγμιοτύπων της κλάσης Student που έχουν δημιουργηθεί. Θεωρώντας τον επαυξημένο ορισμό της κλάσης Student, στην παρακάτω μέθοδο Main, κάνουμε απευθείας αρχικοποίηση των αντικειμένων της κλάσης.

```
public static void Main()
{
    Console.WriteLine("Testing a Single Class with Properties!");

    Student s1 = new Student("FN1", "LN1", 100);
    Student s2 = new Student { No = 200, LName = "LN2", FName = "FN2" };
                                     // direct initialization

    s1.RecordGrade(6);
    s1.RecordGrade(8);

    Console.WriteLine("The GPA of the Student with first name: " +
        s1.FName + " and last name: " + s1.LName +
        " and No: " + s1.No + " so far is: " +
        s1.EvaluateGPA());
    Console.WriteLine("The GPA of the Student with name: " + s2.Name +
        " and No: " + s2.No + " so far is: " +
        s2.EvaluateGPA());

    Console.WriteLine("The number of created students is: " +
        Student.GetNoOfStudents());
}
```

Η εκτύπωση της εκτέλεσης του παραπάνω δίνει:

```
Testing a Single Class with Properties!
Initializing the class!
Constructing a Student!
The Student No is: 100
Created Students so far are: 1
Student constructor doing nothing executing!
The GPA of the Student with first name: FN1, last name: LN1 and No: 100 so far is: 7
The GPA of the Student with name: FN2 LN2 and No: 200 so far is: 0
The number of created students is: 1
```

Στις τρεις τελευταίες γραμμές της παραπάνω εκτύπωσης έχουμε να παρατηρήσουμε τα εξής: οι ιδιότητες και των δύο στιγμιοτύπων της κλάσης Student έχουν αποκτήσει τιμές και από τους δύο τρόπους αρχικοποίησης, όμως, το στατικό πεδίο που αντιστοιχεί στο πλήθος των στιγμιοτύπων της κλάσης Student δεν έχει ενημερωθεί με τη δημιουργία του δεύτερου στιγμιοτύπου, μια που δεν εκτελέστηκε πουθενά κώδικας που να το αυξάνει.

10.1.4.3 Τύποι Δομής

Στην παράγραφο αυτή, θα κάνουμε μια μικρή σύνοψη των χαρακτηριστικών του τύπου δομής και θα δώσουμε κάποια παραδείγματα ορισμού τέτοιων τύπων. Αν θέλουμε να ορίσουμε έναν δικό μας “ελαφρύ” τύπο με συμπεριφορά κατά τιμή (value semantics), μπορούμε να ορίσουμε έναν τύπο δομής χρησιμοποιώντας τη γλωσσική δομή `struct`, αντί για `class`. Οι τύποι δομής, όπως είπαμε, ανήκουν στην κατηγορία τύπων τιμής. Προτείνεται να χρησιμοποιούνται όταν θέλουμε να αναπαραστήσουμε δομήσεις δεδομένων που συνοδεύονται από φτωχή συμπεριφορά. Πολλές φορές, λόγω της φύσης τους, τα δεδομένα που εμπεριέχουν δεν πρόκειται να αλλάξουν, έτσι ένας νέος τύπος δομής μπορεί να προσδιοριστεί ως `readonly`. Επίσης, δεν μπορούμε να ορίσουμε μεθόδους καταστροφής για τους τύπους δομής.

Οι μεταβλητές των τύπων δομής έχουν αποθηκευμένο το ίδιο το αντικείμενο που αποτελεί την τιμή τους και μάλιστα, πάντα, στη στοίβα. Σε περίπτωση ανάθεσης μεταξύ μεταβλητών, αλλά και περάσματος παραμέτρων και επιστροφής μεθόδου, γίνεται αντιγραφή της τιμής στην άλλη μεταβλητή.

Τέλος, οι τύποι δομής δεν συμμετέχουν σε κληρονομικότητα (δεν μπορούμε να ορίσουμε υποτύπους τους) — εκτός από το ότι κληρονομούν από την κλάση `object` λόγω ένταξης στην ιεραρχία των τύπων. Όμως, μπορούν να υλοποιούν τύπους διεπαφής και στις μεταβλητές τους μπορούν να εφαρμοστούν διαδικασίες `boxing/unboxing`, ακόμα και σε τύπους διεπαφής.

Στη συνέχεια, δίνουμε κάποια παραδείγματα ορισμού τύπου δομής. Τα παραδείγματα είναι εμπνευσμένα από τον τύπο “Αεροπορική Πτήση” των πρώτων κεφαλαίων του βιβλίου.

Παράδειγμα ορισμού ενός τύπου δομής

Στο παρακάτω, ορίζεται ένας τύπος δομής `Flight`, ο οποίος έχει τρεις ιδιότητες μόνο και μια μέθοδο αρχικοποίησης. Στη μέθοδο `Main` δημιουργούμε τρεις διαφορετικές μεταβλητές αλλά οι δύο από αυτές έχουν τις ίδιες τιμές.

```
using System;

public class Test
{
    public static void Main()
    {
        Console.WriteLine("Testing a Struct!");
        Flight f1 = new Flight(100, "Athens", "Paris");
        Flight f2 = new Flight(100, "Athens", "Paris");
        Flight f3 = new Flight(500, "Athens", "Heraklion");

        Console.WriteLine(f1.Equals(f2));
        // == is defined for reference types
    }
}

public struct Flight
{
    public int FlightNo { get; }
    public string DepartureCity { get; }
    public string ArrivalCity { get; }

    public Flight(int fno, string depcity, string arrcity)
    {
```

```

        Console.WriteLine("Constructing a Flight!");
        FlightNo = fno;
        DepartureCity = depcity;
        ArrivalCity = arrcity;
    }
}

```

Η εκτύπωση από την εκτέλεση του παραπάνω προγράμματος είναι η εξής:

```

Testing a Struct!
Constructing a Flight!
Constructing a Flight!
Constructing a Flight!
True

```

Παρατηρήστε ότι η σύγκριση τιμών των δύο μεταβλητών `f1` και `f2` δίνει αποτέλεσμα `True`. Αν ο τύπος είχε οριστεί ως τύπος κλάσης μέσω `class`, το αποτέλεσμα θα ήταν `False`. Εξυπακούεται ότι, για την περίπτωση του τύπου κλάσης, το ίδιο θα προέκυπτε στην περίπτωση που η σύγκριση γίνει με χρήση του τελεστή `==`, κάτι που, στην περίπτωση των τύπων δομής, δεν προσφέρεται από τη γλώσσα.

Παράδειγμα ορισμού ενός τύπου δομής και σύνθεση

Στο παρακάτω παράδειγμα προσθέσαμε άλλον έναν τύπο δομής, τον τύπο `MyTime` και επανξήσαμε τον τύπο `Flight` με άλλα δύο πεδία του τύπου αυτού. Με βάση αυτό το παράδειγμα, μπορείτε να δείτε τη συμπεριφορά κατά τιμή ακόμα και στην περίπτωση σύνθεσης. Στο παράδειγμα, βλέπετε και πρόσβαση στα μέλη μελών σύνθετων κλάσεων και χρήση `default` τιμών σε μεθόδους αρχικοποίησης.

```

using System;

public class Test
{
    public static void Main()
    {
        Console.WriteLine("Testing Structs!");
        Flight f1 = new Flight(100, "Athens", "Paris");
        Flight f2 = new Flight(100, "Athens", "Paris");
        Flight f3 = new Flight(500, "Athens", "Heraklion");

        Console.WriteLine(f1.FlightNo + "    " +
                           // member access
                           f1.DepartureCity + "    " +
                           // member access
                           f1.ArrivalCity + "    " +
                           // member access
                           f1.DepartureTime.Hours +
                           // member's member access
                           "    " + f1.DepartureTime.Mins);
                           // member's member access
        Console.WriteLine(f1.Equals(f2));
    }
}

```

```
public struct Flight
{
    public int FlightNo { get; }
    public string DepartureCity { get; }
    public string ArrivalCity { get; }
    public MyTime DepartureTime;
    public MyTime Duration;

    public Flight(int fno, string depcity, string arrcity,
                  int t1h = 0, int t1m = 0, int t2h = 0, int t2m = 0)
        // default values
    {
        Console.WriteLine("Constructing a Flight!");
        DepartureTime = new MyTime(t1h,t1m);
        Duration = new MyTime(t2h,t2m);

        FlightNo = fno;
        DepartureCity = depcity;
        ArrivalCity = arrcity;
    }
}

public struct MyTime
{
    public int Hours { get; }
    public int Mins { get; }

    public MyTime(int h, int m)
    {
        Hours = h;
        Mins = m;
        Console.WriteLine("Creating a MyTime");
    }
}
```

Η εκτύπωση από την εκτέλεση του παραπάνω είναι:

```
Testing Structs!
Constructing a Flight!
Creating a MyTime
Creating a MyTime
Constructing a Flight!
Creating a MyTime
Creating a MyTime
Constructing a Flight!
Creating a MyTime
Creating a MyTime
100 Athens Paris 0 0
True
```

Παρατηρήστε τις εκτυπώσεις που προκύπτουν από την εκτέλεση των μεθόδων αρχικοποίησης. Κυρίως, όμως, την τελευταία γραμμή: το αποτέλεσμα εξακολουθεί να είναι `True`, ακόμα και στην περίπτωση που τα μέλη είναι τύπου δομής.

10.1.5 Κληρονομικότητα

Θα κλείσουμε με μια συζήτηση και παραδείγματα σχετικά με την κληρονομικότητα μεταξύ κλάσεων. Η κληρονομικότητα στη `C#` εκφράζει τη σχέση *isa* μεταξύ τύπων.

Οι περισσότερες παρατηρήσεις σχετικά με την κληρονομικότητα έχουν ήδη γίνει, συζητώντας για τα επίπεδα ορατότητας, καθώς και τις μεθόδους. Να υπενθυμίσουμε τα παρακάτω. Οι μέθοδοι δεν είναι αυτόματα εικονικές. Αν θέλουμε μια μέθοδος να είναι εικονική, πρέπει να την προσδιορίσουμε ως `virtual`. Στην περίπτωση αυτή, εάν επαναορίζεται σε μια υποκλάση, πρέπει στην υποκλάση να το διατυπώσουμε ρητά με χρήση της λέξης-κλειδιού `override`. Άπαξ και μια μέθοδος χαρακτηριστεί ως εικονική, παραμένει έως ότου προσδιοριστεί ως κλειδωμένη με χρήση της λέξης-κλειδιού `sealed`. Τότε, δεν μπορεί να επαναοριστεί στις υποκλάσεις, εκτός εάν με χρήση της λέξης-κλειδιού `new` οριστεί νέα μέθοδος. Με χρήση της λέξης-κλειδιού `sealed` στη δήλωση μιας κλάσης, δηλώνουμε ότι μια κλάση δεν μπορεί να έχει υποκλάσεις. Με χρήση της λέξης-κλειδιού `abstract` στη δήλωση μιας κλάσης, δηλώνουμε ότι δεν μπορούμε να δημιουργήσουμε αντικείμενα της κλάσης αυτής.

Δηλώνουμε ότι μια κλάση είναι υποκλάση μιας άλλης κλάσης με χρήση του συμβόλου `:`. Για παράδειγμα, αν μια κλάση `B` δηλώνεται ως

```
class B : A { ... }
```

τότε έχει δηλωθεί ως υποκλάση της κλάσης `A`, δηλαδή ότι `B isa A`. Το τι σημαίνει αυτό έχει συζητηθεί στα προηγούμενα κεφάλαια και οι παρατηρήσεις που έχουν γίνει περί αναπαράστασης, χρήσης και αρχικοποίησης αντικειμένων και κλάσεων της ιεραρχίας ισχύουν και στην περίπτωση της `C#`.

Στο σημείο αυτό, πρέπει να αναφέρουμε τον ρόλο της λέξης-κλειδιού `base` κατά την κληρονομικότητα. Η λέξη αυτή χρησιμοποιείται κατά τον ορισμό μιας παραγόμενης κλάσης στις εξής περιπτώσεις:

1. Στο σώμα μιας μεθόδου, προκειμένου να κληθεί μια μέθοδος της βασικής κλάσης που έχει επαναοριστεί στην παραγόμενη.
2. Ακολουθεί την επικεφαλίδα μιας μεθόδου αρχικοποίησης, προκειμένου να προσδιοριστεί η μέθοδος αρχικοποίησης της βασικής κλάσης που θα χρησιμοποιηθεί κατά την αρχικοποίηση ενός αντικειμένου της υποκλάσης με χρήση της μεθόδου στην οποία εμφανίζεται.

Παράδειγμα κληρονομικότητας μεταξύ κλάσεων

Στο παρακάτω παράδειγμα, έχουμε προσθέσει στην κλάση `Student` μια ακόμα (δημόσια) μέθοδο στιγμιοτύπου, την `Print`. Επίσης, έχουμε ορίσει μια υποκλάση της, την `WorkingStudent`, που αναπαριστάει την έννοια του “Εργαζόμενου Φοιτητή”. Στην υποκλάση, ορίσαμε μια στατική μέθοδο αρχικοποίησης, προσθέσαμε (ιδιωτικά) πεδία στιγμιοτύπου, μια (δημόσια) μέθοδο στιγμιοτύπου και επαναορίσαμε τη μέθοδο `Print`. Τέλος, ορίσαμε και μια (δημόσια) μέθοδο αρχικοποίησης στιγμιοτύπου. Τόσο η μέθοδος αρχικοποίησης στιγμιοτύπου, όσο και η επαναορισμένη μέθοδος `Print` χρησιμοποιούν τη λέξη-κλειδί `base` για να επιτύχουν τις λειτουργικότητες που προαναφέραμε.

Στην συνάρτηση `Main`, αρχικά δημιουργούμε ένα στιγμιότυπο της κλάσης `Student` και στη συνέχεια ένα στιγμιότυπο της κλάσης `WorkingStudent`. Όπως αναμενόταν, στο δεύτερο μπορούμε να εφαρμόσουμε μεθόδους κληρονομημένες από τη γονεϊκή κλάση, μεθόδους επαναορισμένες στην υποκλάση, τις κληρονομημένες ιδιότητες και τη μέθοδο που προστέθηκε στην υποκλάση. Οι αρχικοποιήσεις πραγματοποιούνται με βάση τις παρατηρήσεις που αναφέραμε για τις κλάσεις στη `C#` και την κληρονομικότητα γενικότερα.

```

using System;

public class Test
{
    public static void Main()
    {
        Console.WriteLine("Testing a SuperClass with Properties!");

        Student s1 = new Student("FN1", "LN1", 100);
        s1.Print();

        WorkingStudent ws1 =
            new WorkingStudent("FN2", "LN2", 200, 800, 6);
        ws1.Print();
        ws1.RecordGrade(7);
        ws1.RecordGrade(8);
        Console.WriteLine("The GPA for Student with Name: " +
            ws1.Name + " is: " + ws1.EvaluateGPA() +
            " and the income is: " + ws1.EvaluateIncome());
    }
}

class Student
{
    ...
    ...

    public void Print()
    {
        Console.WriteLine("The name of the student is " +
            Name + " and the no is: " + No);
    }
}

class WorkingStudent : Student
{
    private static int s_noStudents = 0;
    public new static int GetNoOfStudents() => s_noStudents;

    static WorkingStudent()
    {
        Console.WriteLine("Initializing the subclass!");
    }

    private int _salary;
    private int _months;
}

```

```

public WorkingStudent(string fname, string lname,
                      int nov, int sal, int mon)
    : base(fname, lname, nov) // base constructor use
{
    _salary = sal;
    _months = mon;
    Console.WriteLine("A Working Student has been created!");

    WorkingStudent.s_noStudents++;
    Console.WriteLine("From WS: Created Students so far are: " +
                      Student.s_noStudents);
    Console.WriteLine("Created Working Students so far are: " +
                      WorkingStudent.s_noStudents);
}

public int EvaluateIncome() => _salary * _months; // new method

public new void Print() // override method
{
    base.Print(); // base method call
    Console.WriteLine("The Student's income is: " +
                      EvaluateIncome());
}
}

```

Η εκτύπωση που προκύπτει από το παραπάνω πρόγραμμα είναι η εξής:

Testing a SuperClass with Properties!

Initializing the class!

Constructing a Student!

The Student No is: 100

Created Students so far are: 1

The name of the student is FN1 LN1 and the no is: 100

Initializing the subclass!

Constructing a Student!

The Student No is: 200

Created Students so far are: 2

A Working Student has been created!

From WS: Created Students so far are: 2

Created Working Students so far are: 1

The name of the student is FN2 LN2 and the no is: 200

The Student's income is: 4800

The GPA for Student with Name: FN2 LN2 is: 8 and the income is: 4800

Μια παρατήρηση, που γίνεται εμφανής στην παραπάνω εκτύπωση: οι κλάσεις αρχικοποιούνται μόνο μια φορά. Η κλάση Student αρχικοποιήθηκε άπαξ, εξαιτίας της δημιουργίας του πρώτου αντικειμένου της κλάσης Student.

Παράδειγμα χρήσης μη-εικονικής μεθόδου υπερκλάσης

Σε μια ιεραρχία κλάσεων, μπορούμε να χρησιμοποιήσουμε μια μη-εικονική μέθοδο υπερκλάσης για ένα

αντικείμενο υποκλάσης της, ακόμα και αν είναι επαναορισμένη, κάνοντας μετατροπή τύπου (cast). Αυτό μπορεί να γίνει είτε εκτός της υποκλάσης, σε μεταβλητή αναφοράς της, είτε μέσα στην υποκλάση, με χρήση της λέξης-κλειδιού `this`.

Στο παρακάτω παράδειγμα, έχουμε μια ιεραρχία κλάσεων με τρεις κλάσεις, `C` `isa` `B` `isa` `A`, οι οποίες έχουν (επανα)ορίσει μια μέθοδο `TestF`. Η μέθοδος αυτή εκτυπώνει την τιμή ενός πεδίου στιγμιοτύπου ορισμένου στην κάθε κλάση. Παρατηρούμε ότι μπορούμε να έχουμε πρόσβαση και εκτέλεση ορισμού της μεθόδου από συγκεκριμένη κλάση της ιεραρχίας, στην περίπτωση μας τον ορισμό της κλάσης `A`, τόσο με μετατροπή τύπου σε μεταβλητή αναφοράς — στη μέθοδο `Main` στην κλήση `((A)b).TestF()` — όσο και μέσα από το σώμα μεθόδου, κάνοντας μετατροπή τύπου με χρήση της λέξης-κλειδιού `this` — στην κλήση `((A)this).TestF()` στο σώμα του ορισμού της μεθόδου στην κλάση `C`.

```
using System;

public class Test
{
    public static void Main()
    {
        Console.WriteLine("Testing Inheritance!");
        B b = new B();
        b.TestF();           // Calls the new method of B
        ((A)b).TestF();      // Cast: calls the method of A

        C c = new C();
        c.TestF();           // Calls the new method of C
                           // which calls the method of A
                           // using cast
    }
}

public class A
{
    int _data = 1;

    public void TestF()
    {
        Console.WriteLine("In A! " + _data);
    }
}

public class B : A
{
    int _data = 10;

    public new void TestF()
    {
        Console.WriteLine("In B! " + _data + " ");
    }
}
```

```

public class C : B
{
    int _data = 100;

    public new void TestF()
    {
        ((A)this).TestF();    // Cast
        Console.WriteLine("In C! " + _data);
    }
}

```

Η εκτύπωση της εκτέλεσης του προγράμματος επιβεβαιώνει την παρατήρηση που κάναμε προηγουμένως και είναι η εξής.

Testing Inheritance!

In B! 10

In A! 1

In A! 1

In C! 100

10.1.5.1 Πολλαπλή Κληρονομικότητα, Τύποι Διεπαφής

θα κλείσουμε τη συζήτησή μας για την κληρονομικότητα, κάνοντας μια αναφορά στην πολλαπλή κληρονομικότητα. Πολλαπλή κληρονομικότητα δεν υποστηρίζεται στη C#. Για τον σκοπό αυτό, όπως και στη γλώσσα Java, χρησιμοποιούνται τύποι διεπαφής, οι οποίοι, κατά βάση, δίνουν προδιαγραφές συμπεριφοράς τις οποίες οι κλάσεις (και οι τύποι τιμής) πρέπει να υλοποιήσουν. Μεταξύ τους οι τύποι διεπαφής μπορεί να συνδέονται με κληρονομικότητα. Παρότι υπάρχουν κάποιες διαφορές σε λεπτομέρειες με τους τύπους διεπαφής της Java, η βασική ιδέα και η λειτουργικότητά τους είναι ίδιες, οπότε δεν θα επιμείνουμε περισσότερο.

10.1.6 Κάποια Άλλα Χρήσιμα Στοιχεία για τη C#

Στα προηγούμενα συζητήσαμε περί τύπων που μπορεί να ορίσει ο χρήστης, αλλά εστίασαμε μόνο σε πολύ βασική λειτουργικότητα. Οι δυνατότητες που προσφέρει η C# οι οποίες σχετίζονται με τον ορισμό τύπων και χειρισμό μεταβλητών και χαρακτηριστικών τους είναι πολυάριθμες και, αν ο αναγνώστης το επιθυμεί, μπορεί να αναφερθεί στην τεκμηρίωση της γλώσσας. Στο πλαίσιο της υποστήριξης τύπων, πρέπει να επισημάνουμε την υποστήριξη γενικευμένων τύπων και επαναληπτών, καθώς και τη δυνατότητα ορισμού συμβάντων που προαναφέραμε.

Η C# προσφέρει, επίσης, δυνατότητες χειρισμού εξαιρέσεων σε ένα δομημένο και επεκτάσιμο πλαίσιο. Ακόμα, προσφέρεται η δυνατότητα ορισμού attributes για μέλη, κλάσεις κ.ά., προκειμένου να δηλωθούν επισημειώσεις.

Σημαντικό είναι ότι η C# έχει πολλές δυνατότητες, λόγω ενσωμάτωσης του συναρτησιακού προγραμματισμού. Πέρα από τη δυνατότητα ορισμού λάμδα-εκφράσεων, που ήδη έχουμε αναφέρει, ενδεικτικά θα αναφέρουμε δυνατότητες που προκύπτουν από το ταίριασμα προτύπων και τα δένδρα εκφράσεων (expression trees).

Τέλος, θα κλείσουμε κάνοντας μια αναφορά στο LINQ (Language Integrated Query), το οποίο είναι ένα σύνολο τεχνολογιών του .NET προκειμένου να διατυπώνονται ερωτήματα σε ετερογενείς ή/και καταναεμημένες πηγές δεδομένων με ενιαίο τρόπο. Οι τεχνολογίες αυτές είναι ενσωματωμένες στη C#.

10.2 Η Γλώσσα Python

Στην παράγραφο αυτή, θα ασχοληθούμε με τη γλώσσα Python. Η Python εμφανίστηκε στις αρχές του 1990. Ο εμπνευστής και βασικός προγραμματιστής της, αλλά και πρώτος χρήστης της, είναι ο Ολλανδός μαθηματικός/πληροφορικός Guido van Rossum. Στόχος του σχεδιασμού της ήταν να αποτελεί μια γλώσσα προγραμματισμού “για όλους” και να συμβάλει στην αύξηση της παραγωγικότητας κατά την ανάπτυξη λογισμικού.

Ο λόγος που τη συζητάμε είναι, όχι μόνον λόγω του εύρους της χρήσης της στις μέρες μας, αλλά κυρίως λόγω της φιλοσοφίας της. Από τη μια, είναι μια γλώσσα που μπορεί να διευκολύνει τον προγραμματιστή να επικεντρωθεί στην επίλυση του προβλήματός του, απαλλαγμένος από πολλές τεχνικές λεπτομέρειες, και έτσι προφυλάσσοντάς τον από σχετικά σφάλματα. Αυτό την καθιστά κατάλληλη και για έναν αρχάριο προγραμματιστή. Από την άλλη, όμως, είναι πολύ ενδιαφέρουσα από πλευράς υλοποίησης. Το μοντέλο της επιτρέπει να είναι επεκτάσιμη σε όλα τα επίπεδα, από τους τύπους δεδομένων έως τις διαδικασίες. Η γλώσσα είχε αντικειμενοστραφή χαρακτηριστικά από τον αρχικό σχεδιασμό της.

Η Python είναι μια γλώσσα υψηλού επιπέδου, γενικής χρήσης. Τα βασικά χαρακτηριστικά της είναι τα εξής:

- Υποστηρίζει πολλά είδη προγραμματισμού, όπως διαδικαστικό, συναρτησιακό και αντικειμενοστραφή.
- Είναι διερμηνευόμενη (interpreted) γλώσσα και προσφέρει τη δυνατότητα διαδραστικής επικοινωνίας με τον διερμηνέα. Ως επακόλουθο αυτού, ένα πρόγραμμα Python μπορεί να συνταχθεί ή να επαυξηθεί, ως είσοδος σε έναν κύκλο εκτέλεσης του διερμηνέα.
- Υποστηρίζει τύπους. Οι τύποι, όμως, προκύπτουν δυναμικά (dynamically-typed).
- Στο πλαίσιο της αναγνωσιμότητας των προγραμμάτων Python, γίνεται επιβολή στοίχισης (indentation) των block εντολών, ώστε να γίνεται άμεσα καθαρό ποιες εντολές αποτελούν ένα block.
- Προσφέρει αυτόματη διαδικασία περισυλλογής σκουπιδιών.
- Η καθιερωμένη βιβλιοθήκη της είναι πολύ πλούσια και μεταφέρσιμη σε πλήθος συστημάτων (Unix, Windows και macOS).

Δεδομένου ότι ενδιαφερόμαστε κυρίως για τα αντικειμενοστραφή χαρακτηριστικά της, θα πρέπει να πούμε ότι ως προς το θέμα αυτό έχει μεγαλύτερη εγγύτητα με γλώσσες όπως η Smalltalk και η Modula-3, παρά με τη C++.

Η χρήση της γλώσσας είναι πολύ διαδεδομένη και οι περιοχές των εφαρμογών της εκτείνονται από τον χώρο των επιχειρήσεων και της διακυβέρνησης έως της μηχανικής και των θετικών επιστημών. Δεν πρέπει να παραλείψουμε να τονίσουμε την ευρεία χρήση της σε όλα τα επίπεδα της εκπαίδευσης σχετικά με την Πληροφορική.

10.2.1 Λίγα Λόγια για την Ιστορία της

Όπως είπαμε, ο δημιουργός, βασικός “εξ αεί” συντελεστής και πρώτος χρήστης της Python είναι ο Guido van Rossum. Η ιστορία της, όπως ο ίδιος την αφηγείται, έχει ως εξής. Στο πλαίσιο της διδακτορικής του ενασχόλησης, στο εθνικό ερευνητικό κέντρο της Ολλανδίας CWI, ο Van Rossum είχε συμβάλει στην ανάπτυξη της γλώσσας προγραμματισμού ABC (1982-1986). Η γλώσσα αυτή, παρότι διευκόλυνε τον προγραμματιστή και συνέβαλε στην παραγωγικότητα, εγκαταλείφθηκε, καθώς δεν προωθήθηκε και δεν χρησιμοποιήθηκε ευρέως.

Στη συνέχεια, ο Van Rossum ενεπλάκη σε ένα άλλο έργο του CWI που αφορούσε ένα καταναμημένο λειτουργικό σύστημα, το Amoeba. Στο πλαίσιο του έργου, η ενασχόλησή του αφορούσε ανάπτυξη “εργαλείων” λογισμικού προκειμένου να συνοδεύσουν το σύστημα αυτό. Η γλώσσα που χρησιμοποιούσε ήταν η C, κάτι

που δεν διευκόλυνε την παραγωγικότητά του. Θεωρούσε ότι μια γλώσσα με τα χαρακτηριστικά και την αναγνωσιμότητα της ABC, αλλά πιο κατάλληλη για την ανάπτυξη αυτής της μορφής του λογισμικού, θα ήταν ιδανική. Έτσι, τα Χριστούγεννα του 1989 και για λόγους προσωπικής ευχαρίστησης αποφάσισε να σχεδιάσει και να υλοποιήσει μια γλώσσα επηρεασμένη από την ABC. Σε κάθε περίπτωση, η ενασχόληση με την ανάπτυξη μιας γλώσσας προγραμματισμού του ήταν πιο ελκυστική από την ανάπτυξη “εργαλείων”. Η πρώτη επιδείξιμη μορφή του διερμηνέα της γλώσσας ήταν έτοιμη μέσα σε τρεις μήνες και έτυχε θερμής αποδοχής και από τα άλλα μέλη που συμμετείχαν στο έργο. Η γλώσσα ονομάστηκε “Python” με αφορμή την αγαπημένη τηλεοπτική σειρά του Van Rossum, “Monty Python’s Flying Circus” — αντίθετα από ότι συνηθιζόταν για επιτεύγματα του χώρου της πληροφορικής, στο έργο χρησιμοποιούσαν ονόματα από την κουλτούρα της ποπ και όχι ονόματα σημαντικών ανθρώπων της επιστήμης και της φιλοσοφίας.

Όλη η ομάδα του έργου υπήρξε πολύ πιο παραγωγική τον επόμενο χρόνο. Έναν χρόνο μετά αποφάσισαν να διαθέσουν τον κώδικα ελεύθερα και να προωθήσουν τη γλώσσα μέσω του Usenet. Η γλώσσα είχε μεγάλη απήχηση: απέκτησε πολλούς υποστηρικτές και χρήστες, προγραμματιστές που συνέβαλαν στη διόρθωση σφαλμάτων κ.ά. Η διάδοσή της ήταν εντυπωσιακή. Έως το 2018, ο Van Rossum ήταν ο βασικός συντελεστής της ανάπτυξης της γλώσσας και εκείνος που έπαιρνε αποκλειστικά τις σχετικές αποφάσεις. Το 2018, (προσωρινά) συνταξιοδοτήθηκε. Η δεύτερη έκδοση της Python βγήκε το 2000 και από το 2008, εκδίδονται εκδοχές της τρίτης, στην οποία και παραμένουμε. Παρότι υπάρχουν διάφορες υλοποιήσεις της Python, η βασική είναι στη γλώσσα C (CPython). Μια πιο γρήγορη υλοποίησή της είναι η PyPy, της οποίας η προσέγγιση βασίζεται σε επιτόπου μεταγλώττιση.

10.2.2 Γενικά Χαρακτηριστικά της Γλώσσας

Χρησιμοποιώντας την Python, μπορεί κάποιος να προγραμματίσει πολύ ευκολότερα. Τα γλωσσικά στοιχεία που προσφέρει είναι υψηλού επιπέδου και δεν επιβάλλεται εστίαση σε (τεχνικές) λεπτομέρειες. Από την άλλη μεριά, όμως, το μοντέλο της ίδιας της γλώσσας και τα δυναμικά χαρακτηριστικά του, σε συνδυασμό με τις παροχές από την καθιερωμένη βιβλιοθήκη της γλώσσας, δίνει πολλές δυνατότητες στον προγραμματιστή.

Παρότι δεν θα επιμείνουμε σε λεπτομέρειες, ας κάνουμε μια μικρή σύνοψη της γλώσσας.

10.2.2.1 Η Μορφή ενός Προγράμματος και η Οργάνωση του Κώδικα

Ας δούμε κατ’ αρχήν την Python από την πλευρά του προγραμματιστή. Ένα πρόγραμμα στην Python διατυπώνεται ως σενάριο (script) προς εκτέλεση από τον διερμηνέα της γλώσσας. Η εκτέλεση του κώδικά του ανάγεται στην εκτέλεση των εντολών του από τον διερμηνέα. Μια ευκολία που προσφέρει ο διερμηνέας είναι η δυνατότητα διαδραστικής επικοινωνίας, κάτι που διευκολύνει ιδιαίτερα στην αρχική επαφή με τη γλώσσα και την εκμάθησή της. Μια συνήθης πρώτη επαφή με τη γλώσσα είναι η εκτέλεση της εντολής εκτύπωσης, π.χ. μιας συμβολοσειράς, από την είσοδο του κύκλου εκτέλεσης του διερμηνέα. Αυτό γίνεται ως εξής:

```
>>> print("My first attempt")
```

και το αποτέλεσμα της εκτέλεσης είναι:

```
My first attempt
```

Τόσο απλά! Να πούμε ότι τόσο στο παραπάνω, όσο και στη συνέχεια, στα παραδείγματά μας το >>> αναφέρεται στην αναμονή του κύκλου εκτέλεσης του διερμηνέα για είσοδο από τον χρήστη. Αυτή μπορεί να είναι είτε εντολές προς εκτέλεση, είτε ορισμοί νέων ονομάτων. Για παράδειγμα,

```
>>> def f():
...     print("In a function")
...
>>> f()
In a function
```

Στο παραπάνω παράδειγμα, ορίσαμε και, στη συνέχεια, εκτελέσαμε μια συνάρτηση `f()`. Προσέξτε, δεν υπάρχει τύπος επιστροφής! Αυτό καθώς και γενικότερα περί τύπων των συστατικών του προγράμματός μας θα τα συζητήσουμε στη συνέχεια.

Στο σημείο αυτό, να δώσουμε ένα παράδειγμα που αναδεικνύει ένα ουσιαστικό χαρακτηριστικό των προγραμμάτων Python: τη χρήση της στοίχισης για τον ορισμό block. Ένα σύνολο συνεχόμενων εντολών που αποτελούν ένα block οφείλουν να έχουν την ίδια στοίχιση (και αντίστροφα). Αν σε ένα αρχείο έχουμε τον παρακάτω κώδικα:

```
def f():
    print("Now in a function")
    print("Just printing!")

f()
```

και το “φορτώσουμε” στον διερμηνέα, προκύπτει η εξής εκτύπωση:

```
Now in a function
Just printing!
```

Η εντολή `print("Just printing!")` είναι μέρος του block του σώματος της συνάρτησης `f()`. Αν, όμως, το περιεχόμενο του αρχείου ήταν:

```
def f():
    print("Now in a function")
print("Just printing!")

f()
```

η εκτύπωση είναι:

```
Just printing!
Now in a function
```

Η κλήση `print("Just printing!")` δεν αποτελεί μέρος του ορισμού της συνάρτησης `f()`. Σημείωση: συνήθως στοιχίζουμε τα blocks ανά τέσσερα κενά.

Ένα πρόγραμμα Python αποτελείται από μια ακολουθία εντολών προς εκτέλεση και, ενδεχόμενα, ενός συνόλου ορισμών, π.χ. συναρτήσεων και τύπων. Για παράδειγμα:

```
print("blablabla") # Built in function call

myfirstvar = 50    # Global variable assignment

print(myfirstvar)

def myfirstfun(arg1,arg2): # Function definition
    print("Inside the function!")
    print("arg2")
    print(arg2)
    while arg1 < arg2:
        print("arg1")
        print(arg1)
        arg1 = arg1 + 1 # Local vars
    print("The global var: ", myfirstvar)
```

```
print("At last?", "YES!")

myfirstfun(5,10) # Global function call

print("Last thing:", myfirstvar)
```

Στον παραπάνω κώδικα, έχουμε εκτέλεση σεναρίου αλλά και ορισμό (συνάρτησης).

Δεν υπάρχει καμία απαίτηση να έχει οριστεί κάτι συγκεκριμένο (συνάρτηση ή κλάση). Άρα μπορούμε να γράψουμε κώδικα που να μην έχει καμία κλάση ή, ακόμα, και καμία συνάρτηση ορισμένη. Ποια είναι, λοιπόν, η σχέση της Python με τον αντικειμενοστραφή προγραμματισμό; Πέρα από την προφανή που σχετίζεται με τη δυνατότητα ορισμού κλάσεων, θα κάνουμε μια παρατήρηση που αφορά, όχι απαραίτητα τη μεριά του προγραμματιστή, αλλά το μοντέλο της ίδιας της γλώσσας. Για την ίδια την Python, τα πάντα σε ένα πρόγραμμα είναι αντικείμενα. Αυτό ίσχυε στο μοντέλο της εξαρχής. Εδώ να σημειώσουμε ότι, ενόσω η Python διευκολύνει τον προγραμματισμό, και λόγω της απλότητας στη χρήση της, δίνει πολλές δυνατότητες στον προγραμματιστή για αλληλεπίδραση με το μοντέλο του προγράμματός του — τις οποίες δεν θα συζητήσουμε ιδιαίτερα, όμως.

Υψίστης σημασίας για την Python και τους σχεδιαστικούς της στόχους είναι η οργάνωση του κώδικα. Κατά συνέπεια, η έννοια της εμβέλειας είναι απολύτως κρίσιμη για τον προγραμματισμό σε Python. Οι αναφορές ονομάτων που χρησιμοποιούνται σε ένα πρόγραμμα επιλύονται με βάση την τρέχουσα εμβέλεια. Στην περίπτωση που μπορούμε να ονοματίσουμε εμβέλειες, το όνομά τους μπορεί να χρησιμοποιηθεί για τον προσδιορισμό.

Το προφανές εργαλείο στον διαδικαστικό προγραμματισμό για την οργάνωση κώδικα, ομαδοποιώντας τον ώστε να μπορεί και να επαναχρησιμοποιηθεί, είναι οι συναρτήσεις. Η Python, όμως, δεν περιορίζεται σε αυτό. Στην Python μπορούμε να ομαδοποιήσουμε λογικά σχετιζόμενο κώδικα σε *στοιχεία modules*. Αν σε ένα αρχείο γράψουμε κώδικα που απλά περιγράφει μια διαδικασία, αυτό αποτελεί ένα *σενάριο*. Αν, όμως, σε ένα αρχείο έχουμε συμπεριλάβει ορισμούς, π.χ. συναρτήσεων, κλάσεων κλπ., τότε το αρχείο αποτελεί ένα στοιχείο με όνομα το όνομα του αρχείου. Σε ένα στοιχείο μπορούμε να συμπεριλάβουμε ορισμούς συναρτήσεων, κλάσεων κλπ., αλλά και κώδικα που θα εκτελεστεί ως σενάριο όταν πρωτοφορτωθεί το στοιχείο στον διερμηνέα. Το σενάριο που εκτελείται στο υψηλό επίπεδο (top level) του διερμηνέα, είτε έχει προκύψει από σενάριο στοιχείου, είτε όχι, θεωρείται ότι αποτελεί μέρος ενός στοιχείου που ονομάζεται `__main__`.

Μπορούμε να εισαγάγουμε ονόματα από ένα στοιχείο στον κώδικά μας με δηλώσεις εισαγωγής, `import`. Αυτές μπορεί να αναφέρονται σε όλα τα ονόματα ενός στοιχείου ή σε συγκεκριμένο όνομα. Εισάγοντάς τα, μπορούμε να χρησιμοποιούμε τα ονόματα αυτά, χωρίς τον προσδιορισμό της εμβέλειας, με χρήση του ονόματος του στοιχείου. Στο σημείο αυτό, να κάνουμε μια κρίσιμη επισήμανση: στην Python δεν είναι δυνατή η απόκρυψη πληροφορίας στο επίπεδο της γλώσσας. Προσδιορίζοντας ένα όνομα με τον κατάλληλο τρόπο (π.χ. την εμβέλεια στην οποία ορίζεται), μπορούμε να έχουμε πρόσβαση σε αυτό από οποιοδήποτε σημείο του κώδικά μας.

Περαιτέρω οργάνωση κώδικα για τα προγράμματα Python μπορούμε να επιτύχουμε με χρήση ιεραρχικών καταλόγων που ομαδοποιούν τα στοιχεία μας σε *πακέτα*. Ένα πακέτο είναι ένας κατάλογος που περιέχει στοιχεία ή/και υποπακέτα. Ένα όνομα ενός στοιχείου μπορεί να προσπελαστεί, δίνοντας τα ονόματα των πακέτων στο μονοπάτι του καταλόγου που βρίσκεται το στοιχείο, χωρισμένα με τελεία. Κάθε κατάλογος/πακέτο πρέπει να περιέχει ένα αρχείο με όνομα `__init__.py` ώστε να τον προσδιορίζει ως πακέτο. Το περιεχόμενο του αρχείου αυτού μπορεί να είναι κενό ή να περιέχει κώδικα για την αρχικοποίηση του πακέτου ή δηλώσεις σχετικά με την εισαγωγή ονομάτων από στοιχεία.

Κατά την εκτέλεση ενός προγράμματος, τα ονόματα του προγράμματος οργανώνονται σε *λεξικά χώρων ονομάτων* οι οποίοι δημιουργούνται δυναμικά. Στη γενική περίπτωση, έχουμε τον *τοπικό* χώρο ονομάτων, τον *καθολικό* και τον χώρο ονομάτων των *ενσωματωμένων* (built-in) ονομάτων. Οι χώροι ονομάτων εξυπηρετούν προκειμένου να έχουμε άμεση πρόσβαση σε ονόματα, χωρίς προσδιορισμούς εμβέλειας.

Οι χώροι ονομάτων δημιουργούνται δυναμικά κατά την εκτέλεση ενός προγράμματος και έχουν διαφορε-

τικούς χρόνους ζωής ανάλογα με την εμβέλεια την οποία περιγράφουν. Ο χώρος ονομάτων των ενσωματωμένων ονομάτων δημιουργείται με την έναρξη της εκτέλεσης του διερμηνέα και υφίσταται καθ' όλη τη διάρκεια της εκτέλεσής του. Οι καθολικοί χώροι των ονομάτων στοιχείων δημιουργούνται μόλις γνωστοποιηθεί στον διερμηνέα (εκτελεστεί) ο ορισμός του στοιχείου και παραμένουν και αυτοί εν ζωή διαρκώς. Αντίθετα, οι χώροι ονομάτων σωμάτων συναρτήσεων δημιουργούνται με την κλήση της συνάρτησης και με την ολοκλήρωση της εκτέλεσής της καταστρέφονται (στην περίπτωση αναδρομικής συνάρτησης, δημιουργούνται νέοι χώροι για κάθε κλήση).

Κατά την εκτέλεση ενός προγράμματος, στην περίπτωση χρήσης ονόματος χωρίς προσδιορισμό εμβέλειας, η σειρά με την οποία διερευνώνται οι τρέχοντες χώροι ονομάτων είναι η εξής:

1. Αρχικά, εξετάζονται τα τοπικά ονόματα, στον χώρο ονομάτων της εσωτερικής εμβέλειας.
2. Κατόπιν, εξετάζονται ονόματα από χώρους ονομάτων περικλειουσών εμβελειών (στην περίπτωση εμφώλευσης).
3. Κατόπιν, εξετάζονται τα ονόματα από τον καθολικό χώρο του τρέχοντος στοιχείου.
4. Τέλος, εξετάζεται ο χώρος ονομάτων των ενσωματωμένων ονομάτων.

Η Python δίνει και τη δυνατότητα ορισμού κλάσεων. Έχοντας τη δυνατότητα ορισμού στοιχείων, όμως, μπορούμε να περιορίσουμε τη χρήση κλάσεων μόνο για τη μοντελοποίηση του προβλήματός μας. Για τις κλάσεις στην Python θα μιλήσουμε σε επόμενη παράγραφο.

10.2.2.2 Δεδομένα και Τύποι

Η Python, όπως είπαμε, υποστηρίζει τύπους δεδομένων. Οι τύποι της είναι κλάσεις, υποκλάσεις της κλάσης `object`. Κάθε τι που δημιουργείται από ένα πρόγραμμα Python είναι αντικείμενο μιας κλάσης. Ακόμα και οι συναρτήσεις. Τα αντικείμενα των κλάσεων αποτελούνται από κάποια δεδομένα και σχετίζονται με κάποιες συναρτήσεις, μεθόδους, που καθορίζουν τη λειτουργικότητά τους, τι, δηλαδή, μπορούμε να κάνουμε σε αυτά.

Οι συνήθεις τύποι των αριθμών, συμβολοσειρών κλπ. είναι κλάσεις. Μάλιστα μπορούμε να ορίσουμε υποκλάσεις τους, ώστε να τους εξειδικεύσουμε. Αν θεωρήσετε ότι η συνάρτηση `type` της Python επιστρέφει τον τύπο του ορίσμά της, δείτε τι προκύπτει στα παρακάτω παραδείγματα:

```
>>> type(5)
<class 'int'>
>>> type("kkkk")
<class 'str'>
>>> type('a')
<class 'str'>
```

Τα δεδομένα του προγράμματός μας αποτελούν αντικείμενα κλάσεων που είτε είναι ορισμένες από τη γλώσσα, είτε τις ορίζουμε εμείς. Προκειμένου να χρησιμοποιήσουμε τα αντικείμενα αυτά, συνήθως δημιουργούμε μεταβλητές που αναφέρονται σε αυτά. Αυτές δεν δεσμεύονται να είναι συγκεκριμένου τύπου. Για παράδειγμα, δείτε το παρακάτω μέρος του κύκλου του διερμηνέα:

```
>>> a = 7
>>> a
7
>>> a = "blablabla"
>>> a
'blablabla'
```

Η μεταβλητή `a` την πρώτη φορά αναφέρθηκε σε ακέραιο, αλλά τη δεύτερη σε συμβολοσειρά. Αυτό δεν δημιούργησε πρόβλημα. Αν πάλι θέλουμε να την καταργήσουμε, μπορούμε να χρησιμοποιήσουμε την εντολή `del`,

```
>>> del a
```

Τότε, αν στη συνέχεια ζητήσουμε να δούμε την τιμή της μεταβλητής `a`, θα πάρουμε το μήνυμα `NameError: name 'a' is not defined`.

Οι τύποι δεδομένων της Python διακρίνονται σε *βασικούς* και *ανώτερους*. Τα δεδομένα βασικών τύπων αποθηκεύουν απλές τιμές. Οι βασικοί τύποι δεδομένων είναι: ακέραιοι, πραγματικοί κινητής υποδιαστολής, μιγαδικοί, λογικές τιμές, συμβολοσειρές (σημείωση: για τις συμβολοσειρές, στην Python χρησιμοποιούμε είτε διπλά είτε απλά εισαγωγικά, αλλά και, για την περίπτωση έκτασής τους σε πάνω από μία γραμμές, τριπλά εισαγωγικά, `"""`). Όλοι οι τύποι συνοδεύονται από τις κατάλληλες πράξεις και λειτουργικότητα.

Οι ανώτεροι τύποι αποκαλούνται και *περιέκτες* (containers), διότι εμπεριέχουν (αναφορές σε) άλλα δεδομένα. Τέτοιοι τύποι είναι οι: πλειάδες (tuples), λίστες (lists), διαστήματα (ranges), λεξικά (dictionaries) και σύνολα (sets). Αυτά αποθηκεύουν συλλογές δεδομένων είτε με τη μορφή λίστας είτε με τη μορφή ζεύγους (κλειδί, τιμή). Καθότι ο κάθε τύπος από αυτούς έχει διαφορετική λειτουργικότητα, ανάλογα με τα χαρακτηριστικά των δεδομένων του προβλήματός μας, κάνουμε για το καθένα από αυτά την κατάλληλη επιλογή.

Μια άλλη διάκριση των τύπων αφορά το εάν τα δεδομένα μπορούν να αλλάξουν τιμή ή όχι, είναι δηλαδή *αμετάβλητα* ή *μεταβλητά*. Τα δεδομένα όλων των αριθμητικών τύπων καθώς και των τύπων πλειάδων και εύρους είναι αμετάβλητα. Τα δεδομένα, όμως, των τύπων λίστας, συνόλου και λεξικού είναι μεταβλητά.

Παραδείγματα:

Λίστα

```
>>> alist = ['a', 5, 'blabla']
>>> alist
['a', 5, 'blabla']
```

Πλειάδα

```
>>> data = 11, "fname", "lname"
>>> data
(11, 'fname', 'lname')
```

Σύνολο

```
>>> theset = {1, 3, 1, 5, 1}
>>> theset
{1, 3, 5}
```

Λεξικό

```
>>> dictionnaire = {'cat': 'chat', 'dog': 'chien'}
>>> dictionnaire
{'cat': 'chat', 'dog': 'chien'}
>>> dictionnaire['cat']
'chat'
```


Εύρος

```
>>> r = range(0, 10)
>>> 11 in r
False
>>> 10 in r
False
>>> 9 in r
True
```

Κλείνοντας με τους τύπους της γλώσσας, να κάνουμε μια αναφορά στη δυνατότητα επεξεργασίας λιστών, συνόλων και λεξικών μέσω *comprehensions*, προκειμένου να εφαρμόσουμε επαναληπτικά μια πράξη/έκφραση επάνω στα στοιχεία τους. Η δυνατότητα αυτή οδηγεί σε καθαρότερο και ασφαλέστερο κώδικα, σε περιπτώσεις που διαφορετικά θα αντιμετωπίζαμε μέσω *for*-βρόχων.

10.2.2.3 Δομές Ελέγχου και Συναρτήσεις

Ας προχωρήσουμε στις διαδικασίες, τώρα.

Οι δομές ελέγχου που προσφέρονται από τη γλώσσα εμπίπτουν στις συνηθισμένες περιπτώσεις, με κάποιες διαφοροποιήσεις, και αφορούν επιλογή, επανάληψη και μεταφορά ελέγχου. Ενδιαφέρον έχει ότι προσφέρεται και μια εντολή *pass*, που απλά δεν κάνει τίποτα. Χρησιμεύει για λόγους αναγνωσιμότητας κώδικα ή ενδεχόμενης αλλαγής του.

Δεν θα ασχοληθούμε με τις δομές ελέγχου άλλο, όμως. Θα επικεντρωθούμε στις συναρτήσεις. Πέρα από κάποια χαρακτηριστικά τους που θα σχολιάσουμε, το ενδιαφέρον είναι ότι, στην Python, και οι συναρτήσεις είναι αντικείμενα.

Στην Python, δεν είναι απαραίτητο να γράψουμε τον κώδικά μας μέσα σε συναρτήσεις — παρότι δεν είναι κομψό αυτό, σε καμία περίπτωση. Οι συναρτήσεις, όμως, είναι το βασικό εργαλείο οργάνωσης κώδικα. Ο ορισμός μιας συνάρτησης γίνεται με χρήση της λέξης-κλειδιού *def*, δίνοντας το όνομά της και τα ονόματα για τα ορίσματά της. Δεν δίνεται τύπος επιστροφής, ακόμα και αν επιστρέφεται κάτι από τη συνάρτηση. Τα ορίσματα δεν είναι τίποτα άλλο παρά αναφορές που θα πάρουν τιμή κατά την κλήση. Έτσι, έχουμε πέρασμα παραμέτρων μόνο κατά αναφορά, δεν μπορεί να γίνει κατά τιμή.

Απλό παράδειγμα, ορισμού και χρήσης συνάρτησης:

```
>>> def double_it(num):
...     print(num)
...     rnum = num * 2
...     return rnum
...
>>> double_it(5)
5
10
>>> double_it(5.0)
5.0
10.0
```

Κατά τον ορισμό μιας συνάρτησης, συσχετίζεται το όνομα της συνάρτησης με το αντικείμενο που της αντι-στοιχεί και ενημερώνεται ο τρέχων πίνακας συμβόλων. Η συσχέτιση αυτή χρησιμοποιείται από τον διερμηνέα οποτεδήποτε χρησιμοποιηθεί το όνομα αυτό στην τρέχουσα εμβέλεια. Για την πρόσβαση σε ένα αντικείμενο συνάρτησης, δεν είναι απαραίτητο να έχουμε μόνο ένα όνομα. Και άλλα ονόματα μπορεί να έχουν συσχετιστεί με το αντικείμενο αυτό. Για παράδειγμα:

```
>>> dbit = double_it
>>> dbit(100)
100
200
```

Την ίδια συνάρτηση μπορούμε να προσπελάσουμε είτε με χρήση του ονόματος `double_it` είτε μέσω του ονόματος `dbit`, μια που δώσαμε ως τιμή στο δεύτερο την τιμή του πρώτου.

Μια παρατήρηση ως προς την έκφραση της επιστροφής: αυτή μπορεί να είναι κενή — απλά η εντολή `return` — ή να επιστρέφονται πολλές τιμές. Η πρώτη περίπτωση χρησιμεύει για λόγους διάκρισης `blocks` σε εναλλακτικά.

Να πούμε ακόμα ότι υποστηρίζεται υπερφόρτωση ονομάτων συναρτήσεων και δίνεται η δυνατότητα `default` τιμών για τα ορίσματα συναρτήσεων.

Κλείνοντας τη συζήτηση σχετικά με τις συναρτήσεις στην Python, να κάνουμε μια αναφορά στις *ανώνυμες συναρτήσεις*. Αυτές συμβάλλουν στον χαρακτηρισμό της Python ως γλώσσας υποστήριξης συναρτησιακού προγραμματισμού. Πρόκειται για μικρές συναρτήσεις, που ενδέχεται να χρησιμοποιηθούν μόνο μία φορά, οπότε δεν αξίζει να ονοματοδοτηθούν και να αντιμετωπιστούν σαν κανονικές συναρτήσεις. Απλά, υπάρχει ο περιορισμός γι' αυτές ότι δεν μπορούν να περιέχουν οποιονδήποτε κώδικα, αλλά ορίζονται μέσω εκφράσεων πάνω σε αμετάβλητα δεδομένα. Αποτελούν, δηλαδή, λάμδα-εκφράσεις του συναρτησιακού προγραμματισμού. Παράδειγμα:

```
>>> add = lambda x, y : x + y
>>> add(5,10)
15
```

Στο προηγούμενο, δώσαμε την ανώνυμη συνάρτηση ως τιμή σε ένα όνομα και έτσι δημιουργήθηκε μια κλασική συνάρτηση.

10.2.3 Αντικειμενοστραφής Προγραμματισμός

Στην παράγραφο αυτή θα δούμε τι προσφέρει η γλώσσα στον προγραμματιστή, ώστε να μπορεί να προγραμματίσει αντικειμενοστραφώς. Να υπενθυμίσουμε ότι στην Python δεν έχουμε απόκρυψη πληροφορίας: με χρήση του κατάλληλου προθέματος που προσδιορίζει εμβέλεια, αν χρειάζεται τέτοιο, μπορούμε να προσπελάσουμε οποιοδήποτε όνομα.

Όπως τονίσαμε, παρότι στο μοντέλο της ίδιας της γλώσσας, κάθε τι σε ένα πρόγραμμα που εκτελείται είναι αντικείμενο, στον προγραμματιστή αυτό είναι διάφανο. Μπορεί να προγραμματίζει χωρίς καν να αντιλαμβάνεται τις έννοιες των “αντικειμένων” και των “κλάσεων”, αρκεί να θεωρήσει την έννοια “κλάση” ως συνώνυμο του “τύπου”. Επιπλέον, προκειμένου να οργανώσει τον κώδικά του και να ονοματίσει εμβέλειες, μπορεί να χωρίσει το πρόγραμμά του σε στοιχεία/αρχεία και πακέτα/καταλόγους. Αν όμως, επιθυμεί να αναπαραστήσει τις έννοιες/οντότητες του κόσμου του προβλήματός του σε κλάσεις στο πρόγραμμά του, αυτό είναι δυνατό στην Python, ορίζοντας κλάσεις με χρήση της γλωσσικής δομής `class`.

10.2.3.1 Ορίζοντας μια Απλή Κλάση

Προκειμένου να κάνουμε την παρουσίαση των κλάσεων στην Python, θα δούμε ένα πρώτο παράδειγμα ορισμού κλάσης και χρήσης της απευθείας στον κύκλο εκτέλεσης του διερμηνέα.

Στον κύκλο εκτέλεσης του διερμηνέα της Python, έστω ότι έχουμε το:

```
>>> class Dog:
...     'A demo class'
...
...     print('A Dog class can be used')
```

```
...
...     dog_sound = 'Barking'
...
...     print('Dog sound is: ', dog_sound)
...
...     def bark(self, soundv):
...         print(Dog.dog_sound)
...         self.sound = soundv
...         print(self.sound)
...
```

A Dog class can be used

Dog sound is: Barking

```
>>> azor = Dog()
```

```
>>> azor.bark('Gav Gav')
```

Barking

Gav Gav

Στο παραπάνω ορίζεται μια κλάση Dog η οποία περιέχει τα εξής:

- Μια συμβολοσειρά, 'A demo class', που αποτελεί την τεκμηρίωση της κλάσης — και μπορεί να ανακτηθεί από το πρόγραμμα.
- Ένα σενάριο, που αποτελείται από τα εξής:
 - κλήση μιας ενσωματωμένης συνάρτησης, `print('A Dog class can be used')`,
 - ανάθεση τιμής σε μια μεταβλητή κλάσης: στη μεταβλητή `dog_sound` τη συμβολοσειρά 'Barking' και
 - κλήση μιας ενσωματωμένης συνάρτησης και χρήση της μεταβλητής κλάσης `dog_sound`: `print('Dog sound is: ', dog_sound)`.
- Έναν ορισμό μεθόδου στιγμιοτύπου, της `bark`.
 - Αυτή δέχεται δύο ορίσματα. Το πρώτο αντιστοιχεί στο αντικείμενο επάνω στο οποίο θα εφαρμοστεί η μέθοδος. Κατά την κλήση της θα δεχθεί μόνο ένα όρισμα, το δεύτερο, και το πρώτο θα γραφεί προθεματικά με χρήση τελείας, .. Το όνομα του πρώτου ορίσματος είναι `self`. Εν αντιθέσει με τη λέξη `this` στη C++ και την Java, η λέξη `self` δεν είναι δεσμευμένη στην Python. Αποτελεί μια σύμβαση ονοματοδοσίας για το πρώτο όρισμα των μεθόδων, που αντιστοιχεί στο αντικείμενο που θα εφαρμοστούν και ακολουθείται από όλους τους προγραμματιστές (η αντίστοιχη σύμβαση για τις μεθόδους κλάσης, θα δούμε ότι είναι η λέξη `cls`).
 - Στο σώμα της μεθόδου βλέπουμε τα εξής:
 - * Στην κλήση `print(Dog.dog_sound)` τη χρήση της μεταβλητής κλάσης `dog_sound` με προσδιορισμό της εμβέλειας `Dog`.
 - * Τη δημιουργία μεταβλητής στιγμιοτύπου, `self.sound`, λόγω ανάθεσης (`self.sound = soundv`).
 - * Τη χρήση της μεταβλητής στιγμιοτύπου `print(self.sound)`.

Δεδομένου ότι ο ορισμός της κλάσης δόθηκε απευθείας στον κύκλο εκτέλεσης του διερμηνέα, με την ολοκλήρωσή του, εκτελείται: ενημερώνεται το περιβάλλον και εκτελείται το σενάριο της. Η εκτύπωση είναι:

A Dog class can be used

Dog sound is: Barking

Στη συνέχεια, μπορούμε να ορίσουμε αντικείμενα της κλάσης `Dog`. Αυτό γίνεται με κλήση της κλάσης, δηλαδή, στην προκειμένη περίπτωση `Dog()`. Προκειμένου να έχουμε μια αναφορά στο αντικείμενο που θα δημιουργηθεί, κάνουμε ανάθεση σε μια μεταβλητή, π.χ.,

```
>>> azor = Dog()
```

Μπορούμε να εφαρμόσουμε τη μέθοδο `bark` στο αντικείμενο που αναφέρεται από τη μεταβλητή `azor`, προθεματικά. Δηλαδή:

```
>>> azor.bark('Gav Gav')
```

Η εκτύπωση είναι:

```
Barking
Gav Gav
```

Μπορούμε, όμως, να χρησιμοποιήσουμε και τη μεταβλητή κλάσης `dog_sound`, προσδιορίζοντας την εμβέλεια της κλάσης. Δηλαδή:

```
>>> Dog.dog_sound
```

Η εκτύπωση είναι:

```
'Barking'
```

Μπορούμε ακόμα να προσπελάσουμε τις μεταβλητές στιγμιότυπου του αντικειμένου. Για παράδειγμα:

```
>>> azor.sound
```

Η εκτύπωση είναι:

```
'Gav Gav'
```

Όπως είπαμε, δεν υπάρχει απόκρυψη πληροφορίας. Μπορούμε να προσπελάσουμε κάθε όνομα της κλάσης. Μια ακόμα σύμβαση που υπάρχει είναι η χρήση του `_` ως πρώτο χαρακτήρα ονομάτων για ονόματα που θα μπορούσαν να θεωρηθούν ιδιωτικά, π.χ. ονοματίζουν λεπτομέρειες υλοποίησης. Δηλαδή, αντί για `sound` να γράφαμε `_sound`, αν το όνομα αυτό θεωρείται ιδιωτικό. Απλά, ως σύμβαση, όμως και μόνο. Και πάλι προσπελάζεται και από το υπόλοιπο πρόγραμμα.

Κάτι πολύ ενδιαφέρον αφορά τα δεδομένα στιγμιότυπου ενός αντικειμένου. Αυτά δημιουργούνται δυναμικά κατά την εκτέλεση κώδικα μεθόδων και όχι με τη δημιουργία του αντικειμένου. Λαμβάνοντας υπόψη ότι με χρήση του `__dict__` μπορούμε να δούμε τα δεδομένα αυτά (λεπτομέρειες για τη φύση και χρήση του `__dict__` μπορείτε να δείτε από την τεκμηρίωση της γλώσσας), δείτε το παρακάτω:

```
>>> azor.__dict__
{'sound': 'Gav Gav'}
>>> rex = Dog()
>>> rex.__dict__
{}
>>> rex.bark('Gav Gav Gav Gav')
Barking
Gav Gav Gav Gav
>>> rex.__dict__
{'sound': 'Gav Gav Gav Gav'}
```

Το αντικείμενο στο οποίο αναφέρεται η μεταβλητή `rex` αρχικά δεν είχε μεταβλητή `sound`.

Γενικές Παρατηρήσεις στα Αντικείμενα και στους Ορισμούς Κλάσεων

- Κάθε κλάση μπορεί να περιέχει ορισμούς μεθόδων ή/και σενάριο, αλλά και ορισμούς άλλων (εμφωλευμένων) κλάσεων.
- Οι μεταβλητές μιας κλάσης διακρίνονται σε:
 - μεταβλητές στιγμιοτύπου και
 - μεταβλητές κλάσης.

Κάθε αντικείμενο της κλάσης είναι συσχετισμένο με τις δικές του μεταβλητές στιγμιοτύπου. Αντίθετα, οι μεταβλητές κλάσης είναι συσχετισμένες με την κλάση.

- Διακρίνουμε τις μεθόδους σε:
 - μεθόδους στιγμιοτύπου και
 - μεθόδους κλάσης.

Οι πρώτες εφαρμόζονται σε αντικείμενο της κλάσης. Οι δεύτερες σχετίζονται με την ίδια την κλάση, την οποία χειρίζονται μέσω ορίσματος. Αυτό είναι το πρώτο όρισμα της μεθόδου, το οποίο από σύμβαση ονομάζουμε `cls`. Ιδιαίτερο ενδιαφέρον έχει η συμπεριφορά τους όταν εφαρμόζονται μέσω αντικειμένου στην περίπτωση κληρονομικότητας.

- Οι μέθοδοι στιγμιοτύπου λειτουργούν όπως οι εικονικές συναρτήσεις της C++.
- Υπάρχει και η κατηγορία των *στατικών μεθόδων*. Αυτές είναι μέθοδοι της εμβέλειας της κλάσης, αλλά δεν έχουν όρισμα που να σχετίζεται με την κλάση. Έχουν περιορισμένη χρήση ως συναρτήσεις-εργαλεία εμβέλειας κλάσης.
- Οι μέθοδοι κλάσης και οι στατικές μέθοδοι προσδιορίζονται με χρήση των *decorator* συναρτήσεων `classmethod()` και `staticmethod()`, αντίστοιχα (χρησιμοποιώντας τη γραφή `classmethod` και `staticmethod`). Για παράδειγμα, στην κλάση `Dog`, μπορούμε να προσθέσουμε τον παρακάτω κώδικα:

```
@classmethod
def blaC(cls, val):
    print(val, cls.dog_sound)
```

Τότε μπορούμε να ζητήσουμε την εκτέλεσή του:

```
>>> Dog.blaC('Loud')
```

και το αποτέλεσμα θα είναι

```
Loud Barking
```

- Κάθε αντικείμενο στην Python είναι στιγμιότυπο μιας κλάσης.
- Κάθε αντικείμενο έχει μια ταυτότητα, έναν τύπο και τιμή.
- Κάθε αντικείμενο είναι συσχετισμένο με τις μεταβλητές του και τις μεθόδους του.

- Σε κάθε αντικείμενο μπορούμε να προσθέσουμε ή να αφαιρέσουμε μεταβλητές ανά πάσα στιγμή.
- Οι ίδιες οι κλάσεις είναι στιγμιότυπα μιας μετακλάσης. Η μετακλάση αυτή μπορεί να προσδιοριστεί ως όρισμα κατά τον ορισμό της κλάσης — δίνοντας το όνομά της ως τιμή του ορίσματος `metaclass`.
- Ο ορισμός μιας κλάσης αποτελεί κώδικα δημιουργίας του αντικειμένου που αντιστοιχεί στην κλάση. Κατά την εκτέλεσή του, πέρα από σύνδεση της κλάσης με τα ήδη υπάρχοντα περιβάλλοντα και κλάσεις, εκτελείται το σώμα της και δημιουργείται το αντικείμενο που της αντιστοιχεί.

10.2.3.2 Αρχικοποίηση Αντικειμένων, Ανάθεση και Αρχικοποίηση μέσω Αντιγραφής

Στο παράδειγμα της προηγούμενης παραγράφου, καθώς και τη συζήτηση που ακολούθησε, δεν κάναμε καμιά αναφορά σε διαδικασία αρχικοποίησης αντικειμένων.

Στην Python, τη διαδικασία αρχικοποίησης αντικειμένων αναλαμβάνει μια ενσωματωμένη μέθοδος, η `__init__()` της κλάσης `object`. Για την ακρίβεια, κάθε φορά που δημιουργούμε ένα αντικείμενο μιας κλάσης (καλώντας την κλάση), αρχικά καλείται η μέθοδος `__new__()` για τη δημιουργία του αντικειμένου και στη συνέχεια η μέθοδος `__init__()` για την αρχικοποίησή του. Οπότε, αν θέλουμε να δώσουμε δικές μας διαδικασίες αρχικοποίησης για τα αντικείμενα μιας κλάσης, αρκεί να ορίσουμε μία (ή παραπάνω) μεθόδους `__init__()` για την κλάση αυτή.

Οπότε, στο παράδειγμά μας, θα μπορούσαμε να είχαμε προσθέσει τον παρακάτω κώδικα στον ορισμό της κλάσης `Dog`:

```
... def __init__(self, nam):
...     self.name = nam
...     print(self.name)
```

Τότε, δημιουργώντας ένα αντικείμενο της κλάσης `Dog`, καλείται αυτόματα και η παραπάνω συνάρτηση. Λόγω της εκτέλεσης του σώματός της, το αντικείμενο αυτό συσχετίζεται και με τη μεταβλητή στιγμιότυπου `name`. Βλέπουμε, λοιπόν, την παρακάτω συμπεριφορά:

```
>>> azor = Dog('Azor')
Azor
>>> azor.__dict__
{'name': 'Azor'}
```

Εννοείται ότι, λόγω δυνατότητας υπερφόρτωσης, μπορούμε να έχουμε παραπάνω από έναν υπερφορτωμένους ορισμούς της μεθόδου `__init__()` σε μια κλάση.

Σε εκφράσεις ανάθεσης μεταβλητών με χρήση του τελεστή ανάθεσης = δεν γίνεται αντιγραφή του αντικειμένου, απλά αντιγράφεται η αναφορά σε αυτό. Οπότε, με χρήση της μεταβλητής `azor` που δημιουργήσαμε προηγουμένως, ας κάνουμε τα παρακάτω:

```
>>> id(azor)
1783697340592
>>> azor2 = azor
>>> id(azor2)
1783697340592
```

Με δεδομένο ότι η ενσωματωμένη συνάρτηση `id` επιστρέφει την ταυτότητα ενός αντικειμένου, βλέπουμε ότι με την ανάθεση `azor2 = azor` και οι δύο μεταβλητές `azor2` και `azor` αναφέρονται στο ίδιο αντικείμενο. Αν ο στόχος μας δεν είναι αυτός, αλλά θέλουμε η μεταβλητή `azor2` να αναφέρεται σε αντίγραφο του αντικειμένου που αναφέρεται η μεταβλητή `azor`, πρέπει να δημιουργήσουμε ένα νέο αντικείμενο το οποίο να αρχικοποιηθεί μέσω αντιγραφής. Ο τρόπος να γίνει αυτό είναι μέσω των συναρτήσεων `copy` και `deepcopy`. Η διαφορά

μεταξύ των δύο είναι ότι στην περίπτωση σύνθετων αντικειμένων, με χρήση της συνάρτησης `deepcopy`, η αρχικοποίηση μέσω αντιγραφής γίνεται αναδρομικά σε όλες τις αναφορές, ενόσω με την `copy` το αντίγραφο θα ήταν απλά στο πρώτο επίπεδο. Και αυτές οι συναρτήσεις μπορούν να επαναοριστούν.

Με βάση το παραπάνω, δεδομένης της μορφής των αντικειμένων της κλάσης `Dog`, αρκεί να χρησιμοποιήσουμε τη συνάρτηση `copy` για να δημιουργήσουμε ένα νέο αντικείμενο στο οποίο να αναφερθεί η μεταβλητή `azor2` και το οποίο να είναι αντίγραφο του αντικειμένου στο οποίο αναφέρεται η μεταβλητή `azor`. Δείτε το παρακάτω:

```
>>> from copy import copy
>>> azor2 = copy(azor)
>>> id(azor)
2245561180288
>>> id(azor2)
2245561181440
>>> azor.name
'Azor'
>>> azor2.name
'Azor'
```

Παρότι τα αντικείμενα και των δύο μεταβλητών έχουν την ίδια τιμή, πρόκειται για διαφορετικά αντικείμενα, όπως βλέπετε από την τιμή της ταυτότητάς τους. Μια παρατήρηση: η συνάρτηση είναι ορισμένη σε ένα στοιχείο με όνομα `copy` και έτσι, πριν χρησιμοποιηθεί, πρέπει να κάνουμε εισαγωγή.

Η Python, όπως είπαμε, έχει διαδικασία περισυλλογής σκουπιδιών. Παρ' όλα αυτά, αν κρίνεται απαραίτητο, με επαναορισμό της μεθόδου `__del__()` της κλάσης `object`, ο προγραμματιστής μιας κλάσης μπορεί να ορίσει διαδικασίες που είναι καλό να γίνουν πριν καταστραφεί ένα αντικείμενο της κλάσης. Η μέθοδος αυτή καλείται αυτόματα από την Python, όταν πρόκειται να καταστραφεί ένα αντικείμενο.

10.2.3.3 Μια Υλοποίηση της Έννοιας του "Φοιτητή"

Αφού συζητήσαμε το πώς ορίζουμε κλάσεις και τι μπορεί να περιέχουν, ας δούμε τον ορισμό μιας κλάσης που αντιστοιχεί σε μια έννοια "Φοιτητή", παρόμοια με αυτήν της Παραγράφου 8.2.

```
class Student:
    'A class representing a student'

    _no_students = 0    # Class variable representing
                        # the number of created Students
    print('students created so far: ', _no_students)

    _highest_GPA = 0
    print('highest GPA so far: ', _highest_GPA)

    PASS_GRADE = 5
    TOTAL_COURSES = 40

    def __init__(self, nov):
        self._no = nov
        self._no_completed_courses = 0
        self._completed_courses = []
        Student._no_students = Student._no_students + 1
        print("no", self._no, "\n")
```

```

        print("courses", self._completed_courses, "\n")
        print("students so far: ", Student._no_students)

    def get_no_student(self):
        return self._no

    def record_grade(self, grade):
        if grade >= Student.PASS_GRADE :
            self._no_completed_courses = self._no_completed_courses + 1
            self._completed_courses.append(grade)

    def graduate(self):
        return self._no_completed_courses == Student.TOTAL_COURSES

    def evaluate_GPA(self):
        self._GPA = 0
        if self._no_completed_courses > 0 :
            self._GPA = sum(self._completed_courses) \
                        / self._no_completed_courses
            if self.graduate() :
                if self._GPA > Student._highest_GPA :
                    Student._highest_GPA = self._GPA
        return self._GPA

    @classmethod
    def get_highest_GPA(cls):
        return cls._highest_GPA

    @classmethod
    def get_no_students(cls):
        return cls._no_students

```

Με βάση αυτά που συζητήσαμε στην προηγούμενη παράγραφο, προσπαθήστε να κατανοήσετε τα περιεχόμενα αυτού του ορισμού.

Συγκρίνοντας τον παραπάνω ορισμό με τον ορισμό της κλάσης της Παραγράφου 8.2, θα θέλαμε να παρατηρήσουμε την έλλειψη δήλωσης σταθερών. Ως σύμβαση μόνο και πάλι, τα ονόματα των σταθερών είναι με κεφαλαία γράμματα.

Επίσης, θα θέλαμε να παρατηρήσετε τις μεθόδους κλάσης. Μπορούμε να τις καλέσουμε είτε μέσω του ονόματος της κλάσης είτε μέσω αντικειμένου.

Εκτελώντας τον προηγούμενο ορισμό στον κύκλο εκτέλεσης του διερμηνέα, έχουμε τα εξής:

```

students created so far: 0
highest GPA so far: 0

```

Στη συνέχεια, μπορούν να ακολουθήσουν τα παρακάτω:

```

>>> stud1 = Student(100)
no 100

```



```

courses []

students so far: 1
>>> stud2 = Student(200)
no 200

courses []

students so far: 2
>>> stud1.record_grade(5)
>>> stud2.get_no_student()
200
>>> stud2.evaluate_GPA()
0
>>> stud1.evaluate_GPA()
5.0
>>> stud2.get_no_students()
2
>>> Student.get_no_students()
2

```

Στο σημείο αυτό, να κάνουμε μια αναφορά στη δυνατότητα που δίνει η Python στον προγραμματιστή μιας κλάσης — καθώς και άλλες γλώσσες αντικειμενοστραφούς προγραμματισμού, π.χ. η C# όπως είδαμε — να ορίζει ιδιότητες για τα αντικείμενα της κλάσης. Οι ιδιότητες, εννοιολογικά, αντιστοιχούν σε χαρακτηριστικά των αντικειμένων που συμβάλλουν στην κατάστασή του και συνήθως μπορούν να παρατηρηθούν ή/και να αλλαχθούν. Στην Python, μπορούμε να ορίσουμε ιδιότητες με χρήση της ενσωματωμένης συνάρτησης `property()`. Μάλιστα, λόγω της λειτουργικότητάς της ως decorator, συνήθως χρησιμοποιείται η γραφή `@property`. Χρησιμοποιώντας τη συνάρτηση `property`, μπορούμε να καθορίσουμε μεθόδους ανάκτησης, αλλαγής τιμής κ.ά. για κάποιο δεδομένο στιγμιότυπο, οι οποίες μπορούν να χρησιμοποιούνται από το πρόγραμμα με τη μορφή μεταβλητής. Αυτή μπορεί να θεωρηθεί ως διεπαφή, επιτρέποντας να αλλάξει η υλοποίησή τους, χωρίς να επηρεαστεί το υπόλοιπο πρόγραμμα. Να υπενθυμίσουμε, όμως, ότι από τη στιγμή που δεν υπάρχει απόκρυψη πληροφορίας, ο ορισμός ιδιοτήτων δεν προφυλάσσει από ενδεχόμενη παραβίαση αυτής της σχεδιαστικής αρχής.

Ας δούμε ένα παράδειγμα χρήσης ιδιοτήτων για την κλάση που αναπαριστάει μια πιο απλή έννοια του “Φοιτητή”. Στην εκδοχή αυτή, θεωρούμε ότι ο αριθμός μητρώου του δεν είναι απαραίτητο να δοθεί κατά την αρχικοποίηση — στην περίπτωση αυτή είναι 0 — και δεν μπορεί να είναι αρνητικός αριθμός. Με βάση τις παραδοχές αυτές, ο ορισμός της κλάσης μπορεί να είναι:

```

class Student:
    'A class representing a student'

    _no_students = 0    # Class variable representing
                        # the number of created Students
    print('students created so far: ', _no_students)

    def __init__(self, nov = 0):
        self.no = nov          # Property used!
        Student._no_students = Student._no_students + 1
        print("no", self._no, "\n")    # Still the variable can be used
        print("students so far: ", Student._no_students)

```

```

@property
def no(self):
    return self._no

@no.setter
def no(self, nov):
    print('In the setter!')
    if nov < 0:
        print('The age must be positive')
    else:
        self._no = nov

@classmethod
def get_no_students(cls):
    return cls._no_students

```

Στον παραπάνω ορισμό, ο αριθμός μητρώου του “Φοιτητή” υλοποιείται με χρήση της μεταβλητής στιγμιοτύπου `_no`. Όμως, ορίσαμε και μια ιδιότητα `no` προκειμένου να την προσπελάσουμε και να διασφαλίζεται ότι πάντα θα είναι σε ορθή κατάσταση (τιμή μη αρνητική). Προσέξτε, στη διαδικασία αρχικοποίησης, αλλά και στην παρακάτω χρήση της κλάσης, τόσο τη χρήση της ιδιότητας, αλλά και τη δυνατότητα πρόσβασης στη μεταβλητή αναφοράς, παρά την ύπαρξη ιδιότητας.

```

students created so far: 0
>>> stud1 = Student()
In the setter!
no 0

```

```

students so far: 1
>>> stud2 = Student(200)
In the setter!
no 200

```

```

students so far: 2
>>> stud1.no
0
>>> stud2._no
200
>>> stud2.no
200
>>> stud1.no = 100
In the setter!
>>> stud2.no = -7
In the setter!
The age must be positive
>>> stud2.no
200

```

Ως σχόλιο, να πούμε ότι η χρήση ιδιοτήτων στην Python επιβάλλει επιπλέον κόστος, οπότε, εάν δεν είναι απαραίτητη η λειτουργικότητά τους, καλό είναι να αποφεύγονται. Μια περίπτωση που είναι χρήσιμες είναι

όταν οι αναθέσεις τιμών κλπ. γίνονται υπό προϋποθέσεις, τις οποίες μπορούμε να συμπεριλάβουμε στον ορισμό των μεθόδων των ιδιοτήτων. Μια ακόμα σημείωση: αν ο ορισμός των μεθόδων ανάκτησης είναι απλός, τότε στον ορισμό των ιδιοτήτων, για τη μέθοδο της ανάκτησης, μπορούμε να χρησιμοποιήσουμε λάμδα-εκφράσεις.

10.2.3.4 Σύνθεση Κλάσεων

Όπως είναι αναμενόμενο, μια ή περισσότερες μεταβλητές στιγμιότυπου μιας κλάσης μπορεί να αναφέρονται σε αντικείμενα άλλων κλάσεων. Στην περίπτωση που τα αντικείμενα αυτά και το στιγμιότυπο της κλάσης είναι άρρηκτα συνδεδεμένα μεταξύ τους, οι κλάσεις σχετίζονται με μια σχέση *has a*. Λαμβάνοντας υπόψη τις παρατηρήσεις που έχουμε κάνει σχετικά με την Python, δεν υπάρχει κάτι ιδιαίτερο που να πρέπει να αναπτύξουμε, οπότε περιοριζόμαστε στο να δώσουμε ένα παράδειγμα. Ας υποθέσουμε ότι έχοντας μια κλάση που αντιστοιχεί σε “Ιδιοκτησία”, την `Property`, θέλουμε να υλοποιήσουμε μια κλάση, `Owner`, που να αντιστοιχεί στην έννοια του “Ιδιοκτήτη Περιουσίας”. Η υλοποίηση θα μπορούσε να είναι:

```
class Owner:

    print('An Owner class can be used')

    def __init__(self, id_nov):
        print('Constructing an Owner!')
        self.property = Property(id_nov)

class Property:

    print('A Property class can be used')

    property_value = 'Wealth'

    def __init__(self, id_nov):
        print('Constructing a Property!')
        self.id_no = id_nov
        print(self.id_no)

    def evaluate(self, worthv):
        print(Property.property_value)
        self.worth = worthv
        print(self.worth)
```

Μια ενδεχόμενη χρήση της σύνθετης κλάσης είναι η παρακάτω:

```
An Owner class can be used
A Property class can be used
>>> tim = Owner(123)
Constructing an Owner!
Constructing a Property!
123
>>> tim.property.evaluate('High')
Wealth
High
```

Η μόνη παρατήρηση που έχουμε να κάνουμε στο παραπάνω είναι στη γραμμή `tim.property.evaluate('High')`. Βλέπουμε ότι μπορούμε να έχουμε πρόσβαση στα μέλη των κλάσεων των αντικειμένων-μελών, αρκεί να τα προσδιορίσουμε. Εδώ, εφαρμόσαμε τη μέθοδο στιγμιοτύπου `evaluate` της κλάσης `Property` στο αντικείμενο της κλάσης στο οποίο αναφέρεται η μεταβλητή στιγμιοτύπου `property` της κλάσης `Owner` για το αντικείμενο στο οποίο αναφέρεται η μεταβλητή `tim`.

10.2.3.5 Κληρονομικότητα

Εφόσον συζητήσαμε τον ορισμό των απλών κλάσεων, στην παράγραφο αυτή θα συζητήσουμε σχετικά με την κληρονομικότητα μεταξύ κλάσεων, όπως αυτή υποστηρίζεται από την Python. Παρότι εμείς δεν θα επιμείνουμε στο θέμα αυτό, να πούμε ότι η Python υποστηρίζει πολλαπλή κληρονομικότητα. Μάλιστα, έχει καλά ορισμένη διαδικασία προκειμένου να μην τίθενται θέματα σύγκρουσης κληρονομημένων ονομάτων.

Η κληρονομικότητα μεταξύ δύο κλάσεων αναπαριστάνει τη σχέση *isa*. Ένα αντικείμενο υποκλάσης είναι και αντικείμενο υπερκλάσης της. Κατά συνέπεια, κάθε αντικείμενο μιας κλάσης μπορούμε να το χειριστούμε μέσω των μεθόδων της κλάσης του, αλλά και μέσω των μεθόδων που κληρονομεί η κλάση του από τις υπερκλάσεις της.

Γενικότερα, έχουμε να κάνουμε τις εξής παρατηρήσεις, σχετικά με την κληρονομικότητα:

- Πέρα από τις μεταβλητές στιγμιοτύπου της κλάσης του, ένα αντικείμενο σχετίζεται και με τις μεταβλητές στιγμιοτύπου που κληρονομεί από τις υπερκλάσεις της.
- Μια κλάση κληρονομεί και τις μεταβλητές και τις μεθόδους κλάσης της υπερκλάσης της.
- Όταν πρόκειται να εκτελεστεί ο ορισμός μιας κλάσης από τον διερμηνέα, πρώτα εκτελείται ο ορισμός της υπερκλάσης της κλάσης (αναδρομικά για όλο το βάθος της ιεραρχίας).
- Το ίδιο ισχύει και κατά την αρχικοποίηση ενός αντικειμένου: πρώτα εκτελείται η διαδικασία αρχικοποίησης της υπερκλάσης της κλάσης του.

Στα παραπάνω, δεν αναφερθήκαμε στην περίπτωση που μια κλάση επαναορίζει μεθόδους ή/και μεταβλητές της υπερκλάσης της. Στην περίπτωση αυτή, τα ονόματά τους δεν είναι άμεσα προσβάσιμα στην υποκλάση. Για να εκτελεστεί μια μέθοδος της υπερκλάσης όταν έχει επαναοριστεί στην υποκλάση, κατά την κλήση της από το περιβάλλον της υποκλάσης πρέπει να χρησιμοποιηθεί η συνάρτηση `super()`. Η συνάρτηση αυτή επιστρέφει ένα αντικείμενο που μεταφέρει την κλήση της μεθόδου στη γονεϊκή κλάση. Η συνάρτηση `super()` μπορεί να χρησιμοποιηθεί τόσο για μεθόδους, όσο και για μεταβλητές. Ιδιαίτερη προσοχή χρειάζεται ο ορισμός των διαδικασιών αρχικοποίησης της υποκλάσης, αν υπάρχει ορισμός διαδικασιών αρχικοποίησης στην υπερκλάση (ή ανώτερα στην ιεραρχία). Στο σώμα της μεθόδου `__init__()` που ορίζεται στην υποκλάση πρέπει να γίνει ρητή κλήση στην επιθυμητή μέθοδο αρχικοποίησης της υπερκλάσης με χρήση της συνάρτησης `super()`.

Ας δούμε τα παραπάνω μέσω ενός παραδείγματος. Δεδομένου ότι όνομα έχουν όλα τα (οικόσιτα) ζώα, θα αλλάξουμε λίγο τον ορισμό της κλάσης `Dog` που είδαμε προηγουμένως και θα την ορίσουμε ως υποκλάση μιας κλάσης που αναπαριστάνει τα (οικόσιτα) ζώα, την `Animal`. Οι ορισμοί τους θα μπορούσαν να είναι ως εξής:

```
class Animal:
    'A Super Class'

    print('Animal at large')

    __no_of_insts = 0

    def __init__(self, nam):
        self._name = nam
        print('Constructing an animal!')
```

```
Animal.__no_of_insts = Animal.__no_of_insts + 1

def print(self):
    print('The name is: ', self._name)

@classmethod
def no_insts(cls):
    print('No of created Animals:', cls.__no_of_insts)

#####
class Dog(Animal):
    "A Subclass of Animal"

    print('A Dog class can be used')

    _dog_sound = 'Barking'
    __no_of_insts = 0

    print('Dog sound is: ', _dog_sound)

    def __init__(self, nam):
        super().__init__(nam)
        print('Constructing a Dog!')
        Dog.__no_of_insts = Dog.__no_of_insts + 1

    def bark(self, soundv):
        self._sound = soundv
        print(Dog._dog_sound, self._sound)

    def print(self):
        super().print()
        print('The Dog\'s sound is: ', self._sound)

@classmethod
def no_insts(cls):
    print('No of created Dogs:', cls.__no_of_insts)
```

Στο παραπάνω βλέπουμε ότι η κλάση Dog:

- Ορίζεται ως υποκλάση της κλάσης Animal με την έκφραση `class Dog(Animal)`.
- Προσθέτει μεταβλητές κλάσης, τις `_dog_sound` και νέα `__no_of_insts`.
- Προσθέτει νέα μέθοδο στιγμιοτύπου, την `bark`.
- Επαναορίζει τη μέθοδο στιγμιοτύπου `print`. Στον ορισμό της καλεί την `print` που είναι ορατή στη γονεϊκή της κλάση, με χρήση της συνάρτησης `super()`.
- Στη μέθοδο αρχικοποίησής της, καλεί εκείνη της γονεϊκής της, με χρήση της συνάρτησης `super()`.
- Επαναορίζει τη μέθοδο κλάσης `no_insts`.

Με βάση τους παραπάνω ορισμούς κλάσεων και τις παρατηρήσεις που κάναμε, προσπαθήστε να αιτιολογήσετε την παρακάτω εικόνα από την εκτέλεση του διερμηνέα της Python μετά την εκτέλεση των ορισμών.

```
Animal at large
A Dog class can be used
Dog sound is: Barking
>>> dog1 = Dog('Azor')

Constructing an animal!
Constructing a Dog!
>>> dog1.bark('Loud')

Barking Loud
>>> dog1.print()

The name is: Azor
The Dog's sound is: Loud
>>> dog1.no_insts()
```

```
No of created Dogs: 1
>>> an1 = Animal('Cute')
```

```
Constructing an animal!
>>> an1.print()
```

```
The name is: Cute
>>> an1.no_insts()
```

```
No of created Animals: 2
```

Να κάνουμε μια παρατήρηση που αφορά τα ονόματα που ορίζονται μέσα σε μία κλάση. Θα παρατηρήσατε ότι, για παράδειγμα, το όνομα `__no_of_insts` ξεκινά με `__`. Στην περίπτωση αυτή, η Python τα μετονομάζει προσθέτοντας στο νέο όνομα την κλάση τους ώστε να μη γίνει σύγχυση κατά την κληρονομικότητα. Η τεχνική αυτή ονομάζεται *mangling* και μας εμποδίζει να προσπελάσουμε άμεσα το όνομα που προέρχεται από την υπερκλάση. Καθότι όμως η Python δεν έχει απόκρυψη πληροφορίας, οπότε μπορεί να χρησιμοποιηθεί το τροποποιημένο όνομα, η χρήση της αμφισβητείται, διότι μειώνει την αναγνωσιμότητα του κώδικα.

Θα δώσουμε ακόμα ένα παράδειγμα κληρονομικότητας. Θα ορίσουμε μια κλάση “Εργαζόμενος Φοιτητής” ως υποκλάση μιας παραλλαγής της κλάσης `Student` που ορίσαμε προηγουμένως. Με βάση αυτά που συζητήσαμε προηγουμένως, προσπαθήστε να κατανοήσετε τους παρακάτω ορισμούς.

```
class Student:
    "A Student class"

    _no_insts = 0
    print("students so far: ", _no_insts)

    _highest_GPA = 0
    print("highest GPA so far: ", _highest_GPA)

    PASS_GRADE = 5
```

```

TOTAL_COURSES = 40

def __init__(self, nov):
    self._no = nov
    self._no_completed_courses = 0
    self._completed_courses = []
    Student._no_insts = Student._no_insts + 1
    print("no", self._no, "\n")
    print("courses", self._completed_courses, "\n")
    print("students so far: ", Student._no_insts)

def get_no_student(self):
    return self._no

def record_grade(self, grade):
    if grade >= Student.PASS_GRADE :
        self._no_completed_courses = self._no_completed_courses + 1
        self._completed_courses.append(grade)

def graduate(self):
    return self._no_completed_courses == Student.TOTAL_COURSES

def evaluate_GPA(self):
    self._GPA = 0
    if self._no_completed_courses > 0 :
        self._GPA = sum(self._completed_courses) \
            / self._no_completed_courses
        if self.graduate() :
            if self._GPA > Student._highest_GPA :
                Student._highest_GPA = self._GPA
    return self._GPA

@classmethod
def get_highest_GPA(cls):
    return cls._highest_GPA

@classmethod
def get_no_insts(cls):
    return cls._no_insts

#####
class WorkingStudent(Student):
    "A subclass of Student"

    _no_insts = 0
    print("working students so far: ", _no_insts)

    def __init__(self, nov, salaryv, monthsv):
        super().__init__(nov)

```

```

        WorkingStudent._no_insts = WorkingStudent._no_insts + 1
        print("working students so far: ", WorkingStudent._no_insts)
        print("Students so far from WS with super(): ", \
              super()._no_insts)
        print("Students so far from WS: ", Student._no_insts)
        self._salary = salaryv
        self._months = monthsv

    def get_income(self):
        return self._salary * self._months

```

Παρατηρήστε τη συμπεριφορά της μεθόδου κλάσης `get_no_insts` από την παρακάτω εκτέλεση. Στην περίπτωση που αυτή καλείται μέσω αντικειμένου της θυγατρικής κλάσης, παρότι η μέθοδος ανήκει στη γονεϊκή κλάση, χρησιμοποιείται η μεταβλητή της κλάσης του αντικειμένου.

```

>>> stud1 = Student(100)
no 100

courses []

students so far:  1
>>> ws1 = WorkingStudent(200,800,5)
no 200

courses []

students so far:  2
working students so far:  1
Students so far from WS with super():  2
Students so far from WS:  2
>>> ws2 = WorkingStudent(300,1000,3)
no 300

courses []

students so far:  3
working students so far:  2
Students so far from WS with super():  3
Students so far from WS:  3
>>> ws2.get_no_insts()
2
>>> WorkingStudent.get_no_insts()
2
>>> Student.get_no_insts()
3

```

Αναφερθήκαμε ήδη στη σημασία που δίνει η Python στην πολλαπλή κληρονομικότητα. Αυτή εκφράζεται κατά τον ορισμό μιας κλάσης όταν στην παρένθεση της υπερκλάσης συμπεριλάβουμε πάνω από μία κλάσεις. Π.χ.

```
class Sub(Sup1,Sup2, ..., SupN)
```


Με τον τρόπο αυτό ορίζεται η κλάση *Sub* ως υποκλάση των κλάσεων *Sup1*, *Sup2*, ..., *SupN*. Η σειρά με την οποία γράφουμε τις υπερκλάσεις έχει σημασία. Ως προς τα κληρονομημένα ονόματα, η αναζήτηση γίνεται κατά-βάθος-πρώτα από-αριστερά-προς-τα-δεξιά με παράλειψη διπλής συμμετοχής κλάσεων στη γραμμικοποίηση αυτή. Αν μας ενδιαφέρει να επέμβουμε ως προς την επιλογή, μπορούμε να το κάνουμε χρησιμοποιώντας παραλλαγές της συνάρτησης `super()`.

10.2.4 Τελευταίες Παρατηρήσεις

Η Python προσφέρει πάρα πολλές δυνατότητες στον προγραμματιστή βασισμένες στο αντικειμενοστραφές μοντέλο. Κλασική περίπτωση είναι το σύστημα Εισόδου/Εξόδου της. Επίσης, ο χειρισμός εξαιρέσεων είναι από τα πρωταρχικά της στοιχεία.

Κλείνοντας, να τονίσουμε ότι η Python προσφέρει ένα πλήθος συναρτήσεων που επιτρέπουν να διαχειριστεί ο προγραμματιστής τα αντικείμενα του προγράμματός του. Ως τέτοια είναι και οι ίδιες οι κλάσεις του. Σε αυτό το επίπεδο μεταπρογραμματισμού, η Python δίνει πολλές δυνατότητες και αυτό μπορεί να αποτελέσει ένα κριτήριο για την επιλογή της, αν το αντικείμενο της εφαρμογής προς ανάπτυξη εμπίπτει στην περιοχή αυτή. Ο ενδιαφερόμενος αναγνώστης καλείται να τις μελετήσει από την τεκμηρίωση της γλώσσας.

10.3 Μια Ματιά σε Γλώσσες Συναρτησιακού Προγραμματισμού

Όπως αναφέρθηκε και στην αρχή του βιβλίου, το περιεχόμενό του εστιάζει στον αντικειμενοστραφή διαδικαστικό προγραμματισμό, που είναι η πιο διαδεδομένη μορφή αντικειμενοστραφούς προγραμματισμού. Στις μέρες μας, όμως, μεγάλη άνθηση και χρήση σε εφαρμογές, ακόμα και εμπορικά προϊόντα, έχουν και γλώσσες αντικειμενοστραφούς συναρτησιακού προγραμματισμού. Μια σύνοψη σχετικά με το αντικείμενο του συναρτησιακού προγραμματισμού κάναμε στην Παράγραφο 1.5. Εδώ, θα δώσουμε λίγες ακόμα πληροφορίες γι' αυτό το είδος προγραμματισμού.

Ο “καθαρός” συναρτησιακός προγραμματισμός (ΣΠ), όπως υποστηρίζεται από τις σύγχρονες γλώσσες του είδους του, έχει τα εξής χαρακτηριστικά. Το βασικό εργαλείο του είναι οι (μαθηματικές) *συναρτήσεις*, των οποίων το μαθηματικό υπολογιστικό μοντέλο είναι ο λάμδα λογισμός, *λ-λογισμός*. Ο *λ-λογισμός* προτάθηκε από τον μαθηματικό Alonzo Church τη δεκαετία του '30, στο πλαίσιο της έρευνάς του σχετικά με τις θεμελιώσεις των μαθηματικών. Οι συναρτήσεις του ΣΠ εφαρμόζονται σε μαθηματικές μεταβλητές. Στις μεταβλητές αυτές ανατίθεται μια τιμή άπαξ, είναι, δηλαδή, αμετάβλητες.

Ο ΣΠ θεωρείται είδος *δηλωτικού προγραμματισμού* (*declarative programming*). Προγραμματίζοντας σε μια γλώσσα δηλωτικού προγραμματισμού την επίλυση ενός προβλήματος, περιγράφουμε το *τι ισχύει* στον κόσμο του προβλήματός μας και *όχι ποιες ενέργειες* θα μας οδηγήσουν στη λύση του. Ο ΣΠ και ο Λογικός Προγραμματισμός είναι από τα πιο διαδεδομένα είδη δηλωτικού προγραμματισμού. Ενώ ο ΣΠ βασίζεται στις συναρτήσεις, ο Λογικός Προγραμματισμός βασίζεται στις *σχέσεις* (*relations*).

Ένα πρόγραμμα σε μια γλώσσα ΣΠ είναι μια μαθηματική έκφραση, η οποία εμπλέκει συναρτήσεις και μεταβλητές. Η εκτέλεση του προγράμματος έγκειται στον υπολογισμό της έκφρασης για συγκεκριμένες τιμές των μεταβλητών αυτών. Πέρα από τη διευκόλυνση στην ανάπτυξη και την αναγνωσιμότητα ενός προγράμματος, ο ΣΠ επιτρέπει σταδιακή ανάπτυξη, εύκολη παραλληλοποίηση και, κυρίως, μαθηματική επαλήθευση της ορθότητάς του. Παραδείγματα γλωσσών ΣΠ της κατηγορίας που αναφέραμε είναι η γλώσσα *Hope* (Edinburgh University, δεκαετία του '70), η γλώσσα *Miranda* (δημιουργός David Turner, 1985), η γλώσσα *Haskell* (University of Glasgow, 1990) κ.ά. Εδώ, να κάνουμε την παρατήρηση ότι οι γλώσσες ΣΠ συνήθως υποστηρίζουν “οκνηρό υπολογισμό” (*lazy evaluation*). Δηλαδή, οι υπολογισμοί εκφράσεων γίνονται μόνο στον βαθμό που είναι απαραίτητοι. Στον αντίποδα βρίσκεται η έννοια του “αυστηρού” (*strict*) υπολογισμού, όπου κάθε έκφραση ορίσματος μιας συνάρτησης υπολογίζεται πλήρως, ακόμα και αν αυτό δεν είναι απαραίτητο.

Όπως είπαμε στην Παράγραφο 1.5, ήδη από τη δεκαετία του '70, εμφανίστηκαν γλώσσες συναρτησιακού προγραμματισμού με αντικειμενοστραφή χαρακτηριστικά (π.χ. *Flavors*). Προκειμένου να δώσουμε την ευ-

καιρία στον αναγνώστη να έχει μια πρώτη εικόνα για τη μορφή που έχουν τα αντικειμενοστραφή χαρακτηριστικά ενταγμένα σε σύγχρονες γλώσσες συναρτησιακού προγραμματισμού, στη συνέχεια, θα κάνουμε μια σύντομη αναφορά σε δύο διαδεδομένες γλώσσες του είδους αυτού: τη *Scala* και την *F#*. Πέρα από τη διάδοσή τους, ο λόγος της επιλογής τους σχετίζεται και με το τι έχει προηγηθεί στο βιβλίο. Συγκεκριμένα, η γλώσσα *Scala* μεταφράζεται σε JVM bytecode εντολές και η *F#* είναι συναρτησιακή γλώσσα για το πλαίσιο .NET.

10.3.1 Η Γλώσσα Scala

Η γλώσσα *Scala* είναι μια γλώσσα αντικειμενοστραφούς συναρτησιακού προγραμματισμού. Ο όρος “αντικειμενοστραφής” αναφέρεται τόσο στα προγραμματιστικά χαρακτηριστικά της, όσο και στο μοντέλο της. Σύμφωνα με τον δημιουργό της, τον Martin Odersky, η *Scala* σχεδιάστηκε με στόχο να αποδείξει ότι “η συνένωση συναρτησιακού και αντικειμενοστραφούς προγραμματισμού είναι και εφικτή και πρακτική”. Στις βασικές σχεδιαστικές αρχές της ήταν η απλότητα στον προγραμματισμό και στις απαιτήσεις από το υλικό, αλλά και η κλιμάκωση της πολυπλοκότητάς τους με εκμετάλλευση των δυνατοτήτων της γλώσσας. Εξ ου και το όνομά της, “*Scala*” — λόγω κλιμάκωσης.

Η *Scala* δημιουργήθηκε από τον Γερμανό επιστήμονα υπολογιστών Martin Odersky και πρωτοεμφανίστηκε το 2003. Εξακολουθεί να αναπτύσσεται υπό την επίβλεψή του. Πρόσφατος επανασχεδιασμός της έγινε το 2020. Ο Odersky είχε ενδιαφέρον για τον χώρο των μεταγλωττιστών από φοιτητής. Έκανε το διδακτορικό του με τον Niklaus Wirth στο πολυτεχνείο ETH της Ζυρίχης. Ο Wirth είναι ο “πατέρας” της γλώσσας *Pascal* και της *Modula-2*, γλώσσες με υλοποίηση των οποίων ο Odersky είχε προηγουμένως ασχοληθεί. Κατά τη μεταδιδακτορική ενασχόλησή του, ανέπτυξε δραστηριότητα σχετική με τον συναρτησιακό προγραμματισμό, τον οποίο γνώρισε στο τέλος της δεκαετίας του '80 και ο οποίος τον συνεπήρε. Το διάστημα αυτό ενημερώθηκε για την προσπάθεια σχετικά με τη γλώσσα *Java* και για τη JVM και κατέληξε να συνεργαστεί στην ανάπτυξή τους. Η ενασχόλησή του αφορούσε την ένταξη των γενικευμένων τύπων στην *Java*, αλλά και τον ίδιο τον μεταγλωττιστή της γλώσσας. Όταν η απασχόλησή του ολοκληρώθηκε, συνέχισε την ερευνητική δραστηριότητά του, μεταβαίνοντας στο πολυτεχνείο EPFL της Λωζάνης. Εκεί, απαλλαγμένος από τα σχεδιαστικά πλαίσια της *Java*, ξεκίνησε τις προσπάθειές του για τον σχεδιασμό μιας γλώσσας με τα χαρακτηριστικά που εκείνος ήθελε. Όμως, υιοθέτησε τη χρήση της JVM, μια που είχε μεγάλη εμπειρία σχετικά με αυτήν, αλλά και εκτιμούσε πολύ τις δυνατότητές της. Ξεκινώντας από πιο ακαδημαϊκές γλώσσες που, όμως, δεν θα είχαν μεγάλο πρακτικό ενδιαφέρον, το 2002 ξεκίνησε τον σχεδιασμό μιας γλώσσας πιο κατάλληλης για πραγματικές εφαρμογές, τη *Scala*. Η *Scala* είναι παράγωγο της δραστηριότητας του Odersky στο EPFL, κάτι που αντικατοπτρίζεται και στο λογότυπό της: είναι μια σκάλα του Πολυτεχνείου.

Τα προγράμματα *Scala* μεταφράζονται σε JVM bytecode εντολές και εκτελούνται από τη JVM. Κατά συνέπεια, η *Scala* και η *Java* έχουν κοινό σύστημα εκτέλεσης και βασίζονται στο ίδιο εσωτερικό σύστημα τύπων. Για τον λόγο αυτό, πληθώρα βιβλιοθηκών που έχουν αναπτυχθεί για την *Java* είναι διαθέσιμες και στον προγραμματιστή της *Scala*. Παρότι υπάρχει εύκολη συνεργασία μεταξύ της *Scala* και της *Java*, ο αναγνώστης δεν πρέπει να αναζητά στη *Scala* μια εκδοχή της *Java* για τον συναρτησιακό προγραμματισμό, ούτε να περιμένει απόλυτη ταύτιση των χαρακτηριστικών των δύο γλωσσών. Αργότερα, αναπτύχθηκε ο μεταγλωττιστής *Scala.js*, που μεταφράζει προγράμματα *Scala* σε *JavaScript*, ώστε να μπορούν να εκτελεστούν σε πλοηγούς web.

Όπως είπαμε, στην περίπτωση της *Scala*, η χρήση της JVM ήταν επιλογή για την υποστήριξη μιας γλώσσας με τα χαρακτηριστικά που ήθελε ο Odersky. Αυτό δεν ισχύει στην περίπτωση της *F#*, που θα δούμε στην συνέχεια: το πλαίσιο ήταν δεδομένο, το .NET, και έπρεπε να αναπτυχθεί μια συναρτησιακή γλώσσα για το πλαίσιο αυτό.

Για τη *Scala*, προσφέρεται ένα πλήθος βιβλιοθηκών, π.χ. για Ανάλυση Δεδομένων, Μεγάλα Δεδομένα (Big Data), Τεχνητή Νοημοσύνη και Μηχανική Μάθηση κ.ά.

Η *Scala* υποστηρίζει προγραμματισμό γενικεύσεων. Κρίσιμα χαρακτηριστικά της γλώσσας σχετίζονται με τους γενικευμένους τύπους.

10.3.1.1 Βασικά Στοιχεία της Γλώσσας

Η Scala στοχεύει σε απλότητα στη χρήση της και σε φιλική σύνταξη των γλωσσικών δομών της. Επίσης, η Scala έχει στατικό σύστημα τύπων που συμβάλλει στην ασφάλεια ενός προγράμματος. Μπορούμε να παραλείψουμε να δηλώσουμε τον τύπο μιας προγραμματιστικής οντότητας, όμως. Ο τύπος της θα εξαχθεί αυτόματα κατά τη φάση της μεταγλώττισης. Η Scala δεν επιτρέπει χρήση δεικτών και ρητή διαχείριση μνήμης, κάτι που συμβάλλει επιπλέον στην αποτροπή λαθών.

Κάθε τι στη Scala είναι μια έκφραση που έχει μια τιμή. Όλα τα δεδομένα ενός προγράμματος Scala είναι στιγμιότυπα μιας κλάσης και κάθε ενέργεια (ή τελεστής) που εφαρμόζεται επάνω σε αυτά είναι μέθοδος. Η εκτέλεση ενός προγράμματος ξεκινά με την κλήση μιας μεθόδου που έχει χαρακτηριστεί ως κύρια μέθοδος.

Οι τύποι της Scala δομούν μια ιεραρχία με ρίζα τον τύπο/κλάση `Any`. Κάθε τύπος είναι υποκλάση του `Any`. Υπάρχει και ο τύπος `Nothing`, ο οποίος είναι υποτύπος κάθε τύπου, αλλά καμία τιμή δεν ανήκει στον τύπο αυτό — χρησιμεύει ως τύπος εκφράσεων που χρησιμοποιούνται σε περιπτώσεις ανώμαλης συμπεριφοράς, όπως μη-τερματισμός. Οι υποτύποι του τύπου `Any` διακρίνονται στις κατηγορίες `AnyVal` και `AnyRef`. Ο τύπος `AnyVal` εκφράζει τους τύπους τιμής. Υποτύποι του τύπου `AnyVal` είναι τύποι της γλώσσας, όπως οι τύποι των αριθμών, ο τύπος `Boolean`, ο τύπος `Char` και ο τύπος `Unit`. Ο τύπος `Unit` είναι ο τύπος που εκφράζει έλλειψη πληροφορίας και έχει τιμή μόνο την `()`. Χρησιμοποιείται ως τύπος επιστροφής συναρτήσεων — είναι το αντίστοιχο του τύπου `void`. Ο τύπος `AnyRef` εκφράζει τους τύπους αναφοράς. Στο πλαίσιο του περιβάλλοντος εκτέλεσης της Java, ο τύπος `AnyRef` αντιστοιχεί στον τύπο `java.lang.Object`. Ο τύπος `String` είναι υποτύπος του `AnyRef`. Το ίδιο ισχύει και για κάθε τύπο που ορίζεται από τον προγραμματιστή. Τέλος, υποτύπος όλων των υποτύπων του `AnyRef` είναι ο τύπος `Null`, που έχει μόνο ένα στιγμιότυπο με τιμή `null`. Με βάση την παραπάνω κατηγοριοποίηση τύπων, βλέπουμε ότι η Scala δεν κάνει διάκριση σε πρωτογενείς τύπους και τύπους αναφοράς: τα πάντα βρίσκονται κάτω από τον τύπο `Any`. Από την καθιερωμένη βιβλιοθήκη της Scala προσφέρονται διάφοροι αμετάβλητοι και μεταβλητοί τύποι συλλογής και γενικευμένοι τύποι. Τέλος, να πούμε ότι, για κάποιους τύπους, είναι ορισμένες αυτόματες μετατροπές τύπων.

Τα δεδομένα ενός προγράμματος Scala μπορεί να είναι είτε *αμετάβλητα* είτε *μεταβλητά*. Προκειμένου να δηλώσουμε ότι ένα όνομα ονοματίζει κάτι αμετάβλητο, χρησιμοποιούμε τη λέξη-κλειδί `val`. Για παράδειγμα,

```
val x = 5
```

Με την παραπάνω δήλωση εκφράζουμε ότι το `x` πήρε την τιμή `5` και αυτή δεν μπορεί να αλλάξει. Αν θέλουμε να ορίσουμε μια μεταβλητή που μπορεί να αλλάξει τιμές, πρέπει να χρησιμοποιήσουμε τη λέξη-κλειδί `var`. Για παράδειγμα,

```
var a = 10
```

Η μεταβλητή `a` μπορεί να αλλάξει τιμή στη συνέχεια, για παράδειγμα, να έχουμε την εντολή

```
a += 5
```

Στα παραπάνω δεν δηλώσαμε τύπο — θα εξαχθεί αυτόματα. Αν θέλαμε, όμως, μπορούμε να τον δηλώσουμε ρητά, π.χ.

```
val x: Int = 5
```

σε αυτήν την περίπτωση δηλώσαμε εμείς ότι το `x` έχει τιμή ακεραίου.

Με χρήση της λέξης-κλειδιού `val` μπορούμε να ορίσουμε και *συναρτήσεις* στη Scala. Για παράδειγμα,

```
val double = (x: Int) => 2*x
```

Στο παραπάνω, ορίσαμε μια συνάρτηση, την `double`. Αυτή έχει ένα όρισμα, το `x`, που έχει δηλωθεί ότι είναι τύπου ακεραίου. Η τιμή της είναι η έκφραση που ακολουθεί το σύμβολο `=>`. Με βάση την έκφραση αυτή, θα υπολογιστεί η τιμή της εφαρμογής της σε συγκεκριμένη τιμή του ορίσματος. Οι συναρτήσεις που ορίζονται μέσω `val` έχουν αντίστοιχο νόημα με τις μαθηματικές συναρτήσεις: για δεδομένες τιμές παραμέτρων, έχουν πάντα την ίδια τιμή, αυτή που προκύπτει από τον υπολογισμό της έκφρασης του ορισμού τους. Οι συναρτήσεις είναι αντικείμενα και, ως τέτοια, αποτελούν κύριες οντότητες του προγράμματος.

Στη Scala έχουμε τη δυνατότητα να ορίσουμε *μεθόδους* με χρήση της λέξης-κλειδιού `def`. Οι μέθοδοι ορίζονται μέσα σε κλάσεις και θεωρείται ότι εφαρμόζονται σε στιγμιότυπά τους. Στη Scala, όμως, μπορούμε να ορίσουμε και μεθόδους εκτός κλάσεων. Ως τέτοιες, μπορούμε να δηλώσουμε συναρτήσεις για τις οποίες δεν

μπορούμε να διασφαλίσουμε ότι για τις ίδιες τιμές ορισμάτων, το αποτέλεσμα της αποτίμησής τους είναι πάντα το ίδιο — το αποτέλεσμα της αποτίμησης είναι η τιμή της τελευταίας έκφρασης του ορισμού τους. Στα ορίσματα των μεθόδων, μπορούμε να έχουμε default τιμές. Μπορεί, ακόμα, να μην έχουμε καθόλου ορίσματα. Κατά την κλήση των μεθόδων, μπορούμε να αναφερθούμε στα ορίσματά τους με το όνομά τους. Σε κάποιες περιπτώσεις, τα ορίσματα μπορεί να παραλειφθούν, αν το γενικότερο περιεχόμενο μπορεί να καθορίσει την τιμή τους. Τέλος, να πούμε ότι μπορούμε να έχουμε *πολυμορφικές* μεθόδους, που ορίζονται με χρήση παραμέτρων τύπου.

Τόσο για τις συναρτήσεις, όσο και για τις μεθόδους, με χρήση {, }, μπορούμε να ορίσουμε εκφράσεις ως ομαδοποίηση εκφράσεων. Η Scala προσφέρει διάφορες γλωσσικές δομές για την έκφραση ελέγχου. Σημαντικό, όμως, για τη διαδικασία υπολογισμού εκφράσεων είναι το *ταίριασμα προτύπων*, όπως και σε κάθε άλλη γλώσσα ΣΠ. Αρκετή από τη λειτουργικότητα που προσφέρει η Scala σχετίζεται με το ταίριασμα προτύπων.

Όπως και για κάθε άλλη γλώσσα ΣΠ, σημαντικές είναι και οι δυνατότητες ορισμού συναρτήσεων ανώτερης τάξης (higher order), σύνθεσης συναρτήσεων κ.ά. Στο παρακάτω, δίνουμε ένα παράδειγμα ορισμού μιας συνάρτησης ανώτερης τάξης, την `doublefun`, και την εφαρμογή της για τη συνάρτηση `double` που ορίσαμε προηγουμένως.

```
val doublefun = (x:Int, fun:Int=>Int) => 2*fun(x)
println(doublefun(5,double))
```

Να κλείσουμε την παράγραφο αυτή, αναφέροντας ότι όλος ο κώδικας ενός προγράμματος Scala, καθώς και οι βιβλιοθήκες της, είναι οργανωμένος σε *πακέτα*, προκειμένου να δημιουργηθούν χώροι ονομάτων. Τα πακέτα μπορεί να είναι εμφωλευμένα. Πρόσβαση σε ονόματα ενός πακέτου γίνεται με χρήση του ονόματος του πακέτου. Μπορούμε να απαλλαγούμε από τον προσδιορισμό, κάνοντας *εισαγωγή* του πακέτου ή επιλεκτικά. Στη Scala, για κάθε πακέτο, μπορούμε να ορίσουμε ένα *αντικείμενο πακέτου* (package object), στο οποίο να δίνουμε διάφορους ορισμούς μεταβλητών, μεθόδων κλπ. που να ανήκουν απευθείας στην εμβέλεια του πακέτου. Τα αντικείμενα πακέτου συμπεριφέρονται όπως κάθε οντότητα δηλωμένη με `object`, π.χ. συμμετέχουν σε κληρονομικότητα.

10.3.1.2 Αντικειμενοστραφής Προγραμματισμός

Προκειμένου να υποστηρίξει αντικειμενοστραφή προγραμματισμό σε προγραμματιστικό επίπεδο, η Scala δίνει τη δυνατότητα ορισμού *κλάσεων* και *αφηρημένων κλάσεων*, *τύπων αντικειμένων* (objects), *συντροφικών αντικειμένων* (companion objects) και *τύπων χαρακτηριστικών* (traits). Για εκμετάλλευση του πλαισίου του ΣΠ, προσφέρεται και η δυνατότητα ορισμού *τύπων απαριθμήσεων*, *κλάσεις περιπτώσεων* και *τύποι αντικειμένων περιπτώσεων*. Αυτοί είναι τύποι που συμπεριφέρονται, κυρίως, κατά τιμή και μεγάλος μέρος της συμπεριφοράς τους σχετίζεται με το ταίριασμα προτύπων. Δεν θα συζητήσουμε, λοιπόν, άλλο γι' αυτούς.

Για τις αφηρημένες κλάσεις, η Scala δίνει τη δυνατότητα δήλωσης *αφηρημένων μελών τύπου* (abstract type members), χρησιμοποιώντας μεταβλητές σε τύπους. Οι κλάσεις αυτές αναπαριστούν τύπους μερικά ορισμένους και όχι γενικευμένους.

Η Scala δεν προσφέρει τη δυνατότητα ορισμού στατικών κλάσεων και στατικών μελών κλάσεων με προσδιορισμό τους ως “στατική”/“στατικά”. Η λειτουργικότητα των στατικών κλάσεων επιτυγχάνεται με χρήση των τύπων αντικειμένου. Αντίστοιχα, η λειτουργικότητα των στατικών μελών μιας κλάσης επιτυγχάνεται με ορισμό συντροφικού αντικειμένου για την κλάση αυτή.

Προκειμένου να δημιουργηθούν στιγμιότυπα ενός τύπου, ο προγραμματιστής πρέπει να ορίσει μια κλάση. Κατά τον ορισμό μιας κλάσης, δηλώνεται το όνομά της και ένα πλήθος παραμέτρων, που μπορεί να έχουν default τιμές. Αυτές αντιστοιχούν στα ορίσματα της κύριας μεθόδου *αρχικοποίησης* (primary constructor). Ταυτόχρονα, όμως, δηλώνουν και πεδία της κλάσης, τα οποία θα αρχικοποιηθούν κατά τη δημιουργία του αντικειμένου με τις τιμές των παραμέτρων αυτών.

Στο σώμα της κλάσης μπορούμε να έχουμε ορισμούς μεθόδων, τιμών, μεταβλητών, αντικειμένων, τύπων χαρακτηριστικών και εσωτερικών κλάσεων. Αυτά αποτελούν τα *μέλη* της κλάσης. Μπορούμε, επίσης, να ορί-

σουμε και βοηθητικές μεθόδους αρχικοποίησης με χρήση της λέξης-κλειδιού `this`. Σε μια αφηρημένη κλάση, μπορεί να έχουμε και αφηρημένα μέλη, μη δίνοντας ορισμό τους. Οι εσωτερικές κλάσεις είναι συνδεδεμένες με αντικείμενα των εξωτερικών τους.

Μια πολύ απλή κλάση μπορεί να οριστεί μόνο από το όνομά της ή από το όνομά της συνοδευόμενο από τις παραμέτρους/πεδία. Για παράδειγμα, μπορεί να οριστεί μια κλάση `Bla` χωρίς κανένα μέλος ως εξής:

```
class Bla
```

Στη συνέχεια, μπορούμε να ορίσουμε αντικείμενά της. Αυτό γίνεται χρησιμοποιώντας το όνομα της κλάσης — και στην προκειμένη περίπτωση, με κενή λίστα ορισμάτων —, δηλαδή:

```
val bc = Bla()
```

Στο παραπάνω, το αντικείμενο που δημιουργήθηκε δόθηκε ως τιμή στην `bc`.

Τα επίπεδα ορατότητας μελών κλάσης διακρίνουν τα μέλη μιας κλάσης σε: δημόσια, προστατευόμενα και ιδιωτικά, με τα συνήθη δικαιώματα πρόσβασης. Ο προσδιορισμός γίνεται ανά μέλος. Αν παραλείπεται, το μέλος αυτόματα θεωρείται δημόσιο. Τα πεδία που αντιστοιχούν στις παραμέτρους της κλάσης είναι αυτόματα ιδιωτικά. Αν, όμως, στη δήλωση έχουν προσημανθεί με `var` ή `val`, γίνονται δημόσια, με δικαίωμα αλλαγής τιμής ή όχι, αντίστοιχα. Για παράδειγμα, μπορούμε να έχουμε το παρακάτω:

```
class TestC(var x: Int)

var testVC = TestC(5)
println(testVC.x)
testVC.x += 5
println(testVC.x)
```

Η εκτύπωση θα μας δώσει, διαδοχικά, τις τιμές 5 και 10.

Ας δούμε ένα πιο ολοκληρωμένο παράδειγμα απλής κλάσης.

```
class Animal(var name : String):
    private var _message : String = "NoMessage"
    def print() : Unit =
        println(s"$name say $_message")
    def this(name : String, message: String) =
        this(name)
        _message = message
        println(message)
```

Στο παραπάνω, ορίζουμε μια κλάση `Animal` με ένα όρισμα/πεδίο, μία επιπλέον μέθοδο αρχικοποίησης με δύο ορίσματα και με μία μέθοδο `print`. Επίσης, έχει ένα ιδιωτικό πεδίο, το `_message`. Αν δημιουργήσουμε ένα αντικείμενο με τον παρακάτω τρόπο:

```
var an1 = Animal("AnName", "AnMessage")
an1.print()
```

η εκτύπωση της εκτέλεσης του παραπάνω είναι:

```
AnMessage
AnName say AnMessage
```

Αν δημιουργήσουμε ένα αντικείμενο με τον παρακάτω τρόπο:

```
var an1 = Animal("AnName")
an1.print()
```

η εκτύπωση της εκτέλεσης του παραπάνω κώδικα είναι:

AnName say NoMessage

Ας δούμε τώρα το θέμα της κληρονομικότητας. Στη Scala, η κληρονομικότητα εκφράζει την *isa* σχέση μεταξύ κλάσεων, της οποίας ρητή δήλωση γίνεται με χρήση της λέξης-κλειδιού `extends`. Μια κλάση μπορεί να δηλώσει ρητά ότι είναι υποκλάση το πολύ μιας κλάσης. Στη διεπαφή της δήλωσης, δηλώνουμε και τη μέθοδο αρχικοποίησης της υπερκλάσης που θα χρησιμοποιηθεί, προσδιορίζοντας τα ορίσματά της.

Ας δούμε ένα απλό παράδειγμα κληρονομικότητας. Ας θεωρήσουμε την προηγούμενη κλάση `Animal`. Κάθε ζώο, όμως, ανήκει σε κάποιο είδος, άρα πιο σωστό είναι να μην έχουμε στιγμιότυπά της, οπότε ας τη δηλώσουμε ως αφηρημένη. Ας ορίσουμε μια νέα κλάση `Dog` ως υποκλάση της `Animal`. Η κλάση `Dog` προσθέτει ένα ιδιωτικό πεδίο, το `_sound_level`, και επαναορίζει τη μέθοδο `print`. Προκειμένου να κάνουμε τον επαναορισμό της μεθόδου, πρέπει να χρησιμοποιήσουμε τον προσδιορισμό `override`. Προκειμένου να έχουμε πρόσβαση σε ένα μέλος της υπερκλάσης που δεν είναι άμεσα ορατό, μπορούμε να χρησιμοποιήσουμε τη λέξη-κλειδί `super`. Αυτό κάνουμε μέσα στο σώμα της μεθόδου `print` προκειμένου να έχουμε πρόσβαση στη μέθοδο `print` της κλάσης `Animal`.

```
abstract class Animal(var name : String):
  private var _message : String = "NoMessage"
  def print() : Unit =
    println(s"$name say $_message")
  def this(name : String, message: String) =
    this(name)
    _message = message
    println(message)

class Dog(var sound_level : String, name : String, message: String)
extends Animal(name,message):
  private var _sound_level : String = sound_level
  override def print():Unit =
    super.print()
    println(sound_level)
```

Με βάση το παραπάνω, αν έχουμε επιπλέον τον εξής κώδικα:

```
var azorv = Dog("Loud", "Azor", "Barking")
azorv.print()
```

η εκτύπωση που προκύπτει, αν εκτελεστεί, είναι:

```
Barking
Azor say Barking
Loud
```

Ας αλλάξουμε λίγο τον ορισμό της κλάσης `Dog`, ώστε η μεταβλητή `_message` να αποτελέσει μέλος ενός συντροφικού αντικειμένου της κλάσης, μια που η πληροφορία αυτή σχετίζεται με την κλάση κι όχι ένα συγκεκριμένο στιγμιότυπο. Οι ορισμοί μπορεί να έχουν ως εξής:

```
object Dog:
  private val _message = "Barking"
class Dog(var sound_level : String, name : String)
extends Animal(name,Dog._message) :
  private var _sound_level : String = sound_level
```

```
override def print() : Unit =
  super.print()
  println(s"${Dog._message} $sound_level")
```

Τότε, με τον παρακάτω ορισμό:

```
var azorv = Dog("Loud", "Azor")
azorv.print()
```

η εκτύπωση είναι:

```
Barking
Azor say Barking
Barking Loud
```

Αφήσαμε τελευταίο το πιο κρίσιμο εργαλείο της Scala για τη μοντελοποίηση ενός προβλήματος: τους τύπους χαρακτηριστικών. Αυτοί είναι παρεμφερείς σε λειτουργικότητα με τους τύπους διεπαφής της Java, αλλά στη Scala ο ρόλος τους είναι αναβαθμισμένος. Χρησιμοποιούνται για να περιγράψουν συμπεριφορές τμηματικά. Οι συμπεριφορές οι οποίες περιγράφονται σε τύπους χαρακτηριστικών μπορεί να *αναμειχθούν* (*mixin*) (μεταξύ τους) σε κλάσεις. Προκειμένου να επιτευχθεί αυτό, μια κλάση μπορεί να *αναμειγνύει* τύπους χαρακτηριστικών, με χρήση της λέξης-κλειδιού *with*. Μια κλάση μπορεί να αναμειγνύει οσοσδήποτε τύπους χαρακτηριστικών. Η σειρά με την οποία τους δηλώνουμε έχει σημασία.

Οι τύποι χαρακτηριστικών μπορεί να συνδέονται μεταξύ τους με κληρονομικότητα. Μπορούν, ακόμα, να αποτελέσουν υποτύπους μιας αφηρημένης κλάσης. Οι τύποι χαρακτηριστικών μπορεί να είναι γενικευμένοι.

Στο σώμα τους, οι τύποι χαρακτηριστικών περιέχουν ορισμούς πεδίων και μεθόδων, αφηρημένων και μη.

Για παράδειγμα, ας θεωρήσουμε ότι έχουμε τους παρακάτω τύπους χαρακτηριστικών.

```
trait Cute:
  var cuteness_level : String
  def printCute() : Unit =
    println(s"$cuteness_level as cuteness!")

trait Wild:
  var wildness_level : String
  def printWild() : Unit =
    println(s"$wildness_level as wildness!")
```

Τότε, μπορούμε να έχουμε έναν, ακόμα, διαφορετικό ορισμό για την κλάση *Dog*, ως εξής:

```
object Dog:
  private val _message = "Barking"
class Dog(var sound_level : String, name : String)
extends Animal(name, Dog._message) with Cute with Wild:
  var cuteness_level = "Very Cute"
  var wildness_level = "Quite Wild"
  private var _sound_level : String = sound_level
  override def print() : Unit =
    super.print()
    println(s"${Dog._message} $sound_level and ")
    printCute()
    printWild()
```

Σε αυτήν την περίπτωση, κατά την εκτέλεση του:

```
var azorv = Dog("Loud", "Azor")
azorv.print()
```

η εκτύπωση είναι:

```
Barking
Azor say Barking
Barking Loud and
Very Cute as cuteness!
Quite Wild as wildness!
```

Κλείνοντας τη σύνοψη των αντικειμενοστραφών χαρακτηριστικών της γλώσσας Scala, να πούμε ότι δίνονται πάρα πολλές επιπλέον δυνατότητες που βασίζονται σε αυτά που συζητήσαμε παραπάνω. Για παράδειγμα, μπορούν να οριστούν νέοι τύποι ως ενώσεις τύπων ή ως τομές τύπων. Ακόμα, προσφέρεται η δυνατότητα επέκτασης τύπων με μεθόδους, χωρίς να είναι υποχρεωτικό να οριστεί υποκλάση, ορίζοντας μια μέθοδο επέκτασης (*extension method*) για έναν τύπο. Τέλος, θα αναφέρουμε τους *προσωπικούς τύπους* (*self types*), οι οποίοι επιτρέπουν να αναμειγνύεται ένας τύπος χαρακτηριστικών σε έναν άλλο χωρίς ο ένας να είναι υποτύπος του άλλου.

10.3.2 Η Γλώσσα F#

Η γλώσσα F# είναι η βασική γλώσσα συναρτησιακού προγραμματισμού για το πλαίσιο .NET και υποστηρίζει ορισμό κλάσεων και αντικειμένων. Σχεδιαστής της είναι ο Αυστραλός επιστήμονας υπολογιστών Don Syme, ο οποίος συμμετείχε και στην υλοποίησή της. Η αρχική ενασχόληση του Don Syme στη Microsoft ήταν η ένταξη γενικευμένων τύπων στη C#. Στη συνέχεια, όμως, ασχολήθηκε με τον στόχο της Microsoft να δημιουργηθεί μια πιο απλή γλώσσα προγραμματισμού από τη C# για το πλαίσιο .NET. Όπως ο ίδιος ο Don Syme αναφέρει σε συνέντευξή του, η αρχική ιδέα ήταν να υιοθετηθεί η Haskell, αλλά τα χαρακτηριστικά της δεν ταίριαζαν στο .NET. Παρότι υποστήριζε τύπους, δεν ήταν εύκολο να ενταχθούν αντικείμενα στη γλώσσα. Έτσι, το 2005, δημιουργήθηκε η F#, η οποία “βρίσκεται κάπου ανάμεσα στην Haskell και τη C#” και μπορεί να συνεργαστεί εύκολα με τη δεύτερη. Οι σχεδιαστικές αρχές της F# βασίστηκαν στη γλώσσα OCaml (Objective Caml INRIA - 1996). Βασικός πρόγονός τους ήταν η γλώσσα ML (Robin Milner - University of Edinburgh, 1973). Με βάση τα χαρακτηριστικά της, η F# δεν μπορεί να χαρακτηριστεί απόλυτα γλώσσα “καθαρού” συναρτησιακού προγραμματισμού. Κρίσιμος στόχος στον σχεδιασμό της F# ήταν η απόδοση της γλώσσας. Όπως η Scala, έτσι και η F# έχει στατικό σύστημα τύπων.

Ένας προτεινόμενος συνδυασμός χρήσης F# και C# στην υλοποίηση μιας εφαρμογής είναι να υλοποιηθούν σε F# οι βασικοί αλγόριθμοι και η βασική επεξεργασία δεδομένων και να υλοποιηθούν σε C# τα περιφερειακά και τα μέρη που αφορούν παρουσίαση. Ένα βασικό πλεονέκτημα της προσέγγισης αυτής είναι ότι, λόγω χρήσης του ΣΠ, μπορεί να γίνουν μαθηματικές αποδείξεις της υλοποίησης των αλγορίθμων της εφαρμογής, κάτι που για πολλές περιοχές εφαρμογών, π.χ. οι χρηματοοικονομικές, είναι πολύ κρίσιμο. Να τονίσουμε ότι η F# χρησιμοποιείται ευρέως σε εφαρμογές επιστήμης δεδομένων (*data science*) και μηχανικής μάθησης.

10.3.2.1 Βασικά Στοιχεία της Γλώσσας

Η F# είναι γλώσσα του πλαισίου .NET. Ως τέτοια, οι περισσότερες έννοιες που συζητήσαμε στην παράγραφο για τη C# έχουν και εδώ το ίδιο νόημα. Επίσης, πολλά χαρακτηριστικά της C#, στον βαθμό που ταιριάζουν με τη φιλοσοφία της F#, μεταφέρονται ή προσαρμόζονται σε αυτήν. Έτσι, σημαντικό ρόλο στην οργάνωση ενός προγράμματος F# και στην εμβέλεια των ονομάτων του παίζουν οι *χώροι ονομάτων* και τα *assemblies*. Στην F#, έχουμε και *στοιχεία*, μέσα στα οποία, πέρα από ορισμούς, έχουμε και κώδικα που μπορεί να εκτελεστεί όταν πρωτοχρησιμοποιηθεί το στοιχείο. Κατά βάση, τα στοιχεία είναι στατικές κλάσεις. Πέρα από τα αρχεία

στα οποία γράφουμε τον κώδικά μας (με κατάληξη `.fs`), μπορούμε να έχουμε και αρχεία υπογραφών (με κατάληξη `.fsi`). Για ένα αρχείο κώδικα, μπορεί να έχουμε ένα αρχείο υπογραφής, μέσα στο οποίο δηλώνουμε τις υπογραφές των δημόσιων ονομάτων του.

Ένα αρχείο κώδικα F# αποτελείται, κυρίως, από δεσμεύσεις (bindings) *let* με χρήση της λέξης-κλειδιού *let*. Μια δέσμευση συσχετίζει ένα όνομα με έναν ορισμό. Στη γενική περίπτωση των δεδομένων ενός προγράμματος, πρόκειται για τιμές που ανήκουν σε κάποιον τύπο. Μπορεί όμως να έχουμε και μεταβλητές των οποίων η τιμή μπορεί να μεταβληθεί. Καλό είναι μεταβλητές να χρησιμοποιούνται μόνο ως πεδία τύπων ή τοπικές μεταβλητές, διαφορετικά απομακρυνόμαστε από την ορθή χρήση και εκμετάλλευση της γλώσσας. Στην περίπτωση που θέλουμε να εκτελεστεί κώδικας ο οποίος να μην εμφανίζεται μέσα σε μια συνάρτηση, π.χ. για την αρχικοποίηση ενός στοιχείου, ο κώδικας αυτός γράφεται μέσα σε δεσμεύσεις *do* με χρήση της λέξης-κλειδιού *do*.

Η F# προσφέρει πρωτογενείς τύπους, όπως `bool`, `int` κ.ά., καθώς και τη δυνατότητα ορισμού πλειάδων, πινάκων, λιστών, ακολουθιών (sequences), εγγραφών και τύπων διακρίνουσας ένωσης (discriminated union). Προσφέρονται, επιπλέον, τύποι δομής, τύποι κλάσης, τύποι διεπαφής, delegates κ.ά. Δίνεται η δυνατότητα μετονομασίας τύπων. Ακόμα, υποστηρίζεται έκφραση έλλειψης πληροφορίας τύπου (`unit`) καθώς και γενικευμένοι τύποι δεδομένων. Οι τύποι διακρίνονται, και εδώ, σε τύπους αναφοράς και τύπους τιμής. Στη γενική περίπτωση, οι τύποι ορίζουν αμετάβλητα δεδομένα, αλλά μπορούμε να ορίσουμε και τύπους μεταβλητών δεδομένων. Σε ένα πρόγραμμα F#, μπορούν να οριστούν τύποι από τον προγραμματιστή. Γενικά, μπορεί κανείς να ορίσει τον δικό του τύπο αντικειμένων, όπως σε οποιαδήποτε άλλη γλώσσα που υποστηρίζεται από το πλαίσιο .NET.

Θα μείνουμε λίγο στον τύπο της διακρίνουσας ένωσης, μια που προτείνεται από τη γλώσσα ως εναλλακτικό κλάσεων συνδεδεμένων με κληρονομικότητα. Αυτοί είναι τύποι που ορίζουν τιμές που εμπίπτουν σε επώνυμες εναλλακτικές περιπτώσεις, κάθε μια από τις οποίες έχει διαφορετικό ορισμό. Για παράδειγμα, ο ορισμός του τύπου `Person` ως διακρίνουσα ένωση με εναλλακτικά `Student` και `Employee` μπορεί να είναι:

```
type Person =
| Student of studies : string
| Employee of income : int
```

Ακόμα ένα παράδειγμα χρήσης διακρίνουσας ένωσης είναι ο (αναδρομικός) ορισμός μιας δομής δεδομένων δυαδικού δένδρου ως:

```
type tree =
| Leaf of string
| Node of treenode
and treenode = {value : string ; left : tree ; right : tree}
```

Κατά τον χειρισμό των τύπων αυτών, αλλά και γενικότερα στην F#, γίνεται *ταίριασμα προτύπων*.

Μια παρατήρηση είναι ότι, στην F#, η στοίχιση είναι σημαντική προκειμένου να καθοριστεί ένα block, αλλά μπορεί να παρακαμφθεί δεδομένου ότι υπάρχουν κι άλλες δυνατότητες, π.χ. χρήση *begin-end*.

Σε ένα πρόγραμμα F#, ορίζουμε συναρτήσεις. Για παράδειγμα:

```
let double x = 2 * x
```

Στο παραπάνω, ορίζεται μια συνάρτηση, *double*, που δέχεται ένα όρισμα, το *x*, και επιστρέφει το διπλάσιό του. Ορίζοντας μια συνάρτηση, δεν δηλώνουμε απαραίτητα τον τύπο της, διότι γίνεται *συμπερασμός τύπου* (type inference) — μπορούμε, όμως, αν θέλουμε, να δώσουμε περιορισμούς τύπου (type constraints). Όσον αφορά τον υπολογισμό των ορισμάτων των συναρτήσεων, η F# υποστηρίζει αυστηρό υπολογισμό — αλλά μπορεί να δηλωθεί να αποτιμηθεί μια έκφραση σκληρά.

Τα δεδομένα ενός προγράμματος F# είναι αμετάβλητα. Δίνεται η δυνατότητα, όμως, αν είναι επιθυμητό, να δηλωθεί κάτι ως μεταβλητό, προσδιορίζοντάς το ως *mutable*. Για παράδειγμα, από την εκτέλεση του παρακάτω προγράμματος, εκτυπώνεται: 2

```
let mutable x = 1
x <- x + 1
printfn "%i" x
```

Η F# προσφέρει όλες τις δυνατότητες που δίνονται συνήθως από γλώσσες ΣΠ: ορισμό συναρτήσεων ανώτερης τάξης, σύνθεση συναρτήσεων κ.ά. Ένα παράδειγμα σύνθεσης των συναρτήσεων `square` και `double`, βλέπετε στο παρακάτω:

```
let square x = x * x
let double x = 2 * x
let squaredouble = double >> square

let sqdb2 = squaredouble 2
printfn "%i" sqdb2
```

Η εκτέλεση του προγράμματος εκτυπώνει 16.

Ένα παράδειγμα συνάρτησης ανώτερης τάξης βλέπετε στο παρακάτω. Η συνάρτηση είναι αναδρομική, κάτι που πρέπει να δηλωθεί ρητά (με χρήση της λέξης-κλειδιού `rec`).

```
let rec map f list =
    match list with
    | head :: tail -> f head :: map f tail
    | [] -> []
let square x = x * x

let out1 = map square [1;2;3]
printfn "%A" out1
```

Η συνάρτηση `map` εφαρμόζει το όρισμά της `f` σε όλα τα στοιχεία του ορίσμά της `list` — το οποίο είναι λίστα. Η εκτέλεση του παραπάνω προγράμματος εκτυπώνει `[1; 4; 9]`.

Στην περίπτωση που δεν κρίνεται απαραίτητο να ονοματιστεί μια συνάρτηση, μπορεί να οριστεί και να χρησιμοποιηθεί μια λάμδα-έκφραση. Για παράδειγμα, έχοντας ορισμένη τη συνάρτηση `map`, αν δεν κρίνουμε απαραίτητο να ορίσουμε μια ανεξάρτητη συνάρτηση υπολογισμού του τετραγώνου, θα μπορούσαμε να επιτύχουμε το ίδιο αποτέλεσμα με το παραπάνω με τον παρακάτω ορισμό:

```
let map_sq = map (fun x -> x * x)

let out1 = map_sq [1;2;3]
printfn "%A" out1
```

Η εκτύπωση, πάλι, είναι `[1; 4; 9]`.

Να πούμε ότι η F# δίνει πολλές δυνατότητες για τον ορισμό συναρτήσεων, π.χ. `inlining`, οι οποίες δεν εμπίπτουν στο αντικείμενό μας.

10.3.2.2 Επίπεδα Ορατότητας

Αφού κάναμε μια σύντομη σύνοψη των βασικών οντοτήτων ενός προγράμματος F#, στο σημείο αυτό, θα δούμε τα επίπεδα ορατότητας των ονομάτων τους. Να σημειώσουμε ότι και στην F#, ο κώδικάς μας λειτουργεί μέσα σε εμβέλεις, που, και εδώ, κάποιες μπορεί να έχουν όνομα.

Στην F#, έχουμε έλεγχο πρόσβασης ονομάτων στα εξής επίπεδα ορατότητας:

Δημόσια: για οντότητες που είναι προσβάσιμες από παντού.

Εσωτερικά: για οντότητες που είναι προσβάσιμες μόνο από το ίδιο assembly.

Ιδιωτικά: για οντότητες που είναι προσβάσιμες μόνο από τον περικλείοντα τύπο ή στοιχείο.

Παρατηρείτε ότι δεν υπάρχει διαβάθμιση ορατότητας “προστατευόμενα”. Πράγματι, δεν προσφέρεται. Μόνο εάν επεκτείνονται τύποι από κώδικα γραμμένο σε άλλη γλώσσα και επαναορίζεται προστατευόμενη μέθοδός τους, η μέθοδος παραμένει να έχει αυτό το επίπεδο ορατότητας.

Στην περίπτωση που δεν υπάρχει προσδιορισμός ορατότητας σε μια οντότητα, αυτή είναι δημόσια. Εξαιρέση σε αυτό είναι ονόματα που ορίζονται με δεσμεύσεις με χρήση του `let`: τα ονόματα αυτά είναι ιδιωτικά.

Στη γενική περίπτωση, το επίπεδο ορατότητας του περικλειόντος περιβάλλοντος καθορίζει το επίπεδο ορατότητας και των ονομάτων του περιεχομένου του.

10.3.2.3 Κλάσεις, Αντικείμενα και Κληρονομικότητα στην F#

Η F# δεν είναι γλώσσα αντικειμενοστραφούς προγραμματισμού, με την έννοια ότι δεν είναι οι κλάσεις και τα αντικείμενα το βασικό γλωσσικό της στοιχείο. Δίνει, όμως, τη δυνατότητα στον προγραμματιστή να ορίσει κλάσεις και αντικείμενα, κυρίως για την περίπτωση που χρειάζεται μια ομαλή συνεργασία του κώδικά του με κώδικα γραμμένο σε άλλες γλώσσες του πλαισίου .NET.

Οι κλάσεις μπορεί να προσδιοριστούν και ως αφηρημένες ή να ορίζουν γενικευμένους τύπους. Δηλώνονται ως αφηρημένες είτε έμμεσα, έχοντας μια αφηρημένη μέθοδο χωρίς ορισμό, είτε άμεσα, με χρήση σχετικού attribute.

Κατά τη δήλωση μιας κλάσης, προσδιορίζεται το όνομά της, το επίπεδο ορατότητάς της καθώς και το επίπεδο ορατότητας και τα ορίσματα της κύριας μεθόδου *αρχικοποίησης* (primary constructor) — η F# επιβάλλει τον ορισμό μια κύριας μεθόδου αρχικοποίησης, διαφοροποιούμενη από τις άλλες γλώσσες του πλαισίου .NET. Επίσης, προσδιορίζεται το όνομα που θα χρησιμοποιείται στο σώμα του ορισμού της κλάσης, προκειμένου να αναφερθούμε στο στιγμιότυπό της. Δεν χρησιμοποιείται, δηλαδή, κάποια δεσμευμένη λέξη γι' αυτό, π.χ. `this`. Κατά τη δήλωση της κλάσης, μπορεί να προσδιοριστεί ρητά υπερκλάση γι' αυτήν (με χρήση της λέξης-κλειδιού `inherit`). Μια κλάση μπορεί να δηλώσει ρητά το πολύ μία υπερκλάση. Αν δεν υπάρχει δήλωση υπερκλάσης, υπονοείται η κλάση-ρίζα του συστήματος τύπων, `System.Object`. Αντίθετα, μια κλάση μπορεί να υλοποιεί οσουςδήποτε τύπους διεπαφής. Αυτό γίνεται στον ορισμό της κλάσης, προσδιορίζοντας το όνομα του τύπου διεπαφής, με χρήση της λέξης-κλειδιού `interface` και, στη συνέχεια, δίνοντας τους ορισμούς των μελών του, μετά τη λέξη-κλειδί `with`.

Τα μέλη μιας κλάσης μπορεί να είναι τα εξής: μέθοδοι, ιδιότητες, δεικτοδοτημένες ιδιότητες (δηλαδή, ιδιότητες που επιτρέπεται ο χειρισμός τους σαν να ήταν πίνακες), μέθοδοι αρχικοποίησης, επαναορισμένοι τελεστές, δομές, συμβάντα και πεδία που δεν αρχικοποιούνται ρητά. Στον ορισμό μιας κλάσης, μπορεί να περιέχονται και δεσμεύσεις `let` και `do`. Αυτές αρχικοποιούν (ιδιωτικά) πεδία της κλάσης και δίνουν ορισμούς (ιδιωτικών) συναρτήσεων. Ουσιαστικά, οι δεσμεύσεις `let` και `do` περιγράφουν τον κώδικα της κύριας μεθόδου αρχικοποίησης. Οι δεσμεύσεις `let` και `do` πρέπει να προηγούνται των ορισμών άλλων μελών. Εκτελούνται με τη σειρά που τις γράφουμε. Για στατικά πεδία και αρχικοποίηση, πρέπει να ορίζονται δεσμεύσεις `let` και `do`, αντίστοιχα, οι οποίες πρέπει να έχουν προσδιοριστεί ως `static`. Δεν απαγορεύεται να οριστούν περισσότερες από μία μέθοδοι αρχικοποίησης, αλλά επιβάλλεται να καλούν την κύρια. Ο ορισμός τους γίνεται χρησιμοποιώντας τη λέξη-κλειδί `new`. Προκειμένου οι μέθοδοι στιγμιότυπου μιας κλάσης να λειτουργήσουν ως εικονικές, πρέπει να προσδιοριστούν ως `abstract`. Αν δίνεται ορισμός τους, πρέπει αυτό να προσδιοριστεί με χρήση της λέξης-κλειδιού `default`.

Ας δούμε, όμως, ένα παράδειγμα ορισμού κλάσης. Στο παρακάτω, ορίζεται μια κλάση `Dog`.

```
type Dog(namv:string, brkl:string) as bla = class
    static let sound = "Barking"
    static do printfn "Dogs are %s" sound
    let name = namv
```

```

let barkLevel = brkl
do printfn "Another Dog with name %s %s" name sound
member bla.BarkLevel
    with get () = barkLevel
end

```

Με βάση τον παραπάνω ορισμό, η κλάση Dog

- αναπαριστάνει το στιγμιότυπο μέσα στον ορισμό με τον προσδιοριστή bla,
- έχει ένα στατικό ιδιωτικό πεδίο, sound,
- έχει μια στατική μέθοδο αρχικοποίησης που ο κώδικάς της περιέχει μια εντολή εκτύπωσης,
- έχει μόνο την κύρια μέθοδο αρχικοποίησης, η οποία δέχεται δύο ορίσματα και ο κώδικάς της περιέχει μια εντολή εκτύπωσης,
- έχει δύο ιδιωτικά πεδία στιγμιότυπου, name και barkLevel, και
- έχει μια (δημόσια) ιδιότητα, BarkLevel, δηλωμένη με χρήση της λέξης-κλειδιού member, μόνο για ανάγνωση (ορίζεται μόνο η μέθοδος get).

Άπαξ και οριστεί μια κλάση, μπορούν να δημιουργηθούν αντικείμενά της. Δεν είναι απαραίτητο να χρησιμοποιηθεί η λέξη-κλειδί new για τον σκοπό αυτό. Μπορεί να γίνει και απευθείας με χρήση του ονόματος της κλάσης. Για την κλάση του παραδείγματός μας, μπορούμε ισοδύναμα να δημιουργήσουμε αντικείμενά της με τα εξής εναλλακτικά:

```

let azorv = new Dog("Azor", "loudly")
ή
let azorv = Dog("Azor", "loudly")
ή
let azorv = Dog(brkl = "loudly", namv = "Azor")

```

Αν συμπληρώσουμε τον κώδικα που περιέχει τον ορισμό της κλάσης Dog με τον παρακάτω κώδικα:

```

let azorv = Dog(brkl = "loudly", namv = "Azor")
printfn "%s " azorv.BarkLevel

let rex = Dog("Rex", "quietly")
printfn "%s " rex.BarkLevel

```

η εκτύπωση που προκύπτει κατά την εκτέλεση είναι:

```

Dogs are Barking
Another Dog with name Azor Barking
loudly
Another Dog with name Rex Barking
quietly

```

Παρατηρήστε στην εκτύπωση το μήνυμα από τη στατική μέθοδο αρχικοποίησης της κλάσης Dog, Dogs are Barking: εμφανίζεται στην αρχή, άπαξ, λόγω του ορισμού της κλάσης.

Η κληρονομικότητα στην F# εκφράζει τη σχέση *isa* μεταξύ κλάσεων. Είπαμε ότι κατά τον ορισμό της, μια κλάση μπορεί να δηλωθεί ρητά ως υποκλάση μιας άλλης. Σε αυτήν την περίπτωση, για τα αντικείμενα της υποκλάσης, ισχύουν αυτά που έχουμε συζητήσει, τόσο στην παράγραφο της C#, όσο και πιο αναλυτικά, στα προηγούμενα κεφάλαια.

Να υπενθυμίσουμε ότι από την εμβέλεια μιας υποκλάσης, δεν έχουμε πρόσβαση στα μέλη που συμμετέχουν σε δεσμεύσεις `let` της υπερκλάσης, διότι αυτά είναι ιδιωτικά στην υπερκλάση. Κατά τον ορισμό των μελών της υποκλάσης, μπορούμε να χρησιμοποιήσουμε τη λέξη-κλειδί `base`, προκειμένου να αναφερθούμε στη φύση του στιγμιότυπου της υποκλάσης ως στιγμιότυπο της υπερκλάσης. Προκειμένου να αρχικοποιηθεί ένα αντικείμενο υποκλάσης, πρέπει να χρησιμοποιηθεί και μέθοδος αρχικοποίησης της υπερκλάσης, προκειμένου να αρχικοποιηθεί και αυτή η φύση του. Ο προσδιορισμός της μεθόδου που θα χρησιμοποιηθεί γίνεται με διαφορετικούς τρόπους κατά περίπτωση.

Ας δούμε ένα απλό παράδειγμα κλάσεων με κληρονομικότητα. Ας υποθέσουμε ότι έχουμε και μια κλάση `Animal`, στην οποία μεταφέρουμε την πληροφορία του ονόματος από συγκεκριμένο είδος ζώου, στα (οικόσιτα) ζώα, γενικά. Έτσι η κλάση `Animal` ορίζεται ώστε να περιέχει ένα (ιδιωτικό) πεδίο στιγμιότυπου, που να αντιστοιχεί στο όνομα, και μια (δημόσια) ιδιότητα γι' αυτό. Το πεδίο αρχικοποιείται από την κύρια μέθοδο αρχικοποίησης. Για λόγους επίδειξης, ας προσθέσουμε στην κλάση αυτή και μια στατική μέθοδο αρχικοποίησης. Ο ορισμός της κλάσης `Animal` μπορεί να είναι:

```
type Animal(namv:string) as this = class
  static do printfn "Animals ready!"
  let name = namv
  do printfn "Another Animal with name %s" name
  member this.Name
    with get () = name
end
```

Με δεδομένο τον ορισμό της κλάσης `Animal`, μπορούμε να ορίσουμε την κλάση `Dog` ως εξής:

```
type Dog(namv:string, brkl:string) as this = class
  inherit Animal(namv)
  static let sound = "Barking"
  static do printfn "Dogs are %s" sound
  let barkLevel = brkl
  do printfn "Another Dog with name %s %s" base.Name sound
  member this.BarkLevel
    with get () = barkLevel
end
```

Στον ορισμό αυτό, με την έκφραση `inherit Animal(namv)`, προσδιορίσαμε την υπερκλάση και μεταφέραμε τα ορίσματα στη συνάρτηση αρχικοποίησής της. Προσέξτε, επίσης, στην έκφραση `base.Name` την πρόσβαση στην ιδιότητα `Name` της κλάσης `Animal` με χρήση της λέξης-κλειδιού `base`. Για την πρόσβαση στην ιδιότητα αυτή, όμως, θα μπορούσε να χρησιμοποιηθεί και ο τρέχων προσδιοριστής αντικειμένου — που έχει δηλωθεί να είναι ο `this` — εφόσον δεν υπάρχει νεότερος ορισμός στην υποκλάση.

Με βάση τους παραπάνω ορισμούς, αν συμπληρωθεί ο κώδικας με το παρακάτω:

```
let azorv = Dog(brkl = "loudly", namv = "Azor")
printfn "%s " azorv.BarkLevel
```

η εκτύπωση είναι:

```
Animals ready!
Dogs are Barking
Another Animal with name Azor
Another Dog with name Azor Barking
loudly
```

Εδώ, να πούμε ότι αν χρειαστεί να οριστεί ένα αντικείμενο μιας κλάσης με μικρές διαφορές από μια ήδη υπάρχουσα, αντί να οριστεί νέα κλάση και να συνδεθεί με την υπάρχουσα με κληρονομικότητα, μπορεί το αντικείμενο αυτό να οριστεί μέσω μιας έκφρασης αντικειμένου (*object expression*) που να περιγράφει την κλάση του αποκλειστικά γι' αυτό.

Όπως και στη C#, η πολλαπλή κληρονομικότητα προσομοιώνεται με χρήση τύπων διεπαφής και υλοποίησής τους.

Κλείνοντας τη σύνοψη των αντικειμενοστραφών χαρακτηριστικών της F#, να πούμε ότι παρότι εστίασαμε στις κλάσεις, είπαμε ότι υποστηρίζονται και τύποι δομής. Αυτοί έχουν παρόμοια λειτουργικότητα με τους αντίστοιχους της C#.

10.4 Σύνοψη Κεφαλαίου

C#: Η C# είναι άρρηκτα συνδεδεμένη με το περιβάλλον ανάπτυξης και εκτέλεσης εφαρμογών του Microsoft .NET. Δημιουργός της ήταν ο Anders Hejlsberg και πρωτοεμφανίστηκε το 2000. Έχει πολλά κοινά στοιχεία με τη C++ και την Java. Όλα τα ονόματα που χρησιμοποιούνται σε ένα πρόγραμμα ανήκουν σε μια εμβέλεια. Βασικές έννοιες όσον αφορά την οργάνωση ενός προγράμματος είναι οι χώροι ονομάτων, οι τύποι, τα μέλη των τύπων και τα *assemblies*. Η διάκριση των επιπέδων ορατότητας στη C# δεν αναφέρεται στους χώρους ονομάτων, αλλά στα ονόματα τύπων ή των μελών τους. Η C# προσφέρει 6 επίπεδα ορατότητας. Οι τύποι που προσφέρει η C# ανήκουν σε δύο κατηγορίες: τους τύπους αναφοράς και τους τύπους τιμής. Τύποι αναφοράς είναι οι κλάσεις, οι τύποι διεπαφής, οι πίνακες, οι συμβολοσειρές και οι τύποι *delegates*. Οι υπόλοιποι τύποι είναι τύποι τιμής. Όλο το σύστημα των τύπων της γλώσσας ανήκει στην ιεραρχία τύπων του CTS με ρίζα την κλάση *object*. Οι μέθοδοι στη C# δεν είναι αυτόματα εικονικές. Τα μέλη μιας κλάσης μπορεί να είναι είτε στατικά μέλη είτε μέλη στιγμιοτύπου. Μέλη κλάσης μπορεί να είναι: σταθερές, μεταβλητές, μέθοδοι, ιδιότητες, δεικτοδότες, συμβάντα, τελεστές, μέθοδοι αρχικοποίησης, μέθοδοι καταστροφής και εμφωλευμένοι τύποι. Οι ιδιότητες αποτελούν ένα πρακτικό και συνηθισμένο εργαλείο χειρισμού αντικειμένων. Ένας “ελαφρύς” τύπος με συμπεριφορά κατά τιμή μπορεί να οριστεί ως τύπος δομής. Η κληρονομικότητα στη C# εκφράζει τη σχέση *isa* μεταξύ τύπων. Η πολλαπλή κληρονομικότητα προσομοιώνεται με χρήση τύπων διεπαφής. Η C# προσφέρει πλήθος δυνατοτήτων στον προγραμματιστή.

Python: Είναι άρρηκτα συνδεδεμένη με τον δημιουργό της, τον Guido van Rossum. Η πρώτη επιδείξιμη μορφή της εμφανίστηκε το 1990. Υποστηρίζει διαδικαστικό, συναρτησιακό και αντικειμενοστραφή προγραμματισμό. Είναι διερμηνευόμενη. Έχει δυναμικό σύστημα τύπων. Υπάρχει διαδικασία αυτόματης περισυλλογής σκουπιδιών. Ο κώδικας ενός προγράμματος Python οργανώνεται σε στοιχεία και πακέτα και λειτουργεί μέσα σε εμβέλειες. Δεν υπάρχει απόκρυψη πληροφορίας. Όλοι οι τύποι της Python είναι κλάσεις. Τα πάντα σε ένα πρόγραμμα Python θεωρούνται αντικείμενα, ακόμα και οι συναρτήσεις. Το σύστημα τύπων της Python είναι οργανωμένο σε μια ιεραρχία με ρίζα την κλάση *object*. Όλες οι μεταβλητές της Python είναι μεταβλητές αναφοράς και μπορούν να αναφερθούν ανά πάσα στιγμή σε αντικείμενο οποιουδήποτε τύπου. Σε ένα στοιχείο μπορούμε να ορίσουμε κλάσεις. Μπορούμε να ορίσουμε αντικείμενα κλάσεων. Για την αρχικοποίησή τους μπορούμε να έχουμε ορίσει και δικές μας διαδικασίες αρχικοποίησης. Υπάρχει δυνατότητα αρχικοποίησης μέσω αντιγραφής. Παρότι υπάρχει δυνατότητα ορισμού συνάρτησης καταστροφής, αυτή δεν είναι τόσο απαραίτητη. Οι κλάσεις μπορούν να έχουν υποκλάσεις. Υποστηρίζεται πολλαπλή κληρονομικότητα με πολύ καλά ορισμένη διαδικασία επίλυσης σύγκρουσης ονομάτων. Το μοντέλο της γλώσσας είναι αντικειμενοστραφές. Είναι εύκολα επεκτάσιμη και προσφέρει μεγάλη ποικιλία δυνατοτήτων. Διαρκώς αναπτύσσονται και νέες βιβλιοθήκες κώδικα.

Scala: Γλώσσα αντικειμενοστραφούς συναρτησιακού προγραμματισμού σχεδιασμένη από τον Martin Odersky. Πρωτοεμφανίστηκε το 2003. Μεταφράζεται σε JVM bytecode εντολές. Συνεργάζεται με την Java,

αλλά έχει σχεδιαστεί ανεξάρτητα από εκείνη. Παρέχει μεγάλη λειτουργικότητα, τόσο από πλευράς ΣΠ, όσο και από πλευράς αντικειμενοστραφούς προγραμματισμού. Η φιλοσοφία της βασίζεται στην κλιμάκωση, τόσο από πλευράς πολυπλοκότητας προγραμματισμού, όσο και από πλευράς απαιτήσεων από το σύστημα, εξ ου και το όνομά της.

F#: Γλώσσα συναρτησιακού προγραμματισμού με αντικειμενοστραφή χαρακτηριστικά για το πλαίσιο .NET. Αυτά έχουν συνάφεια με αντίστοιχα της γλώσσας C#. Ο ορισμός κλάσεων και αντικειμένων προτείνεται κυρίως για την περίπτωση συνεργασίας του κώδικα του προγράμματος F# με κώδικες γραμμένους σε άλλες γλώσσες προγραμματισμού του πλαισίου .NET. Εμφανίστηκε το 2005. Δημιουργός της είναι ο Don Syme.

10.5 Βιβλιογραφία

- [1] *C# programming guide*. <https://docs.microsoft.com/en-us/dotnet/csharp/programming-guide/>. Ανάκτηση: 2023-5-30.
- [2] *A tour of the C# language*. <https://docs.microsoft.com/en-us/dotnet/csharp/tour-of-csharp/>. Ανάκτηση: 2023-5-30.
- [3] *.NET fundamentals documentation*. <https://docs.microsoft.com/en-us/dotnet/fundamentals/>. Ανάκτηση: 2023-5-30.
- [4] *Behind the Code with Anders Hejlsberg*. <https://www.microsoft.com/en-us/research/video/behind-the-code-with-anders-hejlsberg/>. Ανάκτηση: 2023-5-30.
- [5] *The C# Design Process - A Conversation with Anders Hejlsberg (Part I)* by Bill Venners with Bruce Eckel, <https://www.artima.com/articles/the-c-design-process>. Ανάκτηση: 2023-5-30.
- [6] ISO/IEC 23271. *Information technology — Common Language Infrastructure (CLI)*. Report ISO/IEC 23271:2012(E). ISO/IEC, 2012.
- [7] *The Python Language Reference*. <https://docs.python.org/3/reference/>. Ανάκτηση: 2023-5-30.
- [8] *The Python Tutorial*. <https://docs.python.org/3/tutorial/>. Ανάκτηση: 2023-5-30.
- [9] *The Python Standard Library*. <https://docs.python.org/3/library/>. Ανάκτηση: 2023-5-30.
- [10] *Guido van Rossum - Brief Bio*. <https://gvanrossum.github.io/bio.html>. Ανάκτηση: 2023-5-30.
- [11] *Q&A with Guido van Rossum, Inventor of Python*. <https://www.youtube.com/watch?v=aYbNh3NS7jA>. Ανάκτηση: 2023-5-30.
- [12] Allen Downey. *Think Python: How to Think Like a Computer Scientist*. 2nd (Version 2.4.0). Green Tea Press, 2015.
- [13] *Tour of Scala*. <https://docs.scala-lang.org/tour/tour-of-scala.html>. Ανάκτηση: 2023-5-30.
- [14] *Learn Scala*. <https://docs.scala-lang.org/>. Ανάκτηση: 2023-5-30.
- [15] *Scala 3 – Book*. <https://docs.scala-lang.org/scala3/book/introduction.html>. Ανάκτηση: 2023-5-30.
- [16] *The Origins of Scala – A Conversation with Martin Odersky, (Part I)*. <https://www.artima.com/articles/the-origins-of-scala>. Ανάκτηση: 2023-5-30.
- [17] *F# documentation*. <https://docs.microsoft.com/en-us/dotnet/fsharp/>. Ανάκτηση: 2023-5-30.
- [18] *Tour of F#*. <https://docs.microsoft.com/en-us/dotnet/fsharp/tour>. Ανάκτηση: 2023-5-30.
- [19] *Don Syme Answering Questions on F#, C#, Haskell and Scala*. <https://www.infoq.com/interviews/F-Sharp-Don-Syme/>. Ανάκτηση: 2023-5-30.

ΠΡΟΒΛΗΜΑΤΑ ΚΑΙ ΕΡΓΑΣΙΕΣ ΚΑΤΑΛΛΗΛΑ ΓΙΑ ΑΝΤΙΚΕΙΜΕΝΟΣΤΡΑΦΗ ΥΛΟΠΟΙΗΣΗ

Σύνοψη

Στο κεφάλαιο αυτό, δίνονται προβλήματα και εργασίες κατάλληλα για ανάπτυξη σε αντικειμενοστραφές περιβάλλον. Η παράθεσή τους γίνεται με σειρά κλιμακούμενης δυσκολίας. Αρχικά, προτείνονται σχετικά απλές ασκήσεις ανάπτυξης κώδικα για εξάσκηση στη μοντελοποίηση με χρήση εννοιών του αντικειμενοστραφούς προγραμματισμού. Αυτές θα μπορούσαν να είναι μέρος γραπτής εξέτασης. Στη συνέχεια, δίνονται πιο περίπλοκες ασκήσεις για εξάσκηση στη μοντελοποίηση και υλοποίηση προβλημάτων που απαιτούν αφαίρεση στα δεδομένα. Ακολουθεί μια σειρά εργασιών ίδιου επιπέδου, που περιέχουν χρήση κληρονομικότητας. Το κεφάλαιο κλείνει με μια σειρά εργασιών σημαντικής πολυπλοκότητας, που προτείνεται να αναπτυχθούν από δύο προγραμματιστές, προκειμένου να γίνει πράξη η ανεξάρτητη υλοποίησή τους.

Προαπαιτούμενη Γνώση

Για την κατανόηση του κεφαλαίου, ο αναγνώστης απαιτείται να έχει γνώσεις διαδικαστικού προγραμματισμού και να έχει μελετήσει τις έννοιες του αντικειμενοστραφούς προγραμματισμού, καθώς και τη C++ από τα προηγούμενα κεφάλαια του βιβλίου.

11.1 Γενικές Παρατηρήσεις επί της Ανάπτυξης

Στο κεφάλαιο αυτό, δίνονται προβλήματα και εργασίες κατάλληλα για ανάπτυξη σε αντικειμενοστραφές περιβάλλον. Οι προδιαγραφές τους είναι τέτοιες ώστε να περιλαμβάνουν σύνθετες οντότητες οι οποίες, συχνά, ενδείκνυται να οργανωθούν σε ιεραρχίες. Η ενασχόληση με τα προβλήματα αυτά και η επιτυχής αντιμετώπισή τους είναι ο σκοπός της μελέτης του βιβλίου. Η ύπαρξη παραδείγματος πλήρους ανάπτυξης εκτενούς κώδικα θα διευκόλυνε περισσότερο την επίτευξη του στόχου αυτού -και στη διδασκαλία σχετικού μαθήματος καλό είναι να συμπεριλαμβάνεται. Παρ' όλα αυτά, στα πιο απλά προβλήματα του κεφαλαίου αυτού, αρκετά συχνά, η μορφή των εκφωνήσεων υποδεικνύει τη μοντελοποίησή τους.

Σε όλες τις ασκήσεις, το βασικό είναι ο καθορισμός των κλάσεων που αναπαριστούν τις οντότητες ή/και τις έννοιες των προβλημάτων και η σωστή υλοποίησή τους. Για να επιτευχθεί αυτό, υπενθυμίζουμε ότι ενεργούμε με τον εξής τρόπο:

1. Μελετάμε την περιγραφή του προβλήματος και εντοπίζουμε τις έννοιες που πρέπει να αναπαραστήσουμε μέσω κλάσεων.
2. Εντοπίζουμε, αν υπάρχουν, κοινά χαρακτηριστικά και ιδιότητες, ώστε να οριστούν υπερκλάσεις των πρωτογενών κλάσεων και να τα συσχετίσουμε με αυτές. Καθορίζουμε τις προδιαγραφές της λειτουργικότητας των κλάσεων και θεωρούμε ότι οι δηλώσεις που τις εκφράζουν θα συμπεριληφθούν στο δημόσιο τμήμα των κλάσεων. Οι κλάσεις που ορίζονται ως υποκλάσεις αναμένεται ότι ακολουθούν τις προδιαγραφές που δηλώνονται στο δημόσιο τμήμα των υπερκλάσεών τους.
3. Αποφασίζουμε τον τρόπο της υλοποίησης της λειτουργικότητας των κλάσεων. Στο πλαίσιο αυτό, αποφασίζουμε και την αναπαράσταση των αντικειμένων τους. Οτιδήποτε δεν αφορά τη δημόσια συμπεριφορά οφείλει να δηλωθεί στο ιδιωτικό τμήμα των κλάσεων. Σε περίπτωση κληρονομικότητας, αποφασίζονται τα προστατευόμενα μέλη.
4. Αποφασίζονται οι τρόποι αρχικοποίησης των αντικειμένων των κλάσεων και υλοποίησης των συναρτήσεων αρχικοποίησης.
5. Ελέγχεται αν χρειάζονται συναρτήσεις καταστροφής, αρχικοποίησης μέσω αντιγραφής (και μετακίνησης) και επαναορισμοί του τελεστή ανάθεσης.

Εννοείται ότι οποτεδήποτε υπάρχει ανάγκη αλγοριθμικής επίλυσης προβλήματος, γίνεται αναζήτηση του βέλτιστου αλγορίθμου και αποδοτική υλοποίησή του. Ακολουθείται, δηλαδή, η μεθοδολογία επίλυσης προβλήματος στο πλαίσιο του διαδικαστικού προγραμματισμού.

Σε ασκήσεις στις οποίες θα χρησιμοποιήσετε κληρονομικότητα μπορεί να χρειάζεται να ορίσετε εικονικές συναρτήσεις ή αφηρημένες κλάσεις. Κάποιες από αυτές τις περιπτώσεις θα σας οδηγήσουν στο να εξοικειωθείτε με μια χαρακτηριστική χρήση των αφηρημένων κλάσεων και των εικονικών συναρτήσεων. Συγκεκριμένα, θα δείτε τη χρήση τους, στην οποία αναφερθήκαμε στο Κεφάλαιο 7, όπου χειριζόμαστε συλλογές από αντικείμενα μέσω δεικτών (ή αναφορών) σε υπερκλάση τους. Εφαρμόζουμε επαναληπτικά στη συλλογή τη συνάρτηση που μας ενδιαφέρει, οπότε, κατά περίπτωση, εκτελείται ο κατάλληλος ορισμός ανάλογα με τον τύπο του αντικειμένου.

Μια άλλη τεχνική που θα χρειαστεί να εφαρμόσετε σε πολλές από τις ασκήσεις είναι, χρησιμοποιώντας τον δείκτη `this`, να μετατρέψετε ένα αντικείμενο από το αντικείμενο εφαρμογής μιας συνάρτησης-μέλους στο όρισμα κλήσης μιας άλλης συνάρτησης.

Και οι δύο παραπάνω περιπτώσεις, στα πλαίσια των ασκήσεων, προκύπτουν από τις προδιαγραφές. Σε συγγράμματα σχεδίασης λογισμικού, όμως, θα τις συναντήσετε με επίσημους όρους προτύπων σχεδίασης (design patterns).

Οι προδιαγραφές των κλάσεων πρέπει να εξαχθούν από μια μη τυπική περιγραφή τους σε φυσική γλώσσα που δίνεται στο πλαίσιο της περιγραφής του προβλήματος. Αυτό συμβάλλει σε μια πρώτη επαφή με την προσπάθεια αντιμετώπισης μη τυπικών περιγραφών. Επίσης, στις περιγραφές των προβλημάτων, υπάρχουν ασαφή και μη πλήρως καθορισμένα σημεία. Αυτά είναι σημεία που δεν ενδιαφέρουν από πλευράς αντικειμενοστραφούς αντιμετώπισης, οπότε στην υλοποίηση μπορεί να ακολουθηθεί οποιαδήποτε εκδοχή δεν παραβιάζει τις προδιαγραφές που προκύπτουν από την υπόλοιπη περιγραφή.

Τέλος, να σημειώσουμε ότι παρότι στις εκφωνήσεις αναφέρεται η C++ ως γλώσσα προς χρήση, ο αναγνώστης θα μπορούσε να επιλέξει μια οποιαδήποτε γλώσσα αντικειμενοστραφούς προγραμματισμού, παραλείποντας κάποιες από τις προδιαγραφές που δεν “ταιριάζουν” με την επιλεγμένη γλώσσα. Για παράδειγμα, μπορεί να παραλειφθεί η εκτύπωση μηνυμάτων κατά την καταστροφή αντικειμένων, αν έχει επιλεγεί η γλώσσα Java. Εκτός από κάποιες λεπτομέρειες, όπως του παραδείγματος αυτού, οι προδιαγραφές είναι τέτοιες ώστε να μην επιβάλλεται συγκεκριμένη γλώσσα υλοποίησης.

11.2 Προβλήματα Μικρής Κλίμακας

Τα προβλήματα που περιλαμβάνονται στο τμήμα αυτό απαιτούν ανάπτυξη κώδικα μικρής κλίμακας, αλλά συνδυάζουν πολλά στοιχεία από τον αντικειμενοστραφή προγραμματισμό. Μπορούν, επίσης, να αποτελέσουν μέρος γραπτής εξέτασης στο αντικείμενο.

Εκφωνήσεις Ασκήσεων

11.2.1 Έστω ότι έχουμε να υλοποιήσουμε σε C++ μία απλουστευμένη προσομοίωση της πρόκρισης στα προημιτελικά από τον πρώτο γύρο στο EURO 2004. Έστω ότι η προσομοίωση αυτή θεωρεί τις εξής κλάσεις:

- κλάση: “Ομάδα” (team) που προσομοιώνει μια ομάδα που συμμετέχει στον πρώτο γύρο.
- κλάση: “Αγώνας” (match) που προσομοιώνει έναν αγώνα του πρώτου γύρου.
- κλάση: “Ομίλος” (group) που προσομοιώνει έναν όμιλο του πρώτου γύρου. Ένας όμιλος αποτελείται από 4 ομάδες που παίζουν μεταξύ τους ανά δύο. Από τον όμιλο, μόνο δύο ομάδες προκρίνονται στα προημιτελικά.
- κλάση: “Πρόκριση” (proceed) που προσομοιώνει το αποτέλεσμα της πρόκρισης του πρώτου γύρου, δηλαδή το σύνολο των ομάδων που προκρίνονται στα προημιτελικά από τους 4 ομίλους του πρώτου γύρου της διοργάνωσης.

Ο αλγόριθμος με τον οποίο (θεωρούμε ότι) αποφασίζονται ποιες είναι οι δύο ομάδες που προκρίνονται στα προημιτελικά από έναν όμιλο είναι ο παρακάτω. Κάθε ομάδα, για κάθε νίκη σε αγώνα του πρώτου γύρου, παίρνει τρεις βαθμούς, για κάθε ισοπαλία, έναν και μηδέν, σε περίπτωση ήττας. Οι δύο πρώτες ομάδες είναι εκείνες που

- έχουν συγκεντρώσει τη μεγαλύτερη βαθμολογία στον όμιλό τους. Σε περίπτωση ισοβαθμίας μεταξύ δύο, τριών ή τεσσάρων ομάδων,
- ελέγχουμε τους μεταξύ τους αγώνες, αν μπορούν να καθορίσουν διάταξη. Σε περίπτωση αποτυχίας καθορισμού διάταξης,
- προσπαθούμε να καθορίσουμε διάταξη με βάση τη διαφορά τερμάτων που πέτυχε μια ομάδα μείον αυτά που δέχθηκε (για όλους τους αγώνες που συμμετείχε). Σε περίπτωση αποτυχίας και αυτού του κριτηρίου,
- προσπαθούμε να καθορίσουμε διάταξη με βάση το σύνολο των τερμάτων που πέτυχε μια ομάδα (για όλους τους αγώνες που συμμετείχε). Σε περίπτωση αποτυχίας και αυτού του κριτηρίου,
- προσπαθούμε να καθορίσουμε διάταξη με βάση τη βαθμολογία που είχε συγκεντρώσει μια ομάδα στα προκριματικά. Σε περίπτωση αποτυχίας και αυτού του κριτηρίου,
- η επιλογή γίνεται τυχαία.

Η κλάση “Ομάδα”:

- Αποθηκεύει τη βαθμολογία της κατά τη διάρκεια του πρώτου γύρου (points).
- Έχει αποθηκευμένη τη βαθμολογία που είχε συγκεντρώσει στα προκριματικά του EURO 2004 (points_in).
- Αποθηκεύει το σύνολο των αγώνων που συμμετείχε στον πρώτο γύρο (matches).

Η κλάση “Ομάδα” χαρακτηρίζεται από την εξής συμπεριφορά:

- Αρχικά, η μόνη πληροφορία που καταχωρείται είναι η βαθμολογία που είχε συγκεντρώσει η ομάδα στα προκριματικά.

- Μπορούμε να μάθουμε τη βαθμολογία που έχει η ομάδα στον όμιλό της (`get_points`).
- Μπορούμε να μάθουμε τη βαθμολογία που είχε η ομάδα στα προκριματικά (`get_points_in`).

Η κλάση “Αγώνας”:

- Αποθηκεύει τον όμιλο στον οποίο ανήκει (`group`).
- Αποθηκεύει τις ομάδες που συμμετέχουν σε αυτόν (`team1, team2`).
- Έχει μια σημαία που εκφράζει αν ο αγώνας έχει διεξαχθεί (αρχικά δεν έχει) (`completed`).
- Μόλις διεξαχθεί ο αγώνας, αυξάνεται ο μετρητής του ομίλου στον οποίο ανήκει ο αγώνας και αντιστοιχεί στους διεξαχθέντες αγώνες (`update_completed`).
- Αποθηκεύει το αποτέλεσμα του αγώνα ως ζεύγος ζευγών ομάδα-τέρματα που πέτυχε η ομάδα στον αγώνα (αρχικά είναι μηδενικό) (`score`).

Η κλάση “Αγώνας” χαρακτηρίζεται από την εξής συμπεριφορά:

- Πέρα από τις ενδεχόμενες αρχικές τιμές, ένας αγώνας αρχικοποιείται με τον όμιλο στον οποίο ανήκει, καθώς και τις ομάδες που παίρνουν μέρος σε αυτόν. Ακολουθεί η ενημέρωση των ομάδων με τον αγώνα αυτό.
- Μπορούμε να καταχωρήσουμε το αποτέλεσμα του αγώνα αυτού. Τότε θα θεωρηθεί ότι ο αγώνας έληξε και θα γίνει και η απαιτούμενη ενημέρωση στον μετρητή των αγώνων του ομίλου (`set_score`).
- Μπορούμε να μάθουμε το αποτέλεσμα του αγώνα αυτού (`get_score`).

Η κλάση “Όμιλος”:

- Αποθηκεύει τις 4 ομάδες που συμμετέχουν σε αυτόν (`teams`).
- Έχει έναν μετρητή που αντιστοιχεί στους διεξαχθέντες αγώνες του ομίλου με αρχική τιμή μηδέν (`completed`).

Η κλάση “Όμιλος” χαρακτηρίζεται από την εξής συμπεριφορά:

- Αρχικοποιείται με τις ομάδες που συμμετέχουν σε αυτόν και τις απαιτούμενες αρχικές τιμές.
- Όταν ολοκληρωθούν οι αγώνες του ομίλου, μπορούμε να εκτυπώσουμε το αποτέλεσμα της πρόκρισης (δηλαδή τις δύο ομάδες που αποφάσισε ο αλγόριθμος πρόκρισης) (`print_group_result`).

Η κλάση “Πρόκριση”:

- Αποθηκεύει τους 4 ομίλους του πρώτου γύρου (`groups`).

Η κλάση “Πρόκριση” χαρακτηρίζεται από την εξής συμπεριφορά:

- Αρχικοποιείται με τους ομίλους των ομάδων.
- Μπορούμε να εκτυπώσουμε το τελικό αποτέλεσμα που αποτελείται από τις 8 ομάδες που προκρίνονται στα προημιτελικά (`print_result`).

Να υλοποιήσετε σε C++ τις παραπάνω κλάσεις και να γράψετε μία συνάρτηση `main` που:

- δημιουργεί 16 ομάδες,
- τις καταχωρεί σε 4 ομίλους,

- δημιουργεί τους απαιτούμενους αγώνες και εισάγει τα αποτελέσματά τους,
- εκτυπώνει τις ομάδες που προκρίνονται στα προημιτελικά.

Τα δεδομένα μπορούν να είναι υποθετικά.

11.2.2 Έστω ότι έχουμε να υλοποιήσουμε σε C++ μία προσομοίωση της φόρτωσης επιβατηγών πλοίων. Προκειμένου να γίνει η υλοποίηση αυτή, θεωρούμε τις κλάσεις: “επιβάτης” (Passenger), “επιβατικό αυτοκίνητο” (Car), “φορτηγό αυτοκίνητο” (Truck) και “επιβατηγό πλοίο” (Ferry_boat).

Η κλάση “επιβάτης”:

- Έχει έναν μοναδικό αριθμό ταυτότητας (id).
- Έχει ένα πλοίο στο οποίο θα επιβιβαστεί (boat).
- Έχει μια ώρα άφιξης στην προβλήτα που είναι αγκυροβολημένο το πλοίο (arrival_time).
- Είτε έχει είτε δεν έχει αγοράσει εισιτήριο για το πλοίο (ticket).

Η κλάση “επιβάτης” χαρακτηρίζεται από την εξής συμπεριφορά:

- Αρχικά ανατίθεται ο αριθμός ταυτότητάς του, το πλοίο, η ώρα άφιξης στην προβλήτα καθώς και το εάν έχει αγοράσει εισιτήριο ή όχι.
- Επιβιβάζεται ένας επιβάτης στο πλοίο, εάν η ώρα άφιξής του είναι μικρότερη από την ώρα αναχώρησης του πλοίου και εάν με την επιβίbasή του, δεν υπερβαίνουμε τον μέγιστο αριθμό επιβατών του πλοίου. Τότε, στην περίπτωση που δεν έχει εισιτήριο, αγοράζει εισιτήριο και αυξάνεται ο μετρητής επιβιbasμένων επιβατών του πλοίου, (board).

Η κλάση “επιβατικό αυτοκίνητο”:

- Έχει έναν μοναδικό αριθμό κυκλοφορίας (id).
- Έχει ένα πλοίο στο οποίο θα επιβιβαστεί (boat).
- Έχει μια ώρα άφιξης στην προβλήτα που είναι αγκυροβολημένο το πλοίο (arrival_time).
- Έχει ένα σύνολο επιβατών που μεταφέρει (passengers), το πολύ επτά, καθώς και το πλήθος τους (passengers_size). Οι επιβάτες του θεωρούμε ότι έχουν αγοράσει εισιτήρια. Η ώρα άφιξης των επιβατών του είναι η ίδια με την ώρα άφιξης του αυτοκινήτου.

Η κλάση “επιβατικό αυτοκίνητο” χαρακτηρίζεται από την εξής συμπεριφορά:

- Αρχικά καταχωρείται ο αριθμός κυκλοφορίας του, το πλοίο, η ώρα άφιξης στην προβλήτα καθώς και οι επιβάτες που μεταφέρει.
- Το αυτοκίνητο επιβιβάζεται στο πλοίο, εάν η ώρα άφιξής του είναι μικρότερη από την ώρα αναχώρησης του πλοίου και εάν με την επιβίbasή του, δεν υπερβαίνουμε τον μέγιστο αριθμό οχημάτων του πλοίου. Τότε, αυξάνεται ο μετρητής επιβιbasμένων οχημάτων του πλοίου και επιβιβάζουμε τους επιβάτες του (board).

Η κλάση “φορτηγό αυτοκίνητο”:

- Έχει έναν μοναδικό αριθμό κυκλοφορίας (id).
- Έχει ένα πλοίο στο οποίο θα επιβιβαστεί (boat).
- Έχει μια ώρα άφιξης στην προβλήτα που είναι αγκυροβολημένο το πλοίο (arrival_time).
- Έχει έναν μόνο επιβάτη, τον οδηγό, που ενδέχεται και να μην έχει εισιτήριο. Η ώρα άφιξης του οδηγού του είναι η ίδια με την ώρα άφιξης του φορτηγού (driver).

Η κλάση “φορτηγό αυτοκίνητο” χαρακτηρίζεται από την εξής συμπεριφορά:

- Αρχικά καταχωρείται ο αριθμός κυκλοφορίας του, το πλοίο, η ώρα άφιξης στην προβλήτα καθώς και ο οδηγός του.
- Το φορτηγό επιβιβάζεται στο πλοίο, εάν η ώρα άφιξης του είναι μικρότερη από την ώρα αναχώρησης του πλοίου και εάν με την επιβίβασή του, δεν υπερβαίνουμε τον μέγιστο αριθμό οχημάτων του πλοίου. Τότε, αυξάνεται ο μετρητής επιβιβασμένων οχημάτων του πλοίου και επιβιβάζουμε τον οδηγό του (board).

Η κλάση “επιβατηγό πλοίο”:

- Έχει έναν μοναδικό αριθμό νηολογίου (id).
- Έχει μια ώρα αναχώρησης (departure_time).
- Έχει έναν μετρητή επιβιβασμένων επιβατών (no_passengers).
- Έχει έναν μέγιστο αριθμό επιβατών (max_no_passengers).
- Έχει έναν μετρητή επιβιβασμένων οχημάτων (no_vehicles).
- Έχει έναν μέγιστο αριθμό οχημάτων (max_no_vehicles).
- Έχει μια ουρά επιβατών που περιμένουν να μπουν στο καράβι (passenger_queue).
- Έχει μια ουρά οχημάτων (επιβατικών αυτοκινήτων ή φορτηγών) που περιμένουν να μπουν στο καράβι (vehicle_queue).

Η κλάση “επιβατηγό πλοίο” χαρακτηρίζεται από την εξής συμπεριφορά:

- Αρχικά καταχωρείται ο αριθμός νηολογίου του, η ώρα αναχώρησης, ο μέγιστος αριθμός επιβατών και ο μέγιστος αριθμός οχημάτων. Αρχικά το πλοίο είναι άδειο και οι ουρές κενές.
- Προστίθεται ένας επιβάτης στην ουρά επιβατών, τοποθετώντας τον στο τέλος της ουράς. Στην περίπτωση που η ώρα άφιξης του επιβάτη είναι μεγαλύτερη από την ώρα αναχώρησης του πλοίου, δεν μπορεί να προστεθεί στην ουρά (passenger_arrive).
- Αφαιρείται ένας επιβάτης από την ουρά επιβατών, διαγράφοντάς τον από την ουρά και επιβιβάζοντάς τον στο πλοίο (passenger_enter).
- Προστίθεται ένα επιβατικό αυτοκίνητο ή φορτηγό στην ουρά οχημάτων, τοποθετώντας το στο τέλος της ουράς. Στην περίπτωση που η ώρα άφιξης του είναι μεγαλύτερη από την ώρα αναχώρησης του πλοίου, δεν μπορεί να προστεθεί στην ουρά (vehicle_arrive).
- Αφαιρείται ένα επιβατικό αυτοκίνητο ή φορτηγό από την ουρά οχημάτων, διαγράφοντάς το από την ουρά και επιβιβάζοντάς το στο πλοίο (vehicle_enter).

Να υλοποιήσετε σε C++ τις παραπάνω κλάσεις, ορίζοντας νέες, όπου δείχνει ενδεικνυόμενο.

11.2.3 Έστω ότι έχουμε να υλοποιήσουμε σε C++ μία αυθαίρετη εκδοχή υπολογισμού στατιστικών περί πωλήσεων εισιτηρίων για τους Ολυμπιακούς Αγώνες της Αθήνας, το 2004. Η πολιτική πώλησης είναι επίσης αυθαίρετη. Προκειμένου να γίνει η υλοποίηση αυτή, θεωρούμε τις εξής κλάσεις:

- κλάση: “Εισιτήριο” (ticket) που προσομοιώνει ένα εισιτήριο που έχει εκδοθεί.
- κλάση: “Εκδοτήριο εισιτηρίων” (ticket_office) που προσομοιώνει ένα από τα περίπτερα έκδοσης εισιτηρίων του “Αθήνα 2004”.
- κλάση: “Υποκατάστημα” (branch) που προσομοιώνει τα υποκαταστήματα τραπεζής από τα οποία μπορούσε κάποιος να προμηθευτεί εισιτήρια.

- κλάση: “Λογαριασμός πελάτη” (`internet_account`) που προσομοιώνει τους προσωπικούς λογαριασμούς πελατών μέσω του δικτυακού τόπου του “Αθήνα 2004”.
- κλάση: “Στατιστική” (`statistics`) που προσομοιώνει το αποτέλεσμα των διαφόρων στατιστικών των πωλήσεων εισιτηρίων.

Η κλάση “Εισιτήριο”:

- Έχει έναν μοναδικό αύξοντα αριθμό (`no`).
- Αποθηκεύει την αγωνιστική περίοδο στην οποία αναφέρεται (`event`). Ως αγωνιστική περίοδος θεωρείται μια σύνθετη πληροφορία που αποτελείται από την ημερομηνία στην οποία διεξάγεται, ώρα έναρξης, ώρα λήξης, είδος του αγωνίσματος (π.χ. “στίβος”), την περιγραφή του (π.χ. “σφαιροβολία γυναικών προκριματικοί, σφαιροβολία ανδρών προκριματικοί”) και την εγκατάσταση στην οποία διεξάγεται (π.χ. “Ολυμπιακό Στάδιο”).
- Το κόστος του εισιτηρίου (`price`). Στην περίπτωση που η είσοδος είναι ελεύθερη, η τιμή είναι μηδέν.
- Η θέση στην οποία αντιστοιχεί (`seat`) (για απλότητα θεωρήστε ότι είναι ένας ακέραιος). Στην περίπτωση που δεν υφίσταται η έννοια της θέσης, η τιμή είναι μηδέν.
- Η ημερομηνία της έκδοσης του εισιτηρίου (`issue_date`)

Η κλάση “Εισιτήριο” χαρακτηρίζεται από την εξής συμπεριφορά:

- Αρχικά, ανατίθεται ο αύξοντας αριθμός του εισιτηρίου ως ο επόμενος από τους αριθμούς των εισιτηρίων που έχουν εκδοθεί. Επίσης, καταχωρείται η αγωνιστική περίοδος στην οποία αναφέρεται. Ακόμα, καταχωρούνται το κόστος και η θέση του εισιτηρίου, αν υπάρχουν. Τέλος, καταχωρείται η ημερομηνία έκδοσης του εισιτηρίου.
- Μπορούμε να παραδώσουμε το εισιτήριο, εκτυπώνοντας όλα τα στοιχεία του (`print`).

Η κλάση “Εκδοτήριο εισιτηρίων”:

- Έχει μια διεύθυνση στην οποία βρίσκεται (`address`), που αναπαριστάνεται ως συμβολοσειρά.
- Αποθηκεύει τα εισιτήρια που έχουν εκδοθεί από αυτό (`issued_tickets`).
- Αποθηκεύει έναν μετρητή που αντιστοιχεί στο πλήθος των εισιτηρίων που έχουν εκδοθεί από το εκδοτήριο αυτό (`tickets_counter`).

Η κλάση “Εκδοτήριο εισιτηρίων” χαρακτηρίζεται από την εξής συμπεριφορά:

- Αρχικά καταχωρείται η διεύθυνση του εκδοτηρίου εισιτηρίων.
- Μπορούμε να εκδώσουμε ένα εισιτήριο από το εκδοτήριο αυτό, δημιουργώντας ένα νέο εισιτήριο με τα απαραίτητα στοιχεία για τη δημιουργία του. Τότε, γίνεται η σχετική ενημέρωση στα εκδοθέντα εισιτήρια και στον μετρητή των εκδοθέντων εισιτηρίων (`issue`).
- Μπορούμε να παραδώσουμε ένα εισιτήριο που έχει ήδη εκδοθεί (`print`).

Η κλάση “Υποκατάστημα”:

- Αποθηκεύει τον αριθμό που το χαρακτηρίζει στην τράπεζα (`branch_no`).
- Αποθηκεύει τα εισιτήρια που έχουν εκδοθεί από αυτό (`issued_tickets`).
- Αποθηκεύει έναν μετρητή που αντιστοιχεί στο πλήθος των εισιτηρίων που έχουν εκδοθεί από το υποκατάστημα αυτό (`tickets_counter`).

Η κλάση “Υποκατάστημα” χαρακτηρίζεται από την εξής συμπεριφορά:

- Αρχικά καταχωρείται ο αριθμός υποκαταστήματος.
- Μπορούμε να εκδώσουμε ένα εισιτήριο από το υποκατάστημα, δημιουργώντας ένα νέο εισιτήριο με τα απαραίτητα στοιχεία για τη δημιουργία του. Τότε, γίνεται η σχετική ενημέρωση στα εκδοθέντα εισιτήρια και στον μετρητή των εκδοθέντων εισιτηρίων (*issue*).
- Μπορούμε να παραδώσουμε ένα εισιτήριο που έχει ήδη εκδοθεί (*print*).

Η κλάση “Λογαριασμός πελάτη”:

- Αποθηκεύει το όνομα χρήστη (*username*).
- Αποθηκεύει τα εισιτήρια που έχει αγοράσει ο χρήστης (*tickets*).
- Αποθηκεύει το συνολικό κόστος των εισιτηρίων που έχει αγοράσει ο χρήστης (*total*).

Η κλάση “Λογαριασμός πελάτη” χαρακτηρίζεται από την εξής συμπεριφορά:

- Αρχικά, καταχωρείται το όνομα χρήστη.
- Μπορούμε να εκδώσουμε ένα εισιτήριο, δημιουργώντας ένα νέο εισιτήριο με τα απαραίτητα στοιχεία για τη δημιουργία του. Τότε, γίνεται η σχετική ενημέρωση στα αγορασθέντα εισιτήρια και στο συνολικό κόστος (*buy*).

Η κλάση “Στατιστική”:

- Αποθηκεύει τον μέγιστο αριθμό εισιτηρίων ανά αγωνιστική περίοδο (*capacities*).

Η κλάση “Στατιστική” χαρακτηρίζεται από την εξής συμπεριφορά:

- Αρχικά, καταχωρεί τους μέγιστους αριθμούς εισιτηρίων ανά αγωνιστική περίοδο.
- Εκτυπώνει, για ένα σημείο έκδοσης εισιτηρίων ή για ένα αγώνισμα ή για μια ημερομηνία έκδοσης, το πλήθος των εισιτηρίων που εκδόθηκαν, δεδομένων των σημείων έκδοσης εισιτηρίων (Εκδοτήρια εισιτηρίων, Υποκαταστήματα, Λογαριασμοί πελατών) (*print_tickets*).
- Δεδομένων των σημείων έκδοσης εισιτηρίων (Εκδοτήρια εισιτηρίων, Υποκαταστήματα, Λογαριασμοί πελατών), υπολογίζει και εκτυπώνει τη μέση πληρότητα των αγωνιστικών περιόδων, για μια συγκεκριμένη ημέρα (*print_average*).

Να υλοποιήσετε σε C++ τις παραπάνω κλάσεις, ορίζοντας νέες όπου δείχνει ενδεικνυόμενο.

11.2.4 Έστω ότι έχουμε να υλοποιήσουμε σε C++ τη συμπεριφορά μιας ομάδας ποδοσφαίρου σε έναν αγώνα, για ένα παιχνίδι Η/Υ. Προκειμένου να γίνει η υλοποίηση αυτή, θεωρούμε τις κλάσεις: “παίκτης” (*Player*), “προπονητής” (*Coach*) και “ομάδα” (*Team*). Επίσης θεωρούμε και τις κλάσεις “επιθετικός παίκτης” (*Attacker*) και “αμυντικός παίκτης” (*Defender*).

Η κλάση “παίκτης”:

- Ανήκει σε μια ομάδα, καταχωρώντας το όνομά της (*team*).
- Έχει έναν αριθμό, μοναδικό για την ομάδα που ανήκει (*number*).
- Έχει επιτύχει έναν αριθμό από γκολ (*goals*).
- Έχει χρεωθεί έναν αριθμό από φάουλ (*fouls*).
- Έχει χρεωθεί έναν αριθμό από κίτρινες κάρτες (*yellow_cards*).
- Έχει ένδειξη αν έχει πάρει κόκκινη κάρτα ή όχι (*red_card*).

- Έχει ένδειξη αν κατέχει την μπάλα ή όχι (`ball`).

Η κλάση “παίκτης” χαρακτηρίζεται από την εξής συμπεριφορά:

- Αρχικά, ανατίθεται το όνομα της ομάδας στην οποία ανήκει, καθώς και ο αριθμός του. Επίσης, θεωρείται ότι ούτε έχει επιτύχει ούτε έχει χρεωθεί τίποτα.
- Βάζει γκολ, αυξάνοντας τα γκολ του, καθώς και εκείνα της ομάδας του κατά ένα (`succeed_goal`).
- Χρεώνεται φάουλ, αυξάνοντας τα φάουλ του, καθώς και εκείνα της ομάδας του κατά ένα (`make_foul`).
- Χρεώνεται κίτρινη κάρτα (`shown_yellow_card`), αυξάνοντας τις κίτρινες κάρτες του, καθώς και εκείνες της ομάδας του κατά μία. Στην περίπτωση που έχει ήδη κίτρινη κάρτα, χρεώνεται και κόκκινη κάρτα.
- Χρεώνεται κόκκινη κάρτα, ενημερώνοντας τη σχετική ένδειξη. Τότε, αποβάλλεται από τους παίκτες της ομάδας και αυξάνονται και οι κόκκινες κάρτες της ομάδας του κατά μία (`shown_red_card`).
- Μεταβιβάζει την μπάλα σε έναν συμπαίκτη του, αν την κατέχει (`pass_ball`).

Η κλάση “επιθετικός παίκτης”, επιπλέον της κλάσης “παίκτης”:

- Κερδίζει φάουλ, αναφωνώντας "`Foul!!`" (`got_foul`).
- Κάνει σουτ, αυξάνοντας τα σουτ της ομάδας του κατά ένα (`shoot`).

Η κλάση “αμυντικός παίκτης”, επιπλέον της κλάσης “παίκτης”:

- Έχει κάποιον παίκτη της αντίπαλης ομάδας τον οποίο μαρκάρει (`pair`). Αυτός ο αντίπαλος παίκτης του ανατίθεται κατά την αρχικοποίηση.
- Αποσπά την μπάλα από τον παίκτη που μαρκάρει, αν αυτός έχει την μπάλα, και πλέον την κατέχει (`get_ball`).
- Κάνει φάουλ στον παίκτη που μαρκάρει, χρεωνόμενος ο ίδιος φάουλ και επιτρέποντας στον άλλο παίκτη να κερδίσει φάουλ (`make_foul`).

Η κλάση “προπονητής”:

- Ανήκει σε μια ομάδα, καταχωρώντας το όνομά της (`team`).

και χαρακτηρίζεται από την εξής συμπεριφορά:

- Αρχικά, ανατίθεται το όνομα της ομάδας στην οποία ανήκει.
- Προτρέπει έναν παίκτη της ομάδας του να κάνει σουτ (`urge_shoot`).
- Προτρέπει έναν παίκτη της ομάδας του να αποσπάσει την μπάλα από τον παίκτη που μαρκάρει (`urge_block`).
- Προτείνει αλλαγή μεταξύ δύο παικτών της ομάδας του, βγάζοντας τον έναν από τον αγώνα και βάζοντας τον άλλο και αυξάνοντας τις αλλαγές της ομάδας του κατά μία (`substitute`).

Η κλάση “ομάδα”:

- Έχει ένα όνομα (`name`).
- Έχει το πολύ έντεκα παίκτες που παίζουν στον αγώνα (`players`).

- Έχει έναν προπονητή (`coach`).
- Έχει έναν αριθμό από γκολ που έχει πετύχει (`goals`).
- Έχει έναν αριθμό από κίτρινες κάρτες που έχουν χρεωθεί οι παίκτες της (`yellow_cards`).
- Έχει έναν αριθμό από κόκκινες κάρτες που έχουν χρεωθεί οι παίκτες της (`red_cards`).
- Έχει έναν αριθμό από φάουλ που έχουν χρεωθεί οι παίκτες της (`fouls`).
- Έχει έναν αριθμό από σουτ που έχουν επιτύχει οι παίκτες της (`shoots`).
- Έχει έναν αριθμό από αποβολές που έχουν γίνει (`offs`).
- Έχει έναν αριθμό από αλλαγές που έχουν γίνει (`substitutions`).

και χαρακτηρίζεται από την εξής συμπεριφορά:

- Αρχικά ανατίθεται το όνομα της ομάδας, ο προπονητής και οι έντεκα παίκτες με τους οποίους ξεκινά τον αγώνα.
- Εκτυπώνει διάφορες πληροφορίες για τον αγώνα (ό,τι θέλετε) (`statistics`).

Να υλοποιήσετε σε C++ τις παραπάνω κλάσεις.

11.2.5 Να υλοποιηθεί σε C++ ένα μέρος της λειτουργικότητας ενός ηλεκτρονικού καταστήματος. Συγκεκριμένα, υλοποιήστε τους πελάτες (`customers`) και τα προϊόντα (`items`) του. Τα προϊόντα συνδέονται ανά ζεύγη, οπότε το ζεύγος προσφέρεται σε ιδιαίτερη (μειωμένη) τιμή. Επίσης, προκειμένου να παρέχει τη δυνατότητα προτάσεων αγοράς (`recommendations`) προς τους πελάτες του, διατηρεί συστάδες συναφών προϊόντων (`item_clusters`). Αυτές έχουν προκύψει από το ιστορικό των αγορών με κοινό σημείο τους πελάτες που τα αγόρασαν. Οι προτάσεις προς έναν πελάτη προκύπτουν ταιριάζοντας τα προϊόντα που έχει αγοράσει ο πελάτης στο παρελθόν με κάποια από τις συστάδες. Οι συστάδες θεωρούνται γνωστές και ο αλγόριθμος κατασκευής τους, καθώς και εκείνος του ταιριάσματος ενός συνόλου προϊόντων με μια συστάδα, δεν αφορά την υλοποίηση.

Η κλάση “Πελάτης”:

- Έχει μια ταυτότητα (`id`).
- Έχει ένα καλάθι προϊόντων που έχει επιλέξει για αγορά (`items_basket`).
- Έχει ένα ποσό που πρέπει να πληρώσει και αντιστοιχεί στην αξία των προϊόντων του καλαθιού (`total_cost`).
- Έχει ένα σύνολο από προϊόντα που έχει αγοράσει στο παρελθόν (`past_items`).

Η κλάση “Πελάτης” χαρακτηρίζεται από την εξής συμπεριφορά:

- Κατά την κατασκευή του, εκχωρείται η ταυτότητά του και το ιστορικό των προϊόντων που έχει αγοράσει. Το καλάθι του είναι κενό και η αντίστοιχη αξία ίση με μηδέν.
- Επιλέγει ένα προϊόν για αγορά. Τότε, αυτό εκτυπώνεται στην οθόνη και, αν υπάρχει απόθεμα από αυτό, προστίθεται στο καλάθι του και αυξάνεται το ποσόν που είναι να πληρώσει κατά την αξία του προϊόντος. Αν επιλέξει το προϊόν και το προϊόν-ταίρι, αφού γίνει έλεγχος αποθέματος και για το ταίρι, προστίθενται και τα δύο στο καλάθι και η συνολική αξία που προστίθεται αφορά τη μειωμένη τιμή.
- Πραγματοποιεί αγορά, μηδενίζοντας το ποσό που πρέπει να πληρώσει, αδειάζοντας το καλάθι και προσθέτοντας τα περιεχόμενα του καλαθιού στο σύνολο των προϊόντων που έχει αγοράσει. Πραγματοποιείται αγορά των προϊόντων του καλαθιού με βάση την ταυτότητά του.

- Με βάση τις συστάδες προϊόντων του καταστήματος, του γίνονται προτάσεις αγοράς ως σύνολο προϊόντων. Οι προτάσεις αυτές παρέχονται από τη χρήση μιας συνάρτησης που ενεργεί με βάση ένα σύνολο προϊόντων και ένα σύνολο συστάδων.

Η κλάση “Προϊόν”:

- Έχει μια ταυτότητα (`id`).
- Έχει μια ονομασία (`name`).
- Έχει μια τιμή (`price`).
- Έχει την ποσότητα του προϊόντος σε απόθεμα (`stock`).
- Έχει ένα ταίρι-προϊόν. Το ταίρι αυτό περιέχει ένα άλλο προϊόν, καθώς και την τιμή που κοστίζει το ζεύγος αγοράς (`partner`).
- Έχει το σύνολο των ταυτοτήτων των πελατών που το έχουν αγοράσει στο παρελθόν (`customers`).

Η κλάση “Προϊόν” χαρακτηρίζεται από την εξής συμπεριφορά:

- Κατά την κατασκευή του, εκχωρείται η ταυτότητά του, η ονομασία του, η τιμή του, το αρχικό απόθεμά του και το ιστορικό των ταυτοτήτων των πελατών που το έχουν αγοράσει. Το ταίρι-προϊόν είναι κενό.
- Ανατίθεται το ταίρι-προϊόν. Κατά την ανάθεση, ενημερώνεται και το αντίστοιχο πεδίο του προϊόντος του ταιριού.
- Ελέγχεται αν υπάρχει απόθεμα από το προϊόν.
- Εκτυπώνεται στην οθόνη, εκτυπώνοντας την ονομασία του, την τιμή του, το απόθεμά του και το ταίρι-προϊόν (όνομα του άλλου προϊόντος και μειωμένη συνολική τιμή).
- Πραγματοποιείται αγορά του προϊόντος, μειώνοντας κατά ένα το απόθεμα — υποθέτουμε ότι η ποσότητα από το κάθε προϊόν που μπορεί να αγοράσει ένας πελάτης είναι μόνο μια μονάδα — και ενημερώνονται οι ταυτότητες των πελατών που το έχουν αγοράσει με την ταυτότητά του πελάτη που το αγοράζει.

Τα προϊόντα υποδιαιρούνται στις παρακάτω κατηγορίες:

“βιβλία” (`books`): αυτά επιπλέον έχουν συγγραφέα (`author`) και έτος έκδοσης (`year`).

“DVDs” (`DVDs`): αυτά επιπλέον έχουν έτος παραγωγής (`year`).

“παιχνίδια και ηλεκτρονικά” (`toys_and_games`): αυτά επιπλέον έχουν εύρος ηλικιών για τα οποία είναι κατάλληλα (`ages`).

Για τις παραπάνω κατηγορίες, η αρχικοποίηση και η εκτύπωση στην οθόνη λαμβάνει υπόψη της και τα παραπάνω στοιχεία.

Οι συστάδες προϊόντων είναι σύνολα προϊόντων.

Να υλοποιήσετε σε C++ τις παραπάνω κλάσεις, ορίζοντας νέες όπου δείχνει ενδεικνυόμενο. Θεωρήστε δεδομένο έναν τύπο λίστας με συναρτήσεις προσθήκης στοιχείου και συνένωσης λιστών. Θεωρήστε δεδομένη τη συνάρτηση παροχής προτάσεων προϊόντων, με βάση ένα σύνολο προϊόντων και ένα σύνολο συστάδων. (Για ενημέρωσή σας, η παραπάνω περιγραφή ανταποκρίνεται στις απαιτήσεις του αλγόριθμου Item-to-Item Collaborative Filtering.)

11.2.6 Να υλοποιηθεί σε C++ η λειτουργικότητα ενός μουσείου. Συγκεκριμένα, υλοποιήστε το μουσείο (museum), τις αίθουσες (rooms) και τους επισκέπτες (visitors).

Η κλάση “Μουσείο”:

- Έχει ένα σύνολο αιθουσών (rooms) και έναν αριθμό αιθουσών.
- Έχει ένα σύνολο επισκεπτών που βρίσκονται στο μουσείο (visitors).
- Έχει μια χωρητικότητα επισκεπτών που μπορούν να βρίσκονται στο μουσείο (capacity).
- Έχει έναν χρόνο αναμονής εισόδου (waiting_time).

Η κλάση “Μουσείο” χαρακτηρίζεται από την εξής συμπεριφορά:

- Κατά την κατασκευή του, δημιουργούνται οι αίθουσές του, με βάση το πλήθος τους και το είδος των ενδιαφερόντων τους. Επίσης, καταχωρείται η χωρητικότητά του, καθώς και ο χρόνος επίσκεψής του. Αρχικά δεν έχει κανένα επισκέπτη.
- Προστίθεται επισκέπτης, αν δεν παραβιάζεται η χωρητικότητα των επισκεπτών. Τότε, ο επισκέπτης προστίθεται στο σύνολο των επισκεπτών που βρίσκονται στο μουσείο.
- Ελέγχει αν πάνω από τις μισές αίθουσές του παρουσιάζουν ένα συγκεκριμένο ενδιαφέρον.
- Επιστρέφει τον χρόνο επίσκεψής του, με βάση τον χρόνο αναμονής του και τον χρόνο επίσκεψης των αιθουσών του.

Η κλάση “Αίθουσα”:

- Έχει ένα σύνολο ενδιαφερόντων που παρουσιάζουν τα εκθέματά της (interests).
- Έχει έναν χρόνο επίσκεψης (time).

Η κλάση “Αίθουσα” χαρακτηρίζεται από την εξής συμπεριφορά:

- Κατά την κατασκευή της, εκχωρείται το σύνολο των ενδιαφερόντων της. Επίσης, εκχωρείται και ο χρόνος επίσκεψής της, ο οποίος υπολογίζεται τυχαία.
- Ελέγχεται αν παρουσιάζει ένα συγκεκριμένο ενδιαφέρον.
- Επιστρέφεται ο χρόνος επίσκεψής της.

Η κλάση “Επισκέπτης”:

- Έχει μια ταυτότητα (id).
- Έχει ένα σύνολο ενδιαφερόντων (interests).
- Έχει έναν διαθέσιμο χρόνο για επίσκεψη σε μουσείο (time).

Η κλάση “Επισκέπτης” χαρακτηρίζεται από την εξής συμπεριφορά:

- Κατά την κατασκευή του, εκχωρείται η ταυτότητά του, τα ενδιαφέροντά του και ο διαθέσιμος χρόνος του.
- Επιλέγει να επισκεφτεί ένα μουσείο, ελέγχοντας αν ικανοποιούνται οι περιορισμοί ενδιαφερόντων και χρόνου.
- Επισκέπτεται ένα μουσείο, προσθέτοντας τον στο μουσείο αυτό.

Να υλοποιήσετε σε C++ τις παραπάνω κλάσεις.

11.2.7 Έστω ότι έχουμε να υλοποιήσουμε σε C++ μία προσομοίωση παιδικής χαράς. Η παιδική χαρά έχει μια μέγιστη χωρητικότητα παιδιών — θεωρούμε μόνο παιδιά —. Αποτελείται από πέντε τραμπάλες. Επίσης, τα παιδιά μπορούν να παίζουν διελκυστίνδα.

Στις τραμπάλες, υποχρεωτικά, κάθεται αγόρι με αγόρι και κορίτσι με κορίτσι.

Στη διελκυστίνδα, τα παιδιά χωρίζονται σε δύο ομάδες και φτιάχνουν δύο αλυσίδες, τραβώντας το ένα το άλλο. Σε κάθε αλυσίδα, το καθένα παιδί τραβιέται από ένα άλλο εκτός από το τελευταίο που δεν τραβιέται από κανένα. Επίσης, το πρώτο τραβά ένα σκοινί από μια άκρη. Την άλλη άκρη του σκοινιού τραβά το πρώτο παιδί της άλλης αλυσίδας. Ο νικητής είναι η ομάδα που, με το τράβηγμα, θα αποσπάσει το σκοινί. Στη διελκυστίνδα, τα παιδιά διατάσσονται στην κάθε αλυσίδα κατά φθίνουσα σειρά δύναμης — το πιο δυνατό τραβά το σκοινί —. Σε κάθε χρονική στιγμή, η μια αλυσίδα μπορεί να έχει το πολύ ένα παιδί περισσότερο από την άλλη. Επίσης, υπάρχει ένα μέγιστο πλήθος παιδιών που μπορούν να παίζουν.

Για την προσομοίωση, να χρησιμοποιηθούν οι κλάσεις “Παιδική Χαρά” (Playground), “Διελκυστίνδα” (war), “Τραμπάλες” (Seesaws) και “Τραμπάλα” (Seesaw).

Η κλάση “Παιδική Χαρά”:

- Έχει μια μέγιστη χωρητικότητα παιδιών (capacity).
- Έχει ένα πλήθος παιδιών που είναι μέσα (children_in).
- Έχει 5 τραμπάλες (seesaws).
- Έχει μια ένδειξη σε διελκυστίνδα (war).

Η κλάση “Παιδική Χαρά” χαρακτηρίζεται από την εξής συμπεριφορά:

- Αρχικά, καταχωρείται η χωρητικότητά της, είναι άδεια από παιδιά, οπότε δεν έχει σχηματιστεί και διελκυστίνδα.
- Ένα παιδί μπαίνει στην παιδική χαρά, αν χωράει. Αν δεν χωράει, εκτυπώνεται σχετικό μήνυμα. Αν χώρεσε, αυξάνεται το πλήθος παιδιών που είναι μέσα κατά μία μονάδα. Αρχικά, δοκιμάζει να μπει στη διελκυστίνδα. Αν δεν χωράει, δοκιμάζει να μπει στις τραμπάλες (enter).
- Γίνεται λήξη διελκυστίνδας, αναδεικνύοντας ποια είναι η νικήτρια αλυσίδα και μηδενίζοντας την ένδειξη στη διελκυστίνδα (war_end).

Η κλάση “Διελκυστίνδα”:

- Έχει μια μέγιστη χωρητικότητα παιδιών (capacity).
- Έχει δύο σύνολα παιδιών (left_children, right_children).

Η κλάση “Διελκυστίνδα” χαρακτηρίζεται από την εξής συμπεριφορά:

- Αρχικά, καταχωρείται η χωρητικότητά της και τα σύνολα παιδιών είναι κενά.
- Ένα παιδί μπαίνει στη διελκυστίνδα, αν δεν υπερβαίνει τη χωρητικότητά της, τοποθετούμενο στην κατάλληλη θέση με βάση τη δύναμή του σε μια από τις δύο αλυσίδες (enter).
- Αναδεικνύεται η νικήτρια αλυσίδα, όπως προκύπτει από το άθροισμα των δυνάμεων των παιδιών που την αποτελούν (decide_winner).

Η κλάση “Τραμπάλες”:

- Έχει 5 τραμπάλες (seesaws).

Η κλάση “Τραμπάλες” χαρακτηρίζεται από την εξής συμπεριφορά:

- Ένα παιδί μπαίνει στις τραμπάλες, αν μπορεί να μπει σε μια τραμπάλα. Αν δεν υπάρχει διαθέσιμη τραμπάλα, εκτυπώνεται σχετικό μήνυμα (enter).

Η κλάση “Τραμπάλα”:

- Έχει δύο ενδείξεις σε παιδιά (left_child, right_child).

Η κλάση “Τραμπάλα” χαρακτηρίζεται από την εξής συμπεριφορά:

- Αρχικά, είναι κενή από παιδιά.
- Ένα παιδί μπαίνει στην τραμπάλα, αν του είναι δυνατό — είτε είναι άδεια είτε μόνο η μία θέση είναι κατειλημμένη και το παιδί που την καταλαμβάνει είναι του ίδιου φύλου —. Αν η εισαγωγή είναι επιτυχής, καταχωρείται στην τραμπάλα μια ένδειξη προς αυτό το παιδί (enter).

Τα “Παιδιά” δομούνται από το φύλο τους, (M ή F), και τη δύναμή τους (ακέραιος).

Να υλοποιήσετε σε C++ τις παραπάνω κλάσεις. Θεωρήστε δεδομένο έναν τύπο “λίστας” όπως σας βολεύει.

11.2.8 Έστω ότι έχουμε να υλοποιήσουμε σε C++ μία απλοϊκή προσομοίωση της εισόδου και εξόδου αυτοκινήτων σε ένα γκαράζ. Στο γκαράζ θεωρούμε ότι σταθμεύουν τόσο (επιβατικά) αυτοκίνητα, όσο και φορτηγά, σε τρεις ορόφους. Το γκαράζ έχει μια μέγιστη χωρητικότητα για στάθμευση αυτοκινήτων και μια μέγιστη χωρητικότητα για στάθμευση φορτηγών, ισοκατανεμημένες στους τρεις ορόφους. Προκειμένου να εισέλθει στο γκαράζ ένα αυτοκίνητο ή ένα φορτηγό, προστίθεται το τέλος μιας ουράς εισόδου που οδηγεί στο γκαράζ. Το πρώτο αυτοκίνητο ή φορτηγό της ουράς αφαιρείται από την ουρά και μπαίνει στο γκαράζ, σταθμεύοντας στον πρώτο όροφο που θα βρεθεί ελεύθερη θέση. Βγαίνει από το γκαράζ το τελευταίο αυτοκίνητο ή φορτηγό (ότι είναι αυτό) που στάθμευσε σε έναν όροφο.

Προκειμένου να γίνει η υλοποίηση αυτή, θεωρούμε τις κλάσεις: “επιβατικό αυτοκίνητο” (Car), “φορτηγό” (Truck), “γκαράζ” (Garage), “όροφος” (Floor) και “ουράΕισόδου” (EntranceQueue).

Η κλάση “επιβατικό αυτοκίνητο”:

- Έχει έναν όροφο στον οποίο έχει σταθμεύσει (floor).

Η κλάση “επιβατικό αυτοκίνητο” χαρακτηρίζεται από την εξής συμπεριφορά:

- Αρχικά, δεν έχει σταθμεύσει σε κάποιον όροφο.
- Το αυτοκίνητο εισέρχεται σε μια ουρά εισόδου γκαράζ, εισάγοντας στην ουρά εισόδου το αυτοκίνητο (enter).
- Το αυτοκίνητο σταθμεύει σε έναν όροφο, εάν επιτρέπεται από τον αντίστοιχο μετρητή, αναθέτοντας τον όροφο αυτόν ως τιμή του ορόφου του και αυξάνοντας τον μετρητή αυτοκινήτων (park).
- Το αυτοκίνητο βγαίνει, αναθέτοντας στον όροφό του μηδενική τιμή και μειώνοντας τον μετρητή αυτοκινήτων του ορόφου στον οποίο είχε σταθμεύσει (exit).
- Το αυτοκίνητο επιστρέφει τον όροφο στον οποίο έχει σταθμεύσει (get_floor).
- Εκτυπώνεται μήνυμα πληρότητας αυτοκινήτων "Car places are full" (print_wait_message).

Η κλάση “φορτηγό αυτοκίνητο”:

- Έχει έναν όροφο στον οποίο έχει σταθμεύσει (floor).

Η κλάση “φορτηγό αυτοκίνητο” χαρακτηρίζεται από την εξής συμπεριφορά:

- Αρχικά, δεν έχει σταθμεύσει σε κάποιον όροφο.
- Το φορτηγό εισέρχεται σε μια ουρά εισόδου γκαράζ, εισάγοντας στην ουρά εισόδου το φορτηγό (enter).
- Το φορτηγό σταθμεύει σε έναν όροφο, εάν επιτρέπεται από τον αντίστοιχο μετρητή, αναθέτοντας τον όροφο αυτόν ως τιμή του ορόφου του και αυξάνοντας τον μετρητή φορτηγών (park).
- Το φορτηγό βγαίνει, αναθέτοντας στον όροφο του μηδενική τιμή και μειώνοντας τον μετρητή φορτηγών του ορόφου στον οποίο είχε σταθμεύσει (exit).
- Το φορτηγό επιστρέφει τον όροφο στον οποίο έχει σταθμεύσει (get_floor).
- Εκτυπώνεται μήνυμα πληρότητας φορτηγών "Truck places are full" (print_wait_message).

Η κλάση “γκαράζ”:

- Έχει τρεις ορόφους (floor1, floor2, floor3).
- Έχει ένα μέγιστο πλήθος αυτοκινήτων (χωρητικότητα αυτοκινήτων) που επιτρέπεται να είναι σταθμευμένα στο γκαράζ (car_max).
- Έχει ένα μέγιστο πλήθος φορτηγών (χωρητικότητα φορτηγών) που επιτρέπεται να είναι σταθμευμένα στο γκαράζ (truck_max).

Η κλάση “γκαράζ” χαρακτηρίζεται από την εξής συμπεριφορά:

- Αρχικά, ανατίθενται οι χωρητικότητες αυτοκινήτων και φορτηγών.
- Ένα αυτοκίνητο ή φορτηγό μπαίνει σε αυτό, μπαίνοντας στον πρώτο όροφο που χωράει. Αν δεν υπάρχει τέτοιος, εκτυπώνεται το σχετικό μήνυμα πληρότητας (enter).
- Ένα αυτοκίνητο ή φορτηγό βγαίνει από αυτό, βγαίνοντας από τον όροφο που είναι σταθμευμένο (exit).

Η κλάση “όροφος”:

- Έχει έναν μετρητή πλήθους αυτοκινήτων που είναι σταθμευμένα στον όροφο (car_counter).
- Έχει έναν μετρητή πλήθους φορτηγών που είναι σταθμευμένα στον όροφο (truck_counter).
- Έχει ένα σύνολο αυτοκινήτων ή φορτηγών που είναι σταθμευμένα σε αυτόν, πλήθους το πολύ όσο το άθροισμα των χωρητικοτήτων του (parked_Car_Trucks).

Η κλάση “όροφος” χαρακτηρίζεται από την εξής συμπεριφορά:

- Αρχικά είναι άδειος.
- Ένα αυτοκίνητο ή φορτηγό μπαίνει σε αυτόν, αν μπορεί να σταθμεύσει, και προστίθεται στο σύνολο των σταθμευμένων οχημάτων (enter).
- Αφαιρείται το τελευταίο σταθμευμένο αυτοκίνητο ή φορτηγό από τον όροφο, μηδενίζοντας τη θέση του και βγάζοντάς το από τον όροφο (exit).

Η κλάση “ουράΕισόδου”:

- Έχει ένα γκαράζ στο οποίο οδηγεί (garage).
- Έχει μια ουρά οχημάτων (queue_Car_Trucks).

Η κλάση “ουράΕισόδου” χαρακτηρίζεται από την εξής συμπεριφορά:

- Αρχικά της ανατίθεται το γκαράζ.
- Ένα αυτοκίνητο ή φορτηγό μπαίνει σε αυτή, προστιθέμενο στο τέλος της ουράς της (enter).
- Το πρώτο αυτοκίνητο ή φορτηγό βγαίνει από αυτή, αφαιρούμενο από την ουρά της και μπαίνοντας στο γκαράζ (exit).

Υλοποιήστε τις κατάλληλες κλάσεις, ορίζοντας μέλη-δεδομένα που χρειάζονται και συναρτήσεις-μέλη που υλοποιούν την παραπάνω συμπεριφορά. Θεωρήστε δεδομένο έναν τύπο ουράς με τις λειτουργίες που χρειάζεστε.

11.2.9 Έστω ότι έχουμε να υλοποιήσουμε σε C++ μία προσομοίωση της αίθουσας αναχωρήσεων ενός αεροδρομίου. Η αίθουσα αποτελείται από ένα σύνολο από “Θέσεις εξυπηρέτησης”. Οι επιβάτες που πρόκειται να αναχωρήσουν εξυπηρετούνται ένας-ένας από τις “Θέσεις Εξυπηρέτησης” στις οποίες σχηματίζουν ουρές. Μια “Θέση Εξυπηρέτησης” εξυπηρετεί είτε τους επιβάτες μιας συγκεκριμένης πτήσης εξωτερικού, είτε οποιαδήποτε πτήση εσωτερικού (Σημείωση: αεροπορικές εταιρείες δεν λαμβάνονται υπόψη στην προσομοίωση). Αν ο χρόνος μιας πτήσης εσωτερικού πλησιάζει (π.χ. απομένει μισή ώρα για την αναχώρησή της), καλούνται οι επιβάτες της να προωθηθούν στις ουρές που περιμένουν. Δηλαδή, φεύγουν από τις θέσεις τους και έρχονται στις πρώτες θέσεις της ουράς που βρίσκονται, ο ένας μετά τον άλλον (με οποιονδήποτε τρόπο).

Για την προσομοίωση, να χρησιμοποιηθούν οι κλάσεις “Αναχωρήσεις” (Departures), “Θέση εξυπηρέτησης” (CheckinCounter), “Επιβάτης” (Passenger) και “Πτήση” (Flight).

Η κλάση “Αναχωρήσεις”:

- Έχει 100 Θέσεις εξυπηρέτησης (counters).
- Έχει ένα ρολόι που καταγράφει τον χρόνο σε λεπτά (time).

Η κλάση “Αναχωρήσεις” χαρακτηρίζεται από την εξής συμπεριφορά:

- Αρχικά ο χρόνος είναι μηδενικός και σε κάθε μια από τις θέσεις εξυπηρέτησης καταχωρείται αν εξυπηρετεί εσωτερικές πτήσεις ή τον αριθμό πτήσης που εξυπηρετεί.
- Γίνεται άφιξη επιβάτη, ο οποίος προστίθεται για να περιμένει στη θέση εξυπηρέτησης, στη μικρότερη ουρά από εκείνες που εξυπηρετούν την πτήση του (arrive).
- Γίνεται εξυπηρέτηση, εξυπηρετώντας επιβάτη από τη θέση εξυπηρέτησης με τους περισσότερους επιβάτες σε αναμονή (serve).
- Γίνεται κλήση εσωτερικής πτήσης, προκειμένου να προωθηθούν οι επιβάτες της στις θέσεις εξυπηρέτησης που περιμένουν, αν ο χρόνος της πτήσης πλησιάζει (announce_flight).

Η κλάση “Θέση εξυπηρέτησης”:

- Για τις πτήσεις εξωτερικού, φυλάσσει τον αριθμό της πτήσης ή το μηδέν αν πρόκειται για πτήσεις εσωτερικού (flightno).
- Έχει μια ουρά από επιβάτες (passenger_queue).
- Έχει έναν μετρητή των επιβατών που περιμένουν στην ουρά (waiting_passengers).

Η κλάση “Θέση εξυπηρέτησης” χαρακτηρίζεται από την εξής συμπεριφορά:

- Αρχικά η ουρά των επιβατών είναι κενή και θεωρείται ότι εξυπηρετεί εσωτερικές πτήσεις.
- Επιτρέπει την αλλαγή της πτήσης που εξυπηρετεί (`set_flightno`).
- Εξυπηρετεί, αφαιρώντας επιβάτη από την ουρά και μειώνοντας τον μετρητή των επιβατών που περιμένουν (`serve`).
- Γίνεται άφιξη επιβάτη, ο οποίος προστίθεται για να περιμένει στο τέλος της ουράς και αυξάνεται ο μετρητής των επιβατών που περιμένουν (`arrive`).
- Αν εξυπηρετεί πτήσεις εσωτερικού, καλεί την πτήση, αν πλησιάζει η ώρα αναχώρησής της, για να προωθηθούν οι επιβάτες της στην ουρά (`announce_flight`).

Οι “Πτήσεις” δομούνται από τον αριθμό τους, από την ένδειξη αν είναι εσωτερικού ή όχι και την ώρα αναχώρησής τους.

Οι “Επιβάτες” δομούνται από τον αριθμό διαβατηρίου τους και τον αριθμό της πτήσης τους.

Να υλοποιήσετε σε C++ τις παραπάνω κλάσεις, ορίζοντας νέες όπου δείχνει ενδεικνυόμενο.

11.2.10 Να υλοποιηθεί σε C++ μια προσομοίωση εκλογικού κέντρου. Το εκλογικό κέντρο αποτελείται από δέκα εκλογικά τμήματα. Στο εκλογικό κέντρο προσέρχονται εκλογείς για να ψηφίσουν, κάνοντας μια επιλογή από ένα (συγκεκριμένο) πλήθος κομματιών, λευκό ή άκυρο (για λόγους απλότητας, θεωρήστε τις δυο τελευταίες επιλογές σαν δυο ακόμα κόμματα). Το εκλογικό κέντρο επιτρέπει την είσοδο εκλογέων μέχρι τις 19:00 και κατόπιν κλείνει. Τα εκλογικά τμήματα, όμως, εξυπηρετούν όλους τους εκλογείς που περιμένουν να ψηφίσουν. Οι εκλογείς αναπαρίστανται από το επίθετό τους και την πρόθεση ψήφου τους. Για την προσομοίωση να χρησιμοποιηθούν οι κλάσεις “Εκλογικό Κέντρο” και “Εκλογικό Τμήμα”.

Η κλάση “Εκλογικό Κέντρο”:

- Έχει ένα ρολόι για να δείχνει την ώρα (`roloi`).
- Έχει έναν αριθμό που αντιστοιχεί στους ψηφίζοντες στο κέντρο (`psifizontes`).
- Έχει έναν αριθμό που αντιστοιχεί στους ψηφίσαντες στο κέντρο (`psifisantes`).
- Φυλάσσει το πλήθος των ψήφων που έχουν πάρει τα κόμματα (`apotelesmata`).
- Έχει δέκα εκλογικά τμήματα (`tmimata`).
- Έχει μια ένδειξη αν είναι ανοικτό ή κλειστό (`open`).

Η κλάση “Εκλογικό Κέντρο” χαρακτηρίζεται από την εξής συμπεριφορά:

- Αρχικά, είναι ανοικτό, η ώρα είναι 6:00, κανείς δεν έχει ψηφίσει, καταχωρεί τον αριθμό των ψηφίζόντων του και καταχωρεί τα εύρη των επιθέτων που εξυπηρετούνται από τα τμήματά του.
- Όταν εισέρχεται ένας νέος ψηφοφόρος, αν το κέντρο είναι ανοικτό, πάει στο κατάλληλο τμήμα και προστίθεται να ψηφίσει. Το τμήμα επιλέγεται με βάση το επίθετο του ψηφοφόρου, λαμβάνοντας υπόψη ότι ένα τμήμα εξυπηρετεί ψηφοφόρους από ένα συγκεκριμένο εύρος επιθέτων.
- Τα αποτελέσματα των κομματιών προκύπτουν από τα επιμέρους αποτελέσματα των τμημάτων.
- Το πλήθος των ψηφισάντων στο κέντρο προκύπτει από τους ψηφίσαντες στα τμήματα.
- Αν είναι κλειστό, και δεν υπάρχει κάποιο τμήμα στο οποίο να περιμένουν ψηφοφόροι, ανακοινώνονται τα αποτελέσματά του (κόμματα, άκυρα, λευκά, αποχή).
- Αν η ώρα είναι 19:00, το κέντρο κλείνει.

Η κλάση “Εκλογικό Τμήμα”:

- Έχει το εύρος επιθέτων που εξυπηρετεί (*arxika*).
- Έχει ένα σύνολο ψηφοφόρων που περιμένουν να ψηφίσουν (*oura_psifoforon*).
- Έχει έναν μετρητή για το πόσοι ψηφοφόροι έχουν ψηφίσει (*psifisantes*).
- Φυλάσσει τα μέχρι τώρα αποτελέσματα (*apotelesmata*).

Η κλάση “Εκλογικό Τμήμα” χαρακτηρίζεται από την εξής συμπεριφορά:

- Κατά την κατασκευή του, δεν έχει ψηφίσει κανείς και δεν περιμένει κανείς να ψηφίσει.
- Καταχωρείται το εύρος των επιθέτων που εξυπηρετεί.
- Ένας νέος ψηφοφόρος προστίθεται να ψηφίσει, περιμένοντας στο τέλος των ψηφοφόρων που περιμένουν να ψηφίσουν, αν το επίθετό του ανήκει στο εύρος επιθέτων που εξυπηρετούνται από το τμήμα.
- Ο πρώτος ψηφοφόρος που περιμένει να ψηφίσει, ψηφίζει αυξάνοντας τους ψηφίσαντες του τμήματος κατά ένα, καθώς και τα αποτελέσματα του κόμματος της επιλογής του. Επίσης, αφαιρείται από την ουρά αναμονής των ψηφοφόρων.

Να υλοποιήσετε σε C++ τις παραπάνω κλάσεις, ορίζοντας νέες όπου δείχνει ενδεικνύμενο.

11.2.11 Έστω ότι έχουμε να υλοποιήσουμε σε C++ μία προσομοίωση της εξυπηρέτησης πελατών στα εκδοτήρια/ταμεία ενός Complex Cinema. Έστω ότι η προσομοίωση αυτή θεωρεί τις εξής κλάσεις:

- κλάση: “Ταμείο” που προσομοιώνει την εξυπηρέτηση σ’ ένα ταμείο/εκδοτήριο.
- κλάση: “Cinema” που προσομοιώνει την κίνηση των ταμείων του Complex Cinema.

Η κλάση “Ταμείο”:

- Έχει μία ουρά πελατών, *customers*, όπου ο κάθε πελάτης αντιπροσωπεύεται μόνο από την ταινία που επιθυμεί να παρακολουθήσει. (Σημείωση: για λόγους απλότητας, η ταινία μπορεί να είναι ένας αριθμός από το 1 έως το 8).

Η κλάση “Ταμείο” χαρακτηρίζεται από την εξής συμπεριφορά:

- Αρχικά, ένα ταμείο είναι άδειο.
- Σε ένα ταμείο, ένας πελάτης προστίθεται (*add*) στο τέλος της ουράς του.
- Στο ταμείο εξυπηρετείται (*serve*) ο πρώτος πελάτης κάθε φορά, συμβάλλοντας στον συνολικό τζίρο του Complex Cinema με το αντίτιμο του εισιτηρίου, και διαγράφεται από την ουρά.

Η κλάση “Cinema”:

- Έχει μια σταθερά που αντιστοιχεί στο αντίτιμο του εισιτηρίου.
- Έχει ένα σύνολο από 10 ταμεία (*cashiers*).
- Αποθηκεύει τον τρέχοντα συνολικό τζίρο του, όπως έχει διαμορφωθεί από τα εισιτήρια που έχουν κοπεί απ’ όλα τα ταμεία (*M*).

Η κλάση “Cinema” χαρακτηρίζεται από την εξής συμπεριφορά:

- Αρχικά, ανατίθεται το αντίτιμο του εισιτηρίου του.

- Αυξάνει τον συνολικό τζίρο κατά το αντίτιμο του εισιτηρίου, όταν ένας πελάτης εξυπηρετείται από ένα ταμείο (`increase_amount`).
- Αναγγέλλεται μια ταινία, δίνοντας τον αριθμό που την αναπαριστά, και τότε, για κάθε ταμείο, οι πελάτες που περιμένουν στην ουρά του και ενδιαφέρονται να δουν την ταινία αυτή, μετακινούνται στην αρχή της ουράς, διατηρώντας τη σχετική διάταξη μεταξύ τους (`announce_film`).

Να υλοποιήσετε σε C++ τις δύο αυτές κλάσεις.

11.2.12 Έστω ότι έχουμε να υλοποιήσουμε σε C++ μία προσομοίωση ενός ταχυφαγείου. Το ταχυφαγείο αποτελείται από ένα σύνολο από “Θέσεις εξυπηρέτησης”. Οι πελάτες που εισέρχονται στο ταχυφαγείο εξυπηρετούνται ένας-ένας από τις “Θέσεις Εξυπηρέτησης”, στις οποίες σχηματίζουν ουρές. Όταν εξυπηρετείται ένας πελάτης, δημιουργείται μια παραγγελία. Επίσης, ο πελάτης αφαιρείται από την ουρά της “Θέσης Εξυπηρέτησης” που περίμενε και προστίθεται σε μια λίστα πελατών του ταχυφαγείου, σε αναμονή της παράδοσης της παραγγελίας του.

Μια παραγγελία αναφέρεται στον κωδικό του μενού που περιέχει και σχετίζεται με τη “Θέση Εξυπηρέτησης” από την οποία παραγγέλθηκε. Τελικά, ανατίθεται σε έναν πελάτη (οποιονδήποτε) που έχει παραγγείλει από τη “Θέση Εξυπηρέτησης” αυτή, μενού με κωδικό ίδιο με τον κωδικό του μενού της παραγγελίας.

Για την προσομοίωση, να χρησιμοποιηθούν οι κλάσεις “Ταχυφαγείο” (`FastFoodRestaurant`), “Θέση εξυπηρέτησης” (`Counter`), “Παραγγελία” (`Order`) και “Πελάτης” (`Customer`).

Η κλάση “Ταχυφαγείο”:

- Έχει 5 Θέσεις εξυπηρέτησης (`counters`).
- Έχει μια λίστα από πελάτες σε αναμονή της παράδοσης της παραγγελίας τους (`waiting_customers`).

Η κλάση “Ταχυφαγείο” χαρακτηρίζεται από την εξής συμπεριφορά:

- Αρχικά η λίστα πελατών σε αναμονή είναι κενή.
- Γίνεται άφιξη πελάτη, ο οποίος προστίθεται για να περιμένει στη θέση εξυπηρέτησης στη μικρότερη ουρά (`arrive`).
- Γίνεται εξυπηρέτηση, εξυπηρετώντας πελάτη από τη θέση εξυπηρέτησης με τους περισσότερους πελάτες σε αναμονή, δημιουργώντας μια νέα παραγγελία που αντιστοιχεί στην επιλογή του πελάτη αυτού, αναθέτοντάς του την παραγγελία και προσθέτοντας τον πελάτη στη λίστα πελατών εν αναμονή (`serve`).
- Παραδίδεται παραγγελία σε έναν πελάτη (οποιονδήποτε) από αυτούς εν αναμονή που ταιριάζει η παραγγελία, αφαιρώντας τον από τη λίστα πελατών εν αναμονή (`deliver`).

Η κλάση “Θέση εξυπηρέτησης”:

- Έχει μια ουρά από πελάτες (`customer_queue`).
- Έχει έναν μετρητή των πελατών που περιμένουν στην ουρά (`waiting_customers`).

Η κλάση “Θέση εξυπηρέτησης” χαρακτηρίζεται από την εξής συμπεριφορά:

- Αρχικά η ουρά των πελατών είναι κενή.
- Εξυπηρετεί, αφαιρώντας πελάτη από την ουρά και μειώνοντας τον μετρητή των πελατών που περιμένουν (`serve`).

- Γίνεται άφιξη πελάτη, ο οποίος προστίθεται για να περιμένει στο τέλος της ουράς και αυξάνεται ο μετρητής των πελατών που περιμένουν (arrive).

Οι “Παραγγελίες” δομούνται από τον κωδικό του μενού τους και τον αριθμό της “Θέση εξυπηρέτησης” από την οποία προέκυψαν.

Οι “Πελάτες” δομούνται από τον αριθμό ταυτότητάς τους και την παραγγελία τους. Αρχικά, η παραγγελία είναι κενή.

Να υλοποιήσετε σε C++ τις παραπάνω κλάσεις, ορίζοντας νέες όπου δείχνει ενδεικνυόμενο.

- 11.2.13** Να υλοποιηθεί σε C++ μια προσομοίωση νεανικού κέντρου διασκέδασης. Το κέντρο διασκέδασης αποτελείται από 100 θέσεις καθισμένων και M όρθιων (είναι, δηλαδή, συνολικής χωρητικότητας $100 + M$ πελατών). Επίσης, σερβίρονται τέσσερα ποτά. Για το κάθε ποτό, υπάρχει μια αρχική ποσότητα, κατανεμημένη ισομερώς σε τρεις μπάρες. Ένας πελάτης αναπαρίσταται από την επιλογή του σε ποτό. Για να μπει στο κέντρο, αρκεί το κέντρο να μην είναι πλήρες και να υπάρχει διαθέσιμο απόθεμα από το ποτό της επιλογής του. Όταν μπει, προστίθεται στην ουρά αναμονής της μπάρας με τη μικρότερη ουρά. Μια εξυπηρέτηση πελάτη γίνεται από την μπάρα με τη μεγαλύτερη ουρά αναμονής και μειώνει τη διαθέσιμη ποσότητα ποτού της επιλογής του κατά μια μονάδα. Κατόπιν, αν υπάρχουν διαθέσιμες θέσεις καθισμένων, ο πελάτης αυτός καταλαμβάνει μια από αυτές, αλλιώς μένει όρθιος, μειώνοντας τις διαθέσιμες θέσεις όρθιων κατά μία. Η έξοδος πελατών δεν ενδιαφέρει την προσομοίωση. Για την προσομοίωση, να χρησιμοποιηθούν οι κλάσεις “Νεανικό Κέντρο Διασκέδασης” και “Μπάρα”.

Η κλάση “Νεανικό Κέντρο Διασκέδασης”:

- Έχει τρεις μπάρες (bars).
- Έχει τη διαθεσιμότητα τεσσάρων ποτών (drinks).
- Έχει μια χωρητικότητα όρθιων (standing_capacity).
- Έχει ένα σύνολο 100 θέσεων με ενδεχόμενα καθισμένους πελάτες (sitting).
- Έχει έναν αριθμό που αντιστοιχεί στους όρθιους στο κέντρο (standing).

Η κλάση “Νεανικό Κέντρο Διασκέδασης” χαρακτηρίζεται από την εξής συμπεριφορά:

- Αρχικά, στις μπάρες δεν περιμένει κανείς, δεν κάθεται κανείς και δεν είναι όρθιος κανείς. Επίσης, προσδιορίζεται η διαθεσιμότητα των τεσσάρων ποτών, που μοιράζεται ισομερώς στις τρεις μπάρες.
- Ένας πελάτης εισέρχεται στο κέντρο, αν αυτό είναι επιτρεπτό, και προστίθεται στην ουρά της μπάρας με τη μικρότερη ουρά αναμονής (σε περίπτωση που οι μπάρες με ουρές ελαχίστου μήκους είναι περισσότερες της μίας, επιλέγεται μια οποιαδήποτε μπάρα από αυτές).
- Γίνεται εξυπηρέτηση, εξυπηρετώντας έναν πελάτη από την μπάρα με τη μεγαλύτερη ουρά αναμονής, βάζοντάς τον να καταλάβει μια θέση καθισμένων, αν υπάρχει διαθέσιμη (σε περίπτωση που οι μπάρες με ουρές μεγίστου μήκους είναι περισσότερες της μίας, επιλέγεται μια οποιαδήποτε μπάρα από αυτές). Σε περίπτωση που τέτοια θέση δεν υπάρχει, τον προσθέτει στον αριθμό των όρθιων.
- Οι διαθεσιμότητες του κέντρου σε ποτά προκύπτουν από τις αντίστοιχες διαθεσιμότητες από τις τρεις μπάρες.

Η κλάση “Μπάρα”:

- Έχει τη διαθεσιμότητα των τεσσάρων ποτών (drinks).
- Έχει μια ουρά πελατών που περιμένουν να εξυπηρετηθούν (waiting_queue).

- Έχει το πλήθος των πελατών που περιμένουν να εξυπηρετηθούν (waiting).

Η κλάση “Μπάρα” χαρακτηρίζεται από την εξής συμπεριφορά:

- Κατά την κατασκευή της, δεν περιμένει κανείς και προσδιορίζεται η διαθεσιμότητα των τεσσάρων ποτών.
- Ένας νέος πελάτης προστίθεται να εξυπηρετηθεί, περιμένοντας στο τέλος των πελατών που περιμένουν να εξυπηρετηθούν.
- Ο πρώτος πελάτης που περιμένει να εξυπηρετηθεί εξυπηρετείται, μειώνοντας τη διαθεσιμότητα του ποτού της επιλογής του κατά μια μονάδα. Επίσης, αφαιρείται από την ουρά αναμονής των πελατών. Σημείωση: η διαθεσιμότητα ενός ποτού σε μια μπάρα μπορεί να πάρει και αρνητική τιμή.

Να υλοποιήσετε σε C++ τις παραπάνω κλάσεις, ορίζοντας νέες όπου δείχνει ενδεικνυόμενο.

- 11.2.14** Να υλοποιηθεί σε C++ ένα μέρος της λειτουργικότητας ενός συστήματος αγγελιών εύρεσης εργασίας. Συγκεκριμένα, υλοποιήστε τις αγγελίες (posts) και τους ενδιαφερόμενους (users). Επίσης, υλοποιήστε και την ιεραρχία επαγγελματιών (ενδεικτικό μέρος της). Το σύστημα αποτελείται από ένα σύνολο ενδιαφερόμενων και ένα σύνολο αγγελιών. Ένας ενδιαφερόμενος μπορεί να επιλέξει το πολύ MAX αριθμό αγγελιών. Η επιλογή είναι επιτρεπτή, αν τα χαρακτηριστικά του ενδιαφερόμενου ταιριάζουν με τις ανάγκες της αγγελίας. Μια αγγελία είναι διαθέσιμη για επιλογή, αν ο αριθμός των ενδιαφερομένων για αυτή δεν έχει ξεπεράσει ένα μέγιστο (MAX_CUST). Η έννοια του επαγγέλματος των ενδιαφερομένων αποτελεί μια ιεραρχία, όπου κάθε ειδικευση στο επάγγελμα συνοδεύεται από συγκεκριμένα χαρακτηριστικά. Για παράδειγμα, ο “Προγραμματιστής εφαρμογών του web” (web_programmer) είναι εξειδίκευση του “Προγραμματιστή” (programmer) ο οποίος με τη σειρά του είναι εξειδίκευση του “Πληροφορικού” (computer_scientist). Ένας “Πληροφορικός” έχει γνώσεις από “υλικό”, “λογισμικό”, “μαθηματικά” και “φυσική”. Ένας “Προγραμματιστής” έχει γνώσεις από κάποιες γλώσσες προγραμματισμού. Ένας “Προγραμματιστής εφαρμογών του web” είναι εξοικειωμένος με κάποια περιβάλλοντα και γλώσσες εφαρμογών του web. Τόσο για τους “Προγραμματιστές”, όσο και για τους “Προγραμματιστές εφαρμογών του web”, οι γνώσεις δεν είναι προκαθορισμένες.

Η κλάση “Αγγελία”:

- Έχει μια ταυτότητα (id).
- Έχει έναν τίτλο (title).
- Έχει ένα σύνολο γνώσεων που απαιτούνται από τους ενδιαφερόμενους (skills).
- Έχει ένα ελάχιστο και ένα μέγιστο ηλικιών που, προαιρετικά, έχουν τιμές (min_age), (max_age).
- Έχει, προαιρετικά, ένα ποσό που αντιστοιχεί στον μισθό (salary).
- Έχει μια ένδειξη αν η αγγελία είναι διαθέσιμη για να την επιλέξει κάποιος ενδιαφερόμενος (availability).
- Έχει ένα σύνολο ενδιαφερομένων που την έχουν επιλέξει (users).

Η κλάση “Αγγελία” χαρακτηρίζεται από την εξής συμπεριφορά:

- Κατά την κατασκευή της, εκχωρείται η ταυτότητά της, ο τίτλος της, το σύνολο των γνώσεων που απαιτούνται από τους ενδιαφερόμενους και οι ηλικίες και ο μισθός. Στην περίπτωση που τα τελευταία δεν προσδιορίζονται, δίνεται η τιμή 0. Αρχικά, η αγγελία είναι διαθέσιμη και το σύνολο των ενδιαφερόμενων που την επέλεξαν είναι κενό.

- Επιλέγεται από έναν ενδιαφερόμενο, αν είναι διαθέσιμη και τα χαρακτηριστικά του ενδιαφερόμενου ταιριάζουν με τις απαιτήσεις της (γνώσεις, ηλικία, μισθό). Τότε, προστίθεται ο ενδιαφερόμενος στο σύνολο των ενδιαφερόμενων. Γίνεται έλεγχος, αν μετά την προσθήκη, η αγγελία παύει να είναι διαθέσιμη, οπότε η διαθεσιμότητα αλλάζει.

Η κλάση “Ενδιαφερόμενος”:

- Έχει μια ταυτότητα (`id`).
- Έχει ένα επάγγελμα (`profession`).
- Έχει μια ηλικία (`age`).
- Έχει μια μισθολογική προτίμηση (`salary`).
- Έχει ένα σύνολο αγγελιών τις οποίες έχει επιλέξει (`posts`).

Η κλάση “Ενδιαφερόμενος” χαρακτηρίζεται από την εξής συμπεριφορά:

- Κατά την κατασκευή του, εκχωρείται η ταυτότητά του, το επάγγελμά του, η ηλικία του και η μισθολογική προτίμηση του. Αρχικά, δεν έχει επιλέξει καμία αγγελία.
- Επιλέγει αγγελία, αν δεν έχει φτάσει στο όριο, επιλέγοντάς την και προσθέτοντας την αγγελία στο σύνολο των αγγελιών που έχει επιλέξει — αν η ενέργεια της επιλογής έγινε επιτυχώς από την αγγελία —.

Τα επαγγέλματα, τώρα, έχουν ένα σύνολο γνώσεων που προσφέρουν. Κάθε εξειδίκευση επαγγέλματος, πέρα από τις γνώσεις που προσφέρουν οι γενικότερες κατηγορίες, προσφέρει και κάποιες επιμέρους. Κάποιες κατηγορίες επαγγελμάτων προσφέρουν συγκεκριμένες γνώσεις, για κάποιες άλλες όμως, οι επιπλέον γνώσεις προσδιορίζονται κατά περίπτωση.

Να υλοποιήσετε σε C++ τις παραπάνω κλάσεις, ορίζοντας νέες όπου δείχνει ενδεικνυόμενο.

11.2.15 Έστω ότι έχουμε να υλοποιήσουμε σε C++ μία σκηνή επιδείξεων παγοδρομιών. Στη σκηνή μπαίνουν παγοδρόμοι που τοποθετούνται ανά ζευγάρια διαφορετικού φύλου. Υπάρχει ένας μέγιστος αριθμός ζευγαριών που μπορούν να σχηματιστούν. Οι επιπλέον παγοδρόμοι που μπαίνουν σχηματίζουν δύο αλυσίδες: μία οι άνδρες και μία οι γυναίκες. Όταν, και αν, οι αλυσίδες αυτές φτάσουν και οι δύο σε ένα μέγιστο μήκος, συνενώνονται και φτιάχνουν μια μεγάλη, διπλάσιου μήκους, όπου οι άνδρες και οι γυναίκες τοποθετούνται εναλλάξ. Δεν επιτρέπεται να μουν επιπλέον παγοδρόμοι. Οι παγοδρόμοι αναχωρούν, πρώτα εκείνοι της αλυσίδας και κατόπιν τα ζευγάρια.

Για την υλοποίηση της παραπάνω συμπεριφοράς, να χρησιμοποιηθούν οι κλάσεις “Σκηνή” (`Scene`), “Παγοδρόμος” (`IceSkater`), “Ζευγάρι” (`Couple`), “Αλυσίδα Ιδίου Φύλου” (`SSChain`) και “Αλυσίδα” (`Chain`).

Η κλάση “Σκηνή”:

- Έχει μια μέγιστη χωρητικότητα παγοδρόμων (`capacity`).
- Έχει ένα σύνολο ζευγαριών (`couples`).
- Έχει μια ένδειξη σε αλυσίδα ανδρών (`menchain`).
- Έχει μια ένδειξη σε αλυσίδα γυναικών (`womenchain`).
- Έχει μια ένδειξη σε αλυσίδα παγοδρόμων (`chain`).

Η κλάση “Σκηνή” χαρακτηρίζεται από την εξής συμπεριφορά:

- Αρχικά, καταχωρείται η χωρητικότητά της και είναι άδεια από παγοδρόμους.

- Ένας παγοδρόμος μπαίνει στη σκηνή, αν χωράει. Αν δεν χωράει, εκτυπώνεται σχετικό μήνυμα. Αν χωράει, γίνεται δοκιμή να τοποθετηθεί σε ζευγάρι. Αν αποτύχει η τοποθέτηση αυτή, τοποθετείται στην αλυσίδα του φύλου του, αν μπορεί (enter).
- Γίνεται συνένωση των αλυσίδων ιδίου φύλου, αφαιρώντας τους παγοδρόμους από τις αλυσίδες ιδίου φύλου, έναν από κάθε μία τη φορά, και βάζοντάς τους στην αλυσίδα παγοδρόμων (populate).
- Γίνεται λήξη της σκηνής, κάνοντας αναχώρηση των παγοδρόμων, πρώτα των αλυσίδων και μετά των ζευγαριών (scene_end).

Η κλάση “Παγοδρόμος”:

- Έχει καταχωρημένο το φύλο του, Μ ή F, (sex).

Η κλάση “Παγοδρόμος” χαρακτηρίζεται από την εξής συμπεριφορά:

- Αρχικά, καταχωρείται το φύλο του.
- Ένας παγοδρόμος αναχωρεί, εκτυπώνοντας το μήνυμα "I am gliding out" και το φύλο του (exit).

Η κλάση “Ζευγάρι”:

- Έχει δύο ενδείξεις σε παγοδρόμους (left, right).

Η κλάση “Ζευγάρι” χαρακτηρίζεται από την εξής συμπεριφορά:

- Αρχικά, είναι κενή από παγοδρόμους.
- Ένας παγοδρόμος γίνεται μέλος στο ζευγάρι, αν του είναι δυνατό, δηλαδή είτε είναι κενές και οι δύο θέσεις είτε μόνο η μια θέση είναι κατειλημμένη και ο παγοδρόμος που την καταλαμβάνει είναι του άλλου φύλου. Αν η εισαγωγή είναι επιτυχής, καταχωρείται στο ζευγάρι μια ένδειξη προς αυτό τον παγοδρόμο (enter).

Η κλάση “ΑλυσίδαΙδίουΦύλου”:

- Έχει μια μέγιστη χωρητικότητα παγοδρόμων (capacity).
- Έχει ένα σύνολο παγοδρόμων (ice_skaters).

Η κλάση “ΑλυσίδαΙδίουΦύλου” χαρακτηρίζεται από την εξής συμπεριφορά:

- Αρχικά, καταχωρείται η χωρητικότητά της και το σύνολο των παγοδρόμων είναι κενό. Η χωρητικότητα υπολογίζεται με βάση τη χωρητικότητα της σκηνής και το πλήθος των ζευγαριών των παγοδρόμων.
- Ένας παγοδρόμος μπαίνει στην αλυσίδα, αν είναι του φύλου του και δεν υπερβαίνει τη χωρητικότητά της. Αν δεν μπορεί να μπει, εκτυπώνεται σχετικό μήνυμα (enter).

Η κλάση “Αλυσίδα”:

- Έχει ένα σύνολο παγοδρόμων (ice_skaters).

Η κλάση “Αλυσίδα” χαρακτηρίζεται από την εξής συμπεριφορά:

- Αρχικά το σύνολο των παγοδρόμων είναι κενό.

- Ένας παγοδρόμος μπαίνει στην αλυσίδα σε άκρη που υπάρχει παγοδρόμος του άλλου φύλου. Αν δεν μπορεί να μπει, εκτυπώνεται σχετικό μήνυμα (enter).

Να υλοποιήσετε σε C++ τις παραπάνω κλάσεις. Θεωρήστε δεδομένο έναν τύπο “λίστας” όπως σας βολεύει, απλώς τεκμηριώστε τον.

11.2.16 Έστω ότι έχουμε να υλοποιήσουμε σε C++ μία προσομοίωση ενός θερινού σινεμά. Έστω ότι η προσομοίωση αυτή θεωρεί τις εξής κλάσεις:

- κλάση: “Ταμείο” που προσομοιώνει την εξυπηρέτηση στο ταμείο του θερινού.
- κλάση: “Bar” που προσομοιώνει την εξυπηρέτηση στο μπαρ του θερινού.
- κλάση: “Cinema” που προσομοιώνει την εξυπηρέτηση στο θερινό σινεμά.

Η κλάση “Ταμείο”:

- Έχει μία ουρά θεατών (customers).

Η κλάση “Ταμείο” χαρακτηρίζεται από την εξής συμπεριφορά:

- Αρχικά, το ταμείο είναι άδειο.
- Στο ταμείο, ένας θεατής προστίθεται (add) στο τέλος της ουράς του.
- Στο ταμείο εξυπηρετείται (serve) ο πρώτος θεατής κάθε φορά, διαγράφοντάς τον από την ουρά.

Η κλάση “Bar”:

- Έχει μία ουρά θεατών (customers).
- Αποθηκεύει τον τζίρο των πωλήσεων (M).

Η κλάση “Bar” χαρακτηρίζεται από την εξής συμπεριφορά:

- Αρχικά, το μπαρ είναι άδειο.
- Στο μπαρ, ένας θεατής προστίθεται (add) στο τέλος της ουράς του.
- Στο μπαρ, εξυπηρετείται (serve) ο πρώτος θεατής κάθε φορά, συμβάλλοντας στον τζίρο με το ποσό αγοράς του, και διαγράφεται από την ουρά.

Η κλάση “Cinema”:

- Έχει ένα ταμείο (cashier).
- Έχει ένα μπαρ (bar).
- Έχει μια σταθερά που αντιστοιχεί στο αντίτιμο του εισιτηρίου (ticket_price).
- Αποθηκεύει τον τρέχοντα συνολικό τζίρο του (M).

Η κλάση “Cinema” χαρακτηρίζεται από την εξής συμπεριφορά:

- Αρχικά, ανατίθεται το αντίτιμο του εισιτηρίου του και ο συνολικός τζίρος είναι μηδέν.
- Κάνει εξυπηρέτηση, δημιουργώντας θεατή και προσθέτοντάς τον στο ταμείο. Κατόπιν, εξυπηρετεί το ταμείο, αυξάνοντας τον συνολικό τζίρο κατά το αντίτιμο του εισιτηρίου. Αν ο θεατής που εξυπηρετήθηκε θέλει να αγοράσει κάτι από το μπαρ, τον προσθέτει στο μπαρ και τυχαία επιλέγει να εξυπηρετήσει το μπαρ ή όχι (serve).

Ο θεατής χαρακτηρίζεται από το ποσό αγοράς του από το μπαρ. Το ποσό αυτό δημιουργείται τυχαία κατά τη δημιουργία του θεατή. Αν δεν θέλει να αγοράσει κάτι από το μπαρ, το ποσό αυτό είναι μηδέν.

Να υλοποιήσετε σε C++ τις παραπάνω κλάσεις.

11.2.17 Έστω ότι έχουμε να υλοποιήσουμε σε C++ μία προσομοίωση εισαγωγής επισκεπτών σε ένα κτήριο δημόσιας υπηρεσίας. Προκειμένου να γίνει η υλοποίηση αυτή θεωρούμε τις κλάσεις: “επισκέπτης” (Visitor), “κτήριο” (Building), “όροφος” (Floor), “χώρος εισόδου” (Entrance) και “γραφείο” (Office).

Η κλάση “επισκέπτης”:

- Έχει ένα κτήριο που θέλει να επισκεφτεί (building).
- Έχει έναν αριθμό γραφείου, του γραφείου που επιθυμεί να επισκεφτεί (no_office).

Η κλάση “επισκέπτης” χαρακτηρίζεται από την εξής συμπεριφορά:

- Αρχικά, ανατίθεται το κτήριο και ο αριθμός του γραφείου που θέλει να επισκεφτεί.
- Μπαίνει στο κτήριο, αν του επιτρέπεται (enter).

Η κλάση “κτήριο”:

- Έχει τέσσερις ορόφους (floors).
- Έχει έναν πίνακα αντιστοίχισης αριθμών γραφείων και αριθμών ορόφων, που συσχετίζει τον αριθμό του γραφείου με τον αριθμό του ορόφου στον οποίο βρίσκεται το γραφείο (table).

Η κλάση “κτήριο” χαρακτηρίζεται από την εξής συμπεριφορά:

- Ένας επισκέπτης μπαίνει στο κτήριο, βρίσκοντας τον αριθμό του ορόφου στον οποίο ανήκει ο αριθμός του γραφείου που θέλει να επισκεφτεί και μπαίνει στον όροφο αυτό, αν αυτό επιτρέπεται (enter).

Η κλάση “όροφος”:

- Έχει έναν χώρο εισόδου (entrance).
- Έχει δέκα γραφεία (offices).

Η κλάση “όροφος” χαρακτηρίζεται από την εξής συμπεριφορά:

- Αρχικά, αναθέτει τις ενδείξεις των γραφείων του στον χώρο εισόδου του.
- Ένας επισκέπτης μπαίνει στον όροφο, μπαίνοντας στον χώρο εισόδου του, αν του επιτρέπεται (enter).

Η κλάση “χώρος εισόδου”:

- Έχει μια μέγιστη χωρητικότητα (capacity).
- Έχει έναν μετρητή για το πλήθος των επισκεπτών που βρίσκονται μέσα σε αυτόν τη στιγμή αυτή (no_of_visitors).
- Έχει ενδείξεις των γραφείων του ορόφου στον οποίο ανήκει (p_offices).

Η κλάση “χώρος εισόδου” χαρακτηρίζεται από την εξής συμπεριφορά:

- Αρχικά, είναι άδειος και του ανατίθεται η μέγιστη χωρητικότητά του.

- Ένας επισκέπτης μπαίνει στον χώρο εισόδου, αν δεν υπερβαίνεται η μέγιστη χωρητικότητά του. Αν του επιτρέπεται να μπει στο γραφείο που επιθυμεί, μπαίνει στο γραφείο. Διαφορετικά, παραμένει στον χώρο εισόδου, αυξάνοντας τον μετρητή του πλήθους των επισκεπτών του (enter).

Η κλάση “γραφείο”:

- Έχει μια μέγιστη χωρητικότητα (capacity).
- Έχει έναν μετρητή για το πλήθος των επισκεπτών που βρίσκονται μέσα σε αυτό τη στιγμή αυτή (no_of_visitors).

Η κλάση “γραφείο” χαρακτηρίζεται από την εξής συμπεριφορά:

- Αρχικά, είναι άδειο και του ανατίθεται η μέγιστη χωρητικότητά του.
- Ένας επισκέπτης μπαίνει στο γραφείο, αν δεν υπερβαίνεται η μέγιστη χωρητικότητά του, αυξάνοντας τον μετρητή του πλήθους των επισκεπτών του (enter).

Υλοποιήστε τα παραπάνω μέσω καταλλήλων κλάσεων, ορίζοντας μέλη-δεδομένα που χρειάζονται και συναρτήσεις-μέλη που υλοποιούν την παραπάνω συμπεριφορά.

11.2.18 Υλοποιήστε σε C++ την προσομοίωση της υπηρεσίας διαλογής ενός κέντρου διανομής Ταχυδρομείου, από το οποίο διανέμονται γράμματα και δέματα.

Κάθε δέμα και κάθε γράμμα έχει μια διεύθυνση παραλήπτη. Αυτή αποτελείται από τον δρόμο και τον αριθμό. Τα γράμματα ενδέχεται να είναι συστημένα ή επείγοντα. Τα δέματα ενδεχόμενα χαρακτηρίζονται ως ογκώδη, οπότε αντί για το ίδιο το δέμα, διανέμεται ένα απλό γράμμα που έχει τη διεύθυνση του δέματος.

Κάθε τι που διανέμεται χαρακτηρίζεται από μονάδες διανομής. Κάθε απλό γράμμα έχει μία μονάδα διανομής. Ένα συστημένο γράμμα ή ένα επείγον γράμμα έχει δύο μονάδες διανομής. Ένα μη ογκώδες δέμα έχει τρεις μονάδες διανομής.

Τα γράμματα και τα δέματα που πρέπει να διανεμηθούν, καθώς αφικνούνται στο ταχυδρομείο, προστίθενται σε μια λίστα διανομής. Τα επείγοντα γράμματα, όμως, φυλάσσονται σε άλλη λίστα διανομής.

Το κέντρο αρχικά δεν έχει τίποτα να διανείμει. Στο κέντρο προστίθεται ένα γράμμα για διανομή (arrive), τοποθετώντας το στην κατάλληλη λίστα για διανομή. Στο κέντρο προστίθεται ένα δέμα για διανομή (arrive), προωθώντας το κατάλληλα για διανομή. Το κέντρο χαρακτηρίζεται από το πλήθος των μονάδων διανομής των αντικειμένων που οφείλει να διανείμει, καθώς και το πλήθος των ογκωδών δεμάτων που φυλάσσει για παράδοση.

Ένα γράμμα αρχικοποιείται, αναθέτοντας τη διεύθυνση παραλήπτη.

Ένα δέμα αρχικοποιείται, αναθέτοντας τη διεύθυνση παραλήπτη.

Υλοποιήστε τα παραπάνω μέσω καταλλήλων κλάσεων, ορίζοντας μέλη-δεδομένα που χρειάζονται και συναρτήσεις-μέλη που υλοποιούν την παραπάνω συμπεριφορά.

11.2.19 Προκειμένου να ολοκληρώσετε τις σπουδές σας, ενδέχεται να εκπονήσετε Πτυχιακή Εργασία. Η εργασία αυτή θα εντάσσεται σε μια επιστημονική περιοχή των σπουδών σας. Κατά την εκπόνηση της εργασίας, θα πρέπει να μελετήσετε και τη σχετική βιβλιογραφία. Όταν ολοκληρωθεί η εργασία σας, θα πρέπει να συντάξετε και να παραδώσετε μια αναφορά (κείμενο), στην οποία να την περιγράφετε. Στην αναφορά αυτή, θα πρέπει να συμπεριλάβετε και τη βιβλιογραφία (references) την οποία μελετήσατε, με τη μορφή συνόλου “βιβλιογραφικών αναφορών”.

Μια βιβλιογραφική αναφορά (reference) μπορεί να είναι είτε κάποιο άρθρο από επιστημονικό περιοδικό (article), είτε εργασία σε πρακτικά συνεδρίου (inproceedings), είτε κεφάλαιο από βιβλίο (inbook), είτε βιβλίο (book), είτε διατριβή (thesis), είτε σελίδα του παγκόσμιου ιστού (url).

Μια βιβλιογραφική αναφορά χαρακτηρίζεται από τρεις λέξεις-κλειδιά που αποτελούν χαρακτηριστικά της περιοχής που εντάσσεται (π.χ. προγραμματισμός, λειτουργικά συστήματα, αλγόριθμοι).

Ένα άρθρο από επιστημονικό περιοδικό συμπεριλαμβάνει τα ονόματα των συγγραφέων, τον τίτλο του άρθρου, τον αριθμό του τόμου του περιοδικού, έναν επιπλέον αριθμό που αφορά υποδιαίρεση του τόμου, τον εκδότη του περιοδικού (π.χ. Elsevier) και χρονολογία έκδοσης του άρθρου.

Μια εργασία σε πρακτικά συνεδρίου συμπεριλαμβάνει τα ονόματα των συγγραφέων, τον τίτλο της εργασίας, τον τίτλο του συνεδρίου, τον εκδότη των πρακτικών και χρονολογία διεξαγωγής του συνεδρίου.

Ένα κεφάλαιο από βιβλίο συμπεριλαμβάνει τα ονόματα των συγγραφέων, τον τίτλο του κεφαλαίου, τον τίτλο του βιβλίου, τον εκδότη του βιβλίου και τη χρονολογία έκδοσης του βιβλίου.

Ένα βιβλίο συμπεριλαμβάνει τα ονόματα των συγγραφέων, τον τίτλο του βιβλίου, τον εκδότη του βιβλίου και τη χρονολογία έκδοσης του βιβλίου.

Μια διατριβή συμπεριλαμβάνει το όνομα του συγγραφέα, τον τίτλο της, το όνομα του ιδρύματος στο οποίο εκπονήθηκε (π.χ. Ε.Κ.Π.Α.) και τη χρονολογία της.

Μια σελίδα παγκόσμιου ιστού συμπεριλαμβάνει τη διεύθυνσή της και την ημερομηνία της τελευταίας επίσκεψής σας σε αυτήν.

Κάθε μία από τις παραπάνω αναφορές αρχικοποιείται με τα στοιχεία που συμπεριλαμβάνει. Δίνεται η δυνατότητα συνολικής εκτύπωσης των στοιχείων της (`print`), καθώς και η δυνατότητα ανάκτησης των λέξεων-κλειδιών που τη χαρακτηρίζει (`get_keywords`).

Για στατιστικούς λόγους, μας ενδιαφέρει να μπορούμε να μάθουμε, πόσες αναφορές από το κάθε είδος έχει η βιβλιογραφία μας (`get_no_article`, `get_no_inproceedings`, `get_no_inbook`, `get_no_book`, `get_no_thesis` και `get_no_url`).

Η βιβλιογραφία μιας εργασίας αρχικά είναι κενή. Δίνεται η δυνατότητα προσθήκης μιας αναφοράς στη βιβλιογραφία (`add_reference`). Στην περίπτωση αυτή, προστίθεται μια ήδη υπάρχουσα αναφορά στη βιβλιογραφία. Μπορεί να γίνει εκτύπωση όλων των αναφορών μιας βιβλιογραφίας (`print`). Επίσης, δίνεται η δυνατότητα εκτύπωσης των στατιστικών της (`print_stats`). Τέλος, δίνεται η δυνατότητα εξαγωγής των τριών κυριαρχουσών λέξεων-κλειδιών των αναφορών της βιβλιογραφίας (`get_keywords`).

Υλοποιήστε την παραπάνω πληροφορία σε C++, υλοποιώντας κατάλληλες κλάσεις.

11.2.20 Υλοποιήστε σε C++ την πληροφορία μιας απλουστευμένης βάσης φιλομορφίας (`examMdb`). Η βάση περιέχει πληροφορίες για (κινηματογραφικές) ταινίες (`films`), ηθοποιούς (`actors`) και σκηνοθέτες (`directors`).

Οι χρήστες μπορούν να ψηφίζουν τους αγαπημένους τους ηθοποιούς και τις αγαπημένες τους ταινίες. Ο ηθοποιός που θα συγκεντρώσει τις περισσότερες ψήφους χαρακτηρίζεται ως ο πλέον δημοφιλής ηθοποιός. Ο σκηνοθέτης ο οποίος έχει σκηνοθετήσει ταινίες που συνολικά έχουν συγκεντρώσει τις περισσότερες ψήφους, χαρακτηρίζεται ως ο πλέον δημοφιλής σκηνοθέτης.

Τόσο οι ηθοποιοί, όσο και οι σκηνοθέτες, έχουν ονοματεπώνυμο, ημερομηνία γέννησης και ένα σύνολο ταινιών στις οποίες έχουν συμμετάσχει. Ένας ηθοποιός έχει καταχωρημένο και το πλήθος ψήφων που έχει λάβει.

Τόσο οι ηθοποιοί, όσο και οι σκηνοθέτες, αρχικοποιούνται με τα στοιχεία του ονοματεπωνύμου τους και της ημερομηνίας γέννησής τους. Αρχικά, θεωρείται ότι δεν έχουν συμμετάσχει σε καμία ταινία. Οι ηθοποιοί, αρχικά, δεν έχουν καμία ψήφο.

Οι ηθοποιοί και οι σκηνοθέτες συμμετέχουν σε μια ταινία (`participates`), προσθέτοντας την ταινία αυτή σε αυτές που έχουν συμμετάσχει.

Οι ηθοποιοί λαμβάνουν ψήφο (`get_voted`), αυξάνοντας το πλήθος των ψήφων τους κατά 1. Το πλήθος αυτό των ψήφων τους μπορεί να ανακτηθεί (`get_votes`).

Οι σκηνοθέτες συλλέγουν ψήφους (`get_votes`), αθροίζοντας τις ψήφους όλων των ταινιών τις οποίες έχουν σκηνοθετήσει.

Οι ταινίες έχουν έναν τίτλο, ένα έτος παραγωγής, έναν σκηνοθέτη, ένα σύνολο ηθοποιών που συμμετέχουν και το πλήθος ψήφων που έχουν λάβει.

Αρχικά, στις ταινίες καταχωρούνται ο τίτλος τους, το έτος παραγωγής τους, οι ηθοποιοί τους και ο σκηνοθέτης τους. Αρχικά, οι ψήφοι των ταινιών είναι μηδέν.

Μια ταινία λαμβάνει ψήφους (`get_voted`), προσθέτοντας μια μονάδα στις ψήφους της. Το πλήθος αυτό των ψήφων τους μπορεί να ανακτηθεί (`get_votes`).

Τόσο οι ηθοποιοί και οι σκηνοθέτες, όσο και οι ταινίες, παρέχουν τη δυνατότητα εκτύπωσης των στοιχείων τους (`print`).

Η βάση της φιλομογραφίας περιέχει ηθοποιούς, σκηνοθέτες και ταινίες. Αρχικοποιείται με κάποιους ηθοποιούς, σκηνοθέτες και ταινίες.

Στη βάση, μπορούμε:

- Να ψηφίσουμε έναν ηθοποιό, ψηφίζοντας τον ηθοποιό (`vote`).
- Να ψηφίσουμε μια ταινία, ψηφίζοντας την ταινία (`vote`).
- Να ανακτήσουμε τον “πλέον δημοφιλή ηθοποιό” (`get_favorite_actor`).
- Να ανακτήσουμε τον “πλέον δημοφιλή σκηνοθέτη” (`get_favorite_director`).

Υλοποιήστε την παραπάνω πληροφορία σε C++, υλοποιώντας κατάλληλες κλάσεις, ορίζοντας τα μέλη-δεδομένα που χρειάζονται και συναρτήσεις-μέλη που υλοποιούν την παραπάνω συμπεριφορά. Θεωρήστε δεδομένο έναν τύπο λίστας.

11.2.21 Υλοποιήστε σε C++ την πληροφορία ενός ταξιδιωτικού πρακτορείου (`TravelAgency`). Το ταξιδιωτικό πρακτορείο έχει πελάτες (`Customer`) και προσφέρει τουριστικά πακέτα (`Package_tour`). Το τουριστικό πακέτο χαρακτηρίζεται από μια σειρά πόλεων (`City`), συνοδευόμενες από τον αριθμό των διανυκτερεύσεων σε κάθε μία από αυτές, και τη σειρά από τις συνδέσεις μετακίνησης μεταξύ πόλεων (`Travel`). Το πρακτορείο δίνει τιμές στα πακέτα. Η τιμή ενός πακέτου προκύπτει από το κόστος του πακέτου, επαυξημένη με ένα ποσοστό που αναλογεί στο κέρδος του πρακτορείου. Το κόστος του πακέτου προκύπτει από το κόστος των συνδέσεων μετακίνησης από πόλη σε πόλη, επαυξημένο με το συνολικό κόστος της διαμονής στις πόλεις. Ανά πάσα στιγμή, το πρακτορείο μπορεί να υπολογίσει το συνολικό κέρδος του από τα πακέτα που έχει πουλήσει.

Ένας πελάτης έχει μια ταυτότητα και χαρακτηρίζεται από το σύνολο των πακέτων που έχει αγοράσει. Αρχικά έχει μόνο ταυτότητα και δεν έχει αγοράσει κανένα πακέτο. Αγοράζει ένα πακέτο (`buy_package`), προσθέτοντας το πακέτο στα πακέτα που έχει αγοράσει. Δίνεται η δυνατότητα να εκτυπωθούν τα πακέτα που έχει αγοράσει (`print`).

Μια πόλη έχει ένα όνομα και ένα κόστος διανυκτέρευσης και αρχικοποιείται με όλη την πληροφορία αυτή. Μπορούμε να ανακτήσουμε το όνομά της (`get_name`). Επίσης, μπορούμε να ανακτήσουμε το κόστος διανυκτέρευσης (`get_cost`).

Μια σύνδεση μετακίνησης μεταξύ πόλεων χαρακτηρίζεται από τις δύο πόλεις που συνδέει και το κόστος της. Η σύνδεση αυτή μπορεί να είναι είτε αεροπορική πτήση, είτε ακτοπλοϊκή σύνδεση, είτε δρομολόγιο τρένου, είτε δρομολόγιο λεωφορείου. Στην περίπτωση της αεροπορικής πτήσης, συμπεριλαμβάνεται και η αεροπορική εταιρεία. Στην περίπτωση της ακτοπλοϊκής σύνδεσης, συμπεριλαμβάνεται και το όνομα του πλοίου. Όλες οι περιπτώσεις σύνδεσης μετακίνησης αρχικοποιούνται με την πληροφορία που τις

χαρακτηρίζει. Πρέπει να παρέχεται η δυνατότητα να ανακτήσουμε το κόστος της (`get_cost`). Επίσης, πρέπει να δίνεται η δυνατότητα εκτύπωσης όλων των στοιχείων της (`print`). Κατά την εκτύπωση, εκτυπώνεται και το μέσο μετακίνησης.

Ένα πακέτο χαρακτηρίζεται από τη σειρά των πόλεων που το απαρτίζουν, όπου η κάθε πόλη συνοδεύεται από τον αριθμό των διανυκτερεύσεων σε αυτές — η πρώτη πόλη και η τελευταία είναι η πόλη αναχώρησης και η πόλη άφιξης, αντίστοιχα, οπότε δεν υπάρχει διανυκτέρευση σε αυτές —. Το πακέτο χαρακτηρίζεται επίσης και από τη σειρά των συνδέσεων μετακίνησης ανάμεσα στις πόλεις του. Ένα πακέτο αρχικοποιείται με όλη την παραπάνω πληροφορία. Για ένα πακέτο, μπορούμε να υπολογίσουμε το κόστος του (`get_cost`), αθροίζοντας τα κόστη διανυκτερεύσεων στις πόλεις που το αποτελούν, καθώς και τα κόστη των μετακινήσεων μεταξύ των πόλεων του. Επίσης, μπορούμε να το εκτυπώσουμε (`print`), εκτυπώνοντας τα ονόματα των πόλεων που το απαρτίζουν, τον αριθμό διανυκτερεύσεων σε κάθε μία από αυτές και κάνοντας εκτύπωση και των συνδέσεων μετακίνησής του.

Το τουριστικό πρακτορείο, αρχικά, δεν έχει κανένα πελάτη και κανένα τουριστικό πακέτο. Μπορεί να προστεθεί πελάτης (`add_customer`), προσθέτοντάς τον στους ήδη υπάρχοντες. Μπορεί να προστεθεί τουριστικό πακέτο (`add_package_tour`), δημιουργώντας το και προσθέτοντάς το στα ήδη υπάρχοντα. Ένας ήδη υπάρχων πελάτης μπορεί να αγοράσει ένα πακέτο (`buy_package`). Αν δεν είναι στους ήδη υπάρχοντες, πρέπει και να προστεθεί σε αυτούς. Για ένα πακέτο, καθορίζουμε την τιμή του (`define_price`), αυξάνοντας το κόστος του κατά ένα ποσοστό κέρδους το οποίο αποτελεί σταθερά του πρακτορείου. Επίσης, μπορεί να υπολογιστεί το κέρδος που έχει το πρακτορείο με βάση τα πακέτα που έχουν αγοράσει οι πελάτες του (`evaluate_profit`). Τέλος, για έναν πελάτη, μπορεί να εκτυπώσει τα πακέτα που έχει αγοράσει (`print`).

Υλοποιήστε την παραπάνω πληροφορία σε C++, υλοποιώντας κατάλληλες κλάσεις, ορίζοντας τα μέλη-δεδομένα που χρειάζονται και συναρτήσεις-μέλη που υλοποιούν την παραπάνω συμπεριφορά. Θεωρήστε δεδομένους τους τύπους δεδομένων που θα χρησιμοποιήσετε.

11.2.22 Ας υποθέσουμε ότι για την εκτίμηση της δυσκολίας (φόρτου) ενός ακαδημαϊκού εξαμήνου σε ένα πανεπιστημιακό τμήμα ισχύουν τα παρακάτω.

Το εξάμηνο αποτελείται από 5 (πέντε) μαθήματα. Το κάθε μάθημα περιλαμβάνει θεωρία (διδασκαλία με τη μορφή παραδόσεων και φροντιστηρίων), αλλά μπορεί να περιλαμβάνει και εργαστήριο ή/και ένα πρακτικό μέρος. Η θεωρία χαρακτηρίζεται από τον αριθμό των ωρών ανά εβδομάδα που διαρκεί η διδασκαλία. Το ίδιο και το εργαστήριο. Το εργαστήριο όμως μπορεί να έχει υποχρεωτική παρακολούθηση ή όχι. Το πρακτικό μέρος αποτελείται από μια σειρά από παραδοτέες ασκήσεις. Η κάθε άσκηση αποτελείται από ένα κείμενο που αντιστοιχεί στην εκφώνησή της.

Ο υπολογισμός της δυσκολίας (`evaluate_load`) του κάθε εξαμήνου προκύπτει από τη συνολική δυσκολία των μαθημάτων του.

Ο υπολογισμός της δυσκολίας (`evaluate_load`) ενός μαθήματος προκύπτει από το άθροισμα της δυσκολίας της θεωρίας και της δυσκολίας του εργαστηρίου του πολλαπλασιασμένο με έναν συντελεστή (σταθερά του μαθήματος). Επιπλέον, προστίθεται και η δυσκολία του πρακτικού μέρους.

Η δυσκολία της θεωρίας (`evaluate_load`) προκύπτει από το πλήθος των ωρών της. Η δυσκολία του εργαστηρίου (`evaluate_load`) προκύπτει από το πλήθος των ωρών του. Αν το εργαστήριο δεν είναι υποχρεωτικό, μειώνεται στο μισό.

Ο υπολογισμός της δυσκολίας (`evaluate_load`) του πρακτικού μέρους προκύπτει από το άθροισμα της δυσκολίας των επιμέρους ασκήσεων. Η δυσκολία μιας άσκησης είναι σταθερά της άσκησης.

Το εξάμηνο αρχικοποιείται με την πληροφορία των μαθημάτων που περιλαμβάνει. Ένα μάθημα αρχικοποιείται με τις πληροφορίες για τη θεωρία του, για το εργαστήριο και για το πρακτικό μέρος. Η θεωρία αρχικοποιείται με το πλήθος των ωρών διδασκαλίας της. Το εργαστήριο αρχικοποιείται με το πλήθος

των ωρών διδασκαλίας του και την ένδειξη αν είναι υποχρεωτικό ή όχι. Το πρακτικό μέρος αρχικοποιείται με τις ασκήσεις που το αποτελούν. Μια τέτοια άσκηση αρχικοποιείται με το κείμενο της εκφώνησής της. Κατά την αρχικοποίηση, παρέχονται οι πληροφορίες και για τις σταθερές, όπου χρειάζεται.

Τόσο για το εξάμηνο, αλλά και για τα μαθήματα, τη θεωρία, το εργαστήριο και το πρακτικό μέρος, να παρέχονται συναρτήσεις εκτύπωσης (print) που να εκτυπώνουν τις πληροφορίες τους.

Να υλοποιήσετε την παραπάνω πληροφορία μέσω κλάσεων C++.

11.3 Μικρές Εργασίες για Εξάσκηση στην Αφαίρεση στα Δεδομένα

Τα προβλήματα του τμήματος αυτού δεν απαιτούν (συνήθως) χρήση κληρονομικότητας. Σκοπό, λοιπόν, έχουν να σας εξασκήσουν στα βασικά θέματα της αφαίρεσης στα δεδομένα που αφορούν σχεδιασμό και υλοποίηση κλάσεων.

Επίσης, οι παρακάτω ασκήσεις έχουν σκοπό να σας εξασκήσουν στις συναρτήσεις κατασκευής-καταστροφής, στην περίπτωση που έχουμε σύνθεση κλάσεων, και στην εμβέλεια και προσβασιμότητα μεταξύ των μελών των κλάσεων.

Εκφωνήσεις Εργασιών

11.3.1 Υλοποιήστε σε C++ μια προσομοίωση πολυκατοικίας. Η δομή της είναι στατική. Δηλαδή, η πολυκατοικία αποτελείται από πέντε ορόφους και κεραμοσκεπή. Κάθε όροφος αποτελείται από δύο διαμερίσματα. Το ένα διαμέρισμα του κάθε ορόφου αποτελείται από τέσσερα δωμάτια και το άλλο διαμέρισμα από δύο.

Στην πολυκατοικία, ανά πάσα στιγμή, βρίσκονται N άτομα ($N \geq 0$). Ο κάθε όροφος φιλοξενεί κάποια από αυτά. Στον κάθε όροφο, τα άτομα βρίσκονται μέσα στα διαμερίσματα του ορόφου. Οι επισκέπτες ενός διαμερίσματος βρίσκονται σε διάφορα δωμάτια του διαμερίσματος.

Κατά την κατασκευή μιας πολυκατοικίας, εκτυπώνεται ένα μήνυμα ότι μια πολυκατοικία κατασκευάζεται. Κατά την κατασκευή ενός ορόφου, εκτυπώνεται ένα μήνυμα ότι ένας όροφος κατασκευάζεται. Κατά την κατασκευή ενός διαμερίσματος, εκτυπώνεται ένα μήνυμα ότι ένα διαμέρισμα κατασκευάζεται. Κατά την κατασκευή ενός δωματίου, εκτυπώνεται ένα μήνυμα ότι ένα δωμάτιο κατασκευάζεται. Κατά την κατασκευή μιας κεραμοσκεπής, εκτυπώνεται ένα μήνυμα ότι μια κεραμοσκεπή κατασκευάζεται. Κατά την κατασκευή τους, τόσο η πολυκατοικία, όσο και οι όροφοι, τα διαμερίσματα και τα δωμάτια δεν περιέχουν κανένα άτομο.

Στην περίπτωση κατεδάφισης (καταστροφής) της πολυκατοικίας, εκτυπώνεται σχετικό μήνυμα. Σχετικά μηνύματα εκτυπώνονται και στις περιπτώσεις καταστροφής ορόφου, διαμερίσματος, δωματίου και κεραμοσκεπής.

Η πολυκατοικία δέχεται επίσκεψη ενός ατόμου (visit) που θα επισκεφθεί ένα διαμέρισμα ενός ορόφου. Τόσο το διαμέρισμα, όσο και ο όροφος, προσδιορίζονται. Το πλήθος των ατόμων που βρίσκονται στην πολυκατοικία αυξάνεται κατά ένα. Αντίστοιχα, ένας επισκέπτης μπορεί να αποχωρήσει (exit) από την πολυκατοικία, προσδιορίζοντας τον όροφο και το διαμέρισμά του από τα οποία θα αποχωρήσει. Σε αυτήν την περίπτωση, το πλήθος των ατόμων που βρίσκονται στην πολυκατοικία μειώνεται κατά ένα.

Η επίσκεψη ενός ατόμου σε έναν όροφο (visit), προσδιορίζοντας το διαμέρισμα που θα επισκεφθεί, γίνεται αυξάνοντας το πλήθος των ατόμων που βρίσκονται στον όροφο και κατόπιν κάνοντας επίσκεψη στο διαμέρισμα. Η αποχώρηση ενός ατόμου από έναν όροφο (exit), δεδομένου ενός διαμερίσματος του ορόφου, γίνεται μειώνοντας το πλήθος των ατόμων που βρίσκονται στον όροφο και κατόπιν αποχωρώντας από το διαμέρισμα.

Η επίσκεψη ενός ατόμου σε ένα διαμέρισμα (visit) γίνεται αυξάνοντας το πλήθος των ατόμων που βρίσκονται στο διαμέρισμα και κατόπιν κάνοντας επίσκεψη σε ένα δωμάτιο του διαμερίσματος

(τυχαίο). Η αποχώρηση από ένα διαμέρισμα (exit) γίνεται με μείωση του πλήθους των ατόμων που βρίσκονται στο διαμέρισμα και με αποχώρηση ενός ατόμου από ένα δωμάτιο (οποιοδήποτε, αρκεί να έχει κάποιο άτομο μέσα).

Η επίσκεψη σε ένα δωμάτιο (visit) γίνεται αυξάνοντας το πλήθος των ατόμων που βρίσκονται στο δωμάτιο. Η αποχώρηση από ένα δωμάτιο (exit) γίνεται μειώνοντας το πλήθος των ατόμων που βρίσκονται στο δωμάτιο.

Υλοποιήστε τα παραπάνω, μέσω των κατάλληλων κλάσεων, ορίζοντας τα μέλη-δεδομένα που χρειάζονται και τις συναρτήσεις-μέλη που υλοποιούν την παραπάνω συμπεριφορά. Να ορίσετε εσείς, όπου κρίνετε αναγκαίο, συναρτήσεις ανάκτησης τιμών (accessors) για τις κλάσεις που θα υλοποιήσετε.

Για να αναδείξετε την προσομοίωσή σας, να υλοποιήσετε μια συνάρτηση main, η οποία να δημιουργεί μια πολυκατοικία και κατόπιν να πραγματοποιεί K επισκέψεις και αποχωρήσεις.

11.3.2 Υλοποιήστε σε C++ μια σκηνή κινουμένων σχεδίων (scene). Η σκηνή εκτυλίσσεται σε ένα δωμάτιο και αποτελείται από τα εξής:

- Το υπόβαθρο του δωματίου (background).
- Μια γιαγιά (old_lady).
- Έναν αρουραίο (rat).

Όταν δημιουργείται μια σκηνή, εκτυπώνεται το μήνυμα "Scene was just created!". Όταν καταστρέφεται μια σκηνή, εκτυπώνεται το μήνυμα "Scene is about to be destroyed". Για τη σκηνή, ορίζεται η συνάρτηση-μέλος που υλοποιεί την πλοκή της (plot). Η πλοκή έχει ως εξής:

*Ο αρουραίος μπαίνει στο δωμάτιο.
Ο αρουραίος οσφραίνεται.
Η γιαγιά μπαίνει στο δωμάτιο.
Η γιαγιά πανικοβάλλεται.
Η γιαγιά πυροβολεί πέντε φορές.
Ο αρουραίος πανικοβάλλεται.
Το ταβάνι του υπόβαθρου καταρρέει.
Η γιαγιά πανικοβάλλεται.*

Οι οντότητες που αποτελούν τη σκηνή περιγράφονται ως εξής:

Το υπόβαθρο αποτελείται από τα εξής:

- Ένα ταβάνι (ceiling).
- Μια πολυθρόνα (chair).
- Ένα τραπέζι (table).
- Ένα παράθυρο (window).
- Ένα τυράκι (cheese).

Αρχικοποιείται, εκτυπώνοντας το μήνυμα "I just created the background!". Καταστρέφεται, εκτυπώνοντας το μήνυμα "Background is about to be destroyed".

Το ταβάνι δεν έχει δεδομένα. Αρχικοποιείται, εκτυπώνοντας το μήνυμα "Ceiling was just created!". Καταστρέφεται, εκτυπώνοντας το μήνυμα "Ceiling is about to be destroyed". Επίσης, το ταβάνι καταρρέει, εκτυπώνοντας το μήνυμα "Ceiling collapses!".

Η πολυθρόνα δεν έχει δεδομένα. Αρχικοποιείται, εκτυπώνοντας το μήνυμα "A chair was just created!". Καταστρέφεται, εκτυπώνοντας το μήνυμα "A chair is about to be destroyed".

Το τραπέζι δεν έχει δεδομένα. Αρχικοποιείται, εκτυπώνοντας το μήνυμα "A table was just created!". Καταστρέφεται, εκτυπώνοντας το μήνυμα "A table is about to be destroyed".

Το παράθυρο δεν έχει δεδομένα. Αρχικοποιείται, εκτυπώνοντας το μήνυμα "A window was just created!". Καταστρέφεται, εκτυπώνοντας το μήνυμα "A window is about to be destroyed".

Το τυράκι δεν έχει δεδομένα. Αρχικοποιείται, εκτυπώνοντας το μήνυμα "Cheese was just created and smells wonderful!". Καταστρέφεται, εκτυπώνοντας το μήνυμα "Cheese is about to be destroyed".

Σημείωση: Στα παραπάνω, δεν θεωρούμε ότι κάποια πληροφορία χρειάζεται απόκρυψη.

Η γιαγιά έχει την πληροφορία σχετικά με το μέγεθός της στην οθόνη (appearance). Αυτό μπορεί να πάρει μια από τις εξής τιμές: small, medium, large και invisible. Η συμπεριφορά της γιαγιάς είναι η εξής:

Κατά την αρχικοποίηση, εκτυπώνεται το μήνυμα "Old lady participates" και το μέγεθός της παίρνει την τιμή medium. Κατά την καταστροφή, εκτυπώνεται το μήνυμα "Old lady cannot be used any longer!". Η γιαγιά εισέρχεται στη σκηνή (enter), εκτυπώνοντας το μήνυμα "Home sweet home ...". Η γιαγιά πυροβολεί (shoot), εκτυπώνοντας το μήνυμα "Bang!". Η γιαγιά πανικοβάλλεται (panic), αλλάζοντας την τιμή της εμφάνισής της σε large και εκτυπώνοντας το μήνυμα "Oh my God!".

Ο αρουραίος έχει την πληροφορία σχετικά με το μέγεθός του στην οθόνη (appearance). Αυτό μπορεί να πάρει μια από τις εξής τιμές: small, medium, large και invisible. Η συμπεριφορά ενός αρουραίου είναι η εξής:

Κατά την αρχικοποίηση, εκτυπώνεται το μήνυμα "A rat participates" και το μέγεθός του παίρνει την τιμή small. Κατά την καταστροφή, εκτυπώνεται το μήνυμα "The rat cannot be used any longer!". Ο αρουραίος εισέρχεται στη σκηνή (enter), εκτυπώνοντας το μήνυμα "Entering this place ...". Ο αρουραίος οσφραίνεται (smell), εκτυπώνοντας το μήνυμα "It smells nice!". Ο αρουραίος πανικοβάλλεται (panic), εκτυπώνοντας το μήνυμα "Panic, panic, panic!" και βγαίνοντας από το δωμάτιο. Ο αρουραίος βγαίνει από το δωμάτιο (exit), εκτυπώνοντας το μήνυμα "Exiting ASAP!" και αλλάζοντας την τιμή της εμφάνισής του σε invisible.

Υλοποιήστε τα παραπάνω, μέσω των κατάλληλων κλάσεων, ορίζοντας τα μέλη-δεδομένα που χρειάζονται και τις συναρτήσεις-μέλη που υλοποιούν την παραπάνω συμπεριφορά.

Για να δοκιμάσετε την υλοποίησή σας, χρησιμοποιήστε την παρακάτω συνάρτηση main

```
int main()
{
    Scene scene;
    scene.plot();

    return 0;
}
```

11.3.3 Υλοποιήστε σε C++ την προσομοίωση ενός επιβατικού πλοίου. Στο πλοίο υπάρχουν τέσσερις όροφοι. Ο κάθε όροφος έχει 25 δίκλινες καμπίνες και 75 τετράκλινες καμπίνες.

Το πλοίο έχει μια χωρητικότητα επιβατών, N . Ο κάθε όροφος έχει χωρητικότητα επιβατών $N/4$ (ίδια για όλους τους ορόφους). Ο επιβάτης αναπαριστάται από τον αριθμό της καμπίνας του. Ο αριθμός αυτός είναι τριψήφιος όπου το πρώτο ψηφίο αντιστοιχεί στον όροφο της καμπίνας. Οι πρώτοι 25

αριθμοί μιας εκατοντάδας αντιστοιχούν σε δίκλινες καμπίνες και οι υπόλοιποι 75 αντιστοιχούν σε τετράκλινες καμπίνες. Αν ο επιβάτης δεν έχει καμπίνα, ο αριθμός αυτός θα είναι το μηδέν και απλά θα βρίσκεται σε έναν όροφο.

Το πλοίο αρχικά δεν έχει κανένα επιβάτη — το ίδιο ισχύει και για τους ορόφους και τις καμπίνες του —. Όταν δημιουργείται ένα πλοίο, εκτυπώνεται το μήνυμα "Ready for embarkation" και ανατίθεται η χωρητικότητά του. Όταν δημιουργείται ένας όροφος, εκτυπώνεται το μήνυμα "A quiet floor!" και ανατίθεται η χωρητικότητά του. Όταν δημιουργείται μια καμπίνα, εκτυπώνεται το μήνυμα "What a clean cabin!". Όταν καταστρέφεται ένα πλοίο, εκτυπώνεται το μήνυμα "End of the trip!". Όταν καταστρέφεται ένας όροφος, εκτυπώνεται το μήνυμα "Everybody out!". Όταν καταστρέφεται μια καμπίνα, εκτυπώνεται το μήνυμα "What a mess!".

Στο πλοίο επιβιβάζεται ένας επιβάτης (enter), αν δεν παραβιάζονται τα όρια χωρητικότητας, τόσο του πλοίου, όσο και του ορόφου, αλλά και της καμπίνας που του αντιστοιχεί. Η επιβίβαση γίνεται εκτυπώνοντας το μήνυμα "Another one on board!", αυξάνοντας τον μετρητή των επιβιβασμένων επιβατών κατά ένα και επιβιβάζοντάς τον στον όροφο που ανήκει η καμπίνα του. Αν δεν έχει καμπίνα, δοκιμάζονται ένας-ένας οι όροφοι του πλοίου. Η επιβίβαση ενός επιβάτη σε όροφο (enter) γίνεται εκτυπώνοντας το μήνυμα "Entering floor number:", ακολουθούμενο από τον αριθμό του ορόφου, αυξάνοντας τον μετρητή των επιβιβασμένων επιβατών του ορόφου κατά ένα και επιβιβάζοντάς τον στην καμπίνα, αν έχει. Η επιβίβαση ενός επιβάτη σε καμπίνα (enter) γίνεται εκτυπώνοντας το μήνυμα "I am in my cabin!" και αυξάνοντας τον μετρητή των επιβιβασμένων επιβατών της καμπίνας κατά ένα. Αν παραβιάζεται το όριο του πλοίου, εκτυπώνεται το μήνυμα "Nobody in: sailing away!". Αν παραβιάζεται το όριο του ορόφου, εκτυπώνεται το μήνυμα "Sorry, no entry!". Αν παραβιάζεται το όριο της καμπίνας, εκτυπώνεται το μήνυμα "Wrong cabin probably!". Στις περιπτώσεις αυτές, η επιβίβαση δεν γίνεται.

Υλοποιήστε τα παραπάνω, μέσω των κατάλληλων κλάσεων, ορίζοντας τα μέλη-δεδομένα που χρειάζονται και τις συναρτήσεις-μέλη που υλοποιούν την παραπάνω συμπεριφορά. Υλοποιήστε, επίσης μια συνάρτηση main στην οποία:

1. Να δημιουργείται ένα πλοίο με χωρητικότητα επιβατών N .
2. Να γίνονται K επιβιβάσεις, με τυχαίες επιλογές καμπίνας.

Τα K και N να δίνονται από τη γραμμή εντολής.

11.3.4 Υλοποιήστε σε C++ την προσομοίωση ενός αεροπλάνου ταχυμεταφορών. Στο αεροπλάνο φορτώνονται το πολύ 6 containers και το πολύ 20 σάκοι φακέλων. Το αεροπλάνο έχει ένα βάρος φορτίου, το οποίο πρέπει να είναι κάτω ενός μέγιστου βάρους που μπορεί να μεταφέρει. Κάθε container περιέχει δέματα και έχει και ένα βάρος φορτίου, που προκύπτει από το συνολικό βάρος των δεμάτων που μεταφέρει και το απόβάρό του. Το κάθε δέμα έχει βάρος, αξία και μέγεθος (μικρό, μεγάλο, κανονικό). Ο κάθε σάκος φακέλων έχει ένα βάρος και μια συνολική αξία των φακέλων που περιέχει.

Το αεροπλάνο αρχικά είναι άδειο και εκχωρείται το μέγιστο βάρος που μπορεί να μεταφέρει. Αν δεν διευκρινίζεται διαφορετικά, η πολιτική της εταιρείας επιβάλλει ένα μέγιστο βάρος $MaxWeight$. Αρχικά το βάρος φορτίου είναι 0. Όταν δημιουργείται ένα αεροπλάνο, εκτυπώνεται το μήνυμα "Getting ready to depart", καθώς και το μέγιστο βάρος που μπορεί να μεταφέρει. Όταν καταστρέφεται το αεροπλάνο, εκτυπώνεται το μήνυμα "The work is over!".

Σε ένα container αρχικά ανατίθενται τα δέματα που περιέχει και το απόβάρό του. Όταν δημιουργείται ένα container, εκτυπώνεται το μήνυμα "Container to load...". Στο container γίνεται υπολογισμός του βάρους του περιεχομένου του (calculate_weight), από τα επιμέρους βάρη των δεμάτων που περιέχει, και το απόβάρό του. Στο container γίνεται υπολογισμός της αξίας του περιεχομένου

του (`calculate_value`), από τις επιμέρους αξίες των δεμάτων που περιέχει. Όταν καταστρέφεται το `container`, εκτυπώνεται το μήνυμα `"Emptying..."`.

Σε ένα δέμα, αρχικά ανατίθεται το βάρος του, η αξία του και το μέγεθός του. Όταν δημιουργείται ένα δέμα, εκτυπώνεται το μήνυμα `"Another parcel in!"`, καθώς και το βάρος του, η αξία του και το μέγεθός του. Όταν καταστρέφεται ένα δέμα, εκτυπώνεται το μήνυμα `"Another parcel out!"`.

Σε έναν σάκο φακέλων, αρχικά ανατίθεται το βάρος του και η συνολική αξία των φακέλων που περιέχει. Όταν δημιουργείται, εκτυπώνεται το μήνυμα `"Bag to load..."`, καθώς και το βάρος και η αξία του. Όταν καταστρέφεται ένας σάκος, εκτυπώνεται το μήνυμα `"Bag out..."`.

Στο αεροπλάνο γίνεται φόρτωση (`load`). Πρώτα φορτώνονται τα `containers` και μετά οι σάκοι. Πριν φορτωθεί το καθένα από αυτά, ελέγχεται αν με το βάρος του υπερβαίνεται το μέγιστο βάρος φορτίου. Αν η φόρτωση του `container` είναι επιτρεπτή, αυξάνεται το βάρος του φορτίου του αεροπλάνου κατά το βάρος του `container` και γίνεται φόρτωση του `container`. Κατά τη φόρτωση του `container` (`load`), εκτυπώνεται το μήνυμα `"Container in"`. Αν η φόρτωση δεν είναι επιτρεπτή, εκτυπώνεται το μήνυμα, `"Container rejected"`. Αν η φόρτωση του σάκου είναι επιτρεπτή, αυξάνεται το βάρος του φορτίου του αεροπλάνου κατά το βάρος του σάκου και γίνεται φόρτωση του σάκου. Κατά τη φόρτωση του σάκου (`load`), εκτυπώνεται το μήνυμα `"Bag in"`. Αν η φόρτωση του σάκου δεν είναι επιτρεπτή, εκτυπώνεται το μήνυμα, `"Bag rejected"`. Τελικά εκτυπώνεται το πλήθος των `containers` και των σάκων που φορτώθηκαν. Για το αεροπλάνο γίνεται και υπολογισμός της συνολικής αξίας του φορτίου που μεταφέρει (`calculate_value`). Για το αεροπλάνο γίνεται και υπολογισμός του βάρους του φορτίου του (`calculate_weight`).

Υλοποιήστε τα παραπάνω, μέσω των κατάλληλων κλάσεων, ορίζοντας τα μέλη-δεδομένα που χρειάζονται και τις συναρτήσεις-μέλη που υλοποιούν την παραπάνω συμπεριφορά. Υλοποιήστε, επίσης μια συνάρτηση `main` στην οποία:

1. Να δημιουργείται ένα αεροπλάνο με μέγιστο βάρος N .
2. Να γίνεται φόρτωση του αεροπλάνου.
3. Να γίνεται υπολογισμός της συνολικής αξίας του φορτίου καθώς και του βάρους του.
4. Να εκτυπώνεται το μήνυμα `"Dispatching to distribute!"`.

Το N να δίνεται από τη γραμμή εντολής. Αν δεν δίνεται, να ακολουθείται η πολιτική της εταιρείας για το βάρος. Τα βάρη και οι αξίες των δεμάτων και των σάκων να είναι τυχαία. Τυχαία επίσης να είναι τα μεγέθη των δεμάτων. Τέλος, τυχαία να είναι και τα πλήθη των σάκων και των δεμάτων που πρόκειται να διανεμηθούν.

11.3.5 Υλοποιήστε σε C++ μια προσομοίωση Έκθεσης του σχολικού μαθήματος Παραγωγής Λόγου.¹ Μια Έκθεση αποτελείται από τον τίτλο, τον πρόλογο, το κυρίως θέμα και τον επίλογο. Ο τίτλος είναι μια συμβολοσειρά. Ο πρόλογος είναι μια Παράγραφος. Το κυρίως θέμα είναι μια ακολουθία πέντε Παραγράφων. Ο επίλογος είναι μια Παράγραφος.

Μια Παράγραφος είναι μια ακολουθία δέκα Προτάσεων. Μια Πρόταση αποτελείται από μια ακολουθία το πολύ N λέξεων και έναν μετρητή που αντιστοιχεί στο πλήθος των λέξεων που περιέχει. Οι λέξεις προέρχονται από ένα λεξικό που περιέχει τις συμβολοσειρές `"All"`, `"Work"`, `"And"`, `"No"`, `"Play"`, `"Makes"`, `"Jack"`, `"A"`, `"Dull"`, `"Boy"`. Κάθε άλλη συμβολοσειρά στη θέση λέξης, αποτελεί ορθογραφικό λάθος.

Κατά τη δημιουργία μιας Έκθεσης, της ανατίθεται ο τίτλος της και εκτυπώνεται στην οθόνη το μήνυμα `"I just composed a composition with title: "` συνοδευόμενο από τον τίτλο. Όταν καταστρέφεται, εκτυπώνεται το μήνυμα `"Another composition is over!"`.

¹Οι παραπάνω προδιαγραφές είναι αυθαίρετες και δεν στηρίζονται σε κάποια επιστημονική αντιμετώπιση.

Μια Παράγραφος όταν δημιουργείται εκτυπώνεται στην οθόνη το μήνυμα "I just composed a paragraph with content...". Όταν καταστρέφεται, εκτυπώνεται το μήνυμα "I can forget this paragraph!".

Όταν δημιουργείται μια Πρόταση, αρχικοποιείται από τυχαίο αριθμό λέξεων (μικρότερο του N) τυχαία παρμένες από το παραπάνω λεξικό ή κάποια άλλη (τυχαία) συμβολοσειρά — οπότε η λέξη αυτή θεωρείται ορθογραφικό λάθος —. Ο αριθμός αυτός καταχωρείται στον μετρητή των λέξεων της Πρότασης. Επίσης, εκτυπώνεται στην οθόνη το μήνυμα "I just composed a phrase with content: " και το περιεχόμενό της. Όταν καταστρέφεται μια Πρόταση, εκτυπώνεται το μήνυμα "Forget phrase: " και το περιεχόμενό της.

Επίσης, τα παραπάνω συνοδεύει και η εξής συμπεριφορά:

Γίνεται διόρθωση Έκθεσης (`spell_correct`), διορθώνοντας τις Παραγράφους της. Επίσης, γίνεται έλεγχος της έκτασής της (`evaluate_length`), κάνοντας μέτρηση του πλήθους των λέξεων του προλόγου, κυρίως θέματος και epilόγου της και ελέγχοντας αν αυτό είναι μέσα σε ένα εύρος *MinWords*, *MaxWords*. Αν είναι μικρότερο, να εκτυπώνεται στην οθόνη το μήνυμα "Length too short ...". Αν είναι μεγαλύτερο, να εκτυπώνεται στην οθόνη το μήνυμα "Length too long ...". Διαφορετικά, να εκτυπώνεται στην οθόνη το μήνυμα "Length OK!!!!".

Γίνεται διόρθωση μιας Παραγράφου (`spell_correct`), διορθώνοντας τις Προτάσεις της. Γίνεται μέτρηση του πλήθους των λέξεων μιας Παραγράφου (`word_count`), κάνοντας μέτρηση του πλήθους των λέξεων των Προτάσεων της.

Γίνεται διόρθωση μιας Πρότασης (`spell_correct`), ελέγχοντας τις λέξεις της. Στην περίπτωση που μια λέξη δεν ανήκει στο παραπάνω λεξικό, αυτή αντικαθίσταται από τη συμβολοσειρά "SPELLING_MISTAKE". Γίνεται μέτρηση του πλήθους των λέξεων μιας Πρότασης (`word_count`), επιστρέφοντας την τιμή του μετρητή που αντιστοιχεί στο πλήθος λέξεων που περιέχει.

Να υλοποιηθούν οι κλάσεις που αντιστοιχούν στην Έκθεση, την Παράγραφο και την Πρόταση και να υλοποιηθεί μια συνάρτηση `main`, που να αναδεικνύει τη συμπεριφορά τους ως εξής:

1. Να δημιουργεί μια έκθεση με τίτλο "The Shining",
2. να κάνει έλεγχο της έκτασής της και
3. να κάνει διόρθωσή της.

Τα N , *MinWords* και *MaxWords* είναι παράμετροι.

11.3.6 Υλοποιήστε σε C++ μια σκηνή κινουμένων σχεδίων (Scene). Η σκηνή αποτελείται από τα εξής:

- Το υπόβαθρο (Background).
- Μια λίμνη (Lake).
- Μια παγοδρόμο (IceSkater).

Όταν δημιουργείται μια σκηνή, εκτυπώνεται το μήνυμα "Scene was just created!". Όταν καταστρέφεται μια σκηνή, εκτυπώνεται το μήνυμα "Scene is about to be destroyed". Για τη σκηνή, ορίζεται η συνάρτηση-μέλος που υλοποιεί την εξέλιξή της (`plot`). Αυτή έχει ως εξής:

Η λίμνη φωτίζεται.
 Η παγοδρόμος ξεκινά την επίδειξή της.
 Η παγοδρόμος γλιστρά έως το κέντρο της λίμνης.
 Η παγοδρόμος περιστρέφεται.
 Η παγοδρόμος πηδά.

*Η παγοδρόμος γλιστρά έως την άλλη άκρη της λίμνης.
Η λίμνη περιστρέφεται.
Η λίμνη σκοτεινιάζει.*

Οι οντότητες που αποτελούν τη σκηνή περιγράφονται ως εξής:

Το υπόβαθρο αποτελείται από τα εξής:

- Μια οροσειρά (Mountains).
- Μια συστοιχία από δέντρα (Trees).
- Ένα σπιτάκι (Little_house).

Αρχικοποιείται, εκτυπώνοντας το μήνυμα "Background was just created!". Καταστρέφεται, εκτυπώνοντας το μήνυμα "Background is about to be destroyed".

Η οροσειρά δεν έχει δεδομένα. Αρχικοποιείται, εκτυπώνοντας το μήνυμα "Mountains were just created!". Καταστρέφεται, εκτυπώνοντας το μήνυμα "Mountains are about to be destroyed".

Η συστοιχία από δέντρα δεν έχει δεδομένα. Αρχικοποιείται, εκτυπώνοντας το μήνυμα "Trees were just created!". Καταστρέφεται, εκτυπώνοντας το μήνυμα "Trees are about to be destroyed".

Το σπιτάκι δεν έχει δεδομένα. Αρχικοποιείται, εκτυπώνοντας το μήνυμα "A little house was just created!". Καταστρέφεται, εκτυπώνοντας το μήνυμα "A little house is about to be destroyed".

Σημείωση: Στα παραπάνω, δεν θεωρούμε ότι κάποια πληροφορία χρειάζεται απόκρυψη.

Η παγοδρόμος έχει την πληροφορία σχετικά με την απόσταση που διανύει σε μια ολίσθηση (distance), καθώς και την απόσταση που βρίσκεται από την αριστερή άκρη της λίμνης (position) (*Σημείωση:* για ευκολία, θεωρήστε ότι η λίμνη έχει σχήμα ορθογώνιο και ότι η παγοδρόμος κινείται κατά μήκος της βάσης του. Οπότε, ως "άκρη" εννοείται η αριστερή κορυφή, "άλλη άκρη" η δεξιά κορυφή και "κέντρο της λίμνης" το μέσο της βάσης του ορθογωνίου αυτού).

Η συμπεριφορά της παγοδρόμου είναι η εξής:

Κατά την αρχικοποίηση, βρίσκεται στην άκρη της λίμνης και ανατίθεται η τιμή της απόστασης που διανύει σε μια ολίσθηση. Επίσης, εκτυπώνεται το μήνυμα "An IceSkater participates". Κατά την καταστροφή, εκτυπώνεται το μήνυμα "An IceSkater will be discarded!". Η παγοδρόμος ξεκινά την επίδειξή της (start), εκτυπώνοντας το μήνυμα "Here I am ...". Η παγοδρόμος γλιστρά κατά μήκος μιας απόστασης (skate), μεταβάλλοντας την απόσταση από την άκρη της λίμνης κατά την απόσταση αυτή και εκτυπώνοντας το μήνυμα "An IceSkater skates", καθώς και το πλήθος των ολισθήσεων που έκανε. Η παγοδρόμος περιστρέφεται (rotate), εκτυπώνοντας το μήνυμα "An IceSkater spins on position" και την απόσταση από την άκρη της λίμνης. Η παγοδρόμος πηδά (jump), εκτυπώνοντας το μήνυμα "An IceSkater jumps on position" και την απόσταση από την άκρη της λίμνης.

Η λίμνη έχει την πληροφορία σχετικά με το μήκος της (το μήκος της βάσης της), καθώς και την απόχρωση του χρώματός της, dark ή light.

Η συμπεριφορά της λίμνης είναι η εξής:

Κατά την αρχικοποίηση, ανατίθεται το μήκος της και η απόχρωση του χρώματός της είναι dark. Επίσης, εκτυπώνεται το μήνυμα "A lake appears". Κατά την καταστροφή, εκτυπώνεται το μήνυμα "A lake will disappear!". Η λίμνη φωτίζεται (light_up), θέτοντας την τιμή light στην απόχρωση και εκτυπώνοντας το μήνυμα "Lights on!". Η λίμνη σκοτεινιάζει (darken), θέτοντας την

τιμή `dark` στην απόχρωση και εκτυπώνοντας το μήνυμα "Lights off!". Η λίμνη περιστρέφεται (rotate), εκτυπώνοντας το μήνυμα "Lake rotated!".

Υλοποιήστε τα παραπάνω, μέσω των κατάλληλων κλάσεων, ορίζοντας τα μέλη-δεδομένα που χρειάζονται και τις συναρτήσεις-μέλη που υλοποιούν την παραπάνω συμπεριφορά.

Για να δοκιμάσετε την υλοποίησή σας, ορίστε μια συνάρτηση `main` που να περιέχει τον παρακάτω κώδικα

```
Scene scene(L,Dist);
scene.plot();
```

Το μήκος της (βάσης της) λίμνης, `L`, και η απόσταση που διανύει σε μια ολίσθηση μια παγοδρόμος, `Dist`, είναι παράμετροι και να δίνονται κατά την κλήση του προγράμματος από τη γραμμή εντολής, με αυτήν τη σειρά.

- 11.3.7** Υλοποιήστε σε C++ μια απλοϊκή προσομοίωση του ατόμου των ισοτόπων του άνθρακα (C), άνθρακα-12, άνθρακα-13 και άνθρακα-14 (^{12}C , ^{13}C , ^{14}C). Στην απλουστευμένη αυτή εκδοχή, ισχύουν τα παρακάτω.

Τα άτομα των στοιχείων, γενικά, είναι οργανωμένα ως εξής. Ένα άτομο (*atom*) αποτελείται από τον πυρήνα του (*nucleus*) και έναν αριθμό ηλεκτρονίων (*electrons*). Ο πυρήνας του ατόμου ενός στοιχείου αποτελείται από έναν αριθμό πρωτονίων (*protons*) και έναν αριθμό νετρονίων (*neutrons*). Στην περίπτωση ενός μη ιονισμένου (μη φορτισμένου) ατόμου, ο αριθμός των ηλεκτρονίων του είναι ίσος με τον αριθμό των πρωτονίων. Ο αριθμός των νετρονίων μπορεί να διαφέρει και με τον τρόπο αυτό καταλήγουμε να έχουμε για το ίδιο στοιχείο διαφορετικά ισότοπα.

Για τα ισότοπα του άνθρακα έχουμε, για τον άνθρακα-12, 6 πρωτόνια και 6 νετρόνια, για τον άνθρακα-13, 6 πρωτόνια και 7 νετρόνια και για τον άνθρακα-14, 6 πρωτόνια και 8 νετρόνια.

Ένα άτομο έχει μαζικό αριθμό (*mass number*) που υπολογίζεται ως ο συνολικός αριθμός πρωτονίων και νετρονίων του πυρήνα του. Ο συνολικός αριθμός των πρωτονίων του αποτελεί τον ατομικό αριθμό του (*atomic number*). Επίσης, ένα άτομο έχει ηλεκτρικό φορτίο (*electric charge*) που προκύπτει από τη διαφορά του συνολικού ηλεκτρικού φορτίου των ηλεκτρονίων του από το συνολικό ηλεκτρικό φορτίο των πρωτονίων του. Όταν δημιουργείται ένα άτομο, του ανατίθεται ο αριθμός των πρωτονίων και των νετρονίων. Επίσης, εκτυπώνεται το μήνυμα "An atom has been constructed" ακολουθούμενο από τους αριθμούς αυτούς. Όταν καταστρέφεται ένα άτομο, εκτυπώνεται το μήνυμα "An atom will be destroyed" ακολουθούμενο από τον ατομικό και τον μαζικό αριθμό του. Για ένα άτομο μπορούμε να υπολογίσουμε τον ατομικό αριθμό του (`get_atomic_number`), τον μαζικό αριθμό του (`get_mass_number`), καθώς και το συνολικό ηλεκτρικό φορτίο του (`get_electric_charge`). Επίσης, μπορούμε να εκτυπώσουμε (`print`) όλη την πληροφορία που το αφορά, δηλαδή, τον ατομικό και τον μαζικό αριθμό του, το φορτίο του και ό,τι μπορεί να εκτυπωθεί από τα συστατικά του.

Ένα πρωτόνιο αποτελείται από τρία quarks, δύο *up quarks* και ένα *down quark*, και τρία γκλουόνια (*gluons*). Ένα πρωτόνιο έχει μάζα (*mass*) ίση με (περίπου) 1.007 u, ιδιοστροφορμή (*spin*) ίση με $\frac{1}{2}$, ηλεκτρικό φορτίο (*electric charge*) ίσο με $+1e$ και υπακούει στη στατιστική (*statistics*) Fermi-Dirac. Όταν δημιουργείται ένα πρωτόνιο, του ανατίθεται η τιμή της μάζας του, η ιδιοστροφορμή του, το ηλεκτρικό φορτίο του και το είδος της στατιστικής στο οποίο υπακούει. Επίσης, εκτυπώνεται το μήνυμα "A proton has been constructed". Όταν καταστρέφεται ένα πρωτόνιο, εκτυπώνεται το μήνυμα "A proton will be destroyed". Για ένα πρωτόνιο, μπορούμε να εκτυπώσουμε (`print`) όλη την πληροφορία που το αφορά, δηλαδή, την τιμή της μάζας του, την ιδιοστροφορμή του, το ηλεκτρικό φορτίο του και το είδος της στατιστικής στο οποίο υπακούει, καθώς και ό,τι μπορεί να εκτυπωθεί από τα συστατικά του.

Ένα νετρόνιο αποτελείται από τρία quarks, δύο *down quarks* και ένα *up quark*, και τρία γκλουόνια (*gluons*). Ένα νετρόνιο έχει μάζα (*mass*) ίση με (περίπου) 1.0085 u, ιδιοστροφορμή (*spin*) ίση με $\frac{1}{2}$, ηλεκτρικό φορτίο (*electric charge*) ίσο με 0e και υπακούει στη στατιστική (*statistics*) Fermi-Dirac. Όταν δημιουργείται ένα νετρόνιο, του ανατίθεται η τιμή της μάζας του, η ιδιοστροφορμή του, το ηλεκτρικό φορτίο του και το είδος της στατιστικής στο οποίο υπακούει. Επίσης, εκτυπώνεται το μήνυμα "A neutron has been constructed". Όταν καταστρέφεται ένα νετρόνιο, εκτυπώνεται το μήνυμα "A neutron will be destroyed". Για ένα νετρόνιο, μπορούμε να εκτυπώσουμε (print) όλη την πληροφορία που το αφορά, δηλαδή, την τιμή της μάζας του, την ιδιοστροφορμή του, το ηλεκτρικό φορτίο του και το είδος της στατιστικής στο οποίο υπακούει, καθώς και ό,τι μπορεί να εκτυπωθεί από τα συστατικά του.

Ένα up quark έχει ηλεκτρικό φορτίο (*electric charge*) ίσο με $+\frac{2}{3}e$, μάζα (*mass*) ίση με (περίπου) 2.01 MeV/c², ιδιοστροφορμή (*spin*) ίση με $\frac{1}{2}$ και υπακούει στη στατιστική (*statistics*) Fermi-Dirac. Όταν δημιουργείται ένα up quark, του ανατίθεται η τιμή της μάζας του, η ιδιοστροφορμή του, το ηλεκτρικό φορτίο του και το είδος της στατιστικής στο οποίο υπακούει. Επίσης, εκτυπώνεται το μήνυμα "An up quark has been constructed". Όταν καταστρέφεται ένα up quark, εκτυπώνεται το μήνυμα "An up quark will be destroyed". Για ένα up quark, μπορούμε να εκτυπώσουμε (print) όλη την πληροφορία που το αφορά, δηλαδή, την τιμή της μάζας του, την ιδιοστροφορμή του, το ηλεκτρικό φορτίο του και το είδος της στατιστικής στο οποίο υπακούει.

Ένα down quark έχει ηλεκτρικό φορτίο (*electric charge*) ίσο με $-\frac{1}{3}e$, μάζα (*mass*) ίση με (περίπου) 4.7 MeV/c², ιδιοστροφορμή (*spin*) ίση με $\frac{1}{2}$ και υπακούει στη στατιστική (*statistics*) Fermi-Dirac. Όταν δημιουργείται ένα down quark, του ανατίθεται η τιμή της μάζας του, η ιδιοστροφορμή του, το ηλεκτρικό φορτίο του και το είδος της στατιστικής στο οποίο υπακούει. Επίσης, εκτυπώνεται το μήνυμα "A down quark has been constructed". Όταν καταστρέφεται ένα down quark, εκτυπώνεται το μήνυμα "A down quark will be destroyed". Για ένα down quark, μπορούμε να εκτυπώσουμε (print) όλη την πληροφορία που το αφορά, δηλαδή, την τιμή της μάζας του, την ιδιοστροφορμή του, το ηλεκτρικό φορτίο του και το είδος της στατιστικής στο οποίο υπακούει.

Ένα γκλουόνιο έχει μάζα (*mass*) ίση με 0, ηλεκτρικό φορτίο (*electric charge*) ίσο με 0e, ιδιοστροφορμή (*spin*) ίση με 1 και υπακούει στη στατιστική (*statistics*) Bose-Einstein. Όταν δημιουργείται ένα γκλουόνιο, του ανατίθεται η τιμή της μάζας του, η ιδιοστροφορμή του, το ηλεκτρικό φορτίο του και το είδος της στατιστικής στο οποίο υπακούει. Επίσης, εκτυπώνεται το μήνυμα "A gluon has been constructed". Όταν καταστρέφεται ένα γκλουόνιο, εκτυπώνεται το μήνυμα "A gluon will be destroyed". Για ένα γκλουόνιο, μπορούμε να εκτυπώσουμε (print) όλη την πληροφορία που το αφορά, δηλαδή, την τιμή της μάζας του, την ιδιοστροφορμή του, το ηλεκτρικό φορτίο του και το είδος της στατιστικής στο οποίο υπακούει.

Ένα ηλεκτρόνιο έχει ηλεκτρικό φορτίο (*electric charge*) ίσο με $-1e$, μάζα (*mass*) ίση με $5.48579909070 \times 10^{-4}$ u, ιδιοστροφορμή (*spin*) ίση με $\frac{1}{2}$ και υπακούει στη στατιστική (*statistics*) Fermi-Dirac. Όταν δημιουργείται ένα ηλεκτρόνιο, του ανατίθεται η τιμή της μάζας του, η ιδιοστροφορμή του, το ηλεκτρικό φορτίο του και το είδος της στατιστικής στο οποίο υπακούει. Επίσης, εκτυπώνεται το μήνυμα "An electron has been constructed". Όταν καταστρέφεται ένα ηλεκτρόνιο, εκτυπώνεται το μήνυμα "An electron will be destroyed". Για ένα ηλεκτρόνιο, μπορούμε να εκτυπώσουμε (print) όλη την πληροφορία που το αφορά, δηλαδή, την τιμή της μάζας του, την ιδιοστροφορμή του, το ηλεκτρικό φορτίο του και το είδος της στατιστικής στο οποίο υπακούει.

Υλοποιήστε τα παραπάνω, μέσω των κατάλληλων κλάσεων, ορίζοντας τα μέλη-δεδομένα που χρειάζονται και τις συναρτήσεις-μέλη που υλοποιούν την παραπάνω συμπεριφορά. Υλοποιήστε επίσης μια συνάρτηση main η οποία:

1. Να δημιουργεί έναν τυχαίο αριθμό ατόμων ισοτόπων άνθρακα, επιλέγοντας τυχαία το είδος τους.

2. Για καθένα από αυτά, να εκτυπώνει τις πληροφορίες του.
3. Να υπολογίζει το σύνολο των πρωτονίων που έχουν δημιουργηθεί.
4. Να υπολογίζει το σύνολο των νετρονίων που έχουν δημιουργηθεί.
5. Να υπολογίζει το συνολικό φορτίο που έχει δημιουργηθεί.

11.3.8 Υλοποιήστε σε C++ την εικόνα ενός discussion forum, π.χ. “Αντικειμενοστραφής Προγραμματισμός”, όπως αυτό περιγράφεται στη συνέχεια. Για λόγους ευκολίας, θεωρήστε ότι δεν υπάρχουν subforums.

Ένα forum αποτελείται από τον “τίτλο” του και ένα πλήθος από threads.

Κάθε thread αποτελείται από ένα “θέμα”, τον “δημιουργό” του, την “ημερομηνία δημιουργίας” του και ένα πλήθος από posts.

Κάθε post αποτελείται από ένα “αριθμητικό αναγνωριστικό” (id, μοναδικό για κάθε post), τον “τίτλο” του, τον “δημιουργό” του, την “ημερομηνία δημιουργίας” του και ένα κείμενο.

Κατά τη δημιουργία ενός post, εκχωρούνται και εκτυπώνονται στην οθόνη οι τιμές των στοιχείων του. Κατά την καταστροφή του, εκτυπώνεται στην οθόνη το μήνυμα, "Post number: $\langle Post_number \rangle$ is about to be destroyed", όπου $\langle Post_number \rangle$ είναι το αριθμητικό αναγνωριστικό του post.

Κατά τη δημιουργία ενός thread, εκχωρείται το θέμα του, ο δημιουργός του και η ημερομηνία δημιουργίας του. Επίσης, εκτυπώνεται στην οθόνη το μήνυμα "Thread with subject: $\langle Subject \rangle$ has just been created!", όπου $\langle Subject \rangle$ είναι το θέμα του thread. Κατά την καταστροφή του, εκτυπώνεται στην οθόνη το μήνυμα, "Thread with subject: $\langle Subject \rangle$ is about to be destroyed", όπου $\langle Subject \rangle$ είναι το θέμα του thread.

Κατά τη δημιουργία ενός forum, εκχωρείται ο τίτλος του και εκτυπώνεται το μήνυμα "Forum with title: $\langle Title \rangle$ has just been created!", όπου $\langle Title \rangle$ είναι ο τίτλος του forum. Κατά την καταστροφή του, εκτυπώνεται στην οθόνη το μήνυμα, "Forum with title: $\langle Title \rangle$ is about to be destroyed", όπου $\langle Title \rangle$ είναι ο τίτλος του forum.

Όσον αφορά τη συμπεριφορά των παραπάνω, ισχύουν τα εξής. Για το forum, να δίνεται η δυνατότητα να εκτυπώνεται, εκτυπώνοντας τους τίτλους των threads του. Επίσης, να δίνεται η δυνατότητα να επιλέγεται ένα thread, προσδιορίζοντας τον τίτλο και τότε να εκτυπώνεται το thread. Τέλος, για το forum, να δίνεται η δυνατότητα να επιλέγεται ένα post, δίνοντας τον αριθμό του και τότε το post αυτό να εκτυπώνεται.

Ένα thread εκτυπώνεται, εκτυπώνοντας το θέμα του, τον δημιουργό του, την ημερομηνία δημιουργίας του και τα posts του.

Ένα post εκτυπώνεται, εκτυπώνοντας τον αριθμό του, τον τίτλο του, τον δημιουργό του, την ημερομηνία δημιουργίας του και το κείμενό του.

Υλοποιήστε τα παραπάνω, μέσω των κατάλληλων κλάσεων, ορίζοντας τα μέλη-δεδομένα που χρειάζονται και τις συναρτήσεις-μέλη που υλοποιούν την παραπάνω συμπεριφορά. Υλοποιήστε επίσης μια συνάρτηση main στην οποία:

1. Να δημιουργείται ένα forum με τίτλο “Αντικειμενοστραφής Προγραμματισμός”.
2. Να εκτυπώνεται το forum.
3. Να εκτυπώνεται το thread με θέμα “Κανόνες και χρήσιμες πληροφορίες”.
4. Να επιλέγονται (τυχαία) κάποια posts και να εκτυπώνονται.

Σημείωση 1: Τα κείμενα για τα posts να τα δημιουργείτε με έναν απλό τρόπο, π.χ. να έχετε έναν πίνακα από εκφράσεις και με τυχαίο τρόπο να επιλέγετε κάποια από αυτές.

Σημείωση 2: Για την αναπαράσταση της ημερομηνίας, θεωρήστε μια δομή.

Σημείωση 3: Όπου χρειάζεστε συμβολοσειρές, να χρησιμοποιείτε τον τύπο `string` από τη Standard βιβλιοθήκη της C++ (οι απαραίτητες δηλώσεις βρίσκονται στο αρχείο επικεφαλίδας `string`).

Σημείωση 4: Οι “δημιουργοί” να αναπαριστούνται με μια συμβολοσειρά που αντιστοιχεί στο όνομά τους στο `forum`.

11.3.9 Υλοποιήστε σε C++ μια προσομοίωση της λειτουργικότητας μιας βιβλιοθήκης. Η βιβλιοθήκη αποτελείται από μια βάση, ένα ντουλάπι και τρία ράφια (με αυτήν τη σειρά). Το ντουλάπι εσωτερικά έχει δύο ράφια.

Στη βιβλιοθήκη, ανά πάσα στιγμή βρίσκονται N βιβλία (το N μπορεί να είναι ίσο και με 0). Ένα βιβλίο δεν είναι τίποτα περισσότερο από μια δόμηση δεδομένων που αποτελείται από τον τίτλο του βιβλίου, το ονοματεπώνυμο του συγγραφέα του και το ISBN του (ακέραιος). Το κάθε ράφι (είτε εξωτερικό είτε εσωτερικό) φιλοξενεί κάποια από τα βιβλία. Κάθε ράφι έχει μέγιστη χωρητικότητα $NMax$. Η χωρητικότητα είναι ίδια για όλα τα ράφια.

Κατά την κατασκευή μιας βιβλιοθήκης, εκτυπώνεται ένα μήνυμα ότι μια βιβλιοθήκη κατασκευάζεται. Κατά την κατασκευή ενός ραφιού, εκτυπώνεται ένα μήνυμα ότι ένα ράφι κατασκευάζεται. Κατά την κατασκευή ενός ντουλαπιού, εκτυπώνεται ένα μήνυμα ότι ένα ντουλάπι κατασκευάζεται. Κατά την κατασκευή μιας βάσης, εκτυπώνεται ένα μήνυμα ότι μια βάση κατασκευάζεται. Κατά την κατασκευή τους, τα ράφια δεν περιέχουν κανένα βιβλίο.

Κατά την καταστροφή της βιβλιοθήκης, εκτυπώνεται σχετικό μήνυμα. Σχετικά μηνύματα εκτυπώνονται και στις περιπτώσεις καταστροφής ραφιών, ντουλαπιών και βάσης βιβλιοθήκης.

Στη βιβλιοθήκη τοποθετούμε ένα βιβλίο (`place_book`), προσδιορίζοντας με έναν αριθμό τον χώρο στον οποίο θα τοποθετηθεί και τοποθετώντας το σε αυτόν τον χώρο. Δηλαδή, αντιστοιχώντας

- 1: στο πάνω ράφι της βιβλιοθήκης.
- 2: στο μεσαίο ράφι της βιβλιοθήκης.
- 3: στο κάτω ράφι της βιβλιοθήκης.
- 4: στο πάνω ράφι του ντουλαπιού της βιβλιοθήκης.
- 5: στο κάτω ράφι του ντουλαπιού της βιβλιοθήκης.

Σε αυτήν την περίπτωση, αυξάνεται κατά 1 ο αριθμός των βιβλίων που είναι τοποθετημένα στη βιβλιοθήκη. Ταυτόχρονα γίνεται εκτύπωση του χώρου στον οποίο γίνεται η τοποθέτηση, π.χ. αν ο προσδιοριστής είναι ο αριθμός 2, η εκτύπωση είναι `"placing book in middle bookcase self"` ή κάτι ανάλογο.

Μπορούμε επίσης να πάρουμε ένα βιβλίο από τη βιβλιοθήκη (`take_book`), προσδιορίζοντας τον χώρο από τον οποίο το αφαιρούμε και αφαιρώντας το από τον χώρο αυτό. Ταυτόχρονα, γίνεται εκτύπωση του χώρου από τον οποίο γίνεται η αφαίρεση. Ο προσδιορισμός του χώρου γίνεται με τρόπο ανάλογο με την τοποθέτηση (Σημείωση: το βιβλίο που αφαιρείται είναι τυχαίο, όχι προσδιορισμένο). Σε αυτήν την περίπτωση, ο αριθμός των βιβλίων που βρίσκονται στη βιβλιοθήκη μειώνεται κατά 1.

Τέλος, γίνεται εκτύπωση (`print`) στη βιβλιοθήκη, κάνοντας εκτύπωση στο ντουλάπι και στα ράφια της.

Η τοποθέτηση ενός βιβλίου στο ντουλάπι (`place_book`) πραγματοποιείται προσδιορίζοντας το ράφι στο οποίο θα γίνει η τοποθέτηση και τοποθετώντας το στο ράφι αυτό. Για τον προσδιορισμό του ραφιού, με 1 προσδιορίζουμε το πάνω ράφι και με 2 το κάτω ράφι του ντουλαπιού της βιβλιοθήκης. Η αφαίρεση ενός βιβλίου από το ντουλάπι (`take_book`), πραγματοποιείται προσδιορίζοντας το ράφι από οποίο θα

γίνει η αφαίρεση και αφαιρώντας το από το ράφι αυτό. Η εκτύπωση του ντουλαπιού (`print`) ανάγεται στις εκτυπώσεις των ραφιών του.

Η τοποθέτηση ενός βιβλίου σε ένα ράφι (`place_book`) πραγματοποιείται αυξάνοντας τον αριθμό των βιβλίων που είναι τοποθετημένα σε αυτό κατά 1, προσθέτοντας το βιβλίο αυτό στα βιβλία που φιλοξενεί και εκτυπώνοντας τα περιεχόμενα του βιβλίου. Αντίστοιχα, η αφαίρεση ενός βιβλίου από ένα ράφι (`take_book`) πραγματοποιείται μειώνοντας τον αριθμό των βιβλίων που είναι τοποθετημένα σε αυτό κατά 1 και αφαιρώντας το βιβλίο από τα βιβλία που φιλοξενεί. Τέλος, γίνεται εκτύπωση (`print`) ενός ραφιού, εκτυπώνοντας τα στοιχεία των βιβλίων που περιέχει. Αν είναι άδειο, να εκτυπώνεται σχετικό μήνυμα.

Στις περιπτώσεις αφαιρέσεων βιβλίων από κενούς χώρους ή τοποθετήσεων σε γεμάτους, να επιστρέφεται από τις αντίστοιχες συναρτήσεις τιμή `false` και να εκτυπώνεται μήνυμα λάθους. Διαφορετικά, να επιστρέφεται η τιμή `true`. Το ίδιο ισχύει και για αφαιρέσεις και τοποθετήσεις βιβλίων στη βιβλιοθήκη.

Η βάση της βιβλιοθήκης έχει μόνο δομικό ρόλο για τη βιβλιοθήκη — απλά συμμετέχει στις διαδικασίες αρχικοποίησης και καταστροφής της βιβλιοθήκης.

Υλοποιήστε τα παραπάνω μέσω των κατάλληλων κλάσεων, ορίζοντας τα μέλη-δεδομένα που χρειάζονται και τις συναρτήσεις-μέλη που υλοποιούν την παραπάνω συμπεριφορά.

Για να αναδείξετε την προσομοίωσή σας, να υλοποιήσετε μια συνάρτηση `main` η οποία:

1. Να δημιουργεί μια βιβλιοθήκη.
2. Να δημιουργεί L βιβλία.
3. Να πραγματοποιεί $K1$ τυχαίες τοποθετήσεις βιβλίων στη βιβλιοθήκη.
4. Να πραγματοποιεί $K2$ τυχαίες αφαιρέσεις βιβλίων από τη βιβλιοθήκη.
5. Να κάνει εκτύπωση στη βιβλιοθήκη.

Σημείωση: Τα $NMax$, L , $K1$ και $K2$ είναι παράμετροι της προσομοίωσης και να διαβάζονται από τη γραμμή εντολής με τη σειρά αυτή.

11.4 Μικρές Εργασίες στην Αφαίρεση στα Δεδομένα: Δομές Δεδομένων

Οι παρακάτω ασκήσεις έχουν στόχο να σας εξασκήσουν στην υλοποίηση δομών δεδομένων χρησιμοποιώντας κλάσεις. Οι κλάσεις αυτές οφείλουν να παρέχουν τη λειτουργικότητα της συμπεριφοράς των δομών, όπως αυτή περιγράφεται στα προβλήματα. Στα πλαίσια του ορισμού των κλάσεων, πρέπει να αποφασιστεί και να υλοποιηθεί η αναπαράσταση των δεδομένων με κατάλληλο τρόπο. Προκειμένου να επιτευχθεί ο στόχος των προβλημάτων αυτών, προτείνεται να μην χρησιμοποιηθούν τύποι βιβλιοθήκης (εκτός από την περίπτωση συμβολοσειρών). Αν είναι, όμως, επιθυμητό να γίνει εξάσκηση και στη χρήση των τύπων βιβλιοθήκης, τότε να υλοποιηθούν και οι δύο εκδοχές τους.

Οι δομές δεδομένων των προβλημάτων μπορεί να αντιστοιχούν σε τυπικούς Αφηρημένους Τύπους Δεδομένων που συναντάμε στη βιβλιογραφία αλλά, συνήθως, αποτελούν τύπους που είναι εξειδικευμένοι για το πρόβλημα. Σε κάθε περίπτωση, οι αρχές που πρέπει να ακολουθηθούν για τις απαιτήσεις που υπάρχουν είναι οι ίδιες.

Εκφωνήσεις Εργασιών

11.4.1 Όταν χορεύουμε παραδοσιακούς χορούς, συνήθως, σχηματίζουμε μία αλυσίδα. Μια τέτοια αλυσίδα μπορεί να θεωρηθεί ως μια δομή λίστας χορευτών. Οι χορευτές μπορεί να είναι άνδρες ή γυναίκες και ο καθένας από αυτούς ξέρει ένα σύνολο χορών. Αν υποθέσουμε ότι:

1. Αρχικά, η αλυσίδα μας είναι είτε κενή είτε περιέχει ένα ζευγάρι χορευτών.

2. Κάποιος που θέλει να μπει στον χορό πηγαίνει στο τέλος της αλυσίδας του χορού.
3. Κάποιος που είναι στο τέλος της αλυσίδας του χορού μπορεί να πάει πρώτος.
4. Η αλυσίδα μας σχηματίζεται εναλλάσσοντας υποχρεωτικά άνδρες και γυναίκες (δηλαδή, απαγορεύεται να έχουμε διαδοχικούς χορευτές του ίδιου φύλου).
5. Όλοι όσοι χορεύουν κάποια στιγμή ξέρουν τον χορό που παίζει το συγκρότημα.

Να γράψετε μια κλάση C++ που να υλοποιεί την παραπάνω συμπεριφορά. Δηλαδή:

1. Την αρχικοποίηση της αλυσίδας των χορευτών.
2. Την εισαγωγή νέου χορευτή στην αλυσίδα.
3. Την αλλαγή της “κεφαλής” του χορού.
4. Την αλλαγή που επιφέρει στην αλυσίδα του χορού κάθε νέος σκοπός που παίζει το συγκρότημα, λαμβάνοντας υπόψη ότι πάντα:
 - (α) αυτοί που χορεύουν ξέρουν τον χορό του σκοπού και
 - (β) απαγορεύεται να βρίσκονται διαδοχικά χορευτές του ίδιου φύλου.

Η έννοια του “χορευτή” να υλοποιηθεί και αυτή ως κλάση που δομείται από το φύλο του χορευτή και τους χορούς που ο χορευτής ξέρει να χορεύει. Θεωρήστε ότι ο χορός τον οποίο παίζει το συγκρότημα φυλάσσεται σε μια καθολική μεταβλητή `tune`.

Υπόδειξη: Χρησιμοποιήστε μια δομή διπλά συνδεδεμένης λίστας, η οποία επιπλέον να παρέχει και τη δυνατότητα να συνδέεται κάθε χορευτής και με τον επόμενο ομόφυλό του.

11.4.2 Τα αυτόματα στοίβας χρησιμοποιούνται για να αναγνωρίσουν συμβολοσειρές από πολύπλοκες γλώσσες και χρησιμοποιούν μία δομή στοίβας για να επιτρέψουν μια “συσχέτιση του παρελθόντος με το μέλλον” (να συνδυάσουμε το τι έχουμε ήδη διαβάσει προσφάτως με το τι πρόκειται να διαβάσουμε στο εγγύς μέλλον).

Τυπικά, ένα αυτόματο στοίβας είναι μία εξάδα

$$M = (K, \Sigma, \Gamma, \Delta, s, F)$$

όπου:

- K είναι ένα πεπερασμένο σύνολο από καταστάσεις.
- Σ είναι ένα αλφάβητο (τα σύμβολα εισόδου).
- Γ είναι ένα αλφάβητο (τα σύμβολα της στοίβας).
- $s \in K$ είναι η αρχική κατάσταση.
- $F \subseteq K$ είναι το σύνολο των τελικών καταστάσεων.
- Δ είναι η σχέση μεταπτώσεων, δηλαδή ένα πεπερασμένο υποσύνολο του

$$(K \times \Sigma^* \times \Gamma^*) \times (K \times \Gamma^*)$$

Σημειώνοντας, κατ' αρχήν, ότι με τον συμβολισμό A^* παριστάνεται το σύνολο όλων των συμβολοσειρών που δημιουργούνται από την παράθεση μηδέν ή περισσότερων στοιχείων από το σύνολο A (από την παράθεση μηδέν στοιχείων προκύπτει η κενή συμβολοσειρά ϵ , η οποία πάντοτε ανήκει στο A^*), δίνουμε στη συνέχεια τη σημασία της σχέσης μεταπτώσεων:

Αν $((p, u, \beta), (q, \gamma)) \in \Delta$, τότε, εάν το M βρίσκεται στην κατάσταση p με τη συμβολοσειρά β στην κορυφή της στοίβας, μπορεί να διαβάσει τη συμβολοσειρά u από την ακολουθία εισόδου,

να αντικαταστήσει την β με τη συμβολοσειρά γ στην κορυφή της στοίβας και να μεταβεί στην κατάσταση q . Εννοείται ότι μπορεί η u να είναι η κενή συμβολοσειρά ϵ , οπότε τότε το αυτόματο κάνει τη μετάπτωση, χωρίς να διαβάσει τίποτα από την ακολουθία εισόδου. Αν η β είναι ϵ , τότε το αυτόματο δεν εξάγει τίποτα από τη στοίβα, απλώς εισάγει την γ . Αν η γ είναι ϵ , τότε το αυτόματο δεν εισάγει τίποτα στη στοίβα, απλώς εξάγει την β .

Μία ειδική κατηγορία αυτομάτων στοίβας είναι τα ντετερμινιστικά αυτόματα στοίβας, για τα οποία ισχύει ότι η σχέση μεταπτώσεων είναι τέτοια, ώστε σε κάθε περίπτωση (κατάστασης, ακολουθίας εισόδου και στοίβας) μία το πολύ μετάπτωση να είναι δυνατή.

Παράδειγμα: $M = (K, \Sigma, \Gamma, \Delta, s, F)$

$$\begin{aligned} \text{με } K &= \{s, f\} \\ \Sigma &= \{a, b, c\} \\ \Gamma &= \{a, b\} \\ F &= \{f\} \\ \Delta &= \{ ((s, a, e), (s, a)), \\ &\quad ((s, b, e), (s, b)), \\ &\quad ((s, c, e), (f, e)) \\ &\quad ((f, a, a), (f, e)) \\ &\quad ((f, b, b), (f, e)) \} \end{aligned}$$

Το παραπάνω αυτόματο αναγνωρίζει όλες τις συμβολοσειρές της μορφής wcw^R , όπου $w \in \{a, b\}^*$ (με τον συμβολισμό w^R παριστάνεται η αντίστροφη συμβολοσειρά της w), π.χ. $ababcbaba$, $baacaab$, $abbaaacaabba$, κοκ .

Να γραφεί ένα πρόγραμμα C++ το οποίο να υλοποιεί ένα ντετερμινιστικό αυτόματο στοίβας. Το πρόγραμμα πρέπει να είναι γενικό ώστε να λειτουργεί για οποιαδήποτε εξάδα αυτομάτου.

11.4.3 Να υλοποιηθεί σε C++ μια προσομοίωση (μιας εκδοχής) του παιχνιδιού με τις “μουσικές καρέκλες”. Στο παιχνίδι αυτό, έχουμε N συμμετέχοντες και $N - 1$ καρέκλες. Οι συμμετέχοντες σχηματίζουν αλυσίδα και γυρνούν γύρω από τις καρέκλες, ενόσω κτυπά ένα τύμπανο (ένα βήμα για κάθε κρούση).

Για το κάθε χτύπημα, θεωρούμε ότι αυτό που αλλάζει είναι η θέση του καθενός μέσα στην αλυσίδα (ο πρώτος γίνεται N -οστός, ο δεύτερος πρώτος, ο i -οστός πάει στη θέση $i - 1$ και ο N -οστός πάει στη θέση $N - 1$). Δηλαδή, αν η τρέχουσα κατάσταση είναι όπως στο Σχήμα 11.1, η κατάσταση μετά από ένα χτύπημα είναι όπως φαίνεται στο Σχήμα 11.2.

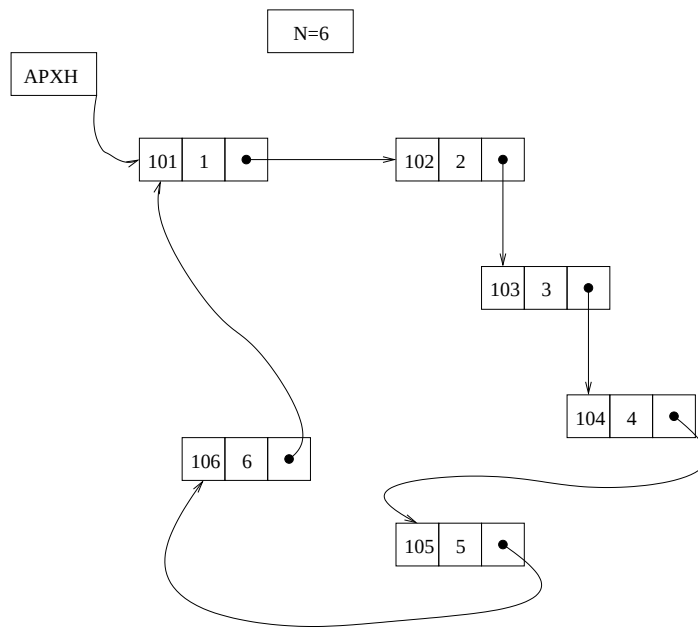
Το τύμπανο κτυπά για τυχαίο αριθμό φορών. Μόλις το τύμπανο σταματήσει, $N - 1$ άτομα καταλαμβάνουν τις καρέκλες και ο j -οστός (τυχαίος) που περισσεύει διαγράφεται από την αλυσίδα. Η αλυσίδα ανασυντάσσεται με τους υπόλοιπους και αφαιρούμε μια καρέκλα. Για λόγους ευκολίας, θεωρούμε ότι διατηρείται η σχετική διάταξη μεταξύ των συμμετεχόντων.

Η διαδικασία τύμπανο-διαγραφή-ανασύνταξη επαναλαμβάνεται, έως ότου μείνει ένας μόνο στην αλυσίδα, ο οποίος είναι και ο νικητής.

Κάθε συμμετέχων στην αλυσίδα έχει μια (αριθμητική) ταυτότητα και καταχωρεί και τη θέση του στη διάταξη της αλυσίδας. Επίσης, συνδέεται (“δείχνει”) στον επόμενο του. Ο τελευταίος (N -οστός) συνδέεται (“δείχνει”) με τον πρώτο συμμετέχοντα.

Η αλυσίδα να αναπαρασταθεί ως κλάση με την εξής συμπεριφορά:

- Αρχικοποιείται με βάση το μήκος της N (ακέραιος). Κατά τη διαδικασία αρχικοποίησης, δημιουργούνται N συμμετέχοντες και συνδέονται μεταξύ τους. Οι ταυτότητές τους είναι τυχαίοι αριθμοί.



Σχήμα 11.1: Η τρέχουσα κατάσταση της αλυσίδας

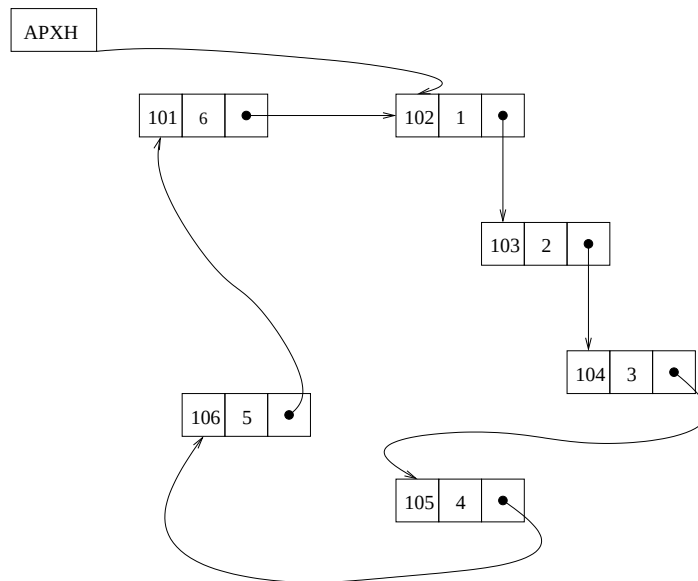
- Ανταποκρίνεται σε ένα χτύπημα του τυμπάνου (hit), αλλάζοντας κατά ένα τη θέση κάθε συμμετέχοντος (όπως περιγράφηκε παραπάνω) και, κατά συνέπεια, την αρχή της αλυσίδας.
- Ανταποκρίνεται στο σταμάτημα του τυμπάνου (stop_tune). Στην περίπτωση αυτή, γίνεται διαγραφή ενός (τυχαίου) συμμετέχοντος, “διορθώνοντας” τη σύνδεση των υπολοίπων και ενημερώνοντας το μήκος αλυσίδας με το πλήθος των συμμετεχόντων που απέμειναν. Επίσης, γίνεται ενημέρωση της αρχής της αλυσίδας, αν είναι απαραίτητο. Τέλος, εκτυπώνεται η ταυτότητα αυτού που διαγράφηκε.
- Η αλυσίδα “ανασυντάσσεται” (resume), διορθώνοντας τις θέσεις των συμμετεχόντων, στην περίπτωση που κάποιοι συνδεδεμένοι δεν έχουν διαδοχικές θέσεις.
- Εκτυπώνονται οι ταυτότητες των συμμετεχόντων της αλυσίδας με τη σειρά που αυτοί εμφανίζονται στην αλυσίδα (print_chain).
- Γίνεται έλεγχος αν το παιχνίδι τελείωσε, δηλαδή αν έχει απομείνει μόνο ένας συμμετέχων στην αλυσίδα (game_over).
- Εμφανίζει τον νικητή, αν το παιχνίδι έχει τελειώσει (winner).

Η κάθε αλυσίδα περιέχει:

- (δείκτη προς) τον πρώτο συμμετέχοντα της αλυσίδας και
- το τρέχον μέγεθος της αλυσίδας.

Προκειμένου να αναδείξετε την προσομοίωσή σας, να υλοποιήσετε μια συνάρτηση main η οποία:

- Αρχικά, να διαβάζει από την προκαθορισμένη είσοδο το πλήθος των συμμετεχόντων N .
- Κατόπιν, να δημιουργεί μια αλυσίδα μήκους N .
- Στη συνέχεια, να επαναλαμβάνει $N - 1$ φορές τη διαδικασία:
 1. Κρούση τυμπάνου (K φορές, όπου K τυχαίος αριθμός από το 1 έως το 10).
 2. Σταμάτημα τυμπάνου.



Σχήμα 11.2: Η κατάσταση της αλυσίδας μετά το χτύπημα του τύμπανου

3. Ανασύνταξη της αλυσίδας.
 4. Εκτύπωση της αλυσίδας.
- Τέλος, να εκτυπώνεται ο νικητής του παιχνιδιού.

(Σημείωση): οι καρέκλες δεν χρειάζεται να αναπαρασταθούν.

11.4.4 Έστω ότι έχουμε ένα σύνολο κανόνων `if_then` της μορφής `if Συνθήκη then Συμπέρασμα`, όπου `Συνθήκη` και `Συμπέρασμα` είναι αλφαριθμητικές ακολουθίες. Π.χ.

`if καλός_καιρός then πάω_στη_θάλασσα`

`if πάω_στη_θάλασσα then κολυμπήσω`

Υιοθετώντας μια κατάλληλη μεθοδολογία-αλγόριθμο της αρεσκείας σας, να υλοποιήσετε ένα πρόγραμμα C++ που, δεδομένου ενός συνόλου `if_then` κανόνων, ενός αλφαριθμητικού που αντιστοιχεί σε κάτι που ισχύει (γεγονός) και ενός άλλου αλφαριθμητικού, να ελέγχει αν το δεύτερο ισχύει ως συμπέρασμα του πρώτου με βάση τους κανόνες. Π.χ. με δεδομένους τους δύο παραπάνω κανόνες και το γεγονός “καλός_καιρός”, το “κολυμπήσω” ισχύει. Στην περίπτωση αυτή, να εκτυπώνει και την αλυσίδα των κανόνων που συνδέουν το γεγονός με το συμπέρασμα. Π.χ.

`καλός_καιρός then πάω_στη_θάλασσα then κολυμπήσω`

Στην περίπτωση που το παραπάνω μπορεί να αποδειχθεί με παραπάνω από μία αλυσίδες κανόνων, να εκτυπώνονται όλες. Στην περίπτωση που το αλφαριθμητικό-ζητούμενο δεν μπορεί να εξαχθεί από το γεγονός με βάση τους κανόνες, να εκτυπώνεται η απάντηση “No solution found”.

Θεωρήστε ότι οι κανόνες δίνονται σε ένα αρχείο με τον εξής τρόπο:

1. Σε κάθε γραμμή του αρχείου αντιστοιχούμε έναν κανόνα.

2. Κάθε κανόνας αναπαριστάται από δύο αλφαριθμητικές ακολουθίες: η πρώτη αντιστοιχεί στη συνθήκη του κανόνα και η δεύτερη στο συμπέρασμά του.

Π.χ.

καλός_καιρός πάω_στη_θάλασσα
πάω_στη_θάλασσα κολυμπήσω

Άλλο παράδειγμα:

Κανόνες:

if Θα ακούσω ειδήσεις then Θα ενημερωθώ για την επικαιρότητα
if Έχει καλοκαιρία then Θα συναντήσω τη Μαίρη
if Θα ανοίξω την τηλεόραση then Θα ακούσω ειδήσεις
if Θα τηλεφωνήσω στη Μαίρη then Θα συναντήσω τη Μαίρη
if Θα τηλεφωνήσω στη Μαίρη then Θα με ενημερώσει η Μαίρη
if Θα διαβάσω εφημερίδα then Θα ενημερωθώ για την επικαιρότητα
if Θα με ενημερώσει η Μαίρη then Θα ενημερωθώ για την επικαιρότητα
if Θα συναντήσω τη Μαίρη then Θα με ενημερώσει η Μαίρη
if Θα ακούσω ραδιόφωνο then Θα ενημερωθώ για την επικαιρότητα

Περιεχόμενα Αρχείου:

Θα_ακούσω_ειδήσεις Θα_ενημερωθώ_για_την_επικαιρότητα
Έχει_καλοκαιρία Θα_συναντήσω_τη_Μαίρη
Θα_ανοίξω_την_τηλεόραση Θα_ακούσω_ειδήσεις
Θα_τηλεφωνήσω_στη_Μαίρη Θα_συναντήσω_τη_Μαίρη
Θα_τηλεφωνήσω_στη_Μαίρη Θα_με_ενημερώσει_η_Μαίρη
Θα_διαβάσω_εφημερίδα Θα_ενημερωθώ_για_την_επικαιρότητα
Θα_με_ενημερώσει_η_Μαίρη Θα_ενημερωθώ_για_την_επικαιρότητα
Θα_συναντήσω_τη_Μαίρη Θα_με_ενημερώσει_η_Μαίρη
Θα_ακούσω_ραδιόφωνο Θα_ενημερωθώ_για_την_επικαιρότητα

Γεγονός: Θα_τηλεφωνήσω_στη_Μαίρη

Ζητούμενο: Θα_ενημερωθώ_για_την_επικαιρότητα

Απάντηση:

1. Θα_τηλεφωνήσω_στη_Μαίρη then
 Θα_με_ενημερώσει_η_Μαίρη then Θα_ενημερωθώ_για_την_επικαιρότητα
2. Θα_τηλεφωνήσω_στη_Μαίρη then Θα_συναντήσω_τη_Μαίρη then
 Θα_με_ενημερώσει_η_Μαίρη then Θα_ενημερωθώ_για_την_επικαιρότητα

(οι δυο αλυσίδες-αποτελέσματα δεν είναι απαραίτητο να εμφανίζονται με αυτή τη σειρά)

Προκειμένου να ολοκληρώσετε την άσκηση αυτή, πρέπει:

1. Να βρείτε μια κατάλληλη αναπαράσταση των κανόνων.
2. Να υιοθετήσετε έναν αλγόριθμο “αναζήτησης”.
3. Να αποφασίσετε τις δομές που απαιτούνται για την υλοποίησή του.

11.4.5 Να υλοποιηθεί σε C++ η προσομοίωση μιας (υποθετικής) λειτουργίας των ασανσέρ του πύργου του Άιφελ στο Παρίσι. Σύμφωνα με αυτή, ισχύουν τα παρακάτω.

Ο πύργος του Άιφελ έχει τρεις (ανοικτούς για το κοινό) ορόφους και τέσσερα ασανσέρ. Κάθε όροφος έχει μια χωρητικότητα επισκεπτών, διαφορετική ο καθένας. Κάθε ασανσέρ έχει μια χωρητικότητα επιβατών, C , ίδια για όλα τα ασανσέρ. Όταν το πλήθος των επισκεπτών ενός ορόφου φτάσει στο μέγιστο της χωρητικότητάς του, το ασανσέρ δεν επιτρέπεται να αποβιβάσει άλλους επισκέπτες στον όροφο αυτό. Οι επισκέπτες αυτοί παραμένουν στο ασανσέρ για να αποβιβαστούν (ενδεχόμενα) κατά την επόμενη διαδρομή.

Κάθε ασανσέρ ανεβαίνει από το ισόγειο στον τρίτο όροφο, σταματώντας στους ενδιάμεσους ορόφους. Στον κάθε όροφο σταματά για χρόνο T , ίδιο για όλους τους ορόφους, για μια διαδρομή ανόδου-καθόδου, εκτός αν υπάρχουν ακόμα επισκέπτες που επιβιβάζονται ή αποβιβάζονται, οπότε ο χρόνος αυτός ενδεχόμενα παρατείνεται, όπως εξηγείται στη συνέχεια. Θεωρούμε ότι ο χρόνος επιβίβασης ενός επισκέπτη είναι ίσος με τον χρόνο αποβίβασης και ισούται με μια χρονική μονάδα. Επίσης, η αποβίβαση επισκεπτών γίνεται ταυτόχρονα με την επιβίβαση — ανοίγουν ταυτόχρονα δυο πόρτες —.

Έστω ότι όταν φτάσει ένα ασανσέρ στον όροφο i , A_i επισκέπτες επιθυμούν να αποβιβαστούν και K_i επισκέπτες επιθυμούν να επιβιβαστούν. Όμως, πραγματικά, θα αποβιβαστούν A'_i επισκέπτες και θα επιβιβαστούν K'_i επισκέπτες όπου

$$A'_i = \min(A_i, C_i - Cur_i + K_i)$$

$$K'_i = \min(K_i, C - Cur + A_i)$$

όπου Cur_i είναι το πλήθος των επισκεπτών του ορόφου i εκείνη τη στιγμή, C_i η χωρητικότητα του ορόφου, Cur το πλήθος των επισκεπτών που βρίσκονται στο ασανσέρ εκείνη τη στιγμή και C η χωρητικότητα του ασανσέρ. Η αιτία ενδεχόμενης μείωσης του A_i και του K_i σε A'_i και K'_i , αντίστοιχα, είναι για να μην παραβιαστεί ο περιορισμός της χωρητικότητας του ορόφου και του ασανσέρ.

Ο χρόνος στάσης σε έναν όροφο ισούται με $\max(T, A'_i, K'_i)$. Ο χρόνος αναμονής T είναι παράμετρος της προσομοίωσης.

Κατά τη στάση ενός ασανσέρ σε έναν όροφο, εκτυπώνονται πόσοι επισκέπτες αποβιβάστηκαν, πόσοι επιβιβάστηκαν, πόσοι είναι οι επισκέπτες του ορόφου και πόσο χρόνο σταμάτησε στον όροφο.

Να σημειωθεί ότι ένας επισκέπτης βγάζει εισιτήριο για έναν μόνο όροφο και μπορεί να επισκεφθεί μόνο αυτόν.

Συγκεκριμένα, ο πύργος του Άιφελ αναπαριστάται από μια κλάση η οποία:

- Έχει τρεις ορόφους.
- Έχει τέσσερα ασανσέρ.
- Έχει ένα σύνολο επισκεπτών που περιμένουν στο ισόγειο να επιβιβαστούν.

Η συμπεριφορά της είναι η εξής:

- Αρχικά, δεν υπάρχει κανένας επισκέπτης.
- Η λειτουργία του πύργου είναι ένας κύκλος κατά τον οποίο επαναλαμβάνονται τα εξής: προστίθενται επισκέπτες στο σύνολο των επισκεπτών που περιμένουν στο ισόγειο να επιβιβαστούν. Γίνεται επιβίβαση επισκεπτών στα ασανσέρ. Τα ασανσέρ ανεβαίνουν. Κατόπιν, τα ασανσέρ κατεβαίνουν. (operate)

Ένας όροφος:

- Έχει μια χωρητικότητα επισκεπτών.
- Έχει έναν αριθμό που αντιστοιχεί στο πλήθος των επισκεπτών που βρίσκονται αυτήν τη στιγμή στον όροφο.
- Έχει έναν αριθμό που αντιστοιχεί στο πλήθος των επισκεπτών που ολοκλήρωσαν την επίσκεψή τους και περιμένουν να κατέβουν με το ασανσέρ.

Αρχικά, δεν υπάρχει κανείς επισκέπτης. Απλά ανατίθεται η τιμή της χωρητικότητάς του.

Ένα ασανσέρ αναπαριστάνεται από μια κλάση η οποία:

- Έχει μια χωρητικότητα επισκεπτών.
- Έχει ένα σύνολο επιβιβασμένων επισκεπτών.

Η συμπεριφορά της είναι η εξής:

- Αρχικά, το ασανσέρ είναι άδειο και ανατίθεται η τιμή της χωρητικότητάς του.
- Ένα ασανσέρ επιβιβάζει από το ισόγειο επισκέπτες πλήθους τόσοι ώστε να μη ξεπερνιέται η χωρητικότητά του. (`load`)
- Ένα ασανσέρ αποβιβάζει στο ισόγειο όλους εκείνους τους επισκέπτες που επισκέφτηκαν τον όροφο που επιθυμούσαν. (`empty`)
- Κατά την άνοδό του, σταματά σε κάθε όροφο. Κατά τη στάση σε όροφο, αποβιβάζει τους επισκέπτες που επιθυμούν να επισκεφτούν τον όροφο και είναι επιτρεπτό και επιβιβάζει αυτούς που επιθυμούν να κατέβουν — και είναι επιτρεπτό, πάλι —. (`ascend`)
- Κατά την κάθοδό του, σταματά σε κάθε όροφο. Κατά τη στάση σε όροφο, επιβιβάζει αυτούς που επιθυμούν να κατέβουν και είναι επιτρεπτό. (`descend`)
- Η αποβίβαση επισκεπτών σε έναν όροφο επιστρέφει το πλήθος των επισκεπτών που τελικά αποβιβάστηκαν στον όροφο αυτό. (`get_out`)
- Η επιβίβαση επισκεπτών από έναν όροφο επιστρέφει το πλήθος των επισκεπτών που τελικά επιβιβάστηκαν από τον όροφο αυτό. (`get_in`)

Οι επισκέπτες αναπαριστώνται από τον όροφο που θέλουν να επισκεφθούν. Κατά την κάθοδό τους, ο όροφος μηδενίζεται.

Να υλοποιήσετε τις απαραίτητες κλάσεις σε C++. Επίσης, να υλοποιηθεί και μια συνάρτηση `main` που να εκτελεί N κύκλους ανόδου-κάθόδου των ασανσέρ, για καθέναν από τους οποίους προστίθενται K νέοι επισκέπτες στο ισόγειο με τυχαίες προτιμήσεις ορόφων.

Τα N , K , T , C και C_i , $i = 1, 2, 3$ είναι παράμετροι της προσομοίωσης και να δίνονται κατά την κλήση του προγράμματος από τη γραμμή εντολής, με αυτήν τη σειρά.

- 11.4.6** Να υλοποιηθεί σε C++ μια προσομοίωση προφορικής εξέτασης παραδοτέων ασκήσεων, όπως περιγράφεται παρακάτω. Η προφορική εξέταση γίνεται από τους Μεταπτυχιακούς Συνεργάτες του μαθήματος που έχουν διορθώσει τα παραδοτέα των φοιτητών.

Ένας Συνεργάτης διορθώνει όλες τις ασκήσεις που παρέδωσε ένας φοιτητής. Η εργασία εκπονείται από δύο φοιτητές, αλλά διορθώνεται από τον Συνεργάτη που έχει αναλάβει τη διόρθωση των ασκήσεων του ενός. Ένας φοιτητής μπορεί να έχει παραδώσει μέρος του Πρακτικού μέρους — κάποιες μόνον από τις ασκήσεις ή/και την εργασία —. Η ανάθεση των παραδοτέων των φοιτητών γίνεται με βάση το λήγον ψηφίο του αριθμού μητρώου των φοιτητών. Κάθε λήγοντας ανατίθεται σε έναν Συνεργάτη. Σε κάθε

Συνεργάτη, μπορεί να έχει ανατεθεί ένας, δύο ή τρεις λήγοντες. Η διόρθωση του Πρακτικού Μέρους και, κατά συνέπεια, η προφορική εξέτασή του, θα γίνει από τέσσερις Μεταπτυχιακούς Συνεργάτες.

Ένας Συνεργάτης χαρακτηρίζεται από το όνομά του, από τους λήγοντες που έχει αναλάβει για διόρθωση, έναν μέσο χρόνο εξέτασης ανά φοιτητή και έναν μέσο χρόνο εξέτασης ανά φοιτητή, για τον τελευταίο κύκλο εξέτασης (ο κύκλος εξέτασης εξηγείται παρακάτω). Επίσης, σχετίζεται με μια ουρά φοιτητών που έχουν μπει μέσα στον χώρο εξέτασης και περιμένουν να εξεταστούν από αυτόν. Αυτή μπορεί να αποτελείται από το πολύ *MaxIn* φοιτητές. Αρχικά, του ανατίθενται οι λήγοντες που εξετάζει και η ουρά είναι κενή.

Ένας Φοιτητής χαρακτηρίζεται από τον αριθμό μητρώου του καθώς και το ποιος από τις ασκήσεις ή/και την εργασία έχει παραδώσει. Σε περίπτωση που έχει παραδώσει εργασία, η πληροφορία γι' αυτή σχετίζεται με τον συμφοιτητή του μαζί με τον οποίο την εκπόνησε.

Η Προφορική Εξέταση έχει μια ουρά φοιτητών που περιμένουν να εξεταστούν έξω από τον χώρο εξέτασης και τους τέσσερις Συνεργάτες που εξετάζουν. Θεωρούμε ότι όλοι οι φοιτητές που έχουν παραδώσει πρακτικό μέρος θα έλθουν να εξεταστούν και είναι πλήθους *TotalNoofStudents*. Η Προφορική Εξέταση γίνεται σε κύκλους. Κάθε κύκλος διαρκεί *T* μονάδες χρόνου. Σε κάθε κύκλο:

- Φτάνουν φοιτητές στο προαύλιο και προστίθενται στην ουρά των φοιτητών που περιμένουν να εξεταστούν. Θεωρούμε ότι σε κάθε κύκλο, ο αριθμός αυτός είναι τυχαίος (οι φοιτητές που έχουν εξεταστεί μέχρι τότε συν αυτούς που προστίθενται δεν πρέπει να ξεπερνούν το *TotalNoofStudents* πλήθος). Γίνεται εκτύπωση του αριθμού μητρώου τους και των άλλων στοιχείων τους και του μηνύματος "Arriving!".
- Γίνεται εξέταση φοιτητών, τυχαίου αριθμού ανά συνεργάτη, οι οποίοι αποχωρούν, αν ολοκληρώθηκε η εξέτασή τους. Γίνεται εκτύπωση του αριθμού μητρώου τους και του μηνύματος "Completed!".
- Εισέρχονται φοιτητές από την ουρά στον χώρο εξέτασης και προστίθενται στις ουρές των Συνεργατών, όσοι μπορούν να προστεθούν σε μια ουρά. Επίσης, εκτός ουράς, εισέρχονται και οι συμφοιτητές των εργασιών τους, οι οποίοι, όμως, μετά την εξέταση επιστρέφουν στη θέση τους στην εξωτερική ουρά. Αν πάλι, η εργασία ενός φοιτητή που μπήκε για να εξεταστεί με βάση τον λήγοντά του έχει διορθωθεί από άλλον Συνεργάτη, ο φοιτητής αυτός μετά την εξέτασή του περιμένει να κληθεί για την εξέταση της εργασίας, εκτός ουράς. Γίνεται εκτύπωση του αριθμού μητρώου τους, το όνομα του Συνεργάτη που θα τους εξετάσει και το μήνυμα "Entering...".
- Για τον κάθε συνεργάτη, ελέγχεται ο μέσος χρόνος προφορικής εξέτασης φοιτητή αλλά και ο μέσος χρόνος εξέτασης ανά φοιτητή, για τον τελευταίο κύκλο εξέτασης. Αν ο πρώτος είναι μεγαλύτερος από *MaxTime* ή αν περιμένουν *M* φοιτητές στην εξωτερική ουρά να εξεταστούν από αυτόν και ο μέσος χρόνος εξέτασης ανά φοιτητή, για τον τελευταίο κύκλο εξέτασης, είναι μεγαλύτερος από *MaxRecentTime*, να γίνεται μια ενημέρωση για να επιταχύνει, εκτυπώνοντας το όνομα του Συνεργάτη και το μήνυμα "...please, hurry up". Το *M* τροποποιείται ανάλογα με το αν ο Συνεργάτης εξετάζει έναν, δύο ή τρεις λήγοντες σε $M \times Quant$, όπου $Quant = 1, 1.5, 2$, αντίστοιχα.

Επίσης, μπορούμε να πάρουμε τα στατιστικά στοιχεία της Προφορικής Εξέτασης, δηλαδή, αριθμό φοιτητών ανά συνεργάτη, μέσο όρο διάρκειας προφορικής εξέτασης ανά συνεργάτη, μέγιστη συνολική διάρκεια εξέτασης, συνολικό αριθμό φοιτητών που εξετάστηκαν.

Αρχικά, στην Προφορική Εξέταση ανατίθεται το πλήθος φοιτητών, *TotalNoofStudents*.

Επίσης, να υλοποιηθεί μια συνάρτηση *main* που:

1. Να δημιουργεί μια Προφορική Εξέταση για *TotalNoofStudents* πλήθος φοιτητών.

2. Να γίνονται τόσοι κύκλοι εξέτασης, ώστε να μην υπάρχουν φοιτητές που δεν έχουν προσέλθει και να μην έχουν ξεπεραστεί 8 ώρες. Σε περίπτωση που λήξουν οι κύκλοι και δεν υπάρχουν φοιτητές που δεν έχουν εξεταστεί, να εκτυπώνονται τα στατιστικά της εξέτασης και να εκτυπώνεται το μήνυμα "All OK!". Σε αντίθετη περίπτωση, να εκτυπώνονται τα στατιστικά, αλλά και το πλήθος των φοιτητών που δεν εξετάστηκαν, καθώς και το μήνυμα "Bad time management, must hurry up! Try again!".

Τα *TotalNoofStudents*, *M*, *MaxIn*, *T*, *MaxTime* και *MaxRecentTime* είναι παράμετροι της προσομοίωσης και να δίνονται κατά την κλήση του προγράμματος από τη γραμμή εντολής, με αυτήν τη σειρά.

11.4.7 Να υλοποιηθεί σε C++ μια σκηνή κινουμένων σχεδίων επιδείξεων σχηματισμών παγοδρόμων που έχει ως εξής:

Θεωρούμε ότι υπάρχει μια λίμνη μήκους L . Για ευκολία, θεωρούμε ότι αυτή έχει σχήμα ορθογώνιο παραλληλόγραμμο και το L αντιστοιχεί στο μήκος της βάσης του.

Θεωρούμε ότι έχουμε και N παγοδρόμους. Οι μισές φοράνε άσπρο κοστούμι και οι άλλες μισές μαύρο. Κάθε παγοδρόμος γεννιάται με τυχαίο χρώμα κοστούμιού. Επίσης, κάθε παγοδρόμος χαρακτηρίζεται από την ταυτότητά της (τυχαίο διακριτό αριθμό) και από την απόσταση της λίμνης που διανύει σε μια ολίσθηση (τυχαίος αριθμός μικρότερος του L). Τέλος, κάθε παγοδρόμος έχει και μια δεξιότητα, είτε *spin* είτε *jump*, που αποκτά κατά τη γέννησή της (τυχαία). Η παγοδρόμος επιδεικνύει (*perform*) τη δεξιότητα αυτή, εκτυπώνοντάς την σε ένα μήνυμα, π.χ. "My skill is" ακολουθούμενο από την τιμή της δεξιότητάς.

Οι παγοδρόμοι γεννώνται μία-μία. Αμέσως μόλις γεννηθεί μια παγοδρόμος, τοποθετείται στο τέλος μιας ουράς που θεωρούμε ότι θα σχηματίσουν. Σε αυτήν την ουρά, τελικά θα τοποθετηθούν όλες οι παγοδρόμοι. Στην ουρά απαιτείται διαδοχικές παγοδρόμοι να φοράνε διαφορετικό χρώμα ρούχα. Αν τύχει και γεννηθούν διαδοχικά παγοδρόμοι με το ίδιο χρώμα με της τελευταίας παγοδρόμου της ουράς, αυτές προστίθενται σε μια δεύτερη ουρά και περιμένουν μέχρι να γεννηθεί και να προστεθεί παγοδρόμος του άλλου χρώματος στην πρώτη. Τότε, η πρώτη παγοδρόμος της ουράς που περιμένει αφαιρείται και προστίθεται στη βασική ουρά.

Στη συνέχεια, θεωρούμε ότι στις τέσσερις άκρες της λίμνης (κορυφές του ορθογώνιου) θα συγκεντρωθούν οι παγοδρόμοι με τη λογική στοίβας. Θεωρούμε ότι σχηματίζεται σε κάθε άκρη της λίμνης από μία στοίβα.

Οι στοίβες γεμίζουν εναλλάξ με κυκλική φορά (δηλαδή πρώτα η κάτω αριστερή, μετά η κάτω δεξιά, μετά η πάνω δεξιά και μετά η πάνω αριστερή κοκ.), αφαιρώντας μία-μία τις παγοδρόμους από την ουρά. Πάλι απαγορεύεται να εισαχθούν διαδοχικά στην ίδια στοίβα παγοδρόμοι με ίδιο χρώμα κοστούμιών (οπότε κάθε παγοδρόμος πάει στην πρώτη που επιτρέπεται να εισαχθεί).

Κατόπιν, όταν θα έχουν τοποθετηθεί όλες οι παγοδρόμοι σε στοίβες, αφαιρούνται κυκλικά πάλι από αυτές, μία μία παγοδρόμος. Κάθε παγοδρόμος, όταν αφαιρείται, επιδεικνύει τη δεξιότητά της και ολισθαίνει έως ότου βγει από την απέναντι πλευρά της λίμνης. Οι παγοδρόμοι βγαίνοντας εκτυπώνουν την ταυτότητά τους, το χρώμα τους και τον αριθμό των διασκελισμών που έκαναν για να διανύσουν το μήκος της λίμνης (για ευκολία, θεωρήστε ότι όλες οι παγοδρόμοι βρίσκονται σε μηδενική απόσταση από την άκρη της λίμνης ανεξάρτητα από τη θέση τους στη στοίβα).

Κατά την προσθήκη και την αφαίρεση παγοδρόμων από τις στοίβες και τις ουρές, να εκτυπώνονται σχετικά μηνύματα, π.χ. "adding IceSkater <id> to the UpperLeftStack", "deleting IceSkater <id> from the MainQueue".

Να υλοποιήσετε τις απαραίτητες κλάσεις σε C++. Επίσης να υλοποιηθεί και μια συνάρτηση *main* που να εκτελεί το παραπάνω σενάριο.

Το L και το N είναι παράμετροι και να δίνονται κατά την κλήση του προγράμματος από τη γραμμή εντολής με τη σειρά αυτή.

11.4.8 Όπως ξέρετε από τη Χημεία του Λυκείου, άτομα άνθρακα συνενώνονται με άλλα άτομα άνθρακα ή άλλων στοιχείων, μέσω κοινών (ζευγαριών) ηλεκτρονίων, προκειμένου να συμπληρωθεί η εξωτερική στοιβάδα του καθενός, και σχηματίζουν ενώσεις. Το κοινό ζεύγος ηλεκτρονίων μεταξύ των ατόμων αποτελεί δεσμό. Μεταξύ των ατόμων, μπορεί να σχηματιστούν απλοί, διπλοί ή τριπλοί δεσμοί, κατά τους οποίους τα άτομα μοιράζονται ένα, δύο ή τρία ζευγάρια ηλεκτρονίων.

Οι υδρογονάνθρακες είναι μια κατηγορία ενώσεων ατόμων άνθρακα με άτομα υδρογόνου. Στην περίπτωση που τα άτομα άνθρακα συνενώνονται έτσι ώστε να δημιουργείται μια μη-κυκλική αλυσίδα, αυτοί λέγονται μη-κυκλικοί υδρογονάνθρακες. Τα αλκάνια, τα αλκένια και τα αλκίνια είναι κατηγορίες μη-κυκλικών υδρογονανθράκων. Τα αλκάνια έχουν γενική μορφή μοριακού τύπου: $C_\nu H_{2\nu+2}$, $\nu \geq 1$, τα αλκένια έχουν γενική μορφή μοριακού τύπου: $C_\nu H_{2\nu}$, $\nu \geq 2$ και τα αλκίνια έχουν γενική μορφή μοριακού τύπου: $C_\nu H_{2\nu-2}$, $\nu \geq 2$.

Αφού αποφασίσετε μια αναπαράσταση των ατόμων του άνθρακα και του υδρογόνου στη C++ ώστε να μπορεί να χρησιμοποιηθεί για την αναπαράσταση των υδρογονανθράκων, να υλοποιήσετε σε C++ μια συνάρτηση `main` που να κάνει τα παρακάτω:

1. Για ένα τυχαίο N , να δημιουργούνται N άτομα άνθρακα.
2. Για καθένα άτομο από αυτά, να δημιουργούνται δεσμοί, τυχαία, είτε με υπάρχοντα άτομα άνθρακα, αν είναι δυνατόν, είτε να δημιουργούνται άτομα υδρογόνου και να συνδέονται με αυτά, έως ότου όλα τα ηλεκτρόνια να ζευγαρώσουν.
3. Να εκτυπώνονται οι ενώσεις των ατόμων.
4. Με βάση τους παραπάνω γενικούς τύπους, να γίνεται αναγνώριση της ένωσης ή των ενώσεων που δημιουργήθηκαν και να εκτυπώνεται η κατηγορία της/τους.

Η διαδικασία αυτή να επαναλαμβάνεται K φορές, όπου το K να δίνεται από τη γραμμή εντολής. Σημείωση: Για διευκόλυνση της υλοποίησης των παραπάνω ενεργειών, στα άτομα μπορείτε να δίνετε μια ταυτότητα. Κατόπιν, για την εκτύπωση, αρκεί να εκτυπώνετε ποιο άτομο είναι συνδεδεμένο με ποια άλλα. Σε περίπτωση διπλών ή τριπλών δεσμών, η εκτύπωση θα το εμφανίζει ρητά.

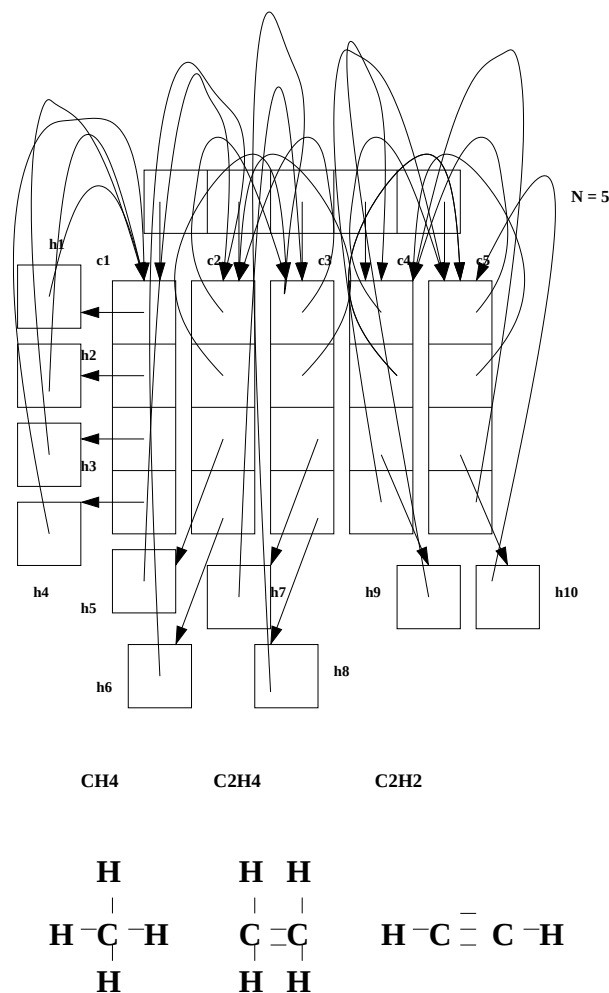
Παράδειγμα για $N = 5$, κατά το οποίο δημιουργήθηκε ένα αλκάνιο (CH_4), ένα αλκένιο (C_2H_4) και ένα αλκίνιο (C_2H_2).

Οι άνθρακες πρέπει να “ζευγαρώσουν” 4 ηλεκτρόνια. Γι’ αυτό και στο σχήμα έχουμε τέσσερα “κουτάκια” (δείκτες) στο κάθε άτομο άνθρακα, ώστε να γνωρίζουμε με ποιο άτομο μοιράζονται το κάθε ηλεκτρόνιο. Αντίστοιχα, στα υδρογόνα έχουμε ένα κουτάκι, που δείχνει με ποιο άτομο μοιράζεται το ηλεκτρόνιό του. Καθότι ένα άτομο άνθρακα μοιράζεται τέσσερα ηλεκτρόνια, τέσσερα άλλα άτομα πρέπει να “δείχνουν” σε αυτό. Οπότε έχουμε συσσώρευση δεικτών στο κάθε άτομο άνθρακα και έτσι καταλήγουμε σε μια τόσο “μπερδεμένη” εικόνα (Σχήμα 11.3).

Απομονώνοντας το αλκένιο (C_2H_4) του Σχήματος 11.3, αυτό αναπαριστάται όπως φαίνεται στο Σχήμα 11.4:

Η εκτύπωση των ενώσεων του παραδείγματος μπορεί να είναι:

```
comp([c1(h1,h2,h3,h4)])
comp([c2(c3(double),h5,h6),c3(c2(double),h7,h8)])
comp([c4(c5(triple),h9),c5(c4(triple),h10)])
```



Σχήμα 11.3: Παράδειγμα για $N = 5$, κατά το οποίο δημιουργήθηκε ένα αλκάνιο (CH_4), ένα αλκένιο (C_2H_4) και ένα αλκίνιο (C_2H_2)

11.4.9 Να υλοποιηθεί σε C++ μια προσομοίωση της εξυπηρέτησης μιας ημέρας ενημέρωσης γονέων μαθητών της δευτεροβάθμιας εκπαίδευσης.

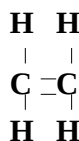
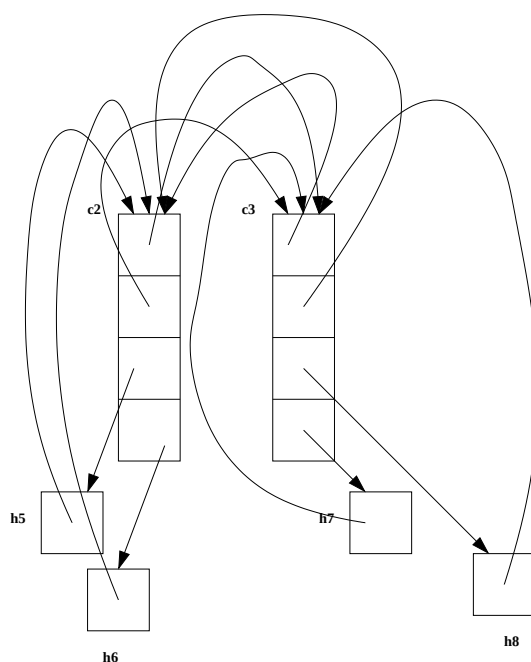
Θεωρούμε ότι η ενημέρωση γίνεται σε μια αίθουσα στην οποία δέχονται N καθηγητές. Οι γονείς επιθυμούν να ενημερωθούν από κάποιους (όχι απαραίτητα από όλους) από αυτούς. Ο κάθε γονέας χαρακτηρίζεται από μια ταυτότητα και τους καθηγητές από τους οποίους θέλει να ενημερωθεί. Ο κάθε καθηγητής χαρακτηρίζεται από έναν αριθμό από το 1 έως το N , που είναι και η ταυτότητά του.

Οι γονείς περιμένουν στην είσοδο της αίθουσας σχηματίζοντας μια ουρά. Όταν έρχεται ένας νέος γονέας, προστίθεται στο τέλος της ουράς. Όταν ένας καθηγητής γίνει διαθέσιμος, ο γονέας που περιμένει στην ουρά της αίθουσας και είναι ο πρώτος που επιθυμεί να ενημερωθεί από τον καθηγητή αυτόν, αφαιρείται από την ουρά και συναντά τον καθηγητή. Όταν ολοκληρωθεί η ενημέρωσή του από τον καθηγητή, ο καθηγητής ξαναγίνεται διαθέσιμος και ο γονέας πηγαίνει στο τέλος της ουράς. Τυχαία, κάποιος γονέας μπορεί να παρανοήσει και να αποπειραθεί να επιστρέψει στην αρχή της ουράς. Τότε οι άλλοι γονείς διαμαρτύρονται και τον επαναφέρουν στην τάξη. Η ουρά και η επανατοποθέτηση γονέα φαίνονται στο Σχήμα 11.5.

Να αναπαραστήσετε τα παραπάνω με χρήση κλάσεων, όπου απαιτείται.

Προκειμένου να αναδείξετε την προσομοίωσή σας, να υλοποιήσετε μια συνάρτηση `main` η οποία:

- Να διαβάζει από την προκαθορισμένη είσοδο το πλήθος N των καθηγητών και ένα πλήθος γονέων K .

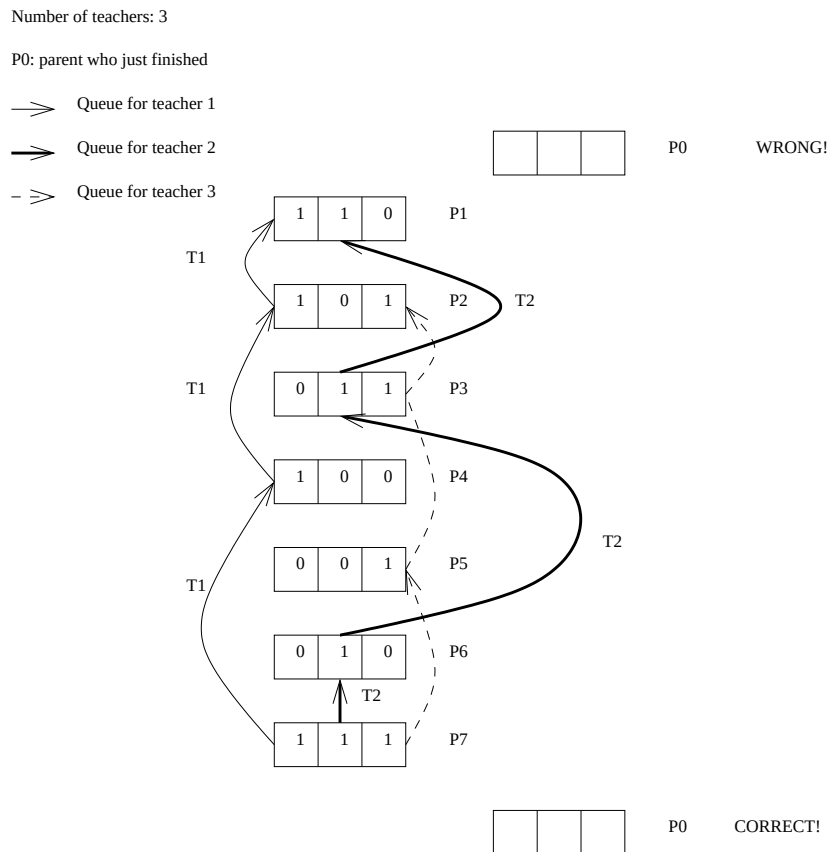


Σχήμα 11.4: Αναπαράσταση ενός αλκένιου (C_2H_4)

- Να δημιουργεί τυχαία K γονείς, αναθέτοντάς τους ταυτότητες καθηγητών από τους οποίους επιθυμούν να ενημερωθούν.
- Ένας-ένας γονέας που δημιουργείται να τοποθετείται στην ουρά της αίθουσας, για τους πρώτους L (τυχαίο) γονείς.
- Να αρχίσει η ενημέρωση γονέων, με τον εξής τρόπο: για τους διαθέσιμους καθηγητές, αφαιρούνται γονείς από την ουρά της αίθουσας. Για καθένα γονέα που αφαιρείται, να προστίθεται (τυχαίο) πλήθος γονέων στο τέλος της ουράς. Μετά την πρόσθεση νέων γονέων στην ουρά, να θεωρείται ότι οι γονείς που ενημερώνονται από καθηγητή ολοκλήρωσαν την ενημέρωσή τους από τον καθηγητή αυτό και είτε επιστρέφουν στην ουρά για να ενημερωθούν και από τους άλλους είτε αποχωρούν, αν δεν επιθυμούν ενημέρωση από άλλους καθηγητές. Σε περίπτωση αποχώρησης, να εκτυπώνεται ανάλογο μήνυμα. Σε περίπτωση επανατοποθέτησής τους στην ουρά, αν αυτό γίνει σωστά, να εκτυπώνεται μήνυμα στωικότητας γονέα, αν γίνει παράτυπα να εκτυπώνεται μήνυμα διαμαρτυρίας γονέων.

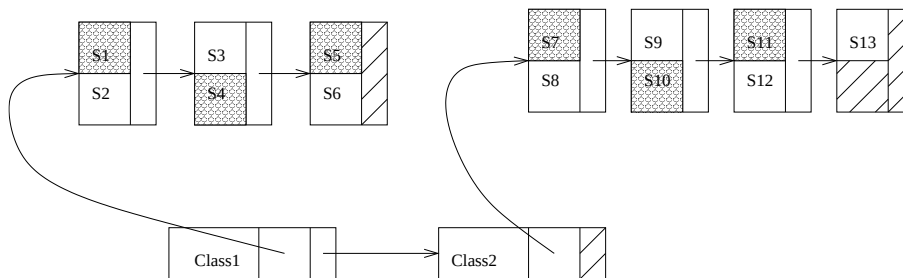
11.4.10 Υλοποιήστε σε C++ μια προσομοίωση της μετακίνησης μαθητών παιδικού σταθμού μεταξύ τμημάτων. Θεωρούμε ότι κάθε τμήμα έχει τον ίδιο αριθμό αγοριών και κοριτσιών, εκτός και αν πρόκειται για περιττό αριθμό, οπότε είναι ένα παραπάνω από τα μεν ή τα δε.

Στον σταθμό αυτό, για να μην δημιουργηθεί αταξία, όταν πρόκειται να μετακινηθούν τα παιδιά ενός τμήματος, σχηματίζουν μια ακολουθία ζευγαριών. Θεωρώντας ότι τα κορίτσια είναι πιο ήσυχα από τα αγόρια (στη γενική περίπτωση :)), τα ζεύγη είναι αγόρι-κορίτσι. Επίσης, το αγόρι του ζευγαριού, στην αντίστοιχη θέση των ζευγαριών που βρίσκονται μπροστά και πίσω του έχει κορίτσια (και ένα κορίτσι, αντίστοιχα, αγόρια). Η ακολουθία χαρακτηρίζεται από τον αριθμό του τμήματος, π.χ. ακέραιος.



Σχήμα 11.5: Ολοκλήρωση ενημέρωσης του γονέα P0 από καθηγητή και επανατοποθέτησή του στην ουρά

Αρχικοποιείται, τοποθετώντας τους μαθητές του τμήματος σε κατάλληλη θέση.



Σχήμα 11.6: Ένα παράδειγμα ακολουθίας ακολουθιών δύο τμημάτων

Όταν πρόκειται να μετακινηθούν όλα τα τμήματα, οι ακολουθίες τους διαδέχονται η μία την άλλη. Μπορείτε να δείτε μια εικόνα ακολουθίας δύο τμημάτων στο Σχήμα 11.6. Αν δύο τμήματα έχουν περιττό αριθμό μαθητών, που όμως οι πλεονάζοντες μαθητές είναι αγόρι-κορίτσι, τότε αυτοί μπαίνουν ως ζευγάρι στην ακολουθία του πρώτου τμήματος.

Παρ' όλα αυτά, αταξίες μπορεί να προκύψουν. Αν μεμονωμένα παιδιά κάνουν αταξίες, αντιμετωπίζονται με άλλα του ίδιου φύλου που επιλέγονται τυχαία από την ακολουθία του τμήματός τους. Αν πρόκειται για ένα ή δύο ζευγάρια, το ίδιο γίνεται για τα παιδιά που τα αποτελούν. Αν, όμως, πρόκειται για περισσότερα ζευγάρια και τα ζευγάρια είναι διαδοχικά, η αντιμετάθεση γίνεται μεταξύ μαθητών διαφορετικών τμημάτων και έτσι διαμοιράζονται στα άλλα τμήματα. Αν τα άτακτα ζευγάρια είναι διάσπαρτα, αρκεί να αντιμετωπιστούν οι μαθητές τους με μαθητές του επόμενου τμήματος. Κάθε τμήμα χαρακτηρίζεται από έναν μετρητή βαθμού αταξίας των μαθητών του. Για κάθε αντιμετάθεση μαθητή, ο μετρητής του τμήματος στο οποίο ανήκει ο μαθητής που την προκάλεσε αυξάνεται κατά έναν βαθμό. Αν προκλήθηκε

αντιμετάθεση μαθητών μεταξύ τμημάτων, η αύξηση είναι διπλή. Αρχικά, κάθε τμήμα έχει μηδενικό βαθμό αταξίας.

Κάθε μαθητής χαρακτηρίζεται και αρχικοποιείται από το όνομά του, το τμήμα και το φύλο του. Δίνεται η δυνατότητα εκτύπωσης των στοιχείων αυτών.

Δίνεται η δυνατότητα εκτύπωσης της ακολουθίας των τμημάτων, εκτυπώνοντας τις ακολουθίες τους. Μια ακολουθία τμήματος εκτυπώνεται, εκτυπώνοντας τα ζεύγη που την αποτελούν. Ένα ζεύγος εκτυπώνεται, εκτυπώνοντας τη θέση του στην ακολουθία και τους μαθητές του. Κατά την εκτύπωση της ακολουθίας τμήματος, αν ο βαθμός αταξίας είναι κάτω από ένα ποσοστό των μαθητών του τμήματος, T_{quiet} , να εκτυπώνεται το μήνυμα "What a quiet class!". Αν είναι πάνω από ένα άλλο (μεγαλύτερο) ποσοστό, T_{messy} , να εκτυπώνεται το μήνυμα "What a mess!". Σε κάθε άλλη περίπτωση, να εκτυπώνεται το μήνυμα "Please, be quiet!".

Υλοποιήστε τα παραπάνω μέσω καταλλήλων κλάσεων, όπου χρειάζεται, οι οποίες σχετίζονται με λειτουργίες που επιτυγχάνουν τις περιγραφόμενες διαδικασίες.

Υλοποιήστε, επίσης, μια συνάρτηση `main` στην οποία:

1. Να δημιουργεί ένα σύνολο μαθητών που ανήκουν σε K τμήματα.
2. Να δημιουργεί ακολουθίες για τα τμήματα στα οποία ανήκουν.
3. Να δημιουργεί την ακολουθία των τμημάτων.
4. Να εκτυπώνει την ακολουθία αυτή.
5. Για L φορές, να επιλέγονται τυχαίου πλήθους τυχαία παιδιά να κάνουν αταξίες. Για κάθε παιδί, να εκτυπώνονται τα στοιχεία του, να γίνονται οι απαραίτητες ενέργειες και να εκτυπώνονται οι ακολουθίες που επηρεάστηκαν από τις ενέργειες αυτές.
6. Τέλος, να εκτυπώνεται η ακολουθία των τμημάτων, όπως αυτή θα έχει διαμορφωθεί.

Οι παράμετροι K και L καθώς και τα κατώφλια ποσοστών, T_{quiet} και T_{messy} , να δίνονται από τη γραμμή εντολής, με τη σειρά αυτή.

11.4.11 Να υλοποιηθεί σε C++ μια προσομοίωση εξυπηρέτησης πελατών σε μια τράπεζα. Η τράπεζα εξυπηρετεί τους πελάτες της μέσω των ταμείων της. Οι πελάτες δεν έχουν ξεχωριστή αναπαράσταση. Απλά, αντικατοπτρίζονται από τον αριθμό προτεραιότητας που παίρνουν κατά την είσοδό τους στην τράπεζα. Για την προσομοίωση, να χρησιμοποιηθούν οι κλάσεις "Τράπεζα" και "Ταμείο".

Η κλάση "Τράπεζα":

- Έχει έναν μετρητή που αυξάνει κάθε φορά που μπαίνει ένας πελάτης και η τιμή του αντιστοιχεί στον αριθμό προτεραιότητας του πελάτη αυτού (`last_customer`) — είναι, δηλαδή, ο αριθμός που εκτυπώνεται στο "χαρτάκι" που παίρνει ο πελάτης όταν μπαίνει.
- Έχει μια ένδειξη που αντιστοιχεί στον αριθμό προτεραιότητας του πελάτη που καλείται να εξυπηρετηθεί (`curr_serving`) — είναι ο αριθμός που εμφανίζεται, συνήθως, σε έναν φωτεινό πίνακα. Σημειώνεται ότι είναι πιθανόν, σε μια δεδομένη χρονική στιγμή, να μην υπάρχει πελάτης στην τράπεζα. Ωστόσο ο αριθμός `curr_serving` είναι ο αριθμός προτεραιότητας του πελάτη που θα κληθεί να εξυπηρετηθεί, έστω και αν αυτός δεν έχει έρθει ακόμα.
- Έχει πέντε ταμεία (`cashiers`).

Η κλάση "Τράπεζα" χαρακτηρίζεται από την εξής συμπεριφορά:

- Αρχικά ανοίγει το πρώτο ταμείο, η τράπεζα δεν έχει κανένα πελάτη και αρχικοποιεί την ένδειξη για τον αριθμό προτεραιότητας, `last_customer`, σε 1.

- Όταν εισέρχεται ένας νέος πελάτης (enter), ελέγχεται αν τα ανοικτά ταμεία επαρκούν για την εξυπηρέτηση των πελατών που περιμένουν. Αν ναι, αυξάνεται η τιμή του μετρητή των αριθμών προτεραιότητας. Αν όχι, ανοίγει το επόμενο κλειστό ταμείο και μετά αυξάνεται η τιμή του μετρητή των αριθμών προτεραιότητας. Αν δεν υπάρχει τέτοιο ταμείο, απλώς εκτυπώνεται το μήνυμα "Sorry you cannot enter until some customers are served!".
- Η εξυπηρέτηση ενός πελάτη (serve) αντιστοιχεί στην εξής διαδικασία.
Ελέγχεται μήπως όλα τα ταμεία είναι κλειστά, οπότε ανοίγει το πρώτο ταμείο και το επιλέγει. Αλλιώς, επιλέγεται το επόμενο ταμείο που είναι ανοικτό και ελεύθερο, έστω το i_{th} . Η επιλογή γίνεται κυκλικά. Δηλαδή, το i_{th} ταμείο είναι το επόμενο ανοικτό ταμείο από αυτό που εξυπηρέτησε την προηγούμενη φορά. Αν αυτό ήταν το τελευταίο, επιλέγεται το πρώτο ανοικτό. (Υπόδειξη: μήπως χρειάζεστε κάποιο επιπλέον μέλος-δεδομένο για την υλοποίηση αυτή;).
Κατόπιν, εκτυπώνονται τα εξής: το μήνυμα "Customer no:", ακολουθεί η τιμή της ένδειξης curr_serving, ακολουθεί το μήνυμα "by cashier no:" και τέλος εκτυπώνεται η τιμή i .
Στη συνέχεια, το επιλεγμένο ταμείο εξυπηρετεί τον πελάτη και κατόπιν ελευθερώνεται.
Στη συνέχεια πραγματοποιείται έξοδος πελάτη.
- Η έξοδος πελάτη (exit) αντιστοιχεί στην αύξηση της τιμής της ένδειξης για το ποιος αριθμός προτεραιότητας καλείται να εξυπηρετηθεί και γίνεται έλεγχος πλεονάσματος ταμείου. Αν τα ταμεία πλεονάζουν, κλείνει το πρώτο ανοικτό ταμείο — για να μη γίνεται "σπατάλη" ταμείων.
- Ο έλεγχος επάρκειας ταμείων (check_to_open) γίνεται ελέγχοντας αν η διαφορά των αριθμών (last_customer – curr_serving) είναι μεγαλύτερος από τον αριθμό των ανοικτών ταμείων επί έναν αριθμό K .
- Ο έλεγχος πλεονάσματος ταμείου (check_to_close) γίνεται ελέγχοντας αν (last_customer – curr_serving) είναι μικρότερος ή ίσος από τον αριθμό των ανοικτών ταμείων μειωμένο κατά ένα, επί τον αριθμό K .
- Ανοίγει ένα ταμείο (open), ανοίγοντας το ταμείο αυτό και εκτυπώνοντας ένα μήνυμα με το ποιο ταμείο άνοιξε.
- Κλείνει ένα ταμείο (close), κλείνοντας το ταμείο αυτό και εκτυπώνοντας ένα μήνυμα με το ποιο ταμείο έκλεισε.
- Ανακοινώνει τον αριθμό των πελατών που περιμένουν να εξυπηρετηθούν (waiting_customers), δηλαδή τη διαφορά των αριθμών (last_customer – curr_serving).
- Ανακοινώνει τον αριθμό των ανοικτών ταμείων (open_cashiers).

Η κλάση "Ταμείο":

- Έχει μια ένδειξη αν είναι ανοικτό ή κλειστό (open_flag).
- Έχει μια ένδειξη αν εξυπηρετεί πελάτη ή όχι (serving_flag).
- Έχει έναν μετρητή για το πόσους πελάτες έχει εξυπηρετήσει χωρίς διακοπή (customers_served).

Η κλάση "Ταμείο" χαρακτηρίζεται από την εξής συμπεριφορά:

- Κατά την κατασκευή του είναι κλειστό και άδειο.
- Ανοίγει (open), ενημερώνοντας τη σχετική ένδειξη και μηδενίζοντας τον μετρητή customers_served.
- Κλείνει (close), ενημερώνοντας τη σχετική ένδειξη.

- Εξυπηρετεί πελάτη (serve), ενημερώνοντας τη σχετική ένδειξη.
- Ελευθερώνεται (free), ενημερώνοντας τη σχετική ένδειξη και αυξάνοντας τον μετρητή customers_served κατά ένα. Αν η τιμή του γίνει ίση με μια σταθερά L , θεωρείται ότι το ταμείο υπερλειτούργησε, οπότε κλείνει και εκτυπώνεται σχετικό μήνυμα.
- Ελέγχεται αν το ταμείο είναι ανοικτό (is_open).
- Ελέγχεται αν το ταμείο είναι ελεύθερο (is_free).

Να υλοποιήσετε τις παραπάνω κλάσεις σε C++. Επίσης να υλοποιηθεί και μια συνάρτηση main που

1. Να δημιουργεί μια τράπεζα.
2. Για κάθε N πελάτες που εισέρχονται στην τράπεζα, αυτή να εξυπηρετεί όσους είναι δυνατόν, με βάση τα ταμεία που είναι ανοικτά (αυτό εξαρτάται από το πλήθος N των πελατών που μπήκαν). Η διαδικασία αυτή να επαναλαμβάνεται M φορές.
3. Στη συνέχεια να εξυπηρετούνται όλοι οι υπόλοιποι πελάτες που περιμένουν να εξυπηρετηθούν.

Τα K , M , N και L είναι παράμετροι της προσομοίωσης και να δίνονται κατά την κλήση του προγράμματος από τη γραμμή εντολής, με αυτήν τη σειρά.

11.5 Μικρές Εργασίες για Εξάσκηση στην Κληρονομικότητα

Οι ασκήσεις στο τμήμα αυτό έχουν ως στόχο να σας εξασκήσουν στον σχεδιασμό και τη συμπεριφορά της κληρονομικότητας μεταξύ κλάσεων, καθώς και στις εικονικές συναρτήσεις και τις αφηρημένες κλάσεις.

Εκφωνήσεις Εργασιών

11.5.1 Δίνεται η παρακάτω συνάρτηση:

```
int main()
{
    Bird Tweety(tweety_speed);
    Cat Silvester(sylvester_init_speed);

    Cartoon* cartoon[2];

    cartoon[0] = &Tweety;
    cartoon[1] = &Silvester;

    while(!felt(cartoon,2)) {
        cartoon[0]->first_move();
        cartoon[1]->first_move();
        cartoon[0]->second_move();
        cartoon[1]->second_move();
    }

    cout << "The End!" << endl;

    return 0;
}
```

Θα υλοποιήσετε σε C++ τη συμπεριφορά κινουμένων σχεδίων (class `Cartoon`) προκειμένου να μπορείτε να εκτελέσετε την παραπάνω συνάρτηση `main()`. Η συνάρτηση αυτή αντιστοιχεί σε μια σκηνή με τον γάτο `Silvester` και το πουλί `Tweety`. Στη σκηνή αυτή, ο `Tweety` μπαίνει σ' ένα δωμάτιο τρέχοντας και ο `Silvester` τον ακολουθεί. Πλησιάζοντας στον απέναντι τοίχο του δωματίου, ο `Tweety` κάνει αναστροφή. Ο `Silvester` κάνει το ίδιο. Η διαδικασία αυτή επαναλαμβάνεται ανάμεσα στους δυο απέναντι τοίχους (έστω ότι η πόρτα έκλεισε με το που μπήκαν), μέχρις ότου ο `Silvester` πέφτει στον τοίχο και η σκηνή τελειώνει.

Ο `Tweety` είναι ένα στιγμιότυπο της κλάσης `Bird` και ο `Silvester` είναι ένα στιγμιότυπο της κλάσης `Cat`. Τόσο η κλάση `Bird` όσο και η `Cat` είναι υποκλάσεις της κλάσης `Cartoon`.

Όλα τα κινούμενα σχέδια έχουν μια μέθοδο `first_move` και μια `second_move`. Η `first_move` αναφέρεται στο τρέξιμο και ουσιαστικά εκτελεί τη μέθοδο `run`. Η `second_move` αναφέρεται στην αναστροφή και ουσιαστικά εκτελεί τη μέθοδο `turn`. Όλα τα κινούμενα σχέδια έχουν ταχύτητα (`velocity`) και κατεύθυνση κίνησης (`direction`). Επίσης, έχουν μια κατάσταση (`felt`), που αντιστοιχεί στο αν έπεσαν σε τοίχο ή όχι. Αν η ταχύτητα ενός κινουμένου σχεδίου ξεπερνά μια σταθερά (`too_fast`), το κινούμενο σχέδιο πέφτει στον τοίχο. Αν ένα κινούμενο σχέδιο κινείται με τη φορά με την οποία μπήκε στο δωμάτιο, εκτυπώνεται στην οθόνη το μήνυμα "`Vrooommmmm...` ". Αν κινείται με την αντίθετη, εκτυπώνεται στην οθόνη το μήνυμα "`Fsssssss...` ".

Θεωρούμε ότι τα πουλιά τρέχουν πάντα με σταθερή ταχύτητα, ίση με την αρχική. Για τις γάτες ισχύει ότι κάθε φορά που κάνουν αναστροφή, η ταχύτητά τους αυξάνεται κατά μια σταθερά (`speed_up`). Κάθε φορά που ένα πουλί κάνει αναστροφή, εκτυπώνεται στην οθόνη το μήνυμα "`I tawt I taw a puddy tat!` ". Σε κάθε περίπτωση, αν η ταχύτητα του κινουμένου σχεδίου ξεπερνά την τιμή της `too_fast`, κατά την (προσπάθεια για) αναστροφή, το κινούμενο σχέδιο πέφτει στον τοίχο και εκτυπώνεται στην οθόνη το μήνυμα ("`Booommmmmmm!` ").

Τα κινούμενα σχέδια αρχικοποιούνται με παράμετρο την αρχική τους ταχύτητα με την οποία μπαίνουν στη σκηνή.

Υλοποιήστε στη C++ τις κλάσεις και την ιεραρχία τους που μοντελοποιούν την παραπάνω συμπεριφορά. Επίσης, συμπληρώστε και τις κατάλληλες καθολικές μεταβλητές και σταθερές καθώς και όποια άλλη συνάρτηση χρειάζεται.

- 11.5.2** Έστω η οντότητα "Εκπαιδευτικό Ίδρυμα". Ένα Εκπαιδευτικό Ίδρυμα αναπαριστάται από το όνομά του, τη διεύθυνσή του, τον αριθμό των ατόμων που φοιτούν σε αυτό, καθώς και από το σύνολο των διδασκόντων σε αυτό. Στην αναπαράσταση των εκπαιδευτικών ιδρυμάτων, εντάσσεται και ο διδάσκων εκείνος που είναι η Κεφαλή του ιδρύματος. Σε ένα Εκπαιδευτικό Ίδρυμα, μπορούμε να εκτυπώσουμε την αναλογία φοιτητών ανά διδάσκοντα εκτυπώνοντας τον λόγο του πλήθους των ατόμων που φοιτούν σε αυτό δια το πλήθος των διδασκόντων. Όλη η πληροφορία που απαιτείται για την αναπαράσταση πρέπει να ανατεθεί (ορθά) κατά τη δημιουργία ενός Εκπαιδευτικού Ιδρύματος. Ένας "Διδάσκων" αναπαριστάται από το ονοματεπώνυμό του και τον αριθμό ταυτότητάς του και υφίσταται, ανεξάρτητα από το Ίδρυμα.

Τα Εκπαιδευτικά Ιδρύματα διακρίνονται σε τρεις κατηγορίες: "Πρωτοβάθμια", "Δευτεροβάθμια", "Τριτοβάθμια".

Στα "Πρωτοβάθμια" Ιδρύματα, προστίθεται η έννοια της "Διδασκαλίας Τάξης", που αναπαριστάει το ποιος διδάσκων διδάσκει σε ποια τάξη (υποθέστε ότι υπάρχει ένα τμήμα ανά τάξη μόνο). Οι απαραίτητες πληροφορίες ανατίθενται κατά τη δημιουργία. Στα ιδρύματα αυτά, μπορούμε να ανακτήσουμε, μέσω της συμπεριφοράς τους, σε ποια τάξη διδάσκει ένας συγκεκριμένος διδάσκων, δίνοντας τον αριθμό της ταυτότητάς του. Η τάξη αναπαριστάται ως ακέραιος.

Στα "Δευτεροβάθμια" Ιδρύματα, προστίθεται η έννοια της "Διδασκαλίας Μαθήματος", που αναπαριστάει το ποιος διδάσκων διδάσκει ποιο μάθημα (υποθέστε ότι: 1) ένα μάθημα ενσωματώνει και την

τάξη, π.χ. “Ιστορία - Α’ Λυκείου”, 2) έχουμε πάλι ένα τμήμα ανά τάξη). Οι απαραίτητες πληροφορίες ανατίθενται κατά τη δημιουργία. Στα ιδρύματα αυτά μπορούμε να ανακτήσουμε, μέσω της συμπεριφοράς τους, ποιο μάθημα διδάσκει ένας συγκεκριμένος διδάσκων, δίνοντας τον αριθμό της ταυτότητάς του.

Στα “Τριτοβάθμια” Ιδρύματα, προστίθενται τα Τμήματα που αποτελούν ένα τέτοιο ίδρυμα (οσαδήποτε). Στα τριτοβάθμια ιδρύματα, προστίθεται η έννοια της “Διδασκαλίας Μαθήματος” που αναπαριστάνει το ποιος διδάσκων διδάσκει ποιο μάθημα. Η συμπεριφορά τους, πέρα από την ανάκτηση της πληροφορίας του μαθήματος που διδάσκει ένας συγκεκριμένος διδάσκων, επαναορίζει την έννοια της αναλογίας φοιτητών ανά διδάσκοντα, εκτυπώνοντας τις αντίστοιχες αναλογίες για κάθε τμήμα του ιδρύματος. Ένα “Τμήμα” είναι μια οντότητα που αναπαριστάνεται από το όνομά του, το πλήθος των φοιτητών του και το σύνολο των διδασκόντων του.

Υλοποιήστε τα παραπάνω μέσω των κατάλληλων κλάσεων και της μεταξύ τους ιεραρχίας, ορίζοντας τα μέλη-δεδομένα που χρειάζονται και τις συναρτήσεις-μέλη που υλοποιούν την παραπάνω συμπεριφορά. Επίσης, ορίστε τις κατάλληλες συναρτήσεις κατασκευής-καταστροφής και κατασκευής αντιγράφων καθώς και τον ορισμό του τελεστή ανάθεσης, όπου είναι απαραίτητο. Να ορίσετε εσείς, όπου κρίνεται αναγκαίο, συναρτήσεις ανάκτησης τιμών (accessors) για τις κλάσεις που θα υλοποιήσετε.

Για να αναδείξετε την υλοποίησή σας, να υλοποιήσετε μια συνάρτηση `main` η οποία:

1. Να δημιουργεί δυναμικά N διδάσκοντες, M_1 πρωτοβάθμια ιδρύματα, M_2 δευτεροβάθμια και M_3 τριτοβάθμια ιδρύματα.
2. Να εκτυπώνει την αναλογία διδασκομένων ανά διδάσκοντα για κάθε ίδρυμα.
3. Για κάθε διδάσκοντα, να ανακτά και να εκτυπώνει την πληροφορία για τη διδασκαλία του.
4. Να αναθέτει το πρώτο ίδρυμα, ανά κατηγορία, σε όλα τα υπόλοιπα της κατηγορίας του και να ξαναεκτυπώνει τα 2 και 3.
5. Να δημιουργεί ένα ίδρυμα ανά κατηγορία μέσω αντιγραφής, κατόπιν να διαγράφει τα αρχικά ιδρύματα και τέλος να εκτυπώνει τα 2 και 3 για τα νέα που ορίστηκαν δι’ αντιγραφής.

Σημείωση: Χωρίς να υλοποιήσετε αυτήν τη δυνατότητα, υποθέστε ότι οι διδάσκοντες ενός ιδρύματος (και ότι τους εμπλέκει) μπορεί να μεταβληθούν.

11.5.3 Υλοποιήστε σε C++ μια προσομοίωση μιας περιοχής από κτήρια. Ένα κτήριο αποτελείται από (οσοσδήποτε) ορόφους. Ένας όροφος αποτελείται από διαμερίσματα. Ένα διαμέρισμα αποτελείται από δωμάτια. Όλοι οι όροφοι έχουν τον ίδιο (οποιοδήποτε) αριθμό διαμερισμάτων. Όλα τα διαμερίσματα έχουν τον ίδιο (οποιοδήποτε) αριθμό δωματίων. Σε ένα κτήριο, ανά πάσα στιγμή βρίσκονται N επισκέπτες ($N \geq 0$). Ο κάθε όροφος φιλοξενεί κάποιους από αυτούς. Στον κάθε όροφο, οι επισκέπτες βρίσκονται μέσα στα διαμερίσματα του ορόφου. Οι επισκέπτες ενός διαμερίσματος βρίσκονται σε διάφορα δωμάτια του διαμερίσματος.

Κατά την κατασκευή ενός κτηρίου, προσδιορίζεται ο αριθμός ορόφων, ο αριθμός διαμερισμάτων ανά όροφο και ο αριθμός δωματίων ανά διαμέρισμα που περιέχει και εκτυπώνεται ένα μήνυμα ότι ένα κτήριο κατασκευάζεται. Κατά την κατασκευή ενός ορόφου, προσδιορίζεται ο αριθμός διαμερισμάτων και ο αριθμός δωματίων ανά διαμέρισμα που περιέχει. Κατά την κατασκευή ενός διαμερίσματος, προσδιορίζεται ο αριθμός δωματίων που περιέχει. Κατά την κατασκευή τους, τόσο το κτήριο, όσο και οι όροφοι, τα διαμερίσματα και τα δωμάτια δεν περιέχουν κανέναν επισκέπτη. Στην περίπτωση κατεδάφισης (καταστροφής) του κτηρίου, εκτυπώνεται σχετικό μήνυμα.

Το κτήριο δέχεται μια επίσκεψη (visit) σε ένα διαμέρισμα ενός ορόφου. Τόσο το διαμέρισμα, όσο και ο όροφος, προσδιορίζονται. Το πλήθος των επισκεπτών που βρίσκονται στο κτήριο αυξάνεται κατά ένα και εκτυπώνεται το μήνυμα "I am entering a building ...". Αντίστοιχα, ένας επισκέπτης

μπορεί να αποχωρήσει (exit) από το κτήριο, προσδιορίζοντας τον όροφο και το διαμέρισμά του από τα οποία θα αποχωρήσει. Σε αυτήν την περίπτωση, το πλήθος των ατόμων που βρίσκονται στο κτήριο μειώνεται κατά ένα και εκτυπώνεται το μήνυμα "I am exiting a building ...".

Η επίσκεψη σε έναν όροφο (visit), προσδιορίζοντας το διαμέρισμα, γίνεται αυξάνοντας το πλήθος των ατόμων που βρίσκονται στον όροφο και κατόπιν κάνοντας επίσκεψη στο διαμέρισμα. Η αποχώρηση από έναν όροφο (exit), δεδομένου ενός διαμερίσματος του ορόφου, γίνεται μειώνοντας το πλήθος των επισκεπτών που βρίσκονται στον όροφο και κατόπιν αποχωρώντας από το διαμέρισμα.

Η επίσκεψη σε ένα διαμέρισμα (visit) γίνεται αυξάνοντας το πλήθος των επισκεπτών που βρίσκονται στο διαμέρισμα και κατόπιν κάνοντας επίσκεψη σε ένα δωμάτιο του διαμερίσματος (τυχαίο). Η αποχώρηση από ένα διαμέρισμα (exit) γίνεται με μείωση του πλήθους των επισκεπτών που βρίσκονται στο διαμέρισμα και με αποχώρηση ενός επισκέπτη από ένα δωμάτιο (οποιοδήποτε, αρκεί να έχει κάποιον μέσα).

Η επίσκεψη σε ένα δωμάτιο (visit) γίνεται αυξάνοντας το πλήθος των επισκεπτών που βρίσκονται στο δωμάτιο. Η αποχώρηση από ένα δωμάτιο (exit) γίνεται μειώνοντας το πλήθος των επισκεπτών που βρίσκονται στο δωμάτιο.

Τα κτήρια διακρίνονται σε τέσσερις κατηγορίες: πολυκατοικίες, κτήρια γραφείων, σταθμούς αυτοκινήτων και πολυκαταστήματα.

Μια πολυκατοικία είναι ένα τυπικό κτήριο. Παρ' όλα αυτά, μια επίσκεψη σε πολυκατοικία, εκτός από τη διαδικασία επίσκεψης κτηρίου, επιπλέον εκτυπώνει και το μήνυμα "Home sweet home!".

Τα κτήρια γραφείων είναι τυπικά κτήρια. Παρ' όλα αυτά, μια αποχώρηση από κτήριο γραφείων, μετά από τη διαδικασία αποχώρησης κτηρίου, αντί του μηνύματος "I am exiting a building ...", εκτυπώνει το μήνυμα "What a day!". Για τα κτήρια γραφείων, ισχύει μια διαδικασία εκκένωσης του κτηρίου (empty), κάθε βράδυ, στην περίπτωση που υπάρχουν ακόμα σε αυτό επισκέπτες. Η διαδικασία αυτή ανάγεται στις απαραίτητες αναχωρήσεις (μέσω της exit).

Ένας σταθμός αυτοκινήτων είναι ένα κτήριο του οποίου ο κάθε όροφος αποτελείται από ένα διαμέρισμα με ένα δωμάτιο μόνο. Ένας σταθμός αυτοκινήτων έχει ένα ταμείο στο οποίο φυλάσσονται οι εισπράξεις από τις επισκέψεις (αυτοκινήτων). Κατά την επίσκεψη ενός σταθμού αυτοκινήτων, εκτυπώνεται το μήνυμα "A new car is entering ... " και αυξάνεται το ταμείο του σταθμού κατά 10 ευρώ.

Ένα πολυκατάστημα είναι ένα κτήριο του οποίου ο κάθε όροφος αποτελείται από ένα διαμέρισμα με ένα δωμάτιο μόνο. Ένα πολυκατάστημα έχει ένα ταμείο στο οποίο φυλάσσονται οι εισπράξεις από τις επισκέψεις. Κατά την επίσκεψη σε ένα πολυκατάστημα, ένας (τυχαίος) αριθμός, που αντιστοιχεί στο ποσό που διέθεσε ο επισκέπτης, προστίθεται στο ταμείο και εκτυπώνεται μαζί με το μήνυμα "Euros spent : ". Για τα πολυκαταστήματα, ισχύει μια διαδικασία εκκένωσης του κτηρίου (empty), κάθε βράδυ, στην περίπτωση που υπάρχουν ακόμα σε αυτό επισκέπτες. Η διαδικασία αυτή ανάγεται στις απαραίτητες αναχωρήσεις (μέσω της exit).

Υλοποιήστε τα παραπάνω μέσω των κατάλληλων κλάσεων και της μεταξύ τους ιεραρχίας, ορίζοντας τα μέλη-δεδομένα που χρειάζονται και τις συναρτήσεις-μέλη που υλοποιούν την παραπάνω συμπεριφορά, ορίζοντάς τις ως virtual, όπου είναι απαραίτητο. Να ορίσετε εσείς, όπου κρίνετε αναγκαίο, αναγνώστριες συναρτήσεις για τις κλάσεις που θα υλοποιήσετε.

Για να αναδείξετε την προσομοίωσή σας, να υλοποιήσετε μια συνάρτηση main η οποία να δημιουργεί ένα σύνολο κτηρίων (πολυκατοικίες, κτήρια γραφείων, σταθμούς αυτοκινήτων και πολυκαταστήματα) και, για M ημέρες, στο καθένα από τα κτήρια αυτά να πραγματοποιεί τυχαίο αριθμό επισκέψεων και τυχαίο αριθμό αποχωρήσεων. Για την κάθε ημέρα (επανάληψη), η διαδικασία των επισκέψεων να ταχειρίζεται μέσω δεικτών σε κτήρια και μόνο η διαδικασία εκκένωσης να χειρίζεται (δείκτες σε) σταθμούς αυτοκινήτων και πολυκαταστήματα.

11.5.4 Υλοποιήστε σε C++ την εξέλιξη μιας σκηνής κινουμένων σχεδίων (scene).

Τα κινούμενα σχέδια (cartoon) διακρίνονται σε αρουραίους (rat) και μάγειρες (cook). Κάθε κινούμενο σχέδιο έχει μια θέση στη σκηνή (position), που προσδιορίζεται από τις συντεταγμένες του στην οθόνη, x και y . Τα κινούμενα σχέδια έχουν μια πληροφορία σχετικά με το μέγεθος της εμφάνισής τους στην οθόνη (appearance). Αυτή μπορεί να πάρει τις τιμές *large* (= 3), *medium* (= 2), *small* (= 1) και *invisible* (= 0).

Αρχικά, τα κινούμενα σχέδια έχουν μέγεθος εμφάνισης *medium*, καταχωρείται η θέση τους στην οθόνη και εκτυπώνεται. Αρχικά, οι αρουραίοι εκτυπώνουν το μήνυμα "I am a rat!". Αρχικά, οι μάγειροι εκτυπώνουν το μήνυμα "I am a cook!". Κατά την καταστροφή τους, εκτυπώνεται το μήνυμα "Destroying cartoon located in " και η θέση τους.

Τα κινούμενα σχέδια εμφανίζονται στην οθόνη (print), λαμβάνοντας υπόψη το μέγεθος της εμφάνισής τους στην οθόνη με τον παρακάτω τρόπο.

Όταν τα κινούμενα σχέδια έχουν εμφάνιση στην οθόνη με τιμή *large*, επιτίθενται. Ένα κινούμενο σχέδιο επιτίθεται (attack), εκτυπώνοντας το μήνυμα "Attack!". Όταν τα κινούμενα σχέδια έχουν εμφάνιση στην οθόνη με τιμή *medium*, εκτυπώνεται το μήνυμα "I am in!".

Όταν οι αρουραίοι έχουν εμφάνιση στην οθόνη με τιμή *small*, καλούν για βοήθεια. Οι αρουραίοι καλούν για βοήθεια (call_for_help), εκτυπώνοντας το μήνυμα "I need help!". Όταν οι αρουραίοι έχουν εμφάνιση στην οθόνη με τιμή *invisible*, πανικοβάλλονται. Ένας αρουραίος πανικοβάλλεται (panic), εκτυπώνοντας το μήνυμα "Panic, panic, panic!" και προσπαθώντας να διαφύγει, αν είναι επιτρεπτό. Ένας αρουραίος διαφεύγει (exit), εκτυπώνοντας το μήνυμα "Exiting ASAP!". Η διαφυγή είναι επιτρεπτή εάν, προς κάποια κατεύθυνση, οριζόντια ή κάθετα, δεν υπάρχει μάγαιρας ή, αν υπάρχει, είναι κρυμμένος. Αν διαφύγει, ο αρουραίος καταστρέφεται.

Όταν οι μάγειροι έχουν εμφάνιση στην οθόνη με τιμή *invisible*, κρύβονται. Ένας μάγαιρας κρύβεται (hide), εκτυπώνοντας το μήνυμα "I am gonna hide!". Ένας μάγαιρας είναι κρυμμένος (is_hidden), αν η εμφάνισή του στην οθόνη έχει τιμή *invisible*.

Η εξέλιξη της σκηνής γίνεται ως εξής.

- Αρχικά, σε ένα πλέγμα $n \times m$ θέσεων, δημιουργούνται τυχαία κινούμενα σχέδια: kr αρουραίοι και kc μάγειροι. Δεν επιτρέπεται δυο κινούμενα σχέδια να έχουν κοινή θέση. Η σκηνή εμφανίζεται στην οθόνη, εμφανίζοντας τις συντεταγμένες κάθε θέσης και το περιεχόμενό της — το κινούμενο σχέδιο, αν υπάρχει.
- Κάθε κινούμενο σχέδιο μπορεί να αλλάξει εμφάνιση στην οθόνη για τον εξής λόγο: έστω ότι βρίσκεται στη θέση (i, j) . Ελέγχονται οι γειτονικές του θέσεις, δηλαδή οι θέσεις $(i - 1, j - 1)$, $(i - 1, j)$, $(i - 1, j + 1)$, $(i, j - 1)$, $(i, j + 1)$, $(i + 1, j - 1)$, $(i + 1, j)$ και $(i + 1, j + 1)$ — όσες από αυτές δεν βγαίνουν εκτός του πλέγματος $(1, 1) - (n, m)$. Για κάθε κινούμενο σχέδιο όμοιου είδους που βρίσκεται σε αυτές, αυξάνει την υποστήριξη (support) κατά την τιμή της εμφάνισης του σχεδίου αυτού στην οθόνη. Για κάθε κινούμενο σχέδιο του άλλου είδους που βρίσκεται σε αυτές, αυξάνει την τιμή της απειλής (threat) κατά την τιμή της εμφάνισης του σχεδίου αυτού στην οθόνη. Αν η τιμή της υποστήριξης είναι μεγαλύτερη από εκείνη της απειλής, η τιμή της εμφάνισης του στην οθόνη αυξάνεται κατά μία μονάδα. Αν η τιμή της απειλής είναι μεγαλύτερη από εκείνη της υποστήριξης, η τιμή της εμφάνισης του στην οθόνη μειώνεται κατά μία μονάδα. Οι τιμές της εμφάνισης δεν μπορούν να γίνουν μικρότερες από *invisible* ούτε μεγαλύτερες από *large*. Οι αλλαγές στην εμφάνιση του κινουμένου σχεδίου δεν γίνονται αμέσως. Εφαρμόζονται κατά την ενημέρωση της σκηνής, η οποία γίνεται αφού έχουν υπολογιστεί οι ενδεχόμενες αλλαγές όλων των κινουμένων σχεδίων στην τρέχουσα κατάσταση.
- Η σκηνή εμφανίζεται στην οθόνη, εμφανίζοντας τις συντεταγμένες κάθε θέσης και το περιεχόμενό της, αφού ενημερωθεί με τη νέα κατάσταση των κινουμένων σχεδίων της.

- Η διαδικασία αυτή συνεχίζεται, έως ότου να μην υπάρχουν αλλαγές στην κατάσταση της.

Να υλοποιήσετε σε C++ το παραπάνω. Για την υλοποίηση, χρησιμοποιήστε τις κλάσεις `cartoon`, `rat` και `cook`, όπως περιγράφηκαν παραπάνω καθώς και την κλάση `scene` της οποίας πιο αναλυτική περιγραφή είναι η εξής.

Η κλάση `scene` αποθηκεύει:

- Τις διαστάσεις του πλέγματος των θέσεων των κινουμένων σχεδίων.
- Το πλέγμα.

Η συμπεριφορά της είναι η εξής:

- Αρχικά δημιουργείται το πλέγμα των $n \times m$ θέσεων κινουμένων σχεδίων με κενές θέσεις. Επιλέγονται τυχαία kr θέσεις και δημιουργούνται σε αυτές αρουραίοι και kc θέσεις και δημιουργούνται σε αυτές μάγειροι. Δεν επιτρέπεται δυο κινούμενα σχέδια να έχουν κοινή θέση. Η σκηνή εμφανίζεται στην οθόνη.
- Εμφανίζεται στην οθόνη, εκτυπώνοντας τις συντεταγμένες κάθε θέσης και το περιεχόμενό της. Αν σε μια θέση βρίσκεται κινούμενο σχέδιο, πρώτα το ενημερώνει με την ενδεχόμενη αλλαγή που υπολογίστηκε στο μέγεθος της εμφάνισής του στην οθόνη. (`print`)
- Για κάθε θέση του πλέγματος, αν εκεί βρίσκεται κινούμενο σχέδιο, υπολογίζεται η ενδεχόμενη αλλαγή στο μέγεθος της εμφάνισής του στην οθόνη, σύμφωνα με τους παραπάνω κανόνες. (`evaluate`)

Επίσης, υλοποιήστε μια συνάρτηση `main` η οποία να υλοποιεί την εξέλιξη της σκηνής. Όταν η εξέλιξη φτάσει σε σταθερό σημείο — δεν υπάρχουν αλλαγές μετά την εφαρμογή του κύκλου `evaluate-print` —, αν το πλήθος των μαγείρων είναι μεγαλύτερο από εκείνο των αρουραίων, να εκτυπώνεται το μήνυμα "Let's go back to work!". Διαφορετικά, να εκτυπώνεται το μήνυμα "Anybody to help us!".

Οι τιμές n , m , kr και kc είναι παράμετροι της υλοποίησης και να δίνονται από τη γραμμή εντολής με αυτήν τη σειρά. Στη γραμμή εντολής να δίνεται και ένα επιπλέον προαιρετικό όρισμα που να αντιστοιχεί στο φύτρο της γεννήτριας τυχαίων αριθμών, προκειμένου να μην έχετε τυχαιότητα στα αποτελέσματα της εκτέλεσης. Αν δεν δοθεί η παράμετρος αυτή, ως φύτρο πάρτε την τρέχουσα χρονική στιγμή.

Για την υλοποίηση, μπορείτε να ορίσετε και επιπλέον τύπους δεδομένων και να επαυξήσετε τους παραπάνω με δεδομένα ή συναρτήσεις, όπου το χρειάζεστε. Για παράδειγμα, στην κλάση `cartoon`, θα χρειαστείτε μια συνάρτηση-μέλος για τον έλεγχο αν το κινούμενο σχέδιο είναι αρουραίος (`is_a_rat`). Στην κλάση `cartoon`, πάλι, πρέπει να προνοήσετε για κάποια θέση που να φυλάτε την ενδεχόμενη αλλαγή στο μέγεθος της εμφάνισής του κινούμενου σχεδίου στην οθόνη.

- 11.5.5** Υλοποιήστε σε C++ την προσομοίωση ενός `ferry boat`. Στο `ferry boat` επιβαίνουν επιβάτες, επιβατικά αυτοκίνητα και φορτηγά. Στα επιβατικά αυτοκίνητα επιβαίνουν το πολύ τέσσερις επιβάτες. Στα φορτηγά επιβαίνει ακριβώς ένας επιβάτης. Το `ferry boat` έχει έναν μέγιστο συνολικό αριθμό οχημάτων που μπορεί να μεταφέρει (`MaxVehicles`), έναν μέγιστο αριθμό επιβατικών αυτοκινήτων (`MaxCars`), έναν μέγιστο αριθμό φορτηγών (`MaxTrucks`) (ισχύει $MaxVehicles < MaxCars + MaxTrucks$) και έναν μέγιστο αριθμό επιβατών (`MaxPassengers`). Επίσης, το `ferry boat`, έχει ένα μέγιστο βάρος που μπορεί να μεταφέρει (`MaxWeight`). Κάθε επιβατικό αυτοκίνητο έχει ένα βάρος και κάθε φορτηγό αυτοκίνητο έχει ένα βάρος. Τόσο τα αυτοκίνητα, όσο και τα φορτηγά, αλλά και οι επιβάτες έχουν μια ταυτότητα.

Το `ferry boat` αρχικά είναι άδειο. Απλά εκτυπώνεται στην οθόνη το μήνυμα "Ready to board". Επίσης, ανατίθενται τα όριά του σε επιβάτες, οχήματα, επιβατικά αυτοκίνητα, φορτηγά και βάρος.

Στο `ferry boat` γίνεται επιβίβαση (`embark`) ενός επιβάτη, προσθέτοντάς τον στους επιβάτες, αυξάνοντας τον αριθμό των επιβιβασμένων επιβατών κατά ένα και επιβιβάζοντας τον επιβάτη. Στο `ferry boat`

γίνεται επιβίβαση (embark) ενός επιβατικού αυτοκινήτου, προσθέτοντας το όχημα στα επιβιβασμένα αυτοκίνητα, αυξάνοντας τον αριθμό των επιβιβασμένων επιβατικών αυτοκινήτων και των οχημάτων κατά ένα και το φορτωμένο βάρος κατά το βάρος του οχήματος. Επίσης, οι επιβάτες του αυτοκινήτου προστίθενται στους επιβάτες του ferry boat και ο αριθμός των επιβιβασμένων επιβατών του ferry boat αυξάνεται κατά το πλήθος τους. Κατόπιν, επιβιβάζεται το επιβατικό αυτοκίνητο. Στο ferry boat γίνεται επιβίβαση (embark) ενός φορτηγού, προσθέτοντας το φορτηγό στα επιβιβασμένα φορτηγά, αυξάνοντας τον αριθμό των επιβιβασμένων φορτηγών και των οχημάτων κατά ένα και το φορτωμένο βάρος κατά το βάρος του φορτηγού. Επίσης, ο επιβάτης του φορτηγού προστίθεται στους επιβάτες του ferry boat και ο αριθμός των επιβιβασμένων επιβατών του ferry boat αυξάνεται κατά ένα. Κατόπιν, επιβιβάζεται το φορτηγό.

Στο ferry boat γίνεται αποβίβαση (disembark) επιβατών, επιβατικών αυτοκινήτων και φορτηγών, ακολουθώντας αντίστροφες διαδικασίες από τις παραπάνω.

Στο ferry boat γίνεται έλεγχος ασφάλειας και αναχώρηση (depart), ελέγχοντας αν τα όρια τηρούνται. Εκτυπώνεται το μήνυμα "Sailing away. Safety check!". Αν ο επιτρεπτός αριθμός των επιβατών παραβιάζεται, εκτυπώνεται το μήνυμα "Too many passengers" και αποβιβάζεται από το ferry boat το πλεόνασμα των επιβατών. Πρώτα αποβιβάζονται από το ferry boat επιβάτες χωρίς όχημα. Αν κάποιος από τους επιβάτες που πρέπει να αποβιβαστεί επιβαίνει σε όχημα, αποβιβάζονται και οι υπόλοιποι επιβάτες και το όχημα από το ferry boat. Αν το πλήθος των οχημάτων παραβιάζεται, εκτυπώνεται το μήνυμα "Too many vehicles" και αποβιβάζονται — πρώτα επιβατικά και μετά φορτηγά — οχήματα από το ferry boat, μέχρι το όριο να τηρείται. Αν το βάρος παραβιάζεται, εκτυπώνεται το μήνυμα "Too heavy" και αποβιβάζονται — πρώτα φορτηγά και μετά επιβατικά — οχήματα από το ferry boat, μέχρι το όριο να τηρείται. Πριν από τον έλεγχο για το πλήθος οχημάτων, ανάλογοι έλεγχοι γίνονται για τα όρια των φορτηγών και των επιβατικών αυτοκινήτων, εκτυπώνοντας μηνύματα "Too many trucks" και "Too many cars", αντίστοιχα. (Σημείωση: όπως περιγράφεται παρακάτω στην αποβίβαση φορτηγών και αυτοκινήτων, όταν γίνονται αυτές, ακολουθεί και αποβίβαση των επιβατών τους.)

Ένας επιβάτης αρχικοποιείται, αναθέτοντας την ταυτότητά του και την πληροφορία για το αν επιβαίνει σε όχημα και ποιο, εκτυπώνοντάς την και εκτυπώνοντας το μήνυμα στην οθόνη "I intend to travel". Επιβιβάζεται (embark), εκτυπώνοντας το μήνυμα "I am in" και την ταυτότητά του. Αποβιβάζεται (disembark), εκτυπώνοντας το μήνυμα "I am out" και την ταυτότητά του.

Ένα επιβατικό αυτοκίνητο αρχικοποιείται, αναθέτοντας την ταυτότητά του, το βάρος του και δημιουργώντας από έναν έως τέσσερις επιβάτες. Εκτυπώνεται το μήνυμα "I want to embark" και η ταυτότητά του. Επιβιβάζεται (embark, εκτυπώνοντας το μήνυμα "Embarking at last..." και την ταυτότητά του και οι επιβάτες του επιβιβάζονται. Αποβιβάζεται (disembark), εκτυπώνοντας το μήνυμα "I can't believe we are out" και την ταυτότητά του και οι επιβάτες του αποβιβάζονται.

Ένα φορτηγό αρχικοποιείται, αναθέτοντας την ταυτότητά του, το βάρος του και δημιουργώντας έναν επιβάτη. Εκτυπώνεται το μήνυμα "Another day at work" και η ταυτότητά του. Επιβιβάζεται (embark), εκτυπώνοντας το μήνυμα "I'll take a nap..." και την ταυτότητά του και ο επιβάτης του επιβιβάζεται. Αποβιβάζεται (disembark), εκτυπώνοντας το μήνυμα "Bad day..." και την ταυτότητά του και ο επιβάτης του αποβιβάζεται.

Υλοποιήστε τα παραπάνω μέσω καταλλήλων κλάσεων, ορίζοντας μέλη-δεδομένα που χρειάζονται και συναρτήσεις-μέλη που υλοποιούν την παραπάνω συμπεριφορά. Κατόπιν, υλοποιήστε μια συνάρτηση main που να πραγματοποιεί τα παρακάτω:

1. Να δημιουργούνται τυχαίος αριθμός επιβατών, επιβατικών αυτοκινήτων και φορτηγών.
2. Να επιβιβάζονται στο ferry boat.
3. Να γίνεται ο έλεγχος ασφάλειας και αναχώρηση.

Τα *MaxVehicles*, *MaxCars*, *MaxTrucks*, *MaxPassengers* και *MaxWeight* να δίνονται από τη γραμμή εντολής, με τη σειρά αυτή.

- 11.5.6** Υλοποιήστε σε C++ την προσομοίωση ενός κέντρου διανομής Ταχυδρομείου. Το κέντρο έχει K διανομείς και σε κάθε χρονική στιγμή πρέπει να διανεμηθούν γράμματα και δέματα. Τα γράμματα μπορεί να είναι συστημένα ή επείγοντα. Τα επείγοντα πρέπει να ανατεθούν πρώτα για διανομή. Τα δέματα ενδεχόμενα χαρακτηρίζονται ως ογκώδη, οπότε αντί για το ίδιο το δέμα, διανέμεται ένα ειδοποιητήριο. Κάθε τι που διανέμεται χαρακτηρίζεται από μονάδες διανομής. Ένας διανομέας δεν πρέπει να ξεπερνά ένα μέγιστο μονάδων διανομής για να διανέμει, αλλά πρέπει και να έχει και κάποιο ελάχιστο. Αν για κάποιο λόγο, οι μονάδες διανομής του είναι κάτω από αυτό το ελάχιστο, η διαφορά προστίθεται στο μέγιστο την επόμενη φορά που του ανατίθεται διανομή. Κάθε απλό γράμμα ή κάθε ειδοποιητήριο έχει μία μονάδα διανομής. Ένα συστημένο γράμμα ή ένα επείγον γράμμα έχει δύο μονάδες διανομής. Ένα μη ογκώδες δέμα έχει τρεις μονάδες διανομής. Κάθε δέμα και κάθε γράμμα έχει μια διεύθυνση παραλήπτη. Αυτή αποτελείται από τον δρόμο και τον αριθμό.

Τα γράμματα και τα δέματα που πρέπει να διανεμηθούν, καθώς αφικνούνται στο ταχυδρομείο, ταξινομούνται με βάση τη διεύθυνση παραλήπτη. Τα επείγοντα διαχωρίζονται. Κατόπιν, ανατίθενται σε διανομείς, με βάση τους παραπάνω κανόνες. Αν μετά το τέλος της διανομής υπάρχουν γράμματα ή/και δέματα που δεν διανεμήθηκαν από τους διανομείς λόγω λήξης του ωραρίου τους, αυτά επιστρέφουν στο κέντρο για διανομή την επόμενη φορά.

Το κέντρο αρχικά δεν έχει τίποτα να διανείμει και έχει τους K διανομείς. Στο κέντρο προστίθεται ένα γράμμα για διανομή (*arrive*), τοποθετώντας το στην κατάλληλη θέση για διανομή. Στο κέντρο προστίθεται ένα δέμα για διανομή (*arrive*), τοποθετώντας το στην κατάλληλη θέση για διανομή. Στο κέντρο γίνεται ανάθεση των γραμμάτων και των δεμάτων διανομής στους διανομείς (*assign*). Στο κέντρο επιστρέφουν οι διανομείς, επιστρέφοντας ό,τι δεν διανεμήθηκε, για να διανεμηθεί την επόμενη φορά (*deliver*).

Ένας διανομέας αρχικά δεν έχει τίποτα να διανείμει και δεν έχει κανένα υπόλοιπο από μονάδες διανομής. Στον διανομέα ανατίθεται ένα σύνολο δεμάτων και γραμμάτων για διανομή, υπό τον όρο ότι δεν παραβιάζονται οι κανόνες μονάδων διανομής (*assign*). Ένας διανομέας επιστρέφει (*finish*), επιστρέφοντας ό,τι δεν διένειμε και ενημερώνοντας για το τυχόν υπόλοιπο μονάδων διανομής του.

Ένα γράμμα αρχικοποιείται, αναθέτοντας τη διεύθυνση παραλήπτη και το είδος του (απλό, επείγον, συστημένο).

Ένα δέμα αρχικοποιείται, αναθέτοντας τη διεύθυνση παραλήπτη και το είδος του (ογκώδες, μη ογκώδες).

Υλοποιήστε τα παραπάνω μέσω καταλλήλων κλάσεων, ορίζοντας μέλη-δεδομένα που χρειάζονται και συναρτήσεις-μέλη που υλοποιούν την παραπάνω συμπεριφορά. Κατόπιν, υλοποιήστε μια συνάρτηση *main* που να πραγματοποιεί τα παρακάτω:

1. Να δημιουργεί ένα κέντρο διανομής.
2. Για *Cycles* βήματα:
 - (α) Να δημιουργούνται τυχαίος αριθμός δεμάτων και γραμμάτων.
 - (β) Να προστίθενται για διανομή στο κέντρο.
 - (γ) Να γίνεται ανάθεση στους διανομείς του κέντρου.
 - (δ) Να γίνεται επιστροφή διανομέων στο κέντρο.

Τα K και *Cycles* δίνονται από τη γραμμή εντολής, καθώς και το μέγιστο και ελάχιστο μονάδων διανομής (*MaxUnits*, *MinUnits*) και με τη σειρά αυτή. Η δημιουργία δεμάτων και γραμμάτων να γίνεται με στοιχεία από τυχαίες επιλογές από τις επιτρεπτές τιμές.

11.5.7 Να υλοποιήσετε σε C++ μια προσομοίωση παγοδρομίου ψυχαγωγίας, το οποίο έχει ως εξής. Το σχήμα του είναι ορθογώνιο παραλληλόγραμμο διαστάσεων $M \times L$, που αναπαριστάνεται ως πλέγμα $(M + 1) \times (L + 1)$ σημείων. Γειτονικά σημεία του πλέγματος (προς κάθε κατεύθυνση ακόμα και διαγώνια) θεωρούμε ότι έχουν απόσταση ίση με 1.

Ανά πάσα στιγμή, σε κάθε σημείο του πλέγματος, μπορεί να υπάρχει το πολύ ένας παγοδρόμος. Αρχικά, υπάρχουν N παγοδρόμοι σε τυχαίες θέσεις του και δεν προστίθενται άλλοι παγοδρόμοι στη συνέχεια. Καθένας από αυτούς θεωρούμε ότι ήδη έχει παγοδρομήσει για κάποιο χρονικό διάστημα.

Κάθε παγοδρόμος έχει μια ταχύτητα ολίσθησης. Ως ταχύτητα ολίσθησης ορίζεται ο αριθμός των σημείων του πλέγματος που επισκέπτεται ο παγοδρόμος στη μονάδα του χρόνου. Οι παγοδρόμοι γλιστρούν στην πίστα, διανύοντας στη μονάδα του χρόνου ευθείες τροχιές, ο καθένας με τη δεδομένη ταχύτητά του. Επίσης, μπορούν να παραμείνουν στο παγοδρόμιο το πολύ για κάποιο χρόνο, διαφορετικό για τον καθένα. Όταν λήξει ο χρόνος του, ο παγοδρόμος αναχωρεί από το παγοδρόμιο (και εκτυπώνεται το μήνυμα "Bye bye..."). Ο κάθε παγοδρόμος έχει μια ταυτότητα.

Αν, κατά τη διαδικασία της ολίσθησης, δύο (ή περισσότεροι) παγοδρόμοι συμπέσουν στο ίδιο σημείο του πλέγματος, πέφτουν. Όταν ένας παγοδρόμος πέφτει, εκτυπώνεται το μήνυμα "I just fell but nice to meet you!". Γενικά, σύγκρουση μεταξύ παγοδρόμων συμβαίνει στις εξής περιπτώσεις: είτε όταν καταλήγουν οι παγοδρόμοι στο ίδιο σημείο, είτε όταν κινούνται στη ίδια κατεύθυνση και προσπερνά ο ένας τον άλλο. Μετά από τη σύγκρουση, ξαναξεκινούν να ολισθαίνουν από τυχαία θέση, είτε γειτονική του σημείου της σύγκρουσης, αν καταλήγουν στο ίδιο σημείο, είτε από ένα τυχαίο σημείο της διαδρομής που θα διάνυναν, αν προσπέρασε ο ένας τον άλλον.

Η κίνηση των παγοδρόμων στο παγοδρόμιο γίνεται με τη φορά των δεικτών του ρολογιού.

Υπάρχει η δυνατότητα παγοδρόμοι να κινούνται σε ζευγάρια. Τότε, οφείλουν να βρίσκονται, ο ένας δίπλα στον άλλο, σε διαδοχικές θέσεις πάντα, και η ταχύτητά τους είναι ίση με του βραδύτερου. Στρίβοντας, ο εσωτερικός παραμένει στη θέση που ήταν και ο άλλος μεταβαίνει ακαριαία στο γειτονικό σημείο που είναι το συμμετρικό ως προς τη διαγώνιο που ορίζεται από τη σταθερή θέση του εσωτερικού και την πλησιέστερη γωνία του παγοδρομίου. Επίσης, αν πέσει ο ένας παγοδρόμος, πέφτει και ο άλλος. Τέλος, αναχωρούν μαζί.

Οι παγοδρόμοι διακρίνονται σε αρχάριους, κοινούς, προχωρημένους και μικρά παιδιά.

Οι κοινόι παγοδρόμοι κινούνται μόνο παράλληλα με τις πλευρές του παγοδρομίου κατά τη φορά των δεικτών του ρολογιού και σε μία απόσταση από τα όρια του πλέγματος τυχαία, ίση με το πολύ 30% του μήκους της κάθετης πλευράς σε εκείνη που κινούνται. Την απόσταση αυτή διατηρούν σταθερή καθ' όλη την ολίσθηση στην κατεύθυνση που κινούνται. Οπότε, πριν φτάσουν στο άκρο της πλευράς αυτής, στρίβουν.

Οι αρχάριοι μπορούν να μεταβούν μόνο σε γειτονική θέση στη μονάδα του χρόνου (έχουν ταχύτητα 1), αλλά προς οποιαδήποτε επιτρεπτή κατεύθυνση. Επίσης, ανά πάσα στιγμή, μπορούν να βρίσκονται σε απόσταση από τα όρια του πλέγματος τυχαία, το πολύ ίση με το 10% του μήκους της κάθετης πλευράς σε εκείνη που κινούνται.

Τα μικρά παιδιά έχουν την ίδια συμπεριφορά με τους αρχάριους, εκτός του ότι επιπλέον το καθένα έχει έναν μέγιστο αριθμό πτώσεων που του επιτρέπεται και αν αυτό το φτάσει, το μικρό παιδί είναι υποχρεωμένο να αναχωρήσει. Ένα μικρό παιδί, όταν αναχωρεί από το παγοδρόμιο, εκτυπώνεται το μήνυμα "I want to stay longer...". Όταν ένα μικρό παιδί πέφτει, εκτυπώνεται το μήνυμα "Crying...".

Οι προχωρημένοι παγοδρόμοι μπορούν να κινηθούν σε οποιαδήποτε περιοχή μέσα στο πλέγμα προς οποιαδήποτε επιτρεπτή κατεύθυνση. Θεωρείται ότι για μια χρονική μονάδα, κινούνται σε σταθερή κατεύθυνση και φορά. Επίσης, δεν συγκρούονται (αν συμπέσουν σε σημείο του πλέγματος με άλλον παγοδρόμο ή τον προσπεράσουν, θεωρούμε ότι δεν πέφτουν και εκτυπώνεται το μήνυμα "I just

overcame the situation!"). Ο άλλος παγοδρόμος πέφτει όμως — εκτός αν είναι και αυτός προχωρημένος —.

Για κάθε μονάδα του χρόνου, το πλέγμα ενημερώνεται με βάση τις μετακινήσεις που έγιναν και εκτυπώνονται στην οθόνη οι συντεταγμένες στις οποίες υπάρχει παγοδρόμος και η ταυτότητά του.

Να υλοποιήσετε μια συνάρτηση *main*, η οποία να δημιουργεί ένα παγοδρόμο με N παγοδρόμους και να το λειτουργεί για T χρονικές μονάδες.

Τα M , L , N και T να είναι παράμετροι και να δίνονται κατά την κλήση του προγράμματος από τη γραμμή εντολής με τη σειρά αυτή.

Υπόδειξη: Ενδείκνυται να αναπαραστήσετε ρητά ένα σημείο του πλέγματος, καταχωρώντας την πληροφορία αν στο σημείο υπάρχει παγοδρόμος, περιλαμβάνοντας (μια ένδειξη προς) τον παγοδρόμο που ενδεχόμενα βρίσκεται εκεί.

Σημείωση: Λεπτομέρειες στην παραπάνω περιγραφή που δεν διασαφηνίζονται είτε δεν παρουσιάζουν ενδιαφέρον για τη σκοπιμότητα της άσκησης, είτε αποτελούν θέμα για προσωπικό προβληματισμό, ζητούμενο της άσκησης. Η διάκριση πρέπει να γίνεται εύκολα αντιληπτή.

- 11.5.8** Θεωρήστε έναν κόσμο σωματιδίων που απλουστεύει τα χαρακτηριστικά και τη συμπεριφορά τους σε σχέση με την επιστημονική εκδοχή τους από τη Φυσική Σωματιδίων. Συγκεκριμένα, θεωρήστε ότι ισχύουν τα παρακάτω.

Κάθε σωματίδιο (*particle*) έχει μάζα (*mass*), ιδιοστροφορμή (*spin*) και ηλεκτρικό φορτίο (*electric charge*). Αν δεν υπάρχει ένδειξη για κάτι διαφορετικό, αυτά έχουν τιμή 0. Ανάλογα με τη στατιστική (*statistics*) που ακολουθούν, διακρίνονται σε φερμιόνια (*fermions*) και μποζόνια (*bosons*). Τα φερμιόνια ακολουθούν τη στατιστική Fermi-Dirac. Τα μποζόνια ακολουθούν τη στατιστική Bose-Einstein.

Ένα πρωτόνιο (*proton*) είναι φερμιόνιο και έχει μάζα (*mass*) ίση με $938.27208 \text{ MeV}/c^2$, ιδιοστροφορμή (*spin*) ίση με $\frac{1}{2}$ και ηλεκτρικό φορτίο (*electric charge*) ίσο με $+1$. Όταν δημιουργείται ένα πρωτόνιο, του ανατίθεται η τιμή της μάζας του, η ιδιοστροφορμή του και το ηλεκτρικό φορτίο του. Επίσης, εκτυπώνεται το μήνυμα "A proton has been constructed" και η στατιστική στην οποία υπακούει. Όταν καταστρέφεται ένα πρωτόνιο, εκτυπώνεται το μήνυμα "A proton will be destroyed".

Ένα νετρόνιο (*neutron*) είναι φερμιόνιο και έχει μάζα (*mass*) ίση με $939.56541 \text{ MeV}/c^2$ και ιδιοστροφορμή (*spin*) ίση με $\frac{1}{2}$. Όταν δημιουργείται ένα νετρόνιο, του ανατίθεται η τιμή της μάζας του, η ιδιοστροφορμή του και το ηλεκτρικό φορτίο του. Επίσης, εκτυπώνεται το μήνυμα "A neutron has been constructed" και η στατιστική στην οποία υπακούει. Όταν καταστρέφεται ένα νετρόνιο, εκτυπώνεται το μήνυμα "A neutron will be destroyed".

Ένα ηλεκτρόνιο (*electron*) είναι φερμιόνιο και έχει ηλεκτρικό φορτίο (*electric charge*) ίσο με -1 , μάζα (*mass*) ίση με $0.510998946 \text{ MeV}/c^2$ και ιδιοστροφορμή (*spin*) ίση με $\frac{1}{2}$. Όταν δημιουργείται ένα ηλεκτρόνιο, του ανατίθεται η τιμή της μάζας του, η ιδιοστροφορμή του και το ηλεκτρικό φορτίο του. Επίσης, εκτυπώνεται το μήνυμα "An electron has been constructed" και η στατιστική στην οποία υπακούει. Όταν καταστρέφεται ένα ηλεκτρόνιο, εκτυπώνεται το μήνυμα "An electron will be destroyed".

Ένα ποζιτρόνιο (*positron*) είναι φερμιόνιο και έχει ηλεκτρικό φορτίο (*electric charge*) ίσο με $+1$, μάζα (*mass*) ίση με $0.510998946 \text{ MeV}/c^2$ και ιδιοστροφορμή (*spin*) ίση με $\frac{1}{2}$. Όταν δημιουργείται ένα ποζιτρόνιο, του ανατίθεται η τιμή της μάζας του, η ιδιοστροφορμή του και το ηλεκτρικό φορτίο του. Επίσης, εκτυπώνεται το μήνυμα "A positron has been constructed" και η στατιστική στην οποία υπακούει. Όταν καταστρέφεται ένα ποζιτρόνιο, εκτυπώνεται το μήνυμα "A positron will be destroyed".

Ένα νετρίνο (neutrino) είναι φερμιόνιο και έχει μάζα (*mass*) ίση με $0.320 \text{ eV}/c^2$ και ιδιοστροφορμή (*spin*) ίση με $\frac{1}{2}$. Όταν δημιουργείται ένα νετρίνο, του ανατίθεται η τιμή της μάζας του, η ιδιοστροφορμή του και το ηλεκτρικό φορτίο του. Επίσης, εκτυπώνεται το μήνυμα "A neutrino has been constructed" και η στατιστική στην οποία υπακούει. Όταν καταστρέφεται ένα νετρίνο, εκτυπώνεται το μήνυμα "A neutrino will be destroyed".

Το αντineutrino (antineutrino) έχει όλα τα χαρακτηριστικά ίδια με το νετρίνο. Όταν δημιουργείται ένα αντineutrino, του ανατίθεται η τιμή της μάζας του, η ιδιοστροφορμή του και το ηλεκτρικό φορτίο του. Επίσης, εκτυπώνεται το μήνυμα "An antineutrino has been constructed" και η στατιστική στην οποία υπακούει. Όταν καταστρέφεται ένα αντineutrino, εκτυπώνεται το μήνυμα "An antineutrino will be destroyed".

Ένα φωτόνιο (photon) είναι μποζόνιο και έχει ιδιοστροφορμή (*spin*) ίση με 1. Όταν δημιουργείται ένα φωτόνιο, του ανατίθεται η τιμή της μάζας του, η ιδιοστροφορμή του και το ηλεκτρικό φορτίο του. Επίσης, εκτυπώνεται το μήνυμα "A photon has been constructed" και η στατιστική στην οποία υπακούει. Όταν καταστρέφεται ένα φωτόνιο, εκτυπώνεται το μήνυμα "A photon will be destroyed".

Μεταξύ των παραπάνω σωματιδίων λαμβάνουν χώρα οι εξής αλληλεπιδράσεις (interactions):

- Από νετρίνιο και νετρίνο, προκύπτουν πρωτόνιο και ηλεκτρόνιο.
- Από πρωτόνιο και αντineutrino, προκύπτουν νετρίνιο και ποζιτρόνιο.
- Από πρωτόνιο και ηλεκτρόνιο, προκύπτουν νετρίνιο και νετρίνο.
- Ένα ηλεκτρόνιο αλληλοαναιρείται με ένα ποζιτρόνιο, παράγοντας δύο φωτόνια.
- Από δύο φωτόνια, μπορεί να παραχθούν ένα ηλεκτρόνιο και ένα ποζιτρόνιο.

Επίσης, μπορεί να συμβούν και οι παρακάτω διασπάσεις (decays):

- Ένα νετρίνιο διασπάται, παράγοντας ένα πρωτόνιο, ένα ηλεκτρόνιο και ένα αντineutrino.
- Ένα πρωτόνιο διασπάται, παράγοντας ένα νετρίνιο, ένα ποζιτρόνιο και ένα νετρίνο.

Υλοποιήστε τα παραπάνω σε C++, μέσω των κατάλληλων κλάσεων, ορίζοντας τα μέλη-δεδομένα που χρειάζονται και τις συναρτήσεις-μέλη που υλοποιούν την παραπάνω συμπεριφορά. Στην παραπάνω περιγραφή μπορείτε να διακρίνετε ότι μπορείτε να χρησιμοποιήσετε κληρονομικότητα, οπότε να το εκμεταλλευτείτε.

Υλοποιήστε επίσης μια συνάρτηση *main* η οποία:

1. Να δημιουργεί *M* αριθμό σωματιδίων, τα οποία να προσπελαύνονται μέσω μιας ακολουθίας *M* ενδείξεων, επιλέγοντας τυχαία το είδος τους.
2. Να εκτυπώνεται το πλήθος από κάθε είδος σωματιδίων που δημιουργήθηκε, καθώς και το συνολικό φορτίο που προκύπτει από τα σωματίδια.
3. Για *N* επαναλήψεις, να επιλέγει τυχαία ένα σωματίδιο και αν αυτό μπορεί να αλληλεπιδράσει με κάποιο από τα γειτονικά του, να γίνεται η αλληλεπίδραση, αλλιώς αν μπορεί να διασπαστεί, να διασπάται. Ως "γειτονικά" θεωρούνται σωματίδια που προσπελαύνονται μέσω διαδοχικών ενδείξεων.
4. Να εκτυπώνεται το πλήθος από κάθε είδος σωματιδίων που τελικά υπάρχουν, καθώς και πόση ποσοστιαία μεταβολή υπήρξε σε σχέση με το αρχικό πλήθος τους. Επίσης, αν το πλήθος των φερμιονίων είναι μεγαλύτερο από εκείνο των μποζονίων, να εκτυπώνεται το μήνυμα "Living in a material world!", διαφορετικά, αν ισχύει το αντίστροφο, να εκτυπώνεται το μήνυμα "Radiation prevails!". Διαφορετικά να εκτυπώνεται το μήνυμα "Equilibrium?".

Ακόμα, να επιβεβαιώνεται ότι το συνολικό φορτίο των σωματιδίων παραμένει σταθερό και να εκτυπώνεται το μήνυμα "Conservation verified!". Τέλος, να υπολογίζεται και να εκτυπώνεται η διαφορά στο άθροισμα της μάζας των σωματιδίων, καθώς και η διαφορά στο άθροισμα των ιδιοστροφορμών και στην περίπτωση που κάποια από αυτές είναι διαφορετική από μηδέν, να εκτυπώνεται το μήνυμα "How it comes!".

Τα M και N να δίνονται από τη γραμμή εντολής.

11.5.9 Υλοποιήστε σε C++ την προσομοίωση ενός κτηρίου δημόσιας υπηρεσίας. Το κτήριο αποτελείται από το ισόγειο, ένα ασανσέρ και άλλους τέσσερις ορόφους. Το ισόγειο αποτελείται από έναν χώρο εισόδου. Ο κάθε όροφος αποτελείται από έναν χώρο εισόδου και δέκα γραφεία.

Το κτήριο έχει μια μέγιστη χωρητικότητα επισκεπτών, N . Ο κάθε όροφος έχει χωρητικότητα επισκεπτών N_f ($< N/3$), ίδια για όλους τους ορόφους. Το ισόγειο έχει μια χωρητικότητα N_g ($< N/2$). Το κάθε γραφείο έχει μια χωρητικότητα N_o ($< N_f$) επισκεπτών και έναν αριθμό. Το ασανσέρ έχει μια χωρητικότητα N_l ($> N_o$) επισκεπτών. Καθένας από τους παραπάνω χώρους φυλάσσει και το πλήθος των επισκεπτών που βρίσκονται ήδη μέσα σε αυτόν.

Ο κάθε επισκέπτης αναπαριστάται από τον όροφο και τον αριθμό του γραφείου που πρέπει να επισκεφθεί. Ένας επισκέπτης μπαίνει στο κτήριο, αν υπάρχει διαθέσιμος χώρος. Διαφορετικά, η ενέργεια αποτυγχάνει και εκτυπώνεται το μήνυμα "Please, come tomorrow". Αν η ενέργεια επιτύχει, ο επισκέπτης μπαίνει στο ισόγειο και περιμένει σε αυτό για να μπει στο ασανσέρ.

Ένας επισκέπτης μπαίνει στο ασανσέρ, αν υπάρχει διαθέσιμος χώρος, εκτυπώνοντας το μήνυμα "Visitor in the lift" και ο αριθμός προτεραιότητάς του. Διαφορετικά, η ενέργεια αποτυγχάνει και εκτυπώνεται το μήνυμα "You are not allowed to enter!" και ο αριθμός προτεραιότητας του επισκέπτη. Ο επισκέπτης παραμένει στον χώρο εισόδου, έως ότου ξανάρθει το ασανσέρ.

Όταν για κάποιο επισκέπτη γίνει αποτυχία επιβίβασης, το ασανσέρ συνεχίζει τη λειτουργία του. Ανεβαίνοντας, το ασανσέρ σταματά σε κάθε όροφο και αποβιβάζει επισκέπτες που επιθυμούν να επισκεφθούν τον όροφο αυτό, με σειρά προτεραιότητας.

Ένας επισκέπτης μπορεί να μπει στον όροφο που επιθυμεί, στην περίπτωση που δεν υπερβαίνεται η μέγιστη χωρητικότητα του ορόφου. Αν έχει εξαντληθεί η χωρητικότητα, εκτυπώνεται το μήνυμα "Sorry, floor number $\langle floor_number \rangle$ is full" και ο αριθμός προτεραιότητας του επισκέπτη. Ο επισκέπτης παραμένει στο ασανσέρ και το ασανσέρ συνεχίζει τη λειτουργία του. Αν ένας επισκέπτης αποβιβαστεί από το ασανσέρ, αποπειράται να μπει στο γραφείο που επιθυμεί — και πάλι αν του το επιτρέπει η μέγιστη χωρητικότητα του γραφείου. Διαφορετικά, η ενέργεια αποτυγχάνει, εκτυπώνεται το μήνυμα "Please, wait outside for entrance in office number $\langle office_number \rangle$ " και ο αριθμός προτεραιότητας του επισκέπτη. Κατόπιν, ο επισκέπτης εισέρχεται στον χώρο εισόδου του ορόφου. Αν οι ενέργειες εισόδου στο γραφείο ή/και στον όροφο επιτύχουν, εκτυπώνεται το μήνυμα "Entering floor number $\langle floor_number \rangle$ " (αντίστοιχα $\langle office_number \rangle$) και ο αριθμός προτεραιότητας του επισκέπτη. Σε κάθε απόπειρα εισόδου (στο ασανσέρ, σε όροφο ή σε γραφείο), η επιλογή επισκέπτη γίνεται με βάση τον αριθμό προτεραιότητας.

Το ασανσέρ, σε έναν κύκλο λειτουργίας του, γεμίζει από επισκέπτες στο ισόγειο και ανεβαίνει, αποβιβάζοντας σε κάθε όροφο τους επισκέπτες που θέλουν (και επιτρέπεται) να αποβιβαστούν στον όροφο αυτό (οι άλλοι παραμένουν στο ασανσέρ). Κατόπιν, κατεβαίνει, επιβιβάζοντας επισκέπτες από κάθε όροφο που θεωρείται ότι έχουν εξυπηρετηθεί. Οι επισκέπτες που θεωρούμε ότι έχουν εξυπηρετηθεί επιλέγονται τυχαία από επισκέπτες που είναι ήδη μέσα σε γραφεία του ορόφου. Ο αριθμός των επισκεπτών που επιλέγονται πρέπει να είναι τέτοιος ώστε να μην ξεπερνά τον διαθέσιμο χώρο στο ασανσέρ εκείνη τη στιγμή (οπότε δεν υπάρχει περίπτωση να ξαναμπούν στον χώρο εισόδου του ορόφου). Κατόπιν, ανάλογα με τον αριθμό των επισκεπτών που εξυπηρετήθηκαν από τα γραφεία του ορόφου και με βάση τον αριθμό προτεραιότητάς τους, επισκέπτες που περιμένουν στον χώρο εισόδου του ορόφου

μπαίνουν στα γραφεία που επιθυμούν. Φτάνοντας στο ισόγειο, το ασανσέρ αδειάζει από επισκέπτες που έχουν εξυπηρετηθεί. Οι επισκέπτες που είναι ήδη στο ασανσέρ αλλά δεν εξυπηρετήθηκαν παραμένουν στο ασανσέρ. Στη συνέχεια, ο κύκλος λειτουργίας του ασανσέρ επαναλαμβάνεται.

Το κτήριο αρχικά δεν έχει κανέναν επισκέπτη. Όταν δημιουργείται, του ανατίθεται η χωρητικότητά του και εκτυπώνεται το μήνυμα "A new building is ready for serving citizens!". Όταν δημιουργείται ένας όροφος (και το ισόγειο), είναι κενός και του ανατίθεται η χωρητικότητά του και ο αριθμός του. Επίσης, εκτυπώνεται το μήνυμα "A Floor has been created with number $\langle floor_number \rangle$ ". Ανάλογα, δημιουργούνται το ασανσέρ και τα γραφεία με μηνύματα "A lift has been created" και "Office number $\langle office_number \rangle$ has been created". Ο χώρος εισόδου δεν έχει μέγιστη χωρητικότητα και όταν δημιουργείται, εκτυπώνεται το μήνυμα "The Entrance has been created!".

Όταν καταστρέφεται ένα γραφείο, εκτυπώνεται το μήνυμα "End of the work!". Όταν καταστρέφεται ένας χώρος εισόδου, εκτυπώνεται το μήνυμα "End of waiting people!". Όταν καταστρέφεται ένα ασανσέρ, εκτυπώνεται το μήνυμα "No more ups and downs!". Όταν καταστρέφεται ένας όροφος (και το ισόγειο), εκτυπώνεται το μήνυμα "End of service!". Όταν καταστρέφεται ένα κτήριο, εκτυπώνεται το μήνυμα "Service not available any longer. Go elsewhere!".

Σε ένα κτήριο, μπαίνει ένας επισκέπτης (enter) και ακολουθείται η διαδικασία που περιγράφηκε πιο πάνω. Σε ένα κτήριο, βγαίνει ένας επισκέπτης (exit), μειώνοντας τον αριθμό των επισκεπτών κατά ένα και εκτυπώνεται το μήνυμα "I cannot believe I finished!" και ο αριθμός προτεραιότητάς του.

Στο ισόγειο μπαίνει ένας επισκέπτης (enter), αυξάνοντας τον αριθμό των επισκεπτών του και μπαίνοντας στον χώρο εισόδου του. Στο ισόγειο περιμένει ένας επισκέπτης στον χώρο εισόδου (wait), παίρνοντας αριθμό προτεραιότητας και εκτυπώνοντας το μήνυμα "Waiting for the lift".

Σε έναν χώρο εισόδου, μπαίνει ένας επισκέπτης (enter) και βγαίνει ένας επισκέπτης (exit). Ο μετρητής επισκεπτών του χώρου εισόδου αυξάνεται και μειώνεται, αντίστοιχα.

Σε έναν όροφο μπαίνει ένας επισκέπτης (enter) και βγαίνει ένας επισκέπτης (exit) που βγήκε από ένα γραφείο. Ο μετρητής επισκεπτών του ορόφου αυξάνεται και μειώνεται, αντίστοιχα.

Σε ένα γραφείο μπαίνει ένας επισκέπτης (enter) και βγαίνει ένας επισκέπτης (exit) που/και θεωρήθηκε ότι έχει εξυπηρετηθεί. Ο μετρητής επισκεπτών του γραφείου αυξάνεται και μειώνεται, αντίστοιχα.

Ένας κύκλος λειτουργίας του ασανσέρ (operate) έχει ως εξής. Αρχικά, μπαίνουν επισκέπτες από το ισόγειο (enter), αυξάνοντας και τον μετρητή των επιβιβασμένων επισκεπτών. Κατόπιν, σταματά σε καθένα από τους τέσσερις ορόφους (stop_up) ανεβαίνοντας. Στη συνέχεια, σταματά σε καθένα από τους τέσσερις ορόφους κατεβαίνοντας (stop_down). Τέλος, αδειάζει στο ισόγειο (empty_all). Όταν μπαίνει ένας επισκέπτης στο ασανσέρ, αυξάνεται ο αριθμός των επιβατών του κατά 1. Όταν σταματά σε όροφο ανεβαίνοντας, αποβιβάζει τους επισκέπτες που θέλουν (και μπορούν) να αποβιβαστούν (exit), μειώνοντας κατάλληλα τον αριθμό των επιβατών του. Όταν σταματά σε όροφο κατεβαίνοντας, οι επισκέπτες που έχουν επιλεγεί (και θεωρούνται ότι εξυπηρετήθηκαν) μπαίνουν στο ασανσέρ. Αδειάζοντας στο ισόγειο, οι επιβάτες που εξυπηρετήθηκαν βγαίνουν και από το κτήριο. Αυτοί που δεν εξυπηρετήθηκαν παραμένουν στο ασανσέρ.

Υλοποιήστε τα παραπάνω μέσω των κατάλληλων κλάσεων, ορίζοντας τα μέλη-δεδομένα που χρειάζονται και τις συναρτήσεις-μέλη που υλοποιούν την παραπάνω συμπεριφορά. Υλοποιήστε επίσης μια συνάρτηση main στην οποία:

1. Να δημιουργείται ένα κτήριο με χωρητικότητα επισκεπτών N και N_f, N_g, N_o και N_l χωρητικότητα ορόφου, ισόγειου, γραφείων και ασανσέρ, αντίστοιχα.
2. Να δημιουργούνται K επισκέπτες, με τυχαίες επιλογές ορόφων και γραφείων.

3. Να γίνεται απόπειρα να μπουν οι επισκέπτες στο κτήριο.
4. Να εκτελούνται L κύκλοι λειτουργίας ασανσέρ.

Τα N , N_f , N_g , N_o , N_l , K και L να δίνονται από τη γραμμή εντολής, με τη σειρά αυτή.

11.5.10 Ως συνέχεια της Άσκησης 11.5.9, θεωρήστε ότι για τους επισκέπτες, επιπλέον της περιγραφής που ήδη έχει δοθεί, ισχύουν και τα παρακάτω. Όλοι οι επισκέπτες έχουν και έναν βαθμό υπομονής και έναν βαθμό θυμού. Επίσης, κάθε επισκέπτης μπορεί να επηρεάσει έναν άλλο (influence) και να επηρεαστεί από έναν άλλο θετικά (get_positive_influence) ή αρνητικά (get_negative_influence).

Πέρα από τη γενική κατηγορία επισκεπτών, διακρίνουμε ακόμα δύο είδη επισκεπτών: τους οξύθυμους επισκέπτες και τους υπομονετικούς επισκέπτες.

Ένας επισκέπτης που ανήκει στη γενική κατηγορία, όταν πρόκειται να επηρεάσει κάποιον άλλο, δεν συμβαίνει τίποτα. Όταν επηρεαστεί από έναν άλλο θετικά, αυξάνεται ο βαθμός υπομονής του κατά 1, όταν επηρεαστεί από έναν άλλο αρνητικά, αυξάνεται ο βαθμός θυμού του κατά 1. Όταν ο βαθμός υπομονής ξεπεράσει κατά πολύ (παράμετρος δική σας) τον βαθμό θυμού, η αρνητική επιρροή δεν έχει καμία επίδραση. Αντίστοιχα, όταν ο βαθμός θυμού ξεπεράσει κατά πολύ (κατά την ίδια παράμετρο) τον βαθμό υπομονής, η θετική επιρροή δεν έχει καμία επίδραση.

Ένας οξύθυμος επισκέπτης, όταν πρόκειται να επηρεάσει κάποιον άλλο, τον επηρεάζει αρνητικά. Όταν επηρεάζεται θετικά από κάποιον άλλο, αυξάνεται ο βαθμός της υπομονής του κατά 1. Όταν επηρεάζεται αρνητικά από κάποιον άλλο, αυξάνεται ο βαθμός θυμού του κατά 2 και μεταδίδει την αρνητική επιρροή στον επόμενο επισκέπτη (κατά σειρά προτεραιότητας) που βρίσκεται στον ίδιο χώρο με αυτόν.

Ένας υπομονετικός επισκέπτης, όταν πρόκειται να επηρεάσει κάποιον άλλο, τον επηρεάζει θετικά. Όταν επηρεάζεται αρνητικά από κάποιον άλλο, εκτυπώνεται το μήνυμα "You won't get me upset!". Όταν επηρεάζεται θετικά από κάποιον άλλο, αυξάνεται ο βαθμός υπομονής του κατά 2 και μεταδίδει τη θετική επιρροή στον επόμενο επισκέπτη (κατά σειρά προτεραιότητας) που βρίσκεται στον ίδιο χώρο με αυτόν.

Ο κύκλος λειτουργίας του ασανσέρ (operate) τροποποιείται με τον εξής τρόπο. Στο κλείσιμο του κύκλου, όπως πριν, αδειάζοντας στο ισόγειο, οι επιβάτες που εξυπηρετήθηκαν βγαίνουν και από το κτήριο και αυτοί που δεν εξυπηρετήθηκαν παραμένουν στο ασανσέρ. Επιπλέον, μπαίνουν επισκέπτες που περιμένουν στο ισόγειο (enter) και μπορούν να επιβιβαστούν. Τέλος, για τα γραφεία και τους χώρους εισόδου, να επιλέγεται τυχαία ένας επισκέπτης και να επηρεάζει τους άμεσα επόμενους του (κατά σειρά προτεραιότητας). Οπότε, για κάθε όροφο, αν ο συνολικός βαθμός υπομονής των επισκεπτών του ξεπερνά κατά πολύ (παράμετρος δική σας) τον συνολικό βαθμό θυμού τους, να εκτυπώνεται το μήνυμα "Peace and serenity in floor number : $\langle floor_number \rangle$ ". Αν, αντίθετα, ο συνολικός βαθμός θυμού των επισκεπτών του ξεπερνά κατά πολύ (η ίδια παράμετρος) τον συνολικό βαθμό υπομονής τους, να εκτυπώνεται το μήνυμα "What a mess in floor number : $\langle floor_number \rangle$ ".

Επίσης, τροποποιείται η απόπειρα εισόδου ενός επισκέπτη στο κτήριο: αν αποτύχει η ενέργεια εισόδου, ο επισκέπτης μπαίνει σε μια λίστα αναμονής με τυχαία σειρά, ώστε να ξαναπροσπαθήσει να μπει αργότερα.

Τέλος, η main να τροποποιηθεί ως εξής:

1. Να δημιουργείται ένα κτήριο με χωρητικότητα επισκεπτών N και N_f , N_g , N_o και N_l χωρητικότητα ορόφου, ισόγειου, γραφείων και ασανσέρ, αντίστοιχα.
2. Να δημιουργούνται K επισκέπτες, με τυχαίες επιλογές ορόφων και γραφείων.
3. Να γίνονται L επαναλήψεις των παρακάτω:

- (α) Να γίνεται απόπειρα να μπουν οι επισκέπτες (που δεν είχαν ήδη μπει) στο κτήριο.
- (β) Να γίνεται ένας κύκλος λειτουργίας του ασανσέρ.

Τα N, N_f, N_g, N_o, N_l, K και L να δίνονται από τη γραμμή εντολής, με τη σειρά αυτή.

11.5.11 Υλοποιήστε σε C++ μια προσομοίωση της πρωινής άφιξης μαθητών σε ένα σχολικό κτήριο και εισόδου τους στις αίθουσες (διδασκαλίας).

Το σχολικό κτήριο αποτελείται από τρεις ορόφους, ένα προαύλιο και ένα κλιμακοστάσιο. Κάθε όροφος αποτελείται από έξι αίθουσες και έναν διάδρομο. Κάθε αίθουσα έχει το πολύ C_{class} μαθητές και έναν δάσκαλο.

Τα στοιχεία ενός μαθητή είναι το όνομά του, τα στοιχεία του τμήματός του (αριθμός ορόφου και αριθμός αίθουσας), μια ένδειξη για το εάν βρίσκεται στην αίθουσά του ή όχι και ένας μετρητής που αντιστοιχεί στον βαθμό κόρασής του.

Τα στοιχεία του δασκάλου είναι το όνομά του, τα στοιχεία του τμήματός του, μια ένδειξη εάν βρίσκεται στην αίθουσά του ή όχι και ένας μετρητής που αντιστοιχεί στον βαθμό κόρασής του.

Οι μαθητές διακρίνονται σε δύο κατηγορίες: τους μαθητές “μικρών τάξεων” (junior) και τους μαθητές “μεγάλων τάξεων” (senior). Οι μαθητές παρακολουθούν μάθημα (attend) για κάποια χρονική διάρκεια (αριθμό ωρών). Για κάθε ώρα παρακολούθησης, ο βαθμός κόρασης των μαθητών των μικρών τάξεων αυξάνεται κατά L_j μονάδες. Αντίστοιχα, ο βαθμός κόρασης των μαθητών των μεγάλων τάξεων αυξάνεται κατά L_s μονάδες.

Οι δάσκαλοι διδάσκουν (teach) για έναν αριθμό ωρών και για κάθε ώρα διδασκαλίας, ο βαθμός κόρασής τους αυξάνεται κατά L_t μονάδες.

Μαθητές μπαίνουν στο σχολικό κτήριο, ένας-ένας. Ένας μαθητής μπαίνει στο σχολικό κτήριο (enter), εκτυπώνοντας το όνομά του συνοδευόμενο από το μήνυμα "... enters school!" και μπαίνοντας στο προαύλιο. Στη συνέχεια, βγαίνει από το προαύλιο και μπαίνει στο κλιμακοστάσιο. Στη συνέχεια βγαίνει από αυτό και μπαίνει στον όροφο του τμήματός του.

Η είσοδος μαθητή στο προαύλιο (enter) γίνεται εκτυπώνοντας το όνομά του συνοδευόμενο από το μήνυμα "... enters schoolyard!". Ένας μαθητής βγαίνει από το προαύλιο (exit), εκτυπώνοντας το όνομά του, συνοδευόμενο από το μήνυμα "... exits schoolyard!" και γίνεται διαθέσιμος στο περιβάλλον που προκάλεσε την έξοδο.

Η είσοδος μαθητή στο κλιμακοστάσιο (enter) γίνεται εκτυπώνοντας το όνομά του συνοδευόμενο από το μήνυμα "... enters stairs!". Ένας μαθητής βγαίνει από το κλιμακοστάσιο (exit), εκτυπώνοντας το όνομά του, συνοδευόμενο από το μήνυμα "... exits stairs!" και γίνεται διαθέσιμος στο περιβάλλον που προκάλεσε την έξοδο.

Ένας μαθητής μπαίνει σε όροφο (enter), εκτυπώνοντας το όνομά του, συνοδευόμενο από το μήνυμα "... enters floor!" και μπαίνοντας στον διάδρομο του ορόφου. Στη συνέχεια, βγαίνει από τον διάδρομο και μπαίνει στην αίθουσα του τμήματός του.

Ένας μαθητής μπαίνει στον διάδρομο (enter), εκτυπώνοντας το όνομά του, συνοδευόμενο από το μήνυμα "... enters corridor!". Ένας μαθητής βγαίνει από τον διάδρομο (exit), εκτυπώνοντας το όνομά του, συνοδευόμενο από το μήνυμα "... exits corridor!" και γίνεται διαθέσιμος στο περιβάλλον που προκάλεσε την έξοδο.

Ένας μαθητής μπαίνει στην αίθουσα του τμήματός του (enter), εκτυπώνοντας το όνομά του και το μήνυμα "... enters classroom!" και προστίθεται στους μαθητές της αίθουσας αυτής (δεν γίνεται έλεγχος αν υπάρχει ήδη δάσκαλος στην αίθουσα αυτή). Πλέον, η σχετική ένδειξη δηλώνει ότι βρίσκεται εντός του τμήματός του.

Ένας δάσκαλος θεωρούμε ότι τοποθετείται (place) στο σχολικό κτήριο. Αυτό γίνεται μέσω τοποθέτησής του (place) στον όροφο που ανήκει το τμήμα του. Το τελευταίο πραγματοποιείται τοποθετώντας τον στην αίθουσα του τμήματός του (place). Πλέον, η σχετική ένδειξη του δασκάλου δηλώνει ότι βρίσκεται εντός του τμήματός του.

Τόσο το σχολικό κτήριο, όσο και οι όροφοι, αλλά και οι αίθουσες, θεωρούμε ότι λειτουργούν για κάποια χρονική διάρκεια (αριθμό ωρών). Το σχολικό κτήριο λειτουργεί (operate) κάποιες ώρες, λειτουργώντας τους ορόφους του τις ώρες αυτές. Ένας όροφος λειτουργεί (operate) κάποιες ώρες, λειτουργώντας τις αίθουσές του τις ώρες αυτές. Όταν μια αίθουσα λειτουργεί (operate) κάποιες ώρες, οι μαθητές της παρακολουθούν μάθημα τις ώρες αυτές και ο δάσκαλός της διδάσκει τις ώρες αυτές.

Μπορούμε να κάνουμε εκτύπωση του σχολικού κτηρίου (print) εκτυπώνοντας το μήνυμα "School life consists of: " και εκτυπώνοντας τους ορόφους. Εκτυπώνουμε έναν όροφο (print), εκτυπώνοντας το μήνυμα "Floor number <floor_number> contains: " και εκτυπώνοντας τις αίθουσες που βρίσκονται σε αυτόν. Εκτυπώνουμε μια αίθουσα (print), εκτυπώνοντας το μήνυμα "People in class <class_number> are: " και εκτυπώνοντας τους μαθητές και τον δάσκαλο που βρίσκονται σε αυτή. Εκτυπώνουμε έναν μαθητή (print), εκτυπώνοντας το όνομά του και την τιμή του βαθμού κούρασής του. Τα ίδια στοιχεία εκτυπώνονται όταν εκτυπώνουμε έναν δάσκαλο (print), απλά εκτυπώνεται πριν και το μήνυμα "The teacher is: ".

Όταν δημιουργείται ένα σχολικό κτήριο, εκτυπώνεται το μήνυμα "A New School has been created!". Αντίστοιχα μηνύματα εκτυπώνονται όταν δημιουργούνται το προαύλιο, το κλιμακοστάσιο, οι όροφοι, οι διάδρομοι και οι αίθουσες. Οι αίθουσες, αρχικά, είναι κενές.

Όταν καταστρέφεται το σχολικό κτήριο, εκτυπώνεται το μήνυμα "A School to be destroyed!". Αντίστοιχα μηνύματα εκτυπώνονται όταν καταστρέφονται το προαύλιο, το κλιμακοστάσιο, οι όροφοι, οι διάδρομοι και οι αίθουσες.

Όταν δημιουργείται ένας νέος μαθητής, αρχικοποιείται με τα στοιχεία του. Αρχικά είναι εκτός αίθουσας και ο μετρητής κούρασής του έχει τιμή μηδέν. Επίσης, εκτυπώνεται το μήνυμα "A New Student has been created!", το όνομά του και τα στοιχεία του τμήματός του. Όταν καταστρέφεται ένας μαθητής, εκτυπώνεται το μήνυμα "A Student to be destroyed!" και τα στοιχεία του.

Όταν δημιουργείται ένας νέος δάσκαλος, αρχικοποιείται με τα στοιχεία του. Αρχικά είναι εκτός αίθουσας και ο μετρητής κούρασής του έχει τιμή μηδέν. Επίσης, εκτυπώνεται το μήνυμα "A New Teacher has been created!", το όνομά του και τα στοιχεία του τμήματός του. Όταν καταστρέφεται ένας δάσκαλος, εκτυπώνεται το μήνυμα "A Teacher to be destroyed!" και τα στοιχεία του.

Από την παραπάνω περιγραφή, θα παρατηρήσατε ότι πολλές οντότητες του κόσμου αυτού έχουν κοινά χαρακτηριστικά/λειτουργικότητα. Για παράδειγμα, υπάρχει μια γενικότερη (και αφηρημένη) έννοια "χώρος", στον οποίο μπαίνουν και βγαίνουν μαθητές. Επίσης, τόσο οι δάσκαλοι, όσο και κάθε είδος μαθητών, είναι εξειδικεύσεις "ανθρώπων", με διαφοροποιήσεις μεταξύ τους στη λειτουργικότητα.

Με δεδομένη την παραπάνω περιγραφή, υλοποιήστε τις κατάλληλες κλάσεις για τον κόσμο που περιγράφεται, εκμεταλλευόμενοι κληρονομικότητα μεταξύ κλάσεων, ορίζοντας κατάλληλες κλάσεις, όπου είναι απαραίτητο. Επίσης, υλοποιήστε μια συνάρτηση main στην οποία:

1. Να δημιουργείται ένα σχολικό κτήριο.
2. Να δημιουργούνται οι μαθητές και οι δάσκαλοί του.
3. Να μπαίνουν μαθητές και να τοποθετούνται δάσκαλοι στο σχολικό κτήριο με τυχαία σειρά.
4. Το σχολικό κτήριο να λειτουργεί για N ώρες.
5. Να εκτυπώνεται το σχολικό κτήριο.

Οι παράμετροι C_{class} , L_j , L_s , L_t και N να δίνονται από τη γραμμή εντολής, με τη σειρά αυτή.

Σημείωση: Θεωρήστε ότι σε κάθε όροφο, τρεις αίθουσες στεγάζουν τμήματα μεγάλων τάξεων και τρεις στεγάζουν τμήματα μικρών τάξεων. Ο μόνος λόγος που γίνεται η διάκριση αυτή είναι για να γίνει η κατάλληλη συσχέτιση μαθητών με αίθουσες κατά τη δημιουργία μαθητών. Δεν υπάρχει διαφοροποίηση όσον αφορά τις αίθουσες καθαυτές.

11.5.12 Έχουμε μια κοινότητα από πλάσματα (`creature`): καλά πλάσματα (`good_creature`) και κακά πλάσματα (`bad_creature`). Τα πλάσματα ζουν το ένα πλάι στο άλλο, σε συγκεκριμένη θέση το καθένα.

Τα πλάσματα έχουν τα εξής χαρακτηριστικά.

- Κάθε πλάσμα έχει ένα όνομα.
- Κάθε πλάσμα έχει μια διάρκεια ζωής που αναμένεται να ζήσει. Αρχικά, όταν γεννιέται είναι L , με $L > 0$ (το L είναι κοινό για όλα τα πλάσματα).
- Όταν η ζωή ενός πλάσματος μηδενιστεί, το πλάσμα πέφτει στην κατάσταση ζόμπι. Στα πλάσματα που βρίσκονται στην κατάσταση ζόμπι, οι παρακάτω ενέργειες δεν έχουν καμιά επίδραση.
- Ένα πλάσμα μπορεί να κλωνοποιηθεί (`clone`) στη θέση ενός άλλου πλάσματος. Σε αυτήν την περίπτωση, το άλλο πλάσμα καταστρέφεται και τη θέση του παίρνει ένα νέο πλάσμα που δημιουργείται ως αντίγραφο του πρώτου.
- Κάθε πλάσμα μπορεί είτε να ευλογηθεί (`bless`), είτε να χτυπηθεί (`beat`). Στην πρώτη περίπτωση, ο χρόνος ζωής του αυξάνεται κατά μια μονάδα. Στη δεύτερη περίπτωση, ελαττώνεται κατά μια μονάδα.

Τα καλά πλάσματα, όταν ευλογηθούν και είναι αρκετά εύρωστα, κλωνοποιούνται, καταλαμβάνοντας την επόμενη θέση στην κοινότητα (η επόμενη θέση της τελευταίας είναι η πρώτη). Θεωρούνται “εύρωστα”, αν η τρέχουσα ζωή τους είναι μεγαλύτερη από ένα κατώφλι (`good_thrsh`), που είναι κοινό για όλα τα καλά πλάσματα.

Τα κακά πλάσματα, όταν ευλογηθούν και είναι αρκετά εύρωστα, κλωνοποιούνται, καταλαμβάνοντας όλες τις συνεχόμενες επόμενες τους θέσεις στην κοινότητα που είναι κατειλημμένες από πλάσματα που βρίσκονται σε κατάσταση ζόμπι (εκτός αν το κακό πλάσμα που ευλογείται βρίσκεται στο τέλος). Θεωρούνται “εύρωστα”, αν η τρέχουσα ζωή τους είναι μεγαλύτερη από ένα κατώφλι (`bad_thrsh`), που είναι κοινό για όλα τα κακά πλάσματα.

Κάθε στιγμή επιλέγεται τυχαία ένα πλάσμα και σ’ αυτό εφαρμόζεται τυχαία μια ενέργεια, ευλογείται ή χτυπιέται. Η διαδικασία αυτή επαναλαμβάνεται M φορές. Είναι ενδεχόμενο το ίδιο πλάσμα να επιλεγεί πάνω από μια φορά.

Υλοποιήστε σε C++ το παραπάνω. Για την υλοποίηση χρησιμοποιήστε τις εξής κλάσεις.

`creature`: Η κλάση αυτή αποθηκεύει:

- Το όνομα του πλάσματος.
- Την αναμενόμενη ζωή του πλάσματος.

Η συμπεριφορά της χαρακτηρίζεται από το ότι:

- Αρχικά, η ζωή του είναι ίση με L , δεν είναι σε κατάσταση ζόμπι, ανήκει σε κάποια θέση σε κάποια κοινότητα και παίρνει το όνομά του.
- Ελέγχεται αν είναι ζόμπι ή όχι, ελέγχοντας αν η ζωή του είναι ίση με 0 (`is_a_zombie`).
- Ελέγχεται αν είναι καλό πλάσμα ή όχι (`is_a_good`), μεταθέτοντας τον έλεγχο στις παράγωγες κλάσεις.

- Κλωνοποιείται, αντικαθιστώντας ένα άλλο πλάσμα, καταστρέφοντάς το και δημιουργώντας ένα αντίγραφό του, για να καταλάβει τη θέση του στην κοινότητα.
- Χτυπιέται, μειώνοντας τη ζωή του κατά μια μονάδα, εάν δεν είναι σε κατάσταση ζόμπι.
- Ευλογείται, αυξάνοντας τη ζωή του κατά μια μονάδα, εάν δεν είναι σε κατάσταση ζόμπι.

`good_creature`: Η κλάση αυτή εξειδικεύει την κλάση `creature` με τα εξής:

- Όταν ευλογείται, επιπλέον από τη συμπεριφορά που επιδεικνύει ως πλάσμα, αν είναι εύρωστο, κλωνοποιείται στην επόμενη του θέση.
- Ο έλεγχος `is_a_good` επιστρέφει `true`.

`bad_creature`: Η κλάση αυτή εξειδικεύει την κλάση `creature` με τα εξής:

- Όταν ευλογείται, επιπλέον από τη συμπεριφορά που επιδεικνύει ως πλάσμα, αν είναι εύρωστο, κλωνοποιείται σε όλες τις επόμενες θέσεις που καταλαμβάνονται από πλάσματα σε κατάσταση ζόμπι και είναι συνεχόμενες από τη δική του (εκτός αν είναι στο τέλος).
- Ο έλεγχος `is_a_good` επιστρέφει `false`.

`creature_society`: Η κλάση αυτή αποθηκεύει:

- Το πλήθος των θέσεων πλασμάτων που έχει.
- Μια ένδειξη στην αρχική θέση της (ακολουθίας της) κοινότητας.

Η συμπεριφορά της είναι:

- Αρχικά, δημιουργούνται N θέσεις πλασμάτων, οι οποίες καταλαμβάνονται από N πλάσματα που δημιουργούνται, επιλέγοντας τυχαία αν θα δημιουργηθεί καλό πλάσμα ή κακό πλάσμα.
- Χτυπιέται η θέση i , χτυπώντας το πλάσμα που βρίσκεται στη θέση αυτή.
- Ευλογείται η θέση i , ευλογώντας το πλάσμα που βρίσκεται στη θέση αυτή.
- Κλωνοποιείται το πλάσμα που βρίσκεται στη θέση i , στην επόμενη του (`clone_next`).
- Κλωνοποιείται το πλάσμα που βρίσκεται στη θέση i , σε όλες τις συνεχόμενες θέσεις από τη θέση i και έπειτα που καταλαμβάνονται από πλάσματα που βρίσκονται σε κατάσταση ζόμπι, — εκτός αν το i αντιστοιχεί στην τελευταία θέση — (`clone_zombies`).
- Επιστρέφεται ο αριθμός των καλών πλασμάτων της κοινότητας (`no_of_good`).
- Επιστρέφεται ο αριθμός των πλασμάτων της κοινότητας που είναι σε κατάσταση ζόμπι (`no_of_zombies`).

Να υλοποιηθεί και μια συνάρτηση `main` η οποία:

- Αρχικά να δημιουργεί μια κοινότητα με N πλάσματα.
- Στη συνέχεια, να επιλέγεται τυχαία μια θέση και να επιλέγεται τυχαία μια ενέργεια (ευλογία ή χτύπημα) να εφαρμοστεί στο πλάσμα που βρίσκεται στη θέση αυτή. Η διαδικασία αυτή να επαναλαμβάνεται M φορές.
- Κατόπιν, αν η κοινότητα αποτελείται μόνο από καλά πλάσματα, να εκτυπώνεται το μήνυμα "Good Dominates in the World!". Αν η κοινότητα αποτελείται από πλάσματα σε κατάσταση ζόμπι, να εκτυπώνεται το μήνυμα "This is a dead society". Διαφορετικά, να εκτυπώνεται το μήνυμα "Try again to improve the world".

Γενικές Παρατηρήσεις:

Τα N , M , L , `good_thrsh` και `bad_thrsh` να δίνονται από τη γραμμή εντολής κατά την κλήση του προγράμματος, με αυτήν τη σειρά.

Από τις συναρτήσεις `bless`, `beat`, `clone`, καθώς και από τις συναρτήσεις κατασκευής και από τις συναρτήσεις καταστροφής, να εκτυπώνονται σχετικά μηνύματα.

Τα ονόματα των πλασμάτων να ανακτώνται από έναν πίνακα ονομάτων για καλά πλάσματα και έναν πίνακα ονομάτων για κακά πλάσματα, τα οποία γίνονται μοναδικά, προσαρτώντας τον αριθμό που αντιστοιχεί στη θέση που καταλαμβάνει το πλάσμα στην κοινότητα.

Για την υλοποίηση, μπορείτε να ορίσετε και επιπλέον τύπους δεδομένων και να επαυξήσετε τους παραπάνω με δεδομένα ή συναρτήσεις, όπου το χρειάζεστε. Για παράδειγμα, θα πρέπει να αποφασίσετε πώς θα συνδέσετε τα στιγμιότυπα της κλάσης `creature` με την κλάση `creature_society` στην οποία ανήκουν. Ανάλογα με την προσέγγισή σας, πρέπει να ορίσετε κατάλληλες συναρτήσεις-μέλη και μέλη-δεδομένα. Εναλλακτικά (αλλά όχι δεσμευτικά) για το παραπάνω μπορεί να είναι:

- Η κοινότητα περιέχει (ένδειξη σε) μια ακολουθία των “θέσεων” που περιέχει. Μια θέση ορίζεται ως δόμηση δεδομένων που περιέχει έναν δείκτη στο πλάσμα που την κατοικεί, έναν δείκτη στην κοινότητα που ανήκει και την τάξη (`index`) της θέσης στην ακολουθία της κοινότητας. Κάθε πλάσμα έχει έναν δείκτη στη θέση που το περιέχει και μέσω αυτού πρόσβαση στην κοινότητα και την τάξη της θέσης του.
- Η κοινότητα περιέχει μια ακολουθία δεικτών σε πλάσματα και τα πλάσματα έναν δείκτη στην κοινότητα που ανήκουν. Τότε, η τάξη της θέσης ενός πλάσματος προκύπτει διατρέχοντας τα στοιχεία της ακολουθίας (έως ότου εντοπίσουμε δείκτη στο πλάσμα που εξετάζουμε).

11.6 Μεγαλύτερες Εργασίες

Τα προβλήματα αυτά είναι εργασίες ανάπτυξης κώδικα μεγαλύτερης κλίμακας και προτείνεται να εκπονηθούν από ομάδες δύο ατόμων. Σκοπός τους είναι, εκτός από την εξάσκηση στην εκμετάλλευση αντικειμενοστραφών χαρακτηριστικών, να αναδειχθεί η διαδικασία ανάπτυξης κώδικα με κατανομή τμημάτων σε διαφορετικούς προγραμματιστές. Εναλλακτικά, μπορεί να γίνει η ανάπτυξη από τον ίδιο προγραμματιστή, με κατανομή της σε διαφορετικά χρονικά διαστήματα. Σε κάθε περίπτωση, είναι κρίσιμο να καθοριστούν τα τμήματα και οι διεπαφές τους. Στη συνέχεια, η ανάπτυξη κάθε τμήματος να γίνει ανεξάρτητα από τα άλλα, διασφαλίζοντας την τήρηση των προδιαγραφών των διεπαφών, με στόχο την επιτυχή τελική ενοποίηση.

Εκφωνήσεις Εργασιών

11.6.1 Ας υποθέσουμε ότι οι φοιτητές που έχουν κωδικούς στο δίκτυο των υπολογιστών ενός πανεπιστημιακού τμήματος εντάσσονται στις εξής κατηγορίες:

1. Προπτυχιακοί φοιτητές.
2. Μεταπτυχιακοί φοιτητές επιπέδου Master’s (μεταπτυχιακοί).
3. Υποψήφιοι διδάκτορες.

Κατά την εγγραφή ενός φοιτητή στο τμήμα, δημιουργείται και ένας κωδικός στο δίκτυο που αποτελεί τον “φοιτητικό” του κωδικό. Για τον κάθε φοιτητή καταγράφεται:

1. Ο αριθμός μητρώου του (4ψήφιος ακέραιος - μοναδικός σε όλους τους φοιτητές).
2. Το ονοματεπώνυμό του (συμβολοσειρά).
3. Το έτος της εγγραφής του (4ψήφιος ακέραιος).
4. Η ημερομηνία της αποφοίτησής του, η οποία θα συμπληρωθεί όταν ο φοιτητής αποφοιτήσει (8ψήφιος ακέραιος — DDMMYYYY —, αρχικά έχει την τιμή 00000000).

Τα στοιχεία αυτά καταγράφονται, ανά κατηγορία φοιτητών, σε αρχεία.

Για τους μεταπτυχιακούς φοιτητές, καταγράφεται επιπλέον και το πρόγραμμα στο οποίο φοιτούν (ακέрайος που αντιστοιχεί σε πρόγραμμα, π.χ. 1 - ΠΜΣ, 2 - ΗΑ, 3 - ΡΗ, 4 - ΜΠΛΑ, 5 - ΓνΕπ, 6 - ΜΕ κλπ.).

Για τους υποψήφιους διδάκτορες, καταγράφεται επιπλέον και ένα εσωτερικό τηλέφωνο επικοινωνίας (4ψήφιος ακέрайος). Επίσης, καταγράφεται και ο παλιός αριθμός μητρώου τους ως μεταπτυχιακοί, αν το διδακτορικό τους είναι συνέχεια των μεταπτυχιακών τους σπουδών.

Τόσο ένας μεταπτυχιακός φοιτητής, όσο και ένας φοιτητής επί διδακτορικό, ενδέχεται να έχουν και άλλους κωδικούς στο δίκτυο, λόγω απασχόλησής τους σε προγράμματα του Τμήματος. Άρα, λοιπόν, όλοι οι φοιτητές αποκτούν τουλάχιστον έναν κωδικό, τον “φοιτητικό” τους. Οι προπτυχιακοί φοιτητές έχουν ακριβώς έναν. Σύμφωνα με τα παραπάνω, οι μεταπτυχιακοί φοιτητές και οι υποψήφιοι διδάκτορες ενδέχεται να έχουν και πάνω από έναν κωδικό. Όμως, στην περίπτωση αυτή, για λόγους διαχείρισης, το πλήθος των κωδικών τους δε μπορεί να ξεπερνά τους πέντε.

Συνοψίζοντας, τα αρχεία στα οποία είναι καταγεγραμμένα τα στοιχεία των φοιτητών έχουν την εξής μορφή:

προπτυχιακοί φοιτητές: (αρχείο: undergraduates.data)

A.M. ΟΝΟΜΑΤΕΠΩΝΥΜΟ ΕΤΟΣ-ΕΓΓΡΑΦΗΣ ΗΜΕΡ.-ΑΠΟΦ. ΚΩΔΙΚΟΣ

μεταπτυχιακοί φοιτητές: (αρχείο: postgraduates.data)

A.M. ΟΝΟΜΑΤΕΠΩΝΥΜΟ ΕΤΟΣ-ΕΓΓΡΑΦΗΣ ΗΜΕΡ.-ΑΠΟΦ. ΜΕΤΑΠΤ.-ΠΡΟΓΡ. ΚΩΔΙΚΟΙ

υποψήφιοι διδάκτορες: (αρχείο: phds.data)

A.M. ΟΝΟΜΑΤΕΠΩΝΥΜΟ ΕΤΟΣ-ΕΓΓΡΑΦΗΣ ΗΜΕΡ.-ΑΠΟΦ. ΕΣΩΤ.-ΤΗΛΕΦΩΝΟ ΚΩΔΙΚΟΙ

Οι κωδικοί των προπτυχιακών φοιτητών είναι της μορφής undXXXX. Οι κωδικοί των μεταπτυχιακών φοιτητών είναι της μορφής postXXXX. Οι κωδικοί των υποψηφίων διδασκόντων είναι της μορφής phdXXXX. Σε κάθε περίπτωση, XXXX είναι ο αριθμός μητρώου του φοιτητή.

Οι διαχειριστές του συστήματος, κατά καιρούς, κάνουν κατάργηση κωδικών που ανήκουν σε φοιτητές. Ένας τέτοιος κωδικός, για να καταργηθεί:

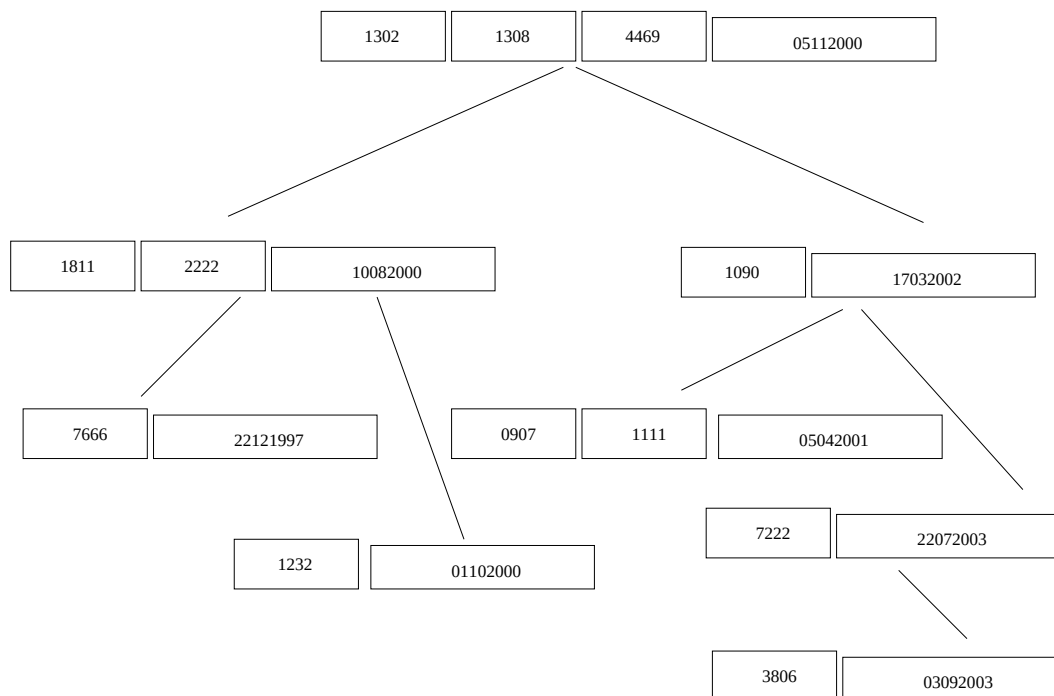
- Αν ανήκει σε προπτυχιακό φοιτητή, πρέπει να έχει περάσει τουλάχιστον ένα έτος από την αποφοίτησή του.
- Αν ανήκει σε μεταπτυχιακό φοιτητή, πρέπει να έχουν περάσει τουλάχιστον δύο έτη από την αποφοίτησή του.
- Αν ανήκει σε υποψήφιο διδάκτορα, πρέπει να έχουν περάσει τουλάχιστον τρία έτη από την αποφοίτησή του.

Να γραφεί ένα πρόγραμμα C++ το οποίο να επιτρέπει στον διαχειριστή του δικτύου:

- Να ελέγχει αν μπορεί να εισαγάγει έναν νέο — μη φοιτητικό — κωδικό για έναν μεταπτυχιακό φοιτητή ή υποψήφιο διδάκτορα, ελέγχοντας ότι, με τον νέο κωδικό, δεν ξεπερνά το όριο των επιτρεπτών κωδικών.
- Με βάση την τρέχουσα ημερομηνία, να γίνεται πρόταση διαγραφής κωδικών των φοιτητών. Η πρόταση να γίνεται εκτυπώνοντας τα στοιχεία των χρηστών των κωδικών που πρόκειται να διαγραφούν, κατά αύξουσα σειρά ημερομηνίας αποφοίτησής τους. Για την ίδια ημερομηνία αποφοίτησης, η εκτύπωση να γίνεται κατά αύξουσα σειρά αριθμού μητρώου. *Παρατήρηση:* Αν ένας φοιτητικός κωδικός που πρόκειται να διαγραφεί ανήκει σε υποψήφιο διδάκτορα, ενδέχεται ο φοιτητής αυτός να υπήρξε και μεταπτυχιακός φοιτητής παλαιότερα και ως τέτοιος να έχει και κωδικό (ή κωδικούς) που να πρέπει να διαγραφούν. Σ’ αυτήν την περίπτωση, κατά την εκτύπωση,

όλοι οι κωδικοί του φοιτητή να είναι ομαδοποιημένοι σε μια εμφάνιση του φοιτητή, η ημερομηνία ταξινόμησης να είναι εκείνη της αποφοίτησης του φοιτητή ως υποψήφιου διδάκτορα και τα υπόλοιπα στοιχεία του ως μεταπτυχιακού (μεταπτυχιακός αριθμός μητρώου, έτος εγγραφής στο μεταπτυχιακό, ημερομηνία αποφοίτησης από το μεταπτυχιακό και μεταπτυχιακό πρόγραμμα) να αγνοούνται.

Για την ταξινόμηση των χρηστών, να χρησιμοποιηθεί μια δομή δυαδικού δένδρου, όπου σε κάθε κόμβο του φυλάσσεται μία ημερομηνία αποφοίτησης και η λίστα των αριθμών μητρώου των φοιτητών που αποφοίτησαν την ημερομηνία αυτή. Η κάθε λίστα να είναι ταξινομημένη κατά αύξουσα σειρά. Σχηματικά, φαίνεται στο Σχήμα 11.7.



Σχήμα 11.7: Η δομή δυαδικού δένδρου για τη φύλαξη των κωδικών των φοιτητών

Στο δένδρο αυτό, για κάθε ημερομηνία-κόμβο του, όλες οι ημερομηνίες που βρίσκονται στο αριστερό υποδένδρο είναι προγενέστερες από αυτήν και όλες οι ημερομηνίες που βρίσκονται στο δεξιό υποδένδρο είναι μεταγενέστερες. Η ιδιότητα αυτή ισχύει αναδρομικά για κάθε ημερομηνία-κόμβο του δένδρου.

11.6.2 Ο Tweety (το πουλί) και ο Sylvester (η γάτα) μπαίνουν σε έναν λαβύρινθο. Ο Tweety μπαίνει πρώτος και προχωρά μερικά (πόσα;) βήματα πριν μπει ο Sylvester. Στον λαβύρινθο κινούνται ανεξάρτητα ο ένας από τον άλλο (δηλαδή, δεν ακολουθούν απαραίτητα την ίδια διαδρομή).

Ο λαβύρινθος έχει μόνο μία είσοδο και μόνο μία έξοδο. Κατά την εμπρόσθια κίνησή τους, τόσο τα Πουλιά, όσο και οι Γάτες, κινούνται από κελί σε κελί του λαβυρίνθου ανάμεσα στα οποία υπάρχει διόδος. Σε κάθε διασταύρωση, τόσο τα Πουλιά, όσο και οι Γάτες, επιλέγουν τυχαία ένα εναλλακτικό. Κάθε εναλλακτικό που επιλέγουν σημειώνεται και έτσι δεν το ξαναεπιλέγουν σε ενδεχόμενη επιστροφή στη διασταύρωση. Κάθε φορά που φτάνουν σε αδιέξοδο, γυρίζουν πίσω στην πιο πρόσφατη διασταύρωση με μη σημειωμένα εναλλακτικά και επιλέγουν, τυχαία πάλι, κάποιο από αυτά (οπισθοδρόμηση).

Όταν ένα Πουλί φτάσει σε αδιέξοδο, εκτυπώνεται στην οθόνη το μήνυμα "Oooooops". Όταν μια Γάτα φτάσει σε αδιέξοδο, εκτυπώνεται στην οθόνη το μήνυμα "Aaaaaah!". Κατά την εμπρόσθια διαδρομή τους, και για τα δύο είδη, εκτυπώνεται στην οθόνη το κελί του λαβυρίνθου στο οποίο μπαίνουν. Αν

κατά την κίνησή τους μέσα στον λαβύρινθο τύχει να συναντηθούν, εκτυπώνεται στην οθόνη το μήνυμα "I tawt I taw a puddy tat!" και ο Tweety οπισθοδρομεί στην πιο πρόσφατη διασταύρωση, αναιρώντας το σημάδι της επίσκεψης για την επιλογή που είχε κάνει.

Κάθε κελί του λαβύρινθου έχει μήκος 1. Τόσο τα Πουλιά, όσο και οι Γάτες, έχουν μια συγκεκριμένη αντοχή, κοινή για όλα τα Πουλιά και κοινή για όλες τις Γάτες. Τα Πουλιά έχουν μεγαλύτερη αντοχή από τις Γάτες. Η αντοχή αυτή ορίζεται ως το μέγιστο μήκος που μπορούν να διατρέξουν συνεχόμενα κατά την εμπρόσθια διαδρομή (δηλαδή, η οπισθοδρόμηση δεν συμβάλλει στην "κούρασή" τους). Όταν η αντοχή τους ξεπεραστεί, τόσο τα Πουλιά, όσο και οι Γάτες, πρέπει να ξεκουραστούν για κάποιο χρονικό διάστημα (μεγαλύτερο για τις Γάτες από ότι για τα Πουλιά).

Ο χρόνος μετρά με βάση ένα (καθολικό) ρολόι, το οποίο αυξάνει τον χρόνο κατά μια μονάδα, για κάθε επανάληψη του ζεύγους: δράση πουλιού - δράση γάτας. Σύμφωνα με τα παραπάνω, μια "δράση πουλιού" μπορεί να είναι ένα από τα:

1. Κίνηση ενός στοιχειώδους τμήματος του λαβυρίνθου σε εμπρόσθια φορά (μετάβαση σε διαδοχικά κελιά).
2. Οπισθοδρόμηση στην πιο πρόσφατη διασταύρωση.
3. Ξεκούραση κατά μια μονάδα χρόνου.

Με τον ίδιο τρόπο, ορίζεται και η "δράση γάτας".

Η σκηνή τελειώνει, εκτυπώνοντας το μήνυμα "The End!", όταν ολοκληρωθεί ένα από τα παρακάτω:

1. Αν ο Tweety βγει πρώτος από τον λαβύρινθο, εκτυπώνεται στην οθόνη το μήνυμα "Clap clap clap!" (και η σκηνή τελειώνει).
2. Αν ο Sylvester βγει πρώτος από τον λαβύρινθο, εκτυπώνεται στην οθόνη το μήνυμα "Bhammm!" — μια πέτρα πέφτει στο κεφάλι του Sylvester — και ο Tweety συνεχίζει, μόνος πια, τη διαδρομή του (ο χρόνος μετρά μόνο για/από αυτόν). Όταν βγει και αυτός από τον λαβύρινθο, εκτυπώνεται στην οθόνη το μήνυμα "Poor puddy tat!" (και η σκηνή τελειώνει).

Επιπλέον εξάσκηση στον διαδικαστικό προγραμματισμό: Γράψτε ένα πρόγραμμα, με όνομα εκτελέσιμου maze, το οποίο να δημιουργεί έναν λαβύρινθο. Θεωρήστε ότι δημιουργούμε ορθογώνιους λαβυρίνθους πλάτους $\langle width \rangle$ και ύψους $\langle height \rangle$, με μία είσοδο, το επάνω αριστερά κελί (με αριθμό 1) και μία έξοδο, το κάτω δεξιά κελί (με αριθμό $\langle width \rangle \times \langle height \rangle$). Τα κελιά αριθμούνται κατά γραμμές. Το πρόγραμμα εκτελείται ως maze $\langle width \rangle \langle height \rangle$ και παράγει ένα πλήθος από γραμμές της μορφής $\langle cell \rangle_1 \langle cell \rangle_2$, όπου αυτό σημαίνει ότι η δίοδος ανάμεσα στα γειτονικά κελιά $\langle cell \rangle_1$ και $\langle cell \rangle_2$ είναι ανοικτή. Το πρόγραμμα, κάθε φορά που εκτελείται, να παράγει τυχαία και έναν άλλο λαβύρινθο. Αν το καλέσετε με κάποιο συγκεκριμένο αριθμό ως τρίτη παράμετρο, να παράγει πάντα τον ίδιο λαβύρινθο. Τέλος, αν δώσετε ως τέταρτη παράμετρο το 1, να παίρνετε και μία γραφική αναπαράσταση του λαβυρίνθου που κατασκευάστηκε. Παράδειγμα:

```
$ maze 6 4 235 1
1 2
2 8
3 4
3 9
4 10
5 6
6 12
7 13
```

```

8   9
10  11
10  16
11  17
12  18
13  14
13  19
14  15
15  21
17  18
17  23
19  20
21  22
22  23
23  24

```

```

+ +--+--+--+--+
|   |   |   |
+--+ + + +--+ +
|   |   |   |
+ +--+--+ + + +
|   |   |   |
+ +--+ +--+ +--+
|   |   |   |
+--+--+--+--+--+

```

11.6.3 Στην εργασία αυτή, έχετε να βοηθήσετε έναν εξόριστο ιππότη να βρει τον δρόμο που οδηγεί στο κάστρο του που έχει καταληφθεί από τους εχθρούς του. Προκειμένου να φτάσει στην πύλη του, όμως, έχει να διανύσει έναν κήπο που οι θάμνοι του σχηματίζουν έναν δαιδαλώδη λαβύρινθο.

Ο κήπος έχει μόνο μια είσοδο και μόνο μια έξοδο, που είναι η πύλη του κάστρου. Ο κάθε χώρος που διαμορφώνεται από τους θάμνους (κελί) του λαβύρινθου:

- Είτε αποτελείται από έναν μικρό λαβύρινθο που σχηματίζεται από δένδρα (με μια μόνο είσοδο και μια μόνο έξοδο).
- Είτε φιλοξενεί έναν δράκο.
- Είτε φιλοξενεί έναν βοηθό.
- Είτε φιλοξενεί μια καλή νεράιδα.
- Είτε φιλοξενεί έναν μονομάχο.
- Είτε τίποτα από τα παραπάνω, δηλαδή είναι ένα ξέφωτο.

Ο ιππότης μπαίνει στον λαβύρινθο με μια αρχική διάρκεια ζωής, N χρόνια, και κινείται από κελί σε κελί ανάμεσα στα οποία υπάρχει δίοδος. Στην περίπτωση που το κελί περιέχει έναν εσωτερικό λαβύρινθο, μπαίνει σε αυτόν και κινείται πάλι από κελί σε κελί του εσωτερικού, μέχρι να βγει από αυτόν. Κατόπιν, είναι σε θέση να βγει από το εξωτερικό κελί και να μεταβεί σε κάποιο άλλο. Για κάθε μετάβαση από κελί σε κελί, χάνει έναν (1) χρόνο από τη ζωή του, είτε πρόκειται για κελιά του λαβυρίνθου του κήπου είτε για κελιά ενός εσωτερικού λαβυρίνθου, εκτός

- εάν το κελί φιλοξενεί έναν δράκο, οπότε χάνει έναν επιπλέον χρόνο,

- εάν το κελί φιλοξενεί έναν βοηθό, οπότε του χαρίζεται ο χρόνος (δεν γίνεται η μείωση κατά 1),
- εάν το κελί φιλοξενεί μια καλή νεράιδα, οπότε του χαρίζονται k_1 χρόνια,
- εάν το κελί φιλοξενεί έναν μονομάχο, οπότε πρέπει να μονομαχήσει μαζί του. Το αποτέλεσμα της μονομαχίας είναι τυχαίο. Εάν κερδίσει, του χαρίζονται k_2 χρόνια. Εάν ηττηθεί, χάνει επιπλέον k_3 χρόνια.

Τα N , k_1 , k_2 , k_3 είναι παράμετροι του προβλήματος (και δίνονται από τη γραμμή εντολής).

Σε κάθε κελί του λαβυρίνθου του κήπου, αντιστοιχεί ένας αριθμός που εκφράζει το πόσο δύσκολο είναι να περάσει από αυτό το κελί. Αυτό προκύπτει, αυθαίρετα, ως εξής:

- Εάν το κελί αποτελείται από έναν εσωτερικό λαβύρινθο, η δυσκολία του είναι ίση με το σύνολο των κελιών του εσωτερικού λαβύρινθου.
- Εάν το κελί φιλοξενεί έναν δράκο, η δυσκολία του είναι ίση με 1.
- Εάν το κελί φιλοξενεί έναν βοηθό, η δυσκολία του είναι ίση με -1 .
- Εάν το κελί φιλοξενεί μια καλή νεράιδα, η δυσκολία του είναι ίση με $-k_1$.
- Εάν το κελί φιλοξενεί έναν μονομάχο, η δυσκολία του είναι ίση με $k_3 - k_2$.
- Εάν το κελί είναι ξέφωτο, η δυσκολία του είναι ίση με 0.

Κατά την *εμπρόσθια κίνησή* του στον λαβύρινθο του κήπου, ο ιππότης επιλέγει να μεταβεί στο κελί με τη μικρότερη δυσκολία, από εκείνα που του είναι διαθέσιμα. Κάθε εναλλακτικό που επιλέγεται σημειώνεται και δεν είναι πλέον διαθέσιμο σε ενδεχόμενη επιστροφή στο κελί. Κάθε φορά που φτάνει σε αδιέξοδο, γυρίζει πίσω στο πιο πρόσφατο κελί με διαθέσιμα εναλλακτικά για μετάβαση (*οπισθοδρόμηση*) και επιλέγει να επισκεφθεί αυτό με τη μικρότερη δυσκολία, από αυτά που δεν έχει επισκεφθεί ήδη. Κατά την οπισθοδρόμηση, δεν χάνει καθόλου ζωή.

Όσον αφορά την κίνηση του ιππότη μέσα σε έναν εσωτερικό λαβύρινθο, αυτή γίνεται με ανάλογο τρόπο με την κίνηση στον λαβύρινθο του κήπου, εκτός του ότι η επιλογή του κελιού για μετάβαση γίνεται τυχαία. Πάλι, μετά την οπισθοδρόμηση, πρέπει να ακολουθήσει νέα εναλλακτική διαδρομή.

Κατά την εμπρόσθια κίνηση του ιππότη, είτε στον λαβύρινθο του κήπου είτε σε εσωτερικό λαβύρινθο, εκτυπώνεται το κελί του λαβύρινθου στο οποίο μπαίνει.

- Εάν πρόκειται για κελί του λαβύρινθου του κήπου που περιέχει εσωτερικό λαβύρινθο, εκτυπώνεται και το μήνυμα "Oh, another maze", πριν αρχίσουν οι εκτυπώσεις του εσωτερικού λαβυρίνθου.
- Εάν πρόκειται για κελί του λαβύρινθου του κήπου που φιλοξενεί δράκο, εκτυπώνεται και το μήνυμα "Oh, Dear!".
- Εάν πρόκειται για κελί του λαβύρινθου του κήπου που φιλοξενεί βοηθό, εκτυπώνεται και το μήνυμα "Go ahead!".
- Εάν πρόκειται για κελί του λαβύρινθου του κήπου που φιλοξενεί καλή νεράιδα, εκτυπώνεται και το μήνυμα "Good luck!".
- Εάν πρόκειται για κελί του λαβύρινθου του κήπου που φιλοξενεί μονομάχο και ο ιππότης ηττηθεί, εκτυπώνεται και το μήνυμα "It is such a pity!".
- Εάν πρόκειται για κελί του λαβύρινθου του κήπου που φιλοξενεί μονομάχο και ο ιππότης νικήσει, εκτυπώνεται και το μήνυμα "I am better now!".

Αν ο ιππότης συναντήσει αδιέξοδο, εκτυπώνεται το μήνυμα "Back again" και το κελί στο οποίο επιστρέφει. Όταν βγει από τον λαβύρινθο, εκτυπώνεται το μήνυμα "Here I am!". Εάν εξαντλήσει τη ζωή του και δεν προλάβει να βγει, εκτυπώνεται το μήνυμα "Game over! Change your parameters to help me, please!".

Να υλοποιήσετε το παραπάνω με C++.

Οι λαβύρινθοι που θα χειρίζεστε είναι ορθογώνιοι πλάτους $\langle width \rangle$ και ύψους $\langle height \rangle$. Τα κελιά τους αριθμούνται κατά γραμμές. Οι λαβύρινθοι διαθέτουν μία είσοδο που είναι στο επάνω αριστερά κελί (με αριθμό 1) και μία έξοδο που βρίσκεται στο κάτω δεξιά κελί (με αριθμό $\langle width \rangle \times \langle height \rangle$). Το πρόγραμμά σας θα πρέπει να διαβάσει τα στοιχεία του λαβυρίνθου από αρχεία της μορφής που περιγράφεται παρακάτω (δίνοντας το σχετικό όνομα αρχείου στη γραμμή εντολής). Στην πρώτη γραμμή του αρχείου, υπάρχουν τρεις ακέραιοι αριθμοί. Ο πρώτος είναι το πλάτος $\langle width \rangle$ του λαβυρίνθου, ο δεύτερος είναι το ύψος του $\langle height \rangle$ και ο τρίτος είναι ο αριθμός $\langle P \rangle$ των ανοικτών διόδων από κελί σε κελί. Έπειτα, ακολουθούν $\langle P \rangle$ γραμμές της μορφής $\langle cell \rangle_1 \langle cell \rangle_2$, που σημαίνουν πως η δίοδος ανάμεσα στα γειτονικά κελιά $\langle cell \rangle_1$ και $\langle cell \rangle_2$ είναι ανοικτή. Τέλος, υπάρχουν $\langle width \rangle \times \langle height \rangle$ περιγραφές για το περιεχόμενο των κελιών του λαβυρίνθου. Κάθε περιγραφή ξεκινά με έναν ακέραιο, που είναι ο αριθμός του κελιού, το περιεχόμενο του οποίου περιγράφεται. Ακολουθείται στην ίδια γραμμή από έναν χαρακτήρα που έχει τις εξής τιμές:

- E: το κελί είναι άδειο (Empty), δηλαδή είναι ένα ξέφωτο.
- G: το κελί φιλοξενεί έναν μονομάχο (Gladiator).
- F: το κελί φιλοξενεί μία νεράιδα (Fairy).
- H: το κελί φιλοξενεί έναν βοηθό (Helper).
- D: το κελί φιλοξενεί έναν δράκο (Dragon).
- M: το κελί αποτελείται από έναν μικρό λαβύρινθο (Maze).

Στις περιπτώσεις των E, G, F, H, D, αυτή είναι η μοναδική γραμμή που περιγράφει το περιεχόμενο του κελιού. Στην περίπτωση του M, η γραμμή αυτή ακολουθείται από μια γραμμή της μορφής $\langle subwidth \rangle \langle subheight \rangle \langle P \rangle$. Οι τρεις αυτοί ακέραιοι συμβολίζουν τις διαστάσεις καθώς και τον αριθμό των ανοικτών διόδων του μικρού λαβυρίνθου, ο οποίος αποτελεί το περιεχόμενο του τρέχοντος κελιού. Μετά από τη γραμμή αυτή, υπάρχουν $\langle P \rangle$ γραμμές της μορφής $\langle cell \rangle_1 \langle cell \rangle_2$, που περιγράφουν τις διόδους ανάμεσα στα γειτονικά κελιά του μικρού λαβυρίνθου.

Επιπλέον εξάσκηση στον διαδικαστικό προγραμματισμό: Να υλοποιήσετε τους παραπάνω λαβυρίνθους σε ένα πρόγραμμα με όνομα εκτελέσιμου maze. Εκτελώντας maze $\langle width \rangle \langle height \rangle \langle subwidth \rangle \langle subheight \rangle$, όπου $\langle width \rangle, \langle height \rangle$ είναι οι διαστάσεις του λαβυρίνθου και $\langle subwidth \rangle, \langle subheight \rangle$ αυτές των μικρών λαβυρίνθων, να παράγετε τυχαίους τέτοιους λαβύρινθους. Αν το καλέσετε με κάποιο συγκεκριμένο αριθμό ως πέμπτη παράμετρο, να παράγει πάντα τον ίδιο λαβύρινθο. Τέλος, αν δώσετε ως έκτη παράμετρο το 1, να παίρνετε και μια γραφική αναπαράσταση των λαβυρίνθων που κατασκευάστηκαν. Παράδειγμα:

```
$ maze 6 4 3 3 233 1
6 4 23
1 7
2 3
2 8
3 9
4 5
4 10
```

5 11
6 12
7 8
9 15
11 12
11 17
12 18
13 14
13 19
14 20
15 21
16 17
16 22
18 24
20 21
21 22
23 24
1 F
2 H
3 D
4 M
3 3 8
1 4
2 3
2 5
3 6
4 7
5 8
6 9
7 8
5 D
6 E
7 E
8 D
9 E
10 E
11 F
12 F
13 E
14 F
15 E
16 G
17 H
18 E
19 E
20 H
21 E
22 E
23 F

Main Maze:

```

+ +-+--+--+--+
| F | H | D | M | D | |
+ + + + + + +
|   D |   | F | F |
+-+--+ +-+ + +
|   F |   | G | H | |
+ + + + +-+ +
|   | H |   | F |   |
+-+--+--+--+ +

```

Submaze in cell 4:

```

+ +-+--+
|   |   |
+ + + +
|   |   |
+ + + +
|   |   |
+-+--+ +

```

11.6.4 Στην εργασία αυτή², πρέπει να υλοποιήσετε με C++ ένα πρόγραμμα διαχείρισης και σύνθεσης ηλεκτρονικών μηνυμάτων. Συγκεκριμένα, να υλοποιήσετε μια εντολή `mine`, η οποία να επιτρέπει τα παρακάτω:

Διαχείριση λίστας εισερχόμενων μηνυμάτων:

- Εμφάνιση της λίστας εισερχόμενων μηνυμάτων, θέτοντας ως τρέχον μήνυμα το πρώτο που δεν έχει διαβαστεί, ταξινομημένα κατά ημερομηνία άφιξης, (`L a`).
- Εμφάνιση της λίστας εισερχόμενων μηνυμάτων, θέτοντας ως τρέχον μήνυμα το πρώτο που δεν έχει διαβαστεί, ταξινομημένα κατά ημερομηνία αποστολής, (`L s`).
- Αλλαγή τρέχοντος μηνύματος, θέτοντας ως τρέχον το υπ' αριθμόν N , (`J N`).
- Διαγραφή τρέχοντος μηνύματος, (`D`), θέτοντας ως τρέχον το επόμενο που δεν έχει διαγραφεί. Αν δεν υπάρχει τέτοιο, επιλέγεται το προηγούμενο που δεν έχει διαγραφεί. Αν δεν υπάρχει ούτε τέτοιο (έχουν διαγραφεί όλα), δεν γίνεται κανένα τρέχον μήνυμα και δεν επιτρέπεται καμία λειτουργία που εφαρμόζεται στο τρέχον μήνυμα.
- Προβολή του τρέχοντος μηνύματος, (`P`), ενημερώνοντας ότι το μήνυμα διαβάστηκε.

Σύνθεση και αποστολή μηνύματος:

- Απάντηση στο τρέχον μήνυμα, (`R`). Κατά τη διαδικασία αυτή, οι επικεφαλίδες προσυμπληρώνονται με τα κατάλληλα στοιχεία. Πιο συγκεκριμένα:
 - Η διεύθυνση του αποστολέα του νέου μηνύματος είναι η διεύθυνση του παραλήπτη του αρχικού μηνύματος.
 - Η διεύθυνση του παραλήπτη του νέου μηνύματος είναι η διεύθυνση του αποστολέα του αρχικού μηνύματος.
 - Το θέμα του μηνύματος προκύπτει από το θέμα του αρχικού μηνύματος, βάζοντάς του το πρόθεμα "Re: ".

²Στην εκφώνηση της εργασίας αυτής, είχε συμβάλει ο Στέφανος Σταμάτης.

- Σύνθεση νέου μηνύματος, (C). Κατά τη διαδικασία αυτή, πρέπει να συμπληρωθούν και τα στοιχεία των επικεφαλίδων.

Με την κλήση του προγράμματος, εμφανίζεται η λίστα εισερχόμενων μηνυμάτων και το πρόγραμμα περιμένει να του προσδιοριστεί από την καθιερωμένη είσοδο κάποια από τις παραπάνω λειτουργίες.

Όλα τα μηνύματα βρίσκονται αποθηκευμένα σε ένα αρχείο γραμματοθυρίδας, το ένα μετά το άλλο.

Κάθε μήνυμα αποτελείται από:

- Τη διεύθυνση του αποστολέα.
- Τη διεύθυνση του παραλήπτη.
- Το θέμα του.
- Την ημερομηνία αποστολής του.
- Την ημερομηνία παραλαβής του.
- Ένδειξη για το αν έχει διαβαστεί ή όχι.
- Το κείμενο του μηνύματος.

Ένα μήνυμα όταν αποθηκεύεται:

- Αρχικά έχει μία γραμμή που ξεκινά με τη λέξη "From", ακολουθούμενη από ένα κενό, τη διεύθυνση του αποστολέα και την ημερομηνία παραλαβής του.
- Στην επόμενη γραμμή, υπάρχει η λέξη "To: ", ακολουθούμενη από ένα κενό και τη διεύθυνση του παραλήπτη.
- Έπεται μία γραμμή που ξεκινά με τη λέξη "Date: ", ακολουθούμενη από ένα κενό και την ημερομηνία αποστολής του.
- Μετά υπάρχει μία γραμμή που ξεκινά με τη λέξη "Subject: ", ακολουθούμενη από ένα κενό και το θέμα του μηνύματος.
- Τέλος υπάρχει μία κενή γραμμή, και ένα πλήθος γραμμών που περιέχουν το σώμα του. Το σώμα του μηνύματος τελειώνει, όταν βρεθεί μία γραμμή που περιέχει μονάχα μία τελεία.

Οι ημερομηνίες έχουν τη μορφή *YYYY MM DD HH MI SE*, όπου

- *YYYY* είναι το έτος,
- *MM* ο μήνας,
- *DD* η ημέρα μέσα στον μήνα,
- *HH* η ώρα,
- *MI* τα λεπτά και
- *SE* τα δευτερόλεπτα.

Παράδειγμα: 2006 04 14 21 05 40

Έτσι, ένα παράδειγμα μηνύματος αποτελεί το ακόλουθο:

```
From someone@di.uoa.gr 2006 04 14 21 05 40
To: kapoios@di.uoa.gr
Date: 2006 04 14 21 03 20
Subject: A sample message
```

This is a sample message to test the
 OOP Project functionality.
 So this is just a test text!
 .

Σημείωση: Για λόγους απλότητας, οι αλλαγές που γίνονται στη λίστα εισερχομένων μηνυμάτων να μην διατηρούνται μετά το τέλος της εκτέλεσης του προγράμματος.

Τόσο κατά την απάντηση σε εισερχόμενο μήνυμα, όσο και κατά τη σύνθεση νέου μηνύματος, ο χρήστης να έχει στη διάθεσή του έναν εκδότη κειμένου γραμμής (line editor) με τις εξής ιδιότητες:

- Δεν υπάρχει περιορισμός στο πλήθος των γραμμών που μπορεί να γράψει.
- Δεν υπάρχει περιορισμός στον αριθμό των χαρακτήρων κάθε γραμμής.
- Το τέλος γραμμής εκφράζεται με τον χαρακτήρα αλλαγής γραμμής.

Ο εκδότης κειμένου να υποστηρίζει τις εξής λειτουργίες. Σημειώστε ότι κάθε στιγμή θεωρούμε ότι υπάρχει μια τρέχουσα γραμμή και σε αυτή ένας τρέχων χαρακτήρας, οπότε έχουμε:

- Προσθήκη κειμένου δεξιά από τον τρέχοντα χαρακτήρα, $i \langle text \rangle$.
- Διαγραφή N χαρακτήρων αριστερά από τον τρέχοντα χαρακτήρα, $x \ N$.
- Μετακίνηση τρέχοντα χαρακτήρα N θέσεις αριστερά (δεξιά), $h \ (1) \ N$.
- Μετακίνηση τρέχοντα χαρακτήρα N θέσεις πάνω (κάτω), $k \ (j) \ N$.
- Διαγραφή N γραμμών από την τρέχουσα γραμμή και κάτω, $d \ N$.
- Αντιγραφή σε προσωρινό χώρο N γραμμών από την τρέχουσα γραμμή και κάτω (πάνω), $y \ (Y) \ N$.
- Επικόλληση των αντιγραμμένων γραμμών από την τρέχουσα γραμμή και κάτω (πάνω), $p \ (P)$.
- Εισαγωγή μιας νέας γραμμής κειμένου κάτω (πάνω) από την τρέχουσα γραμμή, $o \ (O) \langle text \rangle$.

Όταν τελειώσει την επεξεργασία του σώματος του μηνύματος, ο χρήστης να μπορεί να επιλέξει αποστολή του πλήρους μηνύματος ή ακύρωση της αποστολής. Τα μηνύματα κατά την αποστολή τους προσαρτώνται σε ένα αρχείο με όνομα `sentmailqueue`.

Το κείμενο κατά την επεξεργασία του φυλάσσεται σε ένα σύνολο από buffers με τον εξής τρόπο. Έχουμε (άπειρους το πλήθος) buffers σταθερού μεγέθους *Size*. Κάθε buffer φυλάσσει κάποιο τμήμα του κειμένου, έναν χαρακτήρα σε κάθε στοιχείο του. Όμως, το πρώτο στοιχείο του buffer είναι ένας δείκτης στον επόμενο. Το δεύτερο στοιχείο του buffer είναι ένας δείκτης στον προηγούμενο. Το τρίτο στοιχείο φυλάσσει το πλήθος των χαρακτήρων που φυλάσσονται στον buffer. Ακολουθούν οι χαρακτήρες σε *Size* το πολύ θέσεις. Μια γραμμή μπορεί να εκτείνεται σε παραπάνω από ένα buffer. Μετά από χαρακτήρα αλλαγής γραμμής, απαγορεύεται να φυλάσσονται άλλοι χαρακτήρες στον ίδιο buffer.

Επίσης, θεωρούμε μια διπλά συνδεδεμένη λίστα, που κάθε στοιχείο της δείχνει στον πρώτο buffer κάθε γραμμής. Επομένως, έχει τόσα στοιχεία, όσες και οι γραμμές του αρχείου. Ο ένας δείκτης της λίστας δείχνει στην προηγούμενη γραμμή και ο άλλος στην επόμενη. Αυτή μας επιτρέπει να κινούμαστε εύκολα από γραμμή σε γραμμή.

Αρχικά: Η αρχικοποίηση αντιστοιχεί σε μηδενισμούς δεικτών.

Εισαγωγή γραμμής: Για κάθε γραμμή που εισάγεται, φτιάχνουμε τους buffers που της αντιστοιχούν, αν αυτοί δεν υπάρχουν, και τους εισάγουμε στις κατάλληλες θέσεις. Ταυτόχρονα, ενημερώνουμε και τη λίστα γραμμών.

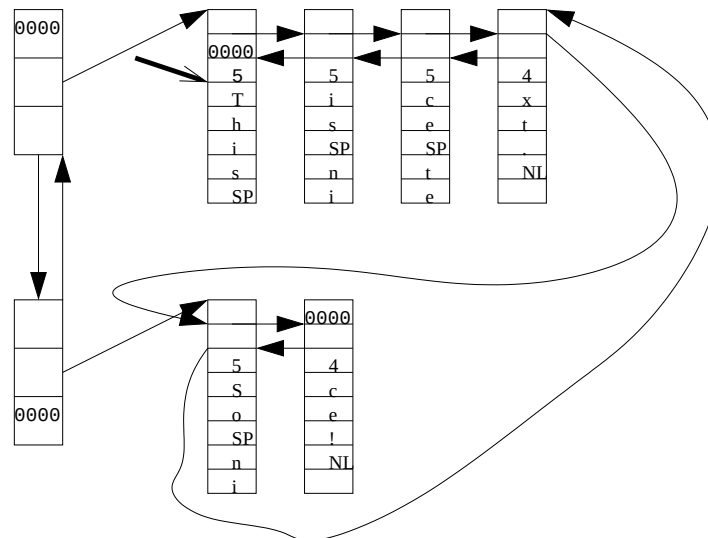
Διαγραφή γραμμής: Αντίστροφη διαδικασία ακολουθούμε στη διαγραφή γραμμών.

Εισαγωγή κειμένου: Για την εισαγωγή κειμένου σε υπάρχουσα γραμμή, αν αυτό εισάγεται στο τέλος buffer, η διαδικασία είναι απλή: γεμίζουμε τον τρέχοντα buffer και φτιάχνουμε άλλους για το υπόλοιπο κείμενο, τους οποίους εισάγουμε. Αν, όμως, η εισαγωγή πρέπει να γίνει ενδιάμεσα στον τρέχοντα buffer, τον σπάμε στο τρέχον σημείο και γεμίζουμε τον πρώτο buffer, ακολουθώντας την προηγούμενη διαδικασία. Το δεύτερο κομμάτι του αρχικού buffer ακολουθεί μετά το σύνολο των νέων buffers.

Διαγραφή κειμένου: Για τη διαγραφή χαρακτήρων, αν αυτοί διαγράφονται από τέλος buffer, απλώς ενημερώνουμε τον μετρητή και τον τρέχοντα χαρακτήρα. Αν η διαγραφή γίνεται ενδιάμεσα σε buffer, τον σπάμε, ώστε οι χαρακτήρες που θα διαγραφούν να είναι από το τέλος buffer. Buffers που ενδεχόμενα αδειάζουν, διαγράφονται.

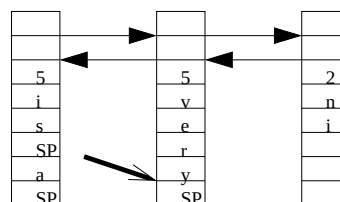
Παραδείγματα λειτουργιών για buffers μεγέθους 5 φαίνονται στο Σχήμα 11.8.

This is nice text.
So nice!



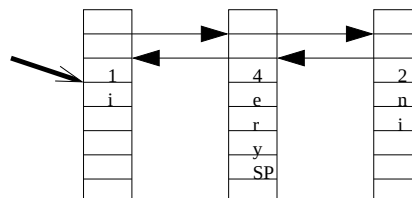
l 7
i "a very "

(second buffer is split, and a new buffer is required which is put between the two parts)



h 3
x 5

(the "verySP" buffer is split into "v" and "erySP" buffers and deleting starts from "v" backwards)



Σχήμα 11.8: Οι λειτουργίες σε buffers μεγέθους 5

11.6.5 Να υλοποιηθεί σε C++ η προσομοίωση μίας υποθετικής λειτουργίας (της μίας κατεύθυνσης) της Αττικής Οδού. Σύμφωνα με αυτή, για τον αυτοκινητόδρομο ισχύουν τα παρακάτω.

Η Αττική Οδός έχει έναν αρχικό κόμβο, έναν τελικό κόμβο και ένα σύνολο ενδιάμεσων κόμβων. Στον αρχικό κόμβο, πραγματοποιείται μόνο είσοδος οχημάτων και στον τελικό κόμβο, μόνο έξοδος. Σε κάθε ενδιάμεσο κόμβο, πραγματοποιείται τόσο είσοδος, όσο και έξοδος οχημάτων.

Η Αττική Οδός αποτελείται από $NSegs$ τμήματα μεταξύ των κόμβων της. Οι κόμβοι της είναι πλήθους $NSegs + 1$, από τον κόμβο 0 (αρχικός) έως τον κόμβο $NSegs$ (τελικός). Κάθε τμήμα του αυτοκινητοδρόμου έχει μία χωρητικότητα, πιθανώς διαφορετική για καθένα από αυτά, που είναι ο μέγιστος αριθμός οχημάτων που μπορεί να βρίσκεται μέσα στο τμήμα.

Σε κάθε κόμβο, πλην του τελικού, υπάρχει ένας αριθμός διόδων για είσοδο στο τμήμα που αρχίζει από τον κόμβο αυτό. Το πλήθος των διόδων μπορεί να είναι διαφορετικό για τους διάφορους κόμβους. Σε κάθε κόμβο, κάποια από τα διόδια είναι με εισπράκτορα και κάποια είναι ηλεκτρονικά.

Σε μία κατάσταση του αυτοκινητοδρόμου, υπάρχει ένας αριθμός οχημάτων που βρίσκεται σε κάθε τμήμα του. Κάποια από αυτά τα οχήματα είναι σε κατάσταση “ετοιμότητας” για έξοδο από το τμήμα. Από αυτά, εκείνα που έχουν ως προορισμό τον κόμβο εξόδου του τμήματος στο οποίο βρίσκονται, στην επόμενη κατάσταση θα βγουν από τον αυτοκινητόδρομο. Εκείνα που έχουν ως προορισμό κάποιον επόμενο κόμβο εξόδου πρέπει τελικά να περάσουν στο επόμενο τμήμα του αυτοκινητοδρόμου και ενδέχεται στην επόμενη κατάσταση να καταστεί εφικτό να το κάνουν. Τα υπόλοιπα οχήματα, που δεν είναι σε κατάσταση “ετοιμότητας”, θα παραμείνουν στο συγκεκριμένο τμήμα και στην επόμενη κατάσταση. Επίσης, υπάρχει και ένας αριθμός οχημάτων που αναμένουν να εισέλθουν στο συγκεκριμένο τμήμα από τα διόδια εισόδου του τμήματος. Σε κάθε δίοδιο του τμήματος (με εισπράκτορα ή ηλεκτρονικό), μπορεί να αναμένει διαφορετικός αριθμός οχημάτων για να εισέλθει σ’ αυτό. Από τα οχήματα που θα βρίσκονται στο τμήμα στη νέα κατάσταση (αυτά που βρίσκόντουσαν σ’ αυτό και στην προηγούμενη, αυτά που πέρασαν σ’ αυτό από το προηγούμενο τμήμα, αν υπάρχει, και αυτά που μπήκαν από τον κόμβο εισόδου του τμήματος), ένα ποσοστό *Percent* από αυτά μεταβαίνει σε κατάσταση “ετοιμότητας” εξόδου από το τμήμα.

Για τη δημιουργία της επόμενης κατάστασης του αυτοκινητοδρόμου, η προσομοίωση δουλεύει από το τέλος του προς την αρχή του. Τα αυτοκίνητα που έχουν ως προορισμό τον τελικό κόμβο του αυτοκινητοδρόμου θα βγουν από αυτόν. Στη συνέχεια, τα οχήματα που είναι σε κατάσταση “ετοιμότητας” από το αμέσως προηγούμενο τμήμα για να περάσουν σε αυτό θα το κάνουν, εφόσον αυτό είναι επιτρεπτό από τη χωρητικότητα του τμήματος. Αν δεν γίνεται να περάσουν όλα, θα το κάνουν μόνο τόσα, ώστε να μην παραβιαστεί η χωρητικότητα. Αν, αφού περάσουν στο τρέχον τμήμα που επεξεργαζόμαστε οχήματα από το προηγούμενο τμήμα, υπάρχει διαθεσιμότητα σ’ αυτό για είσοδο και νέων οχημάτων, αυτό θα γίνει από τα διόδια εισόδου του, επιτρέποντας σε K το πολύ οχήματα να εισέλθουν από διόδια με εισπράκτορα και σε $2K$ το πολύ από ηλεκτρονικά διόδια. Πάλι, δεν πρέπει να παραβιαστεί η χωρητικότητα του τμήματος. Αυτό σημαίνει ότι πιθανώς να επιτραπεί να εισέλθουν στο τμήμα ένα ποσοστό, το ίδιο για όλα τα διόδια, οχημάτων από τα K και $2K$ που αντιστοιχούν στον μέγιστο αριθμό οχημάτων που επιτρέπεται να εισέλθουν στο τμήμα στην τρέχουσα κατάστασή του (τον τρέχοντα κύκλο προσομοίωσης). Αν χρειαστεί να περάσουν λιγότερα οχήματα από τον μέγιστο αριθμό, το K θα πρέπει να μειωθεί κατά 1 για τον συγκεκριμένο κόμβο κατά τον επόμενο κύκλο της προσομοίωσης. Αν εισέλθει στο τμήμα ο μέγιστος αριθμός οχημάτων, K και $2K$ για τους δύο τύπους διόδων, το K θα αυξηθεί κατά 1 για τον συγκεκριμένο κόμβο κατά τον επόμενο κύκλο.

Αφού αποφασισθεί ποια οχήματα θα βρίσκονται σε ένα τμήμα του αυτοκινητοδρόμου στη νέα κατάσταση, με βάση το τι θα ισχύει στο επόμενο τμήμα, η προσομοίωση προχωρά στην εφαρμογή της παραπάνω διαδικασίας στα προηγούμενα τμήματα, μέχρι να φτάσει σταδιακά και στο αρχικό.

Σε μια κατάσταση τμήματος:

- Αν ο αριθμός των οχημάτων που εισέρχονται σε ένα τμήμα από την είσοδό του είναι μικρότερος από το πλήθος των οχημάτων που περιμένουν στην είσοδο, εμφανίζεται το μήνυμα “Καθυστερήσεις στην είσοδο του κόμβου <Αριθμός Κόμβου>”.
- Αν (επιπλέον) ο αριθμός των οχημάτων που μετακινούνται μεταξύ των τμημάτων που συνδέονται με τον κόμβο είναι μικρότερος από αυτά που είναι σε κατάσταση “ετοιμότητας” να μετακινηθούν, εμφανίζεται (και) το μήνυμα “Καθυστερήσεις μετά τον κόμβο <Αριθμός Κόμβου>”.
- Αν δεν ισχύει τίποτα από τα παραπάνω, εμφανίζεται το μήνυμα “Τηρείτε τις αποστάσεις ασφαλείας στο τμήμα μετά τον κόμβο <Αριθμός Κόμβου>”.

Συγκεκριμένα, η Αττική Οδός αναπαριστάται από μια κλάση η οποία:

- Έχει *NSegs* τμήματα.
- Έχει ένα πλήθος οχημάτων που κινούνται συνολικά σε αυτή.

Η συμπεριφορά της είναι η εξής:

- Αρχικά, εκτυπώνεται το μήνυμα “Αυτοκινητόδρομος σε λειτουργία”.
- Η λειτουργία του αυτοκινητόδρομου είναι ένας κύκλος λειτουργίας των τμημάτων της. Τα τμήματα λειτουργούν ακολουθιακά, ξεκινώντας από το τελευταίο. Κάθε φορά, εμφανίζεται το πλήθος των οχημάτων. (operate)

Ένα τμήμα του αυτοκινητόδρομου αναπαριστάται από μια κλάση η οποία:

- Έχει μια είσοδο.
- Έχει ένα σύνολο οχημάτων που κινούνται σε αυτό.
- Έχει μια χωρητικότητα οχημάτων.
- Έχει (ενδεχόμενα) ένα επόμενο τμήμα.
- Έχει (ενδεχόμενα) ένα προηγούμενο τμήμα.

Η συμπεριφορά της είναι η εξής:

- Αρχικά, δημιουργούνται τυχαία αυτοκίνητα που κινούνται σε αυτό.
- Εισέρχονται στο τμήμα τα οχήματα που είναι δυνατόν. (enter)
- Εξέρχονται από τον αυτοκινητόδρομο τα οχήματα που είναι δυνατόν. (exit)
- Εξέρχονται από το τμήμα και μεταβαίνουν στο επόμενο τα οχήματα που είναι δυνατόν. (pass)
- Επιστρέφει το πλήθος των οχημάτων που κινούνται σε αυτό. (get_no_of_vehicles)
- Λειτουργεί, αλλάζοντας το σύνολο των οχημάτων που κινούνται σε αυτό. Στην περίπτωση που δεν είναι δυνατή η μετάβαση όλων των οχημάτων που είναι έτοιμα να βγουν στο επόμενο τμήμα, η επιλογή γίνεται τυχαία. Επίσης, κατά την αλλαγή του συνόλου των οχημάτων, αφαιρούνται και τα εισερχόμενα οχήματα από την είσοδό του. Κατά τη λειτουργία του, εμφανίζονται τα σχετικά μηνύματα. Επίσης, κάποια (τυχαία) οχήματα γίνονται έτοιμα να βγουν από το τμήμα. (operate)

Μια είσοδος αναπαριστάται από μια κλάση η οποία:

- Έχει το όνομα/ταυτότητα του κόμβου.

- Έχει ένα σύνολο διοδίων με υπαλλήλους.
- Έχει ένα σύνολο ηλεκτρονικών διοδίων.

Η συμπεριφορά της είναι η εξής. Αφαιρούνται οχήματα από την είσοδο, αφαιρώντας οχήματα από τα διόδια. Προστίθενται τυχαία οχήματα στα διόδιά της. (operate)

Ένα διόδιο αναπαριστάται από το σύνολο των οχημάτων που περιμένουν να μπουν. Αρχικά, δημιουργούνται τυχαία αυτοκίνητα που περιμένουν σε αυτό. Προστίθενται οχήματα στο διόδιο. Αφαιρούνται οχήματα από το διόδιο.

Τα οχήματα αναπαριστώνται από τον κόμβο εξόδου τους και το τμήμα του αυτοκινητόδρομου στο οποίο κινούνται. Όταν περιμένουν να μπουν στον αυτοκινητόδρομο, η ένδειξη του τμήματος είναι -1. Επίσης, περιλαμβάνουν μια ένδειξη αν είναι έτοιμα να βγουν από το τμήμα που κινούνται (το έχουν διανύσει). Αρχικά, δεν είναι έτοιμα.

Να υλοποιήσετε τις απαραίτητες κλάσεις σε C++. Επίσης, να υλοποιηθεί και μια συνάρτηση `main` που να εκτελεί N κύκλους λειτουργίας του αυτοκινητόδρομου, για καθένα από τους οποίους προστίθενται τυχαία οχήματα στις εισόδους των κόμβων του αυτοκινητόδρομου.

Από τη γραμμή εντολής, κατά την εκτέλεση του προγράμματος, δίνονται τα εξής:

N : Αριθμός κύκλων προσομοίωσης.

N Segs: Πλήθος τμημάτων της Αττικής οδού.

K : Μέγιστος αριθμός οχημάτων που μπορούν να εισέλθουν σ' ένα τμήμα του αυτοκινητοδρόμου από κάποιο διόδιο με εισπράκτορα. Αρχικά, το K είναι ίδιο για όλα τα διόδια με εισπράκτορα όλων των κόμβων (για τα ηλεκτρονικά διόδια είναι $2K$).

Percent: Το ποσοστό των οχημάτων σ' ένα τμήμα, κοινό για όλα τα τμήματα, που μεταβαίνει σε κατάσταση "ετοιμότητας" για έξοδο από το τμήμα στην επόμενη κατάσταση.

Οι χωρητικότητες των τμημάτων της Αττικής Οδού διαβάζονται από την καθιερωμένη είσοδο.

Η αρχική κατάσταση της Αττικής Οδού δημιουργείται με τυχαίο τρόπο. Σε κάθε βήμα της προσομοίωσης, ο αριθμός των οχημάτων που επιθυμούν να εισέλθουν από κάποιο διόδιο στον αυτοκινητόδρομο επιλέγεται με τυχαίο τρόπο.

Όπου χρειάζεται, κάνετε ελεύθερα παραδοχές για τη λειτουργία του αυτοκινητοδρόμου, με βάση τη γενικότερη εμπειρία σας από τον πραγματικό κόσμο, και τεκμηριώστε τις.

Άλλες Περιοχές Εργασιών Επιπλέον των προηγούμενων εργασιών, προτείνεται η ανάπτυξη κώδικα στις περιοχές που αναφέρονται παρακάτω και στις οποίες βασίστηκαν εργασίες του μαθήματος "Αντικειμενοστραφής Προγραμματισμός" που αναφέρθηκε στον Πρόλογο. Τόσο η βασική ιδέα, όσο και οι εκφωνήσεις τους ήταν κατά κύριο λόγο έργο συνεργατών.

Συγκεκριμένα, περιοχές που προτείνονται για ανάπτυξη κώδικα εργασιών αντικειμενοστραφούς προγραμματισμού είναι οι εξής:

- Role Playing Game (Συνεργάτες: Γ. Καστρίνης, Ο. Πολυχρονίου)
- Trading Card Game (Συνεργάτες: Β. -Μ. Αναστασίου, Π. Διαμαντόπουλος, Ε. Ντόνα, Σ. Μανέας)
- Προσομοίωση Οικοσυστήματος (Συνεργάτες: Β. -Μ. Αναστασίου, Π. Διαμαντόπουλος)
- Προσομοίωση Συστήματος Στοιχημάτων (Συνεργάτης: Β. -Μ. Αναστασίου)
- Προσομοίωση Discussion Forum (Συνεργάτες: Δ. Γκανάτσιος, Α. Γλεντής, Δ. Ραβίολος, Γ. Καστρίνης)

11.7 Σύνοψη Κεφαλαίου

Στο κεφάλαιο αυτό, αφού γίνεται μια ανασκόπηση της μεθοδολογίας που προτείνεται να ακολουθηθεί κατά την ανάπτυξη, περιλαμβάνονται ασκήσεις κλιμακούμενης δυσκολίας για εξάσκηση στην ανάπτυξη κώδικα σε αντικειμενοστραφές περιβάλλον. Συγκεκριμένα, κατά τις υλοποιήσεις τους γίνεται εφαρμογή των αρχών της αφαίρεσης στα δεδομένα. Πολλές φορές απαιτείται και εκμετάλλευση κληρονομικότητας.

Το βιβλίο απευθύνεται κυρίως σε προπτυχιακούς φοιτητές που έρχονται για πρώτη φορά σε επαφή με τον αντικειμενοστραφή προγραμματισμό. Προϋποθέτει βασική γνώση του διαδικαστικού προγραμματισμού. Αρχικά, παρουσιάζονται οι έννοιες του αντικειμενοστραφούς προγραμματισμού, οι μηχανισμοί που απαιτούνται για την υποστήριξή του, καθώς και οι ιδιότητές του, ανεξάρτητα από συγκεκριμένη γλώσσα προγραμματισμού. Ο στόχος είναι να αναδειχθεί η καθολικότητά τους. Στη συνέχεια, το βιβλίο εστιάζεται στη γλώσσα C++, παρουσιάζοντας τα παραπάνω μέσω της γλώσσας αυτής. Ξεκινά συνοψίζοντας την υποστήριξη διαδικαστικού προγραμματισμού, με έμφαση σε εξειδικευμένες γλωσσικές δομές της C++. Κατόπιν, συζητιέται η έννοια της εμβέλειας, πηγαίνοντας σταδιακά στην αφαίρεση στα δεδομένα. Κατά την παρουσίαση της αφαίρεσης στα δεδομένα, το ένα από τα δύο βασικά θέματα στον αντικειμενοστραφή προγραμματισμό, αναπτύσσεται η δυνατότητα ορισμού κλάσεων και αντικειμένων, καθώς και σχετιζόμενα θέματα, όπως οι περιορισμοί ορατότητας για απόκρυψη λεπτομερειών υλοποίησης, οι λειτουργίες κατά τη δημιουργία και κατά την καταστροφή των αντικειμένων, αλλά και η λειτουργία κατά την ανάθεση αντικειμένων. Ακολουθεί ξεχωριστή συζήτηση για τη σύνθεση κλάσεων. Στη συνέχεια, γίνεται συζήτηση σχετικά με το δεύτερο βασικό θέμα στον αντικειμενοστραφή προγραμματισμό, που είναι η κληρονομικότητα. Η ενασχόληση με τη C++ ολοκληρώνεται με παρουσίαση επιπλέον γλωσσικών στοιχείων που είναι σχετικά με τις κλάσεις. Ιδιαίτερη έμφαση δίνεται στα στατικά μέλη κλάσεων, η ύπαρξη των οποίων επιτρέπει ομαλή μετάβαση από τη γλώσσα C++ στη γλώσσα Java. Το σύστημα εισόδου/εξόδου της C++ αναφέρεται και ως ενδιαφέρον παράδειγμα χρήσης του αντικειμενοστραφούς προγραμματισμού. Γίνεται, επίσης, αναφορά στον προγραμματισμό γενικεύσεων. Κατόπιν, το βιβλίο στρέφεται στη γλώσσα Java και γίνεται παρουσίαση των αντικειμενοστραφών χαρακτηριστικών της. Ακολουθεί σύνοψη αντικειμενοστραφών στοιχείων από άλλες γλώσσες αντικειμενοστραφούς προγραμματισμού, όχι απαραίτητα διαδικαστικές. Το βιβλίο κλείνει με σύνθετα προβλήματα, τόσο για προσωπική ενασχόληση όσο και για εξάσκηση, αλλά και ως κατάλληλα παραδείγματα ανάγκης χρήσης του αντικειμενοστραφούς προγραμματισμού.

Το παρόν σύγγραμμα δημιουργήθηκε στο πλαίσιο του Έργου ΚΑΛΛΙΠΟΣ+

Χρηματοδότης	ΥΠΟΥΡΓΕΙΟ ΠΑΙΔΕΙΑΣ ΚΑΙ ΘΡΗΣΚΕΥΜΑΤΩΝ, ΠΡΟΓΡΑΜΜΑΤΑ ΠΔΕ, ΕΠΑ 2020-2025
Φορέας υλοποίησης	ΕΛΚΕ ΕΜΠ
Φορέας λειτουργίας	ΣΕΑΒ/ΠΑΡΑΡΤΗΜΑ ΕΜΠ/ΜΟΝΑΔΑ ΕΚΔΟΣΕΩΝ
Διάρκεια 2ης Φάσης	2020-2023
Σκοπός	Η ΔΗΜΙΟΥΡΓΙΑ ΑΚΑΔΗΜΑΪΚΩΝ ΨΗΦΙΑΚΩΝ ΣΥΓΓΡΑΜΜΑΤΩΝ ΑΝΟΙΚΤΗΣ ΠΡΟΣΒΑΣΗΣ (ΠΕΡΙΣΣΟΤΕΡΩΝ ΑΠΟ 700) • ΠΡΟΠΤΥΧΙΑΚΩΝ ΚΑΙ ΜΕΤΑΠΤΥΧΙΑΚΩΝ ΕΓΧΕΙΡΙΔΙΩΝ • ΜΟΝΟΓΡΑΦΙΩΝ • ΜΕΤΑΦΡΑΣΕΩΝ ΑΝΟΙΚΤΩΝ TEXTBOOKS • ΒΙΒΛΙΟΓΡΑΦΙΚΩΝ ΟΔΗΓΩΝ
Επιστημονικά Υπεύθυνος	ΝΙΚΟΛΑΟΣ ΜΗΤΡΟΥ, ΚΑΘΗΓΗΤΗΣ ΣΗΜΜΥ ΕΜΠ
ISBN: 978-618-228-050-8	DOI: http://dx.doi.org/10.57713/kallipos-280

Το παρόν σύγγραμμα χρηματοδοτήθηκε από το Πρόγραμμα Δημοσίων Επενδύσεων του Υπουργείου Παιδείας.